Graph Algorithms From the Ground Up

From Breadth-First Search to Laplacian Solvers

The Advanced Companion to Algorithms From the Ground Up

June 2026

Preface

There is a moment, somewhere past your first algorithms course, when graphs stop being a topic and start being a language. You stop asking "how do I run BFS" and start asking "what is the right graph, and what does its structure already know about my problem." A scheduling constraint becomes a directed edge; a bottleneck becomes a cut; a notion of similarity becomes a weight; a hard combinatorial question dissolves into an eigenvalue. This book is written for the reader who wants to make that shift deliberately, and to be able to prove that the tools work and to say exactly how fast they run.

We start with breadth-first search on a tiny graph you can draw on a napkin, and we work up, justifying each step, to Laplacian solvers that compute electrical flows in nearly linear time. These solvers are at the research frontier and underlie the fastest known algorithms for problems as old as maximum flow. The path is long, but no step is skipped.

Who this book is for

This is the *advanced companion* to a friendly introductory algorithms book. We assume you have met the classics before, even if the memory is hazy: you have seen BFS and DFS, you know what a stack and a queue are, you can read a recursion, you have heard the words "Dijkstra" and "minimum spanning tree" and have a rough sense of what they do. We assume you are comfortable with big-O notation as an idea and will not flinch when we write $\Theta(m + n \log n)$.

What we do *not* assume is that you remember the proofs, or that you have ever seen anything past the introductory layer. We re-derive the foundations at a faster pace and a higher rigor than an intro course, precisely so that the advanced material – blossoms, link-cut trees, treewidth, Cheeger's inequality, electrical flows – rests on proofs you have seen and not on half-remembered intuition. If you can comfortably follow a proof by induction and you are willing to draw the occasional picture, you are ready.

Intuition

A useful self-test. Take a connected undirected graph and remove one edge. When does the graph fall into two pieces? If the phrase "that edge was a bridge" came to mind, you have the right background. If it did not, you will after the chapter on bridges and biconnectivity – but you should expect to work for it. This book rewards the reader who pauses, draws, and re-derives.

What "from the ground up" actually means here

Many advanced books open in the middle. They state a theorem, prove it cleanly, and move on, leaving you to reconstruct the path that a researcher actually walked to arrive there. We do the opposite. For every major algorithm the chapter follows the same arc, and the arc is the pedagogy:

1. **A picture and an intuition.** Before a single line of pseudocode we draw a small graph and ask what *should* happen, building the mental model that the algorithm will later make precise. We put intuition first because it is how you will later *remember* the algorithm and how you will debug it when your implementation misbehaves at 2 a.m.

2. **The algorithm, in clean pseudocode.** Every major algorithm is stated in an `algorithm2e` block precise enough to implement directly, with the data structures named explicitly.

3. **A correctness argument.** We prove the thing works – via a loop invariant, an exchange argument, an augmenting-path argument, a duality, or a potential function. The proof technique is itself a transferable skill, and we name it each time so the catalogue grows in your head.

4. **A tight complexity analysis.** Not "it's polynomial" but the exact bound, with the bottleneck identified and, where it matters, a matching lower bound so you know the analysis is not loose.

5. **A trace by hand.** We run the algorithm on a concrete small graph, showing the data-structure state at each step. This is the single most important habit the book tries to instill.

Why it works

Why insist on intuition *and* full rigor, rather than picking one? Because they fail in opposite directions, and the pair is robust where either alone is brittle. Pure intuition produces algorithms that are "intuitively plausible" and quietly wrong on the adversarial input – the negative-weight cycle, the zero-capacity edge, the disconnected component. Pure formalism produces proofs you can verify line by line but cannot reconstruct, modify, or apply to the next problem. Hold both and you get an algorithm you can re-derive from scratch on a whiteboard *and* defend against an adversary. That combination is what separates someone who has read about algorithms from someone who can build them.

The colored-box language

Throughout the book a small set of recurring boxes carries the load that a good lecturer carries with tone of voice – this part is the heart, this part is a landmine, this is a war story. Learn to read the colors and you can skim or dive at will.

Focus

A **Focus** box frames the one question a section is really about. When you are lost, scroll up to the nearest Focus box; it tells you what the surrounding machinery is *for*.

Intuition

An **Intuition** box is the picture-in-words: the mental model, the analogy, the "what should happen" that precedes the formal statement. These are safe to read first and out of order.

Why it works

A **Why it works** box states the central idea of a correctness proof, stated before the proof so you read the proof knowing where it is going.

Common trap

A **Common trap** box is the subtle bug – the off-by-one, the case the proof quietly assumes away, the input that breaks the naive version. We hunt these on purpose. Most of what you will get wrong in practice is collected here in advance.

A **History** box places an idea in time: Dijkstra at a cafe table in 1956, Edmonds inventing "good characterization" and blossoms, Tarjan turning depth-first search into a versatile search method, Karger randomizing the minimum cut, Spielman and Teng solving Laplacian systems in near-linear time. Algorithms are made by people solving specific problems, and knowing the context clarifies why the algorithm was built.

Cost summary

A **Cost summary** box collects the running time and space of an algorithm in one place, often with the bottleneck circled and variants compared. When you are choosing an algorithm for real work, these are the boxes you want.

Trace it yourself

A **Trace it yourself** box hands you a specific small graph and asks you to run the algorithm by hand before reading our trace. Do these. They are where the understanding actually happens; reading our trace afterward only confirms it.

Application

An **Application** box shows where the algorithm is used in practice – register allocation, image segmentation, recommendation, routing, circuit analysis. Abstractions stick better when anchored to a concrete problem.

Connection

A **Connection** box is the most important box for this companion, because it is the bridge between the three books. It links forward and backward across chapters here, and directly out to the companion *intro algorithms* book and the companion *linear algebra* book. When a graph Laplacian turns out to be a positive semidefinite matrix whose spectrum you already studied, a Connection box says so explicitly.

There are a few more – *Formal* for the precise statement, *Walking question* for something to chew on, *Pause* to recover and consolidate – but the seven above are the main ones.

How the three books fit together

This volume is one of three books, written so that their notation and coverage line up.

Connection

The intro algorithms book is the prequel. It teaches you to read pseudocode, to reason about big-O, to wield recursion and a handful of core data structures, and it introduces BFS, DFS, Dijkstra, and union-find at a gentle pace. Where that book *introduces* a graph algorithm, this book *proves it tight and then goes far past it.* If a derivation here feels too fast, the corresponding section there is the place to slow down.

The linear algebra book supplies the mathematical language for the second half of this volume. The instant we write down the Laplacian $\mathbf{L} = \mathbf{D} - \mathbf{A}$ in the spectral chapters, we are doing spectral theory: eigenvalues, eigenvectors, the Rayleigh quotient, positive semidefiniteness, the quadratic form $x^\top \mathbf{L} x = \sum_{(u,v)\in E}(x_u - x_v)^2$. Every one of those is a concept the linear algebra book builds from scratch. The whole spectral arc – from the Laplacian through Cheeger, expanders, random walks, and Laplacian solvers – is, in a real sense, an extended application of that book to graphs: a graph becomes a matrix, a cut becomes a vector, a bottleneck becomes the second-smallest eigenvalue, and "solve this network" becomes "solve $\mathbf{L}x = b$."

You can read this book on its own – it is self-contained, and the appendix gathers the data-structure and linear-algebra facts we lean on. But the Connection boxes are written assuming the companions exist, and a reader who follows them gets a unified picture instead of three disconnected courses.

The shape of the book: from BFS to Laplacian solvers

It helps to see the whole book on one page before you start, because the later chapters explain the earlier choices. The book moves through five broad territories.

Focus

Territory I — Foundations and traversal. Representations, BFS, DFS, topological order, strongly connected components, minimum spanning trees, and shortest paths from Dijkstra up through Johnson's all-pairs algorithm. We redo this foundation at full rigor: every greedy choice gets an exchange argument, every relaxation gets an invariant.

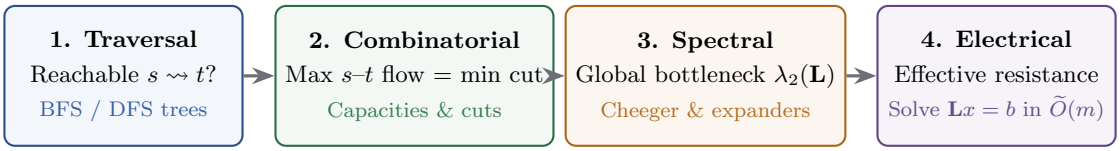
Territory II — Cuts, flows, and matchings. The duality-rich heart of combinatorial optimization: bridges and biconnectivity, global minimum cut, maximum flow and its min-cut dual, push-relabel, min-cost flow, bipartite and weighted bipartite matching, Edmonds' blossom algorithm for general matching, and the art of modeling a problem as a flow.

Territory III — Structure on trees and sparse graphs. Heavy-light and centroid decomposition, Euler tours, link-cut trees, LCA and range structures, treewidth and tree decompositions, fixed-parameter tractability, planarity, separators, coloring, independent sets and cliques, and perfect graphs. Here structure is the resource: exploit it and intractable problems become easy.

Territory IV — The spectral world. The Laplacian, spectral graph theory, spectral clustering, expanders, the deep equivalence between random walks and electrical networks,

and PageRank as a Markov chain. This is where graphs become linear algebra.
Territory V — Hardness, scale, and the frontier. NP-hard graph problems, approximation, dynamic graphs, streaming and massive graphs, and finally near-linear-time Laplacian solvers and electrical flows, where Territory IV's spectral view and Territory II's flow view combine in one of the field's deepest results.

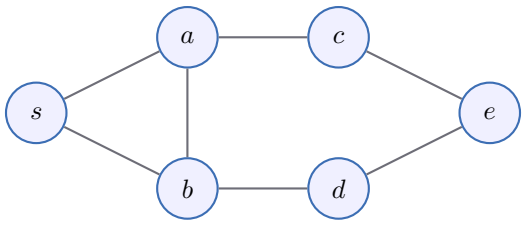
One quantity links the first chapters to the last, and its meaning changes as the book progresses. Connectivity, in the BFS chapter, is a yes/no question answered by BFS: can I reach t from s? In the maximum-flow chapter it has become a number: *how much* can flow from s to t, equal by duality to the lightest cut separating them. In the spectral chapters it has become spectral: how well-connected is the graph *as a whole*, measured by the second eigenvalue λ_2 of the Laplacian, with Cheeger's inequality pinning that eigenvalue to the sparsest cut. And in the random-walks chapter it is physical: connectivity is the conductance of a resistor network, computable by solving a linear system. It is the same question asked in four languages, each more expressive than the last. The figure below summarizes the four stages:



A worked trace, so you know what they look like

The single habit this book most wants you to build is *tracing an algorithm by hand on a small graph*. Reading pseudocode tells you what an algorithm says; tracing it tells you what it *does*. So let us do one now, in the preface, before any chapter – both to set the standard and to dust off your graph intuition. We will trace plain breadth-first search, the most elementary algorithm in the book, because the point here is the *method*, not the difficulty.

Consider this little undirected graph on six vertices. We will run BFS from s and ask which vertices it reaches, in what order, and at what distance.



Intuition

BFS explores in rings. It visits everything at distance 0 (just s), then everything at distance 1 (the neighbors of s), then distance 2, and so on, never touching a vertex at distance $k+1$ before all of distance k is done. A FIFO queue enforces exactly this order for free: a vertex enqueued earlier comes out earlier, and since distance-k vertices are all enqueued before any distance-$(k+1)$ vertex, the rings stay separated. This ring-by-ring order ensures that every

vertex is reached along a path of minimum edge count.

Here is the algorithm we are tracing, stated precisely.

Algorithm 1: Breadth-First Search from a source

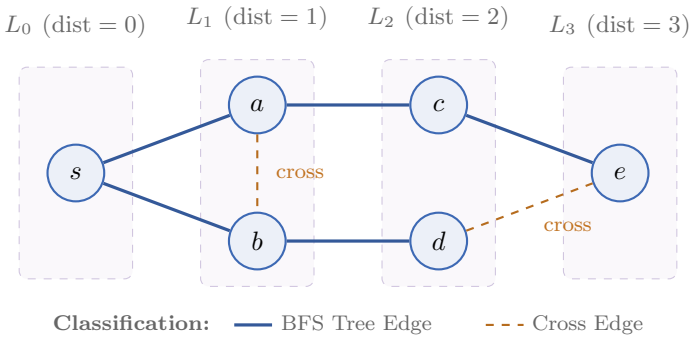
Input : Graph $G = (V, E)$ as adjacency lists; source $s \in V$.
Output: dist$[v]$ = number of edges on a shortest s–v path, or ∞.

```
1  foreach v ∈ V do
2      dist[v] ← ∞
3  dist[s] ← 0;
4  Q ← empty FIFO queue; enqueue s;
5  while Q is not empty do
6      u ← dequeue(Q);
7      foreach neighbor w of u do
8          if dist[w] = ∞ then
9              dist[w] ← dist[u] + 1;
10             enqueue w into Q;
11 return dist;
```

Now the trace. Assume each adjacency list is sorted alphabetically, so neighbors are scanned in that order – pinning down the order makes the trace reproducible, which is exactly what you want when debugging. We show the queue (front on the left) and the distances discovered after each dequeue.

Step	**Queue (front → back)**	**Distances discovered this step**
init	$[\,s\,]$	dist$[s] = 0$
dequeue s	$[\,a, b\,]$	dist$[a] = 1$, dist$[b] = 1$
dequeue a	$[\,b, c\,]$	dist$[c] = 2$ (neighbor b already seen)
dequeue b	$[\,c, d\,]$	dist$[d] = 2$ (neighbor a already seen)
dequeue c	$[\,d, e\,]$	dist$[e] = 3$
dequeue d	$[\,e\,]$	(neighbor e already seen)
dequeue e	$[\,]$	(neighbors c, d already seen)

Redrawing the graph with vertices grouped into columns by distance turns the trace table into a layered BFS tree, separating the solid tree edges that first discover each vertex from the dashed cross edges between already-seen vertices:



The final distances are $\text{dist}[s] = 0$, $\text{dist}[a] = \text{dist}[b] = 1$, $\text{dist}[c] = \text{dist}[d] = 2$, $\text{dist}[e] = 3$. Read the rings straight off the queue: s alone, then $\{a, b\}$, then $\{c, d\}$, then $\{e\}$. Notice the two edges that discovered nothing – the edge a–b when we processed a, and the edge d–e when we processed d. Those are precisely the edges joining vertices in the same ring or an already-finished ring; in the directed-graph chapters that distinction between "tree" and "non-tree" edges becomes the engine behind cycle detection and strongly connected components. A trace makes this split between tree edges and non-tree edges directly visible.

Why it works

Why the distances are correct. The intuition is that BFS explores outward in concentric rings. Every vertex at distance 1 is found directly from the source s; every vertex at distance 2 is found from some vertex at distance 1, and so on. Because vertices are processed in first-in, first-out order, all vertices at distance k are explored before any vertex at distance $k + 1$. When a vertex w is first encountered from an active vertex u, the path from s through u to w has length $\text{dist}[u] + 1$, which is as short as possible. We will prove this ring-by-ring property carefully in Chapter 2 with a formal induction; here, the point is that the FIFO queue determines the discovery order.

Cost summary

Each vertex is enqueued and dequeued at most once – it is enqueued only when its distance flips from ∞, which happens once. So the **while** loop runs $O(n)$ times. Across all iterations the inner **foreach** scans each adjacency list exactly once, totaling $\sum_v \deg(v) = 2m$ edge inspections in an undirected graph. With adjacency lists and an array-backed queue, every individual operation is $O(1)$. Total: $\Theta(n + m)$ time and $\Theta(n)$ extra space. This is optimal – you cannot decide reachability without, in the worst case, looking at every vertex and edge once.

Common trap

The most common BFS bug, and a template for the bugs this book hunts everywhere: marking a vertex as visited *when you dequeue it* instead of *when you enqueue it*. In our trace, b is a neighbor of both s and a. With enqueue-time marking, b is discovered once (by s) and added to the queue once. With dequeue-time marking, both s and a would enqueue b, so b enters the queue twice; you may still get correct distances, but the queue can swell to $\Theta(m)$ and on a dense graph your "linear" algorithm quietly degrades. The guard `if` $\text{dist}[w] = \infty$ at enqueue

time matters. In general, the invariant is a contract the implementation must satisfy at every step.

Every chapter follows this pattern, shown here on the simplest possible example: picture, pseudocode, trace, why-it-works, cost, trap. When the example is Edmonds' blossom contraction or a Cheeger cut instead of BFS, the pattern is the same and the boxes mean the same things. Once you know the pattern, the hard chapters are *longer* but not *unfamiliar*.

How to read this book

Read it with a pen. Every *Trace it yourself* box is an invitation to run the algorithm before reading our version, and the gap between your trace and ours is the most precise feedback you will get about your own understanding. Draw the small graphs; deciding where to place the vertices often reveals the structure.

Reading note

Two reading paths. **Linear** – chapters are ordered so each rests only on earlier ones, and reading straight through is the intended default. **Targeted** – if you came for a specific tool (say, max-flow for a project, or Laplacian solvers for a paper), jump to its chapter and chase the backward Connection boxes to fill gaps; they form a dependency trail you can follow upstream. The final appendix collects the data-structure and linear-algebra prerequisites that the main text uses without comment.

Do not skip the proofs, but do not let them block you either. On a first pass it is legitimate to read a theorem, absorb its *Why it works* box, run the trace, and defer the line-by-line proof to a second pass. Understanding what is true and why it should be true is enough to use an algorithm; understanding the proof in full is what lets you modify it for the variant your problem actually needs – and sooner or later your problem will need a variant.

History

A last word on why the order is what it is. The field did not develop top-down. BFS and DFS were folklore made rigorous; Dijkstra (1956) and Kruskal/Prim gave the greedy method its best-known successes; Ford and Fulkerson (1956) found max-flow/min-cut and with it the duality that organizes half this book; Edmonds (1965) drew the line between polynomial and not and invented blossoms to stay on the right side of it; Tarjan (1970s) showed a single depth-first search could compute almost any structural property of a graph; Fiedler (1973) connected a graph's connectivity to its second eigenvalue; Karger (1990s) let randomness find minimum cuts; and Spielman and Teng (2004) closed the loop by solving Laplacian systems in near-linear time, making the spectral view computationally as cheap as the combinatorial one. We follow roughly that historical arc because the problems came before the solutions, and meeting the problem first is the surest way to understand why anyone bothered to solve it.

Exercises

These are warm-up exercises. They use nothing past this preface, and their only purpose is to get a pen in your hand before Chapter 1. Solutions are not printed; if you can justify every step of your trace, you have checked your answer.

Exercise 0.1 (Trace it cold)**.** Redraw the undirected six-vertex graph from the worked trace, then add one edge: c–d. Re-run BFS from s by hand, writing the queue and the discovered distances after each dequeue exactly as we did. Which distances change, and which do not? Identify, for the new edge, whether it is discovered as a tree edge or found pointing at an already-seen vertex.

Exercise 0.2 (Changing the starting point)**.** On the undirected graph from the worked trace, run the search starting from vertex c instead of s. In what order are the vertices discovered, and what is the maximum distance from c to any reachable vertex?

Exercise 0.3 (Break the algorithm on purpose)**.** Implement the dequeue-time-marking bug described in the *Common trap* box: move the "mark as visited" step from enqueue time to dequeue time. On the undirected graph, exhibit a vertex that enters the queue twice. Then construct, or describe, a family of graphs on n vertices where this bug pushes the queue to size $\Theta(n^2)$ even though the correct algorithm keeps it at $O(n)$.

Exercise 0.4 (Draw your own graph)**.** Draw an undirected connected graph with 6 vertices where every vertex has degree at least 2, but the shortest-path distance between some pair of vertices is at least 3. Trace a search from one endpoint of that diameter to the other.

Exercise 0.5 (A counting sanity check)**.** The *Cost summary* box asserts $\sum_v \deg(v) = 2m$ for an undirected graph. Count the degrees of all six vertices in the worked undirected graph, sum them, and confirm the total is twice the number of edges you drew. Explain in your own words why every undirected edge contributes exactly 2 to the sum of degrees.

The summit – a near-linear-time Laplacian solver computing electrical flows – is demanding, and we will not pretend otherwise. But by the time you reach it you will have built every piece: the graph as a matrix from the Laplacian chapter, the spectrum and Cheeger's inequality from the spectral and expander chapters, the random-walk and electrical correspondence from the random-walks chapter, the flow duality from the maximum-flow chapter, and the sparsification ideas that let a huge linear system be solved by repeatedly solving small ones. The final chapter introduces no new foundations; it combines tools developed in earlier chapters.

Turn the page. Before any algorithm can traverse a graph, we must answer two basic questions: what is a graph mathematically, and how do we store its vertices and edges in computer memory so that looking up neighbors is fast? In Chapter 1, we lay this groundwork by formalizing directed and undirected graphs and evaluating adjacency lists, adjacency matrices, and edge lists under our asymptotic cost model.

Contents

Preface iii

I Foundations 1

1 Graphs and How We Represent Them 3

1.1 What a graph is . . . 3

1.1.1 Degrees and the handshake lemma . . . 4

1.2 Walks, paths, cycles, and connectivity . . . 6

1.3 Trees, forests, and DAGs . . . 7

1.4 Bipartite graphs . . . 9

1.5 Three ways to store a graph . . . 10

1.5.1 Adjacency matrix . . . 10

1.5.2 Adjacency list . . . 11

1.5.3 Edge list . . . 12

1.5.4 The comparison, side by side . . . 12

1.5.5 Building an adjacency list in linear time . . . 13

1.6 Implicit graphs . . . 14

1.7 The cost model . . . 16

1.8 Exercises . . . 17

2 Breadth-First Search and Its Uses 19

2.1 The wave picture . . . 19

2.2 The queue and the algorithm . . . 20

2.2.1 Tracing the queue on G_0 . . . 21

2.3 Correctness: BFS computes shortest paths in edges . . . 22

2.3.1 Complexity . . . 24
2.4 The BFS tree and shortest-path structure . . . 24
2.4.1 The eccentricity and diameter view . . . 25
2.5 Testing bipartiteness . . . 26
2.6 Multi-source BFS . . . 27
2.7 0–1 BFS with a deque . . . 29
2.8 More applications . . . 31
2.9 Summary . . . 32
2.10 Exercises . . . 33

3 Depth-First Search, Timestamps, and Structure 35
3.1 The walk: intuition and a hand trace . . . 35
3.1.1 A directed graph to trace . . . 36
3.2 The algorithm . . . 37
3.2.1 The iterative skeleton . . . 38
3.2.2 Complexity . . . 39
3.3 The parenthesis theorem . . . 40
3.3.1 The white-path theorem . . . 41
3.4 Edge classification . . . 42
3.4.1 Reading the type off the timestamps . . . 43
3.4.2 Verifying on the trace . . . 43
3.5 Cycle detection . . . 44
3.5.1 Directed graphs: back edge $\iff$ cycle . . . 44
3.5.2 Undirected graphs: the parent trap . . . 45
3.5.3 Counting independent cycles . . . 47
3.5.4 A worked undirected trace with low-links . . . 47
3.6 Directed versus undirected: what changes . . . 48
3.7 The structure DFS exposes . . . 49
3.8 Exercises . . . 50

4 Topological Sort and DAGs 53
4.1 What a DAG Is, and Why Acyclicity Unlocks Linear Order . . . 53
4.2 Method 1: Kahn's Algorithm (Indegree Peeling) . . . 55

4.3 Method 2: DFS Finish Times . . . 58
4.4 Uniqueness of the Order . . . 61
4.5 Dynamic Programming on a DAG . . . 62
4.5.1 Counting paths . . . 63
4.5.2 Longest path . . . 64
4.5.3 Why one sweep beats Dijkstra here . . . 65
4.6 Applications: Scheduling and Build Systems . . . 65
4.7 Exercises . . . 66

5 Connectivity and Strongly Connected Components 69

5.1 Undirected connectivity . . . 69
5.2 Strong connectivity: the directed story . . . 70
5.3 Kosaraju's two-pass algorithm . . . 72
5.3.1 Finish times and the key lemma . . . 72
5.3.2 The algorithm . . . 73
5.3.3 Tracing Kosaraju by hand . . . 74
5.4 Tarjan's one-pass algorithm . . . 75
5.4.1 Definitions and the algorithm . . . 76
5.4.2 Why lowlink detects component roots . . . 77
5.4.3 Tracing Tarjan by hand . . . 78
5.5 The condensation DAG . . . 80
5.6 Application: 2-SAT in linear time . . . 81
5.6.1 The implication graph . . . 81
5.6.2 The criterion and the assignment . . . 82
5.7 Eulerian Circuits, Hierholzer's Algorithm, and De Bruijn Genome Assembly . . . 84
5.7.1 Degree conservation and Euler's theorem . . . 84
5.7.2 Hierholzer's cycle-splicing algorithm . . . 85
5.7.3 De Bruijn graphs and DNA shotgun sequencing . . . 87
5.7.4 Trace of a DNA assembly on a De Bruijn graph . . . 87
5.8 Connectivity refinements and edge cases . . . 89
5.9 Exercises . . . 90

II Trees, Paths, and Cuts **93**

6 Minimum Spanning Trees **95**

6.1 The problem and a picture . . . 95
6.2 Two structural laws: the cut and the cycle . . . 96
6.3 The generic greedy algorithm . . . 98
6.4 Kruskal's algorithm . . . 99
6.4.1 Union–find in one paragraph . . . 100
6.5 Prim's algorithm . . . 101
6.6 Boruvka's algorithm . . . 103
6.7 Reverse-delete: the cycle property as an algorithm . . . 105
6.8 Uniqueness of the MST . . . 107
6.9 Verifying a spanning tree in linear time . . . 108
6.10 Randomized linear-time MST . . . 109
6.11 Applications . . . 111
Exercises . . . 112

7 Shortest Paths: Foundations **115**

7.1 The Problem, Stated Carefully . . . 115
7.1.1 The optimal-substructure property . . . 116
7.2 Relaxation: The One Move Every Algorithm Makes . . . 116
7.2.1 Optimality conditions . . . 117
7.3 A Running Example . . . 118
7.4 Dijkstra's Algorithm . . . 119
7.4.1 The greedy idea . . . 119
7.4.2 Correctness . . . 120
7.4.3 Tracing Dijkstra by hand . . . 121
7.4.4 Heap variants and complexity . . . 122
7.4.5 The lazy-deletion implementation . . . 123
7.5 Bellman–Ford and Negative Cycles . . . 124
7.5.1 Tracing Bellman–Ford with a negative edge . . . 125
7.5.2 Complexity and the early-exit refinement . . . 125
7.6 Shortest Paths in a DAG . . . 126

7.7 Potentials and Reduced Costs . . . 127
7.8 Putting the Pieces Together . . . 130
7.9 Exercises . . . 130

8 Advanced Shortest Paths 133

8.1 The cost of being blind: a motivating picture . . . 133
8.2 Goal-directed search: A^* . . . 134
8.2.1 From Dijkstra to A^* . . . 134
8.2.2 Admissibility and consistency . . . 134
8.2.3 The algorithm . . . 136
8.2.4 A worked trace . . . 136
8.3 Bidirectional search . . . 137
8.3.1 Two balls meeting in the middle . . . 137
8.3.2 The stopping rule (and the classic bug) . . . 138
8.3.3 A worked trace . . . 139
8.3.4 Bidirectional A^*: a subtlety . . . 140
8.4 Landmarks and the ALT algorithm . . . 140
8.4.1 Triangle-inequality bounds . . . 140
8.4.2 Preprocessing and the query split . . . 141
8.4.3 Choosing landmarks . . . 141
8.5 Contraction hierarchies . . . 142
8.5.1 The idea: shortcut the unimportant vertices . . . 142
8.5.2 The augmented graph and the upward/downward split . . . 143
8.5.3 Query algorithm . . . 144
8.5.4 Preprocessing: ordering and contraction . . . 144
8.5.5 A worked contraction . . . 146
8.6 When each method wins . . . 147
8.7 Exercises . . . 149

9 All-Pairs Shortest Paths 151

9.1 The Problem and the Multi-Source Dilemma . . . 151
9.2 Floyd–Warshall: shortest paths as a dynamic program . . . 152
9.2.1 The right subproblem . . . 152

9.2.2 The recurrence . . . 152
9.2.3 Collapsing the third dimension . . . 153
9.2.4 A picture and a hand trace . . . 154
9.2.5 Correctness . . . 155
9.2.6 Negative-cycle detection . . . 156
9.3 Transitive closure: reachability with the same loop . . . 156
9.4 The matrix-multiplication view and the min-plus product . . . 157
9.5 Johnson's algorithm: reweighting for sparse graphs . . . 159
9.5.1 The potential trick . . . 159
9.5.2 The algorithm . . . 160
9.5.3 Worked example: same graph, reweighted . . . 160
9.5.4 Correctness . . . 162
9.6 Choosing an algorithm . . . 163
9.7 Path reconstruction and a few extensions . . . 163
9.8 Exercises . . . 164

10 Bridges, Articulation Points, and Biconnectivity 167
10.1 What we are looking for . . . 167
10.2 The DFS tree and why it has no cross edges . . . 168
10.3 Low-link: the highest reachable ancestor . . . 169
10.3.1 Tracing low-link by hand . . . 171
10.4 Finding bridges . . . 172
10.5 Finding articulation points . . . 173
10.6 Biconnected components and the block-cut tree . . . 175
10.6.1 Extracting blocks with an edge stack . . . 175
10.6.2 The block-cut tree . . . 176
10.7 Complexity . . . 177
10.8 2-edge- and 2-vertex-connectivity . . . 178
10.9 Application: network reliability . . . 179
10.10Exercises . . . 180

11 Global Minimum Cut 183
11.1 The problem, precisely . . . 183

11.2 A tiny graph to keep in mind . . . 185
11.3 Stoer–Wagner: greedy orderings find a cut . . . 185
11.3.1 The minimum cut phase . . . 185
11.3.2 The full algorithm . . . 186
11.3.3 Correctness . . . 187
11.3.4 Complexity . . . 189
11.3.5 Tracing Stoer–Wagner by hand . . . 189
11.4 Karger's random contraction . . . 190
11.4.1 Contraction . . . 191
11.4.2 The algorithm . . . 192
11.4.3 Success probability . . . 192
11.4.4 Amplification . . . 193
11.5 Karger–Stein: Branching on the Dangerous Tail . . . 194
11.5.1 The partial-contraction guarantee . . . 194
11.5.2 The recursion . . . 194
11.5.3 Running time . . . 195
11.5.4 Success probability of the recursion . . . 195
11.5.5 Tracing Karger by hand . . . 196
11.6 Applications . . . 197
11.7 Exercises . . . 198

III Flows and Matching **201**

12 Maximum Flow **203**

12.1 Flow Networks . . . 203
12.1.1 A small network we will carry through the chapter . . . 204
12.2 Residual Graphs and Augmenting Paths . . . 204
12.3 Ford–Fulkerson and the Max-Flow Min-Cut Theorem . . . 206
12.3.1 Cuts . . . 207
12.3.2 Tracing Ford–Fulkerson on the running network . . . 208
12.3.3 Does Ford–Fulkerson even terminate? . . . 209
12.4 Edmonds–Karp: Shortest Augmenting Paths . . . 209

12.5 Dinic's Algorithm: Level Graphs and Blocking Flows . . . 211
12.5.1 The level graph . . . 211
12.5.2 The phase structure . . . 212
12.5.3 Finding a blocking flow . . . 213
12.5.4 Special cases where Dinic is much faster . . . 214
12.6 Capacity Scaling . . . 214
12.7 Putting the Bounds Side by Side . . . 216
12.8 Exercises . . . 217

13 Push-Relabel and Modern Max-Flow **219**
13.1 Preflows: dropping conservation . . . 219
13.2 Heights and the two operations . . . 220
13.3 The generic algorithm . . . 222
13.3.1 Correctness . . . 223
13.4 Counting the operations: the generic bound . . . 224
13.5 A worked trace by hand . . . 225
13.5.1 A trace with drain-back . . . 226
13.6 FIFO push-relabel: $O(V^3)$ . . . 228
13.7 Highest-label push-relabel: $O(V^2\sqrt{E})$ . . . 229
13.8 The gap heuristic and why it matters in practice . . . 230
13.9 A note on almost-linear max-flow . . . 232
13.10Exercises . . . 233

14 Minimum-Cost Flow **237**
14.1 The problem, stated cleanly . . . 237
14.1.1 The residual graph carries the cost too . . . 238
14.2 The optimality criterion . . . 239
14.3 Negative-cycle canceling . . . 240
14.3.1 Termination and complexity . . . 240
14.3.2 Canceling by hand . . . 241
14.4 Successive shortest paths . . . 243
14.4.1 Why Dijkstra needs help: potentials and Johnson reweighting . . . 244
14.4.2 A reference implementation . . . 246

14.5 Working a graph by hand . . . 247
14.6 Cost scaling: the idea . . . 249
14.7 Transportation and assignment as special cases . . . 250
14.7.1 The transportation problem . . . 250
14.7.2 The assignment problem . . . 251
14.8 The linear-programming view . . . 251
14.9 Exercises . . . 253

15 Bipartite Matching **255**
15.1 Matchings: the language . . . 255
15.2 Augmenting paths and Berge's theorem . . . 256
15.3 The min-max duality: Kőnig and Hall . . . 258
15.4 The augmenting-path algorithm . . . 261
15.5 Hopcroft–Karp: augment in batches . . . 262
15.5.1 Why the phase count is $O(\sqrt{V})$ . . . 264
15.6 Matching via maximum flow . . . 266
15.7 Putting it together . . . 268
15.8 Exercises . . . 268

16 Weighted Bipartite Matching and the Hungarian Algorithm **271**
16.1 The Assignment Problem . . . 271
16.2 Picture First: Tight Edges and Potentials . . . 272
16.3 Augmenting Paths in the Equality Subgraph . . . 274
16.3.1 Growing a Hungarian tree . . . 274
16.3.2 The dual adjustment . . . 275
16.4 The Hungarian Algorithm . . . 277
16.5 Worked Example, Run by Hand . . . 278
16.5.1 A second trace, narrated in full . . . 280
16.6 Correctness . . . 282
16.7 Complexity . . . 283
16.8 Primal–Dual View and Complementary Slackness . . . 284
16.9 Applications . . . 285
16.10Exercises . . . 286

17 General Matching and Edmonds' Blossom Algorithm 289

17.1 Matchings, Augmenting Paths, and Berge's Theorem . . . 289

17.2 Why Odd Cycles Break the Bipartite Search . . . 291

17.3 Blossoms, Stems, and Contraction . . . 293

17.3.1 Contraction . . . 294

17.4 The Blossom Algorithm . . . 296

17.4.1 A subtle point: why we may fix the root . . . 297

17.5 A Worked Trace . . . 297

17.6 Complexity . . . 299

17.7 The Tutte–Berge Formula . . . 300

17.8 A Word on Weighted General Matching . . . 302

17.9 Summary . . . 304

17.10Exercises . . . 304

18 Modeling with Flows and Cuts 307

18.1 Conventions and the modeling reflex . . . 307

18.1.1 Gadget 1: vertex capacities . . . 308

18.1.2 Gadget 2: lower bounds on edges . . . 309

18.1.3 Gadget 3: antiparallel and undirected edges . . . 311

18.2 Maximum-weight closure and project selection . . . 311

18.2.1 The construction . . . 312

18.3 Image segmentation as an s–t min cut . . . 314

18.3.1 Construction and energy . . . 315

18.4 Baseball elimination . . . 317

18.4.1 Construction . . . 317

18.5 Disjoint paths and Menger's theorem . . . 319

18.5.1 Edge form via unit capacities . . . 320

18.5.2 Vertex form via vertex splitting . . . 321

18.6 A reduction checklist . . . 322

18.7 Exercises . . . 323

IV Decompositions and Dynamic Structures 325

19 Heavy-Light Decomposition 327

19.1 The problem: queries that follow tree paths . . . 327
19.2 Heavy and light: the size rule . . . 328
19.2.1 The key lemma . . . 329
19.3 Laying the chains into an array . . . 330
19.3.1 A tree to carry through the chapter . . . 331
19.3.2 Positions from the heavy-first DFS . . . 332
19.4 Building the decomposition . . . 333
19.5 Answering a path query . . . 333
19.5.1 Tracing a query by hand . . . 335
19.6 Path updates and lazy propagation . . . 335
19.6.1 Edge values vs vertex values . . . 336
19.7 Subtree queries and updates . . . 337
19.8 Complexity, all in one place . . . 337
19.9 Lowest Common Ancestor via Chain Climbs . . . 338
19.10 Worked end-to-end example with values . . . 338
19.11 Reference implementation sketch . . . 339
19.12 Where it gets used . . . 340
19.13 Exercises . . . 341

20 Centroid Decomposition 343

20.1 Tree centroids . . . 343
20.1.1 Finding a centroid in linear time . . . 345
20.2 The decomposition and its depth . . . 346
20.2.1 Tracing it on the nine-vertex tree . . . 347
20.3 Solving path problems by decomposition . . . 349
20.3.1 Pattern A: counting paths of a given length . . . 350
20.3.2 Pattern B: nearest marked vertex under updates . . . 352
20.4 The centroid tree as a query structure . . . 354
20.5 Weighted and rooted variants . . . 355
20.6 Exercises . . . 356

21 The Euler Tour Technique **359**

21.1 Flattening a tree . . . 359

21.1.1 The in/out timestamp encoding . . . 360

21.1.2 The single-array (entry-order) encoding . . . 362

21.2 Subtree queries with a Fenwick tree . . . 363

21.2.1 Subtree updates with a range-update Fenwick or lazy segment tree . . . 364

21.3 Path-to-root queries with the in/out encoding . . . 365

21.3.1 Lowest common ancestor from the Euler tour . . . 366

21.4 Euler tour trees: dynamic connectivity in a forest . . . 367

21.4.1 The representation . . . 367

21.4.2 Link and Cut as sequence surgery . . . 367

21.4.3 Correctness and the connectivity invariant . . . 369

21.4.4 Aggregates over a dynamic subtree . . . 370

21.4.5 A small dynamic trace . . . 370

21.5 Putting it together: a design checklist . . . 371

Exercises . . . 372

22 Link-Cut Trees and Dynamic Trees **375**

22.1 What we are maintaining . . . 376

22.2 Preferred paths: a self-adjusting chain decomposition . . . 377

22.2.1 The Two-Layer Architecture: Represented Tree vs. Auxiliary Splay Forest . . . 377

22.3 The auxiliary structure: one splay tree per preferred path . . . 378

22.3.1 Why a BST keyed by depth . . . 379

22.4 Splay trees, briefly . . . 379

22.5 Access: the master operation . . . 380

22.5.1 How access walks up . . . 380

22.5.2 A hand trace . . . 382

22.6 Everything else in terms of access . . . 383

22.7 Correctness: the invariants never break . . . 385

22.8 The amortized analysis . . . 385

22.8.1 Counting preferred-child changes . . . 386

22.8.2 The splay potential argument . . . 386

22.9 Applications of link-cut trees . . . 388

22.10Implementation checklist . . . 389
22.11Exercises . . . 390

23 Lowest Common Ancestor and Range Structures 393

23.1 Rooted Trees, Depth, and Ancestry . . . 394
23.2 The Naive Climb, and Why We Want Better . . . 395
23.3 Binary Lifting . . . 396
23.3.1 Answering a query . . . 397
23.4 Range Minimum Queries and Sparse Tables . . . 399
23.5 Euler Tour Plus RMQ: LCA in $O(1)$ . . . 401
23.6 Segment Trees: Queries with Updates . . . 403
23.6.1 Lazy propagation: range updates . . . 405
23.7 Fenwick (Binary Indexed) Trees . . . 406
23.8 The LCA $\leftrightarrow$ RMQ Equivalence . . . 408
23.9 Choosing a Structure . . . 410
23.10Exercises . . . 411

24 Tree Decompositions and Treewidth 413

24.1 From Trees to Tree-Like Graphs . . . 414
24.2 The Definition and Why Each Clause Matters . . . 414
24.2.1 The separator property . . . 415
24.2.2 Trees, cycles, and cliques . . . 416
24.3 A Worked Decomposition by Hand . . . 417
24.4 Nice Tree Decompositions . . . 418
24.5 Dynamic Programming over a Tree Decomposition . . . 419
24.5.1 Maximum weight independent set . . . 419
24.5.2 Tracing it by hand . . . 421
24.5.3 Dominating set: a 3-state DP . . . 421
24.6 Computing and Approximating Treewidth . . . 423
24.6.1 Exact, when treewidth is small: Bodlaender's theorem . . . 423
24.6.2 A clean constant-factor approximation . . . 423
24.6.3 Practical heuristics: elimination orderings . . . 424
24.7 Treewidth and Fixed-Parameter Tractability . . . 426

24.8 Pathwidth, Branchwidth, and the Wider Family . . . 426
24.9 Exercises . . . 427

25 Fixed-Parameter Tractability on Graphs 429

25.1 Parameterized problems . . . 429
25.2 A graph to carry through the chapter . . . 431
25.3 Bounded search trees: vertex cover in $O(2^k(n+m))$ . . . 431
25.3.1 The branching rule . . . 431
25.3.2 Trace on the running graph . . . 432
25.4 Kernelization: shrink before you search . . . 434
25.4.1 Buss's elementary kernel . . . 435
25.4.2 The crown reduction . . . 436
25.5 Iterative compression: feedback vertex set . . . 438
25.5.1 The compression framework . . . 438
25.5.2 The compression step . . . 438
25.5.3 Trace iterative compression on the running graph . . . 440
25.6 Color coding: the longest path . . . 441
25.6.1 Colorful-path dynamic programming . . . 441
25.6.2 Trace color coding by hand . . . 442
25.7 When nothing works: the W-hierarchy . . . 443
25.8 Exercises . . . 446

V Structural Graph Theory 449

26 Planar Graphs and Planarity Testing 451

26.1 Drawings, embeddings, and faces . . . 452
26.2 Euler's formula . . . 453
26.2.1 The sparsity bounds . . . 454
26.3 Duality . . . 455
26.4 The forbidden minors: Kuratowski and Wagner . . . 456
26.5 Planarity testing . . . 458
26.5.1 Intuition: add one path at a time . . . 458
26.5.2 A clean recursive scheme . . . 459

26.6 Coloring planar graphs . . . 460
26.7 Planar separators . . . 462
26.8 Summary of planar graph properties . . . 464
26.9 Exercises . . . 465

27 Separators and Divide-and-Conquer on Graphs **467**
27.1 What a separator is, and why balance matters . . . 468
27.1.1 Trees: a warm-up . . . 469
27.1.2 General graphs have no small separators . . . 469
27.2 A grid by hand: the canonical $\sqrt{n}$ separator . . . 470
27.3 The Planar Separator Theorem . . . 471
27.3.1 Step 0: reductions . . . 471
27.3.2 Step 1: BFS levels are separators . . . 471
27.3.3 Step 2: a thin level must exist near the middle . . . 473
27.3.4 Step 3: a fat middle band, cut by a tree path . . . 473
27.4 Separator-driven recurrences . . . 475
27.5 Application 1: planar shortest paths and nested dissection . . . 476
27.5.1 Nested dissection for sparse linear systems . . . 476
27.5.2 Planar single-source shortest paths . . . 477
27.5.3 A tiny worked trace . . . 478
27.6 Application 2: sub-exponential algorithms on planar graphs . . . 478
27.7 Application 3: graph isomorphism on planar graphs . . . 480
27.8 Separators from treewidth . . . 480
27.8.1 Balanced separators from bounded treewidth . . . 481
27.8.2 Treewidth DP: bounded width gives exact poly-time . . . 482
27.9 Tradeoffs: size, balance, findability, and gain . . . 483
27.10Exercises . . . 485

28 Graph Coloring **487**
28.1 Proper Colorings and the Chromatic Number . . . 488
28.2 The Greedy Algorithm and Degeneracy . . . 489
28.3 Bipartite Graphs: Two Colors and a Certificate . . . 491
28.4 Brooks's Theorem . . . 492

28.5 Edge Coloring and Vizing's Theorem . . . 494
28.6 Chordal and Interval Graphs: Optimal Coloring . . . 496
28.7 Coloring Is NP-Hard . . . 498
28.8 Register Allocation and Scheduling . . . 499
28.9 Exercises . . . 501

29 Independent Sets, Cliques, and Vertex Covers 503

29.1 Three Problems and Their Complements . . . 504
29.1.1 Independent set = clique in the complement . . . 505
29.1.2 Gallai: cover and independent set partition the vertices . . . 505
29.2 Enumerating Cliques: Bron–Kerbosch . . . 507
29.2.1 The Pivot: Pruning Redundant Search Branches . . . 507
29.2.2 Tracing it on G_0 . . . 509
29.2.3 Complexity . . . 510
29.3 Vertex Cover: The LP Relaxation and Half-Integrality . . . 511
29.3.1 Reading the half-integral LP on a triangle . . . 512
29.4 Hardness and the Factor-2 Approximation . . . 513
29.4.1 Why these problems are hard . . . 513
29.4.2 The matching lower bound and the greedy 2-approximation . . . 513
29.4.3 The LP-rounding 2-approximation . . . 515
29.5 The Parameterized Angle: Vertex Cover Is FPT . . . 515
29.5.1 Kernelization: shrinking before you branch . . . 516
29.6 Solving the Running Example . . . 517
Exercises . . . 519

30 Perfect Graphs and Structure 521

30.1 What perfection means . . . 522
30.2 The complement, and the Perfect Graph Theorem . . . 523
30.3 Three structured families that are perfect . . . 525
30.4 Chordal graphs: simplicial vertices and elimination orderings . . . 527
30.5 Finding a PEO in linear time: Lexicographic BFS . . . 529
30.6 A worked chordal graph, by hand . . . 531
30.7 Linear-time coloring and maximum clique on a PEO . . . 532

30.8 Maximum independent set and minimum clique cover on a PEO . . . 534
30.9 Why structure gives polynomial time . . . 536
Exercises . . . 537

VI Spectral and Algebraic Graph Theory 539

31 The Graph Laplacian 541
31.1 Three definitions, one matrix . . . 543
31.1.1 The incidence matrix and $\mathbf{L} = \mathbf{A}_B\mathbf{A}_B^\top$ . . . 543
31.1.2 The sum-over-edges form . . . 545
31.2 Smoothness: what $x^\top \mathbf{L}x$ measures . . . 545
31.3 The null space and connected components . . . 547
31.3.1 An algorithm: counting components by Laplacian rank . . . 548
31.4 Weighted Laplacians . . . 549
31.4.1 Cuts as quadratic forms . . . 550
31.5 Eigenvalues of small graphs, by hand . . . 551
31.6 Basic spectral facts to carry forward . . . 552
31.6.1 The normalized Laplacian, briefly . . . 553
31.7 A worked solve: harmonic extension . . . 554
Exercises . . . 556

32 Spectral Graph Theory 559
32.1 Setup: which matrix, which spectrum . . . 560
32.1.1 The variational principles we will reuse . . . 561
32.2 Reading graph structure off the spectrum . . . 562
32.2.1 The kernel: connectivity and components . . . 562
32.2.2 The top of the spectrum and bipartiteness . . . 563
32.2.3 A first worked spectrum: the path P_4 . . . 563
32.2.4 Two more reference spectra . . . 564
32.3 The second eigenvalue: algebraic connectivity . . . 565
32.3.1 Why λ_2 deserves the name "connectivity" . . . 565
32.3.2 The Fiedler vector as an embedding . . . 565
32.4 The Cheeger inequality . . . 566

32.4.1 Conductance: the combinatorial quantity . . . 566
32.4.2 Statement . . . 566
32.4.3 Proof of the easy direction . . . 567
32.4.4 Proof sketch of the hard direction (the sweep) . . . 568
32.5 Worked example: Cheeger on a two-cluster graph . . . 569
32.6 Higher eigenvalues and many-way partitions . . . 573
32.7 Stability: how much does the spectrum move? . . . 573
32.8 Summary . . . 574
32.9 Exercises . . . 575

33 Spectral Clustering and Partitioning **577**
33.1 The cut we wish we could compute . . . 578
33.2 Bisection by the Fiedler vector . . . 580
33.3 Why the relaxation is good: gaps and Cheeger . . . 583
33.4 Normalized cuts and the random-walk view . . . 584
33.5 k-way spectral clustering . . . 585
33.6 Complexity . . . 587
33.7 Building the graph: from data to similarity . . . 588
33.8 Historical Context and Controversies . . . 588
33.8.1 What We Can Now Do and Its Limitations . . . 589
33.9 Exercises . . . 590

34 Expander Graphs **593**
34.1 Two ways to measure a bottleneck . . . 593
34.2 The spectral viewpoint and Cheeger's inequality . . . 596
34.2.1 The adjacency spectrum and the second eigenvalue . . . 598
34.3 The Expander Mixing Lemma . . . 599
34.4 Random graphs are expanders . . . 601
34.5 Explicit constructions, conceptually . . . 602
34.6 Why expanders matter . . . 603
34.6.1 Fast mixing of random walks . . . 603
34.6.2 Robust networks . . . 604
34.6.3 Derandomization . . . 604

34.6.4 Error-correcting codes . . . 605
34.7 A fully worked example . . . 606
34.8 Historical Context and Controversies . . . 608
34.8.1 What We Can Now Do and Its Limitations . . . 608
34.9 Exercises . . . 609

35 Random Walks and Electrical Networks 611
35.1 The simple random walk and its transition matrix . . . 612
35.1.1 The stationary distribution . . . 613
35.2 Hitting times and cover times . . . 614
35.2.1 Hitting times by first-step analysis . . . 614
35.2.2 Bounds on hitting and cover times . . . 615
35.3 The electrical-network analogy . . . 616
35.3.1 Voltages are harmonic functions . . . 617
35.3.2 Effective resistance and energy . . . 618
35.3.3 Resistance via the Laplacian pseudoinverse . . . 619
35.4 Commute time equals $2m$ times resistance . . . 620
35.4.1 Foster's theorem: a global resistance identity . . . 621
35.5 Computing the quantities: an algorithm . . . 622
35.6 Mixing time and the spectral gap . . . 622
35.7 The full picture on one network . . . 623
35.8 Historical Context and Controversies . . . 624
35.8.1 What We Can Now Do and Its Limitations . . . 625
Exercises . . . 625

36 PageRank and Markov Chains on Graphs 627
36.1 The web as a graph . . . 627
36.2 The random-surfer model . . . 628
36.2.1 The transition matrix . . . 628
36.2.2 Two diseases: dangling nodes and traps . . . 629
36.3 PageRank as a stationary distribution . . . 629
36.3.1 Why a unique stationary distribution exists . . . 630
36.3.2 Handling dangling nodes without densifying . . . 631

36.4 Power iteration and convergence . . . 631
36.4.1 The convergence rate is the damping factor . . . 632
36.4.2 The same thing as a linear solve . . . 633
36.5 Tracing PageRank by hand . . . 634
36.6 Personalized PageRank . . . 635
36.6.1 Computing PPR locally: push . . . 636
36.6.2 Tracing push by hand . . . 637
36.7 Spam and robustness . . . 638
36.7.1 The link farm . . . 638
36.7.2 Defenses . . . 639
36.8 The linear-algebra and Markov views, side by side . . . 639
36.9 Historical Context and Controversies . . . 640
36.9.1 What We Can Now Do and Its Limitations . . . 641
36.10Exercises . . . 642

VII Hardness, Approximation, and the Modern Frontier 643

37 NP-Hard Graph Problems and Reductions 645

37.1 What Hardness Means . . . 645
37.2 The Anchor: 3-SAT . . . 647
37.3 Independent Set, Vertex Cover, and Clique . . . 647
37.3.1 The gadget . . . 648
37.4 Graph Coloring . . . 650
37.5 Hamiltonian Cycle and the Traveling Salesman . . . 651
37.6 Longest Simple Path . . . 653
37.7 Max-Cut and Steiner Tree . . . 654
37.8 A Map of the Reductions . . . 656
37.9 The Boundary Between Easy and Hard . . . 656
37.9.1 2-Coloring vs 3-Coloring . . . 656
37.9.2 2-SAT vs 3-SAT . . . 657
37.9.3 Shortest vs Longest Path . . . 657
37.9.4 Min-Cut vs Max-Cut . . . 658

37.9.5 Matching vs 3-Dimensional Matching . . . 658
37.10 What Hardness Means for Algorithm Design . . . 658
37.11 Historical Context and Controversies . . . 659
37.11.1 What We Can Now Do and Its Limitations . . . 660
37.12 Exercises . . . 661

38 Approximation Algorithms on Graphs **663**

38.1 What an approximation guarantee means . . . 663
38.2 Vertex cover: a 2-approximation three ways . . . 664
38.2.1 The maximal-matching algorithm . . . 664
38.2.2 The LP relaxation and rounding . . . 666
38.3 Set cover and the greedy $\ln n$ bound . . . 667
38.4 Metric TSP, the double-tree bound, and Christofides . . . 668
38.4.1 The MST lower bound and double-tree . . . 669
38.4.2 Christofides: fixing the parity instead of doubling . . . 669
38.5 Steiner tree: the metric closure and double-spanning-tree . . . 671
38.6 Max-cut: greedy, local search, and SDP rounding . . . 673
38.6.1 Greedy and local search: factor one half . . . 673
38.6.2 Goemans–Williamson: the 0.878 SDP rounding . . . 673
38.7 Sparsest cut, conductance, and the Leighton–Rao and ARV bounds . . . 676
38.7.1 The spectral bound (Cheeger) and where it fails . . . 676
38.7.2 Leighton–Rao: LP, multicommodity flow, and $O(\log n)$ . . . 676
38.7.3 ARV: the SDP and $O(\sqrt{\log n})$ . . . 677
38.8 A unifying picture . . . 678
38.9 Historical Context and Controversies . . . 679
38.9.1 What We Can Now Do and Its Limitations . . . 680
38.10 Exercises . . . 680

39 Dynamic Graph Algorithms **683**

39.1 A taxonomy of "dynamic" . . . 683
39.2 Warm-up: incremental connectivity and the limits of union-find . . . 684
39.3 Euler-tour trees: connectivity in a forest . . . 685
39.4 Fully dynamic connectivity: the HdLT logarithmic levels . . . 688

39.5 Dynamic minimum spanning tree . . . 693
39.6 Sparsification: trading m for n . . . 695
39.7 Worst-case versus amortized, and a glance at harder problems . . . 695
39.8 Conditional lower bounds: the OMv wall . . . 696
39.9 Putting it together . . . 698
39.10 Historical Context and Controversies . . . 699
39.10.1 What We Can Now Do and Its Limitations . . . 700
39.11 Exercises . . . 700

40 Streaming and Massive Graphs **703**
40.1 The streaming model, precisely . . . 703
40.1.1 A running graph to keep in mind . . . 704
40.2 Connectivity in one pass . . . 704
40.2.1 Bipartiteness and k-edge-connectivity . . . 706
40.3 Spanners: approximate distances in small space . . . 706
40.4 Dynamic streams: when edges get deleted . . . 709
40.4.1 The vertex–edge incidence vector . . . 709
40.4.2 L_0 sampling . . . 710
40.4.3 Connectivity in dynamic streams (AGM sketching) . . . 710
40.5 Approximate triangle counting . . . 712
40.5.1 A clean sampling estimator (one pass) . . . 712
40.5.2 Frequency moments and the AMS sketch . . . 714
40.6 What small space cannot do . . . 715
40.6.1 Summary of the tradeoffs . . . 716
40.7 Historical Context and Controversies . . . 717
40.7.1 What We Can Now Do and Its Limitations . . . 718
40.8 Exercises . . . 719

41 Laplacian Solvers and Max-Flow via Electrical Flows **721**
41.1 The running network . . . 722
41.2 Electrical flows as energy minimization . . . 722
41.2.1 Effective resistance . . . 724
41.3 Solving $\mathbf{L}x = b$ in nearly linear time . . . 725

41.3.1 Preconditioned conjugate gradient . . . 725
41.3.2 Low-stretch spanning trees: the first preconditioner . . . 726
41.3.3 From a tree to an ultrasparsifier . . . 727
41.4 Spectral sparsification . . . 729
41.5 Maximum flow via a sequence of electrical flows . . . 730
41.5.1 Why electrical flow is the right relaxation . . . 730
41.5.2 The multiplicative-weights outer loop . . . 731
41.5.3 The width problem and the $m^{1/3}$ fix . . . 732
41.6 Applications of near-linear Laplacian solvers . . . 733
41.7 Historical Context and Controversies . . . 734
41.7.1 What We Can Now Do and Its Limitations . . . 735
Exercises . . . 735

VIII Background **737**

42 Appendix: Math and Data-Structure Background **739**
42.1 Asymptotics: Big-O, Big-Theta, Big-Omega . . . 739
42.1.1 A hierarchy to memorize . . . 741
42.2 Amortized Analysis . . . 741
42.3 Union-Find (Disjoint-Set Union) . . . 742
42.3.1 Why it is fast . . . 744
42.4 Heaps and Priority Queues . . . 745
42.5 Fenwick Trees (Binary Indexed Trees) . . . 747
42.6 Segment Trees . . . 748
42.6.1 Lazy propagation . . . 750
42.7 Linear Algebra for the Spectral Chapters . . . 751
42.7.1 Symmetric matrices and the spectral theorem . . . 751
42.7.2 Positive semidefiniteness and the Rayleigh quotient . . . 752
42.8 Probability for the Randomized Chapters . . . 753
42.8.1 Expectation, linearity, and variance . . . 753
42.8.2 Concentration: from average to almost-sure . . . 754
42.9 Historical Context, Controversies, and the Unified Curriculum . . . 755

42.9.1 Summary of the Book and Companion Volumes 756
42.10Exercises 757

Part I

Foundations

Chapter 1

Graphs and How We Represent Them

A graph is about as spare an object as computing has: a set of things and a set of relationships between them. In the preface we watched how a traversal explores outward along connected vertices. Before we can analyze or execute any such algorithm, however, we must decide how vertices and relationships are defined mathematically and how they live inside computer memory. A bad representation can turn a linear-time algorithm into a quadratic one without changing a single line of its logic. This opening chapter establishes our core vocabulary, compares the standard data structures for graphs, and fixes the asymptotic cost model that governs every running time in this book.

1.1 What a graph is

Intuition

Draw some dots. Connect some of them with lines. You now have a graph. The dots are *vertices* (or nodes), the lines are *edges*. Everything else — direction, weight, the prohibition or tolerance of repeated edges — is a decision you make about what the dots and lines are allowed to mean. The art of graph modeling is choosing those meanings so that the question you care about becomes a standard graph question with a known answer.

Definition 1.1 (Undirected graph)**.** An *undirected graph* is a pair $G = (V, E)$ where V is a finite set of *vertices* and E is a set of *edges*, each edge being a two-element subset $\{u, v\} \subseteq V$ with $u \neq v$. We write $n = |V|$ and $m = |E|$.

The requirement $u \neq v$ rules out *self-loops* (an edge from a vertex to itself), and using a *set* of edges rules out *parallel edges* (two distinct edges joining the same pair). A graph with neither is called *simple*. When we want loops or parallel edges back we will say *multigraph*; for now, simple is the default, and unless a chapter says otherwise, "graph" means "simple graph."

Definition 1.2 (Directed graph)**.** A *directed graph* (or *digraph*) is a pair $G = (V, E)$ where $E \subseteq V \times V$ is a set of ordered pairs. An edge (u, v) points *from* its *tail* u *to* its *head* v. We draw it as an arrow $u \to v$.

In a digraph the pair is ordered, so (u, v) and (v, u) are different edges and both may be present. The two pictures below show the distinction. On the left an undirected edge between a and b; on

the right two directed edges, one each way, which together carry the same connectivity but twice the bookkeeping.



undirected $\{a, b\}$ directed $(a, b), (b, a)$

Definition 1.3 (Weighted graph)**.** A *weighted graph* adds a function $w : E \to \mathbb{R}$ (or to $\mathbb{R}_{\geq 0}$, or to $\mathbb{Z}$, depending on the setting) assigning each edge a number. We write $w(u, v)$ or w_e. Weights model distance, cost, capacity, similarity, resistance — whatever the application measures along an edge.

Weight is orthogonal to direction: there are weighted undirected graphs (road networks with mileage) and weighted directed graphs (currency exchange rates). Whether weights may be negative matters enormously — Dijkstra needs them nonnegative, Bellman–Ford tolerates negatives but not negative cycles — and we will be careful about that in the shortest-path chapters.

Definition 1.4 (Multigraph)**.** A *multigraph* replaces the edge *set* by an edge *multiset*, or equivalently by a set E of edge identities together with an *endpoint map* ∂ sending each edge to its (un)ordered pair of endpoints. This permits parallel edges and, if we also drop the $u \neq v$ restriction, self-loops.

The endpoint-map formulation is worth a second look because it is the clean way to talk about parallel edges. If three separate cables run between routers u and v, they are three distinct edges e_1, e_2, e_3 with $\partial e_i = \{u, v\}$ for each i. They are not "the same edge"; a flow algorithm can saturate one and leave the others open. Treating E as a plain subset of pairs would silently merge them, which is exactly the bug that makes a naive max-flow implementation under-count capacity.

Formal

Counting conventions. Throughout the book, $n = |V|$ and $m = |E|$. For a multigraph m counts parallel edges with multiplicity. We freely assume $V = \{0, 1, \ldots, n-1\}$ so vertices index into arrays in $O(1)$; relabeling an arbitrary vertex set to $\{0, \ldots, n-1\}$ costs one hash pass, $O(n)$ expected, and we pay it once up front.

1.1.1 Degrees and the handshake lemma

Definition 1.5 (Degree)**.** In an undirected graph the *degree* $\deg(v)$ of a vertex v is the number of edges incident to it (a self-loop, when allowed, counts twice). In a digraph the *in-degree* $\deg^-(v)$ is the number of edges with head v and the *out-degree* $\deg^+(v)$ is the number with tail v.

The single most-used counting fact about graphs is also the first one anybody proves. It is sometimes called the *degree-sum formula*.

Theorem 1.1 (Handshake lemma)**.** *For any undirected multigraph $G = (V, E)$,*

$$\sum_{v \in V} \deg(v) = 2m.$$

Consequently the number of vertices of odd degree is even.

Why it works

The idea is double counting. Build a table with one row per (vertex, incident edge) pair and count it two ways. Summing across rows grouped by vertex gives $\sum_v \deg(v)$. Summing grouped by edge: every edge $\{u, v\}$ touches exactly two vertices, so it contributes exactly 2 rows. The two totals must agree.

Proof. Let $I = \{(v, e) : v \in V,\ e \in E,\ v \text{ is an endpoint of } e\}$ be the set of *incidences*. Count $|I|$ in two ways. Fixing v first, the number of incidences with first coordinate v is exactly $\deg(v)$ (a self-loop contributes 2, matching the degree convention), so $|I| = \sum_v \deg(v)$. Fixing e first, every edge has exactly two endpoints, so it appears in exactly two incidences, giving $|I| = 2m$. Hence $\sum_v \deg(v) = 2m$.

For the parity corollary, split the sum into even-degree and odd-degree vertices:

$$2m = \sum_{\deg(v) \text{ even}} \deg(v) + \sum_{\deg(v) \text{ odd}} \deg(v).$$

The left side is even and the first right-hand sum is even, so the sum over odd-degree vertices is even. A sum of odd numbers is even only when there are an even count of them. □

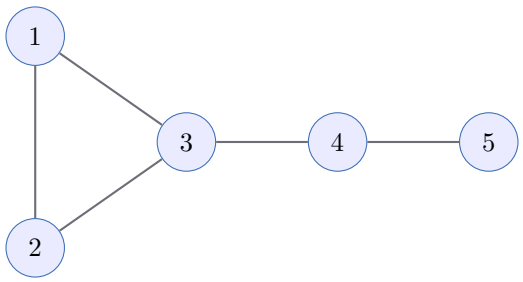
The directed analogue is even simpler and just as useful: every edge has one tail and one head, so

$$\sum_{v \in V} \deg^+(v) \;=\; \sum_{v \in V} \deg^-(v) \;=\; m.$$

We will lean on these identities constantly when we bound running times, because $\sum_v \deg(v) = 2m$ is precisely what lets "do $O(1)$ work per incident edge at each vertex" collapse to a clean $O(n + m)$ total instead of $O(n^2)$.

Trace it yourself

Take the five-vertex graph $V = \{1, 2, 3, 4, 5\}$ with edges $\{1, 2\}, \{1, 3\}, \{2, 3\}, \{3, 4\}, \{4, 5\}$. Read off the degrees: $\deg(1)$=2, $\deg(2)$=2, $\deg(3)$=3, $\deg(4)$=2, $\deg(5)$=1. Their sum is $10 = 2{\cdot}5 = 2m$. Two vertices (3 and 5) have odd degree — an even count, as the lemma promised. This is the running example for the whole chapter; keep it nearby.



1.2 Walks, paths, cycles, and connectivity

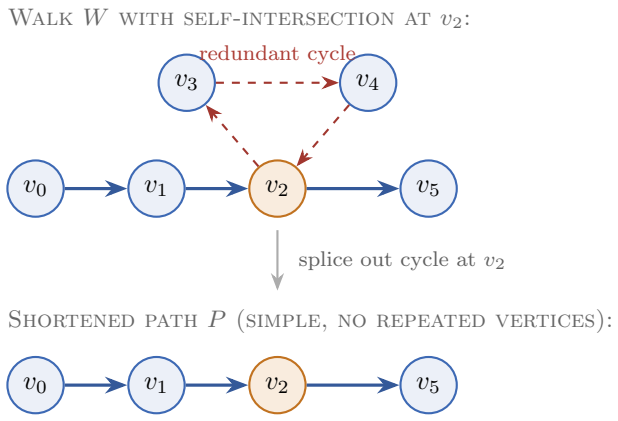
Definition 1.6 (Walk, trail, path)**.** A *walk* of length k is a sequence $v_0, e_1, v_1, e_2, \ldots, e_k, v_k$ where each e_i joins v_{i-1} and v_i (and points $v_{i-1} \to v_i$ in the directed case). A *trail* is a walk with no repeated edge. A *path* is a walk with no repeated vertex (hence no repeated edge). A walk or path is *closed* if $v_0 = v_k$; a closed path of length ≥ 1 (in the directed case) or length ≥ 3 (in the simple undirected case) is a *cycle*.

The distinctions matter more than they look. "There is a walk from s to t" and "there is a path from s to t" are logically equivalent — any walk can be shortened to a path by deleting the loop between two visits to the same vertex — but algorithms produce one or the other, and proofs about lengths care which. The next lemma gives us the basic shortening tool.

Lemma 1.1 (Walk-to-path shortening)**.** *If there is a walk from u to v, then there is a path from u to v, and a shortest such walk is already a path.*

Proof. Take any walk $W = v_0 \ldots v_k$ from $u = v_0$ to $v = v_k$. If some vertex repeats, say $v_i = v_j$ with $i < j$, splice out the closed segment: the shorter walk $v_0 \ldots v_i v_{j+1} \ldots v_k$ still goes from u to v. Repeat until no vertex repeats; the process terminates because each splice strictly shortens W. What remains is a path. For the second claim, a *shortest* walk admits no such splice (the spliced walk would be shorter and still valid), so it has no repeated vertex and is already a path. □

Geometrically, the proof excises each redundant detour loop at a repeated vertex until only a simple path remains:



This is why breadth-first search, which by construction explores along walks of increasing length, returns genuine shortest *paths* in unweighted graphs: the shortest walk is automatically simple.

Definition 1.7 (Connectivity)**.** An undirected graph is *connected* if every pair of vertices is joined by a path. Its maximal connected subgraphs are its *connected components*. A digraph is *strongly connected* if for every ordered pair (u, v) there is a directed path $u \to \cdots \to v$; its maximal strongly connected subgraphs are its *strongly connected components* (SCCs, the subject of Chapter 5).

"Reachable from" — the existence of a walk, equivalently a path — is an equivalence relation in the undirected case (reflexive, symmetric because edges go both ways, transitive by concatenating walks),

and its equivalence classes are exactly the connected components. In the directed case symmetry fails, so reachability is only a preorder, and recovering the symmetric part of it is what makes SCCs subtler than components.

1.3 Trees, forests, and DAGs

Three special shapes recur so often that they earn their own names and their own characterization theorems.

Definition 1.8 (Forest, tree)**.** A *forest* is an undirected graph with no cycle (it is *acyclic*). A *tree* is a connected forest. A vertex of degree 1 in a tree is a *leaf*.

Theorem 1.2 (Tree characterizations)**.** *For an undirected graph T on $n \geq 1$ vertices, the following are equivalent.*

(i) *T is a tree (connected and acyclic).*

(ii) *T is connected and has exactly $n-1$ edges.*

(iii) *T is acyclic and has exactly $n-1$ edges.*

(iv) *Every two vertices are joined by a* unique *path.*

Why it works

The hub fact is: *a connected graph on n vertices has at least $n-1$ edges, with equality exactly when it is acyclic.* Connectivity forces enough edges to tie n vertices together; a cycle is a redundant edge you could delete without disconnecting anything. Trees sit exactly at the boundary — connected but with not one edge to spare.

Proof. We show (i)⇒(iv)⇒(ii)⇒(iii)⇒(i).

(i)⇒(iv). Connectivity gives at least one path between any u, v. If there were two distinct paths $P \neq Q$ from u to v, then walking along P and back along Q reversed gives a non-trivial closed walk; taking the first edge where P and Q diverge and the next vertex where they re-meet extracts a cycle, contradicting acyclicity. So the path is unique.

(iv)⇒(ii). Unique paths imply connected. For the edge count, root the tree at any vertex and map each non-root vertex to its *parent edge* (the first edge on the unique path toward the root). This is an injection from the $n-1$ non-root vertices into E, and it is a surjection because every edge $\{x, y\}$ is the parent edge of whichever of x, y is farther from the root. Hence exactly $n-1$ edges.

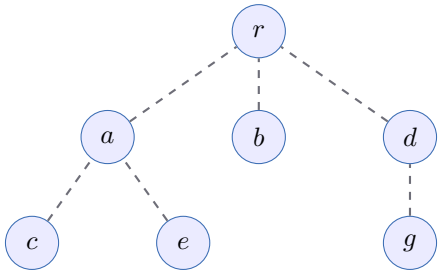
(ii)⇒(iii). Suppose T is connected with $n-1$ edges but contains a cycle. Deleting one cycle edge keeps the graph connected (the rest of the cycle gives a detour), leaving a connected graph on n vertices with $n-2$ edges. But a connected graph needs at least $n-1$ edges: starting from n singleton components and adding edges one at a time, each edge lowers the component count by at most one, so reaching a single component takes at least $n-1$ edges. Contradiction, so T is acyclic.

(iii)⇒(i). Suppose T is acyclic with $n-1$ edges. Let it have c components with sizes $n_1, \ldots, n_c$. Each component is a connected acyclic graph, i.e. a tree, so by the count just established it has $n_i - 1$ edges. Total edges $= \sum_i (n_i - 1) = n - c$. Setting this equal to $n-1$ forces $c = 1$: connected. □

Corollary 1.1 (Forest edge count)**.** *A forest with n vertices and c components has exactly $n - c$ edges. Every tree on ≥ 2 vertices has at least two leaves.*

Proof. The edge count fell out of the proof above. For leaves, if a tree had at most one leaf then at most one vertex has degree 1 and all others have degree ≥ 2 (degree 0 is impossible in a connected graph on ≥ 2 vertices), so $\sum_v \deg(v) \geq 2(n-1) + 1 = 2n - 1$. By the handshake lemma this sum equals $2(n-1) = 2n - 2 < 2n - 1$, a contradiction. □

The figure below makes the counting argument concrete on a seven-vertex tree rooted at r. Each non-root vertex owns the dashed parent edge that points toward r, so the six parent edges match the six non-root vertices one for one — the injection of part (iv)⇒(ii). The four leaves $\{b, c, e, g\}$ sit at the degree-1 fringe, comfortably more than the two the corollary guarantees.



Now the directed cousin.

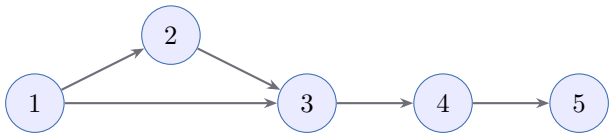
Definition 1.9 (DAG)**.** A *directed acyclic graph* (DAG) is a digraph with no directed cycle. A *topological order* of a digraph is a linear ordering of V such that every edge (u, v) has u before v.

Theorem 1.3 (DAG iff topologically orderable)**.** *A digraph admits a topological order if and only if it is a DAG.*

Proof. (⇒) If a topological order exists, any directed cycle $v_0 \to v_1 \to \cdots \to v_k = v_0$ would force v_0 before v_1 before $\cdots$ before v_0 in the order, impossible. So no cycle: a DAG.

(⇐) Suppose G is acyclic. We claim some vertex has in-degree 0. If not, every vertex has an incoming edge; follow edges backward from any start, and after n steps you have visited $n+1$ vertices, so two coincide, producing a directed cycle — contradiction. Place a source s (in-degree 0) first, delete it, and recurse on the remaining graph; deleting a vertex cannot create a cycle, so the recursion stays valid. This builds a full topological order. □

The constructive direction is exactly Kahn's algorithm, which Chapter 4 turns into linear-time pseudocode. The picture below shows the same five vertices oriented into a DAG and one valid topological order written left to right.



topological order: 1, 2, 3, 4, 5

1.4 Bipartite graphs

Definition 1.10 (Bipartite)**.** A graph is *bipartite* if its vertices can be split into two sets X and Y (the *sides*) so that every edge has one endpoint in X and the other in Y. Equivalently, the vertices can be 2-colored so no edge is monochromatic.

Bipartite graphs model anything with a natural two-sided structure: jobs and workers, students and courses, the two halves of a matching problem. The clean test for bipartiteness is one of the prettiest small theorems in the book.

Theorem 1.4 (Odd-cycle characterization)**.** *A graph is bipartite if and only if it contains no cycle of odd length.*

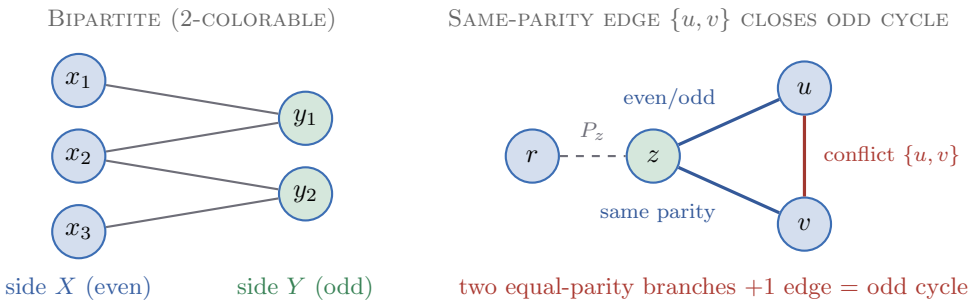
Why it works

Walk around a cycle alternating colors $X, Y, X, Y, \ldots$. To come back to your starting color you must take an even number of steps. So an odd cycle can never be properly 2-colored, and conversely if there is no odd cycle the alternating coloring never contradicts itself.

Proof. ($\Rightarrow$) In a 2-coloring the colors alternate along any walk, so a closed walk returns to its start color only after evenly many steps. Every cycle is a closed walk, hence even.

($\Leftarrow$) Assume no odd cycle; we 2-color each connected component independently. Fix a component, pick a root r, and color each vertex v by the parity of $\operatorname{dist}(r, v)$ (the length of a shortest path, well defined within a connected component). Suppose for contradiction some edge $\{u, v\}$ is monochromatic, i.e. $\operatorname{dist}(r, u)$ and $\operatorname{dist}(r, v)$ have the same parity. Take shortest paths P_u from r to u and P_v from r to v; let z be their last common vertex. The two tails from z to u and z to v have lengths $\operatorname{dist}(r, u) - \operatorname{dist}(r, z)$ and $\operatorname{dist}(r, v) - \operatorname{dist}(r, z)$, of the same parity, so their sum is even; adding the edge $\{u, v\}$ closes a cycle of *odd* length. Contradiction. So no edge is monochromatic and the graph is bipartite. □

This proof is constructive: run a BFS, color by distance parity, and check every edge. That is a linear-time bipartiteness test, and we will reuse the exact idea when BFS gets its own chapter. The two panels below contrast a valid 2-coloring with the odd cycle closed whenever an edge $\{u, v\}$ joins two vertices of the same distance parity from r:



1.5 Three ways to store a graph

We now leave the mathematics and confront the engineering. The same abstract graph (V, E) can be packed into memory in several ways, and the choice governs how fast every later algorithm runs. The three standard structures are the *adjacency matrix*, the *adjacency list*, and the *edge list*. We measure each on four operations that algorithms perform constantly:

- **space**: total memory;
- **edge query** `has_edge(u,v)`: is (u, v) present?
- **neighbor scan**: enumerate all neighbors of a given v;
- **global scan**: enumerate all edges once.

We run all three on the running five-vertex graph $\{1, 2\}, \{1, 3\}, \{2, 3\}, \{3, 4\}, \{4, 5\}$ so you can see the bytes.

1.5.1 Adjacency matrix

Definition 1.11 (Adjacency matrix)**.** The *adjacency matrix* of G on vertex set $\{0, \dots, n-1\}$ is the $n \times n$ matrix $\mathbf{A}$ with $\mathbf{A}_{uv} = 1$ if $(u, v) \in E$ and 0 otherwise (for a weighted graph, store $w(u, v)$ in place of 1, with a sentinel such as $+\infty$ or 0 for absence). For an undirected graph $\mathbf{A}$ is symmetric; for a multigraph store the multiplicity.

For our graph (with vertices $1 \dots 5$ indexing rows and columns), the undirected adjacency matrix is

$$\mathbf{A} = \begin{pmatrix} 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 \end{pmatrix}.$$

Notice the symmetry, and notice the row sums: row i sums to $\deg(i)$, and the grand total of all entries is $\sum_i \deg(i) = 2m = 10$, the handshake lemma showing up as a matrix identity.

Focus

The adjacency matrix turns graph questions into linear algebra. The entry $(\mathbf{A}^k)_{uv}$ counts walks of length exactly k from u to v (prove it by induction: $(\mathbf{A}^k)_{uv} = \sum_z (\mathbf{A}^{k-1})_{uz}\mathbf{A}_{zv}$ is exactly "length-$(k-1)$ walks to a neighbor of v"). This observation is the starting point for the spectral half of the book — eigenvalues of $\mathbf{A}$ and of the Laplacian $\mathbf{L} = \mathbf{D} - \mathbf{A}$ control mixing, expansion, and clustering. See Chapters 31–35.

Costs. Space is $\Theta(n^2)$: every one of the n^2 cells is stored whether or not the edge exists. The edge query is where the matrix excels: `has_edge(u,v)` is a single array lookup, $O(1)$. A neighbor scan, though, is $\Theta(n)$ regardless of how few neighbors v has — you must walk the entire row testing each cell. A global edge scan is $\Theta(n^2)$.

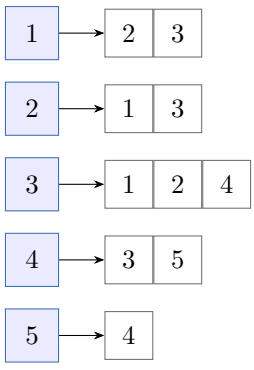
1.5.2 Adjacency list

Definition 1.12 (Adjacency list)**.** The *adjacency list* stores, for each vertex v, a list $\mathrm{Adj}[v]$ of its out-neighbors (in the undirected case, of its neighbors; each undirected edge $\{u, v\}$ appears twice, once in $\mathrm{Adj}[u]$ and once in $\mathrm{Adj}[v]$). For a weighted graph each list entry carries the weight alongside the neighbor.

For our graph:

$$\mathrm{Adj}[1] = [\,2,\,3\,] \quad \mathrm{Adj}[2] = [\,1,\,3\,] \quad \mathrm{Adj}[3] = [\,1,\,2,\,4\,]$$
$$\mathrm{Adj}[4] = [\,3,\,5\,] \quad \mathrm{Adj}[5] = [\,4\,].$$

The total number of list entries is $\sum_v \deg(v) = 2m = 10$: the handshake lemma is now literally the size of the data structure. That is the main reason adjacency lists are the default representation in this book. Drawn as an array of per-vertex headers pointing to their neighbor lists, the five vertices and ten directed incidences look like this:



Costs. Space is $\Theta(n + m)$: one array slot per vertex plus one list entry per edge-endpoint. A neighbor scan of v is $\Theta(1 + \deg(v))$ — you touch exactly the neighbors and nothing else, which is optimal. A global edge scan visits every list once for a total of $\Theta(n + m)$. The weak spot is the edge query: `has_edge(u,v)` must search $\mathrm{Adj}[u]$, costing $O(\deg(u))$ in the worst case (or $O(\log \deg(u))$ if the list is kept sorted, or expected $O(1)$ if you maintain a parallel hash set of edges).

Common trap

The two most common adjacency-list bugs are both about *the two copies of an undirected edge*. (1) Forgetting to add the second copy: you push v into Adj[u] but not u into Adj[v], so BFS from v silently cannot reach u. (2) For algorithms that walk both copies — Euler tours, many flow networks — you must find an edge's *twin* in $O(1)$. The standard trick (used throughout the flow chapters) is to store edges in one array and put copies at indices $2i$ and $2i{+}1$, so the twin of edge e is $e \oplus 1$. Get this wrong and your residual graph leaks capacity.

1.5.3 Edge list

Definition 1.13 (Edge list)**.** The *edge list* is simply the array of edges, each stored as a triple (u, v, w) (or pair (u, v) when unweighted). No per-vertex indexing.

For our graph: $[(1,2),(1,3),(2,3),(3,4),(4,5)]$. That is the whole structure.

Costs. Space $\Theta(n+m)$ (the n is only to know the vertex count). There is no fast neighbor scan or edge query without preprocessing — both are $\Theta(m)$ linear scans. What the edge list is unbeatable at is *processing every edge once, in an order you control.* Kruskal's MST algorithm sorts the edge list by weight and sweeps it; Bellman–Ford relaxes the entire edge list $n-1$ times. For those algorithms the edge list is the most direct representation.

History

The trade-off between matrix and list is older than computer science. The incidence and adjacency matrices go back to the birth of algebraic graph theory in the nineteenth century — Kirchhoff's 1847 work on electrical networks already manipulates what we now call the incidence matrix, and we will meet it again in the Laplacian chapters. Adjacency lists as a deliberate *algorithmic* choice — selected precisely so that running time scales with m rather than n^2 — crystallized in the 1970s with Hopcroft and Tarjan, whose linear-time DFS-based algorithms for planarity and connectivity simply do not work in $O(n^2)$ and forced the field to take representation seriously.

1.5.4 The comparison, side by side

	Adjacency matrix	**Adjacency list**	**Edge list**
space	$\Theta(n^2)$	$\Theta(n+m)$	$\Theta(n+m)$
`has_edge(u,v)`	$O(1)$	$O(\deg u)$	$O(m)$
scan nbrs of v	$\Theta(n)$	$\Theta(1+\deg v)$	$\Theta(m)$
scan all edges	$\Theta(n^2)$	$\Theta(n+m)$	$\Theta(m)$
add an edge	$O(1)$	$O(1)$ (push)	$O(1)$ (push)
delete an edge	$O(1)$	$O(\deg)$	$O(m)$

Intuition

A graph is *dense* when $m = \Theta(n^2)$ and *sparse* when $m = O(n)$ or so. On a dense graph the matrix's $\Theta(n^2)$ space is no worse than the list's $\Theta(n+m) = \Theta(n^2)$, and the $O(1)$ edge query is a real win, so matrices shine. On a sparse graph — which is almost every graph that arises in practice, from road maps to social networks to web crawls — the matrix wastes nearly all of its n^2 cells on zeros, and the adjacency list's $\Theta(n+m)$ is dramatically smaller. **Default to adjacency lists.** Reach for a matrix only when the graph is dense or when an algorithm genuinely needs $O(1)$ random-access edge tests (Floyd–Warshall, some matrix-multiplication shortest-path tricks).

1.5.5 Building an adjacency list in linear time

When graph algorithms meet real hardware, nested pointer-chasing adjacency lists (such as nested vectors or pointer-linked chains) incur significant memory overhead and cache misses. Instead, high-performance graph processing packs the entire graph into two flat, contiguous arrays: the *Compressed Sparse Row* (CSR) representation. An arbitrary edge list can be converted into CSR layout in $\Theta(n+m)$ time without sorting or hashing, using a two-pass counting prefix sum.

Algorithm 2: BUILDCSR: edge list to compressed adjacency list

Input: n, edge list $E = [(u_1, v_1), \ldots, (u_m, v_m)]$ (directed; for undirected, add both (u, v) and (v, u)).

Output: arrays head$[0..n]$ and dest$[0..m-1]$ with Adj$[v]$ = dest[head$[v]$ .. head$[v{+}1]{-}1$].

```
1  for v ← 0 to n do
2      head[v] ← 0;
3  for i ← 1 to m do
4      head[u_i + 1] ← head[u_i + 1] + 1;
   // count out-degrees
5  for v ← 1 to n do
6      head[v] ← head[v] + head[v − 1];
   // prefix sums = block starts
7  cur ← copy(head);
8  for i ← 1 to m do
9      dest[cur[u_i]] ← v_i;
10     cur[u_i] ← cur[u_i] + 1;
11 return (head, dest);
```

Why it works

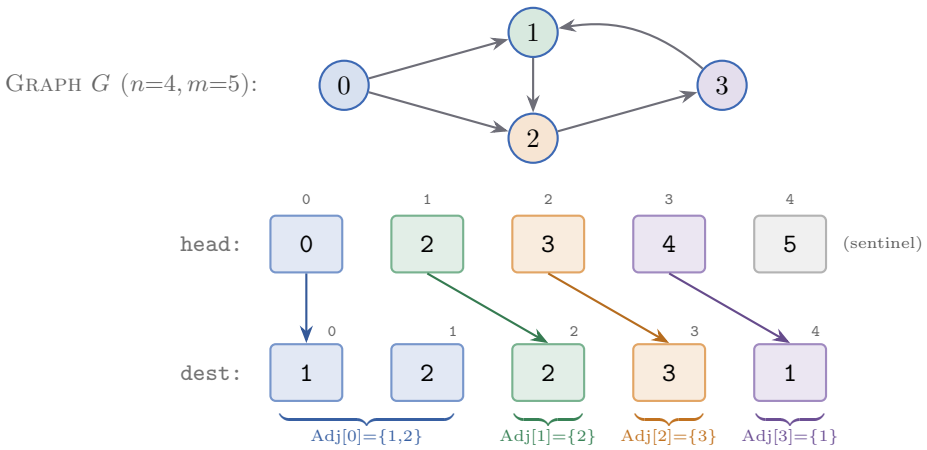
Why it is correct. After the counting loop, head$[u+1]$ holds $\deg^+(u)$. The prefix-sum loop turns these counts into starting offsets: head$[v]$ becomes the index where v's block begins, because the blocks are laid out in vertex order and each occupies $\deg^+$ slots. The placement loop uses cur$[u]$ as a moving write-cursor into u's block; since we increment it after each write and the block has exactly $\deg^+(u)$ slots, the $\deg^+(u)$ edges out of u fill the block exactly, never overflowing into the next vertex's block. Distinct vertices write to disjoint blocks, so nothing

collides. **Why it is fast.** Four loops, each $\Theta(n)$ or $\Theta(m)$, with $O(1)$ work per iteration: $\Theta(n+m)$ total, no comparisons, no hashing.

Trace it yourself

Trace BUILDCSR on the directed edge list $[(0,1),(0,2),(1,2),(2,3),(3,1)]$ with $n=4$. Counting gives out-degrees $\deg^+ = [2,1,1,1]$, so after the count loop head $= [0,2,1,1,1]$ (slot $v{+}1$ holds $\deg^+(v)$). Prefix-summing: head $= [0,2,3,4,5]$. Now place: edge $(0,1)$ writes dest[0]=1, cursor of 0 moves to 1; $(0,2)$ writes dest[1]=2; $(1,2)$ writes dest[2]=2; $(2,3)$ writes dest[3]=3; $(3,1)$ writes dest[4]=1. Final dest $= [1,2,2,3,1]$, and indeed Adj[0] = dest[0..1] $= [1,2]$ and Adj[2] = dest[3..3] $= [3]$. Check it against the edges.

The resulting CSR layout packs all five directed edges contiguously into dest, with each head[v] indexing the start of vertex v's slice:



1.6 Implicit graphs

Sometimes the graph is never stored at all. An *implicit graph* is given by a starting vertex and a function that, asked about any vertex v, returns its neighbors — a successor oracle succ(v) — without any global edge list ever materializing.

Intuition

Think of a Rubik's cube. The "vertices" are the 4.3×10^{19} configurations; the "edges" connect configurations one twist apart. Nobody stores that graph — it would not fit on every hard drive on Earth. But you can still search it, because from any configuration you can *generate* the dozen or so neighbors on demand. The graph exists only as a rule, and BFS or A* explores it one expanded vertex at a time.

Definition 1.14 (Implicit graph)**.** An *implicit graph* is specified by a vertex type, a designated start vertex (or set), and a successor function succ : $V \to 2^V$ computable on demand. The graph (V,E) it represents has $E = \{(v,u) : u \in \text{succ}(v)\}$, but neither V nor E is held in memory; the search visits only the reachable portion it actually touches.

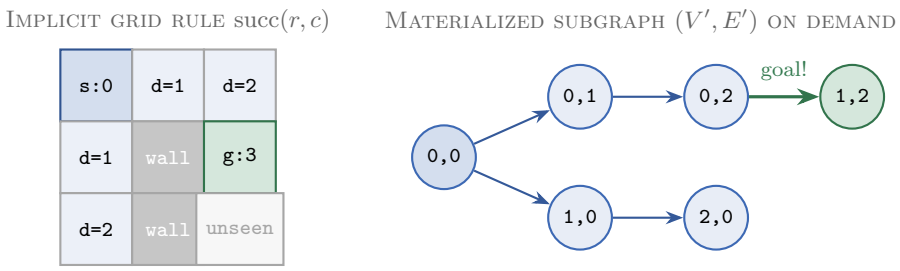
Grid pathfinding is the everyday example: the vertices are cells (r, c) of a maze, and $\text{succ}((r, c))$ returns the up-to-four orthogonal neighbors that are in-bounds and not walls. You would never build an adjacency list; you compute neighbors arithmetically.

Algorithm 3: ImplicitBFS: shortest path on an on-demand graph

Input: start s, goal-test isGoal, successor oracle succ.
Output: the length of a shortest path from s to a goal, or ∞.

```
Q ← empty queue;
seen ← {s};
enqueue (s, 0);
while Q nonempty do
    (v, d) ← dequeue;
    if isGoal(v) then
        return d;
    foreach u ∈ succ(v) do
        if u ∉ seen then
            add u to seen;
            enqueue (u, d + 1);
return ∞;
```

Exploring outward level by level — the principle behind Breadth-First Search, which we develop rigorously in Chapter 2 — works directly on implicit graphs. The only structural adaptation is that seen is maintained as a hash set rather than a length-n array, since n is unknown and the reachable portion may be a tiny fraction of the state space. Just as in standard BFS, states leave the queue in nondecreasing order of distance from s, so the first time a goal is dequeued its associated distance d is the true shortest-path length. The runtime is $O(|V'| + |E'|)$, where V' and E' are the vertices and transitions actually explored — often exponentially smaller than the full state space (V, E). The figure below shows this on a 3×3 grid with two blocked cells: the successor oracle $\text{succ}((r, c))$ generates only valid open neighbors on demand, materializing a small search tree without ever allocating a graph structure for the unvisited region:



Application

Implicit graphs are how search powers puzzle solvers (sliding tiles, Rubik's cube), AI planning (states and actions), game trees (positions and moves), and model checking (program states and transitions). The same BFS/DFS/Dijkstra machinery from the next chapters applies verbatim; you only swap the explicit adjacency structure for a succ oracle. The connection runs the other way too: every explicit graph *is* an implicit graph whose oracle is "look up

Adj[v]."

1.7 The cost model

Every running time in this book is a function of two numbers:

$$n = |V| \quad \text{(vertices)}, \qquad m = |E| \quad \text{(edges)}.$$

We state complexities in terms of both because the relationship between them is where all the interesting trade-offs live. Three facts anchor the whole scale.

Proposition 1.1 (Edge-count bounds)**.** *For a simple undirected graph on n vertices, $0 \le m \le \binom{n}{2} = \frac{n(n-1)}{2}$. For a simple digraph, $0 \le m \le n(n-1)$. If the graph is connected, $m \ge n-1$.*

Proof. The upper bounds count the available vertex pairs (unordered for undirected, ordered for directed; no self-loops). The lower bound for connected graphs is the spanning-tree fact from Theorem 1.2: a connected graph contains a spanning tree, which already has $n-1$ edges. □

Focus

Why we write $O(n+m)$ and not $O(m)$. On a connected graph $m \ge n-1$, so $n+m = \Theta(m)$ and the two are interchangeable. But a graph can have isolated vertices and many components, where m can be as small as 0 while n is large. An algorithm that must at least look at every vertex (BFS initializing a visited array, say) genuinely costs $\Theta(n+m)$, and writing $O(m)$ would be wrong when $m \ll n$. We keep both terms so the bound is correct on *every* input, dense or sparse, connected or shattered.

Common trap

"Linear time" for a graph algorithm means $O(n+m)$, not $O(n)$ and not O(input length) measured in some other unit. An algorithm touching every edge once is linear; one touching every *pair* of vertices is $\Theta(n^2)$ and is linear only on dense graphs. Quote n and m separately and the distinction never bites you.

Cost summary

The yardsticks used throughout the book.

- $O(n+m)$ — "graph-linear." BFS, DFS, topological sort, SCCs, bridges. The gold standard: you cannot beat reading the graph once.
- $O(m \log n)$ — Dijkstra with a binary min-heap, Kruskal/Prim. The factor $\log n$ reflects the tree height $\lfloor \log_2 n \rfloor$ of a binary heap: extracting the minimum-distance vertex or decreasing a key traverses at most one root-to-leaf branch.
- $O(m + n \log n)$ — Dijkstra with a Fibonacci heap. By maintaining a flexible collection of trees with lazy consolidation, Fibonacci heaps achieve amortized $O(1)$ `decrease-key` operations, so the m edge relaxations run in linear time while only the n minimum-

extractions pay the $O(\log n)$ consolidation cost.

- $O(nm)$ — Bellman–Ford, Edmonds–Karp max-flow. Polynomial but the step up from linear is steep.
- $O(n^3)$ — Floyd–Warshall all-pairs, naive matrix methods. Fine for dense graphs of moderate n.
- $\widetilde{O}(m)$ — the modern frontier: near-linear Laplacian solvers and electrical-flow methods (Chapter 41), where $\widetilde{O}$ hides polylogarithmic factors.

We assume the word-RAM model: vertex labels, edge weights, and array indices fit in a machine word, and arithmetic and array access cost $O(1)$.

Connection

The choice of representation matters throughout the rest of the book and in its companion volume on linear algebra. The adjacency matrix $\mathbf{A}$ and its relative the Laplacian $\mathbf{L} = \mathbf{D} - \mathbf{A}$ connect graphs to linear algebra: a graph question becomes a matrix question, and the spectrum of $\mathbf{L}$ encodes connectivity, expansion, and random walks (Chapters 31–36). The sparse adjacency list, meanwhile, makes near-linear algorithms conceivable: operations scan only existing neighbors rather than empty entries.

More immediately, Chapter 2 uses the adjacency list in our first algorithm, Breadth-First Search. We explore the graph level by level, prove the queue invariant that guarantees exact shortest paths on unweighted graphs, and verify that adjacency lists deliver the optimal $O(n+m)$ running time.

1.8 Exercises

Exercise 1.1 (Handshake in practice)**.** A graph has degree sequence $(5, 3, 3, 2, 2, 1)$. Is such a simple graph possible? Either draw one or prove it cannot exist. (Hint: first check the handshake parity, then think about whether the vertex of degree 5 can be realized simply on six vertices.)

Exercise 1.2 (Components by parity)**.** Prove that in any graph the number of vertices of odd degree is even *within each connected component*, not merely globally. Deduce that a connected graph with exactly two odd-degree vertices has an Euler trail between them (you may cite the Euler-trail characterization informally; a full proof comes later).

Exercise 1.3 (Representation surgery)**.** Given a digraph as an adjacency list, give an $O(n + m)$ algorithm that outputs the adjacency list of the *transpose* graph G^{T} (every edge reversed). Prove the bound. Then explain why doing the same from an adjacency matrix is $\Theta(n^2)$ and from an edge list is $\Theta(n + m)$.

Exercise 1.4 (Implement CSR)**.** Implement BuildCSR (Algorithm 2) and test it on the undirected running example by first expanding each undirected edge into two directed copies. Verify that $\mathrm{head}[n] = 2m$ and that scanning every block reproduces all $2m$ directed edges exactly once.

Exercise 1.5 (Break the twin trick)**.** Build an undirected graph as an edge array where the twin of edge i is $i \oplus 1$. Write a routine that, given an edge index, returns its twin, and a routine that walks an Euler circuit using the twins. Then deliberately break the layout (store one edge's copies

at non-adjacent indices) and describe precisely what goes wrong in the Euler walk. Why does the $i \oplus 1$ rule fail the moment the pairing is not at $2i, 2i{+}1$?

Exercise 1.6 (Bipartite test by parity)**.** Using the distance-parity coloring from the proof of Theorem 1.4, write $O(n + m)$ pseudocode that either outputs a valid 2-coloring or returns an odd cycle as a certificate of non-bipartiteness. Argue that the returned cycle really is odd.

Exercise 1.7 (Walks via matrix powers)**.** Prove by induction that $(\mathbf{A}^k)_{uv}$ equals the number of walks of length exactly k from u to v. Use it to compute, for the running five-vertex graph, the number of closed walks of length 3 from vertex 3 to itself, and relate the trace $\sum_v (\mathbf{A}^3)_{vv}$ to the number of triangles in the graph (work out the exact constant of proportionality).

Exercise 1.8 (Tree edge count, two ways)**.** Give a second proof that a tree on n vertices has $n - 1$ edges, this time by induction on n using the existence of a leaf (Corollary 1.1): remove a leaf and recurse. Where exactly does the argument use acyclicity, and where does it use connectivity?

Exercise 1.9 (Dense vs sparse crossover)**.** At what density m (as a function of n) do an adjacency matrix and an adjacency list use asymptotically the same space? For a graph with $m = 4n$ edges and $n = 10^6$, by what factor does the matrix waste memory compared with the list? Give actual numbers.

Exercise 1.10 (Implicit graph blow-up)**.** Consider the implicit graph whose vertices are binary strings of length k and where $\mathrm{succ}(s)$ flips any single bit (the hypercube Q_k). It has $n = 2^k$ vertices and $m = k\,2^{k-1}$ edges. Following the successor rule in Algorithm 3, trace the step-by-step distance expansion from 0^k to 1^k on paper for $k = 3$, grouping vertices by their Hamming distance from 0^k, and confirm the shortest path has length k. Then explain why storing this graph explicitly is impossible for $k = 40$, and contrast the number of states touched to find a short path against storing the full state space.

Chapter 2

Breadth-First Search and Its Uses

Chapter 1 introduced the graph representations – especially the adjacency list – that let an algorithm scan a vertex's incident edges in time proportional to its degree. Now we put those lists to work in our first systematic traversal: Breadth-First Search (BFS). Using a first-in, first-out queue, BFS visits vertices in nondecreasing order of their unweighted edge-distance from a source. This chapter develops that queue discipline, proves the invariant that guarantees exact unweighted shortest paths, and applies it to shortest-path trees, bipartiteness testing, multi-source distance fields, and 0–1 weighted graphs.

2.1 The wave picture

Intuition

Drop a pebble in still water at vertex s. A ring of ripples reaches the neighbors of s first, then their neighbors, then theirs. The ripple touches a vertex exactly when the *shortest* chain of edges from s first reaches it, because the wave moves outward at one edge per tick and never doubles back. BFS is the bookkeeping that makes a computer reproduce that ripple: it processes vertices in nondecreasing order of edge-distance from s, expanding the closest unfinished vertex before any farther one.

Let $G = (V, E)$ be a graph, directed or undirected, with $n = |V|$ vertices and $m = |E|$ edges, and fix a *source* $s \in V$. Define the *breadth-first distance* (or unweighted shortest-path distance)

$$\operatorname{dist}(s, v) \;=\; \text{the minimum number of edges on any walk } s \to v,$$

with $\operatorname{dist}(s, v) = \infty$ if no such walk exists. The set of vertices at a fixed distance ℓ is the *level*

$$L_\ell \;=\; \{\, v \in V : \operatorname{dist}(s, v) = \ell \,\}.$$

So $L_0 = \{s\}$, L_1 is the neighborhood of s, and in general $L_{\ell+1}$ is everything one edge past L_ℓ that we have not already seen. The wave picture is exactly the claim that V partitions into $L_0, L_1, L_2, \ldots$ and that BFS visits the levels in order.

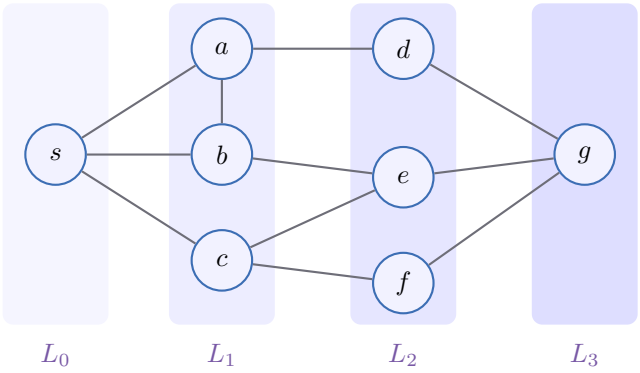


Figure 2.1: The running example graph G_0 drawn by its BFS levels from s. Each shaded column is a level L_ℓ; every edge joins vertices in the same level or in adjacent levels – never two levels apart. That "no edge skips a level" property is the key to the correctness proof.

We will return to G_0 from Figure 2.1 again and again. For the record, its adjacency lists (undirected, so each edge appears twice) are

$$s:\ a,b,c \quad a:\ s,b,d \quad b:\ s,a,e \quad c:\ s,e,f$$
$$d:\ a,g \quad e:\ b,c,g \quad f:\ c,g \quad g:\ d,e,f.$$

Focus

Two facts do all the work in this chapter. **(1)** A FIFO queue processes vertices in nondecreasing distance order. **(2)** In an unweighted graph every edge connects two vertices whose distances from s differ by at most 1. Fact (1) is about the algorithm; fact (2) is about the graph. Together they force BFS to compute $\mathrm{dist}(s,\cdot)$ exactly.

2.2 The queue and the algorithm

The data structure that turns the wave picture into code is the *first-in-first-out queue*. We push the source, then repeatedly pop a vertex u from the front, and for each neighbor v of u that we have never seen, we record its distance as $\mathrm{dist}(u)+1$, mark it seen, and push it to the back. A vertex is marked the instant it is *discovered* (pushed), not when it is later processed; marking on discovery is the single most common place beginners go wrong, and we will dwell on it.

Algorithm 4: BFS(G, s) – breadth-first search from a single source

input : graph $G = (V, E)$ as adjacency lists; source $s \in V$
output: dist$[v]$ and parent$[v]$ for every v reachable from s

```
1  foreach v ∈ V do
2      dist[v] ← ∞;  parent[v] ← NIL
3  dist[s] ← 0
4  Q ← empty FIFO queue;  ENQUEUE(Q, s)
5  while Q is not empty do
6      u ← DEQUEUE(Q)
7      foreach v ∈ adj[u] do
8          if dist[v] = ∞ then
9              dist[v] ← dist[u] + 1
10             parent[v] ← u
11             ENQUEUE(Q, v)
12 return dist, parent
```

Notice that dist$[v] = \infty$ doubles as the "not yet discovered" flag, so we do not need a separate visited array; the first time we touch v we both stamp its distance and remove it from the undiscovered pool. The array parent records, for each discovered v, the vertex from which we first reached it. Those parent pointers, read backwards, spell out a shortest path, and collectively they form the *BFS tree* of Section 2.4.

Common trap

Mark on enqueue, not on dequeue. A tempting but broken variant marks v as visited only when it is dequeued. Then a single vertex can be pushed many times before it is ever processed – once by each neighbor that reaches it while it sits in the queue. On a dense graph this blows the queue up to $\Theta(m)$ entries and, worse, lets a vertex be assigned a larger distance on a later push. Always set dist$[v]$ at the moment you enqueue v, guarded by the dist$[v] = \infty$ test, so each reachable vertex is stamped and enqueued once.

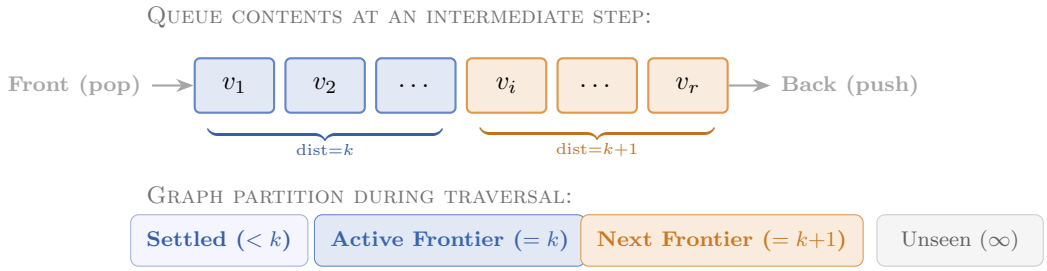
2.2.1 Tracing the queue on G_0

Let us run Algorithm 4 on G_0 from s, processing each adjacency list in the listed order. We track the queue contents (front on the left) and the distance stamps as they are written.

step	dequeue	newly discovered (dist)	queue after
0	–	$s\,(0)$	$[\,s\,]$
1	s	$a\,(1)$, $b\,(1)$, $c\,(1)$	$[\,a, b, c\,]$
2	a	$d\,(2)$	$[\,b, c, d\,]$
3	b	$e\,(2)$	$[\,c, d, e\,]$
4	c	$f\,(2)$	$[\,d, e, f\,]$
5	d	$g\,(3)$	$[\,e, f, g\,]$

6	e	–	$[f, g]$
7	f	–	$[g]$
8	g	–	$[\,]$

At every step of this trace, the FIFO queue holds at most two consecutive distance levels: the remainder of the current frontier at distance k at the front, followed by newly discovered vertices at distance $k+1$ at the back:



When a is processed at step 2 it sees neighbors s (already stamped), b (already stamped, it was discovered at step 1), and d (new). So only d is enqueued. At step 3, b looks at s, a, e and discovers only e. The edge a–b inside level L_1 is examined twice – once from each endpoint – and produces no new discovery either time; those are the *intra-level* edges. The final distance stamps are

$$\text{dist}(s, \cdot): \quad s{=}0,\ a{=}1,\ b{=}1,\ c{=}1,\ d{=}2,\ e{=}2,\ f{=}2,\ g{=}3,$$

matching the level columns in Figure 2.1 exactly. The parent pointers are $\text{parent}[a] = \text{parent}[b] = \text{parent}[c] = s$, $\text{parent}[d] = a$, $\text{parent}[e] = b$, $\text{parent}[f] = c$, and $\text{parent}[g] = d$.

Trace it yourself

Re-run the trace but process each adjacency list in *reverse* order (s expands c, b, a). The distances come out identical – they must, they are a property of the graph – but the parent of g changes, and the BFS tree changes shape. Convince yourself that $\text{dist}[v]$ is canonical while the tree is not. Then run BFS from g instead of s and check that $\text{dist}(g, s) = 3$, confirming symmetry of distance on undirected graphs.

2.3 Correctness: BFS computes shortest paths in edges

We now prove that Algorithm 4 sets $\text{dist}[v] = \text{dist}(s, v)$ for every v. The argument has two halves that mirror the two facts in the focus box: a monotonicity lemma about the queue, and the level structure of the graph.

Lemma 2.1 (Queue monotonicity)**.** *At every moment during the execution, if the queue holds $\langle v_1, v_2, \ldots, v_r \rangle$ from front to back, then*

$$\text{dist}[v_1] \le \text{dist}[v_2] \le \cdots \le \text{dist}[v_r] \qquad \textit{and} \qquad \text{dist}[v_r] \le \text{dist}[v_1] + 1.$$

That is, distances along the queue are nondecreasing and span a range of at most one.

Proof. Induct on the number of queue operations. Initially the queue is $\langle s \rangle$ with $\text{dist}[s] = 0$; both claims hold vacuously. Suppose they hold before an operation.

A dequeue removes v_1. The remaining queue $\langle v_2, \ldots, v_r \rangle$ is still sorted, and we must recheck the span: $\text{dist}[v_r] \le \text{dist}[v_1] + 1 \le \text{dist}[v_2] + 1$, using $\text{dist}[v_1] \le \text{dist}[v_2]$. So the span bound survives.

An enqueue happens only while we are expanding some $u = v_1$ that was just dequeued, and the new vertex w gets $\text{dist}[w] = \text{dist}[u] + 1$. Just before this expansion the queue was $\langle v_2, \ldots, v_r \rangle$ (after u was removed), with $\text{dist}[v_2] \ge \text{dist}[u]$ (sortedness of the old queue) and $\text{dist}[v_r] \le \text{dist}[u] + 1 = \text{dist}[w]$ (old span bound). Appending w keeps the sequence sorted because $\text{dist}[w] = \text{dist}[u] + 1 \ge \text{dist}[v_r]$, and the new span is $\text{dist}[w] - \text{dist}[v_2] \le (\text{dist}[u] + 1) - \text{dist}[u] = 1$. Both invariants are restored. □

A clean corollary: vertices are dequeued in nondecreasing distance order, and the distance stamps written over the whole run are nondecreasing in time. The queue never holds two stamps differing by more than one, which is precisely why BFS processes one entire level before starting the next.

Lemma 2.2 (Adjacent levels)**.** *For any edge (u, v) of G, $|\text{dist}(s, u) - \text{dist}(s, v)| \le 1$. For a directed edge $u \to v$, $\text{dist}(s, v) \le \text{dist}(s, u) + 1$.*

Proof. Take any shortest walk $s \to u$ of length $\text{dist}(s, u)$ and append the edge $u \to v$; this is a walk $s \to v$ of length $\text{dist}(s, u) + 1$, so $\text{dist}(s, v) \le \text{dist}(s, u) + 1$. On an undirected graph the edge is also $v \to u$, so symmetrically $\text{dist}(s, u) \le \text{dist}(s, v) + 1$, and the two inequalities give the absolute-value bound. □

Lemma 2.2 is the formal version of "no edge skips a level" in Figure 2.1: a single edge can keep you in the same level or move you one level, never two. With both lemmas in hand the main theorem is a short induction.

Theorem 2.1 (BFS correctness)**.** *After Algorithm 4 halts, $\text{dist}[v] = \text{dist}(s, v)$ for every $v \in V$, and the value ∞ remains exactly for the vertices unreachable from s. In addition, following parent pointers from any reachable v back to s yields a shortest path.*

Proof. First, every distance stamp written by the algorithm is the length of a genuine s–v walk: when v is stamped from u we extend the parent-chain walk of u by one edge, so by induction the parent chain from v back to s is a real walk of length $\text{dist}[v]$. Hence $\text{dist}[v] \ge \text{dist}(s, v)$ for every discovered v.

It remains to show that every vertex v with $\text{dist}(s, v) = \ell$ is discovered and receives $\text{dist}[v] = \ell$. Induct on ℓ. Base $\ell = 0$: only s, stamped 0 before the loop. Inductive step: let $\text{dist}(s, v) = \ell + 1$. Some shortest path reaches v through a vertex u with $\text{dist}(s, u) = \ell$ and edge $u \to v$. By induction u is discovered with $\text{dist}[u] = \ell$, so u is eventually dequeued. When u is expanded, either v is still undiscovered – in which case v is stamped $\text{dist}[u] + 1 = \ell + 1$ – or v was already discovered earlier. In the second case v was stamped from some vertex dequeued no later than u. By Lemma 2.1 every vertex dequeued up to and including u has distance stamp $\le \text{dist}[u] = \ell$, so v's stamp is at most $\ell + 1$. Combined with $\text{dist}[v] \ge \text{dist}(s, v) = \ell + 1$ from the first paragraph, the two bounds force $\text{dist}[v] = \ell + 1$. Either way v is discovered with the correct stamp.

Finally, a vertex v is reachable iff $\text{dist}(s, v)$ is finite, and the induction discovers exactly those. Unreachable vertices are never enqueued, so their stamp stays ∞. The parent chain of a discovered v has lengths decreasing by exactly one each step down to s, so it is a path of length $\text{dist}[v] = \text{dist}(s, v)$ – a shortest path. □

Why it works

The proof never used edge weights because there are none; it used only that a hop changes distance by one. That is also why BFS *cannot* be salvaged for weighted graphs by the same argument: with weights, a vertex could be reached "early but expensively," the stamp-once-and-freeze rule would lock in a wrong value, and Lemma 2.1 collapses. Dijkstra's algorithm (Chapter 7) fixes exactly that problem by replacing the FIFO queue with a priority queue. BFS is Dijkstra with all weights 1 and a queue instead of a heap.

2.3.1 Complexity

Theorem 2.2 (BFS runs in linear time)**.** *Algorithm 4 runs in $O(n+m)$ time and $O(n)$ extra space on a graph stored as adjacency lists.*

Proof. Initialization touches each vertex once: $O(n)$. Each vertex is enqueued at most once (the $\text{dist}[v] = \infty$ guard fires only on first discovery) and hence dequeued at most once, so the **while** loop iterates $\leq n$ times with $O(1)$ queue work apiece. When u is dequeued we scan $\text{adj}[u]$, doing $O(1)$ per incident edge. Summing over all dequeues, each undirected edge is scanned from both endpoints (twice) and each directed edge once, for $O(m)$ total edge work. The grand total is $O(n+m)$. The arrays dist and parent are $O(n)$, and the queue never holds more than n vertices. □

Cost summary

Single-source BFS. Time $O(n+m)$, space $O(n)$. Optimal: any algorithm that even reads a graph guaranteed to need every edge must spend $\Omega(n+m)$. The hidden constant is tiny – two array writes and one queue push per discovery – which is why BFS is the standard method for unweighted distance, reachability, and connectivity queries, and why it is the inner loop of layered max-flow and bidirectional search.

History

Breadth-first search was published by Edward F. Moore in 1959 as a method for "finding the shortest path through a maze," and independently by C. Y. Lee in 1961 for routing wires on circuit boards – the grid version is still called the *Lee algorithm* in VLSI. Konrad Zuse had described an essentially breadth-first graph traversal in an unpublished 1945 manuscript. The maze framing is the right mental model: a maze is a grid graph, every step costs one, and the ripple from the entrance reaches the exit along a shortest corridor.

2.4 The BFS tree and shortest-path structure

The parent pointers from Algorithm 4 define a subgraph $T = (V_s, E_T)$ on reachable vertices V_s, with tree edges $E_T = \{(\text{parent}[v], v) : v \neq s,\ v \in V_s\}$. Each reachable $v \neq s$ has exactly one parent and the chain terminates at s, so T is a tree rooted at s: the *BFS tree*.

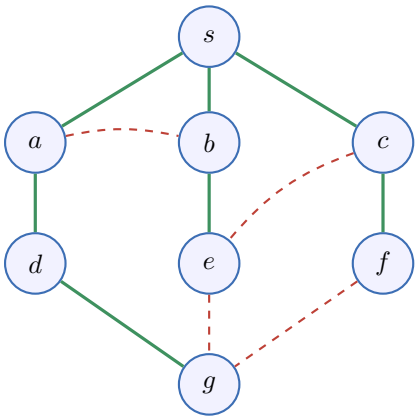


Figure 2.2: The BFS tree of G_0 (green, solid) overlaid with the non-tree edges (red, dashed). Each non-tree edge is either intra-level (a–b joins two vertices of L_1) or joins adjacent levels (c–e joins L_1 to L_2; e–g and f–g join L_2 to L_3). No non-tree edge spans two levels – the visual restatement of Lemma 2.2.

Proposition 2.1 (Tree paths are shortest paths)**.** *For every reachable v, the unique path from s to v in the BFS tree T has length* $\mathrm{dist}(s, v)$*. Consequently T is a* shortest-path tree*: it simultaneously stores a shortest path from s to every vertex in $O(n)$ space.*

Proof. By Theorem 2.1, $\mathrm{dist}[v] = \mathrm{dist}[\mathrm{parent}[v]] + 1$ for every reachable $v \neq s$, so the tree depth of v equals $\mathrm{dist}[v] = \mathrm{dist}(s, v)$. The tree path is therefore a shortest path, and there are at most $n - 1$ tree edges, giving $O(n)$ storage for all n shortest paths at once. □

This is the practical payoff of the whole chapter. To answer "shortest unweighted route from s to v" for many targets v, run one BFS and then walk parent pointers; each query costs only the length of its answer. The classification of non-tree edges in Figure 2.2 also matters: in an undirected graph BFS produces only *tree edges* and *cross edges* joining vertices whose levels differ by 0 or 1 – never a "back edge" to a strict ancestor two or more levels up, and never a "forward edge" skipping a level. (DFS, by contrast, generates back and forward edges; that contrast is the subject of the next chapter.)

Connection

The BFS level structure also connects directly to the graph Laplacian's low-frequency modes. If you order vertices by BFS level and look at the Laplacian $\mathbf{L}$ of G, the level structure bounds how fast a function can vary: a vector that changes gradually across adjacent levels has a bounded Rayleigh quotient, which is one route into the diameter–eigenvalue inequalities of Chapter 32. We will also reuse BFS levels directly to build the *layered residual networks* of Dinic's max-flow algorithm (Chapter 12) and the alternating BFS layers of Hopcroft–Karp matching (Chapter 15).

2.4.1 The eccentricity and diameter view

The largest finite stamp produced by a BFS from s is the *eccentricity* $\mathrm{ecc}(s) = \max_v \mathrm{dist}(s, v)$, the distance to the farthest reachable vertex. The maximum eccentricity over all sources is the *diameter*,

the minimum is the *radius*. On G_0, $\mathrm{ecc}(s) = 3$ (witnessed by g). Computing exact diameter needs a BFS from every vertex ($O(nm)$ on sparse graphs), but a two-sweep heuristic – BFS from any vertex to find a far vertex x, then BFS from x – gives the *exact* diameter on trees and a tight estimate on road networks, a trick we revisit in the exercises.

2.5 Testing bipartiteness

A graph is *bipartite* if its vertices split into two classes so that every edge crosses between classes – a 2-coloring with no monochromatic edge. Equivalently (we prove this), a graph is bipartite iff it has no odd cycle. BFS detects bipartiteness for free: color each vertex by the parity of its level.

Intuition

Walk the wave outward and paint level L_0 white, L_1 black, L_2 white, and so on – color $=$ $\mathrm{dist}(s, v) \bmod 2$. Tree edges always cross colors because they join adjacent levels. The only thing that can ruin the 2-coloring is a non-tree edge joining two vertices in the *same* level, because that edge would join two same-colored vertices. And a same-level edge closes an odd cycle. So the graph is bipartite iff BFS never finds a same-level edge.

Algorithm 5: BipartiteBfs(G) – 2-color by level parity

```
input  : undirected graph G = (V, E)
output: a 2-coloring, or report an odd cycle (not bipartite)
foreach v ∈ V do
    color[v] ← NIL
foreach s ∈ V with color[s] = NIL do
    color[s] ← 0;  Q ← ⟨s⟩
    while Q not empty do
        u ← DEQUEUE(Q)
        foreach v ∈ adj[u] do
            if color[v] = NIL then
                color[v] ← 1 − color[u];  ENQUEUE(Q, v)
            else if color[v] = color[u] then
                return NOT BIPARTITE (odd cycle through u, v)
return color
```

The outer loop over uncolored s handles disconnected graphs: each component gets its own BFS, and bipartiteness is a per-component property.

Theorem 2.3 (Bipartiteness via BFS)**.** *G is bipartite iff Algorithm 5 colors it without ever finding a same-color edge, and this runs in $O(n + m)$. Equivalently, G is bipartite iff it has no odd cycle.*

Proof. If the algorithm succeeds, the coloring is a valid 2-coloring by construction: every edge it accepted joined differently colored endpoints, and it examined every edge. So G is bipartite, and a bipartite graph has no odd cycle (a 2-coloring forces colors to alternate around any cycle, which is impossible at odd length).

If the algorithm fails, it found an edge (u, v) with $\text{color}[u] = \text{color}[v]$, i.e. $\text{dist}(s, u) \equiv \text{dist}(s, v) \pmod 2$ within the same component. Let w be the lowest common ancestor of u and v in the BFS tree. The closed walk $w \rightsquigarrow u \to v \rightsquigarrow w$ has length $(\text{dist}[u] - \text{dist}[w]) + 1 + (\text{dist}[v] - \text{dist}[w]) = \text{dist}[u] + \text{dist}[v] - 2\,\text{dist}[w] + 1$, which is odd because $\text{dist}[u] + \text{dist}[v]$ is even. The two tree paths share only w, so this closed walk contains an odd cycle. An odd cycle blocks any 2-coloring, so G is not bipartite.

The two directions together prove the equivalence with "no odd cycle." The cost is one BFS over each component with an $O(1)$ color check per edge, so $O(n + m)$. □

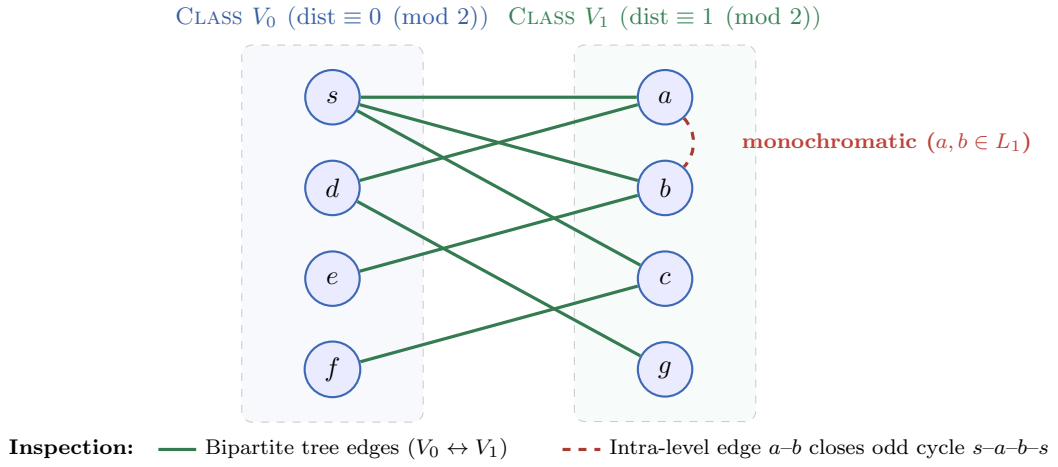
Common trap

Never eyeball a 2-coloring from a drawing; run the algorithm. In our running graph G_0, the levels put a and b both in L_1, yet the edge a–b joins them – a same-level edge. So G_0 is *not* bipartite, and Algorithm 5 certifies the odd cycle $s \to a \to b \to s$ of length 3. A single intra-level edge rules out bipartiteness, and a hand-drawn layout can easily obscure it.

Trace it yourself

Build the 4-cycle $s\,a\,e\,b\,s$ in isolation (edges s–a, a–e, e–b, b–s) and color it from s. Levels are s=0, a=1, b=1, e=2; every edge crosses a color boundary, so the algorithm succeeds and the 4-cycle is bipartite – as every even cycle is. Then add the chord a–b and watch the same-level edge a–b trigger the "not bipartite" report, with odd cycle $s \to a \to b \to s$.

Splitting all eight vertices of G_0 by distance parity into $V_0 = L_0 \cup L_2$ and $V_1 = L_1 \cup L_3$ makes the obstruction visible at a glance: every BFS tree edge crosses between V_0 and V_1, whereas the intra-level edge a–b stays inside V_1:



2.6 Multi-source BFS

Sometimes the natural question is not "how far is v from one source" but "how far is v from the *nearest* of several sources." Think of fire stations on a city grid, each cell wanting its distance to the

closest station; or a flood spreading from many wells at once. The trick costs nothing extra: seed the queue with *all* sources at distance 0, then run ordinary BFS.

Definition 2.1 (Multi-source distance)**.** Given a set $S \subseteq V$ of sources, define

$$\operatorname{dist}(S, v) = \min_{s \in S} \operatorname{dist}(s, v),$$

the edge-distance from v to the nearest source.

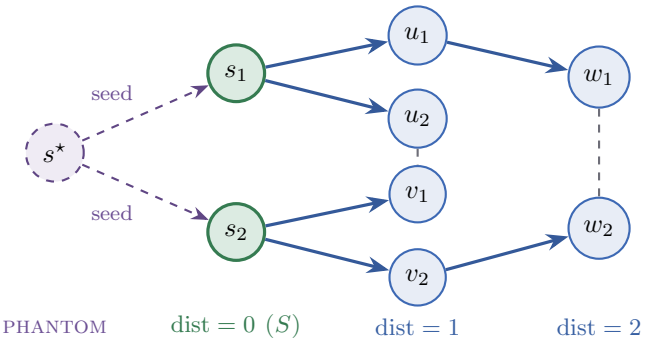
Algorithm 6: MultiSourceBfs(G, S)

```
   input  : graph G = (V, E); source set S ⊆ V
   output: dist[v] = dist(S, v) for all v
 1 foreach v ∈ V do
 2     dist[v] ← ∞
 3 Q ← empty FIFO queue
 4 foreach s ∈ S do
 5     dist[s] ← 0;  enqueue(Q, s)
 6 while Q not empty do
 7     u ← dequeue(Q)
 8     foreach v ∈ adj[u] do
 9         if dist[v] = ∞ then
10             dist[v] ← dist[u] + 1;  enqueue(Q, v)
11 return dist
```

Why it works

Why does seeding everything at level 0 compute the *minimum* over sources? Build a phantom super-source $s^\star$ joined by a single edge to each $s \in S$, and run ordinary single-source BFS from $s^\star$. Then $\operatorname{dist}(s^\star, v) = 1 + \min_{s \in S} \operatorname{dist}(s, v)$ for $v \notin S$: every shortest path from $s^\star$ must pass through one of its neighbors, namely the nearest source. Stripping off the leading phantom edge (subtract 1, and start the real sources at 0 instead of 1) gives exactly Algorithm 6. Because all of S enters the queue before any expansion, Lemma 2.1 still holds – the queue starts as a block of zeros – so the same monotonicity proof certifies correctness, and the cost is still $O(n + m)$ regardless of $|S|$.

The diagram below illustrates the phantom super-source $s^\star$ feeding two real sources $S = \{s_1, s_2\}$ into level 0, so their outward ripples advance in lockstep and partition the graph into nearest-source regions:



Application

Grid distance fields. On an $H \times W$ grid where some cells are sources (stations, fires, exits), multi-source BFS fills the whole distance field in $O(HW)$ – linear in the number of cells, independent of how many sources there are. This is the standard preprocessing step for influence maps in game AI, for *distance transforms* in image processing (the city-block distance to the nearest foreground pixel), and for "rotting oranges"-style spread simulations. The phantom-super-source reduction means you never write special-case code: one BFS, many seeds.

2.7 0–1 BFS with a deque

Plain BFS fails as soon as edges carry arbitrary weights, because the FIFO queue assumes every hop costs one. But a small modification keeps linear time for one class just beyond unweighted graphs: graphs whose edge weights are all 0 or 1. Such graphs appear constantly – a 0-edge models "free" moves (sliding along a conveyor, an already-built road), a 1-edge a costly move (building, turning, crossing). For these, replace the queue with a *double-ended queue* (deque) and the linear-time guarantee survives.

Intuition

The wave still spreads, but now a 0-edge lets the wave reach a neighbor *at no extra distance* – it belongs to the current ripple, not the next one. So push 0-edge neighbors to the *front* of the deque (process them with the current level) and 1-edge neighbors to the *back* (process them with the next level). The deque keeps the same two-value span that ordinary BFS maintains. It is Dijkstra's algorithm, but the priority queue degenerates into a deque because there are only two possible key increments.

Algorithm 7: ZEROONEBFS(G, w, s) – 0–1 BFS with a deque

input : graph $G = (V, E)$ with weights $w(u, v) \in \{0, 1\}$; source s
output: dist$[v]$ = shortest path weight $s \to v$

```
foreach v ∈ V do
    dist[v] ← ∞
dist[s] ← 0;  D ← deque containing s
while D not empty do
    u ← popFront(D)
    foreach v ∈ adj[u] do
        if dist[u] + w(u,v) < dist[v] then
            dist[v] ← dist[u] + w(u,v)
            if w(u,v) = 0 then
                pushFront(D, v)
            else
                pushBack(D, v)
return dist
```

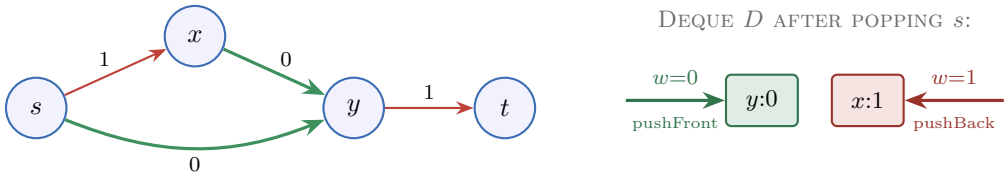


Figure 2.3: Left: the 0–1 BFS example of the lab below (green arrows weight 0, red arrows weight 1). Right: expanding s pushes y (w=0) to the *front* at dist = 0 and x (w=1) to the *back* at dist = 1, so y is popped before x and t finalizes at total weight 1.

Two differences from Algorithm 4 are essential. First, the relax test is dist$[u] + w <$ dist$[v]$, a strict-improvement check, *not* a discovered-yet flag: a vertex may be reached, then reached again more cheaply via a chain of 0-edges, so we must allow re-stamping. Second, because of re-stamping a vertex can sit in the deque more than once. The strict relax test makes the duplicates harmless: once v is popped at its final (smallest) distance, every later relaxation out of a stale copy of v fails the $<$ dist$[\cdot]$ guard and changes nothing, so reprocessing it costs only the scan of its neighbors. Each vertex is finalized once, and the wasted scans are charged to edges, so the work stays linear.

Theorem 2.4 (0–1 BFS correctness and cost)**.** *On a graph with weights in* $\{0, 1\}$*, Algorithm 7 computes* dist$[v] = \text{dist}_w(s, v)$*, the minimum total weight of an* s*–*v *path, in* $O(n + m)$ *time.*

Proof. We claim the deque obeys the same monotonicity invariant as the BFS queue: reading front to back, distances are nondecreasing and span at most 1. The invariant holds initially. A popFront removes the minimum-distance vertex (front), and the span bound is preserved as in Lemma 2.1. A pushFront adds a vertex v with dist$[v]$ = dist$[u]$ (since $w = 0$ and u was the front), so v is a legal new front. A pushBack adds v with dist$[v]$ = dist$[u] + 1$, at most one more than the old back, so it is a legal new back. The span never exceeds 1.

Consequently vertices are popped in nondecreasing dist order, exactly the order Dijkstra would pop them. When a vertex u is popped as the current minimum, no later relaxation can lower dist$[u]$ (all

remaining deque distances are $\geq \text{dist}[u]$, and edge weights are nonnegative), so $\text{dist}[u]$ is final and correct – the standard Dijkstra finalization argument specialized to the two-key deque. Hence every $\text{dist}[v]$ converges to $\text{dist}_w(s, v)$.

For the running time, charge each deque insertion to the relaxation that caused it. Because pops come out in nondecreasing dist order and weights are in $\{0, 1\}$, when an edge $u \to v$ relaxes successfully the value $\text{dist}[v]$ drops to $\text{dist}[u]$ or $\text{dist}[u] + 1$, and u is then final; a second successful relaxation of the same edge would require $\text{dist}[u]$ to have dropped, which cannot happen after u is popped. So each edge contributes $O(1)$ insertions, giving $O(m)$ insertions and pops overall. Each pop scans the popped vertex's adjacency list once, for $O(m)$ scan work in total, and initialization is $O(n)$. Total work $O(n + m)$. □

Trace it yourself

Take the path $s \xrightarrow{1} x \xrightarrow{0} y \xrightarrow{1} t$ together with a direct 0-edge $s \xrightarrow{0} y$, drawn in Figure 2.3. Run 0–1 BFS from s. You should watch y get stamped 0 via the front-push from s, *before* the longer route through x ever finalizes y, and then see t finalize at distance 1 ($y \xrightarrow{1} t$). Compare against what plain BFS (treating every edge as weight 1) returns for t: counting edges, the shortest chain is $s \to y \to t$ of length 2, so plain BFS reports $\text{dist}(t) = 2$, yet the true 0–1 weight along that same chain is only 1 because the $s \to y$ step is free.

Connection

0–1 BFS is the simplest of a family of algorithms. The next, *Dial's algorithm* (Chapter 8), handles small integer weights $0, 1, \ldots, C$ with $C + 1$ buckets instead of a two-ended deque; past that lies Dijkstra with a real heap for arbitrary nonnegative weights (Chapter 7). All three share the finalization invariant proved above – pop the current minimum, freeze it – and differ only in how cheaply the "current minimum" can be extracted. BFS extracts it in $O(1)$ from a queue, 0–1 BFS in $O(1)$ from a deque, Dijkstra in $O(\log n)$ from a heap.

2.8 More applications

Connected components and reachability. One BFS labels an entire connected component; looping over unvisited vertices and launching a BFS from each labels every component in $O(n + m)$. On a directed graph a single BFS from s computes the set of vertices reachable *from* s; running BFS on the reverse graph computes the set that can *reach* s. Their intersection is used in Kosaraju's strongly-connected-components routine of Chapter 5.

Shortest cycle (girth) through unweighted BFS. To find the shortest cycle in an undirected unweighted graph, run a BFS from each vertex r; if while expanding you ever find a non-tree edge (u, v) with both endpoints already discovered in the *current* BFS, it closes a cycle of length $\text{dist}[u] + \text{dist}[v] + 1$. The minimum over all roots and all such edges is the girth, in $O(nm)$.

Word ladders and implicit graphs. BFS does not need the graph in memory. In a word-ladder puzzle (`cat` to `dog`, changing one letter at a time through valid words), vertices are words and edges

connect words differing in one letter. Generate neighbors on the fly inside the inner loop; BFS finds the fewest-step chain. The same *implicit-graph* pattern solves sliding-tile puzzles, Rubik's-cube searches (with bidirectional BFS), and state-space planning.

Bidirectional BFS. For a single s–t query on a huge graph, grow two BFS waves, one from s and one from t, alternating and stopping when they meet. If the graph has branching factor roughly b and the answer is at distance ℓ, one-directional BFS touches $\Theta(b^\ell)$ vertices while the two half-depth waves touch only $\Theta(b^{\ell/2})$ each – a square-root savings that turns hopeless searches tractable. The subtlety is the meeting condition: stop only once the combined frontier radii can no longer improve, then reconstruct the path by splicing the two parent chains at the cheapest meeting vertex.

Application

Maze and grid routing, restated. Moore's and Lee's original use cases are still live. A robot on an occupancy grid runs multi-source BFS from all obstacle cells to get a clearance map, then single-source BFS on the free cells for a shortest collision-free route. The 0–1 variant models terrain where some moves are free (downhill, paved) and others cost one (rough), all without leaving linear time. When weights become genuinely varied, you graduate to Dijkstra – but a surprising amount of real routing lives in the $\{0, 1\}$ and small-integer regime where BFS-family algorithms win on constant factors.

2.9 Summary

BFS expands vertices in nondecreasing edge-distance from the source using a FIFO queue, runs in $O(n+m)$, and produces a shortest-path tree as a side effect. The two key facts are queue monotonicity (Lemma 2.1) and the adjacent-level property of unweighted graphs (Lemma 2.2); together they yield the correctness theorem and every application. Color by level parity to test bipartiteness; seed the queue with many vertices for multi-source distance fields; swap the queue for a deque to handle $\{0, 1\}$ weights. BFS layering reappears wherever this book uses a layered structure – Dinic, Hopcroft–Karp, the Laplacian's smooth modes – so the invariants proved here make those chapters easier to follow.

BFS explores outward uniformly across levels, never advancing deeper until an entire frontier has been cleared. In Chapter 3, we explore the opposite search strategy: Depth-First Search, which follows a single branch until it reaches a dead end and then backtracks. Swapping the FIFO queue for a stack or recursion yields discovery and finishing times, revealing a topological ordering of directed graphs and laying the groundwork for cycle detection and connectivity.

Cost summary

The chapter on one line. Single/multi-source BFS: $O(n + m)$ time, $O(n)$ space, exact unweighted distance and a shortest-path tree. Bipartiteness: $O(n + m)$. 0–1 BFS: $O(n + m)$ for weights in $\{0, 1\}$ via a deque. Bidirectional BFS: $\Theta(b^{\ell/2})$ for a single far query.

2.10 Exercises

Exercise 2.1 (Implement and instrument)**.** Implement Algorithm 4 and, additionally, return the level sizes $|L_0|, |L_1|, \ldots$. Run it on G_0 and confirm the sizes $1, 3, 3, 1$. Then add a counter for how many edges are examined; verify it equals $2m$ on an undirected graph and explain the factor of two.

Exercise 2.2 (Distances are canonical, trees are not)**.** Prove that for a fixed source s, the array $\mathrm{dist}[\cdot]$ produced by BFS is independent of the order in which adjacency lists are scanned, but the $\mathrm{parent}[\cdot]$ array (hence the BFS tree) need not be. Give two distinct BFS trees of G_0 from s.

Exercise 2.3 (Break the mark-on-dequeue bug)**.** Modify Algorithm 4 to mark vertices visited only when dequeued (the trap in Section 2.2). Construct a family of graphs on n vertices for which this buggy version enqueues $\Theta(n^2)$ times, and exhibit a vertex that holds an incorrect (too large) distance at some intermediate moment. Then prove the final distances are still correct despite the blowup, explaining why this is a *performance* bug, not always a correctness bug.

Exercise 2.4 (Two-sweep diameter on trees)**.** Prove that on a *tree*, the following computes the exact diameter: BFS from any vertex to find a farthest vertex x; BFS from x to find a farthest vertex y; return $\mathrm{dist}(x, y)$. Show by counterexample that the same two-sweep method can underestimate the diameter on a general graph with a cycle.

Exercise 2.5 (Odd-cycle certificate)**.** Extend Algorithm 5 so that when it reports "not bipartite," it returns an explicit odd cycle as a vertex list. Prove your cycle has odd length and bound its length by $2\,\mathrm{ecc}(s) + 1$.

Exercise 2.6 (Multi-source distance transform)**.** You are given an $H \times W$ grid with obstacle cells. Using one multi-source BFS, compute for every free cell its distance to the nearest obstacle (the city-block distance transform). State the running time and prove correctness via the phantom super-source reduction of Section 2.6.

Exercise 2.7 (0–1 BFS on a grid with portals)**.** On a grid, moving to an orthogonal neighbor costs 1, but stepping onto a portal teleports to its twin at cost 0. Model this as a $\{0, 1\}$-weighted graph and give the shortest-travel-cost algorithm. Prove that your deque-based solution runs in $O(HW)$ and argue why a plain FIFO queue would give wrong answers.

Exercise 2.8 (Lower bound on insertions)**.** Prove that 0–1 BFS performs $O(m)$ deque insertions even with re-stamping, by charging each insertion to an edge relaxation that strictly decreased some $\mathrm{dist}[v]$. Conclude the $O(n + m)$ bound rigorously, addressing the stale-entry skips.

Exercise 2.9 (Bidirectional correctness)**.** Prove that bidirectional BFS returns a shortest s–t path, being careful about the stopping rule: show that halting at the *first* vertex discovered by both frontiers can overshoot, and give the corrected rule (continue until the sum of the two frontier radii can no longer improve) with a proof.

Exercise 2.10 (BFS and the Laplacian, a first taste)**.** Let G be connected with BFS levels $L_0, \ldots, L_k$ from s. Define $x \in \mathbb{R}^V$ by $x_v = \mathrm{dist}(s, v)$. Writing $\mathbf{L} = \mathbf{D} - \mathbf{A}$, show that $(\mathbf{L}x)_v$ equals the number of neighbors of v in lower levels minus the number in higher levels (same-level neighbors cancel). Compute the Rayleigh quotient $x^\top \mathbf{L}x / x^\top x$ for G_0, and relate its magnitude to the eccentricity, foreshadowing Chapter 32.

Exercise 2.11 (Word ladder)**.** Given a dictionary, build the implicit one-letter-change graph and use BFS to find the shortest transformation from a start word to a target word. Analyze the cost in terms of word length L, alphabet size σ, and dictionary size N, and explain why generating neighbors on the fly beats materializing all $\Theta(N^2)$ potential edges.

Chapter 3

Depth-First Search, Timestamps, and Structure

In Chapter 2, we watched breadth-first search fan out in concentric rings, discovering the geometry of shortest hops. Depth-first search does the opposite. It commits: it walks as far down one branch as it can, and only when that branch dead-ends does it backtrack to the most recent junction to try the next path. That stubbornness looks like a worse search strategy — and for shortest paths it is — but the recursion stack of a DFS records useful structure. The times at which DFS first enters and finally leaves each vertex encode a nesting structure that underpins topological sorting, strongly connected components, bridges, and Euler tours. This chapter records DFS discovery and finish timestamps, proves the parenthesis theorem governing how their intervals nest, classifies every graph edge, and uses that classification for cycle detection.

Focus

By the end you should be able to: run DFS and annotate a graph with $d[\cdot]$ and $f[\cdot]$; state and prove the parenthesis and white-path theorems; classify an edge from the timestamps of its endpoints alone; explain why a back edge means a cycle in a digraph but why undirected graphs need the parent check; and name three later algorithms that are "just" a reading of these timestamps.

3.1 The walk: intuition and a hand trace

Intuition

Imagine exploring a cave system with a ball of string and a lamp. From the entrance you pick a passage and walk down it, unspooling string. At every junction you pick an unexplored passage and keep going. When you hit a chamber with no new exits, you reel the string back to the last junction that still has an unexplored passage and dive down that one. The string is the recursion stack. The moment you *first* enter a chamber is its discovery time; the moment you reel past it for the last time, having exhausted everything reachable beyond it, is its finish time. DFS is exactly this: go deep, backtrack minimally, and timestamp both events.

We maintain a global clock that starts at 0 and ticks by one each time *anything* happens — each discovery and each finish. Every vertex u gets two stamps:

- $d[u]$, the clock value when u is first reached (colored gray, "on the stack");
- $f[u]$, the clock value when DFS finishes exploring everything reachable from u and pops it (colored black, "done").

Vertices start white (undiscovered). With n vertices, the clock runs from 1 to $2n$, and every integer in that range is used exactly once: each vertex contributes one discovery tick and one finish tick.

3.1.1 A directed graph to trace

Consider the directed graph G on six vertices $\{1, 2, 3, 4, 5, 6\}$ with edges

$$1 \to 2, \quad 1 \to 4, \quad 2 \to 3, \quad 3 \to 1, \quad 4 \to 3, \quad 5 \to 4, \quad 5 \to 6, \quad 6 \to 6.$$

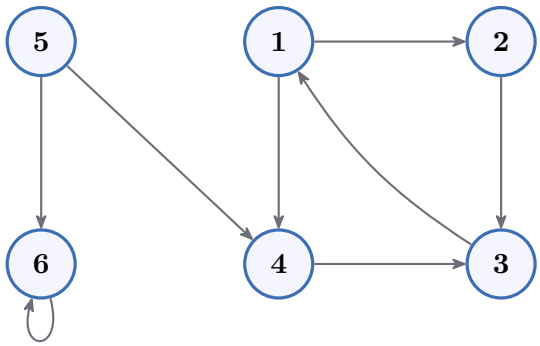


Figure 3.1: The running directed example. Adjacency lists are kept in increasing vertex order, and the outer loop visits roots $1, 2, \ldots, 6$ in order.

We always break ties the same way so the trace is reproducible: the outer loop considers vertices $1, 2, \ldots, 6$ as potential roots, and each adjacency list is scanned in increasing order. Start the clock at 0.

Trace it yourself

Follow along with pencil. Maintain a stack (top on the right) and a clock. Each line below is one clock tick.

1. Root 1 is white. Discover 1: $d[1] = 1$. Stack: $[1]$.
2. 1's first out-neighbor is 2 (white). Discover 2: $d[2] = 2$. Stack: $[1, 2]$.
3. 2's out-neighbor is 3 (white). Discover 3: $d[3] = 3$. Stack: $[1, 2, 3]$.
4. 3's out-neighbor is 1, which is gray (on the stack). This is a *back edge*; do not recurse. 3 has no other out-edges. Finish 3: $f[3] = 4$. Pop. Stack: $[1, 2]$.
5. Back at 2: no more out-edges. Finish 2: $f[2] = 5$. Pop. Stack: $[1]$.
6. Back at 1: next out-neighbor is 4 (white). Discover 4: $d[4] = 6$. Stack: $[1, 4]$.

7. 4's out-neighbor is 3, which is *black* (finished). Edge $4 \to 3$ is a *cross* edge (we will justify the name shortly). 4 has no other out-edges. Finish 4: $f[4] = 7$. Pop. Stack: [1].
8. Back at 1: no more out-edges. Finish 1: $f[1] = 8$. Pop. Stack empty. Tree rooted at 1 is done.
9. Outer loop advances. Vertices $2, 3, 4$ are already black; skip. Vertex 5 is white. Discover 5: $d[5] = 9$. Stack: [5].
10. 5's first out-neighbor is 4 (black). Edge $5 \to 4$ is a cross edge. Next out-neighbor 6 (white). Discover 6: $d[6] = 10$. Stack: [5, 6].
11. 6's only out-edge is the self-loop $6 \to 6$ (6 is gray): a back edge. Finish 6: $f[6] = 11$. Pop. Stack: [5].
12. Finish 5: $f[5] = 12$. Pop. Outer loop finds 6 black, ends. Clock at $12 = 2 \cdot 6$. Done.

Collecting the stamps as (d, f) pairs:

$$1{:}(1,8) \quad 2{:}(2,5) \quad 3{:}(3,4) \quad 4{:}(6,7) \quad 5{:}(9,12) \quad 6{:}(10,11).$$

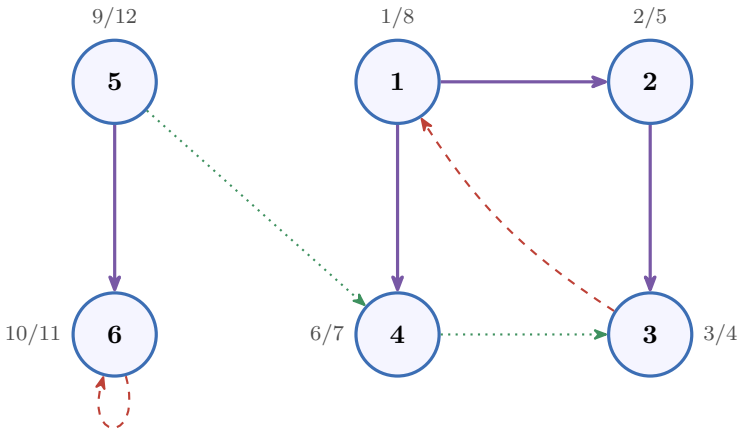


Figure 3.2: The DFS forest with d/f stamps. Tree edges (purple, solid), back edges (red, dashed), cross edges (green, dotted). Two trees: one rooted at 1, one at 5.

Notice the two roots 1 and 5 produced two trees; together they form the *DFS forest*. Notice too that the dashed back edges $3 \to 1$ and $6 \to 6$ each close a cycle, and that no cross edge ever points "into" a still-open subtree. Both facts hold for every DFS, not just this one, and we will prove them.

3.2 The algorithm

Here is DFS with timestamps and edge classification, written recursively. The wrapper loops over all vertices so that disconnected graphs (and digraphs not reachable from one root) are fully covered, yielding a forest rather than a single tree.

Algorithm 8: Depth-first search with timestamps and edge classification

input : Graph $G = (V, E)$ as adjacency lists; directed or undirected.
output: Stamps $d[\cdot]$, $f[\cdot]$, parent $\pi[\cdot]$, and an edge-type label for each edge.

```
1  foreach u ∈ V do
2      color[u] ← WHITE; π[u] ← NIL
3  time ← 0
4  foreach u ∈ V in fixed order do
5      if color[u] = WHITE then DFS-VISIT(u)
6  procedure DFS-VISIT(u)
7      time ← time + 1;  d[u] ← time;  color[u] ← GRAY
8      foreach v ∈ Adj[u] do
9          if color[v] = WHITE then
10             classify (u, v) as tree
11             π[v] ← u;  DFS-VISIT(v)
12         else if color[v] = GRAY then
13             classify (u, v) as back
14         else
15             if d[u] < d[v] then classify (u, v) as forward
16             else classify (u, v) as cross
17     time ← time + 1;  f[u] ← time;  color[u] ← BLACK
```

The three colors hold all the state the algorithm needs. WHITE means undiscovered, GRAY means discovered-but-not-finished (equivalently, *currently on the recursion stack*), and BLACK means finished. An edge (u, v) encountered while scanning u's list is classified purely by the color of v at that instant, with one numeric tiebreak between forward and cross. We prove that this classification is correct and complete in Section 3.4.

Common trap

On large graphs the recursion in Algorithm 8 can blow the call stack: a path graph on $n = 10^6$ vertices recurses a million deep, and most language runtimes overflow well before that. In production code DFS is run with an explicit stack. The subtlety is that a correct iterative DFS must record *where it was* in each adjacency list (an iterator per stack frame) and must run the "finish" bookkeeping when a frame is popped — not when it is first pushed. Iterative versions that timestamp on push compute $d[\cdot]$ correctly but get $f[\cdot]$ and the parenthesis structure wrong. We give a correct iterative skeleton in Exercise 3.3.

3.2.1 The iterative skeleton

Because the recursion above is the part most likely to break on real inputs, it is worth seeing the explicit-stack version in full once, rather than waving at it. The trick that makes it match the recursive timestamps exactly is to store, with each stacked vertex, an *index* into its adjacency list — the cursor marking how far we have scanned — and to emit the finish stamp only when a frame is finally popped because its cursor has run off the end of the list.

Algorithm 9: Iterative DFS with a per-frame adjacency cursor

input : Graph $G = (V, E)$ as adjacency lists; a start vertex s.
output: Stamps $d[\cdot], f[\cdot]$ identical to the recursive DFS rooted at s.

```
1  time ← 0
2  push frame (s, 0) onto stack S
3  time ← time + 1;   d[s] ← time;   color[s] ← GRAY
4  while S nonempty do
5      let (u, i) be the top frame of S
6      if i < |Adj[u]| then
7          v ← Adj[u][i]
8          increment the stored cursor of the top frame to i + 1
9          if color[v] = WHITE then
10             π[v] ← u
11             time ← time + 1;   d[v] ← time;   color[v] ← GRAY
12             push frame (v, 0) onto S
13     else
14         time ← time + 1;   f[u] ← time;   color[u] ← BLACK
15         pop S
```

The single most important line is the **else** branch: a vertex is finished and blackened only when its cursor has walked off the end of its list, i.e. when control would have returned from the recursive call. Push the start onto the stack, advance the top frame's cursor one neighbor at a time, and you reproduce the recursion's control flow exactly — including the order in which finishes happen, which is what determines the parenthesis structure. Run this skeleton on Figure 3.1 starting at 1 and the stamps come out 1:$(1, 8)$, 2:$(2, 5)$, 3:$(3, 4)$, 4:$(6, 7)$, matching the recursive trace tick for tick. Edge classification slots in at the point where v is examined, using the same color test as Algorithm 8.

3.2.2 Complexity

Theorem 3.1 (Linear time)**.** *On a graph with $n = |V|$ vertices and $m = |E|$ edges given as adjacency lists, Algorithm 8 runs in $\Theta(n + m)$ time and $\Theta(n)$ working space beyond the input.*

Proof. DFS-Visit is called exactly once per vertex: it is called only on WHITE vertices, and its first action is to color the vertex GRAY, so no vertex is visited twice. The initialization loop and the outer loop are $\Theta(n)$. Inside the single visit of u, the **for** loop scans $Adj[u]$ once, doing $O(1)$ work per incident edge. Summing the list lengths over all vertices counts each directed edge once, and each undirected edge twice (once from each endpoint), so the total edge-scanning work is $\Theta(m)$ either way. The two timestamp updates per vertex add $\Theta(n)$. Total: $\Theta(n + m)$. The color, d, f, and π arrays are $\Theta(n)$; the recursion stack is at most n deep, also $\Theta(n)$. □

Cost summary

DFS (adjacency lists). Time $\Theta(n+m)$. Extra space $\Theta(n)$ (arrays plus stack depth $\leq n$). On an adjacency *matrix* the edge scan becomes $\Theta(n^2)$ regardless of m, since finding the neighbors of u costs $\Theta(n)$. The timestamps, parent pointers, and edge labels are all produced within

this same bound, at no extra asymptotic cost.

3.3 The parenthesis theorem

The single most useful fact about DFS is that the time intervals $[d[u], f[u]]$ behave like well-formed parentheses: any two are either disjoint or nested, never partially overlapping. Write the interval of u as $I_u = [d[u], f[u]]$.

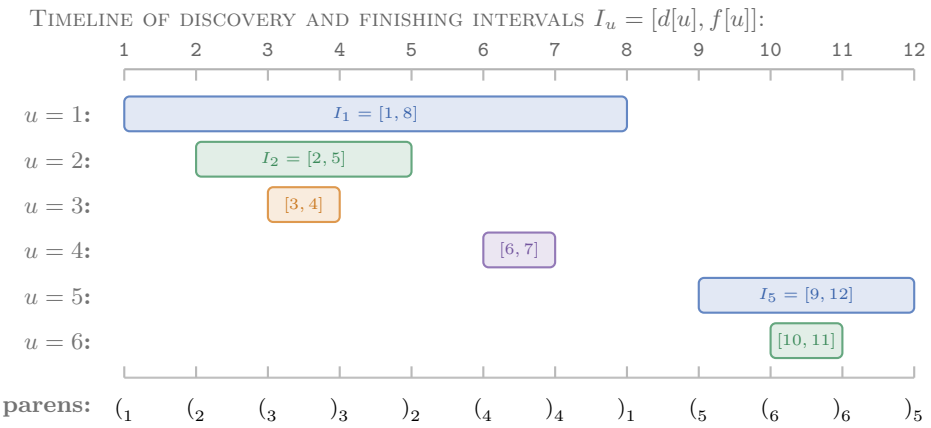
Intuition

Think of $d[u]$ as printing an open paren tagged u and $f[u]$ as printing the matching close paren tagged u. Because DFS is a stack discipline — you cannot finish u until everything you started after entering u has finished — the printed string is a balanced parenthesization. For our trace the string is

$$(_1\ (_2\ (_3\ )_3\ )_2\ (_4\ )_4\ )_1\ (_5\ (_6\ )_6\ )_5$$

and you can read the entire tree shape off the nesting: 3 is inside 2 is inside 1; 4 is inside 1 but beside 2; 6 is inside 5.

Plotting each vertex's active interval $I_u = [d[u], f[u]]$ as a horizontal bar along the clock axis $t = 1, \ldots, 12$ shows this laminar nesting directly above the matching parenthesis string:



Theorem 3.2 (Parenthesis theorem)**.** *In any DFS of a (directed or undirected) graph, for every pair of vertices $u \neq v$ exactly one of the following holds:*

(i) I_u *and* I_v *are disjoint, and neither of* u, v *is a descendant of the other in the DFS forest;*

(ii) $I_u \subset I_v$ *(strictly), and* u *is a descendant of* v*;*

(iii) $I_v \subset I_u$ *(strictly), and* v *is a descendant of* u*.*

In particular the intervals are laminar*: no two of them properly cross.*

Proof. All four numbers $d[u], f[u], d[v], f[v]$ are distinct (every clock value is unique). Without loss of generality assume $d[u] < d[v]$; the other case is symmetric by swapping names. There are two sub-cases depending on whether $d[v] < f[u]$.

Case A: $d[v] < f[u]$. Then v is discovered while u is gray, i.e. while u is still on the stack. By the stack discipline, u cannot be popped (finished) until every vertex discovered after u and still on the stack above it has been finished; v is such a vertex. Hence v finishes before u: $f[v] < f[u]$. Combining, $d[u] < d[v] < f[v] < f[u]$, so $I_v \subset I_u$. In addition, v was reached during the call DFS-Visit(u), before that call returned, so by the recursive structure v lies in the subtree rooted at u — v is a descendant of u. This is case (iii).

Case B: $d[v] > f[u]$. Then v is discovered only after u has already finished, so the entire interval I_u precedes $d[v]$: $d[u] < f[u] < d[v] < f[v]$. The intervals are disjoint. Neither is a descendant of the other: v cannot be a descendant of u because the call DFS-Visit(u) had already returned before v was touched; and u cannot be a descendant of v because u was discovered first. This is case (i).

Since exactly one of $d[v] < f[u]$ or $d[v] > f[u]$ holds (they are distinct), exactly one conclusion holds. Swapping the roles of u, v when $d[v] < d[u]$ gives case (ii). □

Corollary 3.1 (Nesting of descendants)**.** *v is a proper descendant of u in the DFS forest if and only if $d[u] < d[v] < f[v] < f[u]$, i.e. if and only if $I_v \subset I_u$.*

Proof. Immediate from Theorem 3.2: containment of intervals and the ancestor/descendant relation coincide. □

This corollary is our primary structural tool. It turns the question "is v in u's subtree?" — which sounds like it needs a tree traversal — into a pair of integer comparisons on precomputed stamps. The Euler-tour technique of Chapter 21 is built on this idea, and the heavy-light and centroid decompositions lean on it too.

3.3.1 The white-path theorem

The parenthesis theorem describes the output; the white-path theorem describes *why* a vertex ends up where it does, and it is the cleanest tool for proving later facts about reachability.

Theorem 3.3 (White-path theorem)**.** *In a DFS forest of G, vertex v is a descendant of vertex u if and only if, at the moment $d[u]$ when DFS first discovers u, there is a path from u to v consisting entirely of white (as-yet-undiscovered) vertices — u itself permitted as the white start.*

Proof. ($\Rightarrow$) Suppose v is a descendant of u. Let $u = x_0, x_1, \ldots, x_k = v$ be the tree path from u down to v. Each x_i is a tree-child of x_{i-1}, hence discovered during x_{i-1}'s visit, hence after u; so each x_i with $i \geq 1$ is discovered strictly after time $d[u]$, meaning all of $x_1, \ldots, x_k$ are white at time $d[u]$. Together with u itself this is a white path from u to v.

($\Leftarrow$) Suppose at time $d[u]$ there is a white path $u = x_0, x_1, \ldots, x_k = v$. We show by induction on i that each x_i becomes a descendant of u (with u counted as a descendant of itself). The base case $x_0 = u$ is trivial. Inductive step: assume x_{i-1} is a descendant of u, so $d[u] \leq d[x_{i-1}]$ and $I_{x_{i-1}} \subseteq I_u$. Consider the edge $x_{i-1} \to x_i$, and ask when x_i is first discovered.

Suppose, for contradiction, x_i is *not* a descendant of u. By the parenthesis theorem the intervals I_u and I_{x_i} are then disjoint, and since x_i is white at $d[u]$ it is discovered after $d[u]$; disjointness forces I_u to come entirely before I_{x_i}, so $d[x_i] > f[u] \geq f[x_{i-1}]$. But the edge $x_{i-1} \to x_i$ is scanned during the visit of x_{i-1}, at a time strictly before $f[x_{i-1}] < d[x_i]$; at that scan instant x_i has not yet been discovered, so it is white, and DFS would immediately discover x_i as a tree-child of x_{i-1}. That makes

x_i a descendant of x_{i-1}, hence of u, contradicting our assumption. Therefore x_i is a descendant of u, completing the induction. Taking $i = k$ gives that v is a descendant of u. □

Why it works

The white-path theorem says the DFS subtree of u is *exactly the set of vertices reachable from u through still-undiscovered territory at the instant u is opened*. This is why DFS subtrees correspond to "reachable regions," and it is the key step in Tarjan's strongly-connected-components algorithm (Chapter 5): if you arrange the DFS so that the white-reachable region from u is precisely u's SCC, the subtree hands you the component for free. It is also why a single DFS in a DAG, ordered by finishing time, produces a topological order (Chapter 4).

3.4 Edge classification

Run DFS and every edge of G falls into one of four types, determined by the relationship between the endpoints in the DFS forest. For a directed edge (u, v):

- **Tree edge:** (u, v) was used to first discover v; v was white when scanned. These edges form the DFS forest.
- **Back edge:** v is an ancestor of u in the forest (u is in v's subtree). Equivalently v is gray when (u, v) is scanned. A self-loop counts as a back edge.
- **Forward edge:** v is a proper, non-child descendant of u — reached by an already-built path inside u's subtree. Then v is black when scanned and $d[u] < d[v]$.
- **Cross edge:** u and v are unrelated (neither is an ancestor of the other), and the edge points from a later-finished subtree to an earlier-finished one. Then v is black when scanned and $d[v] < d[u]$.

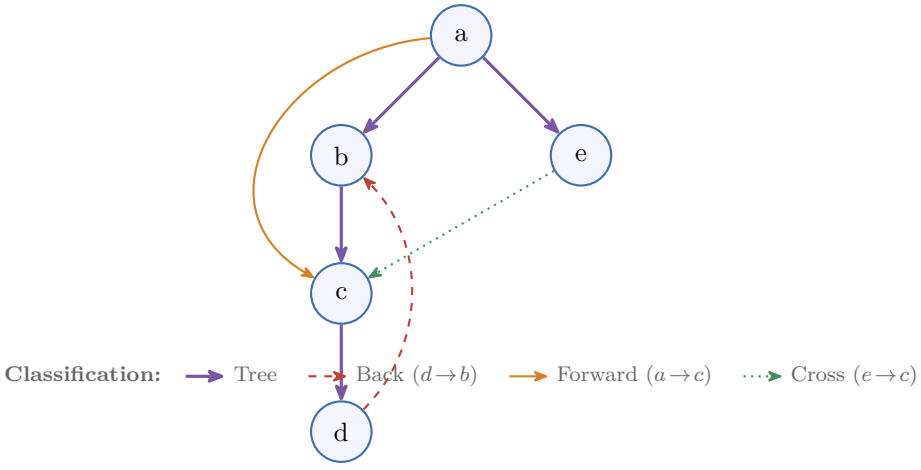


Figure 3.3: The four directed edge types on one forest. Back edges go up to an ancestor; forward edges skip down to a deeper descendant; cross edges jump between unrelated subtrees, always toward something already finished.

3.4.1 Reading the type off the timestamps

The branch logic in Algorithm 8 classifies by color at scan time, but it is often more convenient to classify *after the fact* from the final d/f stamps alone. The following lemma makes that precise and, in passing, proves the algorithm's color-based test is correct.

Lemma 3.1 (Timestamp signature of edge types)**.** *Let (u, v) be a directed edge of G. After DFS completes:*

(T/F) (u, v) is a tree or forward edge $\iff$ $d[u] < d[v] < f[v] < f[u]$ (i.e. $I_v \subset I_u$);

(B) (u, v) is a back edge $\iff$ $d[v] \le d[u] < f[u] \le f[v]$ (i.e. $I_u \subseteq I_v$); a self-loop gives equality;

(C) (u, v) is a cross edge $\iff$ $d[v] < f[v] < d[u] < f[u]$ (i.e. I_v entirely precedes I_u).

Proof. By the parenthesis theorem the two intervals are disjoint or nested, giving exactly three layouts for distinct vertices: $I_v \subset I_u$, $I_u \subset I_v$, or disjoint (and for disjoint, I_v before I_u or after). We match each edge type to a layout.

If (u, v) is a *tree* edge, v is a child of u, so by Corollary 3.1 $I_v \subset I_u$, signature (T/F). If (u, v) is a *forward* edge, v is a deeper descendant of u, again $I_v \subset I_u$. Conversely, suppose $I_v \subset I_u$; then v is a descendant of u, so when u's list is scanned and reaches v, either v is still white (giving a tree edge) or already discovered inside u's subtree (giving a forward edge). Either way it is tree-or-forward. This proves (T/F).

If (u, v) is a *back* edge, v is an ancestor of u (or $u = v$), so $I_u \subseteq I_v$. Conversely if $I_u \subseteq I_v$ with $u \neq v$ then u is a descendant of v; the edge $u \to v$ points up to an ancestor, which is exactly a back edge. Self-loops have $I_u = I_v$. This proves (B).

The only remaining layout is I_v and I_u disjoint. If I_u preceded I_v (so $f[u] < d[v]$), then when u's list was scanned — during I_u, before $d[v]$ — the vertex v was still white, and DFS would have made v a child of u, forcing $I_v \subset I_u$, a contradiction. Hence the only consistent disjoint layout for an actual edge is I_v before I_u, i.e. $d[v] < f[v] < d[u]$. That is the cross signature (C), and it is what is left over. □

Common trap

A common mistake is to believe "edge (u, v) with $d[u] < d[v]$ is a tree or forward edge." That is necessary but not sufficient — the nesting $f[v] < f[u]$ is what seals it. The clean rule is the interval rule of Lemma 3.1: compare *both* endpoints' intervals, not just discovery times. Cross edges always satisfy $d[v] < d[u]$, which is why the algorithm's forward/cross tiebreak compares discovery times.

3.4.2 Verifying on the trace

Return to our digraph timestamp intervals: 1:$(1, 8)$, 2:$(2, 5)$, 3:$(3, 4)$, 4:$(6, 7)$, 5:$(9, 12)$, and 6:$(10, 11)$, and check Lemma 3.1 edge by edge.

edge	I_u	I_v	type
$1 \to 2$	$[1, 8]$	$[2, 5]$	$I_v \subset I_u$, tree
$1 \to 4$	$[1, 8]$	$[6, 7]$	$I_v \subset I_u$, tree
$2 \to 3$	$[2, 5]$	$[3, 4]$	$I_v \subset I_u$, tree
$3 \to 1$	$[3, 4]$	$[1, 8]$	$I_u \subset I_v$, back
$4 \to 3$	$[6, 7]$	$[3, 4]$	I_v before I_u, cross
$5 \to 4$	$[9, 12]$	$[6, 7]$	I_v before I_u, cross
$5 \to 6$	$[9, 12]$	$[10, 11]$	$I_v \subset I_u$, tree
$6 \to 6$	$[10, 11]$	$[10, 11]$	$I_u = I_v$, back (self-loop)

Every label matches what we recorded during the live trace — the after-the-fact interval rule and the on-the-fly color rule agree, as the lemma promised. There are no forward edges in this particular graph; add the edge $1 \to 3$ and it would be classified forward ($I_3 = [3, 4] \subset I_1 = [1, 8]$ but 3 is not a child of 1).

3.5 Cycle detection

DFS turns cycle detection into a search for one edge type. The directed and undirected cases differ in a way that trips up nearly everyone the first time, so we treat them separately and prove both.

3.5.1 Directed graphs: back edge $\iff$ cycle

Theorem 3.4 (Back edges and directed cycles)**.** *A directed graph G contains a (directed) cycle if and only if any DFS of G produces a back edge.*

Proof. ($\Leftarrow$) Suppose DFS produces a back edge (u, v). Then v is an ancestor of u (or $u = v$), so there is a directed tree path $v \rightsquigarrow u$. Appending the edge $u \to v$ closes a directed cycle $v \rightsquigarrow u \to v$.

($\Rightarrow$) Suppose G has a directed cycle C. Let v be the first vertex of C that DFS discovers (the one with smallest d among C's vertices), and let $u \to v$ be the edge of C entering v (its predecessor on the cycle). At time $d[v]$, every other vertex of C is white, and the cycle gives a directed path $v \rightsquigarrow u$ all of whose vertices lie on C, hence are white at $d[v]$. By the white-path theorem, u becomes a descendant of v. But then the edge $u \to v$ points from a descendant up to its ancestor v, which is a back edge. □

This gives a clean linear-time test: run DFS; the moment you scan an edge into a GRAY vertex, report a cycle (and you can recover it by walking the gray stack from u up to v). No need to finish the whole search.

Algorithm 10: Directed cycle detection via gray vertices

input : Directed graph $G = (V, E)$.
output: TRUE with a witness cycle if one exists, else FALSE.

```
1  color all vertices WHITE
2  foreach u ∈ V do
3  |  if color[u] = WHITE then if PROBE(u) then return TRUE
4  |
5  return FALSE
6  procedure PROBE(u)
7  |  color[u] ← GRAY
8  |  foreach v ∈ Adj[u] do
9  |  |  if color[v] = GRAY then
10 |  |  |  return TRUE (back edge u → v; cycle is the gray-stack segment v ⇝ u plus u → v)
11 |  |  else if color[v] = WHITE then
12 |  |  |  if PROBE(v) then return TRUE
13 |  color[u] ← BLACK;   return FALSE
```

3.5.2 Undirected graphs: the parent trap

Common trap

In an undirected graph, the tree edge $u - v$ you just came down appears again when you scan v's list (because the edge $\{u, v\}$ lives on both lists). If you blindly call "an edge to a gray vertex" a back edge, you will declare a cycle for *every* tree edge — a catastrophic false positive. The fix: when scanning v, ignore the single edge back to its DFS parent $\pi[v] = u$. Only an edge to a gray vertex that is *not* the parent signals a real cycle.

There is a sharper subtlety than "skip the parent": you must skip the parent *edge*, not every edge to the parent vertex. If G has *parallel* edges (a multigraph) between u and v, two distinct edges connect them, and the second one is a genuine cycle of length two even though it lands on the parent vertex. Robust code skips by edge identity (or remembers it has already used the one parent edge), not by vertex name.

Theorem 3.5 (Undirected cycles via non-parent back edges)**.** *Let G be a simple undirected graph (no self-loops, no parallel edges). G contains a cycle if and only if a DFS of G encounters an edge $\{u, v\}$ with v gray and $v \neq \pi[u]$.*

Proof. First note that in undirected DFS *every* non-tree edge is a back edge: there are no forward or cross edges. Indeed, suppose $\{u, v\}$ is scanned from u and v is not white. If v were black, then $f[v]$ is in the past, so v finished before u's scan; but the edge $\{u, v\}$ is also on v's list, and when v scanned it earlier, u was either white (making $\{u, v\}$ a tree edge with u a child of v) or gray (making u an ancestor of v) — in both cases u and v are in an ancestor/descendant relation, so the edge is a tree or back edge, not a cross edge. Hence the only non-tree possibility when scanning from u is v gray: an ancestor of u. So undirected DFS yields only tree and back edges (Proposition 3.1 restates this).

($\Leftarrow$) A back edge $\{u, v\}$ with $v \neq \pi[u]$ and v an ancestor of u means there is a tree path $v \rightsquigarrow u$ of length ≥ 2 (since v is not the immediate parent), and adding $\{u, v\}$ closes a cycle of length ≥ 3.

($\Rightarrow$) Suppose G has a cycle $C = w_0 - w_1 - \cdots - w_{k-1} - w_0$ with $k \geq 3$. Let w_i be the first vertex of C discovered. Its two cycle-neighbors are white at $d[w_i]$, and every vertex of C is reachable from w_i along C through vertices white at $d[w_i]$; by the white-path theorem all of C lies in w_i's subtree. Now walk the cycle starting from w_i along one arc; the discovery times need not increase monotonically forever, so let $\{x, y\}$ be the first cycle edge (in tree-walk order from w_i) whose endpoint y was *already discovered* when x scanned it. Such an edge exists because the cycle returns to the already-discovered w_i. At the moment x scans $\{x, y\}$, vertex y is gray (it is an ancestor of x on the tree, being discovered earlier and still open), and $y \neq \pi[x]$ because $\{x, y\}$ is a cycle chord rather than the tree edge by which x was reached. The non-parent gray condition fires. □

Algorithm 11: Undirected cycle detection (skip the entering edge by identity)

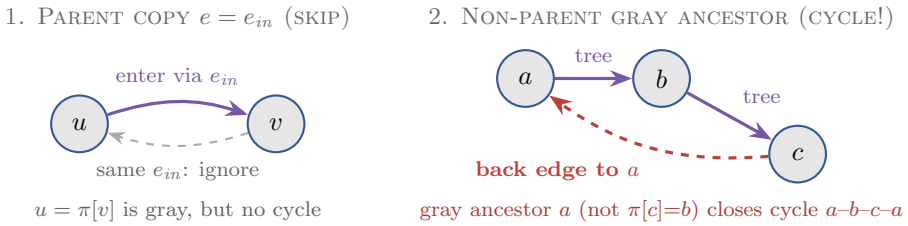
input : Undirected graph $G = (V, E)$ as adjacency lists of (neighbor, edge-id).
output: TRUE if G has a cycle, else FALSE.

```
1  color all WHITE
2  foreach s ∈ V do
3  |  if color[s] = WHITE then if PROBE(s, NIL) then return TRUE
4  |
5  return FALSE
6  procedure PROBE(u, e_in)
7  |  color[u] ← GRAY
8  |  foreach (v, e) ∈ Adj[u] do
9  |  |  if e = e_in then continue (skip the exact edge we entered through)
10 |  |  if color[v] = GRAY then
11 |  |  |  return TRUE
12 |  |  else if color[v] = WHITE then
13 |  |  |  if PROBE(v, e) then return TRUE
14 |  color[u] ← BLACK;   return FALSE
```

Trace it yourself

Trace Algorithm 11 on the triangle $a - b - c - a$. PROBE$(a, \perp)$ grays a, recurses on b via edge ab. PROBE(b, ab) grays b, skips ab, recurses on c via bc. PROBE(c, bc) grays c, skips bc, scans ca: neighbor a is gray and edge $ca \neq bc$ — back edge, return TRUE. Now contrast the path $a - b$: PROBE$(a, \perp)$ recurses on b via ab; PROBE(b, ab) skips ab, finds no other neighbor, returns FALSE. No false alarm.

The two cases of this test are contrasted below: scanning the reverse copy of the entering tree edge $e_{in} = \{u, v\}$ back to the parent $u = \pi[v]$ is filtered out, whereas scanning a distinct edge to an earlier gray ancestor w closes a genuine cycle:



Why it works

Why does the directed test need *no* parent skip while the undirected one does? In a digraph the edge $u \to v$ and the edge $v \to u$ are different objects; coming down a tree edge $\pi[v] \to v$ does not put a $v \to \pi[v]$ edge on v's list. In an undirected graph the single edge $\{u, v\}$ is stored on both endpoints' lists, so traversing it once leaves a phantom "reverse" copy that must be filtered out. The asymmetry is entirely an artifact of how undirected edges are represented, and it is the prototype of a dozen later "don't walk back up the edge you came from" guards (bridges, biconnectivity, Euler tours).

3.5.3 Counting independent cycles

DFS also hands you the dimension of the cycle space cheaply. In an undirected connected graph the DFS tree has $n - 1$ edges; every one of the remaining $m - (n - 1)$ edges is a back edge, and each back edge closes exactly one fundamental cycle. The number $m - n + 1$ is the *cyclomatic number* (first Betti number) of the graph, and the fundamental cycles of the back edges form a basis of the cycle space.

Corollary 3.2. *For an undirected graph with c connected components, the number of independent cycles (back edges across a DFS forest) is $m - n + c$. The graph is a forest iff this is 0, i.e. iff DFS finds no back edge.*

Proof. The DFS forest has one tree per component, with $n - c$ tree edges in total (a tree on n_i vertices has $n_i - 1$ edges, summed over components). By Proposition 3.1 every remaining edge is a back edge, and there are $m - (n - c) = m - n + c$ of them. Each is the unique non-tree edge of its fundamental cycle, and standard linear algebra over GF(2) shows these fundamental cycles are independent and span the cycle space, whose dimension is therefore $m - n + c$. □

3.5.4 A worked undirected trace with low-links

The directed example showed all four edge types; an undirected example shows the cleaner two-type world of Proposition 3.1 and previews the *low*-value bookkeeping that powers Chapter 10. Take the undirected graph on five vertices $\{a, b, c, d, e\}$ with edges $\{a, b\}, \{b, c\}, \{c, a\}, \{c, d\}, \{d, e\}$ — a triangle $a\,b\,c$ with a tail $c - d - e$ hanging off it.

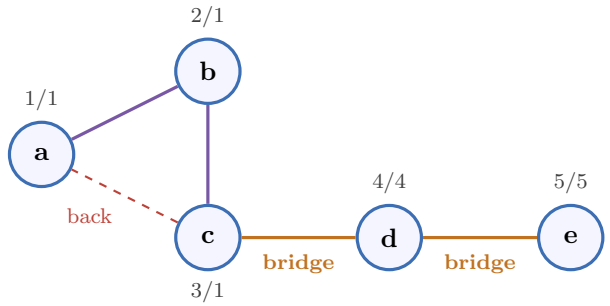


Figure 3.4: Undirected trace started at a, annotated with $d[v]/low[v]$. The back edge c–a (red, dashed) pulls $low[c]$ and $low[b]$ down to 1, protecting the triangle; the tail edges c–d and d–e satisfy $low[v] > d[\pi[v]]$ and are bridges (orange).

Starting the DFS at a and scanning each list alphabetically, the discovery order is a, b, c, d, e. The edge $c-a$ is scanned from c while a is still gray and $a \neq \pi[c] = b$: a genuine back edge, so the triangle is a cycle. Now compute $low[v]$ bottom-up, where $low[v]$ is the smallest discovery time reachable from v's subtree using tree edges downward and at most one back edge:

v	$d[v]$	back edge from subtree	$low[v]$	tree edge $(\pi[v], v)$ a bridge?
e	5	none	5	$low[e] = 5 > d[d] = 4$ **bridge**
d	4	none	4	$low[d] = 4 > d[c] = 3$ **bridge**
c	3	$c-a$ reaches d=1	1	$low[c] = 1 \not> d[b] = 2$
b	2	via c, reaches 1	1	$low[b] = 1 \not> d[a] = 1$
a	1	— (root)	1	root, not applicable

Read off the bridge criterion $low[v] > d[\pi[v]]$: the tail edges $c - d$ and $d - e$ are bridges (removing either disconnects the graph), while the triangle edges $a - b$, $b - c$, $c - a$ are not (each lies on the cycle $a\,b\,c$). This is exactly what Chapter 10's bridge-finding algorithm computes; here we did it by hand from one DFS pass.

3.6 Directed versus undirected: what changes

It is worth pinning down precisely which edge types can occur, because later chapters silently rely on these restrictions.

Proposition 3.1 (Undirected DFS has only tree and back edges)**.** *A DFS of an undirected graph classifies every edge as either a tree edge or a back edge; forward and cross edges never occur.*

Proof. Established inside the proof of Theorem 3.5: an edge to a non-white vertex must reach a gray ancestor, hence a back edge; if it reached white it is a tree edge. The orientation "forward" (down to a deeper descendant) is impossible because such an edge would have been used as a tree edge from the shallower endpoint when that endpoint scanned it first; and "cross" (between unrelated subtrees) is impossible because an undirected edge between u and v forces one to be discovered while scanning the other, creating an ancestor relationship. □

In a *directed* graph all four types appear. The key structural facts:

- **No back edge** $\iff$ the digraph is acyclic (Theorem 3.4). A DAG's DFS therefore has only tree, forward, and cross edges, and *ordering vertices by decreasing finish time topologically sorts it* (Chapter 4).
- **Cross edges always go "right to left"** in time: for a cross edge (u, v), $f[v] < d[u]$, so v's whole subtree finished before u began. This monotonicity is what makes Kosaraju's and Tarjan's SCC algorithms work (Chapter 5).
- In undirected graphs, the absence of cross edges is what lets the single number $low[v]$ — the earliest discovery time reachable from v's subtree via one back edge — summarize all the connectivity you need for bridges and articulation points (Chapter 10).

History

Depth-first search is ancient as a maze procedure — Charles Pierre Trémaux described a string-and-mark method for traversing mazes in the 1880s, and Édouard Lucas reported it. Its transformation from "a way to walk a maze" into "a way to compute global graph structure in linear time" is due chiefly to Robert Tarjan and John Hopcroft around 1971–1973. Tarjan's insight was that the discovery/finish timestamps and the *low*-value bookkeeping expose strong connectivity, biconnectivity, planarity, and dominators, all in $O(n + m)$. The parenthesis structure is implicit in any stack-based recursion, but naming it and reading structural invariants off its intervals turned DFS into a general-purpose foundation for linear-time graph algorithms. Almost every algorithm in Chapters 4, 5, 10, and 21 is a Tarjan-style reading of the timestamps.

3.7 The structure DFS exposes

We close by collecting the structural payoffs, because the entire reason to prefer DFS over BFS for these problems is that DFS produces *more* than a traversal — it produces a forest plus a total order on vertices (by d or by f) that together reveal cycles, components, and connectivity.

Application

Topological sort (Ch. 4). Decreasing finish time is a topological order of any DAG. One DFS, then read $f[\cdot]$ in reverse.

Strongly connected components (Ch. 5). Kosaraju: DFS for finish order, then DFS the transpose in that order; each tree is an SCC. Tarjan: a single DFS with a *low*-link and an auxiliary stack pops one SCC at a time. Both are timestamp readings.

Bridges and articulation points (Ch. 10). $low[v]$ = min of $d[v]$ and the discovery times reachable by one back edge from v's subtree. An edge (u, v) is a bridge iff $low[v] > d[u]$.

Euler tour / subtree-as-interval (Ch. 21, 23). The pair $(d[v], f[v])$ is exactly an interval; "v in u's subtree" becomes "$I_v \subset I_u$," turning subtree queries into range queries on a segment tree.

Dominator trees, planarity testing. Both are built on DFS numbering and the low-link/semidominator machinery.

Connection

The interval picture connects forward to the linear-algebra companion. Order vertices by discovery time and the back edges of an undirected DFS are exactly the entries that make the graph's structure differ from that of a tree; the tree edges form a spanning tree whose incidence matrix has full rank $n-1$, and the $m-n+1$ back edges index a basis of the cycle space — the kernel of the boundary map, dual to the cut space spanned by the tree. When we reach the graph Laplacian $\mathbf{L}$ (Ch. 31) and electrical flows (Ch. 35, 41), the cycle/cut decomposition that DFS exposes here reappears as the orthogonal decomposition of flows into circulations and potential-driven currents. Each back edge corresponds directly to a generator of that cycle space.

Pause

The mental model to keep: DFS gives each vertex an interval, and the intervals *nest*. Two numbers per vertex, laminar by the parenthesis theorem, and four edge types determined by how the intervals sit relative to each other. Cycle detection is "find a back edge" (with a parent guard when edges are undirected). Everything heavier — SCCs, bridges, topological order, Euler tours — is computed from those same two numbers.

In Chapter 4, we turn to the first of these readings: ordering the vertices of a directed acyclic graph. We will prove that sorting vertices by decreasing finish time yields a valid topological sort, compare this against Kahn's indegree-peeling method, and show how topological order gives the evaluation order for dynamic programming on DAGs.

3.8 Exercises

Exercise 3.1 (Trace and classify)**.** Run DFS on the digraph with edges $a \to b,\ a \to c,\ b \to d,\ c \to d,\ d \to a,\ c \to e,\ e \to c$, visiting roots and adjacency lists in alphabetical order. Record $d[\cdot]$ and $f[\cdot]$, draw the DFS forest, and label every edge as tree/back/forward/cross. Verify each label with the interval rule of Lemma 3.1.

Exercise 3.2 (Parenthesis string)**.** Prove that the sequence of discovery and finish events, written as labeled parentheses, is a balanced parenthesization, and that conversely any balanced parenthesization on n symbols is realizable as the event sequence of some DFS on some graph. How many distinct DFS forest shapes (nesting structures) on n labeled vertices are there?

Exercise 3.3 (Correct iterative DFS)**.** Write an explicit-stack DFS that reproduces $d[\cdot]$, $f[\cdot]$, $\pi[\cdot]$, and the edge classification of the recursive version *exactly*, including correct finish times. The trap: each stack frame must store an iterator into its adjacency list so it resumes where it left off, and the finish timestamp must be set when the frame is popped. Test it against Algorithm 8 on the graph of Figure 3.1.

Exercise 3.4 (Break the undirected detector)**.** Exhibit an undirected *multigraph* (two vertices, two parallel edges) on which the "skip the parent vertex" version of cycle detection wrongly reports no cycle, but the "skip the parent edge by identity" version of Algorithm 11 correctly reports one. Explain precisely which line saves it.

Exercise 3.5 (Forward edges require direction)**.** Prove that a forward edge (u, v) can exist only in a directed graph (cf. Proposition 3.1), and give the smallest directed graph (fewest edges) whose DFS, under a stated visiting order, produces a forward edge. Confirm with timestamps.

Exercise 3.6 (Cross-edge monotonicity)**.** Prove that for every cross edge (u, v) in a directed DFS we have $f[v] < d[u]$, and hence $f[v] < f[u]$. Then show that if a digraph's DFS has no back edge, ordering vertices by decreasing f gives an ordering in which every edge goes from earlier to later — a topological sort. (This is the seed of Chapter 4.)

Exercise 3.7 (Counting components and cycles)**.** Using only the DFS forest, give an $O(n + m)$ algorithm that returns the number of connected components c of an undirected graph and the dimension $m - n + c$ of its cycle space, and outputs an explicit fundamental cycle for one chosen back edge. Prove correctness via Corollary 3.2.

Exercise 3.8 (Order-dependence of the forest)**.** The DFS forest and the edge classification depend on the visiting order, but some facts are invariant. Prove that whether a directed graph has a back edge (equivalently, whether it has a cycle) is independent of the visiting order, even though *which* edges are back edges is not. Give a 3-vertex digraph where two visiting orders turn the same edge from a cross edge into a forward edge.

Exercise 3.9 (White-path in action)**.** Use the white-path theorem (Theorem 3.3) to prove the following without re-running DFS: if u is discovered before v and there is a directed path $u \rightsquigarrow v$ whose internal vertices are all undiscovered at time $d[u]$, then $f[v] < f[u]$. Where does the argument break if some internal vertex is already gray at $d[u]$?

Exercise 3.10 (Low-link preview)**.** For an undirected graph define $low[v] = \min\big(\{d[v]\} \cup \{d[w] : \{x, w\} \text{ a back edge with } x \text{ in } v\text{'s subtree}\}\big)$. Give a single-pass DFS that computes $low[v]$ for all v in $O(n + m)$, and prove that a tree edge (u, v) is a bridge iff $low[v] > d[u]$. (Full treatment in Chapter 10; here just establish the recurrence and the bridge criterion.)

Exercise 3.11 (Undirected bridges by hand)**.** Add the edge $\{e, a\}$ to the graph of Figure 3.4, turning the tail into a second cycle $a\,c\,d\,e$. Recompute every $low[v]$ from scratch and show that *no* tree edge now satisfies $low[v] > d[\pi[v]]$, so the augmented graph has no bridges. Explain in one sentence, using the *low* values, why a graph with no bridges cannot be disconnected by deleting any single edge.

Exercise 3.12 (Cursor correctness)**.** Prove that Algorithm 9 assigns exactly the same $d[\cdot]$ and $f[\cdot]$ as the recursive Algorithm 8 when both start from the same root and scan adjacency lists in the same order. The crux is the **else** branch: argue that a frame is popped (and its f stamp set) at precisely the clock tick when the matching recursive call would return. Then modify the skeleton to also emit the four edge labels, and confirm it reproduces the table of Section 3.4.

Exercise 3.13 (Deep recursion stress test)**.** Construct the family of graphs on which recursive DFS recurses to depth exactly n, and a family on which it recurses to depth $O(\log n)$ for all visiting orders. For the first family, estimate the largest n your language's default stack survives, then confirm your iterative DFS from Exercise 3.3 handles $10\times$ that.

Chapter 4

Topological Sort and DAGs

In Chapter 3, depth-first search timestamps showed us how to detect cycles: a directed graph has a cycle if and only if DFS encounters a back edge. When no directed cycles exist, the graph is a directed acyclic graph – a DAG – the core model of dependency. Every time you compile code, resolve package dependencies, or schedule tasks, you are walking a DAG. The central operation on a DAG is to linearize its partial order into a sequence respecting every prerequisite: a topological order. Once vertices are topologically ordered, problems that are NP-hard on general graphs (such as longest paths) collapse into linear-time dynamic programs. This chapter builds topological orderings via DFS finish times and Kahn's indegree-peeling algorithm, proves both methods correct, and applies the order to dynamic programming.

4.1 What a DAG Is, and Why Acyclicity Unlocks Linear Order

Definition 4.1 (Directed acyclic graph)**.** A directed graph $G = (V, E)$ is a *directed acyclic graph* (DAG) if it contains no directed cycle: there is no sequence of vertices $v_0, v_1, \ldots, v_k = v_0$ with $k \geq 1$ and every $(v_{i-1}, v_i) \in E$.

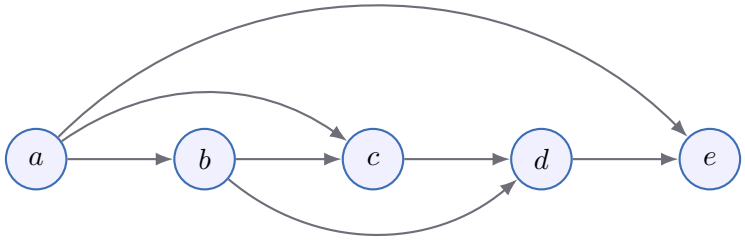
The prohibition is on *directed* cycles. An undirected triangle is fine to orient as $a \to b \to c$, $a \to c$; nothing closes a loop. What kills a DAG is a back-pointing edge that lets you return to where you started.

Intuition

Read each edge $u \to v$ as "u must come before v," or "v depends on u." A directed cycle $u \to \cdots \to u$ then says "u must come strictly before itself," which is impossible to satisfy. So acyclicity is precisely the condition that the constraints can *all* be met at once. A DAG is a consistent set of "before" rules; topological sort is the act of obeying them.

Definition 4.2 (Topological order)**.** A *topological order* of a digraph $G = (V, E)$ is a linear order $\prec$ on V (equivalently, a bijection $\pi \colon V \to \{1, \ldots, n\}$) such that for every edge $(u, v) \in E$ we have $\pi(u) < \pi(v)$: every edge points forward in the order.

If we lay the vertices on a horizontal line in topological order, *every* arrow points to the right. That picture is worth memorizing.



Focus

Two facts drive everything in this chapter:

1. A digraph has a topological order *if and only if* it is a DAG.
2. Both standard algorithms (DFS finish times, Kahn indegree peeling) run in $O(n+m)$ and either produce an order or expose a cycle.

Theorem 4.1 (Existence)**.** *A finite digraph G has a topological order if and only if it is acyclic.*

Proof. ($\Rightarrow$) Suppose π is a topological order and, for contradiction, G has a cycle $v_0 \to v_1 \to \cdots \to v_k = v_0$. Each edge forces $\pi(v_{i-1}) < \pi(v_i)$, so chaining gives $\pi(v_0) < \pi(v_1) < \cdots < \pi(v_k) = \pi(v_0)$, i.e. $\pi(v_0) < \pi(v_0)$, impossible.

($\Leftarrow$) Suppose G is acyclic. We build the order greedily, which also proves the next lemma. Pick any vertex with no incoming edge (Lemma 4.1 below guarantees one exists), assign it the smallest unused position, delete it, and repeat on the rest. Deleting a vertex cannot create a cycle, so every subgraph is again a DAG and again has a source. After n steps every vertex has a position, and any surviving edge (u, v) had u removed before v (we only remove a vertex once it has no in-edges from undeleted vertices), so $\pi(u) < \pi(v)$. □

Lemma 4.1 (Every finite DAG has a source and a sink)**.** *A nonempty finite DAG contains at least one* source *(a vertex of indegree* 0*) and at least one* sink *(a vertex of outdegree* 0*).*

Proof. We show a source exists; the sink argument is symmetric (reverse all edges). Suppose every vertex had an incoming edge. Start at any v_0, follow an in-edge backward to a predecessor v_1, then a predecessor v_2 of v_1, and so on. Each step picks a predecessor, which exists by assumption. After n steps we have visited $n+1$ vertices $v_0, \ldots, v_n$, so by pigeonhole two coincide, say $v_i = v_j$ with $i < j$. The backward walk $v_i \leftarrow \cdots \leftarrow v_j$ reversed is a forward directed cycle $v_j \to \cdots \to v_i = v_j$ – contradicting acyclicity. Hence some vertex has indegree 0. □

Why it works

Lemma 4.1 is the key fact. "A DAG always has somewhere to start" is why both algorithms make progress: peeling needs a source to remove; DFS needs a vertex that finishes with all successors already processed (a "local sink"). Without acyclicity neither holds, and that failure is exactly how the algorithms detect a cycle.

History

Topological ordering was formalized by A. B. Kahn in 1962 in a paper on scheduling project networks (the PERT/CPM tradition of operations research). The depth-first formulation is folklore but is most cleanly stated through the finish-time machinery that Tarjan and Hopcroft popularized in the early 1970s, the same timestamp framework that powers strongly connected components and biconnectivity. The two views – "peel from the front" and "recurse and reverse" – are duals that we will meet again and again.

4.2 Method 1: Kahn's Algorithm (Indegree Peeling)

The constructive proof of Theorem 4.1 *is* an algorithm. Repeatedly: take a vertex with indegree 0, output it, and remove it (which decrements the indegree of its out-neighbors). The only engineering question is how to find zero-indegree vertices fast. The trick: maintain a running indegree count for every vertex and keep a queue of those currently at 0.

Intuition

Think of the DAG as a stack of dependencies and you are clearing a to-do list. A task is *ready* when all its prerequisites are done – i.e. its current indegree is 0. Do a ready task, which may make some other tasks ready, and add those to the ready set. The order in which tasks leave the ready set is a valid topological order. If at some point the list is nonempty but *nothing* is ready, the leftover tasks form a cycle of mutual prerequisites.

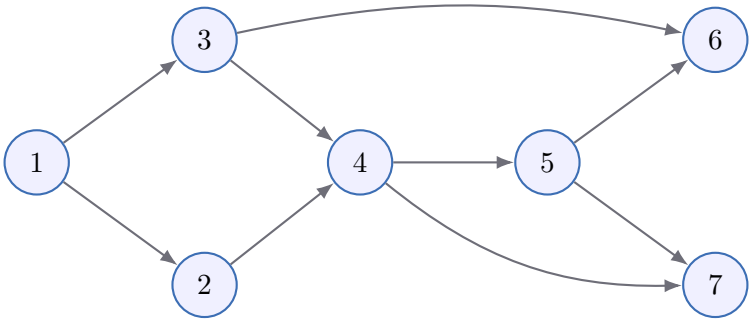
Algorithm 12: KahnTopoSort(G)

Input: Digraph $G = (V, E)$ as adjacency lists, $n = |V|$, $m = |E|$.
Output: A topological order L, or report "cycle."

```
foreach v ∈ V do
    indeg[v] ← 0
foreach edge (u, v) ∈ E do
    indeg[v] ← indeg[v] + 1
Q ← empty queue
foreach v ∈ V with indeg[v] = 0 do
    enqueue v into Q
L ← empty list
while Q is not empty do
    u ← dequeue(Q)
    append u to L
    foreach w with (u, w) ∈ E do
        indeg[w] ← indeg[w] − 1
        if indeg[w] = 0 then
            enqueue w into Q
if |L| = n then
    return L
else
    return "cycle" (the n − |L| unoutput vertices lie on/after a cycle)
```

Worked trace

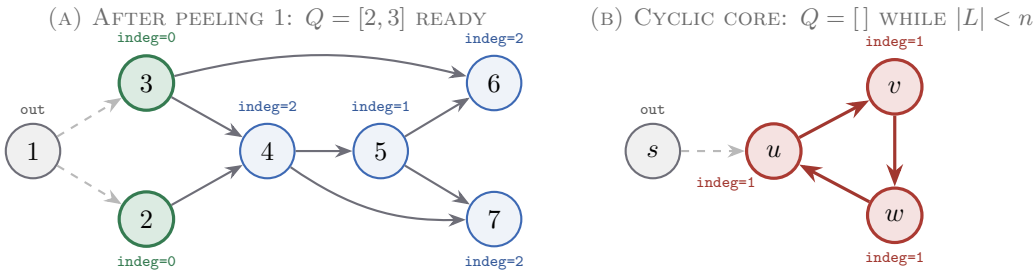
Take a concrete DAG of seven tasks. Read edges as "must precede."



Initial indegrees: 1:0, 2:1, 3:1, 4:2, 5:1, 6:2, 7:2. The only source is 1, so $Q = [1]$. We process FIFO; ties break by smaller vertex number for determinism.

step	dequeue	decrements (new indeg)	queue after
1	1	2:0, 3:0	[2, 3]
2	2	4:1	[3]
3	3	4:0, 6:1	[4]
4	4	5:0, 7:1	[5]
5	5	6:0, 7:0	[6, 7]
6	6	—	[7]
7	7	—	[ ]

Output order: $L = (1, 2, 3, 4, 5, 6, 7)$. All seven vertices appear, so no cycle. Sanity check: every edge points forward, e.g. $3 \to 6$ has 3 before 6, and $4 \to 7$ has 4 before 7. The figure below contrasts how Kahn's indegree peeling wave advances through this DAG (left) against how a cyclic dependency traps its vertices with positive residual indegree so the queue runs dry early (right).



Trace it yourself

Re-run the trace but break ties by *larger* vertex number, or use a stack (LIFO) instead of a queue. You will get a different valid order. Confirm by hand that whatever legal choices you make, the result still has every edge pointing forward. The freedom in tie-breaking is exactly the non-uniqueness we characterize in Section 4.4.

Theorem 4.2 (Correctness of Kahn)**.** *On input G, Algorithm 12 outputs a valid topological order if and only if G is a DAG; otherwise it outputs fewer than n vertices, and the omitted vertices are exactly those that lie on a directed cycle or are reachable from one.*

Proof. **Invariant.** At every moment, for each vertex w, indeg$[w]$ equals the number of in-edges (u, w) whose tail u has *not* yet been appended to L. This holds at start (no vertex output, so indeg is the true indegree) and is preserved: appending u decrements exactly the counts of its out-neighbors, which are precisely the edges leaving the just-output u.

Order is legal. A vertex enters Q only when indeg $= 0$, i.e. when *all* its predecessors are already in L. So when w is appended, every u with $(u, w) \in E$ precedes it: $\pi(u) < \pi(w)$ for all edges. Thus L, read as positions, is a topological order of the vertices it contains.

DAG $\Rightarrow$ all n output. If G is acyclic, suppose some vertices never enter Q; let S be that nonempty set. The induced subgraph $G[S]$ is a DAG, so by Lemma 4.1 it has a vertex s with no in-edge *from within* S. Every in-edge of s thus comes from $V \setminus S$, all of which were output, so by the invariant s's count reached 0 and s would have been enqueued – contradiction. Hence $S = \emptyset$.

Cycle ⇒ some vertex stuck. If G has a cycle C, no vertex of C can ever reach indegree 0: each has a predecessor on C that itself never gets output (induction around the cycle). So at least the vertices of C stay in S, and $|L| < n$. The stuck set is exactly the vertices from which the algorithm cannot "finish," which is the set of vertices lying on or downstream of the residual cyclic core. □

Cost summary

Kahn's algorithm. Computing indegrees scans all edges once: $O(m)$. The seeding loop is $O(n)$. In the main loop each vertex is enqueued/dequeued exactly once ($O(n)$ total) and each edge (u, w) is relaxed exactly once when u is dequeued ($O(m)$ total). Total time $O(n + m)$, extra space $O(n)$ for the indegree array and queue. With a min-heap instead of a queue you get the *lexicographically smallest* topological order in $O(n + m + n \log n)$.

Common trap

The most common bug is using $|L| = n$ versus "queue empty" to decide success. The loop always ends with an empty queue, so emptiness tells you nothing. You must compare the count of *output* vertices against n. A second trap: decrementing an indegree below zero because the input has *parallel edges* or a self-loop (v, v). A self-loop makes v depend on itself – an immediate one-vertex cycle – and correctly leaves v stuck; do not special-case it away. Parallel edges are fine as long as you count each edge once when seeding and once when relaxing.

4.3 Method 2: DFS Finish Times

Depth-first search gives a second route that needs no indegree bookkeeping and naturally reports the cycle as a back edge. The key observation: *a vertex finishes after all vertices it can reach.* So if we order vertices by *decreasing* finish time, every edge points from a vertex that finishes later to one that finishes earlier – forward in the order.

Intuition

Run DFS. When the recursion for u returns, every successor reachable from u has already returned (its subtree of the search was fully explored before we backed out of u). "Returned earlier" will become "later in the topological order." So: push each vertex onto a stack at the *moment it finishes*, then read the stack top-to-bottom. The last vertex to finish sits on top and goes first.

We use the standard DFS coloring: WHITE (undiscovered), GRAY (on the recursion stack, discovered but not finished), BLACK (finished). A *back edge* is an edge from a vertex to a gray ancestor; its existence is equivalent to a cycle.

Algorithm 13: DFSTopoSort(G)

Input: Digraph $G = (V, E)$.
Output: A topological order, or report "cycle."

```
1  foreach v ∈ V do
2      color[v] ← WHITE
3  S ← empty stack
4     cyclic ← false
5  foreach v ∈ V do
6      if color[v] = WHITE then
7          Visit(v)
8  if cyclic then
9      return "cycle"
10 else
11     return S read top-to-bottom

12 Function Visit(u):
13     color[u] ← GRAY
14     foreach w with (u, w) ∈ E do
15         if color[w] = GRAY then
16             cyclic ← true                    // back edge:  w is an active ancestor
17         else if color[w] = WHITE then
18             Visit(w)
19     color[u] ← BLACK
20     push u onto S            // u finishes; it precedes nothing still on the stack
```

Why it works

Why "decreasing finish time" is exactly a topological order. Consider any edge (u, v) at the moment DFS explores it from a gray u. There are three cases for v's color:

- WHITE: v is undiscovered, so DFS recurses into it; v finishes *inside* u's call, hence $f(v) < f(u)$.
- BLACK: v already finished before u explored the edge, so $f(v) < f(u)$.
- GRAY: v is an ancestor still on the stack – this is a back edge and means a cycle $v \rightsquigarrow u \to v$, so no topological order exists.

In the two acyclic cases $f(v) < f(u)$, i.e. u finishes *after* v. Ordering by decreasing f then puts u before v: the edge points forward.

Theorem 4.3 (DFS topological correctness)**.** *Algorithm 13 reports "cycle" iff G has a directed cycle; otherwise the stack, read top to bottom, is a valid topological order.*

Proof. First, *back edge ⇔ cycle.* If the algorithm flags a back edge (u, w) with w gray, then w is an ancestor of u in the DFS tree, so there is a tree path $w \rightsquigarrow u$, and together with (u, w) this is a directed cycle. Conversely, suppose G has a cycle; let x be the first vertex of the cycle that DFS discovers. The other cycle vertices are white when x is discovered, so by the *white-path theorem* (every white vertex reachable from x via a white path becomes a descendant of x) the cycle vertex y

with $(y, x) \in E$ becomes a descendant of x and is gray when its edge (y, x) to the still-gray ancestor x is explored. That edge is flagged.

If no back edge is flagged, every explored edge fell in the white/black cases, giving $f(v) < f(u)$ for every edge (u, v) by the case analysis in the box above. Pushing on finish puts later-finishing vertices nearer the top; reading top-to-bottom is decreasing f, so for every edge the tail (larger f) precedes the head (smaller f). That is a topological order. □

Worked trace

Run DFS on the same seven-task graph, visiting vertices and out-neighbors in increasing numeric order. Discovery/finish times (d/f) on a single global clock:

vertex	1	2	4	5	6	7	3
d	1	2	3	4	5	7	12
f	14	11	10	9	6	8	13

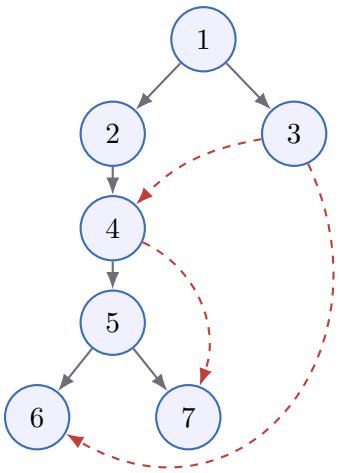
Walking it: start 1 (d=1); go to 2 (d=2); 2 → 4 (d=3); 4 → 5 (d=4); 5 → 6 (d=5, finishes f=6 – no out-edges); 5 → 7 (d=7, 7 has no out-edge, finishes f=8); 5 finishes (f=9); 4 → 7 is black, 4 finishes (f=10); 2 finishes (f=11); back at 1, explore 1 → 3 (d=12): 3 → 4 black, 3 → 6 black, 3 finishes (f=13); finally 1 finishes (f=14).

Push-on-finish stack (bottom → top): $6, 7, 5, 4, 2, 3, 1$. Read top-to-bottom:

$$1,\ 3,\ 2,\ 4,\ 5,\ 7,\ 6.$$

Every edge points forward (check 3 → 6: 3 sits at position 2, 6 at position 7 – forward; 5 → 7: positions 5 then 6 – forward). Note this differs from Kahn's $1, 2, 3, 4, 5, 6, 7$: both are valid, and the difference is exactly the ordering freedom.

The search built the DFS tree below (solid tree edges; dashed non-tree edges, all pointing to already-BLACK vertices, so none is a back edge and the graph is acyclic). Vertex 3 is discovered late, directly from the root 1, only after 2's entire subtree has finished.



Common trap

Two classic DFS mistakes. (1) *Appending on discovery* instead of on finish, then reading the list forward – that gives a *pre*order, which is *not* topological (a child can be discovered before a sibling that must precede it). You must record finish order and reverse it. (2) Treating a gray neighbor the same as a black one. A black neighbor is a harmless cross/forward edge; a gray neighbor is a back edge and the cycle witness. Conflating them either misses cycles or falsely reports them. For *undirected* DFS the rule differs (the edge back to your parent is not a cycle), but in a digraph every gray hit is a real back edge.

Cost summary

DFS topological sort. One DFS over the graph: each vertex colored once, each edge examined once, $O(n + m)$ time. Space $O(n)$ for colors and the output stack, plus $O(n)$ recursion depth (replace with an explicit stack to avoid blowing the call stack on a 10-million-vertex chain). Same asymptotics as Kahn; pick by taste and by what else you need (DFS also hands you the SCC machinery and cycle witness "for free").

Connection

The two methods are dual. Kahn peels *sources* (indegree-0, front of the order); DFS-reverse effectively peels *sinks* (a finished vertex has all successors done, so it is a current sink of the unfinished subgraph). Run DFS on the *reverse* graph and read finish order forward and you recover a source-first order. This same finish-time-reversal idea is the basis of Kosaraju's strongly-connected-components algorithm in Chapter 5: there the condensation of the SCCs is itself a DAG, and you topologically sort *that*.

4.4 Uniqueness of the Order

When is the topological order forced? Intuitively, you have a choice precisely when two vertices are *incomparable* – neither can reach the other – because then you may place either first. The clean characterization is in terms of a *Hamiltonian path*.

Definition 4.3 (Hamiltonian path of a DAG)**.** A directed path that visits every vertex exactly once.

Theorem 4.4 (Uniqueness)**.** *A DAG G has a unique topological order if and only if it has a Hamiltonian path, i.e. iff there is an edge between each pair of consecutive vertices in the (necessarily then unique) order. Equivalently, the order is unique iff at every step of Kahn's algorithm the queue holds exactly one vertex.*

Proof. ($\Leftarrow$) Suppose the order $v_1, \ldots, v_n$ has $(v_i, v_{i+1}) \in E$ for all i. Then for any valid order σ, the edge (v_i, v_{i+1}) forces v_i before v_{i+1} in σ, so σ must list them in the order $v_1, \ldots, v_n$ – identical. Hence unique.

($\Rightarrow$) Contrapositive: suppose no Hamiltonian path exists, i.e. in the order $v_1, \ldots, v_n$ produced by any run there is some i with $(v_i, v_{i+1}) \notin E$. We claim swapping v_i and v_{i+1} yields another valid order, proving non-uniqueness. The swap only reverses the relative order of v_i, v_{i+1}; it threatens validity only via an edge between them. There is no edge $v_i \to v_{i+1}$ by assumption, and no edge $v_{i+1} \to v_i$

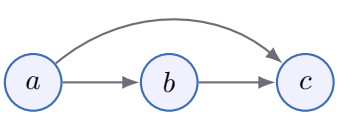
either (that would point backward in the original valid order, impossible). All other edges keep their relative orientation. So the swapped sequence is a second valid order.

The Kahn restatement: the queue having ≥ 2 vertices at some step means two sources of the residual graph are mutually incomparable at that moment, so either can be emitted next – two orders. If the queue is always a singleton, every choice is forced, and the unique consecutive emissions $u_1, u_2, \ldots$ each satisfy: when u_{i+1} became the lone source, it was made ready by removing u_i (else it was ready earlier and would have coexisted in the queue), so $(u_i, u_{i+1}) \in E$ – a Hamiltonian path. □

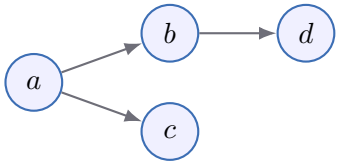
Intuition

The order is unique exactly when the DAG is a single "chain with shortcuts": a backbone $v_1 \to v_2 \to \cdots \to v_n$ (the Hamiltonian path) plus optionally extra forward edges. Any missing backbone link gives you a place to swap, hence a choice. So: *unique order* $\equiv$ *total order already*, and a DAG with a genuine partial order (some incomparable pair) has at least two topological orders. The number of topological orders equals the number of *linear extensions* of the partial order, a quantity that is famously #P-hard to count in general.

The two small DAGs below contrast a unique topological order forced by the Hamiltonian chain $a \to b \to c$ (left) against a branching DAG where incomparable vertices b and c allow three distinct linear extensions (right).



unique: a, b, c



not unique: b, c swap
a, b, c, d *or* a, c, b, d *or* a, b, d, c

4.5 Dynamic Programming on a DAG

A topological order is the natural evaluation order for any quantity that flows along edges: process vertices left to right, and by the time you reach v every predecessor's value is final. Acyclicity guarantees no circular dependency, so a single linear sweep computes recurrences that would loop forever on a cyclic graph. This is dynamic programming where the DAG *is* the DP-state graph.

Focus

The template. Fix a topological order $v_1, \ldots, v_n$. To compute dp[v] defined by a recurrence over v's in-neighbors (or out-neighbors), sweep in topological order (resp. reverse). Each edge is relaxed once, so the whole DP costs $O(n + m)$ – linear, regardless of how many paths the DAG hides.

4.5.1 Counting paths

How many distinct directed paths run from a source s to a target t? On a general graph this can be exponential and tangled by cycles; on a DAG it is one sweep.

Let paths$[v]$ be the number of distinct paths from s to v. Then

$$\text{paths}[s] = 1, \qquad \text{paths}[v] = \sum_{(u,v)\in E} \text{paths}[u].$$

Processing vertices in topological order guarantees every u with $(u, v) \in E$ comes before v, so its count is ready. (A path to v is a path to some predecessor u extended by the edge $u \to v$, and distinct predecessors or distinct paths to them give distinct s–v paths; this partition is exactly the sum.)

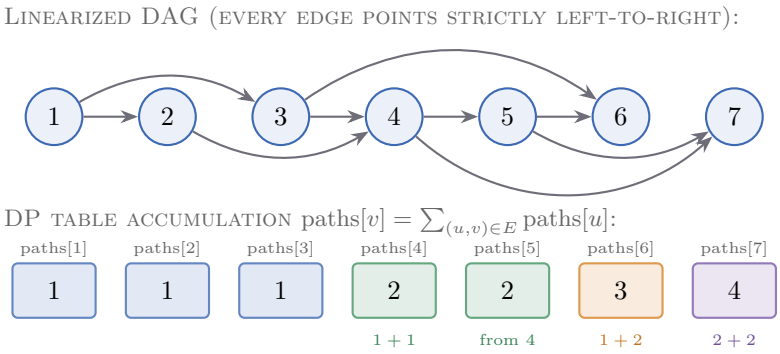
Algorithm 14: CountPaths(G, s, t)

Input: DAG G, source s, target t, topological order $v_1, \ldots, v_n$.
Output: Number of $s \to t$ paths.

```
foreach v ∈ V do
    paths[v] ← 0
paths[s] ← 1
for i ← 1 to n (in topological order) do
    u ← v_i
    foreach w with (u, w) ∈ E do
        paths[w] ← paths[w] + paths[u]
return paths[t]
```

On the seven-task graph with $s = 1$, sweeping order $1, 2, 3, 4, 5, 6, 7$: paths[1] = 1; $\to$ 2:1, 3:1; from 2, 4:1; from 3, 4:2, 6:1; from 4, 5:2, 7:2; from 5, 6:1+2=3, 7:2+2=4. So there are 3 paths from 1 to 6 and 4 from 1 to 7. (Trace 1$\to$6 by hand: 1,3,6; 1,2,4,5,6; 1,3,4,5,6 – three, as computed.)

The diagram below places the seven-task DAG into its linearized left-to-right order alongside the DP table paths$[v]$ accumulated across the sweep.



4.5.2 Longest path

Longest path is NP-hard on general graphs (it encodes Hamiltonicity), yet on a DAG it is linear. The recurrence for the longest path *ending* at v (number of edges, or summed weight w):

$$\text{len}[v] = \max\Big(0,\ \max_{(u,v)\in E}\big(\text{len}[u] + w(u,v)\big)\Big).$$

Why it works

Acyclicity is what makes this work. On a graph with a positive-weight cycle, "longest path" is unbounded (loop forever) and the recurrence has no fixpoint. On a DAG the dependency $\text{len}[v] \leftarrow \text{len}[u]$ only ever points backward in topological order, so the system is triangular and a single sweep solves it. The exact same triangularity is why DAG *shortest* paths need no Dijkstra/Bellman-Ford – one relaxation pass per vertex suffices, and negative weights are no obstacle because there are no cycles to exploit.

Algorithm 15: DAGRELAX(G, w)

Input: Weighted DAG G with weights w, topological order $v_1, \ldots, v_n$.
Output: dist$[\cdot]$ = longest-path (or shortest, swapping max / min) distances from sources.

```
1 foreach v ∈ V do
2     dist[v] ← 0   // longest; use +∞ everywhere but the source for single-source
        shortest
3 for i ← 1 to n (topological order) do
4     u ← v_i
5     foreach x with (u, x) ∈ E do
6         if dist[u] + w(u, x) > dist[x]                    // < for shortest paths
7          then
8             dist[x] ← dist[u] + w(u, x)
9             pred[x] ← u
```

Theorem 4.5 (DAG single-source shortest/longest paths)**.** *Given a topological order, Algorithm 15 computes correct shortest- (or longest-) path distances from the chosen source(s) in $O(n+m)$, for* arbitrary *edge weights (including negatives), with no cycle-handling machinery.*

Proof. We prove the shortest-path version; longest is symmetric. Claim: when the sweep reaches $v = v_i$, dist$[v]$ already equals the true shortest distance $\delta(s, v)$. Induct on i. A shortest s–v path's last edge is (u, v) for some u with $\pi(u) < \pi(v)$ (every edge points forward), so $u = v_j$ with $j < i$. By induction dist$[u] = \delta(s, u)$ was finalized when u was swept, and at that moment we relaxed (u, v), setting dist$[v] \le \delta(s, u) + w(u, v) = \delta(s, v)$. Conversely dist$[v]$ is only ever set to the length of some genuine s–v path, so dist$[v] \ge \delta(s, v)$. Equality holds; and no later relaxation lowers it because all of v's in-edges came from already-swept vertices. Each edge is relaxed once: $O(n+m)$. □

Trace it yourself

Take the seven-task graph as *unweighted* (each edge length 1) and compute the longest path ending at each vertex from source 1, sweeping $1, 2, 3, 4, 5, 6, 7$: len[1]=0, [2]=1, [3]=1, [4]= max(1+1, 1+1)=2, [5]=3, [6]= max(1+1, 3+1)=4, [7]= max(2+1, 3+1)=4. The longest chain to 7 has length 4: $1 \to 2 \to 4 \to 5 \to 7$. This length is the *critical path* – exactly the minimum makespan if every task takes unit time and independent tasks run in parallel.

4.5.3 Why one sweep beats Dijkstra here

Intuition

Dijkstra needs nonnegative weights because it finalizes a vertex's distance as soon as it is the cheapest unsettled vertex, relying on the fact that no later detour can lower that cost – an assumption that fails when an edge has negative weight. Bellman-Ford pays $O(nm)$ to tolerate negatives because it does not know a safe processing order. A DAG *hands you* a safe order: topological. Once you process predecessors before successors, a single relaxation per edge is provably final. The structure of the graph replaces the priority queue.

Cost summary

DP on DAGs. Topological sort $O(n + m)$, then one sweep $O(n + m)$, total $O(n + m)$ and $O(n)$ memo space for: counting paths, longest path / critical path, shortest path with any weights, number of distinct longest/shortest paths, and most "flow-along-edges" recurrences. Compare: general-graph longest path is NP-hard; general shortest path with negatives is Bellman-Ford's $O(nm)$. Acyclicity turns longest path from NP-hard into linear time.

4.6 Applications: Scheduling and Build Systems

Application

Build systems. `make`, Bazel, Ninja, and friends model files as vertices and "a is a prerequisite of b" as an edge $a \to b$. A topological sort yields a legal compile order; the *critical path* (longest weighted path, where weights are per-target build times) is the minimum wall-clock time achievable with unlimited parallelism, and tells you which target to optimize first. A reported cycle is the dreaded "circular dependency" error – and the DFS method even hands the user the offending back edge.

Application

Project scheduling (PERT/CPM). Tasks with durations and precedence constraints form a weighted DAG. DAGRelax in *longest*-path mode computes the earliest start time of every task (longest path from the start node); a reverse sweep computes latest start times; tasks whose earliest and latest starts coincide lie on the critical path and have zero *slack* – delaying

any of them delays the whole project. This is the original setting Kahn was solving in 1962.

Application

Spreadsheets, package managers, dataflow. A spreadsheet recalculates cells in topological order of the formula-dependency DAG; a circular reference is a detected cycle. Package managers (`apt`, `npm`, Cargo) install dependencies bottom-up in topological order. Compilers order instructions and resolve symbol/type dependencies the same way. Whenever a problem requires doing each step only after its prerequisites are complete, it defines a DAG and calls for a topological sort.

Connection

Topological order is used throughout the rest of this book. Strongly connected components (Chapter 5) turn *any* digraph into a DAG of its SCCs and then topologically sort that condensation – so even cyclic graphs get a meaningful linear order at the component level. The DAG shortest-path sweep here is the base case that Dijkstra (Chapter 7) and the all-pairs algorithms (Chapter 9) generalize to cyclic graphs.
In Chapter 5, we explore this condensation directly: we partition general directed graphs into their strongly connected components, see how Kosaraju and Tarjan extract them in linear time, and use the resulting DAG of components to solve 2-SAT and dependency problems.

4.7 Exercises

Exercise 4.1 (Implement and compare)**.** Implement both Algorithm 12 and Algorithm 13. On a randomly generated DAG with $n = 10^6$ vertices and $m = 3 \cdot 10^6$ edges, verify they each run in well under a second and that both outputs are valid topological orders (every edge points forward). Replace the recursive DFS with an explicit stack and confirm it no longer overflows on a path of 10^6 vertices.

Exercise 4.2 (Lexicographically smallest order)**.** Modify Kahn's algorithm to output the lexicographically smallest topological order by replacing the FIFO queue with a min-heap. Prove it is correct (a greedy exchange argument) and give its running time. Then argue why the DFS method cannot easily be coerced into the lexicographically smallest order.

Exercise 4.3 (Cycle witness)**.** Extend the DFS method to *report the actual cycle* (the list of vertices) when it finds a back edge (u, w), not just the boolean. Hint: keep the current gray recursion stack; the cycle is its suffix from w to u plus the edge. Do the same for Kahn: after termination, reconstruct a cycle among the stuck vertices in $O(n + m)$.

Exercise 4.4 (Counting linear extensions)**.** Write a recursive routine that counts *all* topological orders of a small DAG. On the right-hand example of Section 4.4 confirm the count is 3. Explain why your routine is exponential in the worst case and look up why counting linear extensions is #P-complete.

Exercise 4.5 (Unique-order test in linear time)**.** Give an $O(n + m)$ algorithm that decides whether a DAG's topological order is unique, using Theorem 4.4. (Run Kahn and check the queue never holds two vertices, or verify the produced order has an edge between every consecutive pair.) Prove your test correct.

Exercise 4.6 (Longest path, all weights)**.** Prove Theorem 4.5 handles arbitrary (including negative) weights correctly for both shortest and longest paths, and explain precisely where the proof would break if the graph had a single directed cycle. Construct a 3-vertex example showing longest path becomes ill-defined when one edge is reversed to form a cycle.

Exercise 4.7 (Critical path and slack)**.** On a weighted DAG with a single source and single sink, compute for every vertex its earliest start (forward longest-path sweep) and latest start (reverse sweep), and define slack as their difference. Prove a vertex lies on *some* critical (longest source–sink) path iff its slack is 0. Apply it to the seven-task graph with unit weights and list the critical-path vertices.

Exercise 4.8 (Break it: where DFS preorder fails)**.** Exhibit a DAG on which reading the DFS *pre*order (discovery order) forward is *not* a topological order, and pinpoint the edge that points backward. Then show that reversing the *postorder* (finish order) fixes it. Use a 4-vertex graph.

Exercise 4.9 (Minimum path cover, glimpse of matching)**.** A *minimum path cover* of a DAG is the fewest vertex-disjoint directed paths covering all vertices. Show it equals n minus the size of a maximum matching in a bipartite "split" graph (each vertex v becomes v_{out} on the left and v_{in} on the right, with edge $v_{\text{out}}u_{\text{in}}$ for each $(v, u) \in E$). This previews the matching–flow connection of Chapters 15–18; compute a minimum path cover of the seven-task graph by hand.

Exercise 4.10 (Dynamic topological order)**.** Suppose edges are inserted into a DAG one at a time and you must maintain a topological order after each insertion (rejecting any edge that would create a cycle). Describe an algorithm faster than recomputing from scratch each time, and bound its total work over a sequence of m insertions. (Look up the Pearce–Kelly online topological ordering algorithm to check your bound.)

Chapter 5

Connectivity and Strongly Connected Components

In Chapter 4, we saw that a directed graph without cycles admits a topological order, aligning its vertices along a consistent line of dependency. But what happens when cycles are present? On a directed graph, reachability is not symmetric: an edge allows one-way travel, so two vertices belong together only when each can reach the other along directed paths. These maximal mutually reachable sets are *strongly connected components* (SCCs), the directed analogue of connected components. This chapter presents linear-time algorithms for finding SCCs, contracts them to form an acyclic condensation graph, and extends reachability analysis to 2-SAT and Eulerian circuits.

5.1 Undirected connectivity

Let $G = (V, E)$ be an undirected graph. Define a relation on V by $u \sim v$ iff there is a path from u to v. Reflexivity (the trivial length-0 path), symmetry (reverse the path – legal because edges are undirected), and transitivity (concatenate paths) make $\sim$ an equivalence relation. Its classes are the *connected components*.

Definition 5.1 (Connected component)**.** A *connected component* of an undirected graph G is a maximal set $C \subseteq V$ such that every pair of vertices in C is joined by a path. Equivalently, it is an equivalence class of the reachability relation $\sim$.

Intuition

Drop the graph on a table and pull on one vertex. Everything that comes up with it – everything physically attached through a chain of edges – is its connected component. The pieces left lying on the table are the other components. A single graph search starting at any vertex lights up exactly one component; repeat from an unvisited vertex to get the next.

The algorithm is a loop over vertices that launches a DFS (or BFS) from each not-yet-seen vertex, tagging every reached vertex with a component id.

Algorithm 16: Connected components of an undirected graph

Input: Undirected graph $G = (V, E)$ as adjacency lists.
Output: Array comp[·] assigning each vertex a component id.

```
1  c ← 0
2  foreach v ∈ V do comp[v] ← ⊥
3  foreach v ∈ V do
4      if comp[v] = ⊥ then
5          c ← c + 1
6          Explore(v, c)
7  return comp
8  procedure Explore(s, c)
9      push s onto stack S; comp[s] ← c
10     while S nonempty do
11         u ← pop S
12         foreach w ∈ adj(u) do
13             if comp[w] = ⊥ then
14                 comp[w] ← c; push w onto S
```

Why it works

Every vertex is assigned a component id exactly once (the comp$[w] = \perp$ guard), and the outer loop visits every vertex, so the total work is one traversal of each adjacency list: $O(|V| + |E|)$ time and $O(|V|)$ extra space. Correctness: Explore(v, c) assigns id c to exactly the set of vertices reachable from v, which (by symmetry) is precisely v's equivalence class, because the search follows edges in both directions and stops only when no new vertex is reachable.

Connection

A union–find (disjoint set union) data structure gives the same component labels and, unlike a single static traversal, supports adding edges online: union the endpoints. That dynamic view is the entry point to the incremental connectivity and minimum-spanning-tree material later in the book. For now, a static DFS is simpler and strictly linear.

The rest of this chapter deals with directed graphs, where connectivity is more subtle.

5.2 Strong connectivity: the directed story

In a directed graph $G = (V, E)$, "reach" is one-directional. Reachability $u \rightsquigarrow v$ (a directed path from u to v) is reflexive and transitive but *not* symmetric. To recover an equivalence relation we symmetrize it deliberately.

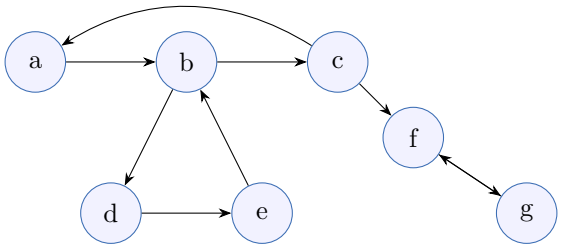
Definition 5.2 (Strong connectivity)**.** Vertices u, v are *strongly connected*, written $u \equiv v$, if $u \rightsquigarrow v$ and $v \rightsquigarrow u$: each can reach the other. The relation $\equiv$ is an equivalence relation; its classes are the *strongly connected components* (SCCs) of G.

Reflexivity and symmetry are immediate. Transitivity: if $u \rightsquigarrow v \rightsquigarrow w$ and $w \rightsquigarrow v \rightsquigarrow u$, concatenate to get $u \rightsquigarrow w$ and $w \rightsquigarrow u$. So $\equiv$ partitions V, and within one class every vertex can reach every other and return.

Intuition

An SCC is a maximal set of vertices that all sit on a common directed cycle structure: from any one you can take a round trip through any other. If the digraph were a city of one-way streets, an SCC is a neighborhood where you can drive from any corner to any other corner and back without ever breaking the law. Between neighborhoods, travel may be one-way only.

Here is a concrete digraph we will return to throughout the chapter. Read the arrows carefully.



Trace it by eye. The triangle $a \to b \to c \to a$ is a cycle, so $\{a, b, c\}$ are mutually reachable. But $b \to d \to e \to b$ is also a cycle, and it shares vertex b with the triangle, so a, b, c, d, e all reach each other: they form one SCC $\{a, b, c, d, e\}$. The pair $f \leftrightarrow g$ is a 2-cycle, hence an SCC $\{f, g\}$. Note $c \to f$ lets the big component reach $\{f, g\}$, but nothing returns from f or g to the rest, so the two components stay separate. Final answer: SCCs $\{a, b, c, d, e\}$ and $\{f, g\}$.

Common trap

A common mistake is to call two vertices strongly connected just because they lie on *a* directed path together. That is one-way reachability, not strong connectivity. You need a path *both ways*. In the picture, a reaches g but g cannot reach a, so they are in different SCCs even though a directed path $a \rightsquigarrow g$ exists.

The naive algorithm is clear: for each pair (u, v) test mutual reachability with two graph searches. That is $O(|V| \cdot (|V| + |E|))$ at best and wasteful. A slightly smarter version computes, for each vertex v, the set of vertices reachable from v (one DFS) and the set that reach v (one DFS in the reversed graph); the SCC of v is the intersection. Still $O(|V|(|V| + |E|))$. We can do all of it in a single linear pass. Two algorithms achieve $O(|V| + |E|)$, and we now develop both.

History

Three linear-time SCC algorithms are standard. Tarjan (1972), in the same paper that systematized depth-first search, gave a single-pass algorithm using a value he called the *lowlink*. Independently, the two-pass method usually attributed to Kosaraju (unpublished, 1978) and to Sharir (1981) trades one extra DFS for conceptual simplicity. Later, Cheriyan–Mehlhorn (1996) and Gabow (1999) gave a path-based single-pass algorithm with two stacks that many find the cleanest of all. We treat Kosaraju and Tarjan; Gabow appears in the exercises.

5.3 Kosaraju's two-pass algorithm

Kosaraju's idea is to run DFS twice: once on G to learn the order in which vertices *finish*, and once on the reverse graph $G^\top$ (every edge flipped) in decreasing finish order. The second pass, started from the right roots, peels off SCCs one at a time.

Intuition

Why does reversing help? Strong connectivity is symmetric in edge direction: $u \equiv v$ in G exactly when $u \equiv v$ in $G^\top$, because reversing every edge reverses every path but leaves "both ways" intact. So G and $G^\top$ have *identical* SCCs. What reversal flips is the direction of travel *between* components. If in the original graph component X can reach component Y but not vice versa, then in the reverse graph Y reaches X. The first DFS's finish times rank the components topologically; processing them in that ranking on the reversed graph traps each search inside a single SCC.

5.3.1 Finish times and the key lemma

Recall from the DFS chapter that each vertex v gets a discovery time $d[v]$ and a finish time $f[v]$, with $d[v] < f[v]$ and the parenthesis property: intervals $[d[v], f[v]]$ are properly nested or disjoint. Extend f to a set of vertices by $f(C) = \max_{v \in C} f[v]$.

Lemma 5.1 (Component finish order)**.** *Let C and C' be distinct SCCs of G. If there is an edge $(u, v) \in E$ with $u \in C$ and $v \in C'$, then $f(C) > f(C')$.*

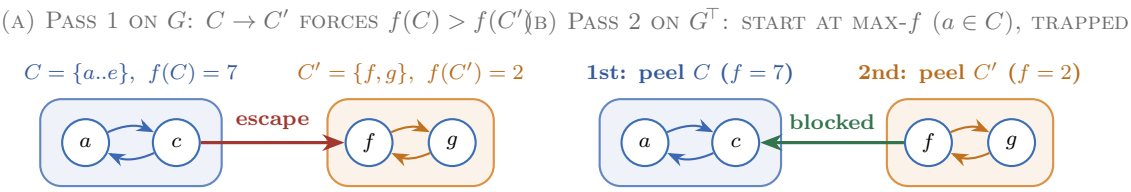
Proof. There are two cases depending on which component DFS enters first.

Case 1: $d(C) < d(C')$, where $d(C) = \min_{w \in C} d[w]$ is the first discovery time in C. Let x be the first vertex discovered in C. At time $d[x]$, every vertex of $C \cup C'$ is white. Within C, x reaches all of C; and through the edge (u, v) it reaches C' (since $u \in C$ is reachable from x inside C, and $v \in C'$ reaches all of C'). By the white-path theorem, every vertex of $C \cup C'$ becomes a descendant of x in the DFS forest, so they all finish before x does: $f[x] = f(C) > f(C')$.

Case 2: $d(C') < d(C)$. Let y be the first vertex discovered in C'. Could y reach C? If it did, then via the edge (u, v) (from C to C') we would have $C' \rightsquigarrow C \rightsquigarrow C'$, making C and C' mutually reachable and hence the same SCC – contradiction. So C' cannot reach C. Thus the entire subtree rooted at y stays inside C' (it cannot escape to C), and finishes before any vertex of C is discovered. Therefore $f(C') < d(C) \leq f(C)$, giving $f(C) > f(C')$ again. □

Corollary 5.1. *If $(u, v) \in E$ joins two different SCCs $C \neq C'$ (with $u \in C$, $v \in C'$), then in the* reverse *graph $G^\top$ the edge points from C' to C, and still $f(C) > f(C')$ where f is computed on G. So in $G^\top$, edges between distinct components always run from a* lower-f *component to a* higher-f *component.*

The second pass relies on this corollary. Order the components by decreasing f. Start a DFS in $G^\top$ from a vertex of the highest-f component $C^\star$. Every edge leaving $C^\star$ in $G^\top$ goes to a component with even higher f – but $C^\star$ already has the maximum, so there are none. The search is trapped inside $C^\star$ and recovers exactly that component. Remove it and repeat. The two panels below show how reversing the inter-component edge turns an outgoing bridge in G into a one-way barrier in $G^\top$.



5.3.2 The algorithm

Algorithm 17: Kosaraju–Sharir SCC

Input: Digraph $G = (V, E)$.
Output: Array scc[·] labeling each vertex with its component.

```
// Pass 1: DFS on G, push vertices on finish
empty stack order; mark all vertices white
foreach v ∈ V do
    if v white then DfsOrder(v)
// Pass 2: DFS on the reverse graph in decreasing finish order
build reverse graph G^⊤; mark all vertices white; c ← 0
while order nonempty do
    v ← pop order
    if v white then
        c ← c + 1
        Assign(v, c)                    // DFS in G^⊤, label all reached with c
return scc
procedure DfsOrder(u)
    mark u gray
    foreach w ∈ adj_G(u) do if w white then DfsOrder(w)
    push u onto order                   // u is now finished
procedure Assign(u, c)
    scc[u] ← c; mark u gray
    foreach w ∈ adj_{G^⊤}(u) do if w white then Assign(w, c)
```

Theorem 5.1 (Correctness of Kosaraju)**.** *Algorithm 17 labels two vertices with the same c if and only if they are in the same SCC.*

Proof. We show that each call Assign(v, c) triggered from the main loop discovers exactly the SCC containing v. Process the components in the order the main loop launches new searches; we argue by induction that before each launch, the still-white vertices are precisely a union of whole SCCs that have not yet been output.

The main loop pops vertices in decreasing f (the stack was filled in increasing finish order). Suppose we are about to launch from v, the highest-f white vertex; let $C = \mathrm{scc}(v)$. By choice of v, $f(C) = f[v]$ is the maximum finish time among white vertices, so among the not-yet-output components, C has the largest f. Now run Assign (a DFS in $G^\top$). It certainly reaches all of C, since C is strongly connected in $G^\top$ as well. Can it escape C? An escaping edge in $G^\top$ goes from C to some other component C'; by Corollary 5.1 that requires $f(C') > f(C)$. But C already has the largest f among

white components, and any component with larger f was output earlier and is no longer white. So every escaping edge leads to a non-white, already-labeled vertex and is ignored. The search is confined to C, labels exactly C with c, and the inductive hypothesis is restored. Hence the labels are exactly the SCCs. □

Cost summary

Two DFS traversals plus building the reverse graph, each $O(|V|+|E|)$. Total time $O(|V|+|E|)$, extra space $O(|V|+|E|)$ for $G^\top$ and the order stack. The constant factor is roughly twice a single DFS, which is why Tarjan's one-pass method is preferred when memory or constants matter.

Trace it yourself

Run Kosaraju on the running digraph. Pass 1 from a (adjacency in alphabetical order) finishes vertices in some order ending with a last; with alphabetical scanning the finish stack (bottom first, top last) is g, f, c, e, d, b, a, so popping gives a first. Pop a: ASSIGN in $G^\top$ from a reaches $\{a, b, c, d, e\}$ (all the reverse edges of the two cycles) and labels them c=1. The original edge $c \to f$ becomes $f \to c$ in $G^\top$, so from a there is no way out to f: the search is trapped inside the first component, exactly as Corollary 5.1 promises. Next white top is f or g; ASSIGN from it labels $\{f, g\}$ as component 2. Two components, matching our hand analysis. Redo it choosing a different pass-1 start vertex and confirm you still get the same two SCCs.

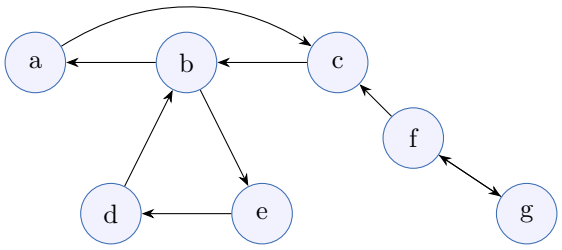
5.3.3 Tracing Kosaraju by hand

It pays to watch the finish times and the two passes line up on the running digraph. Pass 1 is an ordinary DFS on G starting at a, scanning adjacency lists alphabetically. The discovery order is a, b, c along the triangle; from c the alphabetical scan reaches f, g (the side branch $c \to f$) before the search returns to b and descends to d, e. So f and g finish early, deep under c, and the two cycle vertices d, e finish later. The finish times come out as below (smaller finishes first), and we push each vertex onto *order* as it finishes.

Vertex	finish time $f[\cdot]$	pushed onto *order* at
g	1	bottom
f	2	
c	3	
e	4	
d	5	
b	6	
a	7	top

So *order* from top to bottom is a, b, d, e, c, f, g and the main loop pops a first. The exact within-component finish order can vary with tie-breaking, but the *component* maxima are what Lemma 5.1 pins down: $f(\{a, b, c, d, e\}) = 7$ and $f(\{f, g\}) = 2$, so the big component is processed first. Pass 2

runs on the reverse graph $G^\top$, drawn below: every arrow of the running digraph is flipped, so in particular $c \to f$ has become $f \to c$. Starting ASSIGN from a can no longer escape to $\{f, g\}$.



The two cycles are intact (a cycle reversed is still a cycle), so $\{a, b, c, d, e\}$ and $\{f, g\}$ remain the SCCs; only the inter-component link reversed, and it now points $f \to c$ – into the component we visit first, not out of it. Pass 2 then proceeds:

Pop	white?	ASSIGN reaches in $G^\top$	label
a	yes	$\{a, b, c, d, e\}$ (reverse cycle edges; $f \to c$ blocks escape)	1
b	no	— already labeled	—
d, e	no	—	—
c	no	—	—
f	yes	$\{f, g\}$ (reverse 2-cycle; $f \to c$ in G became $c \to f$, dead in $G^\top$ from here)	2
g	no	—	—

Two pops did real work; the rest were absorbed by an earlier ASSIGN. The labels $1, 2$ come out in topological order of the condensation, source first, so the same computation also gives a topological sort of G^{SCC} at no extra cost.

Example 5.1 (A three-component chain)**.** Consider the digraph on $\{p, q, r, s\}$ with edges $p \to q$, $q \to p$, $q \to r$, $r \to s$, $s \to r$. The 2-cycles $\{p, q\}$ and $\{r, s\}$ are the only cycles, and the lone edge $q \to r$ links them one way. Kosaraju's pass 1 (start at p) finishes s, r, q, p so *order* pops p first; ASSIGN in $G^\top$ from p reaches $\{p, q\}$ (the edge $q \to r$ reversed is $r \to q$, which never fires from inside $\{p, q\}$), labels it 1, then r labels $\{r, s\}$ as 2. The condensation is the two-vertex DAG $\{p, q\} \to \{r, s\}$. Run Tarjan on the same graph and confirm it pops $\{r, s\}$ *first* (reverse topological order) and $\{p, q\}$ second – the mirror image of Kosaraju's emission order.

5.4 Tarjan's one-pass algorithm

Tarjan computes all SCCs in a single DFS by maintaining, for each vertex, a *lowlink*: the smallest discovery index reachable from v using tree edges followed by at most one "back-or-cross" edge into a vertex still on the DFS stack. When a vertex's lowlink equals its own index, it is the *root* of an SCC, and that component is exactly the vertices sitting above it on an auxiliary stack.

Intuition

Think of the DFS recursion as exploring a cave with a rope (the stack of vertices currently on the active path, plus everything explored under them that has not yet been packaged into a component). The *index* of a vertex is the timestamp of first visit. The *lowlink* answers: "what is the oldest vertex I can loop back to, staying inside vertices that are still live on my stack?" If a vertex can loop no further back than itself, the rope above it forms a closed pocket – a strongly connected component – and we cut it off.

5.4.1 Definitions and the algorithm

Maintain a global counter giving each vertex a unique index$[v]$ on first visit, set low$[v]$ = index$[v]$ initially, and keep an explicit stack S of vertices that have been discovered but not yet assigned to a finished component, with a boolean onStack$[v]$. When we explore edge (u, w):

- if w is unvisited (a tree edge), recurse, then relax low$[u] \leftarrow \min(\text{low}[u], \text{low}[w])$;
- if w is on the stack (a back or cross edge into a live vertex), relax low$[u] \leftarrow \min(\text{low}[u], \text{index}[w])$;
- if w is visited but not on the stack, it already belongs to a completed SCC – ignore it.

After exploring all of u's edges, if low$[u]$ = index$[u]$ then u is a component root: pop S down to and including u; those popped vertices form one SCC.

Algorithm 18: Tarjan SCC

Input: Digraph $G = (V, E)$.
Output: Array scc[·].

```
1  idx ← 0; c ← 0; empty stack S
2  foreach v ∈ V do index[v] ← ⊥; onStack[v] ← false
3  foreach v ∈ V do
4      if index[v] = ⊥ then StrongConnect(v)
5  return scc
6  procedure StrongConnect(u)
7      index[u] ← idx; low[u] ← idx; idx ← idx + 1
8      push u onto S; onStack[u] ← true
9      foreach w ∈ adj(u) do
10         if index[w] = ⊥ then
11             StrongConnect(w)
12             low[u] ← min(low[u], low[w])                    // tree edge
13         else if onStack[w] then
14             low[u] ← min(low[u], index[w])                  // live back/cross edge
15         else ignore                                         // w in a finished SCC
16
17     if low[u] = index[u] then
18         c ← c + 1
19         repeat
20             w ← pop S; onStack[w] ← false; scc[w] ← c
21         until w = u
```

Common trap

The single most common bug is using low[w] instead of index[w] for the back/cross-edge case (or vice versa). The tree-edge update takes min with low[w] because the child has finished propagating its reach; the non-tree update takes min with index[w], the timestamp of the target, *not* its lowlink. Mixing them can merge components that should stay split, because a cross edge into a vertex w does not give you w's reach – only the fact that you can touch w. The `onStack` test is what keeps cross edges into finished components from corrupting the lowlink: those targets are off the stack and ignored.

5.4.2 Why lowlink detects component roots

Fix the DFS forest. For an SCC C, call its *root* the vertex $r \in C$ with the smallest discovery index. Equivalently r is the first vertex of C that DFS discovers; by the white-path theorem all of C lies in the subtree rooted at r.

Lemma 5.2 (Root characterization)**.** *When* StrongConnect(u) *finishes exploring all outgoing edges of* u, low[u] = index[u] *if and only if* u *is the root of its SCC.*

Proof. First, low[u] equals the minimum index over all vertices reachable from u along a path that uses tree edges down from u and then at most one edge into a vertex that is still on the stack

at the moment that edge is examined. A standard induction on finishing order shows $\text{low}[u] = \min\{\text{index}[v] : v \text{ reachable from } u \text{ by tree edges then } \leq 1 \text{ live non-tree edge}\}$.

($\Leftarrow$) Suppose u is the root of C, so $\text{index}[u]$ is minimal in C. We claim no vertex with index $< \text{index}[u]$ is reachable from u by the lowlink rule, hence $\text{low}[u] = \text{index}[u]$. Any vertex v with $\text{index}[v] < \text{index}[u]$ lies outside C (else u would not be the root). If u could reach such a v through the lowlink mechanism, the touched live vertex w satisfies $u \rightsquigarrow w$ and, being on the stack below or around u, has $w \rightsquigarrow u$ (live stack vertices ancestral-or-cross-reachable establish a path back to u); that would put w in C with smaller index, contradicting minimality. So $\text{low}[u] = \text{index}[u]$.

($\Rightarrow$) Suppose u is *not* the root; let $r \neq u$ be the root of C, so $\text{index}[r] < \text{index}[u]$ and r is a proper ancestor of u in the DFS tree. Since C is strongly connected, there is a path $u \rightsquigarrow r$. Follow it: it descends through the subtree and eventually takes a non-tree edge into an ancestor (or a cross edge into a live vertex) whose index is $< \text{index}[u]$; because r is still on the stack while u's subtree is active, that edge is a live edge and pulls $\text{low}[u]$ down to at most $\text{index}[r] < \text{index}[u]$. Hence $\text{low}[u] < \text{index}[u]$. □

Theorem 5.2 (Correctness of Tarjan)**.** *When* StrongConnect(u) *pops the stack at a root u (i.e. when $\text{low}[u] = \text{index}[u]$), the set of popped vertices is exactly the SCC of u.*

Proof. By Lemma 5.2 the pop happens exactly at roots. Consider the moment u finishes and is found to be a root. Every vertex of $C = \text{scc}(u)$ was discovered after u (since u is the root) and lies in u's subtree, so each was pushed onto S after u and sits above u. None of them has been popped yet: a vertex is popped only when its own root finishes, and the only root in C is u, which is finishing now. Hence all of C is currently above u on S. Conversely, any vertex x strictly above u on S that is *not* in C would belong to some other SCC C' whose root r' lies in u's subtree and has not finished – impossible, because r' would have to finish (and pop C') before u finishes, since r' is a descendant of u. So the vertices above and including u on S are precisely C, and the **repeat** loop pops exactly them. □

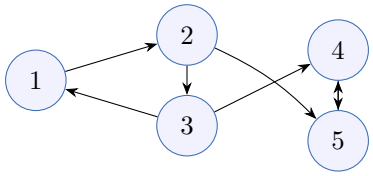
Cost summary

One DFS. Each vertex is pushed and popped once; each edge is examined once; the lowlink updates are $O(1)$. Total $O(|V| + |E|)$ time, $O(|V|)$ extra space for indices, lowlinks, the stack, and the onStack flags. No reverse graph is needed, so the constant factor is about half of Kosaraju's.

5.4.3 Tracing Tarjan by hand

Take a fresh small digraph designed to exercise both cycle types and a cross edge. Vertices $1, 2, 3, 4, 5$ with edges

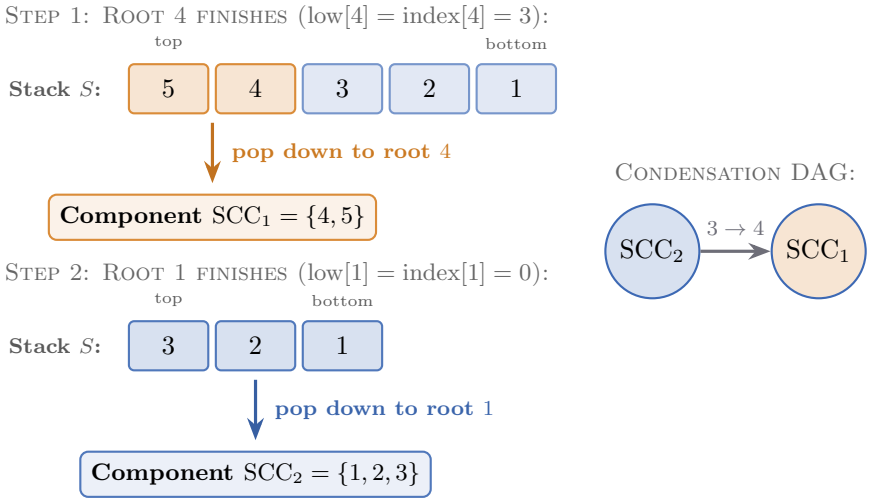
$$1 \to 2, \quad 2 \to 3, \quad 3 \to 1, \quad 3 \to 4, \quad 4 \to 5, \quad 5 \to 4, \quad 2 \to 5.$$



We expect SCCs $\{1,2,3\}$ (the triangle) and $\{4,5\}$ (the 2-cycle); the edge $2 \to 5$ is a cross edge into the $\{4,5\}$ component, and $3 \to 4$ likewise reaches it one-way. Start the DFS at vertex 1, scanning each adjacency list in increasing order. The table shows the state right after each significant event; S is the Tarjan stack listed bottom-to-top.

Event	index	low	S	note
visit 1	index[1]=0	low[1]=0	1	push 1
visit 2	index[2]=1	low[2]=1	1, 2	tree edge $1 \to 2$
visit 3	index[3]=2	low[3]=2	1, 2, 3	tree edge $2 \to 3$
edge $3 \to 1$		low[3]$=\min(2,0)=0$	1, 2, 3	1 on stack: use index[1]=0
visit 4	index[4]=3	low[4]=3	1, 2, 3, 4	tree edge $3 \to 4$
visit 5	index[5]=4	low[5]=4	1, 2, 3, 4, 5	tree edge $4 \to 5$
edge $5 \to 4$		low[5]$=\min(4,3)=3$	1, 2, 3, 4, 5	4 on stack: use index[4]=3
finish 5		low[5]$=3 \neq 4$	1, 2, 3, 4, 5	not a root
back at 4		low[4]$=\min(3,3)=3$	1, 2, 3, 4, 5	absorb child low
finish 4		low[4]=3=index[4]	1, 2, 3	**root**: pop 5,4 $\Rightarrow$ SCC$\{4,5\}$
back at 3		low[3]$=\min(0,3)=0$	1, 2, 3	child 4 done
finish 3		low[3]$=0 \neq 2$	1, 2, 3	not a root
back at 2		low[2]$=\min(1,0)=0$	1, 2, 3	absorb low[3]=0
edge $2 \to 5$		5 not on stack	1, 2, 3	**ignored** (finished SCC)
finish 2		low[2]$=0 \neq 1$	1, 2, 3	not a root
back at 1		low[1]$=\min(0,0)=0$	1, 2, 3	absorb low[2]=0
finish 1		low[1]=0=index[1]	$\varnothing$	**root**: pop 3,2,1 $\Rightarrow$ SCC$\{1,2,3\}$

The diagram below shows the two stack-pop events when roots 4 and 1 finish, along with the resulting two-node condensation DAG.



The trace highlights the two subtleties. When we examined $5 \to 4$, vertex 4 was *on the stack*, so we relaxed with index[4], dragging low[5] to 3 and signaling that 5 is not its own root. When we later examined $2 \to 5$, vertex 5 had already been popped into the finished component $\{4,5\}$, so onStack[5] was false and the edge was correctly ignored – exactly the guard that prevents the cross edge from merging $\{1,2,3\}$ with $\{4,5\}$.

Trace it yourself

Re-run the same digraph but add the edge $5 \to 2$. Now 5 can reach 2, and through the triangle 2 reaches 5, so all five vertices collapse into one SCC. Trace it: you will see that when finishing 4, low[4] no longer equals index[4] (the new edge pulls a lowlink down to 1 via $5 \to 2$), so no premature root pop happens, and the single root at vertex 1 pops all five. Tracing this case shows why the lowlink test is the right condition for popping a component.

Common trap

Recursive Tarjan overflows the call stack on adversarial inputs: a path of a million vertices recurses a million deep. In production, convert StrongConnect to an explicit stack of (vertex, edge-iterator) frames. The logic is identical – push a frame on a tree edge, do the lowlink relaxation when a child frame returns, run the root test when a frame's iterator is exhausted – but you control the memory. The exercises ask you to write this version.

5.5 The condensation DAG

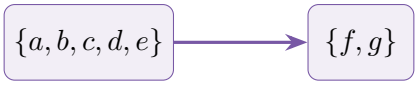
Once you know the SCCs, contract each one to a single super-vertex. The result removes the cycles and keeps only the one-way reachability between components.

Definition 5.3 (Condensation)**.** The *condensation* of a digraph G, written G^{SCC}, has one vertex per SCC of G. There is an edge from component X to component Y (with $X \neq Y$) iff some edge of G goes from a vertex of X to a vertex of Y. (Self-loops and parallel super-edges are dropped.)

Theorem 5.3 (The condensation is acyclic)**.** *G^{SCC} is a directed acyclic graph.*

Proof. Suppose for contradiction G^{SCC} contained a directed cycle $X_1 \to X_2 \to \cdots \to X_k \to X_1$ on distinct components. Each super-edge $X_i \to X_{i+1}$ comes from a real edge between the components, so in G we have $X_1 \rightsquigarrow X_2 \rightsquigarrow \cdots \rightsquigarrow X_k \rightsquigarrow X_1$, and within each strongly connected X_i we can route freely. Then every vertex of $X_1 \cup \cdots \cup X_k$ can reach every other: they all collapse into a single SCC, contradicting that the X_i are distinct maximal SCCs. So no cycle exists. □

Collapsing the two SCCs of our running digraph into super-vertices yields the two-node condensation DAG below.



For our running digraph the condensation is just two super-vertices with one edge – the single one-way street $c \to f$ between neighborhoods. A topological order of G^{SCC} ranks the components so that all super-edges point forward; here $\{a, b, c, d, e\}$ precedes $\{f, g\}$.

Why it works

The two SCC algorithms hand you a topological order of the condensation for free. Tarjan finishes (and pops) components in *reverse* topological order of G^{SCC}: a component is popped only after all components it can reach have been popped. So numbering components $1, 2, 3, \ldots$ in the order Tarjan emits them and then *reversing* gives a valid topological sort. Kosaraju's main loop emits them in topological order directly (highest finish time first is a source of the condensation). Either way, no separate topological sort is needed.

Application

The condensation turns many "directed" problems into easy DAG dynamic programs. Longest path in a general digraph is NP-hard, but *after* condensing, longest path in the DAG (weighting each super-vertex by, say, the number of original vertices it contains) is linear-time by DP over a topological order. Counting reachable vertices, finding a vertex that reaches everything, detecting whether the whole digraph is strongly connected (one SCC), and the 2-SAT solver below all reduce to a trivial pass over G^{SCC}.

Connection

The condensation is the directed-graph quotient that the companion linear-algebra book would describe as collapsing along an equivalence relation. It also previews the block-triangular structure of the adjacency matrix: order vertices by SCC in reverse topological order and $\mathbf{A}$ becomes block upper-triangular, with each diagonal block an irreducible (strongly connected) piece. That block structure is exactly what governs the convergence of Markov chains and PageRank in the spectral chapters: a chain is irreducible precisely when its transition digraph is a single SCC.

5.6 Application: 2-SAT in linear time

A *2-SAT* instance is a Boolean formula in conjunctive normal form where every clause has at most two literals, e.g.

$$(x_1 \vee x_2) \ \wedge \ (\neg x_1 \vee x_3) \ \wedge \ (\neg x_2 \vee \neg x_3) \ \wedge \ (x_3 \vee x_1).$$

We want a truth assignment satisfying every clause, or a proof that none exists. General SAT and even 3-SAT are NP-complete, but 2-SAT is solvable in linear time using strongly connected components.

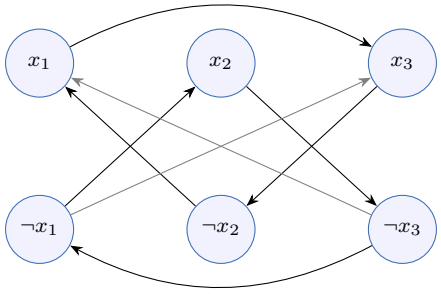
5.6.1 The implication graph

A clause $(\alpha \vee \beta)$ is logically equivalent to the pair of implications $\neg\alpha \Rightarrow \beta$ and $\neg\beta \Rightarrow \alpha$ ("if α is false then β must be true," and symmetrically). This rewriting is the key idea.

Definition 5.4 (Implication graph)**.** Build a digraph on $2n$ vertices, one for each literal x_i and one for $\neg x_i$. For every clause $(\alpha \vee \beta)$ add the two directed edges $\neg\alpha \to \beta$ and $\neg\beta \to \alpha$. (A unit clause (α) is treated as $(\alpha \vee \alpha)$, giving $\neg\alpha \to \alpha$.)

Reachability in this graph is forced truth propagation: if $u \rightsquigarrow v$ and we set u true, then v is forced true. The graph is *skew-symmetric*: edge $\neg\alpha \to \beta$ comes paired with $\neg\beta \to \alpha$, so reversing every edge and swapping each literal with its negation maps the graph to itself.

For the four-clause example formula above, the resulting six-vertex implication graph is drawn below.



5.6.2 The criterion and the assignment

Theorem 5.4 (2-SAT via SCC)**.** *A 2-SAT instance is satisfiable if and only if no variable x_i has x_i and $\neg x_i$ in the same SCC of the implication graph. When satisfiable, setting each literal ℓ to* true *iff* $\mathrm{scc}(\ell)$ *comes* after $\mathrm{scc}(\neg\ell)$ *in a reverse topological order of the condensation yields a valid assignment.*

Proof. ($\Rightarrow$, necessity) If $x_i \equiv \neg x_i$, then $x_i \rightsquigarrow \neg x_i$ and $\neg x_i \rightsquigarrow x_i$. The first path forces: x_i true implies $\neg x_i$ true, a contradiction, so x_i must be false. The second path forces x_i true. No assignment survives both, so the instance is unsatisfiable.

($\Leftarrow$, sufficiency and construction) Assume every variable has x_i and $\neg x_i$ in different SCCs. First a symmetry fact: skew-symmetry of the implication graph means $\mathrm{scc}(\ell)$ and $\mathrm{scc}(\neg\ell)$ are mirror images – if $u \rightsquigarrow v$ then $\neg v \rightsquigarrow \neg u$, so the SCC containing ℓ corresponds to the SCC containing $\neg\ell$ under literal negation, and the condensation has a matching reversal symmetry.

Compute a topological order of the condensation. Assign $\ell \leftarrow$ true iff $\mathrm{scc}(\ell)$ appears later in topological order than $\mathrm{scc}(\neg\ell)$ – equivalently, $\mathrm{scc}(\ell)$ has the larger "topo rank." Since x_i and $\neg x_i$ are in different SCCs, exactly one of them is set true, so the assignment is well-defined and consistent. We must check it satisfies every implication edge $u \to v$: we need "u true $\Rightarrow$ v true," i.e. we must never have u true and v false.

Suppose toward contradiction $u \to v$ with u true and v false. "u true" means $\mathrm{scc}(u)$ ranks after $\mathrm{scc}(\neg u)$. "v false" means $\mathrm{scc}(v)$ ranks before $\mathrm{scc}(\neg v)$. The edge $u \to v$ forces $\mathrm{scc}(u)$ to rank before-or-equal $\mathrm{scc}(v)$ in topological order (edges go forward; if same component, $u \equiv v$). By skew-symmetry the mirror edge $\neg v \to \neg u$ forces $\mathrm{scc}(\neg v)$ before-or-equal $\mathrm{scc}(\neg u)$. Chain the ranks:

$$\mathrm{rank}(\mathrm{scc}(\neg u)) < \mathrm{rank}(\mathrm{scc}(u)) \le \mathrm{rank}(\mathrm{scc}(v)) < \mathrm{rank}(\mathrm{scc}(\neg v)) \le \mathrm{rank}(\mathrm{scc}(\neg u)),$$

where the strict inequalities are "u true" and "v false" and the weak ones are the two implication edges. This says $\mathrm{rank}(\mathrm{scc}(\neg u)) < \mathrm{rank}(\mathrm{scc}(\neg u))$, a contradiction. So no edge is violated, every clause's two implications hold, and the assignment satisfies the formula. □

Why it works

The proof rests on one idea: a literal is true exactly when its node "comes later" than its negation in the DAG of components. Skew-symmetry guarantees that "later" and "earlier" swap perfectly under negation, so the consistency you need on one edge $u \to v$ is mirrored by the consistency on $\neg v \to \neg u$, and the two cannot both fail.

Because Tarjan pops components in reverse topological order, the scc *labels* it produces already satisfy "larger reverse-topo rank = larger label." That makes the assignment a one-liner: set x_i true iff $\text{scc}(x_i) < \text{scc}(\neg x_i)$ when labels increase as Tarjan emits (reverse topological) – or flip the comparison to match your labeling convention. Always sanity-check the direction on a tiny instance.

Algorithm 19: 2-SAT solver

Input: n variables, clauses each a pair of literals.
Output: A satisfying assignment, or UNSAT.

build implication graph H on $2n$ literal vertices
foreach *clause* $(\alpha \vee \beta)$ **do**
  add edges $\neg\alpha \to \beta$ and $\neg\beta \to \alpha$ to H
run Tarjan on H to get $\text{scc}[\cdot]$ (labels in reverse topological order)
foreach *variable* x_i **do**
  if $\text{scc}[x_i] = \text{scc}[\neg x_i]$ **then return** UNSAT
foreach *variable* x_i **do**
  $\text{val}[x_i] \leftarrow (\text{scc}[x_i] < \text{scc}[\neg x_i])$ `// true iff its component is popped later`
return val

Cost summary

The implication graph has $2n$ vertices and $2m$ edges for n variables and m clauses. Building it is $O(n+m)$; Tarjan is $O(n+m)$; reading off the assignment is $O(n)$. Total $O(n+m)$ – linear in the size of the formula. Contrast with general SAT, where the best known algorithms are exponential in the worst case.

Trace it yourself

Solve the four-clause example by hand. Build the eight directed edges, run Tarjan, and check that no x_i shares a component with $\neg x_i$. You should find a satisfying assignment such as $x_1 = \mathsf{true}, x_2 = \mathsf{false}, x_3 = \mathsf{true}$. Verify each clause. Then add the clause $(\neg x_3 \vee \neg x_1)$ together with $(x_3 \vee x_1)$ already present and the forcing $x_1 \to x_3 \to \neg x_1$ and back; trace whether x_1 and $\neg x_1$ now collapse into one SCC, which would report UNSAT.

Application

2-SAT solves a wide range of feasibility problems: placing labels on a map so none overlap (each label has two candidate positions, conflicts become clauses), scheduling with binary either/or constraints, drawing graphs with each edge routed above or below a line, and consistency checking in type systems. Whenever a problem reduces to "each of n binary choices, with pairwise constraints expressible as implications," the SCC method decides it in linear time.

5.7 Eulerian Circuits, Hierholzer's Algorithm, and De Bruijn Genome Assembly

In the summer of 1736, the Swiss mathematician Leonhard Euler presented a paper to the Saint Petersburg Academy that marked the birth of graph theory. The paper addressed a recreational riddle posed by the citizens of Königsberg in Prussia: could one take a stroll through town, crossing each of the seven bridges over the Pregel River exactly once, and return to where one started?

The Pregel River split the city into four landmasses: Kneiphof Island, the northern and southern banks, and a fork between river branches. Euler observed that the precise topography – the lengths of the roads, the geometry of the riverbanks, the walking speed of the traveler – held no bearing on whether such a walk existed. What mattered was the discrete incidence structure: four discrete land regions connected by seven bridges. By stripping away irrelevant geometry, Euler reduced the puzzle to the first graph in mathematical history.

A walk in a graph that traverses every edge exactly once is an *Eulerian walk*; if it begins and ends at the same vertex, it is an *Eulerian circuit*.

If we instead ask for a closed walk that visits every *vertex* of a graph exactly once, we arrive at the *Hamiltonian cycle* problem. As Chapter 37 (and Section 37.5) proves, deciding whether a graph admits a Hamiltonian cycle is NP-complete; the best known worst-case algorithms take exponential time $O(2^{|V|}|V|^2)$ via Held–Karp dynamic programming. Yet deciding whether a graph admits an Eulerian circuit, and finding one, takes $O(|V|+|E|)$ linear time – as fast as a single depth-first search.

This disparity stems from how the two constraints restrict search. A search for a Hamiltonian cycle must remember which arbitrary subset of vertices has already been consumed, creating an exponential state space of $2^{|V|}$ potential partial paths. An Eulerian walk, by contrast, is governed entirely by local degree conservation at each vertex.

5.7.1 Degree conservation and Euler's theorem

Imagine an agent navigating a graph along unvisited edges. Each time the walk passes through an intermediate vertex v, it must enter along one unvisited edge and depart along a different unvisited edge. Incident edges at v are consumed in pairs: one entrance, one exit.

Intuition

Think of edges incident to vertex v as tickets to enter and leave. Entering costs one ticket; leaving costs another. If an agent begins at s, wanders through the graph, and returns to s, every intermediate visit to a vertex consumes two tickets. A closed walk that exhausts all tickets can exist only if every vertex holds an even number of tickets. In a directed graph, each entrance consumes an in-edge and each departure consumes an out-edge, so the number of inbound tickets must equal the number of outbound tickets.

This local balance condition leads directly to the classical characterizations.

Definition 5.5 (Eulerian walk and circuit)**.** Let $G = (V, E)$ be a graph (undirected or directed). An *Eulerian walk* is a walk that traverses every edge in E exactly once. An *Eulerian circuit* (or *Eulerian tour*) is an Eulerian walk that begins and ends at the same vertex.

For an undirected graph, every visit to an intermediate vertex v during an Eulerian circuit contributes 2 to the degree of v in the traversed subgraph. Because the start vertex s has its initial departure balanced by its final return, every vertex with non-zero degree must have even degree.

For an open walk starting at s and ending at $t \neq s$, vertex s has one unmatched departure (odd degree), vertex t has one unmatched arrival (odd degree), and every intermediate vertex $v \in V \setminus \{s, t\}$ has balanced departures and arrivals (even degree).

The exact same reasoning applies to a directed graph:

1. In a directed Eulerian circuit, every vertex v must satisfy $\text{in}(v) = \text{out}(v)$. A digraph satisfying this condition at all vertices is called *balanced*.
2. In a directed open Eulerian walk from s to t, $\text{out}(s) - \text{in}(s) = 1$, $\text{in}(t) - \text{out}(t) = 1$, and $\text{in}(v) = \text{out}(v)$ for every other vertex $v \in V \setminus \{s, t\}$. A digraph satisfying this condition is called *semi-balanced*.

Degree balance alone is necessary, but not sufficient: a graph consisting of two disjoint, disconnected triangles has all even degrees, yet no walk can jump the gap between them. All edges must belong to a single connected component.

Theorem 5.5 (Euler–Hierholzer Theorem)**.** *An undirected multigraph $G = (V, E)$ with $|E| \geq 1$ has an Eulerian circuit if and only if all edges belong to a single connected component and every vertex has even degree. It has an open Eulerian walk if and only if all edges belong to a single connected component and exactly two vertices have odd degree (which are the start and end of the walk).*

A directed multigraph $G = (V, E)$ whose edges belong to a single weakly connected component has an Eulerian circuit if and only if $\text{in}(v) = \text{out}(v)$ for all $v \in V$. It has an open Eulerian walk from s to t ($s \neq t$) if and only if $\text{out}(s) - \text{in}(s) = 1$, $\text{in}(t) - \text{out}(t) = 1$, and $\text{in}(v) = \text{out}(v)$ for all $v \in V \setminus \{s, t\}$.

In the Königsberg bridge problem, Kneiphof Island had degree 5, and the other three landmasses had degree 3. All four vertices had odd degrees. Euler concluded that no pedestrian could traverse every bridge without repeating one, resolving the puzzle without examining every route by trial and error.

5.7.2 Hierholzer's cycle-splicing algorithm

Euler proved that even degrees are necessary for a circuit in 1736. The sufficiency of the condition was established constructively by Carl Hierholzer in 1873.

Suppose we pick a start vertex s and greedily follow unused edges. Because every vertex has balanced degrees, whenever the walk arrives at an intermediate vertex $u \neq s$, there is an unconsumed exit edge available to continue. The walk cannot become stranded at an intermediate vertex; it can terminate only when it returns to s and finds no remaining unused edges incident to s.

This initial greedy traversal produces a simple closed cycle C_1. However, C_1 might not cover every edge in the graph. In a figure-eight graph formed by two triangles joined at a single shared vertex v, a greedy walk starting on the left triangle will traverse the left cycle and return to its start, leaving the right triangle untouched.

Hierholzer recognized that if unvisited edges remain, connectivity ensures that at least one vertex v on C_1 has incident unvisited edges. By launching another greedy walk from v through the unused

edges, degree parity guarantees that this detour must also return to v, generating a second closed cycle C_2. Hierholzer spliced C_2 into C_1 at v:

$$s \rightsquigarrow v \xrightarrow{\text{insert } C_2} v \rightsquigarrow s.$$

Panel (a) of Figure 5.1 illustrates this cycle-splicing principle.

A naive implementation that repeatedly scans the existing tour for vertices with unused edges can degenerate to $O(|E|^2)$ running time. To achieve $O(|V|+|E|)$ time, we implement Hierholzer's method using an explicit walk stack and head pointers into adjacency lists.

Algorithm 20: Hierholzer's linear-time Eulerian circuit algorithm

Input: A directed graph $G = (V, E)$ where $\text{in}(v) = \text{out}(v)$ for all v, with edges in a single connected component.

Output: List of vertices circuit giving an Eulerian circuit.

```
pick a vertex s ∈ V with out(s) > 0
stack S ← [s]
list circuit ← []
initialize an edge pointer head[u] ← 0 for each u ∈ V
while S is not empty do
    u ← top(S)
    if head[u] < |adj[u]| then
        (u, w) ← adj[u][head[u]]
        head[u] ← head[u] + 1
        push w onto S
    else
        pop u from S
        append u to circuit
return reverse of circuit
```

Why it works

Algorithm 20 splices detours in $O(1)$ amortized time per edge. When the forward greedy walk exhausts the exit edges of its current cycle at vertex u, the algorithm pops u and appends it to circuit. The stack unwinds until it reaches an ancestor v that still possesses unvisited outgoing edges. The algorithm then pushes the vertices of that detour cycle onto S and explores them fully. Because the detour is pushed and subsequently popped before v's remaining ancestors unwind, the detour appears immediately prior to v in the reversed list. Every edge is traversed once and pushed onto S once, bounding total runtime by $O(|V| + |E|)$.

Reading note

Carl Hierholzer developed this cycle-splicing construction shortly before his death in 1871; his paper was edited and published posthumously by Chr. Wiener in 1873. For directed graphs, the degree condition appeared in the work of Paul Erdős and Tibor Gallai in the early twentieth century. In 1946, Nicolaas Govert de Bruijn used directed Eulerian circuits to construct circular sequences containing every word of length k over an alphabet. Decades later, Pevzner, Tang, and Waterman recognized that De Bruijn's Eulerian construction provided the

basis for modern genome assembly.

5.7.3 De Bruijn graphs and DNA shotgun sequencing

The most consequential modern application of Eulerian walks is in computational genomics. High-throughput sequencing machines (such as Illumina platforms) cannot read a complete human chromosome of several hundred million base pairs from start to finish. Instead, they extract millions of overlapping fragments called *short reads*, each typically 100 to 250 base pairs in length, drawn randomly across the genome. The computational problem is *de novo genome assembly*: reconstruct the original genomic sequence from these fragmented reads.

In the 1990s, early assemblers relied on the *Overlap-Layout-Consensus* (OLC) approach. The OLC formulation models each sequenced read as a vertex. An edge is drawn from read R_i to read R_j if a long suffix of R_i matches a prefix of R_j. Reconstructing the genome requires finding a path that visits every sequenced read once – a Hamiltonian path. Because Hamiltonian path is NP-complete, assemblers faced severe computational bottlenecks when scaling to tens of millions of reads.

In 2001, Pavel Pevzner, Haixu Tang, and Michael Waterman replaced the overlap graph with a De Bruijn graph. Rather than mapping reads to vertices, they decompose each read into overlapping substrings of length k (k-mers) and length $k-1$ ($(k-1)$-mers).

Definition 5.6 (De Bruijn graph of a sequence)**.** Let $\mathcal{R}$ be a collection of observed k-mers. The *De Bruijn graph* $B_k(\mathcal{R})$ has:

1. **Vertices**: The set of all distinct $(k-1)$-mers appearing as prefixes or suffixes of k-mers in $\mathcal{R}$.
2. **Directed edges**: For each k-mer $w = c_1c_2\ldots c_k \in \mathcal{R}$, a directed edge from prefix $c_1\ldots c_{k-1}$ to suffix $c_2\ldots c_k$, labeled by w.

In this representation, every sequenced k-mer is an *edge*. Assembling the genome requires using every sampled k-mer, which means traversing every edge of the De Bruijn graph exactly once. Genome assembly becomes the problem of finding an *Eulerian walk*.

Shifting the data representation from vertices to edges converts an NP-complete Hamiltonian problem into an $O(|V| + |E|)$ linear-time Eulerian traversal.

5.7.4 Trace of a DNA assembly on a De Bruijn graph

Trace this reduction on a concrete DNA sequence. Consider target sequence $S = \texttt{ATGGCGTGCA}$ of length 10, and choose $k = 3$. The sequence yields $10 - 3 + 1 = 8$ overlapping 3-mers:

$$\mathcal{R} = \{\texttt{ATG}, \texttt{TGG}, \texttt{GGC}, \texttt{GCG}, \texttt{CGT}, \texttt{GTG}, \texttt{TGC}, \texttt{GCA}\}.$$

The $(k-1)$-mers are the 2-character prefixes and suffixes:

$$V = \{\texttt{AT}, \texttt{TG}, \texttt{GG}, \texttt{GC}, \texttt{CG}, \texttt{GT}, \texttt{CA}\}.$$

Each 3-mer introduces a directed edge between its prefix and suffix:

$$\begin{array}{llll} \texttt{ATG}\colon \texttt{AT} \to \texttt{TG}, & \texttt{TGG}\colon \texttt{TG} \to \texttt{GG}, & \texttt{GGC}\colon \texttt{GG} \to \texttt{GC}, & \texttt{GCG}\colon \texttt{GC} \to \texttt{CG}, \\ \texttt{CGT}\colon \texttt{CG} \to \texttt{GT}, & \texttt{GTG}\colon \texttt{GT} \to \texttt{TG}, & \texttt{TGC}\colon \texttt{TG} \to \texttt{GC}, & \texttt{GCA}\colon \texttt{GC} \to \texttt{CA}. \end{array}$$

(a) Hierholzer's cycle-splicing mechanism **(b) De Bruijn graph: $k = 3$ on ATGGCGTGCA**

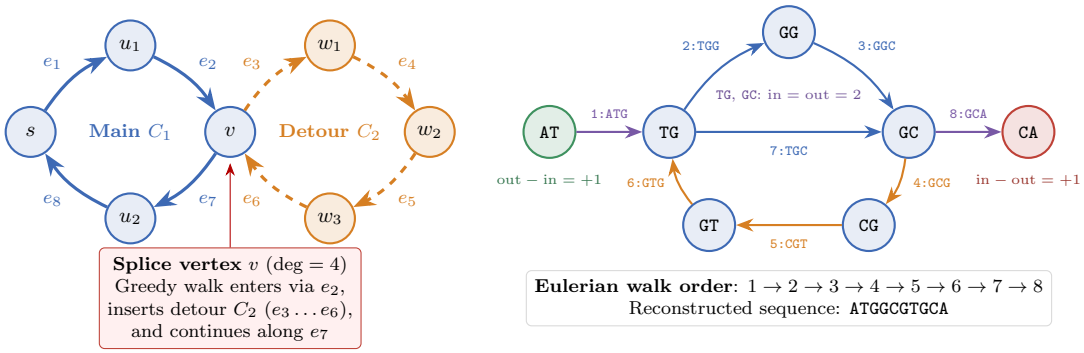


Figure 5.1: Eulerian circuits and genome assembly. (a) Hierholzer's cycle-splicing technique: a greedy walk from s traces cycle C_1; reaching splice vertex v with remaining unused edges launches detour cycle C_2, which is spliced directly into C_1. (b) The De Bruijn graph B_3 for sequence ATGGCGTGCA: overlapping 2-mers form vertices and 3-mers form directed edges. Because degree balances hold (in = out everywhere except start AT and end CA), an Eulerian path traverses every 3-mer edge in linear time, assembling the full DNA sequence.

Examine the in- and out-degrees across the resulting digraph, shown in Figure 5.1(b):

- Vertex AT has $\text{in}(\texttt{AT}) = 0$ and $\text{out}(\texttt{AT}) = 1$. This is the unique start vertex ($\text{out} - \text{in} = 1$).
- Vertex CA has $\text{in}(\texttt{CA}) = 1$ and $\text{out}(\texttt{CA}) = 0$. This is the unique terminal vertex ($\text{in} - \text{out} = 1$).
- Intermediate vertex TG has two incoming edges (ATG, GTG) and two outgoing edges (TGG, TGC), so $\text{in}(\texttt{TG}) = \text{out}(\texttt{TG}) = 2$.
- Intermediate vertex GC has two incoming edges (GGC, TGC) and two outgoing edges (GCG, GCA), so $\text{in}(\texttt{GC}) = \text{out}(\texttt{GC}) = 2$.
- All other vertices (GG, CG, GT) have $\text{in}(v) = \text{out}(v) = 1$.

Because the graph is weakly connected and meets the semi-balanced degree condition of Theorem 5.5, an open Eulerian walk from AT to CA must exist.

Applying Hierholzer's algorithm traces the edges in numerical order:

$$\texttt{AT} \xrightarrow{\texttt{ATG}} \texttt{TG} \xrightarrow{\texttt{TGG}} \texttt{GG} \xrightarrow{\texttt{GGC}} \texttt{GC} \xrightarrow{\texttt{GCG}} \texttt{CG} \xrightarrow{\texttt{CGT}} \texttt{GT} \xrightarrow{\texttt{GTG}} \texttt{TG} \xrightarrow{\texttt{TGC}} \texttt{GC} \xrightarrow{\texttt{GCA}} \texttt{CA}.$$

Notice that the cycle TG → GG → GC → CG → GT → TG forms a detour loop that returns to TG before the traverse exits via TGC → GC → CA. Appending the final character of each traversed k-mer recovers the genomic sequence:

$$\texttt{AT} + \texttt{G} + \texttt{G} + \texttt{C} + \texttt{G} + \texttt{T} + \texttt{G} + \texttt{C} + \texttt{A} = \texttt{ATGGCGTGCA}.$$

Application

Real-world DNA sequencers introduce two biological complications: sequencing errors and genomic repeats. An erroneous base creates isolated dead-end paths called "tips" and false parallel cycles called "bubbles." Repetitive sequences longer than k cause multiple Eulerian

paths through the graph, introducing ambiguity. Modern production assemblers like Velvet and SPAdes use graph simplification algorithms: they prune tips with low read coverage, collapse bubbles where paths re-converge, and use paired-end reads to resolve repeat regions. Beneath these heuristics, the core step is still a linear-time Eulerian traversal of a De Bruijn graph.

5.8 Connectivity refinements and edge cases

This section collects a few further structural facts.

A digraph is strongly connected iff it has exactly one SCC. Testing this needs no full SCC decomposition: pick any vertex s; if a DFS from s in G reaches all vertices *and* a DFS from s in $G^\top$ reaches all vertices, then everything reaches s and s reaches everything, so all pairs are mutually reachable – one SCC. Two traversals, $O(|V| + |E|)$.

Weak connectivity: a digraph is *weakly connected* if the underlying undirected graph (forget directions) is connected. Weak connectivity is strictly easier and strictly coarser than strong connectivity – a directed path $a \to b \to c$ is weakly connected but has three SCCs.

Sources and sinks of the condensation: a *source* component (no incoming super-edges) and a *sink* component (no outgoing super-edges) always exist in a nonempty DAG. A classic interview problem – "add the fewest edges to make a digraph strongly connected" – is answered through the condensation: if it has more than one component, the answer is $\max(\text{sources}, \text{sinks})$, where sources and sinks count source and sink super-vertices; with one component the answer is 0.

Common trap

Self-loops and parallel edges never change SCCs (a self-loop puts a vertex on a trivial cycle with itself, but a single vertex is already its own SCC). Isolated vertices are singleton SCCs. Make sure your implementation treats an isolated vertex as a component of size one rather than skipping it – both Algorithm 17 and Algorithm 18 handle this correctly because the outer loop visits every vertex, but a hand-rolled variant that only starts from in-degree-zero vertices will silently drop them.

Summary

Connected components fall out of one undirected traversal. Strong connectivity is the directed analogue that demands round-trip reachability; its classes, the SCCs, are found in linear time two ways. Kosaraju runs DFS on G to get finish times, then DFS on $G^\top$ in decreasing finish order, peeling SCCs off in topological order of the condensation – and the single lemma about component finish times proves it. Tarjan does it in one pass with the lowlink invariant, popping each SCC when its root is recognized by $\text{low}[u] = \text{index}[u]$, and emitting components in reverse topological order. Contracting SCCs gives the condensation, always a DAG, which exposes one-way reachability and powers DAG dynamic programs. Applying SCCs to 2-SAT solves boolean implication constraints in linear time: build the implication graph, run SCC, declare unsatisfiable iff some variable shares a component with its negation, and otherwise read a satisfying assignment off the topological order.

Finally, Eulerian circuits shift the focus from vertex reachability to edge traversal: under local degree balance, Hierholzer's cycle-splicing algorithm traces an Eulerian circuit in $O(|V|+|E|)$ time, turning shotgun genome assembly on De Bruijn graphs from an NP-complete Hamiltonian path problem into a linear-time edge walk.

This completes our study of unweighted traversals and structural decompositions in Part I. Across these first five chapters, every edge carried unit weight. In Part II, we move to weighted graphs, where edges carry costs, lengths, and capacities. We begin in Chapter 6 with Minimum Spanning Trees: connecting an entire graph at minimum total cost, where greedy choices replace systematic traversals and the cut and cycle properties provide exact certificates of optimality.

5.9 Exercises

Exercise 5.1 (Iterative Tarjan)**.** Rewrite StrongConnect (Algorithm 18) without recursion, using an explicit stack of frames each holding a vertex and an index into its adjacency list. Match the recursive version exactly: perform the lowlink relaxation when a child frame returns, and the root test when a frame's adjacency list is exhausted. Argue your version is still $O(|V| + |E|)$ and uses $O(|V|)$ explicit-stack space.

Exercise 5.2 (Gabow's path-based SCC)**.** Gabow's algorithm replaces the lowlink number with a second stack of "open" vertices. On discovering v, push it on both the main stack S and the path stack P. On a back/cross edge into a live vertex w, pop P down to the vertex with index $\leq \text{index}[w]$. When v finishes and is on top of P, pop S down to v as one SCC and pop v off P. Implement it and prove it computes the same SCCs as Tarjan. Why does P encode exactly the lowlink information?

Exercise 5.3 (Kosaraju needs the reverse pass)**.** Show by a concrete 4-vertex example that running the *second* DFS on G (instead of $G^{\top}$) in decreasing finish order can merge two distinct SCCs into one reported component. Pinpoint which step of the proof of Theorem 5.1 breaks.

Exercise 5.4 (Lowlink vs index)**.** Construct a digraph on which Tarjan, modified to relax with $\text{low}[w]$ instead of $\text{index}[w]$ on non-tree edges, reports the wrong SCCs. Explain in terms of cross edges why the bug appears and why it cannot appear for back edges into a direct ancestor.

Exercise 5.5 (Condensation and longest path)**.** Given a digraph G where each vertex has a positive weight, give an $O(|V|+|E|)$ algorithm for the maximum total weight of vertices on any directed walk (a walk may revisit vertices). Reduce to a DAG problem on the condensation. Prove correctness and explain why allowing revisits makes the condensation the right object.

Exercise 5.6 (Strong connectivity augmentation)**.** Prove that the minimum number of edges to add to a digraph (with more than one SCC) to make it strongly connected equals $\max(s, t)$, where s and t are the numbers of source and sink components in the condensation. Give a construction achieving the bound. (Hint: pair sources with sinks.)

Exercise 5.7 (2-SAT assignment direction)**.** The text claims "set ℓ true iff $\text{scc}(\ell)$ comes later in topological order than $\text{scc}(\neg\ell)$." Tarjan emits components in reverse topological order. Determine precisely whether the correct rule with Tarjan labels is $\text{scc}[\ell] < \text{scc}[\neg\ell]$ or $>$, by tracing a unit-clause instance (x_1) and confirming x_1 comes out true.

Exercise 5.8 (At-most-one constraints)**.** A common modeling primitive is "at most one of $y_1, \ldots, y_k$ is true." Naively adding a clause $(\neg y_i \vee \neg y_j)$ for each pair costs $O(k^2)$ clauses. Design an encoding

with auxiliary variables using only $O(k)$ clauses that is equisatisfiable, and verify it stays within 2-SAT (every clause has ≤ 2 literals). Prove your encoding forbids exactly the assignments with two or more y's true.

Exercise 5.9 (Counting SCCs of a functional graph)**.** A *functional graph* has out-degree exactly one at every vertex (each v has a single successor $f(v)$). Show that every SCC of a functional graph is a single cycle, and give an $O(|V|)$ algorithm that finds all SCCs without a general SCC routine. How does the condensation of a functional graph look?

Exercise 5.10 (Reachability after condensation)**.** After computing the condensation G^{SCC}, you want to answer many queries "can vertex u reach vertex v?" Explain how to reduce each query to reachability between two super-vertices in the DAG, and discuss the time/space tradeoff of precomputing a transitive-closure bitset over the condensation ($O(|V_{\mathrm{SCC}}|^2/64)$ words) versus answering each query with a fresh DAG search.

Exercise 5.11 (Bridges between SCCs)**.** Call an edge of G *critical* if deleting it increases the number of SCCs. Characterize critical edges in terms of the condensation and the DFS tree, and sketch a linear-time algorithm to list them. How does this relate to bridges in the undirected sense covered in a later chapter?

Exercise 5.12 (De Bruijn k-mer spectrum and repeats)**.** Suppose a genome contains a tandem repeat R of length $L > k$. Explain why the De Bruijn graph B_k contains a directed cycle corresponding to R, resulting in multiple valid Eulerian paths that cannot be distinguished from short k-mers alone. Show how a single paired-end read with insert length exceeding L provides the long-range constraint needed to uniquely select the correct Eulerian traversal.

Exercise 5.13 (Counting Eulerian circuits: The BEST Theorem)**.** In a balanced directed graph where every vertex has $\mathrm{in}(v) = \mathrm{out}(v) = d_v$, the number of Eulerian circuits starting at a fixed edge is given by the BEST Theorem (de Bruijn, van Aardenne-Ehrenfest, Smith, and Tutte):

$$\mathrm{ec}(G) = t_r(G) \prod_{v \in V} (d_v - 1)!,$$

where $t_r(G)$ is the number of directed spanning trees directed into any fixed root vertex r. Explain why the product of factorials counts the independent ordering choices of outgoing edges at each vertex, and why a directed spanning tree into r is necessary to guarantee that these local choices form a single connected circuit rather than multiple disjoint cycles.

Part II

Trees, Paths, and Cuts

Chapter 6

Minimum Spanning Trees

In Part I, we traversed unweighted graphs where every edge counted as a single step. In Part II, edges acquire real-valued weights – representing costs, distances, or capacities. When weights measure the cost of constructing or maintaining connections, our goal is to span all n vertices using as few edges as possible while minimizing the total cost. Any minimal connected subgraph is a spanning tree with exactly $n - 1$ edges. This chapter develops the minimum spanning tree (MST) problem from the cut and cycle properties, showing why greedy choices by Kruskal, Prim, and Borůvka are provably optimal.

6.1 The problem and a picture

Definition 6.1 (Spanning tree, MST)**.** Let $G = (V, E)$ be a connected undirected graph with a weight $w(e) \in \mathbb{R}$ on each edge. A *spanning tree* is an acyclic connected subgraph $T = (V, E_T)$ with $E_T \subseteq E$; necessarily $|E_T| = n - 1$ where $n = |V|$. Its weight is $w(T) = \sum_{e \in E_T} w(e)$. A *minimum spanning tree* is a spanning tree of least total weight. If G is disconnected, the analogous object is a *minimum spanning forest*, one tree per connected component.

Intuition

Think of the vertices as towns and the edges as candidate stretches of fibre, each labelled by what it costs to dig. You must be able to route a signal between any two towns, and you are paying by the metre. You do not care about path lengths or redundancy – only that the network is connected and that the total bill is as small as possible. The answer is always a tree: any cycle contains an edge you could delete while staying connected, and deleting a positive-cost edge never hurts. The question is *which* tree.

Here is the running example we will trace by hand throughout the chapter.

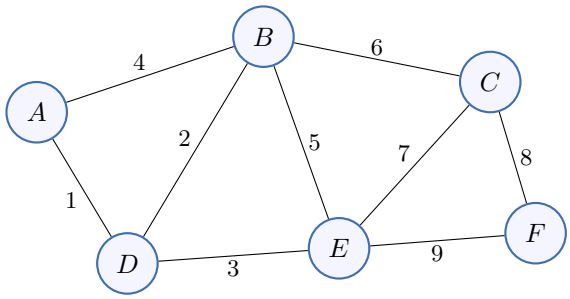


Figure 6.1: The running graph G_0. Six vertices, nine weighted edges, all weights distinct.

We will keep all weights distinct in G_0 on purpose: distinct weights force a *unique* MST, which makes the traces unambiguous. Section 6.8 handles ties. For the record, the MST of G_0 turns out to be $\{AD(1), BD(2), DE(3), BC(6), CF(8)\}$ with total weight 20; every algorithm below will rediscover it.

6.2 Two structural laws: the cut and the cycle

Every correct MST algorithm rests on two facts about which edges *must* or *must not* appear. They are duals of each other: one is phrased over cuts, the other over cycles.

Definition 6.2 (Cut)**.** A *cut* $(S, \overline{S})$ is a partition of V into a nonempty set S and its complement $\overline{S} = V \setminus S$, both nonempty. An edge *crosses* the cut if it has one endpoint in S and one in $\overline{S}$. A cut *respects* a set of edges F if no edge of F crosses it.

Focus

Cut property. For any cut, the cheapest crossing edge can be safely put into an MST.
Cycle property. For any cycle, the most expensive edge on it can be safely left out of every MST.
Both "safely" statements become *exactly one MST forced* when the relevant minimum/maximum is unique.

Theorem 6.1 (Cut property)**.** *Let $F \subseteq E$ be a subset of edges contained in some MST of G. Let $(S, \overline{S})$ be any cut that respects F, and let e be a crossing edge of minimum weight among all crossing edges. Then $F \cup \{e\}$ is also contained in some MST.*

Why it works

The proof is the *exchange argument* (Figure 6.2), the standard way to prove that a greedy choice is optimal. Take any MST T that contains F. If it already uses e, we are done. If not, adding e to T creates exactly one cycle; that cycle must cross the cut a second time (it left S via e, it has to come back), so it contains another crossing edge e'. Swap them: remove e', add e. The result is still a spanning tree, it still contains F (because the cut respected F, so $e' \notin F$), and since e was a cheapest crossing edge, $w(e) \le w(e')$, so the swap did not increase the weight. A tree no heavier than an MST is an MST.

Proof. Let T be an MST with $F \subseteq E_T$. If $e \in E_T$ then $F \cup \{e\} \subseteq E_T$ and we are finished. Otherwise $T + e$ contains a unique cycle γ (adding one edge to a tree always creates exactly one cycle). The endpoints of e lie on opposite sides of the cut $(S, \overline{S})$, so traversing γ we cross from S to $\overline{S}$ at e and must cross back at least once; pick such a return edge $e' \in \gamma$, $e' \neq e$, with e' crossing the cut. Because the cut respects F and e' crosses it, $e' \notin F$.

Form $T' = T - e' + e$. Removing e' from $T + e$ breaks the only cycle, so T' is acyclic; it has $n - 1$ edges and is acyclic, hence a spanning tree. Its weight is $w(T') = w(T) - w(e') + w(e)$. Since e is a minimum-weight crossing edge and e' also crosses, $w(e) \leq w(e')$, so $w(T') \leq w(T)$. But T is minimum, so $w(T') = w(T)$ and T' is also an MST. Finally $F \cup \{e\} \subseteq E_{T'}$: every edge of F survived (we only deleted $e' \notin F$) and e was added. □

Corollary 6.1 (Cut property, sharp form)**.** *If, in addition, e is the* unique *minimum-weight crossing edge of the cut, then* every *MST containing F contains e.*

Proof. Suppose some MST $T \supseteq F$ omits e. Run the exchange as above to get $T' = T - e' + e$ with $w(e) < w(e')$ (strict, by uniqueness), so $w(T') < w(T)$, contradicting minimality of T. □

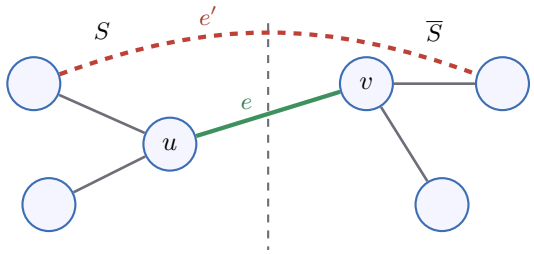


Figure 6.2: The exchange argument for the cut property. The cut (dashed vertical line) splits the tree; e (green) is the cheapest crossing edge. Adding e to the tree closes a cycle that must cross the cut a second time at some tree edge e' (red dashed). Since $w(e) \leq w(e')$, swapping e' out for e does not increase the weight.

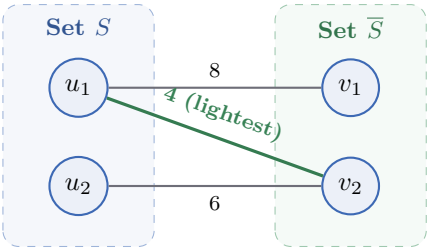
The cycle property is the mirror image and is proved the same way, exchanging in the other direction.

Theorem 6.2 (Cycle property)**.** *Let γ be any cycle in G and let f be an edge of strictly maximum weight on γ. Then f belongs to* no *MST.*

Proof. Suppose an MST T contains f. Deleting f splits T into two components, inducing a cut $(S, \overline{S})$ that f crosses. The cycle γ enters and leaves S, so it contains another crossing edge $g \neq f$ with $g \notin E_T$ (the only T-edge crossing this cut is f). By the choice of f, $w(g) < w(f)$. Then $T - f + g$ is a spanning tree of weight $w(T) - w(f) + w(g) < w(T)$, contradicting minimality. □

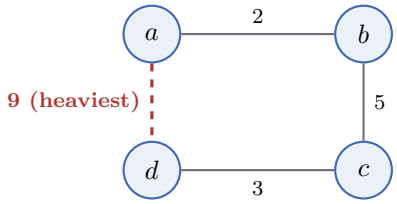
The two panels below summarize this duality between including the lightest edge across a cut and excluding the heaviest edge along a cycle.

Cut Property: Lightest crossing edge included



Cheapest crossing edge $w = 4$ safe to add

Cycle Property: Heaviest edge excluded



Heaviest cycle edge $w = 9$ excluded from MST

Intuition

The cut property tells you what to *include*; the cycle property tells you what to *exclude*. Kruskal and Boruvka are most naturally read as repeated applications of the cut property building the tree up; the cycle property is the tool of choice when you want to argue an edge is *wrong* – which is the basis of MST verification in Section 6.9. The reverse-delete algorithm (sort descending, delete an edge whenever doing so keeps the graph connected) turns the cycle property directly into an algorithm.

Common trap

The cut property does not say "a minimum crossing edge is in *every* MST." Only the *sharp* form, with a *unique* minimum, forces the edge. With ties, a cheapest crossing edge is in *some* MST, which is exactly what a greedy algorithm needs – but two different tie-breaks can build two different trees. Conflating these two readings is the single most common MST proof bug.

6.3 The generic greedy algorithm

All three classical algorithms are instances of one meta-algorithm: maintain a forest F that is a subset of *some* MST (we call such an F *promising*), and repeatedly add a *safe* edge – a minimum-weight edge crossing some cut that respects F – until F spans.

Algorithm 21: Generic-MST – the greedy meta-algorithm

Input: connected weighted graph $G = (V, E, w)$
Output: edge set of an MST

```
F ← ∅;
while F does not span G do
    pick a cut (S, S̄) that respects F;
    let e be a minimum-weight edge crossing (S, S̄);
    F ← F ∪ {e};
return F;
```

Why it works

Correctness of the meta-algorithm. The invariant "F is promising" holds at the start ($\emptyset$ is contained in every MST) and is preserved by every iteration: the chosen cut respects F and e is a minimum crossing edge, so Theorem 6.1 keeps $F \cup \{e\}$ promising. The loop adds one edge per iteration without ever creating a cycle (a safe edge crosses a respecting cut, so it joins two different trees of F), so after $n-1$ iterations F is a spanning tree that is promising – i.e. contained in an MST – and a spanning tree contained in an MST *is* that MST. Every concrete algorithm below differs only in *which cut it picks* and *how it finds the cheapest crossing edge fast*.

The three classical specializations:

- **Kruskal**: consider edges globally cheapest-first; each accepted edge is safe for the cut separating the two components it joins.
- **Prim**: keep one growing tree S; repeatedly add the cheapest edge leaving S.
- **Boruvka**: in parallel, every current component picks its own cheapest outgoing edge; add them all at once.

6.4 Kruskal's algorithm

History

The minimum spanning tree problem has a long history. Otakar Borůvka formulated the problem and discovered the first algorithm in 1926 to design an economical electrical grid for rural Moravia. Vojtěch Jarník discovered the growing-tree method in 1930, later rediscovered independently by Robert Prim in 1957 and Edsger Dijkstra in 1959. Joseph Kruskal proposed the global sorting method in 1956, which became near-linear when paired with the disjoint-set forest (Galler and Fischer, 1964) analyzed by Tarjan in 1975. Later work improved the bounds: Karger, Klein, and Tarjan (1995) achieved randomized linear time $O(m)$, while whether a deterministic comparison-based algorithm can solve the problem in strictly linear time remains one of the deepest open questions in graph algorithms.

Intuition

Sort the edges from cheapest to dearest and walk down the list. Take each edge unless it would close a cycle, i.e. unless its two endpoints are already connected by edges you took earlier. You are greedily buying the cheapest edge that buys you something – that merges two as-yet-separate pieces of the network.

The cycle test "are these two endpoints already connected?" is exactly a *disjoint-set* (union–find) query: maintain the connected components of the chosen-so-far forest, with `find`(v) returning a canonical representative and `union` merging two components.

Algorithm 22: Kruskal-MST

Input: connected weighted graph $G = (V, E, w)$
Output: MST edge set T

```
T ← ∅;
foreach v ∈ V do MAKE-SET(v);
sort E so that w(e_1) ≤ w(e_2) ≤ ··· ≤ w(e_m);
foreach edge e = (u, v) in nondecreasing weight order do
    if FIND(u) ≠ FIND(v) then
        T ← T ∪ {e};
        UNION(u, v);
        if |T| = n − 1 then break;
return T;
```

6.4.1 Union–find in one paragraph

Represent each set as a rooted tree of parent pointers; the root is the representative. `find` walks to the root, and during *path compression* repoints every node it passes directly at the root. `union` links the two roots, attaching the shorter tree under the taller (*union by rank*). With both heuristics, any sequence of m operations on n elements costs $O(m\,\alpha(n))$, where α is the inverse Ackermann function – below 5 for every n writable in this universe. We treat union–find as a black box costing $O(\alpha(n))$ amortized per operation here; the data structure and its analysis get their own chapter.

Trace it yourself

Trace Kruskal on G_0 (Figure 6.1). Sorted edges: $AD(1)$, $BD(2)$, $DE(3)$, $AB(4)$, $BE(5)$, $BC(6)$, $CE(7)$, $CF(8)$, $EF(9)$.

edge	weight	action	components after
AD	1	accept	$\{A, D\}\,\{B\}\,\{C\}\,\{E\}\,\{F\}$
BD	2	accept	$\{A, B, D\}\,\{C\}\,\{E\}\,\{F\}$
DE	3	accept	$\{A, B, D, E\}\,\{C\}\,\{F\}$
AB	4	*reject* (A, B joined)	unchanged
BE	5	*reject* (B, E joined)	unchanged
BC	6	accept	$\{A, B, C, D, E\}\,\{F\}$
CE	7	*reject* (C, E joined)	unchanged
CF	8	accept	$\{A, B, C, D, E, F\}$ – spans, stop

Result $\{AD, BD, DE, BC, CF\}$, weight $1 + 2 + 3 + 6 + 8 = 20$. Note how every *rejected* edge is the heaviest on the cycle it would have closed: $AB(4)$ on $A-D-B-A$, $CE(7)$ on $C-B-D-E-C$. That is the cycle property in action.

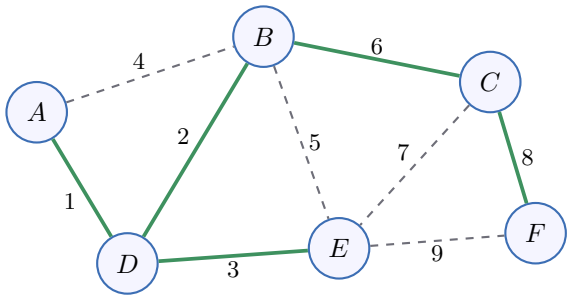


Figure 6.3: The MST of G_0 in green (AD, BD, DE, BC, CF), total weight 20. Dashed edges are excluded; each is the maximum on some cycle.

Why it works

Why Kruskal is correct. Each accepted edge $e = (u, v)$ joins two distinct trees of the current forest F. Take S to be the vertex set of u's tree. The cut $(S, \overline{S})$ respects F (no forest edge crosses it – S is a whole component). Every still-unprocessed edge is at least as heavy as e (we go in sorted order), and every *already*-processed crossing edge was rejected, which only happens inside a single component, so no processed edge crosses $(S, \overline{S})$ at all. Hence e is a minimum-weight crossing edge, and the cut property makes it safe. Kruskal is Generic-MST with the cut chosen retroactively around each accepted edge.

Cost summary

Sorting m edges: $O(m \log m) = O(m \log n)$ (since $m \le n^2$, $\log m \le 2 \log n$). The union–find work: $2m$ finds and $n - 1$ unions, $O(m\, \alpha(n))$. Total $O(m \log n)$, dominated by the sort. If the weights are small integers and can be radix-sorted in $O(m)$, Kruskal runs in $O(m\, \alpha(n))$ – essentially linear.

6.5 Prim's algorithm

Where Kruskal's algorithm grows a forest across the entire graph and merges components in weight order, Prim's algorithm (Jarník, 1930; Prim, 1957; Dijkstra, 1959) grows one connected tree outward from a single starting vertex. Its loop mirrors Dijkstra's shortest-path algorithm, except each outside vertex is prioritized by the weight of its single lightest edge into the tree rather than its accumulated path distance from the root.

Intuition

Grow a single tree like a crystal. Start from any vertex. At each step look at all edges with exactly one endpoint already in the tree – the *frontier* – and pull in the cheapest one, absorbing its outside endpoint. Repeat until everyone is inside. Prim always picks the cut $(S, \overline{S})$ where S is the current tree, so safety is immediate from the cut property.

The efficiency question is: how do we repeatedly find the cheapest frontier edge? Keep, for every vertex $v \notin S$, the cheapest known edge connecting it to S, call it key$[v]$, in a priority queue. When a vertex is absorbed, relax its neighbours' keys, exactly as Dijkstra relaxes distances.

Algorithm 23: Prim-MST (binary-heap version)

Input: connected weighted graph $G = (V, E, w)$, start vertex r
Output: MST given by parent pointers π

```
1 foreach v ∈ V do key[v] ← +∞; π[v] ← NIL; v unmarked;
2 key[r] ← 0;
3 priority queue Q ← V keyed by key;
4 while Q ≠ ∅ do
5     u ← Extract-Min(Q); mark u;
6     foreach edge (u, v) with v ∈ Q unmarked do
7         if w(u, v) < key[v] then
8             key[v] ← w(u, v); π[v] ← u; Decrease-Key(Q, v);
9 return {(v, π[v]) : v ≠ r};
```

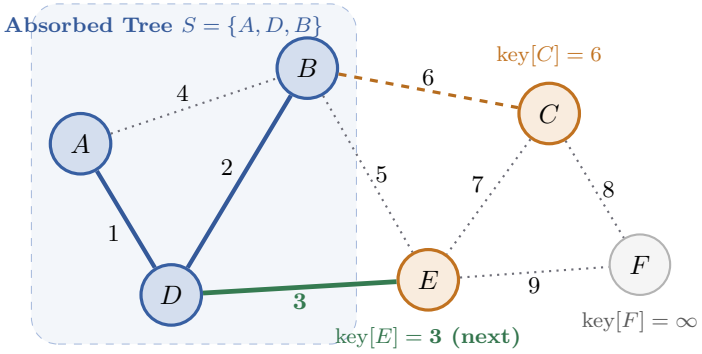
Trace it yourself

Trace Prim on G_0 from $r = A$. Columns are key[·]; • marks the extracted (absorbed) vertex; − means already in the tree; the boldface entry in each row is the minimum key, i.e. the vertex absorbed in the next row.

step (absorb)	A	B	C	D	E	F
init	0	∞	∞	∞	∞	∞
$A^\bullet$	–	4	∞	**1**	∞	∞
$D^\bullet$ (via A)	–	**2**	∞	–	3	∞
$B^\bullet$ (via D)	–	–	6	–	**3**	∞
$E^\bullet$ (via D)	–	–	**6**	–	–	9
$C^\bullet$ (via B)	–	–	–	–	–	**8**
$F^\bullet$ (via C)	–	–	–	–	–	–

The parent edges chosen, in order: $AD(1), DB(2), DE(3), BC(6), CF(8)$ – the same tree, weight 20. Watch the key of C: it first becomes 6 when B is absorbed, and E later offers C a connection of weight 7, which is rejected because $7 > 6$. Watch F: it is offered 9 via E, then improved to 8 via C.

The snapshot below captures Prim's state right after absorbing $S = \{A, D, B\}$: the cut $(S, \overline{S})$ separates the growing tree from the outside vertices, whose priority-queue keys record each vertex's lightest edge into S.



Why it works

Why Prim is correct. Invariant: the marked set S together with the parent edges among them forms a tree that is promising, and for each unmarked v, key$[v]$ is the weight of the lightest edge from v into S (or ∞). When Extract-Min returns u, key$[u]$ is the minimum over *all* unmarked vertices, hence the edge $(\pi[u], u)$ is a minimum-weight edge crossing the cut $(S, \overline{S})$. The cut respects the current tree (S is exactly the marked component). By Theorem 6.1, adding it keeps the tree promising. After n extractions $S = V$ and the promising tree spans, so it is an MST.

Cost summary

With a binary heap: n Extract-Min at $O(\log n)$ each and up to m Decrease-Key at $O(\log n)$ each, total $O((n + m) \log n) = O(m \log n)$ for a connected graph. With a Fibonacci heap, Decrease-Key is $O(1)$ amortized and Extract-Min is $O(\log n)$, giving $O(m+n \log n)$ – asymptotically better for dense graphs. With a simple array (no heap), each Extract-Min is a $O(n)$ scan and Decrease-Key is $O(1)$, giving $O(n^2)$, which is the right choice when $m = \Theta(n^2)$.

Common trap

Prim's priority is the *edge weight* $w(u, v)$, not a running sum key$[u] + w(u, v)$. If you copy Dijkstra and accumulate, you compute a shortest-path tree, not an MST – a different and usually heavier tree. The two algorithms look almost identical in code but compute different trees; confusing them is a classic interview trap.

6.6 Boruvka's algorithm

Borůvka's 1926 algorithm (rediscovered by Sollin in the 1960s) grows components in parallel rounds and serves as the vertex-shrinking step in modern parallel and randomized linear-time MST solvers.

Intuition

In each round, every current component looks at all edges leaving it and selects its single cheapest one. Add all of those selected edges simultaneously. Each round at least halves the number of components, because every component merges with at least one other. After

$O(\log n)$ rounds, one component remains: the MST.

Algorithm 24: Boruvka-MST

Input: connected weighted graph $G = (V, E, w)$ with distinct edge weights
Output: MST edge set T

```
1  T ← ∅; initialize union–find with each v its own component;
2  while number of components > 1 do
3      foreach component C do
4          cheap[C] ← NIL
5      foreach edge (u, v) ∈ E with FIND(u) ≠FIND(v) do
6          foreach C ∈ {FIND(u), FIND(v)} do if cheap[C] = NIL or w(u, v) < w(cheap[C])
             then cheap[C] ← (u, v);
7          ;
8      foreach component C with cheap[C] ≠ NIL do
9          let e = cheap[C] = (u, v);
10         if FIND(u) ≠FIND(v) then
11             T ← T ∪ {e}; UNION(u, v);
12 return T;
```

Why it works

Why Boruvka is correct. For a component C, let e be its cheapest outgoing edge. Take the cut $(C, \overline{C})$. It respects the forest built so far (the forest edges live inside components). e is by definition a minimum-weight crossing edge of this cut, so it is safe by Theorem 6.1. All components apply this simultaneously; with *distinct* weights no harmful conflict arises – if C and C' both pick edge e between them, it is added once (the `find` re-check in the merge phase prevents a duplicate or a cycle). Each chosen edge is independently safe, and the union of safe edges is acyclic because each one strictly merges two components.

Common trap

Why "distinct weights" matters here. Suppose a 4-cycle has all edges of weight 1. Each component's cheapest outgoing edge is some weight-1 edge, and if every component independently picks the edge to its clockwise neighbour, you add all four and create a cycle. The standard fix is a deterministic tie-break: compare $(w(e), \text{edge-id})$ lexicographically, which makes weights effectively distinct and kills the symmetry. With that tie-break Boruvka is correct on arbitrary inputs.

Trace it yourself

Trace Boruvka on G_0.
Round 1. Each singleton picks its cheapest incident edge: $A \to AD(1)$, $B \to BD(2)$, $C \to BC(6)$, $D \to AD(1)$, $E \to DE(3)$, $F \to CF(8)$. The distinct chosen edges are $\{AD, BD, BC, DE, CF\}$ (Figure 6.4). Merging: A, D join (via AD); B joins them (via BD); E joins (via DE); C joins (via BC); F joins (via CF). That already yields one component of

all six vertices.
So G_0 finishes in a single Boruvka round, producing $\{AD, BD, BC, DE, CF\}$, weight 20 – the same MST. (On graphs that do not collapse so fast, you would shrink each component to a supernode and repeat.)

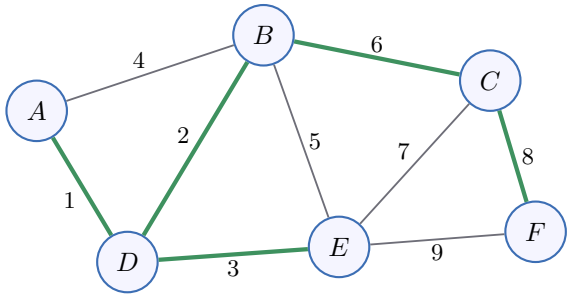


Figure 6.4: Boruvka, round 1 on G_0. Each vertex's cheapest incident edge is highlighted: A and D both choose $AD(1)$, B chooses $BD(2)$, E chooses $DE(3)$, C chooses $BC(6)$, F chooses $CF(8)$. The five distinct chosen edges already merge all six vertices into one component, so the round finishes the MST.

Cost summary

Each round scans all m edges to find every component's cheapest outgoing edge and does $O(m)$ union–find work: $O(m\,\alpha(n))$ per round, or $O(m)$ with a relabel-to-representatives bookkeeping. The component count at least halves each round, so there are $\leq \lceil \log_2 n \rceil$ rounds. Total: $O(m \log n)$. Boruvka's advantage is contraction: after each round you can rebuild the graph on the surviving supernodes (removing self-loops and keeping the lightest parallel edge), and *that* shrinking is what the linear-time algorithms exploit.

Why this matters

All three algorithms apply the same theorem. Kruskal picks cuts globally cheapest-first; Prim fixes one side of the cut and grows it; Boruvka uses every component's cut at once. They are three readings of the cut property, and each is optimal in a different regime: Kruskal when edges are presorted or radix-sortable, Prim with a Fibonacci heap on dense graphs ($O(m + n \log n)$), Boruvka when you need parallelism or contraction. The randomized linear-time algorithm in Section 6.10 is, structurally, Boruvka steps glued to a sampling trick.

6.7 Reverse-delete: the cycle property as an algorithm

Kruskal, Prim, and Boruvka all build *up* from the empty forest using the cut property. The cycle property gives a fourth algorithm that tears *down* from the full graph: sort the edges heaviest-first and delete each one whose removal does not disconnect the graph.

Intuition

Start with every edge present. Walk the edges from most expensive to cheapest. Throw an edge away whenever you can do so without breaking connectivity – if its endpoints are still joined by some other route, the edge was the heaviest rung of a cycle and the cycle property says no MST needs it. Stop when you have removed everything removable; what survives is the MST.

Algorithm 25: Reverse-Delete-MST

Input: connected weighted graph $G = (V, E, w)$
Output: MST edge set T

```
T ← E;
sort E so that w(e_1) ≥ w(e_2) ≥ ··· ≥ w(e_m);
foreach edge e in nonincreasing weight order do
    if (V, T \ {e}) is still connected then
        T ← T \ {e};
return T;
```

Why it works

Why reverse-delete is correct. When we consider e, it is deleted exactly when $T \setminus \{e\}$ is still connected, i.e. when e's endpoints are joined by a path of remaining edges. That path plus e is a cycle, and because we go heaviest-first, every edge still present is no heavier than e – so e is a maximum-weight edge on that cycle. The cycle property (Theorem 6.2) says such an edge is in no MST, so deleting it is safe. Conversely an edge we *keep* is a bridge of the surviving graph at the moment we test it; a bridge lies on no cycle, so removing it would disconnect the graph and it must stay. At the end T is connected, acyclic (any remaining cycle would have a heaviest edge we would have deleted), and spans – an MST.

Trace it yourself

Trace reverse-delete on G_0. Edges heaviest-first: $EF(9)$, $CF(8)$, $CE(7)$, $BC(6)$, $BE(5)$, $AB(4)$, $DE(3)$, $BD(2)$, $AD(1)$.

edge	weight	action
EF	9	delete (F still reached via CF)
CF	8	*keep* (deleting isolates F – bridge)
CE	7	delete (C, E still joined via B, D)
BC	6	*keep* (only link left to C)
BE	5	delete (B, E still joined via D)
AB	4	delete (A, B still joined via D)
DE	3	*keep* (only link left to E)
BD	2	*keep* (bridge)
AD	1	*keep* (bridge)

Survivors $\{CF, BC, DE, BD, AD\}$, weight 20 – the same MST again, this time obtained by deleting edges. The kept edges are precisely the bridges; the deleted edges are exactly the cycle-

maxima Kruskal rejected. Naively the connectivity test is a DFS per edge, giving $O(m(m+n))$; with a dynamic-connectivity structure it drops to $O(m \log^2 n)$, which is why reverse-delete is valued mainly for its proof and is rarely run as written.

6.8 Uniqueness of the MST

Theorem 6.3 (Distinct weights ⇒ unique MST)**.** *If all edge weights of a connected graph G are pairwise distinct, then G has exactly one minimum spanning tree.*

Proof. Suppose $T_1 \neq T_2$ are both MSTs. Let e be the minimum-weight edge in their symmetric difference $T_1 \triangle T_2$ (it is nonzero, and weights are distinct, so e is unique). Without loss of generality $e \in T_1 \setminus T_2$. Adding e to T_2 creates a unique cycle γ. Not all edges of γ can be in T_1 (it is a tree, no cycle), so γ contains some edge $f \in T_2 \setminus T_1 \subseteq T_1 \triangle T_2$. By minimality of e in the symmetric difference, $w(e) < w(f)$. Then $T_2 - f + e$ is a spanning tree of weight $w(T_2) - w(f) + w(e) < w(T_2)$, contradicting that T_2 is an MST. □

The converse fails: a graph with ties can still have a unique MST (if the tied edges happen never to compete). The precise criterion is local.

Proposition 6.1 (When ties create alternatives)**.** *G has more than one MST iff there is an MST T and a non-tree edge $f \notin T$ such that f has the* same *weight as some edge g on the tree path between f's endpoints in T. Swapping g for f produces a different MST of equal weight.*

Why it works

The forward direction is the exchange argument: any two MSTs differ by a sequence of equal-weight swaps. If $T \neq T'$ are MSTs, pick $f \in T' \setminus T$; the T-path between its ends contains an edge $g \notin T'$ on the cycle f closes, and equal total weight forces $w(g) = w(f)$ (if $w(g) < w(f)$ then T' could be improved; if $w(g) > w(f)$ then T could be). The reverse direction is immediate: the swap yields a distinct equal-weight tree.

Remark 6.1. A clean way to detect uniqueness algorithmically: build any MST, then for each non-tree edge f check whether the heaviest edge on the tree-path between its endpoints equals $w(f)$. If some such pair ties, the MST is not unique. The path-max queries are exactly the verification primitive of Section 6.9, so uniqueness testing is no harder than MST verification: $O(m + n)$.

Trace it yourself

A worked tie. Take a triangle A, B, C with $w(AB) = w(BC) = 1$ and $w(AC) = 2$, plus a pendant $w(CD) = 3$. Two MSTs exist: $\{AB, BC, CD\}$ and $\{AB, AC, CD\}$? No – the second uses $AC(2)$, heavier than $BC(1)$, so it is not minimum. The genuine tie needs equal competitors: set $w(AB) = w(BC) = w(AC) = 1$. Now any two of the three triangle edges form a spanning tree of the triangle, all of weight 2, so with the pendant CD there are three MSTs: $\{AB, BC, CD\}$, $\{AB, AC, CD\}$, $\{BC, AC, CD\}$. Proposition 6.1 flags this immediately: in $T = \{AB, BC, CD\}$ the non-tree edge AC has weight 1, equal to the heaviest edge (AB or BC, both 1) on the tree path $A{-}B{-}C$, so a swap exists. Distinct weights would have broken

exactly this symmetry.

Connection

The MST lives inside a *matroid*: the spanning forests of a graph are the independent sets of the *graphic matroid*, and the greedy algorithm is optimal for *every* matroid (the matroid greedy theorem). The cut and cycle properties are the graphic specialization of a matroid's cocircuit and circuit elimination axioms. This is why MST greed works and, say, greedy on shortest paths or on the travelling salesman does not: those feasibility structures are not matroids. The companion linear-algebra book's treatment of the cycle space and cut space of a graph – the kernel and row space of the incidence matrix – is the same duality you met here as cycle property versus cut property.

6.9 Verifying a spanning tree in linear time

A question that sounds easier than building an MST leads to the hardest result in the chapter: *given* a spanning tree T, is it minimum? Recomputing the MST and comparing is too slow, because the randomized algorithm later in the chapter needs verification to run in *linear* time.

Focus

MST verification, the test. A spanning tree T is an MST iff for every non-tree edge $f = (u, v)$, $w(f) \geq \max\{w(g) : g \text{ on the } T\text{-path from } u \text{ to } v\}$. Equivalently: no non-tree edge is lighter than the heaviest tree edge it would replace.

Why it works

This is precisely the cycle property read both ways. If some f is lighter than the heaviest edge g on its tree path, the cycle $f +$ path has its maximum at g (a tree edge), so by Theorem 6.2 g is in no MST, hence T is not minimum – and indeed $T - g + f$ is lighter. Conversely, if the test passes for every f, then T satisfies the cycle-optimality condition for every fundamental cycle, which is known to be equivalent to minimality (each non-tree edge is a maximum on its own fundamental cycle).

So verification reduces to: for each of the m non-tree edges, compute the maximum-weight edge on a tree path – the *tree-path-maximum* problem. Done naively (walk each path) this is $O(nm)$. In fact all m queries together take $O(m + n)$ time: Komlós (1985) proved the comparison bound, and Dixon–Rauch–Tarjan (1992) gave a linear-time implementation.

Intuition

Komlós's insight: you do not answer the m queries one at a time. You push all queries down into the tree at once and answer them in a single sweep, reusing comparisons across queries that share path-prefixes. The number of *comparisons* needed to answer m path-max queries on an n-node tree is $O(n+m)$ – provably, by an amortization that charges each comparison to a query or a tree edge. Turning that comparison bound into a linear-*time* algorithm needs a Boruvka-style reduction to a balanced tree plus careful bit-packing of the answers; we sketch

the structure rather than the full pointer-arithmetic.

Algorithm 26: MST-Verify (Komlós, structure)

Input: graph G, spanning tree T
Output: TRUE iff T is an MST

```
contract T via Boruvka steps into a balanced tree B of depth O(log n) where every internal
  node has ≥ 2 children;
foreach non-tree edge f = (u, v) do
    route the query to the node lca_B(u, v);
process all queries in one bottom-up sweep of B, maintaining for each query the running
  path-maximum, packing the per-edge comparison results into machine words;
foreach non-tree edge f do if w(f) < pathmax(f) then return FALSE;
;
return TRUE;
```

Why it works

Why a *balanced* contracted tree? On a path-shaped tree of depth n, a single query could touch n edges, and m queries naively cost nm. Boruvka contraction replaces T by a tree of height $O(\log n)$ where each node has at least two children and the path-maxima of the original tree are recoverable from path-maxima in the contracted one. On the balanced tree each query path has length $O(\log n)$, and Komlós's amortization brings the *total* comparison count to $O(n+m)$: each tree edge sees $O(1)$ amortized comparisons because the branching factor ≥ 2 geometrically bounds how many queries can "stack up" on any edge. Pack the comparison outcomes along each leaf-to-root path into $O(\log n)$-bit words so that a query reads its answer in $O(1)$ word operations.

Cost summary

MST verification: $O(m+n)$ time on a unit-cost RAM. Number of comparisons (Komlós bound): $O(m+n)$, indeed at most $\approx m \log_2(2+n/m)$ in the refined count. The randomized MST algorithm uses this linear verifier to cheaply discard the edges that provably cannot be in the MST.

6.10 Randomized linear-time MST

Whether MST can be solved in deterministic linear time on a comparison RAM remains open. Karger, Klein and Tarjan (1995) gave a randomized algorithm running in $O(m)$ expected time. Chazelle (2000) gave a deterministic $O(m\,\alpha(m,n))$ algorithm, and Pettie and Ramachandran (2002) found an algorithm provably optimal up to constants. The randomized algorithm is the simplest of these to present in full.

The algorithm rests on one lemma. Sample a random subgraph, compute its minimum spanning *forest*, and use that forest to certify that most edges are useless.

Definition 6.3 (F-heavy and F-light)**.** Let F be a forest in G. An edge $f=(u,v)\in E$ is *F-heavy*

if u, v lie in the same tree of F and $w(f)$ exceeds the maximum weight on the F-path between them; otherwise f is *F-light*. By the cycle property, an F-heavy edge is in no MST of G.

Detecting which edges are F-heavy is exactly the verification primitive: $O(m+n)$ for all edges at once.

Lemma 6.1 (Sampling / DJP–KKT lemma)**.** *Let H be the subgraph obtained by including each edge of G independently with probability p, and let F be the minimum spanning forest of H. Then the expected number of F-light edges of G is at most n/p, where $n = |V|$.*

Why it works

The proof considers edges in increasing weight order and runs Kruskal on G and on H simultaneously. An edge is F-light exactly when, at the moment Kruskal would consider it, it would be *accepted* into the forest of H – it connects two different F-components. Now flip the experiment: reveal each edge's coin only when Kruskal reaches it. An F-light edge that is accepted into F is one of at most $n-1$ such acceptances; an F-light edge that is *not* sampled still "would have been accepted," and each such edge is sampled with probability p independently, so the count of F-light edges is dominated by a sequence of coin flips with at most $n-1$ successes, whose expected length is $\leq (n-1)/p < n/p$. So light edges are rare, because the random forest already connects most of the graph.

Algorithm 27: KKT randomized MST (expected $O(m)$)

Input: connected weighted graph $G = (V, E, w)$
Output: MST edge set

if $|E|$ *is small (base case)* **then**
  return MST by Boruvka/Prim directly
run **3 Boruvka steps** on G, contracting; collect the chosen edges in T_0;
`// now` $\leq n/8$ `vertices remain; call the contracted graph` G'
let H be a random subgraph of G', each edge kept independently w.p. 1/2;
$F \leftarrow \mathrm{MSF}(H)$; `// recursive call`
discard every F-heavy edge of G' ; `// linear verification`
$T_1 \leftarrow$ MSF of the surviving subgraph ; `// recursive call`
return $T_0 \cup T_1$;

Why it works

Why it is correct. The three Boruvka steps add only safe edges (cut property), so T_0 is a subset of an MST, and contracting along them preserves the MST of the rest. Discarding F-heavy edges removes only edges that, by the cycle property, are in no MST – so the surviving graph still contains the MST of G'. The two recursive calls compute spanning forests, and the union $T_0 \cup T_1$ is therefore exactly an MST of G.

Why it is linear in expectation. Let $T(m, n)$ bound the expected time on m edges, n vertices. The Boruvka steps cost $O(m)$ and leave $\leq n/8$ vertices. The first recursion runs on H, which has $\leq m/2$ edges in expectation on $\leq n/8$ vertices. By the sampling lemma with $p = 1/2$, the expected number of F-light survivors is $\leq 2 \cdot (n/8) = n/4$, so the second recursion runs on $\leq n/4$ edges (only F-light edges survive). Verification and discarding are $O(m)$. Writing $E[T(m,n)] \leq c\,m + E[T(m/2,\, n/8)] + E[T(n/4,\, n/8)]$ and solving (the two

subproblem edge-counts sum to $\leq m/2 + n/4 \leq 3m/4$ since $n \leq m$ in a connected graph) gives $E[T(m,n)] = O(m)$ by a standard "shrinking total size" induction.

Cost summary

KKT: $O(m)$ *expected* time, and in fact $O(m)$ with probability $1-e^{-\Omega(m)}$. The three ingredients: Boruvka steps to shrink the vertex set geometrically, random sampling to estimate the MST (sampling lemma), and linear-time verification to prune F-heavy edges using that sample. Each ingredient reuses a result from earlier in this chapter.

Why this matters

The pieces fit together as follows: Boruvka steps contract vertices, random sampling and verification prune edges, and recursion solves the smaller subproblems. The cut property licenses the contraction, the cycle property licenses the deletion. Every main result of this chapter is used in the final algorithm, which is a good reason to build the theory on the cut and cycle properties.

6.11 Applications

Application

Network design. The original motivation: lay the cheapest cabling, piping, or road network that connects every site. MST gives the cost-minimal connected backbone; redundancy is then added on top deliberately.

Application

Clustering (single-linkage). Build the MST, then delete its $k-1$ heaviest edges; the resulting k components are exactly the single-linkage clusters. Because the MST orders merges by increasing dissimilarity, this is hierarchical agglomerative clustering done in $O(m \log n)$ instead of the naive $O(n^3)$. The deleted edges are the largest "gaps" between clusters.

Application

Approximation algorithms. The MST is a 2-approximation skeleton for metric TSP: double the MST, shortcut the resulting Eulerian walk, and you get a tour of length $< 2 \cdot \mathrm{OPT}$. Christofides refines this to 3/2 by adding a minimum matching on odd-degree vertices. The MST also lower-bounds the Steiner tree.

Application

Image segmentation and graph-based vision. Felzenszwalb–Huttenlocher segmentation is essentially Kruskal with a stopping rule, merging pixel regions in increasing edge-weight order. The MST encodes the "most natural" merges.

Connection

To the spectral and electrical material later in the book. The number of spanning trees of G is $\det(\mathbf{L}_{ii})$, any cofactor of the graph Laplacian – the Matrix-Tree Theorem. Random spanning trees (sampled by Wilson's algorithm or by electrical-network methods) connect MSTs to random walks and to the effective resistances you will meet in the Laplacian-solver chapters. The cut/cycle duality of this chapter is the discrete counterpart of decomposing edge space orthogonally into the cut space and cycle space of the incidence matrix.
While a minimum spanning tree minimizes the total weight of all edges across the entire network, it does not minimize the path distance between any individual pair of vertices. If you want the shortest route from a specific source to every other node, a global minimum tree will often lead you down circuitous detours. In Chapter 7, we shift from global network economy to point-to-point distance: we lay the foundations of Shortest Paths, explore edge relaxation, and develop Dijkstra's algorithm for nonnegative weights.

Exercises

Exercise 6.1 (Cut property, by hand)**.** On the graph G_0 of Figure 6.1, take the cut $S = \{A, D, E\}$. List every crossing edge and identify the minimum. Verify that this edge is in the MST of Figure 6.3, as the cut property predicts.

Exercise 6.2 (Reverse-delete)**.** Write pseudocode for the reverse-delete algorithm: sort edges in *decreasing* weight, and delete an edge whenever its removal keeps the graph connected. Prove it correct using the cycle property, and state its running time if connectivity is tested by a single DFS per candidate edge.

Exercise 6.3 (Prim vs. Dijkstra)**.** Construct a small weighted graph on which Prim's tree and Dijkstra's shortest-path tree (from the same root) differ, and exhibit a vertex whose two parent edges differ. Explain the discrepancy in one sentence using the priorities each algorithm minimizes.

Exercise 6.4 (Boruvka ties)**.** Give a 4-vertex graph with repeated weights on which naive Boruvka (no tie-break) can add a cycle. Then show the lexicographic (w, id) tie-break prevents it. How many Boruvka rounds does K_n with all-equal weights take under that tie-break?

Exercise 6.5 (Second-best spanning tree)**.** Design an $O(m \log n)$ algorithm to find the second-minimum spanning tree (the cheapest tree that is not the MST). Hint: it differs from the MST by exactly one swap; for each non-tree edge f, consider replacing the maximum tree edge on f's fundamental cycle, and use the path-max primitive.

Exercise 6.6 (Uniqueness test)**.** Prove that an MST is unique iff for every non-tree edge f, $w(f)$ is strictly greater than the maximum tree-path edge weight between f's endpoints. Turn this into an $O(m + n)$ test assuming an $O(m + n)$ path-max oracle.

Exercise 6.7 (Bottleneck spanning tree)**.** A *minimum bottleneck spanning tree* minimizes its single *heaviest* edge. Prove every MST is also a minimum bottleneck spanning tree, but the converse can fail. Give a counterexample for the converse.

Exercise 6.8 (Edge weight changes)**.** Suppose one edge e's weight changes. (a) If $e \in T$ and its weight *decreases*, argue T stays an MST. (b) If $e \notin T$ and its weight decreases below the path-max on its fundamental cycle, describe the single swap that repairs the MST. Give an $O(n)$ update procedure for both cases.

Exercise 6.9 (Sampling lemma sharpness)**.** In Lemma 6.1, show the bound n/p is tight up to constants by exhibiting a graph and a p where the expected number of F-light edges is $\Omega(n/p)$. Conclude why $p = 1/2$ is a sensible choice in the KKT recursion.

Exercise 6.10 (Implement KKT)**.** Implement Karger–Klein–Tarjan using your union–find and a path-max verifier. Empirically measure the number of edges entering each recursion level on random graphs and check it shrinks geometrically, matching the $O(m)$ expected-time analysis. Where does the constant hide?

Exercise 6.11 (Matroid greedy)**.** State the graphic matroid (ground set E, independent sets = acyclic edge subsets) and prove the exchange axiom. Explain in two sentences why the greedy algorithm optimal for matroids specializes to Kruskal.

Exercise 6.12 (Boruvka contraction cost)**.** Show that after a Boruvka round you can rebuild the contracted multigraph – relabel endpoints to component representatives, drop self-loops, keep one cheapest copy of each parallel edge – in $O(m)$ time. Conclude the standalone Boruvka algorithm runs in $O(m \log n)$.

Chapter 7

Shortest Paths: Foundations

In Chapter 6, we found minimum spanning trees that connect an entire network with minimal total edge weight. But global economy does not guarantee personal convenience: the path between two towns in an MST can be frustratingly circuitous. If your goal is to travel from a specific starting point to every other vertex as quickly or cheaply as possible, you do not want an MST; you want a shortest-path tree. This chapter lays the foundations of shortest-path algorithms: the principle of edge relaxation, Dijkstra's algorithm for nonnegative weights across various heap data structures, and the concept of vertex potentials that bridges into advanced routing.

We work every algorithm by hand on a small graph. Tracing a five-vertex example until the distance labels stop changing teaches more than any amount of staring at pseudocode.

7.1 The Problem, Stated Carefully

Let $G = (V, E)$ be a directed graph with $n = |V|$ vertices and $m = |E|$ edges. A *weight function* $w \colon E \to \mathbb{R}$ assigns a real number to each edge. (Undirected graphs are the special case where every edge $\{u, v\}$ is replaced by the two directed edges (u, v) and (v, u) of equal weight – but beware, this trick is dangerous with negative weights, as we will see.) The *weight of a path* $p = \langle v_0, v_1, \dots, v_k \rangle$ is the sum of its edge weights,

$$w(p) \;=\; \sum_{i=1}^{k} w(v_{i-1}, v_i).$$

The *shortest-path distance* from s to t is

$$\operatorname{dist}(s, t) \;=\; \begin{cases} \min\{w(p) : p \text{ is a path from } s \text{ to } t\}, & \text{if such a path exists,} \\ +\infty, & \text{if } t \text{ is unreachable from } s. \end{cases}$$

Definition 7.1 (Single-source shortest paths, SSSP)**.** Given G, w, and a distinguished *source* $s \in V$, compute $\operatorname{dist}(s, v)$ for every $v \in V$, together with a *shortest-path tree* that records one optimal route to each reachable vertex.

Two subtleties deserve attention before we touch an algorithm.

Common trap

Negative cycles make "shortest" meaningless. Suppose a cycle C reachable from s and able to reach t has total weight $w(C) < 0$. Then for any path from s to t we can splice in another lap of C to lower its weight, without bound. The infimum is $-\infty$ and no shortest *path* exists. Whenever negative weights are in play, "what is the shortest path?" is only well posed once we have ruled out – or detected – a negative cycle on the relevant part of the graph. Bellman–Ford (Section 7.5) does exactly this detection.

Intuition

Even when distances are finite, why should a *shortest path itself* be simple (no repeated vertex)? Because if a path revisits a vertex, it contains a cycle. With no negative cycle anywhere, every cycle has weight ≥ 0, so deleting that cycle cannot increase the weight – the shortened walk is still a path and is no worse. Repeating the argument, some shortest route uses at most $n-1$ edges. That bound – at most $n-1$ edges – is the key fact behind Bellman–Ford.

7.1.1 The optimal-substructure property

Every shortest-path algorithm relies on the following fact.

Lemma 7.1 (Subpaths of shortest paths are shortest)**.** *Let $p = \langle v_0, \ldots, v_k \rangle$ be a shortest path from v_0 to v_k, and let $p_{ij} = \langle v_i, \ldots, v_j \rangle$ be the contiguous subpath from v_i to v_j, where $0 \leq i \leq j \leq k$. Then p_{ij} is a shortest path from v_i to v_j.*

Proof. Decompose p into $v_0 \xrightarrow{p_{0i}} v_i \xrightarrow{p_{ij}} v_j \xrightarrow{p_{jk}} v_k$, so that $w(p) = w(p_{0i}) + w(p_{ij}) + w(p_{jk})$. If some path p'_{ij} from v_i to v_j had $w(p'_{ij}) < w(p_{ij})$, then replacing p_{ij} by p'_{ij} yields a v_0-to-v_k walk of weight strictly less than $w(p)$. With no negative cycle this walk contains a path of no greater weight, contradicting the optimality of p. □

Optimal substructure is what lets us speak of a shortest-path *tree*: if the best route to v ends with the edge (u, v), then the prefix ending at u is itself a best route to u. So we can store, for each vertex, only its predecessor on some optimal path, and the union of these predecessor edges forms a tree rooted at s that simultaneously encodes a shortest path to every vertex.

7.2 Relaxation: The One Move Every Algorithm Makes

Dijkstra, Bellman–Ford, and the DAG algorithm all repeat the same basic step, *relaxation*, and differ only in the order in which they apply it to edges.

We maintain for each vertex v a tentative distance $d[v]$, an over-estimate of $\mathrm{dist}(s, v)$ that we steadily push down, and a predecessor pointer $\pi[v]$. Initialization sets $d[s] = 0$, $d[v] = +\infty$ for $v \neq s$, and all $\pi[v] = \textsc{nil}$.

Algorithm 28: RELAX(u, v, w) – the atomic step

```
if d[u] + w(u, v) < d[v] then
    d[v] ← d[u] + w(u, v)
    π[v] ← u
```

Intuition

Relaxing the edge (u, v) asks one question: "Is going to u and then taking this edge a cheaper way to reach v than anything I have found so far?" If yes, we lower $d[v]$ and remember that we arrived via u. The word "relax" is borrowed from physics: we let a constraint that was held too tight (an over-estimate) loosen toward its true value.

What makes relaxation safe is that no matter *how* you order the steps, a label never drops below the true shortest-path distance. Two invariants make this precise.

Lemma 7.2 (Upper-bound invariant)**.** *At all times, $d[v] \geq \mathrm{dist}(s, v)$ for every v, and once $d[v]$ reaches $\mathrm{dist}(s, v)$ it never changes again.*

Proof. Initially $d[s] = 0 = \mathrm{dist}(s, s)$ (assuming no negative cycle through s) and $d[v] = \infty \geq \mathrm{dist}(s, v)$ for the rest. Relaxation only ever lowers a label, and only to $d[u] + w(u, v)$. By induction $d[u] \geq \mathrm{dist}(s, u)$, so $d[u] + w(u, v) \geq \mathrm{dist}(s, u) + w(u, v) \geq \mathrm{dist}(s, v)$, the last step because $\mathrm{dist}(s, u) + w(u, v)$ is the weight of *some* route to v and the distance is the minimum over all routes. Thus the new value still bounds $\mathrm{dist}(s, v)$ from above. The value $\mathrm{dist}(s, v)$ is therefore a floor the label can never break through, so once reached it is frozen. □

Lemma 7.3 (Convergence / path-relaxation)**.** *Let $p = \langle s = v_0, v_1, \ldots, v_k \rangle$ be a shortest path from s to v_k. If at some point the edges of p are relaxed* in order *$(v_0, v_1), (v_1, v_2), \ldots, (v_{k-1}, v_k)$ – with arbitrary other relaxations interspersed – then afterward $d[v_k] = \mathrm{dist}(s, v_k)$, and it stays there.*

Proof. By induction on i we show $d[v_i] = \mathrm{dist}(s, v_i)$ holds right after (v_{i-1}, v_i) is relaxed. Base: $d[v_0] = d[s] = 0 = \mathrm{dist}(s, s)$ from the start. Step: just before (v_{i-1}, v_i) is relaxed we already have $d[v_{i-1}] = \mathrm{dist}(s, v_{i-1})$, established earlier in the order and frozen by Lemma 7.2. The relaxation tests $d[v_{i-1}] + w(v_{i-1}, v_i) = \mathrm{dist}(s, v_{i-1}) + w(v_{i-1}, v_i) = \mathrm{dist}(s, v_i)$, the last equality because p is a shortest path and its prefix to v_i is shortest (Lemma 7.1). So $d[v_i]$ is set to $\mathrm{dist}(s, v_i)$ (or already equals it), then frozen. □

Together, the two lemmas say that every shortest-path algorithm is a *schedule* of relaxations chosen to guarantee that, for each vertex, the edges of *some* shortest path get relaxed in order. The algorithms differ only in how cleverly – and how cheaply – they arrange that.

7.2.1 Optimality conditions

When have we finished? The following "no edge can be improved" condition is the certificate.

Theorem 7.1 (Shortest-path optimality conditions)**.** *Suppose G contains no negative cycle reachable from s. A labeling $d[\cdot]$ with $d[s] = 0$ and $d[v] < \infty$ exactly for the vertices reachable from s equals the true distances* $\mathrm{dist}(s, \cdot)$ if and only if *it is* feasible*:*

$$d[v] \;\leq\; d[u] + w(u, v) \qquad \text{for every edge } (u, v) \in E. \tag{$\star$}$$

Proof. ($\Rightarrow$) If $d[v] = \mathrm{dist}(s, v)$ then ($\star$) is just the triangle inequality for distances: there is a route to v via u of weight $\mathrm{dist}(s, u) + w(u, v)$, and the distance is no larger.

($\Leftarrow$) Suppose ($\star$) holds. Take any path $p = \langle s, v_1, \ldots, v_k = t\rangle$. Summing ($\star$) along its edges telescopes:

$$d[t] - d[s] = \sum_{i=1}^{k} \big(d[v_i] - d[v_{i-1}]\big) \le \sum_{i=1}^{k} w(v_{i-1}, v_i) = w(p).$$

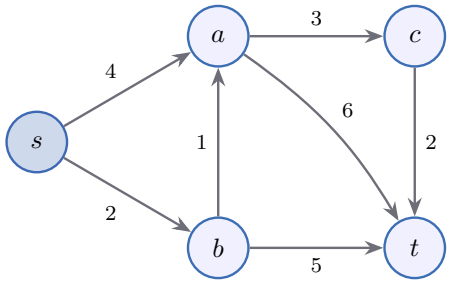
Since $d[s] = 0$ we get $d[t] \le w(p)$ for *every* path, hence $d[t] \le \mathrm{dist}(s, t)$. Combined with the upper bound $d[t] \ge \mathrm{dist}(s, t)$ (which holds for any labeling produced by relaxation from a correct start, Lemma 7.2), equality follows. □

Why it works

Condition ($\star$) is exactly "no edge is still relaxable." So an algorithm is allowed to stop the moment relaxation can make no further progress: feasibility *is* optimality. This is the same telescoping trick that powers linear-programming duality, and when we meet *potentials* (Section 7.7) the quantity $d[v]$ will play the role of a dual variable.

7.3 A Running Example

Here is the graph we will trace by hand throughout the chapter. Source s, five vertices, all weights nonnegative for now (we add a negative edge later).



Edges: $s \to a$ (4), $s \to b$ (2), $b \to a$ (1), $a \to c$ (3), $a \to t$ (6), $b \to t$ (5), $c \to t$ (2). The true distances, which we will rediscover three different ways, are

$$\mathrm{dist}(s, s) = 0, \quad \mathrm{dist}(s, b) = 2, \quad \mathrm{dist}(s, a) = 3, \quad \mathrm{dist}(s, c) = 6, \quad \mathrm{dist}(s, t) = 7.$$

The optimal route to a is $s \to b \to a$ (cost $2 + 1 = 3$, cheaper than the direct edge of weight 4). For t, compare the candidates $s \to b \to t = 2 + 5 = 7$ and $s \to b \to a \to c \to t = 2 + 1 + 3 + 2 = 8$ and $s \to b \to a \to t = 2 + 1 + 6 = 9$; the winner is the direct hop through b, so $\mathrm{dist}(s, t) = 7$. Keep these numbers handy – all three algorithms must agree on them.

7.4 Dijkstra's Algorithm

History

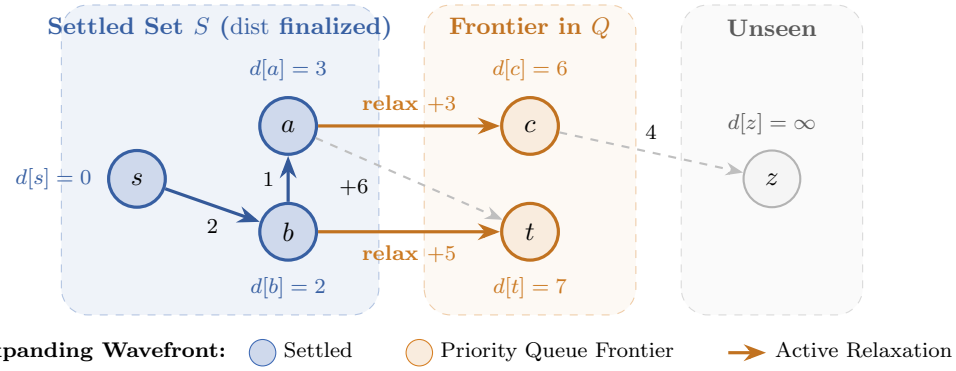
The foundations of shortest paths crystallized in the late 1950s. Edsger Dijkstra described his algorithm in 1959, having designed it in 1956 in about twenty minutes at a cafe table while shopping with his fiancée, as a demonstration for the ARMAC computer. Dijkstra observed that when edge weights are nonnegative, vertices can be finalized in greedy order without ever retracting a distance label. Around the same time, Lester Ford Jr. (1956), Richard Bellman (1958), and Edward F. Moore (1959) developed the iterative edge-relaxation algorithm for general weights, deriving it via Bellman's dynamic programming principle of optimality. Later refinements tackled the priority queue bottleneck: Fredman and Tarjan (1984) invented Fibonacci heaps to achieve $O(m + n \log n)$, while integer priority queues by Thorup and others pushed bounded-weight shortest paths toward linear time.

7.4.1 The greedy idea

Intuition

Imagine fire spreading from s at unit speed along edges whose "length" is their weight. The vertex that catches fire next is always the one with the smallest tentative distance among those not yet burning – because with nonnegative weights, no detour through a *farther* unburned vertex could ever reach it sooner. Dijkstra is exactly this: repeatedly burn (finalize) the nearest unfinished vertex, then relax its outgoing edges to update its neighbors.

The diagram below shows Dijkstra's expanding wavefront at the moment $S = \{s, b, a\}$ has been settled and outgoing edges from S relax the priority-queue frontier $Q = \{c, t\}$.



Maintain a set S of *settled* vertices whose distance is final, and a priority queue Q of the rest keyed by $d[\cdot]$. Each round, extract the minimum-key vertex u from Q, move it to S, and relax every edge leaving u.

Algorithm 29: DIJKSTRA(G, w, s) – nonnegative weights

Input: Directed graph $G = (V, E)$, weights $w \geq 0$, source s.
Output: $d[v] = \text{dist}(s, v)$ and predecessors $\pi[\cdot]$.

```
1  foreach v ∈ V do
2      d[v] ← ∞;   π[v] ← NIL
3  d[s] ← 0
4  S ← ∅;   build min-priority-queue Q over V keyed by d
5  while Q ≠ ∅ do
6      u ← EXTRACT-MIN(Q)
7      S ← S ∪ {u}
8      foreach edge (u, v) ∈ E do
9          if d[u] + w(u, v) < d[v] then
10             d[v] ← d[u] + w(u, v);   π[v] ← u
11             DECREASE-KEY(Q, v, d[v])
```

7.4.2 Correctness

Why it works

The whole proof rests on a single claim: *when a vertex u is extracted, $d[u]$ already equals* $\text{dist}(s, u)$. If that holds, then relaxing u's edges relaxes, in order, the last edge of every shortest path that ends by passing through u, and Lemma 7.3 finishes the argument. Proving the claim is where nonnegativity is needed.

Theorem 7.2 (Dijkstra is correct)**.** *If $w(e) \geq 0$ for all e, then on termination $d[v] = \text{dist}(s, v)$ for every $v \in V$.*

Proof. We prove the invariant: *each vertex u has $d[u] = \text{dist}(s, u)$ at the instant it is extracted from Q.* By Lemma 7.2, $d[u] \geq \text{dist}(s, u)$ always, so we need only rule out $d[u] > \text{dist}(s, u)$ at extraction time.

Suppose, for contradiction, that u is the *first* vertex extracted with $d[u] > \text{dist}(s, u)$. Then $u \neq s$ and u is reachable, so a shortest path $p\colon s \rightsquigarrow u$ exists with $w(p) = \text{dist}(s, u) < \infty$. Walk along p from s; since $s \in S$ already (it was extracted first, correctly) and $u \notin S$ at this moment, p has a first edge (x, y) that leaves S, i.e. $x \in S$ and $y \notin S$ (possibly $y = u$).

Because x was extracted before u and was *not* a counterexample (we took u to be the first), we had $d[x] = \text{dist}(s, x)$ when x was settled, and the edge (x, y) was relaxed at that time. Hence

$$d[y] \;\leq\; d[x] + w(x, y) \;=\; \text{dist}(s, x) + w(x, y) \;=\; \text{dist}(s, y),$$

the final equality because (x, y) lies on a shortest path to u and therefore its prefix to y is shortest (Lemma 7.1). Combined with the upper bound, $d[y] = \text{dist}(s, y)$.

Now use nonnegativity: y lies on p at or before u, and the segment of p from y to u has weight ≥ 0, so $\text{dist}(s, y) \leq \text{dist}(s, u)$. Putting the pieces together,

$$d[y] = \text{dist}(s, y) \leq \text{dist}(s, u) < d[u].$$

But u was chosen as the minimum-key vertex over Q, and $y \in Q$ at extraction time, so $d[u] \le d[y]$ – a contradiction. Hence no such first counterexample exists; every vertex is extracted with its correct distance. After extraction its edges are relaxed, so every shortest path's edges are eventually relaxed in order, and Lemma 7.3 gives $d[v] = \text{dist}(s, v)$ for all v. □

Common trap

Dijkstra and negative edges do not mix. The line "the segment from y to u has weight ≥ 0, so $\text{dist}(s, y) \le \text{dist}(s, u)$" is the *only* place nonnegativity is used – and it is fatal without it. Concretely, take $s \to a$ of weight 1, $s \to b$ of weight 2, and $b \to a$ of weight -2. Dijkstra extracts a first with $d[a] = 1$, settles it, and never revisits; but $\text{dist}(s, a) = 2 - 2 = 0$. The greedy commitment is wrong. Do not "fix" this by re-inserting settled vertices and hoping – that degenerates to exponential time on adversarial graphs. Use Bellman–Ford, or reweight with potentials.

7.4.3 Tracing Dijkstra by hand

Trace it yourself

Run Dijkstra on the graph of Section 7.3. Columns are the labels $d[\cdot]$; "–" means ∞. The settled vertex of each round is starred.

Round	Extract	$d[s]$	$d[a]$	$d[b]$	$d[c]$	$d[t]$
init	–	0	–	–	–	–
1	$s^\star$	0	4	2	–	–
2	$b^\star$	0	3	2	–	7
3	$a^\star$	0	3	2	6	7
4	$c^\star$	0	3	2	6	7
5	$t^\star$	0	3	2	6	7

Walk it: Round 1 settles s, relaxes $s \to a$ (to 4) and $s \to b$ (to 2). Round 2 the queue minimum is $b\,(2)$, not $a\,(4)$ – settle b; relax $b \to a$ giving $2 + 1 = 3 < 4$ (down to 3, predecessor now b) and $b \to t$ giving $2 + 5 = 7$. Round 3 settle $a\,(3)$; relax $a \to c$ to $3 + 3 = 6$ and $a \to t$ to $3 + 6 = 9$, which is *not* less than the current 7, so $d[t]$ stays 7. Round 4 settle $c\,(6)$; relax $c \to t$ to $6 + 2 = 8 \not< 7$, no change. Round 5 settle $t\,(7)$. Final tree edges: $\pi[a] = b$, $\pi[b] = s$, $\pi[c] = a$, $\pi[t] = b$. Notice a's label dropped from 4 to 3 *before* a was settled – that is the decrease-key in action.

The predecessor pointers collect into the shortest-path tree below: each vertex is labeled with its final distance, and the bold arrows are the π edges $s \to b \to a$, $a \to c$, and $b \to t$. The original edges $s \to a$, $a \to t$, and $c \to t$ are dashed – they were relaxed but lost, never becoming part of any optimal route.

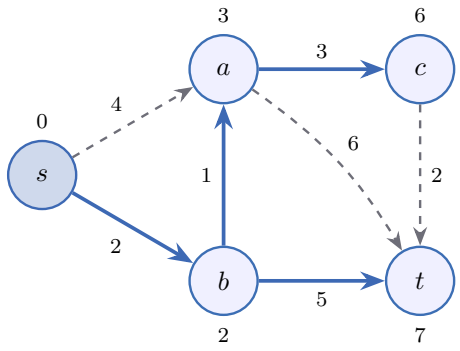


Figure 7.1: Shortest-path tree from Dijkstra on the running graph. Bold blue arrows are the final π edges; dashed gray arrows were relaxed but never won. Vertex labels are final distances.

7.4.4 Heap variants and complexity

Dijkstra performs n EXTRACT-MIN operations, at most m DECREASE-KEY operations, and n inserts. Its running time is therefore

$$T = n \cdot T_{\text{insert}} + n \cdot T_{\text{ext}} + m \cdot T_{\text{dec}},$$

and the priority queue we plug in decides everything. A standard binary min-heap stores elements in a complete binary tree of height $\lfloor \log_2 n \rfloor$ satisfying the heap property (each parent has a smaller key than its children). Inserting a new vertex or decreasing a key sifts up along a single branch in $O(\log n)$ steps, while extracting the minimum replaces the root with the last leaf and sifts down in $O(\log n)$ steps.

Priority queue	Insert	Extract-Min	Decrease-Key	Total
Array / unsorted	$O(1)$	$O(n)$	$O(1)$	$O(n^2 + m)$
Binary heap	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O((n+m)\log n)$
d-ary heap, $d{=}m/n$	$O(\log_d n)$	$O(d \log_d n)$	$O(\log_d n)$	$O(m \log_{m/n} n)$
Fibonacci heap	$O(1)$	$O(\log n)^\dagger$	$O(1)^\dagger$	$O(m + n \log n)$

† amortized.

Intuition

The trade is between extraction and decrease-key. A binary heap makes both $O(\log n)$, fine when $m = \Theta(n)$. But dense graphs have $m = \Theta(n^2)$ decrease-keys, and paying $\log n$ for each hurts. Two escapes. A *d-ary heap* (each node has d children) makes the tree shallower: height $\log_d n$ speeds decrease-key, at the cost of scanning d children per extract. Balancing $d \approx m/n$ gives $O(m \log_{m/n} n)$, which is $O(m)$ for sufficiently dense graphs. A *Fibonacci heap* makes decrease-key $O(1)$ amortized outright, yielding the classic $O(m + n \log n)$ bound that is optimal in the comparison model for this operation mix.

Although Fibonacci heaps achieve the $O(m + n \log n)$ bound by reducing decrease-key to $O(1)$ amortized time, in practice they carry significant pointer overhead and poor cache locality, so engineers

typically prefer binary or 4-ary heaps, or a lazy-deletion binary heap that re-inserts updated keys and discards stale entries.

Connection

The family of heap structures – binary, d-ary, Fibonacci – is developed for its own sake in the companion *Algorithms From the Ground Up* (priority-queue chapter) and in the data-structures appendix of this book. Dijkstra's algorithm is the primary workload motivating all three designs. The exchange argument that proves Dijkstra correct is the same greedy-with-a-cut argument that proves Prim's MST algorithm correct (the minimum-spanning-tree chapter); indeed Prim's algorithm *is* Dijkstra with "$w(u, v)$" in place of "$d[u] + w(u, v)$" as the key.

Cost summary

Dijkstra, SSSP, nonnegative weights:

- Binary heap: $O((n + m) \log n)$ time, $O(n)$ extra space. The default.
- Fibonacci heap: $O(m + n \log n)$ time – best comparison-based bound.
- Array: $O(n^2)$ time – best when $m = \Theta(n^2)$ (dense) and simplest to code.

7.4.5 The lazy-deletion implementation

In practice many languages lack a built-in decrease-key. The standard workaround is to push $(d[v], v)$ afresh every time $d[v]$ drops and to discard stale pairs on pop.

Algorithm 30: DIJKSTRA-LAZY(G, w, s) – binary heap, no decrease-key

```
1  foreach v ∈ V do
2      d[v] ← ∞
3    d[s] ← 0
4  Q ← empty min-heap; push (0, s)
5  while Q ≠ ∅ do
6      (δ, u) ← POP(Q)
7      if δ > d[u] then continue                    // stale entry, skip
8      foreach (u, v) ∈ E do
9          if d[u] + w(u, v) < d[v] then
10             d[v] ← d[u] + w(u, v); π[v] ← u; push (d[v], v)
```

The heap may hold up to m entries, so each operation is $O(\log m) = O(\log n)$ (since $m \le n^2$), and the total is $O(m \log n)$. The stale-skip line is what keeps it correct: a vertex may be popped several times, but only its first (smallest) pop does real work.

7.5 Bellman–Ford and Negative Cycles

Dijkstra's speed comes from settling each vertex once – and that is exactly what breaks under negative edges. Bellman–Ford pays for generality by giving up the greedy order: it simply relaxes *every* edge, over and over, $n-1$ times.

Intuition

Recall (Section 7.1) that with no negative cycle, some shortest path uses at most $n-1$ edges. Lemma 7.3 says that if a shortest path's edges get relaxed *in order*, the endpoint converges. We do not know the order – so we relax all m edges, then all m again, $n-1$ rounds total. After round i, every shortest path of $\le i$ edges has converged: its edges appear in order across the first i sweeps, because each sweep touches every edge. So after $n-1$ rounds, all shortest paths have converged.

Algorithm 31: BELLMAN-FORD(G, w, s) – arbitrary weights

```
1  foreach v ∈ V do
2      d[v] ← ∞; π[v] ← NIL
3    d[s] ← 0
4  for i ← 1 to n − 1 do
5      foreach edge (u, v) ∈ E do
6          if d[u] + w(u, v) < d[v] then
7              d[v] ← d[u] + w(u, v); π[v] ← u
   // Negative-cycle check:  one more sweep must change nothing.
8  foreach edge (u, v) ∈ E do
9      if d[u] + w(u, v) < d[v] then return NegativeCycleDetected
10 return (d, π)
```

Theorem 7.3 (Bellman–Ford correctness)**.** *If G has no negative cycle reachable from s, then after the $n-1$ relaxation rounds, $d[v] = \mathrm{dist}(s, v)$ for every v, and the final check reports no cycle. If a reachable negative cycle* does *exist, the final check returns* NegativeCycleDetected.

Proof. No negative cycle. Fix a vertex v and a shortest path $p = \langle s = v_0, \dots, v_k = v\rangle$ to it with $k \le n-1$ edges (such a simple shortest path exists, Section 7.1). We claim by induction that after round i, $d[v_i] = \mathrm{dist}(s, v_i)$. Each round relaxes *all* edges, in particular (v_{i-1}, v_i). After round 1 the edge (v_0, v_1) has been relaxed with $d[v_0] = 0$ already correct, fixing $d[v_1]$. Assuming $d[v_{i-1}]$ correct after round $i-1$, round i relaxes (v_{i-1}, v_i) and fixes $d[v_i]$ by Lemma 7.3. As $k \le n-1$, after $n-1$ rounds $d[v_k] = \mathrm{dist}(s, v)$. Then $(\star)$ holds for every edge, so the final sweep changes nothing.

Negative cycle exists. Let $C = \langle v_0, \dots, v_\ell = v_0\rangle$ be a reachable cycle with $\sum_i w(v_{i-1}, v_i) < 0$. Suppose the final check found *no* improvable edge; then $d[v_i] \le d[v_{i-1}] + w(v_{i-1}, v_i)$ for all edges of C and all $d[v_i]$ are finite (reachability). Summing around the cycle,

$$\sum_{i=1}^{\ell} d[v_i] \;\le\; \sum_{i=1}^{\ell} d[v_{i-1}] + \sum_{i=1}^{\ell} w(v_{i-1}, v_i).$$

The two distance sums are equal (the cycle returns to its start), so they cancel, leaving $0 \le w(C) < 0$, a contradiction. Hence some edge of C is still improvable and the check fires. □

Why it works

The negative-cycle test is the telescoping argument of Theorem 7.1 run *around a loop*: feasibility ($\star$) summed over a cycle yields $0 \le w(C)$, so feasibility is achievable exactly when every reachable cycle is nonnegative. A surviving improvable edge after $n-1$ rounds is thus a *certificate* of a negative cycle. To extract the cycle itself, follow π pointers back n steps from the improvable edge's head; you are guaranteed to land inside the cycle, and tracing π until a vertex repeats prints it.

7.5.1 Tracing Bellman–Ford with a negative edge

Trace it yourself

Modify the running graph: change $b \to a$ from $+1$ to -1. The graph stays acyclic, so no negative cycle appears. Now $\text{dist}(s,a) = 2 + (-1) = 1$, $\text{dist}(s,c) = 1 + 3 = 4$, and $\text{dist}(s,t) = \min(2+5,\ 1+6,\ 4+2) = 6$. Process edges in the fixed order $sa, sb, ba, ac, at, bt, ct$.

After round	$d[s]$	$d[a]$	$d[b]$	$d[c]$	$d[t]$
init	0	–	–	–	–
1	0	1	2	4	6
2	0	1	2	4	6

Round 1, edge by edge: sa sets $d[a] = 4$; sb sets $d[b] = 2$; ba sets $d[a] = 2 - 1 = 1$; ac sets $d[c] = 1+3 = 4$; at sets $d[t] = 1+6 = 7$; bt tries $2+5 = 7$ (no change); ct sets $d[t] = 4+2 = 6$. Already correct after one round, because this edge order happens to follow the shortest paths. Round 2 changes nothing – and a third sweep (the check) also changes nothing, certifying no negative cycle. The lucky single-round convergence is order-dependent; reverse the edge list and you would need more rounds.

Common trap

An undirected negative edge immediately creates a negative two-cycle. If you model an undirected edge $\{u, v\}$ of weight -3 as the pair $(u, v), (v, u)$ both of weight -3, you have created a 2-cycle of weight -6. Bellman–Ford will (correctly) report a negative cycle. Undirected shortest paths with negative edges is genuinely harder – it reduces to minimum-weight perfect matching, not to this chapter's machinery.

7.5.2 Complexity and the early-exit refinement

Each of the $n-1$ rounds touches all m edges, so Bellman–Ford runs in $O(nm)$ time and $O(n)$ space. A useful refinement: if an entire round relaxes *no* edge, the labels are already feasible and we can stop early. The worst case stays $O(nm)$, but on typical inputs it finishes in a few rounds.

Cost summary

Bellman–Ford, SSSP, arbitrary weights: $O(nm)$ time, $O(n)$ space, detects negative cycles. The *SPFA* heuristic (queue only the vertices whose label changed) has the same $O(nm)$ worst case but is often much faster; do not rely on it against adversarial graphs.

Viewed as a dynamic program, Bellman–Ford computes shortest-path distances stratified by hop count. By defining $\mathrm{dist}^{(i)}(s, v)$ as the shortest distance using at most i edges, the DP recurrence

$$\mathrm{dist}^{(i)}(s,v) = \min\Big(\mathrm{dist}^{(i-1)}(s,v),\ \min_{(u,v)\in E}\big[\mathrm{dist}^{(i-1)}(s,u) + w(u,v)\big]\Big)$$

relaxes all edges in rounds, forming the direct ancestor of the Floyd–Warshall equations.

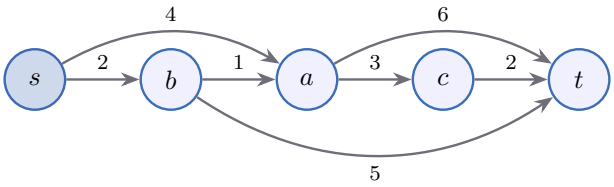
7.6 Shortest Paths in a DAG

When the graph is a directed acyclic graph, negative weights cause no trouble (no cycles, so certainly no negative ones), and we can do better than Bellman–Ford and even avoid the priority queue of Dijkstra. The key is a *topological order*: a linear arrangement of the vertices such that every edge points forward.

Intuition

In topological order, by the time we process a vertex u every edge *into* u has already been relaxed – because those edges come from earlier vertices. So $d[u]$ is final the instant we reach u, with no need to find the global minimum. One left-to-right sweep, relaxing each vertex's outgoing edges as we pass it, settles the whole graph. It is Bellman–Ford's $n-1$ rounds collapsed into a single perfectly-ordered pass.

Placing the vertices of the running graph along a line in topological order s, b, a, c, t makes every edge point strictly from left to right, as shown below.



Algorithm 32: DAG-Shortest-Paths(G, w, s)

```
L ← TopologicalSort(G)                          // e.g. by DFS finish times
foreach v ∈ V do
    d[v] ← ∞; π[v] ← NIL
  d[s] ← 0
foreach u ∈ L in topological order do
    foreach (u, v) ∈ E do
        if d[u] + w(u, v) < d[v] then
            d[v] ← d[u] + w(u, v); π[v] ← u
return (d, π)
```

Theorem 7.4 (DAG correctness)**.** *On a DAG,* DAG-Shortest-Paths *computes* $d[v] = \mathrm{dist}(s, v)$ *for all* v*, even with negative edge weights, in* $O(n + m)$ *time.*

Proof. Process vertices in topological order $u_1, u_2, \ldots, u_n$. We show by induction on the position j that when the sweep finishes processing u_j, $d[u_j] = \mathrm{dist}(s, u_j)$. Any shortest path to u_j arrives via some last edge (u_i, u_j) with $i < j$ (topological order forces predecessors earlier), or $u_j = s$. If $u_j = s$ then $d[s] = 0$. Otherwise, by induction $d[u_i] = \mathrm{dist}(s, u_i)$ was settled when u_i was processed, at which moment the edge (u_i, u_j) was relaxed, giving $d[u_j] \le \mathrm{dist}(s, u_i) + w(u_i, u_j) = \mathrm{dist}(s, u_j)$. With the upper bound this is equality. Topological sort is $O(n+m)$ and the sweep relaxes each edge once, so the total is $O(n + m)$. □

Trace it yourself

The running graph is acyclic; a topological order is s, b, a, c, t (every edge points right in that list: $sa, sb, ba, ac, at, bt, ct$ all go forward). Sweep: process s (relax $sa \to 4$, $sb \to 2$); process b (relax $ba \to 3$, $bt \to 7$); process a (relax $ac \to 6$, $at \to 9 \not< 7$); process c (relax $ct \to 8 \not< 7$); process t (no out-edges). Final $d = (0, 3, 2, 6, 7)$ for (s, a, b, c, t) – identical to Dijkstra, obtained in one pass with no heap. Replace any weight by a negative number and the same single pass still works.

Application

Critical-path / PERT scheduling. Reverse the sign and take *longest* paths in a DAG of tasks (each edge a precedence constraint carrying a duration); the longest source-to-sink path is the project's critical path, the minimum possible completion time. Longest path in a general graph is NP-hard, but in a DAG it is this same linear sweep with max in place of min. This is why project-management tools and build systems (`make`, Bazel) lean on topological order.

7.7 Potentials and Reduced Costs

We close with an idea that turns negative-weight graphs into nonnegative ones *without changing which paths are shortest*, so that we can run fast Dijkstra where slow Bellman–Ford seemed mandatory.

Johnson's all-pairs algorithm and the successive-shortest-path min-cost-flow methods both rely on it.

Definition 7.2 (Potential and reduced cost)**.** A *potential* is any function $h\colon V \to \mathbb{R}$. It induces *reduced edge weights*

$$w_h(u,v) \;=\; w(u,v) + h(u) - h(v).$$

Intuition

Think of $h(v)$ as an altitude assigned to each vertex. Reducing the weight of (u,v) by $h(v) - h(u)$ charges you for climbing and credits you for descending. Along any path the altitude terms telescope, so the *relative* ranking of paths between two fixed endpoints is untouched. We are free to choose altitudes that make every edge's reduced weight nonnegative.

Lemma 7.4 (Reduced costs preserve shortest paths)**.** *For any path p from u to v, $w_h(p) = w(p) + h(u) - h(v)$. Consequently p is a shortest u-v path under w if and only if it is shortest under w_h, and any cycle has the same weight under w and w_h.*

Proof. Sum the definition along $p = \langle u = x_0, \dots, x_k = v\rangle$:

$$w_h(p) = \sum_{i=1}^{k} \big(w(x_{i-1},x_i) + h(x_{i-1}) - h(x_i)\big) = w(p) + h(x_0) - h(x_k) = w(p) + h(u) - h(v),$$

the middle terms telescoping. Since $h(u) - h(v)$ is a constant shared by all u-v paths, it shifts every path's weight identically and cannot change which is least. For a cycle, $u = v$, so $h(u) - h(v) = 0$ and the weight is unchanged. □

Theorem 7.5 (Feasible potentials make weights nonnegative)**.** *The reduced weights w_h are all nonnegative if and only if h is a* feasible potential*: $h(v) \le h(u) + w(u,v)$ for every edge, i.e. h satisfies the optimality condition $(\star)$. Such an h exists if and only if G has no negative cycle.*

Proof. $w_h(u,v) = w(u,v) + h(u) - h(v) \ge 0 \iff h(v) \le h(u) + w(u,v)$, which is exactly $(\star)$. By Theorem 7.1, choosing $h(v) = \mathrm{dist}(s,v)$ from any source s that reaches all of V gives a feasible potential. To reach every vertex, add an artificial source q with a zero-weight edge $q \to v$ to all v; then $h(v) = \mathrm{dist}(q,v)$ is finite and feasible exactly when no negative cycle exists (otherwise some $\mathrm{dist}(q,v) = -\infty$, and Lemma 7.4 shows a negative cycle keeps its negative weight under *any* h, so no feasible h could exist). □

Why it works

This is the bridge to linear-programming duality. The shortest-path problem is a linear program; its dual variables are precisely the potentials $h(v)$, and "feasible potential" is dual feasibility. The condition $(\star)$ – which we first met as "no edge is relaxable" – is dual feasibility, the optimality $d[v] = \mathrm{dist}(s,v)$ is primal–dual complementary slackness, and the negative-cycle obstruction is dual infeasibility. The same three words (feasible, reduced, slack) reappear verbatim in the min-cost-flow chapter.

Trace it yourself

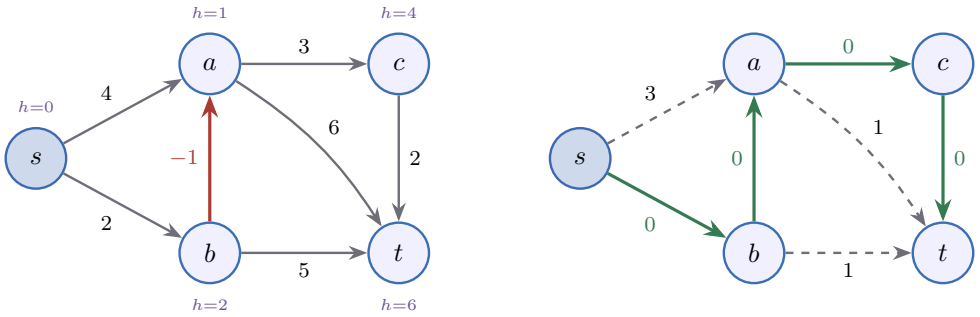
Take the running graph with $b \to a$ set to -1 (the negative variant). Use the potential $h(v) = \text{dist}(s, v) = (0, 1, 2, 4, 6)$ for (s, a, b, c, t). Reduced weights:

$$w_h(s,a) = 4 + 0 - 1 = 3, \quad w_h(s,b) = 2 + 0 - 2 = 0, \quad w_h(b,a) = -1 + 2 - 1 = 0,$$
$$w_h(a,c) = 3 + 1 - 4 = 0, \quad w_h(a,t) = 6 + 1 - 6 = 1, \quad w_h(b,t) = 5 + 2 - 6 = 1,$$
$$w_h(c,t) = 2 + 4 - 6 = 0.$$

Every reduced weight is ≥ 0, as promised, and the edges on the shortest-path tree (sb, ba, ac, ct) have reduced weight 0 – they are the "tight" edges. (Edge bt has reduced weight 1: the route $s \to b \to t$ costs 7, not the optimal 6, so bt is slack, not tight.) Run Dijkstra on w_h: it is now legal, and Lemma 7.4 converts the answers back via $\text{dist}_w(u, v) = \text{dist}_{w_h}(u, v) - h(u) + h(v)$.

The two panels below compare the original negative-weight graph alongside its potential-reweighted counterpart under $h(v) = \text{dist}(s, v)$, where every shortest-path tree edge becomes tight ($w_h = 0$) and all remaining edges have positive slack ($w_h > 0$).

(A) Original weights w and potentials h (B) Reduced costs $w_h(u, v) = w(u, v) + h(u) - h(v) \geq 0$



Connection

Johnson's algorithm (all-pairs chapter) applies this reweighting directly: one Bellman–Ford from an artificial super-source computes $h(v) = \text{dist}(q, v)$ in $O(nm)$, reweights to make everything nonnegative, then runs Dijkstra from every vertex in $O(n(m + n \log n))$ – far better than n Bellman–Fords on sparse graphs. **Min-cost flow's** successive-shortest- path algorithm keeps a potential equal to current distances so that each augmenting search is a nonnegative-weight Dijkstra despite residual edges with negative cost. **A^* search** is Dijkstra with potential $h(v) =$ heuristic estimate of $\text{dist}(v, t)$; an *admissible, consistent* heuristic is exactly a feasible potential, which is why consistent A^* never needs to re-expand a node – it is Dijkstra on reduced costs.

7.8 Putting the Pieces Together

Setting	Algorithm	Time
Nonnegative weights	Dijkstra (binary heap)	$O((n+m)\log n)$
Nonnegative, dense	Dijkstra (Fibonacci/array)	$O(m + n\log n)$ / $O(n^2)$
Arbitrary weights	Bellman–Ford	$O(nm)$
DAG (any weights)	Topological sweep	$O(n+m)$
Arbitrary, all-pairs	Johnson (potentials + Dijkstra)	$O(nm + n^2 \log n)$

The chapter rests on three ideas. Every algorithm in it is a schedule of *relaxation* steps. The *optimality condition* ($\star$) – no edge improvable – is both a stopping rule and, summed around a cycle, the negative-cycle test. And *potentials* recast the problem as a shifted potential field, letting fast nonnegative-weight machinery solve negative-weight problems and tying shortest paths to linear-programming duality and heuristic search.

In Chapter 8, we build on this foundation to explore advanced shortest-path techniques: goal-directed A^* search, bidirectional heuristics, Dial's algorithm for small integer weights, and scaling methods that push past standard comparison bounds.

7.9 Exercises

Exercise 7.1 (Relaxation order)**.** Give a 4-vertex weighted DAG and an order of edge relaxations for which a single pass of Bellman–Ford (one sweep over all edges) does *not* yield correct distances, then give an order on the same graph for which it does. Explain the difference using Lemma 7.3.

Exercise 7.2 (Dijkstra breaks gracefully?)**.** Construct a directed graph with a single negative edge on which Dijkstra (settle-once) returns a *wrong* distance, and a different single-negative-edge graph on which it happens to return the *correct* distances. What property of the negative edge's position decides the outcome?

Exercise 7.3 (Counting shortest paths)**.** Modify Dijkstra to also compute, for every v, the *number* of distinct shortest paths from s to v (assume nonnegative weights and that this count fits in a machine word). Prove your update rule correct and argue it adds only $O(m)$ work. Where does the argument need "settle once"?

Exercise 7.4 (d-ary tuning)**.** For a graph with $n = 10^4$ vertices and $m = 10^6$ edges, compute the heap arity d that minimizes the asymptotic Dijkstra cost $O(nd \log_d n + m \log_d n)$, and compare the resulting bound numerically against a binary heap and a Fibonacci heap.

Exercise 7.5 (Extracting the negative cycle)**.** Prove that if Bellman–Ford's final sweep finds an improvable edge (u, v), then following π pointers backward from v for n steps lands on a vertex that lies on a negative cycle. Turn this into an algorithm that prints the cycle in $O(n)$ time.

Exercise 7.6 (Shortest path with exactly k edges)**.** Adapt the Bellman–Ford DP to compute, for a fixed k, the minimum-weight walk from s to t using *exactly* k edges (walks may repeat vertices). What is the running time, and why can you not simply stop relaxing early?

Exercise 7.7 (Bottleneck shortest paths)**.** Define the weight of a path as the *maximum* edge weight on it (the bottleneck), and ask for the path minimizing this maximum. Show that a Dijkstra-like algorithm with the relaxation $d[v] \leftarrow \min\big(d[v], \max(d[u], w(u,v))\big)$ solves it, prove correctness, and state the time bound. How does this connect to minimum spanning trees?

Exercise 7.8 (Potentials by hand)**.** For the running graph with $a \to t$ changed to weight -4, run one Bellman–Ford from an added super-source to get a feasible potential h, list all reduced weights, verify they are nonnegative, and confirm that Dijkstra on the reduced graph reproduces the true distances after the back-conversion of Lemma 7.4.

Exercise 7.9 (A^* as reduced-cost Dijkstra)**.** Let h be a *consistent* heuristic estimating distance to the target t, meaning $h(u) \le w(u,v) + h(v)$ for every edge and $h(t) = 0$. Show that running Dijkstra with edge weights $w_h(u,v) = w(u,v) - h(u) + h(v)$ is exactly A^* with heuristic h, that all reduced weights are nonnegative, and therefore that consistent A^* never re-expands a settled node.

Exercise 7.10 (Lazy vs. eager heap)**.** Implement both Dijkstra (with decrease-key) and Dijkstra-Lazy (push-stale) and construct a family of graphs on which the lazy heap stores $\Theta(m)$ entries simultaneously. Confirm empirically that both run in $O(m \log n)$ but compare their constant factors and peak memory.

Exercise 7.11 (Why $n-1$ and not fewer)**.** Exhibit, for every n, a graph on n vertices and an edge-processing order for which Bellman–Ford genuinely needs all $n-1$ rounds – the distance of some vertex is still wrong after $n-2$ rounds. (Hint: a path graph processed back to front.)

Chapter 8

Advanced Shortest Paths

The previous chapter built the classical machinery: Dijkstra for nonnegative weights, Bellman–Ford for negative edges, the relaxation invariant, and the priority-queue bookkeeping that makes Dijkstra run in $O((m+n)\log n)$. That machinery answers a single question optimally: *what is the distance from a source to every vertex?* On a continental road network with $n \approx 2 \cdot 10^7$ vertices, "every vertex" is the wrong question. A navigation app issues millions of point-to-point queries per second, and each one wants *this* s-to-t distance, now, ideally in microseconds. Plain Dijkstra settles a ball of vertices whose radius is $\mathrm{dist}(s,t)$; on a road graph that ball can hold most of a country before t is reached. We need to be cleverer.

Instead of exploring blindly in all directions, we preserve the relaxation invariant while restricting our search to vertices that actually lie toward the destination: first with a target-seeking heuristic, then by searching simultaneously from both ends, and finally by precomputing landmark distances and shortcut highways.

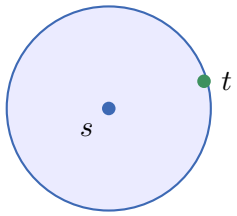
Throughout, $G = (V, E)$ is a directed graph with nonnegative edge weights $w : E \to \mathbb{R}_{\geq 0}$, $n = |V|$, $m = |E|$. We write $\mathrm{dist}(u, v)$ for the true shortest-path distance and reserve $d[\cdot]$ for tentative distances held by an algorithm. For backward search we use the *reverse graph* G^R, which has an edge (v, u) of weight $w(u, v)$ for every edge (u, v) of G; distances in G^R from t are exactly distances in G *to* t.

Intuition

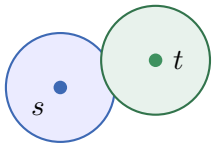
Plain Dijkstra is a blind, expanding circle: it has no idea where t is, so it grows equally in all directions. Every method in this chapter is a way of *telling the search where the target is* – with a geometric guess (A^*), with two circles instead of one (bidirectional), with precomputed distances to anchors (ALT), or by precomputing a road-hierarchy so highways are preferred automatically (CH). Each finds the same shortest path while touching far fewer vertices.

8.1 The cost of being blind: a motivating picture

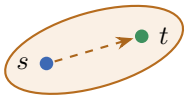
Consider Dijkstra from s on a roughly uniform planar graph (a road map is close enough). By the time t is settled, every vertex v with $\mathrm{dist}(s, v) \leq \mathrm{dist}(s, t)$ has been settled too. The settled set is a disk of radius $r = \mathrm{dist}(s, t)$, and on a planar graph with bounded density it contains $\Theta(r^2)$ vertices. If s and t are far apart, that is an enormous fraction of the map.



DIJKSTRA: AREA πr^2



BIDIRECTIONAL: AREA $\frac{1}{2}\pi r^2$



A^* SEARCH: FOCUSED CORRIDOR

Two half-radius disks have total area $2 \cdot \pi(r/2)^2 = \pi r^2/2$, half the area of the full disk; this is where bidirectional search saves its work. A^* deforms the single disk into an ellipse hugging the s–t axis; with a perfect heuristic it collapses to a thin corridor of $\Theta(r)$ vertices. CH abandons the disk picture entirely: it climbs a precomputed hierarchy so the search reaches t after touching only a few hundred vertices, independent of r.

8.2 Goal-directed search: A^*

8.2.1 From Dijkstra to A^*

Dijkstra orders its priority queue by $d[v]$, the best-known distance from s. A^* adds a *potential* $h(v)$, an estimate of the remaining distance $\text{dist}(v, t)$, and orders by

$$f(v) \;=\; d[v] + h(v).$$

The quantity $f(v)$ estimates the length of the best s–t path that is forced to pass through v. Vertices that look like they lie on a good route to t get small f and are explored first; vertices pointing away from t get large f and wait. When $h \equiv 0$, A^* is exactly Dijkstra.

Intuition

Think of $h(v)$ as a "crow-flies" estimate of how far v still is from t. Dijkstra asks "who is closest to where I started?". A^* asks "who is on the most promising total route to where I am going?". The promise is already-paid distance $d[v]$ plus the optimistic guess $h(v)$ of what remains.

8.2.2 Admissibility and consistency

For A^* to return exact shortest-path distances, $h(v)$ must never overestimate the true remaining distance to t. Two conditions make this precise.

Definition 8.1 (Admissible heuristic)**.** h is *admissible* (for target t) if $h(v) \le \text{dist}(v, t)$ for every v, and $h(t) = 0$. It never overestimates the true remaining distance.

Definition 8.2 (Consistent heuristic)**.** h is *consistent* (or *monotone*) if for every edge (u, v),

$$h(u) \;\le\; w(u, v) + h(v),$$

and $h(t) = 0$. Equivalently, the *reduced cost* $w_h(u, v) := w(u, v) - h(u) + h(v)$ is nonnegative on every edge.

Lemma 8.1 (Consistency implies admissibility)**.** *If h is consistent then h is admissible.*

Proof. Take any v and a shortest path $v = x_0, x_1, \ldots, x_k = t$ of length $\mathrm{dist}(v, t)$. Summing consistency over its edges telescopes:

$$h(v) = h(x_0) \leq \sum_{i=0}^{k-1} w(x_i, x_{i+1}) + h(x_k) = \mathrm{dist}(v, t) + h(t) = \mathrm{dist}(v, t).$$

So $h(v) \leq \mathrm{dist}(v, t)$. □

The reduced-cost view explains why A^* is correct. Define the potential function $\pi(v) = h(v)$ and rewrite each edge weight as $w_h(u, v) = w(u, v) - \pi(u) + \pi(v)$. For *any* path P from s to z, the reduced length telescopes:

$$w_h(P) \;=\; \sum_{(u,v)\in P} \big(w(u, v) - \pi(u) + \pi(v)\big) \;=\; w(P) - \pi(s) + \pi(z).$$

The two endpoint potentials are the same for all s–z paths, so w_h and w rank paths identically: shortest paths are preserved. Consistency says $w_h \geq 0$, which means A^* **on** w **is precisely Dijkstra on the reduced weights** w_h. Every guarantee Dijkstra enjoys – in particular, settling vertices in nondecreasing key order and finalizing each correctly – carries over unchanged. This observation gives a short correctness proof of A^*, and we will reuse it for ALT and for CH's potentials.

Why it works

With a consistent heuristic, A^* is equivalent to running Dijkstra's algorithm on potential-reweighted edges: setting the vertex potential $\pi = h$ replaces each edge weight with $w_h(u, v) = w(u, v) - \pi(u) + \pi(v)$. If the potential is consistent the re-weighted graph has no negative edges, so Dijkstra is correct on it, and because re-weighting shifts every s–t path's length by the same constant $\pi(t) - \pi(s)$, the shortest path is unchanged. Goal-direction is just a smart choice of potential – one that is small near t.

8.2.3 The algorithm

Algorithm 33: A^* search with a consistent heuristic

input : graph G, source s, target t, consistent heuristic h
output: $\mathrm{dist}(s, t)$

```
1  d[v] ← ∞ for all v;   d[s] ← 0
2  priority queue Q keyed by f(v) = d[v] + h(v); insert s with key h(s)
3  while Q not empty do
4      u ← ExtractMin(Q)
5      if u = t then
6          return d[t]                                  // t settled with final distance
7      foreach edge (u, v) ∈ E do
8          if d[u] + w(u, v) < d[v] then
9              d[v] ← d[u] + w(u, v)
10             DecreaseKeyOrInsert(Q, v, d[v] + h(v))
11 return ∞                                             // t unreachable
```

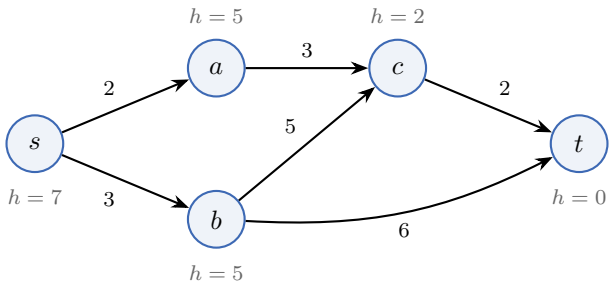
The only structural change from Dijkstra is the key: $d[v] + h(v)$ instead of $d[v]$, plus an early return the moment t is extracted. With a consistent h, no vertex is ever reopened (it is plain Dijkstra on w_h), so the early stop is safe: when t comes off the queue its distance is final.

Common trap

Admissible but not consistent. If h is only admissible, A^* still returns the correct $\mathrm{dist}(s, t)$, but a settled vertex may later be found via a cheaper path and must be *reopened* (reinserted into Q). Skipping the reopen check – treating an extracted vertex as final, as you safely do under consistency – can then yield a wrong, too-large distance. Implementations that assume consistency but are fed a merely-admissible heuristic produce silent errors on a small fraction of queries. Use consistent heuristics, or keep the reopen logic.

8.2.4 A worked trace

Take this small weighted graph. Edge labels are weights; the dashed number under each vertex is the heuristic $h(v)$ (a straight-line lower bound to t).

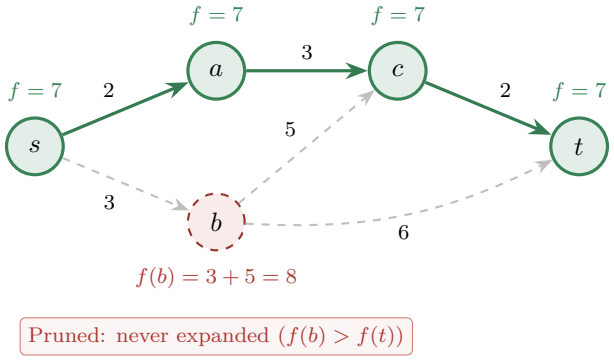


You can check h is consistent: e.g. edge (s, a) needs $h(s) \le w + h(a)$, i.e. $7 \le 2 + 5$, true; edge (a, c) needs $5 \le 3 + 2$, tight with equality; edge (b, t) needs $5 \le 6 + 0$; and so on for every edge. The

true distances to t are $\text{dist}(s,t) = 7$ (via s, a, c, t), $\text{dist}(a,t) = 5$, $\text{dist}(b,t) = 6$, $\text{dist}(c,t) = 2$; note $h \le \text{dist}(\cdot, t)$ everywhere, so it is admissible too.

Run Algorithm 33. The queue holds $(v : f = d + h)$ pairs; **bold** marks the extracted minimum.

step	extract	queue after relaxing (key $= d + h$)
0	–	$\{\mathbf{s{:}0{+}7{=}7}\}$
1	s $(f = 7)$	$a{:}2{+}5{=}7$, $b{:}3{+}5{=}8$
2	a $(f = 7)$	$b{:}8$, $c{:}5{+}2{=}7$
3	c $(f = 7)$	$b{:}8$, $t{:}7{+}0{=}7$
4	t $(f = 7)$	return $d[t] = 7$



A^* extracted s, a, c, t and *never expanded* b: vertex b sat in the queue with key $8 > 7$ and was pruned by the early stop at t. The three on-route vertices s, a, c all carry $f = 7$: along the shortest path every edge is tight ($w_h = 0$), so f holds steady at $\text{dist}(s,t)$, which is exactly the behaviour a consistent heuristic guarantees. Plain Dijkstra, keyed by d alone, would have settled b ($d = 3$) before c ($d = 5$) – one extra expansion. On a real map with thousands of off-route vertices, that "never touched b" multiplies into the elliptical-corridor savings from Section 8.1.

Trace it yourself

Redo the trace with $h \equiv 0$ (plain Dijkstra) and confirm b is extracted at $d = 3$, between a and c. Then inflate the heuristic of a vertex *on* the shortest route – set $h(c) = 20$ (now *inadmissible*, since $\text{dist}(c,t) = 2$) – and watch A^* return the wrong answer 9: vertex c gets key $f = 5 + 20 = 25$ and is buried, so b is expanded, t comes off the queue at $f = 9$ via s, b, t, and the early stop declares it final before the true path through c is ever explored. (Inflating an *off*-route vertex like b does no harm – the damage comes only when you over-estimate a vertex the optimal path needs.) Constructing the failing query by hand shows concretely why admissibility is required.

8.3 Bidirectional search

8.3.1 Two balls meeting in the middle

Run one Dijkstra forward from s in G and, simultaneously, one Dijkstra backward from t in G^R. Each maintains its own distance arrays d_f and d_b and its own queue. Alternate steps between them. The

instant a vertex has been *settled by both* searches, we have enough information to recover $\mathrm{dist}(s,t)$ – though, as we will see, the meeting vertex is not always on the shortest path, and the stopping rule needs care.

Intuition

If the shortest path is $s \rightsquigarrow x \rightsquigarrow t$, then x lies at forward-distance $\mathrm{dist}(s,x)$ and backward-distance $\mathrm{dist}(x,t)$ with $\mathrm{dist}(s,x) + \mathrm{dist}(x,t) = \mathrm{dist}(s,t)$. The forward ball grows to radius about $\mathrm{dist}(s,t)/2$, the backward ball likewise, and they touch near the path's midpoint. Two radius-$r/2$ balls instead of one radius-r ball: on a planar graph that is roughly half the vertices, and the saving grows with dimension.

8.3.2 The stopping rule (and the classic bug)

Let μ be the best s–t path length found so far through any vertex relaxed by both sides; initialize $\mu = \infty$. Whenever a search relaxes an edge (u,v) and v is already settled by the *other* search, update

$$\mu \;\leftarrow\; \min\bigl(\mu,\; d_f[u] + w(u,v) + d_b[v]\bigr)$$

(symmetrically for the backward search). Let top_f and top_b be the current minimum keys of the two queues. **Stop when**

$$\mathrm{top}_f + \mathrm{top}_b \;\geq\; \mu.$$

Common trap

Do not stop at first touch. The tempting rule "halt as soon as some vertex is settled by both searches" is *wrong*. The first commonly-settled vertex may not lie on a shortest path. The classic counterexample: a graph where the two frontiers first meet on a slightly-too-long detour while the true shortest path passes through a vertex each side settles a step later. You must keep going until $\mathrm{top}_f + \mathrm{top}_b \geq \mu$, which certifies that no unexplored path can beat the best μ already recorded.

Theorem 8.1 (Correctness of bidirectional Dijkstra)**.** *When the rule* $\mathrm{top}_f + \mathrm{top}_b \geq \mu$ *first holds,* $\mu = \mathrm{dist}(s,t)$.

Proof. μ is always the length of some genuine s–t path (it is a sum of a real forward distance, a real edge, and a real backward distance), so $\mu \geq \mathrm{dist}(s,t)$ at all times.

For the reverse inequality, fix a shortest s–t path $P : s = p_0, \ldots, p_k = t$. Walk along P and let p_i be the last vertex settled by the forward search at the moment we stop, p_{i+1} the first not settled forward. Because p_i is settled forward, $d_f[p_i] = \mathrm{dist}(s,p_i)$, and the edge (p_i, p_{i+1}) was relaxed, so the forward queue holds p_{i+1} with key $d_f[p_{i+1}] \leq \mathrm{dist}(s,p_{i+1})$; hence $\mathrm{top}_f \leq \mathrm{dist}(s,p_{i+1})$.

Now consider the suffix $p_{i+1}, \ldots, t$. If *every* vertex of the suffix is settled by the backward search, then in particular p_{i+1} is, and at the moment it was settled backward the forward side had already settled p_i and relaxed the edge into p_{i+1}, so μ was updated to at most $d_f[p_i] + w(p_i,p_{i+1}) + d_b[p_{i+1}] = \mathrm{dist}(s,t)$. Then $\mu = \mathrm{dist}(s,t)$, done. Otherwise let p_j ($j \geq i+1$) be the last suffix vertex *not* settled backward; the backward queue holds it with key $\mathrm{top}_b \leq \mathrm{dist}(p_j,t)$. Combining, using that P is shortest so subpath lengths add up,

$$\mathrm{top}_f + \mathrm{top}_b \;\leq\; \mathrm{dist}(s,p_{i+1}) + \mathrm{dist}(p_j,t) \;\leq\; \mathrm{dist}(s,p_{i+1}) + \mathrm{dist}(p_{i+1},t) \;=\; \mathrm{dist}(s,t) \;\leq\; \mu.$$

But the stopping condition is $\text{top}_f + \text{top}_b \geq \mu$, so all the inequalities are equalities, forcing $\mu = \text{dist}(s,t)$. □

Algorithm 34: Bidirectional Dijkstra

input : G, G^R, source s, target t
output: $\text{dist}(s,t)$

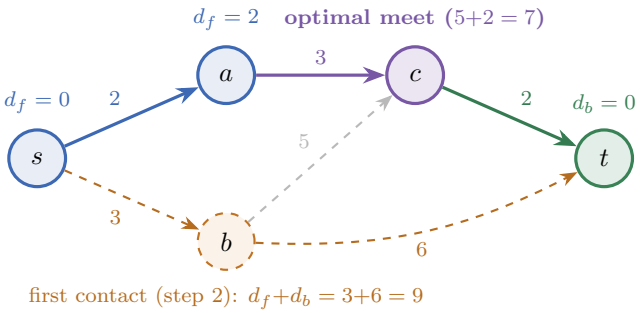
$d_f[\cdot] \leftarrow \infty,\ d_b[\cdot] \leftarrow \infty;\quad d_f[s] \leftarrow 0,\ d_b[t] \leftarrow 0;\quad \mu \leftarrow \infty$
forward queue $Q_f \leftarrow \{s\}$, backward queue $Q_b \leftarrow \{t\}$
while $Q_f \neq \emptyset$ ***and*** $Q_b \neq \emptyset$ ***and*** $\text{top}(Q_f) + \text{top}(Q_b) < \mu$ **do**
    `// alternate; here: expand whichever queue has the smaller top key`
    if $\text{top}(Q_f) \leq \text{top}(Q_b)$ **then**
        $u \leftarrow \textsc{ExtractMin}(Q_f)$; settle u forward
        foreach $(u,v) \in E$ **do**
            relax: **if** $d_f[u] + w(u,v) < d_f[v]$ **then** update $d_f[v]$, push v
            if $d_b[v] < \infty$ **then**
                $\mu \leftarrow \min(\mu,\ d_f[u] + w(u,v) + d_b[v])$
    else
        $u \leftarrow \textsc{ExtractMin}(Q_b)$; settle u backward
        foreach $(v,u) \in E$ `// in-edges of u = out-edges in` G^R
        do
            relax: **if** $d_b[u] + w(v,u) < d_b[v]$ **then** update $d_b[v]$, push v
            if $d_f[v] < \infty$ **then**
                $\mu \leftarrow \min(\mu,\ d_f[v] + w(v,u) + d_b[u])$
return μ

8.3.3 A worked trace

Reuse the graph from Section 8.2.4 (ignore the h labels). Forward distances from s: $d_f[s] = 0$. Backward distances from t in G^R: $d_b[t] = 0$. Alternate, expanding the smaller top key; ties go forward.

step	side	extract	effect
1	F	s (0)	$d_f[a] = 2,\ d_f[b] = 3$
2	B	t (0)	$d_b[c] = 2,\ d_b[b] = 6$; b labelled by F $\Rightarrow \mu = 3 + 6 + 0 = 9$
3	F	a (2)	$d_f[c] = 5$; c labelled by B $\Rightarrow \mu = \min(9, 5 + 2) = 7$
4	B	c (2)	$d_b[a] = 5,\ d_b[b] = \min(6, 2 + 5) = 6$; a labelled by F $\Rightarrow \mu = \min(7, 2 + 3 + 2) = 7$
5	–	–	top_f=3 (b), top_b=5 (a): $3 + 5 = 8 \geq \mu = 7$, stop

After step 4, $\text{top}_f = 3$ (vertex b) and $\text{top}_b = 5$ (vertex a), so $\text{top}_f + \text{top}_b = 8 \geq \mu = 7$ and we stop, returning $\mu = 7 = \text{dist}(s,t)$, matching A^*. Notice in the diagram below that the first value of μ recorded (at step 2, $\mu = 9$, via the detour $s \to b \to t$) was *not* the answer; the sum-of-tops rule kept the search alive until the true path through c was certified. Correctness depended on that rule, not on first contact.



Why it works

The stopping certificate is an LP-duality flavored bound. Any s–t path that μ has not yet seen must leave the forward-settled region across a queued forward vertex (cost $\geq \text{top}_f$) and enter the backward-settled region across a queued backward vertex (cost $\geq \text{top}_b$), so its length is at least $\text{top}_f + \text{top}_b$. Once that lower bound reaches μ, no unseen path can improve on μ, and we are provably done.

8.3.4 Bidirectional A^*: a subtlety

Combining bidirection with A^* is tempting but tricky. If the forward search uses potential h_f (estimating distance to t) and the backward uses h_b (estimating distance from s), the two re-weight edges differently, and the clean "Dijkstra on w_h" picture breaks: a vertex's forward reduced distance and backward reduced distance no longer add up to a quantity you can threshold. The standard fix is the *consistent potential pair* of Ikeda et al.: use $p_f(v) = \frac{1}{2}(h_f(v) - h_b(v))$ forward and $p_b(v) = -p_f(v)$ backward. These satisfy $p_f(v) + p_b(v) = 0$, which makes the two reduced-weight graphs agree on the shared meeting condition, restoring a valid stopping rule at a (usually mild) cost in heuristic sharpness. We will not need the symmetric A^* below, because CH gives a cleaner bidirectional structure, but the lesson – *two potentials must be reconciled* – recurs there.

8.4 Landmarks and the ALT algorithm

A^* is only as good as its heuristic. On a road network the Euclidean straight-line distance is admissible (roads cannot be shorter than crow-flies), but it is loose: detours, mountains, and one-way systems make true distances far longer than the geometric guess, so the corridor stays wide. ALT (**A***, **L**andmarks, **T**riangle inequality) replaces the geometric guess with a graph-aware one, computed from a few precomputed distance tables.

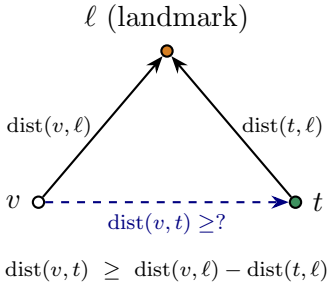
8.4.1 Triangle-inequality bounds

Pick a small set $L \subseteq V$ of *landmarks*. For each landmark ℓ, precompute (one Dijkstra each, forward and backward) the vectors $\text{dist}(\ell, \cdot)$ and $\text{dist}(\cdot, \ell)$. The triangle inequality on the nonnegative-weight metric gives, for any vertex v and target t and any landmark ℓ,

$$\text{dist}(v,t) \;\geq\; \text{dist}(v,\ell) - \text{dist}(t,\ell) \qquad \text{and} \qquad \text{dist}(v,t) \;\geq\; \text{dist}(\ell,t) - \text{dist}(\ell,v).$$

Each is a valid *lower* bound on $\mathrm{dist}(v,t)$. Take the best over all landmarks and both forms:

$$h_L(v) \;=\; \max_{\ell \in L} \max\Big(\mathrm{dist}(v,\ell) - \mathrm{dist}(t,\ell),\;\; \mathrm{dist}(\ell,t) - \mathrm{dist}(\ell,v)\Big).$$



Lemma 8.2 (h_L is feasible for A^*)**.** *For nonnegative weights, h_L is admissible, and each per-landmark term is consistent; the pointwise maximum of consistent heuristics with the same value at t is consistent, so h_L is consistent.*

Proof. Admissibility is the triangle inequality above. For consistency of a single forward term $g_\ell(v) = \mathrm{dist}(v,\ell) - \mathrm{dist}(t,\ell)$, take an edge (u,v). By the triangle inequality $\mathrm{dist}(u,\ell) \le w(u,v) + \mathrm{dist}(v,\ell)$, so $g_\ell(u) = \mathrm{dist}(u,\ell) - \mathrm{dist}(t,\ell) \le w(u,v) + \mathrm{dist}(v,\ell) - \mathrm{dist}(t,\ell) = w(u,v) + g_\ell(v)$. The backward term is symmetric. Finally, if h_1, h_2 are consistent with $h_i(t) = 0$, then for edge (u,v), $\max(h_1(u), h_2(u)) \le \max(w + h_1(v), w + h_2(v)) = w + \max(h_1(v), h_2(v))$, and the max is 0 at t. Induction over $|L|$ terms finishes it. □

So ALT is just A^* (Algorithm 33) with $h = h_L$, evaluated lazily: computing $h_L(v)$ costs $O(|L|)$ table lookups per vertex touched.

8.4.2 Preprocessing and the query split

Algorithm 35: ALT: preprocessing and query

```
Preprocessing (once):
choose landmark set L (see below)
foreach ℓ ∈ L do
    run Dijkstra from ℓ in G to store dist(ℓ, ·)
    run Dijkstra from ℓ in G^R to store dist(·, ℓ)
Query (s, t):
run A* from s to t with heuristic h_L(v) = max_ℓ max(...) above
```

Preprocessing is $O(|L| \cdot (m+n) \log n)$ time and $O(|L| \cdot n)$ space – two distance vectors per landmark. With $|L| = 16$ landmarks the tables for a continental graph fit comfortably in memory and queries speed up by one to two orders of magnitude over plain Dijkstra.

8.4.3 Choosing landmarks

Random landmarks are mediocre. The heuristic $h_L(v)$ is sharp when t lies "behind" a landmark as seen from v – when ℓ, v, t are nearly collinear with the landmark beyond t. The widely used *avoid*

and *farthest* heuristics place landmarks at the corners of the map and at hard-to-reach peripheral points, so that for most query pairs some landmark sits behind the target.

- **Farthest:** start with a random vertex; repeatedly add the vertex maximizing its distance to the current landmark set. Spreads landmarks to the boundary.
- **Avoid:** grow a shortest-path tree from a random root, score each vertex by how much shortest-path "weight" is poorly covered by existing landmarks, and place the next landmark in the worst-covered region.

Trace it yourself

On the trace graph, put a single landmark at b. Precompute $\mathrm{dist}(b, \cdot) = \{b:0, c:5, t:6\}$ and $\mathrm{dist}(\cdot, b) = \{s:3, b:0\}$. Compute $h_{\{b\}}(s) = \max(\mathrm{dist}(s,b) - \mathrm{dist}(t,b), \mathrm{dist}(b,t) - \mathrm{dist}(b,s))$. With $\mathrm{dist}(t,b) = \infty$ (no path $t \to b$ here) the first term is $-\infty$; the second is $6 - \mathrm{dist}(b,s)$, and $\mathrm{dist}(b,s) = \infty$, so $h_{\{b\}}(s) = -\infty$, i.e. useless. Add a landmark at c instead and recompute – you will see why landmarks behind the target dominate, and why one poorly placed landmark contributes nothing.

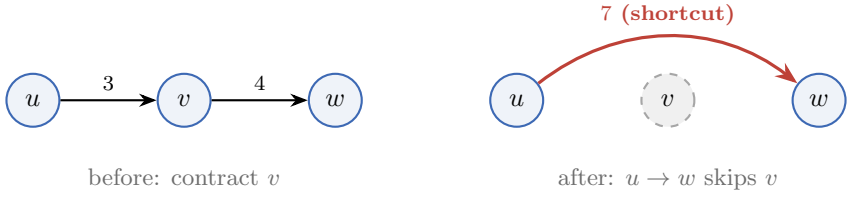
8.5 Contraction hierarchies

ALT trims the search; CH redesigns the graph so the search barely has to run. It is the dominant exact method for static road networks: minutes of preprocessing buy queries that settle a few hundred vertices regardless of how far apart s and t are.

8.5.1 The idea: shortcut the unimportant vertices

Road networks are hierarchical. A trip from a rural house to another rural house climbs onto residential streets, then arterials, then a motorway, cruises the motorway, then descends symmetrically. The vast middle of any long trip uses a small set of "important" roads. CH makes this hierarchy explicit.

Order the vertices by an *importance* rank $\mathrm{rank} : V \to \{1, \ldots, n\}$. Now *contract* vertices one at a time in increasing rank order. To contract v means: remove v, and for every pair of neighbors u, w such that the path $u \to v \to w$ was a shortest path among the remaining vertices, insert a *shortcut* edge $u \to w$ with weight $w(u,v) + w(v,w)$ – but only if no other path of equal or smaller length already connects u to w in the current graph (a *witness*). The shortcut preserves all shortest-path distances among the not-yet-contracted vertices.



Intuition

Contracting a vertex is "routing around" it: if the only reason a path used v was to get from u to w, replace that little detour with one direct shortcut edge and forget v exists. Do this from least-important to most-important vertex, and the shortcuts you accumulate are exactly the on-ramps, motorway segments, and off-ramps – the highway skeleton – expressed as edges. A query then only ever needs to drive "uphill" in importance from s, and "uphill" backward from t, meeting at the most important vertex on the route.

8.5.2 The augmented graph and the upward/downward split

After contracting all vertices, let E^+ be the original edges plus all shortcuts. Orient every edge of E^+ by rank: an edge (x, y) is *upward* if $\text{rank}(x) < \text{rank}(y)$ and *downward* otherwise. Define

$$G_\uparrow = (V,\ \{(x,y) \in E^+ : \text{rank}(x) < \text{rank}(y)\}), \qquad G_\downarrow = (V,\ \{(x,y) \in E^+ : \text{rank}(x) > \text{rank}(y)\}).$$

The query runs a *forward* Dijkstra from s restricted to upward edges $G_\uparrow$, and a *backward* Dijkstra from t restricted to the upward edges of G^R (equivalently, downward edges of $G_\downarrow$ traversed backward). Both searches only ever increase rank. They meet at candidate vertices; combine d_f and d_b as in bidirectional Dijkstra.

Figure 8.1 shows the shape every CH query exploits: the shortest path in E^+ climbs in rank to a single peak, then descends, so the forward search (climbing from s) and the backward search (climbing from t) each cover one half and meet at the peak.

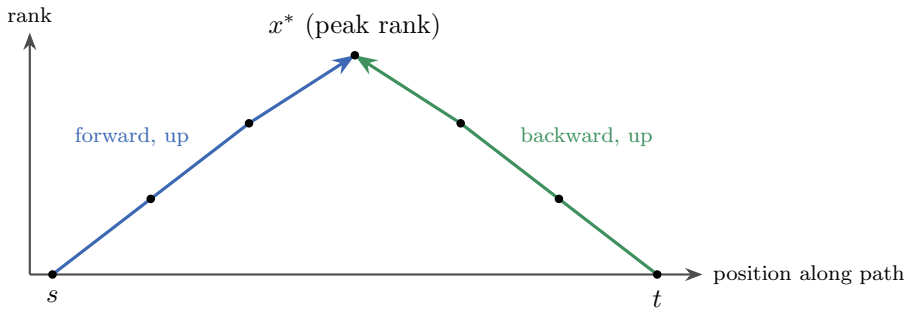


Figure 8.1: The up-down structure of a CH shortest path. Rank increases upward; the path strictly climbs from s to the highest-rank vertex x^*, then strictly descends to t. Each upward search owns one slope and they meet at x^*.

Why it works

Why an up-down path always exists. Claim: for every s, t there is a shortest s–t path in E^+ that first goes strictly up in rank, reaches a single highest-rank vertex x^*, then goes strictly down. Proof sketch by induction on contraction order. Take any shortest path P in the current graph and let v be its *lowest-rank interior* vertex, with neighbors u, w on P. When v was contracted, either a witness path of length $\le w(u,v) + w(v,w)$ already existed (so we may reroute P off v without increasing length), or a shortcut $u \to w$ was added (so we may replace $u \to v \to w$ by the single edge $u \to w$, again not increasing length). Either way we

obtain a shortest path avoiding the lowest interior vertex; repeating removes every "valley", leaving a path that only goes up then down. The peak x^* is the meeting vertex of the two upward searches.

Theorem 8.2 (CH query correctness)**.** *The bidirectional upward search returns* $\mathrm{dist}(s,t)$*: there is a vertex* x *settled by both the forward upward search and the backward upward search with* $d_f[x]+d_b[x] = \mathrm{dist}(s,t)$*, and no settled pair gives a smaller sum.*

Proof. By the up-down lemma in the box, a shortest s–t path P in E^+ decomposes as $P_\uparrow$ (from s up to the peak x^*) followed by $P_\downarrow$ (from x^* down to t). $P_\uparrow$ uses only upward edges, so the forward upward Dijkstra – which is an ordinary correct Dijkstra on the subgraph $G_\uparrow$ – settles x^* with $d_f[x^*] = w(P_\uparrow)$. Reversing $P_\downarrow$ gives an upward path in G^R from t to x^*, so the backward upward Dijkstra settles x^* with $d_b[x^*] = w(P_\downarrow)$. Hence $d_f[x^*] + d_b[x^*] = w(P) = \mathrm{dist}(s,t)$. No common vertex can give a smaller sum, because every common vertex x yields a real s–t walk of length $d_f[x] + d_b[x] \geq \mathrm{dist}(s,t)$ (distances in $G_\uparrow, G_\uparrow^R$ are upper bounds realized by actual paths in E^+, which preserves true distances). □

8.5.3 Query algorithm

Algorithm 36: Contraction-hierarchy query

input : augmented graph E^+ with ranks, source s, target t
output: $\mathrm{dist}(s,t)$

$d_f[\cdot] \leftarrow \infty,\ d_b[\cdot] \leftarrow \infty$; $d_f[s] \leftarrow 0,\ d_b[t] \leftarrow 0$; $\mu \leftarrow \infty$
forward queue $Q_f \leftarrow \{s\}$ (upward edges of E^+)
backward queue $Q_b \leftarrow \{t\}$ (upward edges of E^+ in G^R)
while $Q_f \neq \emptyset$ ***or*** $Q_b \neq \emptyset$ **do**
  foreach *direction* $\mathrm{dir} \in \{f,b\}$ *with nonempty queue and* $\mathrm{top}(Q_{\mathrm{dir}}) \leq \mu$ **do**
    $u \leftarrow \textsc{ExtractMin}(Q_{\mathrm{dir}})$
    if $d_f[u] < \infty$ ***and*** $d_b[u] < \infty$ **then**
      $\mu \leftarrow \min(\mu,\ d_f[u] + d_b[u])$
    foreach ***upward*** *edge* (u,y) *in the relevant graph* **do**
      relax y: **if** $d_{\mathrm{dir}}[u] + w(u,y) < d_{\mathrm{dir}}[y]$ **then** update and push
return μ

The pruning rule $\mathrm{top}(Q_{\mathrm{dir}}) \leq \mu$ stops each direction once its frontier key exceeds the best meeting found, by the same certificate as Theorem 8.1. Because both searches only climb in rank, their queues stay tiny – a few hundred vertices total on a continental graph – which is why CH queries run in microseconds.

8.5.4 Preprocessing: ordering and contraction

The efficiency of preprocessing hinges entirely on the *vertex order*. A bad order (e.g. contract the busiest junction first) creates a combinatorial explosion of shortcuts; a good order keeps the augmented graph sparse. The standard approach is a lazy priority queue keyed by a per-vertex *importance* score, recomputed on demand.

Algorithm 37: CH preprocessing (lazy vertex ordering with witness search)

input : graph G
output: ranks and augmented edge set E^+

$E^+ \leftarrow E$
priority queue P over all v, keyed by IMPORTANCE(v)
$r \leftarrow 0$
while $P \neq \emptyset$ **do**
  $v \leftarrow$ MIN(P)
  recompute IMPORTANCE(v) `// lazy update`
  if IMPORTANCE(v) *still minimal in* P **then**
    $r \leftarrow r + 1$; rank(v) $\leftarrow r$; remove v from P
    foreach *pair of remaining neighbors* (u, w) *with* $u \to v \to w$ **do**
      if *no witness path* $u \rightsquigarrow w$ *of length* $\leq w(u, v) + w(v, w)$ *avoiding* v **then**
        add shortcut (u, w) with weight $w(u, v) + w(v, w)$ to E^+
    update IMPORTANCE of v's former neighbors

Importance terms. The score blends several quantities, all aimed at "contracting this vertex now keeps the graph sparse and balanced":

- **Edge difference** = (shortcuts that would be added) − (edges removed). Negative or small means cheap to contract.
- **Contracted neighbors / deleted-neighbor count:** prefer to spread contraction over the graph rather than hollow out one region (uniformity).
- **Search space / hierarchy depth:** penalize contractions that make long upward chains, which would lengthen queries.

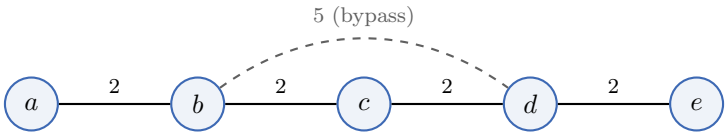
Witness search. Deciding whether a shortcut is necessary is a local shortest-path query: from u, run a Dijkstra in the current graph (excluding v) bounded by distance $w(u, v) + w(v, w)$ and a small hop limit. If w is reached within that bound, a witness exists and no shortcut is needed. The hop limit trades a few unnecessary shortcuts for much faster preprocessing – a rare unnecessary shortcut only slows queries slightly, while an unbounded witness search dominates preprocessing time.

Common trap

The lazy-queue invalidation bug. Importance scores depend on neighbors, so contracting v changes its neighbors' scores. If you contract by a *stale* priority you can pick a vertex that is no longer cheapest, producing many extra shortcuts and a slow hierarchy. The fix is the recompute-on-pop check in Algorithm 37: when v is popped, recompute its score; if it is no longer the minimum, reinsert it and pop again. Skipping this "lazy deletion" check is the single most common CH implementation mistake.

8.5.5 A worked contraction

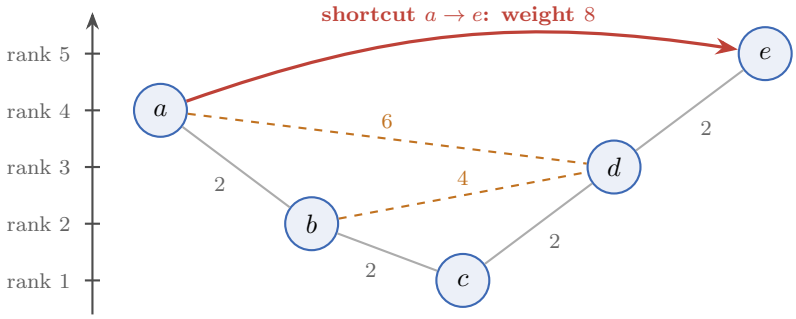
Take an undirected line-with-a-bypass, weights shown, and contract in the order c, b, d, a, e (least to most important; we hand-pick a sensible order).



The chain a–b–c–d–e has each edge of weight 2, plus a bypass b–d of weight 5 (a longer "country road"). True distances are along the chain: $\mathrm{dist}(a, e) = 8$, $\mathrm{dist}(b, d) = 4$ (via c, not the weight-5 bypass).

- **Contract c (rank 1).** Neighbors b, d; the path $b \to c \to d$ has length $2 + 2 = 4$. Witness search from b (excluding c): the bypass $b \to d$ has length $5 > 4$, so *no* witness of length ≤ 4 exists. Add shortcut $b \to d$ of weight 4. Now there are two b–d edges, weights 4 (new) and 5 (bypass); the bypass is dominated.
- **Contract b (rank 2).** Neighbors a and d (via the new shortcut). Path $a \to b \to d$ has length $2 + 4 = 6$. No alternative $a \rightsquigarrow d$ avoiding b exists, so add shortcut $a \to d$ of weight 6.
- **Contract d (rank 3).** Neighbors a (shortcut, 6) and e (2). Path $a \to d \to e = 6 + 2 = 8$. Add shortcut $a \to e$ of weight 8.
- **Contract a (rank 4), then e (rank 5).** No further shortcuts.

Plotting the vertices vertically by rank (1 at the bottom, 5 at the top) reveals the resulting contraction hierarchy E^+:



The augmented graph carries the upward shortcut $a \to e$ of weight 8. A query $\mathrm{dist}(a, e)$ now climbs from a: its only upward edge leads directly to the peak e, where it meets the trivial backward search at e and returns 8 in one hop, never walking the four chain edges. That "jump" is the highway skeleton in miniature.

Trace it yourself

Recompute the contraction with the order reversed – contract the high-degree hub b *first*. Count the shortcuts produced and compare to the four above. You should see the bad order

create more shortcuts and a deeper hierarchy: concrete evidence that CH's speedup comes primarily from the importance ordering rather than the contraction step itself.

History

Point-to-point shortest paths evolved through three distinct waves. In 1968 at the Stanford Research Institute, Peter Hart, Nils Nilsson, and Bertram Raphael invented A^* as the navigation system for Shakey the robot – the first mobile robot capable of reasoning about its own actions. Their paper proved A^* optimal among admissible algorithms: no search guaranteed to find shortest paths can expand fewer states with the same heuristic. For decades, A^* remained primarily an artificial-intelligence heuristic on spatial grids. In 2005, Andrew Goldberg and Chris Harrelson showed with ALT that precomputed triangle-inequality bounds from a handful of landmark vertices could prune Dijkstra searches by orders of magnitude on road networks.

The mid-2000s DIMACS shortest-path challenges accelerated the race. In 2008, Robert Geisberger, Peter Sanders, Dominik Schultes, and Daniel Delling at Karlsruhe introduced contraction hierarchies, distilling earlier highway hierarchies into a clean primitive: order vertices by importance, contract them, add shortcuts, and query bidirectionally upward. Contraction hierarchies became the standard engine behind open-source routing platforms such as OSRM and GraphHopper. In 2014, Julian Dibbelt, Ben Strasser, and Dorothea Wagner introduced Customizable Contraction Hierarchies (CCH), separating the contraction ordering from dynamic travel-time weights so live-traffic updates take seconds rather than hours.

8.6 When each method wins

These methods sit on a preprocessing-versus-query curve. More precomputation buys faster queries but costs memory and rebuild time when the graph changes.

method	preprocessing	query (road graph)	memory	best when
Dijkstra	none	$O((m+n)\log n)$, huge constant	$O(n)$	one-off / arbitrary graph
A^* (geometric h)	none	$<$ Dijkstra, modest gain	$O(n)$	coordinates available, single query
bidirectional	none	$\approx$ half of Dijkstra	$O(n)$	no heuristic, want a free 2×
ALT	$O(\|L\|(m+n)\log n)$	10–100× faster	$O(\|L\|\, n)$	changing weights, moderate prep
CH	minutes, $O(n \operatorname{poly}\log)$ empirically	microseconds	$O(m+\text{shortcuts})$	static road net, many queries

Focus

Decision guide.

- *Single arbitrary query, no coordinates, no time to preprocess:* bidirectional Dijkstra – a free constant-factor win, nothing to tune.
- *Have geometry, want better single-query pruning:* A^* with the straight-line heuristic; upgrade to ALT if you can afford $O(|L|n)$ tables.
- *Edge weights change often (traffic):* ALT or CCH – landmark tables and customizable orders survive perturbation; plain CH must re-contract.

- *Static graph, billions of queries (a routing service):* CH (or CRP/transit-node routing). Pay the preprocessing once; queries are essentially free.
- *Truly enormous, edges streamed or graph nonplanar with no hierarchy:* none of these may help; fall back to the foundations and consider the all-pairs and approximate methods of later chapters.

Connection

The reduced-cost trick ($w_h(u,v) = w - \pi(u) + \pi(v)$) that powers A^* is the same *potential reweighting* behind Johnson's all-pairs algorithm (Chapter 9.5) and behind the price functions of min-cost flow (later chapters): a vertex potential that makes reduced costs nonnegative so a Dijkstra-style method applies. The bidirectional stopping certificate is a direct instance of LP weak duality, which reappears in full as max-flow min-cut. And the "contract the unimportant vertex, keep distances" move of CH is a metric cousin of the vertex-elimination orderings used in nested dissection and in Cholesky/Laplacian solvers: in all of them, eliminating a low-degree vertex and adding a clique/shortcut among its neighbors preserves the quantity of interest (distance here, the Schur complement there). The hierarchy CH discovers is, loosely, a good elimination order on the road metric.

Application

Most consumer routing engines run a variant of the methods in this chapter. OSRM and GraphHopper ship CH; Google and Bing route with internal descendants of transit-node routing and CRP (customizable route planning), which add CH-style shortcuts on top of a multilevel partition so that live traffic re-customizes in under a second. Game AI and robotics still lean on A^* with hand-built heuristics, because their graphs are small and dynamic and preprocessing rarely pays off. The same ALT landmarks reappear in social-network distance estimation, where a few BFS trees from high-degree hubs give sharp distance lower bounds.

Cost summary

Cost summary.

- A^*: $O((m+n)\log n)$ worst case (degenerates to Dijkstra), but settles far fewer vertices with a sharp consistent heuristic. Correct iff h admissible; no reopening iff h consistent.
- Bidirectional Dijkstra: same asymptotics, $\approx 2\times$ fewer vertices settled on planar/road graphs; stop at $\text{top}_f + \text{top}_b \geq \mu$.
- ALT: preprocessing $O(|L|(m+n)\log n)$ time, $O(|L|n)$ space; query is A^* with $O(|L|)$-cost heuristic evaluations.
- CH: preprocessing near-linear in practice (no proven bound for general graphs; provable on graphs of bounded highway dimension); query touches $O(\text{search space})$ vertices, hundreds on continental road networks, with microsecond latency.

Every technique in this chapter accelerated a single point-to-point query $s \to t$. When an application demands shortest paths between all pairs of vertices simultaneously, launching n independent searches repeats work. The next chapter turns to all-pairs shortest paths, showing how dynamic programming and matrix multiplication over the min-plus semiring systematically solve every pair at once.

8.7 Exercises

Exercise 8.1 (Consistency by hand)**.** For the trace graph of Section 8.2.4, verify $h(u) \le w(u,v)+h(v)$ on *every* edge, then exhibit the reduced weights $w_h(u,v)$ and confirm all are nonnegative. Which edge is *tight* ($w_h = 0$)? Explain why tight edges are exactly the edges A^* "wants" to follow.

Exercise 8.2 (Admissible but inconsistent)**.** Construct a 4-vertex weighted graph and a heuristic h that is admissible but not consistent, on which A^* *without* reopening returns a wrong $\mathrm{dist}(s,t)$, yet A^* *with* reopening returns the correct value. Trace both runs.

Exercise 8.3 (A^* optimality)**.** Prove that with a consistent heuristic, A^* never expands a vertex v with $f(v) = d[v] + h(v) > \mathrm{dist}(s,t)$. Conclude that among algorithms that find shortest paths using only the same heuristic, A^* expands a minimal set of vertices (up to ties at $f = \mathrm{dist}(s,t)$).

Exercise 8.4 (First-touch is wrong)**.** Give a concrete weighted graph on which bidirectional Dijkstra's first commonly-settled vertex is *not* on any shortest s–t path, so that the naive "stop at first touch" rule reports a strictly-too-large distance. Show that the $\mathrm{top}_f + \mathrm{top}_b \ge \mu$ rule fixes it.

Exercise 8.5 (Path recovery)**.** Algorithm 34 returns only the distance μ. Extend it to return the actual shortest path. Be careful: the meeting vertex that achieved μ is where you splice the forward and backward parent pointers. Do the same for CH (Algorithm 36), where you must additionally *unpack* shortcut edges back into their two constituent edges, recursively, to recover the original-graph path.

Exercise 8.6 (Landmark sharpness)**.** Prove that the ALT heuristic $h_{\{\ell\}}(v) = \mathrm{dist}(v,\ell) - \mathrm{dist}(t,\ell)$ is *exact* ($= \mathrm{dist}(v,t)$) whenever t lies on a shortest path from v to ℓ. Use this to argue informally why "farthest" landmark placement, which pushes landmarks to the map boundary, makes some landmark nearly collinear-behind the target for most queries.

Exercise 8.7 (Implement ALT)**.** Implement ALT on a road graph (e.g. a DIMACS USA road file). Compare settled-node counts against plain Dijkstra and against A^* with the Euclidean heuristic, for $|L| \in \{1, 4, 16, 64\}$ landmarks chosen by the farthest heuristic. Plot settled nodes versus $|L|$ and find the knee.

Exercise 8.8 (CH shortcut count)**.** For the line-with-bypass graph of Section 8.5.5, compute the total number of shortcuts produced under (a) the order c, b, d, a, e used in the text and (b) the order b, d, c, a, e. Confirm that the worse order produces more shortcuts, and explain which importance term would have steered the greedy ordering away from it.

Exercise 8.9 (Witness hop limit)**.** The witness search in Algorithm 37 is bounded by a hop limit. Show that lowering the hop limit can only *add* (never remove) shortcuts, and argue that adding a spurious shortcut never makes a CH query return an *incorrect* distance – only a (slightly) slower one. Why is preprocessing time so sensitive to this limit?

Exercise 8.10 (Bidirectional A^* potentials)**.** Verify that the consistent potential pair $p_f(v) = \frac{1}{2}(h_f(v) - h_b(v))$, $p_b(v) = -p_f(v)$ from Section 8.3.4 satisfies $p_f(v) + p_b(v) = 0$ for all v, and show that under this pair the forward and backward reduced-weight graphs share a valid common stopping rule. What sharpness do you lose compared to the one-directional A^* that uses h_f directly?

Exercise 8.11 (CH versus ALT under traffic)**.** Suppose 1% of edge weights change every minute (live traffic). Estimate, in big-O terms, the per-update maintenance cost of (a) recomputing all ALT landmark tables versus (b) re-running CH preprocessing, and explain why Customizable Contraction Hierarchies were designed to sit between them. Which method would you ship for a live-traffic routing service, and why?

Chapter 9

All-Pairs Shortest Paths

The previous chapters answered a single-source question: fix a vertex s and find the distance to every other vertex. Here we want the distance between *every* ordered pair (u, v) at once – a full $n \times n$ table of shortest-path costs. This is the all-pairs shortest paths (APSP) problem, and its solutions draw on dynamic programming, matrix algebra, and reweighting.

Two strategies dominate: a global dynamic program that tests every vertex as a potential shortcut in fifteen lines of code (Floyd–Warshall), and a reweighting technique that allows us to run Dijkstra from every vertex even when edges carry negative weights (Johnson's algorithm). Both reduce all-pairs queries to systematic subproblems, connecting shortest paths to linear algebra over the min-plus semiring.

9.1 The Problem and the Multi-Source Dilemma

Focus

We are given a directed graph $G = (V, E)$ with $n = |V|$ vertices, $m = |E|$ edges, and a weight $w(u, v) \in \mathbb{R}$ on each edge. We want $\delta(u, v)$, the weight of a minimum-weight path from u to v, for all n^2 ordered pairs. Edge weights may be negative; if a negative-weight cycle lies on a path between some pair, the relevant distances are $-\infty$ and we want to *report* that rather than return garbage.

The naive reduction is tempting: run a single-source algorithm n times, once per source. On a nonnegative graph each Dijkstra run costs $O(m + n \log n)$ with a Fibonacci heap, so the total is $O(nm + n^2 \log n)$. For sparse graphs ($m = O(n)$) that is $O(n^2 \log n)$, which is excellent. But two things break this plan.

- **Negative edges.** Dijkstra is wrong the moment an edge weight is negative, because its greedy "settle the closest vertex" invariant assumes distances only grow. We would fall back to Bellman–Ford at $O(nm)$ per source, giving $O(n^2 m)$ overall – on a dense graph $m = \Theta(n^2)$ that is $O(n^4)$.
- **Dense graphs.** When $m = \Theta(n^2)$, even n Dijkstras cost $O(n^3 + n^2 \log n) = O(n^3)$. At that density a single $O(n^3)$ dynamic program with a tiny constant and contiguous memory access

– Floyd–Warshall – is simpler and usually faster in practice.

Intuition

Those two regimes call for two different algorithms. **Dense graphs, or negative edges with simple code:** Floyd–Warshall, $O(n^3)$, no heaps, cache-friendly. **Sparse graphs:** reweight once with Johnson's potentials, then run n Dijkstras for $O(nm + n^2 \log n)$. The rest of this chapter develops both methods and shows where each applies.

9.2 Floyd–Warshall: shortest paths as a dynamic program

9.2.1 The right subproblem

The main design decision in a dynamic program is what to vary. For a single path the obvious knob is its *length* (number of edges), and indeed that gives a working but slower recurrence we meet in Section 9.4. Floyd and Warshall found a cleverer knob: restrict which *intermediate* vertices a path is allowed to use.

Number the vertices $1, 2, \ldots, n$. Define

$$d^{(k)}(i, j) = \text{weight of the shortest path from } i \text{ to } j \text{ whose intermediate vertices all lie in } \{1, \ldots, k\}.$$

"Intermediate" means strictly between the endpoints; i and j themselves are always allowed. When $k = 0$ no intermediate vertices are permitted, so the only legal paths are single edges:

$$d^{(0)}(i, j) = \begin{cases} 0 & i = j, \\ w(i, j) & (i, j) \in E, \\ +\infty & \text{otherwise.} \end{cases}$$

When $k = n$ every vertex is allowed as an intermediate, so $d^{(n)}(i, j) = \delta(i, j)$ is exactly the answer we want. The algorithm computes $d^{(0)}, d^{(1)}, \ldots, d^{(n)}$ in turn.

Intuition

Think of the vertices $1, \ldots, n$ as doors that open one at a time. At stage k the first k doors are open. A path may walk through any open door as a waypoint but may not yet use doors $k{+}1, \ldots, n$. Opening door k can create only one genuinely new kind of shortcut: paths that detour *through* k. The recurrence below formalizes this observation.

9.2.2 The recurrence

Fix a pair (i, j) and suppose we know all of $d^{(k-1)}$. Now we open door k. Consider any shortest path P from i to j using intermediates in $\{1, \ldots, k\}$. Exactly one of two things is true:

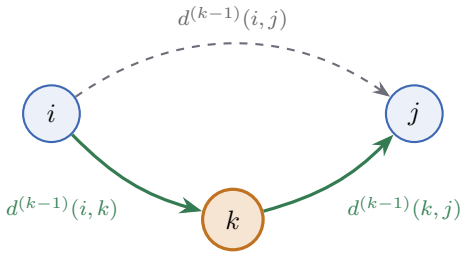
- P does not use vertex k as an intermediate. Then P already lives in $\{1, \ldots, k-1\}$, so its cost is $d^{(k-1)}(i, j)$.

- P uses vertex k once as an intermediate. (Why only once? See the trap box.) Split P at that occurrence into $i \rightsquigarrow k$ then $k \rightsquigarrow j$. Both halves use intermediates only in $\{1, \ldots, k-1\}$, so their best costs are $d^{(k-1)}(i,k)$ and $d^{(k-1)}(k,j)$.

Taking the cheaper of the two options:

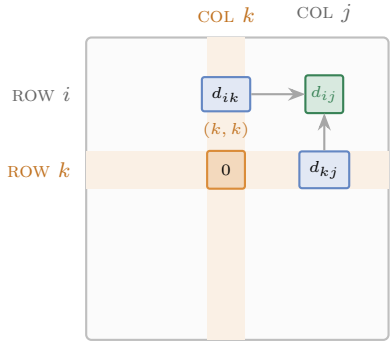
$$d^{(k)}(i,j) = \min\Big(d^{(k-1)}(i,j),\ \ d^{(k-1)}(i,k) + d^{(k-1)}(k,j) \Big). \tag{9.1}$$

Graph View: Detour through Waypoint k



$d^{(k)}(i,j) = \min\big(d^{(k-1)}(i,j),\ d^{(k-1)}(i,k) + d^{(k-1)}(k,j)\big)$

Matrix View: Cross-Product at Cell (i,j)



Row k and column k remain unchanged during step k

Common trap

Why may we assume the through-k path uses k *exactly once*? Because we can take P to be a shortest *walk*, and if there are no negative cycles a shortest walk never repeats a vertex – repeating k would mean a cycle through k, which we could delete to get a no-heavier path. If a negative cycle *is* present the recurrence still runs, but a diagonal entry $d^{(k)}(i,i)$ goes negative; detecting that is exactly how Floyd–Warshall flags negative cycles (Section 9.2.6). Note that both halves $i \to k$ and $k \to j$ avoid k as an *interior* vertex, so they are legitimately scored by $d^{(k-1)}$.

9.2.3 Collapsing the third dimension

Written as stated, $d^{(k)}$ is a stack of $n+1$ matrices – $\Theta(n^3)$ memory. We can do better. Update a single matrix d *in place*, overwriting $d^{(k-1)}$ with $d^{(k)}$ as we sweep k. The worry is that when we read $d(i,k)$ and $d(k,j)$ we might already have overwritten them with their k-th-layer values. But look at what happens to row k and column k at stage k:

$$d^{(k)}(i,k) = \min\big(d^{(k-1)}(i,k),\ d^{(k-1)}(i,k) + d^{(k-1)}(k,k)\big) = d^{(k-1)}(i,k),$$

using $d^{(k-1)}(k,k) = 0$ (true with no negative cycle). The same holds for $d^{(k)}(k,j) = d^{(k-1)}(k,j)$. So the k-th row and column *do not change* when door k opens, and reading them post- or pre-overwrite gives the same number. The in-place update is safe, and memory drops to $\Theta(n^2)$.

Algorithm 38: Floyd–Warshall (distances, predecessors, negative-cycle flag)

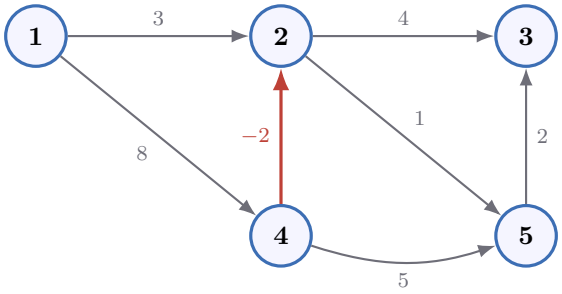
Input: Vertices $1..n$; weight w; $w(i,j) = +\infty$ if $(i,j) \notin E$.
Output: Matrix d with $d(i,j) = \delta(i,j)$; predecessor matrix π.

```
for i ← 1 to n do
    for j ← 1 to n do
        d(i,j) ← w(i,j);   d(i,i) ← 0
        if (i,j) ∈ E and i ≠ j then π(i,j) ← i
        else π(i,j) ← nil
for k ← 1 to n do
    for i ← 1 to n do
        for j ← 1 to n do
            if d(i,k) + d(k,j) < d(i,j) then
                d(i,j) ← d(i,k) + d(k,j)
                π(i,j) ← π(k,j)
for i ← 1 to n do
    if d(i,i) < 0 then report "negative cycle reachable through i"
return d, π
```

The predecessor update $\pi(i,j) \leftarrow \pi(k,j)$ records "the last edge into j on the best i-to-j path now enters j the way the best k-to-j path does." Following π backward from j reconstructs the path, exactly as in single-source algorithms.

9.2.4 A picture and a hand trace

Take a five-vertex directed graph with one negative edge (but no negative cycle).



Edges: $1\to2(3)$, $2\to3(4)$, $1\to4(8)$, $4\to2(-2)$, $2\to5(1)$, $5\to3(2)$, $4\to5(5)$. The starting matrix $d^{(0)}$ (rows = from, columns = to, $\cdot$ means $+\infty$):

$$d^{(0)} = \begin{array}{c|ccccc} & 1 & 2 & 3 & 4 & 5 \\ \hline 1 & 0 & 3 & \cdot & 8 & \cdot \\ 2 & \cdot & 0 & 4 & \cdot & 1 \\ 3 & \cdot & \cdot & 0 & \cdot & \cdot \\ 4 & \cdot & -2 & \cdot & 0 & 5 \\ 5 & \cdot & \cdot & 2 & \cdot & 0 \end{array}$$

Open door $k = 1$. Vertex 1 has no incoming edges, so no path can route through it; nothing changes. $d^{(1)} = d^{(0)}$.

Open door $k = 2$. Now paths may detour through 2. Row 4 reaches 2 cheaply via the -2 edge: $d(4,3) \leftarrow d(4,2)+d(2,3) = -2+4 = 2$, and $d(4,5) \leftarrow \min(5,\ d(4,2)+d(2,5)) = \min(5, -2+1) = -1$. Row 1 already reached 2 directly, and through-2 paths give $d(1,3) \leftarrow d(1,2) + d(2,3) = 3+4 = 7$ and $d(1,5) \leftarrow 3+1 = 4$.

$$d^{(2)} = \begin{array}{c|ccccc} & 1 & 2 & 3 & 4 & 5 \\ \hline 1 & 0 & 3 & 7 & 8 & 4 \\ 2 & \cdot & 0 & 4 & \cdot & 1 \\ 3 & \cdot & \cdot & 0 & \cdot & \cdot \\ 4 & \cdot & -2 & 2 & 0 & -1 \\ 5 & \cdot & \cdot & 2 & \cdot & 0 \end{array}$$

Open door $k = 3$. Vertex 3 is a sink (no outgoing edges), so it is never useful as an intermediate. $d^{(3)} = d^{(2)}$.

Open door $k = 4$. Who can reach 4? Only vertex 1 ($d(1,4) = 8$). Routing 1 through 4 gives $d(1,2) \leftarrow \min(3,\ 8+(-2)) = 3$, $d(1,3) \leftarrow \min(7,\ 8+2) = 7$, $d(1,5) \leftarrow \min(4,\ 8+(-1)) = 4$ – no improvements. Door 4 helps nobody here.

Open door $k = 5$. Paths may detour through 5 to reach 3 cheaply ($5 \to 3$ costs 2): $d(2,3) \leftarrow \min(4,\ d(2,5)+d(5,3)) = \min(4,\ 1+2) = 3$, $d(4,3) \leftarrow \min(2,\ d(4,5)+d(5,3)) = \min(2,\ -1+2) = 1$, and $d(1,3) \leftarrow \min(7,\ d(1,5)+d(5,3)) = \min(7,\ 4+2) = 6$.

$$d^{(5)} = \begin{array}{c|ccccc} & 1 & 2 & 3 & 4 & 5 \\ \hline 1 & 0 & 3 & 6 & 8 & 4 \\ 2 & \cdot & 0 & 3 & \cdot & 1 \\ 3 & \cdot & \cdot & 0 & \cdot & \cdot \\ 4 & \cdot & -2 & 1 & 0 & -1 \\ 5 & \cdot & \cdot & 2 & \cdot & 0 \end{array}$$

This is the final δ matrix. Sanity-check $\delta(1,3) = 6$: the path $1 \to 2 \to 5 \to 3$ costs $3+1+2 = 6$; the direct-looking $1 \to 2 \to 3$ costs $3+4 = 7$. The cheaper one wins, and it is exactly the path the $k = 5$ door opened. Floyd–Warshall needed no special handling for the negative edge $4 \to 2$.

Trace it yourself

Re-run the trace, but add the edge $3 \to 1$ with weight -10. After k reaches the vertices on the cycle $1 \to 2 \to 3 \to 1$ (cost $3+4-10 = -3$), watch $d(1,1)$ drop below 0. That negative diagonal entry signals that a negative cycle is reachable from and back to vertex 1, so the distances among those vertices are $-\infty$. Which diagonal entries go negative, and at which value of k?

9.2.5 Correctness

Theorem 9.1 (Floyd–Warshall correctness)**.** *If G has no negative-weight cycle, then after Algorithm 38 terminates $d(i,j) = \delta(i,j)$ for every pair (i,j).*

Proof. We prove by induction on k that after the outer loop has processed door k, the matrix holds $d^{(k)}(i,j)$: the shortest-path cost from i to j using intermediates only in $\{1,\dots,k\}$.

Base case $k = 0$: the initialization sets $d(i, j) = w(i, j)$ (and 0 on the diagonal), which is $d^{(0)}$ by definition – single edges are the only intermediate-free paths.

Inductive step. Assume the matrix holds $d^{(k-1)}$ before door k is processed. Fix (i, j) and consider an optimal path $P^\star$ from i to j with intermediates in $\{1, \dots, k\}$; since there is no negative cycle we may take $P^\star$ to be simple. If $k \notin P^\star$ then $P^\star$ uses only $\{1, \dots, k-1\}$ and has cost $d^{(k-1)}(i, j)$. If $k \in P^\star$, splitting at k gives two subpaths with interiors in $\{1, \dots, k-1\}$, each itself optimal among such paths (an exchange argument: a cheaper subpath would yield a cheaper $P^\star$). Their costs are $d^{(k-1)}(i, k)$ and $d^{(k-1)}(k, j)$. Hence the true optimum equals $\min(d^{(k-1)}(i, j),\ d^{(k-1)}(i, k) + d^{(k-1)}(k, j))$, which is exactly what the inner update (equation 9.1) computes. By the in-place argument above, reading $d(i, k)$ and $d(k, j)$ returns their $d^{(k-1)}$ values even though we update in place. So after door k the matrix holds $d^{(k)}$. At $k = n$ this is δ. □

9.2.6 Negative-cycle detection

Theorem 9.2. *Run Algorithm 38 on* any *weighted digraph (initializing the diagonal to* 0*). After the triple loop, G contains a negative-weight cycle if and only if $d(i, i) < 0$ for some i.*

Proof. ($\Leftarrow$) If $d(i, i) < 0$ then the recurrence found a closed walk from i back to i of negative total weight; a closed walk of negative weight contains a negative cycle.

($\Rightarrow$) Let $C = v_0 \to v_1 \to \cdots \to v_t = v_0$ be a negative cycle and let v_p be its highest-numbered vertex. Rotate C so it starts and ends at v_p. Every other vertex of C has index $< v_p$, so once door v_p is processed both subpaths $v_p \rightsquigarrow$ and $\rightsquigarrow v_p$ have interiors below v_p, and the update sets $d(v_p, v_p) \leftarrow \min(0, \text{negative total}) < 0$. Once a diagonal goes negative the min keeps it negative. So some diagonal entry is negative at the end. □

Common trap

With negative cycles present, the *off*-diagonal entries are not meaningful: a value like $d(i, j) = 7$ might really be $-\infty$ because a negative cycle sits between i and j. The clean fix is a post-pass: for every k with $d(k, k) < 0$, set $d(i, j) \leftarrow -\infty$ whenever $d(i, k) < \infty$ and $d(k, j) < \infty$ (you can reach k and leave it). Only after this pass should you trust the off-diagonal numbers.

Cost summary

Floyd–Warshall runs in $\Theta(n^3)$ time and $\Theta(n^2)$ space, independent of m. Three nested loops, one comparison and possible addition each. No priority queue, no recursion, and the inner loop is a tight scan over contiguous rows – it vectorizes well and uses the cache efficiently, which is why it routinely beats n Dijkstras on dense graphs even though the asymptotics tie.

9.3 Transitive closure: reachability with the same loop

Drop the weights and ask a yes/no question: is there *any* path from i to j? The answer is the transitive closure G^*, the reachability matrix R with $R(i, j) = 1$ iff j is reachable from i. This is

Warshall's original 1962 algorithm, and it is Floyd–Warshall with $(\min, +)$ replaced by (OR, AND):

$$R^{(k)}(i,j) = R^{(k-1)}(i,j) \ \text{ OR } \ \big(R^{(k-1)}(i,k) \ \text{AND} \ R^{(k-1)}(k,j)\big).$$

"i reaches j already, or i reaches k and k reaches j." Same door-by-door logic.

Algorithm 39: Warshall transitive closure

```
1 R(i,j) ← [ (i,j) ∈ E ] for all i,j;   R(i,i) ← 1
2 for k ← 1 to n do
3     for i ← 1 to n do
4         if R(i,k) then for j ← 1 to n do
5             R(i,j) ← R(i,j) OR R(k,j)
6
7 return R
```

The guard "**if** $R(i,k)$" hoists the test for k out of the innermost loop – when i cannot reach k, the whole j-scan is skipped.

Why it works

Why does the same triple loop solve both problems? Because both are instances of the same algebraic template: a *closed semiring*. Shortest paths use $(\mathbb{R} \cup \{\infty\}, \min, +)$ with identities ∞ and 0; reachability uses $(\{0,1\}, \text{OR}, \text{AND})$ with identities 0 and 1. Floyd–Warshall is really the Gauss–Jordan / Kleene-star computation over a semiring, and any semiring with the right absorption properties drops straight in – bottleneck paths use $(\min, \max)$, path counting uses $(+, \times)$, and regular-expression construction (Kleene's theorem) uses $(\cup, \cdot)$ over languages.

Application

Bit-packing makes closure fast in practice. Store each row of R as a machine-word bitset. The inner "$R(i,\cdot) \mid= R(k,\cdot)$" becomes a word-parallel OR over $\lceil n/w \rceil$ words, cutting the running time to $\Theta(n^3/w)$ for word size $w = 64$. This is the standard trick for reachability in compilers (which definitions reach a program point) and for the meet-in-the-middle phase of many graph queries.

9.4 The matrix-multiplication view and the min-plus product

There is a second, more algebraic, dynamic program for APSP, and it links APSP to matrix multiplication. Define

$$L^{(\ell)}(i,j) = \text{weight of the shortest walk from } i \text{ to } j \text{ using at most } \ell \text{ edges.}$$

A walk with at most ℓ edges is a walk with at most $\ell - 1$ edges followed by one more edge:

$$L^{(\ell)}(i,j) = \min_{1 \le k \le n} \Big(L^{(\ell-1)}(i,k) + w(k,j) \Big), \qquad L^{(1)}(i,j) = w(i,j).$$

This formula is matrix multiplication with $+$ replaced by min and $\times$ replaced by $+$. This is the **min-plus product** (also called the *tropical* or *distance* product). Writing $\otimes$ for it,

$$(A \otimes B)(i,j) = \min_{k} \big(A(i,k) + B(k,j) \big), \qquad L^{(\ell)} = L^{(\ell-1)} \otimes W = W^{\otimes \ell}.$$

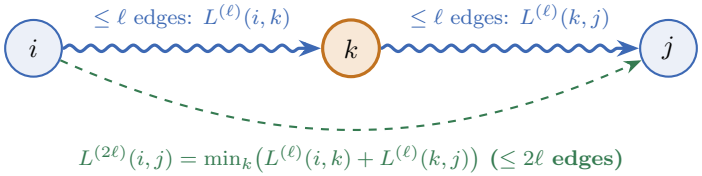
Intuition

Ordinary matrix powering counts walks: $(\mathbf{A}^\ell)(i,j)$ is the number of length-ℓ walks from i to j, because $\sum_k$ adds up ways and $\times$ combines independent choices. Swap the operations to $(\min,+)$ and "count walks" becomes "find the lightest walk" – summation over ways turns into minimization over alternatives, and multiplication of counts turns into addition of lengths. It is the exact same matrix-product loop evaluated over the $(\min,+)$ semiring.

In a graph with no negative cycle, every shortest path is simple and uses at most $n-1$ edges, so $L^{(n-1)} = \delta$. Computing $W^{\otimes(n-1)}$ by repeated squaring needs only $\lceil \log_2(n-1) \rceil$ min-plus products:

$$W^{\otimes 1},\ W^{\otimes 2},\ W^{\otimes 4},\ \ldots,\ W^{\otimes 2^{\lceil \log_2(n-1) \rceil}}.$$

Each squaring step $L^{(2\ell)} = L^{(\ell)} \otimes L^{(\ell)}$ splices two walks of at most ℓ edges at an arbitrary midpoint k, doubling the hop horizon in one matrix product:



Each $\otimes$ is the obvious triple loop at $\Theta(n^3)$, so this APSP route costs $\Theta(n^3 \log n)$ – a $\log n$ factor *worse* than Floyd–Warshall. Why bother?

Why this matters

There are two payoffs. First, writing APSP as a min-plus matrix power connects it directly to fast matrix multiplication, which is where the asymptotically fastest known algorithms come from. If one could compute a min-plus product in $O(n^{3-\varepsilon})$ time, APSP would fall below cubic; the long-standing *APSP conjecture* holds that no truly subcubic ($O(n^{3-\varepsilon})$) combinatorial algorithm exists, and a web of problems (negative triangle, min-plus product, APSP) are known to be subcubic-equivalent. Williams' bound of $n^3/2^{\Theta(\sqrt{\log n})}$ shaves off subpolynomial factors but not a true power. Second, over a *small integer* weight range the min-plus product can be embedded into a few real matrix multiplications and accelerated with Strassen-style $O(n^\omega)$ algebra (Seidel, Zwick), giving genuinely subcubic APSP on unweighted or small-weight graphs.

Connection

The min-plus / matrix-power view connects this chapter to the linear-algebra companion. Counting walks with $\mathbf{A}^\ell$ links to the spectrum of the adjacency matrix (the spectral graph theory chapter); the resolvent $(I - x\mathbf{A})^{-1} = \sum_\ell x^\ell \mathbf{A}^\ell$ is a walk generating function. Replace the field $(\mathbb{R},+,\times)$ by the semiring $(\mathbb{R}\cup\{\infty\},\min,+)$ and matrix *inversion* becomes the Kleene star $W^* = \bigoplus_{\ell \geq 0} W^{\otimes \ell}$ – which is precisely the all-pairs distance matrix. Floyd–Warshall is Gaussian elimination over this semiring. The Laplacian and electrical-flow chapters reuse this "same skeleton, different algebra" idea when they replace combinatorial distance by effective resistance.

9.5 Johnson's algorithm: reweighting for sparse graphs

Floyd–Warshall is $\Theta(n^3)$ no matter how few edges the graph has. On a sparse graph ($m = O(n)$ or so) we would much rather pay $O(nm + n^2 \log n)$ – n runs of Dijkstra. The obstacle is negative edges: Dijkstra needs nonnegative weights. Johnson's algorithm removes the obstacle by *reweighting*.

9.5.1 The potential trick

Intuition

Assign every vertex v a number $h(v)$ – a *height* or *potential*. Reweight each edge by how much it goes uphill:

$$\hat{w}(u, v) = w(u, v) + h(u) - h(v).$$

Now walk any path $u_0 \to u_1 \to \cdots \to u_t$. The heights telescope:

$$\hat{w}(\text{path}) = \sum_i \big(w(u_{i-1}, u_i) + h(u_{i-1}) - h(u_i)\big) = w(\text{path}) + h(u_0) - h(u_t).$$

Every path between a fixed pair (u_0, u_t) shifts by the *same* constant $h(u_0) - h(u_t)$. So reweighting preserves the *ordering* of paths by cost – the shortest path stays the shortest. We have not changed the problem, only the bookkeeping.

We want heights that make every reweighted edge nonnegative:

$$\hat{w}(u, v) = w(u, v) + h(u) - h(v) \geq 0 \quad \Longleftrightarrow \quad h(v) - h(u) \leq w(u, v) \quad \text{for every edge } (u, v).$$

These are exactly the constraints satisfied by shortest-path *distances* from any source: if $h(v) = \delta(s, v)$ then the triangle inequality $\delta(s, v) \leq \delta(s, u) + w(u, v)$ gives precisely $h(v) - h(u) \leq w(u, v)$. So a single-source shortest-path tree hands us a feasible potential – as long as we can compute it, which we can with Bellman–Ford, the one solver that tolerates negative edges.

Common trap

There is a catch: a source s might not reach every vertex, leaving $\delta(s, v) = \infty$ and a useless potential. Johnson's fix is a *virtual super-source* q with a zero-weight edge $q \to v$ to every vertex. Now q reaches everyone, $h(v) = \delta(q, v)$ is finite for all v, and the new edges create no negative cycle (they point only outward from q, which has no incoming edges). If Bellman–Ford from q detects a negative cycle, we abort – APSP is ill-posed.

9.5.2 The algorithm

Algorithm 40: Johnson's all-pairs shortest paths

Input: Digraph $G = (V, E)$, weights w (possibly negative).
Output: $\delta(u, v)$ for all u, v, or report a negative cycle.

Step 1 (super-source). Build G' by adding q and edges $q \to v$ of weight 0 for all $v \in V$
Step 2 (potentials). Run Bellman–Ford from q on G'
if *a negative cycle is found* **then report** and **stop**
$h(v) \leftarrow \delta_{G'}(q, v)$ for every $v \in V$
Step 3 (reweight). for *each edge* $(u, v) \in E$ **do**
    $\hat{w}(u, v) \leftarrow w(u, v) + h(u) - h(v)$ `// ≥ 0`
Step 4 (Dijkstra from each source).
for *each* $u \in V$ **do**
    run Dijkstra from u with weights $\hat{w}$ to get $\hat{\delta}(u, v)$ for all v
    for *each* $v \in V$ **do**
        $\delta(u, v) \leftarrow \hat{\delta}(u, v) - h(u) + h(v)$ `// undo the shift`
return δ

The undo step inverts the telescoping: since $\hat{\delta}(u, v) = \delta(u, v) + h(u) - h(v)$, we recover $\delta(u, v) = \hat{\delta}(u, v) - h(u) + h(v)$.

9.5.3 Worked example: same graph, reweighted

Reuse the five-vertex graph from Section 9.2.4, whose only negative edge is $4 \to 2(-2)$. Add super-source q with zero edges to all five vertices (Figure 9.1) and run Bellman–Ford from q. Each $q \to v$ edge has weight 0, so $h(v)$ is the lightest way to reach v from anywhere (through q). Relaxation propagates through the negative edge, so we follow the chains carefully:

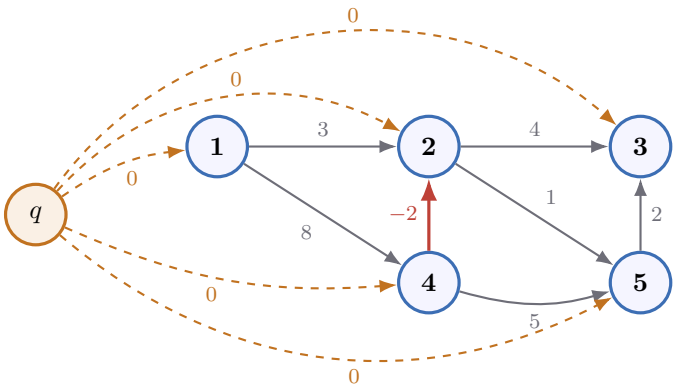


Figure 9.1: The worked graph with Johnson's super-source q (dashed orange edges of weight 0 to every vertex). q has no incoming edges, so it adds no cycle; Bellman–Ford from q yields the potentials $h(v) = \delta(q, v)$.

- $h(1) = 0$ (direct $q \to 1$; nothing reaches 1).

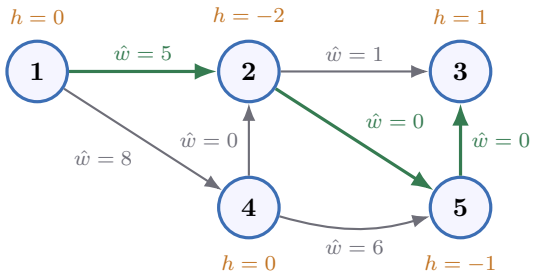
- $h(2) = -2$ via $q \to 4 \to 2$, beating the direct $q \to 2 = 0$.
- $h(5) = -1$ via $q \to 4 \to 2 \to 5 = 0 - 2 + 1$.
- $h(3) = 1$ via $q \to 4 \to 2 \to 5 \to 3 = 0 - 2 + 1 + 2$.
- $h(4) = 0$ (direct $q \to 4$; nothing else reaches 4).

v	1	2	3	4	5
$h(v)$	0	-2	1	0	-1

Now reweight every original edge with $\hat{w}(u,v) = w(u,v) + h(u) - h(v)$:

edge (u,v)	w	$\hat{w} = w + h(u) - h(v)$
$1 \to 2$	3	$3 + 0 - (-2) = 5$
$2 \to 3$	4	$4 + (-2) - 1 = 1$
$1 \to 4$	8	$8 + 0 - 0 = 8$
$4 \to 2$	-2	$-2 + 0 - (-2) = 0$
$2 \to 5$	1	$1 + (-2) - (-1) = 0$
$5 \to 3$	2	$2 + (-1) - 1 = 0$
$4 \to 5$	5	$5 + 0 - (-1) = 6$

Every reweighted edge is nonnegative – exactly as Theorem 9.3 promises – and the negative edge $4 \to 2$ became a harmless 0, as shown in the reweighted graph below:



Run Dijkstra from source 1 on $\hat{w}$. It settles $\hat{\delta}(1,2) = 5$, $\hat{\delta}(1,4) = 8$, and $\hat{\delta}(1,5) = \min(5{+}0,\ 8{+}6) = 5$ (route $1 \to 2 \to 5$). For $1 \to 3$ the contenders are $1 \to 2 \to 3$ at $5 + 1 = 6$ and $1 \to 2 \to 5 \to 3$ at $5 + 0 + 0 = 5$, so $\hat{\delta}(1,3) = 5$. Undo the shift for the $1 \to 3$ answer:

$$\delta(1,3) = \hat{\delta}(1,3) - h(1) + h(3) = 5 - 0 + 1 = 6,$$

matching the $\delta(1,3) = 6$ from Floyd–Warshall. Likewise $\delta(1,5) = \hat{\delta}(1,5) - h(1) + h(5) = 5 - 0 + (-1) = 4$ and $\delta(1,2) = 5 - 0 + (-2) = 3$. Every number agrees.

Trace it yourself

Finish the trace: run Dijkstra from source 4 on $\hat{w}$ (starting across the $4 \to 2(0)$ edge) and undo the shift with $h(4) = 0$. You should recover row 4 of the Floyd–Warshall matrix: $\delta(4,2) = -2$, $\delta(4,3) = 1$, $\delta(4,5) = -1$. Notice the reweighted Dijkstra distances are all ≥ 0 even though the true distances are negative – the potentials carry the negativity.

9.5.4 Correctness

Theorem 9.3 (Johnson reweighting)**.** *Let $h(v) = \delta_{G'}(q, v)$ be finite for all v (guaranteed by the super-source when no negative cycle exists). Then:*

(a) $\hat{w}(u, v) = w(u, v) + h(u) - h(v) \geq 0$ for every edge $(u, v) \in E$;

(b) for every pair (u, v), a path is shortest under w iff it is shortest under $\hat{w}$, and $\delta(u, v) = \hat{\delta}(u, v) - h(u) + h(v)$.

Proof. (a) Since $h(v) = \delta_{G'}(q, v)$ is a shortest-path distance, it satisfies the triangle inequality along the edge (u, v): $\delta_{G'}(q, v) \leq \delta_{G'}(q, u) + w(u, v)$, i.e. $h(v) \leq h(u) + w(u, v)$. Rearranged, $w(u, v) + h(u) - h(v) \geq 0$, which is $\hat{w}(u, v) \geq 0$.

(b) Take any path $P = (u = x_0, x_1, \ldots, x_t = v)$. Summing $\hat{w}$ along P telescopes:

$$\hat{w}(P) = \sum_{i=1}^{t} \big(w(x_{i-1}, x_i) + h(x_{i-1}) - h(x_i)\big) = w(P) + h(u) - h(v).$$

The term $h(u) - h(v)$ depends only on the endpoints, not on the path. Hence for fixed (u, v) the map $P \mapsto \hat{w}(P)$ is $w(P)$ plus a constant, so it preserves the argmin: the same path minimizes both. Taking minima, $\hat{\delta}(u, v) = \delta(u, v) + h(u) - h(v)$, and rearranging gives the recovery formula. Because every $\hat{w} \geq 0$, Dijkstra correctly computes $\hat{\delta}$. □

Why it works

The potential h is an explicit dual *certificate*. Feasibility "$h(v) - h(u) \leq w(u, v)$ for all edges" is the linear-programming dual of shortest paths; a feasible potential exists *iff* there is no negative cycle (Bellman–Ford either finds one or constructs h). Thus Johnson's algorithm solves the shortest-path dual LP once, then reuses that dual solution across all n Dijkstra runs. The same reweighting reappears in the successive-shortest-paths method for min-cost flow, where node potentials keep residual edges nonnegative so each augmentation can use Dijkstra.

Cost summary

Johnson's algorithm: Bellman–Ford once at $O(nm)$, reweighting at $O(m)$, then n Dijkstra runs. With a binary heap each Dijkstra is $O((n+m) \log n)$, total $O(nm \log n)$; with a Fibonacci heap each is $O(m + n \log n)$, total $O(nm + n^2 \log n)$. On sparse graphs $m = O(n)$ this is $O(n^2 \log n)$ – asymptotically beating Floyd–Warshall's $\Theta(n^3)$. On dense graphs $m = \Theta(n^2)$ both are $\Theta(n^3)$ (Fibonacci) and Floyd–Warshall wins on constants.

9.6 Choosing an algorithm

Situation	Algorithm	Time	Notes
Dense, any weights	Floyd–Warshall	$\Theta(n^3)$	simplest; cache-friendly
Sparse, nonnegative	$n\times$ Dijkstra	$O(nm + n^2 \log n)$	no reweighting needed
Sparse, negative edges	Johnson	$O(nm + n^2 \log n)$	reweight, then n Dijkstras
Reachability only	Warshall (bitset)	$\Theta(n^3/w)$	word-parallel OR
Small integer weights	Seidel / Zwick	$O(n^\omega \text{ poly} \log)$	via fast matmul

Intuition

A one-line decision rule. If the graph is dense or you just want the simplest correct code, write Floyd–Warshall. If it is sparse, run Dijkstra from every vertex – and if any edge is negative, prefix that with one Johnson reweighting pass. Everything else is a refinement.

History

All-pairs shortest paths grew out of work in operations research, logic, and algebra. In 1959, Bernard Roy found the intermediate-vertex dynamic programming recurrence while optimizing transit schedules, and Stephen Warshall independently published the boolean reachability version in 1962. Later that year, Robert Floyd adapted Warshall's loop to real weights in a half-page note in *Communications of the ACM*. In 1977, Donald Johnson solved the sparse-graph case: a single Bellman–Ford pass computes vertex potentials that make all edge weights nonnegative without shifting shortest paths, enabling n efficient Dijkstra runs – the same potential trick that governs min-cost flow.

The algebraic perspective links shortest paths to linear algebra. Imre Simon coined the term *tropical semiring* for min-plus algebra. Raimund Seidel (1995) and Uri Zwick (2002) used fast matrix multiplication over small integer weights to break the $O(n^3)$ cubic barrier. For general real weights, the long-standing APSP conjecture posits that no truly subcubic $O(n^{3-\varepsilon})$ combinatorial algorithm exists. In 2014, Ryan Williams evaluated polynomials over circuit-complexity representations to compute real-weighted APSP in $n^3/2^{\Omega(\sqrt{\log n})}$ time, but whether general real-weighted APSP can ever reach $O(n^{3-\varepsilon})$ remains an open problem in fine-grained complexity.

9.7 Path reconstruction and a few extensions

Recovering the actual paths. Both algorithms compute full paths alongside numerical costs. In Floyd–Warshall the predecessor matrix π (Algorithm 38) lets you walk backward from j: repeatedly replace j by $\pi(i, j)$ until you reach i. In Johnson's algorithm each Dijkstra already produces a shortest-path tree; reweighting changes edge costs but not path *structure* (Theorem 9.3(b)), so the tree edges are the genuine shortest paths.

Bottleneck / widest paths. Replace $(\min, +)$ by $(\min, \max)$ in the Floyd–Warshall update – $d(i, j) \leftarrow \min(d(i, j), \max(d(i, k), d(k, j)))$ – and you compute the minimax path between every pair,

the path whose heaviest edge is as light as possible. This is the all-pairs minimum-bottleneck problem (and with $(\max, \min)$, maximum-capacity), useful in network routing.

Counting shortest paths. Carry a count matrix c alongside d. When the update strictly improves $d(i,j)$, set $c(i,j) \leftarrow c(i,k) \cdot c(k,j)$; when it *ties* the current best, add $c(i,j)$ += $c(i,k) \cdot c(k,j)$. This computes the number of distinct shortest paths between every pair, a quantity central to betweenness-centrality computations on networks.

Application

APSP distance matrices feed graph-analytic measures directly. The *Wiener index* $\sum_{i<j} \delta(i,j)$ measures molecular branching in chemistry. *Closeness centrality* $1/\sum_j \delta(i,j)$ ranks how central a vertex is; *betweenness* counts shortest paths through each vertex (the counting extension above). The graph *diameter* $\max_{i,j} \delta(i,j)$ and *radius* fall out of one scan of the matrix. Whenever a pipeline needs "distance between everything," it calls an APSP solver.

All-pairs shortest paths tell us how efficiently information or traffic moves when the entire network is intact. In real-world networks, however, components fail: a single broken link or disabled router can shatter a connected topology into isolated pieces. The next chapter shifts focus from travel distances to structural vulnerability, using depth-first search to uncover the bridges and articulation points whose removal disconnects the graph.

9.8 Exercises

Exercise 9.1. Implement Floyd–Warshall with the predecessor matrix π of Algorithm 38 and a path reconstruction routine. Test it on the five-vertex graph of Section 9.2.4 and confirm it reports the path $1 \to 2 \to 5 \to 3$ for the pair $(1,3)$.

Exercise 9.2. Prove that the in-place update is safe by showing rigorously that row k and column k of d are unchanged during the processing of door k, and explain exactly where the assumption $d^{(k-1)}(k,k) = 0$ is used. What goes wrong if a negative cycle through k exists?

Exercise 9.3. Add the edge $3 \to 1$ with weight -10 to the worked graph, creating the cycle $1 \to 2 \to 3 \to 1$ of weight -3. Trace Floyd–Warshall and identify the first value of k at which some diagonal entry $d(i,i)$ becomes negative. Then implement the $-\infty$ post-pass from the trap box in Section 9.2.6 and report which off-diagonal entries it corrects.

Exercise 9.4. (Min-plus by squaring.) Implement APSP via repeated squaring of the min-plus product (Section 9.4). Verify it agrees with Floyd–Warshall on random graphs, and confirm empirically that it performs $\lceil \log_2(n-1) \rceil$ products. Why does ordinary squaring fail to detect negative cycles the way Floyd–Warshall's diagonal does?

Exercise 9.5. Prove that the min-plus product is associative, so that $W^{\otimes a} \otimes W^{\otimes b} = W^{\otimes (a+b)}$ and repeated squaring is valid. Where in the proof do you use that $(\mathbb{R} \cup \{\infty\}, \min, +)$ is a semiring (in particular, that $+$ distributes over min)?

Exercise 9.6. Show that a feasible potential h with $h(v) - h(u) \le w(u,v)$ for all edges exists if and only if G has no negative cycle. (Hint: one direction is Johnson's super-source construction; for the other, sum the inequalities around a cycle and watch the potentials cancel.)

Exercise 9.7. In Johnson's algorithm, suppose you skip the super-source and instead pick an arbitrary vertex s as the Bellman–Ford source. Construct a small graph where this produces an infinite potential for some vertex and breaks the reweighting. Then argue that the super-source never introduces a negative cycle.

Exercise 9.8. (Bottleneck APSP.) Modify Floyd–Warshall to compute, for every pair, the minimum possible value of the *maximum* edge weight on a path (the minimax path). Prove your recurrence correct by an exchange argument analogous to Theorem 9.1, and state the semiring you are working over.

Exercise 9.9. (Counting shortest paths.) Augment Floyd–Warshall with the count matrix c described in Section 9.7. Prove that on a graph with nonnegative weights and no zero-weight cycles the final $c(i, j)$ equals the number of distinct shortest paths from i to j. Explain why zero-weight cycles can make this count infinite.

Exercise 9.10. (Transitive closure as a special case.) Show that running Floyd–Warshall with edge weights 0 on present edges and $+\infty$ on absent ones, then thresholding $d(i, j) < \infty$, recovers the transitive closure. Compare its constant factor with the bit-packed Warshall closure of Algorithm 39 and explain when the bitset version wins.

Exercise 9.11. (Incremental APSP.) After computing the full distance matrix, a single edge weight $w(a, b)$ is *decreased*. Give an $O(n^2)$ algorithm to update all n^2 distances, and prove it correct. (Hint: the only paths that can improve route through (a, b); relax $d(i, j) \leftarrow \min(d(i, j),\ d(i, a) + w(a, b) + d(b, j))$.) Why does an edge *increase* not admit the same easy update?

Chapter 10

Bridges, Articulation Points, and Biconnectivity

The previous chapter calculated all-pairs shortest paths assuming an intact network. In real networks, however, single components fail: a severed cable or disabled switch can disconnect two communities regardless of how short their previous paths were.

The edges and vertices whose removal disconnects a graph are its structural bottlenecks — bridges and articulation points. A single depth-first search finds every one of them in linear time using a single numeric invariant: the *low-link* value, tracking the highest ancestor reached through back edges. By recording this value in post-order, we can detect bridges and articulation points and decompose the network into biconnected blocks, the basic units of network reliability analysis.

10.1 What we are looking for

Throughout, $G = (V, E)$ is an *undirected* graph with $n = |V|$ vertices and $m = |E|$ edges. We assume G is connected unless stated otherwise; if it is not, every algorithm below runs once per connected component and we union the results.

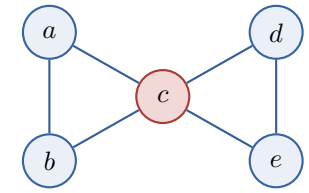
Definition 10.1 (Articulation point)**.** A vertex $v \in V$ is an *articulation point* (or *cut vertex*) if deleting v together with all edges incident to it increases the number of connected components of G. Equivalently, there exist vertices $x, y \neq v$ such that every path from x to y passes through v.

Definition 10.2 (Bridge)**.** An edge $e \in E$ is a *bridge* (or *cut edge*) if deleting e (but keeping both endpoints) increases the number of connected components. Equivalently, $e = \{u, w\}$ is a bridge iff there is no path from u to w in $G - e$, i.e. e lies on no cycle.

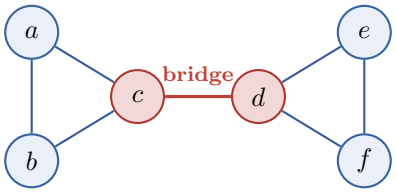
Intuition

A bridge is an edge with no "backup route." If you can get from one endpoint to the other by some other path, that path plus the edge forms a cycle, and removing one edge of a cycle never disconnects anything. So *bridges are precisely the edges that lie on no cycle.* Articulation points are the vertex analogue: a vertex is a cut vertex if some pair of its neighbors can only see each other "through" it.

These two notions are siblings but not identical. The endpoint of a bridge is often, but not always, an articulation point, and an articulation point need not be incident to any bridge. The smallest example that shows the difference is a "bowtie": two triangles sharing a single vertex.



Bowtie: cut vertex c, no bridges



Dumbbell: bridge $\{c, d\}$, cut vertices c, d

Here c is an articulation point (deleting it separates $\{a, b\}$ from $\{d, e\}$), yet *no* edge is a bridge, because every edge lies on a triangle. Conversely, a single edge joining two triangles is a bridge, and both of its endpoints are articulation points. Keep this asymmetry in mind: the algorithms for the two problems share an engine but differ in one comparison ($<$ versus $\leq$).

Definition 10.3 (Biconnected; 2-connected)**.** A connected graph with at least three vertices is *biconnected* (equivalently 2-*vertex-connected*) if it has no articulation point: removing any single vertex leaves it connected. A connected graph is 2-*edge-connected* if it has no bridge: removing any single edge leaves it connected.

Focus

Three connectivity questions, one DFS:

- Which *edges* are bridges? (2-edge-connectivity is "no bridges.")
- Which *vertices* are articulation points? (2-vertex-connectivity is "no cut vertices.")
- How does the graph decompose into maximal pieces that have neither weakness (the *biconnected components*), and how do those pieces fit together (the *block-cut tree*)?

All three are answered by computing, for every vertex, two integers: its discovery time and its low-link value.

10.2 The DFS tree and why it has no cross edges

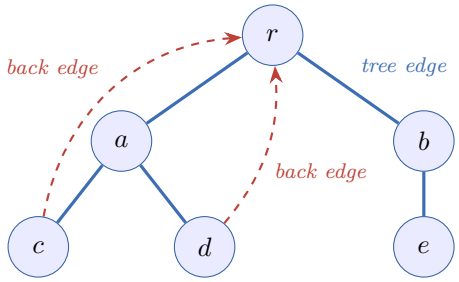
All the algorithms in this chapter rely on one fact about depth-first search in undirected graphs. Run DFS from a root r. As we explore, we assign each vertex a *discovery time* $\mathrm{disc}[v]$, the order in which DFS first visits it $(1, 2, 3, \dots)$. The edges split into two kinds relative to the resulting rooted tree:

- **Tree edges**: the edge $\{u, v\}$ along which DFS first reached v from u. These form the DFS tree T.
- **Back edges**: every other edge. Such an edge $\{u, v\}$ always joins a vertex to one of its *ancestors* in T.

Why it works

No cross edges in undirected DFS. Suppose, when DFS is exploring u, it looks at a non-tree edge $\{u, v\}$. Because the edge exists, v was discovered at some point. If v were neither an ancestor nor a descendant of u, then v would have been fully finished before u started or vice versa; but if v finished before u started, the edge $\{u, v\}$ would have been examined from v's side and would have made u a descendant of v — contradiction. Working through the two cases (which endpoint is discovered first) shows the only possibility for a non-tree edge is that one endpoint is an ancestor of the other. Hence every non-tree edge of an undirected DFS is a *back edge*. This is the property that makes low-link work; it fails for directed graphs, where cross and forward edges appear.

Because there are no cross edges, the cycle structure of G is completely captured by back edges: every cycle uses exactly one back edge together with the tree path between its endpoints. The algorithms below rely on this property.



The solid edges are the tree; the dashed edges go from a vertex up to a proper ancestor. The back edge $c \to r$ creates the cycle r, a, c, r; the back edge $d \to r$ creates the cycle r, a, d, r. Notice $b \to e$ is a tree edge with *no* back edge reaching above b: that tree edge is a bridge.

10.3 Low-link: the highest reachable ancestor

Definition 10.4 (Low-link)**.** Fix a DFS tree. For a vertex v, define

$$\text{low}[v] \;=\; \min\Big(\{\text{disc}[v]\} \;\cup\; \{\text{disc}[w] : \{v, w\} \text{ a back edge}\} \;\cup\; \{\text{low}[c] : c \text{ a child of } v \text{ in } T\}\Big).$$

In words, low$[v]$ is the smallest discovery time reachable from v by going down through zero or more tree edges in the subtree of v and then taking at most one back edge.

Intuition

Read low$[v]$ as: "Starting at v, staying inside my own subtree, and allowed to climb exactly one back edge at the end, how high up the tree can I get?" Low discovery time means high in the tree (the root has time 1). If the highest I can reach is strictly above my parent, my subtree is anchored to the rest of the graph even without the edge from my parent — there is a detour. If I cannot reach above my parent, that single tree edge is the only thing tying my subtree in: cut it and I fall off.

The definition is recursive and bottom-up, which is exactly how DFS finishes vertices: a child's low is known by the time we return to the parent. We compute it during the post-order phase of the same DFS that assigns discovery times.

Algorithm 41: DFS-LowLink: discovery times and low-link in one pass

input : undirected graph $G = (V, E)$ via adjacency lists; global counter $t \leftarrow 0$
output: arrays disc[·] and low[·]

```
1  function Dfs(u, p)
2      t ← t + 1;   disc[u] ← t;   low[u] ← t;
3      foreach w in adj[u] do
4          if edge to w is the (single) parent edge to p then
5              continue;                    // skip exactly one copy of the parent edge
6          if disc[w] = 0 then
7              Dfs(w, u);                                              // tree edge
8              low[u] ← min(low[u], low[w]);
9          else
10             low[u] ← min(low[u], disc[w]);                  // back edge to ancestor w
11 foreach v ∈ V with disc[v] = 0 do
12     Dfs(v, nil)
```

Common trap

The parent edge, and the multigraph subtlety. When scanning u's neighbors we must not treat the edge back to the parent p as a back edge — otherwise low[u] would always reach disc[p] and we would miss every bridge. The naive guard "if $w = p$ skip" is *wrong* when the graph has parallel edges: if there are *two* edges between u and p, one is the tree edge and the other is a genuine back edge that makes $\{u, p\}$ a non-bridge. The correct rule is to skip exactly *one* occurrence of the parent edge, which is why robust implementations track edges by their index rather than by the neighbor label. With distinct neighbor labels and a simple graph, the "skip if $w = p$" shortcut is fine; with multigraphs, identify edges, not vertices.

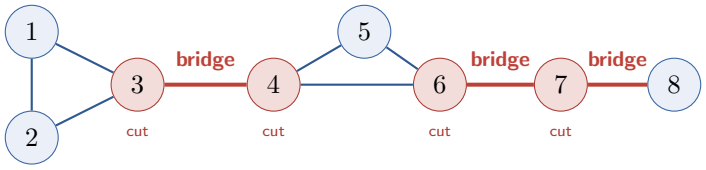
Why it works

Correctness of the recurrence. We prove by induction on finishing order that when Dfs$(u, \cdot)$ returns, low[u] equals the minimum discovery time over all vertices reachable from u by a path that stays in u's subtree except for a single final back edge. The initialization sets low[u] = disc[u] (the empty path). Each back edge $\{u, w\}$ to an ancestor contributes disc[w] directly. Each tree child c is finished before u returns, so by the inductive hypothesis low[c] is already the correct subtree minimum for c; folding it in via min extends the reachable set through c's subtree. Since every in-subtree "descend then climb once" path either is empty, ends with a back edge from u itself, or passes through exactly one child c and continues inside c's subtree, the three cases are exhaustive. Hence the returned value is exactly the claimed minimum.

10.3.1 Tracing low-link by hand

Let us run Algorithm 41 on a concrete graph and watch the numbers. Take vertices $\{1, \dots, 8\}$ with edges

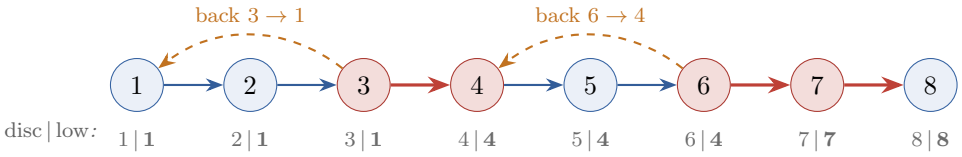
$$\{1,2\}, \{1,3\}, \{2,3\}, \{3,4\}, \{4,5\}, \{4,6\}, \{5,6\}, \{6,7\}, \{7,8\}.$$



The graph is a triangle $\{1,2,3\}$, joined by edge $\{3,4\}$ to a triangle $\{4,5,6\}$, with a tail 6–7–8 hanging off. The thick red edges $\{3,4\}$, $\{6,7\}$, $\{7,8\}$ are the bridges, as we will confirm. We run DFS from vertex 1, always visiting neighbors in increasing label order. The tree edges discovered are $1 \to 2$, $2 \to 3$, $3 \to 4$, $4 \to 5$, $5 \to 6$, $6 \to 7$, $7 \to 8$; the back edges encountered are $3 \to 1$ (closing the first triangle) and $6 \to 4$ (closing the second).

v	1	2	3	4	5	6	7	8
disc$[v]$	1	2	3	4	5	6	7	8
low$[v]$	1	1	1	4	4	4	7	8

The resulting DFS tree is a single spine $1 \to 2 \to \cdots \to 8$ with two back edges $3 \to 1$ and $6 \to 4$; each back edge pulls low up to the ancestor's discovery time across the cycle it closes:



Walk the post-order. Vertex 8 is a leaf with no back edge: low$[8] = 8$. Then 7: its only non-parent neighbor is 8 (a child), so low$[7] = \min(7, \text{low}[8]) = 7$. Then 6: it has child 7 (low$[7] = 7$) and a back edge to 4 (disc$[4] = 4$), so low$[6] = \min(6, 7, 4) = 4$. Vertex 5: child 6 gives low$[5] = \min(5, 4) = 4$. Vertex 4: child 5 gives low$[4] = \min(4, 4) = 4$. Vertex 3: child 4 (low$[4] = 4$) and back edge to 1 (disc$[1] = 1$), so low$[3] = \min(3, 4, 1) = 1$. Vertex 2: child 3 gives $\min(2, 1) = 1$. Vertex 1: child 2 gives $\min(1, 1) = 1$.

Trace it yourself

Redo this trace but start DFS from vertex 4 and visit neighbors in *decreasing* label order. The discovery times change completely, the tree reshapes, yet the set of bridges you read off ($\{3,4\}, \{6,7\}, \{7,8\}$) and the set of articulation points must come out identical. Bridges and cut vertices are properties of G, not of the traversal — a good sanity check that you have implemented the comparisons correctly.

10.4 Finding bridges

Theorem 10.1 (Bridge criterion)**.** *Let* $\{u, v\}$ *be a* tree edge *with* v *the child. Then* $\{u, v\}$ *is a bridge if and only if*

$$\text{low}[v] > \text{disc}[u].$$

No back edge is ever a bridge.

Why it works

Why the comparison is strict. The subtree rooted at v is attached to the rest of G only through the tree edge $\{u, v\}$ *unless* some vertex in v's subtree has a back edge climbing to u or above u. The quantity $\text{low}[v]$ is exactly the highest discovery time any such climb reaches. "Above u or to u" means $\text{low}[v] \leq \text{disc}[u]$ — a detour exists, no bridge. "Strictly below u," i.e. $\text{low}[v] > \text{disc}[u]$, means every escape from the subtree must use the edge $\{u, v\}$ itself, so deleting it severs the subtree. A back edge $\{x, y\}$ can never be a bridge because it closes a cycle (the tree path from x to y plus the edge), and no edge of a cycle is a bridge.

Formal

Proof of Theorem 10.1. ($\Leftarrow$) Suppose $\text{low}[v] > \text{disc}[u]$. Let S be the set of vertices in v's subtree. Every edge of G with one endpoint in S is either the tree edge $\{u, v\}$, an internal tree edge of the subtree, or a back edge from some $x \in S$ to an ancestor y. For such a back edge, $\text{disc}[y] \geq \text{low}[x] \geq \text{low}[v] > \text{disc}[u]$, so y is a *descendant* of u strictly below u; since y is an ancestor of $x \in S$ and lies below u, in fact $y \in S$. Therefore the only edge crossing from S to $V \setminus S$ is $\{u, v\}$. Removing it disconnects S from u; it is a bridge.
($\Rightarrow$) Suppose $\text{low}[v] \leq \text{disc}[u]$. By definition of low there is a vertex x in v's subtree with a back edge to an ancestor y satisfying $\text{disc}[y] = \text{low}[v] \leq \text{disc}[u]$, so y is u or a proper ancestor of u. Then the path

$$u \;\to\; v \;\rightsquigarrow\; x \;\to\; y \;\rightsquigarrow\; u$$

(tree edge, tree path down to x, back edge to y, tree path from y back to u) is a closed walk containing $\{u, v\}$ and using it once. Extracting a simple cycle through $\{u, v\}$ shows the edge lies on a cycle, hence is not a bridge. □

Algorithm 42: BRIDGES: list all bridges in one DFS

```
   output: set B of bridge edges
 1 B ← ∅;  t ← 0;  disc[·] ← 0;
 2 function DFS(u, p_e) ;                          // p_e = edge index used to enter u
 3
 4     t ← t + 1; disc[u] ← low[u] ← t;
 5     foreach edge e = {u, w} in adj[u] with index i do
 6         if i = p_e then continue;
 7         if disc[w] = 0 then
 8             DFS(w, i);  low[u] ← min(low[u], low[w]);
 9             if low[w] > disc[u] then B ← B ∪ {e};
10         else
11             low[u] ← min(low[u], disc[w]);
12 foreach v ∈ V with disc[v] = 0 do DFS(v, −1);
```

On the traced graph: edge $3 \to 4$ has $\text{low}[4] = 4 > \text{disc}[3] = 3$ — bridge. Edge $6 \to 7$ has $\text{low}[7] = 7 > \text{disc}[6] = 6$ — bridge. Edge $7 \to 8$ has $\text{low}[8] = 8 > \text{disc}[7] = 7$ — bridge. Every other tree edge fails the test (e.g. $5 \to 6$ has $\text{low}[6] = 4 \leq 5$), matching the red edges in the picture.

10.5 Finding articulation points

The vertex version uses the same low array with two changes: a non-strict comparison, and a special rule for the root.

Theorem 10.2 (Articulation-point criterion)**.** *Run DFS from a chosen root r.*

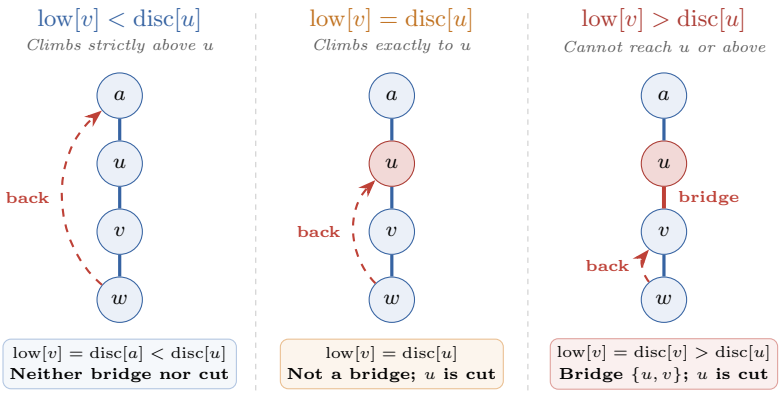
1. *The root r is an articulation point if and only if it has* ***two or more*** *children in the DFS tree.*

2. *A non-root vertex u is an articulation point if and only if it has a child v in the DFS tree with*

$$\text{low}[v] \;\geq\; \text{disc}[u].$$

Intuition

Compare with the bridge test $\text{low}[v] > \text{disc}[u]$. For an edge to be a bridge, the child's escape must land *strictly below* u. For a *vertex* u to be a cut vertex, the child's escape only needs to fail to climb *above* u — landing exactly *at* u ($\text{low}[v] = \text{disc}[u]$) still traps the subtree behind u, because the only way out passes through u itself. Hence $\geq$ for vertices, $>$ for edges. The root is special because it has no parent edge to escape "upward" through; it is a cut vertex exactly when it branches the tree into two or more subtrees that can only meet at r.



Formal

Proof of Theorem 10.2. *Root.* If r has children v_1, v_2 in T, no edge joins the subtree of v_1 to that of v_2 (such an edge would be a non-tree, non-back edge — a cross edge — which undirected DFS forbids). So every v_1–v_2 path uses r; deleting r separates them. Conversely if r has a single child, its whole subtree is the rest of the graph and is connected without r. *Non-root u.* ($\Leftarrow$) Suppose a child v has $\text{low}[v] \geq \text{disc}[u]$. Every back edge leaving v's subtree reaches a vertex with discovery time $\geq \text{disc}[u]$, i.e. u or a descendant of u; none reaches a proper ancestor of u. Thus every path from v's subtree to the part of G above u passes through u. Deleting u separates v's subtree (nonempty, not containing u) from u's parent — so u is a cut vertex. ($\Rightarrow$) Suppose every child v has $\text{low}[v] < \text{disc}[u]$. Then each child's subtree has a back edge to a proper ancestor of u, giving a detour around u that connects that subtree to the part above u. Removing u, all of u's child subtrees stay connected to the ancestor portion (which is connected among itself), so the graph minus u stays connected; u is not a cut vertex. □

Algorithm 43: ARTICULATION: list all cut vertices in one DFS

output: set A of articulation points

```
1  A ← ∅; t ← 0; disc[·] ← 0;
2  function DFS(u, p, isRoot)
3      t ← t + 1; disc[u] ← low[u] ← t; kids ← 0;
4      foreach w ∈ adj[u] do
5          if w = p then continue;
6          if disc[w] = 0 then
7              kids ← kids + 1; DFS(w, u, FALSE);
8              low[u] ← min(low[u], low[w]);
9              if not isRoot and low[w] ≥ disc[u] then A ← A ∪ {u};
10         else
11             low[u] ← min(low[u], disc[w]);
12     if isRoot and kids ≥ 2 then A ← A ∪ {u};
13 foreach v ∈ V with disc[v] = 0 do DFS(v, NIL, TRUE);
```

On the traced graph, root = 1 has a single child, so it is not a cut vertex. Checking non-roots: $u = 3$ has child 4 with $\text{low}[4] = 4 \geq \text{disc}[3] = 3$ — articulation point. $u = 4$ has child 5 with $\text{low}[5] = 4 \geq \text{disc}[4] = 4$ — articulation point. $u = 6$ has child 7 with $\text{low}[7] = 7 \geq 6$ — articulation

point. $u = 7$ has child 8 with $\text{low}[8] = 8 \geq 7$ — articulation point. The cut vertices are $\{3, 4, 6, 7\}$. Vertices 2 and 5 are not (their children climb above them); 1 and 8 are not (root with one child; leaf).

Common trap

Do not apply the $\geq$ rule to the root. If you forget the root special case and use $\text{low}[v] \geq \text{disc}[r]$ on the root, every root with at least one child reports as a cut vertex, since $\text{disc}[r] = 1$ is the global minimum and any $\text{low}[v] \geq 1$. The root genuinely needs the "two children" test. Symmetrically, do not count a child twice in a multigraph: increment kids only on actual tree edges.

10.6 Biconnected components and the block-cut tree

Articulation points and bridges are the weak *spots*; the biconnected components are the maximal *strong pieces* between them.

Definition 10.5 (Biconnected component / block)**.** Define a relation on edges: $e \sim f$ if $e = f$ or e and f lie on a common simple cycle. This is an equivalence relation, and its classes are the *biconnected components* (or *blocks*). Each block is either a maximal 2-connected subgraph or a single bridge edge.

Why it works

Why "share a cycle" is an equivalence relation. Reflexivity and symmetry are immediate. Transitivity is the content: if e, f share a cycle C_1 and f, g share a cycle C_2, then e and g share a cycle. This is a small but real lemma — the two cycles overlap in at least the edge f, and one can splice them along their shared portion into a single cycle containing both e and g. (Splicing along their maximal shared path, rather than assuming a single shared edge, ensures the combined simple cycle encloses both e and g even when C_1, C_2 meet across multiple vertices.) Consequently blocks *partition the edges*. Vertices, by contrast, are *not* partitioned: an articulation point belongs to several blocks at once — it is exactly the vertex shared by two or more of them.

So edges split cleanly into blocks, and the articulation points are precisely the vertices lying in more than one block. A bridge is a block all by itself (a K_2 that is on no cycle). This is the cleanest way to see the relationship: *bridges are the single-edge blocks; articulation points are the shared vertices between blocks.*

10.6.1 Extracting blocks with an edge stack

The same DFS that computes low peels off blocks using an auxiliary stack of edges. Push each edge when first traversed. When the articulation test $\text{low}[w] \geq \text{disc}[u]$ fires at u for child w, every edge currently on the stack down to and including $\{u, w\}$ forms one complete block; pop them off.

Algorithm 44: BiconnectedComponents: peel blocks with an edge stack

```
   output: list of blocks, each a set of edges
 1 t ← 0; disc[·] ← 0; stack S ← empty;
 2 function Dfs(u, p)
 3     t ← t + 1; disc[u] ← low[u] ← t;
 4     foreach w ∈ adj[u] via edge e do
 5         if e is the parent edge then continue;
 6         if disc[w] = 0 then
 7             push e onto S;
 8             Dfs(w, u); low[u] ← min(low[u], low[w]);
 9             if low[w] ≥ disc[u] then
10                 start a new block; pop edges from S until e is popped (inclusive); emit the
                     block;
11         else if disc[w] < disc[u] then
12             push e onto S; low[u] ← min(low[u], disc[w]);          // back edge
13 foreach v ∈ V with disc[v] = 0 do Dfs(v, nil);
```

Why it works

Why the stack respects block boundaries. Edges are pushed in DFS order, so the stack always holds a contiguous "frontier" of edges descending from the current search path. When $\text{low}[w] \geq \text{disc}[u]$, the subtree of w cannot reach above u, so no future edge will ever connect into w's subtree from higher up: every edge of the block that $\{u, w\}$ belongs to has already been pushed and sits at the top of the stack, ending exactly at $\{u, w\}$. Popping down to $\{u, w\}$ therefore emits one complete block and nothing more. The condition matches the $\geq$ test for articulation points: an edge block terminates at u precisely when u serves as the shared cut vertex connecting that block to the rest of the graph.

On our running graph the blocks come out as

$$\{12, 13, 23\} \text{ (triangle)}, \quad \{34\} \text{ (bridge)}, \quad \{45, 46, 56\} \text{ (triangle)}, \quad \{67\} \text{ (bridge)}, \quad \{78\} \text{ (bridge)},$$

five blocks: two triangles and three single-edge bridges. The cut vertices $3, 4, 6, 7$ are exactly the vertices appearing in two blocks each.

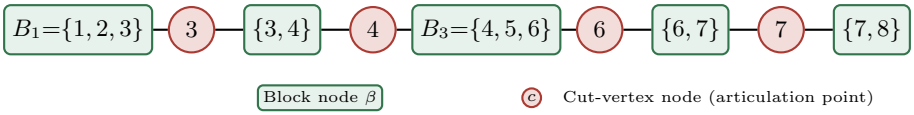
10.6.2 The block-cut tree

The blocks and cut vertices assemble into a tree that records the coarse shape of the graph.

Definition 10.6 (Block-cut tree)**.** The *block-cut tree* $\mathcal{T}$ of a connected graph G has one node per block and one node per articulation point. Connect a cut-vertex node c to a block node β iff c is a vertex of β. The result is a tree (a forest if G is disconnected).

Why it works

Why it is acyclic. Suppose $\mathcal{T}$ contained a cycle. That cycle alternates between cut-vertex nodes and block nodes. Splicing the cycles inside the participating blocks together along the shared cut vertices would merge them all into a single biconnected piece — contradicting that they were *distinct maximal* blocks. So $\mathcal{T}$ has no cycle, and since G is connected $\mathcal{T}$ is connected; hence it is a tree. Its leaves are always blocks (a cut vertex has degree ≥ 2 in $\mathcal{T}$ by definition), a fact used in many reliability and augmentation arguments.



The block-cut tree of the running graph is a path: triangle B_1 – cut 3 – bridge – cut 4 – triangle B_3 – cut 6 – bridge – cut 7 – bridge $\{7, 8\}$. Square nodes are blocks, round red nodes are cut vertices. Reading the tree, the bridges are exactly the single-edge block nodes, and the cut vertices are exactly the round nodes — so the tree shows the graph's connectivity structure at a glance.

Application

The block-cut tree turns "what fails if vertex v dies?" into a tree query. The number of components created by deleting a cut vertex c equals its degree in $\mathcal{T}$. Two query vertices x, y are separated by deleting c iff c lies on the $\mathcal{T}$-path between the blocks containing x and y. Many "is there a path avoiding v" or "which articulation point disconnects x from y" questions reduce to LCA and path queries on $\mathcal{T}$ (see the LCA chapter), answering each in $O(\log n)$ or $O(1)$ after linear preprocessing.

10.7 Complexity

Theorem 10.3. *Bridges, articulation points, all biconnected components, and the block-cut tree can each be computed in $O(n + m)$ time and $O(n)$ extra space (plus $O(m)$ for the block lists or the edge stack).*

Formal

Proof. All four algorithms are a single DFS over G with $O(1)$ extra work per edge examination. Each vertex is entered once (the disc guard) and each edge is scanned at most twice, once from each endpoint, doing constant work: updating a min, comparing two integers, or pushing/popping the edge stack. Summing, the traversal is $O(n+m)$. The arrays disc and low use $O(n)$ words; the edge stack holds at most m edges and each edge is pushed and popped exactly once, so its total handling cost is $O(m)$ and it never exceeds $O(m)$ space. The block-cut tree has at most n cut-vertex nodes and at most m block nodes, so it is built in $O(n+m)$. □

Cost summary

Cost summary.

Task	Time	Space (beyond input)
Discovery times & low-link	$O(n+m)$	$O(n)$
All bridges	$O(n+m)$	$O(n)$
All articulation points	$O(n+m)$	$O(n)$
Biconnected components (edge stack)	$O(n+m)$	$O(m)$
Block-cut tree	$O(n+m)$	$O(n+m)$

A non-recursive (explicit-stack) DFS is advisable in practice: on a path of 10^6 vertices the recursive version overflows the call stack, while the connectivity facts are unchanged.

10.8 2-edge- and 2-vertex-connectivity

The local tests assemble into clean global statements. Recall a graph is 2-edge-connected if it has no bridge, and 2-vertex-connected (biconnected, $n \geq 3$) if it has no articulation point.

Corollary 10.1. *A connected graph G with $m \geq 1$ is 2-edge-connected iff the bridge DFS reports no bridge, iff every edge lies on a cycle, iff G has a single biconnected component containing all edges. All of these are decidable in $O(n+m)$.*

Corollary 10.2. *A connected graph G with $n \geq 3$ is 2-vertex-connected iff the articulation DFS reports no cut vertex, iff between every pair of vertices there are two internally-vertex-disjoint paths, iff the block-cut tree is a single block node.*

The "two disjoint paths" phrasing is the $k = 2$ case of Menger's theorem.

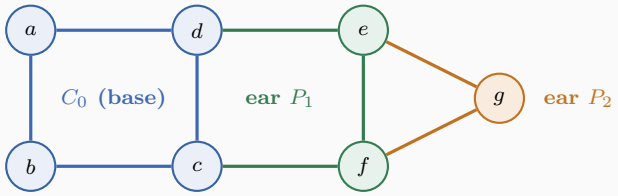
Theorem 10.4 (Menger, edge and vertex forms)**.** *For distinct non-adjacent s, t, the maximum number of pairwise edge-disjoint s–t paths equals the minimum number of edges whose removal separates s from t; and the maximum number of internally-vertex-disjoint s–t paths equals the minimum number of vertices (other than s, t) whose removal separates them.*

Why it works

The $k = 2$ special case is exactly our two tests. Menger says local connectivity equals a global path count. For $k = 2$: "no single edge separates any pair" (no bridge, 2-edge-connected) is equivalent to "every pair has 2 edge-disjoint paths"; "no single vertex separates any pair" (no cut vertex, 2-vertex-connected) is equivalent to "every pair has 2 internally-disjoint paths," which for a graph on ≥ 3 vertices means every pair lies on a common cycle. The general theorem is proved via max-flow min-cut in the flow chapters; here the DFS gives the $k = 2$ case directly and in linear time, without any flow computation.

Formal

Cycle characterization of biconnectivity. We sketch why "no articulation point" (with $n \geq 3$, connected) is equivalent to "every two vertices lie on a common cycle." If every pair lies on a cycle, then for any v and any pair $x, y \neq v$, take a cycle through x, y; it gives an x–y path avoiding v, so v is not a cut vertex. Conversely, suppose G is biconnected and take any x, y. Two internally-disjoint x–y paths (which exist by the vertex Menger bound, since no single vertex separates them) concatenate into a cycle through x and y. An induction building the graph up by "ear decompositions" — repeatedly adding a path (an "ear") whose endpoints are already present — makes this constructive: a graph is 2-connected iff it has an open ear decomposition starting from a cycle (Whitney's theorem). □



History

The mathematical foundations of connectivity began with Karl Menger in 1927, who proved that the minimum number of vertices (or edges) needed to disconnect two non-adjacent vertices equals the maximum number of pairwise internally disjoint paths between them. In 1932, Hassler Whitney systematically defined vertex- and edge-connectivity and characterized 2-connected graphs through ear decompositions. Menger's duality predates the algorithmic max-flow min-cut theorem of Ford and Fulkerson (1956) by three decades, but for forty years testing 2-connectivity relied on repeated reachability searches costing $O(nm)$ or worse.

In 1972, Robert Tarjan introduced the low-link depth-first search, followed in 1973 by John Hopcroft and Tarjan's biconnectivity algorithm in *Communications of the ACM*. Tarjan recognized that discovery times and a single number per vertex — the low-link value, tracking the highest ancestor reachable via back edges — certify the entire separation structure in one $O(n + m)$ pass. This collapsed bridge detection, articulation-point finding, and biconnected decomposition to linear time. The low-link idea reappears in Tarjan's strongly-connected-components algorithm and in Harold Gabow's path-based stack refinements; it remains one of the most widely used techniques in graph algorithms.

10.9 Application: network reliability

Connectivity structure is the first thing a network designer audits. Model a communication or transportation network as a graph: vertices are routers or stations, edges are links.

Application

Single points of failure. A *bridge* is a link whose loss partitions the network — no redundancy, the most fragile element. An *articulation point* is a node whose loss partitions the network — a router whose crash isolates a region. Auditing a network for both is a single

$O(n+m)$ DFS, fast enough to run on million-edge topologies continuously. The block-cut tree visualizes the result: each block is a "robust island" that survives any single-element failure internally, and the tree of blocks shows exactly how those islands hang together through their shared cut vertices and bridges.

Application

Reliability and augmentation. If a network has bridges, how few links must we add to make it 2-edge-connected (survive any single link failure)? Contract every 2-edge-connected component to a point: the graph collapses to its *bridge tree*, and the cheapest augmentation adds $\lceil L/2 \rceil$ edges, where L is the number of leaves of that tree (Eswaran–Tarjan, 1976). The analogous vertex-augmentation problem uses the block-cut tree. Both reductions depend on first computing the decomposition this chapter builds.

Connection

The low-link DFS is the undirected cousin of **Tarjan's strongly connected components** algorithm (Chapter 5), which stacks vertices and uses the very same low recurrence on a directed DFS tree (with cross and forward edges handled carefully). The block-cut tree is queried with the **LCA** structures of Chapter 23. The $k = 2$ Menger statements generalize to all k via the **max-flow min-cut** theorem (Chapters 11–12), where global minimum cut and edge/vertex connectivity become flow computations. And the "how fragile is this graph?" question has a continuous, spectral answer: the second-smallest eigenvalue of the graph Laplacian $\mathbf{L}$ (Chapters 31–32) is a quantitative measure of connectivity, of which "2-edge-connected = no bridge" is the yes-or-no threshold version. Cheeger's inequality (Chapter 32) makes that link precise.

A bridge is an edge cut of size one: removing it cuts the graph in two. When a network has no bridges, its edge connectivity is at least two, but we still want to know how many edges must be removed to disconnect it. In the next chapter, we move from single-edge cuts to the global minimum cut problem: finding the smallest cut of edges that disconnects an arbitrary undirected graph, via Stoer–Wagner's deterministic maximum-adjacency algorithm and Karger's randomized contraction algorithm.

10.10 Exercises

Exercise 10.1. Run Bridges (Algorithm 42) by hand on the cycle C_5 on vertices $1, \ldots, 5$, then on C_5 plus one chord $\{1, 3\}$. Tabulate disc and low and confirm the bridge set is empty in both cases. Then delete edge $\{4, 5\}$ from the chorded graph and recompute: which edges become bridges?

Exercise 10.2. Prove that an endpoint of a bridge is an articulation point if and only if it has degree at least 2. Give a smallest graph where a bridge's endpoint is *not* an articulation point.

Exercise 10.3. (Break it.) Take the multigraph on $\{u, v\}$ with two parallel edges between them. Show that the "skip if $w = p$" parent guard wrongly reports the edge $\{u, v\}$ as a bridge, and that the edge-index guard of Algorithm 42 reports no bridge (correctly). Explain in one sentence what cycle the second parallel edge creates.

Exercise 10.4. Prove that the number of connected components created by deleting an articulation point c from a connected graph equals the degree of c in the block-cut tree.

Exercise 10.5. (Implement.) Convert the recursive BiconnectedComponents (Algorithm 44) into an iterative version using an explicit stack of (vertex, neighbor-iterator) frames, so it runs on a path of 10^6 vertices without overflowing. Verify it emits identical blocks on the running example.

Exercise 10.6. Show that in any DFS of a connected undirected graph, the root is an articulation point iff it has ≥ 2 children, *regardless* of which vertex is chosen as root and in what order neighbors are visited. (Hint: the *set* of articulation points is traversal-independent; only the tree shape changes.)

Exercise 10.7. (Bridge tree.) Describe an $O(n + m)$ algorithm that contracts each 2-edge-connected component of G to a single vertex, producing the *bridge tree* whose edges are exactly the bridges of G. Prove the result is a tree (a forest if G is disconnected).

Exercise 10.8. (Augmentation.) Let the bridge tree of a connected graph have $L \geq 2$ leaves. Prove that adding $\lceil L/2 \rceil$ edges suffices to make the graph 2-edge-connected, and that no smaller number works. (This is the Eswaran–Tarjan bound; pair leaves across the tree.)

Exercise 10.9. (Vertex version of low.) Show that the recurrence used here,

$$\mathrm{low}[u] = \min\Big(\mathrm{disc}[u],\ \min_{\text{back } \{u,w\}} \mathrm{disc}[w],\ \min_{\text{child } c} \mathrm{low}[c]\Big),$$

is equivalent to Definition 10.4, and explain why using $\mathrm{low}[w]$ instead of $\mathrm{disc}[w]$ for back edges would be *wrong* for the articulation test.

Exercise 10.10. (Extend.) A graph is 2-vertex-connected iff it has an open ear decomposition. Use this to prove that every 2-connected graph on $n \geq 3$ vertices has at least n edges, with equality exactly for cycles.

Exercise 10.11. Give a connected graph in which a vertex of maximum degree is *not* an articulation point while a vertex of degree 2 *is*. Conclude that "importance" for connectivity is not the same as degree centrality.

Exercise 10.12. (The $\geq$ vs $>$ boundary.) Construct two graphs that are identical except for one back edge, such that the same tree edge $\{u, v\}$ satisfies $\mathrm{low}[v] = \mathrm{disc}[u]$ in one and $\mathrm{low}[v] < \mathrm{disc}[u]$ in the other. Verify that u is an articulation point in the first but not the second, while $\{u, v\}$ is a bridge in neither.

Chapter 11

Global Minimum Cut

The previous chapter found bridges — single edges whose deletion cuts a connected network in two. A network with no bridges can still be disconnected by removing a small set of edges. Instead of cutting a single edge or specifying two fixed endpoints in advance, we ask a broader, global question:

> What is the cheapest set of edges whose removal disconnects the graph into two nonempty pieces?

This is the *global* minimum cut. It is the weakest point of a network: the smallest amount of capacity you would have to sever to split it in two, no matter where the split happens to fall. The freedom to choose both sides of the cut makes the problem feel unconstrained, yet direct combinatorial algorithms solve it without computing flows. We will see two of them: a deterministic algorithm of Stoer and Wagner that finds the cut via a sequence of greedy maximum-adjacency orderings, and a randomized algorithm of Karger that repeatedly contracts random edges to isolate a minimum cut with clean, provable probability bounds. The Karger–Stein refinement turns that into one of the fastest known min-cut algorithms by branching during the late, error-prone stages of the contraction process.

11.1 The problem, precisely

Let $G = (V, E)$ be a connected undirected graph with $n = |V|$ vertices and $m = |E|$ edges, and let each edge e carry a nonnegative weight $w(e) \geq 0$ (for unweighted graphs read $w(e) = 1$). A *cut* is a partition of the vertices into two nonempty sets $(S, \bar{S})$ with $\bar{S} = V \setminus S$ and $\emptyset \neq S \neq V$. The *weight* (or capacity) of the cut is the total weight of the edges crossing it:

$$w(S, \bar{S}) \;=\; \sum_{\substack{e=\{u,v\}\in E \\ u\in S,\ v\in \bar{S}}} w(e).$$

Definition 11.1 (Global minimum cut)**.** The *global minimum cut* of G is a cut of minimum weight over all partitions $(S, \bar{S})$. Its value is

$$\lambda(G) \;=\; \min_{\emptyset \neq S \subsetneq V} w(S, \bar{S}).$$

For unweighted graphs $\lambda(G)$ is the *edge connectivity*: the fewest edges whose deletion disconnects G.

Intuition

Picture the graph as a sheet of fabric and each edge as a thread of a certain thickness $w(e)$. A cut is a pair of scissors slicing the fabric into two nonempty pieces; its cost is the total thickness of the threads you sever. The global min-cut is the place where the fabric is most flimsy. The s–t version forces you to keep two named points on opposite sides of the slice; the global version lets the scissors fall anywhere, so $\lambda(G) \le w(S, \bar{S})$ for *every* cut, in particular every s–t cut.

Directed cuts versus global cuts. Later, in Chapter 12.1, we will study the s–t cut problem, where two specific endpoints s and t must be separated. If you had an algorithm for that directed separation, you could find the global cut by fixing an arbitrary vertex s and finding the cheapest s–t cut over all $n-1$ possible choices of t:

$$\lambda(G) \;=\; \min_{t \neq s} \big(s\text{–}t \text{ min-cut}\big).$$

Whichever side of the true global cut contains s, the opposite side contains at least one vertex t, so running $n-1$ pairwise cut queries is guaranteed to locate the global minimum. This chapter presents two direct combinatorial algorithms that beat this reduction without computing any flow.

Focus

Three facts we will lean on repeatedly:

1. $\lambda(G)$ is at most the minimum weighted degree $\delta = \min_v \sum_{e \ni v} w(e)$, because isolating a single minimum-degree vertex is a cut.
2. Every cut $(S, \bar{S})$ separates *some* pair $u \in S$, $v \in \bar{S}$. So the global cut equals $\min_{u,v}$ of the u–v min-cut over all $\binom{n}{2}$ pairs — but we will never need all pairs.
3. Contracting an edge $\{u, v\}$ (merging u and v into one vertex, summing parallel edges, and dropping the self-loop) can only destroy cuts that separated u from v. Any cut that kept u and v on the same side survives with its crossing edge weights unchanged. Contraction shrinks the vertex count while preserving all unseparated cuts.

History

The global min-cut problem is old, but its direct combinatorial algorithms are relatively recent. Through the 1980s the standard route was $n-1$ max-flow computations; Hao and Orlin (1992) showed how to share work across them. Then in 1992 Nagamochi and Ibaraki, and independently in their 1997 paper Stoer and Wagner, found a simple flow-free deterministic algorithm. Meanwhile David Karger, as a Stanford graduate student, proposed random edge contraction in 1993; with Clifford Stein in 1996 he refined it to an $O(n^2 \log^3 n)$ algorithm. Karger's 2000 tree-packing algorithm reached near-linear time $O(m \log^3 n)$, and work continued into the 2020s with deterministic near-linear algorithms (Henzinger–Rao–Wang, Li). This chapter covers the two foundational algorithms: Stoer–Wagner and Karger(–Stein).

11.2 A tiny graph to keep in mind

We will trace every algorithm on the same small weighted graph, so keep this picture handy. It is the example from Stoer and Wagner's paper.

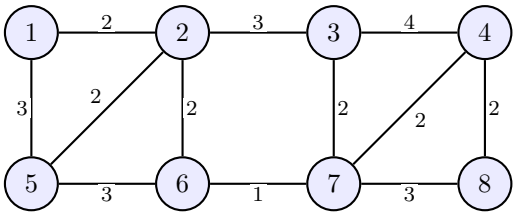


Figure 11.1: Our running example G. Eight vertices, twelve weighted edges. The global min-cut turns out to have weight 4, separating $\{3, 4, 7, 8\}$ from $\{1, 2, 5, 6\}$ (cut edges $\{2, 3\}$ weight 3 and $\{6, 7\}$ weight 1). We will rediscover this two different ways.

Trace it yourself

Before reading on, convince yourself by hand that the cut $\{3, 4, 7, 8\}$ vs. the rest costs $3+1 = 4$. Then try to beat it. Isolating vertex 1 costs $2 + 3 = 5$; isolating 8 costs $2 + 3 = 5$; cutting between columns $\{1, 5\}$ and the rest costs $2 + 2 + 3 = 7$. Nothing you try should drop below 4. Hold onto that number 4 — both algorithms must return it.

11.3 Stoer–Wagner: greedy orderings find a cut

The Stoer–Wagner algorithm is deterministic, runs in $O(nm + n^2 \log n)$ time, and has a compact implementation. Its main subroutine is called the *minimum cut phase* (also known as a *maximum adjacency search*, MA-search, or *legal ordering*) that, in a single greedy sweep, produces a cut isolating the final vertex visited. A lemma guarantees that this cut is a genuine s–t min-cut between the final two vertices s and t.

11.3.1 The minimum cut phase

Start from an arbitrary vertex and grow a set A one vertex at a time. At each step add the vertex *most tightly connected* to the current A: the vertex $v \notin A$ maximizing

$$w(A, v) \;=\; \sum_{\substack{e=\{a,v\}\in E \\ a\in A}} w(e),$$

the total weight of edges from v into A. Ties broken arbitrarily. Continue until all vertices are absorbed. Let s and t be the *second-to-last* and *last* vertices added. The phase returns:

- the *cut-of-the-phase*: the cut $(\{t\},\, V \setminus \{t\})$ that isolates the last vertex, of weight $w(A_{\text{before } t},\, t)$ = weighted degree of t;
- the pair (s, t), which it then *merges* (contracts) for the next phase.

Algorithm 45: MinimumCutPhase(G, w, a) — one MA-search

Input: weighted graph $G = (V, E, w)$, start vertex a
Output: cut-of-the-phase value, and the last two vertices s, t in MA-order

$A \leftarrow \{a\}$
for *each* $v \in V \setminus \{a\}$ **do**
  $key[v] \leftarrow w(\{a\}, v)$ `// edge weight from` v `to` a
$order \leftarrow [a]$
while $|A| < |V|$ **do**
  pick $z \notin A$ with maximum $key[z]$ `// most tightly attached`
  add z to A; append z to *order*
  for *each* $v \notin A$ *with edge* $\{z, v\} \in E$ **do**
    $key[v] \mathrel{+}= w(\{z, v\})$
$s \leftarrow$ second-to-last of *order*; $t \leftarrow$ last of *order*
return $\big(key[t] = w(A \setminus \{t\}, t),\ s,\ t\big)$

This is exactly Prim's MST / Dijkstra's frontier loop with a different priority: where Prim keeps the *minimum* attaching edge and Dijkstra the minimum tentative distance, MA-search keeps the *maximum* total attachment $key[v]$ and adds it up rather than taking a min. With a binary heap each phase costs $O(m \log n)$; with a Fibonacci heap, $O(m + n \log n)$.

11.3.2 The full algorithm

Run $n - 1$ phases. After each phase, record the cut-of-the-phase value, then contract s and t into a single super-vertex (parallel edges to a common neighbor are summed). Each contraction drops the vertex count by one, so after $n - 1$ phases a single vertex remains. The global min-cut is the smallest cut-of-the-phase ever recorded.

Algorithm 46: StoerWagner(G, w)

Input: connected weighted graph $G = (V, E, w)$, $n = |V| \geq 2$
Output: the global minimum cut value $\lambda(G)$ and one cut achieving it

$best \leftarrow +\infty$; $bestSet \leftarrow \varnothing$
fix an arbitrary start vertex $a \in V$
while $|V| > 1$ **do**
  $(cutval, s, t) \leftarrow$ MinimumCutPhase(G, w, a)
  if $cutval < best$ **then**
    $best \leftarrow cutval$
    $bestSet \leftarrow$ set of original vertices currently merged into t
  merge s and t into one super-vertex (sum weights of parallel edges, drop loops)
return $(best, bestSet)$

Intuition

Why should isolating the *last* vertex of a greedy maximum-adjacency walk be a min-cut for the last *two*? Intuition: MA-search adds vertices in order of how strongly the rest of the absorbed graph pulls on them. The last vertex t is the one the graph clung to most reluctantly relative

to its own degree — it was held out until everything else was inside A. Any cut that tries to separate t from the previous vertex s has to "pay" at least as much as simply chopping t off, because the greedy order already certifies that t is well attached. The lemma below makes this precise with an exchange argument.

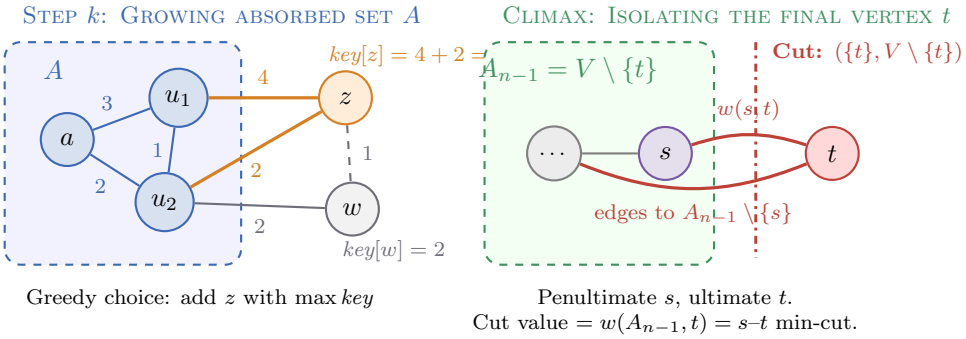


Figure 11.2: The mechanics of a minimum cut phase (MA-search). Left: the absorbed set A expands greedily by annexing whichever unvisited vertex shares the greatest total edge weight with A. Right: the very last vertex t to join is the most weakly attached relative to the whole graph; the cut isolating t has weight $key[t]$, which Lemma 11.1 proves is a true s–t min-cut.

11.3.3 Correctness

Two claims combine into correctness: each phase returns a legitimate s–t min-cut, and looping over contractions never loses the global optimum.

Lemma 11.1 (Cut-of-the-phase is an s–t min-cut)**.** *Let an MA-search on (G, w) visit vertices in order $v_1, v_2, \ldots, v_n$, and let $s = v_{n-1}$, $t = v_n$. Then the cut $(\{t\}, V \setminus \{t\})$ is a minimum-weight cut among all cuts that separate s from t. Its value equals $w(A_{n-1}, t)$, the weighted degree of t, where $A_{n-1} = \{v_1, \ldots, v_{n-1}\}$.*

Proof. Consider any cut $C = (X, \bar{X})$ that separates s from t; we show $w(C) \geq w(\{t\}, V \setminus \{t\})$, i.e. $w(C) \geq key[t]$ at the moment t is absorbed.

Call a vertex v_i (for $i \geq 2$) *active* (with respect to C) if v_i lands on the opposite side of C from the vertex v_{i-1} that immediately precedes it in MA-order. Because C separates $s = v_{n-1}$ from $t = v_n$, the last vertex t is active. For each vertex v_i write $A_i = \{v_1, \ldots, v_i\}$ and let $w(A_{i-1}, v_i)$ be the key of v_i when it is added. Let C_i be the cut of the subgraph induced by A_i obtained by restricting C to A_i. We prove by induction over the active vertices: for every active v_i,

$$w(A_{i-1}, v_i) \;\leq\; w(C_i), \tag{$\star$}$$

the weight of the cut C restricted to A_i.

Base case. Let v_i be the first active vertex. Then $v_1, \ldots, v_{i-1}$ all lie on one side of C and v_i on the other, so every edge from v_i into A_{i-1} crosses C. Hence $w(A_{i-1}, v_i) = w(C_i)$, giving ($\star$).

Inductive step. Let v_i be active and v_j the previous active vertex ($j < i$). Split the attachment of v_i into edges hitting A_{j-1} and edges hitting $\{v_j, \ldots, v_{i-1}\}$:

$$w(A_{i-1}, v_i) \;=\; w(A_{j-1}, v_i) + w(\{v_j, \ldots, v_{i-1}\}, v_i).$$

Second term: every edge from v_i to a vertex on the opposite side of C crosses C, and these are exactly the crossing edges that A_i adds beyond A_{j-1}, so $w(\{v_j, \dots, v_{i-1}\}, v_i) \le w(C_i) - w(C_{j-1})$. First term: because v_j was chosen before v_i, the greedy rule tells us that v_j had an attachment key at least as large as v_i when v_j was added:

$$w(A_{j-1}, v_i) \;\le\; w(A_{j-1}, v_j) \;\le\; w(C_{j-1}),$$

the last inequality being the inductive hypothesis ($\star$) applied to the active vertex v_j. Adding the two bounds:

$$w(A_{i-1}, v_i) \;\le\; w(C_{j-1}) + \big(w(C_i) - w(C_{j-1})\big) \;=\; w(C_i),$$

which proves ($\star$) for v_i. Applying ($\star$) to the last vertex $t = v_n$ (which is active) gives $key[t] = w(A_{n-1}, t) \le w(C_n) = w(C)$. Thus the cut isolating t is no heavier than any s–t cut C, certifying it as an s–t min-cut. □

Why it works

The induction only ever compares an active vertex v_i to the previous active vertex v_j. The key inequality is $w(A_{j-1}, v_i) \le w(A_{j-1}, v_j)$ — expressing that the greedy maximum-adjacency rule preferred v_j over v_i at step j. The remaining algebra merely verifies that crossing edges accumulate monotonically. Informally, the greedy attachment order ensures that the final holdout vertex cannot be sliced away from its immediate predecessor without cutting at least as much capacity as isolating the vertex itself.

Theorem 11.1 (Stoer–Wagner correctness)**.** *Algorithm 46 returns* $\lambda(G)$.

Proof. Induction on $n = |V|$. For $n = 2$ the only cut separates the two vertices and equals their connecting weight; the single phase reports exactly that.

Assume correctness for graphs with fewer than n vertices. Run the first phase, producing s, t and the cut-of-the-phase. There are two cases for the true global min-cut C^* of G.

- C^* **separates** s **from** t**.** By Lemma 11.1 the cut-of-the-phase is an s–t min-cut, so its value is $\le w(C^*)$. Since no cut beats the global min-cut, the cut-of-the-phase value equals $w(C^*) = \lambda(G)$, and the algorithm records it.
- C^* **keeps** s **and** t **together.** Then C^* survives merging s and t: it is a cut of the contracted graph $G/\{s,t\}$ of the same weight, and $G/\{s,t\}$ has no cut cheaper than $\lambda(G)$ (contraction only removes cuts, never creates lighter ones). By induction the recursion on $G/\{s,t\}$ returns $\lambda(G)$.

Either way some phase records $\lambda(G)$, and the algorithm returns the minimum over all phases, which is exactly $\lambda(G)$. □

Common trap

Two classic implementation bugs. *(1) Resetting the start vertex.* The start vertex a of each phase is arbitrary — you may keep the same one or pick fresh — but you must run MA-search on the *current contracted* graph, not the original. Forgetting to contract turns the algorithm into nonsense. *(2) Recovering the actual cut.* `bestSet` must be the set of *original* vertices merged into t at the phase that achieved the minimum, not the single super-vertex label.

Maintain, per super-vertex, the list of originals it represents (a union-find or an explicit list), and snapshot it when you update `best`. People routinely get the value right and the side wrong.

11.3.4 Complexity

Each phase is one MA-search. With a Fibonacci heap, extracting the max-key vertex n times and relaxing each edge once costs $O(m + n \log n)$. There are $n - 1$ phases, and contraction is absorbed into the next phase's adjacency build, so the total is

$$O\big((n-1)(m + n \log n)\big) \;=\; O(nm + n^2 \log n).$$

A binary heap gives $O(nm \log n)$, and a dense $O(n^2)$-per-phase array implementation (scan for the max, no heap) gives $O(n^3)$, which is often the simplest to code and competitive when $m = \Theta(n^2)$.

Cost summary

Implementation	Per phase	Total
Dense array (scan for max)	$O(n^2)$	$O(n^3)$
Binary heap	$O(m \log n)$	$O(nm \log n)$
Fibonacci heap	$O(m + n \log n)$	$O(nm + n^2 \log n)$

Space $O(n + m)$. Fully deterministic; returns both the value and a concrete cut.

11.3.5 Tracing Stoer–Wagner by hand

Run the dense version on Figure 11.1, starting each phase at vertex 1 (relabeling super-vertices by their lowest original index). We list the MA-order and the two key quantities for each phase.

Phase 1. Start $A = \{1\}$. Keys: 2:2, 5:3, rest 0. Pick 5 (key 3). Update: 2:2+2=4, 6:3. Pick 2 (key 4). Update: 3:3, 6:3+2=5. Pick 6 (key 5). Update: 7:1. Pick 3 (key 3). Update: 4:4, 7:1+2=3. Pick 4 (key 4). Update: 7:3+2=5, 8:2. Pick 7 (key 5). Update: 8:2+3=5. Pick 8 (key 5). MA-order: $1, 5, 2, 6, 3, 4, 7, 8$. So $s = 7$, $t = 8$, cut-of-the-phase $= key[8] = 5$ (isolate vertex 8, degree $2 + 3 = 5$). Best so far $= 5$. Merge 7 and 8 into super-vertex "7".

Phase 2. Vertices now $\{1, 2, 3, 4, 5, 6, 7'\}$ where $7' = \{7, 8\}$; edges to $7'$: 3−$7'$:2, 4−$7'$:2+2=4, 6−$7'$:1. Start $A = \{1\}$. Keys: 2:2, 5:3, rest 0. Pick 5 (key 3); update 2:4, 6:3. Pick 2 (key 4); update 3:3, 6:5. Pick 6 (key 5); update $7'$:1. Pick 3 (key 3); update 4:4, $7'$:1+2=3. Pick 4 (key 4); update $7'$:3+4=7. Pick $7'$ (key 7). MA-order $1, 5, 2, 6, 3, 4, 7'$; $s = 4$, $t = 7'$, cut-of-the-phase $= key[7']$ = degree of $7' = 2 + 4 + 1 = 7$. Best stays 5. Merge 4 and $7'$ into super-vertex "4" $= \{4, 7, 8\}$.

Phases 3–7. The remaining phases continue the same pattern: run MA-search on the current contracted graph, isolate the trailing vertex t, record its weighted degree as the cut-of-the-phase, then merge s and t. The whole run is summarized below (super-vertices labelled by their lowest original index, $a = 1$ throughout, ties broken toward the lowest label):

Phase	MA-order	(s, t)	cut-of-phase	vertices of t
1	$1, 5, 2, 6, 3, 4, 7, 8$	$(7, 8)$	5	$\{8\}$
2	$1, 5, 2, 6, 3, 4, 7'$	$(4, 7')$	7	$\{7, 8\}$
3	$1, 5, 2, 6, 3, 4'$	$(3, 4')$	7	$\{4, 7, 8\}$
4	$1, 5, 2, 6, 3''$	$(6, 3'')$	**4**	$\{3, 4, 7, 8\}$
5	$1, 5, 2, 3'''$	$(2, 3''')$	8	$\{3, 4, 6, 7, 8\}$
6	$1, 5, 2''''$	$(5, 2'''')$	7	$\{2, 3, 4, 6, 7, 8\}$
7	$1, 2'''''$	$(1, 2''''')$	5	all but $\{1\}$

Here $4' = \{4, 7, 8\}$ after phase 2, $3'' = \{3, 4, 7, 8\}$ after phase 3, and so on; each prime marks one more merge into the same representative. Phase 4 is the one that matters: by then $4' = \{4, 7, 8\}$ has absorbed into $3'' = \{3, 4, 7, 8\}$, and isolating that super-vertex — which the MA-order forces, since $3''$ is the last vertex absorbed — realizes the partition $\{3, 4, 7, 8\}$ vs. $\{1, 2, 5, 6\}$. Its cut-of-the-phase equals the crossing weight $w(\{2, 3\}) + w(\{6, 7\}) = 3 + 1 = 4$. That updates best to 4. No later phase dips below it (the remaining values are $8, 7, 5$), so the algorithm returns 4 with side $\{3, 4, 7, 8\}$ — the cut drawn in Figure 11.3. The minimum of the cut-of-the-phase column, $\min(5, 7, 7, 4, 8, 7, 5) = 4$, is the answer.

Trace it yourself

Finish the trace. After phase 2 you have six super-vertices. Keep contracting and recording cut-of-the-phase values; the sequence $5, 7, 7, \ldots$ should dip to 4 exactly once and never lower. The phase that hits 4 has t representing $\{3, 4, 7, 8\}$ — that is your `bestSet`. If your by-hand run ever reports a value below 4, you violated Definition 11.1 and made an arithmetic slip; if it never reaches 4, you skipped a contraction.

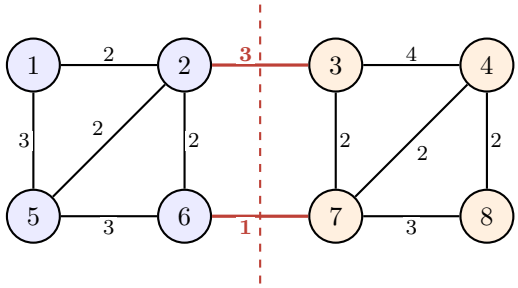


Figure 11.3: The global min-cut of the running graph, value 4. The dashed line separates $\{1, 2, 5, 6\}$ (blue) from $\{3, 4, 7, 8\}$ (orange); only the two thick red edges $\{2, 3\}$ (weight 3) and $\{6, 7\}$ (weight 1) cross it. Stoer–Wagner discovers this at phase 4.

11.4 Karger's random contraction

Stoer–Wagner is greedy and exact. Karger's algorithm goes to the opposite extreme: it makes a random choice at every step, maintains no priority queues, and yet isolates the minimum cut with provable polynomial probability. The algorithm operates by repeated random contraction: pick an edge uniformly at random from the current multigraph, merge its two endpoints into a single super-vertex, and repeat until only two super-vertices remain.

11.4.1 Contraction

To *contract* an edge $e = \{u, v\}$: replace u and v by a single composite vertex uv. Every edge previously incident to u or v now connects to uv. Importantly, maintain all *multi-edges*: if both u and v had edges to a third vertex x, the super-vertex uv now has two parallel edges to x. Parallel edges are preserved because edge multiplicities directly reflect the number of original graph edges spanning those sets of vertices. Self-loops formed by copies of e itself are discarded. Each contraction decreases the vertex count by exactly one. When only two super-vertices remain, the multiset of edges spanning them forms a cut of the original graph: the two super-vertices define the vertex partition.

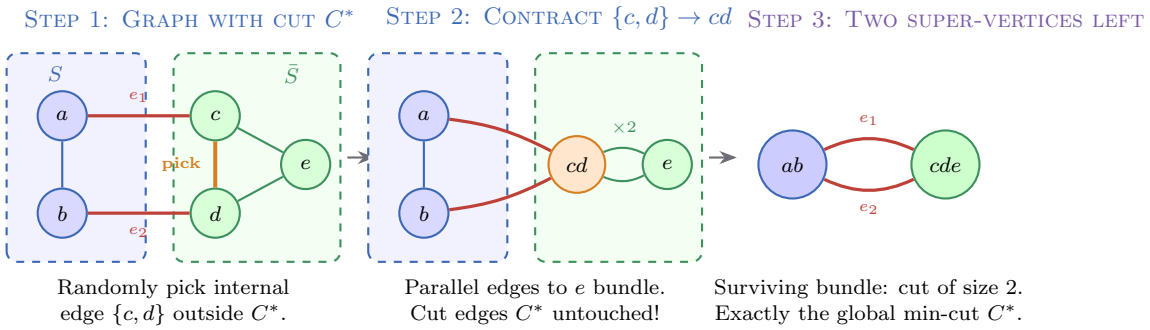


Figure 11.4: The three stages of Karger contraction. Step 1: an edge internal to one side of the min-cut is randomly sampled. Step 2: contracting the edge merges its endpoints; parallel edges to mutual neighbors accumulate rather than being simplified, and self-loops vanish. Step 3: when only two composite vertices remain, the edges between them form the surviving cut.

Lemma 11.2 (Contraction preserves untouched cuts)**.** *Let $G' = G/e$ be G with edge $e = \{u, v\}$ contracted. There is a weight-preserving bijection between cuts of G' and cuts of G that do* not *separate u from v. In particular, the min-cut of G' is at least the min-cut of G: $\lambda(G') \geq \lambda(G)$, with equality unless every min-cut of G separates u and v.*

Proof. A cut $(S', \bar{S}')$ of G' assigns the super-vertex uv entirely to one side. Expand it to G by replacing uv with both u and v on that same side; this is a cut of G that keeps u, v together, and its crossing edges are in bijection with the crossing edges of $(S', \bar{S}')$ (the only edges that disappeared under contraction were copies of e, which never cross a cut keeping u, v together). The map is reversible, hence a weight-preserving bijection. Cuts of G that separated u from v have no preimage, so G' has a subset of G's cuts; the minimum over a subset is no smaller. □

11.4.2 The algorithm

Algorithm 47: KargerContract(G) — one trial (unweighted multigraph)

Input: connected multigraph $G = (V, E)$, $n = |V|$
Output: a cut of G (its value is the number of surviving edges)

```
while |V| > 2 do
    pick an edge e uniformly at random from the current edge multiset E
    G ← G/e          // contract: merge endpoints, keep multi-edges, drop loops
return the cut given by the two remaining super-vertices (value = |E|)
```

For *weighted* graphs, "pick a uniformly random edge" becomes "pick edge e with probability proportional to $w(e)$," and the surviving cut value is the total surviving weight. The analysis below is for the unweighted (multigraph) case; the weighted case follows by treating weight as parallel-edge multiplicity (or directly with the weighted sampling probabilities, see Exercise 11.5).

Intuition

A min-cut has few edges by definition. A uniformly random edge is therefore *unlikely* to be one of the few min-cut edges — the overwhelming majority of edges connect vertices on the same side of the cut. Each contraction of a non-cut edge merges two vertices that the min-cut keeps together, leaving the min-cut intact (Lemma 11.2). If we never choose a min-cut edge across all $n - 2$ contractions, the two surviving super-vertices are precisely the two sides of the min-cut. The analysis below calculates the exact probability of this event.

11.4.3 Success probability

Fix one particular minimum cut C^* of weight $k = \lambda(G)$ (there may be several; we only need one to survive). The trial *succeeds for* C^* if it never contracts an edge of C^*.

Lemma 11.3 (Min-degree bound)**.** *At every stage of contraction, a multigraph on r super-vertices whose min-cut is still $\geq k$ has at least $rk/2$ edges; hence each vertex has degree $\geq k$ and a uniformly random edge crosses C^* with probability $\leq k/(rk/2) = 2/r$.*

Proof. Contraction never decreases the min-cut (Lemma 11.2), so throughout the run every super-vertex has degree at least k (isolating it is a cut, which costs $\geq k$). Summing degrees, $2|E| = \sum_v \deg v \geq rk$, so $|E| \geq rk/2$. The cut C^* contributes exactly k edges to the current multigraph as long as none has been contracted, so the chance the random edge is one of them is $k/|E| \leq k/(rk/2) = 2/r$. □

Theorem 11.2 (Karger success probability)**.** *A single run of Algorithm 47 returns the fixed min-cut C^* with probability at least*

$$\Pr[\textit{success}] \;\geq\; \binom{n}{2}^{-1} \;=\; \frac{2}{n(n-1)}.$$

Proof. Let E_i be the event "the i-th contraction does not destroy C^*," i.e. the chosen edge is not a C^*-edge. Before the i-th contraction there are $r = n - i + 1$ vertices, so by Lemma 11.3,

$$\Pr[E_i \mid E_1 \cap \cdots \cap E_{i-1}] \;\geq\; 1 - \frac{2}{n-i+1} \;=\; \frac{n-i-1}{n-i+1}.$$

The run succeeds iff all $n-2$ events occur. Multiply the conditional probabilities (telescoping product):

$$\Pr[\text{success}] \;\geq\; \prod_{i=1}^{n-2} \frac{n-i-1}{n-i+1} \;=\; \frac{n-2}{n}\cdot\frac{n-3}{n-1}\cdot\frac{n-4}{n-2}\cdots\frac{2}{4}\cdot\frac{1}{3}.$$

Almost everything cancels: numerators $n-2, n-3, \ldots, 1$ against denominators $n, n-1, \ldots, 3$. What survives is $\dfrac{2\cdot 1}{n(n-1)} = \binom{n}{2}^{-1}$. □

Connection

This proof is a pure exercise in conditional probability and the chain rule $\Pr[\bigcap E_i] = \prod \Pr[E_i \mid E_1 \cap \cdots \cap E_{i-1}]$ — exactly the machinery developed in the companion probability book. The telescoping product is the same algebraic move that appears in birthday-paradox and coupon-collector calculations. And Theorem 11.2 yields an immediate structural corollary below: an upper bound on the number of distinct min-cuts, obtained purely from the probability bound, not from any combinatorial enumeration.

Corollary 11.1 (At most $\binom{n}{2}$ minimum cuts)**.** *A graph on n vertices has at most $\binom{n}{2}$ distinct minimum cuts.*

Proof. For each distinct min-cut C_j, the events "a single Karger run outputs C_j" are disjoint (a run outputs exactly one cut). Each has probability $\geq \binom{n}{2}^{-1}$ by Theorem 11.2. Disjoint events have probabilities summing to at most 1, so the number of distinct min-cuts is at most $\binom{n}{2}$. This bound is tight: the cycle C_n has exactly $\binom{n}{2}$ min-cuts (cut any two of its n edges). □

11.4.4 Amplification

A success probability of $\sim 2/n^2$ sounds tiny, but independent repetition drives the failure probability down quickly. Run the trial T times and keep the lightest cut found. The probability that *all* T trials miss C^* is at most $\left(1-\binom{n}{2}^{-1}\right)^T$. Using $1-x \leq e^{-x}$,

$$\Pr[\text{all } T \text{ fail}] \;\leq\; \Big(1-\binom{n}{2}^{-1}\Big)^{T} \;\leq\; \exp\Big(-T/\binom{n}{2}\Big).$$

Choosing $T = \binom{n}{2}\ln n \approx \frac{1}{2}n^2 \ln n$ drives the failure probability below $e^{-\ln n} = 1/n$; choosing $T = c\binom{n}{2}\ln n$ gives failure $\leq n^{-c}$ — high probability for any constant c.

Cost summary

One trial does $n-2$ contractions; a careful adjacency-matrix implementation makes each contraction $O(n)$, so a trial is $O(n^2)$. Amplifying to failure $\leq 1/n$ needs $O(n^2 \log n)$ trials, for a total of

$$O(n^2)\cdot O(n^2 \log n) \;=\; O(n^4 \log n).$$

That is worse than Stoer–Wagner. The fix — Karger–Stein — is to notice that the early contractions are very safe and only the late ones are dangerous, so we should not redo the safe prefix every trial.

11.5 Karger–Stein: Branching on the Dangerous Tail

Consider the survival probability at step r: $1 - 2/r$. When r is large, each contraction is very safe: with $n = 1000$ vertices, the chance of destroying the min-cut is less than $2/1000 = 0.2\%$. But when r shrinks toward small numbers, the failure probability spikes: at $r = 4$, the chance of severing the min-cut on that single step is $2/4 = 50\%$. Almost every failed trial loses the minimum cut during these final few contractions.

Restarting the algorithm from scratch on every failure discards all the safe, expensive early contractions. Karger and Stein observed that we should instead contract down to a threshold where the min-cut is guaranteed to survive with probability at least $1/2$, and then *branch*: generate two independent continuations. By doubling the search effort each time the graph shrinks by a factor of $\sqrt{2}$, the algorithm spends its extra work on the stages where the min-cut is most likely to be lost.

11.5.1 The partial-contraction guarantee

Lemma 11.4 (Surviving to t vertices)**.** *Starting from n vertices and contracting random edges until t vertices remain, the fixed min-cut C^* survives with probability at least*

$$\Pr[\textit{survive to } t] \;\geq\; \frac{\binom{t}{2}}{\binom{n}{2}} \;=\; \frac{t(t-1)}{n(n-1)}.$$

Proof. Same telescoping product as Theorem 11.2, but stop after $n - t$ contractions (when t vertices remain):

$$\prod_{i=1}^{n-t}\Big(1 - \frac{2}{n-i+1}\Big) = \prod_{r=t+1}^{n}\frac{r-2}{r} = \frac{(t-1)t}{n(n-1)} = \frac{\binom{t}{2}}{\binom{n}{2}}.$$

□

Choosing $t = \lceil 1 + n/\sqrt{2} \rceil$ makes $\binom{t}{2}/\binom{n}{2} \geq \frac{1}{2}$: contracting down to a $1/\sqrt{2}$ fraction of the vertices keeps the min-cut alive with probability at least one half.

11.5.2 The recursion

Algorithm 48: KargerStein(G)

Input: multigraph G with $n = |V|$ vertices
Output: a cut of G; with the recursion it is the min-cut w.h.p.

```
if n ≤ 6 then
    return min-cut found by brute force (e.g. Stoer–Wagner) on G
t ← ⌈1 + n/√2⌉
G1 ← contract random edges of G down to t vertices
G2 ← contract random edges of G down to t vertices          // independent of G1
return min(KargerStein(G1), KargerStein(G2))
```

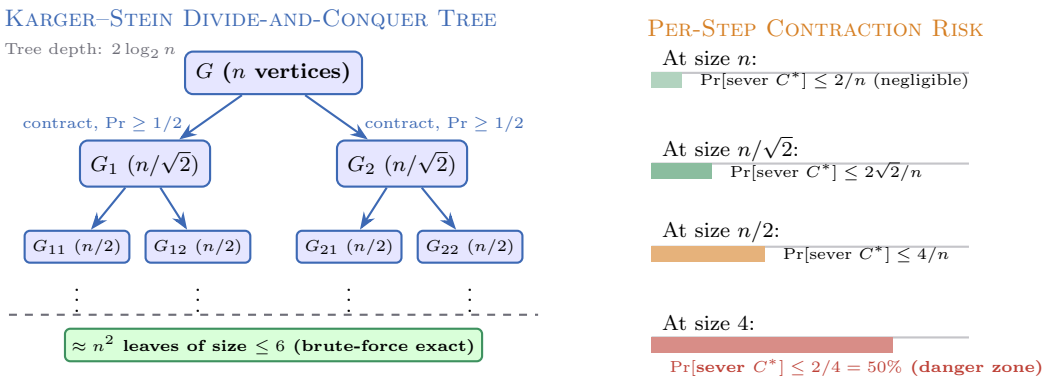


Figure 11.5: The Karger–Stein divide-and-conquer strategy. Left: the algorithm contracts the graph to $t = \lceil 1 + n/\sqrt{2} \rceil$ vertices (survival probability $\geq 1/2$) and spawns two independent recursive continuations, building a binary tree of depth $2\log_2 n$. Right: the per-step risk of severing the min-cut escalates sharply as the graph shrinks; branching ensures that the high-risk late contractions are attempted over $O(n^2)$ independent leaves while the safe early work is reused.

Intuition

The recursion tree is a binary tree of depth $O(\log n)$. Each node shrinks its graph by a factor $\sqrt{2}$ and survives with probability $\geq 1/2$, then hands two independent copies to its children. Two children, each succeeding with probability $\geq 1/2$ on a problem of reduced size, create a branching process where the overall success probability decays only logarithmically with n rather than quadratically.

11.5.3 Running time

Let $T(n)$ be the time for one call. Contracting from n to $t \approx n/\sqrt{2}$ vertices costs $O(n^2)$ (each of $O(n)$ contractions is $O(n)$), done twice, and there are two recursive calls on graphs of size $\approx n/\sqrt{2}$:

$$T(n) = 2\,T\Big(\tfrac{n}{\sqrt{2}} + 1\Big) + O(n^2).$$

By the master theorem with $a = 2$, $b = \sqrt{2}$, so $\log_b a = \log_{\sqrt{2}} 2 = 2$, and the work per level $O(n^2)$ matches $n^{\log_b a} = n^2$: the balanced case. Hence

$$T(n) = O(n^2 \log n).$$

The recursion tree has $O(n^2)$ leaves and $O(\log n)$ levels, $O(n^2)$ total work per level.

11.5.4 Success probability of the recursion

Theorem 11.3 (Karger–Stein success)**.** *Algorithm 48 finds the min-cut with probability* $\Omega\big(1/\log n\big)$.

Proof. Let $P(n)$ lower-bound the probability that a call on n vertices finds C^*. Each of the two children succeeds if (a) the partial contraction to t vertices preserves C^* — probability $\geq 1/2$ by the

choice of t — and (b) the recursive call then finds it — probability $\geq P(t)$. So a single child succeeds with probability $\geq \frac{1}{2}P(t)$, and the parent succeeds if *either* child does:

$$P(n) \;\geq\; 1 - \Bigl(1 - \tfrac{1}{2}P(t)\Bigr)^2, \qquad t \approx \tfrac{n}{\sqrt{2}}.$$

Index the recursion by depth: let p_d be the success probability at a node d levels above the leaves, with $p_0 = \Theta(1)$ (the brute-force base case is exact). Expanding the recurrence, $p_d \geq p_{d-1} - \frac{1}{4}p_{d-1}^2$. Put $q_d = 1/p_d$; from $p = p' - \frac{1}{4}p'^2$ one has $1/p = 1/p' + 1/(4 - p') \leq 1/p' + 1/3$ for $p' \in (0, 1]$, so $q_d \leq q_{d-1} + \frac{1}{3}$. Therefore $q_d \leq q_0 + d/3 = O(\log n)$ since $d = O(\log n)$, giving $p_d = \Omega(1/\log n)$. □

So a *single* Karger–Stein run succeeds with probability $\Omega(1/\log n)$. Repeating it $O(\log^2 n)$ times and keeping the best cut pushes the failure probability below $1/n$ (each run fails with probability $\leq 1 - c/\log n \leq e^{-c/\log n}$; raising to the $O(\log^2 n)$ power gives $e^{-c\log n} = n^{-c}$), for a total of

$$O(n^2 \log n) \cdot O(\log^2 n) \;=\; O(n^2 \log^3 n).$$

Cost summary

Algorithm	Time	Guarantee
Karger, one trial	$O(n^2)$	finds min-cut w.p. $\geq \binom{n}{2}^{-1}$
Karger, amplified	$O(n^4 \log n)$	w.h.p. ($\geq 1 - 1/n$)
Karger–Stein, one run	$O(n^2 \log n)$	w.p. $\Omega(1/\log n)$
Karger–Stein, amplified	$O(n^2 \log^3 n)$	w.h.p.
Stoer–Wagner	$O(nm + n^2 \log n)$	deterministic, exact
Karger tree-packing (2000)	$O(m \log^3 n)$	w.h.p.

Karger–Stein at $O(n^2 \log^3 n)$ on dense graphs beats both naive flow ($O(mn^2)$) and Stoer–Wagner's $O(n^3)$.

Common trap

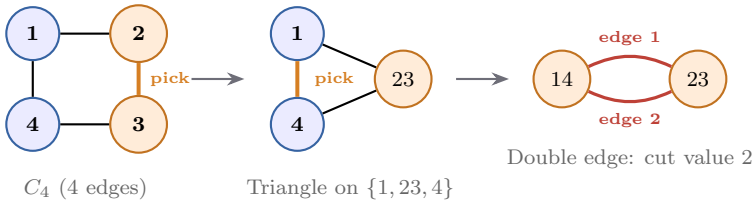
A frequent implementation pitfall is *merging parallel edges* during contraction. If you collapse parallel edges into single edges without weights, you destroy the multiplicities that make a random edge land across the min-cut with probability $k/|E|$ — the entire probability analysis assumes uniform sampling over the multiset of edges. Represent the multigraph with an adjacency *matrix of integer counts* (or edge weights) and add them; never deduplicate edges. A second trap: in Karger–Stein, the two recursive branches must contract *independently* from the parent graph — reusing a single contracted graph across both branches forfeits the independent chance, collapsing the success rate back to $1/n^2$.

11.5.5 Tracing Karger by hand

Take a tiny example where we can compute everything: the 4-cycle C_4 with vertices $1, 2, 3, 4$ and edges $12, 23, 34, 41$, all weight 1. Its min-cut is 2 (removing 12 and 34 separates $\{1, 4\}$ from $\{2, 3\}$, for instance). By Corollary 11.1 there are at most $\binom{4}{2} = 6$ min-cuts; for C_4 every pair of edges removed disconnects it, so there are exactly 6, hitting the bound.

One trial: $n = 4$, we contract twice.

- **Step 1** ($r = 4$, 4 edges). Pick a uniform edge, say 23. Contract: super-vertex 23, edges become 1−23 (the old 12), 23−4 (the old 34), 4−1. Now 3 vertices $\{1, 23, 4\}$ forming a triangle, 3 edges.
- **Step 2** ($r = 3$, 3 edges). Pick a uniform edge, say 4−1. Contract: super-vertex 14. Two vertices $\{14, 23\}$ with edges 14−23: the old 1−23 and 23−4, so a *double* edge. Surviving cut value = 2. Success — this equals $\lambda = 2$, separating $\{1, 4\}$ from $\{2, 3\}$.



On C_4 every trial returns value 2, because every 2-edge removal is a min-cut. Theorem 11.2 promises success $\geq 1/\binom{4}{2} = 1/6$ for each *fixed* cut; summed over the 6 min-cuts that (correctly) permits total success probability up to 1.

Trace it yourself

Build a graph where Karger genuinely fails sometimes: take two triangles $\{a, b, c\}$ and $\{x, y, z\}$ joined by a single edge c−x. The unique min-cut is $\{c, x\}$ (value 1). Trace: a trial succeeds only if the last surviving edge is c−x, i.e. it never contracts c−x and collapses each triangle first. Estimate the success probability by hand and compare to the $\binom{7}{2}^{-1} = 1/21$ guarantee — you should beat the bound comfortably, since the bound is worst-case.

11.6 Applications

Application

Network reliability. The edge connectivity $\lambda(G)$ is the number of link failures a network can tolerate before splitting. Telecom and data-center fabrics are designed so λ exceeds a target; global min-cut finds the cheapest cut, which shows where to add links first. Karger's contraction view also yields the fastest known algorithms for estimating all-terminal network *reliability* (the probability a random subgraph stays connected), because near-minimum cuts dominate the failure probability and Corollary 11.1 extends to bound the number of *near*-minimum cuts.

Image segmentation and clustering. Treat pixels (or data points) as vertices and put a heavy edge between similar neighbors. A light global cut separates dissimilar regions; this is the combinatorial cousin of the spectral "normalized cut" and the starting point for graph-based segmentation. When you want balanced sides you switch to sparsest/normalized cut (the Cheeger and spectral chapters), but unbalanced global min-cut is the exact, polynomial baseline.

VLSI and partitioning. Splitting a circuit netlist across two chips while cutting the fewest wires is a min-cut. The unconstrained version is global min-cut; adding balance constraints makes it NP-hard, which is why heuristics (and spectral relaxations) take over.

A structural bound. Corollary 11.1 — at most $\binom{n}{2}$ min-cuts — is itself the application: it underlies cut-sparsification, the analysis of randomized algorithms, and the fact that you can store *all* min-cuts compactly in the cactus representation.

Connection

Global min-cut connects to several other parts of this book.

- **Max-flow (Chapter 12).** The naive $n-1$ flow reduction and Hao–Orlin connect global cut to s–t duality; Stoer–Wagner is what you reach for to *avoid* flow entirely.
- **Spectral graph theory and the Laplacian L.** The min-cut is the combinatorial extreme; the second Laplacian eigenvalue $\lambda_2(\mathbf{L})$ (Fiedler value) controls the *balanced* cut via Cheeger's inequality. Random contraction and random walks share the same "most edges are internal" intuition that makes $\mathbf{L}$'s spectrum reveal cuts.
- **Probability book.** Theorem 11.2 and Lemma 11.4 are textbook conditional-probability chain-rule arguments; the amplification step is the union/independence bound $1-(1-p)^T \geq 1-e^{-pT}$ used throughout randomized algorithms.

Summary

Global min-cut asks for the cheapest split of a graph with no fixed terminals. You can pay $n-1$ max-flows, but two direct algorithms are better and far more instructive. Stoer–Wagner grows maximum-adjacency orderings; the trailing two vertices of each ordering give a true s–t min-cut (Lemma 11.1), and contracting them across $n-1$ phases sweeps out the global optimum deterministically in $O(nm+n^2\log n)$. Karger contracts a uniformly random edge until two vertices remain; a fixed min-cut survives with probability $\geq \binom{n}{2}^{-1}$ (Theorem 11.2), which both gives a randomized algorithm and proves there are at most $\binom{n}{2}$ min-cuts. Karger–Stein spends two independent recursive continuations on the risky tail of the contraction, reaching $O(n^2\log^3 n)$ with high probability.

Global min-cut asks how cheaply a network can be split. The next chapter asks a related question: how much commodity can we route between a designated source and sink without exceeding channel capacities? It introduces maximum flow and proves the max-flow min-cut theorem, which equates flow volume with cut capacity.

11.7 Exercises

Exercise 11.1 (Implement Stoer–Wagner)**.** Implement the dense $O(n^3)$ version with an $n \times n$ weight matrix and a boolean "merged" array. Run it on Figure 11.1 and confirm it returns 4 with side $\{3,4,7,8\}$. Then add cut recovery: maintain for each super-vertex the list of original vertices it absorbed, and output the actual partition.

Exercise 11.2 (Implement Karger and Karger–Stein)**.** Implement Karger with a count-matrix contraction in $O(n^2)$ per trial. Empirically measure the success rate over many trials on C_n for $n = 5,\ldots,20$ and compare to $\binom{n}{2}^{-1}$. Then implement Karger–Stein and verify the success rate is $\Omega(1/\log n)$ per run.

Exercise 11.3 (The greedy lemma is local)**.** In Lemma 11.1 we only proved the cut-of-the-phase is an s–t min-cut for the *last two* vertices. Give a small weighted graph and an MA-order in which the first cut-of-the-phase is *not* a global min-cut, so the algorithm genuinely needs more than one phase to find λ.

Exercise 11.4 (Counting min-cuts is tight)**.** Prove the cycle C_n has exactly $\binom{n}{2}$ distinct minimum cuts, each of value 2, matching Corollary 11.1. Then exhibit a graph with strictly fewer than $\binom{n}{2}$ min-cuts, confirming the bound is an inequality in general.

Exercise 11.5 (Weighted Karger)**.** Extend Algorithm 47 to weighted graphs by sampling edge e with probability $w(e)/\sum_f w(f)$. Prove that Theorem 11.2 still holds with $k = \lambda(G)$ the weighted min-cut. (Hint: redo Lemma 11.3 with weighted degrees; the bound $2/r$ on the probability of cutting C^* is unchanged.)

Exercise 11.6 (Min-cut from max-flow, carefully)**.** Show that $\lambda(G) = \min_{t \neq s}(s\text{–}t \text{ min-cut})$ for any fixed s. Why is it correct to fix s and vary only t, rather than ranging over all $\binom{n}{2}$ pairs? Then argue that on an undirected graph you need at most $n-1$ flow computations, and sketch why Gomory–Hu trees encode all $\binom{n}{2}$ pairwise min-cuts in only $n-1$ flows.

Exercise 11.7 (Break a buggy contraction)**.** Suppose a colleague's Karger implementation merges parallel edges (keeps each neighbor relation as a single weight-1 edge, discarding multiplicity). Construct a graph on which this bug makes the algorithm return the min-cut with probability bounded *away* from the correct $\binom{n}{2}^{-1}$, or where it systematically returns the wrong value. Explain precisely which step of Lemma 11.3 fails.

Exercise 11.8 (Near-minimum cuts)**.** Generalize Lemma 11.4: show that for any $\alpha \geq 1$, the number of cuts of value at most $\alpha\lambda(G)$ is $O(n^{2\alpha})$. (Hint: stop the contraction earlier, bound the survival probability of a fixed α-near-minimum cut, then use the disjointness argument of Corollary 11.1.)

Exercise 11.9 (Which cut does Stoer–Wagner find?)**.** Stoer–Wagner returns *a* min-cut; which one if there are several? Run it on C_6 (six min-cuts) under different start vertices and tie-breaking rules, and characterize which of the six it reports. Does the choice of start vertex change the returned *value*? Should it?

Exercise 11.10 (A lower-bound intuition)**.** Argue informally why any correct deterministic global min-cut algorithm that only queries edge weights must, in the worst case, "look at" $\Omega(m)$ edges, and why Stoer–Wagner's $O(nm + n^2 \log n)$ is therefore within a near-linear-in-n factor of an obvious barrier. (This is intuition, not a formal lower bound — formal lower bounds for min-cut are subtle.)

Part III

Flows and Matching

Chapter 12

Maximum Flow

Picture a network of pipes welded together at junctions. One junction, the *source*, has water gushing in from outside; another, the *sink*, lets water drain out. Every pipe has a capacity – a maximum number of liters per second it can carry. The water obeys two rules: it never exceeds a pipe's capacity, and at every junction other than the source and sink, whatever flows in must flow out. The question this chapter answers, completely and with proof, is: how much water can you push from source to sink?

That single question – maximum flow – is one of the most widely applicable problems in combinatorial optimization. Bipartite matching, edge-disjoint paths, project selection, image segmentation, baseball elimination, and a long tail of scheduling problems all reduce to it. The answer is given by a central theorem of the subject: the maximum amount you can push equals the minimum total capacity of pipes whose removal separates source from sink. This chapter proves that maximum flow and minimum cut always coincide and turns that proof into fast algorithms.

12.1 Flow Networks

Definition 12.1 (Flow network)**.** A *flow network* is a directed graph $G = (V, E)$ together with a nonnegative *capacity* $c(u, v) \geq 0$ for every ordered pair (u, v), two distinguished vertices the *source* s and the *sink* t with $s \neq t$. We adopt the convention $c(u, v) = 0$ whenever $(u, v) \notin E$. We assume every vertex lies on some s–t path (vertices that do not can be deleted without changing anything).

Definition 12.2 (Flow)**.** A *flow* is a function $f : V \times V \to \mathbb{R}$ satisfying:

- **Capacity:** $f(u, v) \leq c(u, v)$ for all u, v.
- **Skew symmetry:** $f(u, v) = -f(v, u)$ for all u, v.
- **Conservation:** for every $u \in V \setminus \{s, t\}$, $\sum_{v \in V} f(u, v) = 0$.

The *value* of f is the net flow leaving the source,

$$|f| \;=\; \sum_{v \in V} f(s, v).$$

A *maximum flow* is a flow of largest possible value.

Intuition

Skew symmetry is a bookkeeping trick, not extra physics. If 3 units physically travel from u to v, we write $f(u,v) = +3$ and $f(v,u) = -3$. Conservation then reads "the signed flows out of every internal vertex sum to zero," which is exactly Kirchhoff's current law. The trick lets one equation handle both directions of a pipe and makes the residual graph (below) fall out cleanly. When we draw pictures we only label the positive numbers; the negatives are implicit.

A few sanity facts. The empty flow $f \equiv 0$ is always feasible, so a maximum flow exists whenever the value is bounded – and it is, because $|f| \le \sum_v c(s,v)$. With rational capacities the optimum is attained; with real capacities a maximum still exists by compactness (the feasible flows form a closed bounded polytope, and $|f|$ is linear). The value, measured at the source, equals the value measured at the sink:

Lemma 12.1 (Value is conserved end to end)**.** *For any flow f, $|f| = \sum_v f(s,v) = \sum_v f(v,t)$.*

Proof. Sum conservation over all $u \notin \{s,t\}$: $\sum_{u \neq s,t} \sum_v f(u,v) = 0$. Add the source row to both sides: $\sum_{u \neq t} \sum_v f(u,v) = \sum_v f(s,v) = |f|$. Now the left side double-counts: group the terms $f(u,v)$ by unordered pair $\{u,v\}$ with both endpoints in $V \setminus \{t\}$; each such pair contributes $f(u,v) + f(v,u) = 0$ by skew symmetry. What survives are exactly the pairs touching t, i.e. $\sum_{u \neq t} f(u,t) = -\sum_u f(t,u) = \sum_v f(v,t)$. Hence $|f| = \sum_v f(v,t)$. □

12.1.1 A small network we will carry through the chapter

Here is the running example. Six vertices, source s, sink t, the capacities printed on the arcs.

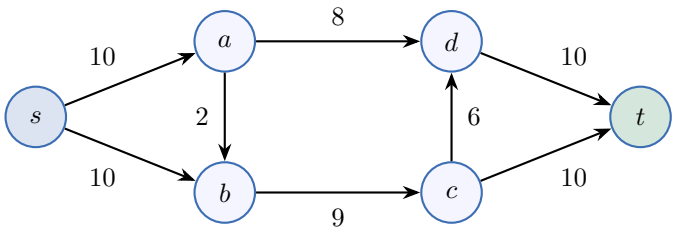


Figure 12.1: The running flow network. Six vertices, eight directed pipes, with capacities labeled on the arcs. Source s is at left, sink t at right.

Eyeballing it: s can emit up to 20. But t absorbs at most $c(c,t) + c(d,t) = 10 + 10 = 20$ as well, so the capacity into t does not bind alone. The actual bottleneck lies elsewhere, and the algorithms below find it. We will compute the maximum flow of this network three different ways.

12.2 Residual Graphs and Augmenting Paths

Almost every max-flow algorithm is built on one idea: given a flow that is not yet maximum, find a way to push a little more, possibly by *rerouting* flow you already committed. The data structure that exposes both options at once is the residual graph.

Definition 12.3 (Residual capacity, residual graph)**.** Given a flow f, the *residual capacity* of an ordered pair is

$$c_f(u, v) \;=\; c(u, v) - f(u, v).$$

The *residual graph* $G_f = (V, E_f)$ has an edge (u, v) exactly when $c_f(u, v) > 0$. An *augmenting path* is a directed s–t path in G_f.

Intuition

Each pipe (u, v) with capacity c carrying f units gives rise to two residual edges. A *forward* edge (u, v) with leftover $c - f$: room to push more. And a *backward* edge (v, u) with residual $c_f(v, u) = c(v, u) - f(v, u) = 0 - (-f) = f$: the ability to *cancel* up to f units of what is already flowing, effectively sending it back. Because of backward edges, a greedy choice can always be corrected later: any commitment can be undone by routing through its reverse.

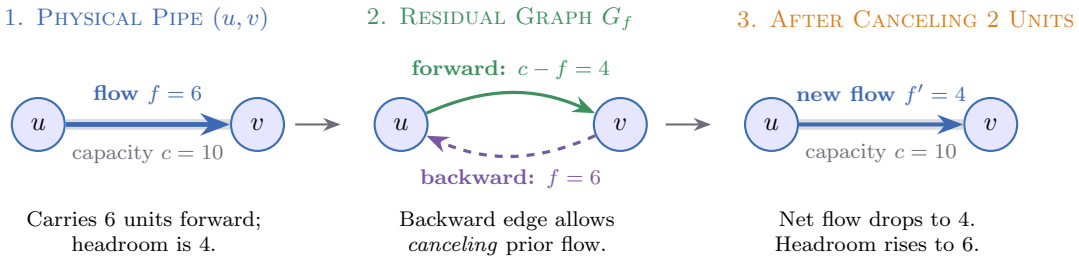


Figure 12.2: The mechanics of residual capacity and flow cancellation. Left: an edge carrying $f = 6$ out of capacity $c = 10$. Center: in the residual graph G_f, the forward arc offers $c - f = 4$ units of additional push, while the backward arc offers $f = 6$ units of cancellation capacity. Right: sending 2 units along the backward arc reduces the net forward flow to 4, freeing up forward headroom without any physical reverse pipe.

If P is an augmenting path, its *bottleneck* is $c_f(P) = \min_{(u,v)\in P} c_f(u, v) > 0$. *Augmenting* along P means

$$f(u, v) \mathrel{+}= c_f(P), \qquad f(v, u) \mathrel{-}= c_f(P) \qquad \text{for each } (u, v) \in P.$$

Figure 12.3 shows what the residual graph of the running network looks like after pushing 8 units along $s \to a \to d \to t$. The three arcs on that path each spawn a dashed backward edge of capacity 8, while the saturated forward arc $a \to d$ has dropped out (residual 0).

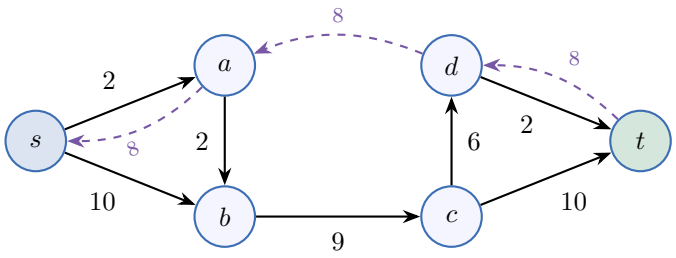


Figure 12.3: Residual graph G_f after pushing 8 units of flow along the augmenting path $s \to a \to d \to t$. Solid arcs show available forward residual capacities $c_f(u,v) = c(u,v) - f(u,v)$. Dashed purple arcs represent backward cancellation capacities $c_f(v,u) = f(u,v) = 8$. The forward arc $a \to d$ is fully saturated and therefore omitted.

Lemma 12.2 (Augmentation preserves feasibility and adds value)**.** *If f is a flow and P an augmenting path with bottleneck $\Delta = c_f(P)$, the function f' produced by augmenting is a flow with $|f'| = |f| + \Delta$.*

Proof. Skew symmetry is maintained because we update (u,v) and (v,u) in lockstep. Capacity: on each $(u,v) \in P$ we raise f by $\Delta \le c_f(u,v) = c(u,v) - f(u,v)$, so $f'(u,v) \le c(u,v)$; the reverse arc only decreases and stays feasible. Conservation at an internal u on P: the path enters u along some arc (x,u) and leaves along some arc (u,y). Augmenting raises $f(u,y)$ by Δ and, since $f(x,u)$ rises by Δ, lowers $f(u,x)$ by Δ. The net outflow $\sum_v f(u,v)$ therefore changes by $+\Delta$ (from $f(u,y)$) plus $-\Delta$ (from $f(u,x)$), which is 0, so conservation still holds. Off the path nothing changes. Finally s is the path's first vertex with exactly one outgoing path-arc, so $|f'| = |f| + \Delta$. □

Common trap

A backward residual edge does *not* mean "a pipe that runs the wrong way." The physical network is unchanged. The residual graph is a momentary accounting of the moves available *right now*; it changes every time you augment. Beginners draw the residual edges once and then forget to recompute them after augmenting. Always update c_f along the path before searching again.

12.3 Ford–Fulkerson and the Max-Flow Min-Cut Theorem

The Ford–Fulkerson method follows a simple rule: *while an augmenting path exists in the residual graph, augment along it.* It is a *method*, not yet an algorithm, because it leaves the choice of path unspecified – and as we will see, that choice determines whether the running time is polynomial, pseudopolynomial, or infinite.

Algorithm 49: FORD–FULKERSON (generic augmenting-path method)

Input: network $G = (V, E, c)$, source s, sink t
Output: a maximum flow f

```
f(u, v) ← 0 for all (u, v);
while G_f contains an s–t path P do
    Δ ← min_{(u,v)∈P} c_f(u, v);
    foreach (u, v) ∈ P do
        f(u, v) += Δ;
        f(v, u) −= Δ;
return f;
```

To prove this terminates with the optimum we need the dual object: cuts.

12.3.1 Cuts

Definition 12.4 (s–t cut)**.** An *s–t cut* is a partition $V = S \cup T$ with $s \in S$, $t \in T$. Its *capacity* is

$$c(S, T) \;=\; \sum_{u \in S} \sum_{v \in T} c(u, v),$$

counting only arcs crossing from S to T. The *net flow across the cut* is $f(S, T) = \sum_{u \in S} \sum_{v \in T} f(u, v)$.

Lemma 12.3 (Flow across any cut equals the value)**.** *For every flow f and every s–t cut (S, T), $f(S, T) = |f|$.*

Proof. Start from $|f| = \sum_v f(s, v)$. For each internal $u \in S$, conservation gives $\sum_v f(u, v) = 0$, so we may add all of them without changing the sum: $|f| = \sum_{u \in S} \sum_{v \in V} f(u, v)$. Split the inner sum by whether $v \in S$ or $v \in T$. The $v \in S$ part pairs each $f(u, v)$ with $f(v, u)$, both endpoints in S; by skew symmetry these cancel. What remains is exactly $\sum_{u \in S} \sum_{v \in T} f(u, v) = f(S, T)$. □

Corollary 12.1 (Weak duality)**.** *For every flow f and every cut (S, T), $|f| \le c(S, T)$.*

Proof. $|f| = f(S, T) = \sum_{u \in S, v \in T} f(u, v) \le \sum_{u \in S, v \in T} c(u, v) = c(S, T)$, using $f \le c$ on each crossing arc. □

So every cut is a certificate of an upper bound on flow, and every flow is a certificate of a lower bound on the minimum cut. If we ever exhibit a flow and a cut with $|f| = c(S, T)$, both are simultaneously optimal. The next theorem says this always happens.

Theorem 12.1 (Max-flow min-cut, Ford–Fulkerson 1956)**.** *Let f be a flow. The following are equivalent:*

1. *f is a maximum flow.*
2. *G_f contains no augmenting path.*
3. *$|f| = c(S, T)$ for some s–t cut (S, T).*

In particular the maximum flow value equals the minimum cut capacity.

Proof. **(1)⇒(2).** If an augmenting path existed, Lemma 12.2 would produce a strictly larger flow, contradicting maximality.

(2)⇒(3). Suppose G_f has no s–t path. Let

$$S = \{v : v \text{ is reachable from } s \text{ in } G_f\}, \qquad T = V \setminus S.$$

Then $s \in S$ and, since t is unreachable, $t \in T$, so (S, T) is a cut. Take any $u \in S$, $v \in T$. Because v is not reachable, $(u, v) \notin E_f$, i.e. $c_f(u, v) = c(u, v) - f(u, v) = 0$, so $f(u, v) = c(u, v)$: every crossing arc is saturated. Therefore $f(S, T) = \sum_{u \in S, v \in T} c(u, v) = c(S, T)$. By Lemma 12.3, $|f| = f(S, T) = c(S, T)$.

(3)⇒(1). If $|f| = c(S, T)$ for some cut, then for every flow g, weak duality gives $|g| \le c(S, T) = |f|$, so f is maximum. □

Why it works

The proof also describes an algorithm. When Ford–Fulkerson stops, it stops precisely because no augmenting path exists – condition (2). The set S of residual-reachable vertices is then a minimum cut, computed for free by one graph search from s in G_f. So you never run a separate min-cut routine: max-flow hands you the certificate. This is a combinatorial instance of LP duality.

Corollary 12.2 (Integrality theorem)**.** *If all capacities are integers, there is a maximum flow in which every $f(u, v)$ is an integer, and Ford–Fulkerson finds one.*

Proof. Start from $f \equiv 0$ (integral). Each bottleneck $\Delta = c_f(P)$ is a minimum of integers, hence a positive integer, and augmentation keeps f integral. The value rises by at least 1 each round and is bounded by any cut capacity, so the process halts at an integral maximum flow. □

This corollary is why flow is such a useful modeling tool: feed it an integer program of the right shape and it returns an integer solution, no rounding.

12.3.2 Tracing Ford–Fulkerson on the running network

Trace it yourself

Capacities as in Section 12.1. We pick augmenting paths greedily by DFS and watch the value climb.

Round 1. Path $s \to a \to d \to t$, residuals $10, 8, 10$, bottleneck 8. Push 8. Now $f(s, a) = 8$, $f(a, d) = 8$, $f(d, t) = 8$; value $= 8$.

Round 2. Path $s \to b \to c \to t$, residuals $10, 9, 10$, bottleneck 9. Push 9. Value $= 17$.

Round 3. Hunt for any augmenting path. Through $c \to t$ we still have $c_f(c, t) = 1$, but reaching c needs $b \to c$, which is saturated ($c_f(b, c) = 0$); the alternative into d is $a \to d$, also saturated ($c_f(a, d) = 0$). From s we can still reach a ($c_f = 2$) and through it b via $a \to b$ ($c_f = 2$), but from b the only forward arc $b \to c$ is dead. No augmenting path survives.

The residual-reachable set from s is $S = \{s, a, b\}$. Crossing arcs $S \to T$: $a \to d$ (cap 8, saturated) and $b \to c$ (cap 9, saturated). Cut capacity $= 8 + 9 = 17 = |f|$. Confirmed maximum.

The maximum flow value is 17, and the minimum cut $\{s, a, b\} \mid \{c, d, t\}$ has capacity 17. Note that neither the source's out-capacity (20) nor the sink's in-capacity (20) was the bottleneck – the constriction sits in the middle, on the two arcs $a \to d$ and $b \to c$.

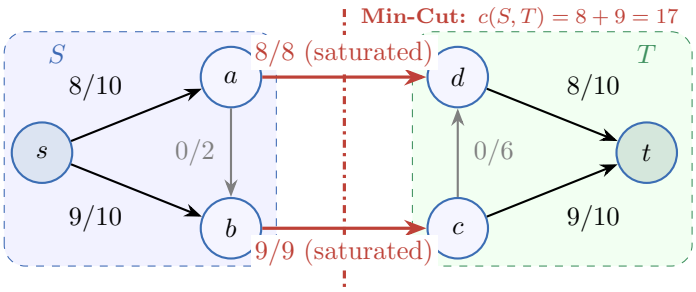


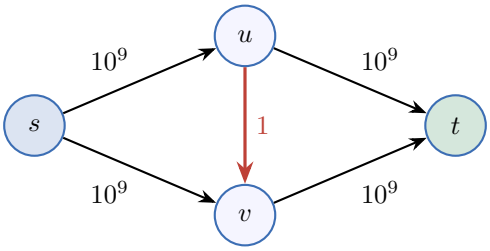
Figure 12.4: The maximum flow $|f| = 17$ alongside its dual minimum cut (S, T) with capacity $c(S, T) = 17$. Flow over capacity (f/c) is labeled on each arc. The two crossing arcs $a \to d$ and $b \to c$ are entirely saturated, certifying that no additional flow can reach T from S.

12.3.3 Does Ford–Fulkerson even terminate?

With integer capacities, yes (Corollary 12.2): the value rises by ≥ 1 per augmentation, so at most $|f^\star|$ rounds occur, where $f^\star$ is the optimum. That gives a running time of $O(E \cdot |f^\star|)$ – each augmenting path costs $O(E)$ to find by BFS/DFS. This is *pseudopolynomial*: fine when $|f^\star|$ is small, terrible when capacities are huge.

Common trap

The classic adversarial network: four vertices s, u, v, t with $c(s, u) = c(s, v) = c(u, t) = c(v, t) = 10^9$ and a single middle arc $c(u, v) = 1$. If your path chooser keeps picking $s \to u \to v \to t$ and then $s \to v \to u \to t$ (using the backward edge of the middle arc), each augmentation moves only 1 unit, and you take 2×10^9 rounds for a flow of value 2×10^9. With *irrational* capacities one can even make Ford–Fulkerson run forever and converge to the wrong value. The fix is to choose augmenting paths by a rule that bounds how many are needed.



12.4 Edmonds–Karp: Shortest Augmenting Paths

The remedy is simple: *always augment along a shortest augmenting path* – a path with the fewest edges, discovered by breadth-first search in G_f. Jack Edmonds and Richard Karp proved in 1972 that this rule guarantees a polynomial running time, independent of the numeric capacities.

Algorithm 50: EDMONDS–KARP

Input: network G, source s, sink t

$f \leftarrow 0$;
while *BFS in* G_f *finds a shortest* s–t *path* P **do**
  augment f along P by its bottleneck $c_f(P)$;
return f;

The analysis rests on how BFS distances in the residual graph behave as we augment. Write $\delta_f(v)$ for the number of edges on a shortest s–v path in G_f (∞ if unreachable).

Lemma 12.4 (Residual distances never decrease)**.** *Under Edmonds–Karp, $\delta_f(v)$ is nondecreasing over time for every v.*

Proof. Suppose augmenting $f \to f'$ strictly decreased some distance; among all such vertices pick v with the smallest $\delta_{f'}(v)$. Let u be the predecessor of v on a shortest s–v path in $G_{f'}$, so $\delta_{f'}(v) = \delta_{f'}(u) + 1$ and $(u, v) \in E_{f'}$. By the minimal choice of v, $\delta_{f'}(u) \geq \delta_f(u)$.

Case A: $(u, v) \in E_f$ already. Then $\delta_f(v) \leq \delta_f(u)+1 \leq \delta_{f'}(u)+1 = \delta_{f'}(v)$, contradicting the decrease.

Case B: $(u, v) \notin E_f$ but $(u, v) \in E_{f'}$. The only way a residual edge appears is that we pushed flow along (v, u), and since Edmonds–Karp augments a *shortest* path, that path used (v, u), giving $\delta_f(v) = \delta_f(u) - 1$. Then $\delta_f(v) = \delta_f(u) - 1 \leq \delta_{f'}(u) - 1 = \delta_{f'}(v) - 2 < \delta_{f'}(v)$, again contradicting a decrease. □

Theorem 12.2 (Edmonds–Karp bound)**.** *Edmonds–Karp performs $O(VE)$ augmentations, hence runs in $O(VE^2)$ time.*

Proof. Call an edge (u, v) *critical* on an augmentation if it is the bottleneck (its residual hits 0 and it leaves E_f). When (u, v) is critical it lies on a shortest path, so $\delta_f(u) + 1 = \delta_f(v)$. The edge can reappear in E_f only after we push flow back along (v, u) in some later augmentation, which (shortest path again) requires $\delta_{f'}(v) + 1 = \delta_{f'}(u)$ at that time. By Lemma 12.4, distances only grew, so

$$\delta_{f'}(u) = \delta_{f'}(v) + 1 \geq \delta_f(v) + 1 = \delta_f(u) + 2.$$

Thus between two successive times (u, v) is critical, $\delta(u)$ rises by at least 2. Since $\delta(u) \in \{0, 1, \ldots, V-1\}$ while u is reachable (once $\delta(u) \geq V$ the vertex is gone for good), each edge is critical $O(V)$ times. Every augmentation makes at least one edge critical, and there are $O(E)$ residual edges, so there are $O(VE)$ augmentations. Each costs $O(E)$ for the BFS, giving $O(VE^2)$. □

Figure 12.5 illustrates why an edge (u, v) requires at least two units of distance growth at u between consecutive criticalities: pushing flow forward requires v to be at level $\delta(u) + 1$, while reactivating (u, v) demands a backward push along (v, u) on some later shortest path, which can only occur once u sits at level $\delta'(v) + 1 \geq \delta(u) + 2$.

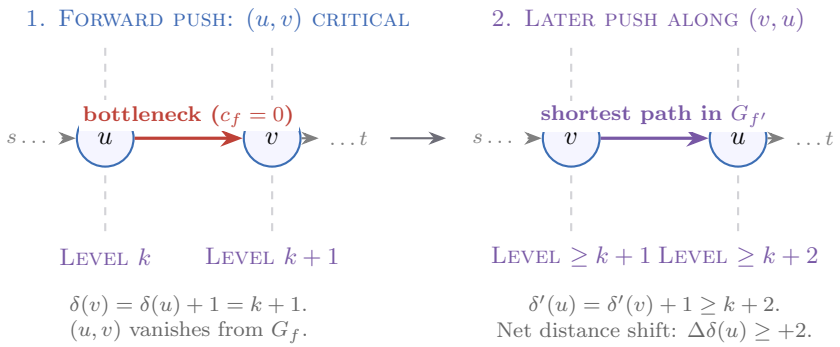


Figure 12.5: The distance leap between successive criticalities of edge (u, v) in Edmonds–Karp. In panel 1, (u, v) is the bottleneck on a shortest augmenting path, so $\delta(v) = \delta(u) + 1 = k + 1$, after which (u, v) drops out of G_f. In panel 2, to reappear in G_f, a subsequent shortest path must push flow backwards along (v, u). Because BFS distances are monotonic ($\delta'(v) \geq \delta(v) = k + 1$), the new distance to u must satisfy $\delta'(u) = \delta'(v) + 1 \geq k + 2$. Hence u's BFS level climbs by at least 2 before (u, v) can be critical again.

Because distance levels cannot exceed $V-1$ while remaining reachable from s, each of the $2E$ residual directed edges is critical at most $(V-1)/2 = O(V)$ times, yielding the $O(VE)$ augmentation bound.

Cost summary

Ford–Fulkerson (generic): $O(E\,|f^\star|)$, pseudopolynomial, may not terminate on irrational data. **Edmonds–Karp:** $O(VE^2)$, strongly polynomial, capacity-independent. The win is purely from picking shortest paths – the same code skeleton, one BFS instead of an arbitrary search.

12.5 Dinic's Algorithm: Level Graphs and Blocking Flows

Edmonds–Karp augments one path per BFS. That is wasteful: a single BFS computes distances to *every* vertex, yet we use it for one path. Dinic's algorithm (Yefim Dinitz, 1970) reuses each BFS to push a whole *blocking flow* – many paths – before recomputing distances. The result is a strict improvement, and on the networks arising from matching it is much faster.

12.5.1 The level graph

Definition 12.5 (Level graph)**.** Given G_f, run BFS from s to get levels $\ell(v) = \delta_f(v)$. The *level graph* G_L keeps only the *advancing* residual edges – those $(u, v) \in E_f$ with $\ell(v) = \ell(u) + 1$.

Every s–t path in G_L has exactly $\ell(t)$ edges, the current shortest distance. So pushing flow within G_L can only use shortest augmenting paths, all of the same length. Figure 12.6 draws the level graph of the running network in Dinic's first phase: the columns are the BFS levels, and only forward-advancing arcs survive.

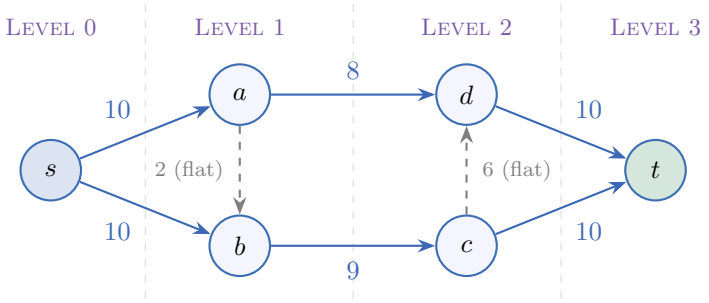


Figure 12.6: Level graph G_L for Dinic's first phase on the running network. Columns display the BFS distance levels from s. Solid blue arcs are strictly advancing ($\ell(v) = \ell(u) + 1$). Dashed gray arcs $a \to b$ and $c \to d$ are level-flat ($\ell(v) = \ell(u)$) and excluded. Every s–t path in this DAG has length exactly $\ell(t) = 3$.

Definition 12.6 (Blocking flow)**.** A flow g in the level graph G_L is *blocking* if every s–t path in G_L contains at least one edge saturated by g. Equivalently: you cannot augment any more *within* G_L.

Intuition

A blocking flow is not a maximum flow of G_L – it can leave usable capacity on non-shortest routes. It only promises that no path of the *current shortest length* survives unsaturated. That is exactly enough: once we saturate all shortest paths, the next BFS will find that $\delta_f(t)$ has strictly increased.

12.5.2 The phase structure

Dinic proceeds in *phases*. Each phase: (1) BFS to build G_L; (2) find a blocking flow in G_L and add it to f. Repeat until t is unreachable.

Lemma 12.5 (Distance strictly increases per phase)**.** *After augmenting f by a blocking flow of its level graph, $\delta_f(t)$ strictly increases.*

Proof. Let the phase start with $\delta_f(t) = k$. By monotonicity (Lemma 12.4, whose proof only used that we augmented along shortest paths – true of every path in G_L), no shortest distance ever shrinks, so afterward $\delta_{f'}(t) \geq k$. Suppose for contradiction $\delta_{f'}(t) = k$. A length-k s–t path in $G_{f'}$ must climb from level 0 to level k in k steps, hence every step goes from level i to level $i + 1$: it is built entirely of advancing edges, so it lives in G_L. (New residual edges created this phase point from level i to $i - 1$ and cannot occur on a length-k ascent.) But the blocking flow saturated some edge on every such path. Contradiction; hence $\delta_{f'}(t) \geq k + 1$. □

Corollary 12.3. *Dinic runs at most $V - 1$ phases.*

Proof. $\delta_f(t)$ starts at ≥ 1, strictly increases each phase, and once it exceeds $V - 1$ no simple s–t path exists, so t is unreachable and we stop. □

12.5.3 Finding a blocking flow

Within a level graph we push flow with a DFS that has one essential optimization: the *current-arc* pointer (dead-end pruning). When a DFS from u finds that an edge leads to a dead end, it never tries that edge again this phase. This is implemented by keeping, for each u, a pointer `it[u]` into its advancing-edge list that only ever moves forward.

Algorithm 51: DINIC – one phase, blocking flow by current-arc DFS

```
   Input: levels ℓ(·), residual c_f, pointers it[·]
 1 Function Dfs(u, pushed):
 2     if u = t then return pushed;
 3     while it[u] points at a valid edge (u, v) do
 4         if c_f(u, v) > 0 and ℓ(v) = ℓ(u) + 1 then
 5             d ← Dfs(v, min(pushed, c_f(u, v)));
 6             if d > 0 then
 7                 f(u, v) += d;   f(v, u) −= d;
 8                 return d ;                  // keep it[u]:  (u, v) may still have room
 9         advance it[u] ;                                  // (u, v) is dead this phase
10     return 0;
11 while Dfs(s, ∞) > 0 do
       ;                                       // push until the level graph is blocked
```

Why it works

Why the current-arc pointer makes a phase cheap. Each DFS call from s either reaches t (a successful push, saturating at least one edge on the path) or hits a dead end at some vertex u, at which point `it[u]` advances past the offending edge *permanently for this phase*. So every residual edge is "retreated past" at most once per phase. The successful paths have length $\leq V$, and at least one edge saturates per successful path, so the total work is bounded by the edge events: $O(VE)$ per phase.

Theorem 12.3 (Dinic complexity)**.** *Dinic's algorithm computes a maximum flow in $O(V^2E)$ time.*

Proof. By Corollary 12.3 there are $O(V)$ phases. We show each phase's blocking flow costs $O(VE)$. In the current-arc DFS, charge work to two kinds of events. *Advances of `it[u]`*: each residual edge is skipped at most once per phase, total $O(E)$. *Saturations*: each successful augmenting DFS path has length $\leq V$ and saturates at least one of its edges (the bottleneck), so the number of successful paths is $O(E)$ and the work along them is $O(VE)$. Summing, one phase is $O(VE)$ and the whole run is $O(V^2E)$. □

Trace it yourself

Dinic on the running network. **Phase 1**: BFS gives levels $\ell(s) = 0$, $\ell(a) = \ell(b) = 1$, $\ell(c) = \ell(d) = 2$, $\ell(t) = 3$. Advancing edges include $s \to a, s \to b, a \to d, b \to c, c \to t, d \to t$ (the arc $a \to b$ is level-flat, dropped; $c \to d$ is level-flat, dropped). The blocking-flow DFS pushes $s \to a \to d \to t$ (bottleneck 8) and $s \to b \to c \to t$ (bottleneck 9), then $s \to a$ has 2 left but

a's only advancing arc $a \to d$ is saturated, dead. Value after phase 1 is 17. **Phase 2**: BFS in the new residual graph cannot reach t (the cut $\{s, a, b\}$ is saturated), so $\delta_f(t) = \infty$ and we stop. Two phases, optimal value 17 – compare the three rounds of greedy Ford–Fulkerson.

History

Maximum flow was first studied during the Cold War. In 1955–1956 at the RAND Corporation, Lester Ford and Delbert Fulkerson studied the throughput of the Soviet railway system to Eastern Europe, asking which rail links an adversary would need to sever to choke off supplies. Their max-flow min-cut theorem showed that the maximum flow volume equals the minimum cut capacity, unifying earlier structural theorems by Karl Menger and Dénes Kőnig. Their original augmenting-path method, however, could spin indefinitely on irrational capacities or take exponential time on pathological networks.

The search for strongly polynomial algorithms produced two independent solutions. In 1970, Yefim Dinitz in Moscow invented level graphs and blocking flows to solve a graph problem set assigned by Georgy Adelson-Velsky. Across the Atlantic, Jack Edmonds and Richard Karp proved in 1972 that choosing shortest augmenting paths via breadth-first search guarantees termination in $O(VE^2)$ time. For two decades, Dinitz's work circulated in Western computer science under the misspelled transliteration "Dinic". His blocking-flow idea led to Alexander Karzanov's $O(V^3)$ algorithm and paved the way for preflow-push methods. Decades later, work toward linear-time algorithms led to the almost-linear-time flow solvers of the 2020s, which are built on interior-point methods and electrical flows.

12.5.4 Special cases where Dinic is much faster

Theorem 12.4 (Unit-capacity networks)**.** *On a network with all capacities* 1*, Dinic runs in* $O(E\sqrt{E})$ *time. On the unit networks arising from bipartite matching it runs in* $O(E\sqrt{V})$*.*

Proof sketch. With unit capacities each blocking flow costs only $O(E)$ (every edge saturates the first time it carries flow, so the DFS is linear). The phase count is the subtle part. Run the first $\sqrt{E}$ phases normally; afterward $\delta_f(t) \geq \sqrt{E}$. The flow still to be pushed, $|f^\star| - |f|$, decomposes into s–t paths in the residual graph, each of length $\geq \sqrt{E}$ and (by unit capacity) edge-disjoint, so there are at most $E/\sqrt{E} = \sqrt{E}$ of them; each remaining phase pushes ≥ 1 unit, bounding the rest to $\sqrt{E}$ phases. Total $O(\sqrt{E})$ phases $\times\, O(E)$ each. For matching networks edge-disjointness becomes vertex-disjointness and $|f^\star| \leq V$, giving the $\sqrt{V}$ bound. □

The $O(E\sqrt{V})$ bound matches Hopcroft–Karp for bipartite matching, which is equivalent to running Dinic's algorithm on the unit-capacity matching network – a connection we develop in Chapter 15.

12.6 Capacity Scaling

There is a different polynomial route that keeps the simple augment-along-a-path structure but tames large capacities directly: *scaling*. The idea is to push fat augmentations first.

Intuition

If you only allow yourself augmenting paths whose bottleneck is at least some threshold Δ, every augmentation moves $\geq \Delta$ units, so there cannot be too many of them. Start Δ at the largest power of two $\leq C_{\max}$, run Ford–Fulkerson restricted to residual edges of capacity $\geq \Delta$, then halve Δ and repeat. By the time $\Delta = 1$ you are doing ordinary Ford–Fulkerson, but the flow is already nearly optimal, so few augmentations remain.

Algorithm 52: Capacity Scaling

Input: network with integer capacities, $C_{\max} = \max_{u,v} c(u, v)$

$f \leftarrow 0$; $\Delta \leftarrow 2^{\lfloor \log_2 C_{\max} \rfloor}$;
while $\Delta \geq 1$ **do**
  while *G_f has an s–t path P using only edges with $c_f \geq \Delta$* **do**
    augment f along P;
  $\Delta \leftarrow \Delta/2$;
return f;

Theorem 12.5 (Scaling bound)**.** *Capacity scaling makes $O(E \log C_{\max})$ augmentations and runs in $O(E^2 \log C_{\max})$ time.*

Proof. There are $1 + \lfloor \log_2 C_{\max} \rfloor = O(\log C_{\max})$ scaling phases. We bound the augmentations *within one phase* at threshold Δ by $O(E)$. At the end of the Δ-phase, no augmenting path uses only edges with $c_f \geq \Delta$; let (S, T) be the residual-reachable cut in the graph of such edges. Every crossing arc has $c_f(u, v) < \Delta$, so the residual capacity across the cut is $< E \cdot \Delta$. Hence $|f^\star| - |f| < E\Delta$ at that moment. Entering the next phase, the threshold is $\Delta/2$, and each augmentation there moves $\geq \Delta/2$ units, so the number of augmentations is at most $(E\Delta)/(\Delta/2) = 2E = O(E)$. Multiplying by $O(\log C_{\max})$ phases gives $O(E \log C_{\max})$ augmentations, each $O(E)$ to find. □

Cost summary

Scaling: $O(E^2 \log C_{\max})$. It is *weakly* polynomial – the $\log C_{\max}$ factor depends on the numbers – whereas Edmonds–Karp and Dinic are *strongly* polynomial. Scaling shines when E is small but capacities huge, and the technique generalizes to min-cost flow (later chapters).

Trace it yourself

Run scaling on the bad network of the trap above ($c(s, u) = c(u, t) = c(s, v) = c(v, t) = 10^9$, middle arc $c(u, v) = 1$). $C_{\max} = 10^9$, so Δ starts at 2^{29}. The first phase, with $\Delta = 2^{29}$, ignores the middle arc entirely (its capacity $1 < \Delta$) and pushes 2^{29} along $s \to u \to t$ and 2^{29} along $s \to v \to t$. Continuing to halve Δ, each phase pushes the two fat disjoint paths and *never* routes through the 1-capacity arc until $\Delta = 1$, where a single unit may cross. The whole computation completes in $O(\log 10^9) \approx 30$ augmentations – compared to 2×10^9 for naive Ford–Fulkerson. Because it pushes large augmentations first, scaling is not slowed down by huge capacities.

Figure 12.7 shows how capacity scaling filters edges on this network: at threshold $\Delta = 2^{29}$, the bottleneck arc $u \to v$ is below threshold and invisible to the search, so flow goes along the two wide

parallel paths instead of zigzagging.

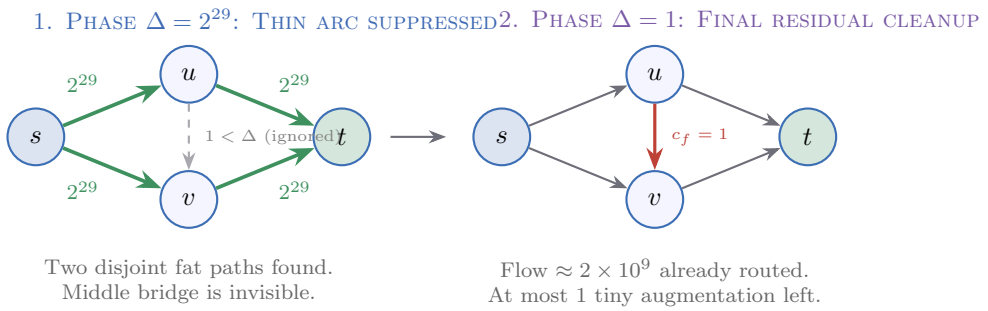


Figure 12.7: Capacity scaling on the pathological 10^9-capacity network. In early phases (panel 1), the Δ-filter suppresses edges with $c_f(e) < \Delta$, hiding the 1-capacity bridge from the search and pushing 2^{29} flow along direct parallel paths without zigzagging. In the final phase ($\Delta = 1$, panel 2), nearly the entire $2 \cdot 10^9$ flow has already settled, requiring at most $O(E)$ minor final adjustments.

Only when Δ has scaled down to 1 is the tiny bridge examined, by which time the bulk flow is already optimal.

12.7 Putting the Bounds Side by Side

Algorithm	Time	Notes
Ford–Fulkerson (generic)	$O(E\,\lvert f^\star\rvert)$	pseudopoly; can loop on reals
Edmonds–Karp	$O(VE^2)$	shortest path, strongly poly
Capacity scaling	$O(E^2 \log C_{\max})$	weakly poly, fat paths first
Dinic	$O(V^2E)$	blocking flows, $O(V)$ phases
Dinic, unit capacity	$O(E\sqrt{E})$	saturate-once DFS
Dinic, bipartite matching	$O(E\sqrt{V})$	= Hopcroft–Karp

For dense graphs Dinic's $O(V^2E)$ beats Edmonds–Karp's $O(VE^2)$; for sparse huge-capacity graphs scaling wins; for unit/matching networks Dinic runs in $O(E\sqrt{E})$ or $O(E\sqrt{V})$ time. Chapter 13 reaches $O(V^3)$ and $O(V^2\sqrt{E})$ via push–relabel preflows.

Connection

Where this plugs into the rest of the book.

- *Min-cut* (Chapter 11.1): the s–t min-cut is a free byproduct here via residual reachability; the global min-cut runs s–t cuts over many sinks or uses Stoer–Wagner.
- *Matching* (bipartite and general): bipartite matching is integral max-flow on a unit network; Dinic gives Hopcroft–Karp's $O(E\sqrt{V})$.
- *Min-cost flow* and *weighted matching*: replace "shortest by edges" with "cheapest by cost," and scaling reappears as cost-scaling.
- *Laplacians and electrical flows* (later chapters): the modern $O(E^{1+o(1)})$ max-flow al-

gorithms route flow by repeatedly solving Laplacian systems – electrical flow is the continuous relaxation of combinatorial flow. Max-flow min-cut is the discrete counterpart of continuous potential-flow duality, and the residual graph is the discrete gradient you will meet again as the **L** operator.

- *LP duality:* Theorem 12.1 is strong duality for the flow LP; weak duality (Corollary 12.1) is the easy half.

Application

Vertex capacities and undirected edges are the two modeling moves you will reach for constantly. To cap a vertex v at κ, split it into $v_{\text{in}} \to v_{\text{out}}$ with capacity κ and route all in-arcs to v_{in}, all out-arcs from v_{out}. To model an undirected edge of capacity c, add both (u, v) and (v, u) with capacity c; a max-flow uses at most one direction net. With these two gadgets and integrality, max-flow solves edge- and vertex-disjoint paths (Menger), bipartite matching (Kőnig), project selection / minimum closure, image segmentation (Boykov–Kolmogorov graph cuts), and baseball elimination. The flow-modeling chapter is a catalog of such reductions.

Every augmenting-path algorithm pushes fluid along an entire route from source to sink before re-evaluating the residual graph. In the next chapter, we abandon global paths entirely. By introducing local *preflows* and height functions, the push–relabel method moves fluid locally from higher vertices to lower vertices, achieving faster bounds without global path searches.

12.8 Exercises

Exercise 12.1 (Trace by hand)**.** On the running network of Section 12.1, run Edmonds–Karp, recording each BFS shortest path, its bottleneck, and the running value. Confirm the final value is 17 and that the residual-reachable set from s is $\{s, a, b\}$. How many augmentations did Edmonds–Karp use, versus the three of the greedy DFS trace?

Exercise 12.2 (Min-cut from a max-flow)**.** Given a maximum flow f, prove that the set S of vertices reachable from s in G_f defines a minimum cut, and that the set of vertices that can *reach* t in G_f defines (by its complement) another, possibly different, minimum cut. Construct a network with at least two distinct minimum cuts.

Exercise 12.3 (Integrality, sharpened)**.** Show that if every capacity is an even integer, some maximum flow assigns even flow to every edge. Generalize: if capacities are multiples of k, a maximum flow exists with all flows multiples of k. Where in the argument does Corollary 12.2 get used?

Exercise 12.4 (Break Ford–Fulkerson)**.** Implement the four-vertex trap network with the 1-capacity middle arc and a path-chooser that alternates $s \to u \to v \to t$ and $s \to v \to u \to t$. Confirm it takes $\Theta(C_{\max})$ augmentations. Then switch to BFS path selection and show it finishes in two augmentations. Explain the difference using Lemma 12.4.

Exercise 12.5 (Edmonds–Karp tightness)**.** The $O(VE^2)$ bound counts $O(VE)$ augmentations. Construct a family of networks (e.g. layered graphs) where the number of augmentations is $\Omega(VE)$, demonstrating that the analysis is tight up to constants. (Hint: force each edge to become critical $\Omega(V)$ times.)

Exercise 12.6 (Blocking flow $\neq$ max flow of the level graph)**.** Give a level graph and a blocking flow g on it that is *not* a maximum flow of the level graph (some s–t capacity remains unused on a longer route). Explain why Dinic does not care, citing Lemma 12.5.

Exercise 12.7 (Current-arc pointer correctness)**.** Prove that resetting the current-arc pointers `it[·]` at the start of each phase (but never within a phase) yields a correct blocking flow. Then exhibit a bug: forgetting to reset between phases. What goes wrong, and is the output still a valid flow (just not maximum), or is it infeasible?

Exercise 12.8 (Unit-capacity phases)**.** Fill in the proof of Theorem 12.4: after $\sqrt{E}$ phases of Dinic on a unit network, $\delta_f(t) \geq \sqrt{E}$; conclude the remaining flow value is $\leq \sqrt{E}$ using edge-disjointness of the residual augmenting paths. Where do you use that capacities are exactly 1?

Exercise 12.9 (Scaling correctness)**.** Prove that capacity scaling terminates with a maximum flow, not merely a large one. (Hint: the final $\Delta = 1$ phase is ordinary Ford–Fulkerson; quote Theorem 12.1.) Then re-derive the $O(E \log C_{\max})$ augmentation bound, being explicit about why the residual cut across the Δ-frontier has capacity $< E\Delta$.

Exercise 12.10 (Vertex capacities and Menger)**.** Using the split-vertex gadget from the application box, model a network where each *vertex* (not edge) has a throughput limit, and prove the gadget preserves the max-flow value exactly. Then formulate Menger's vertex-connectivity theorem as a max-flow statement and prove it as a corollary of Theorem 12.1.

Exercise 12.11 (Antisymmetric vs. nonnegative flows)**.** We defined flow with skew symmetry ($f(u,v) = -f(v,u)$). An alternative is "nonnegative flows": $f(u,v) \geq 0$ on real arcs with conservation $\sum_v f(v,u) = \sum_w f(u,w)$. Show the two formulations describe the same feasible values, and translate the residual-graph construction between them. Which is cleaner for implementation, and why do most libraries store an edge and its reverse adjacently in an edge array?

Exercise 12.12 (Lower bounds on edges)**.** Extend the model so each edge has a *lower bound* $\ell(u,v) \leq f(u,v) \leq c(u,v)$. Describe the standard reduction to ordinary max-flow (a super-source and super-sink absorbing the excess demands) and state when a feasible flow exists. Prove that your feasibility condition is both necessary and sufficient.

Chapter 13

Push-Relabel and Modern Max-Flow

The augmenting-path family of max-flow algorithms – Ford–Fulkerson, Edmonds–Karp, Dinic – all share one organizing idea: keep a *valid* flow at all times, and repeatedly push more of it from source to sink along whole augmenting paths in the residual graph. Every intermediate object the algorithm holds is a feasible flow; the algorithm walks uphill in flow value until no augmenting path remains, and min-cut duality certifies optimality.

Push-relabel, invented by Goldberg and Tarjan in 1986, drops the global path discipline. It maintains an object that is *not* a flow – a *preflow*, in which a vertex may temporarily receive more than it sends out – and it moves flow one edge at a time, governed by vertex *heights* that act like potential energy. Flow only ever runs downhill. Once no active vertices remain, any excess that could not reach the sink has drained back to the source, the preflow becomes a valid flow, and that flow is maximum. The result is a family of purely local algorithms that achieve $O(V^3)$ time under FIFO selection and $O(V^2\sqrt{E})$ under highest-label selection without ever searching for an augmenting path – and that, combined with the gap and global-relabeling heuristics, are among the fastest general-purpose max-flow solvers in practice.

13.1 Preflows: dropping conservation

Fix a flow network: a directed graph $G = (V, E)$ with a source s, a sink t, and a nonnegative integer capacity $c(u, v)$ on every ordered pair, where $c(u, v) = 0$ for non-edges. We use the standard antisymmetric convention for flows: a function $f : V \times V \to \mathbb{R}$ is a *flow* if it obeys capacity ($f(u, v) \le c(u, v)$), skew symmetry ($f(u, v) = -f(v, u)$), and conservation ($\sum_v f(u, v) = 0$ for $u \notin \{s, t\}$).

A *preflow* keeps the first two and relaxes the third in one direction only.

Definition 13.1 (Preflow)**.** A function $f : V \times V \to \mathbb{R}$ is a *preflow* if it satisfies capacity and skew symmetry, and for every vertex $u \in V \setminus \{s\}$ the *excess*

$$e(u) \;=\; \sum_{v \in V} f(v, u) \;=\; (\text{flow in}) - (\text{flow out}) \;\ge\; 0.$$

A vertex $u \notin \{s, t\}$ with $e(u) > 0$ is called *active*.

So a preflow lets flow pile up at interior vertices, but never lets a vertex leak out more than came in. The algorithm's objective is to discharge that excess: push it forward toward t, or, if downstream paths are blocked, push it backward toward s. A preflow with *no* active vertices is an exact flow, and at that moment $e(t)$ equals the flow value $|f|$.

Intuition

Think of the network as a system of pipes laid on a hillside, and think of $e(u)$ as a puddle of water standing at junction u. Water only flows downhill. Push-relabel keeps raising junctions (relabeling) so that some downhill pipe opens up, then lets the puddle slosh down it (pushing). Junctions near s end up high, junctions near t stay low, and water finds its way to the sink. When no puddle can move, the water has settled into a maximum flow, and any water that got stranded behind a ridge runs back down to the source.

The residual graph is exactly as in the augmenting-path world. The residual capacity is $c_f(u,v) = c(u,v) - f(u,v)$, and the residual graph G_f contains every edge (u,v) with $c_f(u,v) > 0$. A residual edge (u,v) either has slack on a forward capacity edge or represents flow on (v,u) that we are allowed to cancel.

13.2 Heights and the two operations

The method has exactly two operations. To control them we attach to every vertex a *height* (also called a label or distance label) $h : V \to \mathbb{N}$.

Definition 13.2 (Height function)**.** A function $h : V \to \mathbb{N}$ is a *valid height function* for a preflow f if

$$h(s) = |V|, \qquad h(t) = 0, \qquad \text{and} \qquad h(u) \le h(v) + 1 \ \text{ for every residual edge } (u,v) \in G_f.$$

The constraint $h(u) \le h(v) + 1$ is the key invariant of the method. It says heights drop by at most one as you cross any residual edge, so heights are a *Lipschitz* function along G_f. Two consequences fall out immediately and we use them constantly:

- If $h(u) > h(v) + 1$ then $(u,v) \notin G_f$, i.e. the edge is saturated (no residual capacity). A steep downhill drop cannot exist.
- There is never a residual path from s to t. Walk any *simple* residual path $s = u_0, u_1, \ldots, u_k = t$. Each step drops height by at most one, so $h(s) \le h(t) + k = k$. But $h(s) = |V|$, forcing $k \ge |V|$, impossible for a simple path on $|V|$ vertices, which has at most $|V| - 1$ edges.

The second bullet is the key guarantee of the method: *as long as a valid height function exists, there is no augmenting path.* So if we can drive the preflow to an actual flow while keeping heights valid, that flow is automatically maximum.

Push

Excess at u can move to a neighbor v only if (a) there is residual capacity on (u,v), and (b) v is strictly lower, $h(u) = h(v) + 1$. Water runs downhill, and the validity constraint guarantees the only downhill residual edges are exactly the ones one step down.

Algorithm 53: PUSH(u, v) – applies when $e(u) > 0$, $c_f(u, v) > 0$, $h(u) = h(v) + 1$

```
1 δ ← min(e(u), c_f(u, v))
2 f(u, v) ← f(u, v) + δ
3 f(v, u) ← f(v, u) − δ                    // maintain skew symmetry
4 e(u) ← e(u) − δ
5 e(v) ← e(v) + δ
```

We call the push *saturating* if $\delta = c_f(u, v)$ (the edge leaves G_f afterward) and *nonsaturating* otherwise (then $e(u)$ becomes 0, so u goes inactive). This distinction drives the entire complexity analysis.

Figure 13.1 shows the two cases: a saturating push uses up the edge's residual capacity, removing the admissible arc from G_f while possibly leaving u active; a nonsaturating push exhausts u's excess completely, deactivating u while preserving leftover capacity on (u, v).

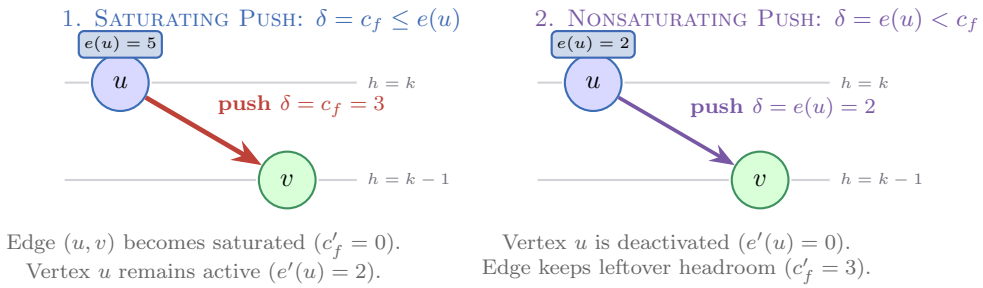


Figure 13.1: The two outcomes of a downhill push. In a saturating push (left), the pushed volume equals the available residual capacity $\delta = c_f(u, v)$, removing the edge from G_f while u retains leftover excess. In a nonsaturating push (right), the entire excess of u is drained ($\delta = e(u)$), deactivating u while the edge survives in G_f with positive residual capacity.

Relabel

An active vertex u with no admissible downhill edge is stuck: it has excess to get rid of but every residual neighbor is at least as high. We lift it just high enough to open one edge.

Algorithm 54: RELABEL(u) – applies when $e(u) > 0$ and $h(u) \le h(v)$ for all $(u, v) \in G_f$

```
1 h(u) ← 1 + min{ h(v) : (u, v) ∈ G_f }
```

The minimum is over residual neighbors of u. Because u is active, it has positive excess; that excess arrived from somewhere, so u has at least one residual edge (the reverse of an edge that delivered flow), and the minimum is well defined. The new height is the smallest value that creates an admissible edge while keeping h valid.

Definition 13.3 (Admissible edge)**.** A residual edge $(u, v) \in G_f$ is *admissible* if $h(u) = h(v) + 1$. PUSH acts only on admissible edges; RELABEL fires exactly when no admissible edge leaves u.

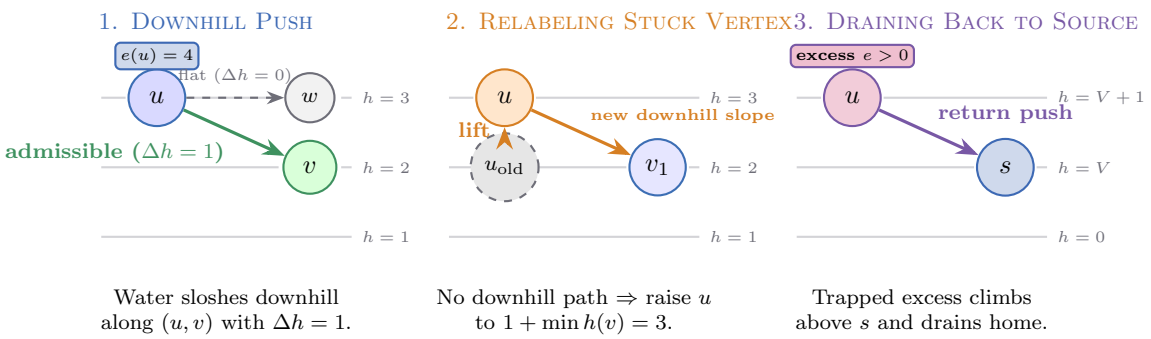


Figure 13.2: Push-relabel viewed as fluid flowing downhill. Panel 1: fluid (excess) at u can only push along residual edges descending exactly one unit of height ($h(u) = h(v) + 1$). Panel 2: when all residual neighbors are level or uphill, u is jacked up via Relabel(u) until a downhill exit opens. Panel 3: excess blocked from reaching the sink eventually climbs above the source ($h > |V|$), tilting the gradient backward so excess drains safely back to s.

13.3 The generic algorithm

Initialize, then repeat any applicable operation until none applies.

Algorithm 55: GenericPushRelabel(G, s, t)

```
foreach u ∈ V do
    h(u) ← 0;   e(u) ← 0
h(s) ← |V|
foreach edge (s, v) ∈ E do
    f(s, v) ← c(s, v);   f(v, s) ← −c(s, v)
    e(v) ← c(s, v);   e(s) ← e(s) − c(s, v)          // saturate every source edge
while some operation Push or Relabel applies do
    select an active vertex u and apply Push(u, ·) if admissible, else Relabel(u)
return f
```

The initialization is deliberate and not optional. Saturating every edge out of s floods the neighbors of s with excess; setting $h(s) = |V|$ both satisfies the height constraint (since all source edges are now saturated, the constraint $h(s) \le h(v) + 1$ has no residual edge to apply to) and guarantees no s–t residual path forever after. The source can never be relabeled, because by convention s is never placed in the active set; its height stays pinned at $|V|$.

Intuition

The opening move dumps a huge amount of water at s's neighbors and sets s on a mountaintop of height $|V|$. From there everything is downhill toward t at height 0. The main loop is just: find a puddle, push it one step downhill if you can, otherwise raise the junction until you can. Eventually the only puddles left are ones trapped behind the source's mountain; relabeling lifts *them* above $|V|$ too, and they drain back to s.

13.3.1 Correctness

We prove three things: the loop maintains the invariants (preflow + valid heights), it terminates, and on termination the preflow is a maximum flow.

Lemma 13.1 (Invariant maintenance)**.** *Throughout* GenericPushRelabel*, f is a preflow and h is a valid height function for f, and vertex heights never decrease.*

Proof. After initialization f is a preflow: every $u \neq s$ either is a saturated source-neighbor with $e(u) = c(s,u) \geq 0$ or has $e(u) = 0$. The height h is valid: $h(s) = |V|$, $h(t) = 0$, and the only vertex with nonzero height initially is s, whose outgoing residual edges were all saturated away, so no constraint $h(s) \leq h(v) + 1$ is active.

Push(u, v) moves $\delta \leq e(u)$ units, so $e(u)$ stays ≥ 0 and $e(v)$ only grows: still a preflow. It can create the new residual edge (v, u). Validity of that new edge needs $h(v) \leq h(u) + 1$; but the push required $h(u) = h(v) + 1$, so $h(v) = h(u) - 1 \leq h(u) + 1$. Other edges are unaffected (pushes only delete edges or use existing valid ones), so h stays valid.

Relabel(u) does not touch f, so the preflow property is untouched. It sets $h(u) = 1 + \min_v\{h(v) : (u,v) \in G_f\}$. For every residual edge (u, v) we now have $h(u) \leq h(v) + 1$ by the choice of minimum. For every residual edge *into* u, say (w, u), raising $h(u)$ only makes $h(w) \leq h(u) + 1$ easier to satisfy. And relabel only fires when $h(u) \leq h(v)$ for all residual neighbors, so the new height strictly exceeds the old: heights never decrease. □

Lemma 13.2 (Heights stay bounded)**.** *At all times, every vertex has $h(u) \leq 2|V| - 1$.*

Proof. A vertex is relabeled only while active, i.e. $e(u) > 0$. We use the fact (Exercise 13.6) that *if u is active then there is a residual path from u back to s in G_f.* Running a flow-decomposition argument on the preflow, any unit of excess at u can be traced back along edges carrying positive flow, and the reverse of each such edge is residual; hence a residual $u \to s$ path exists, and we may take it simple. Along it heights drop by at most 1 per step over at most $|V| - 1$ edges, so $h(u) \leq h(s) + (|V| - 1) = 2|V| - 1$. A relabel raises $h(u)$ to one more than a residual neighbor's height, and that neighbor obeys the same bound, so no height ever exceeds $2|V| - 1$. □

Theorem 13.1 (Correctness of generic push-relabel)**.** *If* GenericPushRelabel *terminates, the preflow it returns is a maximum flow.*

Proof. On termination no operation applies, so no vertex in $V \setminus \{s, t\}$ is active: $e(u) = 0$ for all interior u, which is exactly conservation. So f is a flow. By Lemma 13.1 a valid height function exists, and we showed in Section 13.2 that a valid height function precludes any residual s–t path. By max-flow min-cut (no augmenting path ⇒ optimal), f is a maximum flow. Concretely, let $S = \{u : h(u) \geq |V|\}$ and $T = V \setminus S$; then $s \in S$, $t \in T$, and every edge crossing from S to T is saturated while every edge from T to S carries zero flow, so (S, T) is a minimum cut certifying optimality. □

That leaves termination, which we get for free once we bound the number of operations – the same bounds that give the running time.

13.4 Counting the operations: the generic bound

Three counts: relabels, saturating pushes, nonsaturating pushes.

Lemma 13.3 (Relabel count)**.** *Each vertex is relabeled at most* $2|V|-1$ *times, so the total number of relabels is at most* $(2|V|-1)(|V|-2) < 2|V|^2$*, and the time spent relabeling is* $O(VE)$*.*

Proof. Each relabel of u strictly increases $h(u)$ (Lemma 13.1), and $h(u) \le 2|V|-1$ (Lemma 13.2), with only $V \setminus \{s,t\}$ ever relabeled. Each Relabel(u) scans u's residual edges in time $O(\deg u)$; summing $\deg u$ over $O(V)$ relabels per vertex gives $O(V) \cdot \sum_u \deg u = O(VE)$. □

Lemma 13.4 (Saturating push count)**.** *The number of saturating pushes is at most* $|V||E|$*, i.e.* $O(VE)$*.*

Proof. Consider a fixed edge (u,v). A saturating push on it requires $h(u) = h(v)+1$ and erases (u,v) from G_f. For it to be pushable again we first need a push on (v,u), which requires $h(v) = h(u)+1$ – so $h(v)$ must have risen by at least 2 in between, hence $h(u)$ must rise by at least 2 before the next saturating push on (u,v). Since heights range in $[0, 2|V|-1]$, each edge sees at most $|V|$ saturating pushes. Summing over the $O(E)$ residual edges (both orientations of each pair) gives $O(VE)$. □

Lemma 13.5 (Nonsaturating push count, generic rule)**.** *With arbitrary selection, the number of nonsaturating pushes is* $O(V^2E)$*.*

Proof. Use the potential $\Phi = \sum_{u \text{ active}} h(u)$. Initially $\Phi = 0$ (active vertices have height 0), and $\Phi \ge 0$ always. A *nonsaturating* push on (u,v) drives $e(u)$ to 0, so u leaves the active set, dropping Φ by $h(u)$; it may activate v, raising Φ by $h(v) = h(u)-1$. Net change ≤ -1: every nonsaturating push decreases Φ by at least one.

What can *increase* Φ? A relabel of u raises Φ by the increment in $h(u)$; total relabel increase is at most $\sum_u (2|V|-1) = O(V^2)$. A saturating push on (u,v) can newly activate v, raising Φ by $h(v) \le 2|V|-1$; total saturating increase is $O(VE) \cdot O(V) = O(V^2E)$. Since Φ starts and ends at 0 and each nonsaturating push costs it ≥ 1, the number of nonsaturating pushes is at most the total increase, $O(V^2) + O(V^2E) = O(V^2E)$. □

Theorem 13.2 (Generic running time)**.** GenericPushRelabel *performs* $O(V^2E)$ *operations and runs in* $O(V^2E)$ *time with the right data structures.*

Proof. Relabels $O(V^2)$, saturating pushes $O(VE)$, nonsaturating pushes $O(V^2E)$; the last term dominates. Each push is $O(1)$ work given a current-edge pointer (below); relabel work totals $O(VE)$. Total $O(V^2E)$. Finiteness of all three counts also proves termination. □

Cost summary

Generic push-relabel: $O(V^2E)$ time, $O(E)$ space. The bottleneck is the $O(V^2E)$ nonsaturating pushes. Both improved variants below attack precisely that term by choosing *which* active vertex to discharge.

The current-edge data structure

Implementations never re-scan an edge list from scratch. Each vertex keeps a *current-edge* pointer into its incident-edge list. To discharge u: examine the current edge; if admissible, push; if not, advance the pointer; if you run off the end, relabel u and reset the pointer to the front. The cost of all pointer advances between two consecutive relabels of u is $O(\deg u)$, and there are $O(V)$ relabels per vertex, so the amortized scanning cost folds into the $O(VE)$ already accounted for. This DISCHARGE subroutine is the unit of work for the fast variants.

Algorithm 56: DISCHARGE(u) – push out all of u's excess, relabeling as needed

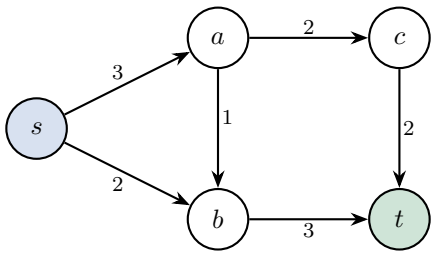
```
while e(u) > 0 do
    if current edge (u, v) is admissible then
        PUSH(u, v)
    else if current edge is not the last then
        advance u's current-edge pointer
    else
        RELABEL(u);   reset u's current-edge pointer to the start
```

Storing the residual graph. The standard representation stores edges in pairs. Each undirected edge slot holds a forward arc and its reverse, kept adjacent in an array so that the reverse of arc index i is index $i \oplus 1$ (XOR with one). An arc records its head vertex and a single residual-capacity field c_f; a PUSH of δ then does exactly two updates, $c_f[i] \mathrel{-}= \delta$ and $c_f[i \oplus 1] \mathrel{+}= \delta$, and skew symmetry is automatic because the two fields always sum to the original capacity of the pair. There is no separate f array at all in fast codes; the residual capacities *are* the state, and the actual flow on an arc is recovered as original-capacity minus current residual. This is why the trap of "forgetting the reverse edge" is so easy to fall into and so damaging: a single missing $c_f[i \oplus 1] \mathrel{+}= \delta$ means flow can never be cancelled, and the algorithm silently computes the wrong answer.

Each vertex stores the index of the first arc in its adjacency list and the current-edge pointer is just an integer offset into that list, so resetting it on relabel is a single assignment. The active set is a queue for FIFO and an array of buckets indexed by height for highest-label; in both cases insertion, removal, and the gap-heuristic's cnt updates are $O(1)$.

13.5 A worked trace by hand

Run the algorithm with FIFO selection on a five-vertex network. Capacities are on the edges.



There are $|V| = 5$ vertices, so $h(s) = 5$. Initialization saturates $s \to a$ ($\delta = 3$) and $s \to b$ ($\delta = 2$):

$$e(a) = 3, \quad e(b) = 2, \quad h = (s{:}5,\ a{:}0,\ b{:}0,\ c{:}0,\ t{:}0).$$

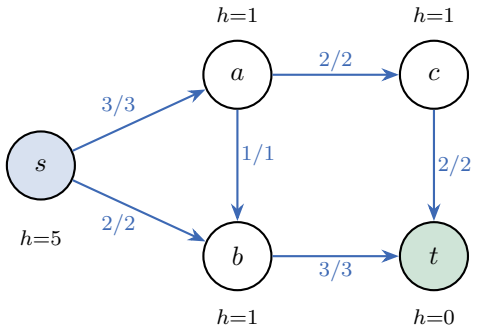
The FIFO queue of active vertices is $[a, b]$. We discharge in queue order.

Discharge a. a has excess 3, but all residual neighbors c, b sit at height $0 = h(a)$: no admissible edge. Relabel: residual neighbors of a are c ($h = 0$), b ($h = 0$), and s via the reverse edge ($h = 5$); the minimum is 0, so $h(a) \leftarrow 1$. Now $a \to c$ and $a \to b$ are admissible. Push to c: $\delta = \min(3, 2) = 2$, saturating; $e(a) = 1$, $e(c) = 2$. Push to b: $\delta = \min(1, 1) = 1$, saturating; $e(a) = 0$, $e(b) = 3$. a is drained. Queue becomes $[b, c]$ (b was already queued; c newly active).

Discharge b. $e(b) = 3$, $h(b) = 0$. Residual neighbors: t ($h = 0$), and reverse edges to s ($h = 5$) and a ($h = 1$). No admissible edge, so relabel: $\min(0, 5, 1) = 0$, $h(b) \leftarrow 1$. Now $b \to t$ admissible. Push to t: $\delta = \min(3, 3) = 3$, saturating; $e(b) = 0$, $e(t) = 3$. b drained. Queue $[c]$.

Discharge c. $e(c) = 2$, $h(c) = 0$. Residual neighbors: t ($h = 0$), reverse to a ($h = 1$). Relabel: $\min(0, 1) = 0$, $h(c) \leftarrow 1$. Push $c \to t$: $\delta = \min(2, 2) = 2$, saturating; $e(c) = 0$, $e(t) = 5$. Queue empty.

No active vertices remain. The preflow is now a flow of value $e(t) = 5$. Heights are $(s{:}5, a{:}1, b{:}1, c{:}1, t{:}0)$.



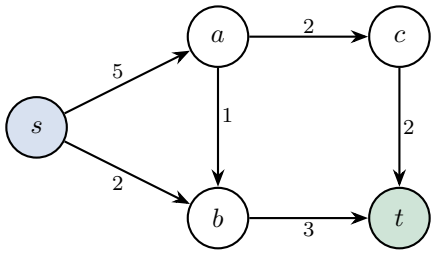
Every edge is saturated; value 5. The min-cut is $\{s\}$ versus the rest, with capacity $c(s, a) + c(s, b) = 5$. Max-flow min-cut confirms optimality.

Trace it yourself

Re-run the same network but discharge in the order b first, then a, then c. You should get the same flow value 5 but a different sequence of pushes, and possibly different final heights – the order changes the trace, never the optimum. Then change $c(s, a)$ from 3 to 5: now a has excess 5, but $a \to c \to t$ can only absorb 2 and $a \to b \to t$ only 1 more after b's edge fills; the leftover 2 units have nowhere to go but back to s. Watch $h(a)$ climb past 5 so that a can push the trapped excess back up to the source. This is the "drain back" phase, and tracing it is the best way to feel why heights can exceed $|V|$.

13.5.1 A trace with drain-back

The first trace was too tidy: every unit of source flow reached t, so the preflow turned into a flow with no excess sent back. That is the exception, not the rule. To see the characteristic "drain back" behaviour – excess that climbs *above* the source and pours back – take the same graph but raise the first capacity to $c(s, a) = 5$. Now a receives more than its downstream edges can carry.



Initialization saturates $s \to a$ ($\delta = 5$) and $s \to b$ ($\delta = 2$), giving $e(a) = 5$, $e(b) = 2$, all interior heights 0, $h(s) = 5$. Queue $[a, b]$.

Discharge a. Relabel a to 1 (c, b both at height 0). Push $a \to c$: saturating, $\delta = 2$, $e(a) = 3$, $e(c) = 2$. Push $a \to b$: saturating, $\delta = 1$, $e(a) = 2$, $e(b) = 3$. Now a still has excess 2 but both forward edges $a \to c$ and $a \to b$ are saturated; its only residual neighbour is s (height 5) via the reverse of $s \to a$. No admissible edge, so relabel: $h(a) \leftarrow 1 + 5 = 6$. The excess at a is now *above* the source, making the reverse edge $a \to s$ admissible ($h(a) = h(s) + 1$). Push $a \to s$: its residual capacity is 5 (the reverse of the saturated edge $s \to a$), but only $\delta = \min(e(a), 5) = 2$ moves, so this push is nonsaturating; $e(a) = 0$, and 2 units return to the source. a drained. Queue $[b, c]$.

Discharge b. $e(b) = 3$, relabel b to 1, push $b \to t$: $\delta = 3$, $e(b) = 0$, $e(t) = 3$. Queue $[c]$.

Discharge c. $e(c) = 2$, relabel c to 1, push $c \to t$: $\delta = 2$, $e(c) = 0$, $e(t) = 5$. Queue empty.

No active vertices remain. Final heights (s:5, a:6, b:1, c:1, t:0), flow value $e(t) = 5$. The two extra units injected at a never reached t; they piled up, lifted a to height 6, and drained back to s exactly as the hillside picture predicts. The min cut is still $\{s\}$ versus the rest – the bottleneck is the pair of source edges, total usable capacity $c(s, a) + c(s, b) = 5$ once we account for the wasted slack on $s \to a$.

Why it works

The drain-back step is what lets push-relabel be greedy at the source without penalty. The opening move floods s's neighbours with the *full* capacity of every source edge, even when downstream cannot absorb it. Any over-injection is recovered automatically, because the same relabel-and-push machinery that pushes excess toward t will, once a vertex climbs above height $|V|$, push it back toward s. A vertex with $h(u) > |V|$ is provably cut off from t (Section 13.2), so the algorithm has lost nothing by lifting it: its excess was never going to reach the sink.

Common trap

The most common implementation bug is forgetting to update the reverse edge $f(v, u) \leftarrow f(v, u) - \delta$ inside PUSH, or storing residual capacities without paired edges. If reverse residual capacity is wrong, the algorithm cannot cancel flow, excess gets stranded, and it either loops forever or returns a non-maximum value. The second classic bug: relabeling s. The source must be excluded from the active set; if your queue ever contains s, $h(s)$ creeps above $|V|$ and silently breaks the min-cut certificate. The third: initializing $h(s) = 0$ instead of $|V|$ – then an s–t residual path can exist initially and the no-augmenting-path invariant fails from the start.

13.6 FIFO push-relabel: $O(V^3)$

The generic bound is dominated by $O(V^2E)$ nonsaturating pushes. The selection rule decides how many of those we actually do. The *FIFO* rule keeps active vertices in a queue: pop the front, DISCHARGE it completely (which may relabel it and enqueue newly activated vertices at the back), repeat.

Algorithm 57: FIFOPUSHRELABEL(G, s, t)

```
initialize f, h, e as in GENERICPUSHRELABEL
Q ← queue of all initially active vertices (the neighbors of s with excess)
while Q ≠ ∅ do
    u ← Q.DEQUEUE()
    DISCHARGE(u)  // enqueue any vertex newly made active, if not already in Q
return f
```

The analysis groups operations into *phases*. Phase 1 is the discharge of the vertices present in Q right after initialization. Phase $i+1$ is the discharge of exactly those vertices that were enqueued during phase i. (Equivalently: imagine a marker at the tail of Q; each time the marker is reached, a new phase begins.)

Lemma 13.6 (Phase count)**.** FIFOPUSHRELABEL *runs for* $O(V^2)$ *phases.*

Proof. Use $\Psi = \max\{h(u) : u \text{ active}\}$ (with $\Psi = 0$ if no vertex is active). Consider a phase with no relabel. Every push in such a phase is from a vertex u to a strictly lower vertex v, and u is emptied; the vertices active at the end of the phase are a subset of the v's that received flow, all of which are strictly below the maximum at the phase's start. So Ψ *strictly decreases* across any relabel-free phase. A phase that *does* contain a relabel raises some $h(u)$, and across the whole run the total relabel increase is $O(V^2)$ (Lemma 13.3), bounding how much Ψ can ever rise. Since Ψ starts and ends at 0, decreases by ≥ 1 in each relabel-free phase, and rises by a total of $O(V^2)$, the number of relabel-free phases is $O(V^2)$. Phases that contain a relabel are at most the number of relabels, $O(V^2)$. Total $O(V^2)$. □

Theorem 13.3 (FIFO running time)**.** FIFOPUSHRELABEL *runs in* $O(V^3)$ *time.*

Proof. Each phase discharges each vertex at most once, so each phase does at most one nonsaturating push per vertex: $O(V)$ nonsaturating pushes per phase. (Within a single discharge of u, only the *last* push – the one that empties u – can be nonsaturating; all earlier pushes from u in that discharge saturate their edges.) Over $O(V^2)$ phases that is $O(V^3)$ nonsaturating pushes. Saturating pushes ($O(VE)$) and relabel work ($O(VE)$) are subsumed since $E \leq V^2$. Total $O(V^3)$. □

Why it works

Why does FIFO beat the generic $O(V^2E)$? The generic analysis charged each nonsaturating push to a global potential and got $O(V^2E)$. FIFO adds structure: a vertex is discharged at most once per phase, so within a phase the nonsaturating pushes number $O(V)$, not $O(E)$. The E in the generic bound came from re-discharging the same vertex many times in close succession; the queue serializes discharges so that re-activation only happens in a *later* phase,

and there are only $O(V^2)$ phases.

13.7 Highest-label push-relabel: $O(V^2\sqrt{E})$

A second natural rule: always discharge an active vertex of *maximum height*. Water at the highest puddle has the longest way to fall, so emptying it first tends to cascade flow far downhill before anything is re-activated above it.

Algorithm 58: HighestLabelPushRelabel(G, s, t)

```
initialize f, h, e
maintain buckets B[0 .. 2|V| − 1] of active vertices indexed by height
while some bucket is nonempty do
    let k be the largest index with B[k] ≠ ∅;   u ← any vertex in B[k]
    Discharge(u)   // moves u between buckets on relabel; inserts newly active
     vertices
return f
```

A pointer scanning down from the top bucket finds the highest active vertex; relabels only move vertices *up*, and the scan pointer rises with them, so the total pointer travel is only $O(V^2)$. The improvement is in the nonsaturating push count.

Figure 13.3 shows this bucket structure: an array of doubly linked lists indexed by height stores active vertices, with a scan pointer `top_bucket` tracking the highest occupied level. Flow is always discharged from the top, cascading downward.

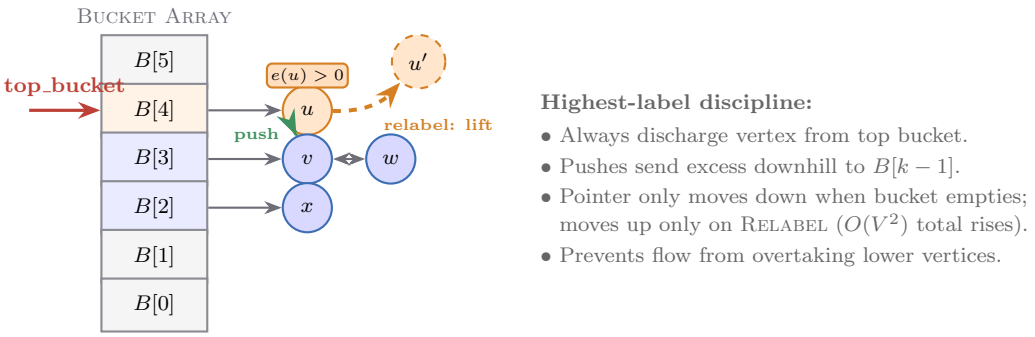


Figure 13.3: Bucket queue implementation of highest-label push-relabel. An array of doubly linked lists partitions active vertices by height. A scanning pointer tracks the maximum active level k. Active vertices in $B[k]$ are discharged; flow cascades downhill into $B[k-1]$, and the pointer only advances upward when a relabel operation lifts a vertex, bounding total pointer motion to $O(V^2)$.

Intuition

Discharging the highest active vertex first is the greedy choice that avoids wasted motion. If you instead emptied a low vertex, the flow it pushes downhill might immediately be overtaken by flow arriving from a higher vertex you had not yet touched, and the low vertex would

re-activate – a second discharge of the same vertex, a second nonsaturating push. By always taking the top, no flow is ever pushed onto a vertex that still has un-discharged excess sitting above it, so a vertex re-activates only after the whole top layer has drained past it. That is exactly why the re-discharge count, and with it the nonsaturating-push count, drops below the FIFO bound.

Theorem 13.4 (Highest-label running time)**.** HighestLabelPushRelabel *performs* $O(V^2\sqrt{E})$ *nonsaturating pushes and runs in* $O(V^2\sqrt{E})$ *time.*

Proof sketch. The argument (Cheriyan–Maheshwari, tightened by Tunçel) again partitions the run into phases delimited by changes in the maximum active height. Set a threshold $K = \sqrt{E}$. Call a phase *cheap* if it contains at most K nonsaturating pushes and *expensive* otherwise. The number of phases is $O(V^2)$ as before, so cheap phases contribute $O(V^2\sqrt{E})$ pushes. For expensive phases, use a potential $\Phi = \frac{1}{K}\sum_{u \text{ active}} |\{w : h(w) \le h(u)\}|$; one shows each expensive phase decreases Φ by an amount proportional to its push count over K, while the total increase of Φ across the whole run is $O(V^2)$. Balancing the two sources of pushes at $K = \sqrt{E}$ yields $O(V^2\sqrt{E})$. Each nonsaturating push is $O(1)$ amortized and the other costs are dominated, so the total time matches. □

History

Andrew Goldberg and Robert Tarjan introduced push-relabel (they called it the "preflow-push" method) in 1986, unifying and simplifying earlier work on Alexander Karzanov's 1974 preflows and Yefim Dinitz's layered blocking flows. Instead of maintaining a valid flow along end-to-end paths, push-relabel moves fluid locally downhill along a discrete height gradient. The generic bound is $O(V^2E)$; FIFO vertex selection achieves $O(V^3)$; and highest-label selection with bucket queues achieves $O(V^2\sqrt{E})$, which stood as the fastest strongly polynomial bound for decades. Goldberg and Tarjan later incorporated Sleator–Tarjan dynamic trees to obtain $O(VE\log(V^2/E))$.

The idea of replacing global paths with local pushes driven by a height potential reappears in later flow algorithms; electrical-flow and interior-point methods follow the same "move locally, certify globally" pattern. In practice, heuristics such as the gap rule and periodic backward-BFS relabeling make highest-label push-relabel one of the fastest general-purpose max-flow algorithms for real-world instances.

13.8 The gap heuristic and why it matters in practice

The worst-case bounds understate push-relabel's practical performance: it is the fastest general max-flow method on most networks, largely because of two heuristics.

Gap heuristic. Maintain a count cnt[k] of vertices at each height k. Suppose at some moment no vertex has height exactly g (a *gap* at g), but some vertices have height $> g$. Any vertex u with $h(u) > g$ is cut off from t: every residual path from u to t would have to descend through height g, but no vertex sits there, and heights drop by at most 1 per edge, so t (at height 0) is unreachable from u in G_f. Such a u can never push its excess to t; the excess is destined to return to s. We

may as well lift u immediately to $|V|+1$ (above the source) so it drains back in one stroke instead of being relabeled one unit at a time.

Algorithm 59: Gap heuristic, invoked right after a relabel that empties height h_{old}

```
if cnt[h_old] = 0 after the relabel and h_old < |V| then
    foreach vertex w with h_old < h(w) < |V| do
        h(w) ← |V| + 1;   update cnt
```

Why it works

The gap heuristic is a correctness-preserving shortcut, not an approximation. If height g becomes empty, the Lipschitz property of h proves that every higher vertex is permanently separated from t in the residual graph – raising them to $|V|+1$ keeps h valid (the only residual edges out of a raised vertex now go to other equally-or-higher vertices or back toward s) and skips a long ladder of unit relabels. On many real instances, especially those with a near-cut found early, the gap heuristic empties whole layers at once and accounts for most of the practical speedup.

To make the gap concrete (Figure 13.4), suppose at some moment in a run on a 7-vertex graph the height-count array reads

$$\text{cnt} = (\,\text{cnt}[0], \ldots, \text{cnt}[6]\,) = (1,\ 2,\ 0,\ 1,\ 1,\ 0,\ 2),$$

so one vertex sits at height 0 (the sink), two at height 1, none at height 2, and the remaining four scattered at heights $3, 4$, and 6 (two at height 6). Height $g = 2$ is empty while vertices live above it, and $g < |V| = 7$. Every vertex at height $3, 4$, or 6 would have to descend through the empty layer 2 to reach t at height 0, but heights fall by at most one per residual edge, so none of them can reach t in G_f – they are stranded behind the gap. Instead of relabeling each one upward a single unit at a time over many iterations, the heuristic lifts all four to $|V|+1 = 8$ in one sweep, after which they drain straight back to s. Without the heuristic, the vertex at height 3 alone might be relabeled $3 \to 8$ in five separate RELABEL calls, each preceded by a wasted scan of its edge list.

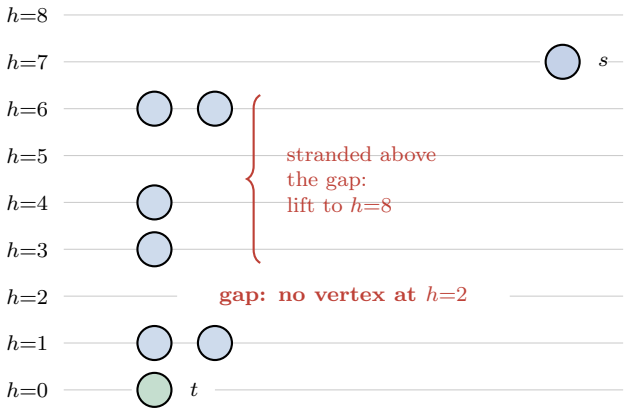


Figure 13.4: The gap at height 2. Every vertex above an empty layer is cut off from t (any residual path to t must pass through the missing layer), so the heuristic lifts all of them to $|V|+1 = 8$ at once instead of relabeling each by one unit at a time.

Global relabeling. Periodically (say every $O(E)$ operations) recompute every height as its exact BFS distance to t in the residual graph, and for vertices that cannot reach t, set $h = |V|$ plus the BFS distance to s. Exact distances are always valid heights and they are the largest valid heights, so global relabeling makes the height function as informative as possible. It costs one $O(E)$ BFS but typically pays for itself many times over by routing excess on near-shortest paths.

Application

Push-relabel with gap and global-relabeling heuristics is the core of Cherkassky and Goldberg's `hi_pr` code, for years the reference max-flow benchmark, and of solvers in the Boykov–Kolmogorov family for computer-vision energy minimization (graph cuts for image segmentation, stereo, and Markov-random-field MAP inference). In those vision instances the graph is a grid with a source and sink wired to every pixel; highest-label push-relabel with gaps routinely beats augmenting-path solvers because the cut is found early and the gap heuristic prunes aggressively. The same machinery underlies project-selection, bipartite matching at scale, and network-reliability computations.

Cost summary

Summary of the variants on $n = |V|$, $m = |E|$:

Variant	Time	Note
Generic (any order)	$O(n^2 m)$	nonsaturating pushes dominate
FIFO	$O(n^3)$	$O(n^2)$ phases, $O(n)$ pushes each
Highest-label	$O(n^2\sqrt{m})$	best simple strongly-poly bound
Goldberg–Tarjan (dynamic trees)	$O(nm \log(n^2/m))$	best with link-cut trees

All variants use $O(m)$ space. Heuristics (gap, global relabel) do not change worst-case bounds but dominate real-world performance.

13.9 A note on almost-linear max-flow

For decades the question "how fast can max-flow be?" inched forward: blocking flows, scaling, dynamic trees, and – starting around 2008 – the *electrical-flow* approach, in which max-flow is solved by a sequence of Laplacian linear systems, each an electrical-flow computation, wrapped in an interior-point or multiplicative-weights outer loop. Christiano–Kelner–Madry–Spielman–Teng (2011) used electrical flows to approximate undirected max-flow in roughly $\tilde{O}(m^{4/3})$ time; Madry pushed exact unit-capacity max-flow to $\tilde{O}(m^{10/7})$; a chain of interior-point refinements followed.

In 2022, Chen, Kyng, Liu, Peng, Probst Gutenberg, and Sachdeva proved that max-flow – indeed min-cost flow, on directed graphs with arbitrary integer capacities – can be solved in

$$m^{1+o(1)} \quad \text{time},$$

almost linear in the number of edges. The algorithm is very different from push-relabel: it runs an interior-point method whose every iteration requires solving a min-ratio cycle problem on a slowly-changing graph, supported by a sophisticated dynamic data structure built on low-stretch spanning trees and the ℓ_1 analogue of Laplacian linear solvers.

Connection

The modern result ties this chapter to the spectral and electrical half of the book. The interior-point outer loop reduces flow to repeatedly solving a *weighted* flow/potential problem; for the ℓ_2 (energy) version that is exactly an electrical flow, i.e. a Laplacian linear system $\mathbf{L}x = b$ (the chapters on the graph Laplacian and on Laplacian solvers and electrical flows). Spielman–Teng's nearly-linear Laplacian solvers are what made electrical flows cheap enough to compute repeatedly. Push-relabel and the almost-linear algorithm sit at opposite poles: one is combinatorial, local, and integral, certified by a height function; the other is continuous, global, and numerical, certified by interior-point duality and electrical energy. Both compute the same max-flow, and seeing that a height function and an electrical potential are two instances of the same vertex-potential idea connects the two approaches directly. For the role of the Laplacian as a discrete operator, see the chapters on random walks and electrical networks.

Focus

Practical guidance. For exact max-flow on a concrete graph today: use highest-label push-relabel with the gap and global-relabeling heuristics, or Dinic's algorithm with scaling. Push-relabel tends to win on dense graphs and on vision-style grids; Dinic often wins on unit-capacity and bipartite-matching instances. The almost-linear algorithm is, as of now, a theoretical breakthrough whose constants and implementation complexity keep it from displacing highest-label and Dinic codes in practice – but it settles the asymptotic question that push-relabel left open.

Push-relabel maximizes flow volume by moving fluid down height gradients, indifferent to cost. In real logistics, however, transmission lines and shipping routes carry per-unit tariffs: each megabit or barrel incurs an edge cost. The next chapter introduces minimum-cost flow, combining the capacity constraints of maximum flow with the cost-minimizing potential machinery of shortest paths.

13.10 Exercises

Exercise 13.1 (Trace it)**.** Run push-relabel with FIFO selection on the network with vertices s, a, b, t and capacities $c(s,a) = 4$, $c(s,b) = 2$, $c(a,b) = 3$, $c(a,t) = 1$, $c(b,t) = 4$. Show h, e, and the queue after every operation, and identify the minimum cut from the final heights.

Exercise 13.2 (Why $|V|$?)**.** Prove that if you initialize $h(s) = |V| - 1$ instead of $|V|$, the algorithm can fail to maintain the invariant "no residual s–t path," and exhibit a small network where it returns a non-maximum value. Then prove that any $h(s) \geq |V|$ is safe.

Exercise 13.3 (Saturating-push bound)**.** Construct a family of networks on which generic push-relabel performs $\Omega(VE)$ saturating pushes, demonstrating that Lemma 13.4 cannot be improved by more than constant factors.

Exercise 13.4 (Nonsaturating pushes for FIFO)**.** Give a family of networks for which FIFO push-relabel performs $\Omega(V^3)$ nonsaturating pushes, so Theorem 13.3 is tight. (Hint: a long chain forces many phases, each with $\Theta(V)$ nonsaturating pushes.)

Exercise 13.5 (Gap correctness)**.** Prove rigorously that the gap heuristic preserves the validity of the height function: if height g is empty and $g < |V|$, then raising every vertex w with $g < h(w) < |V|$ to

$|V|+1$ yields a valid height function for the current preflow. Where exactly do you use the Lipschitz property $h(u) \le h(v) + 1$?

Exercise 13.6 (Excess implies path to source)**.** Fill in the flow-decomposition argument behind Lemma 13.2: show that for any preflow, if $e(u) > 0$ then there is a path from u to s in the residual graph G_f. (Hint: consider the set R of vertices that can reach s in G_f; sum excesses over $V \setminus R$ and derive a contradiction if $u \notin R$.)

Exercise 13.7 (Implement DISCHARGE)**.** Implement the current-edge DISCHARGE routine and the highest-label selection with height buckets. Verify on random graphs that your output value matches a reference Edmonds–Karp implementation, and plot operation counts with and without the gap heuristic to see the practical difference.

Exercise 13.8 (Highest-label phases)**.** Show that under the highest-label rule, between two consecutive relabels of any *fixed* vertex u, the algorithm performs at most $O(V)$ nonsaturating pushes that originate at vertices of height $\ge h(u)$. Use this to give an alternative, self-contained proof that the number of nonsaturating pushes is $O(V^3)$ (a weaker but easier bound than $O(V^2\sqrt{E})$).

Exercise 13.9 (Min-cut from the run)**.** After push-relabel terminates, you have a maximum preflow that is a flow. Prove that $S = \{u : h(u) \ge |V|\}$ and $T = V \setminus S$ is a minimum s–t cut, by showing every edge from S to T is saturated and every edge from T to S carries zero flow. Why does this not require an extra BFS?

Exercise 13.10 (From preflow to flow, explicitly)**.** Some implementations stop as soon as t is unreachable from every active vertex (a maximum *preflow*), skipping the drain-back phase, because the flow *value* is already $e(t)$ and the value is what most applications want. Describe a second phase that converts a maximum preflow into an actual maximum flow of the same value, and bound its running time. (Hint: a decomposition / reverse traversal that cancels excess along residual edges back to s.)

Exercise 13.11 (Connection to electrical flow)**.** The almost-linear algorithm replaces "push flow downhill in height" with "route flow to minimize electrical energy." On the small network of Section 13.5, set every edge's resistance to 1 and compute the electrical flow of value 1 from s to t by solving the Laplacian system $\mathbf{L}\,x = b$ with $b = \mathbf{1}_s - \mathbf{1}_t$. Compare the vertex potentials x to the final push-relabel heights h: both are vertex potentials that "explain" the flow, but one is integral and combinatorial and the other is real-valued and energy-minimizing.

Exercise 13.12 (Discharge invariant)**.** Prove the claim used in Theorem 13.3: within a single call to DISCHARGE(u), every push except possibly the last is saturating. (Hint: a nonsaturating push empties $e(u)$ and ends the call.)

Exercise 13.13 (Drain-back, traced)**.** Reproduce the drain-back trace of Section 13.5.1 for the network with $c(s, a) = 5$, recording h, e, and the queue after every push and relabel. Confirm that exactly 2 units return to s, that $h(a)$ reaches 6, and that the final flow value is 5. Then push the capacity further to $c(s, a) = 9$ and predict, before tracing, how high $h(a)$ must climb and how many units drain back.

Exercise 13.14 (Paired-arc representation)**.** Implement the residual graph with the paired-arc scheme of Section 13.5: store arcs in an array, place each arc next to its reverse, and recover the reverse of index i as $i \oplus 1$. Write PUSH as the two updates $c_f[i]$ $-\!=$ δ, $c_f[i \oplus 1]$ $+\!=$ δ. Then deliberately delete the second update and run it on the network of Section 13.5; report what value the broken code returns and explain, in terms of un-cancellable flow, why it is wrong.

Exercise 13.15 (Gap heuristic on a count array)**.** Given the height-count array $\text{cnt} = (1, 2, 0, 1, 1, 0, 2)$ on a 7-vertex graph, identify every gap, state which vertices the heuristic lifts and to what height, and write out the array cnt after the lift. Then argue that the heuristic never needs to scan below the *lowest* gap, and use that to bound the total work the gap heuristic does over an entire run.

Exercise 13.16 (Highest-label versus FIFO empirically)**.** On the same family of random graphs, run highest-label and FIFO selection and count nonsaturating pushes for each. For which density regime (sparse vs. dense) does highest-label's $O(V^2\sqrt{E})$ advantage over FIFO's $O(V^3)$ actually show up in the counts? Explain the crossover in terms of how $\sqrt{E}$ compares to V.

Chapter 14

Minimum-Cost Flow

The maximum-flow problem asks only about volume: how much flow can we route from s to t? In most settings, routing flow is not free. Each arc carries a per-unit toll, shipping cost, or latency. Among all ways to route the same amount of flow, which one is *cheapest*? That is the minimum-cost flow problem (MCMF), and it is the model behind transportation planning, assignment, scheduling, image segmentation with smoothness priors, and a wide variety of operations-research problems that never mention the word "graph."

This chapter builds MCMF from two complementary methods. The first, *negative-cycle canceling*, starts from any feasible flow and repeatedly reroutes flow around negative-cost cycles in the residual graph until none remain. The second, *successive shortest paths* (SSP), builds the flow up from zero, each time routing the next batch along a minimum-cost residual path. Because residual backward arcs carry negative costs, SSP reuses Johnson's vertex potentials from Chapter 9 to keep reduced arc costs nonnegative so Dijkstra's algorithm applies at every step. We prove both correct with a single optimality criterion, work a small graph by hand twice, sketch the cost-scaling speedup, and derive transportation and assignment as special cases. The chapter closes with the linear-programming duality that explains *why* potentials appear throughout the subject.

Focus

The key idea: **a flow of given value is minimum cost if and only if its residual graph has no negative-cost cycle.** Everything else — the two algorithms, the potentials, the duality — is machinery for finding such a flow or certifying you already have one.

14.1 The problem, stated cleanly

We work with a directed graph $G = (V, E)$. Every arc $(u, v) \in E$ carries two numbers:

- a **capacity** $c(u, v) \geq 0$, the most flow the arc can hold, and
- a **cost** $w(u, v) \in \mathbb{R}$, the price per unit of flow on that arc.

A **flow** is a function $f : E \to \mathbb{R}_{\geq 0}$ with $0 \leq f(u, v) \leq c(u, v)$ (capacity) and, at every vertex except two distinguished ones s (source) and t (sink), inflow equals outflow (conservation). The **value** of

the flow is the net amount leaving s, written $|f|$. Its **cost** is

$$\text{cost}(f) \;=\; \sum_{(u,v)\in E} w(u,v)\, f(u,v).$$

There are two standard ways to phrase the optimization, and you should hold both in mind.

Definition 14.1 (Min-cost max-flow)**.** Among all flows of *maximum* value, find one of minimum cost.

Definition 14.2 (Min-cost flow of value F)**.** Given a target value F, among all flows with $|f| = F$, find one of minimum cost (or report that no such flow exists).

These two are interchangeable in practice. To get min-cost max-flow, first compute the max value $F^\star$ with any max-flow routine, then solve the fixed-value problem at $F = F^\star$. Conversely, to pin the value to a chosen F, add an arc from t back to s with capacity F and a cost low enough to force it full, or simply stop the build-up algorithm once it has pushed F units. We will mostly reason about the fixed-value form because its optimality condition is the cleanest.

Intuition

Think of capacity as the width of each road and cost as the per-car toll. Max-flow asked how many cars per minute the road network can deliver downtown. Min-cost flow asks: now that we know we can deliver F cars, route them so the city collects the smallest total toll. The roads with cheap tolls fill first; expensive detours are used only when the cheap roads are saturated.

14.1.1 The residual graph carries the cost too

Recall the residual construction from the max-flow chapters. For an arc (u, v) with flow $f(u, v)$:

- a **forward** residual arc (u, v) with residual capacity $c(u, v) - f(u, v)$ and cost $w(u, v)$, meaning "you may still push more here, at the listed price";
- a **backward** residual arc (v, u) with residual capacity $f(u, v)$ and cost $-w(u, v)$, meaning "you may cancel a unit already sent, getting the toll *refunded*."

The sign reversal on the backward cost accounts for the refund when flow is cancelled. Sending a unit forward incurs cost w; withdrawing that unit refunds $-w$. Pushing flow around a residual cycle therefore adjusts total cost by exactly the sum of the signed residual costs along that cycle. We write G_f for the residual graph and let it inherit these signed costs. A **negative cycle** in G_f is a directed cycle whose residual costs sum to a negative number: an opportunity to reroute flow and strictly reduce cost without changing the total flow value.

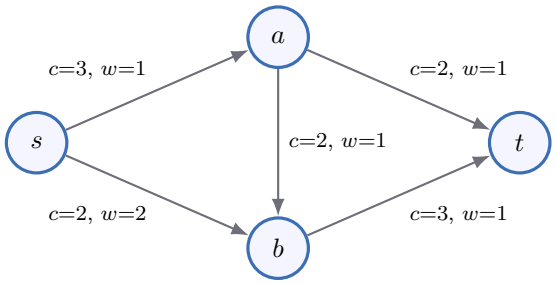


Figure 14.1: A small instance. Each arc shows capacity c and per-unit cost w. We will route flow from s to t and ask for the cheapest way.

14.2 The optimality criterion

Before any algorithm, we want a test: *given* a flow, is it minimum cost for its value? The answer is the following theorem, and both algorithms are two ways of reaching the state it describes.

Theorem 14.1 (Negative-cycle optimality)**.** *A feasible flow f has minimum cost among all flows of the same value $|f|$ if and only if its residual graph G_f contains no negative-cost cycle.*

Why it works

The proof rests on one fact: the difference of two flows of equal value, viewed in the residual graph, decomposes into cycles. So if f is not optimal, the gap to a cheaper flow f' is realized by residual cycles, and at least one of them must be negative. Conversely a negative cycle is an explicit recipe for improvement, so its absence means no improvement exists. This is the flow analogue of "a potential is shortest-path-optimal iff no edge can be relaxed."

Proof. ($\Rightarrow$, by contrapositive.) Suppose G_f has a negative cycle C with minimum residual capacity $\delta > 0$ along it. Push δ units around C. Because C is a cycle, conservation is preserved at every vertex and the value $|f|$ is unchanged; capacities are respected because δ is the bottleneck. The cost changes by $\delta \cdot (\text{sum of residual costs on } C) < 0$. So f was not minimum cost.

($\Leftarrow$.) Suppose G_f has no negative cycle; we show f is optimal. Let f' be any flow with $|f'| = |f|$. Consider the arc-wise difference $g = f' - f$. At every vertex, g satisfies conservation *including* at s and t, because f and f' have equal value, so the net out of s cancels. A conservative (circulation) vector on a graph decomposes into a sum of flows around directed cycles, each cycle using only arcs where g moves flow in a feasible residual direction: wherever $g(u,v) > 0$, f' sends more than f, so (u,v) has positive residual capacity in G_f; wherever $g(u,v) < 0$, f' sends less, so the backward residual arc (v,u) is available. Thus the decomposition uses only arcs of G_f, giving cycles $C_1, \dots, C_k$ in G_f with positive multipliers λ_i such that

$$\text{cost}(f') - \text{cost}(f) \;=\; \sum_{i=1}^{k} \lambda_i \cdot \text{cost}(C_i),$$

where $\text{cost}(C_i)$ is the sum of residual costs around C_i. Since G_f has no negative cycle, every $\text{cost}(C_i) \geq 0$, so the right side is ≥ 0 and $\text{cost}(f') \geq \text{cost}(f)$. Hence f is optimal. $\square$

Common trap

The cycle-decomposition step quietly uses that the difference of two equal-value flows is a *circulation* — it conserves at s and t as well. If $|f'| \neq |f|$ the decomposition picks up s–t paths in addition to cycles, and the clean "cost difference = sum over cycles" identity fails. That is exactly why Theorem 14.1 is a statement about flows *of the same value*.

14.3 Negative-cycle canceling

Theorem 14.1 suggests a direct algorithm: get *a* flow of the right value by any means, then keep finding negative cycles and canceling them until none remain.

Intuition

You have already routed F units somehow — maybe wastefully. Walk the residual graph looking for a loop that, if you shuffle flow around it, lowers the bill. Push as much as the loop allows. Repeat. Each shuffle keeps the value fixed (it is a cycle) and strictly cuts the cost (the cycle was negative). When no money-losing loop is left, Theorem 14.1 certifies you are done.

Algorithm 60: Negative-Cycle Canceling

Input: graph G with capacities c, costs w; a feasible flow f of the desired value F.
Output: a minimum-cost flow of value F.

while G_f *contains a negative-cost cycle* **do**
  find a negative cycle C (Bellman–Ford / SPFA on residual costs);
  $\delta \leftarrow \min_{(u,v)\in C} c_f(u,v)$; `// bottleneck residual capacity on` C
  foreach *arc* $(u,v) \in C$ **do**
    push δ units along (u,v) in G_f (update f and the paired residual arc);
return f;

How do you get the initial feasible flow of value F? Run any max-flow algorithm ignoring costs; that gives a max-value flow, and if you only need value $F < F^\star$ you can cap it. The costs do not affect feasibility, only quality, so starting feasibility is free.

Why it works

Finding a negative cycle is the standard Bellman–Ford byproduct. Run $|V| - 1$ relaxation passes over the residual arcs using w as edge length; if a further pass still relaxes some arc, that arc lies on (or downstream of) a negative cycle, and following predecessor pointers backward recovers an actual cycle. Each search is $O(VE)$.

14.3.1 Termination and complexity

With integer costs, every cancellation drops the total cost by at least 1 (each push moves $\delta \geq 1$ units around a cycle of total cost ≤ -1). The cost is bounded below — it cannot fall past the optimum — so the loop halts.

Cost summary

Let U bound capacities and W bound $|w|$. The initial cost lies in $[-mWU, mWU]$ where $m = |E|$, so there are at most $O(mWU)$ cancellations, each costing $O(VE)$ for the cycle search. The naive bound is $O(VE \cdot mWU)$ — correct but pseudopolynomial. The cure (Goldberg–Tarjan) is to cancel the cycle of *minimum mean cost* each time; that choice forces termination in $O(VE)$ cancellations, giving a strongly polynomial $O(V^2E^2 \log V)$ overall. Plain "any negative cycle" is fine for small inputs and a great teaching tool; minimum-mean canceling is the version you cite in a complexity table.

14.3.2 Canceling by hand

The fastest way to trust the canceler is to watch it repair a deliberately bad flow. Reuse the instance of Figure 14.1, but route value 5 stupidly on purpose. Send 3 units down $s \to a$, but instead of pushing two of them straight to t over the cheap $a \to t$ arc, divert one of those through $a \to b \to t$; then send 2 down $s \to b \to t$. Concretely the starting flow is

$$f_{sa} = 3, \quad f_{at} = 2, \quad f_{ab} = 1, \quad f_{sb} = 2, \quad f_{bt} = 3,$$

which is feasible (every arc within capacity, conservation holds, value $= 5$). Its cost is

$$3(1) + 2(1) + 1(1) + 2(2) + 3(1) = 3 + 2 + 1 + 4 + 3 = 13.$$

That happens to already be the optimum, so to make the canceler do visible work we first *wound* the flow.

Trace it yourself

Perturb the flow: move one unit off the cheap path and onto the expensive one. Drop f_{at} to 1 and raise f_{ab} to 2, then to conserve at b raise f_{bt} to 4 — but $b \to t$ has capacity 3, so instead keep $f_{bt} = 3$ and reduce f_{sb} to 1 so that b still balances (in $= f_{ab} + f_{sb} = 2 + 1 = 3 = f_{bt}$). The wounded flow is $f_{sa} = 3$, $f_{at} = 1$, $f_{ab} = 2$, $f_{sb} = 1$, $f_{bt} = 3$, still value 5. Its cost is $3 + 1 + 2 + 2 + 3 = 11$? Recount with the true costs.

Recounting carefully with $w_{sa} = 1, w_{at} = 1, w_{ab} = 1, w_{sb} = 2, w_{bt} = 1$:

$$3(1) + 1(1) + 2(1) + 1(2) + 3(1) = 3 + 1 + 2 + 2 + 3 = 11.$$

That is *cheaper* than 13, which is impossible for a flow of value 5 on this instance — so the wounded flow is infeasible somewhere. Check conservation at a: inflow $= f_{sa} = 3$, outflow $= f_{at} + f_{ab} = 1 + 2 = 3$. Good. At b: inflow $= f_{ab} + f_{sb} = 2 + 1 = 3$, outflow $= f_{bt} = 3$. Good. At s: outflow $= f_{sa} + f_{sb} = 3 + 1 = 4 \neq 5$. There is the bug: this flow has value 4, not 5. The lesson is worth the detour — when you hand-build a flow it is easy to break conservation at the source, and the value drops silently. We therefore take a genuinely value-5 but suboptimal flow instead.

A clean suboptimal start. Route all 5 units but pay too much on the last one: $f_{sa} = 3$, $f_{at} = 2$, $f_{ab} = 1$, $f_{sb} = 2$, $f_{bt} = 3$ is the optimum at cost 13. Swap the role of one unit so it takes a strictly worse residual loop. Push one unit around the cycle $a \to b \to t \to$ (reverse of $a \to t$) $\to a$: that is, increase f_{ab} by 1 (now 2), increase f_{bt} — but $b \to t$ is already at 3, its capacity. So that loop is

blocked, confirming the optimum is locally rigid. Instead start from a different feasible value-5 flow that routes through b wastefully:

$$f_{sa} = 3,\ f_{at} = 1,\ f_{ab} = 2,\ f_{sb} = 2,\ f_{bt} = 4.$$

But $f_{bt} = 4 > 3 = c_{bt}$ violates capacity. The instance is tight enough that value 5 has an essentially unique feasible routing, which is itself optimal — a fine fact to notice, but unhelpful for demonstrating canceling.

To get room to be wasteful, *raise* a capacity: temporarily set $c_{bt} = 5$. Now a clearly suboptimal value-5 flow exists:

$$f_{sa} = 3,\ f_{at} = 1,\ f_{ab} = 2,\ f_{sb} = 2,\ f_{bt} = 4, \qquad \text{cost} = 3 + 1 + 2 + 4 + 4 = 14.$$

(Conservation: a has 3 in, $1 + 2 = 3$ out; b has $2 + 2 = 4$ in, 4 out; s sends $3 + 2 = 5$. Value 5.) This pays 14, one more than necessary, because it ships a unit over $a \to b \to t$ (cost $1 + 1 = 2$) that could have gone over the now-saturated-in-the-optimum $a \to t$ (cost 1).

Cancellation. Build G_f and look for a negative cycle. The forward arc $a \to t$ still has residual capacity ($c_{at} - f_{at} = 2 - 1 = 1$), cost $+1$. The backward arc $b \to a$ exists (since $f_{ab} = 2 > 0$), cost $-w_{ab} = -1$. The backward arc $t \to b$ exists ($f_{bt} = 4 > 0$), cost -1. Consider the cycle

$$C: \quad a \xrightarrow{+1} t \xrightarrow{-1} b \xrightarrow{-1} a,$$

using forward $a \to t$, backward $t \to b$ (un-shipping a unit of $b \to t$), backward $b \to a$ (un-shipping a unit of $a \to b$). Residual cost $= 1 + (-1) + (-1) = -1 < 0$: a negative cycle (Figure 14.2). Bottleneck $\delta = \min(c_f(a,t), f_{bt}, f_{ab}) = \min(1, 4, 2) = 1$. Push 1 around C: $f_{at}: 1 \to 2$, $f_{bt}: 4 \to 3$, $f_{ab}: 2 \to 1$. New flow $f_{sa} = 3, f_{at} = 2, f_{ab} = 1, f_{sb} = 2, f_{bt} = 3$, cost $14 + (-1) = 13$. Another Bellman–Ford pass finds no negative cycle, so by Theorem 14.1 we are optimal — and the answer matches the SSP build-up below. Restoring $c_{bt} = 3$ leaves this flow feasible and still optimal.

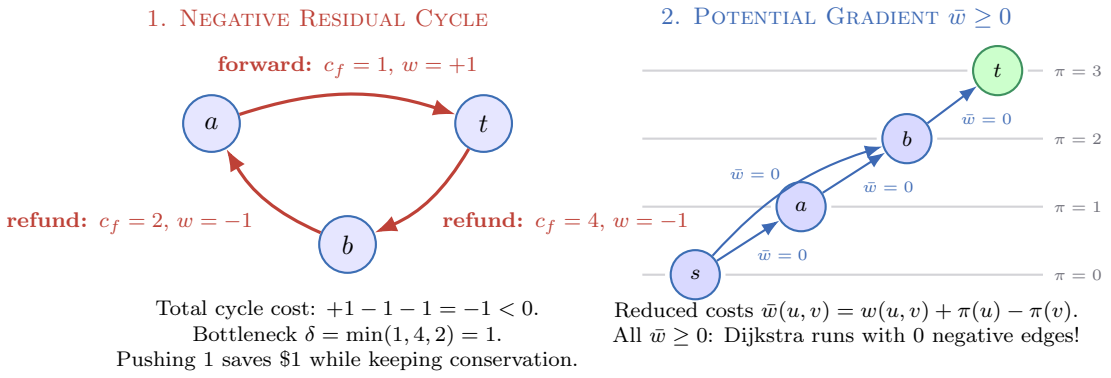


Figure 14.2: The two main tools of minimum-cost flow. Left: the negative residual cycle C in a suboptimal flow (c_{bt} temporarily raised to 5). The forward arc costs $+1$, while the two backward arcs refund -1 each, yielding a net savings of -1 per unit of circulation. Right: vertex potentials $\pi(v)$ transform arc costs into nonnegative reduced costs $\bar{w}(u,v) \geq 0$, so Dijkstra can compute shortest paths without Bellman–Ford.

Common trap

The false starts above illustrate the two mistakes everyone makes the first time. (1) Hand-building a flow and breaking conservation at s so the "value" is wrong; always re-derive $|f|$ from the source arcs, never assume it. (2) Trying to demonstrate improvement on an instance whose optimum is forced, where no negative cycle exists to cancel. When you want to *see* the algorithm move, give it slack (a raised capacity, an extra parallel arc) so a suboptimal flow can exist at all.

14.4 Successive shortest paths

Cycle canceling fixes a wasteful flow. The successive-shortest-paths (SSP) algorithm never makes a wasteful one: it grows the flow from zero, and *each* augmentation goes along a cheapest residual s–t path. The invariant it maintains is exactly the certificate we want.

Intuition

Augmenting paths in plain max-flow could be any s–t path with spare capacity. Here we are pickier: always take the *cheapest* such path. Sending the next unit along the globally cheapest route can never create a negative cycle, so the flow stays optimal at its current value at every step. When the value reaches F, optimality at value F is automatic.

Lemma 14.1 (SSP invariant)**.** *If f is a min-cost flow of value $|f|$, and P is a minimum-cost s–t path in G_f, then augmenting f along P (by its bottleneck) yields a min-cost flow of the new value $|f| + \delta$.*

Proof. Suppose not: the augmented flow f' has a negative cycle C in $G_{f'}$. The arcs of $G_{f'}$ differ from those of G_f only along P (forward arcs of P may have shrunk, their reverses appeared). So C must use at least one reverse arc of P; otherwise C already existed in G_f, contradicting optimality of f. Take such a C. Splicing C with P — cancel the shared reverse-of-P arcs — produces an s–t walk in G_f of cost $\text{cost}(P) + \text{cost}(C) < \text{cost}(P)$, hence a cheaper s–t path than P in G_f (after removing any incidental cycles, which are nonnegative by optimality of f). That contradicts P being a minimum-cost path. Therefore no negative cycle exists and f' is optimal. The base case $f = 0$ has an empty residual graph with no cycles, so it is optimal. By induction the invariant holds throughout. □

Why it works

Lemma 14.1 is the SSP analogue of Theorem 14.1: instead of certifying optimality after the fact, it *preserves* optimality across one augmentation. The key observation is that a hypothetical new negative cycle would have to thread through the reverse arcs of P, and that immediately hands you a cheaper path than the one Dijkstra or Bellman–Ford just declared shortest — a contradiction.

Algorithm 61: Successive Shortest Paths (with potentials)

Input: graph G, capacities c, costs w, source s, sink t, target value F (or ∞ for max-flow).
Output: a min-cost flow of value $\min(F, F^\star)$.

$f \leftarrow 0$;
initialize potentials $\pi(v)$ for all v ; `// Bellman--Ford once if costs may be negative; else 0`
val $\leftarrow 0$;
while val $< F$ ***and*** *t reachable from s in G_f* **do**
  run Dijkstra from s in G_f with reduced costs $\bar{w}(u,v) = w(u,v) + \pi(u) - \pi(v)$;
  let $d(v)$ be the resulting shortest reduced distances;
  foreach *v with $d(v) < \infty$* **do** $\pi(v) \leftarrow \pi(v) + d(v)$;
  let P be the shortest-path tree path from s to t;
  $\delta \leftarrow \min\big(F - \text{val},\ \min_{(u,v)\in P} c_f(u,v)\big)$;
  augment f by δ along P; val $\leftarrow$ val $+ \delta$;
return f;

14.4.1 Why Dijkstra needs help: potentials and Johnson reweighting

Residual graphs have negative arcs — every backward arc carries a negated cost. Dijkstra's correctness guarantee breaks on negative edges. We could fall back to Bellman–Ford on every iteration at $O(VE)$ per path, but there is a much faster way, borrowed from Johnson's all-pairs algorithm in the shortest-path chapters.

Maintain a **potential** $\pi(v)$ at each vertex and define the **reduced cost**

$$\bar{w}(u,v) \;=\; w(u,v) + \pi(u) - \pi(v).$$

Two facts make this work:

Lemma 14.2 (Reduced costs are path-invariant up to endpoints)**.** *For any directed path P from u to v,*

$$\sum_{(x,y)\in P} \bar{w}(x,y) \;=\; \text{cost}(P) + \pi(u) - \pi(v).$$

The internal π-terms telescope. Consequently, for any fixed endpoints, a path that minimizes reduced cost also minimizes original cost: shortest paths are invariant under vertex potential reweighting.

Lemma 14.3 (Reduced costs stay nonnegative)**.** *Set $\pi(v)$ equal to the true shortest-path distance from s to v in the current residual graph. Then for every residual arc (u,v), $\bar{w}(u,v) = w(u,v) + \pi(u) - \pi(v) \geq 0$, because shortest-path distances satisfy the triangle inequality $\pi(v) \leq \pi(u) + w(u,v)$.*

So after each Dijkstra run, we *update* $\pi(v) \mathrel{+}= d(v)$ (where d is the reduced-cost distance found this round). The central claim is that this update keeps reduced costs nonnegative even on the arcs that just appeared in the residual graph.

Lemma 14.4 (Potentials remain valid after augmentation)**.** *After updating potentials and augmenting along a shortest path P, every residual arc — including the newly created reverse arcs along P — has nonnegative reduced cost under the new π.*

Proof. Let $\pi'(v) = \pi(v) + d(v)$ be the updated potential, where d solves the single-source shortest paths under reduced costs $\bar{w}$. For any arc (u, v) that existed and was used by Dijkstra, $d(v) \leq d(u)+\bar{w}(u, v)$, i.e. $d(v) \leq d(u)+w(u, v)+\pi(u)-\pi(v)$, which rearranges to $w(u, v)+\pi'(u)-\pi'(v) \geq 0$. For a reverse arc (v, u) newly created because (u, v) lay on the shortest path P: on P the inequality is tight, $d(v) = d(u) + \bar{w}(u, v)$, so $w(u, v) + \pi'(u) - \pi'(v) = 0$, and the reverse arc has cost $-w(u, v)$ with reduced cost $-w(u, v) + \pi'(v) - \pi'(u) = 0 \geq 0$. Hence validity is preserved. □

Figure 14.3 visualizes why this potential update preserves Dijkstra-readiness: along any shortest path, the distance triangle inequality is tight, which forces the newly created reverse arc to have reduced cost identically equal to zero.

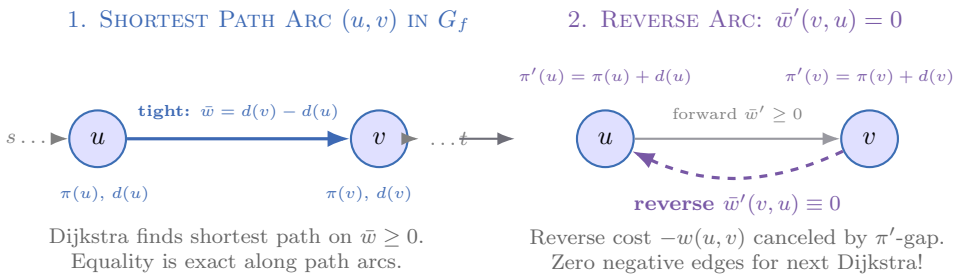


Figure 14.3: Potential invariance under augmentation in Successive Shortest Paths. In panel 1, Dijkstra discovers a shortest path where every traversed arc (u, v) satisfies $d(v) = d(u) + \bar{w}(u, v)$. In panel 2, after pushing flow and updating potentials to $\pi' = \pi + d$, the newly opened reverse arc (v, u) receives reduced cost $\bar{w}'(v, u) = -w(u, v) + \pi'(v) - \pi'(u) = 0$, guaranteeing that no negative reduced costs ever enter $G_{f'}$.

Because every reverse arc emerges with $\bar{w}' = 0$ and all existing arcs maintain $\bar{w}' \geq 0$, no negative reduced costs ever appear, so Dijkstra can be used in every round.

Why it works

This is what makes SSP fast. The first potential pass might need Bellman–Ford ($O(VE)$) if the original costs include negatives, but *every subsequent* shortest-path computation is a plain nonnegative-weight Dijkstra at $O(E \log V)$. The potentials are just the running shortest-path distances from s; Lemma 14.4 guarantees that every later residual graph stays Dijkstra-ready.

Common trap

A frequent bug: recomputing $\bar{w}$ from the *original* π after augmenting, instead of accumulating $\pi(v)$ += $d(v)$. If you forget the update, the newly created reverse arcs can show negative reduced cost on the next iteration and Dijkstra silently returns wrong distances — no crash, just a flow that is not minimum cost. Always update potentials before the next Dijkstra, and use ∞-distance vertices' potentials with care (leave them unchanged; they are unreachable this round).

Cost summary

With integer capacities bounded by U and value F: each augmentation raises the value by at least 1, so at most F augmentations, each a Dijkstra at $O(E \log V)$ (or $O(E + V \log V)$ with Fibonacci heaps). Total: $O\big(F \cdot E \log V\big)$, plus one $O(VE)$ Bellman–Ford for the initial potentials. Bottleneck (scaling-the-bottleneck) augmentation pushes the largest feasible δ each time and reduces the count to $O(E \log U)$ phases. For unit-capacity graphs (matching!), $F \le V$, giving $O(VE \log V)$.

14.4.2 A reference implementation

Pseudocode hides the bookkeeping that bites in practice: paired residual arcs, ∞-distance vertices, and the potential update. Here is a compact but complete SSP with potentials, written so the data structures are explicit. Arcs are stored in an array; arc e and its paired reverse arc $e \oplus 1$ sit next to each other, so flipping the low bit jumps between them.

Listing 14.1: SSP with Johnson potentials and Dijkstra.

```
struct Arc { int to, cap, cost; };       // residual cap and per-unit cost
vector<Arc> arc;                          // arc[e] and arc[e^1] are a pair
vector<vector<int>> g;                    // g[u] = indices of arcs out of u

void add_arc(int u, int v, int cap, int cost) {
    g[u].push_back(arc.size()); arc.push_back({v, cap,  cost});
    g[v].push_back(arc.size()); arc.push_back({u, 0,   -cost}); // reverse
}

// Returns {flow, cost} of a min-cost flow of value up to F from s to t.
pair<long long,long long> min_cost_flow(int n, int s, int t, int F) {
    const long long INF = 1e18;
    vector<long long> pi(n, 0);
    // One Bellman-Ford for initial potentials if negative costs may exist.
    // (Omitted when all costs >= 0: pi stays 0.)
    long long flow = 0, cost = 0;
    while (flow < F) {
        vector<long long> d(n, INF);          // reduced-cost distances
        vector<int> par_arc(n, -1);
        priority_queue<pair<long long,int>,
            vector<pair<long long,int>>, greater<>> pq;
        d[s] = 0; pq.push({0, s});
        while (!pq.empty()) {
            auto [du, u] = pq.top(); pq.pop();
            if (du > d[u]) continue;          // stale heap entry
            for (int e : g[u]) {
                const Arc& a = arc[e];
                if (a.cap <= 0) continue;     // no residual capacity
                long long rw = a.cost + pi[u] - pi[a.to];   // reduced cost
                if (d[u] + rw < d[a.to]) {
                    d[a.to] = d[u] + rw;
                    par_arc[a.to] = e;
                    pq.push({d[a.to], a.to});
                }
```

```
            }
        }
        if (d[t] == INF) break;                // t unreachable: done
        for (int v = 0; v < n; ++v)            // accumulate potentials
            if (d[v] < INF) pi[v] += d[v];
        // Bottleneck along the s->t tree path.
        long long push = F - flow;
        for (int v = t; v != s; ) {
            int e = par_arc[v];
            push = min<long long>(push, arc[e].cap);
            v = arc[e ^ 1].to;
        }
        for (int v = t; v != s; ) {            // apply the augmentation
            int e = par_arc[v];
            arc[e].cap     -= push;
            arc[e ^ 1].cap += push;
            v = arc[e ^ 1].to;
        }
        flow += push;
        cost += push * (pi[t] - pi[s]);        // true cost of this path
    }
    return {flow, cost};
}
```

Why it works

Three lines implement the theory. `rw = a.cost + pi[u] - pi[a.to]` is Johnson's reduced cost (Lemma 14.2); it is provably ≥ 0 (Lemma 14.3), so the priority-queue Dijkstra is valid. `pi[v] += d[v]` is the potential update that keeps the *next* round Dijkstra-ready (Lemma 14.4). And `cost += push * (pi[t] - pi[s])` reads the true path cost straight off the potentials, because after the update $\pi(t) - \pi(s)$ equals the accumulated true shortest distance from s to t — no need to walk the path summing w. Note the reverse arc starts at capacity 0: it only opens up once forward flow has been pushed, which is exactly when un-shipping should become possible.

Common trap

The branch `if (du > d[u]) continue;` discards stale heap entries; without it, a vertex can be expanded many times and the running time silently degrades. Equally important: only vertices with $d[v] < \infty$ get their potential updated. Bumping an unreachable vertex's potential by ∞ corrupts every future reduced cost. If you ever see Dijkstra return a negative distance, the culprit is almost always a skipped potential update or a potential nudged on an unreachable node.

14.5 Working a graph by hand

Take the instance of Figure 14.1 and find a min-cost *max*-flow with SSP. Arcs and their (c, w): $s \to a\,(3,1)$, $s \to b\,(2,2)$, $a \to t\,(2,1)$, $b \to t\,(3,1)$, $a \to b\,(2,1)$. All original costs are nonnegative, so initial potentials are 0 and the first Dijkstra needs no Bellman–Ford.

Trace it yourself

Keep three columns as you read: current flow on each arc, the path Dijkstra finds, and the running cost. Recompute reduced costs after each augmentation using updated π. Try to predict each path before reading on.

Iteration 1. Residual = original. True shortest s–t costs: via a: $s \to a \to t = 1 + 1 = 2$; via b: $s \to b \to t = 2 + 1 = 3$; via $a \to b$: $1 + 1 + 1 = 3$. Cheapest path $P_1 = s \to a \to t$, cost 2/unit. Bottleneck $= \min(c_{sa}, c_{at}) = \min(3, 2) = 2$. Push 2. Now $f_{sa} = 2$, $f_{at} = 2$. Value $= 2$, cost $= 2 \cdot 2 = 4$. Potentials become the distances: $\pi(s) = 0, \pi(a) = 1, \pi(b) = 2, \pi(t) = 2$.

Iteration 2. Residual changes: $a \to t$ is saturated (gone forward), reverse $t \to a$ appears (cost -1); $s \to a$ has residual capacity 1 left. Reduced costs $\bar{w} = w + \pi(u) - \pi(v)$:

$$s \to a: \ 1 + 0 - 1 = 0, \quad s \to b: \ 2 + 0 - 2 = 0, \quad a \to b: \ 1 + 1 - 2 = 0,$$
$$b \to t: \ 1 + 2 - 2 = 1, \quad t \to a: \ -1 + 2 - 1 = 0.$$

All reduced costs are ≥ 0, so Dijkstra applies. Cheapest residual s–t route: $s \to a \to b \to t$ has reduced cost $0 + 0 + 1 = 1$, true cost $1 + 1 + 1 = 3$; alternatively $s \to b \to t$ reduced $0 + 1 = 1$, true 3. Both cost 3/unit. Take $P_2 = s \to a \to b \to t$. Residual bottleneck: $c_f(s, a) = 1$, $c_f(a, b) = 2$, $c_f(b, t) = 3$, so $\delta = 1$. Push 1. Now $f_{sa} = 3, f_{ab} = 1, f_{bt} = 1$. Value $= 3$, cost $= 4 + 3 = 7$. Update potentials by d: $d(s) = 0, d(a) = 0, d(b) = 0, d(t) = 1$, so $\pi = (0, 1, 2, 3)$.

Iteration 3. Can we push more? $s \to a$ is now saturated ($f = 3 = c$); the only remaining source arc with capacity is $s \to b$ ($c = 2, f = 0$, residual 2). Path $s \to b \to t$: $b \to t$ residual $= 3 - 1 = 2$. So $P_3 = s \to b \to t$, true cost/unit $= 2 + 1 = 3$, bottleneck $= \min(2, 2) = 2$. Push 2. Value $= 5$, cost $= 7 + 2 \cdot 3 = 13$.

Iteration 4. Now both source arcs are saturated ($s \to a : 3/3$, $s \to b : 2/2$), so t is unreachable from s in G_f. Stop. Maximum value $= 5$, minimum cost $= 13$.

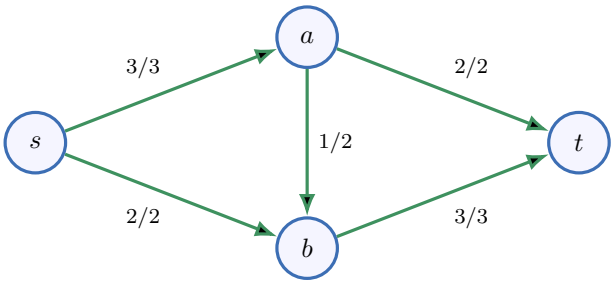


Figure 14.4: Final min-cost max-flow, shown as f/c per arc. Value 5, cost 13. Cheap arcs ($a \to t$) filled first; the costly $s \to b$ used only once the cheaper route saturated.

Trace it yourself

Now confirm optimality the *other* way. Build G_f for the flow above and run Bellman–Ford looking for a negative cycle. List the residual arcs first: every saturated arc ($s \to a$, $s \to b$, $a \to t$, $b \to t$) leaves only its reverse, and $a \to b$ (flow 1 of 2) keeps both directions. Now notice s has no outgoing residual arc and t has no incoming one, so no residual cycle can touch s or t. The only directed cycle left is $a \to b \to a$, with cost $w_{ab} + (-w_{ab}) = 1 + (-1) = 0 \geq 0$. No negative cycle exists, so Theorem 14.1 certifies the same answer the build-up produced.

14.6 Cost scaling: the idea

SSP charges once per augmentation, so its running time scales with the flow value F — pseudopolynomial when capacities are huge. Cost scaling removes that dependence by attacking the costs bit by bit, in the same spirit as capacity scaling for max-flow.

Intuition

Relax optimality. Call a flow ε-optimal if every residual arc has reduced cost $\geq -\varepsilon$ (a little negativity is tolerated). Start with ε large enough that any flow qualifies. Then repeatedly *halve* ε and "refine" the flow back to ε-optimality with push–relabel-style operations on reduced costs. Once $\varepsilon < 1/n$ with integer costs, ε-optimal forces exactly optimal, because a negative cycle would have total reduced cost $\geq -n\varepsilon > -1$, impossible for integers summing to something negative.

Definition 14.3 (ε-optimality)**.** A flow f with potentials π is ε-optimal if every residual arc (u, v) satisfies $\bar{w}(u, v) = w(u, v) + \pi(u) - \pi(v) \geq -\varepsilon$.

Theorem 14.2 (Scaling terminates at the optimum)**.** *With integer costs and $n = |V|$ vertices, any ε-optimal flow with $\varepsilon < 1/n$ is optimal.*

Proof. Take any residual cycle C ($|C| \leq n$ arcs). The π-terms telescope around a cycle, so $\sum_C \bar{w} = \sum_C w = \text{cost}(C)$. By ε-optimality each $\bar{w} \geq -\varepsilon$, so $\text{cost}(C) \geq -|C|\,\varepsilon > -n \cdot (1/n) = -1$. With integer costs $\text{cost}(C)$ is an integer greater than -1, hence ≥ 0. No negative cycle exists, so by Theorem 14.1 f is optimal. □

Each scaling phase ($\varepsilon \to \varepsilon/2$) re-establishes ε-optimality using push and relabel operations that admit only arcs with reduced cost below $-\varepsilon/2$, mirroring the max-flow push–relabel machinery you saw earlier. Goldberg and Tarjan proved each phase costs $O(V^2E)$ (or $O(V^3)$ with the right implementation), and there are $O(\log(nW))$ phases, for a total of $O(V^2E \log(nW))$ or better.

Figure 14.5 illustrates why ε-optimality with $\varepsilon < 1/n$ certifies exact optimality: even if every arc on an n-cycle takes the worst-case negative reduced cost $-\varepsilon$, the cycle sum remains strictly greater than -1, which forces the integer cycle cost to be nonnegative.

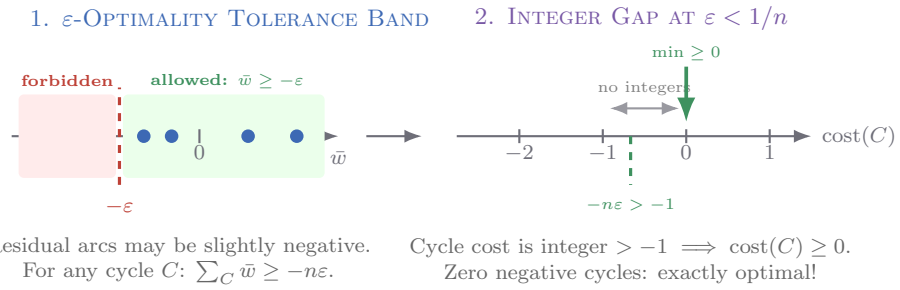


Figure 14.5: The integer squeeze in cost scaling. In panel 1, ε-optimality tolerates small negative reduced costs $\bar{w}(u, v) \geq -\varepsilon$, bounding any directed cycle's sum by $\text{cost}(C) = \sum_C \bar{w} \geq -n\varepsilon$. In panel 2, once scaling drives $\varepsilon < 1/n$, the lower bound strictly exceeds -1. Because all arc costs are integers, $\text{cost}(C)$ cannot lie in $(-1, 0)$ and must be nonnegative, certifying that no negative residual cycles remain.

After $O(\log(nW))$ halvings, $\varepsilon < 1/n$, and Theorem 14.2 shows the flow is exactly optimal.

Cost summary

Algorithm	Running time	Notes
Cycle canceling (any cycle)	$O(VE \cdot mWU)$	pseudopolynomial, simple
Min-mean cycle canceling	$O(V^2E^2 \log V)$	strongly polynomial
SSP (Dijkstra+potentials)	$O(F \cdot E \log V)$	scales with value F
SSP, bottleneck scaling	$O(E \log U \cdot E \log V)$	weakly polynomial
Cost scaling (push–relabel)	$O(V^2E \log(nW))$	method of choice in practice

$m = |E|$, $U =$ max capacity, $W = \max|w|$, $F =$ flow value.

History

Minimum-cost flow developed over several decades by combining shortest paths and network flows. In the late 1950s, Lester Ford and Delbert Fulkerson, William Jewell, Masao Iri, and Robert Busacker and Paul Gowen formulated the successive-shortest-paths framework, repeatedly augmenting flow along cheapest residual paths. In 1967, Morton Klein introduced negative-cycle canceling, proving that a flow is optimal if and only if its residual network contains no directed cycles of negative cost. For twenty years, however, both approaches remained pseudopolynomial, potentially making unbounded numbers of tiny augmentations on large capacities.

Progress toward strongly polynomial algorithms came in the 1970s and 1980s. Nobuaki Tomizawa (1971) and independently Jack Edmonds and Richard Karp (1972) introduced vertex potentials to maintain nonnegative reduced costs, allowing Dijkstra's algorithm to replace Bellman–Ford in successive-shortest-path computations. In 1989, Andrew Goldberg and Robert Tarjan proved that canceling the cycle of minimum mean cost (found in $O(VE)$ time via Richard Karp's algorithm) terminates in $O(V^2E^2 \log V)$ steps, achieving strong polynomiality. Goldberg and Tarjan also adapted push–relabel to cost scaling (1987–1990), which remains the standard practical algorithm. In 2022, Li Chen, Rasmus Kyng, Yang Liu, Richard Peng, Maximilian Probst Gutenberg, and Sushant Sachdeva achieved an almost-linear $O(m^{1+o(1)})$ runtime using continuous interior-point methods and dynamic ℓ_1 data structures.

14.7 Transportation and assignment as special cases

Two classical optimization problems reduce directly to min-cost flow. Recognizing the reduction lets you solve them with the machinery above instead of bespoke code.

14.7.1 The transportation problem

You have warehouses $1, \dots, p$ with supplies a_i and stores $1, \dots, q$ with demands b_j ($\sum a_i = \sum b_j$). Shipping one unit from warehouse i to store j costs w_{ij}. Minimize total shipping cost while meeting all demands.

Model it as a bipartite flow. Add a super-source s with arc (s, i) of capacity a_i, cost 0; a super-sink

t with arc (j, t) of capacity b_j, cost 0; and arcs (i, j) of capacity ∞ (or $\min(a_i, b_j)$), cost w_{ij}. A min-cost max-flow saturates all supply/demand arcs and routes goods at minimum cost.

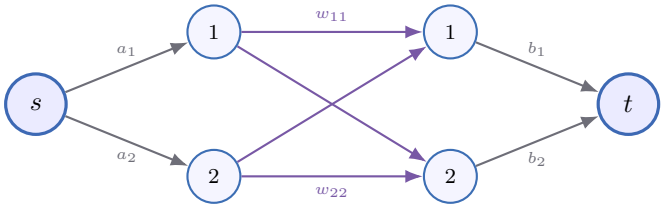


Figure 14.6: Transportation as a flow: capacities on the source/sink arcs encode supplies and demands; middle-arc costs are shipping prices.

14.7.2 The assignment problem

Assignment is transportation with all $a_i = b_j = 1$ and $p = q = n$: match n workers to n jobs, one-to-one, minimizing total cost $\sum w_{i,\sigma(i)}$ over permutations σ. As a flow it is a bipartite graph with unit capacities everywhere; a min-cost max-flow of value n is a minimum-cost perfect matching.

Application

This is precisely the problem the Hungarian algorithm solves, and running SSP on the unit-capacity bipartite network is equivalent step-for-step to the Hungarian algorithm: the potentials π are the dual variables (the "labels"), each shortest-path augmentation matches one more worker, and Lemma 14.4 is the statement that labels stay feasible. The weighted-bipartite-matching chapter develops this view fully; here it is one special case of a general method. Because $F = n$ and capacities are 1, SSP runs in $O(n \cdot E \log n) = O(n^3 \log n)$, matching the classical bound.

Connection

With unit capacities, the integrality of min-cost flow pays off directly: every basic optimal solution is integral, so the min-cost flow LP *automatically* returns a 0/1 matching with no rounding. This is the totally-unimodular structure of the node-arc incidence matrix — the same property that makes max-flow integral. See the bipartite-matching chapters and the LP appendix for total unimodularity.

14.8 The linear-programming view

Min-cost flow is a linear program, and its dual is exactly the potentials we have been carrying around. Making that precise explains every reduced cost in the chapter.

For the fixed-value problem with node demands $d(v)$ (with $d(s) = -F$, $d(t) = +F$, $d(v) = 0$ else),

the primal is

$$\begin{aligned} \text{minimize} \quad & \sum_{(u,v)\in E} w(u,v)\, f(u,v) \\ \text{s.t.} \quad & \sum_{v} f(u,v) - \sum_{v} f(v,u) = -d(u) \quad \forall u \quad \text{(conservation, dual var } \pi_u\text{)} \\ & 0 \le f(u,v) \le c(u,v) \quad \forall (u,v) \quad \text{(capacity, dual var } \rho_{uv} \ge 0\text{)}. \end{aligned}$$

Assigning dual variable π_u to each conservation constraint and $\rho_{uv} \ge 0$ to each upper bound, the dual is

$$\begin{aligned} \text{maximize} \quad & \sum_{u} d(u)\, \pi_u - \sum_{(u,v)} c(u,v)\, \rho_{uv} \\ \text{s.t.} \quad & \pi_v - \pi_u - \rho_{uv} \le w(u,v), \qquad \rho_{uv} \ge 0. \end{aligned}$$

Rearranged, the dual constraint says $\rho_{uv} \ge -\bar{w}(u,v)$ where $\bar{w}(u,v) = w(u,v) + \pi_u - \pi_v$ is exactly our reduced cost. Complementary slackness then reads:

Theorem 14.3 (Optimality via complementary slackness)**.** *A feasible flow f and potentials π are jointly optimal iff for every arc (u,v):*

- *if $\bar{w}(u,v) > 0$ then $f(u,v) = 0$ (too expensive: do not use it);*
- *if $\bar{w}(u,v) < 0$ then $f(u,v) = c(u,v)$ (a bargain: saturate it);*
- *if $0 < f(u,v) < c(u,v)$ then $\bar{w}(u,v) = 0$ (interior flow only on tight arcs).*

Why it works

These three conditions are the same statement as "no negative residual cycle," viewed through duality. An arc with $\bar{w} < 0$ and unsaturated flow, or $\bar{w} > 0$ with positive flow, is precisely a residual arc of negative reduced cost — the seed of an improving cycle. So Theorem 14.3 and Theorem 14.1 are two forms of the same fact. The potentials π that SSP accumulates are an *optimal dual solution* being built incrementally; at termination they satisfy complementary slackness against the flow.

Connection

Strong LP duality guarantees primal optimum = dual optimum, which here specializes to a min-cost-flow analogue of max-flow min-cut. For pure max-flow ($w \equiv 0$, value-maximizing) the dual collapses to the min-cut, recovering the theorem of the previous chapters. The potentials reappear in the Laplacian and electrical-flow chapters as *voltages*: an electrical flow minimizes energy $\sum r_{uv} f_{uv}^2$ subject to conservation, and the optimality condition is again "potential differences drive the flow," now via Ohm's law $f_{uv} = (\pi_u - \pi_v)/r_{uv}$. Min-cost flow is the linear-cost counterpart of that quadratic problem.

Why this matters

Minimum-cost flow can be understood from three perspectives. *Combinatorially*: cancel negative residual cycles until no improving circulation exists. *Algorithmically*: route successive shortest paths and maintain vertex potentials that keep reduced costs nonnegative. *Analytically*: solve a linear program whose dual variables are precisely those node potentials. Each has its use: cycle canceling certifies optimality, successive shortest paths matches the structure

of bipartite matching, and cost scaling gives the best practical performance.

Minimum-cost flow handles capacities and costs simultaneously. When each node needs to pair with at most one partner on an opposing side, the problem specializes to bipartite matching. The next chapter shows how alternating paths and a min-max duality theorem solve this pairing problem.

14.9 Exercises

Exercise 14.1 (Trace the canceler)**.** Take the flow of Figure 14.4 but *perturb* it: reroute one unit from $s \to a \to t$ to $s \to a \to b \to t$ and one unit of $b \to t$ correspondingly, keeping value 5 but raising cost. Build G_f, find the negative cycle Bellman–Ford reports, cancel it, and confirm you return to cost 13.

Exercise 14.2 (Potentials by hand)**.** Re-run the SSP trace of Section 14.5 but introduce a negative cost: set $w(a \to b) = -2$. Compute the initial potentials with one Bellman–Ford pass, then run Dijkstra with reduced costs for each augmentation. Verify Lemma 14.4 numerically at every step.

Exercise 14.3 (Why same value)**.** Give a two-vertex example of flows f, f' with $|f| \neq |f'|$ where $\text{cost}(f') - \text{cost}(f)$ is *not* a sum of residual-cycle costs, illustrating why Theorem 14.1 restricts to equal values.

Exercise 14.4 (Break Dijkstra)**.** Construct a residual graph with a negative arc on which running Dijkstra *without* potentials returns an incorrect shortest path, then show the potential reweighting fixes it. State the smallest such graph you can find.

Exercise 14.5 (Bottleneck scaling)**.** Implement the bottleneck-scaling variant of SSP: maintain a threshold Δ, augment only along paths whose bottleneck is $\geq \Delta$, then halve Δ. Prove the number of phases is $O(\log U)$ and bound the augmentations per phase.

Exercise 14.6 (Assignment = flow)**.** Encode a 3×3 assignment problem (your choice of w_{ij}) as a unit-capacity min-cost flow, run SSP, and read off the optimal permutation. Then identify which potentials correspond to the Hungarian algorithm's row and column labels.

Exercise 14.7 (Lower bounds on arcs)**.** Extend the model to arcs with a lower bound $\ell(u, v) \leq f(u, v) \leq c(u, v)$. Show how to reduce to a standard min-cost flow by sending ℓ units along each arc up front and adjusting supplies/demands, and prove feasibility is preserved.

Exercise 14.8 (Convex costs)**.** Suppose an arc's cost is a *convex* piecewise-linear function of its flow (the more you ship, the more each extra unit costs). Show how to model this with parallel arcs of increasing cost and prove that SSP fills them in the correct order. Where does convexity become essential?

Exercise 14.9 (Min-mean cycle)**.** Implement Karp's $O(VE)$ minimum-mean-cycle algorithm and plug it into Algorithm 60. Empirically compare the number of cancellations against "any negative cycle" on a few random graphs; relate your counts to the Goldberg–Tarjan bound.

Exercise 14.10 (Dual reading)**.** For the final flow of Section 14.5, write down the potentials π at termination and verify all three complementary-slackness conditions of Theorem 14.3 arc by arc. Confirm the dual objective equals the primal cost 13.

Exercise 14.11 (Cancel vs. SSP)**.** Prove that running negative-cycle canceling to completion and running SSP to the same value both yield flows of identical cost (the optimum), even though the intermediate flows differ. Which algorithm can revisit and *undo* earlier routing decisions, and which never does?

Exercise 14.12 (Slack to be wasteful)**.** Section 14.3.2 had to raise a capacity before a suboptimal value-5 flow even existed. Characterize exactly when an instance has a unique feasible flow of a given value (hint: think about the residual graph having no cycle at all). Construct the smallest instance where two distinct flows of equal value exist, one strictly cheaper, and trace one cancellation between them.

Exercise 14.13 (Read the code)**.** In Listing 14.1, the cost of each augmentation is computed as `push * (pi[t] - pi[s])` rather than by summing w along the path. Prove this equals the true path cost after the potential update. Then break it deliberately: remove the `if (d[v] < INF)` guard on the potential update and exhibit a small graph where the reported cost becomes wrong.

Exercise 14.14 (Paired arcs)**.** The reference implementation pairs arc e with arc $e \oplus 1$. Explain why `add_arc` must push the forward and reverse arcs consecutively for the bit trick to work, and what breaks if you instead store reverse arcs in a separate array indexed by a map. Reimplement with an explicit `rev[]` index array and argue the two are equivalent.

Chapter 15

Bipartite Matching

In the previous chapter, we saw how assignment and transportation problems reduce directly to minimum-cost flow. When capacities are all one and costs are uniform, the problem simplifies to pairing compatible items across two disjoint sets so that no vertex is used more than once.

This is *bipartite matching*. A worker takes one job, a student occupies one room, a processor runs one task. Although the problem reduces to unit-capacity network flow, its combinatorial structure yields direct algorithms: alternating paths that flip edges in place, and the Hopcroft–Karp batching strategy that achieves $O(E\sqrt{V})$ time without constructing a flow network.

15.1 Matchings: the language

Definition 15.1 (Bipartite graph)**.** A graph $G = (V, E)$ is *bipartite* if its vertices split into two disjoint sets L and R (the left and right *sides*, or *color classes*) so that every edge has one endpoint in L and one in R. We write $G = (L \cup R, E)$ and set $n = |V| = |L| + |R|$, $m = |E|$.

Equivalently, G is bipartite iff it has no odd cycle (a two-coloring of the vertices exists iff every cycle has even length). A single breadth-first sweep finds the bipartition or an odd cycle, so we will assume L and R are given.

Definition 15.2 (Matching)**.** A *matching* $M \subseteq E$ is a set of edges no two of which share an endpoint. A vertex incident to an edge of M is *matched* (or *saturated*); otherwise it is *free* (or *exposed*). A matching is *maximum* if no matching has more edges, and *perfect* if every vertex is matched (so $|M| = |L| = |R|$).

Intuition

Picture the left column as people and the right column as chairs. A matching is a collection of "person sits in chair" edges where no person sits in two chairs and no chair holds two people. "Maximum" means: seat as many people as possible. The question is never about one edge – it is about the global tradeoff, because greedily grabbing one edge can block two better ones.

Here is the running example for the whole chapter. The left side is $L = \{a, b, c, d\}$, the right side is

$R = \{1, 2, 3, 4\}$, and the edges are

$$E = \{a1,\ a2,\ b1,\ b3,\ c3,\ c4,\ d3\}.$$

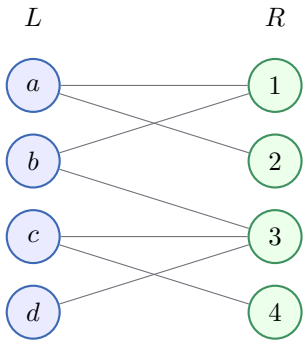


Figure 15.1: The running bipartite graph G. Left $L = \{a, b, c, d\}$, right $R = \{1, 2, 3, 4\}$.

A greedy person scanning edges in the listed order might grab $a1$, then $b3$, then $c4$, and stop: d wants only chair 3, which is taken, and a's alternative chair 2 is now stranded with nobody left to ask cleanly. That greedy run produces $\{a1, b3, c4\}$ of size 3. But $\{a2, b1, c4, d3\}$ has size 4 and is perfect, so greedy fell one short. The rest of the chapter shows how to repair such mistakes systematically.

15.2 Augmenting paths and Berge's theorem

Fix a matching M. The key device is to look at paths that *alternate* between non-matching and matching edges.

Definition 15.3 (Alternating and augmenting paths)**.** Relative to M, a path is *M-alternating* if its edges alternate between $E \setminus M$ and M. An alternating path is *M-augmenting* if it starts and ends at *free* vertices (necessarily one in L, one in R, and the path has odd length).

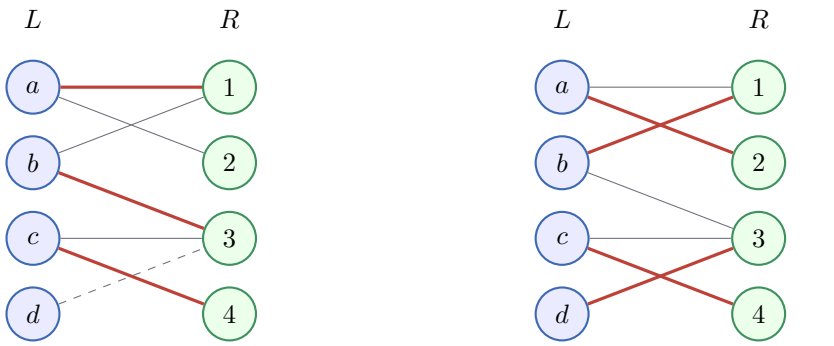
Intuition

An augmenting path is a chain "free person – empty chair – person – their chair – person – their chair – … – empty chair". Walk it and flip every edge: the non-matching edges (the "wants") become matches, the matching edges become free. Because the path starts and ends free and alternates, flipping turns k matched edges into $k+1$ matched edges. Net gain: exactly one. It is a chain of bumps: the first free person takes a chair, bumping its occupant, who takes the next chair, bumping again, until the last bump lands on an empty chair.

Formally, if P is M-augmenting then the symmetric difference

$$M' \ = \ M \oplus P \ = \ (M \setminus P) \cup (P \setminus M)$$

is again a matching, and $|M'| = |M| + 1$. The endpoints of P go from free to matched; every internal vertex keeps exactly one matched edge (its matched edge swaps to the other side of the path), so M' is a valid matching, and P has one more non-matching edge than matching edge.



(a) $M = \{a1, b3, c4\}$ **(b)** after augmenting along P

Figure 15.2: Left: matching $M = \{a1, b3, c4\}$ (thick red) with free vertices $d, 2$ and the augmenting path $P = d{-}3{-}b{-}1{-}a{-}2$ (it alternates $d3 \notin M$, $3b \in M$, $b1 \notin M$, $1a \in M$, $a2 \notin M$). Right: flipping P yields the perfect matching $\{a2, b1, c4, d3\}$ of size 4.

So an augmenting path is a certificate that M is *not* maximum. The harder fact is the converse: if no augmenting path exists, M already is maximum. This is Berge's theorem, and it holds for general (not just bipartite) graphs.

Theorem 15.1 (Berge, 1957)**.** *A matching M in a graph G is maximum if and only if G contains no M-augmenting path.*

Why it works

The forward direction is the easy half: if an augmenting path existed we could flip it and get a strictly larger matching, so a maximum M cannot admit one. For the reverse direction, the key tool is the symmetric difference of two matchings. Compare M with a hypothetical *better* matching M^*. Their symmetric difference is forced into a very rigid shape – disjoint paths and even cycles – and counting edges in those pieces produces an augmenting path whenever M^* is larger.

Proof. ($\Rightarrow$) If P is M-augmenting then $M \oplus P$ is a matching of size $|M| + 1$, so M is not maximum. Contrapositive: maximum $\Rightarrow$ no augmenting path.

($\Leftarrow$) Suppose M is not maximum; let M^* be a matching with $|M^*| > |M|$. Consider the edge set $H = M \oplus M^*$ and the subgraph it induces. Every vertex v is incident to at most one edge of M and at most one of M^*, hence to at most two edges of H. So in H every vertex has degree ≤ 2, which means each connected component of H is either a simple path or a simple cycle.

Within any component the edges alternate between M and M^*: two consecutive H-edges at a vertex cannot both be in M (they would share that vertex, impossible in a matching) nor both in M^*. Consequently every *cycle* component has even length and contains equally many M- and M^*-edges. Since $|M^*| > |M|$ overall, some component must contain strictly more M^*-edges than M-edges. That component cannot be a cycle, so it is a path, and a path with one extra M^*-edge must begin and end with an M^*-edge. Both endpoints of that path are therefore unmatched by M (an M-edge at an endpoint would extend the path), so the path is M-alternating with free ends: an M-augmenting path. Contrapositive gives the claim. □

Common trap

Two pitfalls. (1) Augmenting means flipping a path between *free* endpoints; flipping an alternating path that ends at a matched vertex does *not* increase the matching and can even break it. (2) Berge's theorem says a maximum matching has *no* augmenting path – it does *not* say a greedy/maximal matching has none. A maximal matching (one you cannot extend by adding a single edge) can be half the size of maximum; the running graph's greedy $\{a1, b3, c4\}$ is maximal but not maximum precisely because an augmenting path survives.

Berge's theorem hands us an algorithm template immediately: start from the empty matching and repeatedly find an augmenting path and flip it, until none remains. Each flip adds one edge, so after at most $\min(|L|, |R|)$ flips we stop with a maximum matching. Everything below is about finding those paths fast.

15.3 The min-max duality: Kőnig and Hall

Berge tells us when to stop. The dual theorems tell us *why* the number we stop at is what it is, by exhibiting a small obstruction whose size equals the matching's. This is the same min-max flavor as max-flow/min-cut, and indeed it is a special case of it.

Definition 15.4 (Vertex cover)**.** A *vertex cover* is a set $C \subseteq V$ touching every edge: for each $uv \in E$, $u \in C$ or $v \in C$. Write $\tau(G)$ for the minimum cover size and $\nu(G)$ for the maximum matching size.

For *any* graph $\nu \leq \tau$: a matching has ν disjoint edges, and a cover must spend at least one distinct vertex per matched edge. Kőnig's theorem shows that for bipartite graphs, this inequality is always tight.

Theorem 15.2 (Kőnig, 1931)**.** *In a bipartite graph, $\nu(G) = \tau(G)$: the maximum matching size equals the minimum vertex cover size.*

Proof. We have $\nu \leq \tau$ always. For the reverse, let M be a maximum matching; Berge guarantees no M-augmenting path. Direct every non-matching edge from L to R and every matching edge from R to L, and let Z be the set of vertices reachable by M-alternating paths that *start at a free vertex of* L (do a search from all free left vertices following the alternation rule).

Define the cover

$$C \;=\; (L \setminus Z) \;\cup\; (R \cap Z).$$

We claim C is a vertex cover of size $|M|$.

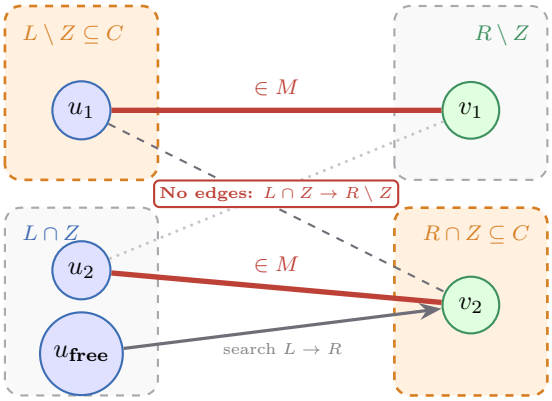
C covers every edge. Take any edge uv with $u \in L$, $v \in R$.

- If $u \notin Z$ then $u \in L \setminus Z \subseteq C$, covered.
- If $u \in Z$, then there is an alternating path from a free left vertex to u. The edge uv is either in M or not. If $uv \notin M$, the search extends the alternating path across uv to v, so $v \in Z$, hence $v \in R \cap Z \subseteq C$. If $uv \in M$, then u was reached by arriving *along* uv from v (the only matched edge at u), so again $v \in Z$ and $v \in C$.

Either way uv is covered, so C is a vertex cover.

$|C| = |M|$. First, every vertex of $L \setminus Z$ is matched: free left vertices are in Z by definition, so a vertex outside Z on the left is not free. Second, every vertex of $R \cap Z$ is matched: a free right vertex in Z would be the end of an M-augmenting path, contradicting maximality. So every vertex of C is saturated by M. We now show each matched edge contributes exactly one endpoint to C. Take a matched edge $u'v'$ with $u' \in L$, $v' \in R$. If $v' \in R \cap Z$, then v' was reached, and the only way the alternating search enters the right side along a matched edge is from its left endpoint, so the path reaches v' via u', forcing $u' \in Z$, i.e. $u' \notin L \setminus Z$. Thus u' and v' are never both in C, and since every vertex of C is the endpoint of a matched edge, $|C| = |M|$. Combined with $\tau \geq \nu = |M|$ we get $\tau = |M| = \nu$. □

Figure 15.3 visualizes this four-quadrant decomposition: the alternating search partitions both L and R by reachability from free left vertices. The minimum vertex cover $C = (L \setminus Z) \cup (R \cap Z)$ takes precisely one endpoint of every matched edge, while no edge can cross from $L \cap Z$ to $R \setminus Z$ without expanding Z.



Cover $C = (L \setminus Z) \cup (R \cap Z)$ picks exactly one vertex per matched edge.
Every non-matched edge is covered; no edges cross to $R \setminus Z$. $|C| = |M|$.

Figure 15.3: Kőnig's minimum vertex cover construction. Alternating reachability from free left vertices partitions the graph into four quadrants. The vertex cover $C = (L \setminus Z) \cup (R \cap Z)$ (shaded orange) contains exactly the left endpoint of matches in the unreached half and the right endpoint of matches in the reached half. By definition of Z, no edge can cross from $L \cap Z$ to $R \setminus Z$, certifying that every edge is covered and $|C| = |M|$.

Every edge in the bipartite graph is therefore covered, and the cover size matches the matching cardinality exactly: $\tau(G) = |C| = |M| = \nu(G)$.

Trace it yourself

Run this on the running graph with maximum matching $M = \{a2, b1, c4, d3\}$. Every left vertex is matched, so there is no free left vertex to seed the search and $Z = \varnothing$. Then $C = (L \setminus \varnothing) \cup \varnothing = L = \{a, b, c, d\}$, size 4. Indeed L itself is a cover of size $4 = \nu$, and no smaller cover exists because the matching already has 4 disjoint edges. Now try the non-maximum $M = \{a1, b3, c4\}$: the alternating search from free vertex d reaches 3 (via $d3$), then b (via matched $b3$), then 1 (via $b1$), then a (via matched $a1$), then 2 (via $a2$) – vertex 2 is free, so the search exposes an augmenting path and certifies that no cover of size 3 exists.

Hall's theorem is the special case that asks when a *perfect* matching of the left side exists. For $S \subseteq L$ let $N(S) = \{r \in R : \exists s \in S,\, sr \in E\}$ be its neighborhood.

Theorem 15.3 (Hall's marriage theorem, 1935)**.** *A bipartite graph has a matching saturating every vertex of L if and only if $|N(S)| \geq |S|$ for every $S \subseteq L$.*

Proof. Necessity. If L is saturated, the $|S|$ matched partners of S are distinct and lie in $N(S)$, so $|N(S)| \geq |S|$.

Sufficiency. Suppose Hall's condition holds but the maximum matching misses some left vertex, so $\nu < |L|$. By Kőnig there is a cover C with $|C| = \nu < |L|$. Split $C = C_L \cup C_R$ with $C_L = C \cap L$, $C_R = C \cap R$. Let $S = L \setminus C_L$; then $|S| = |L| - |C_L| > |C_R|$. Every edge from S must be covered, and no vertex of S is in C, so all such edges are covered on the right by C_R; hence $N(S) \subseteq C_R$ and $|N(S)| \leq |C_R| < |S|$, violating Hall. So $\nu = |L|$ and L is saturated. □

Corollary 15.1 (Defect form / Ore)**.** *The maximum matching saturates all but $\max_{S \subseteq L} \big(|S| - |N(S)|\big)$ vertices of L; equivalently $\nu = |L| - \max_{S \subseteq L}(|S| - |N(S)|)$.*

Figure 15.4 illustrates a Hall bottleneck: a subset $S \subseteq L$ of 3 vertices has only 2 collective neighbors in R, so by the pigeonhole principle no matching saturates L.

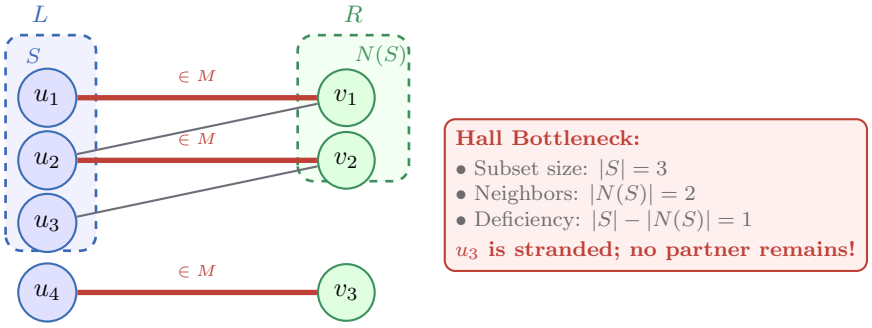


Figure 15.4: Hall's marriage condition violation. The subset $S = \{u_1, u_2, u_3\} \subseteq L$ has collective neighborhood $N(S) = \{v_1, v_2\} \subseteq R$. Because $|N(S)| = 2 < 3 = |S|$, the pigeonhole principle prevents any matching from saturating S. At least one vertex (u_3) is left permanently unmatched, demonstrating the defect bound $\nu \leq |L| - (|S| - |N(S)|)$.

Because no matching can map 3 vertices into 2 distinct targets, the defect is at least $|S| - |N(S)| = 3 - 2 = 1$, directly certifying that the maximum matching size cannot exceed 3.

Connection

Kőnig's theorem is the totally-unimodular special case of LP duality: the matching polytope of a bipartite graph has integral vertices, so the LP relaxation of maximum matching and its dual (fractional vertex cover) both attain integer optima, and strong duality gives $\nu = \tau$. Total unimodularity in the companion linear-algebra volume and the integral-flow theorem in Chapter 12 express the same underlying integrality property. We make the flow connection explicit in Section 15.6.

15.4 The augmenting-path algorithm

Berge converts maximum matching into "repeatedly find an augmenting path." In a bipartite graph an augmenting path is easy to find with a single search rooted at a free left vertex, because of the orientation trick: walk forward along unmatched edges $L \to R$ and backward along matched edges $R \to L$. If such a walk reaches a free right vertex, we have an augmenting path.

The classic implementation (Kuhn's algorithm, the bipartite specialization of the Hungarian method's first phase) does a depth-first search from each left vertex in turn, trying to assign it a partner, recursively bumping current occupants.

Algorithm 62: AUGMENT(u) – try to match left vertex u by DFS

Input: left vertex u; arrays $matchR[\,]$ (current partner of each right vertex, or $\perp$) and $seen[\,]$ (right vertices visited this search).
Output: TRUE if u can be matched, updating $matchR$.

```
foreach r ∈ N(u) do
    if not seen[r] then
        seen[r] ← TRUE
        if matchR[r] = ⊥ or AUGMENT(matchR[r]) then
            matchR[r] ← u
            return TRUE
return FALSE
```

Algorithm 63: MAXBIPARTITEMATCHING(L, R, E)

```
matchR[r] ← ⊥ for all r ∈ R
size ← 0
foreach u ∈ L do
    reset seen[r] ← FALSE for all r ∈ R
    if AUGMENT(u) then
        size ← size + 1
return size and matchR
```

Why it works

Why does the recursive bump find an augmenting path? When AUGMENT(u) visits an unmatched neighbor r it grabs it – an augmenting path of length 1. When it visits a *matched* neighbor r whose current owner is w, it asks w to relocate; if w succeeds, the recursion has built a longer alternating path $u{-}r{-}w{-}\ldots{-}r'$ ending at a free right vertex r', and unwinding the recursion flips exactly that path. The *seen* array stops the DFS from revisiting a right vertex, which both guarantees termination and keeps each search linear.

Why it works

[Correctness of the outer loop] The outer loop processes left vertices one at a time and never *unmatches* anyone permanently: a successful AUGMENT only relocates owners along an aug-

menting path, so the matching size strictly increases and previously matched left vertices stay matched. The invariant is that after processing the first k left vertices, the current matching is maximum on the subgraph induced by those k vertices and all of R. A failed AUGMENT(u) certifies (by the same alternating-reachability argument as Kőnig) that no augmenting path from u exists relative to the current matching, and adding later left vertices can never create one rooted at the already-failed u. So one pass suffices and the final matching is maximum.

Trace on the running graph. Adjacency in listed order: $a : \{1, 2\}$, $b : \{1, 3\}$, $c : \{3, 4\}$, $d : \{3\}$. Process a, b, c, d.

step	call	what happens	*matchR* after
1	AUGMENT(a)	1 free → take it	$1{:}a$
2	AUGMENT(b)	1 owned by a; bump a: a takes 2 (free)	$1{:}b,\ 2{:}a$
3	AUGMENT(c)	3 free → take it	$1{:}b, 2{:}a, 3{:}c$
4	AUGMENT(d)	3 owned by c; bump c: c takes 4 (free)	$1{:}b, 2{:}a, 3{:}d, 4{:}c$

Step 2 is the augmenting path $b-1-a-2$ (length 3); step 4 is $d-3-c-4$. The final matching $\{b1, a2, d3, c4\}$ is perfect, size 4 – exactly the optimum greedy missed.

Cost summary

Each call to MAXBIPARTITEMATCHING runs $|L|$ outer iterations. Each iteration is a DFS that, thanks to *seen*, touches every edge at most once: $O(m)$. Total

$$O(|L| \cdot m) = O(VE).$$

With $V = n$ and $m \le \frac{n^2}{4}$ this is $O(n^3)$ in the dense worst case. Space is $O(V)$ beyond the graph. The bound is tight for adversarial vertex orderings: paths can grow to length $\Theta(V)$ and you may pay $\Theta(V)$ per augmentation.

Common trap

Resetting *seen between* outer iterations (once per left vertex) is correct and necessary; resetting it *inside* the recursion, or forgetting to reset it, both break the algorithm – the first reintroduces exponential blowup, the second makes AUGMENT return spurious failures. Also note AUGMENT marks r as seen *before* recursing, never after; marking after recursion lets two branches chase the same right vertex.

15.5 Hopcroft–Karp: augment in batches

Kuhn pays $O(m)$ per single augmenting path and needs up to ν of them. Hopcroft and Karp (1973) observed that we are wasteful: instead of finding *one* shortest augmenting path per phase, find a *maximal set of vertex-disjoint shortest augmenting paths* all at once, and flip them together.

Two structural facts make this both correct and fast: the shortest augmenting path length strictly increases from phase to phase, and after $O(\sqrt{V})$ phases the matching is maximum.

Intuition

Think of it as flood-then-drain. A breadth-first sweep from all free left vertices, alternating unmatched/matched as it descends, lays down a layered "distance" structure and finds the shortest distance ℓ to any free right vertex. Then a depth-first sweep greedily extracts as many length-ℓ augmenting paths as it can without reusing a vertex, and flips them all. Each BFS sweep raises the shortest-path length, so the lengths go $1, 3, 5, \ldots$ and cannot go on long. Short paths are cheap and plentiful; the expensive long paths are provably few.

Algorithm 64: HopcroftKarp(L, R, E)

```
1 matchL[u] ← ⊥, matchR[r] ← ⊥ for all u, r
2 size ← 0
3 while BFS() reports a free-to-free layered path exists do
4     foreach free u ∈ L do
5         if DFS(u) then
6             size ← size + 1
7 return size
```

Algorithm 65: BFS() – build layers, return whether any augmenting path exists

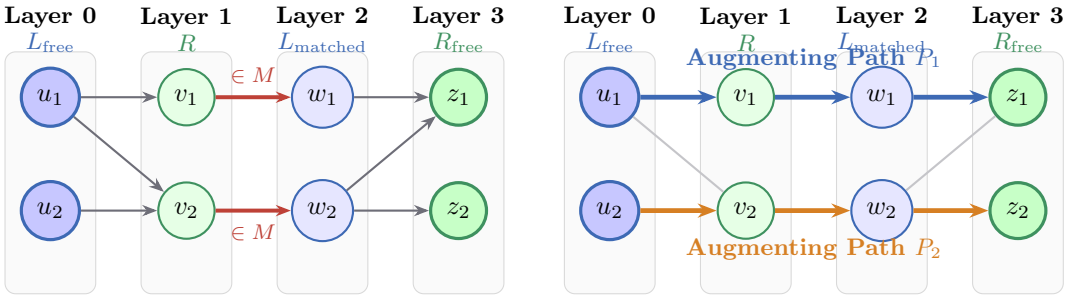
```
1 queue Q ← all free u ∈ L; set dist[u] ← 0 for these, dist[·] ← ∞ otherwise; found ← FALSE
2 while Q nonempty do
3     pop u
4     foreach r ∈ N(u) do
5         w ← matchR[r]
6         if w = ⊥ then
7             found ← TRUE
8         else if dist[w] = ∞ then
9             dist[w] ← dist[u] + 1; push w onto Q
10 return found
```

Algorithm 66: DFS(u) – extract one shortest augmenting path from u

```
1 foreach r ∈ N(u) do
2     w ← matchR[r]
3     if w = ⊥ or (dist[w] = dist[u] + 1 and DFS(w)) then
4         matchR[r] ← u; matchL[u] ← r; return TRUE
5 dist[u] ← ∞; return FALSE
```

The BFS computes $dist[u]$ = length (counted in hops over matched edges) of the shortest alternating path from a free left vertex to u. The DFS only descends along edges that increase $dist$ by exactly one, so it walks shortest augmenting paths; setting $dist[u] \leftarrow \infty$ on failure prunes dead ends so no edge is explored twice in a phase. Figure 15.5 illustrates this two-phase structure: the BFS lays down the level DAG, and the DFS greedily extracts a maximal vertex-disjoint family of augmenting

paths in a single pass. A phase therefore runs in $O(m)$.



(a) BFS layered DAG: alternating paths from free left vertices to free right targets

(b) DFS extraction: maximal vertex-disjoint paths P_1, P_2 flipped simultaneously

Figure 15.5: The two phases of Hopcroft–Karp. (a) A forward breadth-first search starts at free left vertices L_{free} (Layer 0), traversing unmatched edges $L \to R$ and matched edges $R \to L$ to build a layered DAG of shortest alternating distances until reaching free right vertices R_{free} (Layer 3). (b) A depth-first search then traverses only DAG edges with $dist[w] = dist[u] + 1$, greedily collecting a maximal family of vertex-disjoint shortest augmenting paths (P_1 in blue, P_2 in orange) to augment simultaneously in $O(m)$ time.

Trace on the running graph. Start $M = \varnothing$; all of a, b, c, d free.

Phase 1. BFS: every free left vertex has $dist = 0$; all their right neighbors are unmatched, so the shortest augmenting length is 1 ($found =$ TRUE). DFS in order a, b, c, d, each grabbing a disjoint free right neighbor. a takes 1; b takes 3; c's neighbor 3 is used this phase, so c takes 4; d's only neighbor 3 is used this phase, so d fails. After phase 1: $M = \{a1, b3, c4\}$, size 3, d still free.

Phase 2. Only free left vertex is d ($dist[d] = 0$). BFS from d: neighbor 3 is matched to b, set $dist[b] = 1$, push b. From b: neighbor 1 matched to a, set $dist[a] = 2$, push a. From a: neighbor 2 unmatched $\Rightarrow$ $found =$ TRUE (shortest length now 5). DFS from d: $d \to 3$ (owner b, $dist[b] = 1 = dist[d] + 1$), recurse $b \to 1$ (owner a, $dist[a] = 2$), recurse $a \to 2$ (unmatched) – success. Flip the chain $d-3-b-1-a-2$: now $d3, b1, a2$ matched, $c4$ untouched. $M = \{a2, b1, c4, d3\}$, size 4.

Phase 3. No free left vertex remains; BFS returns FALSE; stop. Two phases of work, as the $\sqrt{V}$ bound predicts ($\sqrt{4} = 2$).

15.5.1 Why the phase count is $O(\sqrt{V})$

The analysis rests on a monotonicity lemma and a counting argument. Let ℓ_i be the length (number of edges) of the shortest augmenting path at the start of phase i.

Lemma 15.1 (Lengths strictly increase)**.** *If phase i augments along a maximal set of vertex-disjoint shortest augmenting paths of length ℓ_i, then the shortest augmenting path of the new matching has length $\ell_{i+1} > \ell_i$.*

Proof sketch. Let M be the matching before the phase and M' the matching after flipping the maximal vertex-disjoint family $\mathcal{P}$ of shortest (ℓ_i-length) augmenting paths. Suppose for contradiction M' admits an augmenting path Q with $|Q| \le \ell_i$. Look at $M \oplus M' \oplus Q$. The piece $M \oplus M'$ is exactly

the union of the paths in $\mathcal{P}$ (vertex-disjoint, each of length ℓ_i). The standard Hopcroft–Karp edge count shows $M \oplus Q$ must contain an M-augmenting path that is vertex-disjoint from all of $\mathcal{P}$ and has length $\leq \ell_i$; that path could have been added to $\mathcal{P}$, contradicting its maximality. If instead Q shares a vertex with $\mathcal{P}$, the shared vertex forces Q to reuse a flipped edge, and a parity count on the alternating lengths gives $|Q| > \ell_i$. Either way $\ell_{i+1} > \ell_i$. □

Augmenting paths have odd length, so the lengths run $1, 3, 5, \ldots$, strictly increasing.

Theorem 15.4 (Hopcroft–Karp phase bound)**.** *The algorithm terminates after at most $O(\sqrt{\nu}) = O(\sqrt{V})$ phases, where ν is the size of the maximum matching.*

Proof. Run the algorithm until the shortest augmenting path first has length exceeding $\sqrt{\nu}$. By Lemma 15.1 the lengths are $\geq 1, 3, 5, \ldots$, so this happens within $\lceil \sqrt{\nu} \rceil$ phases. Let M be the current matching at that moment and M^* a maximum matching. As in Berge's proof, $M \oplus M^*$ is a disjoint union of paths and even cycles and contains exactly $\nu - |M|$ vertex-disjoint M-augmenting paths. Each such path now has length $> \sqrt{\nu}$, hence uses more than $\frac{1}{2}\sqrt{\nu}$ edges of M. These paths are vertex-disjoint and all their M-edges lie in the $\leq \nu$ edges of M, so the number of paths is at most $\nu/(\frac{1}{2}\sqrt{\nu}) = 2\sqrt{\nu}$. Thus $\nu - |M| \leq 2\sqrt{\nu}$: at most $2\sqrt{\nu}$ augmentations remain, and each later phase performs at least one, so at most $2\sqrt{\nu}$ further phases finish. Total phases $\leq \lceil \sqrt{\nu} \rceil + 2\sqrt{\nu} = O(\sqrt{\nu}) = O(\sqrt{V})$. □

Cost summary

Each phase is one BFS plus one DFS sweep, each $O(m)$ amortized (the *dist* $\leftarrow \infty$ pruning ensures every edge is examined a constant number of times per phase). With $O(\sqrt{V})$ phases,

$$\boxed{O(E\sqrt{V})}$$

time and $O(V)$ extra space. For dense graphs this is $O(V^{2.5})$ versus Kuhn's $O(V^3)$. On bounded-degree or random graphs the constant is small and Hopcroft–Karp is the practical default.

History

Bipartite matching grew out of both matrix theory and combinatorics. In 1931, Dénes Kőnig established that the maximum size of a matching equals the minimum size of a vertex cover, motivated by Frobenius's work on matrix determinants. In 1935, Philip Hall formulated his marriage theorem for systems of distinct representatives, providing the neighborhood condition for when every element of one bipartition can be matched. Claude Berge in 1957 established the algorithmic viewpoint by proving that a matching is maximum if and only if it admits no augmenting path.

In 1973, John Hopcroft and Richard Karp proved that searching for maximal families of shortest augmenting paths simultaneously in breadth-first layers bounds the number of phases by $O(\sqrt{V})$, achieving an overall running time of $O(E\sqrt{V})$. In the Soviet Union, Alexander Karzanov independently obtained the same bound using unit-capacity blocking flows, reflecting the direct equivalence between Hopcroft–Karp and Dinitz's layered search. For five decades, $O(E\sqrt{V})$ remained the best known general bound. In 2022, Li Chen, Rasmus Kyng, Yang Liu, Richard Peng, Maximilian Probst Gutenberg, and Sushant Sachdeva improved the

asymptotic bound via their $m^{1+o(1)}$ almost-linear flow framework, though Hopcroft–Karp remains the practical implementation of choice on real-world graphs.

15.6 Matching via maximum flow

Every result above has a direct counterpart in network flow, and making the reduction explicit both gives an alternative algorithm and explains *why* the min-max theorems hold.

The construction. Build a directed flow network G':

- a source s and sink t;
- an arc $s \to u$ of capacity 1 for every $u \in L$;
- an arc $u \to r$ of capacity 1 (or ∞) for every edge $ur \in E$, directed $L \to R$;
- an arc $r \to t$ of capacity 1 for every $r \in R$.

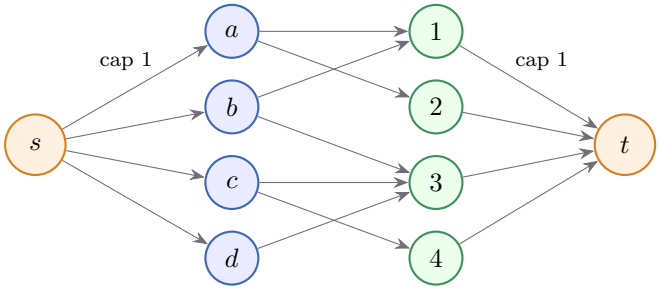


Figure 15.6: The running graph turned into a unit-capacity s–t network. Every s–t path has the form $s \to u \to r \to t$ and uses three unit-capacity arcs.

Theorem 15.5 (Matching = integral max flow)**.** *Integral s–t flows of value k in G' correspond exactly to matchings of size k in G; in particular the maximum flow value equals $\nu(G)$.*

Proof. All capacities are integral, so by the integral flow theorem a maximum flow can be taken integral, with every arc carrying 0 or 1. Given such a flow f, let $M = \{ur \in E : f(u \to r) = 1\}$. Conservation at $u \in L$, whose only in-arc is the capacity-1 arc $s \to u$, forces at most one out-arc of u to carry flow, so u meets at most one edge of M; symmetrically each $r \in R$ feeds the single capacity-1 arc $r \to t$, so r meets at most one edge of M. Thus M is a matching, and $|M| = \sum_u f(s \to u) = |f|$. Conversely a matching M gives an integral flow of value $|M|$ by routing one unit $s \to u \to r \to t$ for each $ur \in M$. The correspondence is value-preserving, so $\max|f| = \nu$. □

Why it works

This is why Kőnig is "just" max-flow/min-cut. Give the middle $L \to R$ arcs capacity ∞, so a finite minimum s–t cut never cuts one; it consists of cut $s \to u$ arcs and cut $r \to t$ arcs only. The corresponding left vertices and right vertices together form a vertex cover, and min-cut = max-flow becomes $\tau = \nu$. Hall is the "max flow saturates all of L" specialization. The flow

viewpoint hands you the dual certificate for free.

Why it works

[Augmenting paths are augmenting paths] A residual s–t path in G' alternates: forward $s \to u$, forward $u \to r$ (an unused matching edge), backward $r \to u'$ (a used edge, i.e. matched, traversed against its flow), forward $u' \to r'$, ..., forward $r'' \to t$. Strip the s and t endpoints and this is precisely an M-augmenting path in G: forward arcs are non-matching edges, backward arcs are matching edges, and the endpoints u, r'' are free. Ford–Fulkerson on G' *is* Berge's algorithm; BFS augmenting (Edmonds–Karp) on unit capacities reproduces Hopcroft–Karp's phase structure (Dinic blocking flow). The matching algorithms and the flow algorithms are the same algorithms.

Cost summary

Plain Ford–Fulkerson with BFS on this unit-capacity network gives the same $O(VE)$ as Kuhn. Dinic's blocking-flow algorithm on a unit-capacity network runs in $O(E\sqrt{V})$ – identical to Hopcroft–Karp, and for the same reason (the $\sqrt{V}$-phase bound is a theorem about unit-capacity blocking flows). Thus reducing bipartite matching to unit-capacity flow runs the same augmenting phases with extra source and sink vertices. The flow framing wins when the model needs capacities > 1 (e.g. one job can absorb b workers), which a plain matching cannot express.

Application

The flow reduction makes bipartite matching a versatile modeling tool. Assigning radio frequencies to towers, scheduling exams to rooms and time slots, routing virtual machines to physical hosts, recovering a perfect matching in a Latin-square completion, and the "system of distinct representatives" problems all become a single max-flow call once phrased as left/right compatibility. Capacitated variants – a hospital that can absorb q residents, a warehouse that ships c units – are immediate by raising the $s \to u$ or $r \to t$ capacities, and the integral-flow theorem keeps the answer whole. The flow-modeling chapter of this book gives the full catalogue.

Connection

The matching polytope of a bipartite graph is the set of fractional matchings $\{x \geq 0 : \sum_{e \ni v} x_e \leq 1\}$, and its constraint matrix is the incidence matrix of a bipartite graph – which is *totally unimodular*. Total unimodularity forces every vertex of the polytope to be integral, so the LP and the combinatorial problem coincide. This is a natural entry point to the polyhedral and spectral material of the book: the same incidence matrix, its associated Laplacian $\mathbf{L} = BB^\top$ for incidence matrix B, and the rank facts that drive total unimodularity all reappear in the chapter on the graph Laplacian. Matching is one of the clearest places where combinatorial optimization and linear algebra meet.

15.7 Putting it together

algorithm	time	idea
Kuhn / augmenting DFS	$O(VE)$	one augmenting path per left vertex
Hopcroft–Karp	$O(E\sqrt{V})$	maximal disjoint shortest paths per phase
Dinic on unit network	$O(E\sqrt{V})$	blocking flow = Hopcroft–Karp

The central result is Berge's theorem (Theorem 15.1): a matching is maximum iff no augmenting path survives. Kőnig (Theorem 15.2) and Hall (Theorem 15.3) give the dual obstruction – a minimum vertex cover of equal size, and the neighborhood condition for saturating one side. The algorithms differ only in how they search for augmenting paths, and the flow reduction (Theorem 15.5) shows they all execute the same underlying residual-graph search.

Pause

Before the exercises, re-derive the fact the whole chapter depends on: take any two matchings, look at their symmetric difference, and convince yourself it is a disjoint union of paths and even cycles. Every theorem in this chapter – Berge, Kőnig (via the alternating-reachability set Z), Hall (via Kőnig), and the Hopcroft–Karp phase bound – comes from this picture with a different counting argument.

Every matching in this chapter treated edges identically, aiming only to pair as many vertices as possible. In practical assignments, compatibility comes in degrees: applicants have test scores, drivers have passenger ratings, and workers have wage expectations. The next chapter assigns each edge a numerical weight, introducing the Hungarian algorithm and primal–dual methods to find maximum-weight bipartite matchings.

15.8 Exercises

Exercise 15.1 (Greedy is a 2-approximation)**.** A *maximal* matching is one you cannot enlarge by adding a single edge. Prove that any maximal matching has at least $\nu/2$ edges, and exhibit a bipartite graph where greedy returns exactly $\nu/2$. (Hint: each greedy edge can block at most two optimal edges.)

Exercise 15.2 (Implement and instrument Kuhn)**.** Implement Algorithm 63. Add a counter of edges examined and run it on a graph designed to force $\Theta(VE)$ work. Confirm empirically that reordering L by ascending degree usually slashes the runtime, even though the worst case is unchanged.

Exercise 15.3 (Break a buggy SEEN reset)**.** Modify AUGMENT (Algorithm 62) to reset $seen[r]$ to FALSE just before returning TRUE. Find a small graph on which the modified routine reports a matching smaller than maximum, and explain which augmenting path it fails to find.

Exercise 15.4 (Kőnig from a run)**.** After computing a maximum matching with any algorithm, give an $O(V+E)$ procedure that outputs a minimum vertex cover of equal size, following the reachability set Z in the proof of Theorem 15.2. Run it on the chapter's graph and verify you recover a cover of size 4.

Exercise 15.5 (Hall, tightly)**.** Construct a bipartite graph with $|L| = |R| = n$ in which exactly one subset $S \subseteq L$ violates Hall's condition, with $|N(S)| = |S| - 1$. What is the maximum matching size, and which vertices does Corollary 15.1 predict are left unsaturated?

Exercise 15.6 (Phase lengths in Hopcroft–Karp)**.** Build a bipartite graph on which Hopcroft–Karp uses augmenting paths of lengths $1, 3, 5$ across three distinct phases (so the bound is exercised, not just the trivial single-phase case). Trace the BFS distance labels at the start of each phase using Algorithm 65.

Exercise 15.7 (Tightness of $O(E\sqrt{V})$)**.** It is known that on some bipartite graphs Hopcroft–Karp truly needs $\Theta(\sqrt{V})$ phases. Sketch a "staircase" of nested long alternating paths and argue that the shortest augmenting length grows by exactly 2 each phase, so $\Theta(\sqrt{V})$ phases occur.

Exercise 15.8 (Capacitated assignment via flow)**.** Suppose each right vertex r can be matched to up to q_r left vertices (a *b-matching* on the right). Show how to model this with the network of Section 15.6 by changing one set of capacities, and argue the integral-flow theorem still returns an integer solution. What is the analogue of Kőnig's theorem here?

Exercise 15.9 (Augmenting paths fail the easy way in general graphs)**.** Give a non-bipartite graph (an odd cycle plus a pendant edge will do) and a matching where the naive "DFS alternating from a free vertex" of Algorithm 62 reports failure even though an augmenting path exists. Explain why the odd cycle defeats the bipartite reachability argument – this is the blossom that Chapter 17 contracts.

Exercise 15.10 (Matching and the incidence matrix)**.** Let B be the $|V| \times |E|$ vertex-edge incidence matrix of a bipartite graph. Prove B is totally unimodular (every square submatrix has determinant in $\{-1, 0, 1\}$), and deduce that the fractional matching LP has an integral optimum. Relate this to detecting a perfect matching via the determinant of the bipartite biadjacency matrix (the Edmonds matrix idea).

Exercise 15.11 (Online matching, a glimpse ahead)**.** Left vertices arrive one at a time and must be matched irrevocably on arrival, or dropped. Show that the deterministic greedy rule is 2-competitive and no better, then look up and summarize the RANKING algorithm's $1 - 1/e$ guarantee. Why does the offline $O(E\sqrt{V})$ machinery not apply?

Chapter 16

Weighted Bipartite Matching and the Hungarian Algorithm

The previous chapter asked a pure cardinality question about bipartite graphs: how many disjoint pairs can we form? Now we attach a number to every possible pairing and ask for the *best* set of disjoint pairs. Workers and jobs, taxis and riders, antennas and frequencies – whenever each assignment carries a cost or a profit and each side must be used at most once, you are looking at the *assignment problem*. The algorithm that solves it, the Hungarian method of Kuhn and Munkres, is a standard example of *primal–dual* reasoning in combinatorial optimization. It maintains a feasible solution to a dual (a set of *potentials*), and uses that dual both to certify optimality and to drive the search. It is the method behind successive shortest paths for min-cost flow, specialized to bipartite graphs, where it is simple enough to run by hand.

16.1 The Assignment Problem

Focus

We are given a complete weighted bipartite graph on two sides L and R with $|L| = |R| = n$, and a cost $c(i, j) \in \mathbb{R}$ for assigning $i \in L$ to $j \in R$. A *perfect matching* (or *assignment*) is a bijection $\pi : L \to R$. We want the assignment of minimum total cost

$$\min_{\pi} \sum_{i \in L} c\big(i, \pi(i)\big).$$

Equivalently, choosing exactly one entry from each row and each column of an $n \times n$ cost matrix C, minimize the sum of chosen entries.

Two framings travel together throughout this chapter, and it pays to keep both in view.

- **Graph view.** A bipartite graph $G = (L \cup R, E)$ with edge weights. We seek a maximum-weight (or minimum-cost) perfect matching. This is the view in which *augmenting paths* live.
- **Matrix view.** An $n \times n$ matrix C, from which we pick a *transversal*: one cell per row, one per column. This is the view in which the *row/column reductions* of the original Hungarian

method live.

Intuition

Why not just try every assignment? There are $n!$ of them. For $n = 15$ that is already more than a trillion. Greedy – repeatedly grab the cheapest available cell – fails badly: committing i to its personal favourite j can strand a second worker who had *only* j as a cheap option, forcing an expensive repair later. The difficulty is that a locally cheap choice can be globally expensive. We need a way to make commitments that we can later *revoke for free*. That is exactly what augmenting paths and dual potentials give us.

Max-weight vs. min-cost; non-square inputs. Maximizing total weight $\sum w(i, \pi(i))$ and minimizing total cost $\sum c(i, \pi(i))$ are the same problem under $c = W - w$ for any large constant W (a perfect matching uses exactly n edges, so shifting every weight by W shifts every assignment's total by nW). If the two sides differ in size, pad the smaller side with dummy vertices joined by zero-cost edges; if a perfect matching need not exist or some pairings are forbidden, set forbidden costs to $+\infty$ (or a number larger than any achievable finite total). We therefore lose no generality assuming a *square, complete, finite-cost* instance, and we phrase the core algorithm as minimization.

History

Harold Kuhn published the algorithm in 1955 and named it the "Hungarian method" to honor mathematicians Dénes Kőnig and Jenő Egerváry. In 1931, Egerváry proved that the maximum weight of a bipartite matching equals the minimum sum of vertex weights that cover the edges – a result that anticipates strong linear programming duality by two decades. James Munkres reorganized Kuhn's matrix version in 1957 into the standard implementation used today. In 2006 it came to light that Carl Gustav Jacobi had formulated an essentially identical method in an 1840s Latin manuscript on systems of differential equations, where finding an optimal transversal determined the order of a differential system, though his work lay largely forgotten for over a century. Kuhn's algorithm led to the general *primal–dual method* in combinatorial optimization, inspiring Lester Ford and Delbert Fulkerson's network flow algorithms and Jack Edmonds's blossom algorithm. Even today, closing the complexity gap between bipartite matching and general min-cost flow for dense matrices remains an open research problem.

16.2 Picture First: Tight Edges and Potentials

Assign to each left vertex i a number u_i and to each right vertex j a number v_j. Think of u_i as "how much i has already been promised" and v_j as "how much j has already been promised." Call the pair (u, v) a *feasible potential* (or feasible *labeling*) if it never over-promises against the cost of any edge:

$$u_i + v_j \;\le\; c(i, j) \qquad \text{for every } i \in L,\ j \in R.$$

An edge (i, j) is *tight* (or *admissible*) when its slack is zero:

$$\text{slack}(i, j) \;=\; c(i, j) - u_i - v_j \;=\; 0.$$

Intuition

Feasibility says: the potentials together can "claim" no more than the edge actually costs. The *equality subgraph* $G_=$ consists of exactly the tight edges – those where the claim is exact, with no slack left over. The strategy is: *find a perfect matching using only tight edges.* If you can, the matching is optimal, because the potentials hand you a matching certificate of optimality. When you cannot, you nudge the potentials – carefully, keeping feasibility – so that at least one new edge becomes tight, growing $G_=$ until a perfect matching appears inside it.

Here is the lower bound that makes potentials useful. Take *any* perfect matching π and *any* feasible potential (u, v). Summing the feasibility inequality over the n matched edges, and using that π hits each left vertex once and each right vertex once,

$$\sum_i c\big(i, \pi(i)\big) \;\geq\; \sum_i \big(u_i + v_{\pi(i)}\big) \;=\; \sum_{i \in L} u_i + \sum_{j \in R} v_j.$$

The right-hand side does not depend on π. So:

Why it works

Weak duality. For every feasible potential (u, v) and every perfect matching π,

$$\underbrace{\sum_i c\big(i, \pi(i)\big)}_{\text{primal: cost of an assignment}} \qquad \geq \qquad \underbrace{\sum_i u_i + \sum_j v_j}_{\text{dual: total potential}} \;.$$

Hence *any* feasible potential lower-bounds the optimum, and *any* matching upper-bounds it. If we ever produce a matching and a potential with *equal* value, both are optimal simultaneously, and each certifies the other. Equality forces every matched edge to satisfy $u_i + v_{\pi(i)} = c(i, \pi(i))$ – every matched edge has zero slack. This condition is *complementary slackness*: whenever a primal variable $x_{ij} = 1$ is active, the corresponding dual constraint $u_i + v_j \leq c(i, j)$ must be tight. The algorithm preserves this tightness invariant on matched edges while growing the matching until it is perfect.

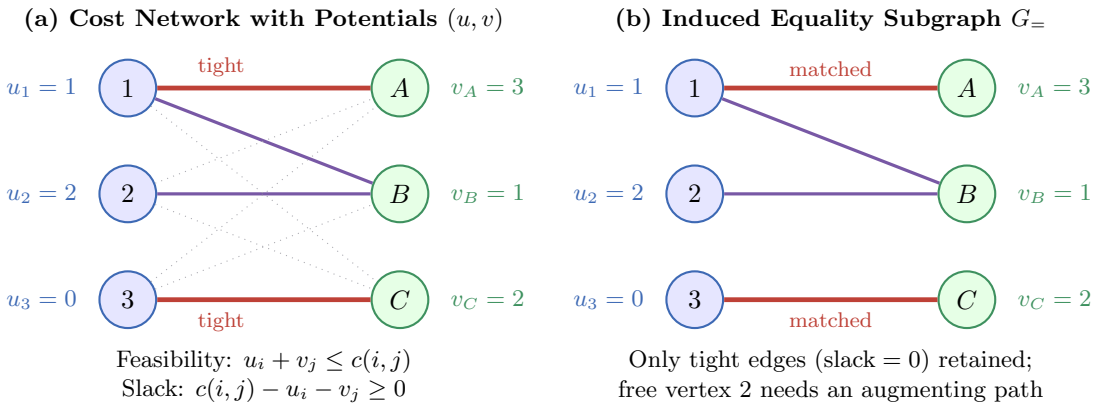


Figure 16.1: The role of vertex potentials and the equality subgraph. (a) Feasible potentials (u_i, v_j) satisfy $u_i + v_j \leq c(i, j)$ across all edges. Tight edges (purple/red) satisfy $u_i + v_j = c(i, j)$ with zero slack, while dotted edges have strictly positive slack. (b) The equality subgraph $G_=$ discards all positive-slack edges. Matched edges (thick red) satisfy complementary slackness. Any augmenting path in $G_=$ immediately preserves dual feasibility while growing the cardinality of the matching.

In the picture the matching $\{(1, A), (3, C)\}$ lives among tight edges, and 2 still needs a tight option to complete a perfect matching; the algorithm's job is to reshape potentials until vertex 2 acquires a tight edge that finishes the assignment. We now describe the algorithm that does this.

16.3 Augmenting Paths in the Equality Subgraph

Recall the matching vocabulary from the unweighted chapter. Given a matching M, a vertex is *free* if unmatched. An *alternating path* alternates between non-matching and matching edges; an *augmenting path* is an alternating path between two free vertices. Flipping (XOR-ing) M along an augmenting path increases $|M|$ by one. The new wrinkle: we only ever walk along *tight* edges, so that whatever matching we build automatically satisfies complementary slackness.

Focus

Invariant maintained throughout.

1. (u, v) is a feasible potential: $u_i + v_j \leq c(i, j)$ for all i, j.
2. M is a matching, and *every* edge of M is tight ($u_i + v_j = c(i, j)$).

When M becomes perfect, invariant (2) is exactly complementary slackness, and weak duality (Section 16.2) makes M optimal. The algorithm's only moves are: (a) augment along a tight alternating path, growing M; (b) adjust potentials to add a tight edge without breaking feasibility.

16.3.1 Growing a Hungarian tree

To match one more left vertex r (currently free), grow an alternating *tree* rooted at r inside the equality subgraph, exactly as in unweighted bipartite matching. Maintain two sets:

- $S \subseteq L$: left vertices reached by the tree (including the root r);
- $T \subseteq R$: right vertices reached by the tree.

A right vertex enters T via a tight edge from S; once in T, if it is matched, its matched partner enters S (the only way to leave a matched right vertex along an alternating path is through its matching edge). If some right vertex in our reach is *free*, we have found an augmenting path – flip it and we are done with this phase.

The danger: every tight edge out of S might land in T already. Then the tree is stuck inside the current equality subgraph. This is where the dual potentials come in.

16.3.2 The dual adjustment

Suppose the tree is stuck: there is no tight edge from S to $R \setminus T$. Compute the smallest slack across that frontier,

$$\delta \;=\; \min_{i \in S,\; j \in R \setminus T} \big(c(i,j) - u_i - v_j\big) \;>\; 0,$$

and update

$$u_i \mathrel{+}= \delta \;\; (i \in S), \qquad v_j \mathrel{-}= \delta \;\; (j \in T).$$

Why it works

Why this preserves feasibility *and* progress – check each edge class (i, j):

- $i \in S,\ j \in T$: u_i rises by δ, v_j falls by δ; slack unchanged. *Tree edges and matched edges stay tight.*
- $i \in S,\ j \notin T$: u_i rises by δ, v_j fixed; slack drops by δ and remains ≥ 0 by choice of δ. *At least one frontier edge becomes newly tight.*
- $i \notin S,\ j \in T$: u_i fixed, v_j falls; slack rises. Still feasible.
- $i \notin S,\ j \notin T$: untouched.

So feasibility holds, all tree/matched edges stay tight (the tree survives intact), and the frontier gains at least one tight edge – letting the tree grow by at least one more right vertex. After at most n such adjustments the tree must reach a free right vertex, and we augment.

Common trap

Three classic bugs.

1. **Updating the wrong side of T.** You add δ to u on S and subtract from v on T. Subtracting on all of R, or adding to u on all of L, breaks feasibility for edges with one endpoint outside the tree.
2. **Recomputing δ from scratch every time** – correct, but $O(n^2)$ per adjustment, giving $O(n^4)$ overall. Maintain a *slack array* $\text{slk}[j] = \min_{i \in S} \text{slack}(i, j)$ for $j \notin T$ and update it incrementally; this is the difference between $O(n^4)$ and $O(n^3)$ (Section 16.7).
3. **Forgetting that $\delta > 0$ strictly.** If a zero-slack frontier edge existed, the tree was not actually stuck – you should have grown it first. Mixing these up loops forever.

Pause

Pause and think: what happens to the dual objective during an adjustment? The sum of potentials changes by $\sum_{i \in S} \delta - \sum_{j \in T} \delta = (|S| - |T|)\delta$. Because the root $r \in S$ is unmatched and every other vertex in S is matched to a vertex in T, we have $|S| = |T| + 1$. Thus the total dual objective strictly increases by $\delta > 0$ at every adjustment, driving the potentials steadily toward the minimum matching cost.

Figure 16.2 shows the dual adjustment. Before the step, the Hungarian alternating tree spans S on the left and T on the right; the frontier edges from S to $R \setminus T$ all carry positive slack, the smallest of which is δ. Raising every u_i for $i \in S$ by δ and lowering every v_j for $j \in T$ by δ keeps every tree edge at zero slack while the frontier slacks drop by δ, so at least one frontier edge enters the equality subgraph.

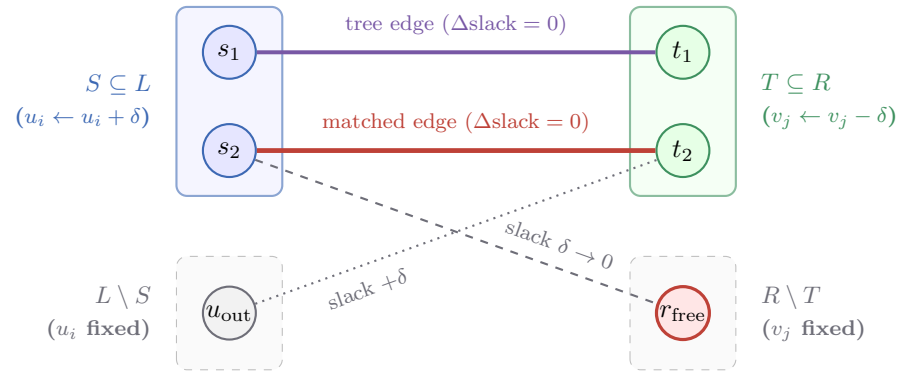


Figure 16.2: Dual adjustment when the Hungarian tree is stuck. When no tight edge crosses from S to $R \setminus T$, potentials are shifted by the minimum frontier slack $\delta = \min_{i \in S, j \notin T}(c(i,j) - u_i - v_j) > 0$. Increasing u_i by δ on S and decreasing v_j by δ on T preserves exact zero slack for all internal tree and matched edges ($S \to T$), while decreasing frontier slacks ($S \to R \setminus T$) by δ. The edge achieving δ becomes tight, expanding the alternating tree without violating feasibility anywhere.

Walking question

Why must δ be finite – why is there always a frontier edge to measure? Because the instance is complete: every $i \in S$ has an edge to every $j \in R \setminus T$, and as long as the matching is not yet perfect there is at least one right vertex outside T (otherwise $T = R$ would already contain a free right vertex, and we would have augmented instead of adjusting). So the minimum in the definition of δ ranges over a nonempty set and is a genuine finite number. On a *sparse* instance you must instead treat missing edges as cost $+\infty$, and then guard against an all-infinite frontier, which signals that no perfect matching exists.

16.4 The Hungarian Algorithm

We give the $O(n^3)$ form directly: match left vertices one at a time, and within each phase maintain the slack array so each phase costs $O(n^2)$. The implementation below uses the standard "virtual root" / column-driven Jonker–Volgenant style augmentation, which is both fast and short. We write matchR$[j]$ for the left vertex matched to right vertex j (0 if free), and reconstruct the path with a parent array prev$[j]$.

Algorithm 67: Hungarian algorithm (min-cost perfect assignment, $O(n^3)$)

Input: $n \times n$ cost matrix c (1-indexed); rows $L = \{1..n\}$, columns $R = \{1..n\}$.
Output: potentials u, v and assignment matchR of minimum total cost.

```
1  u[0..n] ← 0;  v[0..n] ← 0;  matchR[0..n] ← 0
2  for r ← 1 to n do
3      j0 ← 0;  matchR[0] ← r      // column 0 is the virtual root carrying new row r
4      slk[j] ← +∞, prev[j] ← 0 for all j ∈ R;  used[0..n] ← FALSE
5      repeat
6          used[j0] ← TRUE;  i ← matchR[j0];  δ ← +∞;  j1 ← −1
7          for j ∈ R with used[j] = FALSE do
8              cur ← c[i][j] − u[i] − v[j]                  // current slack of edge (i, j)
9              if cur < slk[j] then
10                 slk[j] ← cur;  prev[j] ← j0
11             if slk[j] < δ then
12                 δ ← slk[j];  j1 ← j
13         for j ∈ {0..n} do
14             if used[j] then
15                 u[matchR[j]] += δ;  v[j] −= δ
16             else
17                 slk[j] −= δ
18         j0 ← j1                                  // step to the newly tightened column
19     until matchR[j0] = 0
20     while j0 ≠ 0 do
21         j1 ← prev[j0];  matchR[j0] ← matchR[j1];  j0 ← j1          // flip the path
22 return u, v, matchR
```

Intuition

Read the inner **Repeat** as "grow the Hungarian tree by one column per iteration." The virtual column 0 holds the freshly added row r. Each iteration: scan slacks of the current row i = matchR$[j_0]$ into the slack array, find the minimum residual slack δ over unused columns, push *all* potentials by δ (this is the dual adjustment, applied lazily through the slack array), and step to the newly tightened column j_1. The loop stops the moment it steps onto a *free* column (matchR$[j_1] = 0$); then the **While** walks prev back to the root, flipping matched/unmatched edges – a complete augmentation. Notice potentials are updated *inside* the search, so they stay consistent with the partial tree.

Why it works

Why the lazy potential update is correct. We maintain $\text{slk}[j] = \min_{i \in S} \text{slack}(i, j)$ for $j \notin T$, where S is the set of rows pulled into the tree and T the set of used columns. A dual adjustment of size δ must (i) leave tree columns (used$[j]$, i.e. $j \in T$) tight, and (ii) reduce every outside column's recorded slack by δ. The second inner loop does exactly that: for used columns it shifts u of the matched row up and v down (so the slack of tree edges is unchanged), and for unused columns it decrements slk$[j]$ by δ. Whenever some slk$[j]$ reaches 0, the edge into j has become tight – and that is the j_1 we step to next. This is the Section 16.3 adjustment executed in $O(n)$ per step rather than $O(n^2)$.

16.5 Worked Example, Run by Hand

Take the 3×3 cost matrix (minimize). Rows are workers $1, 2, 3$; columns are jobs A, B, C.

$$C = \begin{array}{c|ccc} & A & B & C \\ \hline 1 & 4 & 1 & 3 \\ 2 & 2 & 0 & 5 \\ 3 & 3 & 2 & 2 \end{array}$$

Trace it yourself

Before running the full algorithm, get the answer by brute force so we have something to check against. The six assignments and their costs:

$$1A, 2B, 3C: \ 4+0+2=6 \qquad 1A, 2C, 3B: \ 4+5+2=11$$
$$1B, 2A, 3C: \ 1+2+2=5 \qquad 1B, 2C, 3A: \ 1+5+3=9$$
$$1C, 2A, 3B: \ 3+2+2=7 \qquad 1C, 2B, 3A: \ 3+0+3=6$$

The minimum is 5, achieved by $1 \to B,\ 2 \to A,\ 3 \to C$. Keep 5 in your pocket.

Now run the Hungarian method in the dual-potential form. It is easiest to track by hand using the classic "reduce then cover" presentation, which is the same algorithm: subtracting the row minimum from a row decreases c by a constant on that row, i.e. raises u_i; subtracting a column minimum raises v_j. Tight edges are the zeros.

Step 1 – row reduction (initialize u_i to the row minimum). Row minima are $u_1 = 1,\ u_2 = 0,\ u_3 = 2$. Subtract:

$$C^{(1)} = \begin{array}{c|ccc} & A & B & C \\ \hline 1 & 3 & \mathbf{0} & 2 \\ 2 & 2 & \mathbf{0} & 5 \\ 3 & 1 & \mathbf{0} & \mathbf{0} \end{array} \qquad u = (1, 0, 2),\ v = (0, 0, 0).$$

Bold zeros are tight edges. Note all three rows put their zero in B – a collision.

Step 2 – column reduction (initialize v_j). Column minima of $C^{(1)}$: $v_A = 1,\ v_B = 0,\ v_C = 0$.

Subtract from column A:

$$C^{(2)} = \begin{array}{c|ccc} & A & B & C \\ \hline 1 & 2 & \mathbf{0} & 2 \\ 2 & 1 & \mathbf{0} & 5 \\ 3 & \mathbf{0} & \mathbf{0} & \mathbf{0} \end{array} \qquad u = (1, 0, 2),\ v = (1, 0, 0).$$

Step 3 – try to match on the zeros (equality subgraph). Tight edges now: $1{-}B$, $2{-}B$, $3{-}A$, $3{-}B$, $3{-}C$. Greedily match: take $3{-}A$, then $1{-}B$. Vertex 2 has only $2{-}B$ tight, but B is taken. Grow a Hungarian tree from free worker 2:

$$S = \{2\}, \quad 2{-}B \text{ tight} \Rightarrow B \in T, \quad B \text{ matched to } 1 \Rightarrow 1 \in S.$$

Now $S = \{1, 2\}$, $T = \{B\}$. Tight edges from S to $R \setminus T = \{A, C\}$? From 1: $1{-}A$ slack 2, $1{-}C$ slack 2. From 2: $2{-}A$ slack 1, $2{-}C$ slack 5. None is tight – the tree is stuck, as shown in Figure 16.3, where every frontier edge out of $S = \{1, 2\}$ carries positive slack and $(2, A)$ achieves the minimum $\delta = 1$.

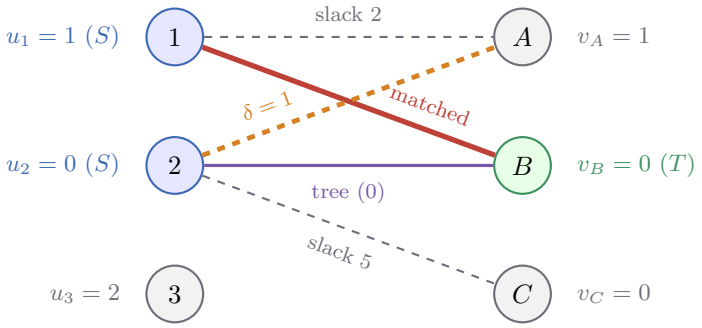


Figure 16.3: Step 3 of the worked example: the Hungarian tree rooted at free worker 2 reaches B via tight edge $(2, B)$ and pulls matched worker 1 into S, stalling at $S = \{1, 2\}$ and $T = \{B\}$. Among the frontier edges from S to $R \setminus T = \{A, C\}$, edge $(2, A)$ has the smallest positive slack $\delta = 1$.

Step 4 – dual adjustment.

$$\delta = \min\{2, 2, 1, 5\} = 1 \quad (\text{edge } 2{-}A).$$

Raise u on $S = \{1, 2\}$ by 1, lower v on $T = \{B\}$ by 1:

$$u = (2, 1, 2), \quad v = (1, -1, 0).$$

Recompute slacks (equivalently: subtract δ from uncovered cells, add it to doubly-covered cells):

$$C^{(3)} = \begin{array}{c|ccc} & A & B & C \\ \hline 1 & 1 & \mathbf{0} & 1 \\ 2 & \mathbf{0} & \mathbf{0} & 4 \\ 3 & \mathbf{0} & 1 & \mathbf{0} \end{array}$$

Edge $2{-}A$ is now tight. The tree extends: $A \in T$ via $2{-}A$; A is matched to 3, so $3 \in S$. From 3, edge $3{-}C$ is tight and C is *free*. Augmenting path found:

$$2 \xrightarrow{\text{free}} A \xrightarrow{\text{match}} 3 \xrightarrow{\text{free}} C.$$

Figure 16.4 contrasts the equality subgraph $G_=$ right after this dual step – showing the newly tightened edge $(2, A)$ completing the augmenting path $2 \to A \to 3 \to C$ – with the optimal perfect matching obtained after flipping that path in Step 5.

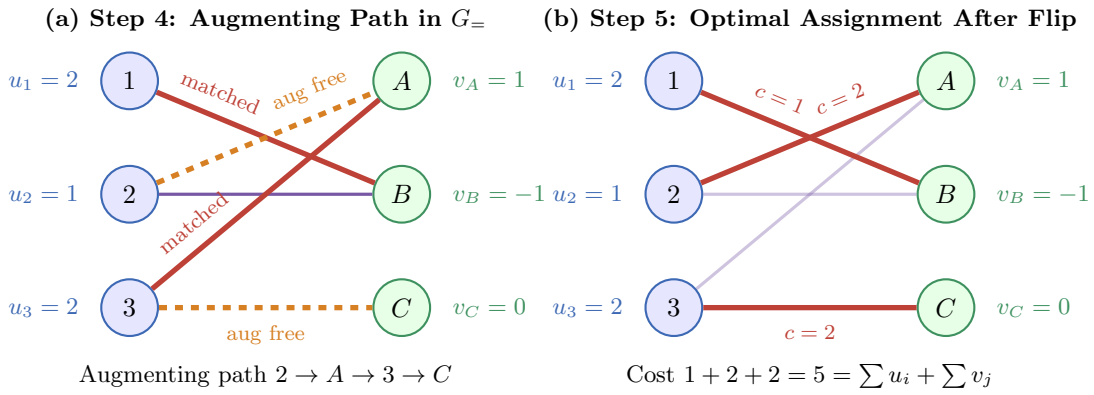


Figure 16.4: Completing the worked example. (a) The equality subgraph after the $\delta = 1$ dual adjustment, with augmenting path $2 \to A \to 3 \to C$. (b) Flipping the path replaces $(3, A)$ with $(2, A)$ and $(3, C)$, yielding the optimal assignment of cost 5.

Step 5 – flip the path. Currently $3-A$ is matched; flipping along $2-A-3-C$ makes $2-A$ and $3-C$ matched and frees A from 3. Keeping $1-B$, the final matching is

$$\boxed{1 \to B, \quad 2 \to A, \quad 3 \to C} \qquad \text{cost } 1 + 2 + 2 = 5.$$

Step 6 – read the certificate. Final potentials $u = (2, 1, 2)$, $v = (1, -1, 0)$. Dual value

$$\sum_i u_i + \sum_j v_j = (2 + 1 + 2) + (1 - 1 + 0) = 5 + 0 = 5,$$

matching the primal cost 5. Weak duality with equality $\Rightarrow$ optimal. Every matched edge is tight: $c(1, B) - u_1 - v_B = 1 - 2 - (-1) = 0$, $c(2, A) - u_2 - v_A = 2 - 1 - 1 = 0$, $c(3, C) - u_3 - v_C = 2 - 2 - 0 = 0$. Complementary slackness holds on the nose.

Trace it yourself

Trace it yourself. Re-run on $\begin{bmatrix} 7 & 5 & 11 \\ 9 & 8 & 5 \\ 4 & 6 & 5 \end{bmatrix}$ (minimize). Predict: row reduction gives $u = (5, 5, 4)$, and here the reduced zeros already land in distinct columns, so no dual step is needed at all. Brute-force the six assignments first, then confirm your final u, v sum to the optimum. (Spoiler: the optimum is 14, via $1 \to B$, $2 \to C$, $3 \to A$.)

16.5.1 A second trace, narrated in full

Before leaving the hand trace, let us run the matrix from the box above to see what happens when row reduction alone already isolates a full transversal of zeros – and why small cost perturbations

leave that transversal intact.

$$C = \begin{array}{c|ccc} & A & B & C \\ \hline 1 & 7 & 5 & 11 \\ 2 & 9 & 8 & 5 \\ 3 & 4 & 6 & 5 \end{array}$$

Row reduction subtracts $u = (5, 5, 4)$:

$$C^{(1)} = \begin{array}{c|ccc} & A & B & C \\ \hline 1 & 2 & \mathbf{0} & 6 \\ 2 & 4 & 3 & \mathbf{0} \\ 3 & \mathbf{0} & 2 & 1 \end{array} \qquad u = (5, 5, 4),\ v = (0, 0, 0).$$

Each column already contains a zero, so column reduction adds nothing: $v = (0, 0, 0)$. The tight edges are $1{-}B$, $2{-}C$, $3{-}A$ – one per row and per column, a perfect matching on the nose. There is no collision, so the algorithm halts immediately with

$$\boxed{1 \to B,\ 2 \to C,\ 3 \to A}, \qquad \text{cost } 5 + 5 + 4 = 14,$$

and the dual value $\sum u_i + \sum v_j = 14 + 0 = 14$ confirms it. (v is all zero here, so the row potentials alone carry the bound.) Here row reduction alone already exposed a perfect matching. Even if we raise $c(2, C)$ from 5 to 7, row 2's minimum simply shifts from 5 to 7:

$$C' = \begin{bmatrix} 7 & 5 & 11 \\ 9 & 8 & 7 \\ 4 & 6 & 5 \end{bmatrix} \xrightarrow{u=(5,7,4)} C'^{(1)} = \begin{array}{c|ccc} & A & B & C \\ \hline 1 & 2 & \mathbf{0} & 6 \\ 2 & 2 & 1 & \mathbf{0} \\ 3 & \mathbf{0} & 2 & 1 \end{array}$$

Now the zeros still land in three distinct columns, so again no dual step is needed – the assignment $1 \to B, 2 \to C, 3 \to A$ stays optimal at cost $5 + 7 + 4 = 16$. The general point: *whenever the reduced matrix has a transversal of zeros, you are already done*, and the dual adjustments exist to create such a transversal when row/column reduction alone does not. The first worked example needed one adjustment; some instances need up to $n - 1$ per phase.

Common trap

Hand-execution pitfalls.

1. **Declaring victory at n zeros.** Having n (or more) zeros is necessary but not sufficient – they must form a *transversal* (no two in the same row or column). The first example had four zeros after step 2 but they shared column B, so no transversal existed yet.
2. **Forgetting to update v negative.** The duals on the equality constraints are free, so $v_j < 0$ is normal and not a sign of error. Clamping v at 0 silently breaks the certificate.
3. **Re-reducing rows mid-run.** After the initial reduction, you never subtract a fresh row minimum again; all further changes come through the structured S/T dual step. A stray re-reduction can break invariant (2).

Pause

Pause and think: why does matrix reduction never change the optimal assignment? Every candidate assignment chooses exactly one entry from each row i and each column j. If you subtract u_i from row i and v_j from column j, the cost of *every* candidate assignment changes by the exact same constant $\sum_i u_i + \sum_j v_j$. So potential shifts do not change the ranking of assignments.

16.6 Correctness

Theorem 16.1 (Optimality of the Hungarian algorithm)**.** *On a square instance with finite costs, the algorithm terminates with a perfect matching π and a feasible potential (u, v) such that $\sum_i c(i, \pi(i)) = \sum_i u_i + \sum_j v_j$. Consequently π is a minimum-cost perfect matching.*

Proof. We verify the two invariants of Section 16.3 are preserved, that each phase terminates, and that the final equality yields optimality.

Feasibility (invariant 1). Initialization sets $u_i = \min_j c(i, j)$ and then $v_j = \min_i(c(i, j) - u_i) \geq 0$ shifts, giving $u_i + v_j \leq c(i, j)$ for all i, j. Each dual adjustment of size $\delta = \min_{i \in S,\, j \notin T} \text{slack}(i, j)$ keeps feasibility by the case analysis in the "Why it works" box of Section 16.3: only S-rows rise and only T-columns fall, and the binding constraints are the frontier $S \times (R \setminus T)$, whose slacks drop by exactly $\delta \geq 0$ and stay nonnegative.

Tight matching (invariant 2). Initially $M = \varnothing$ satisfies it vacuously. Dual adjustments leave all matched edges tight: a matched edge has both endpoints in the tree ($i \in S,\ j \in T$, slack unchanged). Augmentation flips along an alternating path of tight edges, so every new matched edge is tight.

Each phase terminates. A phase grows a tree rooted at the new free vertex r. Each growth step adds a right vertex to T (and, if it is matched, its mate to S); each dual adjustment makes at least one frontier edge tight, enabling a growth step. Since $|T| \leq n$, after at most n adjustments and n growth steps the tree reaches a free right vertex. That such a vertex is reachable is guaranteed by a Kőnig-style counting argument: as long as no free right vertex is reachable, T is exactly the set of mates of $S \setminus \{r\}$, so $|T| = |S| - 1$; the dual step strictly enlarges the equality subgraph on the frontier, and since the instance is complete, S cannot grow past n without exposing a free right vertex. Augmentation then increases $|M|$ by 1.

Overall termination and optimality. There are exactly n phases, each increasing $|M|$ by one, so M becomes perfect. At that point invariant (2) gives $u_i + v_{\pi(i)} = c(i, \pi(i))$ for every matched edge. Summing over the n matched edges,

$$\sum_i c(i, \pi(i)) = \sum_i (u_i + v_{\pi(i)}) = \sum_i u_i + \sum_j v_j.$$

Combined with weak duality (Section 16.2) – the left side is $\geq$ the right for every matching, and the right side is $\leq$ the optimum for every feasible potential – both are optimal: no matching is cheaper and no potential is larger. □

Why it works

The exchange view of optimality, for readers who prefer it to duality. Suppose π is the matching the algorithm returns and some other perfect matching π' is strictly cheaper. The symmetric difference $\pi \oplus \pi'$ is a disjoint union of alternating cycles. Walk around any such cycle: swapping π-edges for π'-edges changes the cost by $\sum(\pi'\text{-edges}) - \sum(\pi\text{-edges})$. Now telescope the potentials around the cycle – each vertex is entered and left once, so the u, v contributions cancel, giving

$$\sum(\pi'\text{-edges}) - \sum(\pi\text{-edges}) = \sum_{e \in \pi' \cap \text{cycle}} \text{slack}(e) - \sum_{e \in \pi \cap \text{cycle}} \text{slack}(e).$$

Every π-edge is tight (slack $= 0$) and every π'-edge has slack ≥ 0, so the right side is ≥ 0: no cycle can decrease the cost. Hence π is optimal. This is min-cost flow's "no negative residual cycle" criterion, specialized to matchings.

16.7 Complexity

Theorem 16.2 (Running time)**.** *The algorithm of Section 16.4 runs in $O(n^3)$ time and $O(n^2)$ space (or $O(n)$ extra space beyond the input matrix).*

Proof. There are n phases (one per left vertex). Within a phase, the inner **Repeat** executes at most n times: each iteration marks a previously-unused column j_0 as used and never unmarks it, so with n columns the loop steps at most n times before hitting a free column. Each iteration does $O(n)$ work – the slack scan over unused columns is $O(n)$, and the potential-update loop over all columns is $O(n)$. So a phase costs $O(n^2)$, and the whole run $O(n \cdot n^2) = O(n^3)$. The path-flip **While** is $O(n)$ per phase, dominated. Space: $u, v, \text{matchR}, \text{slk}, \text{prev}, \text{used}$ are $O(n)$ each, plus the $O(n^2)$ cost matrix. □

Cost summary

Variant	Time
Naive "reduce & cover" (recompute δ each step)	$O(n^4)$
Hungarian with slack array (this chapter)	$O(n^3)$
Min-cost flow, n shortest-path phases (dense)	$O(n(m + n \log n)) = O(n^3)$
Jonker–Volgenant (shortest-augmenting refinements)	$O(n^3)$ worst, faster in practice
Scaling (Gabow–Tarjan), integer costs in $[0, C]$	$O(\sqrt{n}\, m \log(nC))$

Bottom line: $O(n^3)$ is the standard, and is excellent up to a few thousand on each side. For sparse or huge instances, route through min-cost flow (next box) or a scaling algorithm.

Connection

The Hungarian algorithm as a specialization of min-cost flow (Chapter 14). Build a flow network: source $s \to$ each $i \in L$ with capacity 1, cost 0; each edge $i \to j$ with capacity 1, cost $c(i, j)$; each $j \in R \to t$ with capacity 1, cost 0. A min-cost integral flow of value n from s

to t is a min-cost perfect matching. The potentials u, v are precisely the *node potentials* of the successive-shortest-paths algorithm, and the dual adjustment is the reduced-cost reweighting (Johnson's potentials) that keeps every reduced cost nonnegative so Dijkstra applies. Each phase = one shortest augmenting path. The Hungarian algorithm is the bipartite-assignment specialization where the layered structure turns the shortest-path tree into the Hungarian tree, and a Bellman–Ford-free Dijkstra runs in $O(n^2)$ per phase.

16.8 Primal–Dual View and Complementary Slackness

The assignment problem has a natural linear programming formulation that explains where the vertex potentials come from. Let $x_{ij} \in \{0, 1\}$ indicate whether worker i is assigned to job j. The integer programming formulation is:

$$\min \sum_{i,j} c_{ij} x_{ij} \quad \text{s.t.} \quad \sum_{j} x_{ij} = 1 \ (\forall i \in L), \quad \sum_{i} x_{ij} = 1 \ (\forall j \in R), \quad x_{ij} \geq 0.$$

The constraint matrix is the vertex-edge incidence matrix of the complete bipartite graph $K_{n,n}$. Because $K_{n,n}$ is bipartite, this matrix is *totally unimodular* (every square submatrix has determinant in $\{-1, 0, 1\}$). Consequently, the polyhedron defined by the linear relaxation $x_{ij} \geq 0$ has integer vertices, ensuring that an optimal basic feasible solution is automatically an integer matching vector $x \in \{0, 1\}^{n \times n}$. Continuous LP algorithms and combinatorial matching algorithms optimize over the exact same convex hull.

Focus

Primal LP (P)

$$\begin{aligned} \min \ & \textstyle\sum_{ij} c_{ij} x_{ij} \\ & \textstyle\sum_{j} x_{ij} = 1 \quad (u_i) \\ & \textstyle\sum_{i} x_{ij} = 1 \quad (v_j) \\ & x_{ij} \geq 0 \end{aligned}$$

Dual LP (D)

$$\begin{aligned} \max \ & \textstyle\sum_{i} u_i + \sum_{j} v_j \\ & u_i + v_j \leq c_{ij} \qquad (x_{ij}) \\ & u_i, v_j \text{ free} \end{aligned}$$

The dual variables are exactly our potentials, and the dual constraint $u_i + v_j \leq c_{ij}$ is exactly feasibility. The equality-constraint duals are free (unsigned), which is why v_B went negative in the worked example without alarm. LP duality gives *strong* duality: max(D) = min(P). The Hungarian algorithm is a constructive proof of strong duality for this specific LP.

Why it works

Complementary slackness, stated for this pair. A primal-feasible x and dual-feasible (u, v) are simultaneously optimal iff for every edge

$$x_{ij} > 0 \implies u_i + v_j = c_{ij} \qquad (\text{"every used edge is tight"}).$$

(The dual-slack $\times$ primal-slack condition on the constraints is automatic, since the primal constraints are equalities.) The algorithm maintains dual feasibility *always* and primal feasibility only *at the end* (when M is perfect); it engineers the run so that the moment primal feasibility is reached, CS already holds because M uses only tight edges. This is the design

principle of *primal–dual* algorithms: keep one side feasible, drive the other toward feasibility while never violating complementary slackness, and you arrive at a matched optimal pair.

Egerváry's theorem as strong duality. Egerváry's 1931 theorem is precisely strong duality for this LP, proved combinatorially two decades before linear programming had a name. Kőnig's theorem (max matching = min vertex cover in bipartite graphs) is the special case where all costs are 0/1: the potentials become a 0/1 vertex cover. The Hungarian algorithm thus generalizes Kőnig's min-cover / max-matching duality to arbitrary edge weights, with potentials playing the role of a fractional, real-valued cover.

16.9 Applications

Application

Job assignment / personnel scheduling. The textbook use: c_{ij} = time (or cost, or $-$profit) for worker i on task j; minimize the total. Because a perfect matching uses each worker and each task once, the Hungarian algorithm returns the globally cheapest one-to-one assignment in $O(n^3)$. It generalizes to *bottleneck assignment* (minimize the largest edge used) by binary-searching a threshold and testing for a perfect matching among edges below it (Chapter 15).

Application

Tracking and data association. In multi-object tracking, each frame's detections must be matched to existing tracks; c_{ij} is a distance (Mahalanobis, IoU, appearance feature). The Hungarian algorithm produces the maximum-likelihood association each frame, and it is the core of classic trackers (the assignment step inside SORT and Kalman-filter trackers). Gating sets impossible matches to $+\infty$.

Application

Optimal transport, discretely. When two distributions are supported on n points each with equal masses, the optimal-transport (Earth Mover) plan between them is exactly a min-cost *assignment* with c_{ij} = ground distance. The dual potentials u, v are the *Kantorovich potentials*. Sinkhorn's entropic regularization is the fast approximate cousin; the Hungarian algorithm is the exact solver for the unregularized, balanced, equal-mass case.

Application

Frequency / channel assignment, antenna pointing, taxi dispatch, wiring placement. Any setting with two finite sides, a one-to-one constraint, and an additive objective is in scope. When the sides are large but most pairs are forbidden, model as min-cost flow on the sparse graph instead of running dense Hungarian.

Connection

The dual potentials reappear across this book. In Chapter 14 they are min-cost-flow node potentials; in Chapter 35 reduced costs play a voltage-like role; and the LP-duality template here – maintain dual feasibility, enforce complementary slackness – is the same template behind the primal–dual approximation algorithms of Chapter 38. Total unimodularity, the reason the LP is integral, is the bipartite specialization of the integrality you will meet when matchings stop being bipartite (Chapter 17).

Why this matters

Even where faster scaling solvers exist for huge sparse graphs, the Hungarian algorithm remains the clearest setting in which the primal–dual method is completely visible: a dual object you can write on a napkin (one number per vertex), a feasibility rule with a one-line proof, a search that the dual itself steers, and a stopping test that doubles as a certificate. Once you understand it, min-cost flow, weighted matching in general graphs, and primal–dual approximation algorithms are variations on the same method.

Looking ahead: Every technique in this chapter relies on the two-sided geometry of bipartite graphs: alternating paths cannot loop back on themselves in odd cycles, and total unimodularity guarantees integral vertices for free. In general graphs, where any vertex can pair with any other, these guarantees fail. In Chapter 17, odd cycles – which Jack Edmonds called *blossoms* – break the two-coloring of alternating trees, requiring a cycle-contraction step to restore alternating search.

16.10 Exercises

Exercise 16.1 (Quick mental check: initial feasibility)**.** Suppose all edge costs in C are strictly positive ($c_{ij} > 0$). Is the all-zero potential assignment $u_i = 0$ for all $i \in L$ and $v_j = 0$ for all $j \in R$ feasible? Can the Hungarian algorithm begin with these potentials, and if it does, what is the initial equality subgraph?

Exercise 16.2 (Quick mental check: zero-cost shift)**.** If you subtract the row minimum $\min_j c_{ij}$ from every row and then the column minimum from every column, explain in one sentence why every resulting entry is non-negative and at least one entry in each row and column is zero.

Exercise 16.3. Run the Hungarian algorithm by hand on $\left[\begin{smallmatrix} 9 & 11 & 14 \\ 6 & 15 & 13 \\ 12 & 13 & 6 \end{smallmatrix}\right]$ (minimize). Show every row/column reduction, each dual adjustment with its δ and the sets S, T, the augmenting path, and the final u, v. Verify that $\sum u_i + \sum v_j$ equals the matching cost.

Exercise 16.4. Prove that adding a constant to an entire row (or column) of the cost matrix does not change which assignment is optimal, only its value. Use this to explain why initial row/column reduction is legitimate, and why max-weight and min-cost assignment are interconvertible.

Exercise 16.5. (Break it.) Construct a 3×3 instance where the greedy rule "repeatedly pick the cheapest unused cell whose row and column are still free" returns a strictly suboptimal assignment. Quantify the gap, and identify the alternating cycle the Hungarian algorithm uses to repair greedy's mistake.

Exercise 16.6. Implement the $O(n^3)$ algorithm of Section 16.4. Test it against a brute-force $O(n!)$ checker for all $n \le 8$ on random integer matrices, asserting that (a) the costs agree and (b) your returned (u, v) are feasible and sum to the cost. Then stress-test $n = 500$ and report the runtime.

Exercise 16.7. Show how to handle a *rectangular* instance with $|L| = p < |R| = q$ (assign every left vertex, each right vertex at most once) by padding to a square. Prove your padding does not change the optimal assignment of the real vertices. What costs do the dummy rows need?

Exercise 16.8. (Min-cost flow reduction.) Write out the flow network of the *Connection* box in Section 16.7 for the worked example of Section 16.5. Run successive-shortest-paths with Johnson potentials and check that the node potentials you obtain match u, v up to the standard sign/offset convention.

Exercise 16.9. (Total unimodularity.) Prove that the constraint matrix of the assignment LP is totally unimodular. (Hint: every square submatrix can be expanded along a row or column with at most two ± 1 entries; induct, using bipartiteness to two-colour the rows.) Conclude that the LP relaxation has an integral optimum.

Exercise 16.10. (Sensitivity.) After solving an instance optimally with potentials (u, v), suppose one cost $c_{i^\star j^\star}$ decreases by $t > 0$. Give an $O(n^2)$ procedure to update the optimal matching (hint: at most one augmenting search is needed). For which t does the current matching stay optimal? Express the threshold using the slack of $(i^\star, j^\star)$.

Exercise 16.11. (Bottleneck assignment.) You want the perfect matching minimizing the *maximum* edge cost used, not the sum. Give an $O(n^2 \log n)$ algorithm by binary-searching the threshold and testing for a perfect matching among edges below it (using unweighted bipartite matching, Chapter 15). Prove correctness.

Exercise 16.12. (Stability vs. optimality.) Contrast min-cost assignment with a *stable* matching (Gale–Shapley) on preference-derived costs. Give a small instance whose min-cost matching is not stable, and one whose stable matching is not min-cost. Explain in one paragraph why the two objectives optimize different things.

Exercise 16.13. (Counting dual steps.) For the 3×3 worked example of Section 16.5, exactly one dual adjustment occurred. Construct, for each $k \in \{0, 1, 2\}$, a 3×3 instance whose run uses exactly k dual adjustments in total across all phases. Then argue that on an $n \times n$ instance the total number of dual adjustments is at most $n(n-1)/2$, and exhibit a family approaching that bound.

Exercise 16.14. (Certificate checking.) You are handed a claimed solution: an assignment π and potentials (u, v). Write the $O(n^2)$ checker that decides whether the pair certifies optimality, and state precisely the three conditions it must verify (feasibility, tightness of matched edges, value agreement). Explain why a verifier needs no access to the algorithm that produced the answer – this is the practical payoff of carrying a dual.

Chapter 17

General Matching and Edmonds' Blossom Algorithm

A matching in a graph is a set of edges, no two of which share a vertex. Every reader of the previous two chapters knows how to find a maximum matching in a *bipartite* graph: orient the edges, push flow, or chase augmenting paths along the two-sided structure. The natural question is whether the same methods work on an arbitrary graph. They almost do. The augmenting-path idea still works; what breaks is the search that finds those paths. A single odd cycle is enough to mislead the breadth-first search that works on bipartite graphs, and this chapter shows how to repair that failure.

The repair is Jack Edmonds' 1965 *blossom* algorithm, which detects odd cycles during alternating search and contracts them into single nodes to reveal hidden augmenting paths. We trace the contraction and lifting mechanics, prove Berge's theorem and the Tutte–Berge duality formula, and conclude with Edmonds' primal–dual weighted matching.

17.1 Matchings, Augmenting Paths, and Berge's Theorem

Fix an undirected graph $G = (V, E)$ with $n = |V|$ and $m = |E|$. We make no bipartiteness assumption.

Definition 17.1 (Matching)**.** A *matching* $M \subseteq E$ is a set of edges with no shared endpoints. A vertex incident to an edge of M is *matched* (or *covered*); otherwise it is *free* (or *exposed*). M is *maximum* if no matching has more edges, and *perfect* if it covers every vertex.

Maximum and *maximal* are different words. A maximal matching is one you cannot enlarge by adding a single edge; a maximum matching is globally largest. A path graph $a-b-c-d$ has the maximal matching $\{bc\}$ of size one, but the maximum matching $\{ab, cd\}$ has size two. The greedy "add any edge that fits" procedure gives a maximal matching, and it can be off by a factor of two. We want maximum.

Intuition

Think of a matching as a set of committed monogamous pairs. To grow it, we want to find a free person, lead a chain of swaps through the currently-paired people, and arrive at another free person. If such a chain exists, we can re-pair everybody along it and net one extra couple. That chain is an augmenting path, and most of matching theory is about finding one or proving that none exists.

Definition 17.2 (Alternating and augmenting paths)**.** Given a matching M, a path is *M-alternating* if its edges alternate between $E \setminus M$ and M. An alternating path is *M-augmenting* if it is *simple* and both endpoints are free.

An augmenting path necessarily has odd length: it starts and ends with a non-matching edge, so the pattern is $\notin M, \in M, \notin M, \ldots, \notin M$. If P is augmenting, flipping every edge of P (matched becomes unmatched and vice versa) produces a matching $M \oplus P$ – the symmetric difference – with exactly one more edge. The two endpoints become matched and every interior vertex stays matched, just to a different partner.

Theorem 17.1 (Berge, 1957)**.** *A matching M is maximum if and only if G contains no M-augmenting path.*

Why it works

The forward direction is immediate: an augmenting path lets us build a strictly larger matching, so a maximum M admits none. The reverse direction is the substantive one, and its proof uses a fact about the symmetric difference of two matchings that we will reuse constantly.

Proof. ($\Rightarrow$) If P is M-augmenting then $|M \oplus P| = |M| + 1$, so M is not maximum. Contrapositive gives the claim.

($\Leftarrow$) Suppose M is not maximum; let $M^\star$ be a larger matching, so $|M^\star| > |M|$. Consider the edge set $H = M \oplus M^\star$ and the subgraph it spans, illustrated in Figure 17.1. Every vertex has degree at most one in M and at most one in $M^\star$, hence degree at most two in H. A graph of maximum degree two is a disjoint union of simple paths and cycles. Along any such component the edges alternate between M and $M^\star$ (two M-edges at a vertex would violate the matching property, likewise two $M^\star$-edges). Therefore every cycle has even length and contains equally many M- and $M^\star$-edges. Since $M^\star$ contributes more edges overall than M, some path component must contain more $M^\star$-edges than M-edges. Such a path begins and ends with an $M^\star$-edge; its endpoints are thus free in M (they are covered in $M^\star$, not in M). That path is alternating with respect to M and has both ends free: an M-augmenting path. Contradiction. $\square$

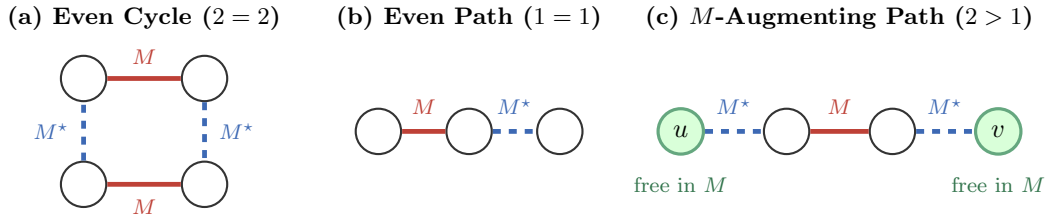


Figure 17.1: Components of the symmetric difference $H = M \oplus M^\star$ when $|M^\star| > |M|$. Every vertex has degree at most 2, so H decomposes into (a) even alternating cycles and (b) even-length paths where $|M| = |M^\star|$, plus at least one (c) odd-length path with one extra $M^\star$-edge, whose endpoints u, v are free in M – an M-augmenting path.

Berge's theorem reduces "find a maximum matching" to the loop:

while an augmenting path exists, find one and flip it.

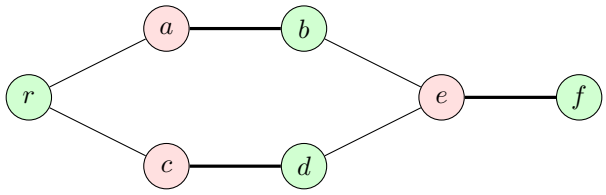
Each flip raises $|M|$ by one, so the loop runs at most $n/2$ times. Everything now hinges on a single subroutine: FindAugmentingPath. In a bipartite graph that subroutine is an ordinary search. In a general graph, finding that path requires handling odd cycles.

History

Claude Berge proved in 1957 that augmenting paths characterize maximum matchings in any graph, but left open how to find them efficiently when odd cycles break bipartiteness. Jack Edmonds solved this in 1965 with his blossom algorithm, introducing the idea of contracting odd cycles into pseudovertices. Edmonds's paper, "Paths, Trees, and Flowers," faced initial skepticism from referees for spending space arguing that polynomial time ($O(n^c)$) represents the true dividing line between practical and intractable algorithms; his definition laid the foundation for computational complexity theory years before Cook and Karp formalized NP-completeness. Later, Harold Gabow (1976) used union-find trees to implement blossom contraction in $O(m\alpha(m, n))$ per phase, while Silvio Micali and Vijay Vazirani (1980) achieved the $O(m\sqrt{n})$ bound by synchronizing search levels across phases. Modern research continues to explore whether general matching can achieve truly linear time $O(m)$ or whether algebraic methods based on randomized matrix determinants can match blossom-based combinatorial bounds in practice.

17.2 Why Odd Cycles Break the Bipartite Search

To find an augmenting path we grow a search tree from a free vertex, alternating as we go. Root the tree at a free vertex r. Label r *even* (an even number of edges from the root). From an even vertex we step along a *non-matching* edge to a neighbour and label it *odd*; from an odd vertex we step along the unique *matching* edge to its partner and label it *even*. An even-labelled vertex is reached by an even-length alternating path from r, so if we ever reach a *free* vertex with an even label we have closed an augmenting path. Every augmenting-path matching algorithm uses this search; in the literature the search tree is called an *alternating tree*.



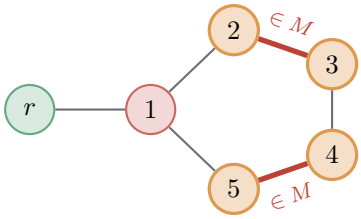
Green discs are even vertices, red discs are odd, thick edges are matched. So far the tree behaves itself: from the root r we reached b and d (even) along matched edges through a, c (odd), and the search would continue from b and d.

Now look at what happens at e. From even b we step along the non-matching edge $b-e$ and want to call e odd. But from even d we also reach e along the non-matching edge $d-e$, and that call also wants e odd. The two even vertices b and d are joined (through e) by an even-length alternating path inside the tree plus the edges be and de – together with e they form an odd closed walk b–e–d–$\cdots$–b. The vertex e can be labelled *both* even and odd depending on which root-path you take to it. This ambiguity is what makes general matching hard.

Intuition

In a bipartite graph this never happens. Bipartite graphs are exactly the graphs with no odd cycle, so the BFS layering into even and odd is consistent – a vertex's parity is determined by the graph, not by the path you took. The moment an *odd* cycle appears, a vertex can be reached by both an even and an odd alternating path, and the even/odd labelling is no longer well defined.

A standard example of this ambiguity is a five-cycle carrying a near-perfect matching, with one free vertex on the rim and a free pendant attached to it, as shown in Figure 17.2.



Rim vertices $2, 3, 4, 5$ possess *both* even and odd parity
depending on whether the cycle is traversed clockwise or counterclockwise

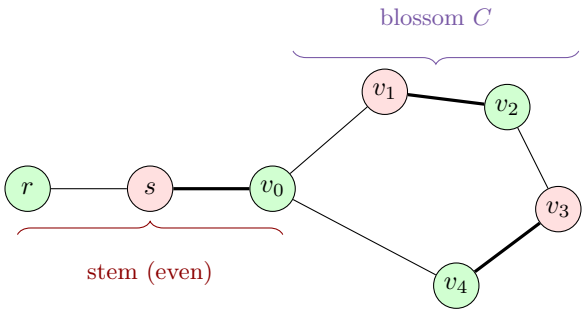
Figure 17.2: The parity ambiguity on an odd cycle. Starting an alternating breadth-first tree from free root r, vertex 1 is reached via unmatched edge $r1$ and labelled odd. Traversing clockwise, 2 is even and 3 is odd. But traversing counterclockwise, 5 is even and 4 is odd. Because the cycle length is odd (5), each rim vertex can be reached by both an even-length and an odd-length alternating path, defeating standard bipartite BFS level assignments.

Here $M = \{23, 45\}$, with vertices 1 and r free. Grow the tree from r: r is even, 1 is odd via the stem $r1$, but 1 is also reachable as an *even* vertex by walking the cycle: 1 2 3 makes 1 even, 2 odd, 3 even; and 1 5 4 makes 1 even, 5 odd, 4 even. The five-cycle relabels its rim vertices with both parities. A naive search that commits to the first label it assigns will either loop or miss the augmenting path that runs through the cycle. The cure is to recognize the odd cycle and collapse it.

17.3 Blossoms, Stems, and Contraction

Definition 17.3 (Blossom)**.** Let M be a matching and let the alternating-tree search be in progress. A *blossom* is an odd-length cycle $C = v_0 v_1 \cdots v_{2k} v_0$ with exactly k matched edges, in which the vertex v_0 (the *base*) is the unique vertex not matched *inside* C. Equivalently, C has $2k+1$ edges, k of them in M, and every vertex except the base is matched along C.

The base v_0 is either free or matched by an edge leaving C (the matched edge that connects the blossom to the rest of the tree). The alternating path from the root r down to the base is the *stem*; it has even length and ends with a matched edge into v_0 (or is empty if $v_0 = r$). Stem plus blossom is a *flower*.



A blossom here is the five-cycle $v_0 v_1 v_2 v_3 v_4 v_0$ with base v_0, matched edges $v_1 v_2$ and $v_3 v_4$, and base reached from r by the stem $r - s - v_0$ (matched edge $s v_0$).

The key fact is that *every* vertex of a blossom is reachable from the base by an alternating path of *even* length that ends with a non-matching edge. Because you can go around the odd cycle in two directions, each non-base vertex can be reached with both parities. This is the ambiguity that broke the search; inside a blossom it is useful, because it means we can splice into the cycle from either side.

Pause

Pause and think: why does an odd cycle give every vertex an even alternating path? Walk along the cycle in both directions from the base v_0. Since the cycle has odd length $2k+1$ and contains k disjoint matching edges, exactly one vertex (v_0) is matched outside the cycle. For any other vertex v, walking clockwise gives an alternating path of some length; walking counter-clockwise visits the complementary cycle arc, whose length has opposite parity. One of those two paths has even length, starting at v_0 with an unmatched edge and ending with a matched edge into v.

Lemma 17.1 (Even reachability inside a blossom)**.** *Let C be a blossom with base v_0. For every vertex $u \in C$ there is an M-alternating path inside C from v_0 to u of even length (possibly length 0 for $u = v_0$) whose first edge (the one leaving the base) is unmatched and whose final edge (the one entering u) is matched whenever $u \neq v_0$.*

Proof. Index the cycle $v_0 v_1 \cdots v_{2k} v_0$ so that the matched edges are $v_1 v_2, v_3 v_4, \ldots, v_{2k-1} v_{2k}$. Going *clockwise* $v_0 v_1 v_2 \cdots v_i$ the edges alternate starting with the unmatched edge $v_0 v_1$; this path has

length i. Going *counterclockwise* $v_0 v_{2k} v_{2k-1} \cdots v_i$ starts with the unmatched edge $v_0 v_{2k}$ and has length $2k+1-i$. The two lengths i and $2k+1-i$ have opposite parities (their sum is the odd number $2k+1$), so exactly one of the two routes reaches v_i in an even number of steps. Take that route; by construction it starts at the base with an unmatched edge and, being alternating of even length, ends with a matched edge into v_i (an even-length path that opens with an unmatched edge closes with a matched one). □

17.3.1 Contraction

Edmonds' insight was that when the search discovers a blossom C, it should *contract* it. Replace all vertices of C by a single super-vertex B. Every edge from a vertex outside C to a vertex inside C now attaches to B (parallel edges are allowed and harmless); edges inside C vanish. Call the smaller graph G/B and let M/B be the matching with the blossom's internal edges removed – the k internal matched edges disappear, while the base's external matched edge (if any) survives and now matches the super-vertex B. The super-vertex is matched iff the base was matched, and it inherits the base's even label in the tree.

Figure 17.3 illustrates contraction and path lifting.

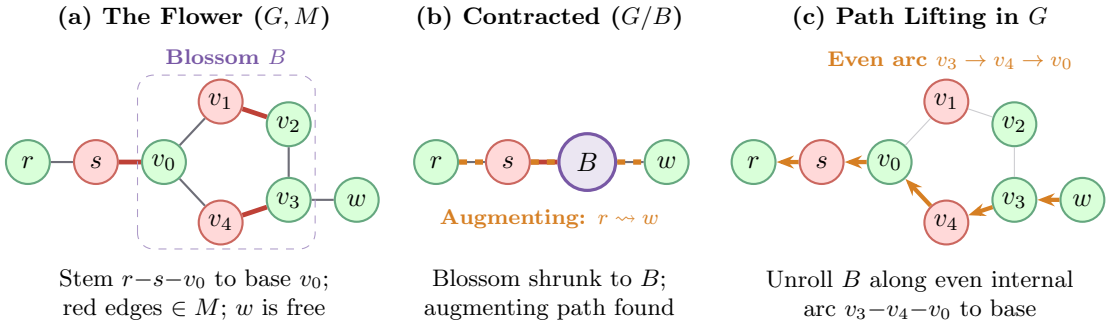


Figure 17.3: Blossom contraction and path lifting. (a) A flower consists of an alternating stem from free root r to base v_0, and an odd cycle B with near-perfect matching. An external free vertex w connects to v_3. (b) Contracting blossom B into a single pseudo-vertex collapses all internal cycle edges. In the reduced graph G/B, the alternating search finds the augmenting path $r - s - B - w$. (c) Lifting the path back to G: upon entering the blossom at v_3, the path routes around the odd cycle via the even alternating path $v_3 - v_4 - v_0$ guaranteed by Lemma 17.1, exiting at base v_0 and continuing along the stem to free root r.

Focus

Contraction turns the odd cycle into a single even-labelled vertex. The labelling ambiguity that broke the search is gone because there is no longer a cycle to walk two ways around. We run the ordinary alternating search on the smaller graph; if it finds an augmenting path through B, we *expand* the blossom and re-route the path through whichever side of the cycle restores alternation. Lemma 17.1 is exactly what guarantees a valid re-route exists.

Correctness of the algorithm rests on the next theorem.

Theorem 17.2 (Blossom contraction is matching-preserving)**.** *Let C be a blossom for matching M with the base reachable from a free root by an even alternating stem. Then G has an M-augmenting*

path if and only if G/C *has an* (M/C)*-augmenting path. In addition, an augmenting path in* G/C *lifts to one in* G *in linear time.*

Why it works

Informally, *a blossom may as well be a single free-or-matched super-vertex.* Any augmenting path can be made to enter and leave the blossom only at its base, and Lemma 17.1 fills in the missing arc inside the cycle. The proof handles the two directions separately; the lift direction is the constructive one used by the implementation.

Proof. ($\Leftarrow$, the lift) Let P' be an (M/C)-augmenting path in G/C. If P' avoids the super-vertex c, it is already an M-augmenting path in G (none of its edges or vertices were touched by contraction). So suppose P' passes through c. The base v_0 is matched in M exactly when c is matched in M/C, so the matched edge of P' at c (if any) is the base's external matched edge $e^\star = v_0w$. Walk P' from one free endpoint; let it reach c along an edge xv_i that, before contraction, hit the blossom at the specific vertex v_i. There are two cases.

If c is matched in P', then P' enters c along the unmatched edge xv_i and leaves along the matched edge $e^\star = v_0w$. Replace the visit to c by $\dots x\, v_i \rightsquigarrow v_0\, w \dots$, where $v_i \rightsquigarrow v_0$ is the even alternating path inside C from v_i to the base given by Lemma 17.1, read backwards. Its first edge xv_i is unmatched, the internal path is alternating and of even length, then $e^\star$ continues as a matched edge. Alternation holds throughout.

If c is free in P' (so c is an endpoint of P', since augmenting paths have free endpoints), then P' enters c along an unmatched edge xv_i and stops. Replace the endpoint c by the even internal path $v_i \rightsquigarrow v_0$; its far end v_0 is free in M (the base is free because c was free), so the lifted path ends at a free vertex. Alternation again holds. Either way the lifted path is simple (the internal arc visits distinct blossom vertices) and M-augmenting.

($\Rightarrow$) Suppose G has an M-augmenting path P. If P misses C entirely it survives contraction. Otherwise let P meet C. Among all augmenting paths choose one meeting C in the fewest vertices; standard surgery (replace any returning excursion by the shorter internal arc, again via Lemma 17.1) shows P can be taken to enter C, possibly traverse part of it, and exit, with all of that collapsing to a single pass through the base. Contracting that pass to the super-vertex c yields an alternating path P' in G/C whose endpoints are the (unchanged) free endpoints of P, except that if an endpoint lay inside C then the base was free and c becomes the free endpoint. In all cases P' is (M/C)-augmenting. A counting view confirms it without the surgery: a maximum matching of G uses exactly k edges inside any factor-critical blossom that it covers, so $\nu(G) = \nu(G/C) + k$ and likewise $|M| = |M/C| + k$; hence G has a larger matching than M iff G/C has a larger matching than M/C, which by Berge (Theorem 17.1) is the augmenting-path statement. □

Common trap

A frequent bug is contracting a cycle that is *not* a blossom – for instance an even cycle, or an odd cycle whose base is not reachable by an even stem. Theorem 17.2 needs *both* conditions: an odd cycle with a single base unmatched inside it, *and* an even alternating stem from a free root. Contracting an arbitrary odd cycle can destroy augmenting paths. The search structure guarantees these conditions automatically: a blossom is detected only when two even-labelled tree vertices are joined by a non-matching edge, and their lowest common ancestor in the

alternating tree is the base.

17.4 The Blossom Algorithm

We now assemble the pieces. The outer loop is Berge's: repeatedly search for an augmenting path from some free vertex and flip it. The inner search grows an alternating tree, contracting blossoms as it meets them, until it either finds an augmenting path (success) or exhausts the tree (this free vertex can never be matched in a larger matching, by Berge restricted to its tree).

We describe the classical $O(V^3)$ version. Each vertex carries a label in $\{\text{EVEN}, \text{ODD}, \text{UNLABELLED}\}$, a tree parent, and a $\mathsf{base}[\cdot]$ pointer giving the base of the (possibly contracted) blossom it currently belongs to – implemented with a union-find-like structure but, in the simple cubic version, with a plain array recomputed per contraction.

Algorithm 68: FindAugmentingPath(G, M, r) – alternating search with blossom contraction

Input: graph $G = (V, E)$, matching M, free root r
Output: an M-augmenting path from r, or none

```
label[v] ← UNLABELLED, parent[v] ← nil, base[v] ← v for all v
label[r] ← EVEN; enqueue r in Q
while Q nonempty do
    u ← dequeue(Q)                                              // u is EVEN
    foreach edge uv ∈ E do
        if base[u] = base[v] then
            continue                                            // edge inside a blossom
        if label[v] = UNLABELLED and v is free then
            return reconstruct path r ⇝ u, v                    // augmenting path found
        else if label[v] = UNLABELLED then
            label[v] ← ODD; parent[v] ← u
            w ← mate[v]; label[w] ← EVEN; enqueue w
        else if label[base[v]] = EVEN then
            b ← LCA(base[u], base[v]) in the alternating tree
            Contract(u, v, b): relabel every ODD vertex on both tree paths up to b as EVEN,
              enqueue them, set base[·] ← b along the cycle
return none
```

Algorithm 69: MaximumMatching(G) – Edmonds' outer loop

```
M ← ∅                                          // or any greedy matching, as a warm start
foreach free vertex r ∈ V do
    P ← FindAugmentingPath(G, M, r)
    if P ≠ none then
        M ← M ⊕ P                              // flip the path; |M| increases by one
return M
```

A few implementation remarks. The CONTRACT step does not physically merge vertices; it marks all vertices around the discovered odd cycle as belonging to base b (via $\mathsf{base}[\cdot]$) and flips every ODD vertex on the two tree-paths from u and v up to b into EVEN, enqueuing it. That relabelling is the algorithmic content of "contraction": once a vertex is even, the search may continue from it. The LCA on the alternating tree finds the base b as the lowest common ancestor of u and v; walking both parent-chains and marking gives the cycle in $O(n)$. Reconstructing the final augmenting path walks parent pointers and, whenever it crosses a contracted vertex, splices in the even internal arc from Lemma 17.1.

Why it works

Why does the search terminate with the right answer for the root r? The alternating tree it grows is *maximal*: every even vertex has had all its edges scanned, and every reachable vertex has been absorbed (as odd, or swallowed into a blossom and made even). If no free vertex was reached from an even vertex, then there is no augmenting path *from* r. By the lifting direction of Theorem 17.2, augmenting paths are preserved under all the contractions performed, so "no augmenting path from r in the contracted graph" implies "no augmenting path from r in G." Combined over all free roots and with Berge's theorem, the returned M is maximum.

17.4.1 A subtle point: why we may fix the root

In the bipartite world one runs a single multi-source search. Here we re-root for each free vertex. The reason is the parity asymmetry: EVEN/ODD labels are relative to a particular free root, and a blossom's validity (Theorem 17.2) depends on the even stem from *that* root. There exist multi-source variants – the Micali–Vazirani algorithm runs all searches in lockstep – but the simplest correctness argument fixes one root at a time. Once a vertex becomes matched it never becomes free again (flips only swap partners along augmenting paths), so a vertex that fails to be matched from its own root will never be matchable later; we need not revisit it. That is why the outer loop is a single pass over free vertices, not a fixpoint.

17.5 A Worked Trace

To see a blossom *form*, we need an even–even collision: a non-matching edge between two vertices that the search has already labelled even. A suitable instance has a triangle that the search reaches from a free root through an even stem. Take vertices $\{1, 2, 3, 4, 5, 6, 7\}$ and edges

$$E = \{12,\ 23,\ 34,\ 45,\ 53,\ 46,\ 67\},$$

with warm-start matching $M = \{23,\ 45,\ 67\}$. Vertex 1 is the only free vertex; all of $2, 3, 4, 5, 6, 7$ are matched. The triangle $3\,4\,5$ carries the matched edge 45 and will become a blossom with base 3. Search from root $r = 1$.

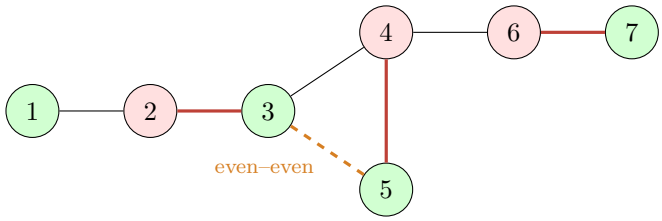


Figure 17.4: The 7-vertex running graph with matching $M = \{23, 45, 67\}$ and free root 1. Scanning from even vertex 3 labels 4 odd and its mate 5 even; scanning edge 35 then reveals an even–even collision between 3 and 5.

Trace it yourself

Keep three columns as you read: the queue Q, the label of each vertex, and the base array. Update them line by line. The point is to see the blossom appear as a forced double-even collision and watching vertex 4 flip from odd to even.

We have $\mathsf{mate} = \{2\to3, 3\to2, 4\to5, 5\to4, 6\to7, 7\to6\}$.

- **Init.** $\mathsf{label}[1] = \textsc{even}$, all else unlabelled, $\mathsf{base} = \mathrm{id}$, $Q = [1]$.
- **Dequeue 1 (even).** Edge 12: 2 unlabelled and matched, so label 2 ODD (parent 1); mate 3 becomes EVEN, enqueue. $Q = [3]$.
- **Dequeue 3 (even).** Edge 32: 2 is odd, skip. Edge 34: 4 unlabelled, matched, so label 4 ODD (parent 3); mate 5 becomes EVEN, enqueue. Edge 35: now $\mathsf{base}[5]$ is EVEN and 3 is EVEN – an **even–even collision on the non-matching edge** 35. A blossom is found.
- **Contract.** LCA of $\mathsf{base}[3] = 3$ and $\mathsf{base}[5] = 5$ in the tree: the tree path of 5 is $5{-}4{-}3$ and of 3 is just 3, so the base is $b = 3$. The odd cycle is $3{-}4{-}5{-}3$ (edges $34 \notin M$, $45 \in M$, $53 \notin M$): a triangle with one matched edge 45, base 3. Relabel the ODD vertex 4 on the cycle to EVEN and enqueue it; set $\mathsf{base}[3] = \mathsf{base}[4] = \mathsf{base}[5] = 3$. Queue now $Q = [5, 4]$.
- **Dequeue 5 (even).** Edges 54 and 53 both have $\mathsf{base} = 3 = \mathsf{base}[5]$, inside the blossom, skip. Nothing new.
- **Dequeue 4 (now even).** Edges 43, 45 inside blossom, skip. Edge 46: 6 unlabelled, matched, so label 6 ODD (parent 4); mate 7 becomes EVEN, enqueue. $Q = [7]$.
- **Dequeue 7 (even).** Edge 76: 6 odd, skip. No other edges. Q empties.

The search ends with no augmenting path from 1, and that is the correct answer: $M = \{23, 45, 67\}$ already has size 3, the graph has 7 vertices (odd), so one vertex must stay exposed and M is maximum. The point of the trace is the even–even collision at edge 35 and the relabelling of 4 to even. Without the blossom move, vertex 4 would have stayed ODD, its edge 46 would never be scanned from an even endpoint, and the search would have stalled with a wrong claim. The contraction let the search continue from the far side of the triangle and confirm that 6 and 7 offer no escape.

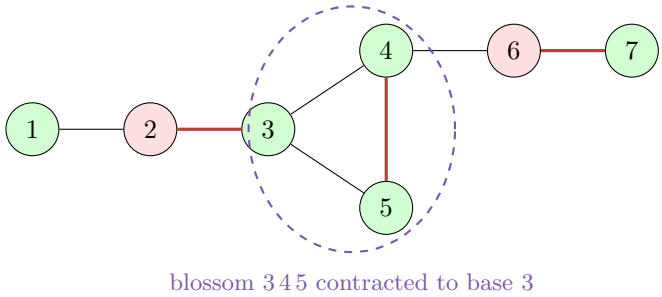


Figure 17.5: Blossom contraction in the worked trace. Triangle $3-4-5$ is contracted into base 3; vertex 4 is relabelled even, allowing the search to continue scanning its external edge 46.

To see an augmenting path *found* through a blossom, change the warm start by dropping the matched edge 67, so $M = \{23, 45\}$ and vertices 1, 6, and 7 are free. The search proceeds identically up to the contraction; then, when vertex 4 (now even) is dequeued, scanning edge 46 finds 6 unlabelled and *free*, so we immediately report an augmenting path. Reconstruct through the contracted blossom: from 6 back to 4, then the even internal arc inside the triangle from 4 to the base 3 (namely $4-5-3$ via Lemma 17.1, the route of even length), then 3's mate 2, then 2's parent 1. The lifted path is

$$1 \;-\; 2 \;-\; 3 \;-\; 5 \;-\; 4 \;-\; 6,$$

alternating $12 \notin M$, $23 \in M$, $35 \notin M$, $54 \in M$, $46 \notin M$, free ends 1 and 6. Flipping it gives $M = \{12, 35, 46\}$ of size 3, leaving only vertex 7 exposed – a maximum matching for this 7-vertex graph.

(a) Lifted Augmenting Path $1{-}2{-}3{-}5{-}4{-}6$ **(b) Augmented Matching** $M = \{12, 35, 46\}$

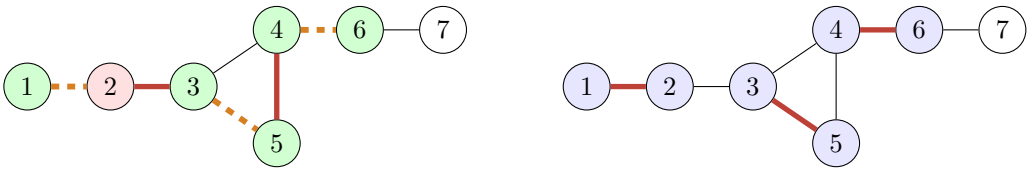


Figure 17.6: Completing the modified trace. (a) Opening the blossom routes around the even internal segment $3-5-4$, linking free root 1 to free vertex 6 via $1-2-3-5-4-6$ (alternating between dashed orange non-matching edges and solid red matching edges). (b) Flipping the path yields the maximum matching $M = \{12, 35, 46\}$ of size 3, leaving only vertex 7 exposed.

The thick orange track in Figure 17.6 is the lifted augmenting path; it enters the triangle at the base 3, swings around the far side $3{-}5{-}4$ (the even arc of Lemma 17.1), and exits at 4 toward the now-free vertex 6. Vertex 7 remains the single exposed vertex.

17.6 Complexity

Theorem 17.3 (Cubic bound)**.** *Algorithm 69 computes a maximum matching in $O(V^3)$ time.*

Proof. The outer loop performs at most $O(V)$ augmentations (each raises $|M|$ by one, capped at $\lfloor n/2 \rfloor$), plus at most $O(V)$ failed searches. Each call to FindAugmentingPath runs one alternating

search. Within a search, every vertex is labelled at most $O(1)$ times *except* for blossom relabellings; each contraction relabels at most $O(n)$ vertices and recomputes an $O(n)$ LCA. The number of contractions per search is at most $O(n)$ (each strictly reduces the number of distinct bases by at least one, and bases never split during a single search). So the work per search is dominated by scanning edges, $O(m)$, plus $O(n)$ contractions at $O(n)$ each, giving $O(m + n^2) = O(n^2)$ per search. Over $O(n)$ searches that is $O(n^3)$. □

Cost summary

Plain Edmonds (this chapter). $O(V^3)$ time, $O(V+E)$ space. Each of $O(V)$ augmentations costs $O(V^2)$ for the search-with-contractions.
Gabow's refinement. With a clever union-find for the base pointers, a single search is $O(E\,\alpha(E,V))$, giving $O(VE\,\alpha)$ overall.
Micali–Vazirani (1980). Phased search in the style of Hopcroft–Karp finds a maximal set of shortest augmenting paths per phase, yielding $O(E\sqrt{V})$ – the best known for cardinality general matching. Its correctness proof (blossoms nested by phase) is famously intricate.
Weighted general matching. Edmonds' primal–dual algorithm runs in $O(V^3)$; Galil–Micali –Gabow improved the constants and the asymptotics for sparse graphs.

The recurring $O(V^3)$ bound. The $O(V^3)$ bound matches the Hungarian algorithm for weighted bipartite matching, matches Gabow's implementation of weighted general matching, and arises naturally from running $O(V)$ augmentation searches that each take $O(V^2)$ work on dense alternating trees. Gabow's 1976 work and his later $O(VE\log V)$ and $O(V(E + V\log V))$ scaling algorithms refined the weighted case, while Micali and Vazirani reached the $O(E\sqrt{V})$ bound for the cardinality case, matching what Hopcroft and Karp had done for bipartite graphs.

17.7 The Tutte–Berge Formula

Berge's theorem certifies maximality by the *absence* of an augmenting path – a negative condition. The Tutte–Berge formula turns it into a min-max identity, the general-graph analogue of Kőnig's theorem. It says exactly how large a maximum matching is, and it gives a short *certificate* of optimality you can hand to a skeptic.

For a vertex set $S \subseteq V$, write $o(G - S)$ for the number of *odd* components (connected components with an odd number of vertices) of the graph obtained by deleting S. Write $\nu(G)$ for the size of a maximum matching.

Theorem 17.4 (Tutte–Berge formula)**.** *The maximum matching of $G = (V, E)$ has size*

$$\nu(G) \;=\; \frac{1}{2}\left(n \;-\; \max_{S \subseteq V} \big(o(G - S) - |S| \big) \right).$$

Equivalently, the number of vertices left exposed by a maximum matching is $\max_{S} \big(o(G - S) - |S| \big)$.

Intuition

Delete a set S of vertices. Each odd component of what remains must leave at least one of its vertices unmatched *unless* that vertex is matched across into S. But S has only $|S|$ vertices to offer, so at most $|S|$ of the odd components can be "rescued" through S. The remaining $o(G-S)-|S|$ odd components each strand a vertex, as shown in Figure 17.7. The formula says the best such set S gives the deficiency exactly. Tutte's perfect-matching theorem is the special case $\nu(G)=n/2$: a perfect matching exists iff $o(G-S)\le|S|$ for every S.

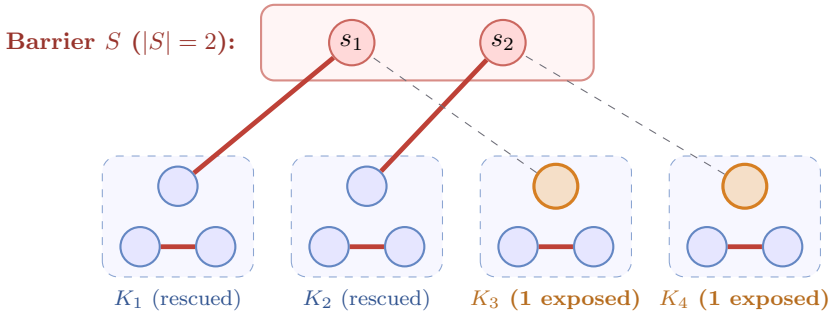


Figure 17.7: The Tutte–Berge counting bound. Deleting $|S|=2$ vertices leaves $o(G-S)=4$ odd components $K_1,\dots,K_4$ (here of size 3 each). Internal matching edges can pair at most two vertices per triangle; the third vertex in each component must either match into S or remain exposed. Because S has only 2 vertices, at most 2 odd components (K_1,K_2) can be rescued, forcing at least $o(G-S)-|S|=4-2=2$ exposed vertices (orange) in any matching.

Proof. (Every S upper-bounds ν.) Fix any S and any matching M. Delete S. In each odd component K of $G-S$, the edges of M lying entirely inside K cover an even number of K's vertices, so at least one vertex of K is either unmatched or matched by an edge into S. Charge that vertex to K. Distinct odd components charge distinct vertices of S (each S-vertex is matched at most once), so at most $|S|$ odd components are rescued through S; the other $o(G-S)-|S|$ components each contribute an exposed vertex. Hence $n-2|M|\ge o(G-S)-|S|$, i.e. $|M|\le\frac{1}{2}(n-(o(G-S)-|S|))$. Taking S to maximize the right side gives $\nu(G)\le\frac{1}{2}(n-\max_S(\dots))$.

(A tight $S^\star$ exists.) We exhibit a set $S^\star$ for which a maximum matching leaves exactly $o(G-S^\star)-|S^\star|$ vertices exposed. Run Edmonds' algorithm to a maximum M and grow the final alternating forest from all exposed vertices, with all blossoms conceptually expanded. Let $S^\star$ be the set of ODD ("inner") vertices, and let the EVEN ("outer") vertices form the stranded components. The Gallai–Edmonds structure theorem makes this precise: each outer odd component is *factor-critical* (deleting any one of its vertices leaves a perfect matching of the rest), each is matched into $S^\star$ at most once, and the number of exposed vertices equals $o(G-S^\star)-|S^\star|$. For this $S^\star$ the inequality above is tight, so the maximum over S is attained and equals the deficiency of M. Therefore $\nu(G)=\frac{1}{2}(n-\max_S(o(G-S)-|S|))$. □

Why it works

The harder half is the existence of a tight $S^\star$, and Edmonds' algorithm *constructs* it: the odd-labelled vertices of the final alternating forest are the Tutte set, and the factor-critical outer blossoms are exactly the odd components. This is the content of the Gallai–Edmonds

decomposition. So the same search that finds the matching also constructs the certificate proving it maximum – a direct combinatorial instance of strong LP duality.

Example 17.1 (A certificate)**.** Take the 7-vertex graph of the trace, with maximum matching $\{23, 45, 67\}$ leaving vertex 1 exposed. Choose $S^\star = \emptyset$. Then $G - S^\star = G$ is connected with 7 vertices, an odd number, so $o(G - \emptyset) = 1$ and $o - |S| = 1 - 0 = 1$. The formula gives deficiency 1, hence $\nu = \frac{1}{2}(7 - 1) = 3$, matching what we found. The empty set already certifies that one vertex must be exposed, because the whole graph is a single odd component.

Application

General matching is used in pairing problems with no two-sided structure (roommate assignment, kidney-exchange chains where any patient may pair with any compatible donor); the Christofides 3/2-approximation for metric TSP, whose bottleneck step is a *minimum-weight perfect matching* on the odd-degree vertices of an MST; switch scheduling and frequency assignment; and the T-join / Chinese-postman problem, which reduces to minimum-weight matching on odd-degree vertices. Whenever a problem says "pair things up to optimize" and the compatibility graph is not bipartite, blossoms appear.

17.8 A Word on Weighted General Matching

Replace cardinality by weight: each edge e has weight w_e and we seek a matching (or perfect matching) of maximum total weight. The augmenting-path structure survives but must respect weights, and the right tool is linear-programming duality, run combinatorially – a *primal–dual* method, just as the Hungarian algorithm is for the bipartite case.

The matching polytope of a general graph is *not* described by the degree and nonnegativity constraints alone; Edmonds' companion result is that it needs the *odd-set* (blossom) inequalities:

$$\sum_{e \subseteq U} x_e \le \frac{|U| - 1}{2} \qquad \text{for every odd } U \subseteq V,\ |U| \ge 3,$$

together with $x_e \ge 0$ and $\sum_{e \ni v} x_e \le 1$. Each blossom inequality carries a dual variable. The weighted blossom algorithm maintains dual variables y_v on vertices and $z_U \ge 0$ on odd sets U (the blossoms it has formed), keeping primal–dual complementary slackness:

- every edge has reduced weight $y_u + y_v + \sum_{U:\, e \subseteq U} z_U - w_{uv} \ge 0$ (dual feasibility);
- matched edges are *tight* (reduced weight zero);
- only blossoms with $z_U > 0$ are kept contracted.

The algorithm grows alternating trees in the tight (zero reduced-weight) subgraph, forms blossoms with $z_U = 0$ as in the cardinality case, and when the tight subgraph admits no further progress it performs a *dual adjustment*: shift the y's and z's by the largest δ that keeps feasibility, which tightens a new edge (extending a tree), or drives some z_U to zero (*expanding* a blossom), or saturates a vertex. Expansion is the new ingredient absent from the cardinality version: a blossom formed earlier may need to be taken apart when its dual hits zero.

Algorithm 70: WeightedMatching – primal–dual skeleton (one phase)

```
initialize y_v ← ½ max_e w_e, all z_U ← 0, M ← ∅
while some vertex is free and improvable do
    grow alternating trees in the tight subgraph from free vertices
    if augmenting path found then
        flip it; reset trees
    else if blossom found (even–even tight edge) then
        contract it, create z_U = 0
    else
        compute δ = largest dual step keeping feasibility
        update y_v += ±δ on tree vertices, z_U −= 2δ on odd inner blossoms
        if some z_U hits 0 then
            expand blossom U
return M
```

Theorem 17.5 (Weighted bound)**.** *Maximum-weight matching (and maximum-weight perfect matching) in a general graph can be solved in $O(V^3)$ time.*

Why it works

The argument mirrors the cardinality count. There are $O(V)$ *augmentation phases* – each either matches one more vertex or, for the perfect case, reduces the number of free vertices. Between augmentations, each *dual adjustment* either tightens an edge (growing a tree), forms a blossom, or expands one; the number of such events between two augmentations is $O(V)$ because the tree gains a vertex or the blossom count changes monotonically. Each event costs $O(V)$ to find the critical δ and update labels with the right data structures, so a phase is $O(V^2)$ and the total is $O(V^3)$. Complementary slackness at termination certifies optimality: the dual (y, z) exactly proves the matching is heaviest, which is the LP-duality content of Edmonds' blossom inequalities.

Connection

The dual variables (y, z) are the explicit variables of the linear-programming dual studied in the companion *Linear Algebra* book; the odd-set inequalities are the facets of the matching polytope, and "the matching polytope is integral" is the geometric statement that blossoms suffice. The same primal–dual pattern underlies the Hungarian algorithm of the previous chapter (bipartite, no z variables needed) and the min-cost-flow algorithms two chapters back. Minimum-weight perfect matching on odd-degree vertices is the key step in Christofides' TSP bound and of the T-join problem; factor-critical blossoms reappear as the "odd components" in the Gallai–Edmonds decomposition that also underlies the rank of the Tutte matrix – a bridge to the algebraic, randomized matching algorithms (Lovász, Mucha–Sankowski) that compute $\nu(G)$ via a symbolic determinant in $O(V^\omega)$ time, linking matching to fast matrix multiplication.

17.9 Summary

The augmenting-path method (Berge) reduces maximum matching to repeatedly finding an alternating path between two free vertices. In bipartite graphs a plain search finds such a path because the even/odd labelling is globally consistent. Odd cycles destroy that consistency: a vertex can be reached by both an even and an odd alternating path. Edmonds' insight is that the offending odd cycle is a *blossom* – and a blossom can be contracted to a single super-vertex without changing whether an augmenting path exists (Theorem 17.2), because every blossom vertex is reachable from the base by an even alternating arc (Lemma 17.1). Contract on collision, expand on success, flip the lifted path; repeat. The simple version runs in $O(V^3)$, refined to $O(E\sqrt{V})$ by Micali–Vazirani. The Tutte–Berge formula states the matching number as a min-max over vertex-deletion sets, and Edmonds' own algorithm constructs the optimal Tutte set as a byproduct. Weighted general matching wraps the same blossom moves in a primal–dual loop over the odd-set inequalities, still $O(V^3)$.

This completes our treatment of matching. Chapter 18 returns to network flows and cuts, showing how flow modeling gives exact algorithms for project selection, image segmentation, sports elimination, and scheduling.

17.10 Exercises

Exercise 17.1 (Quick mental check: blossom edge count)**.** If a blossom contains $2k+1$ vertices, how many edges of the current matching M lie completely inside the blossom? Why must the remaining unmatched vertex be the base v_0?

Exercise 17.2 (Quick mental check: even collision)**.** During an alternating BFS, you discover a non-tree edge between an even vertex and an odd vertex. Does this close an odd cycle or an even cycle? Can it ever lead to a blossom contraction?

Exercise 17.3 (Maximal vs. maximum)**.** Prove that any maximal matching has size at least $\frac{1}{2}\nu(G)$, and give a family of graphs achieving the factor 2 exactly. [Hint: each maximum-matching edge can be "blocked" by at most two maximal-matching edges.]

Exercise 17.4 (Berge by symmetric difference)**.** Prove carefully that for any two matchings $M, M^\star$ each connected component of $M \oplus M^\star$ is a simple path or an even cycle. Where exactly does the argument use that no vertex meets two edges of the same matching?

Exercise 17.5 (Why even cycles never need contracting)**.** Show that if the alternating search ever closes an *even* cycle (an even–odd collision rather than even–even), no blossom is formed and the search may simply ignore the extra edge. Conclude that only odd cycles trigger contraction, and relate this to bipartiteness.

Exercise 17.6 (Lift correctness)**.** Implement the path-lifting step of Theorem 17.2: given an augmenting path in G/C that passes through the super-vertex c, output the corresponding path in G. Prove your output is simple and alternating, using Lemma 17.1.

Exercise 17.7 (Break a wrong contraction)**.** Construct a small graph, a matching, and an *odd* cycle whose base is *not* reachable by an even alternating stem, such that contracting that cycle destroys an augmenting path. This shows both hypotheses of Theorem 17.2 are necessary.

Exercise 17.8 (Trace a nested blossom)**.** Build a graph in which a blossom forms and then the contracted super-vertex participates in a *second*, larger blossom (a blossom inside a blossom). Run Algorithm 68 by hand and record the `base` array after each contraction. What is the maximum nesting depth as a function of n?

Exercise 17.9 (Tutte's theorem from Tutte–Berge)**.** Derive Tutte's perfect-matching theorem – G has a perfect matching iff $o(G - S) \le |S|$ for all S – from Theorem 17.4. Then use it to prove that every bridgeless cubic graph has a perfect matching (Petersen's theorem). [Hint: bound $o(G-S)$ by counting edges leaving odd components.]

Exercise 17.10 (Find the Tutte set)**.** For the 7-vertex trace graph, list *all* sets S that attain the Tutte–Berge maximum $o(G - S) - |S| = 1$. Relate them to the odd-labelled vertices of the final alternating forest.

Exercise 17.11 (Factor-critical blossoms)**.** Prove that the vertex set of any blossom is *factor-critical*: deleting any single vertex of the blossom leaves a graph with a perfect matching on the rest. (This is the structural reason blossoms are the right objects to contract.)

Exercise 17.12 (Weighted complementary slackness)**.** For the weighted algorithm, write out the complementary-slackness conditions in full and prove that a matching together with feasible duals (y, z) satisfying them is simultaneously a maximum-weight matching and a minimum dual – i.e., prove weak and then strong duality for the matching LP with blossom inequalities.

Exercise 17.13 (Christofides connection)**.** Recall Christofides' algorithm: build an MST T, find a minimum-weight perfect matching J on the odd-degree vertices of T, and shortcut an Eulerian tour of $T \cup J$. Prove $w(J) \le \frac{1}{2}\,\mathrm{OPT}$ for a metric instance, and explain precisely which step needs *general* (not bipartite) weighted matching.

Exercise 17.14 (Implement and stress-test)**.** Implement Algorithm 69 and validate it against a brute-force maximum matching on all graphs up to 8 vertices. Then instrument it to also output a Tutte set $S^\star$ and verify $|M| = \frac{1}{2}(n - (o(G - S^\star) - |S^\star|))$ on every instance.

Chapter 18

Modeling with Flows and Cuts

The previous chapters built the solvers: augmenting paths, push–relabel, min-cost flow, and bipartite and general matching. This chapter is about the modeling step – recognizing when a problem can be cast as a flow, cut, or matching problem, and constructing the graph so that an off-the-shelf solver answers the original question.

A reduction is a translation. You take a problem stated in the language of projects and prerequisites, or pixels and labels, or teams and remaining games, and you produce a graph whose maximum flow, minimum cut, or maximum matching answers the original question. The translation has to hold in both directions: every legal solution of the original must correspond to a flow/cut of the graph with a matching objective value, and conversely. Every reduction proof must establish both directions.

Focus

We assume you already know how to *compute* a max flow, a min cut, and a maximum matching, and that you know the max-flow min-cut theorem. This chapter never re-proves those. It proves that particular *models* are correct: that the min cut of the graph we build equals the cost of the best project portfolio, the best image segmentation, the maximum set of disjoint paths, and so on. The recurring proof obligation is a two-sided correspondence between feasible solutions and flows/cuts.

We will collect a small set of gadgets – vertex capacities, lower bounds, antiparallel edges – then spend the bulk of the chapter on four classic modeling applications (counting edge- and vertex-disjoint paths as a pair): project selection / maximum-weight closure, image segmentation as an s–t min cut, baseball elimination, and edge- and vertex-disjoint paths via Menger's theorem. Each gets a hand-traced small graph.

18.1 Conventions and the modeling reflex

Fix notation for the whole chapter. A flow network is a directed graph $G = (V, E)$ with a *source* s, a *sink* t, and a nonnegative integer (or rational) capacity $c(u, v)$ on each edge. A flow f assigns $0 \le f(u, v) \le c(u, v)$ to each edge and conserves flow at every vertex except s and t. Its value $|f|$ is the net flow out of s. An s–t cut (S, T) partitions V with $s \in S$, $t \in T$; its capacity is $c(S, T) = \sum_{u \in S, v \in T} c(u, v)$, counting only edges crossing *forward* from S to T. Max-flow min-cut

says $\max_f |f| = \min_{(S,T)} c(S,T)$.

Intuition

Three reflexes cover almost every modeling problem.

- **"Choose a subset, pay for crossings"** ⇒ *minimum cut.* A cut *is* a partition of the vertices into two sides. If your problem asks you to partition items into two groups while minimizing the penalty of split pairs, it models directly as a minimum cut.
- **"Route as much / as many as possible through a bottlenecked system"** ⇒ *maximum flow.* Capacities are the bottlenecks; the value is the throughput.
- **"Pair up / cover / select with conflicts"** ⇒ *matching* (a special flow). Disjointness constraints become unit capacities.

In every reduction, each feasible decision must map to a valid flow or cut, and the capacities must be chosen so that the solver's optimum is also the optimum of the original problem.

18.1.1 Gadget 1: vertex capacities

Standard flow puts capacities on edges. Many problems want to limit how much passes *through a vertex* – a router that can forward at most k packets, an intersection that can carry at most c cars. The fix is vertex splitting, shown in Figure 18.1.

Focus

Vertex split. Replace each capacitated vertex v by two copies v_{in} and v_{out} joined by a single edge $v_{\text{in}} \to v_{\text{out}}$ with capacity equal to the vertex capacity $c(v)$. Every original edge (u, v) becomes $(u_{\text{out}}, v_{\text{in}})$; every (v, w) becomes $(v_{\text{out}}, w_{\text{in}})$.

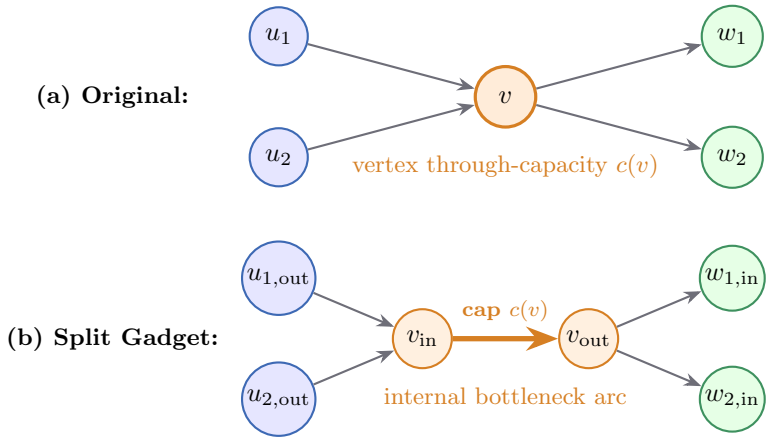


Figure 18.1: The vertex-splitting gadget. (a) A vertex v with through-capacity $c(v)$ limits the total flow passing through it. (b) Replacing v with a directed pair $(v_{\text{in}}, v_{\text{out}})$ connected by a single arc of capacity $c(v)$ forces all flow traversing v across the internal bottleneck edge, converting vertex capacities into standard edge capacities.

Why it works

Every unit of flow that visits v in the original must traverse the unique internal edge $v_{\text{in}} \to v_{\text{out}}$ in the split graph, because that edge is the *only* way to get from any v_{in} to any v_{out}. So the through-flow at v is exactly the flow on that edge, which is capped at $c(v)$. Conversely, any flow in the split graph projects to a flow in the original obeying the vertex bound. The two networks have identical max-flow values, and a min cut that uses an internal edge corresponds to deleting the vertex. The construction at most doubles $|V|$ and adds $|V|$ edges, so it is free asymptotically.

18.1.2 Gadget 2: lower bounds on edges

Sometimes an edge must carry *at least* $\ell(u, v)$ units, not just at most $c(u, v)$. Think of a delivery contract that obligates a minimum shipment, or a scheduling constraint that a machine run at least some minimum load. A flow with lower bounds need not even exist, so the first question is feasibility.

Focus

Demand reduction. Given bounds $\ell(u, v) \le f(u, v) \le c(u, v)$, substitute $f = \ell + g$ where $0 \le g(u, v) \le c(u, v) - \ell(u, v)$. Conservation of f becomes conservation of g plus a fixed *excess* at each vertex:

$$d(v) \;=\; \sum_{u} \ell(u, v) \;-\; \sum_{w} \ell(v, w),$$

the surplus of forced inflow over forced outflow. We must source this surplus or sink this deficit.

Algorithm 71: Feasible circulation with lower bounds

Input: Digraph with bounds $\ell(u, v) \le f(u, v) \le c(u, v)$.
Output: A feasible flow f, or report infeasible.

Build new graph on the same vertices; add super-source S and super-sink T.
for *each edge* (u, v) **do**
  Add edge (u, v) with capacity $c(u, v) - \ell(u, v)$.
for *each vertex* v **do**
  Compute $d(v) = \sum_u \ell(u, v) - \sum_w \ell(v, w)$.
  if $d(v) > 0$ **then** add $S \to v$ with capacity $d(v)$
  if $d(v) < 0$ **then** add $v \to T$ with capacity $-d(v)$
Compute max flow $S \rightsquigarrow T$.
if *every S-edge is saturated (flow $= \sum_{d(v)>0} d(v)$)* **then**
  return $f(u, v) = \ell(u, v) + g(u, v)$ for each original edge.
else
  return infeasible.

Why it works

The super-source S injects exactly the forced surplus into each deficit-of-outflow vertex, and the super-sink T absorbs exactly the forced excess elsewhere. A feasible g exists iff S can push all of its supply to T, i.e. iff the max flow saturates every S-edge. Saturation is equivalent to value $= \sum_{d(v)>0} d(v)$. Reconstructing $f = \ell + g$ restores the lower bounds while preserving the upper bounds because $g \le c - \ell$. To turn this from a feasibility test into a max-flow-with-lower-bounds (between a real s and t), add an edge $t \to s$ of capacity ∞ to convert the problem to a circulation, find a feasible circulation, then maximize the residual $s \rightsquigarrow t$ flow on top.

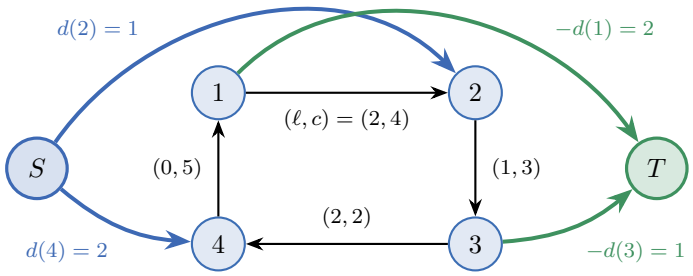


Figure 18.2: The lower-bound demand reduction on a 4-cycle circulation. Each original edge carries lower and upper bounds (ℓ, c). Non-zero demand values $d(v)$ are balanced by auxiliary arcs from super-source S to supply nodes ($d(v) > 0$) and from demand nodes ($d(v) < 0$) to super-sink T.

Trace it yourself

A four-cycle circulation, no real source or sink (Figure 18.2 shows the cycle with (ℓ, c) labels and the super-source/sink attachments computed below). Vertices $\{1, 2, 3, 4\}$; edges with (ℓ, c): $1 \to 2$ is $(2, 4)$, $2 \to 3$ is $(1, 3)$, $3 \to 4$ is $(2, 2)$, $4 \to 1$ is $(0, 5)$. Every vertex has one incoming and one outgoing edge, so compute each $d(v) =$ (forced in) $-$ (forced out):

$$d(1) = \ell(4{\to}1) - \ell(1{\to}2) = 0 - 2 = -2, \quad d(2) = 2 - 1 = +1,$$

$$d(3) = 1 - 2 = -1, \quad d(4) = 2 - 0 = +2.$$

The positive-d vertices (2 and 4) get edges *from* the super-source S with capacities 1 and 2; the negative-d vertices (1 and 3) get edges *to* the super-sink T with capacities 2 and 1. Total supply $\sum_{d>0} d = 1 + 2 = 3$, total demand $\sum_{d<0} |d| = 2 + 1 = 3$ – they always match, which is a useful arithmetic sanity check. The residual capacities $c - \ell$ on the original edges are $1 \to 2 : 2$, $2 \to 3 : 2$, $3 \to 4 : 0$, $4 \to 1 : 5$.

Now push S's 3 units to T. The wiring is $S \to 2$ (cap 1), $S \to 4$ (cap 2) on the supply side, and $1 \to T$ (cap 2), $3 \to T$ (cap 1) on the demand side, with the four original edges carrying their residual capacities. The edge $3 \to 4$ has residual capacity 0, so the route through it is closed, but the remaining residual edges suffice. A feasible routing: $S \to 4 \to 1 \to T$ carries 2 units (uses residual $4 \to 1 = 5$, delivers 2 to vertex 1's sink edge, exactly its cap), and $S \to 2 \to 3 \to T$ carries 1 unit (uses residual $2 \to 3 = 2$, delivers to vertex 3's cap-1 sink edge). All 3 source units placed, every S-edge saturated: **feasible**. Reconstruct $f = \ell + g$: $f(1{\to}2) = 2 + 0 = 2$, $f(2{\to}3) = 1 + 1 = 2$, $f(3{\to}4) = 2 + 0 = 2$, $f(4{\to}1) = 0 + 2 = 2$. Check

conservation: every vertex carries 2 in and 2 out. A uniform circulation of value 2 satisfies all the lower bounds.

18.1.3 Gadget 3: antiparallel and undirected edges

Two edges (u, v) and (v, u) both present ("antiparallel") confuse the textbook residual-graph bookkeeping, which wants a single residual capacity per ordered pair. Split one of them: replace (u, v) by $u \to x \to v$ through a fresh vertex x, each new edge inheriting $c(u, v)$. For an *undirected* edge of capacity c, model it as two directed edges (u, v) and (v, u) each of capacity c; a flow uses at most one direction at full strength in any optimal solution, and the cut capacity counts the edge once per crossing direction – exactly right, since an undirected edge crossing a cut should be paid for once.

Common trap

For *undirected min cut*, both directed copies have capacity c, and a cut that separates the endpoints pays c (one forward copy crosses). Do *not* also count the backward copy: in an s–t cut (S, T) only $S \to T$ edges are charged, so the $v \to u$ copy with $v \in T, u \in S$ is a backward edge and contributes 0. The single undirected edge is therefore charged exactly c. People who symmetrize sloppily double every cut.

18.2 Maximum-weight closure and project selection

Now the first substantial reduction. You have a set of projects. Each project i has a profit p_i (possibly negative – it might be a pure cost). Some projects *require* others: to do i you must also do every project it depends on. Choose a subset maximizing total profit subject to all prerequisites being satisfied. This is the canonical "open-pit mining" / "task selection" problem, and it is a minimum cut.

Definition 18.1 (Closure)**.** Let $G = (V, E)$ be a directed graph with vertex weights $w : V \to \mathbb{R}$. A subset $C \subseteq V$ is *closed* if every edge leaving C stays inside C: whenever $u \in C$ and $(u, v) \in E$, then $v \in C$. The *maximum-weight closure* maximizes $\sum_{v \in C} w(v)$ over closed C.

Read the edges as "u needs v." Closure says: if you take u, you take everything u points to. That is exactly the prerequisite rule.

Intuition

Picture two heaps: projects you *select* (side S, with the source) and projects you *reject* (side T, with the sink). Profitable projects "want" to be on the source side; costly ones want the sink side. A prerequisite edge $u \to v$ must never go from S to T (that would mean taking u but not its prerequisite v). Give that edge infinite capacity so no finite-capacity cut can cross it. Then minimizing the cut balances the forgone profit of rejected good projects against the incurred cost of accepted bad projects – which is precisely the negation of total profit.

18.2.1 The construction

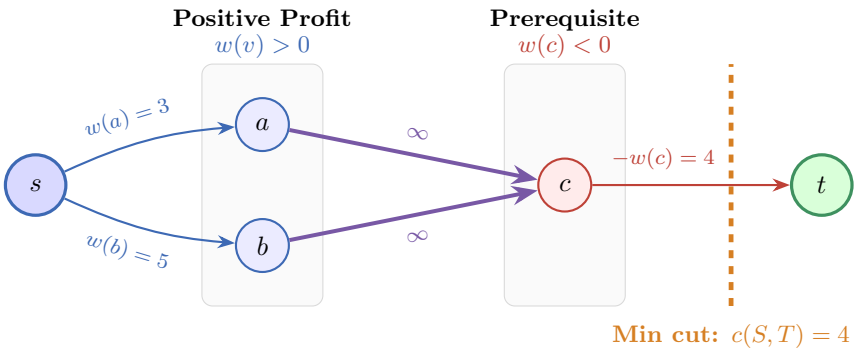
Focus

Build a flow network. Add source s and sink t.

- For each vertex v with $w(v) > 0$: edge $s \to v$ with capacity $w(v)$.
- For each vertex v with $w(v) < 0$: edge $v \to t$ with capacity $-w(v)$.
- For each prerequisite edge $(u, v) \in E$: edge $u \to v$ with capacity ∞.

Let $P = \sum_{v:\, w(v)>0} w(v)$ be the total positive weight. Then the maximum closure value equals $P - (\min\ s\text{–}t\ \text{cut})$, and the selected set is $S \setminus \{s\}$ for a min cut (S, T).

Figure 18.3 shows this construction.



Prerequisites ($a \to c$, $b \to c$) carry capacity ∞, preventing non-closed cuts.
Min cut $S = \{s, a, b, c\}$, $T = \{t\}$ cuts $c \to t$ (cap 4), yielding max closure profit $P - c(S,T) = 8 - 4 = 4$.

Figure 18.3: The maximum-weight closure / project selection reduction. Source s connects to all projects with positive profit ($p_i > 0$) with capacity p_i, representing potential earnings. Dependency arcs $u \to v$ have capacity ∞, preventing finite cuts from separating a project from its prerequisites. Costly prerequisite projects ($p_j < 0$) connect to sink t with capacity $-p_j$. A minimum s–t cut minimizes the sum of forfeited profits on unselected projects and incurred costs on selected prerequisites, achieving the maximum net closure profit $P - c(S,T)$.

Theorem 18.1 (Closure as min cut)**.** *With the construction above, an s–t cut has* finite *capacity iff its source side $S \setminus \{s\}$ is a closed set, and in that case*

$$c(S,T) \;=\; P \;-\; \sum_{v \in S \setminus \{s\}} w(v).$$

Consequently the minimum cut yields the maximum-weight closure, with value $P - c(S,T)_{\min}$.

Proof. Let $C = S \setminus \{s\}$. A cut is finite iff no infinite edge crosses forward. The infinite edges are the prerequisite edges $u \to v$; one crosses forward iff $u \in C$ and $v \notin C$. So finiteness is exactly the closure condition: every prerequisite of a selected vertex is selected.

Now compute the finite cut capacity. The only finite edges are the $s \to v$ edges (capacity $w(v)$ for $w(v) > 0$) and the $v \to t$ edges (capacity $-w(v)$ for $w(v) < 0$).

- An edge $s \to v$ crosses forward iff $v \in T$, i.e. $v \notin C$. It contributes $w(v)$. Summing over positive-weight $v \notin C$ gives $\sum_{v \notin C,\, w(v)>0} w(v) = P - \sum_{v \in C,\, w(v)>0} w(v)$.

- An edge $v \to t$ crosses forward iff $v \in S$, i.e. $v \in C$. It contributes $-w(v)$. Summing over negative-weight $v \in C$ gives $\sum_{v \in C,\, w(v)<0} (-w(v)) = -\sum_{v \in C,\, w(v)<0} w(v)$.

Adding,

$$c(S,T) = P - \sum_{v \in C,\, w(v)>0} w(v) - \sum_{v \in C,\, w(v)<0} w(v) = P - \sum_{v \in C} w(v).$$

Since P is a constant, minimizing $c(S,T)$ maximizes $\sum_{v \in C} w(v)$ over closed C. The infinite edges guarantee the minimizer is closed. □

Trace it yourself

Run it on the picture. $P = w(a) + w(b) = 3 + 5 = 8$. The min s–t cut: the only finite edges are $s \to a$ (3), $s \to b$ (5), $c \to t$ (4). Options for a closed set C:

- $C = \emptyset$: cut cuts $s \to a, s \to b$, capacity 8. Closure value $8 - 8 = 0$.
- $C = \{a, b, c\}$: closed (both prerequisites of c present). Cut cuts only $c \to t$, capacity 4. Closure value $8 - 4 = 4$.
- $C = \{a, c\}$: closed. Cut cuts $s \to b$ (5) and $c \to t$ (4), capacity 9. Value $8 - 9 = -1$.

Minimum cut is 4, achieved by $C = \{a, b, c\}$: take all three, profit $3 + 5 - 4 = 4$. The cut 4 matches $P - 4 = 4$. Note $C = \{a\}$ is *not* closed ($a \to c$ leaves), so the infinite edge $a \to c$ forbids it – any cut separating a from c is infinite.

Pause

Pause and think: why does minimizing cut capacity maximize net profit? A cut (S,T) partitions projects into chosen $(S \setminus \{s\})$ and rejected $(T \setminus \{t\})$. Forward cut edges comprise: (1) $s \to i$ for rejected profitable projects ($i \in T$), costing p_i in forfeited revenue, and (2) $j \to t$ for accepted costly projects ($j \in S$), costing $-p_j$ in incurred costs. Prerequisite edges carry ∞ capacity and never cross forward. Thus, the cut capacity is:

$$c(S,T) = \sum_{i \in T, p_i>0} p_i + \sum_{j \in S, p_j<0} (-p_j) = \sum_{p_i>0} p_i - \text{Profit}(S \setminus \{s\}).$$

Because total positive revenue $P = \sum_{p_i>0} p_i$ is a fixed constant, minimizing $c(S,T)$ maximizes net profit.

Algorithm 72: Project selection via maximum-weight closure

Input: Projects $1..n$ with profits p_i; prerequisite pairs (i, j) meaning i needs j.
Output: Subset maximizing total profit subject to prerequisites.

```
P ← 0; create source s, sink t.
for i = 1 to n do
    if p_i > 0 then add s → i with capacity p_i;  P += p_i
    else if p_i < 0 then add i → t with capacity −p_i
foreach prerequisite (i, j) do add i → j with capacity ∞
(F, (S, T)) ← MaxFlowMinCut(s, t).
return S \ {s} as the selected projects; profit = P − |F|.
```

Cost summary

The network has $n + 2$ vertices and at most $n + |E|$ edges. With Dinic's algorithm the cost is $O\big((n + |E|) \cdot |V|^2\big)$ in general, but on unit-ish or bounded-capacity instances it is far better in practice. Memory is $O(n + |E|)$. The ∞ capacities can be taken as $1 + \sum_i |p_i|$ to keep arithmetic bounded – no finite optimal cut ever uses one.

Origins in open-pit mining. The maximum-closure model is due to Jean-Claude Picard (1976), building on the open-pit mining formulation of Helmut Lerchs and Ingo Grossmann (1965). Mining engineers wanted to decide which blocks of ore to excavate: digging a deep block forces removing the blocks above it (the prerequisites), each block has a value (ore revenue minus extraction cost), and the optimal pit is a maximum-weight closure. The same template reappears as "task scheduling with precedence," "flow-based feature selection," and Markov-random-field energy minimization.

Application

Beyond mining: choosing which experiments to fund when some share expensive shared equipment; selecting a profitable set of software features where features depend on shared libraries; and the densest-subgraph and maximum-density problems, which reduce to a parametric family of closure/min-cut instances (Exercise 18.6).

18.3 Image segmentation as an s–t min cut

Foreground/background segmentation: each pixel must be labeled *object* or *background*. Two forces compete. A *data* term says how much each pixel, on its own evidence (color, intensity), prefers each label. A *smoothness* term penalizes neighboring pixels that get different labels, because real objects are mostly contiguous. Minimizing data + smoothness is, for two labels, exactly a minimum cut.

Intuition

Put the source s on the "object" side and the sink t on the "background" side. A cut assigns every pixel to one side – a labeling. Edge $s \to p$ with weight a_p = "cost of *not* calling p object" (paid when p lands on the background side). Edge $p \to t$ with weight b_p = "cost of *not* calling p background." A neighbor edge p–q with weight λ_{pq} is paid exactly when p and q are split.

The min cut minimizes total penalty: the optimal labeling.

18.3.1 Construction and energy

Let the pixels be vertices. Choose data costs: $a_p \geq 0$ is the penalty for labeling p background, $b_p \geq 0$ for labeling p object. For each neighboring pair $\{p, q\}$ choose a separation penalty $\lambda_{pq} \geq 0$.

Focus

Segmentation network.

- Source s (object), sink t (background).
- Edge $s \to p$ with capacity a_p for every pixel p.
- Edge $p \to t$ with capacity b_p for every pixel p.
- For each neighbor pair $\{p, q\}$, *two* directed edges $p \to q$ and $q \to p$, each capacity λ_{pq}.

A cut (S, T) labels p *object* if $p \in S$, *background* if $p \in T$.

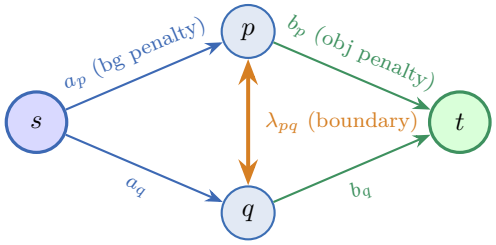


Figure 18.4: The binary image segmentation graph gadget. Pixel affinities to the foreground object source s (a_p) and background sink t (b_p) compete against pairwise spatial coherence terms λ_{pq}. An s–t min cut directly minimizes the combined Markov Random Field energy $E(O, B)$.

Theorem 18.2 (Segmentation energy equals cut capacity)**.** *For any labeling with object set O (the pixels in S) and background set B (the pixels in T), the cut capacity equals the energy*

$$E(O, B) \;=\; \sum_{p \in B} a_p \;+\; \sum_{p \in O} b_p \;+\; \sum_{\{p,q\}:\, split} \lambda_{pq}.$$

Hence the minimum cut is the minimum-energy two-label segmentation.

Proof. Tally forward-crossing edges of the cut (S, T), $s \in S$, $t \in T$, $O = S \setminus \{s\}$, $B = T \setminus \{t\}$.

- $s \to p$ crosses iff $p \in T$, i.e. $p \in B$: contributes a_p. Total $\sum_{p \in B} a_p$.
- $p \to t$ crosses iff $p \in S$, i.e. $p \in O$: contributes b_p. Total $\sum_{p \in O} b_p$.
- For a neighbor pair $\{p, q\}$: if both on the same side, neither directed copy crosses forward. If split, exactly one copy crosses forward (the one going from the S-side endpoint to the T-side endpoint), contributing λ_{pq} once.

Summing gives $E(O, B)$. Minimizing cut capacity minimizes energy. □

Why it works

Why does the two-edge gadget pay λ_{pq} *once* per split and *zero* when not split? Because a cut only charges $S \to T$ edges. If $p, q \in S$, both copies are inside S, charged 0. If $p, q \in T$, both inside T, charged 0. If $p \in S, q \in T$, the copy $p \to q$ is forward (charged λ_{pq}) while $q \to p$ is backward (charged 0). Symmetric in the other split case. This is the undirected-edge trick of Section 18.1.3 doing exactly the right accounting.

Trace it yourself

Three pixels in a row, $p_1 - p_2 - p_3$, neighbors $\{p_1, p_2\}$ and $\{p_2, p_3\}$, all $\lambda = 2$. Data: $a = (5, 1, 0)$ (object-affinity), $b = (0, 4, 6)$ (background-affinity). So p_1 strongly wants object ($a_1 = 5$ penalty to put it in background), p_3 strongly wants background.
Try labeling $O = \{p_1\}$, $B = \{p_2, p_3\}$. Split pairs: $\{p_1, p_2\}$ only. Energy $= (a_2 + a_3) + (b_1) + \lambda = (1 + 0) + 0 + 2 = 3$.
Try $O = \{p_1, p_2\}$, $B = \{p_3\}$. Split pairs: $\{p_2, p_3\}$. Energy $= a_3 + (b_1 + b_2) + \lambda = 0 + (0 + 4) + 2 = 6$.
Try all object: energy $= 0 + (b_1 + b_2 + b_3) + 0 = 10$. Try all background: $(a_1 + a_2 + a_3) + 0 + 0 = 6$.
The winner is $O = \{p_1\}, B = \{p_2, p_3\}$ at energy 3: p_1 is foreground, the rest background, with one boundary paid. A min-cut solver finds this as the cut $\{s, p_1\} \,|\, \{p_2, p_3, t\}$ of capacity 3.

Common trap

Min cut handles *exactly two* labels. With three or more labels the energy minimization is NP-hard in general (it is multiway cut). The α-expansion and $\alpha\beta$-swap algorithms of Boykov–Veksler–Zabih attack the multilabel case by solving a *sequence* of two-label min cuts, each one a binary "keep your label vs. switch to α" decision. They give a 2-approximation under a metric smoothness condition, not the exact optimum (Exercise 18.8).

Common trap

The exact-cut reduction needs the pairwise term to be *submodular*: the penalty for "different" must be at least the penalty for "same." Here "same" costs 0 and "different" costs $\lambda_{pq} \geq 0$, so submodularity holds automatically. If you tried to *reward* disagreement (negative λ), the problem stops being a min cut and becomes NP-hard. Submodularity is the dividing line between "min-cut easy" and "hard" (Exercise 18.7).

From statistical physics to computer vision. Margaret Greig, Allan Porteous, and Allan Seheult (1989) first observed that the maximum a-posteriori estimate of a binary image under an Ising-type prior is an exact min cut – a result that sat almost unused for a decade until Yuri Boykov and Vladimir Kolmogorov (2001–2004) engineered specialized max-flow algorithms fast enough for high-resolution vision and developed the α-expansion framework. Today graph cuts are a standard building block in stereo reconstruction, interactive segmentation, and surface stitching.

Connection

The energy $\sum_p(\text{data}) + \sum_{p\sim q}\lambda_{pq}[x_p \neq x_q]$ is, up to constants, a weighted graph *cut function*, and minimizing $[x_p \neq x_q]$ summed over edges is the quadratic form $\frac{1}{4}x^\top \mathbf{L}x$ on $\{\pm 1\}$ labels, where $\mathbf{L}$ is the graph Laplacian (Chapter 31). The data terms shift it to a linear-plus-quadratic energy. The s–t cut solves this Boolean quadratic exactly because the pairwise part is submodular; the spectral relaxation (Chapter 32) drops the integrality and minimizes the Rayleigh quotient instead, giving Cheeger-type cuts. Both use the same Laplacian; they differ only in the relaxation.

18.4 Baseball elimination

This problem does not look like a flow problem at all from its statement. It is late in the season. Team x has won w_x games and has r_x games left. Between every pair $\{i, j\}$ a known number g_{ij} of games remain to be played. Question: can team x *still* finish in first place (tie allowed), assuming the remaining games can come out any way we like? If not, x is mathematically eliminated.

Intuition

Team x does best if it wins all r_x remaining games, reaching $W = w_x + r_x$. For x to finish first, *no other team* may exceed W wins. So every other team i can win at most $W - w_i$ more games – call this its *slack*. The remaining games among the other teams must be distributed without busting any slack. "Distribute games subject to capacity" is a flow: route each remaining game to one of its two teams, respecting each team's slack as a capacity into the sink.

18.4.1 Construction

Fix the team x we are testing. Let $W = w_x + r_x$. If some other team already has $w_i > W$, then x is eliminated immediately. Otherwise build:

Focus

Elimination network for team x.

- Source s, sink t.
- A *game node* g_{ij} for each unordered pair $i \neq j$ of teams other than x with $g_{ij} > 0$. Edge $s \to g_{ij}$ with capacity g_{ij} (these games must be played and assigned).
- A *team node* for each team $i \neq x$. Edges $g_{ij} \to i$ and $g_{ij} \to j$, each capacity ∞ (a game between i and j is won by one of them).
- Edge $i \to t$ with capacity $W - w_i$ (team i's slack).

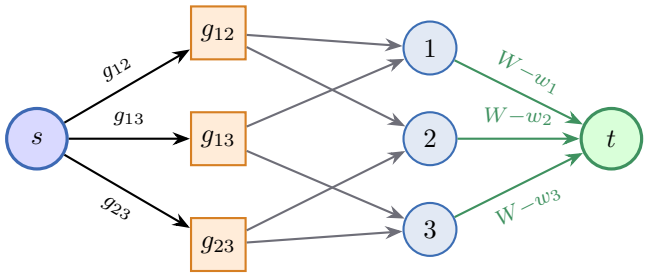


Figure 18.5: The baseball elimination network for evaluating team x. Matchup nodes g_{ij} receive games that must be distributed to opponents i and j, while opponent nodes connect to sink t with capacity bounded by their win slacks $W - w_i$.

Theorem 18.3 (Baseball elimination)**.** *Let $R = \sum_{i<j,\, i,j\neq x} g_{ij}$ be the total number of remaining games among teams other than x. Team x is* not *eliminated iff the max flow in the network saturates the source, i.e. has value R (equivalently, the min cut has capacity R).*

Proof. ($\Leftarrow$) Suppose max flow $= R$. Every game edge $s \to g_{ij}$ is saturated, so all g_{ij} units leave g_{ij} along $g_{ij} \to i$ and $g_{ij} \to j$. Interpret the flow on $g_{ij} \to i$ as "i wins that many of the g_{ij} games." By conservation, the wins assigned to each team total $\sum_j f(g_{ij} \to i)$, which is the flow into team node i, hence at most its capacity $W - w_i$. So i's final win total $w_i + (\text{assigned wins}) \leq w_i + (W - w_i) = W$. Let x win all its remaining games to reach W; then no team exceeds W, and x ties or leads. Not eliminated.

($\Rightarrow$) Suppose x is not eliminated, so there is an outcome of *all* remaining games with x on top at some total $\leq W$ (and x can always be assumed to win all its own games, pushing it to exactly W and only helping). In that outcome every other team finishes with $\leq W$ wins, so the games it won among the others-only games number at most $W - w_i$. Route each others-only game to its winner: this is a feasible flow saturating every $s \to g_{ij}$ edge (each game is won by someone), with team-inflows $\leq W - w_i$ respecting the sink capacities. Its value is R. So max flow $= R$. □

Trace it yourself

Three other teams $\{1, 2, 3\}$, current wins $w = (83, 80, 78)$. We test team x with $w_x = 81$, $r_x = 2$, so $W = 83$. Slacks: $W - w_1 = 0$, $W - w_2 = 3$, $W - w_3 = 5$. Remaining others-only games: $g_{12} = 1, g_{13} = 0, g_{23} = 4$, so $R = 5$.
Source edges: $s \to g_{12}$ (cap 1), $s \to g_{23}$ (cap 4). Team 1 has slack 0: edge $1 \to t$ has capacity 0. The game g_{12} must send its 1 unit to team 1 or team 2; team 1 is full (capacity 0), so the unit goes to team 2. The game g_{23} sends 4 units split between teams 2 and 3. Team 2 already took 1 unit, leaving slack $3 - 1 = 2$; team 3 has slack 5. Together they absorb $2 + 5 = 7 \geq 4$. So all 5 units route through: max flow $= 5 = R$. Team x is *not* eliminated.
Now suppose instead $w_x = 79, r_x = 2$, so $W = 81$, slacks $(W - w_i) = (-2, 1, 3)$. Team 1 already has $83 > 81$: trivial elimination, no flow needed.

Why it works

The min cut tells you *why* a team is eliminated. If the max flow is $< R$, the min cut (S, T) exposes a set of teams $A = \{i : i \in S\}$ whose combined remaining games among themselves cannot fit under their combined slack. Concretely, eliminate x iff there is a subset A of the

others with

$$\frac{w(A) + g(A)}{|A|} > W,$$

where $w(A) = \sum_{i \in A} w_i$ and $g(A)$ counts games with both teams in A: their games *must* be won by members of A, so the average final win count of A is forced above W. The flow finds this certificate of elimination automatically – it is the team subset on the source side of the min cut.

Algorithm 73: Is team x eliminated?

Input: Wins w_i, remaining pairwise games g_{ij}, the team x to test.
Output: ELIMINATED or ALIVE, with a certificate set if eliminated.

```
W ← w_x + r_x where r_x = Σ_j g_xj.
if ∃ i ≠ x : w_i > W then return ELIMINATED (trivial, certificate {i})
Build the elimination network; R ← Σ_{i<j, i,j≠x} g_ij.
(F, (S, T)) ← MAXFLOWMINCUT(s, t).
if |F| = R then
    return ALIVE
else
    return ELIMINATED, certificate A = {i : i ∈ S}.
```

Cost summary

For one team: $O(n^2)$ game nodes, $O(n^2)$ edges, max flow with Dinic in $O(|V|^2|E|) = O(n^6)$ worst case but vastly faster in practice (the graph is unit-ish and shallow). Testing all n teams multiplies by n. Hoffman and Rivlin's classical condition sometimes lets you skip the flow, but the flow formulation is the clean general answer.

Sports elimination and flow certificates. The problem and a combinatorial criterion go back to Benjamin Schwartz (1966) and to Alan Hoffman and T. J. Rivlin (1967). The crisp max-flow formulation with the average-wins certificate became a textbook staple through Kevin Wayne's exposition. It is a popular example because the reduction is so unexpected: "can my team still win the pennant" has no edges or capacities in its statement.

18.5 Disjoint paths and Menger's theorem

A classical use of flow is connectivity. How many *edge-disjoint* paths run from s to t? How many *internally vertex-disjoint* ones? Menger's theorem answers both, and max-flow min-cut is the modern proof.

Definition 18.2 (Disjoint paths and separators)**.** Two s–t paths are *edge-disjoint* if they share no edge, *internally vertex-disjoint* if they share no vertex except s and t. An *edge cut* is a set of edges whose removal leaves no s–t path; a *vertex separator* is a set of vertices (other than s, t) whose removal leaves no s–t path.

Theorem 18.4 (Menger, edge form)**.** *In any graph, the maximum number of pairwise edge-disjoint s–t paths equals the minimum number of edges in an s–t edge cut.*

Theorem 18.5 (Menger, vertex form)**.** *If s, t are non-adjacent, the maximum number of internally vertex-disjoint s–t paths equals the minimum size of an s–t vertex separator.*

18.5.1 Edge form via unit capacities

Intuition

Give every edge capacity 1. A flow of value k is, by integrality of max flow, k units that decompose into k edge-disjoint paths (each unit reuses no edge, since capacity is 1). A cut of capacity k is literally a set of k edges whose removal disconnects s from t. Max-flow min-cut equates the two. That is the entire proof.

Proof of Theorem 18.4. Set $c(e) = 1$ for all edges. By the integral flow theorem (max flow with integer capacities has an integral optimum), there is a max flow f with $f(e) \in \{0, 1\}$ and value $k = |f|$. Decompose f into paths and cycles; discard cycles (they do not change the value). Since each edge carries at most 1 unit, no two of the resulting s–t paths share an edge: k edge-disjoint paths exist.

Conversely, any k edge-disjoint paths give a feasible flow of value k (one unit per path), so $\max f \geq$ (max edge-disjoint paths); the decomposition gives the reverse, hence equality between max flow and max edge-disjoint paths. Min-cut capacity, with unit capacities, counts edges, and a finite-capacity s–t cut is exactly an s–t edge cut. Max-flow min-cut closes the loop: max edge-disjoint paths $=$ max f $=$ min cut $=$ min edge cut. □

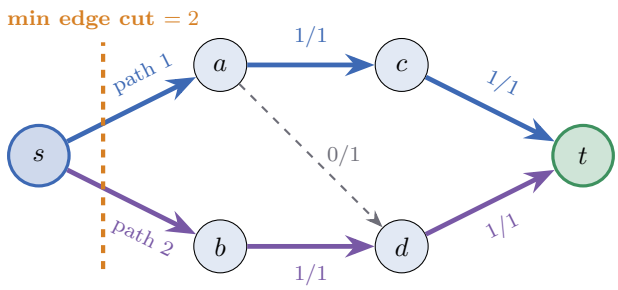


Figure 18.6: Edge-disjoint paths in a unit-capacity directed network. Two edge-disjoint paths $s \to a \to c \to t$ (blue) and $s \to b \to d \to t$ (purple) match the minimum s–t edge cut $\{s \to a, s \to b\}$ of capacity 2 (orange dashed).

Trace it yourself

Trace the picture, all capacities 1. Two edge-disjoint s–t paths: $s \to a \to c \to t$ and $s \to b \to d \to t$. Can we get a third? Every s–t path must use $s \to a$ or $s \to b$; there are only those two edges out of s, so at most 2 edge-disjoint paths leave s. Indeed the cut $\{s \to a,\ s \to b\}$ has size 2 and removing it isolates s. Max flow $= 2 =$ min cut. The diagonal $a \to d$ does not create a third disjoint path because the source still has out-degree 2.

18.5.2 Vertex form via vertex splitting

For internally vertex-disjoint paths, paths may share edges only if they also share vertices, so the binding constraint is on vertices. Apply the vertex-split gadget of Section 18.1.1 with every internal vertex capacity 1.

Focus

Vertex-disjoint network. Split each vertex $v \notin \{s, t\}$ into $v_{\text{in}} \to v_{\text{out}}$ with capacity 1; leave s, t uncapacitated. Original edge (u, v) becomes $(u_{\text{out}}, v_{\text{in}})$ with capacity ∞ (the vertex caps already do the limiting). Max flow = max internally vertex-disjoint paths; a finite min cut must cut internal vertex edges and so equals a min vertex separator.

Proof of Theorem 18.5. After splitting, every flow path traverses each internal vertex's unit-capacity internal edge $v_{\text{in}} \to v_{\text{out}}$, and the aggregate flow across that edge is at most 1, so a flow of value k decomposes into k paths that pairwise share no internal vertex: k internally vertex-disjoint paths. Conversely k vertex-disjoint paths give a flow of value k. So max flow = max vertex-disjoint paths.

A min cut in this network: edges crossing forward are either original ∞-edges (never in a finite cut) or internal $v_{\text{in}} \to v_{\text{out}}$ edges of capacity 1. A finite min cut therefore selects a set of internal vertex-edges whose removal disconnects s from t – that is exactly a vertex separator, of size equal to the cut capacity. Max-flow min-cut gives max vertex-disjoint paths = min separator. Non-adjacency of s, t ensures no direct $s \to t$ edge bypasses all internal vertices (which would make the separator undefined). □

Trace it yourself

Take the diamond with edges $s \to a$, $s \to b$, $a \to t$, $b \to t$, and a cross edge $a \to b$, shown in Figure 18.7. Internally vertex-disjoint s–t paths: $s \to a \to t$ and $s \to b \to t$ are vertex-disjoint (share only s, t). A third would need a new internal vertex, but only a, b exist, both used. The minimum separator is $\{a, b\}$, size 2: deleting both disconnects every s–t path. After splitting, the internal edges $a_{\text{in}} \to a_{\text{out}}$ and $b_{\text{in}} \to b_{\text{out}}$ (capacity 1 each) form the min cut of capacity 2 = max flow. The cross edge $a \to b$ becomes $a_{\text{out}} \to b_{\text{in}}$ and cannot manufacture a third disjoint path, because any path through it still consumes both a's and b's unit capacity.

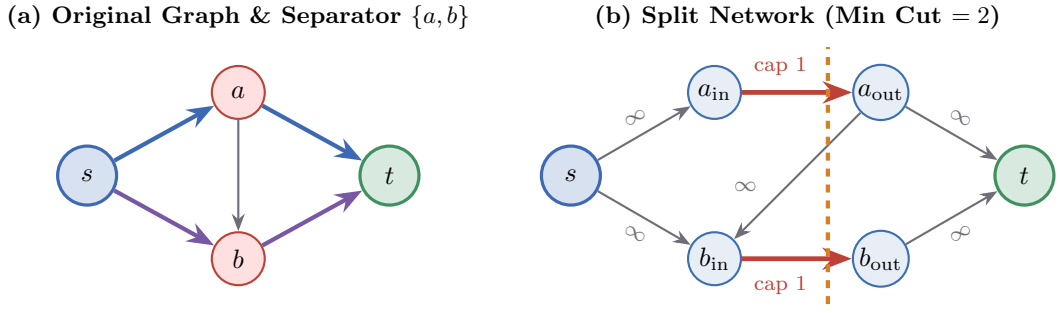


Figure 18.7: Menger's vertex form via vertex splitting. (a) The diamond network has two internally vertex-disjoint paths ($s \to a \to t$ and $s \to b \to t$) and minimum vertex separator $\{a, b\}$ of size 2. (b) Splitting a and b into unit-capacity internal arcs while setting original edges to ∞ turns the vertex separator $\{a, b\}$ into a minimum edge cut of capacity 2 (orange dashed).

Why it works

Why does integrality matter so much here? The whole equivalence "flow value = number of disjoint paths" rests on max flow having an *integer* optimum when capacities are integers (proved in the max-flow chapter via augmenting paths preserving integrality). Without it, a fractional flow could split a unit across several edges and you could not read off disjoint paths. Every disjoint-paths and matching reduction relies on unit capacities plus integrality.

Connection

Menger is the local (s–t) version of global connectivity. The minimum over *all* pairs s, t of the min s–t edge cut equals the global edge connectivity, computed in Chapter 11 (global min cut, Stoer–Wagner / Karger). The vertex form underlies k-connectivity testing and is the combinatorial basis of the max-flow proof of Hall's theorem and Kőnig's theorem for bipartite matching (Chapter 15): a bipartite matching is just a unit-capacity s–t flow, and Kőnig's min-vertex-cover = max-matching is Menger's vertex form on the bipartite flow network (Exercise 18.11).

History

Karl Menger proved his path-separator duality theorem in 1927, establishing that edge and vertex bottlenecks govern connectivity long before the concept of network flow existed. In 1956, Lester Ford and Delbert Fulkerson published their max-flow min-cut theorem at the RAND Corporation, proving that Menger's theorem, Dénes Kőnig's matching theorem, and Robert Dilworth's poset chain theorem all follow from a single linear programming duality. Over subsequent decades, researchers realized that flows and cuts could solve problems bearing no surface resemblance to networks: Jean-Claude Picard (1976) reduced precedence-constrained scheduling and open-pit mining to min-cuts; Alan Hoffman and Benjamin Schwartz applied flows to sports elimination; and in 1989, Margaret Greig, Allan Porteous, and Allan Seheult showed that maximum a posteriori image segmentation under binary Markov random fields is an exact min-cut. Today, graph cuts remain essential throughout combinatorial optimization and computer vision, while developing near-linear-time algorithms for multi-terminal and hypergraph cuts remains an active research area.

18.6 A reduction checklist

Focus

When you suspect a problem is a flow/cut/matching, run this checklist.

1. **Binary choice with split penalties?** Min cut. Vertices = items, source/sink = the two labels, neighbor edges = separation penalties (must be nonnegative: submodular).
2. **Prerequisites / "take this forces that"?** Maximum-weight closure ⇒ min cut, with ∞ on the forcing edges.
3. **Throughput through bottlenecks?** Max flow. Vertex limits ⇒ split vertices. Minimum loads ⇒ lower bounds.

4. **Assign items to bins under per-bin quotas?** Max flow: item→bin edges, bin→ t capacity = quota (baseball is this).
5. **Disjointness / connectivity?** Unit capacities ⇒ Menger. Vertex-disjoint ⇒ split.

Common trap

The single most common modeling bug is an *orientation* or *capacity-sign* error that makes the min cut measure the wrong thing – e.g. putting profit on the sink side, or forgetting the ∞ on a forcing edge so the cut "cheats" by severing a prerequisite. Always verify the two-sided correspondence on a tiny instance by hand (as in every Trace it yourself box above) before trusting the solver. If your hand trace and the solver disagree, the model is wrong, not the solver (Exercise 18.13).

Why this matters

The same theorem, max-flow min-cut, solves five problems that share no surface vocabulary: mining schedules, image pixels, pennant races, network reliability, and graph connectivity. These reductions work because "partition into two sides and pay for crossings" and "route as much as possible through bottlenecks" recur across combinatorial optimization well beyond physical networks. Once you recognize those two structures, many seemingly unrelated optimization problems reduce to a single max-flow call.

Where We Go Next (Part IV: Tree Structures and Decompositions): Across Parts I–III, our graph algorithms operated on flat, static graphs where every query inspects the entire network or rebuilds a search from scratch. Part IV turns to trees and dynamic queries. When a graph is a tree – or can be organized into one – how do we answer path queries and update edge weights in logarithmic time without re-running whole traversals? In Chapter 19, we begin with *Heavy-Light Decomposition*, partitioning any tree into heavy paths that turn complex branch-to-branch queries into fast interval operations.

18.7 Exercises

Exercise 18.1 (Quick mental check: infinite capacities)**.** In maximum-weight closure, why do prerequisite edges receive capacity ∞? What bug would occur if we replaced ∞ with a small constant like 1?

Exercise 18.2 (Quick mental check: vertex capacity)**.** When splitting a capacitated vertex v into $v_{\text{in}} \to v_{\text{out}}$, what should the capacity of the internal edge $(v_{\text{in}}, v_{\text{out}})$ be, and what capacities do the incoming and outgoing edges receive?

Exercise 18.3 (Vertex-capacitated max flow)**.** You are given a directed network where both edges and vertices have capacities. Describe the splitting construction precisely and prove that the resulting edge-capacitated max flow equals the original vertex-and-edge-capacitated max flow. What is the blow-up in $|V|$ and $|E|$?

Exercise 18.4 (Lower bounds, end to end)**.** Run the lower-bound feasibility reduction of Algorithm 71 on the following circulation: vertices $\{1, 2, 3\}$, edges $1 \to 2$ with $\ell = 1, c = 3$; $2 \to 3$ with

$\ell = 2, c = 2$; $3 \to 1$ with $\ell = 0, c = 5$. Compute every $d(v)$, build the super-source/super-sink network, and decide feasibility by hand. Then exhibit a feasible flow or a certificate of infeasibility.

Exercise 18.5 (Closure at the extremes)**.** Suppose every vertex weight is negative. What is the maximum-weight closure, and what min cut does Theorem 18.1 produce? Now suppose every weight is positive: same questions. Use these extremes to sanity-check that an implementation handles the empty and full closures.

Exercise 18.6 (Densest subgraph)**.** The density of a vertex subset A in an undirected graph is $|E(A)|/|A|$ (edges inside A over its size). Show that "is there a subset of density $\geq g$?" reduces to a maximum-weight closure / min-cut instance with a parameter g, and describe how a binary (or parametric) search over g finds the densest subgraph. (Hint: make each edge a positive-weight "project" requiring both its endpoints, each endpoint a cost-g prerequisite.)

Exercise 18.7 (Segmentation hardness boundary)**.** Prove that if some pairwise term *rewards* disagreement – i.e. a neighbor pair pays a negative λ_{pq} when split – then the energy is no longer the capacity of any s–t cut with nonnegative capacities. Explain in one paragraph why this corresponds to non-submodularity and why min cut can no longer be used directly.

Exercise 18.8 (Multiway labels)**.** Three labels $\{R, G, B\}$, energy with data terms and a Potts smoothness (λ paid per disagreeing neighbor pair). Show that the natural "one source per label" min-cut construction does *not* in general give the optimum (find a 3-pixel counterexample), and explain how α-expansion sidesteps this with a sequence of binary cuts.

Exercise 18.9 (Baseball certificate)**.** Standings $w = (75, 71, 69, 63)$, remaining games $g_{12} = 2, g_{13} = 1, g_{14} = 0, g_{23} = 4, g_{24} = 0, g_{34} = 3$. Test team 4 assuming it can reach $W = 71$ in the best case. Build the elimination network for $x = 4$, compute the max flow versus R, and – if eliminated – produce the subset-A certificate from the min cut and verify $(w(A) + g(A))/|A| > W$.

Exercise 18.10 (Menger by hand)**.** For the graph with undirected unit-capacity edges sa, sb, ab, at, bc, ct, ac, find the maximum number of edge-disjoint s–t paths and a minimum edge cut, and verify they are equal. Then redo it for internally vertex-disjoint paths and minimum vertex separators using the vertex-split network.

Exercise 18.11 (Kőnig from Menger)**.** Derive Kőnig's theorem (in a bipartite graph, max matching = min vertex cover) from Menger's vertex form (Theorem 18.5) applied to the standard bipartite flow network ($s \to$ left $\to$ right $\to t$, unit capacities). Identify what an internally vertex-disjoint s–t path is, and what a vertex separator is, in graph-matching terms.

Exercise 18.12 (Project selection with mutual exclusion)**.** Extend the project-selection model so that some pairs of projects are *mutually exclusive* (at most one of the pair may be chosen). Show that a naive ∞-edge gadget cannot encode "at most one," and discuss why mutual exclusion can make the problem NP-hard (relate it to the non-submodular / max-cut boundary). Give one special case where it stays polynomial.

Exercise 18.13 (Break the model)**.** Take the project-selection construction of Algorithm 72 and *remove* the ∞ on prerequisite edges, replacing it with a large finite capacity M. Construct an instance where the min cut now "cheats" by severing a prerequisite edge, returning an infeasible (non-closed) selection. State the smallest M for which your instance still misbehaves, and explain the fix.

Part IV

Decompositions and Dynamic Structures

Chapter 19

Heavy-Light Decomposition

In the preceding chapters we explored global problems on flat networks – pushing flows and finding cuts across entire graphs. Part IV turns to trees and dynamic queries. A balanced binary search tree answers range queries on an *array* in $O(\log n)$ time: a *segment tree* or Fenwick tree turns "sum or minimum of an interval" into a logarithmic walk. Trees in the graph sense are not arrays, though. A query like "what is the maximum edge weight on the path between vertices u and v?" or "add 5 to every vertex in the subtree rooted at w" has no obvious contiguous home in an array. The path between u and v bends through the tree, meeting at their *lowest common ancestor* (LCA), and there are $\Theta(n^2)$ such paths.

Heavy-light decomposition (HLD) closes this gap. It splits any tree into a small family of vertical *paths*, lays each path down as a contiguous segment of a single array, and proves that any root-to-node path – and therefore any node-to-node path – crosses only $O(\log n)$ of these segments. Once the tree is sliced this way, every off-the-shelf range structure you already own (segment tree, Fenwick tree, sparse table) becomes a tree-path structure. Path queries and path updates cost $O(\log^2 n)$, while subtree queries map directly to single contiguous intervals.

This chapter builds the decomposition from the size-of-subtree intuition, proves the $O(\log n)$-chains bound rigorously, traces the whole construction by hand on a concrete 11-vertex tree, gives pseudocode for the build and for both kinds of query, and analyzes the complexity. We close with the subtree trick and the connection to Euler tours, link-cut trees, and centroid decomposition.

19.1 The problem: queries that follow tree paths

Fix a rooted tree T on n vertices, root r. Each vertex (or each edge) carries a value, and we want to support operations of two flavors along *paths* and *subtrees*:

- **Path query** $\textsc{Query}(u, v)$: aggregate the values on the vertices (or edges) lying on the unique simple path from u to v – sum, max, min, gcd, anything associative.
- **Path update** $\textsc{Update}(u, v, \delta)$: add δ to (or assign δ on) every vertex on that path.
- **Subtree query / update**: the same, restricted to the subtree rooted at a given w.
- **Point operations**: read or write a single vertex; a degenerate path with $u = v$.

Intuition

On an array, the path between positions i and j is just the contiguous interval $[i, j]$, and a segment tree aggregates it in $O(\log n)$. On a tree, the unique simple path between u and v climbs from u up to their *lowest common ancestor* $\ell = \mathrm{LCA}(u, v)$ – the deepest node that is an ancestor of both – and then descends to v. (While dedicated static LCA algorithms are studied in Chapter 23, HLD actually computes LCAs for free: repeatedly jumping up chains until u and v share a chain immediately reveals their LCA at the shallower node.) We would love to order the tree vertices in an array so that every path becomes a single contiguous index range. While no single ordering can do this for all paths simultaneously, heavy-light decomposition guarantees that any path is covered by at most $O(\log n)$ contiguous intervals.

The naive approaches set the bar. Re-walking the path vertex by vertex costs $O(n)$ per query: fine for one query, too slow for 10^5 of them. Precomputing all pairs costs $\Theta(n^2)$ memory. We want preprocessing near-linear and each operation polylogarithmic, while supporting *updates*, which rules out static precomputation alone.

Focus

The chapter rests on one fact. Color each tree edge *heavy* or *light* by a rule that depends only on subtree sizes. Then any root-to-leaf walk uses at most $\lfloor \log_2 n \rfloor$ light edges. Heavy edges chain together into vertical paths; light edges are the rare jumps between chains. "Few light edges" is exactly "few chains crossed," and "few chains" is exactly "few segment-tree ranges."

19.2 Heavy and light: the size rule

For a vertex v, let $\mathrm{sz}(v)$ be the number of vertices in the subtree rooted at v (counting v itself). For a non-leaf v, its *heavy child* is a child c maximizing $\mathrm{sz}(c)$; ties are broken arbitrarily but consistently (say, the first such child in adjacency order). The edge (v, c) to the heavy child is the *heavy edge*; every other child-edge of v is *light*.

Definition 19.1 (Heavy edge, light edge)**.** Let v have children $c_1, \ldots, c_k$ in the rooted tree. The heavy child is $\arg\max_i \mathrm{sz}(c_i)$ (lowest index on ties). The edge from v to its heavy child is heavy; the edges from v to its other children are light. Every vertex (except the root, which has no parent edge) sits at the bottom of exactly one parent edge, which is heavy or light accordingly.

Definition 19.2 (Heavy path / chain)**.** A maximal connected set of vertices joined by heavy edges is a *heavy path* (or *chain*). Because each vertex has at most one heavy edge going down (to its heavy child) and at most one coming from above (it is its parent's heavy child or not), the heavy edges form a disjoint union of vertical paths. Each chain has a *top* (highest vertex, whose parent edge is light or absent) and runs downward through a sequence of heavy children.

Intuition

Picture water poured at the root, always flowing into the fattest child. The streams it traces are the heavy chains: long vertical runs down the bulkiest part of the tree. A light edge is a step into a small side-branch – and stepping into a small branch is something you cannot do too many times, because each light step at least halves the subtree size beneath you. The

proof below rests on this halving.

19.2.1 The key lemma

Lemma 19.1 (Light edges halve the subtree)**.** *If (v, c) is a light edge (so c is a non-heavy child of v), then* $\mathrm{sz}(c) \le \mathrm{sz}(v)/2$.

Why it works

Let h be the heavy child of v. By the choice of heavy child, $\mathrm{sz}(c) \le \mathrm{sz}(h)$ for every other child c. Now v's subtree contains v, the subtree of h, the subtree of c, and possibly more, all disjoint, so

$$\mathrm{sz}(v) \;\ge\; \mathrm{sz}(h) + \mathrm{sz}(c) \;\ge\; \mathrm{sz}(c) + \mathrm{sz}(c) \;=\; 2\,\mathrm{sz}(c).$$

Hence $\mathrm{sz}(c) \le \mathrm{sz}(v)/2$. The only fact used is that the heavy child is at least as big as c; everything else only makes $\mathrm{sz}(v)$ larger. □

Theorem 19.1 (Few light edges on any vertical path)**.** *On the path from the root to any vertex u, the number of light edges is at most $\lfloor \log_2 n \rfloor$. Consequently the root-to-u path meets at most $\lfloor \log_2 n \rfloor + 1$ distinct heavy chains.*

Why it works

Walk from the root down to u. Whenever we traverse a light edge (v, c), Lemma 19.1 says the subtree size at least halves: $\mathrm{sz}(c) \le \mathrm{sz}(v)/2$. Heavy edges never increase the subtree size as we descend either. Start with $\mathrm{sz}(\text{root}) = n$ and end with $\mathrm{sz}(u) \ge 1$. If the path uses k light edges, then

$$1 \;\le\; \mathrm{sz}(u) \;\le\; \frac{n}{2^k},$$

because each of the k light edges contributes a factor of at least 2 to the denominator and heavy edges only shrink things further. Rearranging, $2^k \le n$, so $k \le \log_2 n$, i.e. $k \le \lfloor \log_2 n \rfloor$ since k is an integer.

For the chain count: a vertical root-to-u path is partitioned into maximal heavy runs separated by light edges. With k light edges there are at most $k+1$ heavy runs, and each run lies inside a single chain. So the path touches at most $k + 1 \le \lfloor \log_2 n \rfloor + 1$ chains. □

Common trap

The bound is about *light* edges, not chains directly, and it is a property of *vertical* root-descending paths. A general $u \to v$ path is two vertical paths glued at the LCA, so it meets at most $2(\lfloor \log_2 n \rfloor + 1) = O(\log n)$ chains – still logarithmic. A common mistake is to claim each chain is short. Chains can be as long as n (a path graph is one giant chain). What is bounded is how *many different* chains a path visits, not their lengths.

Pause

Pause and think: can a tree have zero light edges? Yes! If the tree is a path (a line graph $1 - 2 - \cdots - n$), every child is an only child and therefore heavy. There is only one heavy chain spanning the entire tree and zero light edges. Conversely, in a star graph with $n - 1$ leaves attached to the root, only one leaf edge is heavy; the other $n - 2$ edges are light. Yet any root-to-leaf path uses only one edge.

19.3 Laying the chains into an array

Knowing the chains is not enough. To reuse a segment tree we must assign each vertex a *position* $\text{pos}(v) \in \{0, 1, \ldots, n - 1\}$ so that:

1. every chain occupies a *contiguous* block of positions, with the chain's top at the smallest position and proceeding downward; and

2. each subtree occupies a *contiguous* block of positions (this gives subtree queries for free).

Both are achieved by a single DFS that, at each vertex, recurses into the heavy child *first*, then into the light children, assigning positions in visit order. Visiting the heavy child first keeps a chain's vertices consecutive; visiting all of a vertex's subtree before backtracking keeps each subtree consecutive. This is just an Euler-style preorder with a deliberate child ordering.

Intuition

Think of two passes. The first DFS computes $\text{sz}(\cdot)$ and, while it is at it, records each vertex's heavy child. The second DFS does the relabeling: it stamps the current vertex with the next free position, then dives heavy-first so an entire chain gets a run of consecutive stamps before any light branch starts. Each vertex also remembers the *top* of its chain, which is what lets a query jump from chain to chain.

19.3.1 A tree to carry through the chapter

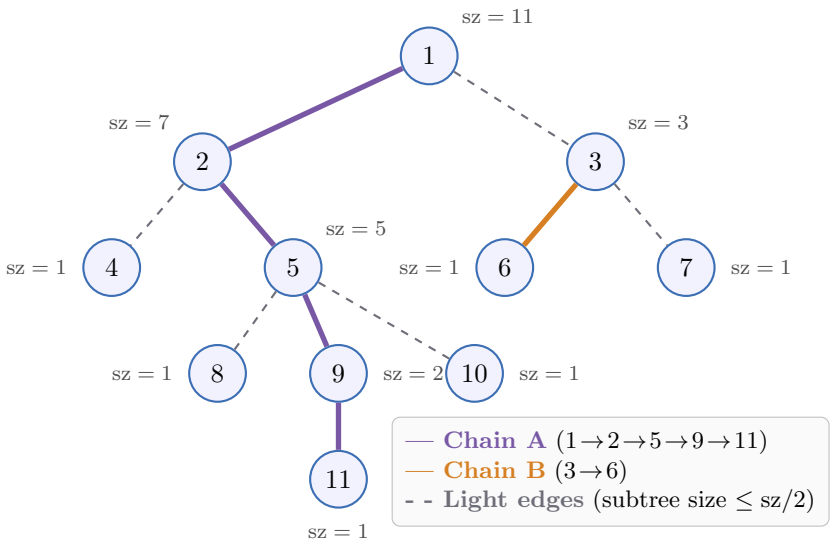


Figure 19.1: The running tree on 11 vertices with heavy and light edge classifications. At each node v, the heavy child maximizes $\mathrm{sz}(c)$. Thick purple edges form Chain A, thick orange edges form Chain B, and dashed edges are light transitions.

The adjacency (children, in the order we will scan them) is: $1\colon \{2,3\}$, $2\colon \{4,5\}$, $3\colon \{6,7\}$, $5\colon \{8,9,10\}$, $9\colon \{11\}$, and $4, 6, 7, 8, 10, 11$ are leaves.

Trace it yourself

Subtree sizes (post-order). Leaves first: $\mathrm{sz}(4) = \mathrm{sz}(6) = \mathrm{sz}(7) = \mathrm{sz}(8) = \mathrm{sz}(10) = \mathrm{sz}(11) = 1$. Then

$$\mathrm{sz}(9) = 1 + \mathrm{sz}(11) = 2, \quad \mathrm{sz}(5) = 1 + \mathrm{sz}(8) + \mathrm{sz}(9) + \mathrm{sz}(10) = 1 + 1 + 2 + 1 = 5,$$

$$\mathrm{sz}(2) = 1 + \mathrm{sz}(4) + \mathrm{sz}(5) = 1 + 1 + 5 = 7, \qquad \mathrm{sz}(3) = 1 + \mathrm{sz}(6) + \mathrm{sz}(7) = 3,$$

$$\mathrm{sz}(1) = 1 + \mathrm{sz}(2) + \mathrm{sz}(3) = 1 + 7 + 3 = 11 = n. \ \checkmark$$

Heavy children. At 1: children $2\,(7)$ vs $3\,(3) \Rightarrow$ heavy child 2. At 2: $4\,(1)$ vs $5\,(5) \Rightarrow 5$. At 3: $6\,(1)$ vs $7\,(1) \Rightarrow 6$ (tie, lowest index). At 5: $8\,(1), 9\,(2), 10\,(1) \Rightarrow 9$. At 9: only $11 \Rightarrow 11$. This matches the thick edges in Figure 19.1.

Chains. Following heavy edges from each chain top:

$$\text{Chain A} : 1 \to 2 \to 5 \to 9 \to 11, \qquad \text{Chain B} : 3 \to 6,$$

$$\text{singletons} : \{4\},\ \{7\},\ \{8\},\ \{10\}.$$

Their tops are $1, 3, 4, 7, 8, 10$ respectively (a vertex is a chain top iff its parent edge is light or it is the root).

19.3.2 Positions from the heavy-first DFS

We now run the second DFS from the root, stamping positions $0, 1, 2, \ldots$, always recursing into the heavy child before the light ones. Whenever we enter a vertex through a light edge (or it is the root), that vertex starts a new chain, so we set its top to itself; a heavy step inherits the parent's top.

Trace it yourself

Order of first-visits, heavy-first at every vertex:

$$1,\ 2,\ 5,\ 9,\ 11,\ 8,\ 10,\ 4,\ 3,\ 6,\ 7.$$

Reading the stamps off:

vertex v	1	2	5	9	11	8	10	4	3	6	7
pos(v)	0	1	2	3	4	5	6	7	8	9	10
top(v)	1	1	1	1	1	8	10	4	3	3	7

Chain A is exactly positions $[0, 4]$ – one contiguous block, top at 0. Chain B is positions $[8, 9]$. The singleton chains land at $5, 6, 7, 10$. Figure 19.2 shows the layout. Notice subtrees are contiguous too: subtree of 5 is $\{5, 9, 11, 8, 10\}$ at positions $\{2, 3, 4, 5, 6\} = [2, 6]$, and subtree of 2 is positions $[1, 7]$. We will lean on that for subtree operations in Section 19.7.

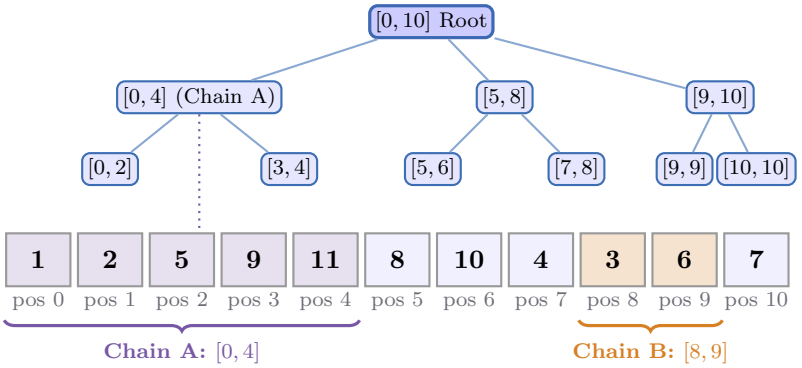


Figure 19.2: The HLD position array layout with the segment tree interval hierarchy built directly above it. Heavy chains map to contiguous blocks (Chain A to $[0, 4]$, Chain B to $[8, 9]$), so standard segment tree canonical range queries QUERY(L, R) resolve tree-path segments in $O(\log n)$ time each.

Why it works

Why is a chain contiguous? When the DFS first reaches a chain top t, it stamps t, then immediately recurses into t's heavy child (the next chain vertex), stamps it, recurses into *its* heavy child, and so on – no light branch is touched until the heavy descent of the chain has been fully stamped, because heavy-first ordering puts the heavy subtree entirely before any light subtree. Therefore the chain's vertices receive a run of consecutive numbers starting at pos(t). Why is a subtree contiguous? Standard preorder: a vertex's stamp is followed by the stamps of its entire subtree before the DFS backtracks, regardless of child order.

19.4 Building the decomposition

Two DFS passes, then we hand the position array to whatever range structure we like. The first pass computes sizes, parents, depths, and heavy children; the second assigns positions and chain tops.

Algorithm 74: DECOMPOSE: build heavy-light positions

Input: Rooted tree T on n vertices, root r, adjacency lists.
Output: Arrays par, dep, sz, heavy, top, pos.

```
Function DfsSize(v, p)
  par[v] ← p;  sz[v] ← 1;  heavy[v] ← −1;  best ← 0
  for c ∈ children(v) (neighbors ≠ p) do
      dep[c] ← dep[v] + 1;  DfsSize(c, v);  sz[v] += sz[c]
      if sz[c] > best then
          best ← sz[c];  heavy[v] ← c

Function DfsHld(v, t)   (t = current chain top)
  top[v] ← t;  pos[v] ← timer;  timer += 1
  if heavy[v] ≠ −1 then
      DfsHld(heavy[v], t)    // stay on the chain
  for c ∈ children(v) with c ≠ heavy[v] do
      DfsHld(c, c)    // light edge: c starts a new chain

dep[r] ← 0;  DfsSize(r, −1);  timer ← 0;  DfsHld(r, r)
build a segment tree (or Fenwick tree) over an array of length n indexed by pos
```

Cost summary

DfsSize and DfsHld each visit every vertex and edge once: $O(n)$ time, $O(n)$ space for the six arrays. Building the segment tree is $O(n)$. Total preprocessing $O(n)$. (Recursion depth can be $\Theta(n)$ on a path graph; in practice convert to an explicit stack or raise the recursion limit.)

Common trap

If your tree is given with a recursion depth that can reach n (a caterpillar or a path), a recursive DfsSize can blow the stack at $n \approx 10^5$. Either iterate with an explicit stack, or bump the system recursion limit. This is an implementation hazard, not an algorithmic one, but it sinks more contest submissions than any logic bug in HLD.

19.5 Answering a path query

To aggregate over the path $u \to v$, repeatedly fold in the top of whichever endpoint sits in the deeper chain, then jump to the chain top's parent. We measure "deeper chain" by the depth of the chain *top*: whichever of $\text{top}[u], \text{top}[v]$ is deeper gets processed, because its chain cannot contain the other endpoint. We fold the contiguous segment from that top down to the endpoint, hop above the top via its parent (a light edge), and continue until both endpoints share a chain, where one final range

from the shallower to the deeper finishes the job.

Algorithm 75: PATHQUERY(u, v) over vertex values

Input: u, v; segment tree S supporting range aggregation $\oplus$ over positions.
Output: $\bigoplus$ of values on the $u \to v$ path.

```
res ← identity of ⊕
while top[u] ≠ top[v] do
    if dep[top[u]] < dep[top[v]] then
        swap u ↔ v
    // top[u] is the deeper chain top; fold its segment
    res ← res ⊕ S.QUERY(pos[top[u]], pos[u])
    u ← par[top[u]]   // climb the light edge above the chain
// now u, v share a chain; fold the segment between them
if dep[u] > dep[v] then
    swap u ↔ v
res ← res ⊕ S.QUERY(pos[u], pos[v])
return res
```

Why it works

Correctness. Invariant: at the top of the loop, res holds the aggregate of all path vertices already passed, and the still-unprocessed part of the path is exactly the path between the *current* u and v. When $\text{top}[u] \neq \text{top}[v]$, the deeper top, say $\text{top}[u]$, cannot be an ancestor of v (if it were, v would lie in u's chain or below a shallower top), so the entire segment from $\text{top}[u]$ down to u belongs to the path and is a contiguous index range $[\text{pos}[\text{top}[u]], \text{pos}[u]]$. Folding it and moving u to $\text{par}[\text{top}[u]]$ removes exactly that segment from the remaining path and preserves the invariant. The loop ends when both share a chain, i.e. both lie on one contiguous block; the final range from the shallower to the deeper closes the path, whose shallower endpoint is by then the LCA. □

Why it works

Why it is fast. Each loop iteration moves one endpoint above its current chain top across a *light* edge. By Theorem 19.1, each of the two vertical halves (endpoint up to LCA) crosses at most $\lfloor \log_2 n \rfloor$ light edges, so the loop runs $O(\log n)$ times. Each iteration issues one segment-tree range query at $O(\log n)$. Total $O(\log^2 n)$ per path query.

Pause

Pause and think: why must we compare depths of chain tops rather than depths of nodes? If node u is deeper than node v, its chain top $\text{top}[u]$ might still be shallower than $\text{top}[v]$. If we moved u simply because $\text{dep}[u] > \text{dep}[v]$, we could leap past the lowest common ancestor across an entire chain. Comparing $\text{dep}[\text{top}[u]]$ ensures we only advance the endpoint whose chain top is strictly deeper, which can never overshoot the LCA.

19.5.1 Tracing a query by hand

Trace it yourself

Query the path $8 \to 7$ on the running tree (Figure 19.1), using the positions and tops from Section 19.3. Recall top[8] = 8, top[7] = 7, depths dep[8] = 3, dep[7] = 2, and dep[top[8]] = 3, dep[top[7]] = 2. The true path is $8 \to 5 \to 2 \to 1 \to 3 \to 7$.
Iteration 1. top[8] = 8 ≠ 7 = top[7]. Deeper top is 8 (depth 3 > 2). Fold range [pos[8], pos[8]] = [5, 5], i.e. just vertex 8. Climb: $u \leftarrow$ par[top[8]] = par[8] = 5.
Iteration 2. Now $u = 5$, $v = 7$: top[5] = 1, top[7] = 7, with dep[top[5]] = dep[1] = 0, dep[top[7]] = 2. Deeper top is 7, so swap to process the v-side: fold [pos[7], pos[7]] = [10, 10] (vertex 7). Climb: $v \leftarrow$ par[7] = 3.
Iteration 3. $u = 5$, $v = 3$: top[5] = 1, top[3] = 3, depths of tops 0 vs 1. Deeper top is 3; fold [pos[3], pos[3]] = [8, 8] (vertex 3). Climb: $v \leftarrow$ par[3] = 1.
Iteration 4. $u = 5$, $v = 1$: top[5] = top[1] = 1. Loop exits. Final segment: dep[5] = 2 > dep[1] = 0, so swap; fold [pos[1], pos[5]] = [0, 2], i.e. vertices 1, 2, 5.
Collected ranges: {8}, {7}, {3}, {1, 2, 5}, whose union is {1, 2, 3, 5, 7, 8} – exactly the path 8, 5, 2, 1, 3, 7 (Figure 19.3). Four range queries, matching the $O(\log n)$ chain bound for $n = 11$ ($\lfloor \log_2 11 \rfloor + 1 = 4$). Each range query is a logarithmic segment-tree call.

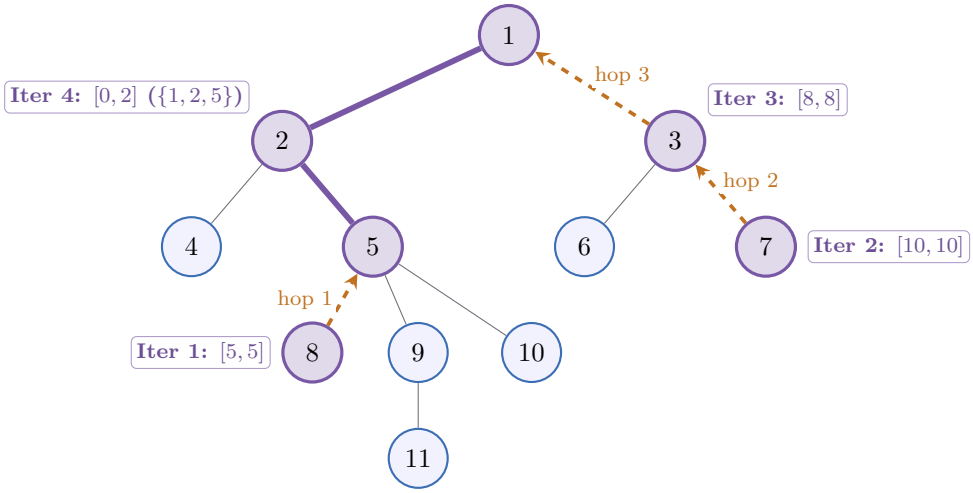


Figure 19.3: The path $8 \to 7$ (purple vertices) split into the four folded ranges: Iter 1 folds {8} ([5, 5]), Iter 2 folds {7} ([10, 10]), Iter 3 folds {3} ([8, 8]), and Iter 4 folds {1, 2, 5} ([0, 2] on Chain A). Orange dashed arrows show the three light-edge hops ($u, v \leftarrow$ par[top]) that advance between chains.

19.6 Path updates and lazy propagation

A path *update* is identical in structure: decompose the path into the same $O(\log n)$ contiguous ranges and apply a range update to each, using a lazy-propagating segment tree. Replace each $S.\textsc{query}(\cdot, \cdot)$ in Algorithm 75 with $S.\textsc{rangeUpdate}(\cdot, \cdot, \delta)$ and drop the accumulator.

Algorithm 76: PATHUPDATE(u, v, δ) – add δ on the path

```
while top[u] ≠ top[v] do
    if dep[top[u]] < dep[top[v]] then
        swap u ↔ v
    S.RANGEUPDATE(pos[top[u]], pos[u], δ)
    u ← par[top[u]]
if dep[u] > dep[v] then
    swap u ↔ v
S.RANGEUPDATE(pos[u], pos[v], δ)
```

Cost summary

$O(\log n)$ chain segments, each a lazy range update at $O(\log n)$: $O(\log^2 n)$ per path update. With a Fenwick tree and a difference trick you can sometimes shave the inner factor for specific aggregates, but $O(\log^2 n)$ is the standard, safe bound.

19.6.1 Edge values vs vertex values

Many problems put values on *edges*, not vertices. Map each non-root vertex v to its parent edge $(\text{par}[v], v)$ and store that edge's value at $\text{pos}[v]$. The root owns no parent edge, so its slot stays at the identity. The only change to a path query/update is the final within-chain range: after the loop, u is the LCA, and the LCA's vertex slot is *not* an edge of the path, so fold $[\text{pos}[u]+1,\ \text{pos}[v]]$ instead of $[\text{pos}[u], \text{pos}[v]]$ (and skip entirely if $u = v$). Figure 19.4 contrasts the final chain segment on Chain A for both conventions so you can see why the $+1$ shift is mandatory.

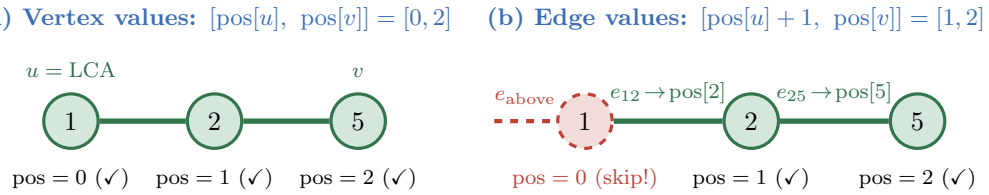


Figure 19.4: Final chain range between $u = \text{LCA} = 1$ and $v = 5$ on Chain A. (a) For vertex queries, vertex 1 lies on the path, so range $[0, 2]$ includes $\text{pos}[1] = 0$. (b) For edge queries, each edge is stored at its deeper endpoint ($\text{pos}[2] = 1$ holds e_{12}, $\text{pos}[5] = 2$ holds e_{25}), while $\text{pos}[1] = 0$ holds the edge above the LCA, so the query must fold $[\text{pos}[u] + 1, \text{pos}[v]] = [1, 2]$.

Common trap

The off-by-one at the LCA is the single most common HLD bug. For *vertex* values, the LCA is on the path and its slot must be included. For *edge* values, the LCA's slot represents the edge *above* the LCA, which is off the path, so it must be excluded with the $+1$. Mixing these up gives answers that are wrong by exactly one vertex/edge and are hard to spot because short paths look correct.

19.7 Subtree queries and updates

Subtrees are easier than paths, because of the contiguity we already proved: the subtree of v is the position range $[\text{pos}[v],\ \text{pos}[v] + \text{sz}[v] - 1]$. So a subtree query or update is a *single* range operation – no chain walking at all.

Why it works

The heavy-first DFS is still a preorder, so when it stamps v at position $\text{pos}[v]$ it then stamps every descendant of v before returning, giving them positions $\text{pos}[v] + 1, \ldots, \text{pos}[v] + \text{sz}[v] - 1$ contiguously. Hence SubtreeQuery$(v) = S$.query$(\text{pos}[v],\ \text{pos}[v] + \text{sz}[v] - 1)$ and the analogous range update. Both cost $O(\log n)$.

Trace it yourself

On the running tree, subtree of 5 has $\text{pos}[5] = 2$, $\text{sz}[5] = 5$, so range $[2, 6] = \{5, 9, 11, 8, 10\}$ – correct. Subtree of 2: $\text{pos}[2] = 1$, $\text{sz}[2] = 7$, range $[1, 7] = \{2, 5, 9, 11, 8, 10, 4\}$. A single segment-tree call covers either, no per-chain loop.

Why this matters

The same pos ordering gives both $O(\log^2 n)$ path operations and $O(\log n)$ subtree operations. Every DFS preorder makes subtrees contiguous; visiting the heavy child first chooses the specific preorder that also keeps each heavy chain contiguous, so one segment tree supports both.

19.8 Complexity, all in one place

Cost summary

Let n be the number of vertices and Q the number of operations.

Operation	Time	Why
Preprocessing (two DFS + segtree)	$O(n)$	each vertex/edge touched $O(1)$ times
Memory	$O(n)$	six arrays + segment tree
Path query / update	$O(\log^2 n)$	$O(\log n)$ chains $\times$ $O(\log n)$ segtree
Subtree query / update	$O(\log n)$	one contiguous range
Point query / update	$O(\log n)$	one segtree leaf path
LCA via the path loop	$O(\log n)$	last shared top is the LCA

Total over Q operations: $O(n + Q \log^2 n)$. The two log factors come from two independent logarithmic structures stacked: the HLD chain count and the segment tree depth.

Why it works

Can we drop a log? For commutative-group aggregates (sum), pairing HLD with a Fenwick tree keeps $O(\log^2 n)$ but with smaller constants. For *static* path *queries* (no updates), replace the segment tree by an $O(1)$ sparse table over each chain and the cost drops to $O(\log n)$ per query – $O(\log n)$ chains, $O(1)$ each. With updates the inner log is genuinely needed, so $O(\log^2 n)$ is the right target for the dynamic case. A different decomposition, link-cut trees (Chapter 22), achieves $O(\log n)$ *amortized* per path operation even with structural changes, at the cost of much heavier machinery.

19.9 Lowest Common Ancestor via Chain Climbs

The path loop already walks both endpoints up to their meeting chain, so it also computes $\mathrm{LCA}(u, v)$: when the loop exits, the shallower of u, v is the LCA.

Algorithm 77: LCA(u, v) via heavy paths

```
while top[u] ≠ top[v] do
    if dep[top[u]] < dep[top[v]] then
        swap u ↔ v
    u ← par[top[u]]
if dep[u] ≤ dep[v] then return u
else return v
```

Trace it yourself

LCA$(8, 7)$: tops $8, 7$, deeper $8 \Rightarrow u \leftarrow \mathrm{par}[8] = 5$. Now tops $1, 7$, deeper $7 \Rightarrow v \leftarrow \mathrm{par}[7] = 3$. Tops $1, 3$, deeper $3 \Rightarrow v \leftarrow \mathrm{par}[3] = 1$. Tops $1, 1$ equal, exit. Shallower of $5, 1$ is 1. Indeed $\mathrm{LCA}(8, 7) = 1$, matching the trace in Section 19.5. Cost $O(\log n)$: pure pointer climbing, no segment tree.

19.10 Worked end-to-end example with values

Put a value on each vertex and run a real mix of operations, so the segment tree side is concrete. Initialize vertex i's value to i (so val $= [1, \ldots, 11]$ indexed by vertex). Lay them by position using Section 19.3's table; the segment-tree array $a[\,]$ indexed by position is

position p	0	1	2	3	4	5	6	7	8	9	10
vertex	1	2	5	9	11	8	10	4	3	6	7
$a[p]$	1	2	5	9	11	8	10	4	3	6	7

Trace it yourself

(1) Sum on path $11 \to 10$**.** Path is $11 \to 9 \to 5 \to 10$. Run the loop. Initially top[11] = 1 (depth 0), top[10] = 10 (depth 3): deeper top 10, swap so we fold the 10-side: range $[\text{pos}[10], \text{pos}[10]] = [6, 6]$, value $a[6] = 10$; climb $v \leftarrow \text{par}[10] = 5$. Now $u = 11, v = 5$, tops 1 and 1 equal – exit. Final range: $\text{dep}[11] = 4 > \text{dep}[5] = 2$ so swap; fold $[\text{pos}[5], \text{pos}[11]] = [2, 4]$, values $a[2] + a[3] + a[4] = 5 + 9 + 11 = 25$. Total $= 10 + 25 = 35$. Check directly: $\text{val}(11) + \text{val}(9) + \text{val}(5) + \text{val}(10) = 11 + 9 + 5 + 10 = 35$. ✓
(2) Add $\delta = 100$ **on path** $4 \to 6$**.** Path is $4 \to 2 \to 1 \to 3 \to 6$. Loop: tops top[4] = 4 (depth 2), top[6] = 3 (depth 1): deeper top 4, range $[\text{pos}[4], \text{pos}[4]] = [7, 7]$ gets +100 (vertex 4); climb $u \leftarrow \text{par}[4] = 2$. Now $u = 2$ (top = 1, depth 0), $v = 6$ (top = 3, depth 1): deeper top 3, swap; range $[\text{pos}[3], \text{pos}[6]] = [8, 9]$ gets +100 (vertices 3, 6); climb $v \leftarrow \text{par}[3] = 1$. Now $u = 2, v = 1$ tops 1, 1 equal – exit. Final: $\text{dep}[2] = 1 > 0$, swap; range $[\text{pos}[1], \text{pos}[2]] = [0, 1]$ gets +100 (vertices 1, 2). Updated vertices: $\{4, 3, 6, 1, 2\}$, exactly the path. New values: $a[0] = 101, a[1] = 102, a[7] = 104, a[8] = 103, a[9] = 106$.
(3) Sum on subtree of 2**.** Range $[\text{pos}[2], \text{pos}[2] + \text{sz}[2] - 1] = [1, 7]$. After update (2) this is $a[1{:}7] = 102 + 5 + 9 + 11 + 8 + 10 + 104 = 249$. Vertices $\{2, 5, 9, 11, 8, 10, 4\}$; original sum $2 + 5 + 9 + 11 + 8 + 10 + 4 = 49$, and the two of them on path $4 \to 6$ that got +100 are 2 and 4, i.e. +200, giving $49 + 200 = 249$. ✓ (Vertices $1, 3, 6$ are not in the subtree of 2.)

19.11 Reference implementation sketch

The pieces fit into a compact program: two DFS for the layout, a lazy segment tree for the array, and the chain-walking wrappers. The skeleton below assumes a 0-indexed segment tree exposing QUERY(l, r) and UPDATE(l, r, δ) over inclusive ranges.

```
// par[], dep[], sz[], heavy[], top[], pos[] filled by Algorithm 19.1.
long long pathQuery(int u, int v) {
    long long res = 0;                      // identity of the aggregate
    while (top[u] != top[v]) {
        if (dep[top[u]] < dep[top[v]]) swap(u, v);
        res += seg.query(pos[top[u]], pos[u]);
        u = par[top[u]];
    }
    if (dep[u] > dep[v]) swap(u, v);
    res += seg.query(pos[u], pos[v]);       // pos[u]+1 for EDGE values
    return res;
}
void pathUpdate(int u, int v, long long d) {
    while (top[u] != top[v]) {
        if (dep[top[u]] < dep[top[v]]) swap(u, v);
        seg.update(pos[top[u]], pos[u], d);
        u = par[top[u]];
    }
    if (dep[u] > dep[v]) swap(u, v);
    seg.update(pos[u], pos[v], d);          // pos[u]+1 for EDGE values
}
long long subtreeQuery(int v) {             // contiguous range, one call
    return seg.query(pos[v], pos[v] + sz[v] - 1);
}
```

Common trap

Three recurring pitfalls. (i) Comparing dep[u] instead of dep[top[u]] inside the loop – the loop condition is about which *chain* is deeper, decided by the top's depth. (ii) The edge-value +1 at the LCA, already flagged. (iii) Forgetting that the final range needs the shallower endpoint first; if pos[u] > pos[v] your segment tree may silently return the identity or crash. Swap by depth before the final call.

19.12 Where it gets used

Application

Network paths. "Maximum congested link on the route between two routers, with link costs changing over time" is a path-max query with point updates – HLD plus a max segment tree.
Accumulation on org charts and phylogenies. "Add a bonus to everyone on the chain of command from employee u up to manager v" is a path add; "total payroll under manager w" is a subtree sum. Both are exactly the operations above.
Competitive programming. HLD is the standard answer to tree path queries with updates (e.g. SPOJ `QTREE`, many Codeforces tree problems). When the values are on edges and the query is path-max, HLD over a max segment tree is the textbook solution.
Auxiliary structure. HLD underlies fast implementations of tree-path versions of problems that on arrays use segment trees: range assignment, range gcd, range affine transforms – each lifts to trees by swapping the underlying segment tree.

History

The heavy-light decomposition idea originated in Daniel Sleator and Robert Tarjan's 1983 paper on *link-cut trees*, where dynamic heavy paths (called "preferred paths") are maintained in splay trees to support tree modifications in logarithmic amortized time. In 1984, Dov Harel and Tarjan adapted heavy-path partitioning to develop the first linear-time preprocessing, constant-time query algorithm for lowest common ancestors, settling a major open question in data structures. During the early 2000s, competitive programmers and researchers adapted Tarjan's static heavy-path partitioning to pair with standard array range structures like segment trees, naming the result "Heavy-Light Decomposition." This gave a simple way to perform path updates and subtree operations without the complex rotations of dynamic trees. An open practical problem is reducing the $O(\log^2 n)$ factor to $O(\log n)$ for general non-invertible semigroup updates on static trees without incurring large constant factors or non-cache-friendly pointer-chasing.

Connection

Range structures (Chapter 23). HLD is a reduction: it converts tree-path problems into the array-range problems solved by segment trees, Fenwick trees, and sparse tables. Everything you know about lazy propagation, range-assign, and merge-sort trees transfers verbatim to

tree paths.
Euler tour technique (Chapter 21). The heavy-first DFS *is* an Euler/preorder tour; the subtree-contiguity we used is the same in/out-time trick that the Euler tour chapter develops for subtree queries. HLD adds the chain alignment on top.
Link-cut trees (Chapter 22). LCTs maintain heavy (preferred) paths dynamically under `link`/`cut`, achieving $O(\log n)$ amortized where static HLD needs the tree fixed. Static HLD is the right tool when the tree shape never changes; LCTs when it does.
Centroid decomposition (Chapter 20). A sibling reduction that also yields $O(\log n)$ recursion depth, but partitions by *centroids* for path-*counting* problems rather than laying paths into an array. The two solve different query shapes; knowing both lets you pick.
Where We Go Next (Centroid Decomposition): In this chapter, we decomposed a tree vertically into heavy paths to turn single-path queries into array ranges. But what if a problem asks not about a single path between two endpoints, but about all paths in the tree satisfying a property (such as counting all pairs at distance exactly k)? In Chapter 20, we turn from vertical path decompositions to recursive divide-and-conquer, splitting trees at balanced vertex centroids.

19.13 Exercises

Exercise 19.1 (Quick mental check: heavy child uniqueness)**.** Can a vertex with three children have two heavy children? What happens if two children tie for the maximum subtree size?

Exercise 19.2 (Quick mental check: light edge bound)**.** Explain in your own words why stepping across a light edge from parent v to child c must cut the subtree size strictly in half ($\text{sz}(c) < \text{sz}(v)/2$).

Exercise 19.3 (Trace another path)**.** On the running tree (Figure 19.1), trace $\textsc{PathQuery}(4, 11)$ step by step. List every chain range folded, the value of u, v after each iteration, and confirm the union of ranges is the true path $4 \to 2 \to 5 \to 9 \to 11$.

Exercise 19.4 (Edge values)**.** Reimplement PathQuery for values stored on *edges* (each edge's value at the child's position). Prove that the only change is the final within-chain range, which becomes $[\text{pos}[u] + 1, \text{pos}[v]]$, and that the result is empty when $u = v$. Trace path-max on edges for the pair $(8, 7)$ assuming each edge (p, c) has weight $\text{dep}[c]$.

Exercise 19.5 (Break the tie rule)**.** Show that choosing the heavy child by *smallest* subtree instead of largest can violate Lemma 19.1. Give a concrete small tree where the light-edge count on some root-to-leaf path exceeds $\lfloor \log_2 n \rfloor$ under the wrong rule, and explain which inequality in the proof fails.

Exercise 19.6 (Subtree add, path sum)**.** Design a structure supporting subtree-add updates and path-sum queries together. Argue that a single lazy segment tree over the HLD positions handles both, since a subtree is one range and a path is $O(\log n)$ ranges. State the per-operation cost and prove correctness of combining the two range families on one tree.

Exercise 19.7 (Static queries, one log)**.** Suppose there are no updates, only path-max queries. Replace the segment tree with a sparse table over the whole position array. Prove that each path query then costs $O(\log n)$ total, not $O(\log^2 n)$, and identify exactly where the second log disappears.

Exercise 19.8 (Kth vertex on a path)**.** Extend HLD to answer "the k-th vertex on the path from u to v" in $O(\log n)$. Hint: split at the LCA using depths, then locate within chains by position arithmetic. Prove your index computation handles the LCA boundary correctly.

Exercise 19.9 (Distance and the LCA loop)**.** Using only dep and Algorithm 77, give an $O(\log n)$ formula for the number of edges on the path $u \to v$. Prove it from $\mathrm{dist}(u, v) = \mathrm{dep}[u] + \mathrm{dep}[v] - 2\,\mathrm{dep}[\mathrm{LCA}(u, v)]$.

Exercise 19.10 (Worst case is one chain)**.** Prove that on a path graph (a line of n vertices) the entire tree is a single heavy chain, so every path query is a *single* segment-tree range and costs $O(\log n)$, not $O(\log^2 n)$. Conversely, construct a tree where some path query truly forces $\Theta(\log n)$ distinct chain ranges, showing the $O(\log^2 n)$ bound is tight in the worst case.

Exercise 19.11 (Iterative build)**.** Rewrite DfsSize and DfsHld (Algorithm 74) iteratively with an explicit stack so they survive a path graph of $n = 10^6$ without stack overflow. Verify your positions and tops match the recursive version on the running tree.

Exercise 19.12 (Forest, shared array)**.** You are given a forest. Adapt the layout so all components share one position array (one segment tree), with each component's vertices contiguous. Show path queries within a component still cost $O(\log^2 n)$ and that no query ever crosses components.

Chapter 20

Centroid Decomposition

Heavy-light decomposition, in the previous chapter, took a tree and sliced it into chains so that root-to-vertex paths cross only $O(\log n)$ of them. That is the right tool when each query follows a single path in the tree – update a vertex, ask for the maximum on the path from u to v. Many tree problems, however, ask about *all* paths at once. How many pairs of vertices are at distance exactly k? For each vertex, what is the closest red vertex anywhere in the tree? Add a vertex to a set; report the nearest set-member to a query vertex. These ask about global path structure across the whole tree, where a chain decomposition does not help.

Centroid decomposition handles this class of problems. The idea is simple: find a vertex whose removal splits the tree into pieces each no larger than half the tree – a *centroid* – delete it, and recurse on the pieces. Because every split at least halves the part you recurse into, the recursion is only $O(\log n)$ deep. The centroids, linked parent-to-child by the recursion, form a new tree of logarithmic height: the *centroid tree*. Its key property is that *every* path in the original tree passes through the centroid-tree ancestor of its two endpoints. That single fact converts "reason about all $\binom{n}{2}$ paths" into "for each of n centroids, reason about the paths that pass through it," and the halving guarantees the bookkeeping stays cheap.

We will define centroids and prove they always exist, build the decomposition and prove its depth bound, then focus on the technique that makes the decomposition so useful: solving path and distance problems by attributing each path to the unique centroid that owns it. A small tree is traced by hand throughout.

20.1 Tree centroids

Throughout, $T = (V, E)$ is a tree on $n = |V|$ vertices: connected, acyclic, $n - 1$ edges. Deleting a vertex v from a tree disconnects it into one component per neighbor of v; we call these the *branches* hanging off v, and write their sizes as $s_1, \ldots, s_{\deg v}$ with $\sum_i s_i = n - 1$.

Definition 20.1 (Centroid)**.** A vertex $c \in V$ is a *centroid* of T if removing c leaves every resulting component with at most $\lfloor n/2 \rfloor$ vertices. Equivalently, every branch hanging off c has size at most $\lfloor n/2 \rfloor$.

Intuition

A centroid is the tree's balance point. Stand on a vertex and look down each branch; if one branch holds more than half of everything, you are off-center – step toward that heavy branch and you improve. A centroid is the place where no single step toward any branch helps, because no branch is heavy enough to be worth walking into. Contrast the *root* of a tree, which you pick arbitrarily and which can be wildly unbalanced (a path rooted at its end). The centroid is forced by the tree's shape, not chosen.

Here is the small tree we will use for the entire chapter, shown in Figure 20.1. It has $n = 9$ vertices.

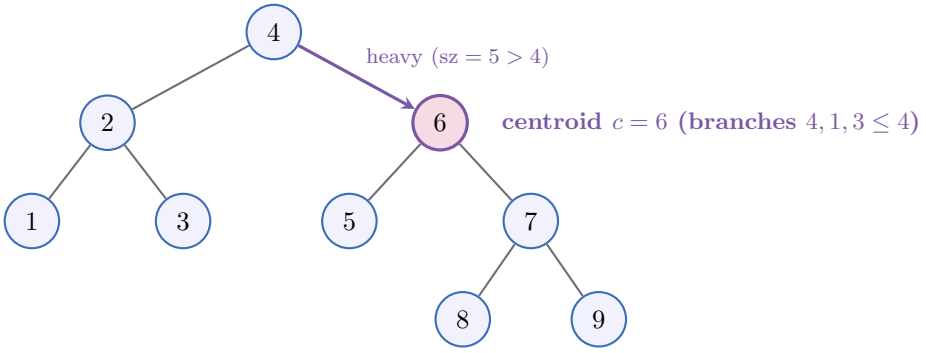


Figure 20.1: The running 9-vertex tree. Removing node 4 leaves a heavy component of size 5 ($\{5, 6, 7, 8, 9\}$). Stepping toward the heavy branch into node 6 yields three components of sizes 4 ($\{1, 2, 3, 4\}$), 1 ($\{5\}$), and 3 ($\{7, 8, 9\}$) – each at most $\lfloor 9/2 \rfloor = 4$. Thus vertex 6 is the centroid.

The edge set, pinned down so the hand-traces are unambiguous: 4 is adjacent to 2 and 6; 2 is adjacent to $1, 3, 4$; 6 is adjacent to $4, 5, 7$; 7 is adjacent to $6, 8, 9$.

Let us locate a centroid by hand. Total $n = 9$, so a branch is "heavy" if it holds more than $\lfloor 9/2 \rfloor = 4$ vertices. Try vertex 4: removing it leaves the component $\{2, 1, 3\}$ of size 3 and the component $\{6, 5, 7, 8, 9\}$ of size 5. The second branch has $5 > 4$ vertices, so 4 is *not* a centroid; we should step toward the heavy side, into 6. Remove 6: components are $\{4, 2, 1, 3\}$ (size 4), $\{5\}$ (size 1), $\{7, 8, 9\}$ (size 3). Every piece is ≤ 4, so **vertex 6 is the centroid**.

Why it works

The "walk toward the heavy branch" idea is the existence proof. Define a potential $\Phi(v) =$ size of the largest branch hanging off v. If v is not a centroid, some branch B has $|B| > \lfloor n/2 \rfloor \geq (n-1)/2$, hence $|B| \geq \lfloor n/2 \rfloor + 1 > n - |B|$, so B holds a strict majority. Step from v to its neighbor w inside B. From w's vantage, the branch back toward v contains everything outside B plus v itself: $n - |B| < |B|$ vertices, strictly fewer than $\Phi(v)$. The other branches off w are sub-pieces of $B \setminus \{w\}$, each strictly smaller than $|B| = \Phi(v)$. So $\Phi(w) < \Phi(v)$. The potential is a positive integer that strictly decreases each step, so the walk cannot loop and must halt – and it can only halt at a centroid.

Theorem 20.1 (Existence of a centroid)**.** *Every tree on $n \geq 1$ vertices has at least one centroid. It has at most two, and if it has two they are adjacent.*

Proof. Existence. The walk in the box above starts at any vertex and strictly decreases the integer

potential Φ each step, so after finitely many steps it reaches a vertex with no heavy branch – a centroid.

At most two, and adjacent. Suppose c is a centroid and $c' \neq c$ is also a centroid. Root the tree at c and let B be the branch off c that contains c'. Because c is a centroid, $|B| \leq \lfloor n/2 \rfloor$. Now view things from c': one of c''s branches is the part of the tree on c's side, which has size $n - |\text{subtree}(c')|$ where the subtree is taken in the c-rooted tree. Since c' is a centroid this side has size $\leq \lfloor n/2 \rfloor$, giving $|\text{subtree}(c')| \geq n - \lfloor n/2 \rfloor = \lceil n/2 \rceil$. But $\text{subtree}(c') \subseteq B$, so $\lceil n/2 \rceil \leq |B| \leq \lfloor n/2 \rfloor$, forcing $|B| = \lceil n/2 \rceil = \lfloor n/2 \rfloor$, which needs n even and pins the subtree of c' to be exactly B. Then the c-to-c' branch has size exactly $n/2$ on both ends, and any vertex strictly between c and c' on the path would push the larger side past $n/2$ – contradiction unless c and c' are adjacent. □

Common trap

The centroid is not the same as the *center* (the vertex minimizing eccentricity, i.e. the middle of a longest path), nor the *barycenter* of a weighted embedding. On the path graph 1–2–3–4–5 all three coincide at 3, which lulls people into thinking they are interchangeable. They are not: take a star with one long whisker and the center and centroid drift apart. For centroid decomposition you specifically need the size-balance property of Definition 20.1; eccentricity is irrelevant.

Pause

Pause and think: why does a tree with an odd number of vertices have a unique centroid? If T had two distinct centroids c and c', the edge (c, c') would partition T into two components of size $n/2$ each. But if n is odd, $n/2$ is not an integer, so a tree with an odd number of vertices has a unique centroid.

20.1.1 Finding a centroid in linear time

The walk-toward-the-heavy-branch argument is constructive but a little awkward to code directly. The clean implementation roots the (sub)tree arbitrarily, computes subtree sizes with one DFS, then in a second pass walks down from the root always stepping into a child whose subtree exceeds half, stopping when none does.

Algorithm 78: Find the centroid of a tree (or subtree) of size m

Input: Tree given by adjacency lists; a start vertex r; a removed[$\cdot$] flag marking deleted vertices; the count m of live vertices reachable from r.
Output: A centroid of that component.

```
1  procedure SubtreeSizes(u, p)
2      sz[u] ← 1
3      foreach w ∈ adj(u) with w ≠ p and not removed[w] do
4          sz[u] ← sz[u] + SubtreeSizes(w, u)
5      return sz[u]
6  procedure FindCentroid(u, p, m)
7      foreach w ∈ adj(u) with w ≠ p and not removed[w] do
8          if sz[w] > m/2 then
9              return FindCentroid(w, u, m)
10     return u
11 SubtreeSizes(r, ⊥)
12 return FindCentroid(r, ⊥, m)
```

Why it works

After SubtreeSizes$(r, \cdot)$, sz$[w]$ is the size of w's subtree in the r-rooted component. At a vertex u, the branch hanging off u toward parent p has size $m - \text{sz}[u]$, and each child-branch has size sz$[w]$. FindCentroid descends only into a child whose subtree $> m/2$. Two facts make this correct. First, at most one child can have sz$[w] > m/2$, since two such would already overflow m; so the descent is deterministic. Second, when we step from u into such a w, the new parent-branch (back toward u) has size $m - \text{sz}[w] < m/2$, so it can never again be the heavy branch – we only ever move "inward" and sizes shrink. Termination at u means no child exceeds $m/2$ and the parent-branch $m - \text{sz}[u] \le m/2$, so every branch off u is $\le \lfloor m/2 \rfloor$ and u is a centroid. Both passes are a single DFS over the m live vertices: $O(m)$ time.

20.2 The decomposition and its depth

Now recurse. Find a centroid c of the whole tree, record it as the root of the centroid tree, mark it removed, and recurse on each remaining component; the centroid found in a component becomes a *child* of c in the centroid tree.

Algorithm 79: Build the centroid decomposition

Input: Tree $T = (V, E)$ as adjacency lists.
Output: par[·]: parent of each vertex in the centroid tree.

```
foreach v ∈ V do
    removed[v] ← FALSE
procedure DECOMPOSE(r, cpar)
    m ← number of live vertices in r's component (one DFS)
    c ← FINDCENTROID(r, ⊥, m)                          // Algorithm 78
    par[c] ← cpar
    removed[c] ← TRUE
    foreach w ∈ adj(c) with not removed[w] do
        DECOMPOSE(w, c)
DECOMPOSE(any vertex, ⊥)
return par
```

20.2.1 Tracing it on the nine-vertex tree

Trace it yourself

Run Algorithm 79 on our tree. We found the top centroid is 6. Remove 6; three live components remain.

- Component $\{4, 2, 1, 3\}$, $m = 4$, "heavy" threshold > 2. Root at 4: subtree of 2 is $\{2, 1, 3\}$, size $3 > 2$, step into 2; from 2 the parent-branch is $\{4\}$ size 1, children $\{1\}, \{3\}$ size 1 each – none > 2. **Centroid** $= 2$, child of 6. Recurse: removing 2 gives $\{4\}, \{1\}, \{3\}$, each a single vertex and its own centroid, children of 2.
- Component $\{5\}$, $m = 1$. **Centroid** $= 5$, child of 6 (a leaf).
- Component $\{7, 8, 9\}$, $m = 3$, threshold > 1. Root at 7: children $\{8\}, \{9\}$ size 1, parent-branch empty. **Centroid** $= 7$, child of 6. Recurse: $\{8\}, \{9\}$ become children of 7.

The recursive component splitting process and the resulting balanced centroid tree are shown in Figure 20.2.

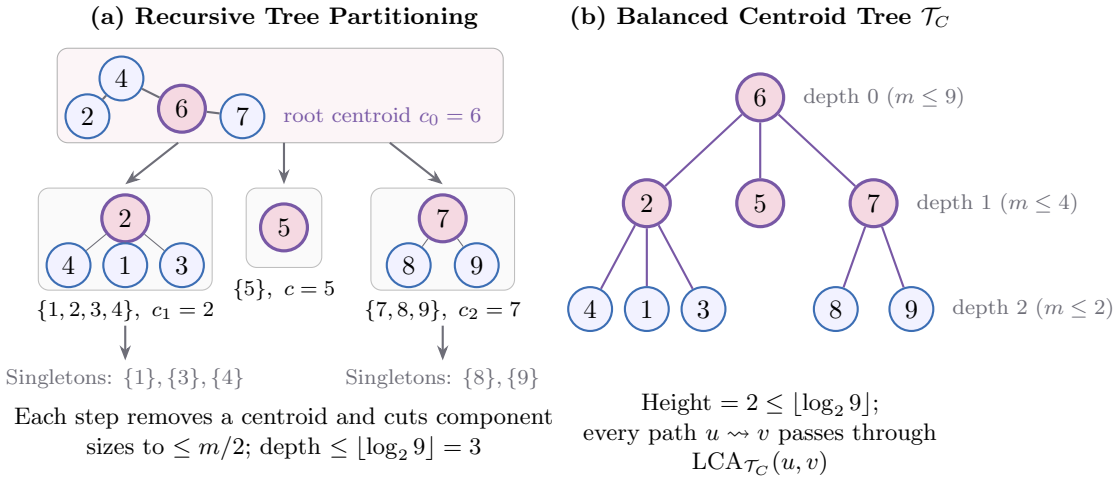


Figure 20.2: The centroid tree recursion hierarchy. (a) Recursive decomposition: at each stage, removing the centroid c guarantees that all disconnected components have at most $\lfloor m/2 \rfloor$ vertices, driving logarithmic recursion depth. (b) The resulting centroid tree $\mathcal{T}_C$ has height $2 \leq \log_2 n$. The unique lowest common ancestor $a = \mathrm{LCA}_{\mathcal{T}_C}(u, v)$ in the centroid tree is the unique centroid that separates and owns the path between u and v.

Notice this is a different tree on the same nine vertices. It is broad and shallow: height 2 (edges $6 \to 2 \to 4$). The original tree had height 3. In general the centroid tree's height is $O(\log n)$, which we now prove.

Theorem 20.2 (Logarithmic depth)**.** *The centroid tree produced by Algorithm 79 has height at most $\lfloor \log_2 n \rfloor$. Consequently every vertex has at most $\lfloor \log_2 n \rfloor + 1$ ancestors in the centroid tree (itself included).*

Proof. When DECOMPOSE processes a component of size m and removes its centroid c, every child component has size $\leq \lfloor m/2 \rfloor$ by Definition 20.1. The component handled at centroid-tree depth d (root has depth 0) therefore has size at most $\lfloor n/2^d \rfloor \leq n/2^d$. A component must contain at least one vertex to exist, so $n/2^d \geq 1$, i.e. $d \leq \log_2 n$. Depth is an integer, so $d \leq \lfloor \log_2 n \rfloor$. □

Cost summary

Build time. Let $T(n)$ be the work to decompose a component of size n. Finding the centroid and counting the component is $O(n)$ (one DFS), and the children's sizes sum to $\leq n-1$. So the total work at each *level* of the centroid tree is $O(\text{sum of component sizes at that level}) = O(n)$, and there are $O(\log n)$ levels (Theorem 20.2). Hence

$$T(n) = O(n \log n) \quad \text{time}, \qquad O(n) \text{ extra space.}$$

The recurrence $T(n) \leq O(n) + \sum_i T(s_i)$ with $\sum_i s_i \leq n - 1$ and each $s_i \leq n/2$ solves to the same bound. The centroid tree itself stores one parent pointer per vertex: $O(n)$.

Common trap

A common bug: computing component sizes by running DFS over the *whole original tree* each time instead of only the live component. If SubtreeSizes ignores removed and walks the full tree, every call is $O(n)$ and you do n of them in the worst case (a path), so build degrades to $O(n^2)$. The $O(n \log n)$ bound relies on each DFS touching only *currently live* vertices, which is why the removed[·] guard appears in every loop of Algorithm 78. Always re-measure the local component; never the global tree.

Pause

Pause and think: why does each level of decomposition take $O(n)$ work? At any single recursion depth, the tree is partitioned into disjoint components whose vertex sets partition a subset of V. The DFS to find centroids and subtree sizes runs in time proportional to each component's size. Summing across all components at that depth, the work is at most $\sum O(|V_i|) = O(n)$. With $O(\log n)$ levels, the total construction time is $O(n \log n)$.

History

Centroids of trees were first identified by Camille Jordan in 1869, who proved that every tree possesses at least one center and centroid while studying the symmetry groups of chemical isomers – the same French mathematician known for the Jordan curve theorem. The recursive decomposition method emerged in the 1980s as the simplest 1D case of graph separator theory (formalized by Richard Lipton and Robert Tarjan). In the 2000s, algorithmic problem solvers popularized the centroid tree data structure for path-counting and nearest-marked-node queries, turning quadratic tree searches into $O(n \log n)$ divide-and-conquer passes. This led to fast algorithms for dynamic diameter and distributed network routing. An open challenge remains designing fully dynamic centroid trees that support edge insertions and deletions in sub-polynomial update time without rebuilding entire subtrees from scratch.

20.3 Solving path problems by decomposition

The construction above exists to support the following lemma, which is what makes the technique work.

Lemma 20.1 (Path–centroid lemma)**.** *Let $u, v \in V$ and let a be their lowest common ancestor in the centroid tree. Then the unique tree path $u \rightsquigarrow v$ in T passes through a, and a is the* only *vertex on that path that is an ancestor of both u and v in the centroid tree.*

Proof. Consider the moment Decompose acts on the component that contains both u and v and is about to remove its centroid. While $a = \mathrm{lca}(u, v)$ has not yet been removed, u and v live in the same component (that is what being descendants of the same centroid-tree node means). At the step that removes a, either $a \in \{u, v\}$, or a splits this component so that u and v fall into *different* child components – otherwise their lowest common ancestor would lie deeper, inside that single child, contradicting $a = \mathrm{lca}(u, v)$. In a tree, if deleting a separates u from v, then a lies on the path $u \rightsquigarrow v$ (the path is the only u–v route, and it is cut). So a is on the path.

Uniqueness: suppose some other vertex $b \neq a$ on the path were a common ancestor of u and v in the centroid tree. A common ancestor of u and v is, in particular, an ancestor of $a = \mathrm{lca}(u, v)$ or equals a; if $b \neq a$ then b is a strict ancestor of a, so b was removed at a shallower centroid level, *before* a. But at the moment b was removed, u and v still lay in the same component (they are removed even later), so b did not separate them, hence b is not on the path $u \rightsquigarrow v$ – contradiction. Thus a is the only such vertex. □

Intuition

Read the lemma as an *ownership* rule. Every one of the $\binom{n}{2}$ paths in T is owned by exactly one centroid: the highest centroid that the path touches, equivalently lca of its endpoints in the centroid tree. If you want to process all paths, process each centroid c once and handle only the paths that pass through c and live entirely within c's component. No path is missed (every path has an owner) and no path is double-counted (the owner is unique). The $O(\log n)$ depth means each vertex belongs to only $O(\log n)$ components, so the total "path through c" work, summed over all centroids, stays small.

The standard recipe. For each centroid c, every relevant path through c is the concatenation of two "arms": a path from c down into one branch, and a path from c down into another (or the trivial empty arm ending at c). Distances along an arm are just distances to c inside c's component, which one BFS/DFS from c computes. The problem then becomes: combine arms from *different* branches. Two classic combination patterns follow.

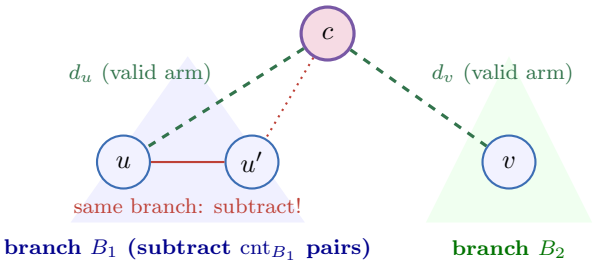


Figure 20.3: A path owned by centroid c splits into two arms in distinct branches: $d_u = \mathrm{dist}(c, u)$ and $d_v = \mathrm{dist}(c, v)$, with total length $d_u + d_v$. Two arms u, u' in the *same* branch B_1 do not form a simple path through c and are removed by the per-branch inclusion–exclusion subtraction.

20.3.1 Pattern A: counting paths of a given length

Problem: count unordered pairs $\{u, v\}$ with $\mathrm{dist}_T(u, v) = k$ (here dist is the number of edges). Each such pair is owned by one centroid $a = \mathrm{lca}(u, v)$, and the path splits at a into two arms of lengths $d_u = \mathrm{dist}(a, u)$ and $d_v = \mathrm{dist}(a, v)$ with $d_u + d_v = k$, the two arms living in different branches off a (or one arm empty when $a \in \{u, v\}$); see Figure 20.3.

So at centroid c: run a DFS to get, for every live vertex x, its depth $\mathrm{dist}(c, x)$ and which branch of c it lies in. Build a histogram $\mathrm{cnt}[d]$ = number of live vertices at depth d from c. The number of *ordered* arm pairs summing to k is $\sum_d \mathrm{cnt}[d] \cdot \mathrm{cnt}[k-d]$, but this overcounts: it includes pairs of arms in the *same* branch (whose true path does not pass through c). Subtract them with the standard inclusion–exclusion: for each branch B, build $\mathrm{cnt}_B[\cdot]$ and subtract $\sum_d \mathrm{cnt}_B[d] \cdot \mathrm{cnt}_B[k-d]$. Finally divide the ordered count by two.

Algorithm 80: Count pairs at distance exactly k via centroid decomposition

Input: Tree T; target distance k.
Output: ans $= |\{\{u, v\} : \text{dist}_T(u, v) = k\}|$.

```
ans ← 0
procedure CountPaths(cnt[·], sign)
    // add sign · (ordered arm pairs summing to k)
    for d ← 0 to k do
        ans ← ans + sign · cnt[d] · cnt[k − d]
procedure Solve(r)
    m ← size of r's live component
    c ← FindCentroid(r, ⊥, m); removed[c] ← true
    cnt ← depth histogram of whole component from c (cnt[0] = 1 for c)
    CountPaths(cnt, +1)
    foreach branch B off c (each live neighbor's subtree) do
        cnt_B ← depth histogram of B from c (depths ≥ 1)
        CountPaths(cnt_B, −1)
    foreach w ∈ adj(c) with not removed[w] do
        Solve(w)
Solve(any vertex)
return ans/2
```

Why it works

The whole-component histogram counts every ordered pair of arms summing to k, including illegal same-branch pairs and including the cnt$[0] = 1$ slot for c itself. Subtracting each branch's self-pairs removes exactly the same-branch pairs: two arms in branch B both contribute to cnt_B and get cancelled. What survives is cross-branch ordered pairs, plus the two ordered pairs (c, x) and (x, c) for each x at depth k – those use the $d = 0$ slot, which is present only in the full histogram (each cnt_B starts at depth 1). So a distance-k path with one endpoint $= c$ is also counted twice. Each legitimate path through c is thus counted exactly twice, and the final division by 2 is exact. By Lemma 20.1 every distance-k pair is owned by exactly one centroid, so summing over centroids counts each pair exactly once.

Cost summary

At centroid c the work is $O(m_c)$ for the DFS plus $O(\min(k, m_c))$ per histogram convolution; summed naively the convolutions cost $O(\sum_c k)$, which can be $O(nk)$. Capping each histogram at the branch's max depth makes total convolution work $O(\sum_c m_c) = O(n \log n)$, since each vertex appears in $O(\log n)$ components and contributes $O(1)$ to one histogram slot per component. Overall $O(n \log n)$ time, $O(n)$ space – versus the $O(n^2)$ of checking all pairs.

Trace it yourself

Trace Algorithm 80 with $k = 2$ at the top centroid $c = 6$. DFS from 6 over the whole tree gives depths dist$(6, \cdot)$: $6 : 0$; $4, 5, 7 : 1$; $2, 8, 9 : 2$; $1, 3 : 3$. Histogram cnt $= [1, 3, 3, 2]$ for depths $0, 1, 2, 3$. Ordered pairs summing to $k = 2$ from the full histogram: cnt$[0]$cnt$[2] +$ cnt$[1]$cnt$[1] +$

cnt[2]cnt[0] $= 3 + 9 + 3 = 15$. Now the three branches off 6. Branch $\{4, 2, 1, 3\}$ has depths 4:1, 2:2, 1:3, 3:3, so cnt_B has 1 at depth 1, 1 at depth 2, 2 at depth 3; ordered pairs summing to 2: $\text{cnt}_B[1]^2 = 1$. Branch $\{5\}$: depth 1, gives 1. Branch $\{7, 8, 9\}$: depths $7:1$, $8:2$, $9:2$, gives $\text{cnt}_B[1]^2 = 1$. Subtract $1 + 1 + 1 = 3$, leaving $15 - 3 = 12$ ordered pairs owned by 6, i.e. 6 unordered distance-2 pairs through 6: the three "via 6" pairs $\{4, 5\}, \{4, 7\}, \{5, 7\}$ and the three depth-2 endpoints-with-6 pairs $\{6, 2\}, \{6, 8\}, \{6, 9\}$. The remaining distance-2 pairs ($\{1, 3\}$ via 2, $\{8, 9\}$ via 7) are owned by the deeper centroids 2 and 7 and are counted when SOLVE recurses there. Summing each centroid's contribution and dividing by 2 gives the total.

20.3.2 Pattern B: nearest marked vertex under updates

Problem: maintain a set $S \subseteq V$ (initially empty); support MARK(v) adding v to S, and QUERY(v) returning $\min_{s \in S} \text{dist}_T(v, s)$. This is the standard *online* centroid-tree application, and it uses the centroid tree as a data structure rather than for a one-shot count.

Store at each centroid c a value $\text{best}[c] = \min_{s \in S \cap \text{comp}(c)} \text{dist}_T(c, s)$, the distance from c to the nearest marked vertex in c's component (where $\text{comp}(c)$ is the set of vertices for which c is a centroid-tree ancestor). To support distance lookups we precompute, for every vertex x and every centroid-tree ancestor a of x, the value $\text{dist}_T(x, a)$ – there are only $O(\log n)$ ancestors per vertex, so this is an $n \times O(\log n)$ table built during the decomposition, where each centroid's BFS over its live component records $\text{dist}_T(x, c)$ for every x it reaches.

Algorithm 81: Nearest marked vertex (Pattern B)

Data: par[·] centroid tree; $D[x][a] = \text{dist}_T(x, a)$ for each centroid-ancestor a of x; best[c] $\leftarrow +\infty$ for all c.

```
1  procedure MARK(v)
2      a ← v
3      while a ≠ ⊥ do
4          best[a] ← min(best[a], D[v][a])
5          a ← par[a]
6  procedure QUERY(v)
7      a ← v; res ← +∞
8      while a ≠ ⊥ do
9          res ← min(res, D[v][a] + best[a])
10         a ← par[a]
11     return res
```

Why it works

MARK(v) walks v's $O(\log n)$ centroid ancestors and tells each ancestor a "a marked vertex now sits at distance $D[v][a]$ from you," updating best[a]. QUERY(v) walks the same ancestors and, for each a, considers routing through a: the nearest marked vertex *in a's component* is best[a] away from a, and v is $D[v][a]$ away from a, giving a candidate path of length $D[v][a] + \text{best}[a]$. Correctness is exactly Lemma 20.1: the true nearest marked vertex s^* and v have a centroid-tree lca a^* lying on the v–s^* path, so $\text{dist}_T(v, s^*) = \text{dist}_T(v, a^*) + \text{dist}_T(a^*, s^*) =$

$D[v][a^*] + \text{dist}_T(a^*, s^*) \geq D[v][a^*] + \text{best}[a^*]$, and the query examines a^*. Every candidate $D[v][a] + \text{best}[a]$ is the length of an actual walk $v \to a \to s$, hence an upper bound on $\text{dist}_T(v, s) \geq \min_{s \in S} \text{dist}_T(v, s)$; and the bound is met at $a = a^*$. So the query returns the exact minimum. Each operation is $O(\log n)$.

Common trap

The classic mistake is to store, at best$[c]$, the nearest marked vertex *in the original subtree of* c (an HLD-style rooted-subtree notion) rather than in c's *centroid component* – the set of vertices for which c is a centroid ancestor. Those are different sets, and using the wrong one makes Query miss valid marked vertices. A second hazard is an inexact distance table: $D[x][a]$ must be the *true* tree distance, computed by each centroid's BFS over its live component during the build. If D is right, the candidate $D[v][a] + \text{best}[a]$ is always a real walk length, so the algorithm cannot overshoot; the only way to a wrong answer is a wrong D.

Pause

Pause and think: why do we only check centroid ancestors during nearest marked queries? For any marked node m and query node v, their path passes through their lowest common ancestor $a = \text{LCA}_{\text{centroid}}(m, v)$. By definition, a is a centroid-tree ancestor of m (so best$[a]$ records $D[m][a]$) and also an ancestor of v. Thus, testing all centroid-tree ancestors of v is guaranteed to examine the true distance $D[v][a] + D[m][a]$ without scanning the rest of the tree.

Trace it yourself

On our tree, Mark(1). Vertex 1's centroid ancestors are $1 \to 2 \to 6$ (parents read off the centroid tree). The true tree distances: $D[1][1] = 0$, $D[1][2] = \text{dist}_T(1, 2) = 1$ (edge 1-2), and $D[1][6] = \text{dist}_T(1, 6) = 3$ (path $1-2-4-6$). So after the mark, best$[1] = 0$, best$[2] = 1$, best$[6] = 3$. Now Query(9): ancestors $9 \to 7 \to 6$ with $D[9][9] = 0$, $D[9][7] = 1$ (edge 9-7), $D[9][6] = 2$ (path $9-7-6$). Candidates: $D[9][9] + \text{best}[9] = 0 + \infty$; $D[9][7] + \text{best}[7] = 1 + \infty$; $D[9][6] + \text{best}[6] = 2 + 3 = 5$. The answer 5 matches the tree path $9-7-6-4-2-1$, which has 5 edges. The two arms $9-7-6$ and $6-4-2-1$ are vertex-disjoint except at the meeting centroid 6, which is precisely why adding $D[9][6]$ and best$[6]$ gives the true distance and not an overcount (Figure 20.4).

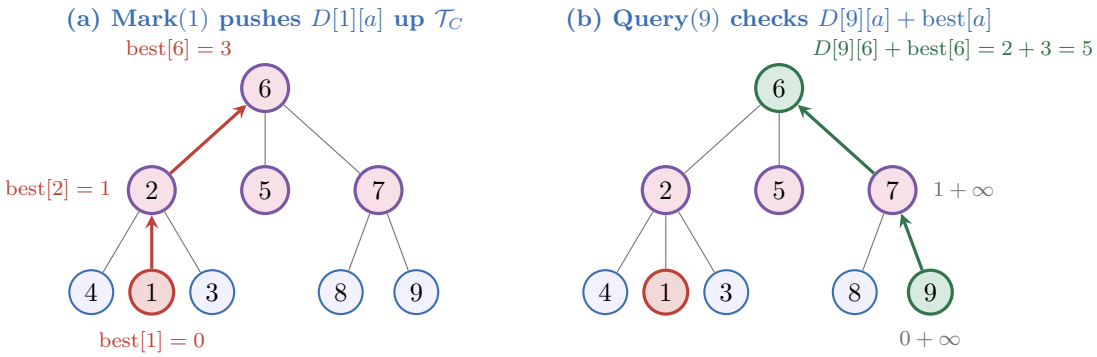


Figure 20.4: Tracing Pattern B on the centroid tree $\mathcal{T}_C$. (a) Mark(1) climbs $1 \to 2 \to 6$ and records the true tree distances best[1] $= 0$, best[2] $= 1$, and best[6] $= 3$. (b) Query(9) climbs $9 \to 7 \to 6$ and meets the marked ancestor chain at $a^* = \mathrm{LCA}_{\mathcal{T}_C}(1, 9) = 6$, returning $D[9][6] + \text{best}[6] = 2 + 3 = 5$.

20.4 The centroid tree as a query structure

The centroid tree is a balanced auxiliary tree on the same vertex set with three important properties.

1. **Logarithmic height** (Theorem 20.2): every vertex has $O(\log n)$ ancestors, so per-vertex ancestor walks are cheap.
2. **Path ownership** (Lemma 20.1): the lca in the centroid tree lies on the original tree path and uniquely owns it.
3. **Component nesting**: the components processed down a root-to-vertex path of the centroid tree are nested and shrink by at least half, so each vertex lives in only $O(\log n)$ components.

Together they give a general template: precompute $D[x][a]$ (distance to each centroid ancestor) during the build; keep an aggregate agg$[c]$ at each centroid summarizing the marked/active vertices in c's component; an update walks $O(\log n)$ ancestors pushing information up, a query walks the same ancestors pulling it down and combining via the path-through-c decomposition. Mark/Query above is the additive instance; with a Fenwick tree or balanced multiset at each centroid you get range-distance counts, k-th nearest, sum of distances within radius r, and so on, all in $O(\log^2 n)$ per operation.

Application

Centroid decomposition is the standard tool behind several recurring problems:

- **"Count paths of length k"** and weighted variants (count paths with weight $\le W$, using a Fenwick over arm lengths) – Pattern A.
- **Nearest active facility on a tree under toggles** (telecom or CDN placement on a tree-shaped network) – Pattern B.
- **Sum of pairwise distances** $\sum_{u<v} \mathrm{dist}_T(u, v)$: a single pass attributing each path to its centroid.
- **Point updates with path-aggregate queries between arbitrary pairs** where the

queries are not along a single tree path (so HLD does not apply).

To choose between this and heavy-light decomposition: use HLD when queries follow one tree path; use centroid decomposition when queries range over many paths or ask "nearest / all within distance."

Connection

A centroid is the cheapest *vertex separator* of a tree – one vertex whose deletion balances the pieces. The planar-separator and treewidth chapters generalize this: a planar graph has an $O(\sqrt{n})$-vertex balanced separator, and recursing on separators (separator decomposition) is the exact analogue of the centroid tree, giving nested-dissection orderings for fast linear solves. The $O(\log n)$-depth recursion tree here is the simplest member of that family. The centroid tree's logarithmic height also makes it a convenient base for the Euler-tour and LCA structures of the next chapters. And when you later see Laplacian solvers built on recursive vertex elimination, recall that eliminating a balanced separator first is precisely what keeps fill-in – and cost – under control: centroid decomposition is the tree-only special case of that idea.

20.5 Weighted and rooted variants

Two extensions appear constantly and need only small changes.

Edge weights. If edges carry nonnegative lengths and "distance" means summed weight, nothing structural changes: centroids are still defined by *vertex counts* (so the $O(\log n)$ depth is untouched), and the only edit is that the BFS from each centroid becomes a weighted DFS accumulating edge lengths into $D[x][c]$. Pattern A's histogram becomes indexed by weight, often needing a Fenwick tree to count arms with weight in a range; Pattern B is unchanged.

Counting paths with length in a range $[\ell, r]$. Replace the exact-k convolution with a two-pointer or Fenwick query: for each arm of length a, count opposite arms of length in $[\ell - a, r - a]$. Same inclusion–exclusion over branches.

Formal

Why vertex-count balance, not weight balance, is the right invariant. The depth bound (Theorem 20.2) needs each child component to have at most half as many *vertices*, because the recursion bottoms out when a component has one vertex. If you tried to balance by total edge weight, a tree with one enormously heavy edge could force unbalanced vertex splits and blow the depth to $\Theta(n)$. Keep the balancing criterion on counts; let weights live only in the distance values $D[\cdot][\cdot]$.

Summary

A centroid splits a tree into pieces of at most half the size; recursing on the pieces builds the centroid tree, a balanced auxiliary tree of height $O(\log n)$ computable in $O(n \log n)$. Its defining property is that the centroid-tree lca of any two vertices lies on – and uniquely owns – the tree path between

them. That ownership turns global "all paths" questions into per-centroid local work, and the logarithmic depth keeps the bookkeeping at $O(n \log n)$ for offline counting (Pattern A) or $O(\log n)$ per operation for online nearest-marked queries (Pattern B). Use it whenever tree queries range over many paths rather than following a single one.

Where We Go Next (Euler Tour Technique): In Chapters 19 and 20, we developed two divide-and-conquer decompositions: heavy paths for intervalizing individual path queries, and centroid trees for balancing all-paths divide-and-conquer. But what if we do not need path queries, but instead want to query entire subtrees or compute lowest common ancestors as simply as possible? In Chapter 21, we introduce the *Euler Tour Technique*, flattening a tree into a 1D sequence via a single depth-first traversal and turning entire subtrees into contiguous array intervals.

20.6 Exercises

Exercise 20.1 (Quick mental check: balance factor)**.** If a tree has $n = 10$ vertices, what is the maximum size of any component left when the centroid is deleted? If $n = 11$?

Exercise 20.2 (Quick mental check: centroid tree height)**.** If a tree is a linear path graph of $n = 1024$ vertices, what is the height of its centroid tree? Why does maximal tree unbalance not hurt centroid recursion depth?

Exercise 20.3 (Implement the build)**.** Implement Algorithm 79 and verify on random trees that the resulting centroid tree has height $\leq \lfloor \log_2 n \rfloor$. Instrument the code to assert that every SubtreeSizes call touches only live vertices, and measure total DFS work to confirm it is $O(n \log n)$, not $O(n^2)$.

Exercise 20.4 (Two centroids)**.** Sharpen Theorem 20.1: characterize exactly which trees have two centroids. (Hint: an edge whose removal splits the tree into two halves of size $n/2$ each; what must n be?) Give the smallest tree with two centroids and the smallest with a unique centroid, for each parity of n.

Exercise 20.5 (Sum of all pairwise distances)**.** Use centroid decomposition to find $\sum_{u<v} \text{dist}_T(u, v)$ in $O(n \log n)$ time. Then find an $O(n)$ algorithm using a single tree DP (each edge contributes $\text{size} \cdot (n - \text{size})$). Explain when each approach is preferable and why the simpler $O(n)$ method does not generalize to "distance exactly k."

Exercise 20.6 (Break the naive size count)**.** Construct a family of trees on which a centroid-decomposition implementation that recomputes subtree sizes over the *entire* tree at every level runs in $\Theta(n^2)$. Confirm empirically, then fix it with the live-component guard and show the curve flattens to $O(n \log n)$.

Exercise 20.7 (Paths in a length range)**.** Extend Algorithm 80 to count pairs with $\text{dist}_T(u, v) \in [\ell, r]$ using a Fenwick tree over arm lengths instead of an exact-k convolution. State and justify the resulting running time, including the extra logarithmic factor.

Exercise 20.8 (Weighted nearest marked)**.** Adapt Pattern B (Algorithm 81) to nonnegative edge weights. Prove that the $D[x][a]$ table, built by a weighted traversal from each centroid, still has only $O(\log n)$ entries per vertex, and that Query remains exact. Where does the proof use Lemma 20.1?

Exercise 20.9 (k-th nearest marked vertex)**.** Augment each centroid with a sorted multiset (or Fenwick over distances) of the distances from c to marked vertices in its component, so Query(v)

can return the number of marked vertices within distance r of v. Achieve $O(\log^2 n)$ per operation and prove the bound (one logarithm for centroid ancestors, one for the per-centroid structure). Watch for double counting across overlapping centroid components and explain how a per-branch correction avoids it.

Exercise 20.10 (Centroid vs. center)**.** Give a tree where the centroid and the center (the eccentricity minimizer) are different vertices, and one where they coincide. Conclude with an explicit example that the center is not a substitute for the centroid in the decomposition: exhibit a tree whose center, used as the split vertex, produces a child component of size $> n/2$ and thus a recursion deeper than $O(\log n)$.

Exercise 20.11 (Why static)**.** Centroid decomposition as built here is static – inserting or deleting a tree edge can change centroids globally. Show that a single edge change can move the top centroid arbitrarily far, sketch why a fully dynamic centroid decomposition is hard, and contrast with link-cut trees (later chapters) which *do* support edge updates. For which of Pattern A / Pattern B does staticness matter, and why?

Exercise 20.12 (Lower-bound the depth)**.** Show the height bound $\lfloor \log_2 n \rfloor$ is essentially tight: exhibit a tree whose centroid tree has height $\Theta(\log n)$ (e.g. a complete binary tree), and argue no tree forces height $\omega(\log n)$. Then show a *path* graph still yields centroid-tree height $\Theta(\log n)$ despite being maximally unbalanced as a rooted tree – explaining why "unbalanced as a rooted tree" and "deep centroid decomposition" are unrelated notions.

Chapter 21

The Euler Tour Technique

The previous chapters split trees at centroids and into heavy-light chains. Many tree tasks need less than that: they only inspect or update an entire subtree at once.

A tree branches in two dimensions, with subtrees nested within subtrees. An array is one-dimensional: a flat sequence of cells that supports instant slicing and interval queries. The Euler tour technique converts one into the other. It takes a rooted tree, walks around it in a single fixed order, and writes down what it sees, producing an array in which *every subtree occupies one contiguous block*. Once a subtree is a contiguous block, "update everything below v" and "sum everything below v" become range operations, and you can deploy the full range of array data structures – prefix sums, Fenwick trees (BITs), segment trees, sparse tables – at problems that were phrased on a tree.

The same walk, recorded slightly differently, gives the *Euler tour trees* of Henzinger and King: a representation of a forest as a balanced binary search tree keyed on the tour, in which linking two trees, cutting an edge, and testing whether two vertices are connected each cost $O(\log n)$. Euler tour trees are the standard data structure for dynamic connectivity in forests.

This chapter builds both layers from the ground up. We flatten a small tree by hand, watch the in/out timestamps appear, prove the contiguous-subtree theorem, combine the flattening with a Fenwick tree to answer subtree and (with a twist) path queries, and finally develop Euler tour trees as a dynamic forest structure.

21.1 Flattening a tree

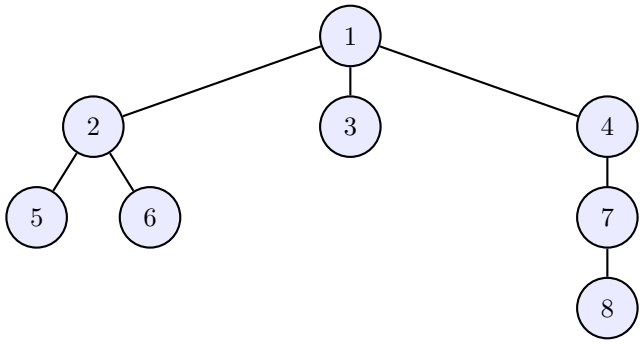
Fix a rooted tree $T = (V, E)$ with $n = |V|$ vertices, root r, and for each vertex a fixed order on its children (any order will do; we just commit to one). Run a depth first search from r. Two natural things happen as the DFS proceeds: we *enter* a vertex when we first reach it, and we *leave* it when we have finished all of its descendants and are about to return to its parent.

Intuition

Imagine walking around the outside of the tree with your left hand always touching an edge, the way you would trace a maze wall. You enter a vertex from above, explore its leftmost subtree, return, explore the next subtree, and so on, and only when every subtree is exhausted

do you climb back to the parent. The list of "first time I touched each vertex," read in the order you touched them, is the Euler tour. The key fact: while you are anywhere inside v's subtree, you have entered v but not yet left it. Entry and exit *bracket* the subtree like matched parentheses.

Here is the running example we will use throughout. The root is 1; children are listed left to right in increasing order.



There are two common ways to record the walk, and they serve different purposes. We treat each in turn.

21.1.1 The in/out timestamp encoding

Maintain a global counter `timer`, starting at 1. When DFS first enters a vertex v, assign $\texttt{tin}[v] \leftarrow \texttt{timer}$ and increment the timer. When DFS finishes v (after all children are done), assign $\texttt{tout}[v] \leftarrow \texttt{timer}$ and increment again. Every vertex thus receives two distinct stamps, and the counter runs from 1 to $2n$.

Algorithm 82: In/out timestamps via DFS

Input: rooted tree T, root r
Output: arrays $\texttt{tin}, \texttt{tout}$

```
timer ← 1
function DFS(v, parent p):
    tin[v] ← timer;   timer ← timer + 1
    foreach child c of v with c ≠ p do
        DFS(c, v)
    tout[v] ← timer;   timer ← timer + 1
DFS(r, NIL)
```

Run it by hand on the example. The DFS visits $1, 2, 5, 6, 3, 4, 7, 8$ on the way down and unwinds in the mirror order. The counter steps once on every enter and once on every leave, so the sequence of events in time order is

$$1_{\text{in}}, 2_{\text{in}}, 5_{\text{in}}, 5_{\text{out}}, 6_{\text{in}}, 6_{\text{out}}, 2_{\text{out}}, 3_{\text{in}}, 3_{\text{out}}, 4_{\text{in}}, 7_{\text{in}}, 8_{\text{in}}, 8_{\text{out}}, 7_{\text{out}}, 4_{\text{out}}, 1_{\text{out}},$$

which assigns the stamps $1, 2, \ldots, 16$ in that order:

vertex v	1	2	3	4	5	6	7	8
$\texttt{tin}[v]$	1	2	8	10	3	5	11	12
$\texttt{tout}[v]$	16	7	9	15	4	6	14	13

Every pair $(\texttt{tin}[v], \texttt{tout}[v])$ is a matched bracket, and the brackets nest. Vertex 4 opens at time 10 and closes at 15; inside that window sit 7 (11–14) and 8 (12–13), exactly the descendants of 4. The picture as a balanced-parenthesis string makes the nesting visible:

$$\underbrace{(_1\ \underbrace{(_2\ (_5\ )_5\ (_6\ )_6}_{\text{subtree 2}}\ )_2\ \underbrace{(_3\ )_3}_{\text{subtree 3}}\ \underbrace{(_4\ \underbrace{(_7\ (_8\ )_8\ )_7}_{\text{subtree 7}}\ )_4}_{\text{subtree 4}}\ )_1}_{\text{subtree 1}}$$

Figure 21.1 visualizes how this bracket nesting translates between the 2D tree topology and the 1D event array.

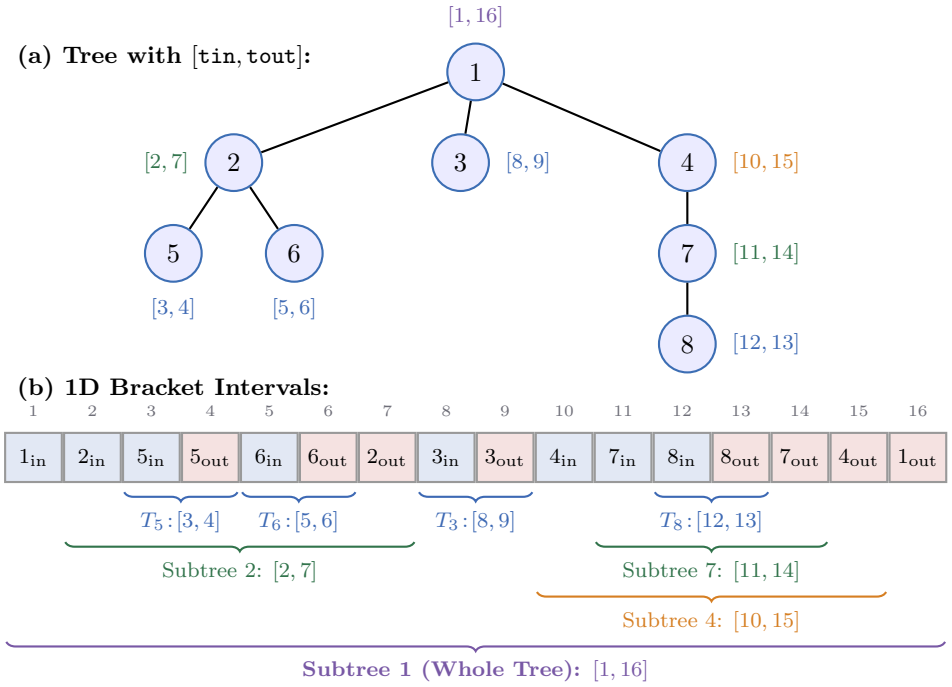


Figure 21.1: The Euler tour tree flattening into 1D bracket intervals. (a) The 8-vertex tree labeled with each node's entry and exit timestamps $[\texttt{tin}[v], \texttt{tout}[v]]$. (b) The corresponding 1D array of $2n = 16$ events. Entry timestamps (v_{in}, blue) and exit timestamps (v_{out}, red) form matched, perfectly nested bracket intervals $[\texttt{tin}[v], \texttt{tout}[v]]$. A node w belongs to the subtree of v if and only if $[\texttt{tin}[w], \texttt{tout}[w]] \subseteq [\texttt{tin}[v], \texttt{tout}[v]]$.

Common trap

The single most common slip is an off-by-one in the unwinding: writing $\texttt{tout}[7] = 14$ but then forgetting that 8 closed at 13 *before* 7 closes, or accidentally reusing a stamp. Sanity check:

the multiset of all $2n$ stamps must be exactly $\{1, 2, \ldots, 2n\}$ with no repeats, and for every v you must have $\texttt{tin}[v] < \texttt{tout}[v]$. Both hold above.

Theorem 21.1 (Subtree–ancestor characterization)**.** *For vertices u, v of T, u is an ancestor of v (with $u = v$ allowed) if and only if*

$$\texttt{tin}[u] \le \texttt{tin}[v] \quad \text{and} \quad \texttt{tout}[v] \le \texttt{tout}[u].$$

Equivalently, $\texttt{tin}[u] \le \texttt{tin}[v] \le \texttt{tout}[u]$, since exit stamps exceed entry stamps and the intervals never cross.

Proof. The DFS enters u, then recurses fully on u's subtree, then exits u. The time interval $[\texttt{tin}[u], \texttt{tout}[u]]$ is precisely the span during which the DFS stack contains u. A vertex v is in u's subtree iff the DFS reaches v while u is on the stack, i.e. iff v's entire enter/leave window lies within u's window: $\texttt{tin}[u] \le \texttt{tin}[v]$ and $\texttt{tout}[v] \le \texttt{tout}[u]$. For two vertices on different branches, one finishes before the other starts, so the intervals are disjoint; they never partially overlap. Thus the single inequality $\texttt{tin}[u] \le \texttt{tin}[v] \le \texttt{tout}[u]$ (combined with u being entered no later than v) already certifies the ancestor relation, because $\texttt{tin}[v]$ landing inside u's window forces $\texttt{tout}[v]$ inside it too. □

Why it works

The whole technique rests on one structural fact: *a DFS visits a subtree as a single uninterrupted run.* It never half-explores v's subtree, wanders off to a cousin, and comes back. That is what makes the enter/leave stamps nest like parentheses, and nesting is what turns "descendant of v" into "inside an interval." Every result in this chapter follows from this fact.

Pause

Pause and think: why can two subtree intervals never partially overlap? In depth-first search, if u is visited before v, either v is a descendant of u (in which case v's entire traversal finishes before u exits, so $[\texttt{tin}[v], \texttt{tout}[v]] \subset [\texttt{tin}[u], \texttt{tout}[u]]$), or v is in an entirely separate branch (in which case u finishes completely before v is entered, so $[\texttt{tin}[u], \texttt{tout}[u]]$ and $[\texttt{tin}[v], \texttt{tout}[v]]$ are strictly disjoint). Partial overlapping would imply the DFS left u's subtree before finishing it, violating the stack discipline of tree traversal!

21.1.2 The single-array (entry-order) encoding

The in/out encoding uses $2n$ slots. For *subtree* problems a leaner encoding suffices: record each vertex once, at its entry, and remember the subtree size. Let $\texttt{ord}[v] \in \{1, \ldots, n\}$ be the rank of v in entry order (its position in the DFS preorder), and let $\texttt{sz}[v]$ be the number of vertices in v's subtree.

Algorithm 83: Entry-order flattening with subtree sizes

```
1 timer ← 1
2 function DFS(v, p):
3     ord[v] ← timer;  node[timer] ← v;  timer ← timer + 1
4     sz[v] ← 1
5     foreach child c ≠ p of v do
6         DFS(c, v);  sz[v] ← sz[v] + sz[c]
7 DFS(r, NIL)
```

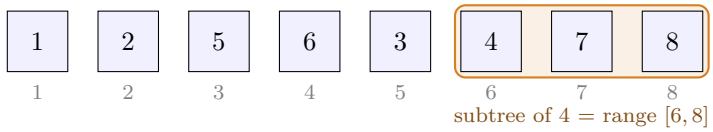
On the example, the preorder is $1, 2, 5, 6, 3, 4, 7, 8$, so `ord` assigns $1 \mapsto 1$, $2 \mapsto 2$, $5 \mapsto 3$, $6 \mapsto 4$, $3 \mapsto 5$, $4 \mapsto 6$, $7 \mapsto 7$, $8 \mapsto 8$, and the subtree sizes are $\texttt{sz}[1] = 8$, $\texttt{sz}[2] = 3$, $\texttt{sz}[3] = 1$, $\texttt{sz}[4] = 3$, $\texttt{sz}[5] = \texttt{sz}[6] = 1$, $\texttt{sz}[7] = 2$, $\texttt{sz}[8] = 1$.

Corollary 21.1 (Subtree as a contiguous range)**.** *In the entry-order array, the subtree of v occupies exactly the index range*

$$\big[\, \mathit{ord}[v],\ \mathit{ord}[v] + \mathit{sz}[v] - 1 \,\big].$$

Proof. By Theorem 21.1 the descendants of v are exactly the vertices u with $\texttt{tin}[v] \le \texttt{tin}[u] \le \texttt{tout}[v]$, and entry order is the order of `tin`. Since the DFS visits the subtree as one run, those descendants receive consecutive entry ranks; there are $\texttt{sz}[v]$ of them, the first being v itself at rank $\texttt{ord}[v]$. □

For the example, the subtree of 4 is the range $[\texttt{ord}[4], \texttt{ord}[4] + \texttt{sz}[4] - 1] = [6, 8]$, holding $\{4, 7, 8\}$ – exactly right. The flattened array, with the subtree of 4 boxed, looks like this:



Focus

Two encodings, two jobs.

- **Entry-order** (n cells): each vertex once. Perfect for *subtree* aggregate queries and subtree updates, because a subtree is one range.
- **In/out** ($2n$ cells, sometimes written as "$+v$ on enter, $-v$ on leave"): each vertex twice. Needed when you want *path-to-root* sums to telescope, or when you must distinguish entering from leaving (Euler tour trees below).

Pick the one whose ranges line up with the queries you must answer.

21.2 Subtree queries with a Fenwick tree

Suppose each vertex v carries a value a_v, and we must support:

- PointUpdate(v, δ): add δ to a_v;

- SUBTREESUM(v): return $\sum_{u \in \text{subtree}(v)} a_u$.

Flatten once with Algorithm 83. Place value a_v at array position `ord`$[v]$ and lay a Fenwick tree (binary indexed tree) over the array. (A Fenwick tree is a compact array structure maintaining prefix sums via power-of-two bit index jumps in $O(\log n)$ time; we explore static range trees comprehensively in Chapter 23.) A point update on v is a point update at index `ord`$[v]$; a subtree sum on v is the range sum over [`ord`$[v]$, `ord`$[v]$ + `sz`$[v] - 1$], which a Fenwick tree answers as PREFIX(r) − PREFIX($\ell - 1$).

Algorithm 84: Subtree sum / point update over the Euler flattening

```
1 Preprocess: run Algorithm 83; build empty Fenwick tree B of size n
2 for each vertex v do
3     B.ADD(ord[v], a_v)
4 function POINTUPDATE(v, δ):
5     B.ADD(ord[v], δ)
6 function SUBTREESUM(v):
7     ℓ ← ord[v];   r ← ord[v] + sz[v] − 1
8     return B.PREFIX(r) − B.PREFIX(ℓ − 1)
```

Trace it yourself

Take the example tree, all $a_v = 1$. Build the Fenwick tree over the eight entry positions. Query SUBTREESUM(2): range [`ord`$[2]$, `ord`$[2]$+`sz`$[2]-1$] = $[2, 4]$, holding nodes $2, 5, 6$, sum = 3. Now POINTUPDATE$(6, +10)$ and re-ask: the range $[2, 4]$ now sums to 13, while SUBTREESUM(3) on range $[5, 5]$ is unaffected and stays 1. Trace the Fenwick `add`/ `prefix` index walks to convince yourself the boundary $\ell - 1 = 1$ is handled correctly when v is not the root.

Cost summary

Preprocess: one DFS, $O(n)$, plus n Fenwick inserts, $O(n \log n)$ (or $O(n)$ with the linear Fenwick build). Each POINTUPDATE and SUBTREESUM is $O(\log n)$. Space $O(n)$. Swapping the Fenwick tree for a segment tree keeps the same bounds and additionally supports range-assign or min/max with lazy propagation.

21.2.1 Subtree updates with a range-update Fenwick or lazy segment tree

The mirror problem – *add δ to every vertex of a subtree*, then query a single vertex – is just as direct. "Add to a range, query a point" is the classic difference-array trick: a Fenwick tree over the difference array supports range-add via two point updates ($+\delta$ at ℓ, $-\delta$ at $r+1$) and point-query via a prefix sum. With the Euler flattening, the subtree of v is the range [`ord`$[v]$, `ord`$[v]$ + `sz`$[v] - 1$], so:

operation on tree	range on array	Fenwick primitive
add δ to subtree(v)	$[\ell, r]$	range-add
read a_v	index `ord`$[v]$	point-query (prefix)

If you need *both* subtree-update and subtree-query simultaneously, use the range-update/range-query

Fenwick (two BITs) or a lazy segment tree over the same flattening; nothing about the tree-to-array map changes.

Common trap

A frequent bug: building the Fenwick tree indexed by vertex id instead of by `ord`$[v]$. Then a subtree is *not* a contiguous range and the query is silently wrong. Always translate through `ord`, and remember that `ord` depends on the chosen child order; if you rebuild the DFS with a different order, every range moves.

21.3 Path-to-root queries with the in/out encoding

Subtree queries wanted entry order. *Path-to-root* queries want the in/out encoding, because of a telescoping cancellation. Suppose we must support "add δ to a_v" updates and answer "what is the sum of values on the path from the root down to v?" Place each vertex twice on a length-$2n$ array: at position `tin`$[v]$ write $+a_v$, and at position `tout`$[v]$ write $-a_v$.

Theorem 21.2 (Root-path sum telescopes)**.** *With the $\pm a$ placement above, for any vertex v,*

$$\textsc{prefix}\big(\texttt{tin}[v]\big) \;=\; \sum_{u \text{ on root-to-}v \text{ path}} a_u.$$

Proof. A vertex u contributes $+a_u$ at `tin`$[u]$ and $-a_u$ at `tout`$[u]$. Sum the array over indices $1 \ldots$ `tin`$[v]$. For each u, its net contribution to that prefix is:

- $+a_u$ if `tin`$[u] \le$ `tin`$[v] <$ `tout`$[u]$, i.e. u's window is still open at time `tin`$[v]$ – which by Theorem 21.1 means exactly that u is an ancestor of v (or v itself);
- 0 if both stamps of u are $\le$ `tin`$[v]$ (window already closed) or both are $>$ `tin`$[v]$ (window not yet opened).

The vertices with open windows at time `tin`$[v]$ are precisely those on the DFS stack, which is the root-to-v path. Summing their $+a_u$ gives the claim. □

So with a single Fenwick tree over the $2n$-length array, "add δ to a_v" becomes two point updates ($+\delta$ at `tin`$[v]$, $-\delta$ at `tout`$[v]$), and a root-path query is one prefix sum. This is the standard way to support *subtree-add, path-to-root-query* (and, by swapping the roles, the dual): you are choosing which operation gets the $\pm$ trick.

To see the cancellation concretely, take the running example with $a_v = v$ and query the path to vertex 8. Lay $+a_v$ at `tin`$[v]$ and $-a_v$ at `tout`$[v]$ across the 16 positions, then sum the prefix up to `tin`$[8] = 12$ (Figure 21.2). Every vertex whose window has already closed (its $+$ and $-$ both lie in the prefix) cancels to zero; the survivors are exactly $1, 4, 7, 8$, whose windows are still open at time 12 – the root-to-8 path – giving prefix $1 + 4 + 7 + 8 = 20$.

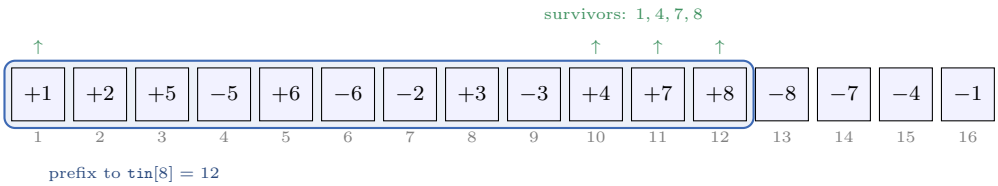


Figure 21.2: Root-path sum by prefix telescoping. With $+a_v$ at $\texttt{tin}[v]$ and $-a_v$ at $\texttt{tout}[v]$ ($a_v = v$ here), the prefix up to $\texttt{tin}[8] = 12$ (shaded) cancels every closed window and leaves the open ones, marked ↑ at positions $1, 10, 11, 12$ – the entries of $1, 4, 7, 8$, summing to 20.

Connection

The path-to-root telescope is a baby version of what **heavy-light decomposition** and **link-cut trees** do for arbitrary u–v paths. The Euler tour handles root paths for free because a root path is exactly "the set of currently-open brackets," but it cannot decompose a general u–v path into $O(\log n)$ ranges. For that you split the path at the lowest common ancestor and need HLD's chain ranges or LCT's preferred paths. The Euler tour is the lightweight tool; reach for it first, and escalate only when root paths are not enough.

21.3.1 Lowest common ancestor from the Euler tour

There is a third, slightly different Euler walk that drives LCA queries: record a vertex *every time the DFS touches it*, including each return from a child. This walk has length $2n - 1$ (each of the $n - 1$ edges is crossed twice, plus the starting vertex). Alongside it record the depth of each visited vertex. The LCA of u and v is the shallowest vertex appearing between any occurrence of u and any occurrence of v in this walk – a range-minimum query on the depth array. Building a sparse table over depths answers LCA in $O(1)$ after $O(n \log n)$ preprocessing; the specialized ± 1 RMQ structure pushes preprocessing to $O(n)$. We develop this fully in Chapter 23; here it is one more application of the same tour.

History

The term honors Leonhard Euler's 1736 paper on the bridges of Königsberg, which established graph theory by characterizing walks that traverse every edge once. The modern algorithmic technique was created by Robert Tarjan and Uzi Vishkin in 1985 as a strategy to parallelize tree computations on PRAM architectures; by flattening trees into edge tours, they converted irregular pointer trees into 1D arrays amenable to fast parallel prefix sums and list ranking. In 1995, Monika Henzinger and Valerie King introduced *Euler tour trees* (ETTs), maintaining tour sequences within balanced search trees to track dynamic graph connectivity under edge insertions and deletions in $O(\log^2 n)$ time. Link-cut trees handle path queries better, but Euler tour trees gave the first poly-logarithmic algorithms for fully dynamic general graph connectivity and subtree aggregation. An active frontier remains designing cache-oblivious, pointer-free dynamic tour trees with optimal worst-case bounds.

21.4 Euler tour trees: dynamic connectivity in a forest

Everything so far assumed a *static* tree: flatten once, query forever. Now we let the forest change. We must support, on a forest F (a set of vertex-disjoint trees):

- LINK(u, v): add edge uv, merging two trees into one (precondition: u, v in different trees, so no cycle is created);
- CUT(u, v): remove an existing edge uv, splitting one tree into two;
- CONNECTED(u, v): are u and v in the same tree?
- plus subtree/aggregate queries, if values are attached.

A static flattening cannot survive a LINK or CUT: those operations splice the tour, moving $\Theta(n)$ array entries in the worst case. The fix is to stop storing the tour as an array and store it as a *balanced binary search tree keyed by tour position*, where "splice" becomes split/merge in $O(\log n)$.

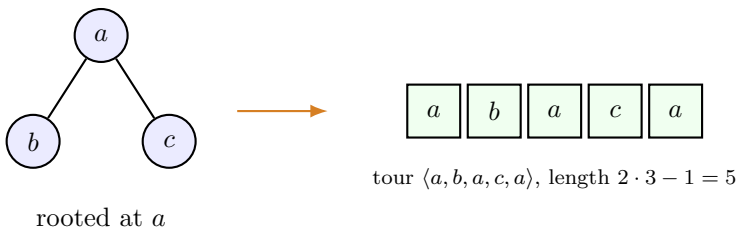
21.4.1 The representation

For each tree in the forest, fix an arbitrary root and take its Euler tour as a sequence of *visits*. The cleanest version for connectivity stores *occurrences*: each vertex v with c children appears $c+1$ times in the tour (once before its first child, once between consecutive children, once after its last child; a leaf appears once). The total length of the tour of a tree on k vertices is $2k-1$ occurrences.

Store this occurrence sequence in a balanced BST – a treap, a splay tree, or a red-black tree – keyed *implicitly by position* (the BST is a sequence container, not a value-ordered set; its in-order traversal is the tour). Keep, for each vertex, a pointer to one of its occurrences (any one). The key insight:

Why it works

Two vertices are in the same tree *iff their occurrence nodes live in the same BST*. To test that, walk from each occurrence to the root of its BST and compare the roots in $O(\log n)$. No traversal of the tree is needed – connectivity is a single "which container am I in" check. The Euler tour has converted a tree-shaped question into a sequence-membership question.



rooted at a

tour $\langle a, b, a, c, a\rangle$, length $2 \cdot 3 - 1 = 5$

21.4.2 Link and Cut as sequence surgery

The operations become careful cut-and-paste on the occurrence sequence. We use the standard *reroot-then-splice* formulation, which keeps the bookkeeping uniform.

Rerooting. To reroot a tree's tour at vertex x: the tour is cyclic in spirit (it starts and ends at the old root), so to make x the start, split the sequence just before the first occurrence of x and swap the two halves – [prefix | from x onward] becomes [from x onward | prefix], with one occurrence of x appended so the tour still begins and ends at x. This is a constant number of split/merge operations, $O(\log n)$.

Link(u, v)**.** Reroot u's tree at u and v's tree at v. Now both tours begin at their respective new roots. Splice: concatenate u's tour, then a fresh occurrence of v, then v's tour body, then a fresh occurrence of u. Concretely you merge the two sequences and add two new occurrences (descend $u \to v$, return $v \to u$) so that u gains v as a child. A constant number of split/merge, $O(\log n)$.

Cut(u, v)**.** The edge uv corresponds to two "crossing" positions in the tour: where the DFS descends $u \to v$ and where it returns $v \to u$. Let these bracket a contiguous block – that block is exactly v's subtree's tour. Split it out (two splits), then merge the remaining two pieces back together (one merge). The extracted block is v's new tree; the merged remainder is u's. $O(\log n)$.

Algorithm 85: Euler tour tree operations (sketch over an implicit balanced BST)

```
function Reroot(x):
    S ← sequence (BST) containing x's occurrences
    split S just before the first occurrence of x into A ‖ B
    drop the now-duplicate boundary occurrence; return merge(B, A, ⟨x⟩)
function Link(u, v):
    Reroot(u);  Reroot(v)
    return merge(tour(u), ⟨v⟩, body of tour(v), ⟨u⟩)
function Cut(u, v):
    locate the two occurrences bracketing v's block (the u→v and v→u crossings)
    split them out: S = A ‖ (block v) ‖ C
    tree(v) ← block v (cleaned of the crossing occurrences)
    tree(u) ← merge(A, C)
function Connected(u, v):
    return root(occ(u)) = root(occ(v))
```

Why it works

Why is the answer to Cut a single contiguous block? Because of the very same bracketing that powered subtree ranges in the static case. The descent into v opens v's window and the return closes it; everything the tour visits in between is, by Theorem 21.1, a descendant of v. Removing edge uv detaches exactly that subtree, so it is exactly that contiguous tour block. Static range queries and dynamic tour splicing both rest on this same subtree-contiguity theorem.

Figure 21.3 shows one Cut in full. The tree is rooted at u with children x, v, y and v carrying one child w; its tour is $\langle u, x, u, v, w, v, u, y, u \rangle$. The two crossings of edge uv – the descent $u \to v$ and the return $v \to u$ – bracket the block $\langle v, w, v \rangle$. Splitting that block out leaves v's new tree and, after merging the two surrounding pieces (which adjoins the doubled u into one occurrence), the tour $\langle u, x, u, y, u \rangle$ of u's shrunken tree.

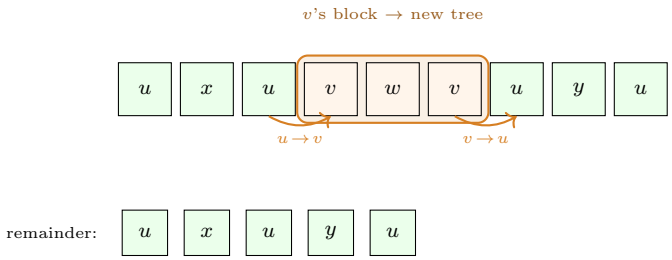


Figure 21.3: CUT(u, v) as sequence surgery. The edge uv is crossed at the two arrows; they bracket the block $\langle v, w, v\rangle$ (shaded), which splits off as v's tree. Merging the remaining pieces fuses the doubled u into the tour $\langle u, x, u, y, u\rangle$ (bottom row).

21.4.3 Correctness and the connectivity invariant

Theorem 21.3 (ETT maintains the forest correctly)**.** *Maintain, for each tree of the forest, one balanced BST holding its Euler tour occurrences in tour order. Then after any sequence of* LINK *and* CUT *operations performed as in Algorithm 85:*

1. *the BSTs are in bijection with the trees of the current forest, and*
2. *the in-order sequence of each BST is a valid Euler tour of its tree.*

Consequently CONNECTED(u, v) *is correct.*

Proof. Both claims hold initially (singletons: one occurrence each). We show each operation preserves the invariant.

Link. Rerooting permutes a tour into another valid tour of the same tree (a cyclic rotation of the closed walk that an Euler tour is, with endpoints fixed up). After rerooting u's tree at u and v's at v, splicing inserts the directed traversal $u \to v$, then v's whole tour, then the return $v \to u$, at the front of u's tour. This is exactly the Euler tour of the tree obtained by attaching v's tree under u via edge uv – the DFS from u would now descend into v first, do v's subtree, and return. So the merged BST's in-order is a valid tour of the merged tree, and there is now one BST for the one merged tree.

Cut. The two occurrences bracketing v's block are the $u \to v$ descent and the $v \to u$ return (these exist and are unique because uv is a tree edge, traversed exactly twice). Splitting the block out and discarding those two crossing occurrences yields, as in-order, exactly v's subtree tour (a valid tour of the component containing v after deletion) and, after merging the surrounding pieces, a valid tour of the component containing u. Two BSTs for the two new trees.

Both invariants are thus maintained, and since each tree owns exactly one BST, u, v are co-tree iff their occurrence nodes share a BST root, which is what CONNECTED tests. □

Cost summary

With a balanced BST (treap/splay/red-black) supporting SPLIT and MERGE in $O(\log n)$, every ETT operation – LINK, CUT, CONNECTED, REROOT – runs in $O(\log n)$ (amortized, for splay). Each tree on k vertices stores $2k-1$ occurrences, so total space is $O(n)$. To read `occ` pointers and roots we keep parent pointers (or use splay's amortization). This matches the best known

bounds for connectivity in a *forest* under edge updates.

21.4.4 Aggregates over a dynamic subtree

Augment each BST node with a subtree aggregate (sum, max, count of marked vertices) maintained on SPLIT/MERGE, the way order-statistics or interval treaps carry subtree sums. Because a vertex's tree-subtree is a contiguous block in the tour, a dynamic "sum over the subtree of v" is again a range aggregate over the BST – reroot at v (or locate v's bracket) and read the aggregate of its block. This is the mechanism Henzinger–King and later Holm–de Lichtenberg–Thorup use to maintain, e.g., the number of incident non-tree edges per subtree, the key subroutine in fully dynamic connectivity for *general* graphs.

Connection

Euler tour trees solve dynamic connectivity for *forests*. General-graph dynamic connectivity (Chapter 39) is built on top: keep a spanning forest in an ETT, and when a tree edge is cut, search a logarithmic hierarchy of non-tree edges for a replacement. The subtree-aggregate augmentation above is what makes "count non-tree edges crossing this subtree" an $O(\log n)$ query, giving the celebrated $O(\log^2 n)$ amortized update of Holm–de Lichtenberg–Thorup.

Common trap

ETT maintains *connectivity*, not *depth or path information*. Rerooting freely scrambles the notion of "parent," so ETT cannot answer "what is the distance from u to v" or "find the LCA" under updates. That is the job of **link-cut trees**, which maintain rooted-path structure at the cost of a more delicate preferred-path machinery. Choose ETT when you only need connectivity and subtree aggregates; choose LCT when you need path aggregates or rooted depth.

21.4.5 A small dynamic trace

Trace it yourself

Start with three singleton vertices a, b, c – three BSTs, occurrences $\langle a\rangle, \langle b\rangle, \langle c\rangle$. LINK$(a, b)$: reroot is trivial, splice to get tour $\langle a, b, a\rangle$ in one BST; $\langle c\rangle$ stays separate. Now CONNECTED(a, b) = true, CONNECTED(a, c) = false. LINK(b, c): reroot the ab-tree at b giving $\langle b, a, b\rangle$, splice c to get $\langle b, a, b, c, b\rangle$; now all three share a BST. Finally CUT(a, b): the $b \to a$ and $a \to b$ crossings bracket the block $\langle a\rangle$, which splits off as a's tree; the remainder merges to $\langle b, c, b\rangle$. End state: $\{a\}$ and $\{b, c\}$. Verify CONNECTED(b, c) = true, CONNECTED(a, c) = false. Figure 21.4 shows the forest and its Euler tour occurrence sequences after each step.

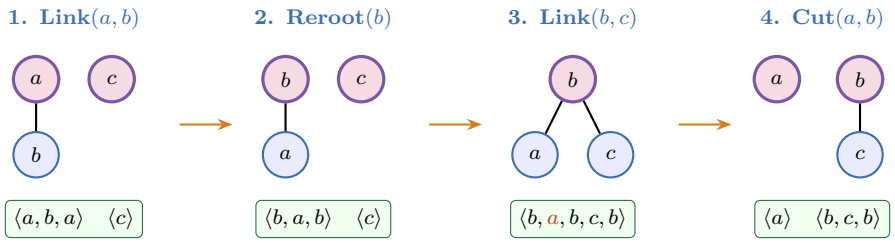


Figure 21.4: Tracing dynamic forest operations in an Euler tour tree (roots shaded purple). Link(b, c) first reroots the $\{a, b\}$ tree at b ($\langle a, b, a\rangle \to \langle b, a, b\rangle$) before splicing c; Cut(a, b) excises a's bracketed block and fuses the surrounding b's.

21.5 Putting it together: a design checklist

When a problem is phrased on a rooted tree, ask in order:

1. **Static or dynamic shape?** If edges never change, flatten once (Section 21.1). If the forest is edited by link/cut, reach for Euler tour trees (Section 21.4).
2. **Subtree or path queries?** Subtree aggregates ⇒ entry-order flattening + range structure (Section 21.2). Root-path aggregates ⇒ in/out ± encoding (Section 21.3). General u–v path ⇒ this technique is not enough; split at the LCA and use HLD or LCT.
3. **Which array structure?** Sum/point ⇒ Fenwick. Range-assign, min/max, lazy ⇒ segment tree. Immutable + idempotent (min/gcd) ⇒ sparse table.

Application

The Euler tour flattening is used for: subtree-sum / subtree-add queries in competitive programming; "mark a subtree dirty" in incremental compilers and scene graphs; aggregating metrics over an organizational or file-system hierarchy with a single prefix-sum array; offline LCA via the depth-RMQ tour; and – through Euler tour trees – the spanning-forest layer of fully dynamic graph connectivity used in network reliability and dynamic graph databases.

Cost summary

Static flattening. Build $O(n)$; subtree point-update/sum $O(\log n)$ with a Fenwick tree; root-path update/query $O(\log n)$ with the ± encoding; space $O(n)$. **Euler tour trees.** Link, Cut, Connected, subtree aggregate each $O(\log n)$; space $O(n)$.

Connection

Where We Go Next (Link-Cut Trees): In Section 21.4, we saw how Euler tour trees represent dynamic forests by maintaining tour sequences inside balanced search trees, supporting link, cut, and subtree queries in $O(\log n)$. But Euler tour trees face a limitation: because an Euler tour traverses edges in tour order rather than along vertical paths, aggregating values along a simple path between two arbitrary nodes u and v cannot be done in a single contiguous slice. When a dynamic forest must support fast *path* queries and path updates, we need a

different decomposition. In Chapter 22, we study *Link-Cut Trees* (Sleator and Tarjan), which dynamically maintain heavy-light path partitions in splay trees to support both path and edge modifications in $O(\log n)$ amortized time.

Exercises

Exercise 21.1 (Quick mental check: tour length)**.** In an entry-exit Euler tour that writes down each vertex once when entering and once when leaving, what is the exact length of the resulting array for a tree on n vertices?

Exercise 21.2 (Quick mental check: ancestor detection)**.** Suppose $\texttt{tin}[u] < \texttt{tin}[v] < \texttt{tout}[v] < \texttt{tout}[u]$. What is the ancestor relationship between u and v? Could $\texttt{tout}[u]$ ever fall between $\texttt{tin}[v]$ and $\texttt{tout}[v]$?

Exercise 21.3. Recompute `tin` and `tout` for the running example if the children of vertex 1 are visited in the order $4, 3, 2$ instead of $2, 3, 4$. Verify that every subtree is still a contiguous range in the new entry order, and give the new range of subtree(2).

Exercise 21.4. Prove that in the in/out encoding, vertex u is a *proper* ancestor of v iff $\texttt{tin}[u] < \texttt{tin}[v]$ and $\texttt{tout}[v] < \texttt{tout}[u]$, and that u, v are unrelated (neither is an ancestor of the other) iff their intervals are disjoint. Conclude that two intervals from a DFS tour are never properly crossing.

Exercise 21.5. You are given a rooted tree and must support PaintSubtree(v, c) (assign color c to every vertex of subtree(v)) and Color(v) (read v's color). Design an $O(\log n)$-per-operation solution using the Euler flattening, naming the exact array structure and the range it touches.

Exercise 21.6. Using the $\pm a$ in/out encoding (Theorem 21.2), implement *subtree-add, single-vertex-read*: adding δ to all of subtree(v), then reading a_w for a single w. Which positions get $\pm\delta$, and why does the point read become a prefix sum up to $\texttt{tin}[w]$? Prove correctness.

Exercise 21.7. Show that the Euler tour technique *cannot* reduce a general u–v path sum to $O(1)$ ranges in entry order, by exhibiting a tree and a path whose vertices land at $\Theta(n)$ disjoint, non-adjacent entry positions. (This is the gap that heavy-light decomposition and link-cut trees close.)

Exercise 21.8. Augment the static flattening to answer SubtreeMax(v) and PointAssign(v, x) in $O(\log n)$. Explain why a plain Fenwick tree does not suffice for max under assignment and what structure you use instead.

Exercise 21.9. Implement Reroot(x) for an Euler tour tree over an implicit treap, and prove that the resulting in-order sequence is a valid Euler tour rooted at x. Count the exact number of split/merge calls.

Exercise 21.10. In an Euler tour tree, augment each occurrence node with a subtree sum of vertex values (each vertex's value attributed to exactly one of its occurrences). Describe how to answer "sum over the tree-subtree of v" in $O(\log n)$ under Link and Cut, and prove that rerooting does not corrupt the per-vertex single-attribution bookkeeping.

Exercise 21.11. Give a sequence of Link/Cut/Connected operations on five vertices that produces a forest with two trees, one of which is a path of length 3. Trace the occurrence-sequence BST after each operation, and at the end verify Connected answers for all $\binom{5}{2}$ vertex pairs.

Exercise 21.12. (Counting.) Prove that the Euler tour of a tree on k vertices used by ETT has exactly $2k-1$ occurrences when each vertex of degree d contributes $d+1$ occurrences. Then show the directed-edge formulation (record each of the $k-1$ edges in both directions) has length $2(k-1)$, and reconcile the two counts.

Exercise 21.13. (Break it.) A student builds the subtree-sum structure but indexes the Fenwick tree by *post*order rank instead of *pre*order rank. For which trees does SubtreeSum still return the right answer, and for which does it fail? Give a concrete failing tree with the wrong numerical answer.

Chapter 22

Link-Cut Trees and Dynamic Trees

Heavy-light decomposition (Chapter 19) taught us to answer path queries on a *fixed* tree in $O(\log^2 n)$ or $O(\log n)$ time. But what if the tree itself changes? Suppose edges are inserted and deleted online, the forest reshapes between every query, and we still want to ask "what is the maximum weight on the path from u to v?" or "are u and v in the same tree?" Heavy-light decomposition does not handle changes: the decomposition into heavy chains is a global property of the whole tree, and a single edge insertion can invalidate it.

The *link-cut tree* of Sleator and Tarjan solves this problem. It maintains a forest of rooted trees under two structural operations,

$$\textsc{Link}(u, v) \quad \text{(make } u \text{ a child of } v\text{)} \qquad \text{and} \qquad \textsc{Cut}(u) \quad \text{(detach } u \text{ from its parent)},$$

together with path operations (query the path to the root, add a constant along it, find the root, find the minimum), each in $O(\log n)$ *amortized* time. It is used in the fastest versions of Dinic-style blocking-flow algorithms, in online dynamic connectivity on forests, and in many other dynamic-tree problems in algorithm design and competitive programming.

This chapter builds the structure from the ground up. We start with the idea of *preferred paths* – a self-adjusting cousin of heavy-light chains – show how to store each path in a balanced BST (a splay tree), define the single master operation Access from which everything else follows, and then prove the $O(\log n)$ amortized bound. We trace a small forest by hand the whole way through.

History

The link-cut tree was invented in 1983 by Daniel Sleator and Robert Tarjan at Bell Labs and Stanford. Their driving motivation was not an abstract tree puzzle, but maximum network flow. In Dinic's blocking-flow algorithm (1970) and Karzanov's preflow-push methods (1974), each phase repeatedly pushes flow along augmenting paths in a layered graph. With standard depth-first search, finding and bottlenecking paths took $O(V \cdot E)$ per phase, totaling $O(V^2 E)$. Sleator and Tarjan realized that if one could maintain the flow-carrying paths as a collection of dynamic trees under edge insertions (Link) and edge deletions (Cut) in $O(\log V)$ time per update, Dinic's overall bound would drop to $O(VE \log V)$.

The obvious approaches did not work. In the late 1970s, the known balanced tree machinery (AVL trees, 2-3 trees) balanced binary search trees by local rotations. But if you try to balance the represented tree directly, a single Cut near the root can orphan $\Theta(n)$ nodes, and local

rotations would change the vertex degrees and topology of the graph being modeled, which changes the problem itself.
Sleator and Tarjan instead separated two levels: leave the represented tree's graph topology completely untouched, but decompose its paths into depth-ordered chains. Each chain is stored inside an auxiliary balanced tree keyed strictly by depth. To keep the auxiliary trees balanced without complex rebalancing invariants across splices, they invented *Splay Trees* (published in 1985), relying on self-adjusting rotations that amortize to $O(\log n)$ per operation. While biased search trees provided worst-case bounds in their 1983 paper, the splay-based version became the standard for dynamic tree algorithms. An open challenge remains obtaining worst-case logarithmic bounds with data structures as simple and cache-efficient as splay-based link-cut trees.

22.1 What we are maintaining

Focus

We maintain a forest F of rooted trees on n nodes. Each node may carry a value; we support:

- MakeTree(v): create a one-node tree (a new root).
- Link(u, v): attach root u as a child of node v. Requires that u is a tree root and u, v lie in different trees.
- Cut(u): remove the edge from u to its parent, splitting one tree into two. Requires that u is not a root.
- FindRoot(v): return the root of v's tree (so FindRoot(u) = FindRoot(v) tests connectivity).
- PathAggregate(v): return an aggregate (sum, min, max) of the values on the path from v up to its root, and optionally update them.
- Evert(v) (re-root): make v the root of its tree, reversing the orientation of the path from v to the old root. This upgrades the "path to root" primitives into "path between any u and v."

These are the operations on the *represented forest* – the actual forest the user cares about. We never store the represented forest directly with parent pointers, because a single Cut near the top could orphan $\Theta(n)$ descendants and a single path query could walk $\Theta(n)$ edges. Instead we store an *auxiliary* structure whose shape only loosely follows the represented forest, and we re-balance it lazily.

Intuition

Think of two layers.

- The **represented forest**: the rooted trees the algorithm above us reasons about. Nodes here have a true parent.
- The **auxiliary forest**: a collection of balanced binary search trees, one per "preferred

path," glued together by special *path-parent* pointers. This is what we physically store and rotate.

Every represented-forest operation is translated into a handful of operations on the auxiliary forest. The key guarantee is that, with splay trees as the auxiliary BSTs, every translation costs $O(\log n)$ amortized.

22.2 Preferred paths: a self-adjusting chain decomposition

Recall heavy-light decomposition. Each node points to exactly one "heavy" child (the one with the largest subtree), and the heavy edges link up into chains that cover the tree. A root-to-node path crosses $O(\log n)$ chains because each time you leave a chain you at least double the subtree size.

Link-cut trees use the same "one special child per node" idea, but the choice of special child is driven by *access history* rather than subtree size.

Definition 22.1 (Preferred child and preferred path)**.** At each node v we designate at most one child as the *preferred child*. Concretely, the preferred child of v is the child through which the most recent Access to a descendant of v descended; if the last access to v's subtree ended at v itself (or no such access has happened), v has no preferred child. The *preferred edges* are the edges from each node to its preferred child. They decompose the tree into vertex-disjoint *preferred paths*.

So preferred paths are exactly heavy chains in spirit, but they migrate. After you query the path to node x, the entire root-to-x path becomes one preferred path; everything that used to branch off it loses its preferred-edge status.

Intuition

A preferred path is the "hot" route your last query took. Storing each hot route in a balanced BST lets you grab any contiguous chunk of it – a path query – in $O(\log n)$. When the next query takes a different route, you splice the BSTs accordingly. Self-adjusting access patterns are precisely what splay trees were designed to exploit, which is why they are the BST of choice here.

22.2.1 The Two-Layer Architecture: Represented Tree vs. Auxiliary Splay Forest

Figure 22.1 shows the two layers. In the represented forest (Lane 1), a tree of nine nodes has solid blue edges marking the current preferred path $1-3-6-7$ (the route traversed by the most recent access to node 7). Dashed gray edges are non-preferred branches.

In the auxiliary forest (Lane 2), each preferred path lives in its own binary search tree, keyed strictly by depth in the represented forest. Shallower nodes sort to the left; deeper nodes sort to the right. Dashed red arrows show path-parent pointers: each leaves the root of an auxiliary BST and points to the represented-forest parent of the shallowest node on that path.

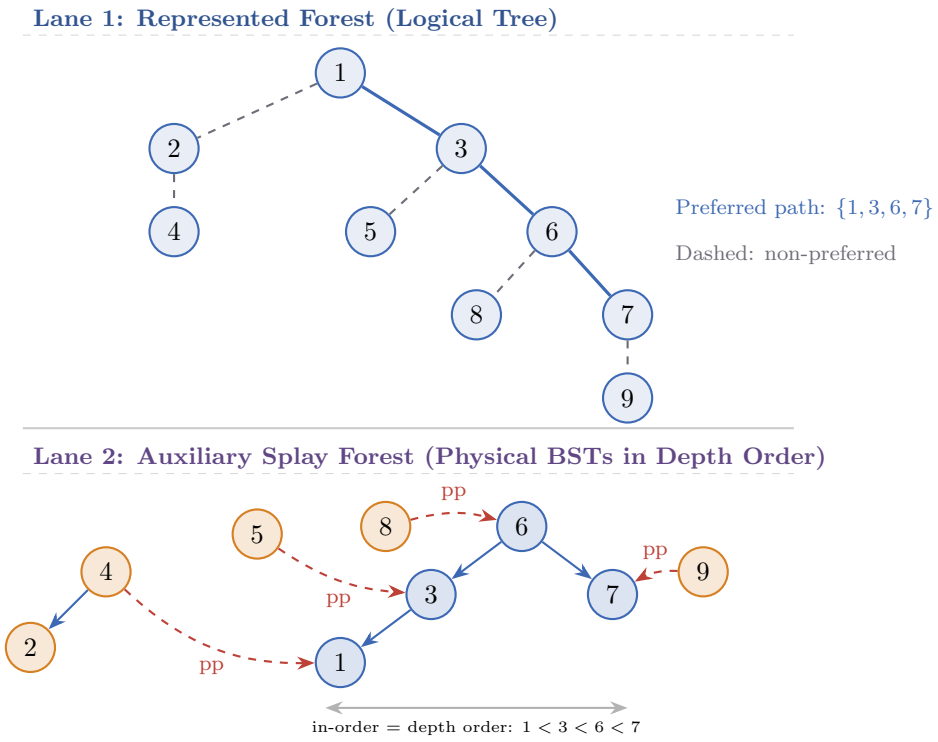


Figure 22.1: The two layers of a Link-Cut Tree. **Top (Lane 1):** The represented forest where thick blue edges mark the current preferred path $\{1, 3, 6, 7\}$, and dashed gray edges are non-preferred branches. **Bottom (Lane 2):** The auxiliary forest storing each preferred path as an independent splay tree ordered strictly by depth (shallowest at far left, deepest at far right). Dashed red arrows are unidirectional path-parent pointers leaving auxiliary roots.

The preferred paths here are $\{1, 3, 6, 7\}$, $\{2, 4\}$, $\{5\}$, $\{8\}$, $\{9\}$. Five paths, five auxiliary BSTs. The dashed edges become path-parent pointers: the BST holding $\{2, 4\}$ knows that its topmost node (2) hangs off node 1 in the represented forest, but node 1 does *not* point back down to it.

22.3 The auxiliary structure: one splay tree per preferred path

We store each preferred path in a binary search tree keyed by *depth in the represented forest*: shallower nodes are "smaller." So an in-order traversal of the auxiliary BST visits the path's nodes from the root end to the deep end. The leftmost node of the BST is the shallowest node of the path; the rightmost is the deepest.

Formal

Invariant (ordering). For a preferred path p, its auxiliary BST stores exactly the nodes of p, ordered by represented-forest depth. The *symmetric order* (in-order) of the BST equals the top-to-bottom order along p.

Invariant (gluing). The root r of each auxiliary BST stores a *path-parent pointer* to the represented-forest parent of the shallowest node on its path (or `null` if that node is a forest root). No path-parent pointer points *into* the middle of another BST; it points at a node,

and that node's BST is the parent path. These one-way pointers are the only links between distinct auxiliary trees.

The two kinds of pointers are easy to confuse, so pin them down:

- **BST child pointers** (`left`, `right`) live *inside* one auxiliary tree and obey the depth ordering. They are rotated freely by splay operations and carry no represented-forest meaning beyond the ordering invariant.
- **Path-parent pointers** live *between* auxiliary trees. They encode the non-preferred edges of the represented forest.

Common trap

A path-parent pointer is stored at the *root* of an auxiliary tree and is unidirectional: child path knows its parent path, but the parent stores no list of its hanging children. This asymmetry is what keeps Cut cheap – you never have to scan or update a parent's child list – but it also means "which preferred paths hang off me?" is not a question the structure answers directly. You only ever walk path-parent pointers *upward*.

The dashed path-parent arrows are the only links between distinct trees, and they run strictly upward toward the represented root. Walking them upward is exactly what Access does.

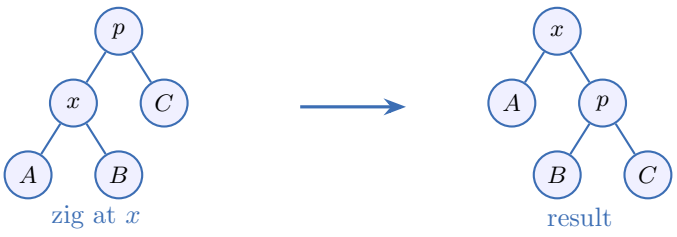
22.3.1 Why a BST keyed by depth

Because the key is depth, a contiguous prefix of the path (everything shallower than some node v) is exactly the set of BST nodes less than v – the left subtree after we bring v to the root. That is what makes path-to-root aggregates $O(\log n)$: splay v to the top of its auxiliary tree, and its left subtree is precisely the part of the current preferred path that lies above v.

22.4 Splay trees, briefly

The auxiliary BSTs are *splay trees*: self-adjusting binary search trees that move an accessed node to the root by a sequence of rotations called *splaying*. We will not re-derive the full splay analysis here (the companion intro book covers basic BSTs), but we need the operation and its amortized guarantee.

A single rotation at x (with parent p) is the usual BST rotation; splaying repeats double rotations until x is the root:



Formal

Splay amortized bound. Assign each node x a weight $w(x) > 0$ and define its *size* $s(x) = \sum_{y \in \text{subtree}(x)} w(y)$ and *rank* $r(x) = \log_2 s(x)$. The potential of a splay tree is $\Phi = \sum_x r(x)$. *Access Lemma:* splaying a node x in a tree with root t costs at most $3\big(r(t) - r(x)\big) + 1$ amortized rotations. Consequently a sequence of m operations on a tree of n nodes runs in $O\big((m+n)\log n\big)$.

We will use the freedom to choose weights in the Access Lemma: choosing $w(x)$ to count the *whole represented subtree* of x, not just its auxiliary subtree, is what turns the per-splay bound into a per-Access bound across all the auxiliary trees at once.

22.5 Access: the master operation

Everything reduces to one operation. Access(v) does two things:

1. It makes the path from v up to the root of its represented tree the *preferred path*, and makes v the deepest node on a preferred path that starts at the represented root.

2. It splays v to the root of its auxiliary tree, so that afterwards the single auxiliary tree containing the represented root has v at its top, and v's left subtree is exactly the represented path from the root down to v.

After Access(v), the node v has no path-parent pointer (it is on the path containing the represented root), and the auxiliary tree rooted at v *is* the represented root-to-v path stored in depth order.

22.5.1 How access walks up

We splay v within its own auxiliary tree, then repeatedly hop up a path-parent pointer, splay the parent, and splice. Each hop turns one non-preferred edge into a preferred edge (and the old preferred child of the parent into a non-preferred one). The splicing is pure BST surgery: at the parent w we splay w, detach its current right subtree (the part of w's path *deeper* than w, which is losing preferred status – it becomes a separate auxiliary tree with a path-parent pointer back to w), and attach the tree we just came from as w's new right subtree.

Algorithm 86: ACCESS(v) – make v the deep end of the root's preferred path

```
1  Splay(v)
2  if v.right ≠ NULL then
       // detach v's path below v:  it becomes its own preferred path
3      v.right.pathparent ← v
4      v.right.parent ← NULL
5      v.right ← NULL
6  recompute aggregate at v
7  while v.pathparent ≠ NULL do
8      w ← v.pathparent
9      Splay(w)
10     if w.right ≠ NULL then
           // w's old preferred child below it loses preferred status
11         w.right.pathparent ← w
12         w.right.parent ← NULL
13     w.right ← v
14     v.parent ← w
15     v.pathparent ← NULL
16     recompute aggregate at w
17     Splay(v)                                   // bring v back to the top
18 return v
```

Why it works

Why does this leave the invariants intact? Each iteration of the loop dissolves one non-preferred edge (the path-parent from v's path to w) and creates one (from w's old right child back to w). The depth ordering is preserved because w is shallower than every node of v's former tree (it is v's represented-forest ancestor through the spliced edge), so attaching v's tree as w's *right* subtree keeps in-order = top-to-bottom. When the loop ends, v has no path-parent, meaning its auxiliary tree contains the represented root; and after the final SPLAY, v sits at that tree's root with the whole root-to-v path in its left subtree.

Pause

Pause and think: what does Access(v) accomplish in the represented tree? In the represented tree, ACCESS(v) makes every edge along the path from the root down to v preferred, while making the edge leaving v toward any deeper child non-preferred. In the auxiliary forest, this merges all the depth-ordered splay trees along the root-to-v path into a single splay tree, places v at the root of that tree, and ensures that v's right child is empty (no deeper nodes on the preferred path). Everything on the root-to-v path now lives in v and its left subtree.

22.5.2 A hand trace

Trace it yourself

Start from the nine-node forest of Section 22.2, with preferred path $1-3-6-7$. The auxiliary trees, drawn as their in-order (equivalently, depth order):

preferred path	auxiliary BST and its path-parent
$\{1,3,6,7\}$	BST over $1<3<6<7$; path-parent = NULL (holds the root)
$\{2,4\}$	BST over $2<4$; path-parent of top node 2 is 1
$\{5\}$	singleton; path-parent of 5 is 3
$\{8\}$	singleton; path-parent of 8 is 6
$\{9\}$	singleton; path-parent of 9 is 7

Now call ACCESS(4). Walk it:

1. Splay 4 inside its tree $\{2,4\}$; now 4 is the BST root with left child 2. Node 4 has no right child, so nothing to detach.
2. The path-parent now sits at the root 4 and equals 1 (it followed the topmost path node 2's old edge to 1). Hop to $w=1$.
3. Splay 1 in the big tree $\{1,3,6,7\}$. Now 1 is the BST root; its right subtree is $\{3,6,7\}$ (everything deeper than 1).
4. 1 has a right child, so the path $\{3,6,7\}$ loses preferred status: set its root's path-parent to 1, detach it. Attach 4's tree as 1's new right child. The preferred path containing the root is now $1-2-4$.
5. 4 has no further path-parent. Splay 4 to the top. Done.

Two path-parent rewires, one access. The resulting preferred edges:

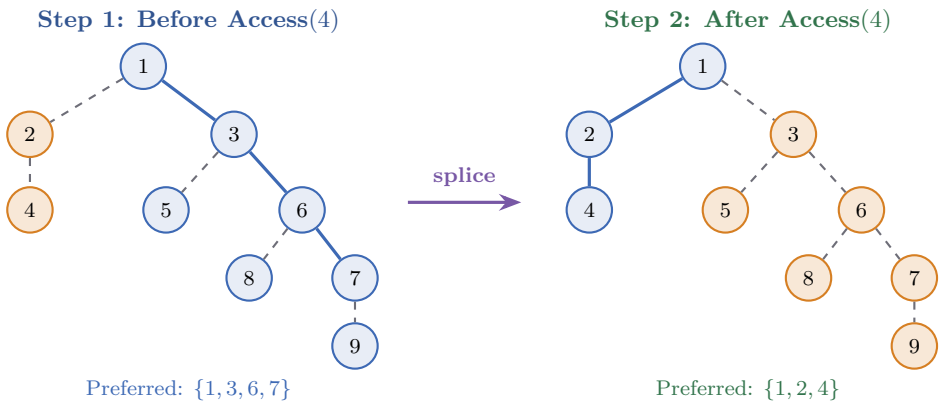


Figure 22.2: The self-adjusting transition performed by ACCESS(4). **Left:** Before access, the root path $1-3-6-7$ is preferred. **Right:** Splaying 4 and ascending path-parent pointers to 1 swaps the preferred child of 1 from 3 to 2. The old chain $\{3,6,7\}$ is severed into a separate auxiliary tree hanging off 1 via a path-parent pointer, and $1-2-4$ becomes the active preferred path.

22.6 Everything else in terms of access

Once ACCESS is in hand, the represented-forest operations are short.

FindRoot. ACCESS(v) puts v's root path into one auxiliary tree with v at the top. The represented root is the shallowest node, i.e. the leftmost node of that tree. Walk left children to the bottom, then splay it (to keep the tree balanced and the amortized bound intact) and return it.

Algorithm 87: FINDROOT(v)

```
Access(v)
r ← v
while r.left ≠ NULL do
    PUSH(r); r ← r.left                    // descend to the shallowest node
Splay(r)
return r
```

Link. LINK(u, v) with u a represented root: in the re-rooting variant, EVERT u so it becomes the root of its own tree, access v, then set u's path-parent to v, hanging u's whole tree off v as a non-preferred child.

Algorithm 88: LINK(u, v) – attach u under v (re-rooting variant)

```
Evert(u)                                   // make u the root of its own tree
Access(v)
u.pathparent ← v                           // u hangs off v as a non-preferred child
```

Cut. CUT(u) removes the edge to u's parent. Access u; its left subtree is exactly the represented path strictly above u, i.e. u's ancestors. Detach that left subtree – it becomes its own tree, rooted at the old represented root – and u becomes a new represented root.

Algorithm 89: CUT(u) – detach u from its parent

```
Access(u)
if u.left ≠ NULL then
    u.left.parent ← NULL
    u.left ← NULL
recompute aggregate at u
```

Evert (re-root). To make v the root, access v (bringing the root-to-v path into one tree with v at the top and the path in its left subtree), then *reverse* that path. Reversal is the standard lazy "flip" bit on the auxiliary tree root that swaps left and right subtrees on push-down – the same trick used in a treap or splay tree to support sequence reversal. After flipping at v, depth order is inverted, so v (formerly deepest) becomes shallowest, i.e. the new root.

Algorithm 90: EVERT(v) – make v the root of its tree

```
1 Access(v)
2 v.flip ← ¬ v.flip                          // lazily reverse the root-to-v path
```

Path queries. For a query on the path between u and v (sum, min, max, or a range update): EVERT(u) then ACCESS(v). Now the auxiliary tree rooted at v holds *exactly* the nodes of the u–v path, in order. Read the aggregate stored at the root v; for an update, set a lazy tag at v. If you only need root-to-v aggregates (rooted variant, no re-rooting), a bare ACCESS(v) already isolates the path in v's tree. Figure 22.3 shows how CUT(u) and EVERT(u) + ACCESS(v) act directly on the auxiliary splay tree.

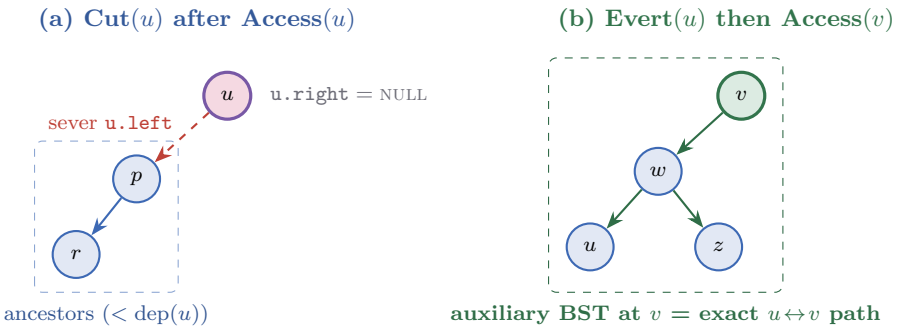


Figure 22.3: Auxiliary-tree surgery for CUT and path queries. (a) After ACCESS(u), u sits at the auxiliary root with `u.right` = NULL and all ancestors on the root-to-p path in `u.left`; nulling `u.left` cuts edge (p, u). (b) EVERT(u) makes u the represented root (flipping its splay tree), after which ACCESS(v) gathers the entire $u \leftrightarrow v$ path into the single auxiliary BST rooted at v.

Common trap

EVERT only works if your aggregate is *reversible*. Sum, min, and max are symmetric, so flipping the path order does not change them. But a *non-commutative* aggregate (matrix products, or "leftmost occurrence") must store *two* accumulators per subtree – one for each direction – and swap them when the flip bit is pushed, exactly as with a reversible segment tree. Forgetting this is the classic link-cut bug: queries silently return the answer for the reversed path.

Why it works

Why is EVERT(u); ACCESS(v) exactly the u–v path? After EVERT, u is the root of the represented tree, so the represented path from the root to v *is* the path from u to v. ACCESS(v) then gathers precisely the root-to-v path into one auxiliary tree with v on top. The aggregate at v summarizes its whole auxiliary subtree, which is that path and nothing else.

Pause

Pause and think: why does swapping left and right subtrees reverse the path? In an auxiliary splay tree, the binary search tree key is the depth in the represented tree: in-order

traversal lists nodes from shallowest to deepest. Flipping the left and right children of every node inverts the in-order sequence completely. The deepest node (v) becomes the shallowest, and parent-child orientations along the entire path invert without touching any off-path nodes!

22.7 Correctness: the invariants never break

Theorem 22.1 (Structural correctness)**.** *Starting from singletons, any sequence of* MakeTree, Link, Cut, Evert, Access *(with legal preconditions) maintains the two invariants of Section 22.3, and the represented forest defined by "preferred edges + path-parent edges" equals the abstract forest these operations specify.*

Proof sketch. We argue each primitive preserves both invariants; Access is the only nontrivial one and the others are built on it.

Splay preserves invariants. Rotations inside an auxiliary tree change BST shape but not the set of nodes nor their symmetric order, so the depth-ordering invariant holds. The path-parent of a tree must always sit at the root; whenever splaying moves a node to the root we transfer the path-parent pointer with it (a constant-time fix in the Splay wrapper). The gluing invariant is otherwise untouched: no path-parent target moves between trees.

Access preserves invariants. By the loop-invariant argument in Section 22.5: each iteration converts one non-preferred edge into a preferred edge and one preferred edge into a non-preferred one, with the path-parent pointer relocated to the new tree root. Attaching the incoming tree as the parent's *right* subtree respects depth order because the parent is strictly shallower than every incoming node. The set of represented edges (preferred $\cup$ path-parent) is unchanged throughout – we only relabel which are preferred – so the represented forest is invariant.

Link, Cut, Evert are compositions of Access with one $O(1)$ pointer edit (set a path-parent, null a left child, toggle a flip bit), each of which matches the abstract semantics: Link adds the single edge $u \to v$ as a path-parent; Cut deletes the single edge above u by nulling the left subtree that holds u's ancestors; Evert reverses the root-to-v chain, which is precisely re-rooting at v. Lazy flip bits are pushed down before any structural read, so reads see the correct orientation. □

Remark 22.1. The lazy flip bit deserves the same discipline as in a balanced-BST sequence structure: *push down* (apply and clear the tag, swapping children, and forwarding the tag to both children) before you recurse into or rotate around a node. Splay's rotations must push down the two nodes involved first, or the ordering invariant silently corrupts.

22.8 The amortized analysis

We now show that each Access costs $O(\log n)$ amortized, so every represented-forest operation does too.

The cost of one Access has two parts: (a) the number of splay steps, dominated by the number of *preferred-child changes* (one splice per loop iteration), plus (b) the splay work itself. We bound both.

22.8.1 Counting preferred-child changes

Lemma 22.1 (Few preferred changes, amortized)**.** *Over any sequence of m accesses on an n-node forest built by links and cuts, the total number of preferred-child changes is $O\big((m+n)\log n\big)$.*

The proof is a *heavy-light* counting argument. We give the intuition; the full charge is folded into the splay potential below.

Intuition

Classify each represented edge as *heavy* (the child's subtree is more than half the parent's subtree) or *light* otherwise, exactly as in heavy-light decomposition. Any root-to-node path crosses $O(\log n)$ light edges, because each light edge at least halves the subtree size as you go up. Charge each preferred-child change either to the splay potential (if it happens along a heavy edge – and there are few heavy edges on any path) or to a light edge it crosses. Summing over the path gives $O(\log n)$ changes per access plus a global term for heavy edges, which telescopes to $O((m+n)\log n)$ overall.

22.8.2 The splay potential argument

Weight every node by 1, and define the *size* $s(x)$ of an auxiliary-tree node x to be the total number of *represented* descendants reachable through x – including those in auxiliary subtrees hanging off x by path-parent pointers. With this choice $s(\text{represented root}) = n$. Put $r(x) = \log_2 s(x)$ and $\Phi = \sum_x r(x)$, summed over all nodes of all auxiliary trees.

Theorem 22.2 (Sleator–Tarjan)**.** *With this potential, one* Access *on an n-node link-cut forest costs $O(\log n)$ amortized. Hence* FindRoot*,* Link*,* Cut*,* Evert*, and a path query each run in $O(\log n)$ amortized time, and a sequence of m operations runs in $O\big((m+n)\log n\big)$ total.*

Proof idea. Fix the access to v. The work is a sum of splays, one per preferred path on the root-to-v route. Apply the Access Lemma to each splay: the splay of v inside a path whose auxiliary root is t costs at most $3\big(r(t) - r(v)\big) + 1$ amortized. When we then splice v's tree under the parent w and splay w, the rank $r(w)$ is at least the old $r(t)$ (it is shallower, hence has a larger represented subtree), so consecutive $3(r(\cdot) - r(\cdot))$ terms *telescope*:

$$\sum_{\text{paths } t_1, t_2, \ldots \text{ up to the root}} 3\big(r(t_i) - r(v)\big) \;\le\; 3\big(r(\text{represented root}) - r(v)\big) \;+\; (\#\text{splices}).$$

Since $s(\text{represented root}) = n$, we get $r(\text{root}) \le \log_2 n$, so the telescoped sum is $O(\log n)$ plus one unit per splice. The number of splices is the number of preferred-child changes on this access; the $+1$ units are absorbed by Lemma 22.1, whose total over all m accesses is $O\big((m+n)\log n\big)$. Dividing by m gives $O(\log n)$ amortized per access. Each derived operation is one access plus $O(1)$ extra splay or pointer work, so the bound carries over.

The $+n$ term accounts for the initial potential and for Links, which can raise a represented subtree (and hence ranks) by an amount bounded by the $\log n$ rank ceiling. □

Why it works

The single most important line is the telescoping. Because depth order makes each spliced-in parent shallower, its rank only goes *up* as we climb, so the $-r(v)$ from one path cancels against the $+r(t)$ of the next. The cost of an entire access collapses to "rank of the root minus rank of v," a quantity bounded by $\log_2 n$. The same argument makes a single splay $O(\log n)$; link-cut trees just run it once per preferred path and let the ranks chain together.

Pause

Pause and think: why does access pay for both preferred and non-preferred splices? Each step of Access that flips a non-preferred edge into a preferred edge costs real work, but decreases the number of heavy non-preferred edges along the path. By defining the potential as $\sum \log \text{size}(x)$, the telescoping sum of splay operations absorbs the cost of all preferred updates, proving that the amortized cost per operation remains $O(\log n)$ regardless of tree shape!

Cost summary

Per operation (amortized):

Access, FindRoot	$O(\log n)$
Link, Cut, Evert	$O(\log n)$
path aggregate / path update	$O(\log n)$

Space: $O(n)$ – a constant number of pointers, an aggregate, and a lazy tag per node. **Worst case for a single op:** $\Theta(n)$ (a splay can be linear once), but the amortized average is $O(\log n)$. Variants built on globally biased or balanced BSTs achieve $O(\log n)$ *worst case* per operation at the cost of much heavier bookkeeping.

Theoretical milestones and variants. Daniel Sleator and Robert Tarjan introduced link-cut trees in "A Data Structure for Dynamic Trees" (JCSS, 1983) to speed up the blocking-flow phases of maximum-flow algorithms. The companion paper that gave us splay trees (Sleator–Tarjan, 1985) supplied the self-adjusting auxiliary structure that makes the modern link-cut tree so concise to implement. The original 1983 version used *biased* balanced binary trees for a worst-case bound; the splay-based version is the one almost everyone codes today. Tarjan and Werneck's experimental study (2009) compares link-cut trees against Euler-tour trees and top trees and is the standard reference for practical dynamic-tree performance.

22.9 Applications of link-cut trees

Application

[Blocking flows in $O(VE \log V)$] Dinic's algorithm finds a blocking flow in each phase by repeatedly pushing along augmenting paths in the level graph. The naive blocking-flow inner loop is $O(VE)$ per phase. Sleator and Tarjan's original motivation: maintain the current set of admissible edges as a link-cut forest where each node points to the next node on a potential augmenting path, store residual capacities as edge values, and use PathAggregate to find the bottleneck (minimum residual) along the whole path at once, then a path update to subtract the flow, then a Cut on every saturated edge. This drops the blocking-flow phase to $O(E \log V)$ and the whole max-flow to $O(VE \log V)$, beating plain $O(V^2E)$ Dinic on dense graphs. See Chapter 12.

Application

[Online dynamic connectivity on forests] For a forest under edge insert/delete with connectivity queries, FindRoot(u) = FindRoot(v) answers "same tree?" in $O(\log n)$. With Evert you also get $O(\log n)$ tree-path queries (distance, max edge, sum). This is the forest case of the general dynamic connectivity problem; for *general* graphs one layers a spanning forest of link-cut (or Euler-tour) trees underneath, replacing tree edges with non-tree edges on deletion (Holm–de Lichtenberg–Thorup), which Chapter 39 develops further.

Application

[Online LCA, MST verification, and more] Link-cut trees give online *lowest common ancestor*: Access(u) then Access(v) – the last node at which the second access performed a splice is the LCA. They support online *maximum-edge-on-path* queries, which is the inner loop of incremental minimum-spanning-tree maintenance and of MST verification (is edge e heavier than every tree edge on the cycle it would close?). Whenever a problem is "trees that change, plus path questions," link-cut trees are the standard tool.

Connection

The line from heavy-light decomposition (Chapter 19) to link-cut trees is a single conceptual move: replace the *static* heavy-chain decomposition with a *self-adjusting* preferred-path decomposition, and replace the per-chain segment tree with a per-path splay tree. Euler-tour trees (Chapter 21) solve a *subset* of the same problems (subtree aggregates, connectivity) but cannot do arbitrary path aggregates – the two structures are complementary, and *top trees* (a generalization built on top of link-cut trees) unify them.

The amortized argument here – a potential equal to a sum of logs of subtree sizes, telescoping along a root path – is the same one used for splay trees and, loosely, for the union-by-rank/path-compression analysis of disjoint-set forests. When you reach Chapters 31–35 and 41, note the contrast: link-cut trees are a *combinatorial* dynamic-tree structure, whereas dynamic *electrical* quantities (effective resistances on a changing graph) need the Laplacian machinery, where "path" is replaced by "current" and "min on a path" by "potential drop."

22.10 Implementation checklist

Focus

A correct link-cut tree node carries:

- `parent`, `left`, `right` – auxiliary-tree links. Whether a node is a splay root is detected by "my parent's children do not include me," so `parent` doubles as the path-parent at a root.
- `value` and an `aggregate` over the auxiliary subtree (sum/min/max), recomputed on every structural change via PULL.
- a `flip` lazy bit (for EVERT) and any range-update lazy tags, applied via PUSH before recursing or rotating.

The two functions you must call religiously: PUSH(x) (apply pending lazy tags, propagate to children) *before* reading children, and PULL(x) (recompute the aggregate from children) *after* changing them. Splay rotations must PUSH both nodes first and PULL both after, bottom node before top.

Common trap

Three recurring bugs:

1. **Forgetting the root test.** Many implementations fold the path-parent into `parent` and decide "is this node a splay root" by checking whether the parent claims it as a child. Get this predicate wrong and splay either stops early (leaving the wrong node at the top) or rotates across a path-parent edge (corrupting the gluing invariant).
2. **Pushing in the wrong order during rotation.** You must PUSH grandparent, parent, then child top-down before rotating, or a stale `flip` bit reverses a subtree you have already read.
3. **Non-reversible aggregates under Evert.** As warned above, store both directions for non-commutative monoids.

Connection

What This Made Possible: Link-cut trees were the first practical $O(\log n)$ amortized dynamic forest data structure. Besides speeding up maximum network flow to $O(VE \log V)$, they made possible online dynamic minimum spanning tree maintenance in $O(m \log n)$ time (greedily cutting the heaviest edge on a cycle whenever a new edge is added), and online connectivity queries for dynamic forests.

Limitations:

- *Amortized, not worst-case:* Splay-based link-cut trees achieve $O(\log n)$ amortized cost. While Sleator and Tarjan later developed a worst-case $O(\log n)$ variant using globally biased search trees, it is notoriously complicated, with formidable constant factors and code complexity that deter practical adoption.
- *Subtrees are difficult:* Link-cut trees natively navigate root-to-node paths. Maintaining aggregates over *subtrees* (such as the size or sum of an entire branch) requires tracking

"virtual subtree" pointers that must be laboriously updated on every preferred-child splice.

- *Forests only:* Link-cut trees cannot accommodate cycles. A single cycle causes ACCESS to loop endlessly across path-parent edges.

Where We Go Next (Lowest Common Ancestor and Range Structures): Throughout our study of tree decompositions – from heavy-light chains to Euler tours and dynamic link-cut trees – one primitive has appeared repeatedly: finding the *Lowest Common Ancestor* (LCA) of two nodes. While link-cut trees compute LCAs dynamically through successive accesses, static trees admit simpler solutions: binary lifting in $O(\log n)$ query time, and the Bender–Farach-Colton reduction to ± 1 Range Minimum Queries, answering LCA queries in $O(1)$ worst-case time after linear $O(n)$ preprocessing. In Chapter 23, we examine these static range structures and LCA algorithms, completing our repertoire for tree queries.

22.11 Exercises

Exercise 22.1 (Quick mental check: splay key invariant)**.** What is the search key in an auxiliary splay tree? Can a node ever have a smaller search key than its ancestor in the represented tree?

Exercise 22.2 (Quick mental check: path-parent vs parent)**.** In a link-cut tree node, what distinguishes a normal parent pointer from a path-parent pointer? How does a splay root detect that it is at the root of an auxiliary tree?

Exercise 22.3. Implement the splay-based link-cut tree with MAKETREE, LINK, CUT, FINDROOT, EVERT, and a path-sum query. Test it against a brute-force forest (recompute paths by walking parent pointers) on 10^4 random operations.

Exercise 22.4. Prove that after ACCESS(v) the leftmost node of v's auxiliary tree is the represented root of v's tree. Where in the proof do you use the depth-ordering invariant?

Exercise 22.5. Trace ACCESS(9) on the post-ACCESS(4) forest drawn in Section 22.5. List every path-parent rewire and draw the final preferred-edge set. How many preferred-child changes occurred?

Exercise 22.6. (Break it.) Implement EVERT via the flip bit but *forget* to push the flip down inside SPLAY's rotation. Construct a small sequence of operations on which a path-sum query returns a wrong answer, and explain which invariant was violated.

Exercise 22.7. Extend the structure to support a *path range-add*: add a constant c to every node on the u–v path. Specify the lazy tag, its PUSH rule, and how it interacts with the `flip` bit. Prove the aggregate stays correct.

Exercise 22.8. Show how to support *subtree* aggregates (the sum over the whole represented subtree of v) in a link-cut tree. (Hint: this is genuinely harder than path aggregates; you must maintain a separate "virtual subtree" aggregate that the ACCESS splices keep up to date. Compare with Euler-tour trees, which handle subtrees natively but not paths.)

Exercise 22.9. Give the LCA algorithm using two accesses precisely. Prove that the node returned is the lowest common ancestor, and bound its cost. What goes wrong if the tree is unrooted because you have been calling EVERT?

Exercise 22.10. Use a link-cut tree to maintain the maximum edge weight on the path between two nodes of a growing tree, and use it to verify in $O(m \log n)$ whether a given spanning tree is a minimum spanning tree of an m-edge graph. Describe the values you store – recall link-cut trees store *node* values, so how do you encode edge weights?

Exercise 22.11. (Worst case.) The splay-based structure is only $O(\log n)$ *amortized*. Sketch how biased binary trees give an $O(\log n)$ *worst-case* bound per operation, and explain why the amortized version is nonetheless preferred in practice.

Exercise 22.12. Prove the telescoping step of Theorem 22.2 carefully: state precisely why $r(w) \geq r(t)$ when w is the spliced-in parent of a path with auxiliary root t, and where the "+1 per splice" units come from in the Access Lemma.

Chapter 23

Lowest Common Ancestor and Range Structures

Chapter 22 handled trees that change under link and cut operations. When the tree is fixed in advance, precomputed static structures are both faster and simpler.

Take a rooted tree and pick two nodes u and v. Trace the path from each up toward the root. The two paths run separately for a while, then meet, and from the meeting point upward they coincide. That first shared node – the deepest node that is an ancestor of both – is the *lowest common ancestor* of u and v, written $\mathrm{lca}(u, v)$.

The LCA is one of the most useful primitives on rooted trees. The distance between two nodes in a weighted tree is $\mathrm{dep}(u) + \mathrm{dep}(v) - 2\,\mathrm{dep}(\ell)$ where $\ell = \mathrm{lca}(u, v)$. "Is u an ancestor of v?" is just "$\mathrm{lca}(u, v) = u$?". Path queries, auto-completion tries, suffix-tree string matching, phylogenetics, and many tree problems in competitive programming reduce to LCA calls. So we want it fast: after linear or near-linear preprocessing, answer each query in $O(\log n)$ or even $O(1)$ time.

This chapter covers two topics. The first is LCA itself, by two routes: *binary lifting*, which precomputes ancestor pointers at power-of-two heights and climbs in $O(\log n)$ per query; and the *Euler tour* reduction, which flattens the tree into an array and turns each LCA query into a *range-minimum query* (RMQ) over that array. The second is the family of *range structures* – sparse tables, segment trees, and Fenwick (binary indexed) trees – that answer "summarize this contiguous block" queries. RMQ is one such question; sums, maxima, and assignment-on-ranges are others. Nearly every tree algorithm in this part of the book uses these structures, so we build them carefully and prove their bounds. The chapter closes by showing that LCA and RMQ are *the same problem* up to linear-time transformations in either direction.

We trace a concrete eleven-node tree by hand, all the way through, watching every table fill in.

History

The Lowest Common Ancestor problem emerged in compiler design and evolutionary biology. In 1973, Alfred Aho, John Hopcroft, and Jeffrey Ullman identified LCA as the computational bottleneck in compiler optimization (finding dominators in control-flow graphs) and phylogenetics. They proposed an algorithm with $O(n \log n)$ preprocessing and $O(1)$ query time using tree contractions.

In 1984, Dov Harel and Robert Tarjan showed that LCA could be solved in strictly linear preprocessing $O(n)$ and $O(1)$ worst-case query time. However, their construction was notoriously intricate: it required depth-first Cartesian tree decompositions, bitwise interleaving, and recursive tree contractions that few practitioners could implement without bugs. For sixteen years, it was widely regarded as a "galactic algorithm" – theoretically optimal, yet avoided in practice in favor of binary lifting ($O(n \log n)$ preprocessing, $O(\log n)$ query).
The breakthrough that made $O(n)$ preprocessing practical arrived in 2000, when Michael Bender and Martin Farach-Colton realized that an Euler tour flattens the tree into an array where adjacent depths differ by *exactly* ± 1. Because differences are constrained to ± 1, micro-blocks of length $b = \frac{1}{2}\log_2 n$ have only $2^b = \sqrt{n}$ distinct normalized shapes. By the Method of the Four Russians, one can precompute a tiny lookup table for all $\sqrt{n}$ shapes, achieving $O(n)$ preprocessing and $O(1)$ queries in a simple, fifty-line algorithm. Today, whether dynamic trees can support worst-case $O(1)$ LCA queries under online edge insertions without amortized rebuilding remains a central open question in data structures.

23.1 Rooted Trees, Depth, and Ancestry

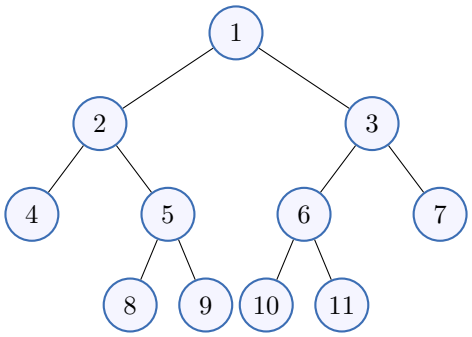
Fix a tree $T = (V, E)$ with $n = |V|$ vertices, rooted at a chosen vertex r. Rooting orients every edge: each non-root vertex v has a unique *parent* $\mathrm{par}(v)$, the neighbor of v on the path to r, and the other neighbors are its *children*. The *depth* $\mathrm{dep}(v)$ is the number of edges on the path from r to v, so $\mathrm{dep}(r) = 0$.

Definition 23.1 (Ancestor, descendant, LCA)**.** Vertex a is an *ancestor* of v if a lies on the (unique) path from r to v; equivalently $a = v$ or a is an ancestor of $\mathrm{par}(v)$. Then v is a *descendant* of a. The *lowest common ancestor* $\mathrm{lca}(u, v)$ is the ancestor of both u and v of maximum depth.

Intuition

Why is the LCA well defined and unique? The set of common ancestors of u and v always contains the root, so it is nonempty. The ancestors of any vertex form a chain ordered by depth (the root-to-vertex path), so the common ancestors – being a subset of both chains – also form a chain. A finite nonempty chain has a unique deepest element. That element is the LCA.

Throughout we run our methods on the same example so the data structures can be compared side by side.



root $r = 1$; depths:
$\mathrm{dep}(1){=}0$; $\mathrm{dep}(2){=}\mathrm{dep}(3){=}1$;
$\mathrm{dep}(4){=}\mathrm{dep}(5){=}\mathrm{dep}(6){=}\mathrm{dep}(7){=}2$;
$\mathrm{dep}(8){=}\mathrm{dep}(9){=}\mathrm{dep}(10){=}\mathrm{dep}(11){=}3$.

Figure 23.1: The running tree T. For example $\mathrm{lca}(8,9) = 5$, $\mathrm{lca}(8,4) = 2$, $\mathrm{lca}(10,7) = 3$, and $\mathrm{lca}(8,11) = 1$.

We write par for the parent array. For T,

$$\mathrm{par} = [\,-,\ 1,\ 1,\ 2,\ 2,\ 3,\ 3,\ 5,\ 5,\ 6,\ 6\,],$$

indexed $1 \ldots 11$ with par(1) undefined (the root). Computing par and dep is one depth-first or breadth-first sweep, $O(n)$ time. Everything below assumes these two arrays are in hand.

23.2 The Naive Climb, and Why We Want Better

The definition hands us an algorithm. To find $\mathrm{lca}(u, v)$, first lift the deeper of the two up to the depth of the shallower by repeatedly following parent pointers; now u and v sit at equal depth. Then move both up one step at a time until they collide.

Algorithm 91: NaiveLCA(u, v) – climb one step at a time

```
while dep(u) > dep(v) do
    u ← par(u)
while dep(v) > dep(u) do
    v ← par(v)
while u ≠ v do
    u ← par(u)
    v ← par(v)
return u
```

This is correct – once the depths agree, the two climbers stay at equal depth and their ancestor chains must meet exactly at the LCA – but a single query costs $O(n)$ on a path graph (a "bamboo"). With q queries that is $O(nq)$, hopeless for the large n and q these structures are meant for. The two big ideas of the chapter both attack the slow $O(n)$ climb: binary lifting climbs in jumps of $1, 2, 4, 8, \ldots$; the Euler tour avoids climbing entirely.

23.3 Binary Lifting

Intuition

Climbing one parent at a time is wasteful. Any number of steps k has a binary expansion: $13 = 8 + 4 + 1$. If we could jump up by exactly 8, then 4, then 1 ancestors in one move each, we would reach the 13th ancestor in three moves instead of thirteen. So precompute, for every vertex, its 2^j-th ancestor for each $j = 0, 1, \ldots, \lfloor \log_2 n \rfloor$. Then *any* climb of k steps decomposes into at most $\lceil \log_2 n \rceil$ power-of-two jumps.

Let $\mathrm{up}[v][j]$ denote the 2^j-th ancestor of v (or a sentinel 0, meaning "above the root", when no such ancestor exists). The base case is $\mathrm{up}[v][0] = \mathrm{par}(v)$. The method rests on one recurrence: the 2^j-th ancestor is the 2^{j-1}-th ancestor *of* the 2^{j-1}-th ancestor,

$$\mathrm{up}[v][j] \;=\; \mathrm{up}[\,\mathrm{up}[v][j-1]\,][\,j-1\,]. \tag{23.1}$$

Two half-jumps of length 2^{j-1} compose into one full jump of length 2^j. We fill the table column by column (all j=0 first, then all j=1, ...) so that the right-hand side of (23.1) is always already computed.

Algorithm 92: BUILDLIFTING$(\mathrm{par}, \mathrm{dep}, n)$

```
L ← ⌊log2 n⌋
for v ← 1 to n do
    up[v][0] ← par(v)                          // par(r) = 0, the sentinel
for j ← 1 to L do
    for v ← 1 to n do
        up[v][j] ← up[ up[v][j − 1] ][ j − 1 ]
return up
```

The table has $n(L{+}1) = \Theta(n \log n)$ entries, each filled in $O(1)$, so preprocessing is $\Theta(n \log n)$ time and space. We adopt the convention $\mathrm{up}[0][j] = 0$: jumping above the root stays at the sentinel, which makes the recurrence total without special cases.

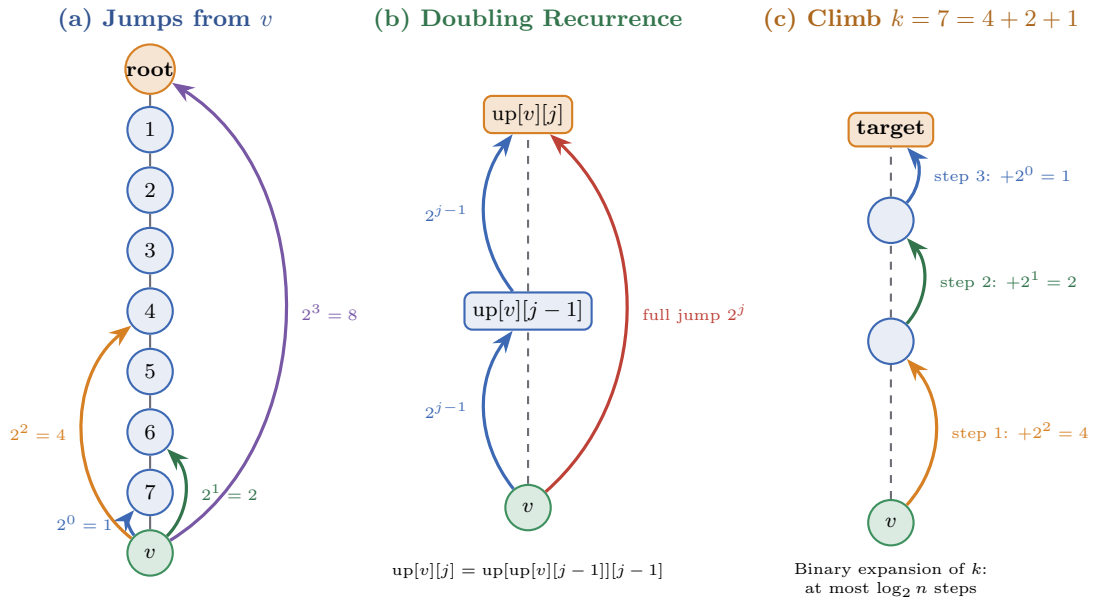


Figure 23.2: The geometry of binary lifting. **Panel A:** From any vertex v, pointers store ancestors at exponential power-of-two heights $2^0, 2^1, 2^2, 2^3, \ldots$. **Panel B:** The doubling recurrence $\text{up}[v][j] = \text{up}[\text{up}[v][j-1]][j-1]$ composes two half-jumps of length 2^{j-1} into one jump of length 2^j. **Panel C:** Climbing any distance k decomposes into the set bits of k (e.g., $7 = 4+2+1$), reaching the ancestor in at most $\lceil \log_2 n \rceil$ hops.

23.3.1 Answering a query

To compute $\text{lca}(u, v)$: first lift the deeper vertex to the depth of the shallower, using the binary expansion of the depth difference. Now both are at the same depth. If they are already equal, that is the answer. Otherwise climb both *in lockstep* by the largest jumps that keep them apart, from $j = L$ down to 0: if $\text{up}[u][j] \neq \text{up}[v][j]$ the 2^j-ancestors are still below the LCA, so jump both there; if they are equal we would overshoot, so skip that bit. After the loop, u and v sit exactly one step below the LCA, and $\text{par}(u)$ is the answer.

Algorithm 93: LIFTLCA(u, v)

```
if dep(u) < dep(v) then
    swap u ↔ v                                  // make u the deeper one
Δ ← dep(u) − dep(v)
for j ← L to 0 do
    if bit j of Δ is set then u ← up[u][j]
if u = v then return u
for j ← L to 0 do
    if up[u][j] ≠ up[v][j] then
        u ← up[u][j]
        v ← up[v][j]
return up[u][0]
```

Why it works

Why the lockstep descent lands exactly below the LCA. Let $\ell = \mathrm{lca}(u, v)$ after the depths are equalized; say both vertices are at depth d and ℓ is at depth $d' \le d$, so $k = d - d'$ steps separate each of u, v from ℓ. We claim the second loop drives both to depth $d'+1$, i.e. to the two distinct children of ℓ on the respective paths.
The invariant maintained is: *u and v stay at equal depth, and neither has reached ℓ (so $u \neq v$, and $\mathrm{up}[u][\cdot]$, $\mathrm{up}[v][\cdot]$ are genuine ancestors).* A jump at height j is taken exactly when $\mathrm{up}[u][j] \neq \mathrm{up}[v][j]$. If the two 2^j-ancestors differ they are both strictly below ℓ (equal-depth proper ancestors that are not yet the LCA cannot have met), so jumping preserves the invariant. If they are equal, that common ancestor is at or above ℓ, and jumping would overshoot, so we correctly skip. Processing j from high to low realizes the binary expansion of $k - 1$: the loop reduces the remaining gap to exactly 1. Hence u, v end as the two children of ℓ, and $\mathrm{up}[u][0] = \ell$.

Each loop runs $L+1$ times with $O(1)$ work, so a query is $O(\log n)$.

Pause

Pause and think: why does binary lifting aim for the children of the LCA, rather than the LCA itself? If we jumped whenever $\mathrm{up}[u][j] = \mathrm{up}[v][j]$, any jump that lands at or above the LCA would look identical! For example, the root is an ancestor of every node, so $\mathrm{up}[u][L] = \mathrm{up}[v][L] = \text{root}$ tells us nothing about whether we jumped too far. By jumping only when $\mathrm{up}[u][j] \neq \mathrm{up}[v][j]$, we are guaranteed that both nodes remain strictly below the LCA. After testing all powers of two from L down to 0, the nodes must stop at distance exactly 1 below the LCA, making their parent $\mathrm{up}[u][0]$ the exact answer.

Trace it yourself

Build the lifting table for T (Figure 23.1). Here $n = 11$ so $L = \lfloor \log_2 11 \rfloor = 3$. Column j=0 is just par. Column j=1 applies the recurrence $\mathrm{up}[v][1] = \mathrm{par}(\mathrm{par}(v))$, and so on:

v	1	2	3	4	5	6	7	8	9	10	11
up[v][0]	0	1	1	2	2	3	3	5	5	6	6
up[v][1]	0	0	0	1	1	1	1	2	2	3	3
up[v][2]	0	0	0	0	0	0	0	0	0	0	0
up[v][3]	0	0	0	0	0	0	0	0	0	0	0

(The whole tree has depth 3, so the 4th ancestor of every vertex is already above the root: rows j=2 and j=3 are all sentinel.) Now query $\mathrm{lca}(8, 11)$. Both have depth 3, so $\Delta = 0$ and the first loop does nothing; they are unequal. Second loop, $j = 3$: $\mathrm{up}[8][3] = \mathrm{up}[11][3] = 0$ equal, skip. $j = 2$: $\mathrm{up}[8][2] = \mathrm{up}[11][2] = 0$ equal, skip. $j = 1$: $\mathrm{up}[8][1] = 2 \neq 3 = \mathrm{up}[11][1]$, jump: $u \leftarrow 2$, $v \leftarrow 3$. $j = 0$: $\mathrm{up}[2][0] = \mathrm{up}[3][0] = 1$ equal, skip. Answer: $\mathrm{up}[u][0] = \mathrm{up}[2][0] = 1$. Correct.
Try $\mathrm{lca}(10, 7)$ yourself: lift 10 (depth 3) up by $\Delta = 1$ to its parent 6 (depth 2), then lockstep with 7 to reach 3.

Common trap

Three classic binary-lifting bugs. (1) Forgetting the sentinel: if $\mathrm{par}(r)$ is left undefined rather than set to a node 0 with $\mathrm{up}[0][j] = 0$, the recurrence dereferences garbage when a jump exceeds the root. (2) Equalizing depths using the *wrong* vertex – you must lift the *deeper* one; swap first. (3) In the second loop, jumping when $\mathrm{up}[u][j] = \mathrm{up}[v][j]$. That moves both to a common ancestor at or above the LCA and you lose the answer; the test must be "jump only when they *differ*."

Application

Binary lifting is not only for LCA. The same up table answers "the k-th ancestor of v" in $O(\log n)$ by reading off the bits of k, which powers level-ancestor queries, jump pointers in tries, and the "walk up to the highest ancestor with property P" searches inside heavy-light decomposition (Chapter 19) and on functional graphs (where each node has exactly one outgoing edge and $\mathrm{up}[v][j]$ is the 2^j-th iterate of the successor function).

23.4 Range Minimum Queries and Sparse Tables

The second LCA method relies on range minimum queries. The *range minimum query* problem: preprocess an array $A[1 \ldots m]$ so that for any query interval $[\ell, r]$ we can return $\min_{\ell \le i \le r} A[i]$ (and, for LCA, the *position* achieving it) quickly.

Intuition

Minimum has a property that sum does not: it is *idempotent*, $\min(x, x) = x$. So overlapping ranges may be combined freely – covering $[\ell, r]$ with two blocks that *overlap* in the middle changes nothing, because counting an element twice in a min is harmless. This is why RMQ can be answered in $O(1)$ after $O(m \log m)$ preprocessing; sums cannot be handled this way.

The *sparse table* precomputes the minimum of every block whose length is a power of two. Let $\mathrm{sp}[i][j] = \min\{A[i], \ldots, A[i + 2^j - 1]\}$, the minimum of the length-2^j block starting at i. The doubling recurrence mirrors binary lifting: a block of length 2^j is two abutting blocks of length 2^{j-1},

$$\mathrm{sp}[i][j] \;=\; \min\big(\,\mathrm{sp}[i][j-1],\; \mathrm{sp}[i + 2^{j-1}][j-1]\,\big). \tag{23.2}$$

Algorithm 94: BuildSparse(A, m)

```
L ← ⌊log2 m⌋
for i ← 1 to m do sp[i][0] ← A[i]
for j ← 1 to L do
    for i ← 1 to m − 2^j + 1 do
        sp[i][j] ← min(sp[i][j − 1], sp[i + 2^(j−1)][j − 1])
return sp
```

To answer $[\ell, r]$, let $len = r - \ell + 1$ and $j = \lfloor \log_2 len \rfloor$. Two blocks of length 2^j – one anchored at ℓ, one ending at r – together cover $[\ell, r]$, since $2^j > len/2$ means each block covers more than half the

interval and they overlap in the middle. By idempotency the overlap is harmless:

$$\textsc{Query}(\ell, r) \;=\; \min\big(\,\mathrm{sp}[\ell][j],\; \mathrm{sp}[r - 2^j + 1][j]\,\big). \tag{23.3}$$

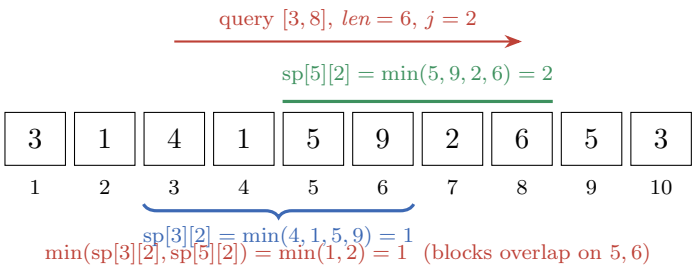


Figure 23.3: A sparse-table query on $A = [3, 1, 4, 1, 5, 9, 2, 6, 5, 3]$. The two length-4 blocks covering $[3, 8]$ overlap on positions $5, 6$; idempotency of min makes the double-count harmless.

Why it works

Why two blocks always suffice. With $j = \lfloor \log_2 \mathit{len} \rfloor$ we have $2^j \le \mathit{len} < 2^{j+1}$, so $2^j > \mathit{len}/2$. The first block covers $[\ell, \ell + 2^j - 1]$ and the second covers $[r - 2^j + 1, r]$. Their union starts at ℓ and ends at r; they overlap because $\ell + 2^j - 1 \ge r - 2^j + 1$ rearranges to $2 \cdot 2^j \ge \mathit{len} + 1$, which holds since $2^j > \mathit{len}/2$ and the two sides differ by an integer. No gap, hence full coverage, hence the answer is exactly (23.3).

Cost summary

Sparse-table RMQ: $\Theta(m \log m)$ preprocessing time and space; $O(1)$ per query (one lookup of $\lfloor \log_2 \mathit{len} \rfloor$, precomputable into a small array, plus one min). The catch: it is *static*. Updating an array entry can invalidate $\Theta(\log m)$ table cells with no cheap repair; sparse tables are for immutable arrays. For updates, use a segment tree (Section 23.6). Idempotency is essential – a sparse table computes min, max, gcd, bitwise-and/or in $O(1)$, but *not* sum, because double-counting the overlap would corrupt a sum.

Pause

Pause and think: why does idempotency allow $O(1)$ queries? An operator $\oplus$ is idempotent if $x \oplus x = x$. Because $\min(x, x) = x$, we can cover an interval $[\ell, r]$ with two overlapping dyadic blocks of length 2^j. Elements that appear in both blocks are evaluated twice, but the minimum is unchanged! For non-idempotent operations like addition ($x + x \ne x$), overlapping ranges double-count the shared elements, forcing Range Sum Queries to partition $[\ell, r]$ into disjoint blocks (costing $O(\log n)$ via segment trees or Fenwick trees).

Trace it yourself

On $A = [3, 1, 4, 1, 5, 9, 2, 6, 5, 3]$ ($m = 10$, $L = 3$), the $j = 1$ row of pairwise minima is $\mathrm{sp}[\cdot][1] = [1, 1, 1, 1, 5, 2, 2, 5, 3]$ and the $j = 2$ row of length-4 minima is $\mathrm{sp}[\cdot][2] = [1, 1, 1, 1, 2, 2, 2]$. Query $[2, 9]$: $\mathit{len} = 8$, $j = 3$, $\min(\mathrm{sp}[2][3], \mathrm{sp}[2][3]) = \mathrm{sp}[2][3] = \min(A[2..9]) = 1$. Query $[7, 10]$: $\mathit{len} = 4$, $j = 2$, $\min(\mathrm{sp}[7][2], \mathrm{sp}[7][2]) = 2$. Recompute $[3, 8]$ and confirm it equals 1 as in

Figure 23.3.

23.5 Euler Tour Plus RMQ: LCA in $O(1)$

Binary lifting answers in $O(\log n)$. The Euler-tour method answers in $O(1)$ by reducing LCA to RMQ and dropping a sparse table on top.

Intuition

Walk the tree with a depth-first traversal, but write down a vertex *every time you touch it* – when you first enter, and again each time you return after finishing a child. This *Euler tour* is a sequence that goes down and up the tree. The key observation: on the segment of the tour between an occurrence of u and an occurrence of v, you cannot have left the subtree of $\mathrm{lca}(u, v)$, and you must have touched $\mathrm{lca}(u, v)$ itself (the only way from u's region to v's region is through their common ancestor). And the LCA is the *shallowest* vertex on that segment – everything else between them is deeper. So the LCA is the minimum-depth entry in a range of the tour. That is an RMQ. Figure 23.4 draws the tour as a depth profile so this "shallowest point in the range" reading is visible at a glance.

Concretely, run a DFS from r. Maintain three arrays:

- $\mathrm{E}[1 \dots 2n-1]$: the vertex visited at each tour step (the *Euler array*);
- $\mathrm{H}[1 \dots 2n-1]$: $\mathrm{H}[k] = \mathrm{dep}(\mathrm{E}[k])$, the depth at that step;
- $\mathrm{first}[v]$: the index of the *first* occurrence of v in E.

A DFS on T produces a tour of exactly $2n-1$ entries: each of the $n-1$ edges is traversed twice (down and up), giving $2(n-1)$ steps after the root, plus the initial visit to the root, total $2n-1$.

Algorithm 95: EulerTour(v) – emit into E, H and record first

```
append v to E; append dep(v) to H; let k be the new last index
if first[v] undefined then first[v] ← k
for each child c of v do
    EulerTour(c)
    append v to E; append dep(v) to H          // re-touch v on the way back up
```

Theorem 23.1 (LCA is RMQ on the Euler tour)**.** *For any u, v, let $a = \mathrm{first}[u]$ and $b = \mathrm{first}[v]$ with (without loss of generality) $a \le b$. Let $k^\star$ be an index in $[a, b]$ minimizing $\mathrm{H}[\cdot]$. Then $\mathrm{E}[k^\star] = \mathrm{lca}(u, v)$.*

Proof. Let $\ell = \mathrm{lca}(u, v)$. Two claims.

(i) An occurrence of ℓ lies in $[a, b]$. The tour enters the subtree of ℓ exactly once (at ℓ's first occurrence) and leaves it once, and between those moments it visits every descendant of ℓ and only those, in one contiguous block of tour-time. Both u and v are descendants of ℓ, so their first occurrences a, b both lie inside that block, which begins and ends at occurrences of ℓ. To travel in the tour from the visit of u to the visit of v, the walk must climb to ℓ and descend again, because u

and v sit in different child-subtrees of ℓ (or one is ℓ); either way some occurrence of ℓ falls in $[a, b]$. Hence $\ell = \mathrm{E}[k]$ for some $k \in [a, b]$, with $\mathrm{H}[k] = \mathrm{dep}(\ell)$.

(ii) No vertex in $[a, b]$ is shallower than ℓ. Every vertex touched between times a and b is a descendant of ℓ (the walk never leaves ℓ's subtree in that interval, by the contiguity in (i)). Descendants of ℓ have depth $\geq \mathrm{dep}(\ell)$, with equality only for ℓ itself. So $\min_{k \in [a,b]} \mathrm{H}[k] = \mathrm{dep}(\ell)$, attained exactly at occurrences of ℓ. Therefore any minimizing index $k^\star$ has $\mathrm{E}[k^\star] = \ell$. □

The recipe: build E, H, first in one DFS ($O(n)$), build a sparse table over H ($O(n \log n)$), and answer each $\mathrm{lca}(u, v)$ by an RMQ on $[\min(\mathrm{first}[u], \mathrm{first}[v]), \max(\mathrm{first}[u], \mathrm{first}[v])]$ in $O(1)$, returning the *vertex* $\mathrm{E}[\cdot]$ at the minimizing index.

Trace it yourself

Euler tour of T. A DFS visiting children in increasing order produces (with root 1):

k	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21
E	1	2	4	2	5	8	5	9	5	2	1	3	6	10	6	11	6	3	7	3	1
H	0	1	2	1	2	3	2	3	2	1	0	1	2	3	2	3	2	1	2	1	0

($2n - 1 = 21$ entries, good.) First occurrences, for $v = 1 \ldots 11$: first $= [1, 2, 12, 3, 5, 13, 19, 6, 8, 14, 16]$.
Query $\mathrm{lca}(8, 11)$: $\mathrm{first}[8] = 6$, $\mathrm{first}[11] = 16$, so RMQ on $\mathrm{H}[6 \ldots 16] = [3, 2, 3, 2, 1, 0, 1, 2, 3, 2, 3]$. The minimum is 0 at $k = 11$, and $\mathrm{E}[11] = 1$. Answer 1 – matches binary lifting.
Query $\mathrm{lca}(10, 7)$: $\mathrm{first}[10] = 14$, $\mathrm{first}[7] = 19$, RMQ on $\mathrm{H}[14 \ldots 19] = [3, 2, 3, 2, 1, 2]$, minimum 1 at $k = 18$, $\mathrm{E}[18] = 3$. Answer 3. Correct.

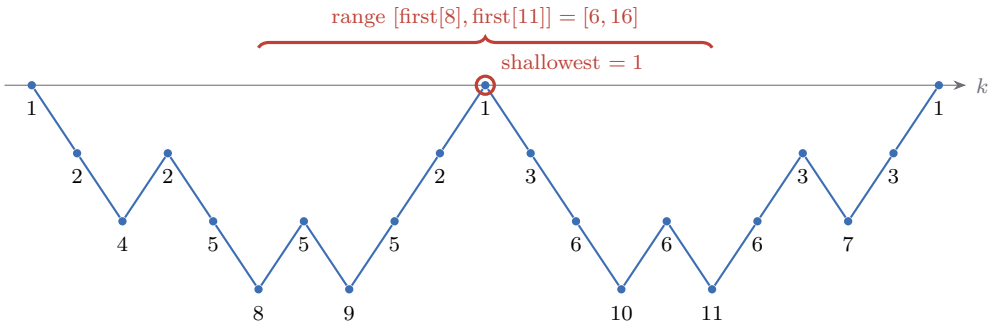


Figure 23.4: The Euler tour of T drawn as a depth profile (down is deeper). Each dot is a tour step k, plotted at height $-\mathrm{H}[k]$ and labelled with $\mathrm{E}[k]$. The query $\mathrm{lca}(8, 11)$ scans the range $[6, 16]$; its shallowest (topmost) point sits at $k = 11$, where $\mathrm{E}[11] = 1$ – the LCA.

Common trap

Query the right range. You must use *first* occurrences and order them with min / max; mixing a first occurrence of one vertex with a *later* occurrence of the other can skip past the LCA. Using $\mathrm{first}[u]$ and $\mathrm{first}[v]$ (sorted, in either original order) is what the proof relies on. Note too that the Euler array here has length $2n - 1$; a common variant emits a vertex on

every edge traversal with a trailing copy, or uses entry/exit timestamps, landing at $2n$ entries – pick one convention and never mix them.

From sparse tables to linear-time ±1 RMQ. In practice, most implementations stop at the $\langle O(n \log n), O(1)\rangle$ sparse table over the Euler tour, which takes under twenty lines of code. Closing the remaining $\log n$ gap in preprocessing without the bit-manipulation machinery of Harel–Tarjan (1984) or Schieber–Vishkin (1988) comes down to one extra property of H: every step in the Euler tour moves between a node and its parent, so consecutive depths differ by *exactly* ± 1.

Why this matters

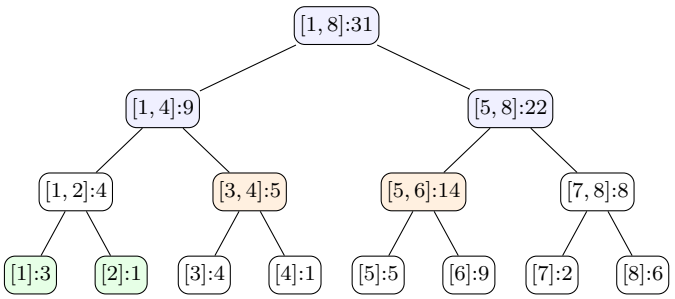
That ± 1 constraint is what enables linear preprocessing. Because adjacent depths in H differ by $+1$ or -1, a block of size $b = \frac{1}{2}\log_2 m$ is determined up to an additive offset by a sign pattern of $b-1$ bits, leaving only $2^{b-1} = O(\sqrt{m})$ distinct normalized block shapes. Precompute the in-block RMQ table for every shape once, store one minimum per block in a sparse table over the $O(m/\log m)$ blocks, and total preprocessing drops to $O(m)$ while queries stay $O(1)$. This "tabulate every small normalized block" technique is the Four-Russians method, and it recurs across algorithm design.

23.6 Segment Trees: Queries with Updates

The sparse table is static. When the array can change, we use a *segment tree*: $O(\log m)$ for both point updates and range queries, and – with lazy propagation – range updates too. It is a general-purpose range structure, working for any associative combine operation (sum, min, max, gcd, matrix product), idempotent or not.

Intuition

Recursively cut the array in half. The root owns $[1, m]$; its children own the two halves; leaves own single cells. Each internal node stores the combined value (say sum, or min) of its segment. A query interval $[\ell, r]$ splits into $O(\log m)$ *canonical* segments – whole nodes that tile $[\ell, r]$ – and we combine their stored values. An update to one cell touches only the $O(\log m)$ nodes on the root-to-leaf path above it.



***sum** query $[3, 6]$ splits into the two shaded canonical nodes $[3, 4]$=5 and $[5, 6]$=14; answer 19.*

Figure 23.5: Segment tree over $A = [3, 1, 4, 1, 5, 9, 2, 6]$ storing subtree sums. The interval $[3, 6]$ decomposes into two canonical nodes (orange).

We store the tree in a flat array $t[1 \dots 4m]$: node 1 is the root, node p's children are $2p$ and $2p{+}1$. (Size $4m$ safely covers any m; the tree has at most $2 \cdot 2^{\lceil \log_2 m \rceil} \le 4m$ nodes.) Let $\oplus$ be the combine operator with identity e (e.g. 0 for sum, $+\infty$ for min). Build recursively:

Algorithm 96: BUILD(p, ℓ, r) – node p owns $[\ell, r]$

```
1 if ℓ = r then
2 |   t[p] ← A[ℓ]
3 |   return
4 mid ← ⌊(ℓ + r)/2⌋
5 BUILD(2p, ℓ, mid)
6 BUILD(2p+1, mid+1, r)
7 t[p] ← t[2p] ⊕ t[2p+1]
```

Algorithm 97: QUERY$(p, \ell, r, q_\ell, q_r)$ – combine $A[q_\ell \dots q_r]$

```
1 if q_r < ℓ or r < q_ℓ then return e                    // node disjoint from query
2
3 if q_ℓ ≤ ℓ and r ≤ q_r then return t[p]                 // node inside query
4
5 mid ← ⌊(ℓ + r)/2⌋
6 return QUERY(2p, ℓ, mid, q_ℓ, q_r) ⊕ QUERY(2p+1, mid+1, r, q_ℓ, q_r)
```

Algorithm 98: UPDATE(p, ℓ, r, i, x) – set $A[i] \leftarrow x$

```
1 if ℓ = r then
2 |   t[p] ← x
3 |   return
4 mid ← ⌊(ℓ + r)/2⌋
5 if i ≤ mid then UPDATE(2p, ℓ, mid, i, x)
6 else UPDATE(2p+1, mid+1, r, i, x)
7 t[p] ← t[2p] ⊕ t[2p+1]
```

Why it works

Why a query touches only $O(\log m)$ nodes. Call a node *partial* if the query interval cuts through its segment without containing it (the recursion continues), *total* if its segment is contained (returns $t[p]$, no recursion), *null* if disjoint (returns e). Recursion continues only at partial nodes. Claim: at each depth of the tree, at most two nodes are partial. The query interval $[q_\ell, q_r]$ has a left endpoint and a right endpoint; a node at a given depth is partial only if one of these endpoints lies strictly inside its segment. The segments at one depth are disjoint, so q_ℓ falls in at most one and q_r in at most one – at most two partial nodes per depth. Each partial node does $O(1)$ work and examines at most two children, so the total node count is $O(\text{depth}) = O(\log m)$. UPDATE follows a single root-to-leaf path, also $O(\log m)$.

23.6.1 Lazy propagation: range updates

A point update is easy; a *range* update – "add x to every $A[i]$ with $\ell \le i \le r$" – would naively touch $\Theta(r-\ell)$ leaves. *Lazy propagation* fixes this: when an update fully covers a node's segment, apply it to the node's aggregate and park a *pending tag* on the node instead of recursing. The tag is pushed down to children only when a later operation needs to descend through the node.

Algorithm 99: RangeAdd$(p, \ell, r,\ q_\ell, q_r,\ x)$ – add x to $A[q_\ell \dots q_r]$

```
if q_r < ℓ or r < q_ℓ then return
if q_ℓ ≤ ℓ and r ≤ q_r then
    t[p] += (r − ℓ + 1) · x
    lazy[p] += x
    return
PushDown(p, ℓ, r)
mid ← ⌊(ℓ + r)/2⌋
RangeAdd(2p, ℓ, mid, q_ℓ, q_r, x)
RangeAdd(2p+1, mid+1, r, q_ℓ, q_r, x)
t[p] ← t[2p] + t[2p+1]
```

PushDown(p, ℓ, r) takes the tag lazy$[p]$, adds lazy$[p]\cdot$(child size) to each child's t and lazy$[p]$ to each child's tag, then clears lazy$[p]$. Because a tag on a fully covered node already reflects the correct aggregate in $t[p]$, and is pushed only when we descend, the invariant "$t[p]$ is correct *including* the effect of lazy$[p]$, while lazy$[p]$ holds updates not yet reflected in the children" is maintained throughout. Range update and range query are each $O(\log m)$ by the same two-partial-nodes-per-depth argument: pushing down adds $O(1)$ per visited node (Figure 23.6).

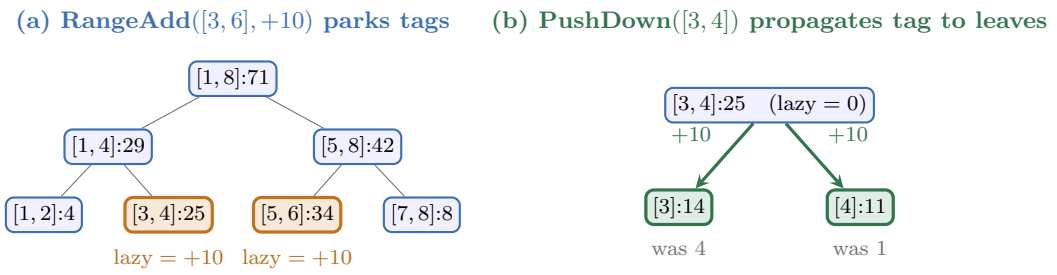


Figure 23.6: Lazy propagation on $A = [3, 1, 4, 1, 5, 9, 2, 6]$. (a) RangeAdd$([3, 6], +10)$ stops at the two canonical nodes $[3, 4]$ and $[5, 6]$, adding $2 \cdot 10 = 20$ to their sums and parking lazy $= +10$ without visiting their leaves. (b) When a later operation descends through $[3, 4]$, PushDown propagates $+10$ to leaves $[3]$ and $[4]$ and clears the tag on $[3, 4]$.

Common trap

The lazy invariant is delicate. Two ways to corrupt it: (1) querying or updating a child without PushDown-ing the parent first – the child still hides a stale tag; (2) forgetting that a node's stored aggregate $t[p]$ must *already include* its own tag. Rule of thumb: a node's $t[p]$ is always up to date; a node's lazy$[p]$ records pending updates not yet pushed to its children, propagated only on the way down. For non-commuting tag compositions (range *assign* mixed

with range *add*) the order in which composed tags apply matters; get it wrong and the bug is maddening to find.

Cost summary

Segment tree: $\Theta(m)$ build, $O(\log m)$ per point update / range query; with lazy propagation $O(\log m)$ per range update too. Space $\Theta(m)$. Works for any associative $\oplus$. As an RMQ structure it gives $\langle O(m), O(\log m)\rangle$ – slower queries than a sparse table, but it supports updates, which the sparse table cannot.

Pause

Pause and think: why does a segment tree query visit at most $2\log_2 m$ canonical nodes? At each level of the segment tree, the query range $[L, R]$ can completely cover at most two canonical intervals without their parent being completely covered. Intervals that are completely covered return immediately; only intervals that partially overlap $[L, R]$ recurse further. Since only the two boundary endpoints of $[L, R]$ can cut through intervals, at most two nodes at each of the $\log_2 m$ levels are split, bounding the query work to $O(\log m)$!

23.7 Fenwick (Binary Indexed) Trees

For the common special case of *prefix-sum* queries with point updates, the Fenwick tree is a lighter alternative: a single array, a handful of bit-twiddling lines, $O(\log m)$ per operation, and a constant factor that beats the segment tree.

Intuition

Index i in binary ends in some run of zeros; let $\text{lowbit}(i) = i \,\&\, (-i)$ be the value of its lowest set bit. Fenwick cell $F[i]$ stores the sum of the $\text{lowbit}(i)$ array elements ending at i, i.e. $A[i - \text{lowbit}(i) + 1 \ldots i]$. Then any prefix sum $A[1 \ldots i]$ is assembled by peeling off lowest bits: $F[i]$ covers a suffix ending at i, then jump to $i - \text{lowbit}(i)$ and add its block, and so on down to 0. The binary expansion of i has at most $\log_2 m$ ones, so a prefix sum is at most $\log_2 m$ additions.

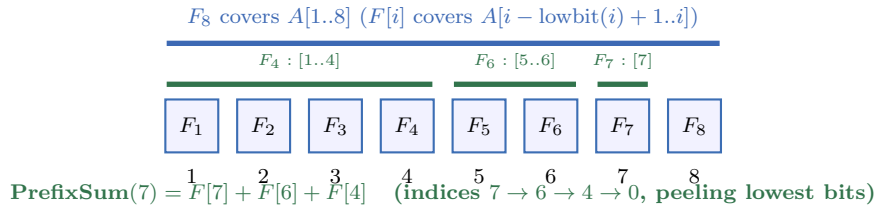


Figure 23.7: Fenwick coverage. $F[8]$ covers all of $A[1..8]$, $F[4]$ covers $A[1..4]$, $F[6]$ covers $A[5..6]$, $F[7]$ covers just $A[7]$.

Algorithm 100: Fenwick operations on $F[1 \dots m]$ (all zero initially)

```
1  function Add(i, x)                                   // A[i] += x
2  :
3      while i ≤ m do
4          F[i] += x
5          i ← i + lowbit(i)
6  function PrefixSum(i)                                // returns A[1] + ··· + A[i]
7  :
8      s ← 0
9      while i > 0 do
10         s += F[i]
11         i ← i − lowbit(i)
12     return s
```

A range sum $A[\ell \dots r]$ is $\textsc{PrefixSum}(r) - \textsc{PrefixSum}(\ell - 1)$. The two loops walk *opposite* bit-jumps: queries strip the lowest set bit ($i \to i - \text{lowbit}(i)$), updates climb to the next cell that contains i ($i \to i + \text{lowbit}(i)$). Each visits at most $O(\log m)$ cells because each step clears or sets a distinct bit.

Why it works

Why Add hits exactly the cells containing i. Cell $F[j]$ contains $A[i]$ iff $j - \text{lowbit}(j) < i \le j$. The smallest such $j \ge i$ is i itself; the next is obtained by $j \mapsto j + \text{lowbit}(j)$, which jumps to the smallest larger index whose block reaches back to i, and iterating gives precisely the increasing chain of containing cells, ending once it exceeds m. Dually, $i \mapsto i - \text{lowbit}(i)$ in PrefixSum partitions $[1, i]$ into the disjoint blocks $F[i], F[i - \text{lowbit}(i)], \dots$ whose lengths are the powers of two in the binary representation of i, so their union is exactly $[1, i]$ and their sum is the prefix sum.

Trace it yourself

Trace on $A = [3, 1, 4, 1, 5, 9, 2, 6]$. Building by $\textsc{Add}(i, A[i])$ for each i yields $F = [3, 4, 4, 9, 5, 14, 2, 31]$. Check: $F[8] = A[1..8] = 31$? The sum is $3{+}1{+}4{+}1{+}5{+}9{+}2{+}6 = 31$, yes. $F[6] = A[5]{+}A[6] = 14$, $F[7] = A[7] = 2$, $F[4] = A[1..4] = 9$. $\textsc{PrefixSum}(7) = F[7] + F[6] + F[4] = 2 + 14 + 9 = 25$, and indeed $A[1..7] = 25$. Now $\textsc{Add}(3, +10)$ visits $3 \to 4 \to 8$: $F[3]{=}14, F[4]{=}19, F[8]{=}41$. Re-query $[3, 6] = \textsc{PrefixSum}(6) - \textsc{PrefixSum}(2) = (F[6]{+}F[4]) - (F[2]) = (14{+}19) - 4 = 29$, matching $A[3..6]$ after the update: $14{+}1{+}5{+}9 = 29$. Correct.

Common trap

Fenwick is 1-indexed. The jumps $i \pm \text{lowbit}(i)$ break at $i = 0$ ($\text{lowbit}(0) = 0$ loops forever); always index from 1, and let PrefixSum stop at $i = 0$. A second trap: a plain Fenwick tree handles *prefix-decomposable* aggregates – sums, xor, counts. It does *not* handle range *min*, because min has no inverse, so you cannot subtract one prefix from another. (Prefix-min from index 1 is fine; arbitrary range-min needs a sparse table or segment tree.)

Cost summary

Fenwick: $\Theta(m)$ build (or $\Theta(m)$ in-place), $O(\log m)$ point update and prefix/range sum, $\Theta(m)$ space with the smallest constant of any structure here. Range-update / point-query and range-update / range-query variants exist via one or two Fenwick trees over difference arrays, still $O(\log m)$.

Pause

Pause and think: why does subtracting the lowest set bit navigate to the parent prefix? In two's-complement arithmetic, $i \,\&\, (-i)$ isolates the lowest set bit (the largest power of 2 dividing i). Cell $C[i]$ in a Fenwick tree stores the sum of the interval $(i - \text{lowbit}(i), i]$. Subtracting $\text{lowbit}(i)$ removes the contribution of cell $C[i]$ and lands on the index representing the remaining prefix, accumulating the entire prefix sum $A[1..i]$ in at most $\log_2 i$ steps!

Application

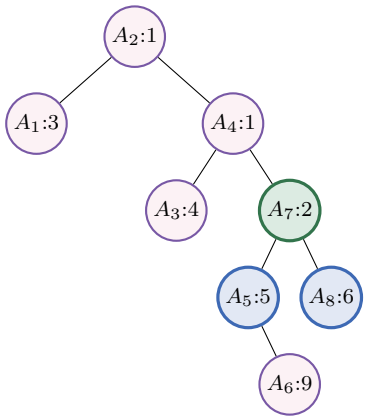
Fenwick trees are the standard way to count *inversions* (pairs $i < j$ with $A[i] > A[j]$): sweep left to right, query how many already-seen values exceed the current one. Combined with the Euler tour of Section 23.5, a Fenwick tree over entry/exit timestamps answers subtree-sum and subtree-update queries, which connects this chapter to Chapter 21's Euler-tour-on-subtrees technique.

23.8 The LCA $\leftrightarrow$ RMQ Equivalence

We reduced LCA to RMQ (Theorem 23.1). The reduction runs the other way too, so the two problems are, up to linear-time transformation, the *same*. As a consequence, a result for either problem transfers to the other, and it explains why both admit $\langle O(n), O(1)\rangle$ solutions.

Theorem 23.2 (RMQ reduces to LCA via the Cartesian tree)**.** *Given an array $A[1 \dots m]$, build its* Cartesian tree *C: the root is the position of the global minimum; its left subtree is the Cartesian tree of the elements to the left of that minimum, its right subtree that of the elements to the right (ties broken, say, leftmost). Then for any $\ell \le r$, the position of the minimum of $A[\ell \dots r]$ equals $\text{lca}_C(\ell, r)$.*

Proof. The Cartesian tree has the heap property on values (a node's value is $\le$ its descendants', since each subtree is built around its own minimum) and the binary-search-tree property on *positions* (the left subtree holds smaller indices, the right subtree larger). Take ℓ, r and let $w = \text{lca}_C(\ell, r)$ sit at position p. By the BST property $p \in [\ell, r]$: otherwise both ℓ and r would lie in the same child subtree of w, contradicting w being their LCA. By the heap property, $A[p]$ is $\le$ every value in the subtree rooted at w, which contains all positions in $[\ell, r]$ (they all descend from their LCA). Hence $A[p] = \min A[\ell \dots r]$, and p is a minimizing position. $\square$



Cartesian tree of $A = [3, 1, 4, 1, 5, 9, 2, 6]$. $\mathrm{lca}(5, 8)$ *is the node* A_7:2 *(green), so* $\min A[5..8] = 2$ *at position* 7.

Figure 23.8: Cartesian tree: RMQ on A becomes LCA on C. Node labels are position:value; the tree is min-heap-ordered on value and BST-ordered on position.

The Cartesian tree of $A[1 \dots m]$ is built in $O(m)$ by a single left-to-right scan with a stack: maintain the right spine of the tree built so far; to insert $A[i]$, pop spine elements whose value exceeds $A[i]$ (they become $A[i]$'s left subtree), then attach $A[i]$ as the right child of whatever remains on top.

Algorithm 101: BUILDCARTESIAN(A, m) – $O(m)$ stack scan

```
1 stack ← empty
2 for i ← 1 to m do
3 |   last ← NIL
4 |   while stack nonempty and A[stack.top] > A[i] do
5 |   |   last ← POP(stack)
6 |   if last ≠ NIL then LEFTCHILD(i) ← last
7 |   if stack nonempty then RIGHTCHILD(stack.top) ← i
8 |   PUSH(stack, i)
9 return root = bottom of stack
```

Each index is pushed and popped at most once, so the build is $O(m)$ amortized.

Why it works

Closing the loop. Put the two reductions together. $LCA \to RMQ$: the Euler tour turns a tree on n nodes into a ± 1 array of length $2n - 1$ ($O(n)$). $RMQ \to LCA$: the Cartesian tree turns an array of length m into a tree on m nodes ($O(m)$). So an $\langle f(n), g(n) \rangle$ algorithm for either problem yields an $\langle O(n) + f(O(n)),\ O(1) + g(O(n)) \rangle$ algorithm for the other. Plug in the ± 1-RMQ $\langle O(n), O(1) \rangle$ result to get $\langle O(n), O(1) \rangle$ LCA – and then, via Cartesian trees, $\langle O(n), O(1) \rangle$ *general* RMQ. Any improvement for one problem gives the same improvement for the other.

Connection

The tools of this chapter appear throughout the book. The Euler tour appears in Chapter 21 as the basis for subtree and path queries; binary lifting is the static cousin of the link-cut trees of Chapter 22; and the heavy-light decomposition of Chapter 19 uses exactly the segment trees

built here to answer path-aggregate queries. On the spectral side, the Laplacian $\mathbf{L} = \mathbf{D} - \mathbf{A}$ of a tree has quadratic form $x^\top \mathbf{L} x = \sum_{(u,v)\in E}(x_u - x_v)^2$, which telescopes along root-to-LCA paths, so LCA queries appear inside electrical-flow computations on trees (see Chapter 41 and the companion *Linear Algebra* volume).

23.9 Choosing a Structure

Structure	Preprocess	Query	Update	Operations
Sparse table	$O(m \log m)$	$O(1)$	–	idempotent (min/max/gcd)
Segment tree	$O(m)$	$O(\log m)$	$O(\log m)$	any associative $\oplus$
Segment tree + lazy	$O(m)$	$O(\log m)$	$O(\log m)$ range	associative + tags
Fenwick tree	$O(m)$	$O(\log m)$	$O(\log m)$	invertible (sum/xor)
Binary-lifting LCA	$O(n \log n)$	$O(\log n)$	–	LCA, k-th ancestor
Euler + sparse LCA	$O(n \log n)$	$O(1)$	–	LCA (static tree)

Focus

The decision tree in practice: *Static array, idempotent query, want $O(1)$?* sparse table. *Updates needed, or non-idempotent combine?* segment tree (add lazy tags for range updates). *Only prefix/range sums with point updates?* Fenwick – least code, smallest constant. *LCA with an $O(\log n)$ budget and you also want k-th ancestor?* binary lifting. *LCA queries dominate and the tree is fixed?* Euler tour plus sparse table for $O(1)$.

Connection

What This Made Possible: Static LCA and range structures let us query tree distances, ancestor relationships, and path bottlenecks in $O(1)$ or $O(\log n)$ time without touching intermediate tree nodes. Paired with Cartesian trees, RMQ allows constant-time longest common extension (LCE) queries on strings, while segment and Fenwick trees handle dynamic point updates and range queries in $O(\log n)$.

Limitations:

- *Completely static:* Binary lifting tables (up$[v][j]$) and Euler tour arrays rely on a fixed tree topology. Adding or deleting a single tree edge corrupts the ancestry structure, requiring an $\Omega(n)$ or $\Omega(n \log n)$ rebuild.
- *Memory footprint and cache behavior:* An $O(n \log n)$ table for binary lifting or sparse tables requires 16–24 bytes per entry. For large trees ($n \geq 10^7$), the table requires gigabytes of RAM, causing random L3 cache misses during the binary search jumps.
- *Trees only:* These primitives exploit the unique path property of trees. In graphs with multiple paths or cycles, the concept of a unique lowest common ancestor breaks down entirely.

Where We Go Next (Tree Decompositions and Treewidth): With static lowest com-

mon ancestors and range structures in hand, our survey of exact tree algorithms in Part IV is complete: from heavy-light decomposition to dynamic link-cut trees and constant-time LCA queries. But what happens when our input graph is *not* a tree, but contains cycles, meshes, and dense cross-edges? Can we still exploit the fast, clean dynamic programming of tree structures? In **Chapter 24 (Tree Decompositions and Treewidth)**, we generalize tree algorithms to arbitrary graphs by grouping vertices into overlapping clusters called *bags* arranged as a tree. The width of these bags – the graph's *treewidth* – measures how close a graph is to a tree, opening the door to linear-time algorithms for NP-hard problems on graphs with bounded treewidth.

23.10 Exercises

Exercise 23.1 (Quick mental check: binary lifting memory)**.** For a tree on n vertices with maximum power-of-two jump $L = \lfloor \log_2 n \rfloor$, how many total entries are stored in the up table? What is the meaning of up$[v][0]$?

Exercise 23.2 (Quick mental check: sparse table idempotence)**.** Why can Range Minimum Queries be answered in $O(1)$ time by evaluating two overlapping blocks of length 2^j, whereas Range Sum Queries cannot?

Exercise 23.3 (Distance and the LCA)**.** In a tree with nonnegative edge weights, let $D(v)$ be the weighted distance from the root to v. Prove that the weighted distance between u and v equals $D(u)+D(v)-2D(\mathrm{lca}(u,v))$. Then describe an $O(\log n)$-per-query algorithm combining binary lifting with a precomputed D array.

Exercise 23.4 (Implement binary lifting)**.** Implement BuildLifting and LiftLCA and test them against NaiveLCA on random trees of up to 10^5 nodes. Add a function that returns the k-th ancestor of a node, or the root's sentinel if $k > \mathrm{dep}(v)$, using only the up table.

Exercise 23.5 (Break the lockstep)**.** A student writes the second loop of LiftLCA to jump when up$[u][j]$ = up$[v][j]$ (the negation of the correct test). Exhibit a small tree and a query where this returns a wrong answer, and say in one sentence what the buggy loop actually computes.

Exercise 23.6 (Sparse table for gcd)**.** Argue that gcd is associative and idempotent, so a sparse table answers range-gcd in $O(1)$ comparisons. If entries are big integers of B bits, what is the true per-query time, and how does that change the bound? Contrast with range-*sum*, and explain precisely why the overlapping-block trick fails for sums.

Exercise 23.7 (Euler-tour length and bracketing)**.** Prove that the Euler tour produced by EulerTour has exactly $2n-1$ entries for a tree on n nodes, by a counting argument over edge traversals. Then prove that the first and last occurrences of any vertex v bracket exactly the tour steps spent inside v's subtree.

Exercise 23.8 (Lazy assign *and* add)**.** Extend the lazy segment tree to support two range updates – "assign x to $A[\ell \dots r]$" and "add x to $A[\ell \dots r]$," interleaved arbitrarily – with range-sum queries. Define the tag type and its composition rule, and explain why the order of applying a pending assign versus a pending add matters. Give the PushDown that respects it.

Exercise 23.9 (Inversions with Fenwick)**.** Using a Fenwick tree, count the inversions of a permutation of $1 \dots m$ in $O(m \log m)$. Then modify your code to compute, for each i, the number of $j > i$ with $A[j] < A[i]$ (the per-element inversion count), still in $O(m \log m)$.

Exercise 23.10 (Cartesian tree by hand)**.** Build the Cartesian tree of $A = [5, 2, 8, 1, 9, 3, 7, 4]$ using the stack scan, drawing the right spine after each insertion. Verify on three ranges that $\mathrm{lca}_C(\ell, r)$ is the position of $\min A[\ell \dots r]$.

Exercise 23.11 (± 1 RMQ in $O(n)$ preprocessing)**.** Implement the Bender–Farach-Colton ± 1 RMQ: split the depth array into blocks of size $b = \frac{1}{2}\log_2 m$, build a sparse table over the $O(m/b)$ block minima, and tabulate in-block answers for all $O(\sqrt{m})$ distinct ± 1 patterns. Prove that the total preprocessing is $O(n)$ and queries are $O(1)$, and use it to get linear-preprocessing LCA.

Exercise 23.12 (Offline LCA, Tarjan)**.** Look up Tarjan's *offline* LCA algorithm, which answers a batch of queries during a single DFS using a union-find structure. Implement it, and prove its running time is $O((n + q)\,\alpha(n))$ where α is the inverse Ackermann function. When is the offline method preferable to the online structures of this chapter?

Exercise 23.13 (Persistent segment tree)**.** A *persistent* segment tree keeps every past version by copying only the $O(\log m)$ nodes on an update path. Implement one supporting point updates, and use m stacked versions to answer "the k-th smallest value in $A[\ell \dots r]$" (a static range-kth query) in $O(\log m)$ per query. Explain why the total space is $O(m \log m)$.

Chapter 24

Tree Decompositions and Treewidth

In the preceding chapters, we saw how trees yield fast queries and clean dynamic programs because every edge is a cut and ancestors order subproblems neatly. But most networks contain cycles. Tree decompositions carry the tree's hierarchical structure over to such graphs. They measure, with an integer called *treewidth*, how far a graph deviates from a tree structure – and turn that measurement into an algorithm. A graph of treewidth k inherits tree-like dynamic programming, paying an overhead that depends on k rather than n.

This is the structural foundation of *fixed-parameter tractability*, the subject of the next chapter. Here we build the machinery: what a tree decomposition is, why the two technical conditions on it are exactly the right ones, how to run dynamic programming over a decomposition (we will trace independent set and dominating set in full, by hand, on a small graph), and how to compute or approximate treewidth itself. By the end you should recognize a low-treewidth graph as one that splits into small, nearly independent pieces that a dynamic program can solve efficiently.

History

Tree decompositions were introduced in 1984 by Neil Robertson and Paul Seymour in Paper II of their landmark twenty-paper *Graph Minors* project, which ultimately proved Wagner's Conjecture: every minor-closed family of finite graphs is characterized by a finite set of forbidden minors (where a graph H is a minor of G if H can be formed by deleting vertices, deleting edges, and contracting edges). Independently, related ideas surfaced in Umberto Bertelé and Francesco Brioschi's work on non-serial dynamic programming (1972), Rudolf Halin's study of S-functions (1976), and the partial k-trees of Stefan Arnborg and Andrzej Proskurowski (1989). Robertson and Seymour needed a structural invariant to show that any graph avoiding a fixed planar minor decomposes into tree-like pieces – the celebrated Excluded Grid Theorem. Early attempts to extend tree algorithms to graphs centered on deleting feedback vertex sets (sets of vertices whose removal leaves an acyclic graph). But that strategy is brittle: an $n \times n$ grid requires deleting $\Theta(n^2)$ vertices to kill all cycles, destroying all local structure even though the grid is locally simple. Recursive partitioning also struggled, because unconstrained cuts caused dynamic programming states to blow up.

Robertson and Seymour's insight was that the decomposition itself must form an abstract tree of overlapping vertex subsets ("bags"). The critical condition is the *running intersection property* (every vertex induces a connected subtree of bags), guaranteeing that bags act as separators that isolate past choices from future ones. In 1990, Bruno Courcelle proved that

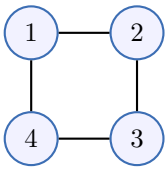
every graph property expressible in Monadic Second-Order Logic (MSO_2, logic quantifying over vertices, edges, and sets of them) can be decided in linear time $f(k) \cdot n$ on graphs of bounded treewidth k. In 1996, Hans Bodlaender showed that deciding whether $\mathrm{tw}(G) \le k$ and finding an optimal decomposition is also solvable in linear time $f(k) \cdot n$ for fixed k. Whether treewidth can be approximated within a constant factor in polynomial time remains a central open problem in graph algorithms.

24.1 From Trees to Tree-Like Graphs

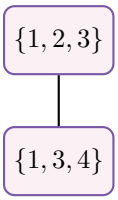
Intuition

A tree is easy to work with for two reasons. First, a tree has tiny *separators*: delete one vertex and the tree falls into independent pieces, so a subproblem in one piece interacts with the rest only through that single cut vertex. Second, those separators are organized hierarchically – the recursion tree of "cut here, recurse on the pieces" is itself a tree, so a dynamic program can sweep it once. A tree decomposition provides both properties for a general graph. It groups the vertices into overlapping sets called *bags*, arranges the bags in a tree, and guarantees that each bag acts as a small separator between the parts of the graph on either side of it. The largest bag, minus one, is the treewidth, which governs the cost of tree-like dynamic programming.

Consider the cycle C_4 on vertices $\{1, 2, 3, 4\}$ with edges $12, 23, 34, 41$. It is not a tree – it has a cycle – but it is barely not a tree. Cut it at any two opposite vertices and it falls apart. We can capture "barely not a tree" by covering the graph with bags of size three, arranged in a path:



the graph C_4



a tree decomposition, width 2

Two bags, $\{1, 2, 3\}$ and $\{1, 3, 4\}$, joined by an edge. Every vertex appears in some bag. Every edge of C_4 – including the pairs $12, 23$ in the first bag and $34, 41$ in the second – sits inside a single bag. And the shared pair $\{1, 3\}$ is exactly the separator that breaks the cycle. The widest bag has three vertices, so the *width* is $3 - 1 = 2$. We will prove shortly that C_4 has treewidth exactly 2, and more generally that trees are precisely the graphs of treewidth 1.

24.2 The Definition and Why Each Clause Matters

Definition 24.1 (Tree decomposition)**.** A *tree decomposition* of a graph $G = (V, E)$ is a pair $(\mathcal{T}, \{B_x\}_{x \in \mathcal{T}})$ where $\mathcal{T}$ is a tree and each node x of $\mathcal{T}$ carries a *bag* $B_x \subseteq V$, subject to three

conditions:

(V) *Vertex cover:* $\bigcup_{x \in \mathcal{T}} B_x = V$. Every vertex lives in at least one bag.

(E) *Edge cover:* for every edge $\{u, v\} \in E$ there is a node x with $\{u, v\} \subseteq B_x$. Every edge is realized inside some bag.

(C) *Connectivity (coherence):* for every vertex $v \in V$, the set of nodes $\{x : v \in B_x\}$ induces a *connected subtree* of $\mathcal{T}$.

The *width* of the decomposition is $\max_x |B_x| - 1$. The *treewidth* $\mathrm{tw}(G)$ is the minimum width over all tree decompositions.

The "-1" is a convention chosen so that trees come out with treewidth 1 rather than 2; see Proposition 24.1. To keep the names straight: nodes (of $\mathcal{T}$) carry bags; vertices (of G) live inside bags. We reserve "node" for $\mathcal{T}$ and "vertex" for G.

Focus

Each of the three conditions has a purpose. **(V)** and **(E)** say the decomposition does not lose information: you could reconstruct G from the bags. The essential condition is **(C)**. It forces each vertex to occupy a connected subtree of $\mathcal{T}$, and that connectedness is what makes bags act as separators. Without **(C)**, a vertex could appear in two distant bags without appearing along the path between them, creating dependencies across subtrees and breaking dynamic programming. Condition **(C)** says each vertex enters the decomposition once, stays present across a single connected region of $\mathcal{T}$, and never reappears after leaving.

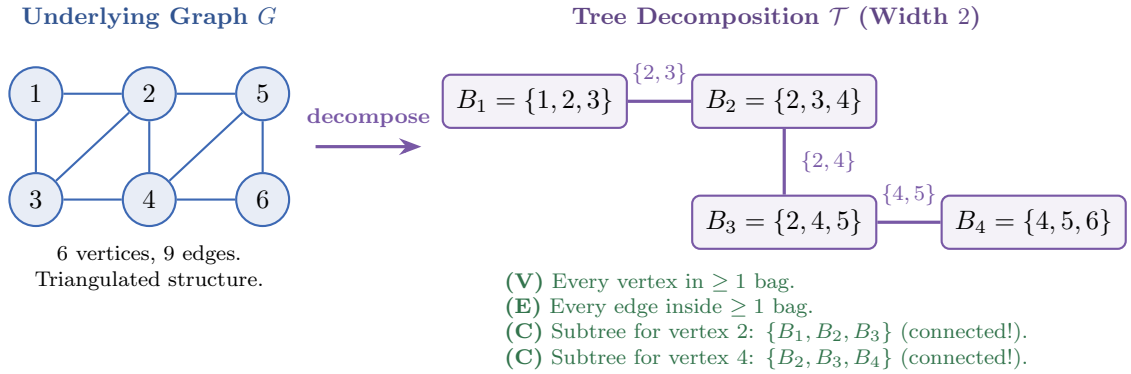


Figure 24.1: A tree decomposition of width 2 on a 6-vertex graph. Every edge of G is contained in at least one bag. The set of bags containing vertex 2 forms a connected subpath ($B_1 - B_2 - B_3$) in the tree, and similarly for vertex 4 ($B_2 - B_3 - B_4$). The intersection of adjacent bags $B_2 \cap B_3 = \{2, 4\}$ acts as a balanced separator cutting the left vertices $\{1, 3\}$ from the right vertex $\{6\}$.

24.2.1 The separator property

Every algorithm in this chapter rests on the following property. Removing the vertices of a single bag disconnects the rest of the graph along the tree.

Lemma 24.1 (Bags are separators)**.** *Let $(\mathcal{T}, \{B_x\})$ be a tree decomposition of G, and let $\{x, y\}$ be an edge of $\mathcal{T}$. Deleting that edge splits $\mathcal{T}$ into two subtrees $\mathcal{T}_x \ni x$ and $\mathcal{T}_y \ni y$. Let*

$$A = \Big(\bigcup_{z \in \mathcal{T}_x} B_z\Big) \setminus (B_x \cap B_y), \qquad C = \Big(\bigcup_{z \in \mathcal{T}_y} B_z\Big) \setminus (B_x \cap B_y).$$

Then $S := B_x \cap B_y$ separates A from C: no edge of G joins a vertex of A to a vertex of C.

Proof. Suppose for contradiction $\{a, c\} \in E$ with $a \in A$, $c \in C$. By **(E)** some bag B_z contains both a and c. The node z lies in $\mathcal{T}_x$ or in $\mathcal{T}_y$; say $z \in \mathcal{T}_y$ (the other case is symmetric). Now $a \in A$ means a occurs in some bag of $\mathcal{T}_x$, and $a \in B_z$ means it occurs in a bag of $\mathcal{T}_y$. By **(C)** the nodes containing a form a connected subtree, and any path in $\mathcal{T}$ between a node of $\mathcal{T}_x$ and a node of $\mathcal{T}_y$ must cross the edge $\{x, y\}$. Hence both x and y contain a, so $a \in B_x \cap B_y = S$. But A was defined to exclude S – contradiction. □

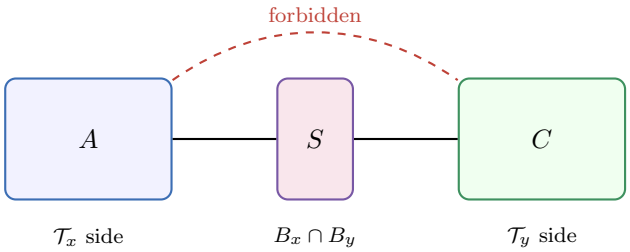


Figure 24.2: Lemma 24.1 in one picture. Deleting tree edge $\{x, y\}$ partitions the graph vertices into A, the separator $S = B_x \cap B_y$, and C. Edges may run A–S or S–C, but never directly A–C: every A–C interaction is forced through S.

Why it works

Lemma 24.1 (Figure 24.2) is the central mechanism. A dynamic program processes the tree one node at a time; when it crosses an edge of $\mathcal{T}$, everything it needs to remember about the already-processed side is summarized by how the solution behaves on the separator $S = B_x \cap B_y$, a set of at most tw $+1$ vertices. The number of possible behaviors on S is bounded by a function of tw alone – $2^{|S|}$ for independent set, $3^{|S|}$ for dominating set, $k^{|S|}$ for k-coloring. That is why small treewidth makes hard problems tractable: the "interface" between subproblems is provably small.

24.2.2 Trees, cycles, and cliques

Proposition 24.1 (Treewidth 1 means forest)**.** $\mathrm{tw}(G) \le 1$ *if and only if G is a forest (acyclic). A graph with at least one edge has* $\mathrm{tw} \ge 1$.

Proof. ($\Leftarrow$) Root each tree of the forest. Make one decomposition node per *edge* $\{u, \mathrm{parent}(u)\}$ with bag $\{u, \mathrm{parent}(u)\}$, and hang these nodes in the shape of the tree (the node for u is a child of the node for $\mathrm{parent}(u)$); add a singleton bag for the root. Each bag has size ≤ 2, so width ≤ 1. Conditions **(V)**,**(E)** are immediate, and **(C)** holds because u appears only in its own edge-node and in those of its children – a connected subtree.

($\Rightarrow$) Suppose G has a cycle $v_1 v_2 \cdots v_\ell v_1$ but $\mathrm{tw}(G) \le 1$, so all bags have size ≤ 2. The cycle vertices form subtrees $T_{v_i} = \{x : v_i \in B_x\}$, and consecutive ones intersect (each edge of the cycle lies in a bag). Walking the cycle, these subtrees chain around and the intersection structure of a cycle has Helly number 3, which forces some bag to contain three cycle vertices, i.e. width ≥ 2. Hence a cycle implies $\mathrm{tw} \ge 2$, so $\mathrm{tw} \le 1$ implies acyclic. The last sentence holds because a single edge $\{u, v\}$ must, by **(E)**, lie in a bag of size ≥ 2. □

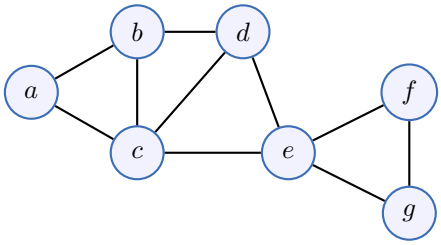
Proposition 24.2 (Cliques are unavoidable)**.** *If $K \subseteq V$ induces a clique in G, then some bag contains all of K. Consequently* $\mathrm{tw}(G) \ge \omega(G) - 1$*, where ω is the clique number, and* $\mathrm{tw}(K_n) = n - 1$.

Proof. This is the *Helly property* for subtrees of a tree: a family of pairwise intersecting subtrees has a common node. For each $v \in K$ let $T_v = \{x : v \in B_x\}$, a subtree by **(C)**. For any two $u, v \in K$ the edge $\{u, v\}$ is in some bag (clique, so the edge exists; **(E)**), hence $T_u \cap T_v \neq \emptyset$ – the subtrees pairwise intersect. The Helly property for subtrees of a tree gives a common node $x \in \bigcap_{v \in K} T_v$, and then $K \subseteq B_x$. So $|B_x| \ge |K|$, width $\ge |K| - 1$. For K_n this forces a bag with all n vertices, width $n - 1$; the one-bag decomposition achieves it. □

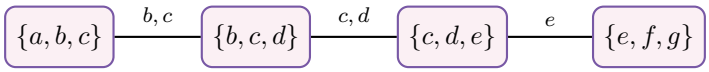
24.3 A Worked Decomposition by Hand

Let us fix a running graph and find a good tree decomposition for it. Take G on seven vertices $\{a, b, c, d, e, f, g\}$ with edges

$$ab,\ ac,\ bc,\ bd,\ cd,\ ce,\ de,\ ef,\ eg,\ fg.$$



Here $\{a, b, c\}$, $\{b, c, d\}$, $\{c, d, e\}$ form a chain of triangles sharing edges, then $\{e, f, g\}$ is a triangle hanging off e. This "chain of overlapping triangles" is exactly a graph of treewidth 2. A natural decomposition reads the chain off directly:



Check the conditions. **(V)**: all seven vertices appear. **(E)**: every edge lives in a bag – e.g. $ab, ac, bc \subseteq \{a, b, c\}$; $bd, cd \subseteq \{b, c, d\}$; $ce, de \subseteq \{c, d, e\}$; $ef, eg, fg \subseteq \{e, f, g\}$. **(C)**: trace each vertex. c appears in the first three bags – contiguous. d in bags $2, 3$ – contiguous. e in bags $3, 4$ – contiguous. Good. Widest bag has 3 vertices, so this decomposition has width 2, and since G contains a triangle (clique of size 3), Proposition 24.2 gives $\mathrm{tw}(G) \ge 2$. Therefore $\mathrm{tw}(G) = 2$ exactly.

Trace it yourself

Build a tree decomposition of the 3×3 grid graph (vertices a 3×3 lattice, edges between orthogonal neighbors). Try to get width 3: use bags that are "two adjacent columns" sliding across the grid, $\{\text{col } i, \text{col } i{+}1\}$. Verify **(C)** holds for the middle column. Then convince yourself you cannot do better than width 3 – the 3×3 grid has treewidth exactly 3, and in general the $n \times n$ grid has treewidth n, the canonical example of a graph that is sparse yet has unbounded treewidth.

24.4 Nice Tree Decompositions

Dynamic programming wants a tidy tree where each step does exactly one small thing. The raw decomposition above has bags changing by several vertices across a single edge, which makes the transition rules messy. We standardize.

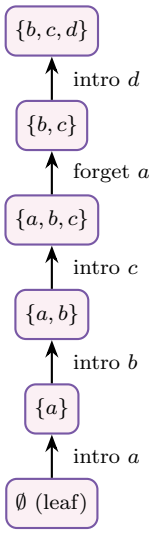
Definition 24.2 (Nice tree decomposition)**.** A tree decomposition is *nice* if $\mathcal{T}$ is rooted, the root bag and every leaf bag are empty, and every non-leaf node is one of:

- **Introduce** v: one child y, and $B_x = B_y \cup \{v\}$ for a single $v \notin B_y$.
- **Forget** v: one child y, and $B_x = B_y \setminus \{v\}$ for a single $v \in B_y$.
- **Join**: two children y, z with $B_x = B_y = B_z$.

Proposition 24.3 (Niceness is free)**.** *Any width-k tree decomposition with $O(n)$ nodes can be converted to a nice one of width k with $O(k \cdot n)$ nodes in $O(k^2 n)$ time.*

Proof sketch. Root the tree arbitrarily. For an edge from parent x to child y where the bags differ, insert a chain of introduce/forget nodes that adds the vertices of $B_x \setminus B_y$ one at a time and removes those of $B_y \setminus B_x$ one at a time; each step changes the bag by one vertex, at most $|B_x| + |B_y| \le 2(k{+}1)$ steps. For a node with ≥ 2 children, replace it by a binary tree of join nodes (all sharing the same bag), then attach the children. Finally pad the top with forget nodes down to an empty root and cap each leaf with introduce nodes from an empty bag. Every inserted bag is a subset of an existing one, so width is unchanged; the total node count is $O(kn)$. □

A nice tree decomposition can be drawn as a chain of one-vertex changes. Here is a nice version of the bottom of the spine $\{a, b, c\} - \{b, c, d\}$ from our running graph:



The arrows point from child to parent (leaves at the bottom, root at top), the direction the dynamic program flows.

24.5 Dynamic Programming over a Tree Decomposition

The dynamic programming template is always the same: at each node x we maintain a table indexed by the possible "states" of the bag B_x, where the value stored is the best (or the count, or feasibility) of a partial solution on the part of the graph already absorbed below x, *conditioned* on the bag being in that state. Lemma 24.1 guarantees the bag's state is a complete interface: nothing below x talks to anything above x except through B_x.

24.5.1 Maximum weight independent set

An *independent set* is a set of pairwise non-adjacent vertices. Given vertex weights $w(v) \geq 0$, we want a maximum-weight independent set (MWIS). NP-hard in general; here is the bounded-treewidth algorithm.

For a node x with bag B_x, a *state* is a subset $S \subseteq B_x$ of bag vertices we tentatively include, required to be independent inside the bag. Let V_x denote the union of all bags in the subtree rooted at x (the vertices "introduced so far"). Define

$$\mathrm{dp}[x][S] \;=\; \max\Big\{\; w(I) \;:\; I \subseteq V_x \text{ independent},\; I \cap B_x = S \;\Big\},$$

the best weight of an independent set on V_x whose intersection with the current bag is exactly S (and $-\infty$ if S itself is not independent in G). The transition rules:

- **Leaf** ($B_x = \emptyset$): the only state is $S = \emptyset$, with $\mathrm{dp}[x][\emptyset] = 0$.
- **Introduce** v (child y): for each independent $S \subseteq B_x$,

$$\mathrm{dp}[x][S] = \begin{cases} \mathrm{dp}[y][S] & v \notin S, \\ \mathrm{dp}[y][S \setminus \{v\}] + w(v) & v \in S, \end{cases}$$

provided S stays independent; if $v \in S$ but S contains a neighbor of v, set $\mathrm{dp}[x][S] = -\infty$.

- **Forget** v (child y): for each $S \subseteq B_x$,

$$\mathrm{dp}[x][S] = \max\big(\mathrm{dp}[y][S],\ \mathrm{dp}[y][S \cup \{v\}]\big).$$

  We project away v, keeping the better of "v out" and "v in."

- **Join** (children y, z, $B_x = B_y = B_z$): for each independent S,

$$\mathrm{dp}[x][S] = \mathrm{dp}[y][S] + \mathrm{dp}[z][S] - w(S),$$

  where $w(S) = \sum_{v \in S} w(v)$. The two subtrees agree on S, so we add their values but subtract the weight of S counted twice.

The answer is $\mathrm{dp}[r][\emptyset]$ at the empty root r.

Algorithm 102: MWIS on a nice tree decomposition

Input: nice tree decomposition $(\mathcal{T}, \{B_x\})$ rooted at r, weights w
Output: maximum weight of an independent set of G

```
foreach node x of T in post-order do
    if x is a leaf then
        dp[x][∅] ← 0;
    else if x introduces v over child y then
        foreach S ⊆ B_x do
            if v ∈ S and S is independent in G then
                dp[x][S] ← dp[y][S \ {v}] + w(v);
            else if v ∉ S then
                dp[x][S] ← dp[y][S];
            else
                dp[x][S] ← −∞;
    else if x forgets v over child y then
        foreach S ⊆ B_x do
            dp[x][S] ← max(dp[y][S], dp[y][S ∪ {v}]);
    else // join with children y, z
        foreach independent S ⊆ B_x do
            dp[x][S] ← dp[y][S] + dp[z][S] − w(S);
return dp[r][∅];
```

Why it works

Why is the join rule subtract-the-overlap and not something fancier? Because the separator B_x splits V_x into the y-side and the z-side, sharing only B_x itself (Lemma 24.1). Fix the bag's chosen set S. Any independent set on the whole subtree restricted to "$\cap B_x = S$" decomposes uniquely into an independent set on the y-side and one on the z-side, each intersecting the bag in S, and the separation guarantee ensures there are *no edges* between the private parts of the two sides to forbid – so we may combine them freely. Their weights add, but the bag vertices S were paid for in both tables, hence $-w(S)$. The independence *within* S was already

enforced when S was introduced, and there are no other cross edges to check. That is the entire content of the rule.

Theorem 24.1 (Correctness of Algorithm 102)**.** *For every node x and every $S \subseteq B_x$, Algorithm 102 computes* $\mathrm{dp}[x][S]$ *equal to the maximum weight of an independent set $I \subseteq V_x$ with $I \cap B_x = S$ (or $-\infty$ if none exists). In particular* $\mathrm{dp}[r][\emptyset]$ *is the MWIS weight of G.*

Proof. Induction on the post-order. *Leaf:* $V_x = \emptyset$, only the empty set qualifies, value 0. *Introduce v:* by **(C)**, v is brand new ($v \notin V_y$ since $v \notin B_y$ and the subtree of $\mathcal{T}$ containing v is rooted here), so every edge incident to v within V_x has its other endpoint in B_x – this is the consequence of the separator property. If $v \in S$ we must check S is independent (done) and then v's contribution is additive over the child's best set with v removed; if $v \notin S$, v contributes nothing and any I counted by the child with $I \cap B_y = S$ still has $I \cap B_x = S$. *Forget v:* an independent set $I \subseteq V_x = V_y$ with $I \cap B_x = S$ either contains v (then $I \cap B_y = S \cup \{v\}$) or not (then $I \cap B_y = S$); taking the max over the two child entries is exactly right, and no new edges appear because $V_x = V_y$. *Join:* by Lemma 24.1, uniqueness of the split uses that the y-side private vertices and z-side private vertices have no edge between them (separator), so the combined set is independent iff each side is and they agree on S. Maximizing each side independently and adding (correcting the double count) yields the optimum. □

24.5.2 Tracing it by hand

Take the path P_3: vertices $1-2-3$, edges $12, 23$, all weights $w = 1$. The MWIS is $\{1, 3\}$ with weight 2. We run the DP on the nice decomposition with spine that introduces 1, introduces 2, forgets 1, introduces 3, forgets 2, forgets 3. We track the table as we climb.

node	bag	table (state $\mapsto$ value)
leaf	$\emptyset$	$\emptyset \mapsto 0$
intro 1	$\{1\}$	$\emptyset \mapsto 0,\ \{1\} \mapsto 1$
intro 2	$\{1,2\}$	$\emptyset \mapsto 0,\ \{1\} \mapsto 1,\ \{2\} \mapsto 1,\ \{1,2\} \mapsto -\infty$
forget 1	$\{2\}$	$\emptyset \mapsto \max(0,1) = 1,\ \{2\} \mapsto \max(1,-\infty) = 1$
intro 3	$\{2,3\}$	$\emptyset \mapsto 1,\ \{3\} \mapsto 1{+}1{=}2,\ \{2\} \mapsto 1,\ \{2,3\} \mapsto -\infty$
forget 2	$\{3\}$	$\emptyset \mapsto \max(1,1) = 1,\ \{3\} \mapsto \max(2,-\infty) = 2$
forget 3 (root)	$\emptyset$	$\emptyset \mapsto \max(1,2) = 2$

The root reports 2. At "intro 3" the state $\{3\}$ takes the child's $\emptyset \mapsto 1$ (which already encodes the choice "1 in, 2 out") and adds $w(3) = 1$, giving 2 – the program has discovered $\{1, 3\}$ without ever storing it explicitly. The $-\infty$ entries are the edges 12 and 23 doing their job, forbidding adjacent vertices from being chosen together.

24.5.3 Dominating set: a 3-state DP

A *dominating set* $D \subseteq V$ is a set such that every vertex is in D or adjacent to a vertex in D. Minimum dominating set is NP-hard. The bag state now needs three colors per vertex, because a

vertex's domination can be settled by a neighbor that has not been processed yet.

For each vertex v in the bag, assign one of:

- **1** (*black*): $v \in D$.
- **0** (*white-dominated*): $v \notin D$ and v is already dominated by some chosen vertex in V_x.
- $\hat{\mathbf{0}}$ (*white-pending*): $v \notin D$ and v is *not yet* dominated – it is still waiting for a future neighbor (one introduced above x) to dominate it.

A coloring $\phi : B_x \to \{1, 0, \hat{0}\}$ is a state. Define $\mathrm{dp}[x][\phi]$ as the minimum size of a set $D \subseteq V_x$ that (i) restricts to the black vertices of ϕ on B_x, (ii) dominates every *forgotten* vertex (every vertex of $V_x \setminus B_x$), and (iii) dominates each white-0 bag vertex already, while white-$\hat{0}$ bag vertices are explicitly left undominated for now. The transitions (minimizing):

- **Leaf:** $\mathrm{dp}[x][\,] = 0$ (empty coloring).
- **Introduce v:** v has no forgotten neighbors yet, so it starts either black (+1 to the size; this also upgrades any bag-neighbor that was $\hat{0}$ to 0), or white-pending $\hat{0}$ (cost 0, undominated so far). White-0 is *not* a legal fresh start for v unless an already-black bag-neighbor dominates it.
- **Forget v:** we drop v, but only from states where v is *not* pending: v must be black or white-0 (it had its last chance to be dominated). We take the min over those child states. A child state with $v = \hat{0}$ is discarded – a pending vertex that is forgotten can never be dominated.
- **Join:** children y, z with the same bag. Two colorings combine into ϕ when they agree on the black vertices, a bag vertex is white-0 in ϕ iff it is white-0 in *at least one* child (dominated on either side counts), and white-$\hat{0}$ in ϕ iff it is $\hat{0}$ in *both*. Add the sizes and subtract $|\{v : \phi(v) = 1\}|$ for the double-counted black bag vertices.

The answer is $\mathrm{dp}[r][\,]$ at the empty root. Figure 24.3 illustrates how the three states interact across introduce, forget, and join steps.

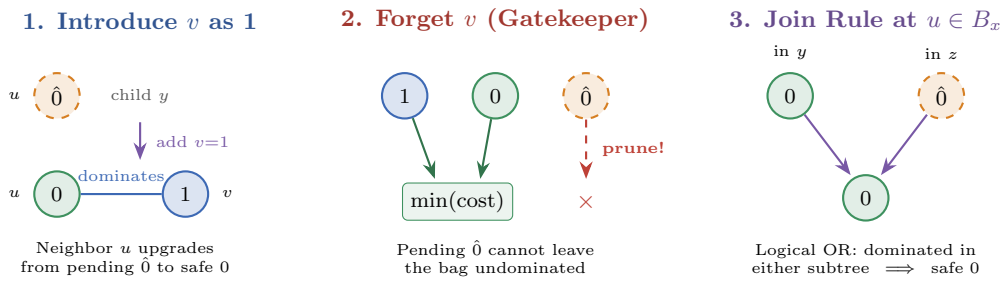


Figure 24.3: How the three dominating-set states – **1** (in D), **0** (already dominated), and $\hat{\mathbf{0}}$ (still awaiting a dominator) – flow up a nice tree decomposition. Introducing a black vertex **1** immediately upgrades its pending bag neighbors from $\hat{\mathbf{0}}$ to **0**; forgetting a vertex discards any state where that vertex is still $\hat{\mathbf{0}}$; joining two subtrees takes the logical OR of domination on white vertices.

Common trap

The classic bug is collapsing white-0 and white-$\hat{0}$ into one "white" state. Then the forget rule cannot tell whether a vertex about to vanish was ever dominated, and the DP will happily

produce sets that miss a vertex. The distinction between the two kinds of white is the main subtlety of dominating set: "not in D and already safe" versus "not in D and still owing a dominator." A second trap is the join rule – domination is a logical OR across the two subtrees (dominated on either side suffices), which is why white-0 in the parent needs white-0 in *just one* child, while pending requires pending in *both*.

Cost summary

Per node, MWIS examines $2^{|B_x|} \le 2^{k+1}$ states; introduce/forget cost $O(2^k)$ and join costs $O(2^k)$ table combines. Over $O(kn)$ nodes the total is $O(2^k \cdot kn)$, i.e. *linear in* n for fixed k. Dominating set has $3^{|B_x|}$ states; a naive join is $O(3^k \cdot 3^k)$ but with the subset-convolution / fast-coloring trick it drops to $O(3^k \cdot k)$ per join, giving $O(3^k \cdot kn)$ overall. In general a problem whose "interface" has c states per vertex runs in $O(c^k \cdot \mathrm{poly}(k) \cdot n)$ time on a width-k decomposition: single-exponential in treewidth, linear in the graph.

24.6 Computing and Approximating Treewidth

So far we assumed a decomposition was handed to us. Where does it come from? Deciding $\mathrm{tw}(G) \le k$ is NP-complete (Arnborg–Corneil–Proskurowski, 1987), but the parameter k rescues us in three different ways.

24.6.1 Exact, when treewidth is small: Bodlaender's theorem

Theorem 24.2 (Bodlaender, 1996)**.** *There is an algorithm that, given G and k, in time $2^{O(k^3)} \cdot n$ either outputs a tree decomposition of width $\le k$ or reports $\mathrm{tw}(G) > k$.*

This is *fixed-parameter tractable*: linear in n for each fixed k, with all the dependence shoved into the $2^{O(k^3)}$ factor. The constant is far too large in practice, so the result is mainly of theoretical interest: it shows that "find a good decomposition" is not the bottleneck for the FPT program. In practice one uses heuristics or the approximation below.

24.6.2 A clean constant-factor approximation

A standard approximation routine, due in this form to Robertson–Seymour and sharpened by many, builds a decomposition of width $O(k)$ whenever $\mathrm{tw} \le k$, using *separators*. It relies on the fact that bounded-treewidth graphs have balanced separators.

Lemma 24.2 (Balanced separator from treewidth)**.** *If $\mathrm{tw}(G) \le k$ then for any weight on the vertices, there is a bag B_x of some optimal decomposition that is a $\frac{1}{2}$-balanced separator: each component of $G \setminus B_x$ carries at most half the total weight. In particular every $W \subseteq V$ admits a separator of size $\le k+1$ splitting W in balanced fashion.*

Proof idea. Root an optimal decomposition and push toward the heavier side: starting at the root, repeatedly move to the child whose subtree carries more than half the weight. This must stop at

a node x where no single child subtree exceeds half – its bag B_x, of size $\leq k+1$, is the balanced separator, because each component of $G \setminus B_x$ lies on one side of the tree split and inherits the $\leq \frac{1}{2}$ bound from Lemma 24.1. □

The recursive scheme then reads:

Algorithm 103: ApproxDecompose(W) – decomposition with boundary W

Input: a set $W \subseteq V$ with $|W| \leq 3k+4$ (the boundary so far)
Output: a tree decomposition of the relevant part of G of width $O(k)$

```
if the remaining interior plus W fits in one bag of size ≤ 4k + 4 then
    return single bag;
find a separator S, |S| ≤ k + 1, that is 2/3-balanced for W;
// exists by Lemma 24.2 when tw ≤ k, else report failure
foreach component Γ of G \ (W ∪ S) do
    W_Γ ← N(Γ) ∩ (W ∪ S);                          // boundary inherited by Γ
    T_Γ ← ApproxDecompose(W_Γ ∪ S);
create a root bag W ∪ S and attach each T_Γ as a child;
return the assembled tree;
```

Theorem 24.3 ($O(1)$-approximation of treewidth)**.** *If $\mathrm{tw}(G) \leq k$, Algorithm 103 (started with $W = \emptyset$) returns a tree decomposition of width $O(k)$. With the right separator routine it runs in $2^{O(k)} \cdot n$ time and yields width $\leq 4k+4$.*

Why it works

Why does keeping $|W| \leq 3k+4$ work? Each recursive boundary W_Γ is a subset of $W \cup S$ that touches component Γ. Balanced separation guarantees no child inherits more than $\frac{2}{3}$ of W, and we add S of size $\leq k+1$; the arithmetic $\frac{2}{3}(3k+4) + (k+1) = 3k+4$ shows the bound is self-sustaining – boundaries never grow past $3k+4$, so every bag $W \cup S$ stays $O(k)$. The balance also caps the recursion depth at $O(\log n)$, keeping the size linear. The factor lost (here, roughly 4) comes from carrying the boundary plus the fresh separator in each bag.

24.6.3 Practical heuristics: elimination orderings

Almost every practical solver uses a concrete equivalent of treewidth: *vertex elimination*. Eliminate a vertex v by deleting it and adding edges to make all its former neighbors a clique ("fill-in"). An *elimination ordering* π eliminates vertices one by one; its *width* is the largest neighbor-clique size encountered, minus one.

Theorem 24.4 (Treewidth equals minimum elimination width)**.** $\mathrm{tw}(G)$ *equals the minimum over all orderings π of the elimination width of π. In addition, an ordering of width k yields a tree decomposition of width k (and vice versa) in linear time.*

Why the two notions coincide. Given a width-k decomposition, repeatedly take a leaf bag B_x and eliminate a vertex v that appears *only* in B_x (it exists by **(C)** unless the leaf duplicates its parent, in which case delete the leaf): v's neighbors all sit in B_x by the separator property, so the fill-in clique has size $\leq |B_x| - 1 \leq k$. This produces an ordering of width $\leq k$. Conversely, given an elimination

ordering, the bag of v is $\{v\} \cup$ (neighbors of v at its elimination, in the fill-in graph); arranging these bags by the ordering and connecting each v to the bag of its earliest-eliminated fill-in neighbor yields a valid tree decomposition of the same width. The fill-in edges are exactly the edges that **(E)** forces a bag to realize. □

Figure 24.4 shows a single elimination step and why the order in which vertices are eliminated governs the fill-in.

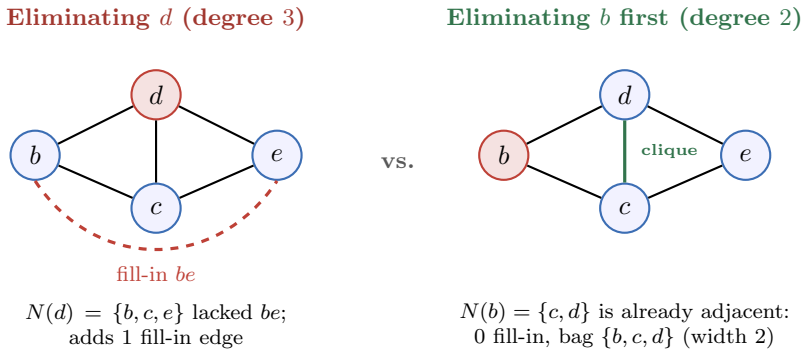


Figure 24.4: Eliminating a vertex connects all of its current neighbors into a clique. On the core $\{b, c, d, e\}$ of our running graph, eliminating d while b, c, e remain requires adding the dashed fill-in edge be. Eliminating the degree-2 vertex b first needs no fill-in because its neighbors $\{c, d\}$ already share an edge, immediately yielding the width-2 bag $\{b, c, d\}$ followed by $\{c, d, e\}$.

Heuristics now pick the ordering greedily:

- **Min-degree:** always eliminate a vertex of smallest current degree. Cheap, surprisingly effective.
- **Min-fill:** eliminate the vertex whose elimination adds the *fewest* fill-in edges. Slower per step, usually tighter width.

Neither is guaranteed to be optimal – finding a minimum-width elimination ordering is equivalent to computing treewidth itself, and therefore NP-hard – but on the structured instances that arise in practice (constraint networks, probabilistic graphical models, register allocation) they routinely find near-optimal decompositions.

Trace it yourself

Run min-degree elimination on our seven-vertex running graph. Degrees: $\deg a = 2, \deg f = 2, \deg g = 2$ are the smallest. Eliminate a first: its neighbors $\{b, c\}$ are already adjacent, so *no* fill-in, clique size 2. Then eliminate f (neighbors $\{e, g\}$, already adjacent, no fill-in). Then g (neighbor $\{e\}$, clique size 1). Now after removing a, f, g, vertex d has neighbors $\{b, c, e\}$; edges bc and ce exist, be does not, so eliminating d adds one fill edge be, clique size 3. The maximum clique size seen is 3, width $3 - 1 = 2$. The ordering certifies $\mathrm{tw} = 2$, matching our hand decomposition.

24.7 Treewidth and Fixed-Parameter Tractability

The dynamic programs in this chapter are special cases of a general theorem, which the next chapter develops in full.

Theorem 24.5 (Courcelle, 1990)**.** *Every graph property expressible in* monadic second-order logic *(MSO_2: quantification over vertices, edges, and sets thereof, with adjacency and incidence) can be decided in time $f(\mathrm{tw}, |\varphi|)\cdot n$ on graphs given with a width-*tw *tree decomposition, for some computable f.*

Independent set ("there is a vertex set I such that no edge has both endpoints in I"), dominating set, 3-colorability, Hamiltonicity – all are MSO_2-expressible, so all are linear-time on bounded-treewidth graphs by a single meta-theorem. Our hand-built DPs are just Courcelle's automaton spelled out for specific properties, usually with far better constants than the generic construction.

Connection

The bag-as-separator idea recurs throughout the book. The separator framing connects forward to Chapter 27 (planar graphs have treewidth $O(\sqrt{n})$, which is why "divide along a small separator" gives subexponential planar algorithms). It connects to the dynamic programming on trees you saw with heavy-light and centroid decompositions – those are treewidth-1 decompositions specialized for fast path and subtree queries. And it connects to linear algebra: a tree decomposition is exactly a sparse *elimination ordering* for Gaussian elimination, where the fill-in clique is the set of nonzeros created when you pivot. Minimizing treewidth is minimizing fill-in, the central question of sparse direct solvers and of nested dissection for solving $\mathbf{L}x = b$ on structured graphs. When the companion *Linear Algebra* volume discusses Cholesky fill-in and the elimination tree, it is discussing treewidth in matrix language.

Application

Bounded treewidth matters in practice. *Probabilistic inference* in Bayesian networks and Markov random fields is exactly DP over a tree decomposition of the model graph (the "junction tree" algorithm); its running time is exponential in the treewidth, which is why practitioners work hard at finding low-width elimination orders. *Register allocation* models live ranges as an interference graph; chordal interference graphs (treewidth = clique number -1) color optimally in polynomial time. *Constraint satisfaction* and SQL *query optimization* use the treewidth (or its cousin, hypertree width) of the constraint/join graph to bound evaluation cost. And many *road-network* and *VLSI* instances have small treewidth in practice, which is why preprocessing-based shortest-path engines use tree- and path-decompositions.

24.8 Pathwidth, Branchwidth, and the Wider Family

Two close relatives deserve a mention. A *path decomposition* is a tree decomposition whose tree is a path; its minimum width is the *pathwidth* $\mathrm{pw}(G) \geq \mathrm{tw}(G)$. Pathwidth corresponds to a linear sweep with a sliding window – the spine decomposition of our running graph was already a path decomposition, so it has pathwidth 2 as well. *Branchwidth* routes the graph's edges through a ternary tree and differs from treewidth by at most a constant factor: $\mathrm{bw} \leq \mathrm{tw}+1 \leq \frac{3}{2}\,\mathrm{bw}$. These all travel

together, and the deepest fact in the area is the Grid Minor Theorem (Robertson–Seymour): a graph of large treewidth contains a large grid as a minor, and grids have large treewidth, so treewidth is, up to polynomial factors, the same as "largest grid you can find inside." This correspondence between a structural obstruction (the grid) and a width parameter is the basis of bidimensionality theory and subexponential FPT algorithms on minor-closed classes.

Connection

What This Made Possible: Treewidth connects polynomial-time tree algorithms to NP-hard problems on general graphs. When a graph has bounded treewidth ($k = O(1)$), NP-hard problems like Maximum Independent Set, 3-Coloring, Dominating Set, and Hamiltonian Cycle become solvable in linear time $2^{O(k)} \cdot n$. This gives exact solutions for real-world graphs with tree-like structure, including control-flow graphs of structured programs, telecommunication backbones, and RNA secondary structures.

Limitations:

- *Exponential explosion in width:* The running time scales as $2^{O(k)}$ or $3^{O(k)}$. When $k \geq 30$, memory requirements and transition tables exceed practical computational limits.
- *Meshes, grids, and dense graphs fail:* An $n \times n$ planar grid has treewidth $\Theta(n)$, and general planar graphs can have treewidth $\Theta(\sqrt{n})$. Expander graphs and dense cliques have treewidth $\Theta(n)$, reducing the dynamic program to $2^{\Theta(n)}$ brute force.
- *Computing treewidth is NP-hard:* Determining whether $\mathrm{tw}(G) \leq k$ is NP-complete in general. While Bodlaender's algorithm can decide it in linear time for constant k, its constant factor $2^{\Omega(k^3)}$ is astronomical for $k \geq 5$.

Where We Go Next: Treewidth is only one structural parameter. When treewidth is large, can we parameterize by solution size k, or preprocess the graph to remove easy parts? In **Chapter 25 (Fixed-Parameter Tractability on Graphs)**, we develop the broader theory of FPT: kernelization, crown decompositions, and bounded search trees. For planar graphs where treewidth is $\Theta(\sqrt{n})$, we develop topological tools in **Chapter 26 (Planar Graphs)** and **Chapter 27 (Separators)**.

24.9 Exercises

Exercise 24.1 (Treewidth of small graphs)**.** Compute the treewidth of: (a) the cycle C_n, (b) the complete bipartite graph $K_{2,n}$, (c) the 3-dimensional hypercube Q_3, (d) a single triangle with a pendant path of length 5 attached. Prove both the upper bound (exhibit a decomposition) and the lower bound (exhibit a clique or use Proposition 24.1).

Exercise 24.2 (The separator lemma, sharpened)**.** Prove a converse flavor of Lemma 24.1: if every bag of a purported decomposition acts as the stated separator and conditions **(V)**,**(E)** hold, must condition **(C)** hold? Either prove it or give a counterexample.

Exercise 24.3 (Implement MWIS)**.** Implement Algorithm 102 end to end: a min-degree heuristic to get an elimination ordering, conversion to a nice tree decomposition, and the DP. Test it against brute force on random graphs of 12 vertices, and on a 4×4 grid (treewidth 4). Report the empirical 2^k scaling.

Exercise 24.4 (Reconstruct the witness)**.** Algorithm 102 returns only the optimum *value*. Augment

it to return an actual maximum-weight independent set by storing back-pointers and walking down from the root. Prove your reconstruction is correct.

Exercise 24.5 (Dominating set transitions in full)**.** Write out the complete introduce/forget/join tables for the dominating-set DP of Section 24.5, including the bookkeeping that upgrades a bag-neighbor from white-$\hat{0}$ to white-0 when a black vertex is introduced. Trace your rules on the path P_4 and confirm the minimum dominating set has size 2.

Exercise 24.6 (Three-coloring)**.** Design a DP over a nice tree decomposition that decides 3-colorability in $O(3^k \cdot \mathrm{poly}(k) \cdot n)$ time. The bag state is a coloring of B_x with $\{1, 2, 3\}$; state the introduce, forget, and join rules and prove the join rule correct using the separator property.

Exercise 24.7 (Break a fake decomposition)**.** Give three bags B_1, B_2, B_3 on a path $B_1 - B_2 - B_3$ that satisfy **(V)** and **(E)** for some graph but violate **(C)**, and exhibit a concrete edge of the graph for which the bag-as-separator lemma fails. This shows **(C)** is not redundant.

Exercise 24.8 (Elimination = fill-in)**.** Prove the two directions of Theorem 24.4 in full detail for a worked 6-vertex graph: pick an ordering, compute the fill-in graph and its width, then read off the corresponding tree decomposition and verify all three conditions hold with the same width.

Exercise 24.9 (Series-parallel graphs)**.** A graph is *series-parallel* if it is built from a single edge by series and parallel compositions. Prove every series-parallel graph has treewidth ≤ 2, and conversely that every graph of treewidth ≤ 2 is a subgraph of a series-parallel graph. (Hint: treewidth ≤ 2 graphs are exactly the subgraphs of partial 2-trees.)

Exercise 24.10 (Pathwidth gap)**.** Exhibit a family of trees (treewidth 1) whose pathwidth grows like $\Theta(\log n)$. The complete binary tree of depth d is the canonical example: prove its pathwidth is roughly $\lceil d/2 \rceil$. This shows pathwidth can be unboundedly larger than treewidth.

Exercise 24.11 (Grid lower bound)**.** Prove that the $n \times n$ grid has treewidth $\geq n$. (Hint: any balanced separator of the grid has size $\geq n$, and Lemma 24.2 turns small treewidth into small balanced separators; contrapose.)

Exercise 24.12 (Treewidth under operations)**.** Show how treewidth changes under: adding one edge (tw rises by at most 1), contracting an edge (tw does not increase), and taking the disjoint union (tw is the max). Use these to bound the treewidth of K_n minus a perfect matching.

Chapter 25

Fixed-Parameter Tractability on Graphs

In the preceding chapter, we saw how bounding treewidth lets us solve NP-hard problems in linear time by pushing the exponential cost onto the width of a tree decomposition. Treewidth is one parameter that makes hard problems tractable. But what if a graph does not have small treewidth? Can we still isolate the exponential blowup by measuring some other feature of the instance? Parameterized complexity provides the broader framework for this question.

Most classic graph optimization problems are NP-hard: vertex cover, feedback vertex set, longest path, clique, dominating set. Standard NP-completeness treats all instances equally and says no more once an algorithm cannot be polynomial in n. Parameterized complexity asks a sharper question: *where* does the exponential blow-up really live? For vertex cover, one often seeks a cover of size at most k, where k is small. If we can push the entire combinatorial explosion onto k and keep the dependence on n polynomial, then for small k the problem becomes practical. Here we examine the four core techniques that achieve this: bounded search trees, kernelization, iterative compression, and color coding.

25.1 Parameterized problems

Fix a problem and a way to read a second number k off each instance. Formally a *parameterized problem* is a language $L \subseteq \Sigma^* \times \mathbb{N}$; an instance is a pair (x, k) where x is the usual input (a graph, say) of size $n = |x|$ and k is the parameter.

Definition 25.1 (Fixed-parameter tractable)**.** A parameterized problem is *fixed-parameter tractable* (FPT) if there is an algorithm deciding $(x, k) \in L$ in time

$$f(k) \cdot |x|^c$$

for some computable function f and a constant c independent of both n and k. The class of all such problems is denoted FPT.

The function f is allowed to grow rapidly — 2^k, $k!$, 2^{2^k} — but it touches only the parameter. The dependence on the input size is a fixed polynomial. Contrast this with the obvious brute force.

Definition 25.2 (Slice-wise polynomial)**.** A problem is in XP if it can be solved in time $f(k) \cdot |x|^{g(k)}$ for computable f, g. Equivalently, for every fixed k there is a polynomial-time algorithm, but the degree of the polynomial is allowed to grow with k.

Intuition

The difference between FPT and XP is the difference between $2^k \cdot n$ and n^k. Both are "polynomial for fixed k," but only one is usable. Take $n = 10^6$ and $k = 20$. The FPT bound $2^{20} \cdot 10^6 \approx 10^{12}$ is a long afternoon. The XP bound $n^k = (10^6)^{20} = 10^{120}$ is more operations than there are atoms in the observable universe, many times over. FPT moves the parameter out of the exponent of n; XP leaves k in the degree of n, and that difference decides what can actually run.

Focus

Facts we use throughout.

- FPT $\subseteq$ XP, and the inclusion is strict (there are problems solvable in $n^{f(k)}$ time but provably not in $f(k) \cdot n^{O(1)}$).
- Brute-force "try all $\binom{n}{k}$ subsets" is the canonical XP algorithm: $O(n^k)$. The goal is to replace it by $f(k) \cdot n^{O(1)}$.
- A problem is FPT *if and only if* it admits a kernel (Theorem 25.1). This equivalence is central to the field.
- The choice of parameter matters enormously. Vertex cover parameterized by solution size is FPT; parameterized by treewidth it is also FPT; clique parameterized by solution size is believed *not* to be FPT. The graphs are the same; only the parameter differs.

History

Parameterized complexity was founded in the late 1980s and 1990s by Rod Downey and Michael Fellows. In practical applications ranging from VLSI circuit design to computational biology and compiler register allocation, engineers repeatedly faced NP-hard problems where a critical budget or threshold parameter k was small ($k \leq 15$), while the graph size n was massive ($n \geq 10^6$). Classical Cook–Karp NP-completeness declared all these problems flatly intractable, offering no guidance on why some small-parameter instances were easily solved while others resisted all computation.

A puzzle in Karp's original 1972 reductions motivated the theory: Vertex Cover and Independent Set are polynomial-time equivalent via the trivial complement map (S is an independent set if and only if $V \setminus S$ is a vertex cover). Yet Vertex Cover possessed an $O(2^k n)$ bounded search tree, while Independent Set seemed to require $O(n^k)$ brute-force search. Early researchers spent years trying to adapt Vertex Cover branching and kernelization rules to Independent Set and Clique, but every attempt failed.

Downey and Fellows resolved the puzzle by observing that standard polynomial-time Karp reductions do not preserve parameter size: a cover of size k maps to an independent set of size $n - k$, turning a small parameter into a large one. To separate these behaviors, they introduced *parameterized reductions* (requiring the new parameter $k' \leq g(k)$) and defined the W-hierarchy $(W[1], W[2], \dots)$. By proving that k-Clique is $W[1]$-hard, they provided the first

mathematical proof that Vertex Cover and Independent Set belong to fundamentally distinct complexity classes under parameterization.

25.2 A graph to carry through the chapter

We will keep returning to one small graph G on seven vertices.

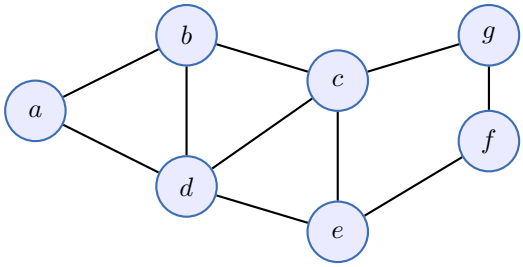


Figure 25.1: Running example $G = (V, E)$ with $V = \{a, b, c, d, e, f, g\}$ and ten edges. The triangles $a\,b\,d$, $b\,c\,d$, $c\,d\,e$ and the 4-cycle $c\,e\,f\,g$ overlap, which makes it a useful test bed for both vertex cover and feedback vertex set.

Trace it yourself

Before the algorithms arrive, find a vertex cover of G by hand — a set of vertices touching every edge. Try $\{b, c, d, e\}$: edge $f\,g$ is left uncovered, so no. Now hunt for size 4. Test $\{b, c, d, f\}$ against all ten edges: ab (b), ad (d), bc (b), bd (b), cd (c), ce (c), cg (c), de (d), ef (f), fg (f). All covered. So $\{b, c, d, f\}$ is a cover of size 4. Keep this number 4 in mind.

25.3 Bounded search trees: vertex cover in $O(2^k(n + m))$

The first technique is the most elementary and the most broadly useful. The idea: make a *branching* decision that is forced to be small (at most a constant number of options) and that decreases the parameter by at least one on every branch. The recursion then has bounded depth k and bounded fan-out, so the tree of recursive calls has size $f(k)$, and each node does polynomial work.

25.3.1 The branching rule

Vertex Cover: given $G = (V, E)$ and an integer k, is there a set $S \subseteq V$ with $|S| \leq k$ covering every edge?

Pick any edge $\{u, v\}$ that is still uncovered. Any vertex cover must contain u or v — otherwise that edge goes uncovered. That single observation gives a two-way branch:

$$\text{either } u \in S \quad \text{or} \quad v \in S.$$

In the first branch we put u into the cover, delete u (and all its incident edges), and recurse with

budget $k-1$. In the second we do the same with v. Each branch spends one unit of budget. After at most k levels we either run out of edges (success) or run out of budget (failure on this path).

Algorithm 104: VC-Branch(G, k) — bounded search tree for vertex cover

Input: graph $G = (V, E)$, budget $k \geq 0$
Output: a vertex cover of size $\leq k$, or None

```
if E = ∅ then
    return ∅                                   // all edges covered
if k = 0 then
    return None                                // out of budget but edges remain
pick any edge {u, v} ∈ E
S_u ← VC-Branch(G − u, k − 1)
if S_u ≠ None then return S_u ∪ {u}
S_v ← VC-Branch(G − v, k − 1)
if S_v ≠ None then return S_v ∪ {v}
return None
```

Here $G - u$ denotes G with vertex u and its incident edges removed.

Why it works

Correctness. We argue VC-Branch(G, k) returns a cover of size $\leq k$ iff one exists. The base cases are immediate: no edges means the empty set covers everything; no budget but a surviving edge means failure on this branch. For the inductive step, fix an edge $\{u, v\}$. Every vertex cover contains u or v. If a cover $S^\star$ of size $\leq k$ exists and $u \in S^\star$, then $S^\star \setminus \{u\}$ is a cover of $G-u$ of size $\leq k-1$, which the recursion on the u-branch finds by induction; symmetrically if $v \in S^\star$. Conversely any set the algorithm returns is, by construction, a union of branch vertices that together hit every deleted edge, hence a genuine cover. The two branches are exhaustive because the alternatives "$u \in S$" and "$v \in S$" cover all possibilities (they may overlap; that is fine, we only need completeness).

Why it works

Complexity. Let $T(k)$ be the number of leaves of the search tree. The recurrence is $T(k) = 2\,T(k-1)$ with $T(0) = 1$, so $T(k) \leq 2^k$. The tree has $O(2^k)$ nodes. At each node we find an uncovered edge and delete a vertex, $O(n+m)$ work with adjacency lists. Total:

$$O\big(2^k(n+m)\big).$$

For $k = 20$ on a million-edge graph this is about 10^{12} basic steps — feasible. The naive $\binom{n}{k}$ enumeration would be $\binom{10^6}{20} \approx 10^{100}$. The branching has moved the exponent entirely onto k.

25.3.2 Trace on the running graph

Run VC-Branch$(G, 4)$ on Figure 25.1. Suppose the edge picker always returns the lexicographically first uncovered edge.

Trace it yourself

Step by step. The algorithm explores the first endpoint of each chosen edge first, so we follow that branch and watch it succeed.

- First edge: ab. Take the a-branch: delete a, budget 3. Edges left: $bc, bd, cd, ce, cg, de, ef, fg$ (the edges ab, ad vanished with a).
- First edge now bc. Take the c-branch: delete c, budget 2. Edges left: bd, de, ef, fg (the edges bc, cd, ce, cg vanished with c).
- First edge bd. Take the d-branch: delete d, budget 1. Edges left: ef, fg (the edges bd, de vanished with d).
- First edge ef. Take the f-branch: delete f, budget 0. Edges left: $\emptyset$. Success with $\{a, c, d, f\}$, a size-4 cover.

The search tree visits at most $2^4 = 16$ leaves; the winning path is one root-to-leaf walk of length 4. The algorithm returns the first size-4 cover it reaches; here that is $\{a, c, d, f\}$ rather than the $\{b, c, d, f\}$ we found by hand earlier, but both are valid covers of size 4.

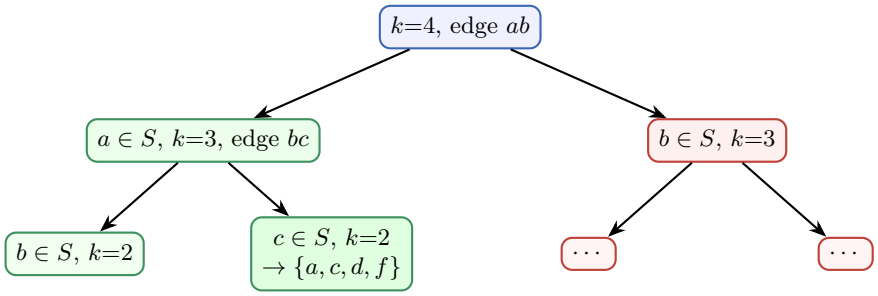


Figure 25.2: Top of the bounded search tree for VC-Branch$(G, 4)$. Each branch fixes one endpoint of an uncovered edge and spends one unit of budget; depth is at most $k = 4$, fan-out is 2, so at most 2^4 leaves. The algorithm explores the a-branch first; the green path reaches the cover $\{a, c, d, f\}$, so the b-branch is never visited.

Common trap

A tempting "optimization" is to branch greedily on a maximum-degree vertex: "either the high-degree vertex u is in the cover, or it is not." But the second branch "$u \notin S$" is *not* free: it forces *all* neighbors of u into the cover at once. That is actually a good rule — it gives the famous $O(1.47^k)$ branching — but only if you remember to put every neighbor in and decrease k by $\deg(u)$ on that branch. Writing "$u \notin S$, recurse with k unchanged" is the classic bug: it never decreases the budget on one branch, the recursion does not terminate with a 2^k bound, and worse, it is simply wrong, because not taking u obliges you to take its neighborhood. Branching rules must be both exhaustive and budget-decreasing on every branch.

Why this matters

The $O(2^k)$ bound is the simplest version. With smarter branching — handling degree-1 and degree-2 vertices by reduction rules, then branching on a degree-≥ 3 vertex either "in" or

"out (so all ≥ 3 neighbors in)" — the base of the exponent drops. The recurrence $T(k) = T(k-1) + T(k-3)$ solves to $O(1.466^k)$, and the current record is around $O(1.2^k)$. The lesson generalizes: every clever case analysis you can prove is a branching rule with a better characteristic root, and the running time is governed by that root.

25.4 Kernelization: shrink before you search

Branching attacks a hard instance directly. Kernelization does the opposite: it spends polynomial time *pre-processing* the instance down to an equivalent one whose size is bounded by a function of k alone. Then you can run any algorithm — even brute force — on the tiny remainder.

Definition 25.3 (Kernel)**.** A *kernelization* for a parameterized problem L is a polynomial-time algorithm that maps an instance (x, k) to an instance (x', k') such that

1. $(x, k) \in L \iff (x', k') \in L$ (equivalence),
2. $|x'| \leq g(k)$ and $k' \leq k$ for some computable function g (the *kernel size*).

The kernel is *polynomial* (or *linear*) if g is a polynomial (linear) function.

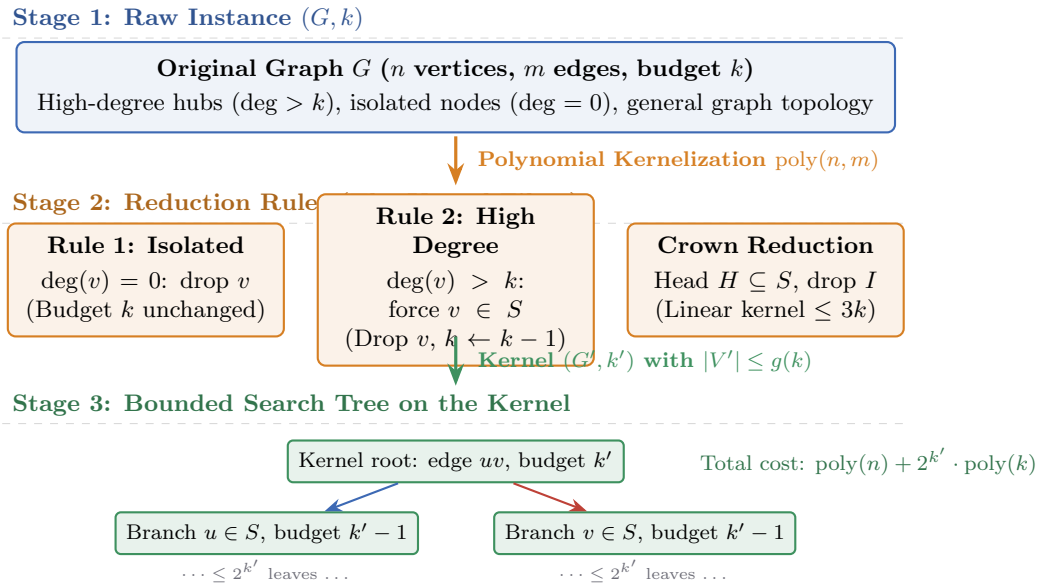


Figure 25.3: The two-stage FPT architecture. **Stage 1:** The raw problem instance (G, k) may have millions of vertices. **Stage 2:** Polynomial-time reduction rules prune isolated vertices and force high-degree vertices into the cover, shrinking the graph to an equivalent kernel G' whose size depends only on k. **Stage 3:** The bounded search tree branches only on the compact kernel, confining exponential work strictly to 2^k.

Theorem 25.1 (Kernelization characterizes FPT)**.** *A decidable parameterized problem is in FPT if and only if it admits a kernelization.*

Why it works

Proof. ($\Leftarrow$) Kernelize in time $p(n)$ to size $g(k)$, then solve the kernel by brute force in time $h(g(k))$ for some computable h (decidability gives a finite, hence bounded, exhaustive procedure once the size is bounded by $g(k)$). Total $p(n) + h(g(k)) = f(k) \cdot n^{O(1)}$, so the problem is FPT.
($\Rightarrow$) Suppose an FPT algorithm runs in time $f(k) \cdot n^c$. Run it for n^{c+1} steps. *Case 1:* it finishes — then we know the answer and output a trivial constant-size yes- or no-instance accordingly. *Case 2:* it does not finish within n^{c+1} steps — then $f(k) \cdot n^c > n^{c+1}$, i.e. $f(k) > n$, so $n < f(k)$ and the input itself is already smaller than $f(k)$; output (x, k) unchanged as the kernel. Either way we produce, in polynomial time, an equivalent instance of size bounded by a function of k. □

This equivalence is clean theoretically, though the kernel its proof produces is useless in practice (it leaves the instance untouched whenever $n \leq f(k)$). Useful kernels come from problem-specific *reduction rules*. We develop two for vertex cover.

25.4.1 Buss's elementary kernel

Focus

Buss reduction rules for Vertex Cover with budget k.

Rule 1 (isolated): delete any vertex of degree 0 (it covers nothing, costs nothing). k unchanged.

Rule 2 (high degree): if some vertex v has $\deg(v) > k$, then v *must* be in every cover of size $\leq k$ — otherwise all $> k$ of its edges would need $> k$ other endpoints. Put v in the cover, delete v, set $k \leftarrow k - 1$.

Apply both until neither fires.

Why it works

Why Rule 2 is safe. If $v \notin S$ for a cover S, then S must contain all $\deg(v) > k$ neighbors of v to cover the edges at v, forcing $|S| > k$. Contrapositive: any cover of size $\leq k$ contains v. So forcing v into the cover loses no solutions.
Kernel size. After the rules saturate, every remaining vertex has degree $\leq k$. If the reduced graph still has more than k^2 edges, answer No immediately: a cover of size $\leq k$, each vertex covering $\leq k$ edges, can cover at most $k \cdot k = k^2$ edges, so $> k^2$ edges with max-degree $\leq k$ is infeasible. If it has $\leq k^2$ edges, then (ignoring the already-deleted isolated vertices) it has at most $2k^2$ non-isolated vertices. The kernel has $O(k^2)$ vertices and $O(k^2)$ edges.

Trace it yourself

Run Buss on G with $k = 3$ (we expect a No, since the optimum is 4). Degrees: a=2, b=3, c=4, d=4, e=3, f=2, g=2. With $k = 3$, vertices c and d have degree $4 > 3$, so Rule 2 forces both into the cover. Delete c, d; now $k = 1$ and the surviving edges are ab, ef, fg (everything

touching c or d is gone). Three edges with budget 1: no single vertex covers the disjoint edges ab and fg, and $3 > k^2 = 1$, so the "too many edges" rule fires: No. Correct — there is no cover of size 3.

25.4.2 The crown reduction

Buss gives a quadratic kernel cheaply. The *crown reduction* gives a linear kernel. A *crown* is a structure that lets you greedily commit part of the cover.

Definition 25.4 (Crown)**.** A *crown* of $G = (V, E)$ is a pair (I, H) of disjoint vertex sets with:

1. $I \neq \emptyset$ is an independent set (no edges inside I);
2. $H = N(I)$ is exactly the set of neighbors of I (the *head*);
3. there is a matching of H into I that saturates H (every head vertex is matched to a private I-vertex).

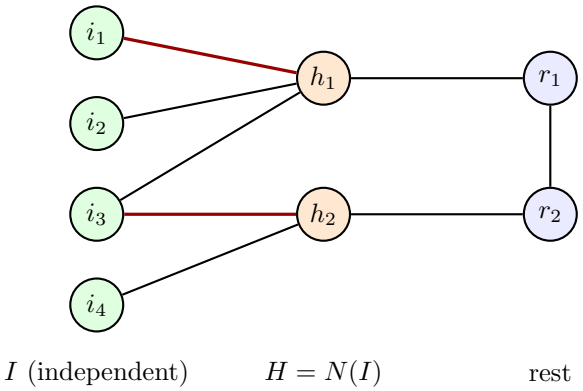


Figure 25.4: A crown (I, H). The independent set I on the left, its neighborhood H in the middle, and a matching (thick red) that saturates H into I. Every edge incident to I must be covered; the saturating matching certifies that any cover needs at least $|H|$ vertices for these edges, and taking all of H achieves it.

Focus

Crown reduction rule. If G has a crown (I, H), then there is a minimum vertex cover containing all of H and none of I. Remove $I \cup H$ from G and set $k \leftarrow k - |H|$.

Why it works

Why it is safe. The saturating matching M pairs each $h \in H$ with a distinct $i \in I$. These $|H|$ matching edges are vertex-disjoint, so *any* cover must spend at least $|H|$ vertices to cover them. Now compare two ways to cover all edges incident to I:

- Take all of H: cost $|H|$. Since $H = N(I)$, every edge touching I has an endpoint in H,

so this covers them all.

- Take some vertices of I instead: the matching already forces $\geq |H|$ vertices among $I \cup H$, and I is independent, so I-vertices cover *only* edges going to H — they buy you nothing outside the crown.

Hence given any cover S, the set $(S \setminus I) \cup H$ is a cover of size $\leq |S|$ by the matching lower bound. So an optimum cover taking all of H and none of I exists, and we may commit to it. □

How do we *find* a crown? A clean route uses maximum matchings and the Kőnig/Dulmage–Mendelsohn decomposition.

Algorithm 105: FindCrown(G) — crown via two matchings

Input: graph $G = (V, E)$
Output: a crown (I, H), or report "no crown found"

```
M1 ← a maximum matching of G
if M1 saturates V then return "no crown"
O ← set of M1-unmatched vertices
M2 ← a maximum matching of the bipartite graph between O and N(O)
H ← vertices of N(O) matched by M2
I ← vertices reachable from O by M2-alternating paths, restricted to the O-side
return (I, H)
```

The construction guarantees that whenever G has more than $3k$ vertices after Buss, a crown exists and is found, driving the kernel to at most $3k$ vertices.

Theorem 25.2 (Linear crown kernel)**.** *Exhaustively applying Buss's rules and the crown reduction yields, in polynomial time, an equivalent vertex-cover instance with at most* $3k$ *vertices, or correctly reports* No.

Connection

The strongest known vertex-cover kernel comes not from crowns but from *linear programming*. Relax the integer program to $x_v \in [0, 1]$ with $x_u + x_v \geq 1$ per edge; by the Nemhauser–Trotter theorem the LP has a half-integral optimum $x_v \in \{0, \frac{1}{2}, 1\}$, and there is an optimum integral cover taking every v with $x_v = 1$ and no v with $x_v = 0$. Removing those decided vertices leaves only the all-$\frac{1}{2}$ core, of size $\leq 2k$. The LP is solvable by a single bipartite matching (LP duality = Kőnig), so this $2k$ kernel is computable in near-linear time. The same half-integrality underlies the "above-LP" FPT algorithm of Lokshtanov, Narayanaswamy, Raman, Ramanujan and Saurabh. Here the polyhedral viewpoint of the companion *Linear Algebra* volume is directly useful: the geometry of the LP polytope gives the kernel.

Cost summary

Vertex cover, the parameterized scorecard.

Technique	Result	Cost
Naive enumeration	XP	$O(n^k)$
Bounded search tree	FPT	$O(2^k(n+m))$
Refined branching	FPT	$O(1.466^k \text{ poly})$, best $\approx 1.2^k$
Buss kernel	quadratic	$O(k^2)$ vertices/edges
Crown kernel	linear	$\le 3k$ vertices
LP / Nemhauser–Trotter kernel	linear	$\le 2k$ vertices

Best practice: kernelize to $2k$ vertices, *then* branch on the kernel. The branching cost becomes $O(1.2^k \cdot k^{O(1)})$ with no n anywhere near the exponent.

25.5 Iterative compression: feedback vertex set

The third technique answers a recurring difficulty: many problems are easy to *verify and shrink* a candidate solution but hard to build one from scratch. *Iterative compression*, introduced by Reed, Smith and Vetta for odd cycle transversal, turns "I have a solution of size $k+1$, find one of size k" into the core step of an FPT algorithm. We use it on Feedback Vertex Set (FVS): find $\le k$ vertices whose deletion makes the graph a forest (acyclic).

25.5.1 The compression framework

Build the graph up one vertex at a time. Maintain the invariant: *after processing the first i vertices, we hold a feedback vertex set S_i of size $\le k$ for the induced subgraph G_i.* Add the next vertex v_{i+1}. Then $S_i \cup \{v_{i+1}\}$ is an FVS of G_{i+1} of size $\le k+1$ (adding a vertex to an FVS keeps it an FVS). Now run a *compression* routine: given an FVS of size $k+1$, either produce one of size $\le k$, or prove none exists — in which case the whole instance is a No.

Algorithm 106: IterativeFVS(G, k) — build-and-compress

Input: graph $G = (V, E)$ with $V = \{v_1, \dots, v_n\}$, budget k
Output: an FVS of size $\le k$, or None

```
S ← ∅;   V_0 ← ∅
for i ← 1 to n do
    V_i ← V_{i−1} ∪ {v_i};   G_i ← G[V_i]
    W ← S ∪ {v_i}                                   // FVS of G_i of size ≤ k+1
    S ← Compress(G_i, W, k)
    if S = None then return None
return S
```

The outer loop runs n times; if each Compress call is FPT in k, the whole thing is FPT.

25.5.2 The compression step

We are given an FVS W with $|W| = k+1$ and want a smaller one. The trick is to guess the *intersection* of the new solution with the old one. Iterate over all subsets $Z \subseteq W$ that the new

solution S keeps (so $Z = S \cap W$); there are 2^{k+1} of them. Vertices in $W \setminus Z$ are then *forbidden* from S. After committing Z to the solution and deleting it, we must find an FVS of size $\le k - |Z|$ in the remaining graph that uses *no* vertex of $W \setminus Z$. This is the *disjoint compression* problem:

> DISJOINT-FVS: given G and a feedback vertex set W' (here $W \setminus Z$) such that $G - W'$ is a forest, find an FVS of size $\le k'$ that is *disjoint* from W'.

Why it works

Why is DISJOINT-FVS easier? Because the forbidden set W' induces a forest *and* $G - W'$ is a forest. So G is the union of two forests plus the edges between them — a highly constrained structure (Figure 25.5). The solution we seek lives entirely in the forest $F = G - W'$, and we must break every cycle of G that survives when we are not allowed to touch W'. A vertex of F with ≥ 2 edges into W', or one lying on a path between two W'-vertices through F, is a natural candidate for the solution. A small further branching — repeatedly take a leaf of F into the solution or push it to the W'-side — solves DISJOINT-FVS in $O(c_0^k \,\mathrm{poly})$, and modern analyses reach $O(4^k \,\mathrm{poly})$ or better.

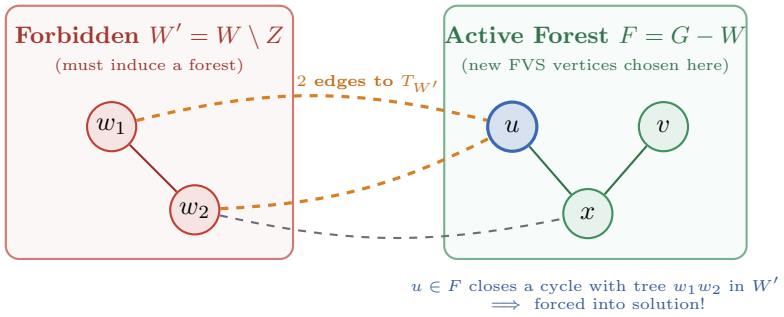


Figure 25.5: The DISJOINT-FVS structure after guessing $Z = S \cap W$ and deleting Z. Both the forbidden remainder $W' = W \setminus Z$ and the active subgraph $F = G - W$ are forests, so every surviving cycle must alternate between W' and F. Any vertex $u \in F$ with two edges into the same tree of W' immediately closes a cycle and must be taken into the new FVS.

Algorithm 107: COMPRESS(G, W, k) — shrink an FVS by guessing its overlap

Input: graph G, FVS W with $|W| = k + 1$, budget k
Output: an FVS of size $\le k$, or NONE

```
for each subset Z ⊆ W with |Z| ≤ k do
    W' ← W \ Z                          // kept from old solution; forbidden for new
    if G[W'] is acyclic then
        R ← DisjointFVS(G − Z, W', k − |Z|)
        if R ≠ None then return Z ∪ R
return None
```

Why it works

Correctness of compression. Suppose G_i has an FVS $S^\star$ of size $\le k$. Let $Z^\star = S^\star \cap W$. The loop eventually tries $Z = Z^\star$. Then $W' = W \setminus Z^\star$ consists of old-solution vertices not reused; since $S^\star \cap W' = \emptyset$, the set $S^\star \setminus Z^\star$ is an FVS of $G - Z^\star$ disjoint from W' of size $\le k - |Z^\star|$ — exactly what DisjointFVS searches for, and finds. Conversely any $Z \cup R$ returned is an FVS of size $\le k$ by construction. The guard "$G[W']$ acyclic" is needed because the disjoint subproblem assumes the forbidden set induces a forest; if it does not, no disjoint solution exists for that Z and we skip it.

Cost summary

Compress tries 2^{k+1} subsets Z; each calls DisjointFVS, FPT in k at cost $O(c_0^k \,\mathrm{poly})$. So one compression costs $O(2^{k+1} c_0^k \,\mathrm{poly}) = O(c^k \,\mathrm{poly})$ for a constant c. The outer loop multiplies by n. Total IterativeFVS: $O(c^k \cdot n \cdot \mathrm{poly})$, fixed-parameter tractable. The classical bound from this framework is around $O(5^k \cdot kn)$; refined branchings reach $O(3.46^k)$ and randomized methods $O(3^k)$.

25.5.3 Trace iterative compression on the running graph

Take G from Figure 25.1 with $k = 2$. Its short cycles are the triangles $a\,b\,d$, $b\,c\,d$, $c\,d\,e$ and the 4-cycle $c\,e\,f\,g$. Vertex d sits on the first three; the fourth avoids d entirely, which is what forces the second solution vertex.

Trace it yourself

Add vertices in order a, b, c, d, e, f, g.

- After a, b: only edge ab, no cycle. $S = \emptyset$.
- Add c: edges ab, bc. Still a tree. $S = \emptyset$.
- Add d: two triangles appear at once, $a\,b\,d$ (edges ab, ad, bd) and $b\,c\,d$ (edges bc, bd, cd), sharing the edge bd. $W = \{d\}$, size $1 \le k + 1$. Compress with budget 2: d lies on both triangles, so $\{d\}$ breaks both and has size $1 \le 2$. $S = \{d\}$.
- Add e: triangle $c\,d\,e$ appears. Does $\{d\}$ still work? Deleting d kills abd, bcd and cde at once. Yes. $S = \{d\}$.
- Add f: edge ef, f a leaf, no new cycle. $S = \{d\}$.
- Add g: edges cg, fg, the 4-cycle $c\,e\,f\,g$ appears and does *not* contain d. So $\{d\}$ fails. Set $W = \{d, g\}$ (size $2 \le k + 1$). Compress with budget 2: $\{d, g\}$ hits abd (d), bcd (d), cde (d), $cefg$ (g) — all cycles broken, size $2 \le k$. $S = \{d, g\}$.

So G has an FVS of size 2, e.g. $\{d, g\}$. This is in fact optimal: no single vertex breaks every cycle, because triangle $a\,b\,d$ and the 4-cycle $c\,e\,f\,g$ are vertex-disjoint, so any FVS needs at least one vertex from each, hence size ≥ 2. (A tempting guess is $\{c\}$, since c lies on bcd, cde and $cefg$; but c misses triangle abd, so $\{c\}$ leaves the cycle $a\,b\,d$ intact.) Iterative compression certifies a solution of size $\le k$ for the k you ask for; to get the minimum, run it with the target k (or binary-search k upward).

25.6 Color coding: the longest path

The fourth technique uses randomization. Longest Path (does G contain a simple path on k vertices?) is NP-hard for general k, and the obstacle is the word "simple": a walk of length k is trivial by matrix powering, but forbidding repeated vertices is what makes it hard. *Color coding*, due to Alon, Yuster and Zwick, sidesteps the repetition problem with randomness: color the vertices with k colors at random and look only for *colorful* paths — paths using each color exactly once. A colorful path is automatically simple, and colorfulness is exactly what dynamic programming over color *subsets* can track.

Intuition

Searching for a simple k-path directly forces you to remember which vertices you have used — a set of up to n possibilities, exponential. Random coloring replaces "which vertices" with "which colors," and there are only k colors, so the bookkeeping set has 2^k states instead of 2^n. The price is randomness: a fixed k-path becomes colorful only some of the time. But the probability is good enough, $k!/k^k \approx e^{-k}$, that $O(e^k)$ independent colorings catch it with high probability. We pay a factor of e^k to replace 2^n states by 2^k.

25.6.1 Colorful-path dynamic programming

Color each vertex with a uniformly random color in $\{1, \dots, k\}$. A path is *colorful* if its k vertices carry k distinct colors. We seek a colorful path on k vertices. Dynamic program: for each vertex v and each color set $C \subseteq \{1, \dots, k\}$, let

$$D[v, C] = \text{true} \iff \text{there is a colorful path ending at } v \text{ using exactly the color set } C.$$

Then $D[v, \{\text{color}(v)\}] = \text{true}$ for every v, and

$$D[v, C] = \bigvee_{\substack{u \sim v \\ \text{color}(v) \in C}} D[u,\ C \setminus \{\text{color}(v)\}].$$

A colorful k-path exists iff $D[v, \{1, \dots, k\}] = \text{true}$ for some v.

Algorithm 108: ColorfulPath(G, k, color) — DP over color subsets

Input: graph $G = (V, E)$, target length k, a coloring $\text{color} : V \to \{1, \dots, k\}$
Output: True if a colorful k-path exists under this coloring

```
foreach v ∈ V do  set D[v, {color(v)}] ← True, all other D[v, ·] ← False
for s ← 2 to k do
    foreach color set C with |C| = s do
        foreach v with color(v) ∈ C do
            foreach neighbor u ∼ v do
                if D[u, C \ {color(v)}] then  set D[v, C] ← True
return ⋁_v D[v, {1, ..., k}]
```

Why it works

Correctness of the DP. Induction on path length s. Base $s = 1$: the single-vertex path at v uses color set $\{\text{color}(v)\}$. Step: a colorful path of s vertices ending at v with color set C is a colorful path of $s-1$ vertices ending at some neighbor u with color set $C \setminus \{\text{color}(v)\}$, extended by the edge uv — and $\text{color}(v)$ must lie in C but not in the prefix's set, which is precisely the recurrence. Colorfulness is enforced because C is a *set*: re-using a color cannot grow $|C|$, so a path that revisits a color never reaches $|C| = s$. A colorful path is simple because distinct colors force distinct vertices. □

Why it works

Success probability. Fix any simple k-path P in G (if one exists). Under a uniform random coloring, the k vertices of P get k distinct colors with probability

$$\Pr[P \text{ colorful}] = \frac{k!}{k^k} \geq \frac{\sqrt{2\pi k}\,(k/e)^k}{k^k} = e^{-k}\sqrt{2\pi k} \;\geq\; e^{-k},$$

using Stirling. So one random coloring detects a fixed target path with probability $\geq e^{-k}$. Repeating with $N = \lceil e^k \rceil$ independent colorings, the chance of missing P in *all* of them is $\leq (1 - e^{-k})^{e^k} \leq e^{-1}$. Taking $N = \lceil c \cdot e^k \rceil$ colorings drops the failure probability to e^{-c}, as small as we like. The algorithm has one-sided error: it never reports a path that is not there (the DP only finds genuine colorful, hence simple, paths), and it finds an existing path with probability $\geq 1 - e^{-c}$.

Cost summary

One DP pass fills a table of $O(2^k \cdot n)$ entries, each touching $O(\deg)$ neighbors, for $O(2^k \cdot m)$ time. Repeating $O(e^k)$ times for high probability gives

$$O\big((2e)^k \cdot m \cdot \log(1/\delta)\big) = 2^{O(k)} \cdot m,$$

fixed-parameter tractable in k, with failure probability δ. Derandomization via (n, k)-*perfect hash families* replaces the random colorings by a deterministic family of $O(e^k k^{O(\log k)} \log n)$ colorings guaranteed to make any fixed k-set colorful at least once, yielding a deterministic $2^{O(k)}\,\text{poly}(n)$ algorithm. The algebraic descendant of Koutis and Williams pushes the base to 2^k.

25.6.2 Trace color coding by hand

Look for a path on $k = 4$ vertices in G (Figure 25.1). A real one exists, e.g. $a\,b\,c\,g$.

Trace it yourself

Pick the coloring $\text{color}(a) = 1,\ b = 2,\ c = 3,\ d = 1,\ e = 4,\ f = 3,\ g = 4$. Hunt for a colorful 4-path (colors $\{1, 2, 3, 4\}$ all distinct).

- Length 1: $D[a, \{1\}], D[b, \{2\}], D[c, \{3\}], D[d, \{1\}], D[e, \{4\}], D[f, \{3\}], D[g, \{4\}]$ all true.

- Length 2: edge $a\,b$ gives $D[b, \{1,2\}]$; edge $b\,c$ gives $D[c, \{2,3\}]$; edge $c\,g$ gives $D[g, \{3,4\}]$; edge $d\,e$ gives $D[e, \{1,4\}]$.
- Length 3: extend $D[b, \{1,2\}]$ along $b\,c$ ($\text{color}(c) = 3$): $D[c, \{1,2,3\}]$ true (path $a\,b\,c$). Extend $D[c, \{2,3\}]$ along $c\,g$: $D[g, \{2,3,4\}]$ true (path $b\,c\,g$).
- Length 4: extend $D[c, \{1,2,3\}]$ (path $a\,b\,c$) along $c\,g$, $\text{color}(g) = 4$: $D[g, \{1,2,3,4\}]$ true. Colorful 4-path found: $a\,b\,c\,g$ with colors $1, 2, 3, 4$.

This coloring happened to make $a\,b\,c\,g$ colorful, so one pass succeeded (Figure 25.6). With $k = 4$ the per-path success chance is $4!/4^4 = 24/256 \approx 0.094$, so about 11 colorings suffice on average to hit a given path; had this one failed we would simply re-roll.

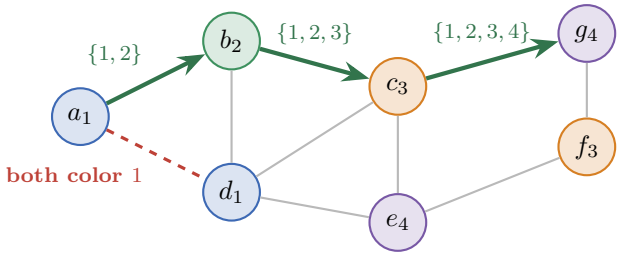


Figure 25.6: Color coding on G for $k = 4$, with vertex subscripts indicating random colors in $\{1, 2, 3, 4\}$. The subset DP builds $a_1 \to b_2 \to c_3 \to g_4$ (solid green arrows), growing the color set from $\{1\}$ to $\{1, 2, 3, 4\}$. Meanwhile the edge a_1–d_1 (dashed red) cannot extend a colorful path because both endpoints share color 1.

Application

Color coding is a standard tool for *subgraph counting* in biological networks. To estimate how often a small motif (a k-vertex pattern, like a feed-forward loop) occurs in a protein-interaction or gene-regulatory graph, color the host graph and count colorful occurrences of the motif by exactly this subset DP; an unbiased estimator divides by the colorful probability. It also underlies FPT algorithms for k-Path, Tree Subgraph Isomorphism and Graph Motif, and its algebraic descendant — replacing colors by random field elements and detecting a multilinear monomial via a determinant over a polynomial ring (Koutis–Williams) — gives the $O(2^k)$ longest-path algorithm that beats naive color coding.

25.7 When nothing works: the W-hierarchy

Some parameterized problems resist all four techniques. Clique (is there a clique of size k?) and Dominating Set (is there a dominating set of size k?) have no known $f(k) \cdot \text{poly}(n)$ algorithm, and the best anyone has is the XP brute force $O(n^k)$. We would like a hardness theory explaining why — a parameterized analogue of NP-completeness.

Intuition

NP-completeness rests on polynomial-time reductions and a complete problem (SAT). Parameterized hardness rests on *FPT reductions* — reductions that run in FPT time and map parameter k to a new parameter $k' \le g(k)$ bounded by a function of k only — and a hierarchy of increasingly hard problems indexed by the "depth" of the circuits needed to verify a solution. That hierarchy is the W-hierarchy:

$$\text{FPT} \subseteq \text{W[1]} \subseteq \text{W[2]} \subseteq \cdots \subseteq \text{W[P]} \subseteq \text{XP}.$$

Each containment is conjectured strict. W[1] is the parameterized world's NP: "W[1]-hard" is the verdict "almost certainly not FPT."

Definition 25.5 (FPT reduction)**.** An *FPT reduction* from (A, κ_A) to (B, κ_B) maps an instance (x, k) to an instance (x', k') such that: (i) $(x, k) \in A \iff (x', k') \in B$; (ii) the map runs in time $f(k) \cdot |x|^{O(1)}$; (iii) $k' \le g(k)$ for some computable g. If A FPT-reduces to B and $B \in \text{FPT}$, then $A \in \text{FPT}$.

Why it works

Why must k' stay bounded by $g(k)$? Because if a reduction were allowed to inflate the parameter to, say, $k' = n$, then an $f(k')$ poly algorithm for B would cost $f(n)$ poly on A — not FPT at all. The parameter is the resource being tracked, so an FPT reduction cannot hide dependence on n inside k'. This requirement is what makes the theory coherent, and it plays the same role as the kernel definition's "$k' \le k$."

The hierarchy is defined via weighted circuit satisfiability. Weighted Circuit-SAT: given a Boolean circuit C and integer k, is there an input of *weight* exactly k (exactly k ones) that satisfies C? Parameterizing by k and restricting the *weft* (the number of large-fan-in gates on any input-to-output path) to t defines W[t].

Focus

Canonical complete problems.

- Independent Set and Clique are W[1]-complete (under FPT reductions). They are the "parameterized SAT" of the field: a problem is "probably not FPT" precisely when it is W[1]-hard.
- Dominating Set is W[2]-complete — one level harder, reflecting the extra alternation in "every vertex is dominated."
- Vertex Cover, FVS, Longest Path are in FPT (this chapter), the easy end of the spectrum.

Why it works

Why Clique is believed not FPT. Suppose Clique parameterized by k were FPT, solvable in $f(k)n^c$. The *Exponential Time Hypothesis* (ETH) of Impagliazzo and Paturi posits that 3-SAT on n variables cannot be solved in $2^{o(n)}$ time. A classical chain of reductions shows that even an $n^{o(k)}$ algorithm for Clique would yield a $2^{o(n)}$ algorithm for 3-SAT, contradicting

ETH. So under ETH, Clique has no $n^{o(k)}$ algorithm, let alone an FPT one; the brute-force $O(n^k)$ is essentially optimal. W[1]-hardness packages this kind of evidence into a reusable certificate: prove your problem W[1]-hard and you have shown it is, modulo widely-believed conjectures, not FPT.

Common trap

A frequent confusion: "W[1]-hard" does not mean "not in XP." Clique *is* in XP (the n^k brute force). W[1]-hardness is a statement about the *exponent's location*: k cannot be removed from the exponent of n. Likewise, "FPT" does not mean "practical" — a $2^{2^k}n$ algorithm is FPT and hopeless for $k = 6$. And the parameter is a modeling choice: Dominating Set is W[2]-hard parameterized by solution size but FPT parameterized by treewidth. Always state the parameter before claiming hardness or tractability.

Connection

The treewidth chapter (Chapter 24) and this one are closely related. Courcelle's theorem says every problem expressible in monadic second-order logic is FPT parameterized by treewidth — a single meta-theorem subsuming vertex cover, FVS, dominating set, 3-coloring and hundreds more, all FPT *when the graph is tree-like*. The bounded-search-tree and color-coding results here parameterize by *solution size* instead, a different parameter that works on arbitrary graphs. Parameterizing by structure (treewidth, planarity, genus) or by solution size makes many NP-hard graph problems tractable. The W-hierarchy marks the boundary where neither parameter is known to help.

Summary

Technique	Idea	Flagship result
Bounded search tree	branch, shrink k each branch	VC in $O(2^k(n+m))$
Kernelization	poly-time shrink to size $g(k)$	VC kernel $\leq 2k$ vertices
Iterative compression	build up, compress $k{+}1 \to k$	FVS in $O(c^k\, kn)$
Color coding	random colors make paths simple	k-Path in $2^{O(k)}m$
W-hierarchy	FPT reductions, weft levels	Clique W[1]-hard

All four techniques follow the same pattern: find the parameter that captures the hardness, then arrange for the exponential dependence to involve only that parameter. When this works, you get FPT. When it provably cannot (W[1]-hardness, under ETH), you have learned that the chosen parameter is the wrong one and should look for another.

Connection

What This Made Possible: Parameterized complexity gave practical algorithms for dozens of NP-hard problems. By confining exponential growth to an auxiliary parameter k, algorithms like bounded search trees, kernelization (Buss and Crown rules), iterative compression, and color coding run in polynomial time for small k on massive graphs ($n \geq 10^6$). Kernelization

provides a mathematically rigorous foundation for data reduction and preprocessing pipelines.
Limitations:

- *Small parameters only:* FPT algorithms are practical only when k is small (typically $k \leq 20$–40). When $k = \Omega(n)$ or $k = \Omega(\log n)$ with superexponential $f(k)$, the running time degenerates into brute force.
- *W-hardness barriers:* Many critical problems (k-Clique, k-Independent Set, k-Dominating Set) are $W[1]$- or $W[2]$-hard, provably precluding $f(k) \cdot n^{O(1)}$ algorithms under standard complexity conjectures (like the Exponential Time Hypothesis).
- *Non-polynomial kernels:* Many FPT problems admit kernels of exponential size $2^{O(k)}$ but provably have no polynomial-size kernel unless NP $\subseteq$ coNP/poly.

Where We Go Next: When parameterization by solution size fails on general graphs, geometric and topological constraints can help. If a graph can be drawn in the plane without crossing edges, topological properties strongly restrict its structure. In **Chapter 26 (Planar Graphs and Planarity Testing)**, we explore Euler's formula $V-E+F = 2$ and Kuratowski's theorem, which lay the groundwork for bidimensionality theory and subexponential $2^{O(\sqrt{k})} \cdot n$ algorithms on planar graphs.

25.8 Exercises

Exercise 25.1 (Refined VC branching)**.** Prove that the branching rule "pick a vertex v of degree ≥ 3; branch (a) $v \in S$, recurse on $G - v$ with $k-1$, or (b) $v \notin S$, put all of $N(v)$ in S, recurse on $G-v-N(v)$ with $k-\deg(v)$" has running time $O(1.466^k \text{ poly})$ once degree-≤ 2 vertices are eliminated by reduction. Solve the recurrence $T(k) = T(k-1) + T(k-3)$ and identify the characteristic root.

Exercise 25.2 (Buss kernel by hand)**.** Run Buss's reduction on the graph of Figure 25.1 with $k = 4$. Which vertices (if any) are forced by Rule 2? What is the resulting kernel, and does the "$> k^2$ edges" rule fire? Conclude whether a size-4 cover exists.

Exercise 25.3 (Crown correctness, sharpened)**.** Prove that if (I, H) is a crown then *every* minimum vertex cover that takes some vertex of I can be transformed into one of equal size taking all of H and none of I. Where exactly does the saturating matching get used in your argument?

Exercise 25.4 (No crown, no shrink)**.** Construct a graph on which Buss's rules and the crown reduction both fail to reduce the instance below $2k$ vertices for some k, yet the LP/Nemhauser–Trotter kernel still shrinks it. (Hint: a graph whose LP optimum is all-$\frac{1}{2}$.)

Exercise 25.5 (Iterative compression for OCT)**.** Adapt the iterative-compression framework to Odd Cycle Transversal: delete $\leq k$ vertices to make the graph bipartite. Describe the disjoint-compression subproblem and explain why it reduces to a sequence of min-cut / max-flow computations. (This is the original Reed–Smith–Vetta result.)

Exercise 25.6 (Why colorful equals simple)**.** Prove carefully that in the color-coding DP, every path counted by $D[v, C]$ with $|C| = s$ visits s distinct vertices. Then exhibit a (non-colorful) coloring under which a real simple k-path is missed, and compute the probability of that miss for $k = 5$.

Exercise 25.7 (Derandomizing color coding)**.** State the definition of an (n, k)-perfect hash family and show that, given such a family of size $N(n, k)$, the randomized color-coding algorithm becomes

deterministic at a multiplicative cost of $N(n, k)$. Look up the best known bound on $N(n, k)$ and write down the resulting deterministic running time for k-Path.

Exercise 25.8 (FPT reductions preserve FPT)**.** Prove Definition 25.5's closure claim: if A FPT-reduces to B and $B \in$ FPT, then $A \in$ FPT. Where in the argument do you use $k' \leq g(k)$? Show by example that dropping this condition breaks the implication.

Exercise 25.9 (Clique and independent set)**.** Give an FPT reduction from Independent Set (parameter k) to Clique (parameter k) and back, using the complement graph. Why does this show the two are equivalent under FPT reductions, and why does it *not* by itself prove either is W[1]-hard?

Exercise 25.10 (Parameter matters)**.** Dominating Set is W[2]-hard parameterized by solution size. Show it is FPT parameterized by treewidth by sketching a DP over a tree decomposition with states recording, for each bag vertex, whether it is in the set / dominated / not-yet-dominated. What is the dependence on treewidth w? (Expect $O(3^w \cdot \text{poly})$ or so.)

Exercise 25.11 (Break the budget bug)**.** Take the buggy branching discussed after Algorithm 104 ("$u \notin S$, recurse with k unchanged"). Construct a small graph and budget k on which this buggy algorithm either over-explores without terminating in the claimed bound or returns a wrong answer, and explain precisely which invariant it violates.

Part V

Structural Graph Theory

Chapter 26

Planar Graphs and Planarity Testing

In the preceding chapter, we saw how parameterizing by solution size or treewidth makes NP-hard problems tractable. But graph structure can also arise from topology. Some graphs can be drawn on a flat sheet of paper without any two edges crossing, and some cannot. This geometric constraint has many algorithmic consequences. Planar graphs are sparse ($m \leq 3n - 6$), possess small balanced separators, can be colored with four colors, and admit fast algorithms for problems that are otherwise intractable on general graphs. Euler's 1750 polyhedral identity $V - E + F = 2$ is the basis of this theory. Here we trace how Euler's formula bounds edge counts, why K_5 and $K_{3,3}$ are the unique obstructions to planarity, how linear-time testing works, and how planarity leads directly to the separator theorems developed in the next chapter.

History

The mathematics of planar graphs originated in polyhedral geometry. In 1750, Leonhard Euler wrote to Christian Goldbach stating his polyhedral formula: every convex polyhedron satisfies $V - E + F = 2$. Euler confirmed it on pyramids, prisms, and Platonic solids, though he lacked topological definitions. In 1813, Augustin-Louis Cauchy published a proof by removing one face and flattening the remaining surface into the plane. But Cauchy tacitly assumed faces are topological discs and that removing them preserves connectivity. In 1812 and subsequent decades, Simon Lhuilier found counterexamples: polyhedra with holes or tunnels (like a hollow picture frame) satisfy $V - E + F = 0$, establishing that the alternating sum depends on the *genus* (the number of donut-like holes in the surface). This historical struggle inspired Imre Lakatos's classic philosophical study *Proofs and Refutations*.

Map coloring opened a second major line of inquiry. In 1852, Francis Guthrie conjectured that any planar map could be colored with four colors. Alfred Kempe claimed a proof in 1879 using *Kempe chains* (paths alternating between two colors to swap choices locally). In 1890, Percy Heawood uncovered a flaw in Kempe's chain-swapping argument, though he salvaged the technique to prove the Five Color Theorem. The Four Color Theorem finally yielded in 1976 to Kenneth Appel and Wolfgang Haken, who verified 1,936 reducible configurations using 1,200 hours of supercomputer calculations. Because no human could verify the computer outputs by hand, the proof caused controversy over the nature of mathematical certainty. Robertson, Sanders, Seymour, and Thomas simplified the discharging argument in 1997, and Georges Gonthier formalized a computer-verified proof in Coq in 2005, though a conceptual, hand-checkable proof remains an open challenge.

Algorithmic planarity matured in parallel. In 1930, Kazimierz Kuratowski characterized planar graphs by forbidden subgraphs (no subdivisions of K_5 or $K_{3,3}$, where a subdivision replaces edges with simple paths), and Klaus Wagner proved the minor-contraction analogue in 1937 – foreshadowing Robertson and Seymour's Graph Minor Theorem (1983–2004) showing that every minor-closed family is characterized by a finite list of forbidden minors. Testing planarity by searching for Kuratowski subgraphs directly was inefficient. In 1974, John Hopcroft and Robert Tarjan introduced the first $O(n)$ linear-time planarity test using depth-first search to embed paths incrementally, an achievement recognized by their 1986 ACM Turing Award. John Boyer and Wendy Myrvold (2004) later introduced an edge-addition method that is easier to implement and extracts explicit Kuratowski witnesses when planarity fails.

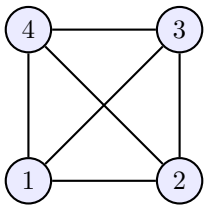
26.1 Drawings, embeddings, and faces

Throughout, $G = (V, E)$ is a finite undirected graph with $n = |V|$ and $m = |E|$. We forbid loops and, for most of the chapter, multiple edges – they never help a graph become planar and only clutter the counting.

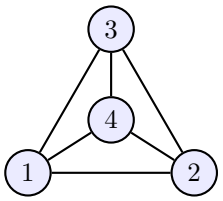
Definition 26.1 (Planar drawing)**.** A *drawing* of G in the plane $\mathbb{R}^2$ maps each vertex to a distinct point and each edge $\{u, v\}$ to a simple (non-self-intersecting) curve joining the images of u and v, such that the curve meets no other vertex and any two curves meet only at shared endpoints. A graph is *planar* if it admits such a drawing; a fixed drawing is a *plane graph*, or a *planar embedding*.

Intuition

"Planar" is a property of the abstract graph: *can it be drawn flatly?* A "plane graph" is one specific flat drawing. The same planar graph has many essentially different embeddings, just as a cube can be unfolded many ways. The crossings you see in a careless drawing are an artifact of *that* drawing, not of the graph. K_4 looks like it must cross if you draw it as a square with both diagonals, but slide one vertex inside the triangle of the other three and the crossing vanishes.



drawn with a crossing



same K_4, planar

A plane graph cuts the plane into connected regions called *faces*. One of them is unbounded – the *outer* (or infinite) face. The boundary of a face is a closed walk along edges; an edge can appear on the boundary of one face twice (a bridge), or border two distinct faces.

Definition 26.2 (Face and its degree)**.** Let H be a plane graph. A *face* is a connected component of $\mathbb{R}^2 \setminus H$ (the plane minus all the drawn vertices and edges). The *degree* $\deg(f)$ of a face f is the number of edges on its boundary walk, counting an edge twice if both of its sides touch f.

Focus

Three quantities govern every connected plane graph in this chapter: n vertices, m edges, and f faces. Euler's formula ties them directly via $n - m + f = 2$, so fixing any two determines the third.

The two-sided counting of edges around faces is the fundamental accounting identity. Each edge borders exactly two face-sides (it has a left and a right as you walk along it), so summing face degrees double-counts edges:

$$\sum_{f} \deg(f) \;=\; 2m. \tag{$\star$}$$

This is the planar cousin of the handshake lemma $\sum_v \deg(v) = 2m$, and we will use it often.

26.2 Euler's formula

Theorem 26.1 (Euler's formula)**.** *For any connected plane graph with n vertices, m edges, and f faces,*

$$n - m + f = 2.$$

Intuition

Grow the graph one edge at a time and watch the bookkeeping. Start from a single vertex: $n = 1$, $m = 0$, $f = 1$ (just the outer face), and $1 - 0 + 1 = 2$. Now add edges one by one. Every new edge either reaches a brand-new vertex (a *tree* edge) or closes a cycle (a *non-tree* edge). A tree edge bumps n and m each by one: $n - m + f$ is unchanged. A cycle-closing edge splits one face into two, bumping m and f each by one: again $n - m + f$ is unchanged. The quantity starts at 2 and never moves.

Proof. Induct on m. *Base.* If $m = 0$ then G is a single vertex (connected), so $n = 1$, $f = 1$, and $1 - 0 + 1 = 2$.

Step. Take a connected plane graph with $m \geq 1$ edges and pick any edge e.

Case 1: e lies on a cycle. Then e borders two distinct faces (its two sides are in different regions, since the rest of the cycle separates them by the Jordan curve theorem). Deleting e merges those two faces into one and keeps the graph connected. The new graph has n vertices, $m - 1$ edges, $f - 1$ faces; by induction $n - (m - 1) + (f - 1) = 2$, hence $n - m + f = 2$.

Case 2: e lies on no cycle (a bridge). Then e has the same face on both sides, so deleting it does not change f, but it disconnects G into two components with $n_1 + n_2 = n$, $m_1 + m_2 = m - 1$ edges, and f_1, f_2 faces. Each component is a connected plane graph, but they *share* the outer face, so $f_1 + f_2 = f + 1$ (the outer face is counted once in G but once in each piece). Applying induction to each piece, $n_i - m_i + f_i = 2$, and summing,

$$n - (m - 1) + (f + 1) = 4 \implies n - m + f = 2. \qquad \square$$

Why it works

The induction is on *a* drawing, but the conclusion $f = 2 - n + m$ shows f does not depend on which planar drawing you chose – the number of faces is an invariant of the abstract planar graph (for connected graphs). So although faces seem to be a property of the drawing, their number is determined by n and m.

26.2.1 The sparsity bounds

The most frequently used consequences of Euler's formula are the sparsity bounds.

Theorem 26.2 (Edge bound for simple planar graphs)**.** *If G is simple, planar, and connected with $n \geq 3$, then*

$$m \leq 3n - 6.$$

If in addition G is bipartite (girth ≥ 4), then $m \leq 2n - 4$.

Proof. Fix a plane embedding with f faces. Because G is simple and $n \geq 3$, every face is bounded by at least three edges, so $\deg(f) \geq 3$ for every face f. Summing and using $(\star)$,

$$2m \;=\; \sum_{f} \deg(f) \;\geq\; 3f \quad\Longrightarrow\quad f \leq \tfrac{2}{3}m.$$

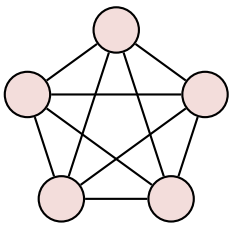
Plug into Euler's formula $f = 2 - n + m$:

$$2 - n + m \leq \tfrac{2}{3}m \implies m \leq 3n - 6.$$

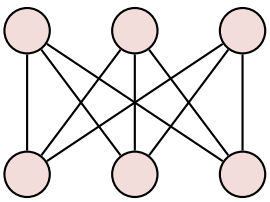
If G is bipartite it has no odd cycles, so no triangular faces; every face has $\deg(f) \geq 4$, giving $2m \geq 4f$, $f \leq m/2$, and $2 - n + m \leq m/2$, i.e. $m \leq 2n - 4$. □

Corollary 26.1 (Two non-planar graphs)**.** *K_5 and $K_{3,3}$ are not planar.*

Proof. K_5 has $n = 5$, $m = 10$. The bound $3n - 6 = 9 < 10$ is violated, so K_5 cannot be planar. $K_{3,3}$ is bipartite with $n = 6$, $m = 9$. The bipartite bound $2n - 4 = 8 < 9$ is violated, so $K_{3,3}$ is not planar. □



K_5: $m = 10 > 9$



$K_{3,3}$: $m = 9 > 8$

Common trap

The bound $m \le 3n - 6$ is necessary, not sufficient. A graph can satisfy it and still be non-planar – glue a disjoint K_5 to a long path and the average gets diluted below the threshold while the K_5 keeps it non-planar. Sparsity rules graphs *out*; it never rules them *in*. Also, the bound is for *simple* graphs with $n \ge 3$. For $n = 1, 2$ it fails ($n = 2$, $m = 1$ gives $3n - 6 = 0$). And it assumes connectivity for the clean Euler count; for a graph with c components the formula is $n - m + f = 1 + c$.

Corollary 26.2 (A low-degree vertex always exists)**.** *Every simple planar graph has a vertex of degree at most* 5*.*

Proof. If every vertex had degree ≥ 6, then $2m = \sum_v \deg(v) \ge 6n$, so $m \ge 3n > 3n-6$, contradicting Theorem 26.2. (For $n < 3$ the claim is trivial.) □

This corollary underlies both the five-color theorem and linear-time planar algorithms: planar graphs always have a cheap vertex to peel off, so greedy/recursive strategies terminate fast.

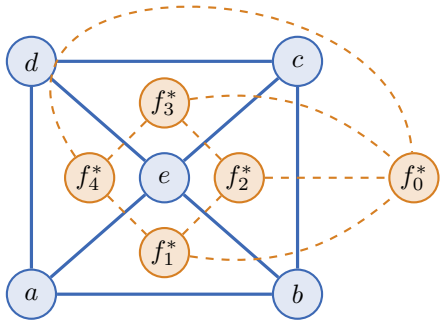
Trace it yourself

Trace the edge-growing proof on the cube graph Q_3 ($n = 8$, $m = 12$). Euler predicts $f = 2-8+12 = 6$ faces – the six faces of an actual cube, plus you reinterpret one as the outer face when you flatten it. Now check the bound: $3n - 6 = 18 \ge 12$, consistent. Check ($\star$): each of the 6 faces is a 4-cycle, so $\sum_f \deg(f) = 6 \cdot 4 = 24 = 2m$. All the counts agree.

26.3 Duality

Treating each face of a planar embedding as a vertex gives the dual graph.

Definition 26.3 (Dual graph)**.** Let H be a connected plane graph. Its *dual* H^* has one vertex per face of H, and for each edge e of H a dual edge e^* joining the two faces that e borders (a loop if e borders the same face on both sides).



Solid blue: Primal plane graph H $(n = 5, m = 8, f = 5)$
Dashed orange: Orthogonal dual graph H^* $(n^* = 5, m^* = 8, f^* = 5)$

Figure 26.1: A planar graph H and its orthogonal dual H^*. Every face of H (four interior triangular faces f_1, f_2, f_3, f_4 and one exterior face f_0) contains one dual vertex f^*. Every primal edge e is crossed orthogonally by exactly one dual edge e^* connecting the two faces that e borders. Vertices and faces swap roles while the edge count is preserved: $n^* = f$, $f^* = n$, and $m^* = m$.

Each primal edge crosses exactly its dual edge, so H and H^* have the same number of edges. Vertices and faces swap roles: $n^* = f$, $f^* = n$, $m^* = m$. Euler's formula is symmetric under this swap – duality is an involution on connected plane graphs, $(H^*)^* = H$.

Connection

Duality connects to the flow material in Chapters 11–12. In a planar graph, a *cut* separating s from t corresponds to a *cycle* in the dual separating the dual faces, and a shortest such cycle is a minimum s–t cut. This is how planar min-cut reduces to planar shortest paths (Reif, Hassin), and why planar max-flow runs much faster than the general bound. Cuts $\leftrightarrow$ cycles, flows $\leftrightarrow$ potentials: this is a geometric form of the primal/dual duality from linear programming.

Remark 26.1. The dual depends on the specific plane embedding rather than the abstract graph alone: a 3-connected planar graph has a unique embedding (Whitney's theorem) and hence a unique dual, but graphs with low connectivity can have several inequivalent embeddings and several duals.

26.4 The forbidden minors: Kuratowski and Wagner

Corollary 26.1 says K_5 and $K_{3,3}$ are non-planar. The deep theorem is the converse: *these are the only obstructions*, provided you look for them in the right way. Two ways turn out to be equivalent.

Definition 26.4 (Subdivision)**.** A *subdivision* of G replaces some edges by internally disjoint paths (inserting degree-2 vertices along them). Subdividing never affects planarity.

Definition 26.5 (Minor)**.** A graph H is a *minor* of G if H can be obtained from a subgraph of G by contracting edges. Equivalently, H is a minor iff G contains $|V(H)|$ disjoint connected "branch sets," one per vertex of H, with an edge of G between branch sets whenever H has the corresponding edge.

Theorem 26.3 (Kuratowski 1930; Wagner 1937)**.** *A graph G is planar if and only if it contains* ***no subdivision*** *of K_5 or $K_{3,3}$ (Kuratowski). Equivalently, G is planar iff it has* ***no minor*** *isomorphic to K_5 or $K_{3,3}$ (Wagner).*

Figure 26.2 contrasts the two operations: subdividing an edge inserts degree-2 stepping-stone vertices along a path, whereas a minor allows each vertex of H to expand into a connected branch set that contracts back to a single vertex.

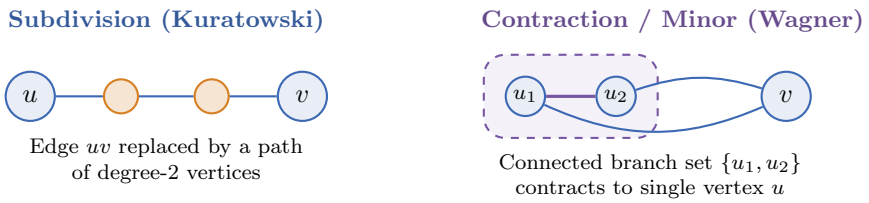


Figure 26.2: The two ways a forbidden obstruction can hide inside G: as a subdivision (edges stretched into internally disjoint paths) or as a minor (connected branch sets contracted along internal edges).

Intuition

Both statements say the same thing from two angles. "Subdivision" looks for K_5 or $K_{3,3}$ literally drawn out, with the 5 or 6 branch vertices and edge-paths between them threaded through the graph. "Minor" is more permissive about how the pieces sit: each branch vertex can be a whole connected blob you contract to a point. For K_5 and $K_{3,3}$ specifically (all branch degrees ≤ 3 in $K_{3,3}$; degree 4 in K_5), the two notions coincide, which is why Kuratowski and Wagner are interchangeable here. They diverge for larger forbidden graphs.

Why it works

Why necessity is easy, sufficiency is hard. The "only if" direction is just Corollary 26.1 plus the observation that planarity is closed under taking subgraphs, subdivisions, and contractions: if G were planar it could not hide a non-planar K_5 or $K_{3,3}$ inside. The "if" direction – that avoiding these two patterns *guarantees* a flat drawing – is the content. The cleanest modern proof goes through 3-connected graphs: by a theorem of Tutte, every 3-connected planar graph has a convex straight-line drawing, and one shows any 3-connected graph with no $K_5/K_{3,3}$ minor is planar by induction on contractions, reducing to K_4 as the base. Lower connectivity is handled by gluing embeddings along cut vertices and separating pairs.

We will not reproduce the full 3-connected case analysis, but the inductive skeleton is worth seeing because the planarity-testing algorithm mirrors it.

Lemma 26.1 (Reduction to 3-connected pieces)**.** *G is planar iff every one of its blocks (maximal 2-connected subgraphs, see Chapter 10) is planar, and a 2-connected graph is planar iff each of the 3-connected components in its SPQR/triconnected decomposition is planar.*

The point is that planarity is a *local* property of the connectivity skeleton: you may test it block by block, and within a block, 3-connected chunk by chunk. Embeddings glue back together along cut vertices (free choice of which side) and separation pairs (a binary flip, recorded in the SPQR tree).

26.5 Planarity testing

Could we test planarity by hunting for a K_5 or $K_{3,3}$ subdivision directly? In principle yes, but the obvious search is exponential. Three linear-time approaches exist; all are intricate, so we build intuition for one (the DFS / path-addition family) and state the guarantees.

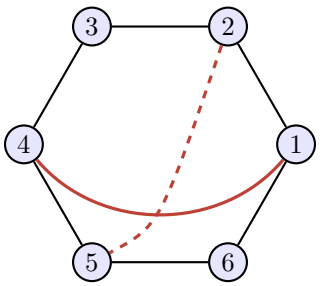
Focus

Goal. Decide in $O(n)$ time whether G is planar (and, if so, output an embedding – a cyclic edge order at each vertex). Note $m \le 3n-6$ for any candidate, so we may assume $m = O(n)$: if $m > 3n-6$ we instantly answer "non-planar" by Theorem 26.2. From here on $m = O(n)$, so $O(n)$ and $O(m)$ are the same.

26.5.1 Intuition: add one path at a time

The path-addition / DFS view (Auslander–Parter, refined by Hopcroft–Tarjan to linear time) goes like this. Run a DFS, producing a spanning tree plus back edges. Every back edge closes a cycle with tree edges. Build the embedding incrementally: maintain a partial planar embedding of what you have placed so far, with a known *outer boundary*, and try to route each remaining path either inside or outside the current drawing.

The obstruction is a conflict: two pieces that each must be drawn on the same side of an already-placed cycle, but cannot coexist there. Whether two pieces conflict is captured by a 2-*coloring* (inside/outside) of an auxiliary "interlacement" graph: pieces that interlace around the cycle must lie on opposite sides. The graph is planar iff every such interlacement graph is bipartite; any odd cycle in an interlacement graph yields a K_5 or $K_{3,3}$ subdivision.



chords $\{1,4\}$ and $\{2,5\}$ *interlace*: their endpoints alternate, so opposite sides are forced

Intuition

Picture the spanning-tree DFS cycle as a circle. Each remaining piece attaches to the circle at two or more points. Two pieces *interlace* if their attachment points alternate around the circle (like $1,2,4,5$ where one piece uses $\{1,4\}$ and the other $\{2,5\}$). Interlacing pieces cannot both go inside (they would cross) and cannot both go outside, so one goes in and one goes out. That is a 2-coloring constraint. A consistent assignment is a planar embedding; an inconsistency (odd cycle of forced-opposite pieces) is non-planarity.

26.5.2 A clean recursive scheme

Here is the Auslander–Parter–Goldstein recursion that makes the idea precise. It is cubic as written; Hopcroft–Tarjan's bookkeeping squeezes it to linear, but the logic is the same.

Algorithm 109: PLANARTEST(G) – path-addition skeleton

Input: a biconnected graph G (test each biconnected block separately).
Output: PLANAR or NONPLANAR.

if $m > 3n - 6$ **then**
  return NONPLANAR; `// sparsity gate, Thm 26.2`
if *G is a single cycle or has ≤ 2 edges* **then**
  return PLANAR
Find a cycle C in G;
Remove C; let the connected components of the rest, with their attachments to C, be the *bridges* (pieces) $B_1, \dots, B_k$;
foreach *piece B_i* **do**
  recursively embed $B_i \cup$ (its attachment arcs on C);
  if *that sub-instance is non-planar* **then return** NONPLANAR;
Build the *interlacement graph* I: a vertex per piece, an edge between pieces whose attachment points interlace around C;
if *I is bipartite* **then**
  return PLANAR; `// 2-color = inside/outside assignment`
else
  return NONPLANAR; `// odd cycle ⇒ K5 or K3,3`

Why it works

Why bipartiteness of the interlacement graph is exactly planarity (around one cycle). Each piece must be drawn entirely inside C or entirely outside (a piece is connected and attaches only on C, so it cannot straddle). Interlacing pieces on the same side cross; non-interlacing pieces on the same side nest peacefully. So a valid drawing is precisely a 2-coloring of I with "same color $\Rightarrow$ non-interlacing," i.e. a proper 2-coloring – which exists iff I is bipartite. An odd cycle in I is a minimal set of pieces no side-assignment can satisfy, and unwinding it produces a $K_{3,3}$ or K_5 subdivision, recovering Kuratowski. The recursion handles pieces that are themselves complex by re-running on each piece against its own representative cycle.

Cost summary

The skeleton recursion is $O(n^3)$ (or $O(n^2)$ with care): $O(n)$ levels, each building an interlacement graph of size $O(n)$ and bipartiteness-checking it. The **Hopcroft–Tarjan** algorithm (1974) reorganizes the same path-additions with a clever DFS ordering and a stack of conflicts, achieving $O(n)$. The independent **vertex-addition** method of Lempel–Even–Cederbaum, sped up by Booth–Lueker's PQ trees, also runs in $O(n)$. The modern favorite, **Boyer–Myrvold** (2004), is a left/right path-addition DFS that is linear, simple to implement, and emits a Kuratowski subgraph as a certificate of non-planarity.

Common trap

Two classic implementation traps. (1) *Forgetting the sparsity gate.* Without the $m \le 3n - 6$ check, a dense input makes the interlacement graphs huge and the running time blows up before you even start; always discard $m > 3n - 6$ first. (2) *Testing biconnectivity wrong.* Planarity testing is run per biconnected block; if you feed a graph with a cut vertex straight in, the "single cycle" decomposition can miss pieces. Decompose into blocks (Chapter 10) and test each. A graph is planar iff every block is.

Trace it yourself

Run the skeleton on K_4. Pick the cycle $C = 1\,2\,3\,1$. The remaining vertex 4 with its three edges to $1, 2, 3$ is one piece B_1, attaching at $\{1, 2, 3\}$. There is only one piece, so the interlacement graph I has a single vertex and no edges, which is immediately bipartite. Answer: PLANAR. Now run it on K_5. Take $C = 1\,2\,3\,4\,5\,1$. The chords are $\{1, 3\}, \{2, 4\}, \{3, 5\}, \{4, 1\}, \{5, 2\}$, each its own piece (a single edge). Pieces $\{1, 3\}$ and $\{2, 4\}$ interlace; $\{2, 4\}$ and $\{3, 5\}$ interlace; and so on around a 5-cycle of forced-opposite pieces – an *odd* cycle in I. Not bipartite. Answer: NONPLANAR, with the odd 5-cycle reconstructing the K_5 obstruction.

26.6 Coloring planar graphs

The most famous fact about planar graphs is that four colors suffice.

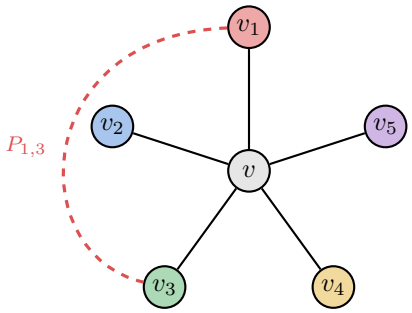
Theorem 26.4 (Four Color Theorem; Appel–Haken 1976)**.** *Every planar graph is properly* 4*-colorable: its vertices can be colored with four colors so that adjacent vertices differ.*

The four-color theorem is hard to prove. The five-color theorem has a one-page proof that uses Corollary 26.2 and *Kempe chains*, and it is algorithmic.

Theorem 26.5 (Five Color Theorem; Heawood 1890)**.** *Every simple planar graph is properly* 5*-colorable, and a coloring can be found in* $O(n)$ *time.*

Intuition

Peel a low-degree vertex, color the rest recursively, then put it back. By Corollary 26.2 there is always a vertex v of degree ≤ 5. If $\deg(v) \le 4$, its ≤ 4 neighbors use at most 4 colors and a fifth is free for v. The only hard case is $\deg(v) = 5$ with all five colors used by the neighbors. Here we *recolor* part of the graph to free a color, using a connectivity argument that planarity makes available – the Kempe chain.



$\deg(v) = 5$ uses all colors; $P_{1,3}$ encloses v_2 and blocks any 2–4 chain

Proof. Induct on n. For $n \leq 5$ color arbitrarily. For $n > 5$, pick v with $\deg(v) \leq 5$ (Corollary 26.2). Delete v, 5-color $G - v$ by induction, and try to extend.

If $\deg(v) \leq 4$, or if $\deg(v) = 5$ but the neighbors use ≤ 4 colors, a free color exists; assign it to v. The remaining case: $\deg(v) = 5$ and its neighbors $v_1, \ldots, v_5$ (in clockwise embedding order) carry all five distinct colors $1, 2, 3, 4, 5$.

Kempe chain. Look at the subgraph $H_{1,3}$ induced by colors 1 and 3. Consider the connected component of $H_{1,3}$ containing v_1 (color 1).

Case A: v_3 (color 3) is not *in that component.* Swap colors $1 \leftrightarrow 3$ throughout that component. This keeps a proper coloring (the component's boundary touches only colors $\neq 1, 3$), and now v_1 has color 3. Color 1 is freed at v; assign it.

Case B: v_3 is in that component. Then there is a path $P_{1,3}$ from v_1 to v_3 using only colors 1 and 3. Together with v this path forms a closed curve. By the Jordan curve theorem, it separates v_2 from v_4: one is strictly inside the loop, the other outside. Now look at $H_{2,4}$, the subgraph on colors 2 and 4, and the component containing v_2. Any 2–4 path from v_2 to v_4 would have to cross $P_{1,3}$ – but crossing is impossible in a plane graph and the colors $\{2, 4\}$ and $\{1, 3\}$ are disjoint, so no such path exists. Hence v_2 and v_4 lie in different components of $H_{2,4}$. Swap $2 \leftrightarrow 4$ in v_2's component; color 2 is freed at v. Assign it. □

Why it works

The Jordan-curve step is where planarity is used. In a non-planar graph the 1–3 chain need not separate v_2 from v_4, and the argument collapses – which is why 5 (and 4) colors are special to the plane. Kempe's 1879 "proof" of *four* colors tried to run two chain swaps at once; Heawood's counterexample showed the two swaps can interfere. For *five* colors one swap always suffices, and there is no interference.

Algorithm 110: FiveColor(G) – recursive Kempe-chain coloring

Input: simple planar graph G.
Output: a proper coloring $c : V \to \{1, 2, 3, 4, 5\}$.

```
if |V| ≤ 5 then
    color all vertices distinctly; return
Find v with deg(v) ≤ 5;                          // exists by Cor. 26.2
c ← FiveColor(G − v);
if some color in {1, ..., 5} is missing among v's neighbors then
    assign v that color;
else
    v_1, ..., v_5 ← neighbors in embedding order, with distinct colors;
    run the Kempe swap on chain {c(v_1), c(v_3)} (else {c(v_2), c(v_4)}) to free a color;
    assign v the freed color;
return c;
```

Cost summary

Each recursion deletes one vertex, so there are n levels. The work per level is finding a low-degree vertex (amortized $O(1)$ with a degree-bucket queue) and at most one Kempe swap, which is a BFS/DFS over a 2-colored subgraph in $O(n)$. Naively $O(n^2)$; with care about which chains get re-traversed, $O(n)$ total. Four-coloring in polynomial time is also known (a quadratic algorithm follows from the RSST proof), but it is vastly more involved.

Common trap

A tempting wrong simplification: "every planar graph is 4-colorable, so just be greedy and back off." Greedy coloring in a bad vertex order can need up to $\Delta + 1$ colors even on a planar graph – it is not guaranteed to find a 4- or 5-coloring on its own. The five-color guarantee needs the *specific* peel-lowest-degree order *and* the Kempe-chain repair. Drop either and the bound breaks.

Application

Coloring planar graphs is the math behind frequency assignment on a map of transmitters (adjacent regions need different frequencies), register interference for certain structured programs, and – historically – coloring actual maps, which is the dual statement: countries become vertices, shared borders become edges, and a map is always planar.

26.7 Planar separators

One more structural consequence of planarity is that planar graphs admit small balanced cuts: you can always split them into balanced pieces by deleting only $O(\sqrt{n})$ vertices. (The dedicated separator chapter, Chapter 27, develops this in full; here we state it and read off consequences.)

Theorem 26.6 (Lipton–Tarjan Planar Separator Theorem, 1979)**.** *Let G be an n-vertex planar graph. There is a partition $V = A \cup S \cup B$ such that*

- *no edge joins A and B (so S* separates *them),*
- $|A|, |B| \le \frac{2}{3}n$ *(the parts are balanced),*
- $|S| \le 2\sqrt{2}\,\sqrt{n} = O(\sqrt{n})$ *(the separator is small),*

and A, S, B can be found in $O(n)$ time.

Intuition

Run a BFS from any root. The graph stratifies into levels $L_0, L_1, \ldots$ Two facts combine to give the theorem. First, since $\sum_i |L_i| = n$, *some* level is small – there is a level with $O(\sqrt{n})$ vertices not too far from the median, and deleting one level disconnects the levels above from those below. Second, the levels between two such small levels form a planar graph of bounded "radius," and a bounded-radius planar graph has a small separator because you can contract the inside to a point and use the fact that a planar graph with a spanning tree of depth d has a fundamental-cycle separator of size $O(d)$. Balancing the two effects gives $O(\sqrt{n})$. The square root is the perimeter-vs-area scaling of the plane: a region of n cells has boundary $\sim \sqrt{n}$.

Why it works

Why $\sqrt{n}$ and not less. The $\sqrt{n} \times \sqrt{n}$ grid is planar, and any balanced separator of it must contain $\Omega(\sqrt{n})$ vertices – a horizontal or vertical cut already costs $\sqrt{n}$, and one can show no cleverer cut does asymptotically better. So Lipton–Tarjan is tight up to the constant. Compare: a tree (planar, but a special case) has a *single-vertex* centroid separator; general graphs (e.g. expanders, Chapter 34) need $\Omega(n)$. Planar graphs lie in between.

Corollary 26.3 (Divide and conquer for free)**.** *Recursively applying the separator yields a separator hierarchy. This drives:*

- *exact and approximate algorithms running in $2^{O(\sqrt{n})}$ for problems that are $2^{O(n)}$ in general (planar TSP, planar independent set, planar Hamiltonicity – the "square-root phenomenon");*
- *nested dissection: solving an n-variable planar linear system (e.g. a planar Laplacian) in $O(n^{3/2})$ time and $O(n \log n)$ fill, by eliminating along the separator tree;*
- *polynomial-time approximation schemes via Baker's layering technique.*

Connection

The separator theorem connects planar graphs to Chapters 31–35 and 41. Nested dissection (Corollary 26.3) is the fastest classical direct solver for planar Laplacian systems and feeds straight into the electrical-flow material of Chapter 35. And the $O(\sqrt{n})$ bound is mirrored spectrally: planar graphs have a small second Laplacian eigenvalue, so Cheeger's inequality (Chapter 32) predicts a sparse cut; the separator and the small spectral gap both reflect planarity.

Application

Planar separators are used in route planners (road networks are nearly planar; contraction hierarchies and customizable route planning exploit small separators), VLSI layout (Lipton

and Tarjan's original motivation was circuit area lower bounds), and mesh partitioning for finite-element solvers, where a good separator is a good way to split work across processors.

26.8 Summary of planar graph properties

Quantity / problem	General graphs	Planar graphs
Edges	up to $\binom{n}{2}$	$\leq 3n-6$
Min vertex degree	up to $n-1$	≤ 5
Chromatic number	up to n	≤ 4
Balanced separator	up to $\Omega(n)$	$O(\sqrt{n})$
Planarity test	—	$O(n)$
Max-flow / min-cut	$O(nm)$-ish	near-linear (via duality)
Shortest paths (neg. ok)	$O(nm)$	near-linear
Independent set, TSP	$2^{O(n)}$	$2^{O(\sqrt{n})}$

Every row of the table traces back to Euler's formula. Sparsity ($m \leq 3n-6$) comes straight from it; the low-degree vertex and the colorings come from sparsity; planarity testing exploits the bounded structure; duality turns cuts into cycles; and separators exploit the plane's area-perimeter scaling.

Connection

What This Made Possible: Planarity reduces the cost of many classical graph problems. Sparsity ($m \leq 3n-6$) means algorithms that are $O(m)$ run in strict $O(n)$ linear time. Planar duality turns minimum cuts into shortest cycles in the dual graph, allowing near-linear min-cut and max-flow computations. In addition, planar graphs guarantee vertices of degree at most 5, enabling fast recursive reductions and the Four-Color Theorem.

Limitations:

- *Fragility under minor perturbations:* Planarity is topologically brittle. Adding even two or three crossing edges (such as a $K_{3,3}$ subdivision) destroys planarity, invalidating Euler's formula and face duality.
- *Non-uniqueness of embeddings:* A 2-connected planar graph can have exponentially many distinct planar embeddings (differing in face structures), meaning dual-graph algorithms must fix an embedding first. Only 3-connected planar graphs enjoy unique embeddings (Whitney 1932).
- *Still NP-hard:* Many hard problems remain NP-hard on planar graphs: 3-Coloring, Maximum Independent Set, Hamiltonian Cycle, and Vertex Cover remain strictly NP-complete.

Where We Go Next: While planar problems remain NP-hard, their planar geometry enables divide-and-conquer strategies that run in subexponential time $2^{O(\sqrt{n})}$. In **Chapter 27 (Separators)**, we develop the Lipton–Tarjan Planar Separator Theorem, showing that every planar graph can be split into balanced pieces by deleting only $O(\sqrt{n})$ vertices. And in **Chapter 28 (Graph Coloring)**, we study coloring in general, including Kempe chains and the Four-Color Theorem.

26.9 Exercises

Exercise 26.1 (Euler bookkeeping)**.** A connected planar graph has every face of degree exactly 5 and every vertex of degree exactly 3. Use ($\star$), the handshake lemma, and Euler's formula to solve for n, m, f. (You should recognize the dodecahedron.)

Exercise 26.2 (The component correction)**.** Prove that for a planar graph with c connected components, $n - m + f = 1 + c$. Where in the proof of Theorem 26.1 does connectivity get used, and how does adding a component change the face count?

Exercise 26.3 (Girth bound)**.** Generalize Theorem 26.2: if a simple planar graph has girth (shortest cycle length) $g \geq 3$, prove $m \leq \frac{g}{g-2}(n-2)$. Recover the $3n-6$ and $2n-4$ bounds as special cases. Use it to give a one-line proof that the Petersen graph ($n = 10$, $m = 15$, girth 5) is non-planar.

Exercise 26.4 (Both obstructions are needed)**.** Exhibit a graph that contains a K_5 subdivision but no $K_{3,3}$ subdivision, and another with a $K_{3,3}$ subdivision but no K_5 subdivision. Conclude that neither forbidden graph alone characterizes planarity.

Exercise 26.5 (Maximal planar = triangulation)**.** A planar graph is *maximal planar* if adding any edge destroys planarity. Prove that a maximal planar graph on $n \geq 3$ vertices has exactly $m = 3n-6$ edges and every face (including the outer one) is a triangle. (Hint: a non-triangular face has a missing chord you could add.)

Exercise 26.6 (Self-dual graphs)**.** Show that for a self-dual connected plane graph ($H \cong H^*$) we have $m = 2n-2$. Find a small self-dual example. (Hint: $n = f$ for self-duality; combine with Euler.)

Exercise 26.7 (Implement five-coloring)**.** Implement Algorithm 110. Maintain a bucket queue keyed by current degree so that finding a deg ≤ 5 vertex is $O(1)$ amortized, and implement the Kempe swap as a BFS over the relevant two-color class. Test it on a random maximal planar graph (build one by repeatedly inserting a vertex into a random triangle) and verify the coloring is proper.

Exercise 26.8 (Break a naive tester)**.** Consider the "tester" that returns PLANAR iff $m \leq 3n-6$. Construct an infinite family of non-planar graphs that this tester wrongly accepts, with m as small as $2n$. Why does the sparsity gate in Algorithm 109 not suffer from this – what does the rest of the algorithm add?

Exercise 26.9 (Kempe's mistake)**.** Read Heawood's counterexample to Kempe's four-color argument. Describe, on a concrete small configuration, two Kempe chains whose simultaneous swap interferes (one swap undoes the gain of the other). Explain why a single swap in the five-color proof never has this problem.

Exercise 26.10 (Separator on the grid)**.** For the $\sqrt{n} \times \sqrt{n}$ grid, prove that any partition $V = A \cup S \cup B$ with $|A|, |B| \geq n/3$ and no A–B edge has $|S| = \Omega(\sqrt{n})$. (Hint: count rows/columns that must contain a separator vertex.) Conclude that Theorem 26.6 is tight up to the constant.

Exercise 26.11 (Outerplanar warm-up)**.** A graph is *outerplanar* if it has a planar drawing with every vertex on the outer face. Prove that an outerplanar simple graph satisfies $m \leq 2n-3$, and that the forbidden minors for outerplanarity are K_4 and $K_{2,3}$. (Reduce to the planar case by adding a new vertex adjacent to all others and applying Kuratowski.)

Exercise 26.12 (Dual of a cut)**.** Let G be a connected plane graph and H^* its dual. Prove that a set of edges forms a minimal edge cut (bond) in G if and only if the corresponding dual edges form a simple cycle in H^*. Explain how this lets you compute a minimum s–t cut in a planar graph by a shortest-path computation in the dual.

Chapter 27

Separators and Divide-and-Conquer on Graphs

In the preceding chapter, we saw how planarity restricts edge density to $m \leq 3n - 6$. Planarity also guarantees that any planar graph can be split into balanced pieces along a small boundary. Divide-and-conquer relies on cutting an input into balanced pieces with a small interface. On lines or trees, this is straightforward; on general graphs, the boundary between two halves can be as large as $\Omega(n^2)$ edges or $\Omega(n)$ vertices. Planar graphs, by contrast, behave like two-dimensional geometric sheets: they always admit balanced separators of size $O(\sqrt{n})$.

A *separator* is a set of vertices whose removal disconnects the graph into components. When small balanced separators exist, many hard problems become easier: planar graphs admit sub-exponential $2^{O(\sqrt{n})}$ exact algorithms, fast shortest-path queries via recursive divisions, and efficient sparse matrix factorization. Here we develop the planar separator theorem of Lipton and Tarjan, prove its linear-time construction, and show how both planarity and treewidth provide small separators for divide-and-conquer.

History

The planar separator theorem was born in numerical linear algebra. In the early 1970s, engineers using finite element methods to simulate stresses on physical structures (airplane wings, bridges, turbine blades) were constrained by computational limits. Solving the resulting sparse linear systems $Ax = b$ using Gaussian elimination created severe "fill-in" — zero matrix entries turning non-zero during row reductions. On a planar 2D mesh with n nodes, naive Gaussian elimination created $\Omega(n^2)$ fill-in and required $\Omega(n^3)$ arithmetic operations, exceeding computer memory.

In 1973, J. Alan George invented *nested dissection*: find a small balanced vertex separator S, eliminate the disconnected halves A and B independently, and eliminate S last. George proved that if every n-node subgraph has an $O(\sqrt{n})$ balanced separator, fill-in drops to $O(n \log n)$ and computation to $O(n^{3/2})$. However, George only knew how to construct such separators for regular square grids. Did general, irregular planar meshes always possess $O(\sqrt{n})$ balanced separators?

Early attempts to prove this used single BFS level cuts, but in graphs with dense central hubs, individual levels can contain $\Theta(n)$ vertices. Peter Ungar (1951) had proved an existential separator bound using geometric area-perimeter inequalities, but his methods offered no efficient

algorithm. In 1979, Richard Lipton and Robert Tarjan solved the problem completely: they designed an $O(n)$ linear-time algorithm guaranteeing a balanced separator of size $\leq 2\sqrt{2}\sqrt{n}$. Their insight was a two-phase strategy: concentric BFS levels bracket the graph between two thin levels bounding a middle band of size $\leq \sqrt{n}$, and a fundamental cycle in the condensed dual tree cuts through the middle band to balance the remaining weights.
Hristo Djidjev (1982) tightened the separator constant toward $\sqrt{6} \approx 2.45$, and Greg Frederickson (1987) introduced *r-divisions* (partitioning a planar graph into pieces of size r with boundary $O(\sqrt{r})$), which led to near-linear shortest path algorithms. Noga Alon, Paul Seymour, and Robin Thomas (1990) extended the $O(\sqrt{n})$ bound to any graph class excluding a fixed minor H. Beyond matrix factorization, separators led to linear-time planar graph isomorphism (John Hopcroft and Robert Tarjan, 1972) and polynomial isomorphism on graphs of bounded genus and treewidth. In contrast, expander graphs (graphs where every cut has boundary proportional to its volume) provably require $\Omega(n)$ balanced separators. Finding an optimal planar separator in near-linear time remains an active area of research.

27.1 What a separator is, and why balance matters

Definition 27.1 (Vertex separator)**.** Let $G = (V, E)$ be an undirected graph and let $\alpha \in [\frac{1}{2}, 1)$ be a balance constant. A set $S \subseteq V$ is an *α-balanced (vertex) separator* if the vertices $V \setminus S$ can be partitioned into two sets A, B with

$$A \cup B = V \setminus S, \qquad A \cap B = \emptyset, \qquad \text{no edge joins } A \text{ to } B,$$

and $|A| \leq \alpha\,|V|$, $|B| \leq \alpha\,|V|$. The *size* of the separator is $|S|$.

The two conditions pull in opposite directions. "No edge joins A to B" means S genuinely *separates*: every path from A to B must pass through a deleted vertex. "$|A|, |B| \leq \alpha n$" means the separation is *balanced*: neither leftover piece is almost the whole graph. We want both, with $|S|$ as small as possible.

Intuition

Think of the graph drawn on the floor and imagine pouring a thin wall of removed vertices across it. The wall (S) cuts the floor into two regions (A and B). A *good* wall is short (few vertices) and splits the floor area fairly evenly. A long, winding wall is useless even when balanced because boundary work is too high; a one-brick wall that walls off a single corner vertex is tiny, yet useless because the split is completely unbalanced ($|A| = 1$, $|B| = n - 2$). The separator theorem promises a wall that is both short and fair.

Why insist on balance? Because divide-and-conquer's running time is governed by how fast the subproblems shrink. If we recurse on A and B and the larger of them can be as big as $n - 1$, the recursion depth is n and we have gained nothing. Balance ($|A|, |B| \leq \frac{2}{3}n$, say) guarantees the larger child is a constant factor smaller, so the recursion has depth $O(\log n)$. The separator size $|S|$ controls how much "boundary work" each level costs. The two together — small size and good balance — are what make a separator theorem an *algorithmic* theorem rather than a curiosity.

Focus

The three numbers that describe a separator family, and what each one buys:

- **Size** $f(n)$: how many vertices you delete. Smaller is better; $O(\sqrt{n})$ for planar, $O(1)$ (a single vertex) for trees, $\mathrm{tw}(G)+1$ via treewidth.
- **Balance** α: the larger leftover piece has at most αn vertices. We will mostly hit $\alpha = \frac{2}{3}$; the exact constant rarely matters, only that $\alpha < 1$.
- **Recursiveness**: ideally the class is closed under taking subgraphs, so the children are again separable by the same theorem. Planar graphs and bounded-treewidth graphs both have this; it is what lets the recursion repeat all the way down.

27.1.1 Trees: a warm-up

We start with the easiest separator theorem in graph theory. Recall the *centroid* of a tree (Chapter 20): a vertex whose removal leaves every component with at most $n/2$ vertices.

Theorem 27.1 (Tree separator)**.** *Every tree on $n \geq 1$ vertices has a single vertex c whose deletion splits it into components each of size at most $\lfloor n/2 \rfloor$. Thus a tree has a $\frac{1}{2}$-balanced separator of size 1.*

Proof. Root the tree anywhere and let $sz(v)$ be its subtree size. Start at the root and repeatedly walk to the child whose subtree is largest, stopping at the first vertex c all of whose children u satisfy $sz(u) \leq n/2$. At c, every "downward" component (a child subtree) has size $\leq n/2$ by the stopping rule, and the single "upward" component has size $n - sz(c)$. We claim $n - sz(c) \leq n/2$. We moved to c from its parent because $sz(c) > n/2$ (otherwise we would have stopped earlier, at the parent), so $n - sz(c) < n/2$. Hence all components are $\leq n/2$. □

This is the basis of *centroid decomposition*: split at the centroid, recurse on each component, get a recursion tree of depth $O(\log n)$. Trees are the friendliest case because a single vertex always suffices. The question this chapter answers is how far that idea reaches when the graph has cycles.

27.1.2 General graphs have no small separators

You cannot hope for $O(\sqrt{n})$ separators in every graph. The obstruction is *expansion* — the property studied in the chapter on expanders (Chapter 34). For now, the simplest witness:

Proposition 27.1 (Cliques and expanders resist separation)**.** *In the complete graph K_n, any α-balanced separator with $\alpha < 1$ has size $|S| \geq (1-\alpha)n = \Omega(n)$: since every two vertices are adjacent, A and B cannot both be nonempty, so one side is empty and $|S| \geq n - \max(|A|, |B|) \geq n - \alpha n$.*

Proof. In K_n every pair of vertices is joined by an edge. If A and B are both nonempty, pick $a \in A, b \in B$; the edge ab joins A to B, contradicting the no-edge condition. So one side, say B, is empty, forcing $A \cup S = V$ and $|S| = n - |A| \geq n - \alpha n = (1-\alpha)n$. □

A clique is dense, but density is not the real issue — *expansion* is. There exist sparse graphs (constant degree, only $O(n)$ edges) called *expanders* in which every balanced separator still has size

$\Omega(n)$. Removing a sublinear set of vertices from an expander always leaves one giant connected lump. So the planar separator theorem does not extend to all sparse graphs; it depends on planarity.

27.2 A grid by hand: the canonical $\sqrt{n}$ separator

The clearest example is the $k \times k$ grid graph $G_{k,k}$, with $n = k^2$ vertices arranged in a square lattice and edges between orthogonal neighbours. Here the $\sqrt{n}$ separator is directly visible.

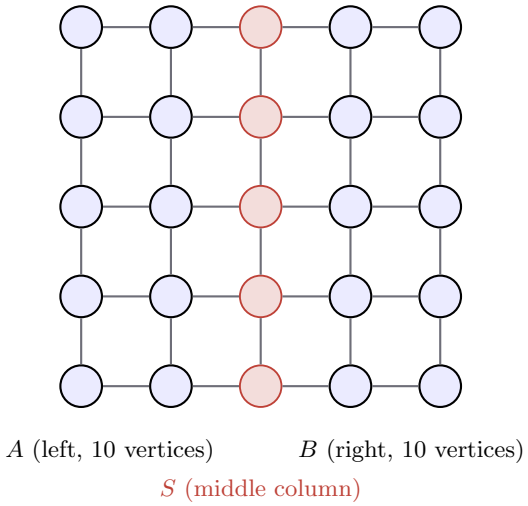


Figure 27.1: The 5×5 grid, $n = 25$. Deleting the middle column S (red, $5 = \sqrt{n}$ vertices) leaves A (two left columns, 10 vertices) and B (two right columns, 10 vertices) with no edge between them. A perfectly balanced separator of size $\sqrt{n}$.

Trace it yourself

Verify Figure 27.1 by hand. The middle column has $k = 5$ vertices. Every horizontal edge that would cross from a left column to a right column must pass through the middle column — there is no edge skipping over it, because grid edges only join immediate neighbours. So removing the middle column severs all left–right connections. The leftover sides have $2k = 10$ vertices each, both $\leq \frac{2}{5}n$. Now ask: could you do it with fewer than k deleted vertices? Intuitively no — any wall separating left from right must block all k rows, and the rows are vertex-disjoint paths, so by Menger's theorem (Chapter 12) you need at least k vertices. The grid's separator number is exactly $\sqrt{n}$.

Why it works

Why is the grid the "hardest easy" case? Because it is the densest planar mesh: it has $\Theta(n)$ edges and uniform local connectivity, so it looks locally like an expander. Yet globally it has a $\sqrt{n}$ separator. The planar separator theorem says this is not a quirk of the grid's symmetry — *every* planar graph, however irregular, can be cut as cleanly as a grid (up to constants). The grid is the extremal example that fixes the $\sqrt{n}$ scaling: you cannot beat $\sqrt{n}$ in general, because the grid already needs that many.

The grid also explains the recursion. A $\sqrt{n}$ separator splits n vertices into two halves of $\leq \frac{2}{3}n$. Recurse. The boundary work at the top level is $O(\sqrt{n})$, at the next level $O(\sqrt{2n/3})$ on each of two pieces, and so on. The geometric decay of $\sqrt{\cdot}$ down a depth-$O(\log n)$ tree is exactly what makes separator-based recurrences sum to small totals — we make this precise in Section 27.4.

27.3 The Planar Separator Theorem

Theorem 27.2 (Lipton–Tarjan, 1979)**.** *Let $G = (V, E)$ be a planar graph on n vertices. Then there is a partition of V into three sets A, S, B such that*

1. *no edge joins a vertex of A to a vertex of B;*
2. *$|A| \leq \frac{2}{3}n$ and $|B| \leq \frac{2}{3}n$;*
3. *$|S| \leq 2\sqrt{2}\,\sqrt{n} \approx 2.83\sqrt{n}$.*

The sets A, S, and B can be computed in $O(n)$ time.

The proof is constructive and is based on breadth-first search. We build it in three steps: (i) reduce to triangulated graphs, (ii) find a good *level* of a BFS to use as the separator backbone, (iii) if no single level is small enough, cut the graph between two levels and patch with a spanning-tree path. We give the argument in enough detail to implement.

Intuition

Run a BFS from any root. This partitions the vertices into *levels* $L_0, L_1, L_2, \ldots$ by distance from the root. Each level is a "moat" — removing all vertices of one level disconnects the inside (smaller-numbered levels) from the outside (larger-numbered levels), because every edge of a graph joins vertices in the same or adjacent levels. So *any single BFS level is automatically a separator*; the only question is whether some level is both small ($O(\sqrt{n})$) and well-placed (balanced). Most of the proof is bookkeeping to guarantee such a level exists, using planarity (via Euler's formula) to bound how the levels can grow.

27.3.1 Step 0: reductions

We may assume G is connected (else separate the largest component using the rest as ballast; standard packing makes the pieces balanced). We may also assume each vertex has a nonnegative *weight* summing to 1, and ask only that each of A, B carries weight $\leq \frac{2}{3}$; the unit-weight case is what we stated. Finally we may assume G is *triangulated* (a *maximal* planar graph, every face a triangle): adding edges only makes separation harder, and a triangulation can be computed in linear time. From here on G is a connected triangulated planar graph with $m \leq 3n - 6$ edges (Euler).

27.3.2 Step 1: BFS levels are separators

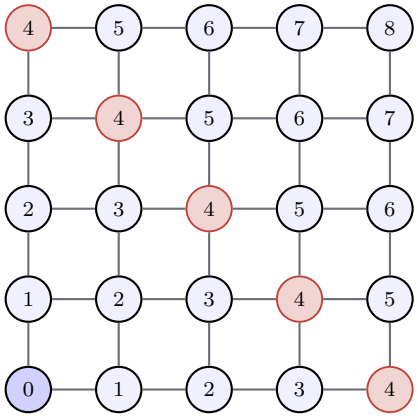
Run BFS from an arbitrary root r. Let $L_i = \{v : \mathrm{dist}(r, v) = i\}$ be the i-th level and let ℓ be the last nonempty level. Write $L_{<i} = \bigcup_{j<i} L_j$ and $L_{>i} = \bigcup_{j>i} L_j$.

Lemma 27.1 (Levels separate)**.** *For every i, the level L_i is a separator: there is no edge between $L_{<i}$ and $L_{>i}$.*

Proof. An edge $\{u, v\}$ of G joins vertices whose BFS distances differ by at most 1 (BFS distances are a metric and adjacent vertices are within 1). So if $u \in L_{<i}$, i.e. $\mathrm{dist}(r, u) \le i - 1$, then $\mathrm{dist}(r, v) \le i$, hence $v \in L_{<i} \cup L_i$, never in $L_{>i}$. □

So if some level L_i has $|L_i| \le c\sqrt{n}$ *and* both $|L_{<i}|$ and $|L_{>i}|$ are $\le \frac{2}{3}n$, we are done with $S = L_i$. Pick the *middle level* L_{i^*} at the median vertex: let i^* be the smallest i with $|L_{\le i}| \ge n/2$. Then both sides have $\le n/2 \le \frac{2}{3}n$ vertices, so balance is automatic. The only thing that can go wrong is that this particular level is *fat* — it could have many more than $\sqrt{n}$ vertices.

The picture to keep in mind is a set of concentric moats around the root. Every edge stays inside one moat or hops to the next; none leaps across. Figure 27.2 draws the levels of the 5×5 grid rooted at a corner, with the median level shaded.



root at lower-left; label = $\mathrm{dist}(r, \cdot) = r + c$

Figure 27.2: BFS levels of the 5×5 grid rooted at the lower-left corner. Each vertex is labelled with its level $\mathrm{dist}(r, \cdot)$; on this grid the level is just the anti-diagonal index $r + c$, running from 0 to 8. Level sizes are $1, 2, 3, 4, \mathbf{5}, 4, 3, 2, 1$. The median level L_4 (red, the long anti-diagonal) holds $5 = \sqrt{n}$ vertices and splits the 25 vertices into 10 below and 10 above — a thin median, so Step 1 alone already separates the grid.

Trace it yourself

Read the level sizes straight off Figure 27.2 and watch the proof run. The cumulative counts are $1, 3, 6, 10, 15, 19, 22, 24, 25$. The first cumulative count to reach $n/2 = 12.5$ is at level 4 (cumulative 15), so $i^* = 4$: the median level is the long anti-diagonal $L_4 = \{(4, 0), (3, 1), (2, 2), (1, 3), (0, 4)\}$, exactly $\sqrt{n}$ vertices. Below it sit levels $0, 1, 2, 3$ with $1 + 2 + 3 + 4 = 10$ vertices; above it sit levels $5, 6, 7, 8$ with $4 + 3 + 2 + 1 = 10$. No edge of the grid joins a ≤ 3 vertex to a ≥ 5 vertex — check any edge and its two endpoints differ in level by exactly 1. So $S = L_4$ is a balanced separator of size $\sqrt{n}$, and the grid never needs the harder Steps 2–3. The fat-median case only bites when the level-size profile spikes in the middle instead of forming this gentle tent; Exercise 27.4 builds such a spike on purpose.

27.3.3 Step 2: a thin level must exist near the middle

Here is where $\sqrt{n}$ enters. Suppose the median level L_{i^*} is fat. We look for a *thin* level above and a thin level below to sandwich it.

Lemma 27.2 (A thin level nearby)**.** *There exist levels $a \le i^* \le b$ with $|L_a| \le \sqrt{n}$ and $|L_b| \le \sqrt{n}$, with a within $\sqrt{n}$ levels above i^* and b within $\sqrt{n}$ levels below, such that cutting at L_a and L_b isolates the "middle band" $M = \bigcup_{a<j<b} L_j$ from the rest. In particular $|L_a| + |L_b| \le 2\sqrt{n}$ and $b - a \le 2\sqrt{n}$.*

Proof. Walk upward from i^* toward the root. If every one of the levels $L_{i^*}, L_{i^*-1}, \ldots, L_{i^*-\sqrt{n}}$ had more than $\sqrt{n}$ vertices, those $\sqrt{n}+1$ disjoint levels would together contain more than $\sqrt{n} \cdot \sqrt{n} = n$ vertices, impossible. So within $\sqrt{n}$ steps upward there is a level L_a with $|L_a| \le \sqrt{n}$. The same averaging argument applied downward from i^* produces a thin level $L_b \le \sqrt{n}$ within $\sqrt{n}$ steps below. Hence $b - a \le 2\sqrt{n}$ and $|L_a| + |L_b| \le 2\sqrt{n}$. □

After Lemma 27.2 we have three regions: the *top* $T = L_{<a}$, the *middle band* $M = \bigcup_{a<j<b} L_j$, and the *bottom* $B' = L_{>b}$, separated by the two thin levels L_a and L_b (total $\le 2\sqrt{n}$). If the middle band M has $|M| \le \frac{2}{3}n$, set $S = L_a \cup L_b$, $A = M$, $B = T \cup B'$. The top T and bottom B' are not adjacent (they are separated by M and by the thin levels), so packing them together on one side is legal, and both sides are $\le \frac{2}{3}n$. Done, with $|S| \le 2\sqrt{n}$.

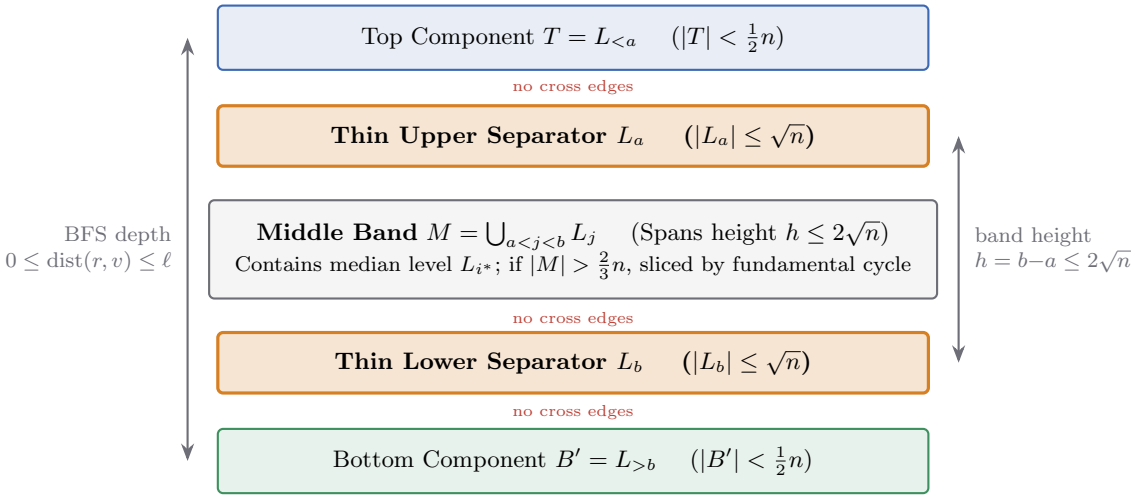


Figure 27.3: BFS levels in the Lipton–Tarjan separator theorem. BFS levels form concentric moats where every graph edge connects vertices within the same level or adjacent levels. An averaging argument guarantees two thin levels L_a and L_b of size at most $\sqrt{n}$ sandwiching the median i^*. If the middle band M has weight $\le \frac{2}{3}n$, the separator is simply $S = L_a \cup L_b$ of total size $\le 2\sqrt{n}$.

27.3.4 Step 3: a fat middle band, cut by a tree path

The remaining case is a middle band M with more than $\frac{2}{3}n$ vertices but spanning only "few" levels (at most $b - a \le 2\sqrt{n}$ of them). This is where planarity is needed. Contract everything above L_a to a single root and discard everything below L_b; the band M together with L_a, L_b is planar and triangulated, and its BFS depth from the contracted root is at most the number of band levels, $h \le 2\sqrt{n}$.

Lemma 27.3 (Fundamental-cycle separator)**.** *In a triangulated planar graph with a rooted spanning tree of height h, there is a non-tree edge whose induced* fundamental cycle *(the tree path between its endpoints plus the edge) is a separator with both sides carrying $\leq \frac{2}{3}$ of the weight. The cycle has length $\leq 2h + 1$.*

Idea. Each non-tree edge e closes a fundamental cycle C_e with the tree (Figure 27.4). In a planar embedding, C_e is a closed curve and, by the Jordan curve theorem, splits the plane — hence the faces, hence the vertices — into an inside and an outside. Walk over the faces: because the graph is triangulated, moving the dividing cycle across one triangular face changes the inside-weight by a bounded amount, so some non-tree edge yields a cycle with inside- and outside-weight both $\leq \frac{2}{3}n$. Its vertex set is the separator, of size $\leq 2h + 1$, since the cycle consists of two root-to-endpoint tree paths (each $\leq h$) plus the single non-tree edge. □

Apply Lemma 27.3 inside the band. The effective height is $h \leq 2\sqrt{n}$, so the cycle separator has $\leq 2(2\sqrt{n}) + 1 = O(\sqrt{n})$ vertices. Add the two thin levels L_a, L_b already counted; the grand total stays $O(\sqrt{n})$, and the original paper's careful accounting yields $|S| \leq 2\sqrt{2}\sqrt{n}$. Every step — BFS, level counting, choosing the thin levels, the fundamental-cycle search — is linear time, proving the $O(n)$ bound.

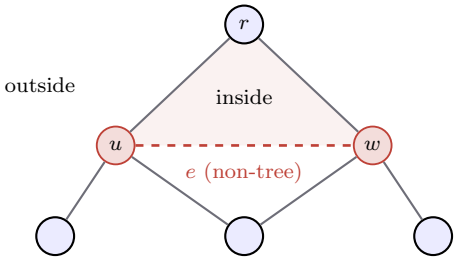


Figure 27.4: A fundamental-cycle separator. The non-tree edge $e = \{u, w\}$ (dashed red) together with the two tree paths from u and w up to their meeting point r forms a cycle $C_e = r\,u\,w\,r$ (red). In the plane drawing C_e bounds an inside and an outside (Jordan curve theorem); its vertices $\{r, u, w\}$ are the separator. Sweeping e over triangular faces changes the inside-weight by a bounded amount, so some choice of e balances the two sides to $\leq \frac{2}{3}n$ each. The cycle length is at most two tree-path lengths plus one, i.e. $\leq 2h + 1$.

Why it works

Where exactly was planarity used? Twice. First, in the edge bound $m \leq 3n - 6$ from Euler's formula, which forces BFS levels to grow "slowly" on average — in a general graph a single BFS level could be all of $V \setminus \{r\}$ (a star), with no thin neighbour. Second, and more deeply, in Lemma 27.3: a cycle in a *plane* drawing has a well-defined inside and outside (the Jordan curve theorem), which is what lets a single fundamental cycle act as a separator. Without a plane embedding, a cycle need not separate anything.

Common trap

A tempting misreading: "the smallest BFS level is a balanced separator." False. The smallest level might be L_ℓ, a single far-flung vertex — size 1, but wildly unbalanced (A = everything,

$B = \emptyset$). Balance must be enforced by choosing the level near the *median* of the vertex count, not the level of minimum size. The main difficulty in Steps 2–3 is reconciling "small" with "balanced" when the median level happens to be fat. Confusing the two is the classic first-attempt bug.

Algorithm 111: PLANARSEPARATOR(G) — Lipton–Tarjan outline

Input: connected planar graph $G = (V, E)$, $n = |V|$
Output: separator S, parts A, B with $|A|, |B| \le \frac{2}{3}n$, $|S| = O(\sqrt{n})$

Triangulate G (linear time); fix a planar embedding
Run BFS from an arbitrary root r; record levels $L_0, L_1, \ldots, L_\ell$
Let i^* be the least i with $|L_{\le i}| \ge n/2$ `// median level`
if $|L_{i^*}| \le 2\sqrt{n}$ **then**
  return $S \leftarrow L_{i^*}$, $A \leftarrow L_{<i^*}$, $B \leftarrow L_{>i^*}$ `// thin median: done`
Find $a \le i^*$ (nearest) with $|L_a| \le \sqrt{n}$
Find $b \ge i^*$ (nearest) with $|L_b| \le \sqrt{n}$ `// thin sandwich,` $b - a = O(\sqrt{n})$
$T \leftarrow L_{<a}$; $M \leftarrow L_{(a,b)}$; $B' \leftarrow L_{>b}$
if $|M| \le \frac{2}{3}n$ **then**
  return $S \leftarrow L_a \cup L_b$, $A \leftarrow M$, $B \leftarrow T \cup B'$
Within the band $M \cup L_a \cup L_b$ build a BFS tree and run FUNDAMENTALCYCLESEP (Lemma 27.3)
return cycle C unioned with $L_a \cup L_b$ as S; balanced face-partition gives A, B

27.4 Separator-driven recurrences

A separator theorem turns a problem on G into the same problem on two graphs each $\le \frac{2}{3}n$ vertices, plus boundary work proportional to $|S|$. The exact speedup depends on how the boundary work interacts with the recursion. Three recurrence patterns appear repeatedly.

Linear divide-and-conquer. If recombining costs $O(n)$ and the separator is $O(\sqrt{n})$,

$$T(n) \le T(n_1) + T(n_2) + O(n), \qquad n_1 + n_2 \le n,\ n_1, n_2 \le \tfrac{2}{3}n.$$

This solves to $T(n) = O(n \log n)$, exactly like mergesort: the work per level is $O(n)$ (summed across all pieces at that depth) and there are $O(\log n)$ levels. The $\sqrt{n}$ does not even show up — it is dominated by the $O(n)$ merge.

Boundary-dominated work. When the cost at a node is proportional to the *separator size* rather than the subproblem size — common when the separator vertices become "interface" variables — the recurrence is

$$T(n) \le T(n_1) + T(n_2) + O(\sqrt{n}).$$

The key inequality is concavity of the square root:

$$\sqrt{n_1} + \sqrt{n_2} \le \sqrt{2}\,\sqrt{n} \quad \text{when } n_1 + n_2 = n,$$

so each level multiplies the total boundary by at most $\sqrt{2}$, while the pieces shrink by a constant factor. Over $O(\log n)$ levels this gives $O(\sqrt{n}\log n)$ for the unbalanced split and $O(\sqrt{n})$ when the recursion is stopped early (as in r-divisions).

Why it works

The recurrence $T(n) = T(2n/3) + T(n/3) + O(\sqrt{n})$ is worth solving once by hand. Guess $T(n) \le c\sqrt{n}$. Then $T(n) \le c\sqrt{2n/3} + c\sqrt{n/3} + O(\sqrt{n}) = c\sqrt{n}(\sqrt{2/3} + \sqrt{1/3}) + O(\sqrt{n})$. Now $\sqrt{2/3} + \sqrt{1/3} = 0.816 + 0.577 = 1.394 > 1$, so the homogeneous part *grows*; the guess $c\sqrt{n}$ fails and the true solution carries a log factor: $T(n) = O(\sqrt{n}\log n)$. The lesson: $\sqrt{n}$ separators do not automatically give $O(\sqrt{n})$ total work — you usually pay one or two log factors, and you should always check the constant $\sqrt{\alpha} + \sqrt{1-\alpha}$ against 1.

Sub-exponential exact algorithms. The most important separator recurrence appears in *exponential-time* algorithms. Suppose solving a problem (graph colouring, independent set, Hamiltonicity) on G means trying all assignments to the separator S and recursing. With $|S| = O(\sqrt{n})$ and q choices per separator vertex, the recurrence is

$$T(n) \le q^{c\sqrt{n}} \cdot \big(T(n_1) + T(n_2)\big).$$

We solve it in Section 27.6 and find $T(n) = 2^{O(\sqrt{n})}$ — sub-exponential, a large speedup over the $2^{\Theta(n)}$ brute force. This is the structural reason planar NP-hard problems are so much easier than general ones.

Cost summary

Quick reference for separator recurrences (α-balanced, separator size $f(n)$):

Per-node cost	Recurrence	Solution
$O(n)$ merge	$T(n_1) + T(n_2) + O(n)$	$O(n \log n)$
$O(\sqrt{n})$ boundary	$T(\frac{2}{3}n) + T(\frac{1}{3}n) + O(\sqrt{n})$	$O(\sqrt{n}\log n)$
$2^{O(\sqrt{n})}$ enumerate	$2^{c\sqrt{n}}\,[T(n_1) + T(n_2)]$	$2^{O(\sqrt{n})}$
treewidth w DP	per bag q^w, $O(n)$ bags	$O(q^w\ \mathrm{poly}(w)\, n)$

The rule of thumb: a size-$f(n)$ separator buys you a $2^{O(f(n))}$ exact algorithm and an $O(f(n))$-factor in divide-and-conquer.

27.5 Application 1: planar shortest paths and nested dissection

The earliest and still most important use of planar separators is *nested dissection* — ordering the vertices so that recursive elimination touches as little of the graph as possible.

27.5.1 Nested dissection for sparse linear systems

To solve $\mathbf{L}x = b$ for a planar graph Laplacian $\mathbf{L}$ (Chapter 31), Gaussian elimination's cost is dominated by *fill-in*: zero entries that become nonzero as you eliminate. Nested dissection eliminates the

two halves A, B *before* the separator S. Because A and B share no edges, eliminating an A-vertex never creates fill between A and B — the separator S shields them. Recursing, the fill-in obeys a separator recurrence and one proves:

Theorem 27.3 (George 1973; Lipton–Rose–Tarjan 1979)**.** *A planar Laplacian system can be solved by nested dissection with $O(n \log n)$ fill-in and $O(n^{3/2})$ arithmetic operations, versus $O(n^3)$ for naive elimination on a dense matrix.*

Connection

This connects to the companion *Linear Algebra* volume and to Chapters 31 (Laplacians), 32 (spectra), and 41 (Laplacian solvers). Nested dissection is why solving a planar Laplacian system via direct elimination is far cheaper than solving a dense system: the geometry of the graph controls fill-in during Gaussian elimination. When you reach electrical networks (Chapter 35) and near-linear Laplacian solvers (Chapter 41), the $O(n^{3/2})$ nested-dissection method here serves as the direct-elimination ancestor of those iterative solvers.

27.5.2 Planar single-source shortest paths

For shortest paths, separators give the celebrated $O(n)$-time planar SSSP of Henzinger, Klein, Rao, and Subramanian, and more practically the *r-division* approach to all-pairs queries and distance oracles. The skeleton:

1. Compute an r-division: $O(n/r)$ pieces, each $\le r$ vertices, each with a *boundary* of $O(\sqrt{r})$ separator vertices shared with neighbours.
2. Inside each piece, precompute shortest paths between its boundary vertices — a *dense distance graph* on $O(\sqrt{r})$ nodes per piece.
3. Glue the dense distance graphs across the global boundary (total $O(n/\sqrt{r})$ boundary vertices) and run a shortest-path computation on this much smaller *condensed* graph.

Intuition

You are planning a road trip across a continent divided into provinces. Within each province you precompute the driving time between its border crossings only — you do not care about the interior towns once you are passing through. Then you plan the cross-country route purely on the network of border crossings. Because each province has only $O(\sqrt{r})$ crossings (its separator), the inter-province network is tiny compared to the full road map, and the global plan is cheap. The separator is precisely the set of border crossings; planarity guarantees each province has few of them.

Application

This is used in practice. Real route planners on road networks (which are nearly planar) use exactly this hierarchy: *customizable route planning* and *multi-level Dijkstra* build separator-based overlays so that a continental shortest-path query touches only the boundary networks, answering in microseconds. The separator is the algorithmic content; planarity (approximately, via low highway dimension) is why it is small.

27.5.3 A tiny worked trace

Take the 4×4 grid ($n = 16$), source s at the top-left corner. Use column 3 as the separator S (four vertices, splitting columns $1, 2$ from column 4).

Trace it yourself

Do it on paper. Label vertices (r, c), $r, c \in \{1, 2, 3, 4\}$, unit edge weights, source $(1, 1)$.

1. **Separator** $S = \{(1, 3), (2, 3), (3, 3), (4, 3)\}$ (column 3). Left piece A = columns $1, 2$ (8 vertices), right piece B = column 4 (4 vertices).
2. **Inside** A: run Dijkstra from s. Distances to the four separator vertices of column 3 equal their grid Manhattan distance from the corner: $\mathrm{dist}((1, 3)) = 2$, $\mathrm{dist}((2, 3)) = 3$, $\mathrm{dist}((3, 3)) = 4$, $\mathrm{dist}((4, 3)) = 5$.
3. **Across** S: the only way into B (column 4) is through column 3. So $\mathrm{dist}((r, 4)) = \min_{r'} \big(\mathrm{dist}((r', 3)) + |r - r'| + 1\big)$, taking the cheapest border crossing.
4. For $(1, 4)$: best is via $(1, 3)$: $2 + 0 + 1 = 3$. For $(4, 4)$: via $(4, 3)$: $5 + 0 + 1 = 6$, or via $(1, 3)$: $2 + 3 + 1 = 6$ — a tie, both Manhattan-correct.

You never needed interior-to-interior distances across the boundary: every A-to-B shortest path is captured by the four border values (Figure 27.5). This is the separator idea on a small scale.

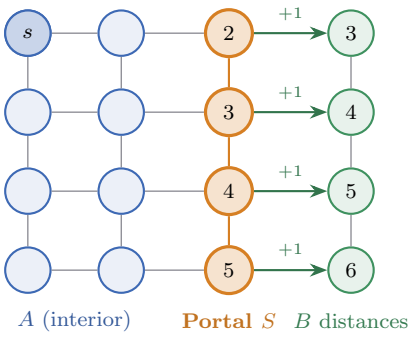


Figure 27.5: Boundary-only shortest paths on the 4×4 grid from $s = (1, 1)$. Once the separator S (column 3, orange) records its distances $\{2, 3, 4, 5\}$ from s, every distance into B (column 4, green) is determined strictly through S, with no further access to the interior of A.

27.6 Application 2: sub-exponential algorithms on planar graphs

Many NP-hard problems — *maximum independent set*, *vertex cover*, *3-colouring*, *Hamiltonian cycle* — are still NP-hard restricted to planar graphs. But on planar inputs they admit $2^{O(\sqrt{n})}$ algorithms, versus $2^{\Theta(n)}$ in general, because of the small separator.

Theorem 27.4 (Sub-exponential planar algorithms)**.** *Maximum independent set (and many similar problems) on an n-vertex planar graph can be solved exactly in time $2^{O(\sqrt{n})} \cdot \mathrm{poly}(n)$.*

Proof. Find a separator S, $|S| \le c\sqrt{n}$, splitting V into A, B with $|A|, |B| \le \frac{2}{3}n$. The optimal

independent set I^* interacts with S in some way: $I^* \cap S$ is one of at most $2^{|S|} = 2^{c\sqrt{n}}$ subsets of S. *Enumerate* all independent subsets $X \subseteq S$. For each such X:

- Forbid the neighbours of X in A and in B (they cannot be in I^*).
- Because no edge crosses A–B, the best extension into A and the best extension into B are *independent subproblems*: recurse on $G[A]$ and $G[B]$ with the forbidden vertices removed.
- Total for this X: $|X| + \mathrm{MIS}(A') + \mathrm{MIS}(B')$.

Take the best over all X. The recurrence is

$$T(n) \le 2^{c\sqrt{n}} \left(T(\tfrac{2}{3}n) + T(\tfrac{1}{3}n)\right) + \mathrm{poly}(n).$$

Let $T(n) = 2^{g(n)}$. The dominant term gives $g(n) \le c\sqrt{n} + g(\frac{2}{3}n)$ (the larger child dominates). Unrolling, $g(n) \le c\sqrt{n}\sum_{k\ge 0}(\sqrt{2/3})^k = c\sqrt{n} \cdot \frac{1}{1-\sqrt{2/3}} = O(\sqrt{n})$, because $\sqrt{2/3} < 1$ makes the geometric series converge. Hence $T(n) = 2^{O(\sqrt{n})}$. □

Why it works

The geometric series is the key step, and it is where the $O(\sqrt{n})$ planar separator bound is used. The separator size shrinks by $\sqrt{2/3} \approx 0.816$ each level (since $n \mapsto \frac{2}{3}n$ and we take square roots), and a geometric series with ratio < 1 sums to a constant times its first term $c\sqrt{n}$. If the separator were size $\Theta(n)$ (as in a general or expander graph), each level would contribute $\Theta(n)$ to the exponent and the sum would be $\Theta(n \log n)$ — worse than brute force. The sub-exponential bound is *exactly* the statement that the separator-size series converges, which is *exactly* the statement that separators are sublinear. Without sublinear separators, this approach gives no sub-exponential algorithm.

Algorithm 112: PlanarMIS(G) — sub-exponential max independent set

Input: planar graph $G = (V, E)$, $n = |V|$
Output: size of a maximum independent set

```
if n ≤ n_0 then
    return brute force
(A, S, B) ← PlanarSeparator(G)
best ← 0
for each independent subset X ⊆ S do
    A' ← A \ N(X);   B' ← B \ N(X)          // delete forbidden neighbours
    v ← |X| + PlanarMIS(G[A']) + PlanarMIS(G[B'])
    best ← max(best, v)
return best
```

Application

The same template — "enumerate the separator, recurse on the independent halves" — gives $2^{O(\sqrt{n})}$ algorithms for planar Hamiltonicity, planar TSP, planar Steiner tree, planar dominating set, and more. It is also the basis of *Baker's technique* for PTASs on planar graphs (peel off BFS layers mod k to get bounded-treewidth slabs, solve each by DP, glue) — a separator-

flavoured method we revisit in the approximation chapter (Chapter 38). The slogan "planar problems are sub-exponential" is a clear example of graph structure being used to beat brute force.

27.7 Application 3: graph isomorphism on planar graphs

Separators were among Lipton and Tarjan's original applications: a fast planar graph isomorphism test. General graph isomorphism has no known polynomial algorithm (Babai's 2015 quasi-polynomial bound is the frontier), but *planar* isomorphism is solvable in linear time — and the early algorithms went through separators with a divide-and-conquer that matches the two graphs separator-first.

Intuition

To decide whether two planar maps are the same, find a small separator in each. The separator splits each map into a few balanced pieces. Try to match a separator of G_1 to a separator of G_2 (only $O(\sqrt{n})$ vertices, so few alignments survive when you also exploit the embedding), then recursively match the pieces. Because the pieces are a constant factor smaller and the boundary is small, the matching recursion terminates fast. Modern planar isomorphism uses canonical embeddings and tree decompositions instead, but the separator viewpoint is the historical and conceptual root.

27.8 Separators from treewidth

The second main source of separators is *treewidth* (Chapter 24). Where planarity gives $O(\sqrt{n})$ separators via geometry, a tree decomposition gives separators of size $\mathrm{tw}(G)+1$ via structure — and these separators are *recursive by construction*.

Definition 27.2 (Recap: tree decomposition)**.** A *tree decomposition* of $G=(V,E)$ is a tree $\mathcal{T}$ whose nodes carry *bags* $B_t \subseteq V$ such that (i) every vertex appears in some bag, (ii) every edge $\{u,v\}$ has both ends in a common bag, and (iii) for each vertex v, the bags containing v form a connected subtree of $\mathcal{T}$. The *width* is $\max_t |B_t|-1$, and $\mathrm{tw}(G)$ is the minimum width over all decompositions.

The connectivity condition (iii) is exactly what makes every bag a separator.

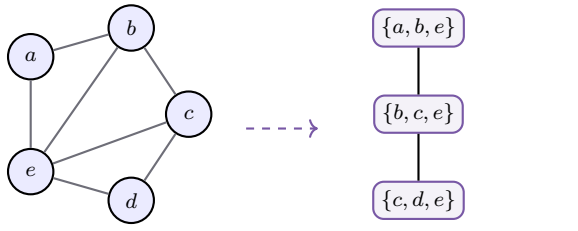


Figure 27.6: A graph of treewidth 2 and a width-2 tree decomposition. The shared bag $\{b,e\}$ between the top two bags is a separator: deleting $\{b,e\}$ disconnects $\{a\}$ from $\{c,d\}$. Every bag separates the graph into the parts hanging off the corresponding tree node.

Theorem 27.5 (Bags are separators)**.** *Let $\mathcal{T}$ be a tree decomposition of G of width w. For any tree edge $\{t, t'\}$ of $\mathcal{T}$, removing the bag intersection $B_t \cap B_{t'}$ separates the vertices "on the t side" from those "on the t' side." In particular, every bag B_t is a separator of size $\leq w + 1$ whose removal disconnects the graph into the parts corresponding to the components of $\mathcal{T} - t$.*

Proof. Deleting tree edge $\{t, t'\}$ splits $\mathcal{T}$ into two subtrees, $\mathcal{T}_t$ and $\mathcal{T}_{t'}$. Let A = vertices appearing only in bags of $\mathcal{T}_t$, B = vertices appearing only in bags of $\mathcal{T}_{t'}$, and $S = B_t \cap B_{t'}$. Suppose for contradiction an edge $\{u, v\} \in E$ has $u \in A, v \in B$. By property (ii) some bag B_x contains both u and v. Say $x \in \mathcal{T}_t$. Then $v \in B_x$ with $x \in \mathcal{T}_t$, so v appears in a bag of $\mathcal{T}_t$. But v also appears in some bag of $\mathcal{T}_{t'}$ (as $v \in B$); by property (iii) the bags containing v form a connected subtree, which therefore spans both sides of edge $\{t, t'\}$, forcing $v \in B_t \cap B_{t'} = S$ — contradicting $v \in B$ (which excludes S). Hence no A–B edge exists, and S separates A from B with $|S| \leq |B_t| \leq w + 1$. □

Why it works

Property (iii) — the running-intersection (connected-subtree) condition — is the essential requirement. It says each vertex "lives" in a connected region of the tree, so to pass from a vertex living strictly on one side to one living strictly on the other, you must cross through a vertex living *on the boundary bag*. That boundary bag is the separator. This is the same reason a single vertex separates a tree (Theorem 27.1): a tree is exactly a width-1 decomposition of itself, and the centroid is the balanced bag.

27.8.1 Balanced separators from bounded treewidth

Theorem 27.5 gives *some* separator; to get a *balanced* one, find the centroid bag of the decomposition tree.

Theorem 27.6 (Balanced separator from treewidth)**.** *A graph of treewidth w has a $\frac{1}{2}$-balanced separator S with $|S| \leq w + 1$: every component of $G - S$ has $\leq \frac{1}{2}n$ vertices.*

Proof. Take a tree decomposition $\mathcal{T}$ of width w. Weight each tree node t by the number of vertices appearing *first* in B_t when $\mathcal{T}$ is rooted (so the total weight is n and each vertex is counted once). Find the centroid bag t^* of $\mathcal{T}$ under these weights, as in Theorem 27.1: removing t^* from $\mathcal{T}$ leaves subtrees each carrying $\leq \frac{1}{2}n$ weight. Set $S = B_{t^*}$. By Theorem 27.5, S separates the graph into the parts corresponding to those subtrees, each of size $\leq \frac{1}{2}n$, and $|S| \leq w + 1$. □

Connection

Separators and treewidth are closely linked. *Treewidth is small if and only if the graph has small balanced separators recursively.* One direction is Theorem 27.6 (small treewidth ⇒ small balanced separators). The converse also holds up to constants: if every subgraph has a balanced separator of size $\leq k$, then $\mathrm{tw}(G) = O(k \log n)$ (and $O(k)$ for nice families). So "has small separators" and "has small treewidth" are essentially the same structural property viewed from two angles — separators are the *local* witness, a tree decomposition is the *global* certificate assembling them. Planar graphs have $\mathrm{tw} = O(\sqrt{n})$ precisely because of the $O(\sqrt{n})$ separator theorem, and this is what makes the planar DP and FPT machinery of Chapters 24–26 run.

Example 27.1 (Planar treewidth bound)**.** Applying the $O(\sqrt{n})$ planar separator recursively bounds planar treewidth: $\mathrm{tw}(G) = O(\sqrt{n})$ for planar G. The recursion $f(n) \le f(\frac{2}{3}n) + O(\sqrt{n})$ for the width of the assembled decomposition solves to $O(\sqrt{n})$. Combined with bounded-treewidth DP (cost $2^{O(\mathrm{tw})}\,\mathrm{poly}\,n$), this is a *second* route to the $2^{O(\sqrt{n})}$ planar algorithms of Section 27.6: build a width-$O(\sqrt{n})$ decomposition, then run the standard treewidth DP. The bound is the same, but the structure is cleaner: you compute the decomposition once and then never separate again.

27.8.2 Treewidth DP: bounded width gives exact poly-time

When w is a *constant* (or small parameter), the separator is constant-sized and the recursion becomes a polynomial-time dynamic program over the bags — the FPT technique of Chapter 25.

Algorithm 113: TreewidthDP skeleton — weighted independent set, width w

Input: nice tree decomposition $(\mathcal{T}, \{B_t\})$ of width w, rooted
Output: max weight of an independent set

```
for each node t of T in post-order do
    for each independent subset X ⊆ B_t do
        // dp[t][X] = best weight in the subtree below t with B_t ∩ I = X
        combine children's dp tables consistent with X on the shared (separator) vertices
return max_X dp[root][X]
```

Cost summary

The DP keeps, per bag, a table indexed by subsets of the bag — $O(2^w)$ states for independent set, $2^{O(w)}$ for connectivity problems like Hamiltonicity (via the Cut&Count / rank-based methods of Chapter 25). With $O(n)$ bags, total time is $O(2^w\, w\, n)$ for independent set — *linear in* n for fixed w. The separator size $w+1$ is exactly the table's exponent: every join over a tree edge synchronizes on the shared separator $B_t \cap B_{t'}$, so the separator is what you enumerate, just as in the planar MIS algorithm. Treewidth DP *is* separator-based divide-and-conquer, with the decomposition precomputing all the separators at once.

Trace it yourself

Run the DP by hand on the graph of Figure 27.6, vertices $\{a, b, c, d, e\}$ with edges $ab, bc, cd, de, ea, be, ce$, all weights 1, maximizing the size of an independent set. The non-edges are exactly ac, ad, bd, so the true answer is a 2-set such as $\{a, c\}$, $\{a, d\}$, or $\{b, d\}$ — no 3-set is independent (check: any third vertex hits an edge). We will watch the DP rediscover **2** touching only $\le w + 1 = 3$ vertices at a time. Use the path decomposition $B_3 = \{c, d, e\}$, $B_2 = \{b, c, e\}$, $B_1 = \{a, b, e\}$, rooted at B_1 so post-order is B_3, B_2, B_1. The shared separators are $B_2 \cap B_3 = \{c, e\}$ and $B_1 \cap B_2 = \{b, e\}$. Vertex d is forgotten when we leave B_3; c is forgotten when we leave B_2; a is introduced only at the root. Each $dp[B_t][X]$ stores the best independent set in the subtree at or below t whose intersection with B_t equals X.

Leaf $B_3 = \{c, d, e\}$**.** Edges inside: cd, de, ce — a triangle, so only singletons and $\emptyset$ are independent. $dp[B_3]$: $dp[\emptyset] = 0$, $dp[\{c\}] = 1$, $dp[\{d\}] = 1$, $dp[\{e\}] = 1$. Forgetting d, we project onto the separator $\{c, e\}$, keeping the best value per pattern: $\{c, e\}{=}\emptyset \to 1$ (realized by $\{d\}$), $\{c\} \to 1$, $\{e\} \to 1$.

Internal $B_2 = \{b, c, e\}$. Edges inside: bc, ce, be — again a triangle, only singletons. The forget-d projection above tells us how much weight the subtree below contributes for each choice on the shared $\{c, e\}$: pattern $\emptyset \to 1$ (the set $\{d\}$), $\{c\} \to 1$ (the set $\{c\}$), $\{e\} \to 1$ (the set $\{e\}$). For each independent $X \subseteq B_2$ we glue to the child pattern whose $\{c, e\}$ choice *agrees* with X (so we never combine states that disagree on a separator vertex), and we add only the weight of the vertices of X *not already counted in the child* (the shared separator vertices are counted once): $X = \emptyset \to 0 + (\text{pattern } \emptyset) = 0 + 1 = 1$; $X = \{b\} \to w(b) + (\text{pattern } \emptyset) = 1 + 1 = 2$, since b is new (it is not in the child's $\{c, e\}$); $X = \{c\} \to (\text{pattern } \{c\}) = 1$, the shared c already counted in the child; $X = \{e\} \to (\text{pattern } \{e\}) = 1$, the shared e already counted. Now forget c, projecting onto $\{b, e\}$ by taking, per pattern, the best value over X that excludes c: $\emptyset \to 1$ (from $X = \emptyset$, beating $X = \{c\}$=1), $\{b\} \to 2$ (from $X = \{b\}$), $\{e\} \to 1$ (from $X = \{e\}$).
Root $B_1 = \{a, b, e\}$. Edges inside: ab, be, ea — triangle, only singletons. Vertex a is introduced here; it is non-adjacent to the already-forgotten c, d, so adding a never conflicts below. Combine each independent $X \subseteq B_1$ with the child's $\{b, e\}$-pattern that agrees with X, again adding only the weight of new (non-shared) vertices: $X = \emptyset \to 0 + (\text{pattern } \emptyset) = 0 + 1 = 1$; $X = \{a\} \to w(a) + (\text{pattern } \emptyset) = 1 + 1 = 2$, with a new and the child realizing pattern $\emptyset$ by $\{d\}$, so the set is $\{a, d\}$; $X = \{b\} \to (\text{pattern } \{b\}) = 2$, the shared b already counted in the child pattern $\{b\}$ (realized by $\{b, d\}$), and $a \notin X$ so no ab conflict arises; $X = \{e\} \to (\text{pattern } \{e\}) = 1$, shared e counted once. The maximum over the root is **2**, realized by $\{a, d\}$ (from $X = \{a\}$, inheriting d from B_3) or by $\{b, d\}$ (from $X = \{b\}$). The key point: each join is a *constrained* merge on the shared separator ($\{c, e\}$, then $\{b, e\}$) — only states agreeing on every separator vertex may glue — which is exactly why the separator size is the table's exponent and why bounded width gives linear-time exact answers.

27.9 Tradeoffs: size, balance, findability, and gain

Separator theorems come in several forms, and the choices interact; picking the wrong tradeoff wastes the available structure. The main axes:

- **Size vs. balance.** You can often trade a smaller separator for worse balance or vice versa. A $\frac{1}{2}$-balanced separator may be larger than a $\frac{2}{3}$-balanced one. For divide-and-conquer the constant α barely matters (any $\alpha < 1$ gives $O(\log n)$ depth), so prefer the smallest separator with *some* constant balance.
- **Vertex vs. edge separators.** We used vertex separators throughout. *Edge separators* (delete edges to disconnect, à la sparsest cut, Chapter 33) are different and generally larger for the same balance; a bounded-degree graph converts between them up to a degree factor. Spectral methods (Cheeger, Chapter 32) find approximate *edge* separators; planar/treewidth methods find exact *vertex* separators.
- **Findability.** A separator theorem is only algorithmic if you can *find* the separator fast. Planar separators: $O(n)$. Treewidth-w separators: need a decomposition, computable in $2^{O(w)}n$ (Bodlaender) or approximately faster. General *balanced* separator (sparsest cut) is NP-hard exactly, with an $O(\sqrt{\log n})$-approximation (Arora–Rao–Vazirani, Chapter 33) — the gap between "exists" and "findable" is itself a research frontier.
- **Recursiveness.** The class must be closed under the recursion. Planar minus a separator is still planar; treewidth-w minus a bag is still $\leq w$. General graphs offer no such closure, which

is why the recursion does not start.

- **What the gain is.** Match the separator to the goal: $O(\sqrt{n})$ buys $O(n \log n)$ D&C, $2^{O(\sqrt{n})}$ exact, and $O(n^{3/2})$ elimination; constant treewidth buys linear-time exact DP; sublinear-but-not-$\sqrt{n}$ separators (e.g. $O(n^{2/3})$ for some bounded-genus or low-dimensional graphs) buy correspondingly weaker but still sub-exponential bounds.

Common trap

A separator theorem with no efficient algorithm is a trap. Many graph classes *have* small balanced separators that are NP-hard to find optimally (general graphs with small separators, via sparsest cut). Do not assume "small separator exists" implies "divide-and-conquer is efficient" — you also need a fast (possibly approximate) separator oracle. Conversely, a fast oracle giving a *slightly* larger separator (say $O(\sqrt{n} \log n)$) usually costs you only a log factor in the recurrence and is well worth it. Optimize for *found* size, not existential size.

Why this matters

Across the book, small separators appear from three distinct viewpoints. *Geometrically* (planar graphs), a small separator is a short cycle splitting a drawing into balanced regions. *Structurally* (treewidth), it is a central bag in a tree decomposition. *Spectrally* (Chapters 32–34), it is a sparse cut exposed by the second Laplacian eigenvector. Each viewpoint asks the same algorithmic question: where is the graph's narrowest balanced bottleneck, and how can we recurse across it? A separator theorem guarantees that such a bottleneck exists and gives an algorithm to find it. When sublinear separators exist recursively, divide-and-conquer and dynamic programming work well; when no sparse cut exists at all (expander graphs, Chapter 34), recursive decomposition fails. The contrast between separability and expansion recurs throughout modern graph theory.

Connection

What This Made Possible: The Planar Separator Theorem made divide-and-conquer on graphs practical. It extended Alan George's nested dissection to arbitrary planar finite-element meshes, solving $n \times n$ sparse systems in $O(n^{3/2})$ arithmetic operations and $O(n \log n)$ fill-in rather than $O(n^3)$ dense elimination. For NP-hard problems, it enabled sub-exponential $2^{O(\sqrt{n})}$ exact algorithms for planar independent set, planar dominating set, and planar TSP, replacing $2^{\Omega(n)}$ brute force.

Limitations:

- *Strictly 2D topological scaling:* The $O(\sqrt{n})$ separator size is tied to the two-dimensional area-to-perimeter ratio. For 3D meshes (e.g. volumetric finite elements), separators grow to $\Theta(n^{2/3})$, increasing nested dissection cost to $O(n^2)$.
- *Expanders have no small separators:* For dense graphs, random graphs, or expander graphs, any balanced separator requires deleting $\Omega(n)$ vertices. On such graphs, divide-and-conquer completely breaks down.
- *Constant factors:* The theoretical bound $2\sqrt{2}\sqrt{n} \approx 2.83\sqrt{n}$ means that for $n = 100$, the separator may contain up to 28 vertices, leaving only modest subproblems until n is moderately large.

Where We Go Next: Separators slice a graph apart into independent components. But how do we understand local constraints that prevent independent sets, such as coloring conflicts? In **Chapter 28 (Graph Coloring)**, we investigate vertex colorings, Kempe chains, and the Four-Color Theorem on planar graphs. Later, when graphs are non-planar and lack topological separators, how do we find small cuts? In **Chapter 31 (The Graph Laplacian)** and **Chapter 32 (Spectral Graph Theory)**, we turn to continuous linear algebra: the second Laplacian eigenvector (the Fiedler vector) reveals hidden cuts and bottlenecks spectrally.

27.10 Exercises

Exercise 27.1 (Grid lower bound)**.** Prove that the $k \times k$ grid ($n = k^2$) has no $\frac{1}{2}$-balanced separator of size smaller than $k = \sqrt{n}$. (Hint: a balanced separator must intersect $\Omega(k)$ of the k row-paths or k column-paths; use Menger / vertex-disjoint paths to lower-bound the intersection.)

Exercise 27.2 (Levels are separators)**.** Implement BFS-level separators: given any graph, output, for each BFS level L_i, the sizes $|L_{<i}|, |L_i|, |L_{>i}|$, and identify the median level. On a random planar graph, plot $|L_i|$ versus i and locate the thinnest level near the median. Where does balance fail, and where does size fail?

Exercise 27.3 (Fundamental-cycle separator)**.** For a triangulated planar graph with a rooted spanning tree, write code that, for each non-tree edge, computes the inside-weight of its fundamental cycle, and returns the edge whose cycle is most balanced. Prove that the resulting cycle has both sides $\leq \frac{2}{3}n$ and length $O(h)$ where h is the tree height.

Exercise 27.4 (Break the median heuristic)**.** Construct a planar graph where the median BFS level has $\Theta(n)$ vertices (so the "just take the median level" shortcut fails), forcing Steps 2–3 of the proof. (Hint: a graph whose BFS levels form a single fat level in the middle — think of two large fans joined at a wide waist.)

Exercise 27.5 (Treewidth $\Leftrightarrow$ separators)**.** Prove that if every subgraph of G has a $\frac{2}{3}$-balanced separator of size $\leq k$, then $\mathrm{tw}(G) = O(k \log n)$. (Build a tree decomposition top-down: separate, recurse, and add the separator to every descendant bag; bound the bag size by the recursion depth.)

Exercise 27.6 (Sub-exponential vertex cover)**.** Adapt Algorithm 112 to compute a minimum *vertex cover* on planar graphs in $2^{O(\sqrt{n})}$ time, and verify the recurrence solves to sub-exponential. Then explain why the same approach gives a $3^{O(\sqrt{n})}$ algorithm for 3-*colouring*, with 3 in place of 2 per separator vertex.

Exercise 27.7 (Nested dissection fill-in)**.** On the $k \times k$ grid, order the vertices by nested dissection (separator last, recursively). Prove the fill-in obeys $F(n) \leq 2F(n/2) + O(n) = O(n \log n)$ and the operation count is $O(n^{3/2})$. Compare experimentally against a naive row-major ordering's fill-in.

Exercise 27.8 (r-division by hand)**.** On a 6×6 grid, construct an r-division with $r = 9$: partition into pieces of ≤ 9 vertices, each with $O(\sqrt{r}) = O(3)$ boundary vertices. Count the total boundary size and verify it is $O(n/\sqrt{r})$. Then answer a single shortest-path query between two corners using only the per-piece boundary distances.

Exercise 27.9 (Expanders have no small separators)**.** Using the (edge) expansion definition $h(G) = \min_{|S| \leq n/2} \frac{e(S, \bar{S})}{|S|} \geq \beta$ for a constant $\beta > 0$, prove that a constant-degree expander has every $\frac{2}{3}$-balanced *vertex* separator of size $\Omega(n)$. Contrast with the planar theorem and explain why no $O(\sqrt{n})$-separator algorithm can exist for general bounded-degree graphs.

Exercise 27.10 (Cut between two thin levels)**.** In the planar proof we sometimes use two thin levels L_a, L_b as the separator. Prove that $L_a \cup L_b$ is indeed a separator (no edge from $L_{<a}$ to $L_{>b}$ avoiding the band), and that if the middle band M has $|M| \le \frac{2}{3}n$ we are immediately done. Identify precisely which case forces the fundamental-cycle step instead.

Exercise 27.11 (Trace the median-level argument)**.** Reproduce Figure 27.2 for the 6×6 grid ($n = 36$) rooted at a corner. Write out the eleven level sizes $|L_0|, \dots, |L_{10}|$, the cumulative counts, and identify the median level i^*. Show that $|L_{i^*}| = 6 = \sqrt{n}$ and that both sides have $\le n/2$ vertices, so Step 1 already separates the grid. Then change the root to a *center* vertex and recompute: do the level sizes still peak at $\sqrt{n}$, and is the median level still thin?

Exercise 27.12 (Treewidth DP table sizes)**.** Continue the worked DP trace (the worked example after Algorithm 113) on the graph of Figure 27.6, but now *maximize a weighted* independent set with weights $w(a) = 3$ and $w(b) = w(c) = w(d) = w(e) = 1$. Recompute all three bag tables and the final answer. Then count: how many table entries does each bag store as a function of the width $w = 2$, and how does that count grow if you instead solve *dominating set*, where each bag vertex carries one of three states (in the set / dominated / undominated)? Relate the 3^{w+1} blow-up to the separator size.

Exercise 27.13 (Edge vs. vertex separators on the grid)**.** The $k \times k$ grid has a vertex separator of size k (a middle column) and also an *edge* separator of size k (the k horizontal edges crossing the midline). Show both achieve $\frac{1}{2}$-balance. Now take a bounded-degree graph of max degree Δ and prove that any edge separator of size s yields a vertex separator of size $\le s$ (take one endpoint per cut edge), and that any vertex separator of size t yields an edge separator of size $\le \Delta t$. Where does the grid sit in these bounds, and why is the conversion lossless there?

Chapter 28

Graph Coloring

In the preceding chapter, we saw how separators split a graph along narrow boundaries. Now we turn from cutting graphs to coloring them. Color a map of the counties of England so that no two counties sharing a border get the same color, and you will never need more than four colors. Reframe the map as a graph – one vertex per county, an edge between counties that touch – and the puzzle becomes the central problem of this chapter: assign a color to every vertex so that adjacent vertices differ, using as few colors as possible. That minimum number is the *chromatic number* $\chi(G)$.

In coloring problems an edge no longer means "you can travel here," but "these two endpoints cannot make the same choice." Compilers mapping variables to CPU registers, universities assigning exams to timeslots, and mobile towers sharing frequencies are all coloring problems. Here we examine greedy orderings, Brooks's and Vizing's degree bounds, the linear-time coloring of chordal graphs, and the complexity barriers that make general coloring NP-hard.

History

The mathematical theory of graph coloring began with a cartographic observation. In October 1852, Francis Guthrie, an undergraduate student at University College London, noticed while coloring a map of England's counties that four colors always seemed sufficient. Guthrie asked his brother Frederick, who consulted Augustus De Morgan. In 1879, Alfred Kempe published a claimed proof introducing *Kempe chains* – paths alternating between two colors that allow swapping colors locally. In 1890, Percy Heawood discovered that Kempe's swap could accidentally intertwine independent chains in degree-5 configurations, breaking the proof, though Heawood salvaged the idea to prove the Five Color Theorem.

Progress on general graphs soon followed. In 1941, R. Leonard Brooks proved that every connected graph has $\chi(G) \leq \Delta(G)$ unless it is a complete graph or an odd cycle (László Lovász later gave a slick ordering proof, the ancestor of modern algorithmic recoloring). In 1964, Vadim Vizing proved that edges can always be colored using either Δ or $\Delta + 1$ colors. The Four Color Theorem itself was finally settled in 1976 by Kenneth Appel and Wolfgang Haken, who used 1,200 hours of computer time to verify 1,936 reducible configurations. The result provoked intense philosophical debate over whether computer-assisted proofs that no human could verify by hand constitute genuine mathematical knowledge.

On the computational side, Richard Karp placed Chromatic Number on his original 1972 list of 21 NP-complete problems. In the late 1990s, Uriel Feige, Joe Kilian, and Johan Håstad proved via the PCP theorem that χ cannot even be approximated within $n^{1-\varepsilon}$ in polynomial

time unless P = NP. Today, whether a 3-colorable graph can be colored with $O(1)$ colors in polynomial time remains one of the major open problems in algorithm design.

28.1 Proper Colorings and the Chromatic Number

Definition 28.1 (Proper coloring)**.** Let $G = (V, E)$ be an undirected graph with no self-loops. A *k-coloring* is a function $c : V \to \{1, 2, \ldots, k\}$. It is *proper* if $c(u) \neq c(v)$ for every edge $uv \in E$. The graph is *k-colorable* if a proper k-coloring exists. The *chromatic number* $\chi(G)$ is the least k for which G is k-colorable.

The colors $1, \ldots, k$ are just labels; nothing about them is ordered or numeric. A proper k-coloring is the same data as a partition of V into k (possibly empty) *color classes* $V_1, \ldots, V_k$, where each $V_i = c^{-1}(i)$ is an *independent set* – a set of vertices with no edge inside it. So coloring is exactly the problem of covering the vertex set by as few independent sets as possible. We will use this view often: $\chi(G)$ is the minimum number of independent sets needed to partition V.

Intuition

Three pictures to keep in your head. A path or any tree needs 2 colors – walk along it flipping a switch. A triangle needs 3: every pair of its vertices is adjacent, so no two may share a color. And an odd cycle C_{2k+1} needs 3 even though it looks like it ought to alternate: try to two-color it and the alternation collides with itself when the cycle closes. The jump from "looks alternating" to "actually needs three" is why bipartiteness is special.

A few exact values are worth knowing. The complete graph K_n has every pair adjacent, so all n vertices need distinct colors and $\chi(K_n) = n$. A graph with no edges has $\chi = 1$. A cycle C_n has $\chi(C_n) = 2$ if n is even and 3 if n is odd. Any tree with at least one edge has $\chi = 2$.

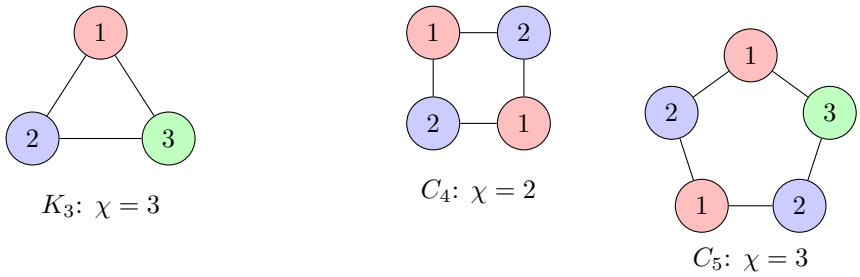


Figure 28.1: A triangle forces three colors; an even cycle alternates with two; an odd cycle cannot alternate consistently and is forced up to three. The third color in C_5 resolves the parity clash where the cycle closes.

Two structural lower bounds fall out immediately and are worth naming because they are the only general handles we get for free.

Proposition 28.1 (Clique and independence-set bounds)**.** *For every graph G on n vertices,*

$$\omega(G) \;\le\; \chi(G) \;\le\; n, \qquad \text{and} \qquad \chi(G) \;\ge\; \frac{n}{\alpha(G)},$$

where $\omega(G)$ is the size of a largest clique and $\alpha(G)$ the size of a largest independent set.

Proof. A clique of size ω has all pairs adjacent, so its vertices receive ω distinct colors; hence $\chi \ge \omega$. For the right inequality, color all n vertices distinctly. For the last bound: in a proper coloring each color class is independent, so each has at most α vertices; χ classes cover at most $\chi \cdot \alpha$ vertices, and this must be at least n. □

Common trap

Do not mistake $\omega(G) \le \chi(G)$ for an equality. It is tempting because it holds for many friendly graphs, but the gap can be arbitrarily large. The odd cycle C_5 already shows it: $\omega(C_5) = 2$ (no triangle) yet $\chi(C_5) = 3$. Worse, there exist *triangle-free* graphs ($\omega = 2$) with chromatic number as large as you like – Mycielski's construction builds them. Graphs where $\omega = \chi$ holds for the graph *and all its induced subgraphs* are exactly the *perfect graphs*, the subject of Chapter 30; most graphs are not perfect.

28.2 The Greedy Algorithm and Degeneracy

The most basic coloring algorithm is the one a child uses: take the vertices in some order, and give each the smallest color not already used by an already-colored neighbor. It is myopic and it is not optimal, but it never fails to produce a *proper* coloring, and how many colors it spends depends entirely on the order you feed it.

Algorithm 114: GreedyColor(G, π) – greedy coloring in a fixed order

Input: Graph $G = (V, E)$; an ordering $\pi = (v_1, \dots, v_n)$ of V.
Output: A proper coloring $c : V \to \{1, 2, \dots\}$.

```
for i ← 1 to n do
    S ← { c(u) : u ∈ N(v_i), u already colored };
    c(v_i) ← smallest positive integer not in S;
return c;
```

Why it works

Why is the output always proper? When we color v_i, we explicitly forbid every color currently assigned to a neighbor of v_i. A neighbor v_j either is colored already (we avoid its color now) or comes later (it will avoid *our* color when its turn arrives). Either way the edge $v_i v_j$ ends up bichromatic. So correctness is automatic; the only question is the *number* of colors.

A clean upper bound comes from looking at how many *earlier* neighbors each vertex has.

Proposition 28.2 (Greedy degree bound)**.** *Running Algorithm 114 uses at most $\Delta(G) + 1$ colors, where Δ is the maximum degree. More sharply, if every vertex v_i has at most d neighbors among $v_1, \dots, v_{i-1}$ (its* back-degree*), then at most $d + 1$ colors are used.*

Proof. When v_i is colored, the forbidden set S has size at most the number of already-colored neighbors of v_i, which is its back-degree, $\le d$. The smallest color avoiding a set of size $\le d$ is at most $d+1$. Since back-degree never exceeds total degree $\le \Delta$, the global bound $\Delta+1$ follows. □

The sharper, back-degree version is the key to everything that follows, because it says: *we get to choose the order, so we should choose one that keeps back-degrees small.* That motivates a graph parameter.

Definition 28.2 (Degeneracy)**.** A graph is *d-degenerate* if every subgraph has a vertex of degree at most d. The *degeneracy* $\mathrm{degen}(G)$ is the smallest such d.

Equivalently, a graph is d-degenerate iff you can repeatedly delete a vertex of degree $\le d$ until nothing is left. Reversing that deletion order gives an ordering in which every vertex has back-degree $\le d$. This is the *degeneracy ordering*, and it is the order you want to hand the greedy algorithm.

Algorithm 115: DegeneracyOrder(G) – smallest-last ordering

Input: Graph $G = (V, E)$.
Output: Ordering $v_1, \dots, v_n$ with back-degrees $\le \mathrm{degen}(G)$, and $d = \mathrm{degen}(G)$.

```
Maintain buckets B_0, B_1, ... where B_j holds current vertices of residual degree j;
d ← 0;   let deg[v] be the current degree of each v;
for i ← n, n − 1, ..., 1 do
    let j* be the smallest nonempty bucket index; remove some v from B_{j*};
    d ← max(d, j*);   v_i ← v;
    foreach still-present neighbor u of v do
        move u from B_{deg[u]} to B_{deg[u]−1};   deg[u] −= 1;
return (v_1, ..., v_n), d;
```

Why it works

The smallest-last rule – repeatedly peel off a minimum-degree vertex of the *current* graph and place it last – realizes the degeneracy definition directly. When vertex v is peeled at residual degree $j^\star$, every vertex remaining (which is exactly $v_1, \dots, v_{i-1}$) is an earlier vertex, so v's back-neighbors are precisely its still-present neighbors, of which there are $j^\star \le d$. The maximum $j^\star$ over the whole peel is the degeneracy: it is the largest minimum-degree you are ever forced to confront, which is exactly $\max_{H \subseteq G} \delta(H) = \mathrm{degen}(G)$. Using bucket lists, each edge is touched a constant number of times, so the whole procedure runs in $O(n+m)$.

Corollary 28.1 (Degeneracy coloring bound)**.** *Every graph satisfies $\chi(G) \le \mathrm{degen}(G) + 1$, and greedy coloring on the degeneracy ordering achieves $\mathrm{degen}(G)+1$ colors in $O(n+m)$ time.*

Proof. Order by Algorithm 115; back-degrees are $\le \mathrm{degen}(G)$; apply Proposition 28.2. The bucket peel is $O(n+m)$, and a single greedy pass where each vertex scans its earlier neighbors is also $O(n+m)$ amortized. □

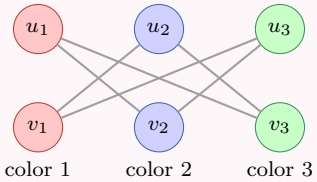
Degeneracy is sharper than Δ wherever the graph is locally dense but globally sparse. A star $K_{1,n}$ has $\Delta = n$ but degeneracy 1 (peel the leaves), so $\chi \le 2$ – correct. Planar graphs are 5-degenerate, giving the easy "six-color theorem" for planar maps in one line.

Trace it yourself

Run greedy on the 4-cycle a-c-b-d-a (edges ac, cb, bd, da), which is bipartite, so $\chi = 2$. Two orders:

Order a, c, b, d	$c(a) = 1$; c adj $a \Rightarrow 2$; b adj $c \Rightarrow 1$; d adj $a, b \Rightarrow 2$ (2 colors).
Order a, b, c, d	$c(a) = 1$; b (no colored nbr)$\Rightarrow 1$; c adj $a, b \Rightarrow 2$; d adj $a, b \Rightarrow 2$ (2 colors).

This 4-cycle is too forgiving to expose greedy. The classic worst case is the *crown graph*: take $K_{n,n}$ minus a perfect matching; it is bipartite ($\chi = 2$), yet the adversarial order $u_1, v_1, u_2, v_2, \ldots$ forces greedy to use n colors. Build a small crown ($n = 3$) by hand and watch greedy spiral: with the matching u_iv_i removed, u_i joins every v_j with $j \neq i$, so when greedy hits u_i (and then v_i) their already-colored neighbors are $v_1, \ldots, v_{i-1}$ (respectively $u_1, \ldots, u_{i-1}$), carrying colors $1, \ldots, i-1$, and the pair u_i, v_i takes a fresh color i. The lesson: greedy's color count is an ordering artifact, never an invariant of the graph.



The crown S_3^0: each u_i misses only its partner v_i. Feeding greedy the order $u_1, v_1, u_2, v_2, u_3, v_3$ paints all three v_i different colors.

Common trap

There *always* exists an ordering on which greedy uses exactly $\chi(G)$ colors – take any optimal coloring and list color class 1 first, then class 2, and so on. So greedy is "optimal with the right order." This is useless algorithmically: finding that order is as hard as coloring itself. Never conclude from "greedy can be optimal" that "greedy is good"; the missing piece is the order, and the order hides all the difficulty.

28.3 Bipartite Graphs: Two Colors and a Certificate

The simplest tractable case is $\chi \leq 2$. A graph is 2-colorable exactly when it is *bipartite*, and there is a perfect, checkable obstruction: the odd cycle.

Theorem 28.1 (Konig's characterization)**.** *For a graph G the following are equivalent:*

(1) $\chi(G) \leq 2$*;*

(2) G *is bipartite (its vertices split into two independent sets);*

(3) G *contains no cycle of odd length.*

Proof. (1)$\Leftrightarrow$(2) is the definition of a 2-coloring read as a bipartition. For (2)$\Rightarrow$(3): in a bipartition (X, Y) every edge crosses, so any closed walk alternates sides and must have even length; an odd cycle is an odd closed walk and cannot exist. For (3)$\Rightarrow$(2): assume G connected (otherwise treat

components separately). Pick a root r, run BFS, and color v by the parity of its distance $\mathrm{dist}(r, v)$: even $\to X$, odd $\to Y$. Suppose some edge uv joins two vertices of equal parity. The tree paths $r \to u$ and $r \to v$ together with edge uv form a closed walk of length $\mathrm{dist}(r, u) + \mathrm{dist}(r, v) + 1$, which is odd; a standard argument extracts an odd cycle from any odd closed walk, contradicting (3). So every edge crosses the bipartition. □

The proof is constructive: BFS two-colors a bipartite graph and, when it fails, hands back an odd cycle as a *certificate* of non-bipartiteness. That certificate is what makes "is this 2-colorable?" decidable in linear time, in sharp contrast to "is this 3-colorable?", which we will see is NP-complete.

Algorithm 116: TwoColor(G) – BFS bipartiteness test and coloring

Input: Connected graph $G = (V, E)$.
Output: A proper 2-coloring, or an odd cycle.

```
c[v] ← ⊥ for all v; pick a start s; c[s] ← 0; queue Q ← {s};
while Q nonempty do
    u ← dequeue Q;
    foreach neighbor w of u do
        if c[w] = ⊥ then
            c[w] ← 1 − c[u]; enqueue w;
        else if c[w] = c[u] then
            return odd cycle through u, w via BFS-tree paths;
return coloring c;
```

Cost summary

TwoColor is one BFS: $O(n + m)$ time, $O(n)$ space. It both decides bipartiteness and produces a witness in either direction – a 2-coloring or an odd cycle. This is the best a coloring algorithm can offer, and beyond two colors it is almost never achievable.

Connection

Bipartite graphs are where coloring, matching, and flow all meet. Konig's theorem (this one) is the coloring counterpart of *Konig's matching theorem* – in a bipartite graph the maximum matching equals the minimum vertex cover – which Chapter 15 proves via flow. Two-colorability also shows up in the spectrum: a connected bipartite graph is precisely one whose largest *adjacency* eigenvalue $\lambda_{\max}$ is matched by $-\lambda_{\max}$ as the smallest, and whose Laplacian spectrum has a corresponding symmetry. Chapter 32 develops this connection.

28.4 Brooks's Theorem

The bound $\chi \le \Delta + 1$ from greedy is tight – complete graphs and odd cycles hit it. Brooks's theorem says those two families are the *only* obstructions: for everything else you can shave one color off.

Theorem 28.2 (Brooks, 1941)**.** *Let G be a connected graph with maximum degree Δ. Then $\chi(G) \le \Delta$ unless G is a complete graph $K_{\Delta+1}$ or an odd cycle, in which case $\chi(G) = \Delta + 1$.*

Intuition

Why should removing the two exceptional families buy a color? Greedy spends $\Delta + 1$ only if some vertex sees all Δ colors already used among its neighbors at the moment it is colored. If we could always arrange that *at least two* neighbors of the troublesome last vertex already share a color – so the neighbors waste a color on a repeat – then Δ colors suffice. The proof engineers exactly such an ordering, and the only graphs where no such arrangement exists are the rigid ones: cliques (every neighbor pair is adjacent, no repeats possible) and odd cycles (too thin to create a repeat).

We give the clean ordering-based proof for the main case.

Proof (main case). Assume G is connected, not complete, not an odd cycle, with $\Delta \geq 3$. (The cases $\Delta \leq 2$ are direct: $\Delta = 0, 1$ are trivial, and a connected $\Delta = 2$ graph is a path or a cycle, where only odd cycles need 3 colors.)

Case A: G is not Δ-regular. Then some vertex v_n has degree $< \Delta$. Build a BFS tree rooted at v_n and order the vertices in *decreasing* BFS depth, breaking ties arbitrarily, so v_n comes last. Every v_i with $i < n$ has at least one neighbor strictly closer to v_n (its BFS parent), which comes *later* in this order; hence v_i's back-degree is at most $\deg(v_i) - 1 \leq \Delta - 1$. And v_n itself has degree $< \Delta$. Greedy in this order never sees Δ forbidden colors, so it uses $\leq \Delta$.

Case B: G is Δ-regular. Here every vertex has exactly Δ neighbors, so the parent trick alone leaves the last vertex exposed. The fix is to plant a deliberate color repeat. One can show that a connected, Δ-regular, non-complete graph with $\Delta \geq 3$ contains a vertex v_n with two non-adjacent neighbors $a = v_1$ and $b = v_2$ such that $G - \{a, b\}$ stays connected. (If G is 2-connected this is a short argument; if it has a cut vertex one descends into the blocks and inducts.) Order $v_3, \ldots, v_{n-1}$ by decreasing BFS depth from v_n inside $G - \{a, b\}$, ending with v_n.

Now run greedy. Give a and b the same color 1 (legal, they are non-adjacent). Every internal v_i ($3 \leq i \leq n-1$) has a later neighbor toward v_n, hence back-degree $\leq \Delta - 1$, so $\leq \Delta$ colors suffice for it. Finally v_n has Δ neighbors, but two of them, a and b, share color 1; so v_n's neighbors use at most $\Delta - 1$ distinct colors, and a free color in $\{1, \ldots, \Delta\}$ remains (Figure 28.2). Thus $\chi(G) \leq \Delta$. □

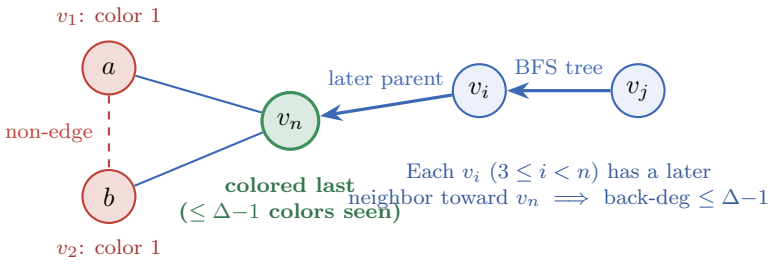


Figure 28.2: Brooks's theorem (Case B). Non-adjacent neighbors $a = v_1$ and $b = v_2$ of v_n both receive color 1 first. Ordering the remaining vertices along a spanning tree rooted at v_n guarantees every intermediate vertex v_i has at least one uncolored parent neighbor toward v_n, while v_n itself sees color 1 repeated twice among its Δ neighbors.

Common trap

The delicate clause is "find v_n with two non-adjacent neighbors keeping $G - \{a, b\}$ connected." If you skip the connectivity requirement, the decreasing-depth ordering can strand some vertex with all Δ neighbors earlier, and the saved color is lost. The full proof must also rule out the case where every two neighbors of every vertex are adjacent – but that forces a clique on each neighborhood, and a connected such graph is itself complete, already excluded. This neighborhood-clique trap is exactly where Kempe's 1879 four-color "proof" went wrong in spirit: an unjustified assumption that a convenient local configuration always exists.

28.5 Edge Coloring and Vizing's Theorem

Shift attention from vertices to edges. An *edge coloring* assigns colors to edges so that edges sharing an endpoint differ; the minimum number of colors is the *chromatic index* $\chi'(G)$. The model is scheduling matches in a tournament: edges meeting at a vertex are games involving the same player and cannot run at the same time, so each color is a round of simultaneous, non-conflicting games.

Intuition

Edge coloring of G is exactly vertex coloring of its *line graph* $L(G)$, where each edge of G becomes a vertex and two are adjacent iff they shared an endpoint. So edge coloring is a special case of vertex coloring, but a much more constrained one. A vertex of degree Δ in G has Δ incident edges, all mutually conflicting: a clique of size Δ in $L(G)$. That forces $\chi' \geq \Delta$. Vizing's theorem says you never need more than one extra color.

Theorem 28.3 (Vizing, 1964)**.** *For every simple graph G,*

$$\Delta(G) \;\leq\; \chi'(G) \;\leq\; \Delta(G) + 1.$$

The lower bound is the clique-in-the-line-graph argument above. The upper bound is the deep half, and its proof is a piece of recoloring via *alternating paths* of two colors – a *Kempe chain*. The classification into "Class 1" graphs ($\chi' = \Delta$) and "Class 2" graphs ($\chi' = \Delta + 1$) is itself NP-hard to decide, so Vizing's theorem is the rare situation where you know the answer to within 1 but pinning it down exactly is intractable.

Proof of the upper bound. We color edges one at a time, maintaining a partial proper edge coloring with the palette $\{1, \ldots, \Delta + 1\}$, and show any uncolored edge can be absorbed.

For a vertex x, call a color *missing at* x if no edge incident to x currently has that color. Since $\deg(x) \leq \Delta$ but the palette has $\Delta + 1$ colors, every vertex has at least one missing color at every stage.

Take an uncolored edge xy_0. Build a *fan*: a maximal sequence of distinct neighbors $y_0, y_1, \ldots, y_k$ of x where, for each $i \geq 1$, the edge xy_i is colored with a color that is *missing at* y_{i-1}. Let α be a color missing at x and β a color missing at y_k.

Easy case: β is also missing at x. Then "rotate" the fan: recolor $xy_i \leftarrow$ color of xy_{i+1} for $i = 0, \ldots, k-1$, which stays proper because each shift uses a color missing at the receiving neighbor, and finally color xy_k with β. The edge xy_0 is now colored and the coloring is still proper.

Hard case: β is used at x. Consider the maximal path P of edges alternating colors α and β starting at x (a Kempe chain in $\{\alpha, \beta\}$). Because α is missing at x, vertex x is an endpoint of P. Swap $\alpha \leftrightarrow \beta$ along P. This keeps the coloring proper and alters which colors are missing only at the chain's far endpoint. A short case check on where P ends (at y_k, at the relevant fan vertex, or elsewhere) shows that after the swap, one of two truncated fans falls into the easy case, so a rotation finishes the job. Inducting on the number of colored edges completes the proof. □

Why it works

The key tool is the two-color Kempe chain. Restricting to two colors α, β, each vertex meets at most one edge of each, so the $\{\alpha, \beta\}$-subgraph is a disjoint union of paths and even cycles. Swapping the two colors along one component keeps the coloring proper everywhere and only disturbs the missing-color pattern at the two endpoints of that path. This local control – "I can flip α and β on one path without breaking anything except locally" – is the same idea used in Brooks's theorem, the four-color attempts, and the augmenting-path arguments of matching.

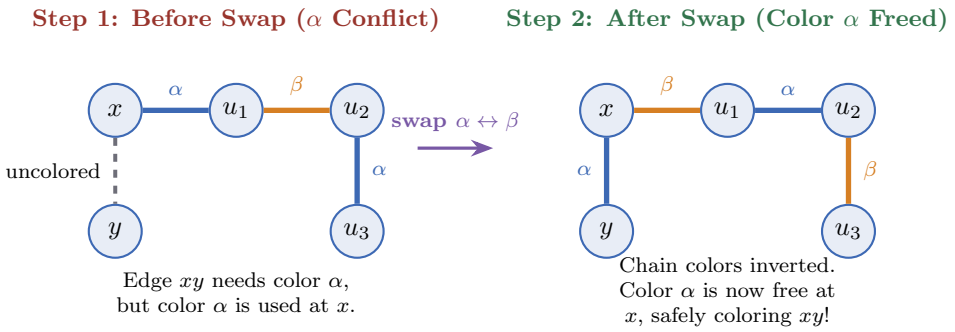


Figure 28.3: The two-color Kempe chain swap in Vizing's theorem. **Left:** The uncolored edge xy seeks color α, but α is already incident to x along an alternating (α, β) path. **Right:** Swapping colors $\alpha \leftrightarrow \beta$ along the connected Kempe chain inverts the alternating edges without creating conflicts anywhere along the path, freeing color α at vertex x.

Vizing's proof is algorithmic. The fan and the single Kempe swap per edge cost $O(n)$ each, over m edges, giving a polynomial edge-coloring algorithm.

Algorithm 117: VizingEdgeColor(G) – $(\Delta+1)$-edge-coloring

Input: Simple graph $G=(V,E)$ with maximum degree Δ.
Output: Proper edge coloring with palette $\{1,\dots,\Delta+1\}$.

```
foreach edge e = xy_0 ∈ E (in any order) do
    build a maximal fan y_0, ..., y_k at x;
    α ← a color missing at x;   β ← a color missing at y_k;
    if β missing at x then
        rotate the fan and set c(xy_k) ← β;
    else
        swap colors α, β along the Kempe chain from x;
        truncate the fan at the right point and rotate; assign the freed color;
return c;
```

Cost summary

The fan has length $\le \deg(x) \le \Delta$, and the Kempe chain has length $\le n$. Per edge the work is $O(n)$, so the whole edge coloring runs in $O(nm)$ time and produces at most $\Delta+1$ colors – provably within one of optimal. For bipartite graphs the bound improves: *Konig's edge-coloring theorem* gives $\chi' = \Delta$ exactly, achievable via repeated perfect matchings (each color class a matching that saturates every maximum-degree vertex).

Application

Round-robin scheduling. To schedule a round-robin among n players (every pair plays once) in the fewest rounds with no player playing twice in a round, edge color K_n. For even n, $\Delta = n-1$ and K_n is Class 1, so $n-1$ rounds suffice – each round a perfect matching. For odd n, $\chi'(K_n) = n$, meaning one player sits out each of n rounds. The "circle method" tournament organizers use by hand is exactly this edge coloring made explicit.

28.6 Chordal and Interval Graphs: Optimal Coloring

So far we have worked around NP-hardness with bounds and special structure. Two graph classes can be colored *optimally* in linear time: interval graphs and, more generally, chordal graphs. They are where $\omega = \chi$ holds with an algorithm to match.

Definition 28.3 (Interval graph)**.** A graph is an *interval graph* if its vertices can be put in correspondence with intervals on the real line so that two vertices are adjacent iff their intervals overlap.

Definition 28.4 (Chordal graph)**.** A graph is *chordal* (triangulated) if every cycle of length ≥ 4 has a *chord* – an edge joining two non-consecutive cycle vertices. Equivalently, it has no induced cycle C_k for $k \ge 4$.

Every interval graph is chordal; chordal graphs are the strictly larger class. Chordal graphs can be colored optimally in a single pass because they admit a clean elimination order.

Theorem 28.4 (Perfect elimination ordering)**.** *A graph is chordal iff it has a* perfect elimination ordering *(PEO): an ordering $v_1,\dots,v_n$ such that for each i, the* later *neighbors of v_i – those among $v_{i+1},\dots,v_n$ – form a clique. Such an ordering is found in $O(n+m)$ by* Lexicographic BFS.

Intuition

A vertex whose neighbors are all mutually adjacent is called *simplicial*. Chordal graphs are exactly the graphs you can dismantle by repeatedly deleting a simplicial vertex – every chordal graph has at least one, and deleting it leaves a chordal graph. The deletion order, reversed, is the PEO. Think of an interval graph: the interval that ends earliest overlaps only intervals that all contain its right endpoint, and those mutually overlap there – so the earliest-ending interval is simplicial. Sorting intervals by right endpoint *is* a PEO.

Greedy coloring in PEO order is optimal.

Theorem 28.5 (Optimal chordal coloring)**.** *Let $v_1, \dots, v_n$ be a PEO of a chordal graph G. Greedy coloring in the order $v_1, \dots, v_n$ uses exactly $\omega(G) = \chi(G)$ colors.*

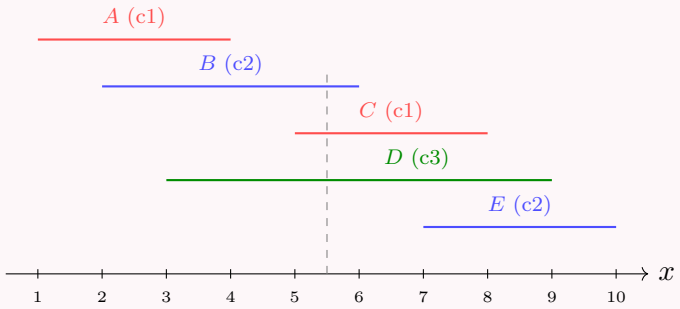
Proof. Process and color in PEO order $v_1, \dots, v_n$. When v_i is colored, its already-colored neighbors are exactly its neighbors among $v_1, \dots, v_{i-1}$ – its *earlier* neighbors. Suppose v_i receives color t. Then greedy was forced to skip colors $1, \dots, t-1$, so v_i has earlier neighbors colored with each of $1, \dots, t-1$; pick one such neighbor for each, giving $t-1$ earlier neighbors of distinct colors. We claim these $t-1$ neighbors are pairwise adjacent. Take two of them, v_p and v_q with $p < q < i$, and apply the PEO at the earlier one, v_p: its later neighbors form a clique. Both v_q and v_i are later neighbors of v_p (each has larger index than p and each is adjacent to v_p), so both lie in that clique, which forces $v_p v_q \in E$. Hence the $t-1$ chosen neighbors together with v_i form a clique of size t. So every color greedy introduces is witnessed by a clique that large, and the number of colors is at most $\omega(G)$. Since $\chi \geq \omega$ always, equality holds and the coloring is optimal. □

Why it works

The reason chordal graphs are easy and general graphs are hard sits in one phrase: in a chordal graph, *every color greedy is forced to introduce is witnessed by a clique of that size.* There is no "wasted" color, no parity surprise like the odd cycle. The clique lower bound and the greedy upper bound coincide. This is the defining feature of *perfect graphs*, and chordal graphs are the most useful perfect subclass.

Trace it yourself

Color this interval graph by hand. Intervals: $A = [1,4]$, $B = [2,6]$, $C = [5,8]$, $D = [3,9]$, $E = [7,10]$. Overlaps: A–B, A–D, B–C, B–D, C–D, C–E, D–E. Sort by right endpoint: $A(4), B(6), C(8), D(9), E(10)$ – that is already a PEO read forward. Greedy: $A \to 1$; B adj $A \to 2$; C adj $B \to 1$; D adj $A, B, C \to 3$; E adj $C, D \to 2$. Three colors. The clique $\{B, C, D\}$ (all overlap near $x = 5.5$) certifies $\omega = 3$, so the coloring is optimal – three was forced.



The dashed line at $x = 5.5$ stabs B, C, D – three pairwise overlapping intervals, the clique that forces three colors.

Application

Interval scheduling / room assignment. Tasks are intervals in time; two tasks conflict iff they overlap; the minimum number of rooms (or machines) needed is exactly χ of the interval graph, which equals the maximum number of tasks live at any instant (ω). The greedy "assign each task to any free room, sorted by start time" is precisely the PEO coloring. This is why airport gate assignment, conference room booking, and bandwidth allocation on a line all admit clean optimal greedy solutions, while their two-dimensional cousins (which are not interval graphs) become hard.

28.7 Coloring Is NP-Hard

Everything tractable in this chapter exploited special structure – two colors, bounded degree, an elimination ordering. Without such structure, coloring is one of the canonical hard problems.

Theorem 28.6 (Hardness of coloring)**.** *Deciding whether $\chi(G) \le k$ is NP-complete for every fixed $k \ge 3$. In particular* 3-Coloring *is NP-complete. Computing $\chi(G)$ exactly is NP-hard, and unless* $\mathsf{P} = \mathsf{NP}$ *no polynomial algorithm approximates $\chi(G)$ within a factor $n^{1-\varepsilon}$ for any $\varepsilon > 0$.*

Intuition

The jump from 2 colors to 3 is one of the sharpest complexity transitions in graph algorithms. Two-coloring reduces to a single BFS, whereas three-coloring is NP-complete. There is no gradual ramp-up in difficulty – tractability breaks immediately at $k = 3$. The reason is expressive power: two colors only propagate forced parities along paths, whereas three colors let you build logical gadgets (a vertex's color encodes a Boolean value, triangles enforce "not equal," and shared vertices propagate constraints) rich enough to simulate any 3-SAT formula.

We sketch the gadget reduction to see why three colors already suffice to encode Boolean logic; the full proof appears in Chapter 37.

Proof idea: 3-SAT $\le_p$ 3-Coloring. Fix three palette colors and name them T, F, N (true, false, neutral). Add a base triangle on three vertices named T, F, N so the names are pinned to actual colors. For each variable x add two vertices $x, \bar{x}$ joined to each other and to N; this forces

$\{c(x), c(\bar{x})\} = \{\mathsf{T}, \mathsf{F}\}$, i.e. a Boolean value. For each clause attach an "OR gadget" (a small fixed graph on six vertices) wiring the three literal vertices to the base triangle so that the gadget is 3-colorable *iff at least one literal is colored* T. The whole graph is 3-colorable iff the formula is satisfiable. The construction is polynomial, so 3-Coloring is NP-hard; membership in NP is clear (a coloring is a checkable certificate). □

Connection

NP-hardness still leaves options. Three approaches recur in this book. *Structure:* bounded-treewidth graphs admit exact coloring by dynamic programming over a tree decomposition in time $O^\star(k^{\mathrm{tw}})$ – see Chapters 24 and 25. *Geometry/sparsity:* planar graphs are 4-colorable (and 5-colorable easily), though 3-colorability stays NP-complete even for planar graphs of bounded degree. *Relaxation:* the *Lovasz theta function* ϑ, a semidefinite program computable in polynomial time, is sandwiched between clique and chromatic data and underlies the $O(n^{0.2})$-coloring of 3-colorable graphs via *vector colorings*, connected to the spectral and SDP relaxations of Chapters 32 and 38.

28.8 Register Allocation and Scheduling

Application

Register allocation. A compiler must map the unboundedly many *variables* of a program onto the few physical *registers* of the CPU. Two variables that are simultaneously *live* – both needed at some future point – cannot share a register. Build the *interference graph*: one vertex per variable, an edge between variables whose live ranges overlap. A proper k-coloring is an assignment of variables to k registers with no clash. So register allocation *is* graph coloring, with k the number of machine registers.

Chaitin's insight (1981) was to attack it with iterated greedy. Repeatedly remove a vertex of degree $< k$ (it can always be colored later, since fewer than k neighbors leave a free register) and push it on a stack – this is exactly building a degeneracy-style ordering. If at some point every remaining vertex has degree $\geq k$, *spill* one variable to memory (delete it from the graph and reload it around its uses), then continue. Pop the stack and color greedily. When live ranges are computed on a structured (reducible) control-flow graph in SSA form, the interference graph is chordal, and then this is the *optimal* chordal coloring of Theorem 28.5 – which is why SSA-based allocators can drop spilling heuristics and color provably optimally in linear time. The $\Delta + 1$ greedy bound, the degeneracy ordering, and the chordal theorem of this chapter are used directly in real compilers.

Application

Timetabling and frequency assignment. Exams that share a student cannot run in the same slot: vertices are exams, edges are shared students, colors are time slots, and χ is the shortest exam timetable. Mobile transmitters too close together cannot reuse a frequency: vertices are transmitters, edges are interfering pairs, colors are frequencies. Both are coloring, both are NP-hard in general, and both are attacked in practice by the methods developed here – greedy on a good (degeneracy or saturation-degree "DSATUR") order, Brooks-style one-color

savings when no clique forces the worst case, and chordal/interval optimality when the conflict structure happens to be an interval graph (e.g. a single linear corridor of transmitters).

Cost summary

Chapter cost summary.

Task	Method	Cost / guarantee
Proper coloring (any)	greedy, fixed order	$O(n+m)$, $\leq \Delta + 1$ colors
Best greedy bound	degeneracy ordering	$O(n+m)$, $\leq$ degen $+1$ colors
2-colorability	BFS	$O(n+m)$, exact + certificate
$\leq \Delta$ colors	Brooks (non-clique, non-odd-cycle)	polynomial, exact bound
Edge coloring	Vizing fans + Kempe chains	$O(nm)$, $\leq \Delta + 1$ colors
Chordal / interval	PEO via Lex-BFS, then greedy	$O(n+m)$, *optimal*
χ exactly, general	(NP-hard) DP on treewidth	$O^\star(k^{\mathrm{tw}})$
$\chi \leq k$, $k \geq 3$	—	NP-complete

Connection

What This Made Possible: Graph coloring gives a mathematical model of pairwise conflicts. Brooks's and Vizing's theorems established tight upper bounds based on local maximum degrees, while Lex-BFS and Perfect Elimination Orderings gave exact linear-time coloring for chordal and interval graphs. Gregory Chaitin (1981) applied graph coloring to compiler register allocation, replacing ad-hoc heuristics with a principled graph algorithm.

Limitations:

- *Hardness of approximation:* Deciding whether a planar graph of maximum degree 4 is 3-colorable is NP-complete (Garey, Johnson, Stockmeyer 1976). On general graphs, coloring cannot be approximated within $n^{1-\varepsilon}$ for any $\varepsilon > 0$ unless P = NP (Zuckerman 2007).
- *Order-sensitivity of greedy heuristics:* As shown in Exercise 28.1, a bad ordering can force the greedy algorithm to use $n/2$ colors on a graph with $\chi(G) = 2$.
- *Local vs. global gap:* A graph can have no triangles ($\omega(G) = 2$) yet have an arbitrarily large chromatic number $\chi(G)$ (Mycielski's construction), showing that local clique checks cannot detect global coloring obstructions.

Where We Go Next: A proper coloring partitions the vertex set into independent sets. How do we find the largest possible independent set, or dually, the largest clique? In **Chapter 29 (Independent Sets, Cliques, and Vertex Covers)**, we examine exact algorithms and the Bron–Kerbosch pivoting algorithm. Then, for which broad family of graphs is the chromatic number $\chi(H)$ always equal to the clique number $\omega(H)$ on every induced subgraph? That is the subject of **Chapter 30 (Perfect Graphs)**.

28.9 Exercises

Exercise 28.1 (Greedy adversary)**.** The *crown graph* S_n^0 is $K_{n,n}$ with a perfect matching removed. Prove $\chi(S_n^0) = 2$, then exhibit a vertex order on which Algorithm 114 uses n colors. Conclude that the gap between greedy's worst order and χ is unbounded.

Exercise 28.2 (Degeneracy versus max degree)**.** Show $\text{degen}(G) \le \Delta(G)$ always, and construct, for each d, a graph with degen $= d$ but Δ arbitrarily large. Then prove every simple planar graph is 5-degenerate (use that it has $\le 3n - 6$ edges, so average degree < 6), and deduce the six-color theorem for planar graphs.

Exercise 28.3 (Odd-cycle certificate)**.** Modify Algorithm 116 so that, on failure, it outputs an actual odd cycle (not merely the conflicting edge). Prove your reconstruction is correct and runs in $O(n + m)$. Why does this certificate constitute a co-NP-style proof of non-2-colorability?

Exercise 28.4 (Brooks by hand)**.** Take the 3-cube graph Q_3 ($\Delta = 3$, 3-regular, bipartite). Brooks guarantees $\chi \le 3$; bipartiteness guarantees $\chi = 2$. Reconcile these and exhibit the 2-coloring. Then find a connected 3-regular graph that is *not* bipartite and color it with 3 colors using the ordering trick from the proof of Theorem 28.2.

Exercise 28.5 (Vizing classes)**.** Compute χ' for K_4, K_5, the Petersen graph, and C_5. Which are Class 1 ($\chi' = \Delta$) and which Class 2 ($\chi' = \Delta + 1$)? Prove that every connected Δ-regular graph with an *odd* number of vertices is Class 2. (Hint: a color class is a matching, which saturates an even number of vertices.)

Exercise 28.6 (Line graphs and edge coloring)**.** Prove $\chi'(G) = \chi(L(G))$ where L is the line graph. Using Vizing, deduce $\chi(L(G)) \le \Delta(G) + 1$. Give an example where $L(G)$ is *not* perfect, showing edge coloring's good behavior is special to line graphs and not a consequence of perfection.

Exercise 28.7 (Simplicial vertices)**.** Prove that every chordal graph with at least two vertices has at least two simplicial vertices, and that if it is not complete the two can be chosen non-adjacent. Use this to give an alternative inductive proof of Theorem 28.5 that does not invoke Lex-BFS.

Exercise 28.8 (Interval coloring is room scheduling)**.** Given n intervals, design an $O(n \log n)$ algorithm that assigns each a color (room) using exactly the maximum overlap depth ω many colors. Prove optimality by exhibiting a point stabbed by ω intervals. Where does the argument break if the "intervals" are arcs on a circle rather than segments on a line?

Exercise 28.9 (Mycielski's triangle-free tower)**.** The Mycielskian $M(G)$ adds, for each vertex v, a twin v' joined to v's neighbors, plus one apex w joined to all twins. Prove $M(G)$ is triangle-free if G is, and that $\chi(M(G)) = \chi(G) + 1$. Starting from K_2, build a triangle-free graph with $\chi = 4$ and report its vertex count.

Exercise 28.10 (3-coloring gadget)**.** Design the OR gadget used in Theorem 28.6: a small graph attached to the base triangle and to three literal vertices that is 3-colorable iff at least one literal is colored T. Verify your gadget by checking all 8 truth assignments of the literals. Conclude the reduction is polynomial in formula size.

Exercise 28.11 (Register allocation with spilling)**.** Implement Chaitin-style allocation: given an interference graph and k registers, repeatedly remove a vertex of degree $< k$ (stack it); if stuck, spill a highest-degree vertex; finally pop and color. Prove that if the interference graph is chordal, the "stuck" case never arises for $k = \omega$, so no spilling is needed and the allocation is optimal. Connect this to Theorem 28.5.

Chapter 29

Independent Sets, Cliques, and Vertex Covers

In the preceding chapter, we saw that coloring a graph partitions its vertices into conflict-free subsets. Each color class is an independent set. Naturally, this leads to an optimization question: how large can a single conflict-free set be? Three fundamental optimization problems on a graph turn out to be direct translations of one another. Pick a graph. What is the largest set of vertices with *no* edge between any two of them? What is the largest set of vertices with *every* edge present between them? And what is the smallest set of vertices that *touches* every edge? The first is the *independent set*, the second the *clique*, the third the *vertex cover*. A solution to any one of them gives, with two lines of reasoning, solutions to all three.

These three problems are among the most basic NP-hard problems on graphs. There is no flow trick, no augmenting path, no spanning-tree exchange that finds a maximum clique in polynomial time. But vertex cover admits three algorithmic approaches: a factor-2 approximation, a half-integral linear programming relaxation, and a fixed-parameter tractable branch-and-bound algorithm when the cover is small. Here we examine all three tools on this shared foundation.

History

The trio of independent set, clique, and vertex cover represents the historical core of combinatorial optimization. In 1959, Hungarian mathematician Tibor Gallai established the fundamental identities $\alpha(G)+\tau(G) = n$ and $\alpha'(G)+\tau'(G) = n$, proving that maximum independent sets and minimum vertex covers are algebraic complements. In 1972, Richard Karp included all three problems in his foundational list of 21 NP-complete problems, proving that exact polynomial-time optimization is intractable in general.

Early heuristics attempted greedy degree-based selection, but such choices can get trapped in small sub-optimal sets. In 1965, J.W. Moon and Leo Moser proved that an n-vertex graph can contain up to $3^{n/3} \approx 1.442^n$ maximal cliques. In 1973, Coenraad Bron and Joep Kerbosch introduced their classic recursive backtracking algorithm with a pivot rule that avoids redundant searches and matches the Moon–Moser worst-case bound.

Approximation and linear programming opened a new chapter. In the 1970s, Fanica Gavril and Mihalis Yannakakis observed that a maximal matching immediately yields a 2-approximation for vertex cover by taking both endpoints of each matched edge. George Nemhauser and Leslie

Trotter (1975) proved that the standard LP relaxation is half-integral: every variable takes value in $\{0, \frac{1}{2}, 1\}$, and variables assigned integer values are guaranteed to agree with some optimal integer solution, providing an exact kernelization reduction. Whether vertex cover can be approximated within $2 - \varepsilon$ remains a major open problem, conjectured to be NP-hard under Subhash Khot's Unique Games Conjecture (2002).

29.1 Three Problems and Their Complements

Fix a simple undirected graph $G = (V, E)$ with $n = |V|$ vertices and $m = |E|$ edges. We need three definitions, and then one picture ties them together for the rest of the chapter.

Definition 29.1 (Independent set)**.** A set $S \subseteq V$ is *independent* (or *stable*) if no edge of G has both endpoints in S: for all $u, v \in S$, $\{u, v\} \notin E$. The *independence number* $\alpha(G)$ is the size of a largest independent set.

Definition 29.2 (Clique)**.** A set $K \subseteq V$ is a *clique* if every pair of its vertices is joined by an edge: for all distinct $u, v \in K$, $\{u, v\} \in E$. The *clique number* $\omega(G)$ is the size of a largest clique.

Definition 29.3 (Vertex cover)**.** A set $C \subseteq V$ is a *vertex cover* if every edge has at least one endpoint in C: for all $\{u, v\} \in E$, $u \in C$ or $v \in C$. The *vertex cover number* $\tau(G)$ is the size of a smallest vertex cover.

Intuition

Read the three definitions as three sentences about edges. An independent set says "no edge lives inside me." A clique says "every possible edge lives inside me." A vertex cover says "no edge escapes me." The first and third are about the *same* edges: if S contains no edge, then every edge must have an endpoint outside S, i.e. in the complement $V \setminus S$. This observation is the Gallai identity below.

Here is the running graph for the chapter. Call it G_0 (Figure 29.1): six vertices, drawn so that the three structures are visible at a glance.

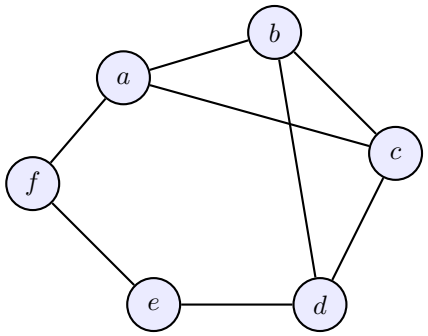


Figure 29.1: The running graph G_0 with eight edges. The triangles $\{a, b, c\}$ and $\{b, c, d\}$ are its cliques of size 3; the path $f\,e\,d$ and the pendant edges af, de, ef hang off them.

The triangle $\{a, b, c\}$ is a clique of size 3. The set $\{a, d\}$ is independent (no edge between them), and so is $\{b, e\}$. Let us not give away all the answers yet; we trace them by hand in Section 29.2.2. First, we relate the three problems.

29.1.1 Independent set = clique in the complement

Definition 29.4 (Complement graph)**.** The *complement* $\overline{G} = (V, \overline{E})$ has the same vertices and exactly the non-edges of G: $\{u, v\} \in \overline{E}$ if and only if $u \neq v$ and $\{u, v\} \notin E$.

Proposition 29.1 (Independent–clique duality)**.** *A set $S \subseteq V$ is independent in G if and only if S is a clique in $\overline{G}$. Consequently $\alpha(G) = \omega(\overline{G})$ and $\omega(G) = \alpha(\overline{G})$.*

Proof. S independent in G means no pair inside S is an edge of G, i.e. every pair inside S is a non-edge of G, i.e. every pair inside S is an edge of $\overline{G}$ – which is exactly the statement that S is a clique in $\overline{G}$. The numerical identities follow by taking the largest such S, and the second comes from $\overline{\overline{G}} = G$. □

This duality has *algorithmic* consequences. Any algorithm for maximum clique runs unchanged as a maximum independent set algorithm on the complement, and vice versa. The Bron–Kerbosch enumeration we build in Section 29.2 is universally phrased for cliques; feed it $\overline{G}$ and it lists maximal independent sets. The only cost is that complementation flips edge density: a sparse G has a dense $\overline{G}$, which matters for the running time but never for correctness.

29.1.2 Gallai: cover and independent set partition the vertices

Theorem 29.1 (Gallai's identity)**.** *$S \subseteq V$ is independent in G if and only if its complement $C = V \setminus S$ is a vertex cover. Hence for every graph on n vertices,*

$$\alpha(G) + \tau(G) = n.$$

Proof. Suppose S is independent. Take any edge $\{u, v\} \in E$. Since S contains no edge, u and v are not both in S, so at least one lies in $C = V \setminus S$. Thus C covers every edge: C is a vertex cover. Conversely, suppose C is a vertex cover and let $S = V \setminus C$. If some edge $\{u, v\}$ had both endpoints in S, then *neither* endpoint is in C, contradicting that C covers $\{u, v\}$. So S is independent.

The map $S \mapsto V \setminus S$ is a bijection between independent sets and vertex covers that sends a set of size k to a set of size $n - k$. A largest independent set therefore corresponds to a smallest vertex cover, giving $\alpha(G) = n - \tau(G)$ (Figure 29.2). □

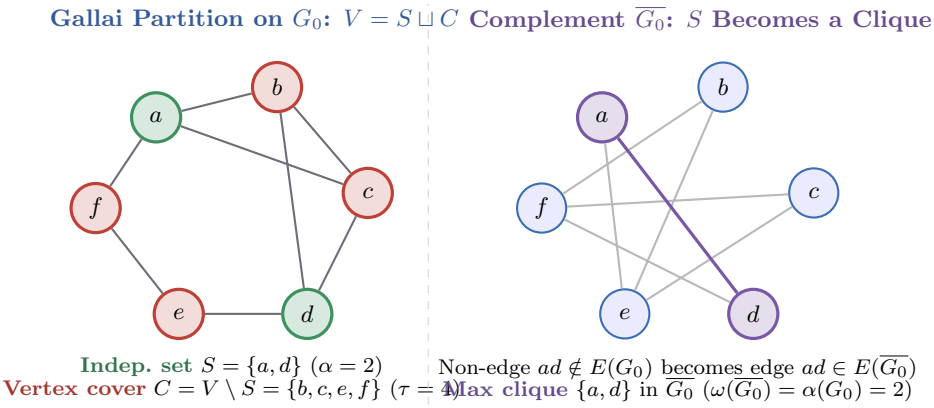


Figure 29.2: The two structural relationships on G_0. **Left (Gallai):** Every edge of G_0 has at least one red endpoint in the cover $C = \{b, c, e, f\}$, leaving the green independent set $S = \{a, d\}$ completely edge-free ($\alpha + \tau = 2 + 4 = 6$). **Right (Complement):** Flipping edges and non-edges turns the independent set $S = \{a, d\}$ of G_0 into a clique in $\overline{G_0}$.

Focus

Three numbers, one identity each, all on G_0:

$$\alpha(G_0) + \tau(G_0) = 6, \qquad \alpha(G_0) = \omega(\overline{G_0}), \qquad \omega(G_0) = \alpha(\overline{G_0}).$$

The first is Gallai (complement *within* G_0); the second and third are the complement-graph duality (a *different* graph). Do not conflate "the complement of a set" with "the complement of a graph." Gallai uses the former, Proposition 29.1 the latter.

Common trap

Gallai's $\alpha + \tau = n$ is exact and holds for *every* graph – it is a genuine equation, not an inequality or an approximation. The temptation is to expect a parallel identity tying ω to the other two. There is none of the same simplicity: ω relates to the rest only through the complement graph, where $\omega(G) = \alpha(\overline{G}) = n - \tau(\overline{G})$. So clique size is a statement about $\overline{G}$'s cover, not G's. Mixing these up produces "identities" that are false on the very first triangle you test.

Connection

On *bipartite* graphs these three problems become tractable. König's theorem (Chapter 15) says $\tau(G) = \nu(G)$, the size of a maximum matching, whenever G is bipartite. Through Gallai, $\alpha(G) = n - \nu(G)$, so maximum independent set on bipartite graphs is just maximum matching – solvable by augmenting paths in polynomial time. The hardness we meet below is therefore a non-bipartite, odd-cycle phenomenon, the same obstruction that forced Edmonds to invent blossoms for general matching.

29.2 Enumerating Cliques: Bron–Kerbosch

Finding one maximum clique is NP-hard (Section 29.4). But many applications – community detection, computational chemistry, the analysis of social networks – want *all* maximal cliques, and there a simple recursion does the job. The Bron–Kerbosch algorithm enumerates every maximal clique exactly once, and on real instances it is fast despite the worst case being unavoidable (a graph can have $3^{n/3}$ maximal cliques, and you cannot list them in less time than it takes to write them down).

Definition 29.5 (Maximal vs. maximum)**.** A clique is *maximal* if no vertex can be added to it while keeping it a clique. It is *maximum* if it is the largest clique in the graph. Maximum implies maximal; the converse fails – a maximal clique can be small if it is "stuck."

Intuition

The recursion grows a clique vertex by vertex. It keeps three sets: R, the vertices already committed to the current clique; P, the candidates that are adjacent to all of R and could still extend it; and X, vertices that are also adjacent to all of R but have *already* been used as the start of some branch. When both P and X are empty, R cannot be extended by anyone – it is maximal, and we report it. X is the bookkeeping that prevents the same maximal clique from being printed twice: it remembers "we already explored that direction."

Algorithm 118: BronKerbosch(R, P, X) – enumerate maximal cliques

input : committed set R, candidates P, excluded X; initial call $(\emptyset, V, \emptyset)$
output: every maximal clique, each printed once

```
if P = ∅ and X = ∅ then
    report R as a maximal clique
    return
choose a pivot u ∈ P ∪ X maximizing |P ∩ N(u)|
foreach v ∈ P \ N(u) do
    BronKerbosch(R ∪ {v}, P ∩ N(v), X ∩ N(v))
    P ← P \ {v}
    X ← X ∪ {v}
```

Here $N(u)$ is the open neighborhood of u. The recursion branches on v, moves v into the committed set R, and restricts the candidate and excluded sets to v's neighbors – because any clique containing $R \cup \{v\}$ can only use vertices adjacent to v. After exploring the branch through v, we retire v from P into X: future branches must avoid v, or they would re-list cliques already found.

29.2.1 The Pivot: Pruning Redundant Search Branches

The naive version omits the pivot and simply loops over all $v \in P$. It is correct but slow, because it explores branches that cannot possibly yield a *new* maximal clique. The pivot fixes this.

Why it works

Fix any maximal clique $K \supseteq R$ that we still hope to find from (R, P, X). Let $u \in P \cup X$ be the pivot. Either $u \in K$, or some non-neighbor of u is in K – because K is maximal, so if u

itself were addable it would be forced in, meaning K must contain a vertex that blocks u, i.e. a non-neighbor of u. In both cases K contains a vertex of $\{u\} \cup (P \setminus N(u))$. The branches we actually take are exactly $P \setminus N(u)$ (and the pivot's own fate is handled because u stays available in those sub-branches). So looping over $P \setminus N(u)$ misses nothing: every maximal clique reachable from here passes through one of these branches. Choosing u to *maximize* $|P \cap N(u)|$ makes $P \setminus N(u)$ as small as possible – the fewest branches.

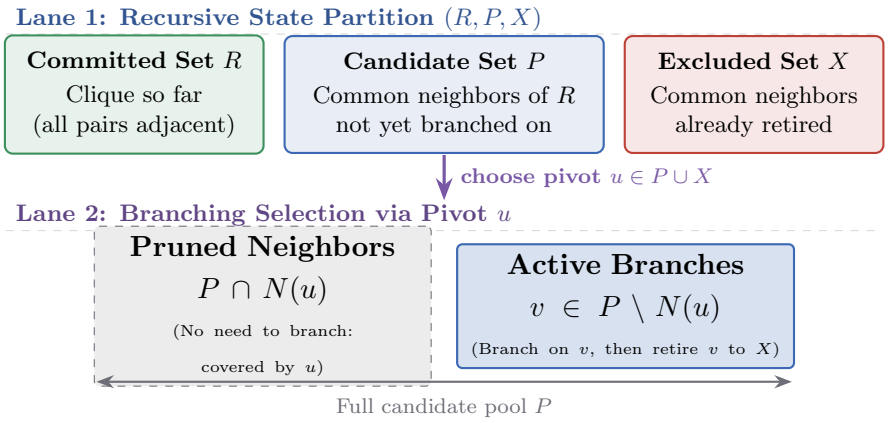


Figure 29.3: The Bron–Kerbosch state partition and pivot optimization. **Top (Lane 1):** The vertex set is divided into committed clique vertices R, candidates P adjacent to all of R, and already-explored vertices X. **Bottom (Lane 2):** Selecting a pivot u maximizing $|P \cap N(u)|$ proves that every maximal clique containing R either contains u or a non-neighbor of u. The algorithm branches only on the restricted set $v \in P \setminus N(u)$, pruning all neighbors of u from branching and drastically collapsing the search tree.

Lemma 29.1 (Correctness of Bron–Kerbosch)**.** *Algorithm 118 reports a set R if and only if R is a maximal clique, and reports each maximal clique exactly once.*

Proof. Invariant. At every call (R, P, X), the set R is a clique, and $P \cup X$ is exactly the set of vertices adjacent to every vertex of R (the common neighborhood). P holds those not yet branched on at an ancestor; X holds those already retired. This holds at the root $(\emptyset, V, \emptyset)$ and is preserved by the recursion, since restricting to $N(v)$ keeps precisely the common neighbors of $R \cup \{v\}$, and the moves $P \to X$ respect the split.

Reports only maximal cliques. When $P = X = \emptyset$, no vertex is adjacent to all of R, so R cannot be extended: it is maximal.

Reports every maximal clique once. Let K be maximal. Existence of a branch that builds it follows from the pivot argument above: at each level a vertex of K lies in the looped set $P \setminus N(u)$, so the recursion can descend into K, committing its vertices one at a time until $P = X = \emptyset$ and $R = K$. Uniqueness follows from X: once a vertex v is retired into X, every descendant call carries v in its X component whenever v is a common neighbor, so the maximality test $X = \emptyset$ forbids re-reporting any clique that could have included v earlier. Hence each maximal clique is emitted in exactly one leaf. □

29.2.2 Tracing it on G_0

Take the running graph. The adjacency lists are

$$N(a) = \{b, c, f\}, \quad N(b) = \{a, c, d\}, \quad N(c) = \{a, b, d\},$$
$$N(d) = \{b, c, e\}, \quad N(e) = \{d, f\}, \quad N(f) = \{a, e\}.$$

Start with $R = \emptyset$, $P = \{a, b, c, d, e, f\}$, $X = \emptyset$. Pick a pivot in $P \cup X$ maximizing $|P \cap N(u)|$: vertices a, b, c, d each have degree 3, so any of them ties; take $u = c$, giving $P \cap N(c) = \{a, b, d\}$ and a loop over $P \setminus N(c) = \{c, e, f\}$ (note c itself is a non-neighbor of itself under the open-neighborhood convention, so it appears in the loop). The loop processes c, then e, then f, retiring each into X as it goes.

The table below follows the recursion in execution order, one row per call. The *depth* column tracks how deep in the recursion the call sits; the *pivot, loop* column shows the pivot chosen at that call and the resulting loop set; a leaf with empty P and X reports the clique R.

depth	v	call $(R; P; X)$	pivot, loop	event
0	–	$(\emptyset;\ abcdef;\ \emptyset)$	c, loop $\{c, e, f\}$	
1	c	$(c;\ abd;\ \emptyset)$	b, loop $\{b\}$	
2	b	$(bc;\ ad;\ \emptyset)$	a, loop $\{a, d\}$	
3	a	$(abc;\ \emptyset;\ \emptyset)$	–	**report** $\{a, b, c\}$
3	d	$(bcd;\ \emptyset;\ \emptyset)$	–	**report** $\{b, c, d\}$
1	e	$(e;\ df;\ \emptyset)$	d, loop $\{d, f\}$	
2	d	$(de;\ \emptyset;\ \emptyset)$	–	**report** $\{d, e\}$
2	f	$(ef;\ \emptyset;\ \emptyset)$	–	**report** $\{e, f\}$
1	f	$(f;\ a;\ e)$	a, loop $\{a\}$	
2	a	$(af;\ \emptyset;\ \emptyset)$	–	**report** $\{a, f\}$

So the full census of maximal cliques of G_0 is

$$\{a, b, c\}, \quad \{b, c, d\}, \quad \{d, e\}, \quad \{e, f\}, \quad \{a, f\},$$

five in all, each reported once. The triangles $\{a, b, c\}$ and $\{b, c, d\}$ are the two maximum cliques, so $\omega(G_0) = 3$. Note the effect of the pivot. At the root, choosing c keeps only the three branches $\{c, e, f\}$ instead of all six vertices; both triangles then fall out of the single c-branch, because once c is committed the candidate set $\{a, b, d\}$ already contains everything the triangles need. The excluded set X prevents duplicate cliques in the last branch: the call $(f;\ a;\ e)$ carries e in X because e is a common neighbor of f already explored through the e-rooted branch, so the clique $\{e, f\}$ found there is not reported a second time from underneath f.

The complement $\overline{G_0}$ makes the independent-set side visible. Its edges are exactly the seven non-edges of G_0, and a maximal clique of $\overline{G_0}$ is a maximal independent set of G_0 (Proposition 29.1). Drawn below, $\overline{G_0}$ has no triangle, so its largest clique – and hence $\alpha(G_0)$ – is just 2.

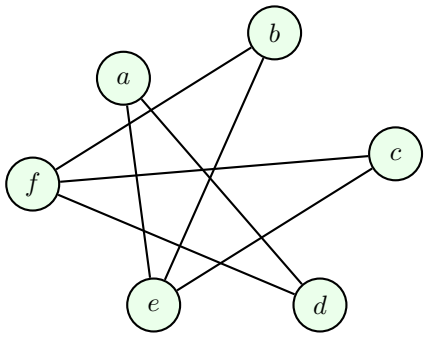


Figure 29.4: The complement $\overline{G_0}$: its seven edges are the non-edges of G_0. It is triangle-free, so $\omega(\overline{G_0}) = \alpha(G_0) = 2$.

Trace it yourself

Re-run the recursion yourself, this time choosing a different root pivot among the degree-3 vertices (say a instead of c) and confirm you still recover exactly the five cliques above with no repeats – X is precisely what suppresses those duplicates. Then take the complement $\overline{G_0}$ of Figure 29.4 and re-run Bron–Kerbosch on it: the maximal cliques you find are the maximal *independent sets* of G_0. You should find that every maximal independent set has size 2 – for instance $\{a, d\}$ and $\{a, e\}$ both fail to extend – so $\alpha(G_0) = 2$. Which vertex blocks each attempt to grow $\{a, e\}$ to size three?

29.2.3 Complexity

Theorem 29.2 (Tomita–Tanaka–Takahashi bound)**.** *With pivoting, Bron–Kerbosch runs in $O(3^{n/3})$ time.*

Idea. The number of leaves of the recursion tree is bounded by the number of maximal cliques, and Moon and Moser proved a graph on n vertices has at most $3^{n/3}$ maximal cliques (a disjoint union of triangles achieves it). The pivot guarantees each internal node's branching factor is controlled so that the total work is within a polynomial factor of the leaf count, and a careful accounting collapses that to $O(3^{n/3})$ overall. □

Cost summary

Bron–Kerbosch with pivot. Worst-case time $O(3^{n/3})$, optimal in the output-size sense (you cannot beat the number of cliques you must print). Memory is $O(n^2)$ for the adjacency structure plus $O(n)$ recursion depth times the set representations. A degeneracy-ordering variant achieves $O(d\, n\, 3^{d/3})$ where d is the graph's *degeneracy*, near-linear in n for sparse graphs of small degeneracy – the version used in practice on large social networks. Representing P, X as bitsets gives a constant-factor gain that matters a great deal in practice.

Application

Bron–Kerbosch is the standard algorithm behind clique-based community detection, protein–protein interaction analysis, and the "maximum common subgraph" matching used in cheminformatics (mapping one molecule's structure onto another's reduces to finding large cliques in a product graph). The degeneracy-ordered variant of Eppstein, Löffler, and Strash (2010) is what lets it scale to graphs with millions of edges.

29.3 Vertex Cover: The LP Relaxation and Half-Integrality

Vertex cover is the most tractable of the three, because of a linear program with a useful property: its optimal solutions can always be taken to use only the values 0, $\frac{1}{2}$, and 1. This half-integrality underlies both the 2-approximation and the parameterized algorithm, so we build it carefully.

Assign a variable $x_v \in \{0,1\}$ to each vertex, meaning "v is in the cover." The covering requirement – every edge has an endpoint in C – is $x_u + x_v \geq 1$ for each edge $\{u,v\}$. Minimizing the size of the cover is minimizing $\sum_v x_v$. So the exact problem is the integer program

$$\tau(G) \;=\; \min \sum_{v \in V} x_v \quad \text{s.t.} \quad x_u + x_v \geq 1 \;\; \forall \{u,v\} \in E, \quad x_v \in \{0,1\}. \tag{29.1}$$

The integrality constraint $x_v \in \{0,1\}$ is what makes this NP-hard. Relax it to $x_v \geq 0$ – the constraints never push a variable above 1, so $[0,1]$ and $[0,\infty)$ give the same optimum – and we get the *LP relaxation*:

$$\tau^*(G) \;=\; \min \sum_{v \in V} x_v \quad \text{s.t.} \quad x_u + x_v \geq 1 \;\; \forall \{u,v\} \in E, \quad x_v \geq 0. \tag{29.2}$$

Any integral cover is a feasible LP point, so $\tau^*(G) \leq \tau(G)$: the LP *lower-bounds* the true cover. Bounding the gap between them is the central task in LP-based approximation.

Intuition

A fractional cover sounds nonsensical – you cannot put half a guard on a vertex. But geometrically the LP optimum sits at a vertex of the feasible polytope, and the structure of that polytope forces a strong restriction: every coordinate of an optimal vertex is 0, $\frac{1}{2}$, or 1. The half-values are exactly the odd cycles, where you genuinely cannot decide integrally without "spending" a little extra. A triangle needs 2 integral guards but only $\frac{3}{2}$ fractional ones – three guards at weight $\frac{1}{2}$ each. Those $\frac{1}{2}$ weights appear precisely because the cycle has odd length.

Theorem 29.3 (Nemhauser–Trotter half-integrality)**.** *The LP (29.2) has an optimal solution $x^\star$ in which every $x_v^\star \in \{0, \frac{1}{2}, 1\}$.*

Proof. Let x be any optimal solution. Split the vertices by value: $V_+ = \{v : x_v > \frac{1}{2}\}$ and $V_- = \{v :$

$x_v < \frac{1}{2}\}$, leaving the rest at exactly $\frac{1}{2}$. Pick a small $\varepsilon > 0$ and define two perturbations:

$$y_v = \begin{cases} x_v + \varepsilon & v \in V_+, \\ x_v - \varepsilon & v \in V_-, \\ x_v & \text{else}, \end{cases} \qquad z_v = \begin{cases} x_v - \varepsilon & v \in V_+, \\ x_v + \varepsilon & v \in V_-, \\ x_v & \text{else}. \end{cases}$$

For ε small enough both y and z stay in $[0, 1]$. Check feasibility of each edge $\{u, v\}$. If both endpoints sit at $\frac{1}{2}$, the constraint $x_u + x_v = 1$ is untouched. If one endpoint is in V_- (value $< \frac{1}{2}$), then to satisfy $x_u + x_v \geq 1$ the other endpoint must be in V_+ (value $> \frac{1}{2}$); in y the V_- end drops by ε but the V_+ end rises by ε, so the sum is preserved – and likewise in z. In every remaining edge neither endpoint lies in V_-, so both endpoints are $\geq \frac{1}{2}$ with at least one strictly above $\frac{1}{2}$; that edge has *slack* $x_u + x_v > 1$, and for ε small enough the at-most-2ε change in either direction keeps the sum ≥ 1. Hence y and z are both feasible.

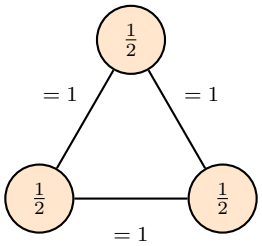
Now $x = \frac{1}{2}(y + z)$, and the objective is linear, so $\sum_v x_v = \frac{1}{2}(\sum_v y_v + \sum_v z_v)$. Since x is optimal (minimal), neither y nor z can be strictly smaller, so both are also optimal. Now push ε up along y: the V_+ coordinates rise and the V_- coordinates fall, so y stays optimal until the first moment some V_+ coordinate hits 1 or some V_- coordinate hits 0. At that boundary we have a new optimal solution with at least one more coordinate pinned at $\{0, 1\}$ and none unpinned from $\{0, \frac{1}{2}, 1\}$. Drop the pinned coordinates and repeat on the rest; the process terminates at an optimum entirely supported on $\{0, \frac{1}{2}, 1\}$. □

Why it works

The proof is a *rounding-toward-the-extremes* argument. Whenever a fractional solution has wiggle room – variables strictly between $0, \frac{1}{2}, 1$ – you can trade weight between the high group and the low group without changing the objective or breaking any constraint, because every binding edge pairs a high vertex with a low one. Each trade drives a variable to a half-integral value and never increases cost. Linearity guarantees the average of two optima is optimal, and the whole argument relies on this.

29.3.1 Reading the half-integral LP on a triangle

Take the triangle $T = \{a, b, c\}$ with all three edges. Constraints: $x_a + x_b \geq 1$, $x_b + x_c \geq 1$, $x_a + x_c \geq 1$. The integral optimum is any two vertices, cost 2. The fractional optimum sets $x_a = x_b = x_c = \frac{1}{2}$: every constraint reads $\frac{1}{2} + \frac{1}{2} = 1$, tight, with objective $\frac{3}{2}$. So $\tau^*(T) = \frac{3}{2} < 2 = \tau(T)$ – the integrality gap is real, here exactly $\frac{4}{3}$. The $\frac{1}{2}$'s are forced precisely because the triangle is an *odd* cycle.



Connection

On bipartite graphs the LP (29.2) is *integral*: every vertex of the polytope is a 0/1 point, no $\frac{1}{2}$'s appear, and $\tau^* = \tau$. This is the LP form of König's theorem, and it is not a coincidence – the constraint matrix of vertex cover is the incidence matrix of G, which is totally unimodular exactly when G is bipartite. The odd cycles that break total unimodularity are the same odd cycles that produce $\frac{1}{2}$-valued extreme points. Total unimodularity of the incidence matrix is the exact algebraic counterpart of bipartiteness.

29.4 Hardness and the Factor-2 Approximation

29.4.1 Why these problems are hard

Theorem 29.4 (NP-completeness)**.** *Deciding whether G has a vertex cover of size $\leq k$ is NP-complete; so is deciding $\alpha(G) \geq k$ and $\omega(G) \geq k$.*

Sketch. Vertex cover is in NP – a candidate cover is checkable in $O(m)$ time. For hardness, reduce from 3-SAT via the classical clause-and-variable gadget (Karp, 1972): each variable becomes an edge (forcing one of two truth literals into the cover), each clause becomes a triangle (forcing two of its three literal vertices in), and consistency edges tie literals to clauses; a formula is satisfiable iff a cover of a specific size exists. By Gallai (Theorem 29.1), independent set is then NP-complete (it is the set complement), and by Proposition 29.1 clique is too (it is independent set on the complement graph). □

Common trap

The three problems are equivalent under *polynomial-time* reductions, so all are NP-hard. But they are *not* equivalent under *approximation*. Vertex cover has a 2-approximation. Independent set and clique have *no* constant-factor approximation unless P = NP – in fact no $n^{1-\varepsilon}$ approximation (Håstad). The Gallai bijection preserves exact optima but wrecks ratios: a cover that is 2 times too big leaves an independent set that can be arbitrarily far from α. "Equivalent" is reduction-relative; always state the reduction.

29.4.2 The matching lower bound and the greedy 2-approximation

The cleanest 2-approximation for vertex cover does not even look at the LP – it just grabs both endpoints of edges in a maximal matching. The LP gives a second, rounding-based route to the same factor; we present both because each generalizes differently.

Algorithm 119: MATCHINGVC(G) – 2-approximate vertex cover

```
C ← ∅; E′ ← E
while E′ ≠ ∅ do
    pick any edge {u, v} ∈ E′
    C ← C ∪ {u, v}
    remove from E′ every edge incident to u or v
return C
```

Lemma 29.2 (Correctness and ratio)**.** MATCHINGVC *returns a vertex cover C with $|C| \le 2\,\tau(G)$, in $O(n+m)$ time.*

Proof. *Cover.* The loop ends only when $E' = \emptyset$, i.e. every edge has been removed; an edge is removed only when one of its endpoints entered C. So every edge has an endpoint in C.

Ratio. The edges picked in the loop form a *matching* M: each picked edge's endpoints are added to C and all their incident edges deleted, so no two picked edges share a vertex. Now $|C| = 2|M|$, since each picked edge contributes two fresh vertices. The key bound: *any* vertex cover must contain at least one endpoint of each edge of M, and since the edges of M are vertex-disjoint those endpoints are distinct – so $\tau(G) \ge |M|$. Combining, $|C| = 2|M| \le 2\,\tau(G)$.

Time. Each edge is examined a constant number of times (once when scanned, once when deleted as incident), giving $O(n+m)$ with adjacency lists (Figure 29.5). □

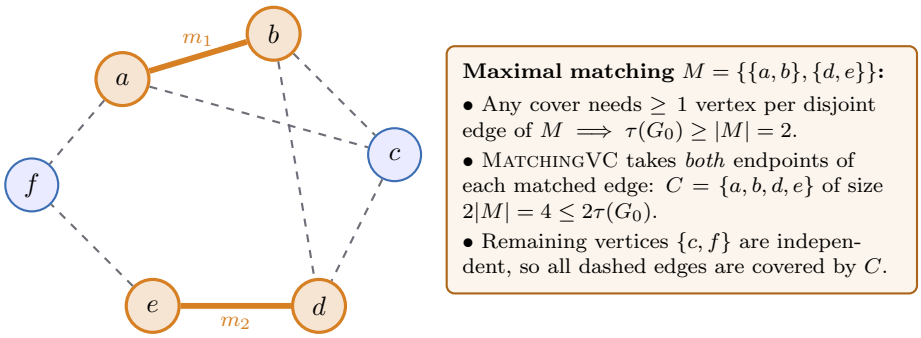


Figure 29.5: MATCHINGVC on G_0. Picking vertex-disjoint edges $m_1 = \{a,b\}$ and $m_2 = \{d,e\}$ forms a maximal matching M of size 2 and returns the 4-vertex cover $C = \{a,b,d,e\}$, certifying the 2-approximation sandwich $|M| \le \tau(G_0) \le 2|M|$.

Why it works

The lower bound is the key step, and it is a small instance of duality. A matching is a set of edges no cover can handle with a single shared vertex; it forces the cover to spend one vertex *per matching edge*. So $\nu(G) \le \tau(G)$ always. The algorithm pays two vertices per matching edge, the cover must pay at least one, hence the factor 2. The slack – paying 2 where one might suffice – is exactly the integrality gap of the LP, and it cannot be closed by any greedy trick: 2 is, under the Unique Games Conjecture, the best constant achievable in polynomial time.

29.4.3 The LP-rounding 2-approximation

The half-integral LP gives the same guarantee with a one-line rounding rule, and its analysis is the template for many other LP-rounding approximations.

Algorithm 120: RoundLPVC(G) – LP rounding

```
solve LP (29.2) for an optimum x* (half-integral, Thm 29.3)
C ← {v : x*_v ≥ 1/2}
return C
```

Lemma 29.3 (LP rounding ratio)**.** *$C = \{v : x^\star_v \geq \frac{1}{2}\}$ is a vertex cover with $|C| \leq 2\,\tau^*(G) \leq 2\,\tau(G)$.*

Proof. Cover. For any edge, $x^\star_u + x^\star_v \geq 1$ forces at least one of $x^\star_u, x^\star_v$ to be $\geq \frac{1}{2}$, so at least one endpoint is in C. *Ratio.* Rounding $x^\star_v \geq \frac{1}{2}$ up to 1 at most doubles its contribution: $1 \leq 2x^\star_v$ when $x^\star_v \geq \frac{1}{2}$, and vertices with $x^\star_v < \frac{1}{2}$ contribute $0 \leq 2x^\star_v$. Summing, $|C| = \sum_{v \in C} 1 \leq \sum_v 2x^\star_v = 2\tau^*(G) \leq 2\tau(G)$. □

Cost summary

Vertex cover approximations. Both MatchingVC and RoundLPVC give ratio 2. MatchingVC runs in $O(n+m)$ with no LP solver. RoundLPVC costs an LP solve, but the vertex-cover LP can be solved combinatorially as a bipartite matching / min-cut on a doubled graph in $O(m\sqrt{n})$, so the half-integral optimum is reachable without a general LP library. The best known ratio is $2 - \Theta\big(1/\sqrt{\log n}\big)$ (Karakostas); no polynomial algorithm beats 1.36 unless P = NP, and none beats $2 - \varepsilon$ under the Unique Games Conjecture.

29.5 The Parameterized Angle: Vertex Cover Is FPT

NP-hardness says vertex cover is hard *in* n. But many applications only care about *small* covers – a network where a handful of nodes touch every link, a conflict graph where a few deletions resolve all conflicts. If the cover size k is small, can we solve the problem exactly and fast? Yes: vertex cover is the textbook example of a fixed-parameter tractable problem, solvable in time $O(2^k \cdot (n+m))$ – exponential only in the parameter k, linear in the graph size.

Intuition

The branching rule is simple. Take any edge $\{u, v\}$. A vertex cover of size $\leq k$ must contain u or v – it has no third option. So branch two ways: "put u in the cover, recurse on $G - u$ with budget $k-1$" or "put v in, recurse on $G - v$ with budget $k-1$." Each branch spends one unit of budget, so the recursion tree has depth k and branching factor 2: at most 2^k leaves. At a leaf, either the budget hit zero with edges remaining (this branch fails) or no edges remain (success). That is the entire algorithm.

Algorithm 121: VCBranch(G, k) – decide vertex cover $\leq k$

```
output: a cover of size ≤ k, or None
if G has no edges then
    return ∅
if k = 0 then
    return None
pick any edge {u, v} of G
C_u ← VCBranch(G − u, k − 1)
if C_u ≠ None then return C_u ∪ {u}
C_v ← VCBranch(G − v, k − 1)
if C_v ≠ None then return C_v ∪ {v}
return None
```

Theorem 29.5 (Vertex cover is FPT)**.** VCBranch *correctly decides whether $\tau(G) \leq k$ and, if so, returns a witness, in time $O(2^k \cdot (n+m))$.*

Proof. Correctness. By induction on k. If G has no edges, $\emptyset$ covers it. If $k = 0$ but edges remain, no cover of size 0 exists, so None is right. Otherwise fix an edge $\{u, v\}$. Any cover C of size $\leq k$ contains u or v. If $u \in C$, then $C \setminus \{u\}$ covers $G - u$ with size $\leq k - 1$, found by the first recursive call (induction); the symmetric statement holds for v. Conversely any returned set is genuinely a cover, since we union back the deleted vertex. So the disjunction of the two branches is exactly the decision.

Time. The recursion tree is binary of depth at most k, so it has at most $2^{k+1} - 1$ nodes. Each node does $O(n+m)$ work to pick an edge and build the reduced graph. Total $O(2^k(n+m))$. □

Why it works

What makes this work where a branching algorithm for, say, clique would not? The *measure* that drops. Each branch decreases the budget k by exactly one, and the budget is bounded by the answer size we are testing – not by n. The search tree's size depends on k alone. For maximum clique there is no such small certifying parameter: a maximum clique can be of size $\Theta(n)$, so the analogous branching depth is $\Theta(n)$ and you gain nothing. Designing FPT algorithms means finding a parameter that shrinks under branching and is small on the instances you care about.

29.5.1 Kernelization: shrinking before you branch

The 2^k factor multiplies a graph of size $n+m$. We can first *shrink* the graph to size depending only on k – a *kernel* – so the expensive branching runs on a tiny instance. Two reduction rules suffice.

Lemma 29.4 (Buss kernelization)**.** *If a graph has a vertex cover of size $\leq k$, the following rules reduce it, in polynomial time, to an equivalent instance with at most $k^2 + k$ vertices and at most k^2 edges:*

1. ***High-degree rule.*** *If $\deg(v) > k$, then v must be in every cover of size $\leq k$; put v in the cover, delete it, decrement k.*

2. ***Isolated rule.*** *Delete any vertex of degree* 0 *(it covers nothing and need not be in any cover).*

After exhaustive application, if more than k^2 *edges remain, answer* No.

Proof. High-degree rule. If v has more than k neighbors and v is left out of the cover, then *all* $> k$ of its neighbors must be in the cover to cover v's edges – exceeding budget k. So v is forced in; removing it and decrementing k is exact. *Isolated rule* is immediate. *Size bound.* After the high-degree rule every remaining vertex has degree $\leq k$. A cover of $\leq k$ vertices, each of degree $\leq k$, touches at most $k \cdot k = k^2$ edges; if more remain, no size-k cover exists. Each covered edge has an endpoint among the $\leq k$ cover vertices, and the other endpoints number at most k^2, so at most $k^2 + k$ non-isolated vertices survive. □

Focus

Kernelization is a standard FPT technique: *provably* compress the instance to a size that is a function of k alone, in polynomial time, with no loss of optimality. Then any exact method – the 2^k branching, or even brute force – runs on the kernel. Combining Buss's $O(k^2)$ kernel with the $O(2^k \cdot (n+m))$ branching gives the full pipeline. The Nemhauser–Trotter theorem upgrades this to a kernel of at most $2k$ *vertices* – the LP's half-integral structure says the vertices set to 0 and 1 can be peeled off optimally, leaving only the $\frac{1}{2}$-core of size $\leq 2k$.

Cost summary

FPT vertex cover. Branching: $O(2^k(n+m))$. With kernelization first: $O(n+m)+O(2^k \cdot k^2)$ using Buss, or kernel down to $2k$ vertices via Nemhauser–Trotter. Sharper branching (on degree-1, degree-2, and dominated vertices) brings the base below 2; the best published bound is around $O(1.2^k + kn)$. Yet no such FPT algorithm is known or possible for clique parameterized by solution size – clique is W[1]-hard, the parameterized analogue of NP-hardness, so the three problems differ here as well.

Connection

The W[1]-hardness of clique (and hence independent set) parameterized by solution size is the parameterized-complexity counterpart of the approximation split. Three problems, polynomially equivalent in exact terms, diverge under *both* finer measures: vertex cover is approximable to within 2 and is FPT; clique and independent set are inapproximable to any constant and are W[1]-hard. The Gallai bijection that ties their optima together does not preserve these finer properties: whether two problems count as "the same" depends on which resource you are counting.

29.6 Solving the Running Example

We now finish the analysis of G_0. Bron–Kerbosch gave $\omega(G_0) = 3$. The matching heuristic picks, say, edges $\{a, b\}$ and $\{d, e\}$ – a maximal matching of size 2 – and returns the cover $\{a, b, d, e\}$ of size 4, certifying $\tau \leq 4$ and (matching lower bound) $\tau \geq 2$. Can we do better than four? The set $\{b, c, e\}$ covers ab, bc, ac, cd, bd (via b, c) and de (via e), but misses ef and af. Add f: $\{b, c, e, f\}$ is size 4 again. Try $\{a, c, d, f\}$: covers ab, ac (via a), bc, cd (via c), bd (via d), de (via d), ef, af (via f) – size

4 once more. The cleanest way to pin the exact value is Gallai: the largest independent set of G_0 has size $\alpha(G_0) = 2$ (every maximal independent set, such as $\{a, d\}$ or $\{b, e\}$, has just two vertices, as the complement has no triangle), so $\tau(G_0) = 6 - \alpha(G_0) = 4$. Every size-4 cover above is therefore optimal.

Trace it yourself

Finish G_0 by hand. (i) List all maximal independent sets by complementing the clique census of Section 29.2.2; you should find seven, all of size 2. (ii) Confirm $\alpha(G_0) = 2$ and verify $\alpha + \tau = 6$ (Gallai), giving $\tau(G_0) = 4$. (iii) Solve the vertex-cover LP on G_0: which vertices come out $\frac{1}{2}$, and which triangle forces them? (iv) Run VCBranch with $k = 4$, branching on edge $\{d, e\}$ first, and watch the tree; check that $k = 3$ fails. You should land on $\tau(G_0) = 4$ with a concrete witness, hence $\alpha(G_0) = 2$ – for instance the independent set $\{a, d\}$ paired with the cover $V \setminus \{a, d\} = \{b, c, e, f\}$.

Why this matters

This chapter used one graph, three numbers, and three approaches to intractability. Gallai ties independent set and vertex cover into a single quantity; the complement graph connects both to clique. From there the chapter is a tour of what to do when a problem is NP-hard: enumerate cleverly (Bron–Kerbosch and its $3^{n/3}$ optimal output), relax and round (the half-integral LP and its 2-approximation), or parameterize and branch (the 2^k algorithm and its k^2 kernel). That these three responses can all be exhibited on *one* problem family, with full proofs and a six-vertex graph you can hold in your head, makes these problems a good place to learn the techniques.

Connection

What This Made Possible: Independent set, clique, and vertex cover motivated several modern approaches to intractable problems. Vertex cover was an early showcase for the primal-dual method and the 2-approximation via maximal matchings; half-integrality led to Nemhauser–Trotter kernelization; and the Bron–Kerbosch pivoting algorithm made maximal clique enumeration practical for computational biology, community detection, and cheminformatics.

Limitations:

- *Inapproximability barriers:* While Vertex Cover admits a simple factor-2 approximation, achieving any approximation factor below $2 - \varepsilon$ is NP-hard under the Unique Games Conjecture (Khot–Regev 2008). For Maximum Independent Set and Maximum Clique, inapproximability is far starker: neither can be approximated within $n^{1-\varepsilon}$ for any $\varepsilon > 0$ unless P = NP (Håstad 1999).
- *Output explosion:* The number of maximal cliques can reach $3^{n/3}$. Even with perfect pivoting, any enumeration algorithm must spend exponential time on dense graphs with exponentially many maximal cliques.

Where We Go Next: Is there a broad structural class of graphs where Maximum Independent Set, Maximum Clique, and Minimum Vertex Coloring can all be solved in exact polynomial time? In **Chapter 30 (Perfect Graphs)**, we explore the rich class of graphs where $\chi(H) = \omega(H)$ for every induced subgraph, uniting chordal, bipartite, and comparability

graphs using Lovász's geometric methods.

Exercises

Exercise 29.1. Prove that for any graph G, $\nu(G) \le \tau(G) \le 2\nu(G)$, where ν is the maximum matching size. Give a graph achieving each side with equality. Which graphs satisfy $\tau = \nu$? (Hint: König.)

Exercise 29.2. Compute α, ω, τ for the cycle C_n and for the complete bipartite graph $K_{p,q}$. Verify Gallai's identity in each case, and confirm that for odd cycles the vertex-cover LP has a strictly fractional optimum.

Exercise 29.3. Implement Bron–Kerbosch with pivoting and run it on the Petersen graph. How many maximal cliques does it find, and what is ω? Then run the no-pivot version and count the recursive calls each makes – quantify the pivot's savings.

Exercise 29.4. (Break it.) A student proposes the following 2-approximation for vertex cover: "repeatedly take the highest-degree vertex into the cover and delete it, until no edges remain." Show this greedy can be a factor $\Theta(\log n)$ away from optimal, not 2. Construct the bad instance. (This is a graph specialization of the classic $\Theta(\log n)$ lower bound for greedy set cover.)

Exercise 29.5. Prove the Moon–Moser bound: a graph on n vertices has at most $3^{n/3}$ maximal cliques, and the disjoint union of $n/3$ triangles attains it. Conclude that no algorithm can list all maximal cliques in time $o(3^{n/3})$ in the worst case.

Exercise 29.6. Show the vertex-cover LP (29.2) is integral on bipartite graphs by exhibiting the reduction to a min-cut / maximum matching, and conclude $\tau^* = \tau = \nu$ there. Where exactly does the argument break for an odd cycle?

Exercise 29.7. (Extend.) Improve VCBranch with a degree rule: if some vertex has degree 1 with neighbor w, argue there is an optimal cover containing w (not the leaf), so that branch can be taken without doubling. Then show that branching on a degree-2 vertex yields the recurrence $T(k) \le T(k-1) + T(k-2)$, giving base ≈ 1.618 rather than 2.

Exercise 29.8. Prove the Nemhauser–Trotter theorem operationally: given the half-integral LP optimum $x^\star$, show that $V_1 = \{v : x^\star_v = 1\}$ can be placed in the cover and $V_0 = \{v : x^\star_v = 0\}$ discarded, with some *optimal* cover respecting this split. Conclude a kernel of at most $2k$ vertices for parameter k.

Exercise 29.9. (Hardness intuition.) Using the Gallai bijection, show that a ρ-approximation for independent set does *not* yield a ρ-approximation for vertex cover. Give explicit numbers: a graph where the independent set returned is within a factor 2 of α but the induced cover is a factor $\Theta(n)$ from τ.

Exercise 29.10. Let G have degeneracy d (every subgraph has a vertex of degree $\le d$). Prove $\omega(G) \le d + 1$, and use a degeneracy ordering to bound the running time of Bron–Kerbosch by $O(d\, n\, 3^{d/3})$. Why is this near-linear in n for bounded-degeneracy families such as planar graphs?

Chapter 30

Perfect Graphs and Structure

In the preceding chapters, we saw that graph coloring, maximum clique, and maximum independent set are NP-hard on general graphs. One cannot expect $\chi(G) = \omega(G)$ in general; indeed, Mycielski's construction produces triangle-free graphs with arbitrarily high chromatic number. This chapter studies a class where these problems become tractable: the *perfect graphs*, where $\chi(H) = \omega(H)$ holds for every induced subgraph H. On perfect graphs, coloring, maximum clique, maximum independent set, and minimum clique cover are all solvable in polynomial time. We examine the structural characterizations of Berge and Lovász, focus on chordal graphs and their linear-time elimination orderings, and see how continuous convex relaxations solve combinatorial problems.

History

The concept of perfect graphs was born in information theory. In 1956, Claude Shannon posed the fundamental problem of calculating the *zero-error capacity* of a noisy channel: transmitting symbols over a communications channel where certain pairs of signals can be confused by receiver noise. Shannon proved that the maximum number of mutually distinguishable single-letter messages is exactly the independence number $\alpha(G)$ of the confusability graph.

In 1961, French mathematician Claude Berge observed that for several well-behaved graph families — bipartite graphs (Kőnig 1931), interval graphs (Hajnal–Surányi 1958), and comparability graphs of posets (Dilworth 1950) — the chromatic number $\chi(H)$ consistently equaled the clique number $\omega(H)$ for every induced subgraph H. Berge called these graphs *perfect* and formulated two famous conjectures: the Weak Perfect Graph Conjecture (a graph is perfect if and only if its complement is perfect) and the Strong Perfect Graph Conjecture (a graph is perfect if and only if neither G nor $\overline{G}$ contains an induced odd cycle of length at least 5).

In 1972, 24-year-old Hungarian mathematician László Lovász proved the Weak Conjecture using his Replication Lemma. Lovász later introduced the semidefinite ϑ function, which Martin Grötschel, Lovász, and Alexander Schrijver (1981) optimized via the ellipsoid method to give the first polynomial-time algorithms for coloring, clique, and independent set on any perfect graph. The Strong Conjecture was finally proved in 2002 (published 2006) by Maria Chudnovsky, Neil Robertson, Paul Seymour, and Robin Thomas in a landmark 178-page decomposition theorem. Polynomial-time recognition of perfect graphs was established shortly after by Chudnovsky, Gérard Cornuéjols, Xinming Liu, Seymour, and Kristina Vušković (2005). Today, finding a purely combinatorial polynomial-time coloring algorithm for general perfect graphs that avoids semidefinite programming remains a major open problem.

30.1 What perfection means

Start with two numbers attached to every graph $G = (V, E)$.

Definition 30.1 (Clique and chromatic numbers)**.** The *clique number* $\omega(G)$ is the size of the largest set of pairwise-adjacent vertices. The *chromatic number* $\chi(G)$ is the fewest colors needed so that adjacent vertices get different colors.

There is always a one-sided relationship between them.

Proposition 30.1 (The trivial bound)**.** *For every graph G, $\chi(G) \geq \omega(G)$.*

Proof. A clique of size ω is a set of ω mutually adjacent vertices. In any proper coloring no two of them may share a color, so at least ω colors are spent on the clique alone. Hence any proper coloring uses $\geq \omega$ colors, and the minimum χ is $\geq \omega$. □

So ω is a *lower bound* on χ: you cannot color with fewer colors than the size of your biggest clique forces. The natural question is whether the clique is the *only* obstruction. Sometimes it is not.

Intuition

Think of $\omega(G)$ as the "obvious reason" a graph needs many colors: a big clique simply cannot be squeezed into few color classes. The gap $\chi - \omega$ measures coloring difficulty that has *no* local clique witness — difficulty that comes from the global shape of the graph, like a long odd cycle that locally looks tree-like but globally refuses two colors. A *perfect* graph is one with no such hidden difficulty anywhere: the obvious lower bound is always tight.

The smallest example where the bound is loose is the 5-cycle.

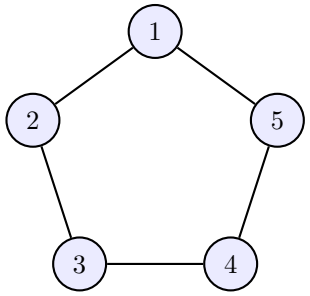


Figure 30.1: The cycle C_5. It has no triangle, so $\omega(C_5) = 2$. But it has an odd number of vertices, so two colors fail: $\chi(C_5) = 3$. The gap $\chi - \omega = 1$ certifies that C_5 is *not* perfect.

In C_5 the largest clique is a single edge, $\omega = 2$. Yet you cannot 2-color an odd cycle: walk around it alternating colors and you arrive back at the start needing the color you began with to differ from itself. So $\chi(C_5) = 3 > 2 = \omega(C_5)$. The trouble is global — every local neighborhood is just a path — which is exactly the kind of hidden difficulty perfection forbids.

Perfection asks for the bound to be tight not only on the whole graph but on every piece of it.

Definition 30.2 (Perfect graph)**.** A graph G is *perfect* if $\chi(H) = \omega(H)$ for every induced subgraph $H \subseteq G$ (including $H = G$ itself).

The induced-subgraph quantifier is essential and easy to underestimate. Requiring $\chi = \omega$ on the whole graph alone is a weak condition that many graphs satisfy by accident. Requiring it on *every* induced subgraph is far more restrictive, and that restriction is what makes polynomial-time algorithms possible.

Common trap

"$\chi(G) = \omega(G)$ for the whole graph" does *not* make G perfect. Consider C_5 together with a disjoint triangle K_3: the whole graph has $\omega = 3$ and can be 3-colored, so $\chi = \omega = 3$ globally — yet the induced C_5 still has $\chi = 3 > 2 = \omega$. The defect hides in a subgraph. Perfection is a hereditary property; you must check all induced subgraphs, and that is precisely why it is strong enough to be useful.

Focus

Why should an algorithmist care about $\chi = \omega$ everywhere? Because it ties four separately-hard quantities together. We will see that on a perfect graph you can, in polynomial time, find:

- a maximum clique (size ω),
- a maximum independent set (size α),
- an optimal coloring (using $\chi = \omega$ colors),
- a minimum clique cover (using $\bar{\chi} = \alpha$ cliques).

On a *general* graph all four are NP-hard. Perfection is the structural assumption that makes all four tractable.

30.2 The complement, and the Perfect Graph Theorem

There is a symmetry hiding in Definition 30.2 that took two decades and a deep proof to confirm. Recall the *complement* $\bar{G}$: same vertices, and uv is an edge of $\bar{G}$ exactly when it is *not* an edge of G.

Cliques and independent sets swap under complementation. A set of pairwise-adjacent vertices in G is a set of pairwise-*non*-adjacent vertices in $\bar{G}$, so

$$\omega(G) = \alpha(\bar{G}), \qquad \alpha(G) = \omega(\bar{G}).$$

Coloring swaps too, into something less familiar. A proper coloring of $\bar{G}$ partitions V into independent sets of $\bar{G}$, which are cliques of G. So $\chi(\bar{G})$ is the minimum number of cliques of G needed to cover all vertices — the *clique-cover number* $\bar{\chi}(G)$. Putting these together, "$\chi = \omega$ on $\bar{G}$" reads, in the language of G, as "the minimum clique cover equals the maximum independent set," i.e. $\bar{\chi}(G) = \alpha(G)$.

Intuition

Perfection of G is a statement about coloring versus cliques. Perfection of $\bar{G}$ is a statement about covering by cliques versus independent sets. On the surface, they look like completely different conditions. Lovász proved, surprisingly, that they are equivalent: G is perfect if and only if $\bar{G}$ is.

Theorem 30.1 (Perfect Graph Theorem; Lovász 1972)**.** *A graph G is perfect if and only if its complement $\bar{G}$ is perfect.*

We will not need the full machinery of ϑ, but Theorem 30.1 has a self-contained proof through a single lemma. The lemma says you may freely *duplicate* a vertex and the result is still perfect.

Definition 30.3 (Vertex replication)**.** To *replicate* a vertex v of G, add a new vertex v' adjacent to v and to all neighbors of v (so v, v' are "twins" joined by an edge). Replicating with k copies means doing this $k-1$ times, producing a clique $\{v = v_1, \dots, v_k\}$ all sharing v's outside neighborhood.

Lemma 30.1 (Replication Lemma)**.** *If G is perfect and G' is obtained from G by replicating a vertex, then G' is perfect.*

Proof idea. Replication is a local move, so by induction it suffices to handle a single new twin v' of v in a perfect G and show $\chi(G') = \omega(G')$; the same holds on induced subgraphs since they are again single replications of induced subgraphs of G. Color G optimally with $\omega(G)$ colors. Let C be the color class containing v. Either some maximum clique of G avoids v, in which case adding the twin v' does not raise ω and a short recoloring of $C \setminus \{v\}$ keeps the color count at ω; or every maximum clique meets v, in which case $\{v, v'\}$ together with that clique's other vertices forms a clique one larger, so $\omega(G') = \omega(G) + 1$ and we may simply spend one fresh color on v'. Either way $\chi(G') = \omega(G')$. The full argument is a careful but elementary induction; see the exercises. □

Proof of Theorem 30.1, sketch. By symmetry it suffices to show: G perfect $\implies$ $\bar{G}$ perfect. Argue by induction on $|V|$, assuming every *proper* induced subgraph of $\bar{G}$ already satisfies $\chi = \omega$. What remains is the single equation

$$\chi(\bar{G}) = \omega(\bar{G}), \qquad \text{i.e.} \qquad \bar{\chi}(G) = \alpha(G),$$

namely that V can be covered by $\alpha = \alpha(G)$ cliques of G. Suppose not, and take G minimal. The contradiction comes from a counting argument: replicate each vertex v a number of times $t_v \geq 0$ chosen so that, in the blown-up graph G^*, every maximum independent set of G is "hit" the right number of times. By the Replication Lemma 30.1, G^* stays perfect, so $\chi(G^*) = \omega(G^*)$. Counting vertices of G^* two ways — once as $\sum_v t_v$ and once through the bound $|V(G^*)| \leq \omega(G^*) \cdot \alpha(G^*)$ that perfection forces (Exercise) — yields $\omega(G^*) \cdot \alpha(G) \geq \sum_v t_v$ on one side and a strictly larger lower bound on the other whenever $\bar{\chi}(G) > \alpha(G)$. The two estimates contradict each other, so the assumption was false and $\bar{\chi}(G) = \alpha(G)$. That vertex-blowup step relies directly on Lemma 30.1; the remaining counting details are walked through in the exercises. □

Why it works

The key fact in the proof is that replication preserves perfection. Once you can blow each vertex up into a clique of any size, you can convert a hypothetical violation of $\bar{\chi} = \alpha$ into a counting impossibility: a perfect graph cannot have more vertices than $\omega \cdot \alpha$, but a clique cover that is too large would force exactly that. The complement symmetry of perfection thus comes down to the bound $|V| \leq \omega \cdot \alpha$ holding on every induced subgraph.

The Strong Perfect Graph Theorem

Berge's deeper conjecture pins down *exactly* which graphs are perfect, by forbidding two families of induced subgraphs.

Definition 30.4 (Holes and antiholes)**.** An *odd hole* is an induced cycle C_{2k+1} with $2k+1 \geq 5$. An *odd antihole* is the complement of an odd hole, $\overline{C_{2k+1}}$ with $2k+1 \geq 5$. A graph with no odd hole and no odd antihole as an induced subgraph is called *Berge*.

Both families are imperfect for the reason C_5 was: an odd hole C_{2k+1} has $\omega = 2$ but $\chi = 3$, and an odd antihole $\overline{C_{2k+1}}$ has $\omega = k$ but $\chi = k+1$. So no perfect graph can contain either. Berge conjectured the converse.

Theorem 30.2 (Strong Perfect Graph Theorem; Chudnovsky–Robertson–Seymour–Thomas 2006)**.** *A graph is perfect if and only if it is Berge: it contains no odd hole and no odd antihole as an induced subgraph.*

Connection

The Lovász theta function $\vartheta(G)$ connects perfect graphs to Chapters 31, 32, and 38. Defined by a semidefinite program, it sits between ω and χ of the complement: $\omega(G) \leq \vartheta(\bar{G}) \leq \chi(G)$. The Grötschel–Lovász–Schrijver algorithm computes ϑ via the ellipsoid method, and on perfect graphs $\vartheta(\bar{G}) = \omega(G) = \chi(G)$ exactly. This is a rare case where a polynomial-time algorithm for an exact combinatorial invariant runs *through* continuous optimization rather than around it. The combinatorial algorithms in the rest of this chapter avoid SDPs entirely — but only for the structured subclasses, where elimination orderings replace the ellipsoid.

30.3 Three structured families that are perfect

The general perfect-graph algorithms are polynomial but heavy. Fortunately, the perfect graphs that arise in practice usually belong to a structured subclass with a fast combinatorial algorithm. Three families are the most common.

Definition 30.5 (Interval graph)**.** G is an *interval graph* if each vertex v can be assigned a closed real interval I_v so that $uv \in E$ iff $I_u \cap I_v \neq \emptyset$. The intervals are a *representation* of G.

Definition 30.6 (Comparability graph)**.** G is a *comparability graph* if its edges can be oriented transitively: there is an acyclic orientation such that whenever $u \to v$ and $v \to w$ are arcs, $u \to w$ is also an arc. Equivalently, E is the comparability relation of some partial order on V.

Definition 30.7 (Chordal graph)**.** G is *chordal* (or triangulated) if every cycle of length ≥ 4 has a *chord*: an edge joining two vertices that are non-consecutive on the cycle. Equivalently, G has no induced cycle C_k for any $k \geq 4$.

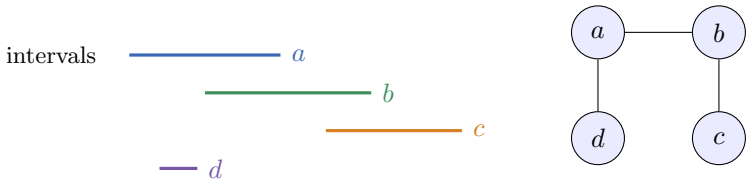


Figure 30.2: Left: four intervals on the line. a overlaps b and d; b overlaps a and c; c overlaps b; d overlaps a. Right: the resulting interval graph (a path d–a–b–c). Interval graphs are chordal, and chordal graphs are perfect.

These three families are not the same, but they nest and overlap in a way worth memorizing.

Proposition 30.2 (Containments)**.** *Every interval graph is chordal. Every interval graph is also the complement of a comparability graph (a "co-comparability" graph). Every chordal graph and every comparability graph is perfect.*

We prove the chordal case below, because the proof gives our main algorithm directly. The interval-implies-chordal direction is a quick exercise: a chordless cycle of length ≥ 4 among intervals cannot be realized on the line. The comparability-is-perfect direction follows from Mirsky's theorem (a longest chain equals the minimum antichain cover), an instance of Dilworth-type duality.

Application

Interval graphs model *scheduling*: tasks occupy time intervals, two tasks conflict iff their intervals overlap, and a proper coloring with χ colors is an assignment of tasks to the fewest machines so that no machine runs two overlapping tasks. Because interval graphs are perfect, $\chi = \omega$: the number of machines you need equals the maximum number of tasks live at any single instant — the obvious bound is achievable, and greedy by start time achieves it. Chordal graphs model *sparse Gaussian elimination*: the "fill-in" you incur eliminating a symmetric matrix in a given order is exactly the set of edges you must add to make the graph chordal, and a perfect elimination ordering is a fill-free pivot order. Comparability graphs encode *partial orders*: Dilworth's theorem (minimum chain cover = maximum antichain) is the perfection statement for them.

A comparability graph, concretely. Take the partial order on $\{1, 2, 3, 4, 6, 12\}$ given by divisibility: $x < y$ when x divides y and $x \neq y$. Its comparability graph joins two numbers iff one divides the other. A *chain* (totally ordered subset) is a clique; an *antichain* (pairwise incomparable subset) is an independent set. Here $\{1, 2, 4, 12\}$ is a chain of length 4, and $\{4, 6\}$ is an antichain — 4 and 6 divide neither one another.

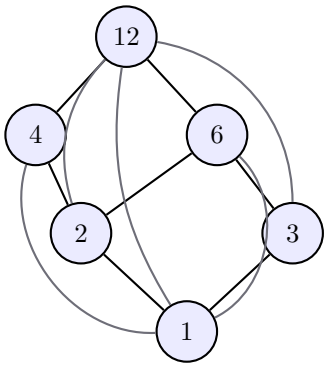


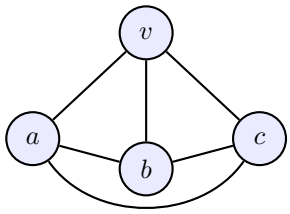
Figure 30.3: The divisibility order on $\{1, 2, 3, 4, 6, 12\}$, drawn with cover edges dark and transitive comparabilities in gray. Every comparable pair is an edge of the comparability graph. The longest chain $1 \mid 2 \mid 4 \mid 12$ (or $1 \mid 2 \mid 6 \mid 12$, $1 \mid 3 \mid 6 \mid 12$) has length 4, so $\omega = 4$ and $\chi = 4$; the largest antichain is $\{4, 6\}$ (or $\{3, 4\}$), so $\alpha = 2$ and the minimum chain cover uses 2 chains. Both equalities $\chi = \omega$ and $\bar{\chi} = \alpha$ hold — the graph is perfect.

Mirsky's theorem says the height (longest chain) equals the minimum number of antichains needed to cover the order, which is the statement $\chi = \omega$ for the comparability graph. Dilworth's theorem is its dual: the width (largest antichain) equals the minimum number of chains needed to cover the order, which is $\bar{\chi} = \alpha$. The two dualities together are precisely perfection of the comparability graph and of its complement — and by the Perfect Graph Theorem each implies the other.

30.4 Chordal graphs: simplicial vertices and elimination orderings

Chordal graphs are broad enough to include trees, interval graphs, and k-trees, yet structured enough that all four problems are solvable in linear time. The structure rests on one local notion.

Definition 30.8 (Simplicial vertex)**.** A vertex v is *simplicial* if its neighborhood $N(v)$ is a clique — every two neighbors of v are adjacent. Equivalently, v together with $N(v)$ forms a clique.



$N(v) = \{a, b, c\}$ is a triangle, so v is simplicial

Figure 30.4: A simplicial vertex v: its three neighbors are pairwise adjacent, so $\{v, a, b, c\}$ is a clique. Since v lies inside a clique, it cannot be an interior vertex of any chordless cycle.

The reason simplicial vertices matter is a theorem of Dirac.

Theorem 30.3 (Dirac 1961)**.** *Every chordal graph with at least one vertex has a simplicial vertex. If the graph is not complete, it has* two *non-adjacent simplicial vertices.*

Proof. Induct on $|V|$. A graph on one vertex is immediate, and a complete graph has every vertex simplicial. Let G be chordal and not complete, and pick two non-adjacent vertices a, b. Take a minimal vertex separator S between them (a minimal set whose removal puts a, b in different components). Let A be the component of $G - S$ containing a and B the one containing b.

First, S is a clique. If two vertices $x, y \in S$ were non-adjacent, then by minimality of S both x and y have a neighbor in A and a neighbor in B; stitching a shortest x–y path through A with a shortest one through B produces an induced cycle of length ≥ 4 with no chord, contradicting chordality.

Now apply the induction hypothesis to the chordal subgraph $G[A \cup S]$. It has a simplicial vertex; if it is complete, any vertex of A works, and otherwise it has two non-adjacent simplicial vertices, at least one of which lies in A (since S is a clique, the two cannot both lie in S). A vertex $v \in A$ that is simplicial in $G[A \cup S]$ is simplicial in all of G, because v's only neighbors lie in $A \cup S$ — none in B. Symmetrically B yields a simplicial vertex of G. The two lie in different components of $G - S$, so they are non-adjacent. □

Dirac's theorem says you can always strip a chordal graph one simplicial vertex at a time. Record the stripping order and you get the central object of the chapter.

Definition 30.9 (Perfect elimination ordering)**.** A *perfect elimination ordering* (PEO) of G is an ordering $v_1, v_2, \ldots, v_n$ of the vertices such that for each i, v_i is simplicial in the subgraph induced on $\{v_i, v_{i+1}, \ldots, v_n\}$ — that is, the *later* neighbors of v_i form a clique.

Theorem 30.4 (Fulkerson–Gross 1965)**.** *A graph is chordal if and only if it has a perfect elimination ordering.*

Proof. ($\Rightarrow$) If G is chordal, repeatedly apply Dirac's Theorem 30.3: pull out a simplicial vertex, call it v_1, and recurse on $G - v_1$, which is again chordal (deleting a vertex cannot create a chordless cycle). The order of removal is a PEO by construction.

($\Leftarrow$) Suppose G has a PEO $v_1, \ldots, v_n$ but, for contradiction, contains a chordless cycle C of length ≥ 4. Let v_i be the lowest-indexed vertex of C. Its two neighbors on C, call them x and y, both come later in the order (they have higher index, since v_i is lowest on C). They are both later-neighbors of v_i, so by the PEO property they must be adjacent. But x and y are non-consecutive on C (they are the two neighbors of v_i, hence at distance 2 on the cycle), so the edge xy is a chord of C — contradicting that C is chordless. Hence no such cycle exists and G is chordal. □

Why it works

A PEO turns "no chordless cycle anywhere" — a global, hard-to-check condition — into a *local* elimination rule: process vertices in this fixed order, and at the moment you touch each one its surviving neighbors already form a clique. Every algorithm in this section is just "walk the PEO and do something cheap at each vertex." The hard part is finding the order; once you have it, coloring, max-clique, and max-independent-set fall out in a single linear pass each.

30.5 Finding a PEO in linear time: Lexicographic BFS

How do we get the ordering? Dirac's proof gives an $O(nm)$ method (repeatedly scan for a simplicial vertex). A much faster method is *Lexicographic Breadth-First Search* (Lex-BFS) of Rose, Tarjan, and Lueker (1976), which produces a candidate ordering in $O(n + m)$ time; a second linear pass verifies whether it is a genuine PEO and hence whether the graph is chordal.

Intuition

Ordinary BFS breaks ties among the frontier arbitrarily. Lex-BFS breaks them with a rule: each unvisited vertex carries a *label* listing which already-numbered vertices it is adjacent to, written in decreasing order of number. At each step pick the unvisited vertex whose label is lexicographically largest. Vertices adjacent to recently chosen, high-numbered vertices float to the front. The effect is to keep cliques together and visit a vertex right after its "clique-mates," which is exactly what a (reversed) PEO needs.

Algorithm 122: Lex-BFS: produces an ordering σ (a reversed-PEO candidate)

Input: Graph $G = (V, E)$ with n vertices, m edges.
Output: An ordering $\sigma(n), \sigma(n-1), \ldots, \sigma(1)$ of V.

```
Assign every vertex the empty label ε;
for i ← n to 1 do
    Pick an unnumbered vertex v of lexicographically largest label;
    σ(i) ← v; mark v numbered with number i;
    foreach unnumbered neighbor w of v do
        append i to the label of w;
return σ;
```

Two implementation facts make this run in linear time. First, labels never need character-by-character comparison if you keep the unnumbered vertices in a *partition refinement* structure: a doubly linked list of "classes," each class being a set of vertices with identical labels so far, ordered by label. When vertex v gets numbered, each class splits into (neighbors of v, non-neighbors), with the neighbor part promoted ahead. Each edge causes one split-membership move, so the total work is $O(n+m)$. Second, the ordering Lex-BFS returns is a PEO — when read in the elimination direction — *if and only if* the graph is chordal.

Algorithm 123: Verify a PEO (and thus chordality) in linear time

Input: Ordering $v_1, \dots, v_n$ (eliminate v_1 first), graph G.
Output: TRUE if it is a PEO, else a chordless-cycle witness.

```
for i ← 1 to n do
    A(v_i) ← ∅;
for i ← 1 to n do
    Let later(v_i) = { v_j : v_i v_j ∈ E, j > i };
    if later(v_i) ≠ ∅ then
        Let u be the earliest (smallest index) vertex in later(v_i);
        A(u) ← A(u) ∪ (later(v_i) \ {u});
    if A(v_i) ⊈ later(v_i) then
        return chordless cycle through v_i (a required neighbor is missing);
return TRUE;
```

The verification idea: for each v_i, its later-neighbors must be a clique. Rather than test all pairs (quadratic), pass the responsibility to the earliest later-neighbor u: we only need u to be adjacent to all the *other* later-neighbors of v_i. Accumulating these requirements in $A(u)$ and checking, when we reach u, that $A(u)$ really sits inside u's later-neighbors gives an $O(n+m)$ test. If every check passes, the ordering is a PEO and G is chordal; the first failure exhibits two later-neighbors of some v_i that are non-adjacent, which expands to an induced C_4 or longer.

Trace it yourself

Run Lex-BFS on the graph in Figure 30.5 below by hand. Keep the partition of unnumbered vertices as an ordered list of classes. At each step you should see one class split into the neighbors and non-neighbors of the vertex you just numbered. Stop when you have all six numbers, then read off the elimination order and check that each vertex's later-neighbors really do form a clique. The order you get should match the trace in Table 30.1.

Why it works

The partition-refinement structure is worth a second look, because it is the reason the whole chapter runs in linear time rather than quadratic. Picture the unnumbered vertices as a left-to-right sequence of *classes*, every vertex in a class sharing the same label so far, and classes ordered by label from large (left) to small (right). Numbering a vertex v does exactly one thing to this sequence: it walks v's adjacency list and, for every class that contains a neighbor of v, splits that class into its neighbor part and non-neighbor part, sliding the neighbor part one notch to the left (toward the larger labels). A vertex moves only when an incident edge is processed, and each edge is processed once, so the splits cost $O(m)$ in total; the n pick-

and-remove steps cost $O(n)$. No label is ever compared symbol by symbol — the ordering of classes *is* the lexicographic order, maintained incrementally. In short, "compare strings" is replaced by "keep buckets sorted as you go."

30.6 A worked chordal graph, by hand

Here is a six-vertex chordal graph. We will find a PEO with Lex-BFS, then color it, find its maximum clique, and find a maximum independent set — all from the single ordering.

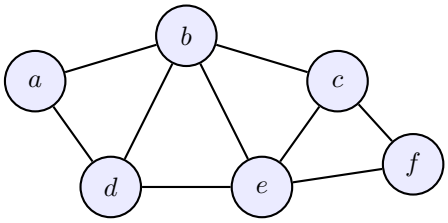


Figure 30.5: A chordal graph on six vertices. The triangles are $\{a,b,d\}$, $\{b,d,e\}$, $\{b,c,e\}$, $\{c,e,f\}$. There is no chordless cycle of length ≥ 4: every such cycle here has a chord. Maximal cliques are the four triangles; $\omega = 3$.

Lex-BFS trace. We number from 6 down to 1 and break label ties by alphabetical order, starting from a. The label of a vertex records the numbers of its already-numbered neighbors, high-to-low.

step	numbered	labels of remaining (high-to-low)	ordered classes
6	$a{=}6$	b:6, d:6; c, e, f:ε	$\{b,d\} \mid \{c,e,f\}$
5	$b{=}5$	d:6, 5; c:5, e:5; f:ε	$\{d\} \mid \{c,e\} \mid \{f\}$
4	$d{=}4$	e:5, 4; c:5; f:ε	$\{e\} \mid \{c\} \mid \{f\}$
3	$e{=}3$	c:5, 3; f:3	$\{c\} \mid \{f\}$
2	$c{=}2$	f:3, 2	$\{f\}$
1	$f{=}1$	—	—

Table 30.1: Lex-BFS on Figure 30.5. After numbering $a = 6$, its neighbors b, d get label "6" and jump ahead of c, e, f. Then $b = 5$ (largest label, ties broken alphabetically among $\{b, d\}$). d's label becomes "6, 5," beating c, e ("5"). And so on. The numbering, high to low, is a, b, d, e, c, f.

The *elimination order* is the increasing-number order: $f(1), c(2), e(3), d(4), b(5), a(6)$. Equivalently, eliminate f first, then c, e, d, b, a. Let us verify it is a PEO by checking later-neighbors (higher in elimination order = later) form a clique.

vertex	elim. #	later-neighbors	clique?
f	1	$\{c, e\}$	$ce \in E$? yes ✓
c	2	$\{e, b\}$	$eb \in E$? yes ✓
e	3	$\{d, b\}$	$db \in E$? yes ✓
d	4	$\{b, a\}$	$ba \in E$? yes ✓
b	5	$\{a\}$	singleton ✓
a	6	$\emptyset$	✓

Table 30.2: Every vertex's later-neighbors form a clique, so f, c, e, d, b, a is a perfect elimination ordering. Theorem 30.4 confirms the graph is chordal.

The four maximal cliques of this graph assemble into a *clique tree*, the object that connects chordality to treewidth. Arrange the maximal cliques as tree nodes so that, for each vertex, the cliques containing it form a connected subtree (the running-intersection property). Here the tree is a single path.

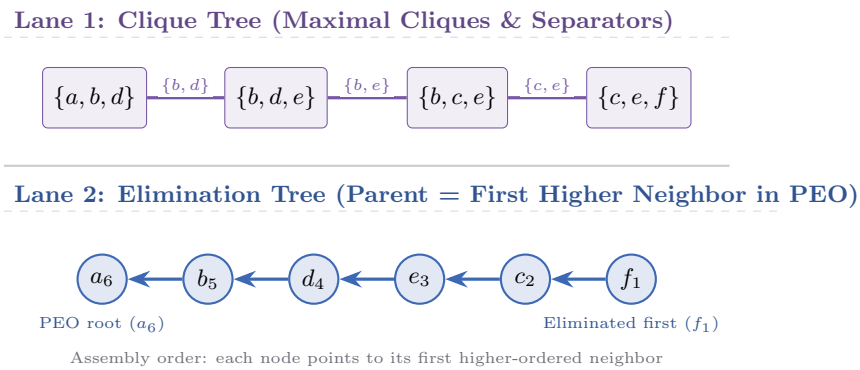


Figure 30.6: Structural representations of the chordal graph. **Top (Lane 1):** The clique tree arranging the four maximal cliques into a tree decomposition of width $\omega - 1 = 2$. Shared vertices on edges are minimal separators. **Bottom (Lane 2):** The elimination tree derived from the PEO $f(1), c(2), e(3), d(4), b(5), a(6)$, where each vertex directs an edge to its first higher-indexed neighbor. In Cholesky factorization, this tree captures all numerical dependencies without creating fill-in.

30.7 Linear-time coloring and maximum clique on a PEO

With a PEO in hand, coloring is a one-pass greedy — but applied in *reverse* elimination order (eliminate-last vertex gets colored first), so that whenever we color a vertex its already-colored neighbors are exactly its later-neighbors, a clique.

Algorithm 124: Optimal greedy coloring of a chordal graph from a PEO

Input: Chordal G, PEO $v_1, \dots, v_n$ (eliminate v_1 first).
Output: A proper coloring using exactly $\omega(G)$ colors.

```
for i ← n to 1 do
    F ← { color(v_j) : v_i v_j ∈ E, j > i } ;        // colors of already-colored (later) neighbors
    color(v_i) ← smallest positive integer not in F;
return color;
```

Theorem 30.5 (Chordal greedy is optimal)**.** *Algorithm 124 produces a proper coloring with exactly $\omega(G)$ colors, and the maximum color used equals the size of the largest clique. Hence for chordal graphs $\chi = \omega$ and both are found in $O(n+m)$ time.*

Proof. Properness is immediate: when we color v_i we avoid the colors of all neighbors already colored, and the neighbors colored later get colored avoiding v_i's color in turn (each edge is resolved when its second endpoint is colored).

Color count. Process in reverse elimination order. When v_i is colored, its already-colored neighbors are precisely its later-neighbors $\text{later}(v_i)$, which by the PEO property form a clique. Say $|\text{later}(v_i)| = k$. Those k neighbors are pairwise adjacent and pairwise differently colored, so they occupy k distinct colors; the greedy rule gives v_i a color in $\{1, \dots, k+1\}$. Therefore v_i receives color $\le k+1$, and the total number of colors used is at most $1 + \max_i |\text{later}(v_i)|$.

Now $\{v_i\} \cup \text{later}(v_i)$ is itself a clique of size $k+1$ (the later-neighbors are mutually adjacent and all adjacent to v_i). So $\max_i(|\text{later}(v_i)| + 1) \le \omega(G)$. Combining, the algorithm uses at most $\omega(G)$ colors. By Proposition 30.1 it uses at least $\omega(G)$. Hence exactly $\omega(G)$, and $\chi = \omega$.

Max clique. The largest clique is realized at the vertex v_i maximizing $|\text{later}(v_i)|$: that vertex with its later-neighbors is a clique of size ω, and no clique can be larger, since any clique's lowest-indexed vertex v_j has the rest of the clique among its later-neighbors. So scanning the later sizes finds ω and an actual maximum clique. The running time is one pass with neighbor scans, $O(n+m)$, given the PEO; Lex-BFS supplies the PEO in $O(n+m)$. □

Coloring our example. Using the PEO f, c, e, d, b, a, we color in reverse: a first, then b, d, e, c, f.

color step (reverse PEO)	already-colored later-neighbors and their colors	color chosen
a	none	1
b	a=1	2
d	a=1, b=2	3
e	b=2, d=3	1
c	b=2, e=1	3
f	c=3, e=1	2

Table 30.3: Greedy coloring along the reverse PEO. Colors used: $\{1,2,3\}$, so $\chi = 3$. The largest later-set had size 2 (e.g. d has later-neighbors $\{b, a\}$), giving clique $\{d, b, a\}$ of size $3 = \omega$. Indeed $\chi = \omega = 3$.

Common trap

The reverse order matters. If you greedily color in the *forward* elimination order (f first), a vertex's already-colored neighbors are its *earlier*-neighbors, which need not be a clique, and the bound $\leq k+1$ fails — greedy can then use more than ω colors even on a chordal graph. The PEO guarantees the clique property only for the *later*-neighbors, so you must color last-eliminated first. A second classic slip: forgetting that the same Lex-BFS output, read forward versus backward, plays both roles — it is the elimination order one way and the coloring order the other.

30.8 Maximum independent set and minimum clique cover on a PEO

The independent-set problem is solved by a greedy that runs in the *forward* elimination order. Take the first vertex of the PEO into the solution, delete it and all its neighbors, and repeat on what remains.

Algorithm 125: Maximum independent set of a chordal graph from a PEO

Input: Chordal G, PEO $v_1, \ldots, v_n$ (eliminate v_1 first).
Output: A maximum independent set S and a clique cover of equal size.

```
S ← ∅; mark all vertices alive;
for i ← 1 to n do
    if v_i is alive then
        S ← S ∪ {v_i};
        Form clique K_i ← {v_i} ∪ {alive later-neighbors of v_i};
        mark v_i and all its alive neighbors dead (assign them to cover-clique K_i);
return S and the cover {K_i};
```

Theorem 30.6 (Chordal MIS and clique cover)**.** *Algorithm 125 returns a maximum independent set S and a clique cover using exactly $|S|$ cliques. Hence on chordal graphs $\alpha = \bar{\chi}$, both computable in $O(n+m)$.*

Proof. S is independent. When v_i is added we kill all its alive neighbors, so no two chosen vertices are adjacent.

The cover is valid and has size $|S|$. Each time we add v_i to S we create one clique $K_i = \{v_i\} \cup$ (alive later-neighbors). Why is K_i a clique? Because at the moment we pick v_i, its alive neighbors are all later-neighbors (the earlier ones were killed by previous picks), and by the PEO those form a clique. Every vertex is killed exactly once, assigned to the K_i of the picked vertex that killed it, so the K_i partition V into $|S|$ cliques.

Optimality, by duality. We have produced an independent set S and a clique cover $\mathcal{K}$ with $|S| = |\mathcal{K}|$. But *any* independent set meets each clique at most once (two independent vertices cannot both sit in one clique), so $\alpha(G) \leq \bar{\chi}(G)$ always. Our construction achieves $|S| = |\mathcal{K}| \geq \bar{\chi} \geq \alpha \geq |S|$, forcing equality throughout: S is maximum, $\mathcal{K}$ is minimum, and $\alpha = \bar{\chi}$. Each vertex is scanned once with a neighbor sweep, $O(n+m)$. □

Why it works

The proof is a small duality argument, of the same form as max-flow/min-cut or LP duality. The "primal" object S (an independent set) and the "dual" object $\mathcal{K}$ (a clique cover) satisfy a weak inequality $|S| \le |\mathcal{K}|$ for free, just from the meaning of the words. The greedy algorithm produces a matched pair, and a matched pair certifies that *both* are optimal. This self-certifying property is what makes perfection useful algorithmically: a structured graph lets you build the optimum and its proof in the same pass.

Independent set on our example. PEO order f, c, e, d, b, a (eliminate f first).

- f alive $\Rightarrow$ take f. Kill f and its neighbors c, e. Clique $K_f = \{f, c, e\}$. Alive: d, b, a.
- c dead, skip. e dead, skip.
- d alive $\Rightarrow$ take d. Kill d and its alive neighbors b, a. Clique $K_d = \{d, b, a\}$. Alive: none.
- b, a dead, skip.

Result $S = \{f, d\}$, $|S| = 2 = \alpha$. Clique cover $\{f, c, e\}$ and $\{d, b, a\}$, two cliques, $\bar{\chi} = 2$. Check: $\{f, d\}$ is independent ($fd \notin E$), and you cannot find three pairwise non-adjacent vertices here, so $\alpha = 2$. The duality $\alpha = \bar{\chi} = 2$ holds (Figure 30.7).

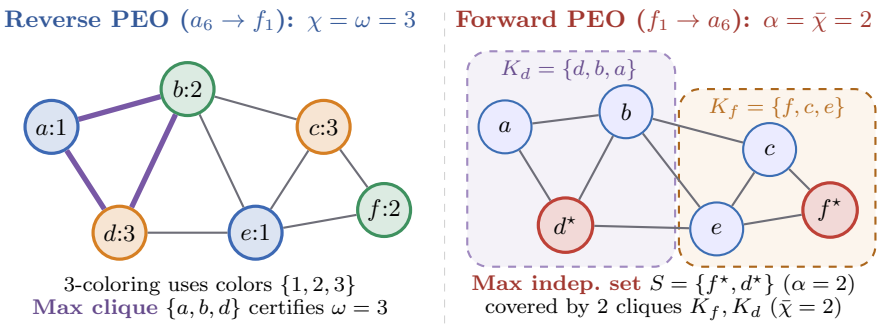


Figure 30.7: Both dualities realized on the six-vertex chordal graph from a single PEO. **Left:** Coloring in reverse elimination order ($a_6 \to f_1$) uses $\chi = 3$ colors, matched by the purple 3-clique $\{a, b, d\}$. **Right:** Sweeping in forward elimination order ($f_1 \to a_6$) picks independent vertices $f^\star$ and $d^\star$ ($\alpha = 2$) and partitions V into their later-neighbor cliques $K_f = \{f, c, e\}$ and $K_d = \{d, b, a\}$ ($\bar{\chi} = 2$).

Cost summary

On a chordal graph, given G by adjacency lists:

task	time
Find a PEO / decide chordality (Lex-BFS + verify)	$O(n + m)$
Optimal coloring ($\chi = \omega$ colors)	$O(n + m)$
Maximum clique (size ω)	$O(n + m)$
Maximum independent set (size α)	$O(n + m)$
Minimum clique cover (size $\bar{\chi} = \alpha$)	$O(n + m)$

Every one of these is NP-hard on a general graph. All of these rely on a single $O(n + m)$ ordering. Interval graphs inherit all of this (they are chordal) and additionally admit an even simpler "sort by left endpoint" sweep for coloring.

30.9 Why structure gives polynomial time

The algorithmic pattern here recurs throughout the book.

Why this matters

The general optimization problems — χ, ω, α, $\bar{\chi}$ — are NP-hard because an arbitrary graph can encode arbitrary constraint satisfaction; there is no order to exploit. *Perfection* is the promise that the simplest lower bound is always tight, which removes the gap that hardness hides in. *Chordality* is a still stronger, purely combinatorial promise: the vertices can be linearly ordered so that each one is locally trivial (simplicial) at the moment it is processed. A linear order under which a global problem becomes a sequence of trivial local decisions is the recurring shape of efficient graph algorithms — topological order for DAG dynamic programming, BFS layers for shortest paths, the PEO here, and tree-decomposition orders for bounded treewidth. Once the order is found, the algorithm is usually straightforward.

Connection

What This Made Possible: Perfect graphs form a class on which four problems that are NP-hard in general — $\chi(G)$, $\omega(G)$, $\alpha(G)$, and clique cover $\bar{\chi}(G)$ — become solvable in polynomial time. On chordal and interval graphs, Lex-BFS and Perfect Elimination Orderings yield optimal linear-time $O(n + m)$ algorithms. On general perfect graphs, the Grötschel–Lovász–Schrijver (1981) framework uses semidefinite programming and the Lovász ϑ function to solve all four invariants in polynomial time. In addition, a chordal graph's clique tree is a tree decomposition with width $\omega - 1$ and zero fill-in, linking perfection directly to sparse Cholesky factorization and Gaussian elimination.

Limitations:

- *Extreme sensitivity to induced subgraphs:* Perfection requires $\chi(H) = \omega(H)$ for *every* induced subgraph. Adding or deleting a single edge that introduces an induced C_5 destroys perfection instantly, rendering the combinatorial guarantees void.
- *Semidefinite programming overhead:* While chordal graphs admit fast $O(n + m)$ algorithms, computing χ or α on general non-chordal perfect graphs requires the ellipsoid method on the Lovász ϑ function, which is polynomial ($O(n^3)$ or $O(n^4)$) but practically heavy.

Where We Go Next: Lovász's work on perfect graphs showed that discrete graph problems can be solved by continuous geometric relaxations and eigenvalues. How do we translate a discrete graph into continuous physics and linear algebra? In **Chapter 31 (The Graph Laplacian)**, we connect graphs to continuous potential theory: representing edges as physical springs and matrices as diffusion operators.

Reading note

The hierarchy to keep in your head: *trees* $\subset$ *interval* $\subset$ *chordal* $\subset$ *perfect*, with *bipartite* and *comparability* graphs also sitting inside perfect by clean structural theorems. The Strong Perfect Graph Theorem says *all* perfect graphs are exactly the Berge graphs (no odd hole, no odd antihole). For algorithms you almost always want the strongest structure you can prove your input has: chordality if you can get it (linear time), perfection as a fallback (polynomial via the Lovász theta SDP).

Exercises

Exercise 30.1 (Imperfection certificates)**.** Show directly that the odd antihole $\overline{C_7}$ has $\omega = 3$ and $\chi = 4$, certifying it is not perfect. (Hint: a clique in $\overline{C_7}$ is an independent set in C_7; the largest is 3. For χ, a proper coloring of $\overline{C_7}$ is a partition of C_7 into cliques of C_7, i.e. edges and single vertices — count how many you need to cover 7 vertices.)

Exercise 30.2 (The product bound)**.** Prove that for every perfect graph $|V| \leq \omega(G) \cdot \alpha(G)$, and show this can fail for imperfect graphs by exhibiting a graph with $|V| > \omega\alpha$. (This inequality is the arithmetic core of the Perfect Graph Theorem; C_5 is the smallest counterexample on the imperfect side.)

Exercise 30.3 (Replication, in full)**.** Complete the proof of the Replication Lemma 30.1: starting from an optimal ω-coloring of the perfect graph G, give the full case analysis that yields an $\omega(G')$-coloring of the replicated graph G'. Conclude by induction that replicating any vertex any number of times preserves perfection.

Exercise 30.4 (Interval $\Rightarrow$ chordal)**.** Prove that every interval graph is chordal. (Hint: suppose intervals realize a chordless cycle $v_1 v_2 \cdots v_k v_1$ with $k \geq 4$. Consider the vertex whose interval has the leftmost right-endpoint and derive a contradiction from the cycle's non-adjacencies.)

Exercise 30.5 (Trees are chordal, and more)**.** Show every tree is chordal and exhibit a PEO of a tree directly (no Lex-BFS): repeatedly remove a leaf. What are the maximal cliques of a tree, and what does Algorithm 124 reduce to? Confirm $\chi = \omega = 2$ for any tree with an edge.

Exercise 30.6 (Break the forward greedy)**.** Find a chordal graph and a PEO for which coloring greedily in *forward* elimination order (eliminate-first colored first) uses strictly more than ω colors. This shows the reverse-order requirement of Algorithm 124 is essential, not a convenience.

Exercise 30.7 (Lex-BFS by hand)**.** Run Lex-BFS on the 5-cycle C_5 and on C_5 plus one chord. For each, write the elimination order it returns and run the PEO verifier (Algorithm 123). The verifier should fail on C_5 (not chordal); determine whether a single chord makes C_5 chordal, and confirm the verifier's verdict.

Exercise 30.8 (Clique tree)**.** For the chordal graph of Figure 30.5, list the four maximal cliques and build a *clique tree*: a tree whose nodes are the maximal cliques such that for every vertex v, the cliques containing v form a connected subtree. Verify the "running intersection" property and explain why this tree is a tree decomposition of width $\omega - 1 = 2$. (Figure 30.6 shows one such tree; confirm it and decide whether any other clique tree exists here.)

Exercise 30.9 (Comparability and Dilworth)**.** A comparability graph comes from a partial order P. State Dilworth's theorem (minimum chain cover = maximum antichain) and explain why it is exactly the assertion $\bar{\chi} = \alpha$ for the comparability graph of P. Deduce, using the Perfect Graph Theorem 30.1, that comparability graphs are perfect.

Exercise 30.10 (Recognizing interval graphs)**.** Sketch how to combine a chordality test with a check that the maximal cliques can be linearly ordered (the "consecutive cliques" property) to recognize interval graphs in polynomial time, and explain how a valid ordering of maximal cliques yields an interval representation. (Booth–Lueker's PQ-trees do this in linear time; you need only describe the high-level reduction.)

Exercise 30.11 (Divisibility order)**.** For the divisibility comparability graph of Figure 30.3 on $\{1, 2, 3, 4, 6, 12\}$, exhibit an optimal coloring with $\chi = \omega = 4$ colors and a minimum chain cover with $\bar{\chi} = \alpha = 2$ chains. Then extend the ground set to $\{1, 2, \ldots, 12\}$ under divisibility, and compute ω (longest chain) and α (largest antichain) for the enlarged order. (Hint for α: the integers in $\{7, 8, \ldots, 12\}$ are pairwise non-dividing.)

Exercise 30.12 (Partition refinement by hand)**.** Implement, on paper, the partition-refinement view of Lex-BFS: maintain the unnumbered vertices as an ordered list of classes and, at each numbering step, perform the neighbor/non-neighbor split described in the text. Run it on the graph of Figure 30.5 and confirm you reproduce Table 30.1 class-by-class. Then argue that the total number of split operations over the whole run is $O(m)$, so the algorithm is $O(n + m)$.

Exercise 30.13 (Perfection is not closed under everything)**.** Show that perfection is preserved by taking induced subgraphs and by complementation (the latter is Theorem 30.1), but is *not* preserved by adding an arbitrary edge or by taking subgraphs that are not induced. Give a small concrete example for each failure. (Hint: a path P_5 is perfect; what single edge turns part of it into an induced C_5 inside a larger graph?)

Part VI

Spectral and Algebraic Graph Theory

Chapter 31

The Graph Laplacian

In the preceding chapter, we saw how continuous semidefinite relaxations like the Lovász ϑ function solved discrete combinatorial problems on perfect graphs. Now we step fully into linear algebra and physics. Every chapter so far has treated a graph as a purely combinatorial object: vertices, edges, and discrete traversals. This chapter introduces the central algebraic object connecting graphs to continuous mathematics: the *Laplacian* matrix $\mathbf{L}$. Its eigenvalues measure connectivity, its quadratic form measures smoothness, its null space counts connected components, its pseudoinverse computes electrical resistances and hitting times of random walks, and the linear system $\mathbf{L}x = b$ is the main computational step in spectral clustering and graph sparsification.

The Laplacian connects the discrete graph algorithms of earlier parts to the spectra, electrical flows, and random walks of the chapters that follow. We define $\mathbf{L} = \mathbf{A}_D - \mathbf{A}$ three different ways, prove they coincide, and derive the rest from one identity: $x^\top \mathbf{L} x$ equals the sum over edges of squared differences. Most of spectral graph theory rests on that identity.

Intuition

Think of the graph as a network of springs. Put a bead at each vertex and let bead i sit at height x_i on a vertical wire. Every edge is an ideal spring of stiffness w_{ij} connecting the two beads it touches. The total elastic energy stored in the network, when the beads are pinned at heights $x = (x_1, \ldots, x_n)$, is exactly

$$\tfrac{1}{2}\, x^\top \mathbf{L} x \;=\; \tfrac{1}{2} \sum_{\{i,j\} \in E} w_{ij}\,(x_i - x_j)^2.$$

The Laplacian *is* the energy of the spring network. Everything in this chapter — smoothness, connectivity, the smallest eigenvalues — is a statement about which height assignments store little energy. Flat assignments (all beads at the same height) store none; that is why the all-ones vector is in the null space.

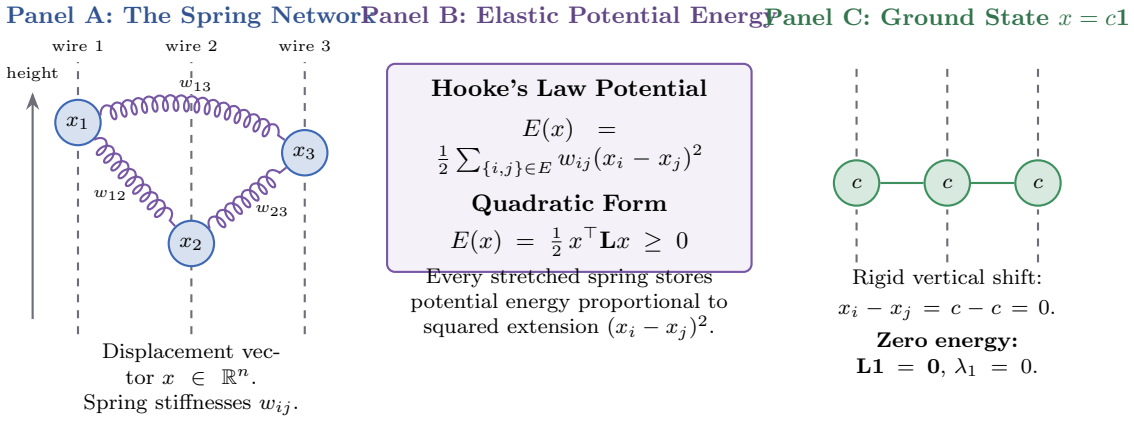


Figure 31.1: The physical intuition of the Graph Laplacian. **Panel A:** Each vertex is a bead constrained to slide vertically along a wire; each edge is an ideal spring of stiffness w_{ij}. **Panel B:** Displacing the beads to heights $x = (x_1, \ldots, x_n)$ stretches each spring by $(x_i - x_j)$, storing total elastic energy $\frac{1}{2}x^\top \mathbf{L}x$. **Panel C:** A uniform shift $x = c\mathbf{1}$ moves all beads in tandem, producing zero spring elongation and zero energy: $x^\top \mathbf{L}x = 0$, explaining why $\mathbf{1}$ is the ground-state null eigenvector.

History

The Laplacian matrix originated in nineteenth-century physics. In 1847, twenty-three-year-old German physicist Gustav Kirchhoff was analyzing networks of electrical resistors driven by batteries. Formulating Kirchhoff's Current and Voltage Laws, he sought an algebraic method to calculate electrical potentials from injected currents, discovering that the conductance matrix relating them was $\mathbf{L} = \mathbf{A}_D - \mathbf{A}$. In doing so, Kirchhoff proved the celebrated *Matrix-Tree Theorem*: every cofactor of $\mathbf{L}$ counts the exact number of spanning trees in the graph. The term *Laplacian* itself draws an analogy to the continuous differential operator $\Delta f = \nabla^2 f$, which measures how much a function deviates from its local average, with $x^\top \mathbf{L}x = \sum_{\{i,j\}}(x_i - x_j)^2$ serving as the discrete analogue of continuous Dirichlet energy $\int \|\nabla f\|^2$.

For over a century, mathematicians viewed $\mathbf{L}$ primarily as an electrical tool and determinant curiosity, while graph theorists focused almost exclusively on the adjacency matrix $\mathbf{A}$. But the adjacency matrix has structural limitations: isolated vertices add spurious zero rows, and scaling vertex degrees distorts its eigenvalues without reflecting bottleneck structure.

The modern spectral perspective arrived in 1973, when Czech mathematician Miroslav Fiedler investigated $\mathbf{L}$ through quadratic forms. Fiedler recognized that $x^\top \mathbf{L}x = \sum_{\{i,j\}}(x_i - x_j)^2$ is positive semi-definite, directly capturing the elastic potential energy of physical springs. Fiedler proved that the second-smallest eigenvalue λ_2 (which he named *algebraic connectivity*) is strictly positive if and only if the graph is connected. In addition, he showed that the corresponding eigenvector (the *Fiedler vector*) provides a one-dimensional coordinate layout that places tightly connected clusters together while pulling weakly connected components apart. In the 2000s, Daniel Spielman and Shang-Hua Teng used this perspective to develop nearly linear-time solvers for Laplacian systems $\mathbf{L}x = b$. Designing deterministic nearly linear-time solvers for directed and non-symmetric Laplacians remains an active open research challenge.

31.1 Three definitions, one matrix

Fix an undirected graph $G = (V, E)$ with $n = |V|$ vertices, labeled $1, \dots, n$, and $m = |E|$ edges. For now take the graph unweighted and simple (no loops, no parallel edges); we generalize to weights in Section 31.4. Recall two familiar matrices.

The *adjacency matrix* $\mathbf{A} \in \mathbb{R}^{n \times n}$ has $\mathbf{A}_{ij} = 1$ when $\{i, j\} \in E$ and 0 otherwise. It is symmetric with zero diagonal. The *degree matrix* $\mathbf{A}_D \in \mathbb{R}^{n \times n}$ is diagonal, with $(\mathbf{A}_D)_{ii} = \deg(i) = \sum_j \mathbf{A}_{ij}$ on the diagonal and zeros off it.

Definition 31.1 (Combinatorial Laplacian)**.** The *Laplacian* of G is the $n \times n$ matrix

$$\mathbf{L} \;=\; \mathbf{A}_D - \mathbf{A}, \qquad \mathbf{L}_{ij} \;=\; \begin{cases} \deg(i) & i = j, \\ -1 & \{i, j\} \in E, \\ 0 & \text{otherwise.} \end{cases}$$

Each diagonal entry holds a vertex's degree; each off-diagonal entry is -1 exactly when the two vertices are adjacent. Notice that every row sums to zero: the diagonal $\deg(i)$ is canceled by the $\deg(i)$ many -1's in that row. We will see shortly that much of the theory follows from this observation.

Example 31.1 (A five-vertex graph)**.** We will trace the whole chapter on one little graph: a 4-cycle $1{-}2{-}3{-}4{-}1$ with a pendant vertex 5 hanging off vertex 3.

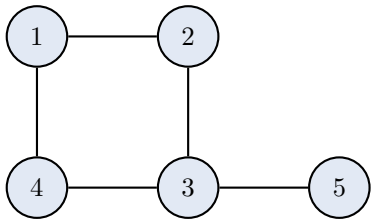


Figure 31.2: Our running graph G: a 4-cycle with a pendant. Degrees are $\deg(1) = \deg(2) = \deg(4) = 2$, $\deg(3) = 3$, $\deg(5) = 1$.

Reading the edges $\{1,2\}, \{2,3\}, \{3,4\}, \{1,4\}, \{3,5\}$ straight into Definition 31.1:

$$\mathbf{A} = \begin{pmatrix} 0 & 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \end{pmatrix}, \qquad \mathbf{L} = \begin{pmatrix} 2 & -1 & 0 & -1 & 0 \\ -1 & 2 & -1 & 0 & 0 \\ 0 & -1 & 3 & -1 & -1 \\ -1 & 0 & -1 & 2 & 0 \\ 0 & 0 & -1 & 0 & 1 \end{pmatrix}.$$

Check the rows sum to zero. The diagonal $(2, 2, 3, 2, 1)$ is exactly the degree sequence. Keep this $\mathbf{L}$ in view; we will compute its quadratic form, its null space, and its eigenvalues by hand before the chapter ends.

31.1.1 The incidence matrix and $\mathbf{L} = \mathbf{A}_B \mathbf{A}_B^\top$

The second definition is the one that explains *why* $\mathbf{L}$ is positive semidefinite, and it is the one we use for proofs. To write it we must orient each edge — pick, arbitrarily, a head and a tail. The orientation is a bookkeeping device; the final matrix $\mathbf{L}$ does not depend on it, as we will verify.

Definition 31.2 (Incidence matrix)**.** Orient each edge e from a tail $t(e)$ to a head $h(e)$. The (signed, vertex–edge) *incidence matrix* $\mathbf{A}_B \in \mathbb{R}^{n\times m}$ has one row per vertex and one column per edge, with

$$(\mathbf{A}_B)_{v,e} = \begin{cases} +1 & v = h(e), \\ -1 & v = t(e), \\ 0 & \text{otherwise.} \end{cases}$$

Each column of $\mathbf{A}_B$ has exactly one $+1$ and one -1: it is the vector $\mathbf{e}_{h(e)} - \mathbf{e}_{t(e)}$, where $\mathbf{e}_v$ is the standard basis vector. Geometrically the column of edge $e = (u \to v)$ is the "discrete gradient direction" along that edge.

Theorem 31.1 (Incidence factorization)**.** *For any orientation of the edges,* $\mathbf{L} = \mathbf{A}_B\mathbf{A}_B^\top$.

Why it works

Idea. $\mathbf{A}_B\mathbf{A}_B^\top$ sums, over all edges, the outer product of each column with itself. A column for edge $\{u, v\}$ is $\mathbf{e}_u - \mathbf{e}_v$ (up to sign, which squares away), and $(\mathbf{e}_u - \mathbf{e}_v)(\mathbf{e}_u - \mathbf{e}_v)^\top$ places $+1$ at (u, u) and (v, v) and -1 at (u, v) and (v, u). Summing over edges builds up exactly the degree on the diagonal and a -1 for each adjacency. That is $\mathbf{L}$.

Proof. Write $\mathbf{b}_e \in \mathbb{R}^n$ for the column of $\mathbf{A}_B$ indexed by edge e, so $\mathbf{b}_e = \mathbf{e}_{h(e)} - \mathbf{e}_{t(e)}$. Matrix multiplication gives

$$\mathbf{A}_B\mathbf{A}_B^\top = \sum_{e\in E} \mathbf{b}_e\mathbf{b}_e^\top.$$

For a single edge e with endpoints u, v (in either orientation),

$$\mathbf{b}_e\mathbf{b}_e^\top = (\mathbf{e}_u - \mathbf{e}_v)(\mathbf{e}_u - \mathbf{e}_v)^\top = \mathbf{e}_u\mathbf{e}_u^\top + \mathbf{e}_v\mathbf{e}_v^\top - \mathbf{e}_u\mathbf{e}_v^\top - \mathbf{e}_v\mathbf{e}_u^\top.$$

The sign of $\mathbf{b}_e$ does not matter: flipping the orientation negates $\mathbf{b}_e$, and $(-\mathbf{b}_e)(-\mathbf{b}_e)^\top = \mathbf{b}_e\mathbf{b}_e^\top$. Now sum over edges. The term $\mathbf{e}_u\mathbf{e}_u^\top$ contributes $+1$ to entry (u, u), once for each edge incident to u; the total at (u, u) is therefore $\deg(u)$. The cross terms $-\mathbf{e}_u\mathbf{e}_v^\top - \mathbf{e}_v\mathbf{e}_u^\top$ contribute -1 to entries (u, v) and (v, u), once for the single edge $\{u, v\}$ (the graph is simple). So $\left(\mathbf{A}_B\mathbf{A}_B^\top\right)_{ii} = \deg(i)$ and $\left(\mathbf{A}_B\mathbf{A}_B^\top\right)_{ij} = -1$ iff $\{i, j\} \in E$. That is precisely $\mathbf{L}$. □

Example 31.2 (Incidence matrix of the running graph)**.** Orient each edge from its smaller to its larger label: $e_1 = (1 \to 2)$, $e_2 = (2 \to 3)$, $e_3 = (3 \to 4)$, $e_4 = (1 \to 4)$, $e_5 = (3 \to 5)$. Then

$$\mathbf{A}_B = \begin{array}{c} \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{array} \begin{array}{c} \begin{array}{ccccc} e_1 & e_2 & e_3 & e_4 & e_5 \end{array} \\ \begin{pmatrix} -1 & 0 & 0 & -1 & 0 \\ +1 & -1 & 0 & 0 & 0 \\ 0 & +1 & -1 & 0 & -1 \\ 0 & 0 & +1 & +1 & 0 \\ 0 & 0 & 0 & 0 & +1 \end{pmatrix} \end{array}.$$

You should multiply $\mathbf{A}_B\mathbf{A}_B^\top$ and recover the $\mathbf{L}$ of Example 31.1; for instance row 3 of $\mathbf{A}_B$ has three nonzeros, so $(\mathbf{A}_B\mathbf{A}_B^\top)_{33} = 3 = \deg(3)$, and vertices $3, 4$ share only edge e_3, where row 3 holds -1 and row 4 holds $+1$, so $(\mathbf{A}_B\mathbf{A}_B^\top)_{34} = (-1)(+1) = -1$ as required.

31.1.2 The sum-over-edges form

The third definition skips matrices entirely and writes down what the quadratic form $x^\top \mathbf{L} x$ *does* to a vector. It is the most useful of the three.

Theorem 31.2 (The quadratic form)**.** *For every $x \in \mathbb{R}^n$,*

$$x^\top \mathbf{L} x \;=\; \sum_{\{i,j\}\in E} (x_i - x_j)^2 \;\geq\; 0.$$

Proof. Use the factorization of Theorem 31.1: $x^\top \mathbf{L} x = x^\top \mathbf{A}_B \mathbf{A}_B^\top x = (\mathbf{A}_B^\top x)^\top (\mathbf{A}_B^\top x) = \|\mathbf{A}_B^\top x\|^2$. The vector $\mathbf{A}_B^\top x \in \mathbb{R}^m$ has, in the coordinate of edge $e = (u \to v)$, the value $\mathbf{b}_e^\top x = x_v - x_u$. Hence

$$\|\mathbf{A}_B^\top x\|^2 = \sum_{e\in E} (x_{h(e)} - x_{t(e)})^2 = \sum_{\{i,j\}\in E} (x_i - x_j)^2.$$

The squares are orientation-independent and nonnegative, so $x^\top \mathbf{L} x \geq 0$. □

Focus

Three forms of the same matrix, used for three purposes:

- $\mathbf{L} = \mathbf{A}_D - \mathbf{A}$ — for *building* $\mathbf{L}$ from a graph and reading off entries.
- $\mathbf{L} = \mathbf{A}_B \mathbf{A}_B^\top$ — for *proving* structural facts (PSD, rank, null space).
- $x^\top \mathbf{L} x = \sum_{\{i,j\}\in E} (x_i - x_j)^2$ — for *reasoning* about smoothness, cuts, and eigenvalues.

Memorize the third. Almost every estimate in spectral graph theory begins by writing $x^\top \mathbf{L} x$ as a sum of squared edge-differences and bounding it.

31.2 Smoothness: what $x^\top \mathbf{L} x$ measures

Read x as a *signal* on the vertices: a real number x_i assigned to each vertex i. Theorem 31.2 says $x^\top \mathbf{L} x$ penalizes disagreement across edges. A signal that is nearly constant on each tightly connected region has small Laplacian quadratic form; a signal that jumps wildly between neighbors has large form. That penalty is why the Laplacian appears in clustering, semi-supervised learning, mesh processing, and image segmentation.

Intuition

The quantity $x^\top \mathbf{L} x$ is a *roughness meter*. Feed it a constant signal and the needle reads zero — there is no disagreement anywhere. Feed it a signal that respects the graph's community structure (high inside clusters, low between them, but changing only across the few inter-cluster edges) and the needle stays low. Feed it a checkerboard signal that flips sign across every edge and the needle pins to the maximum. The eigenvectors of $\mathbf{L}$ are, in order, the smoothest possible signals: the very smoothest is constant, the next is the gentlest non-constant variation the graph allows, and so on up to the roughest.

Let us make "smoothest non-constant signal" precise, because that is the Fiedler vector used throughout the next chapters.

Formal

The Spectral Foundations in One Place.

- **Eigenvectors and eigenvalues:** For a symmetric matrix $\mathbf{L} \in \mathbb{R}^{n\times n}$, a nonzero vector v satisfying $\mathbf{L}v = \lambda v$ is an *eigenvector*, and the scalar λ is its *eigenvalue*. Matrix multiplication simply dilates v by factor λ without altering its direction.
- **Positive Semidefiniteness (PSD):** A symmetric matrix is PSD ($\mathbf{L} \succeq 0$) if its quadratic form never goes negative: $x^\top \mathbf{L}x \geq 0$ for all $x \in \mathbb{R}^n$. Because Theorem 31.2 expressed $x^\top \mathbf{L}x = \sum w_{ij}(x_i - x_j)^2$ as a sum of nonnegative squares, the Graph Laplacian is automatically PSD. A fundamental theorem of linear algebra guarantees that every real symmetric PSD matrix has n mutually orthogonal eigenvectors and real, nonnegative eigenvalues: $0 \leq \lambda_1 \leq \lambda_2 \leq \cdots \leq \lambda_n$.
- **The Rayleigh quotient:** For any nonzero vector x, the ratio

$$R_{\mathbf{L}}(x) \;=\; \frac{x^\top \mathbf{L}x}{x^\top x} \;=\; \frac{\sum_{\{i,j\}\in E} w_{ij}(x_i - x_j)^2}{\sum_{i=1}^n x_i^2}$$

  normalizes the quadratic energy by the total squared length of the signal. If x is an eigenvector with eigenvalue λ, its Rayleigh quotient is exactly $R_{\mathbf{L}}(x) = \frac{x^\top(\lambda x)}{x^\top x} = \lambda$.
- **Courant–Fischer theorem:** The minimum of $R_{\mathbf{L}}(x)$ over all nonzero vectors is the ground state $\lambda_1 = 0$, achieved when $x = \mathbf{1}$. When we constrain x to be orthogonal to the constant vector $\mathbf{1}$ (meaning $\mathbf{1}^\top x = \sum_i x_i = 0$, so the signal has zero mean), the minimum of the Rayleigh quotient defines the second eigenvalue:

$$\lambda_2 \;=\; \min_{\substack{x\perp\mathbf{1}\\ x\neq 0}} \frac{x^\top \mathbf{L}x}{x^\top x}.$$

So λ_2 is the least roughness achievable by any signal that is orthogonal to constants (i.e. has the constant component removed). Its minimizer v_2 is the *Fiedler vector*: the smoothest way to assign real numbers to the vertices without making them all equal. We return to it at length in Chapter 32 on spectral partitioning; here we only need to see that the Laplacian's small eigenvalues encode the gentle, large-scale structure of the graph.

Connection

This whole section applies the companion *Linear Algebra* volume's treatment of *symmetric PSD matrices and the spectral theorem*. Three imported facts do all the work: (1) a real symmetric matrix has a real orthonormal eigenbasis; (2) it is PSD iff $x^\top Mx \geq 0$ for all x iff all eigenvalues are ≥ 0; (3) the Rayleigh quotient $x^\top Mx/x^\top x$ ranges over $[\lambda_{\min}, \lambda_{\max}]$, with the constrained minima given by Courant–Fischer. The graph Laplacian is the standard example that makes those abstractions concrete and combinatorial: eigenvalues measure connectivity, and eigenvectors give embeddings.

Common trap

$\mathbf{L}$ is positive *semi*definite, never positive definite, because $\mathbf{1}$ is always in its null space. So $\mathbf{L}$ is singular — $\det \mathbf{L} = 0$ always — and you cannot invert it. Algorithms that "solve $\mathbf{L}x = b$" really solve on the subspace $\mathbf{1}^{\perp}$, using the Moore–Penrose pseudoinverse $\mathbf{L}^{+}$, and they require the consistency condition $\mathbf{1}^{\top} b = 0$ (the entries of b must sum to zero). Forgetting that the system is singular is the single most common Laplacian bug. We meet $\mathbf{L}^{+}$ properly in Chapter 35.

31.3 The null space and connected components

The flat direction $\mathbf{1}$ is in the kernel of every Laplacian. The key fact is that the *entire* kernel is built from flat directions, one per connected component, and so the dimension of the null space counts the components. This is the first place where a purely algebraic quantity (a matrix rank) recovers a purely combinatorial one (the number of pieces).

Theorem 31.3 (Kernel counts components)**.** *Let G have connected components $C_1, \dots, C_k$. Then the null space of $\mathbf{L}$ has dimension exactly k, and is spanned by the indicator vectors $\mathbf{1}_{C_1}, \dots, \mathbf{1}_{C_k}$ (the vector $\mathbf{1}_{C_\ell}$ is 1 on the vertices of C_ℓ and 0 elsewhere). Equivalently,* $\operatorname{rank} \mathbf{L} = n - k$*. In particular G is connected iff $\lambda_2 > 0$, and the multiplicity of the eigenvalue 0 equals the number of components.*

Why it works

Idea. $\mathbf{L}x = 0$ forces $x^{\top}\mathbf{L}x = 0$, and by the sum-of-squares form that forces $x_i = x_j$ across every edge. A signal that never changes along an edge is constant on each connected component but free to take a different value per component. So the kernel is exactly the k-dimensional space of "locally constant" signals.

Proof. We show $\ker \mathbf{L} = \operatorname{span}\{\mathbf{1}_{C_1}, \dots, \mathbf{1}_{C_k}\}$.

($\supseteq$) Each $\mathbf{1}_{C_\ell}$ is constant on its component and zero elsewhere, so $(\mathbf{1}_{C_\ell})_i - (\mathbf{1}_{C_\ell})_j = 0$ for every edge $\{i, j\}$ (both endpoints lie in the same component, hence both get 1, or both get 0). By Theorem 31.2, $\mathbf{1}_{C_\ell}^{\top}\mathbf{L}\mathbf{1}_{C_\ell} = 0$. Because $\mathbf{L}$ is PSD, $x^{\top}\mathbf{L}x = 0 \iff \mathbf{L}x = 0$: indeed $0 = x^{\top}\mathbf{L}x = \|\mathbf{A}_B^{\top}x\|^2$ gives $\mathbf{A}_B^{\top}x = 0$, hence $\mathbf{L}x = \mathbf{A}_B(\mathbf{A}_B^{\top}x) = 0$. So each $\mathbf{1}_{C_\ell} \in \ker \mathbf{L}$.

($\subseteq$) Suppose $\mathbf{L}x = 0$. Then $x^{\top}\mathbf{L}x = 0$, so $\sum_{\{i,j\}\in E}(x_i - x_j)^2 = 0$, which forces $x_i = x_j$ for every edge $\{i, j\}$. Pick any vertex and follow paths: within a connected component every pair of vertices is joined by a path, and x is unchanged along each path's edges, so x is constant on the component. Therefore $x = \sum_\ell c_\ell \mathbf{1}_{C_\ell}$ for some scalars c_ℓ (the value x takes on C_ℓ). Hence $x \in \operatorname{span}\{\mathbf{1}_{C_\ell}\}$.

The indicators $\mathbf{1}_{C_1}, \dots, \mathbf{1}_{C_k}$ have disjoint supports, so they are linearly independent; thus $\dim \ker \mathbf{L} = k$. By rank–nullity $\operatorname{rank} \mathbf{L} = n - k$. The eigenvalue 0 has multiplicity $\dim \ker \mathbf{L} = k$; ordering eigenvalues $0 = \lambda_1 \le \cdots \le \lambda_n$, connectivity ($k = 1$) is equivalent to $\lambda_2 > 0$. □

Trace it yourself

Run this on Example 31.1. The graph is connected, so $k = 1$ and the kernel should be one-dimensional, spanned by $\mathbf{1} = (1, 1, 1, 1, 1)^{\top}$. Check: each row of $\mathbf{L}$ sums to zero, which is exactly the statement $\mathbf{L}\mathbf{1} = 0$. Now imagine deleting the single edge $\{3, 5\}$, which strands

vertex 5 on its own. With two components the kernel jumps to dimension 2, spanned by $\mathbf{1}_{\{1,2,3,4\}}$ and $\mathbf{1}_{\{5\}}$, and $\operatorname{rank}\mathbf{L}$ falls to $n-2=3$. The rank of $\mathbf{L}$ drops by exactly one for each new component.

31.3.1 An algorithm: counting components by Laplacian rank

Theorem 31.3 turns "how many connected components?" into "what is the rank of $\mathbf{L}$?". For a real graph you would just run BFS/DFS in $O(n+m)$ — nobody computes a rank to count components. But the reduction is conceptually important (it is how spectral methods "see" connectivity), and the same Gaussian elimination underlies the Laplacian solvers to come. Here is the explicit procedure.

Algorithm 126: ComponentsByRank(G) — count components via $\dim\ker\mathbf{L}$

Input: Graph $G=(V,E)$ on n vertices.
Output: Number of connected components k.

Build $\mathbf{L}=\mathbf{A}_D-\mathbf{A}\in\mathbb{R}^{n\times n}$.
$r\leftarrow 0$ `// rank accumulator`
for $c\leftarrow 1$ **to** n *(sweep columns)* **do**
  Find a pivot row p with $|\mathbf{L}_{p,c}|$ maximal among rows not yet used
  if $|\mathbf{L}_{p,c}|>\tau$ `// numerical zero tolerance` τ
  then
    Mark row p used; $r\leftarrow r+1$
    for *every unused row* q **do**
      $\mathbf{L}_{q,\bullet}\leftarrow\mathbf{L}_{q,\bullet}-\dfrac{\mathbf{L}_{q,c}}{\mathbf{L}_{p,c}}\mathbf{L}_{p,\bullet}$ `// eliminate column` c `below the pivot`
return $k\leftarrow n-r$

Why it works

The algorithm is plain Gaussian elimination with partial pivoting; the number of pivots it finds is the rank r of $\mathbf{L}$. By Theorem 31.3, $\operatorname{rank}\mathbf{L}=n-k$, so the missing $n-r$ pivots are exactly the k components. Each "rank-deficient" column — a column with no pivot — corresponds to one degree of freedom in the null space, i.e. one free component value. Correctness is therefore immediate from the theorem; the only subtlety is numerical (choosing τ so that genuine zeros are not mistaken for tiny nonzeros), which is why partial pivoting matters.

Cost summary

Building $\mathbf{L}$ costs $O(n^2)$ to allocate plus $O(m)$ to fill. Dense Gaussian elimination on an $n\times n$ matrix is $O(n^3)$ time and $O(n^2)$ space — dominant. So ComponentsByRank runs in $\Theta(n^3)$. This is much worse than the $O(n+m)$ of BFS, which is the point: *never* use rank to count components in practice. The value of the reduction is conceptual and it foreshadows fast Laplacian system solvers, which exploit sparsity to beat $O(n^3)$ by orders of magnitude.

Trace it yourself

Trace ComponentsByRank on the running graph's $\mathbf{L}$ to watch the rank come out at $4 = n - 1$. We sweep columns left to right; after each pivot we clear the entries below it. Starting from

$$\mathbf{L} = \begin{pmatrix} 2 & -1 & 0 & -1 & 0 \\ -1 & 2 & -1 & 0 & 0 \\ 0 & -1 & 3 & -1 & -1 \\ -1 & 0 & -1 & 2 & 0 \\ 0 & 0 & -1 & 0 & 1 \end{pmatrix},$$

column 1 pivots on the 2 in row 1; eliminating below gives rows $2, 4$ updated and a 2×2 Schur block forming. Carrying the elimination through columns 1–4 (each finds a pivot above tolerance τ) yields an upper-triangular shape with exactly four nonzero pivots and a final all-zero row:

$$\xrightarrow{\text{eliminate}} \begin{pmatrix} 2 & -1 & 0 & -1 & 0 \\ 0 & \frac{3}{2} & -1 & -\frac{1}{2} & 0 \\ 0 & 0 & \frac{7}{3} & -\frac{4}{3} & -1 \\ 0 & 0 & 0 & \frac{4}{7} & -\frac{4}{7} \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix}.$$

Four pivots $(2, \frac{3}{2}, \frac{7}{3}, \frac{4}{7})$, so $r = 4$ and $k = n - r = 5 - 4 = 1$: one component, as expected. The zero last row corresponds to the all-ones null direction, the single "free" degree of freedom predicted by the theorem. Add a second component (snip vertex 5 free) and a second zero pivot appears.

31.4 Weighted Laplacians

Real networks carry weights: edge $\{i, j\}$ has a conductance $w_{ij} > 0$ (think spring stiffness, pipe capacity, or similarity score). Everything above generalizes with the weights tucked into the right places, and no proof changes in substance.

Definition 31.3 (Weighted Laplacian)**.** Let $w : E \to \mathbb{R}_{>0}$ assign a positive weight to each edge. The *weighted degree* of i is $d_i = \sum_{j:\{i,j\}\in E} w_{ij}$. The *weighted adjacency* $\mathbf{A}$ has $\mathbf{A}_{ij} = w_{ij}$ for edges and 0 otherwise; the *weighted degree matrix* $\mathbf{A}_D$ is diagonal with d_i. The weighted Laplacian is

$$\mathbf{L} = \mathbf{A}_D - \mathbf{A}, \qquad \mathbf{L}_{ij} = \begin{cases} d_i & i = j, \\ -w_{ij} & \{i, j\} \in E, \\ 0 & \text{otherwise.} \end{cases}$$

The factorization survives with a diagonal weight matrix in the middle. Let $\mathbf{A}_W = \operatorname{diag}(w_e)_{e\in E} \in \mathbb{R}^{m\times m}$ hold the edge weights and let $\mathbf{A}_B$ be the (unweighted) signed incidence matrix.

Theorem 31.4 (Weighted factorization and quadratic form)**.** *With $\mathbf{A}_B, \mathbf{A}_W$ as above,*

$$\mathbf{L} = \mathbf{A}_B\, \mathbf{A}_W\, \mathbf{A}_B^\top = \sum_{e\in E} w_e\, \mathbf{b}_e \mathbf{b}_e^\top, \qquad x^\top \mathbf{L} x = \sum_{\{i,j\}\in E} w_{ij}\, (x_i - x_j)^2 \geq 0.$$

Consequently $\mathbf{L}$ is PSD, its kernel is spanned by the component indicators (Theorem 31.3 holds verbatim, since $w_{ij} > 0$ makes $(x_i - x_j)^2 = 0$ exactly when adjacent), and the spring-energy reading $\frac{1}{2} x^\top \mathbf{L} x$ is the elastic energy with stiffnesses w_{ij}.

Proof. $\mathbf{A}_B \mathbf{A}_W \mathbf{A}_B^\top = \sum_e w_e \, \mathbf{b}_e \mathbf{b}_e^\top$ because $\mathbf{A}_W$ is diagonal: the e-th column of $\mathbf{A}_B$ gets scaled by w_e before the outer product. Each $w_e \, \mathbf{b}_e \mathbf{b}_e^\top$ places $+w_e$ on the two diagonal entries of its endpoints and $-w_e$ on the two off-diagonal entries; summing yields d_i on the diagonal and $-w_{ij}$ off it, matching Definition 31.3. The quadratic form follows as in Theorem 31.2: writing $\mathbf{A}_W^{1/2} \mathbf{A}_B^\top x$ and taking its squared norm gives $\sum_e w_e (x_{h(e)} - x_{t(e)})^2$. Since each $w_e > 0$, the form vanishes iff every adjacent pair agrees, and the kernel argument of Theorem 31.3 carries over unchanged. □

Example 31.3 (A weighted twist on the running graph)**.** Keep the graph of Figure 31.2 but weight the cycle edges by 2 and the pendant edge $\{3, 5\}$ by 5:

$$w_{12} = w_{23} = w_{34} = w_{14} = 2, \quad w_{35} = 5.$$

Then $d_1 = d_2 = d_4 = 2 + 2 = 4$, $d_3 = 2 + 2 + 5 = 9$, $d_5 = 5$, and

$$\mathbf{L} = \begin{pmatrix} 4 & -2 & 0 & -2 & 0 \\ -2 & 4 & -2 & 0 & 0 \\ 0 & -2 & 9 & -2 & -5 \\ -2 & 0 & -2 & 4 & 0 \\ 0 & 0 & -5 & 0 & 5 \end{pmatrix}.$$

Rows still sum to zero, so $\mathbf{L}\mathbf{1} = 0$. Take the signal $x = (1, 1, 1, 1, 0)^\top$ that flags the pendant vertex. Then

$$x^\top \mathbf{L} x = \sum_{\{i,j\}} w_{ij} (x_i - x_j)^2 = w_{35}(1 - 0)^2 = 5,$$

because the only edge whose endpoints disagree is $\{3, 5\}$. Every cycle edge contributes 0. The form reads off the cost of separating vertex 5 — a preview of the cut interpretation below.

31.4.1 Cuts as quadratic forms

A special signal makes the cut connection exact. Given a vertex set $S \subseteq V$, let $\mathbf{1}_S \in \{0, 1\}^n$ be its indicator. Plug it in:

$$\mathbf{1}_S^\top \mathbf{L} \, \mathbf{1}_S = \sum_{\{i,j\} \in E} w_{ij} \big((\mathbf{1}_S)_i - (\mathbf{1}_S)_j \big)^2.$$

An edge contributes w_{ij} exactly when one endpoint is in S and the other is not — that is, exactly when the edge *crosses* the cut $(S, \bar{S})$. So

$$\boxed{\mathbf{1}_S^\top \mathbf{L} \, \mathbf{1}_S = w(S, \bar{S})}$$

the Laplacian quadratic form evaluated at a 0/1 indicator equals the weight of the corresponding cut (Figure 31.3). This is the starting point for spectral cut bounds: relaxing $\mathbf{1}_S \in \{0, 1\}^n$ to a real vector $x \perp \mathbf{1}$ turns the combinatorial minimum cut into the eigenvalue λ_2, and the gap between them is governed by Cheeger's inequality (Chapters 32–34). Minimum cuts, eigenvalues, and electrical flows can all be expressed through this quadratic form.

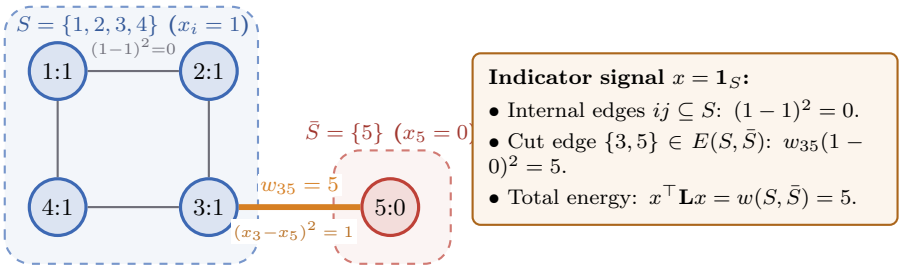


Figure 31.3: Cut weight as a Laplacian quadratic form on Example 31.3. Evaluating $x^\top \mathbf{L}x$ on the indicator vector $\mathbf{1}_S = (1,1,1,1,0)^\top$ zeroes out every internal edge where $x_i = x_j$ and sums the weights of only the edges crossing the cut $(S, \bar{S})$.

Application

This identity is why the Laplacian appears in so many applications.

- **Spectral clustering and image segmentation.** Minimizing a normalized version of $\mathbf{1}_S^\top \mathbf{L}\mathbf{1}_S$ over balanced S is NP-hard; relaxing the indicator to the Fiedler vector v_2 gives a polynomial-time cut that Cheeger's inequality certifies as near-optimal. Shi–Malik's normalized cuts segment images exactly this way.
- **Semi-supervised learning.** Given a few labeled vertices, the "harmonic" extension minimizes $x^\top \mathbf{L}x$ subject to matching the labels — the smoothest signal consistent with what you know. This is a single (constrained) Laplacian solve.
- **Graph drawing.** Plotting each vertex at coordinates $(v_2(i), v_3(i))$ from the second and third eigenvectors produces the classic "spectral layout" that lays out meshes and social graphs with edges short and clusters visible.
- **Mesh processing.** The cotangent Laplacian (a weighted Laplacian with geometry-derived w_{ij}) drives smoothing, parameterization, and the discrete heat equation in computer graphics.

31.5 Eigenvalues of small graphs, by hand

Computing a few spectra makes the abstract bounds concrete. We do two by hand and read the running example off them.

Example 31.4 (The path and the cycle)**.** For the cycle C_n every vertex has degree 2, so $\mathbf{L} = 2I - \mathbf{A}$ where $\mathbf{A}$ is the cyclic shift plus its inverse. The eigenvectors are the discrete Fourier modes $v_j^{(\omega)} = e^{2\pi i \omega j/n}$ for $\omega = 0, 1, \ldots, n-1$, and a direct computation gives

$$\lambda_\omega = 2 - 2\cos\Big(\frac{2\pi\omega}{n}\Big), \qquad \omega = 0, \ldots, n-1.$$

The smallest is $\lambda_0 = 0$ (the constant mode), and the second smallest is $\lambda_2 = 2 - 2\cos(2\pi/n) \approx (2\pi/n)^2$ for large n — the algebraic connectivity of a long cycle shrinks like $1/n^2$, matching the intuition that a big ring is only weakly connected end-to-end. For the path P_n the eigenvalues are $2 - 2\cos(\pi\omega/n)$, $\omega = 0, \ldots, n-1$, via cosine modes that respect the free (zero-flux) boundary.

Example 31.5 (The complete graph and a star)**.** For K_n every off-diagonal entry is -1 and every diagonal is $n-1$, so $\mathbf{L} = nI - \mathbf{1}\mathbf{1}^\top$. The vector $\mathbf{1}$ gives eigenvalue 0; any vector orthogonal to $\mathbf{1}$ gives $\mathbf{L}x = nx$. Hence the spectrum is 0 once and n with multiplicity $n-1$. The huge spectral gap $\lambda_2 = n$ says K_n is maximally well connected — you cannot cut it cheaply. The star $K_{1,n-1}$ has spectrum 0, then 1 with multiplicity $n-2$, then n once; the lone large eigenvalue is the center's high degree.

Trace it yourself

Now pin down the running graph (Example 31.1). It is connected, so $\lambda_1 = 0 < \lambda_2$. Two quick sanity checks bracket the spectrum without solving the characteristic polynomial:

- **Trace** $= \sum_i \lambda_i = \sum_i \deg(i) = 2+2+3+2+1 = 10 = 2m$. (The trace of $\mathbf{L}$ is $\sum_i \deg i = 2m$, always.)
- **Largest eigenvalue** $\lambda_n \le 2\,\Delta = 6$ where $\Delta = \max_i \deg i = 3$ (a standard Gershgorin bound: every Gershgorin disc is centered at $\deg i$ with radius $\deg i$, so $\lambda \le 2\deg i$). Also $\lambda_n \ge \Delta + 1 = 4$ for any graph with a vertex of degree Δ.

Solving numerically, the five eigenvalues are approximately $\{0,\ 0.83,\ 2.00,\ 2.69,\ 4.48\}$. They sum to 10 (trace check passes), the gap $\lambda_2 \approx 0.83$ is small because the pendant vertex 5 is loosely attached, and the Fiedler vector v_2 has its extreme entry at vertex 5 — the natural "odd one out" that the eigenvector pulls farthest from the cycle. Confirm the sum yourself; finding v_2 by hand is Exercise 31.6.

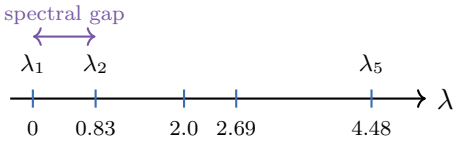


Figure 31.4: The Laplacian spectrum of the running graph on the real line. The gap from $\lambda_1 = 0$ to $\lambda_2 \approx 0.83$ is the algebraic connectivity; the small gap reflects the weakly attached pendant.

31.6 Basic spectral facts to carry forward

We collect the elementary identities that every later chapter assumes. They all fall out of the three definitions.

Proposition 31.1 (Spectral toolbox)**.** *Let* $\mathbf{L}$ *be the (weighted) Laplacian of* G *with eigenvalues* $0 = \lambda_1 \le \cdots \le \lambda_n$*. Then:*

1. $\sum_i \lambda_i = \operatorname{tr}\mathbf{L} = \sum_i d_i = 2\sum_e w_e$ *(twice the total edge weight).*
2. $\lambda_n \le 2\max_i d_i$*, and* $\lambda_n \ge \max_i d_i + 1$ *for unweighted graphs with at least one edge.*
3. $\lambda_2 > 0 \iff$ *G is connected; the multiplicity of 0 is the number of components.*
4. *(Monotonicity) Adding an edge or increasing a weight can only increase each* λ_i*: if* $\mathbf{L}' = \mathbf{L} + w\,\mathbf{b}_e\mathbf{b}_e^\top$ *with* $w \ge 0$ *then* $\lambda_i(\mathbf{L}') \ge \lambda_i(\mathbf{L})$ *for all* i*.*
5. *(Kirchhoff / Matrix–Tree) The number of spanning trees of a connected* G *equals* $\frac{1}{n}\prod_{i=2}^{n}\lambda_i$*, equivalently any cofactor of* $\mathbf{L}$*.*

Proof sketches. (1) The trace of $\mathbf{L}$ is $\sum_i d_i$ by Definition 31.3, and the trace equals the eigenvalue sum for any symmetric matrix. Each edge contributes its weight to two diagonal entries, so $\sum_i d_i = 2\sum_e w_e$. (2) By Gershgorin, every eigenvalue lies in some disc centered at d_i with radius $\sum_{j\neq i}|\mathbf{L}_{ij}| = d_i$, so $\lambda \le 2d_i \le 2\max_i d_i$. The lower bound follows from interlacing with the star subgraph at a max-degree vertex. (3) Theorem 31.3. (4) Adding $w\,\mathbf{b}_e\mathbf{b}_e^\top$ adds a PSD rank-one matrix; by the Courant–Fischer min–max characterization, adding a PSD matrix raises every λ_i (Weyl's monotonicity). Concretely the Rayleigh quotient $x^\top\mathbf{L}x/x^\top x$ increases pointwise. (5) This is Kirchhoff's theorem; its proof via the Cauchy–Binet expansion of a cofactor of $\mathbf{A}_B\mathbf{A}_B^\top$ appears in the spanning-trees material. The product form follows because the nonzero eigenvalues are $\lambda_2,\dots,\lambda_n$ and a cofactor equals $\frac{1}{n}\prod_{i\ge 2}\lambda_i$. □

Connection

Item (5) ties this chapter directly to three others. Cauchy–Binet on $\mathbf{A}_B\mathbf{A}_B^\top$ proves Kirchhoff's Matrix–Tree Theorem above; Chapter 35 (*Random Walks and Electrical Networks*) interprets $\mathbf{L}^+$ via effective resistance and shows that an edge's effective resistance equals the fraction of spanning trees containing it; and *Karger's* random-contraction bound (Chapter 11) also admits an effective-resistance view through $\mathbf{L}$. The companion *Linear Algebra* volume supplies the spectral theorem, Courant–Fischer, Weyl monotonicity, and the Moore–Penrose pseudoinverse on which these results rest.

31.6.1 The normalized Laplacian, briefly

Two variants normalize away the degrees, which matters when degrees vary wildly (web graphs, social networks). With $\mathbf{A}_D$ the degree matrix,

$$\mathbf{L}_{\text{sym}} = \mathbf{A}_D^{-1/2}\,\mathbf{L}\,\mathbf{A}_D^{-1/2} = I - \mathbf{A}_D^{-1/2}\mathbf{A}\,\mathbf{A}_D^{-1/2}, \qquad \mathbf{L}_{\text{rw}} = \mathbf{A}_D^{-1}\mathbf{L} = I - \mathbf{A}_D^{-1}\mathbf{A}.$$

$\mathbf{L}_{\text{sym}}$ is symmetric PSD with eigenvalues in $[0,2]$; $\mathbf{L}_{\text{rw}}$ is the generator of the lazy random walk and shares the same eigenvalues (they are similar matrices). Its quadratic form is

$$x^\top\mathbf{L}_{\text{sym}}\,x = \sum_{\{i,j\}\in E} w_{ij}\Big(\frac{x_i}{\sqrt{d_i}} - \frac{x_j}{\sqrt{d_j}}\Big)^2,$$

so a signal is "smooth" relative to degree. The normalized spectrum is what Cheeger's inequality and the random-walk mixing bounds actually use, so we flag it here and develop it fully in Chapters 32, 33, and 35.

Common trap

$\mathbf{L}_{\text{rw}} = \mathbf{A}_D^{-1}\mathbf{L}$ is *not symmetric*, so you must not feed it to a symmetric eigensolver or assume orthogonal eigenvectors. It is similar to the symmetric $\mathbf{L}_{\text{sym}}$ via $\mathbf{L}_{\text{rw}} = \mathbf{A}_D^{-1/2}\mathbf{L}_{\text{sym}}\mathbf{A}_D^{1/2}$, so they share eigenvalues, but their eigenvectors differ by the factor $\mathbf{A}_D^{\pm 1/2}$. Compute with $\mathbf{L}_{\text{sym}}$, then transform. Also: $\mathbf{A}_D^{-1/2}$ needs every degree positive, so isolated vertices (degree 0) must be removed first.

31.7 A worked solve: harmonic extension

As an application, we solve a small Laplacian system by hand: the harmonic-extension problem behind semi-supervised learning and electrical potentials. Fix the values of x on a set $B \subset V$ of "boundary" (labeled) vertices and choose the rest to minimize $x^\top \mathbf{L} x$. The minimizer is *harmonic* on the interior $I = V \setminus B$: each interior value equals the (weighted) average of its neighbors.

Theorem 31.5 (Harmonic extension)**.** *Partition* $\mathbf{L}$ *in block form by interior* I *and boundary* B*:*

$$\mathbf{L} = \begin{pmatrix} \mathbf{L}_{II} & \mathbf{L}_{IB} \\ \mathbf{L}_{BI} & \mathbf{L}_{BB} \end{pmatrix}, \qquad x = \begin{pmatrix} x_I \\ x_B \end{pmatrix}.$$

With x_B *fixed,* $x^\top \mathbf{L} x$ *is minimized by the unique solution of* $\mathbf{L}_{II}\, x_I = -\mathbf{L}_{IB}\, x_B$*, provided every component of* G *touches* B *(so* $\mathbf{L}_{II}$ *is invertible). At the optimum* $(\mathbf{L}x)_i = 0$ *for all* $i \in I$*: each interior vertex satisfies* $d_i x_i = \sum_{j \sim i} w_{ij} x_j$*, the weighted-average (harmonic) condition.*

Proof. Expand $f(x_I) = x^\top \mathbf{L} x = x_I^\top \mathbf{L}_{II} x_I + 2 x_I^\top \mathbf{L}_{IB} x_B + x_B^\top \mathbf{L}_{BB} x_B$ (using symmetry $\mathbf{L}_{BI} = \mathbf{L}_{IB}^\top$). This is a convex quadratic in x_I since $\mathbf{L}_{II} \succeq 0$. Setting the gradient to zero, $\nabla_{x_I} f = 2\mathbf{L}_{II} x_I + 2\mathbf{L}_{IB} x_B = 0$, i.e. $\mathbf{L}_{II} x_I = -\mathbf{L}_{IB} x_B$. If every component meets B, then $\mathbf{L}_{II}$ is a principal submatrix of $\mathbf{L}$ with no all-zero-row support left in the kernel, hence positive *definite* and invertible, giving a unique minimizer. The optimality condition $(\mathbf{L}x)_i = 0$ for $i \in I$ unpacks to $d_i x_i - \sum_{j \sim i} w_{ij} x_j = 0$. □

Example 31.6 (Solving on the running graph)**.** Use the unweighted $\mathbf{L}$ of Example 31.1. Pin the boundary $B = \{1, 5\}$ with $x_1 = 0$ and $x_5 = 1$ (think: vertex 1 grounded, vertex 5 at unit potential), and solve for the interior $I = \{2, 3, 4\}$. The interior block and coupling are

$$\mathbf{L}_{II} = \begin{pmatrix} 2 & -1 & 0 \\ -1 & 3 & -1 \\ 0 & -1 & 2 \end{pmatrix} \text{ (rows } 2,3,4), \qquad -\mathbf{L}_{IB} x_B = -\begin{pmatrix} -1 & 0 \\ 0 & -1 \\ -1 & 0 \end{pmatrix} \begin{pmatrix} 0 \\ 1 \end{pmatrix} = \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix}.$$

(The columns of $\mathbf{L}_{IB}$ are vertices 1 and 5; vertex 3 couples to vertex 5 with -1.) Solving $\mathbf{L}_{II} x_I = (0, 1, 0)^\top$:

$$2x_2 = x_3, \quad 3x_3 = x_2 + x_4 + 1, \quad 2x_4 = x_3.$$

By symmetry $x_2 = x_4$, so $x_3 = 2x_2$ and $3(2x_2) = 2x_2 + 1 \Rightarrow 4x_2 = 1 \Rightarrow x_2 = x_4 = \frac{1}{4}$, $x_3 = \frac{1}{2}$. The harmonic signal is $x = (0,\ \frac{1}{4},\ \frac{1}{2},\ \frac{1}{4},\ 1)$. Sanity check at the interior: vertex 3 has $d_3 = 3$ and neighbors $2, 4, 5$ with values $\frac{1}{4}, \frac{1}{4}, 1$, summing to $\frac{3}{2} = 3 \cdot \frac{1}{2} = d_3 x_3$, as required. This is exactly the electrical potential when a unit voltage is applied across vertices 1 and 5; the energy $x^\top \mathbf{L} x$ is the dissipated power, and its reciprocal is the effective conductance (see Chapter 35).

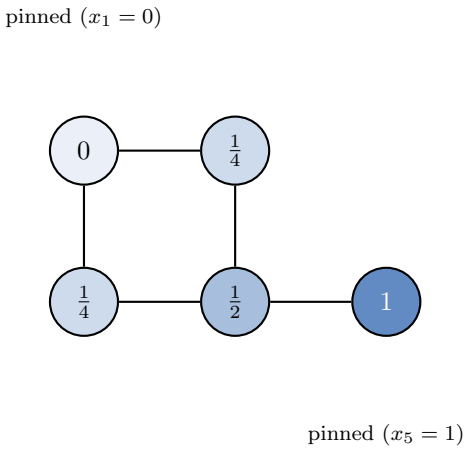


Figure 31.5: The harmonic extension of Example 31.6. Boundary vertices 1 and 5 are pinned at 0 and 1 (darker shading is higher potential); each interior value is the average of its neighbors. Read the picture as the steady voltage when a one-volt battery is wired across vertices 1 and 5 through unit resistors.

Algorithm 127: HARMONICEXTEND($\mathbf{L}, B, x_B$) — smoothest interior given boundary

Input: Laplacian $\mathbf{L}$; boundary set B with fixed values x_B; interior $I = V \setminus B$.
Output: Signal x minimizing $x^\top \mathbf{L} x$ subject to $x|_B = x_B$.

1 Form blocks $\mathbf{L}_{II}$ (rows/cols I) and $\mathbf{L}_{IB}$ (rows I, cols B).
2 Assert every component of G meets B `// else` $\mathbf{L}_{II}$ `singular`
3 Solve the SPD system $\mathbf{L}_{II}\, x_I = -\mathbf{L}_{IB}\, x_B$ `// Cholesky, CG, or a fast Laplacian solver`
4 $x \leftarrow$ concatenate x_I and x_B in vertex order
5 **return** x

Cost summary

The cost is one solve of an SPD system of size $|I| \le n$. Dense Cholesky is $O(|I|^3)$. Conjugate gradient runs in $O(m\sqrt{\kappa})$ where κ is the condition number of $\mathbf{L}_{II}$, which can be large. The central result of modern spectral graph theory — Spielman–Teng and successors — gives *nearly-linear* solvers running in $O(m \log^c n \log(1/\varepsilon))$ time for $\mathbf{L}x = b$ to accuracy ε, by preconditioning with a spectral sparsifier or a low-stretch spanning tree. Those solvers are developed in Chapter 41.

Connection

What This Made Possible: The Graph Laplacian connects combinatorial graph theory with physics and numerical linear algebra. It enabled Kirchhoff's Matrix-Tree Theorem, Tutte's spring embeddings for graph layout, electrical network analysis (effective resistance and random walk commute times), and modern near-linear time symmetric diagonally domi-

nant (SDDM) linear system solvers (Spielman–Teng 2004).

Limitations:

- *Continuous relaxation gap:* Solving $\mathbf{L}x = b$ produces continuous real-valued signals $x \in \mathbb{R}^n$. Converting a continuous potential vector back into discrete graph cuts or vertex partitions inevitably introduces rounding and relaxation errors.
- *Eigenvalue sensitivity:* Graph eigenvalues can be hypersensitive to edge additions. Adding a single edge can shift high-frequency eigenvalues significantly while leaving algebraic connectivity λ_2 nearly unchanged.
- *Computational overhead on small graphs:* For basic connectivity or shortest paths, discrete BFS/DFS runs in $O(n+m)$ integer steps with trivial cache lines, whereas computing Laplacian eigenvectors requires numerical eigensolvers (Lanczos, LOBPCG) that require floating-point arithmetic and iterative convergence.

Where We Go Next: We have established that $\lambda_2 > 0$ if and only if a graph is connected. But what does the exact magnitude of λ_2 tell us about the graph's bottleneck? In **Chapter 32 (Spectral Graph Theory)**, we prove the discrete Cheeger inequality, establishing that sweeping across the coordinates of the Fiedler vector guarantees a near-optimal combinatorial cut.

Exercises

Exercise 31.1 (Build it)**.** Write down the Laplacian of the 3×3 grid graph (nine vertices, edges between orthogonally adjacent cells). Verify by hand that every row sums to zero and that the diagonal is the degree sequence $(2, 3, 2, 3, 4, 3, 2, 3, 2)$. Confirm $\operatorname{tr} \mathbf{L} = 2m$ by counting edges.

Exercise 31.2 (Orientation independence)**.** Prove directly from Definition 31.2 that $\mathbf{A}_B \mathbf{A}_B^\top$ does not depend on the chosen edge orientations, without invoking Theorem 31.1. (Hint: flipping one edge negates one column of $\mathbf{A}_B$; show the outer product is unchanged.)

Exercise 31.3 (Disconnect and watch the rank)**.** Take the running graph (Example 31.1) and delete edges until it has exactly three components. Predict $\dim \ker \mathbf{L}$ and $\operatorname{rank} \mathbf{L}$ before computing, then verify by row-reducing the new $\mathbf{L}$. Identify the three indicator vectors spanning the kernel.

Exercise 31.4 (Cut = quadratic form)**.** For the weighted graph of Example 31.3, compute $\mathbf{1}_S^\top \mathbf{L}\, \mathbf{1}_S$ for $S = \{1, 2\}$, $S = \{5\}$, and $S = \{1, 2, 4\}$. Check each against the total weight of edges crossing the cut, and find the minimum-weight cut by trying all $2^4 - 1$ nontrivial vertex sets containing vertex 1.

Exercise 31.5 (Cycle spectrum)**.** Verify the cycle eigenvalue formula $\lambda_\omega = 2 - 2\cos(2\pi\omega/n)$ of Example 31.4 for C_4 by computing the 4×4 Laplacian's characteristic polynomial. Confirm the spectrum is $\{0, 2, 2, 4\}$, and identify which eigenvector is the Fiedler vector. Why is λ_2 degenerate (multiplicity 2) here?

Exercise 31.6 (Fiedler vector by hand)**.** For the running graph, the claim was $\lambda_2 \approx 0.83$ with the Fiedler vector taking its extreme value at vertex 5. Set up the eigenproblem $\mathbf{L}v = \lambda v$ with $v \perp \mathbf{1}$, exploit the symmetry $x_2 = x_4$, and reduce to a small polynomial. Solve for λ_2 to two decimals and confirm the sign pattern of v_2 separates the pendant from the cycle.

Exercise 31.7 (PSD without incidence)**.** Prove $\mathbf{L} \succeq 0$ directly from Definition 31.1 (degree minus adjacency), by completing the square on $x^\top \mathbf{L} x = \sum_i \deg(i) x_i^2 - 2\sum_{\{i,j\}} x_i x_j$. (You should recover $\sum_{\{i,j\}} (x_i - x_j)^2$.)

Exercise 31.8 (Weighted monotonicity)**.** Prove Proposition 31.1(4): if $\mathbf{L}' = \mathbf{L} + w\,\mathbf{b}_e \mathbf{b}_e^\top$ with $w \geq 0$ then $\lambda_i(\mathbf{L}') \geq \lambda_i(\mathbf{L})$ for all i. Use the Courant–Fischer min–max formula. Then exhibit a graph where adding one edge strictly increases λ_2 and another where it leaves λ_2 unchanged.

Exercise 31.9 (Harmonic = electrical)**.** Recompute the harmonic extension of Example 31.6 but ground vertex 4 instead of vertex 1 (set $x_4 = 0$, $x_5 = 1$). Interpret the result as voltages in a unit-resistance network, compute the energy $x^\top \mathbf{L} x$, and deduce the effective resistance between vertices 4 and 5.

Exercise 31.10 (Normalized quadratic form)**.** Prove $x^\top \mathbf{L}_{\mathrm{sym}} x = \sum_{\{i,j\} \in E} w_{ij} \big(x_i/\sqrt{d_i} - x_j/\sqrt{d_j}\big)^2$. Conclude that $\mathbf{L}_{\mathrm{sym}} \succeq 0$ and that its smallest eigenvalue is 0 with eigenvector $\mathbf{A}_D^{1/2}\mathbf{1}$. Why is this eigenvector, rather than $\mathbf{1}$, the "constant" of the normalized world?

Exercise 31.11 (Spectral upper bound on the cut)**.** Let $S \subset V$ with $|S| = s$. Using $\mathbf{1}_S^\top \mathbf{L}\,\mathbf{1}_S = w(S, \bar{S})$ and the Rayleigh quotient, prove $w(S, \bar{S}) \geq \lambda_2 \cdot \frac{s(n-s)}{n}$ for an unweighted graph. (Hint: project $\mathbf{1}_S$ onto $\mathbf{1}^\perp$ and bound its Rayleigh quotient below by λ_2.) This is the easy direction; the hard direction is Cheeger's inequality.

Exercise 31.12 (Implement and time it)**.** Implement ComponentsByRank (Algorithm 126) and compare it to BFS on random graphs of increasing size. Plot running time versus n and confirm the $\Theta(n^3)$ versus $O(n+m)$ gap. Then build $\mathbf{L}$ as a sparse matrix and time a single HarmonicExtend solve using a conjugate-gradient routine; observe how the iteration count grows with λ_n/λ_2.

Exercise 31.13 (Trace the elimination)**.** Reproduce the worked Gaussian-elimination trace following Algorithm 126: carry the running graph's $\mathbf{L}$ to upper-triangular form by hand, recording each pivot. Confirm the four pivots $2, \frac{3}{2}, \frac{7}{3}, \frac{4}{7}$ and the single zero row. Then delete edge $\{3, 5\}$ and re-run; show a second zero row appears and the pivot product drops, matching $k = 2$.

Exercise 31.14 (Spring energy minimization)**.** Read Figure 31.1 literally. Pin beads 1 and 3 at heights 0 and 1 in a triangle K_3 with unit springs and let bead 2 settle. Write the energy $\frac{1}{2} x^\top \mathbf{L} x$ as a function of x_2, minimize it, and check that the minimizer is the harmonic value $x_2 = \frac{1}{2}$. Relate the stored energy at the minimum to the effective resistance between the two pinned beads.

Chapter 32

Spectral Graph Theory

A graph is a combinatorial object: a set of vertices, a set of edges, no numbers anywhere unless you put them there. Yet the moment you write the graph down as a matrix – the adjacency matrix, or better, the Laplacian – you have handed it to linear algebra, and linear algebra hands back a list of real numbers, the *eigenvalues*. Those numbers encode a great deal of structure: whether the graph is connected, how many pieces it splits into, whether it is bipartite, how well-knit it is, and – the deepest fact in this chapter – whether it can be cut into two large pieces by severing only a few edges. Spectral graph theory is the study of the dictionary between the combinatorics of G and the spectrum of its matrices.

The previous chapter (Chapter 31) built the Laplacian $\mathbf{L} = \mathbf{D} - \mathbf{A}$ and proved its basic identity, the quadratic form $x^\top \mathbf{L} x = \sum_{(i,j)\in E}(x_i - x_j)^2$. We lean on that identity constantly here. This chapter does two things. First, we read graph structure straight off the spectrum: the multiplicity of the eigenvalue 0 counts connected components, the largest eigenvalue of the normalized Laplacian touches 2 exactly when a component is bipartite, and the *second* eigenvalue – the algebraic connectivity, or Fiedler value – measures how robustly the graph hangs together. Second, we prove the *Cheeger inequality*, the theorem that ties that second eigenvalue to the existence of a sparse cut, and whose constructive direction underlies spectral clustering (Chapter 33) and the analysis of expanders (Chapter 34).

We keep everything self-contained and we trace a real little graph by hand. By the end you should be able to take a six-vertex graph, write its Laplacian, find its spectrum, point at the Fiedler vector, and read off the near-optimal cut it suggests.

History

Spectral graph theory has roots in both differential geometry and discrete computer science. In 1970, American mathematician Jeff Cheeger was investigating Riemannian manifolds (smooth curved spaces equipped with notions of distance, angle, and volume), asking: how does the lowest non-zero eigenvalue λ_1 of the continuous Laplace–Beltrami operator (the smooth differential operator describing heat diffusion and membrane vibration) relate to the narrowest bottleneck or geometric constriction of the manifold? Cheeger proved the fundamental continuous inequality $h(M) \geq \frac{\lambda_1}{2}$, tying eigenvalues directly to isoperimetric geometry.
Simultaneously, computer scientists designing VLSI circuits and telephone switching networks faced the Sparsest Cut and Minimum Bisection problems: dividing a circuit graph into two

balanced halves while severing as few interconnect wires as possible. Sparsest cut is NP-hard, and early heuristic searches (such as the Kernighan–Lin algorithm) constantly became trapped in poor local minima. In 1972, William Donath and Alan Hoffman proposed relaxing the discrete binary cut assignment $x_i \in \{-1, +1\}$ to a continuous real vector $x \in \mathbb{R}^n$. Miroslav Fiedler (1973) recognized that this continuous quadratic relaxation is minimized precisely by the second Laplacian eigenvector v_2 (the Fiedler vector).

One question remained open: because v_2 is a continuous real vector, could its coordinates be misleading? What guarantee existed that rounding v_2 to a discrete cut would yield a cut anywhere near the true combinatorial optimum? In 1984–1985, Józef Dodziuk, and independently Noga Alon and Vitali Milman (and in 1989 Alistair Sinclair and Mark Jerrum for Markov chain mixing times) proved the *discrete Cheeger inequality*:

$$\tfrac{\mu_2}{2} \;\le\; \phi(G) \;\le\; \sqrt{2\,\mu_2}.$$

They proved that sorting vertices by their Fiedler vector coordinates and evaluating the $n-1$ sweep cuts provably guarantees a cut satisfying this quadratic bound. This turned spectral partitioning from a heuristic into a method with a proven guarantee, and the same ideas later fed into expander constructions, algorithm design, and nearly-linear-time Laplacian solvers (Chapter 41). What remains an active research frontier is obtaining tight, dimension-free higher-order Cheeger inequalities for simultaneous k-way graph partitioning.

32.1 Setup: which matrix, which spectrum

Fix an undirected graph $G = (V, E)$ with $n = |V|$ vertices and $m = |E|$ edges, possibly weighted with nonnegative weights $w_{ij} \ge 0$ on edge (i, j). Let $d_i = \sum_j w_{ij}$ be the (weighted) degree of vertex i and $\mathbf{D} = \operatorname{diag}(d_1, \dots, d_n)$. We will use three matrices.

Definition 32.1 (The three matrices)**.** The *adjacency matrix* $\mathbf{A}$ has $\mathbf{A}_{ij} = w_{ij}$ (and 0 on the diagonal, no self-loops). The *combinatorial Laplacian* is

$$\mathbf{L} \;=\; \mathbf{D} - \mathbf{A}.$$

The *normalized Laplacian* (assuming no isolated vertices, so every $d_i > 0$) is

$$\mathcal{L} \;=\; \mathbf{D}^{-1/2}\,\mathbf{L}\,\mathbf{D}^{-1/2} \;=\; \mathbf{I} - \mathbf{D}^{-1/2}\mathbf{A}\,\mathbf{D}^{-1/2}.$$

Both $\mathbf{L}$ and $\mathcal{L}$ are real symmetric, so by the spectral theorem each has n real eigenvalues and an orthonormal basis of real eigenvectors. We order the Laplacian eigenvalues

$$0 = \lambda_1 \le \lambda_2 \le \cdots \le \lambda_n$$

and the normalized ones

$$0 = \mu_1 \le \mu_2 \le \cdots \le \mu_n \le 2.$$

(We will earn each of these endpoints below; that $\lambda_1 = 0$ and $\mu_1 = 0$ comes from the all-ones vector, and $\mu_n \le 2$ is Lemma 32.3.)

Intuition

Think of a vector $x \in \mathbb{R}^V$ as an assignment of a real number to each vertex – a "signal" on the graph. The Laplacian quadratic form

$$x^\top \mathbf{L} x \;=\; \sum_{(i,j)\in E} w_{ij}\,(x_i - x_j)^2$$

measures how *rough* the signal is: it is large when the signal jumps across edges and zero when the signal is constant on each connected piece. Eigenvectors of small eigenvalue are the smoothest non-constant signals the graph admits. "Smooth" here means: neighbors get nearly equal values. The rest of the chapter works out the consequences of this idea.

Why two Laplacians? The combinatorial $\mathbf{L}$ is the natural one for many proofs and for the matrix-tree theorem; the normalized $\mathcal{L}$ is the right one for cuts and random walks, because it weights each vertex by its degree. When the graph is d-regular (every $d_i = d$) the two coincide up to scaling: $\mathcal{L} = \frac{1}{d}\mathbf{L}$, so $\mu_k = \lambda_k/d$. We will state the structural facts for whichever matrix makes them cleanest and translate when it matters.

Focus

Three questions organize the chapter.

1. **Structure from the spectrum.** What do λ_1, λ_n, multiplicities, and the bipartite case tell us about G? (Section 32.2)
2. **The second eigenvalue.** Why is λ_2 – the algebraic connectivity, the Fiedler value – the single most informative number, and what does its eigenvector look like? (Section 32.3)
3. **Cheeger.** How does λ_2 (or μ_2) sandwich the sparsest cut, and how do we turn the eigenvector into an actual cut? (Section 32.4)

32.1.1 The variational principles we will reuse

Everything spectral here flows from two standard facts about symmetric matrices. We state them once and cite them by name.

Lemma 32.1 (Rayleigh quotient and the bottom of the spectrum)**.** *Let M be symmetric with eigenvalues $\nu_1 \le \cdots \le \nu_n$ and orthonormal eigenvectors $v_1, \dots, v_n$. The* Rayleigh quotient *is $R_M(x) = \dfrac{x^\top M x}{x^\top x}$ for $x \neq 0$. Then*

$$\nu_1 = \min_{x\neq 0} R_M(x), \qquad \nu_k = \min_{\substack{x\neq 0 \\ x\perp v_1,\dots,v_{k-1}}} R_M(x),$$

and the minimizer of the k-th problem is v_k.

Lemma 32.2 (Courant–Fischer min–max)**.** *With the same notation,*

$$\nu_k \;=\; \min_{\dim S = k}\; \max_{x\in S\setminus\{0\}} R_M(x) \;=\; \max_{\dim S = n-k+1}\; \min_{x\in S\setminus\{0\}} R_M(x),$$

the optimum over k-dimensional subspaces $S \subseteq \mathbb{R}^n$.

Both are proved in any linear algebra text and in the companion *Linear Algebra* volume; we use them as tools. The first gives us λ_2 as a constrained minimum of $\sum(x_i - x_j)^2$ over signals orthogonal to the constant vector. The second lets us compare eigenvalues across edge insertions and across the two Laplacians without ever computing them.

Connection

The Rayleigh-quotient view is the direct link to the companion *Linear Algebra* volume's treatment of symmetric matrices and the min–max characterization, and to Chapter 31's identity. It is also the bridge to Chapter 35 (random walks and electrical networks): the eigenvectors of $\mathcal{L}$ are, up to the $\mathbf{D}^{1/2}$ change of variables, the eigenvectors of the random-walk transition matrix $\mathbf{P} = \mathbf{D}^{-1}\mathbf{A}$, and the quadratic form is the energy dissipated by a unit current. Keep that triangle – Laplacian, random walk, electrical energy – in view.

32.2 Reading graph structure off the spectrum

32.2.1 The kernel: connectivity and components

Theorem 32.1 (Multiplicity of 0 counts components)**.** *For the combinatorial Laplacian* $\mathbf{L}$ *of an undirected graph, the multiplicity of the eigenvalue* 0 *equals the number* c *of connected components of* G*. The same holds for* $\mathcal{L}$ *(with no isolated vertices). In particular* $\lambda_2 > 0$ *if and only if* G *is connected.*

Proof. $\mathbf{L}$ is positive semidefinite because $x^\top \mathbf{L} x = \sum_{(i,j)\in E} w_{ij}(x_i - x_j)^2 \geq 0$ for all x; hence every eigenvalue is ≥ 0, and $\lambda = 0$ has eigenvectors exactly the x with $x^\top \mathbf{L} x = 0$, i.e. the kernel of $\mathbf{L}$.

Now $x^\top \mathbf{L} x = 0$ forces $x_i = x_j$ for every edge (i, j) with $w_{ij} > 0$. Walking along edges, x is constant on each connected component. Conversely any vector that is constant on each component makes every term vanish and so lies in the kernel. The indicator vectors $\mathbf{1}_{C_1}, \ldots, \mathbf{1}_{C_c}$ of the c components are therefore a basis of the kernel: they are linearly independent (by disjoint coordinate supports) and they span it (any kernel vector is a combination of them). Hence $\dim\ker \mathbf{L} = c$, which is the multiplicity of 0. For $\mathcal{L}$, note $\mathcal{L} = \mathbf{D}^{-1/2}\mathbf{L}\mathbf{D}^{-1/2}$ is a congruence by an invertible matrix, so it has the same rank and the same number of zero eigenvalues; the kernel vectors are $\mathbf{D}^{1/2}\mathbf{1}_{C_\ell}$. □

The all-ones vector $\mathbf{1}$ is always in the kernel of $\mathbf{L}$ (each row sums to 0), which is why $\lambda_1 = 0$ for every graph. The content of the theorem is that 0 has *no other* eigenvectors precisely when the graph is connected.

Common trap

A graph with λ_2 very small but positive is connected, yet only barely – it has a near-disconnection, a bottleneck. Beginners read $\lambda_2 = 0$ as "disconnected" and stop there. The interesting regime is λ_2 tiny but nonzero: the graph is one piece, but a thin one, and the Fiedler vector points right at the weak seam. Cheeger's inequality is about this regime.

32.2.2 The top of the spectrum and bipartiteness

For the normalized Laplacian the top eigenvalue μ_n has its own clean meaning.

Lemma 32.3 ($\mu_n \le 2$, with equality for bipartite)**.** *For the normalized Laplacian, $0 \le \mu_k \le 2$ for all k. On a connected graph, $\mu_n = 2$ if and only if G is bipartite.*

Proof. Write $y = \mathbf{D}^{-1/2}x$ so that, with the substitution, the normalized Rayleigh quotient becomes

$$\frac{x^\top \mathcal{L} x}{x^\top x} = \frac{\sum_{(i,j)\in E} w_{ij}(y_i - y_j)^2}{\sum_i d_i\, y_i^2}.$$

By $(y_i - y_j)^2 \le 2(y_i^2 + y_j^2)$ and $\sum_{(i,j)} w_{ij}(y_i^2 + y_j^2) = \sum_i d_i y_i^2$, the numerator is at most 2 times the denominator, so $\mu_n \le 2$. Equality needs $(y_i + y_j)^2 = 0$ on every edge, i.e. $y_i = -y_j$ across each edge with a consistent sign; this two-colors the vertices, so the graph is bipartite. Conversely if G is bipartite with sides A, B, take $y_i = +1$ on A and $y_i = -1$ on B; then every $(y_i - y_j)^2 = 4 = 2(y_i^2 + y_j^2)$, achieving the bound, so $\mu_n = 2$. □

Intuition

$\mu_n = 2$ is the spectral fingerprint of bipartiteness: there is a signal that flips sign across *every* edge, the roughest possible signal. "Almost bipartite" graphs have μ_n close to 2 but below it. So the normalized spectrum brackets the graph from both ends: μ_2 small says "almost disconnected"; $2 - \mu_n$ small says "almost bipartite." Both are kinds of weak structure, and both are caught spectrally.

32.2.3 A first worked spectrum: the path P_4

Let us pin all of this to a concrete graph before going further. Take the path on four vertices, P_4: edges 1–2, 2–3, 3–4, unit weights.



The degrees are $1, 2, 2, 1$, so

$$\mathbf{L}(P_4) = \begin{pmatrix} 1 & -1 & 0 & 0 \\ -1 & 2 & -1 & 0 \\ 0 & -1 & 2 & -1 \\ 0 & 0 & -1 & 1 \end{pmatrix}.$$

The eigenvalues of the path Laplacian are known in closed form: $\lambda_k = 2 - 2\cos\big(\frac{(k-1)\pi}{n}\big)$ for $k = 1, \dots, n$. With $n = 4$:

$$\lambda_1 = 0, \quad \lambda_2 = 2 - 2\cos\tfrac{\pi}{4} = 2 - \sqrt{2} \approx 0.586, \quad \lambda_3 = 2, \quad \lambda_4 = 2 + \sqrt{2} \approx 3.414.$$

The Fiedler vector (eigenvector for λ_2) is $v_2 = (\cos\frac{\pi}{8}, \cos\frac{3\pi}{8}, \cos\frac{5\pi}{8}, \cos\frac{7\pi}{8}) \approx (0.92, 0.38, -0.38, -0.92)$. Plotting the eigenvectors as vertical bar signals over the four vertices makes the roughness dictionary concrete: as λ_k grows, the signal oscillates faster across edges, and v_2 is the smoothest non-constant

wave – monotone along the path and changing sign exactly once, between vertices 2 and 3, which is precisely where you would cut P_4 into two equal halves.

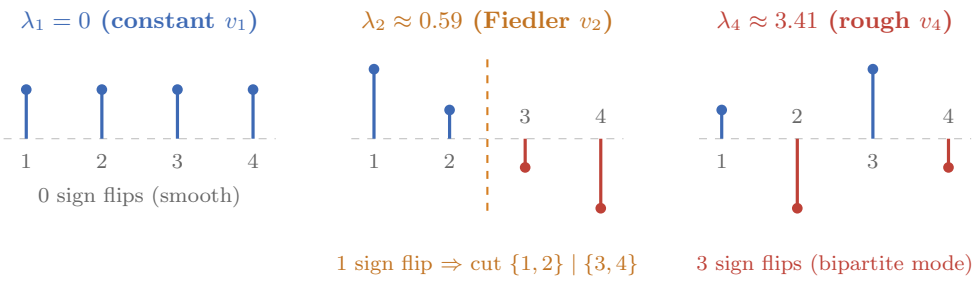


Figure 32.1: Three Laplacian eigenvectors of P_4 plotted as vertex signals. Higher eigenvalues correspond to more sign flips across edges; the Fiedler vector v_2 crosses zero once, bisecting the path.

That single sign change in Figure 32.1 is what spectral partitioning exploits. For trees one can prove the Fiedler vector always has exactly one sign change (its positive and negative nodal domains are connected subtrees).

Trace it yourself

Confirm $\lambda_2 = 2 - \sqrt{2}$ by plugging v_2 into $\mathbf{L}v_2 = \lambda_2 v_2$ for, say, the first coordinate: row 1 gives $v_{2,1} - v_{2,2} = 0.92 - 0.38 = 0.54$, and $\lambda_2 v_{2,1} = 0.586 \cdot 0.92 \approx 0.54$. Match. Now do the same hand-check for the second coordinate (row 2): you should get $-v_{2,1} + 2v_{2,2} - v_{2,3} = -0.92 + 0.76 + 0.38 = 0.22$ against $0.586 \cdot 0.38 \approx 0.22$.

32.2.4 Two more reference spectra

Memorizing a handful of spectra builds intuition fast.

The complete graph K_n. Here $\mathbf{L} = n\mathbf{I} - \mathbf{J}$ where $\mathbf{J}$ is the all-ones matrix. So $\mathbf{L}\mathbf{1} = 0$, and on $\mathbf{1}^\perp$ we have $\mathbf{J}x = 0$, hence $\mathbf{L}x = nx$. The spectrum is

$$\lambda_1 = 0, \qquad \lambda_2 = \lambda_3 = \cdots = \lambda_n = n.$$

The algebraic connectivity is as large as it can be, $\lambda_2 = n$: K_n is the most robustly connected graph on n vertices, with no weak cut anywhere.

The cycle C_n. By circulant diagonalization the eigenvalues are $\lambda_k = 2 - 2\cos\left(\frac{2\pi(k-1)}{n}\right)$, $k = 1, \ldots, n$. For C_4: $0, 2, 2, 4$. The algebraic connectivity $\lambda_2 = 2 - 2\cos\frac{2\pi}{n} \approx \frac{4\pi^2}{n^2}$ for large n – it decays like $1/n^2$, reflecting that a long cycle is easy to cut (snip any two edges).

Reading note

Graph spectra originated in 1930s–50s chemistry and molecular physics, where adjacency eigenvalues approximate π-electron orbital energies. In 1966, Mark Kac famously asked "Can one hear the shape of a drum?" For graphs the answer is no: there exist *cospectral* non-isomorphic graphs (graphs with identical eigenvalues that are not isomorphic; the smallest pair has 6 vertices). The spectrum captures connectivity, but it does not determine the graph.

32.3 The second eigenvalue: algebraic connectivity

Definition 32.2 (Algebraic connectivity / Fiedler value)**.** The *algebraic connectivity* of G is $a(G) = \lambda_2(\mathbf{L})$, the second-smallest Laplacian eigenvalue. A unit eigenvector for λ_2 is a *Fiedler vector*.

By Lemma 32.1 and the fact that $v_1 = \mathbf{1}/\sqrt{n}$,

$$\lambda_2 \;=\; \min_{\substack{x \neq 0 \\ x \perp \mathbf{1}}} \frac{\sum_{(i,j)\in E} w_{ij}(x_i - x_j)^2}{\sum_i x_i^2}. \tag{32.1}$$

In words: among all signals whose entries sum to zero – the "balanced" signals – λ_2 is the smallest possible roughness per unit energy. A graph with a thin bottleneck admits a balanced signal that is nearly constant on each side of the bottleneck and only changes across the few cut edges; such a signal has tiny numerator and order-1 denominator, forcing λ_2 small. Cheeger's inequality makes this intuition quantitative.

32.3.1 Why λ_2 deserves the name "connectivity"

Proposition 32.1 (Monotonicity and basic bounds)**.** *Algebraic connectivity behaves like a connectivity measure should.*

1. (Edge monotonicity.) *Adding an edge (or increasing a weight) never decreases any λ_k; in particular λ_2 can only grow. Removing an edge can only shrink it.*
2. (Bounded by vertex connectivity.) *$\lambda_2(G) \le \kappa(G)$, the vertex connectivity (minimum number of vertices whose deletion disconnects G), for $G \neq K_n$. Hence $\lambda_2 \le \delta(G)$, the minimum degree.*
3. (Subgraph by deletion.) *If G' is G with a vertex deleted, $\lambda_2(G') \ge \lambda_2(G) - 1$.*

Proof of (1). Adding edge $e = (p, q)$ with weight w replaces $\mathbf{L}$ by $\mathbf{L} + w\,\mathbf{L}_e$, where $\mathbf{L}_e = (\mathbf{e}_p - \mathbf{e}_q)(\mathbf{e}_p - \mathbf{e}_q)^\top$ is itself positive semidefinite. For any subspace S and any $x \in S$, $x^\top(\mathbf{L} + w\mathbf{L}_e)x \ge x^\top \mathbf{L} x$, so the Courant–Fischer max over S only increases; taking the min over S gives $\lambda_k(\mathbf{L} + w\mathbf{L}_e) \ge \lambda_k(\mathbf{L})$. (This is Weyl monotonicity for a PSD perturbation.) □

Part (1) is the cleanest evidence that λ_2 is a connectivity. More edges ⇒ harder to disconnect ⇒ larger λ_2, and the inequality is exact and one-line. Part (2), Fiedler's original bound $\lambda_2 \le \kappa$, says the spectral measure never overstates how connected the graph is; we omit its proof (a clean argument tests the Rayleigh quotient on a vector supported away from a vertex cut).

32.3.2 The Fiedler vector as an embedding

Equation (32.1) says the Fiedler vector v_2 places each vertex on the real line so that adjacent vertices stay close (small $\sum(x_i - x_j)^2$) while the configuration stays spread out and balanced ($x \perp \mathbf{1}$, fixed energy). That is a one-dimensional *embedding* of the graph that respects adjacency. Sorting vertices by their Fiedler coordinate and cutting the order somewhere is the *spectral ordering*; it is the standard heuristic for graph partitioning, sparse-matrix reordering (envelope/bandwidth reduction), and is the seed of Chapter 33.

Why it works

Why does minimizing $\sum_{(i,j)}(x_i - x_j)^2$ subject to $x \perp \mathbf{1}$ and $\|x\| = 1$ give a good cut? Because the objective is a sum over edges of squared coordinate gaps. To make it small you must give the two endpoints of almost every edge nearly equal coordinates – so densely connected clumps collapse to nearly a single point on the line, and the only place coordinates are allowed to differ a lot is across the *few* edges joining different clumps. The constraint $x \perp \mathbf{1}$ forbids the trivial all-equal solution and forces the clumps to spread to opposite signs. So the sign of v_2 is, heuristically, a label for "which clump."

32.4 The Cheeger inequality

The central theorem of the chapter turns the coordinate-gap heuristic above into a quantitative bound, and its proof yields an explicit rounding algorithm.

32.4.1 Conductance: the combinatorial quantity

Definition 32.3 (Cut, volume, conductance)**.** For $S \subseteq V$, write $S^c = V \setminus S$. The *edge boundary* is $\partial S = \{(i,j) \in E : i \in S, j \in S^c\}$, with cut weight $w(\partial S) = \sum_{(i,j)\in\partial S} w_{ij}$. The *volume* of S is $\mathrm{vol}(S) = \sum_{i\in S} d_i$. The *conductance* (a.k.a. Cheeger ratio, sparsity) of the cut is

$$\phi(S) = \frac{w(\partial S)}{\min\{\mathrm{vol}(S),\, \mathrm{vol}(S^c)\}},$$

and the *conductance of the graph* is $\phi(G) = \min_{\emptyset \neq S \subsetneq V} \phi(S)$.

Small $\phi(G)$ means there is a way to split the vertices into two pieces, each of substantial volume, with only a few edges crossing – a sparse cut, a bottleneck. Large $\phi(G)$ means no such split exists; the graph is an *expander* (Chapter 34). Computing $\phi(G)$ exactly is NP-hard (it is essentially sparsest cut), so a polynomial-time two-sided estimate is valuable. Cheeger's inequality provides one.

Intuition

Why divide by the smaller volume rather than by $|S|$? Because we want to forbid trivial cuts that peel off a tiny set (a single vertex has cut $= d_i$ but is no real bottleneck). Normalizing by volume charges each side for the degree it contains, so a sparse cut must isolate a genuinely *large* (high-volume) region cheaply. This is exactly the normalization the random walk sees, which is why μ_2 – not λ_2 – is the eigenvalue that brackets ϕ.

32.4.2 Statement

Theorem 32.2 (Cheeger inequality for graphs)**.** *Let μ_2 be the second-smallest eigenvalue of the normalized Laplacian $\mathcal{L}$ of a connected graph. Then*

$$\frac{\mu_2}{2} \;\le\; \phi(G) \;\le\; \sqrt{2\,\mu_2}.$$

The two inequalities go in opposite directions and have completely different characters.

- The **easy direction** $\frac{\mu_2}{2} \le \phi(G)$ says "a good cut forces a small eigenvalue." Its proof is a one-line Rayleigh-quotient test: plug the indicator of the sparse cut into the variational formula for μ_2.
- The **hard direction** $\phi(G) \le \sqrt{2\mu_2}$ says "a small eigenvalue forces a good cut," and its proof is *constructive*: sweeping the sorted coordinates of the eigenvector (Cheeger rounding) produces a prefix cut of conductance at most $\sqrt{2\mu_2}$.

We prove both directions in turn.

32.4.3 Proof of the easy direction

$\frac{\mu_2}{2} \le \phi(G)$. Let S achieve $\phi(G)$, with $\mathrm{vol}(S) \le \mathrm{vol}(S^c)$ (relabel if needed). We exhibit a test vector for the variational characterization of μ_2. In the y-variables of Lemma 32.3,

$$\mu_2 \;=\; \min_{\substack{y \neq 0 \\ \sum_i d_i y_i = 0}} \frac{\sum_{(i,j)\in E} w_{ij}(y_i - y_j)^2}{\sum_i d_i\, y_i^2},$$

where the constraint $\sum_i d_i y_i = 0$ is "$x \perp \mathbf{D}^{1/2}\mathbf{1}$," the orthogonality to the bottom eigenvector of $\mathcal{L}$. Choose

$$y_i \;=\; \begin{cases} \dfrac{1}{\mathrm{vol}(S)}, & i \in S, \\[2mm] -\dfrac{1}{\mathrm{vol}(S^c)}, & i \in S^c. \end{cases}$$

This y is constant on each side, so $(y_i - y_j)^2$ is nonzero only on boundary edges, each contributing $\big(\frac{1}{\mathrm{vol}(S)} + \frac{1}{\mathrm{vol}(S^c)}\big)^2$. The constraint holds: $\sum_i d_i y_i = \mathrm{vol}(S)\frac{1}{\mathrm{vol}(S)} - \mathrm{vol}(S^c)\frac{1}{\mathrm{vol}(S^c)} = 0$. Compute both pieces:

$$\text{numerator} = w(\partial S)\Big(\tfrac{1}{\mathrm{vol}(S)} + \tfrac{1}{\mathrm{vol}(S^c)}\Big)^2, \qquad \text{denominator} = \tfrac{1}{\mathrm{vol}(S)} + \tfrac{1}{\mathrm{vol}(S^c)}$$

(the denominator because $\sum_i d_i y_i^2 = \mathrm{vol}(S)\frac{1}{\mathrm{vol}(S)^2} + \mathrm{vol}(S^c)\frac{1}{\mathrm{vol}(S^c)^2}$). Their ratio is

$$\mu_2 \le \frac{w(\partial S)\big(\frac{1}{\mathrm{vol}(S)} + \frac{1}{\mathrm{vol}(S^c)}\big)^2}{\frac{1}{\mathrm{vol}(S)} + \frac{1}{\mathrm{vol}(S^c)}} = w(\partial S)\Big(\tfrac{1}{\mathrm{vol}(S)} + \tfrac{1}{\mathrm{vol}(S^c)}\Big) \le w(\partial S)\cdot\frac{2}{\mathrm{vol}(S)} = 2\,\phi(G),$$

using $\mathrm{vol}(S) \le \mathrm{vol}(S^c)$ so that $\frac{1}{\mathrm{vol}(S^c)} \le \frac{1}{\mathrm{vol}(S)}$. Rearranging, $\mu_2/2 \le \phi(G)$. □

Intuition

The easy direction is just "the cut indicator is a valid test signal." A sparse cut gives a near-piecewise-constant balanced signal of small roughness, and μ_2 is by definition the minimum roughness, so μ_2 is pushed down. The factor 2 is the slack between a sharp 0/1 indicator and the genuinely smoothest signal.

32.4.4 Proof sketch of the hard direction (the sweep)

The hard direction turns the eigenvector into a cut. Let $f = v_2$ be the Fiedler vector of $\mathcal{L}$, written back in vertex space as $g = \mathbf{D}^{-1/2} f$ so that the relevant Rayleigh quotient is

$$\mu_2 \;=\; \frac{\sum_{(i,j)\in E} w_{ij}(g_i - g_j)^2}{\sum_i d_i g_i^2}, \qquad \sum_i d_i g_i = 0.$$

We will produce a *threshold cut*: sort the vertices by their value g_i, and consider the $n-1$ cuts $S_t = \{i : g_i \le \theta_t\}$ as the threshold θ_t sweeps through the sorted values. The claim is that the *best* of these sweep cuts already has conductance $\le \sqrt{2\mu_2}$.

Algorithm 128: SpectralSweep – rounding a Fiedler vector to a cut

Input: Connected graph $G = (V, E, w)$, normalized Fiedler vector $f = v_2$ of $\mathcal{L}$.
Output: A vertex set $S^\star$ with $\phi(S^\star) \le \sqrt{2\,\mu_2}$.

```
g ← D^{-1/2} f;
Sort vertices so that g_{σ(1)} ≤ g_{σ(2)} ≤ ··· ≤ g_{σ(n)};
φ* ← +∞;   S* ← ∅;
for t = 1 to n − 1 do
    S_t ← {σ(1), ..., σ(t)};                          // prefix in sorted order
    Update w(∂S_t) and vol(S_t) incrementally from S_{t−1};
    φ(S_t) ← w(∂S_t) / min{vol(S_t), vol(V) − vol(S_t)};
    if φ(S_t) < φ* then
        φ* ← φ(S_t);   S* ← S_t;
return S*;
```

Theorem 32.3 (Sweep guarantee, the hard direction)**.** SpectralSweep *returns a set $S^\star$ with $\phi(S^\star) \le \sqrt{2\,\mu_2}$. In particular $\phi(G) \le \sqrt{2\,\mu_2}$.*

Proof sketch. The proof averages over random thresholds. Shift and scale g so that the median vertex (by volume) sits at 0 and so that $\sum_i d_i g_i^2$ is normalized; one then splits g into its positive part g^+ and negative part g^- and works with whichever side carries at most half the volume – call that vector h, supported on a set of volume $\le \frac{1}{2}\operatorname{vol}(V)$, with $\dfrac{\sum_{(i,j)} w_{ij}(h_i - h_j)^2}{\sum_i d_i h_i^2} \le \mu_2$ still holding by a short calculation. Write ρ for this Rayleigh quotient, so $\rho \le \mu_2$.

Now consider drawing a random threshold θ with density proportional to $2|\theta|$ over the range of h (equivalently, threshold the values h_i^2 uniformly). For such a random threshold define the sweep set $S_\theta = \{i : h_i^2 > \theta\}$ – a prefix set in the sorted order. Two expectation computations finish it:

- *Expected boundary.* A boundary edge (i,j) of S_θ is exactly one with θ between h_i^2 and h_j^2, which happens with probability $|h_i^2 - h_j^2| = |h_i - h_j|\,|h_i + h_j|$. By Cauchy–Schwarz over edges,

$$\mathbb{E}\, w(\partial S_\theta) = \sum_{(i,j)} w_{ij}|h_i - h_j||h_i + h_j| \le \sqrt{\sum w_{ij}(h_i - h_j)^2}\;\sqrt{\sum w_{ij}(h_i + h_j)^2}.$$

  The first factor is $\sqrt{\rho \sum d_i h_i^2}$. The second is $\le \sqrt{2\sum d_i h_i^2}$ by $(h_i + h_j)^2 \le 2(h_i^2 + h_j^2)$. So $\mathbb{E}\, w(\partial S_\theta) \le \sqrt{2\rho}\,\sum_i d_i h_i^2$.

- *Expected volume.* A vertex i lies in S_θ with probability $h_i^2/(\max_k h_k^2)$; scaling so $\max_k h_k^2 = 1$, $\mathbb{E}\,\mathrm{vol}(S_\theta) = \sum_i d_i h_i^2$, and every S_θ has volume $\le \frac{1}{2}\,\mathrm{vol}(V)$ because h is supported on the light side. Hence $\min\{\mathrm{vol}(S_\theta), \mathrm{vol}(S_\theta^c)\} = \mathrm{vol}(S_\theta)$.

Therefore

$$\mathbb{E}\, w(\partial S_\theta) \;\le\; \sqrt{2\rho}\,\mathbb{E}\,\mathrm{vol}(S_\theta).$$

Because the left side is a nonnegative-weighted average of $w(\partial S_\theta)$ and the right side the same average of $\sqrt{2\rho}\,\mathrm{vol}(S_\theta)$, there must exist a particular threshold θ with $w(\partial S_\theta) \le \sqrt{2\rho}\,\mathrm{vol}(S_\theta)$, i.e. $\phi(S_\theta) \le \sqrt{2\rho} \le \sqrt{2\mu_2}$. That S_θ is one of the $n-1$ prefix sets the algorithm tries, so $S^\star$ is at least as good. (The averaging step is exactly the standard "if $\mathbb{E}[A] \le \sqrt{2\rho}\,\mathbb{E}[B]$ with $B > 0$ then some outcome has $A \le \sqrt{2\rho}\,B$" applied to the two correlated quantities boundary and volume.) □

Why it works

The square root in $\sqrt{2\mu_2}$ – rather than μ_2 – comes from the Cauchy–Schwarz step. The loss cannot be avoided, and the cycle C_n shows why. Its normalized eigenvalue is $\mu_2 = 1 - \cos(2\pi/n) \approx \frac{1}{2}(2\pi/n)^2 = \Theta(1/n^2)$, while its sparsest cut splits the cycle into two equal arcs: 2 edges cross and each side has volume n, so $\phi(C_n) = 2/n = \Theta(1/n)$. Thus $\phi \approx \sqrt{\mu_2}$ up to constants, not $\phi \approx \mu_2$. The square-root loss comes from rounding a real-valued eigenvector to a cut instead of solving the combinatorial problem exactly.

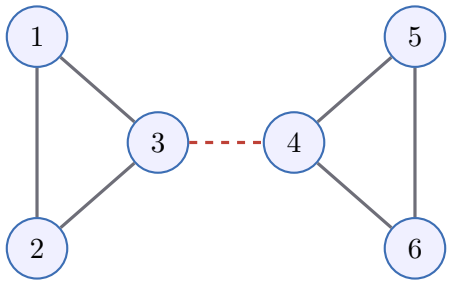
Cost summary

Given the Fiedler vector, SpectralSweep costs $O(m + n\log n)$: an $O(n\log n)$ sort, then a single linear sweep maintaining $w(\partial S_t)$ and $\mathrm{vol}(S_t)$ incrementally (each vertex enters once; touch its incident edges, flipping each from "crossing" to "internal" or vice versa, in total $O(m)$). Computing the Fiedler vector itself dominates: a few iterations of the Lanczos method or a Laplacian linear solve, $\widetilde{O}(m)$ per matrix–vector product times the iteration count, or $\widetilde{O}(m)$ overall with the nearly-linear-time solvers of Chapter 41. The guarantee $\phi(S^\star) \le \sqrt{2\mu_2} \le \sqrt{2}\sqrt{2\phi(G)} = 2\sqrt{\phi(G)}$ makes spectral partitioning an $O(1/\sqrt{\phi})$-approximation to sparsest cut – weak in the worst case but excellent on graphs with a clear bottleneck.

32.5 Worked example: Cheeger on a two-cluster graph

Let us run the entire pipeline on a graph engineered to have one obvious bottleneck, so we can watch μ_2, the Fiedler vector, and the sweep all point at the same cut.

Take six vertices in two triangles, $\{1,2,3\}$ and $\{4,5,6\}$, each a complete triangle, joined by a single bridge edge 3–4. All weights 1.



Degrees: $d_1 = d_2 = 2$, $d_3 = 3$, $d_4 = 3$, $d_5 = d_6 = 2$, so $\mathrm{vol}(V) = 14 = 2m$ with $m = 7$. The natural cut $S = \{1, 2, 3\}$ has $w(\partial S) = 1$ (the bridge) and $\mathrm{vol}(S) = 7 = \mathrm{vol}(S^c)$, hence

$$\phi(\{1, 2, 3\}) = \frac{1}{\min\{7, 7\}} = \frac{1}{7} \approx 0.143,$$

and one checks no other cut beats it, so $\phi(G) = 1/7$.

The combinatorial Laplacian.

$$\mathbf{L} = \begin{pmatrix} 2 & -1 & -1 & 0 & 0 & 0 \\ -1 & 2 & -1 & 0 & 0 & 0 \\ -1 & -1 & 3 & -1 & 0 & 0 \\ 0 & 0 & -1 & 3 & -1 & -1 \\ 0 & 0 & 0 & -1 & 2 & -1 \\ 0 & 0 & 0 & -1 & -1 & 2 \end{pmatrix}.$$

By the left–right symmetry (swap $1 \leftrightarrow 6$, $2 \leftrightarrow 5$, $3 \leftrightarrow 4$) every eigenvector is symmetric or antisymmetric. The Fiedler vector is the antisymmetric one that is constant on each triangle except for the bridge vertices: solving $\mathbf{L}v_2 = \lambda_2 v_2$ with the ansatz $v_2 \propto (b, b, a, -a, -b, -b)$, the equations decouple to a small 2×2 system in a, b, and we record the outcome:

$$\lambda_2 = \tfrac{5-\sqrt{17}}{2} \approx 0.438, \qquad v_2 \approx (0.47, 0.47, 0.26, -0.26, -0.47, -0.47).$$

The Fiedler vector is positive on the left triangle, negative on the right, and crosses zero exactly on the bridge. (The other eigenvalues are $\lambda_1 = 0$, then $\lambda_2 \approx 0.44$, and the rest clustered near 3–4.6, the "internal" frequencies of the triangles.)

Let us actually carry out the 2×2 reduction, because it shows where that number comes from and why the symmetry ansatz is forced rather than guessed. Write the eigenvector as $v = (b, b, a, -a, -b, -b)$ – the antisymmetric pattern that is constant on the two non-bridge vertices of each triangle. Two of the six eigen-equations are independent; the rest are copies by symmetry. Row 1 (vertex 1, degree 2, neighbors 2 and 3) reads

$$2b - b - a \;=\; \lambda b \qquad \Longrightarrow \qquad b - a \;=\; \lambda b.$$

Row 3 (vertex 3, degree 3, neighbors $1, 2, 4$, with $v_4 = -a$) reads

$$3a - b - b - (-a) \;=\; \lambda a \qquad \Longrightarrow \qquad 4a - 2b \;=\; \lambda a.$$

These are two linear equations in (a, b); a nonzero solution needs the determinant of

$$\begin{pmatrix} 1-\lambda & -1 \\ -2 & 4-\lambda \end{pmatrix}$$

to vanish, i.e. $(1-\lambda)(4-\lambda)-2=0$, which is $\lambda^2-5\lambda+2=0$. The two roots are $\lambda=\frac{5\pm\sqrt{17}}{2}$, giving $\lambda\approx 0.438$ and $\lambda\approx 4.56$. The smaller root, $\lambda_2=\frac{5-\sqrt{17}}{2}\approx 0.438$, is the antisymmetric Fiedler value; the larger is one of the internal triangle frequencies. Back-substituting $\lambda\approx 0.438$ into $b-a=\lambda b$ gives $a=(1-\lambda)b\approx 0.562\,b$, so with the normalization $\|v\|=1$ one recovers $v_2\approx(0.47,0.47,0.26,-0.26,-0.47,-0.47)$, matching the shape recorded above. More generally, a graph with a symmetry group lets you block-diagonalize the Laplacian by symmetry type and read each block's spectrum off a tiny matrix, no numerical solver required.

Normalized version and Cheeger check. With $\mathbf{D}^{-1/2}$ rescaling, the second normalized eigenvalue comes out to $\mu_2\approx 0.205$. Cheeger then brackets the true conductance $\phi(G)=1/7\approx 0.143$:

$$\underbrace{\tfrac{\mu_2}{2}\approx 0.102}_{\text{lower}}\ \le\ \phi(G)\approx 0.143\ \le\ \underbrace{\sqrt{2\mu_2}\approx 0.640}_{\text{upper}}.$$

Both inequalities hold, as promised, and the gap is exactly the square-root looseness of the hard direction.

Running the sweep by hand. Sort vertices by $g_i=\mathbf{D}^{-1/2}v_2$ value; ascending, the order groups the right triangle (negative coordinates) before the left (positive), so the meaningful break is between the two triangles. Sweep the prefix cuts:

t	prefix S_t (by sorted g)	$w(\partial S_t)$	min vol	$\phi(S_t)$
1	$\{5\}$	2	2	1.00
2	$\{5,6\}$	2	4	0.50
3	$\{4,5,6\}$	1	7	**0.143**
4	$\{3,4,5,6\}$	2	4	0.50
5	$\{1,4,5,6\}\ \ldots$	–	–	(worse)

The minimum over the sweep is at $t=3$, the cut $\{4,5,6\}$ versus $\{1,2,3\}$, conductance $1/7$ – the exact sparsest cut. The sweep succeeds here because the graph has a clean spectral gap (a big jump from $\lambda_2\approx 0.44$ to the next eigenvalue), the regime where spectral partitioning works best. The full combinatorial spectrum makes the gap visible at a glance (Figure 32.2).



Figure 32.2: Combinatorial Laplacian spectrum of the two-triangle graph: 0, $\frac{5-\sqrt{17}}{2}\approx 0.44$, $3,3,3$, $\frac{5+\sqrt{17}}{2}\approx 4.56$. The wide gap between λ_2 and the cluster at 3 signals that the graph splits cleanly into exactly two pieces.

The embedding the Fiedler vector produces is worth seeing. Plot each vertex at its coordinate g_i on a single axis: the six vertices fall into two tight clumps with a wide gap around zero, and the sweep simply scans left to right, trying a cut in each gap. The widest gap, the one between the two clumps, is the cut the algorithm reports.

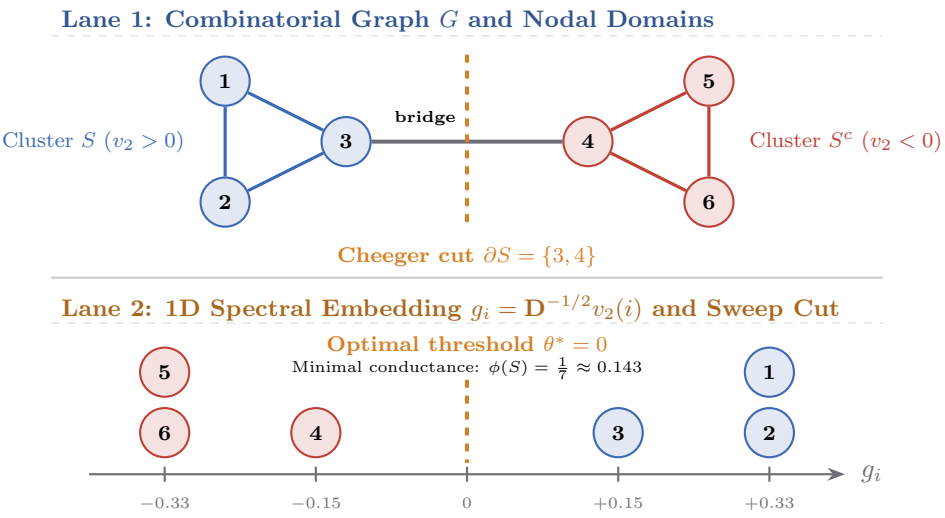


Figure 32.3: The Cheeger nodal cut along the Fiedler vector for the two-triangle graph. **Top (Lane 1):** The graph vertices partition into two nodal domains according to the sign of the Fiedler vector coordinates: $S = \{1, 2, 3\}$ (positive, blue) and $S^c = \{4, 5, 6\}$ (negative, red), severing only the bridge edge $\{3, 4\}$. **Bottom (Lane 2):** Embedding vertices on the line at $g_i = \mathbf{D}^{-1/2} v_2(i)$ clumps the two clusters on opposite sides of zero. Sweeping the threshold across the gap isolates the exact sparsest cut with conductance $\phi = 1/7$.

The dashed line is the winning threshold: every edge except the bridge has both endpoints on the same side, so only one edge crosses, and that is the conductance-1/7 cut the table found at $t = 3$. This is how spectral partitioning works in general: the eigenvector turns a combinatorial cut problem into a one-dimensional gap-finding problem.

Trace it yourself

Redo this with the bridge *doubled* (weight 2 on edge 3–4, or a second parallel bridge). Recompute ∂S for $S = \{1, 2, 3\}$, the new vol, and predict the direction λ_2 moves before computing it. (It must increase, by edge monotonicity, Proposition 32.1.) Then verify the sweep still picks $\{1, 2, 3\}$ but with larger conductance. This is the cleanest hands-on way to feel "more bridge $\Rightarrow$ less of a bottleneck $\Rightarrow$ larger λ_2."

Application

Spectral partitioning via the sweep is the standard first cut in: *image segmentation* (the normalized cuts algorithm of Shi and Malik is exactly $\mathcal{L}$'s Fiedler vector on a pixel-similarity graph); *scientific computing* (partitioning a finite-element mesh across processors to minimize inter-processor communication, where the mesh graph's bottleneck is the seam to cut); *community detection* in social and biological networks; and *sparse-matrix reordering* to reduce fill-in. In each case the win is the same: a hard combinatorial cut problem is replaced by an eigenvector computation plus a linear sweep, with a Cheeger guarantee on the result.

32.6 Higher eigenvalues and many-way partitions

Higher eigenvectors carry information too. The eigenvectors $v_2, v_3, \dots, v_k$ together give a $(k-1)$-dimensional embedding of the graph – map vertex i to the point $(v_2(i), v_3(i), \dots, v_k(i))$ – and clustering that point cloud (e.g. with k-means) is the k-way spectral clustering of Chapter 33. The theoretical justification is a higher-order Cheeger inequality.

Theorem 32.4 (Higher-order Cheeger, informal)**.** *Let $\rho_k(G) = \min\left\{ \max_{1\le i\le k} \phi(S_i) \right\}$ over partitions of V into k nonempty disjoint sets $S_1, \dots, S_k$ (the k-way expansion). Then there are universal constants such that*

$$\frac{\mu_k}{2} \;\le\; \rho_k(G) \;\le\; C\, k^2 \sqrt{\mu_k},$$

and a refined bound replaces k^2 by $O(k)$ using $O(k)$ eigenvectors. Consequently G has a meaningful k-clustering exactly when there is a large gap $\mu_{k+1} \gg \mu_k$ in the spectrum.

We state this without proof (it is due to Lee, Oveis Gharan, and Trevisan, 2012); the takeaway is the *spectral-gap heuristic*: *the number of clusters to ask for is the index k at which the eigenvalue sequence $\mu_2, \mu_3, \dots$ first jumps.* In the two-triangle example there is a big jump after μ_2, signalling exactly two clusters. We return to this in the next chapter.

Connection

The eigenvalue sequence $0 = \mu_1 \le \mu_2 \le \cdots$ is the same object that governs the *mixing time* of the random walk (Chapters 35 and 36) – the walk mixes in roughly $O(\frac{1}{\mu_2} \log n)$ steps, so a small μ_2 (a bottleneck) means slow mixing, the random-walk counterpart of Cheeger's inequality. It is also the quantity expander constructions (Chapter 34) are built to control: an expander is precisely a sparse graph with μ_2 bounded away from 0 (equivalently $\phi(G) = \Omega(1)$), and Cheeger is what certifies the two notions agree. The same spectrum governs the convergence of the Laplacian solvers of Chapter 41.

32.7 Stability: how much does the spectrum move?

A reason the spectrum is useful in practice is that it is *stable*: small changes to the graph make small changes to the eigenvalues. This is Weyl's inequality applied to the Laplacian.

Proposition 32.2 (Weyl stability)**.** *If $\mathbf{L}$ and $\mathbf{L}'$ are Laplacians of two weighted graphs on the same vertex set, then for every k,*

$$\left|\lambda_k(\mathbf{L}) - \lambda_k(\mathbf{L}')\right| \;\le\; \|\mathbf{L} - \mathbf{L}'\|_2,$$

the spectral norm of the difference. In particular, adding or deleting a single unit edge changes each λ_k by at most 2 (the spectral norm of one edge Laplacian $\mathbf{L}_e$ is 2).

Proof. Weyl's inequality for symmetric matrices states $\lambda_k(M) + \lambda_{\min}(E) \le \lambda_k(M+E) \le \lambda_k(M) + \lambda_{\max}(E)$; setting $E = \mathbf{L}' - \mathbf{L}$ gives $|\lambda_k(\mathbf{L}') - \lambda_k(\mathbf{L})| \le \max(|\lambda_{\max}(E)|, |\lambda_{\min}(E)|) = \|E\|_2$. For a single edge $e = (p, q)$, $\mathbf{L}_e = (\mathbf{e}_p - \mathbf{e}_q)(\mathbf{e}_p - \mathbf{e}_q)^\top$ has one nonzero eigenvalue $\|\mathbf{e}_p - \mathbf{e}_q\|^2 = 2$. □

This robustness is what makes spectral methods workable on noisy real-world graphs and what underlies *spectral sparsification* (Chapter 41): one can replace a dense graph by a sparse weighted

one whose Laplacian is spectrally close, $(1-\varepsilon)\mathbf{L} \preceq \widetilde{\mathbf{L}} \preceq (1+\varepsilon)\mathbf{L}$, and every eigenvalue – hence every Cheeger estimate, every cut value up to $1\pm\varepsilon$, every effective resistance – is preserved. The spectrum is a useful measure precisely because it changes continuously with the graph.

Common trap

Stability of *eigenvalues* does not imply stability of *eigenvectors*. When μ_2 and μ_3 are nearly equal (a small spectral gap), the Fiedler vector can swing wildly under a tiny perturbation, because the relevant eigenspace is nearly two-dimensional and the choice of axis within it is ill-conditioned. The Davis–Kahan $\sin\Theta$ theorem makes this precise: eigenvector stability is governed by the *gap* to neighboring eigenvalues, not by the perturbation size alone. So always check the spectral gap before trusting a single Fiedler cut; if the gap is small, the graph does not have a single well-defined bottleneck and you should ask for more clusters.

32.8 Summary

The dictionary we built:

- $\lambda_1 = \mu_1 = 0$ always (the constant vector); multiplicity of 0 counts connected components (Theorem 32.1).
- $0 \le \mu_k \le 2$; $\mu_n = 2$ iff a component is bipartite (Lemma 32.3).
- λ_2, the algebraic connectivity / Fiedler value, measures how robustly the graph hangs together: monotone in edges, $\le$ vertex connectivity, $= 0$ iff disconnected (Definition 32.2, Proposition 32.1).
- Cheeger: $\frac{\mu_2}{2} \le \phi(G) \le \sqrt{2\mu_2}$ (Theorem 32.2); the upper bound is constructive via the eigenvector sweep (Theorem 32.3, Algorithm 128), giving a polynomial-time sparse-cut approximation.
- Higher eigenvalues give k-way partitions; the spectral gap tells you how many clusters there are (Theorem 32.4).
- The spectrum is Weyl-stable under edge perturbations and sparsification, but eigenvectors are only stable when the spectral gap is large (Proposition 32.2).

The two-triangle worked example tied it together: μ_2 small, a clean spectral gap, a Fiedler vector that changes sign exactly across the bridge, and a sweep that recovers the exact sparsest cut. The next chapter turns this from one cut into a clustering pipeline; Chapter 34 studies the graphs where ϕ is bounded below (expanders); and Chapter 41 makes the eigenvector and solver computations nearly linear time.

Connection

What This Made Possible: Spectral graph theory turned NP-hard graph partitioning into continuous eigenvalue optimization. The discrete Cheeger inequality guaranteed that sorting coordinates of the Fiedler vector yields an $O(\sqrt{\mu_2})$ approximation to the sparsest cut in polynomial time. This led to spectral graph bisection, image segmentation (Normalized Cuts by Shi and Malik 2000), random-walk mixing time analysis, and the foundational theory of expander graphs.

Limitations:

- *The square-root gap:* Cheeger's inequality has an inherent quadratic loss: $\frac{\mu_2}{2} \le \phi(G) \le \sqrt{2\mu_2}$. When the true conductance ϕ is small (e.g. on cycles C_n or hypercubes Q_d), the spectral sweep guarantees only an $O(\sqrt{\phi})$ cut rather than an $O(\phi)$ cut. The Arora–Rao–Vazirani (2009) algorithm uses semidefinite programming and ℓ_2^2 embeddings to bridge this gap to $O(\sqrt{\log n})$.
- *Multi-cluster blind spot:* A single Fiedler vector partitions a graph into only two pieces. It cannot natively separate graphs with $k > 2$ clusters or complex hierarchical communities without recursive sweeps or multiple eigenvectors.

Where We Go Next: How do we partition a graph into $k \ge 3$ natural clusters simultaneously? In **Chapter 33 (Spectral Clustering)**, we embed vertices into $\mathbb{R}^k$ using the bottom k Laplacian eigenvectors and cluster them via k-means. And what happens when $\mu_2 \ge c > 0$ is bounded away from zero regardless of graph size? Such graphs have no bottlenecks and mix rapidly: they are **Chapter 34's Expander Graphs**, which are central tools in derandomization, coding theory, and theoretical computer science.

32.9 Exercises

Exercise 32.1 (Star and its spectrum)**.** Compute the Laplacian spectrum of the star $K_{1,n-1}$ (one center joined to $n-1$ leaves). Show $\lambda_2 = 1$ and $\lambda_n = n$, with $\lambda = 1$ of multiplicity $n-2$. Interpret $\lambda_2 = 1$: how sparse is the sparsest cut, and does Cheeger's bracket contain it?

Exercise 32.2 (Disjoint union)**.** Prove that if G is the disjoint union of G_1 and G_2, then the Laplacian spectrum of G is the multiset union of the two spectra. Deduce the component-counting theorem (Theorem 32.1) as a corollary.

Exercise 32.3 (Edge monotonicity is tight)**.** Give a graph and an edge whose addition raises λ_2 by exactly the maximum allowed by Weyl (+2), and another whose addition raises λ_2 by almost nothing. Explain combinatorially what distinguishes the two edges.

Exercise 32.4 (Bipartite double cover)**.** For any graph G, its bipartite double cover $G \times K_2$ has vertex set $V \times \{0,1\}$ with edges $((u,0),(v,1))$ for each $uv \in E$. Show its normalized spectrum is $\{\mu_k\} \cup \{2 - \mu_k\}$. Conclude $\mu_n(G \times K_2) = 2$ always, consistent with Lemma 32.3.

Exercise 32.5 (Cheeger lower bound by hand)**.** Take the path P_4 (Section 32.2.3). Compute $\phi(P_4)$ exactly (try all cuts), compute μ_2, and verify $\frac{\mu_2}{2} \le \phi \le \sqrt{2\mu_2}$ numerically. Which side of Cheeger is closer to tight here, and why?

Exercise 32.6 (Implement the sweep)**.** Implement Algorithm 128. Feed it a graph that is two cliques K_t joined by a single edge. Verify that the returned cut is the bridge cut and that its conductance matches $1/(t(t-1)+1)$ up to your normalization. Then make the bridge r parallel edges and watch ϕ and μ_2 both rise.

Exercise 32.7 (Break the sweep)**.** Construct a graph where the *global* sparsest cut is *not* a prefix of the Fiedler ordering, so the single sweep misses it. (Hint: nest two bottlenecks at different scales, or use a graph whose Fiedler vector has its small spectral gap so v_2 mixes two cuts.) Estimate how far off the sweep's conductance is, and relate it to the spectral gap $\mu_3 - \mu_2$.

Exercise 32.8 (Fiedler nodal domains)**.** Prove that on a *tree*, the Fiedler vector v_2 has the "one sign change" property: the vertices with $v_2(i) > 0$ form a connected subtree and likewise for $v_2(i) < 0$. (This is Fiedler's nodal-domain theorem for trees; use the Rayleigh-quotient characterization and an exchange argument.) Show by example it can fail on a general graph.

Exercise 32.9 (Algebraic vs. vertex connectivity)**.** Verify Fiedler's bound $\lambda_2 \leq \kappa(G)$ on three graphs: C_n, $K_{3,3}$, and the Petersen graph. For which is the bound tight? Then exhibit a graph where λ_2 is much smaller than κ, demonstrating that algebraic connectivity can badly underestimate vertex connectivity.

Exercise 32.10 (Weyl in action)**.** Let G be the cycle C_{100}. Predict, using Proposition 32.2, the maximum possible change in λ_2 when you add a single "shortcut" chord. Then compute the actual change for the chord joining antipodal vertices and explain why it is so much larger than for a short chord – connect this to the chord's effect on the bottleneck.

Exercise 32.11 (Normalized vs. combinatorial)**.** Build a graph with a highly skewed degree distribution (one high-degree hub, many low-degree leaves) where the combinatorial Fiedler vector and the normalized Fiedler vector suggest *different* cuts. Explain which one respects volume and why the normalized version is the right one for conductance.

Exercise 32.12 (Cheeger constant of the hypercube)**.** The hypercube Q_d has 2^d vertices and is d-regular with $\mu_2 = 2/d$. Use Cheeger to bound $\phi(Q_d)$ from both sides, then compare with the exact isoperimetric fact $\phi(Q_d) = 1/d$ (the cut splitting on one coordinate). Which Cheeger direction is tight, and what does that say about the square-root loss for this graph?

Exercise 32.13 (Two-triangle spectrum by symmetry)**.** Redo the 2×2 reduction of Section 32.5 for the *symmetric* eigenvectors $v \propto (b, b, a, a, b, b)$ (constant across the bridge instead of flipping). Write down the two eigen-equations, find the resulting quadratic, and confirm that one of its roots is $\lambda_1 = 0$ (the all-ones direction) so the symmetric block contributes the trivial eigenvalue plus one nontrivial value. Combined with the antisymmetric block you computed in the text, you now have four of the six eigenvalues from two 2×2 determinants; identify where the remaining two (the pure within-triangle modes orthogonal to both a and b patterns) come from.

Exercise 32.14 (Conductance versus the unnormalized cut ratio)**.** Define the unnormalized ratio $h(S) = w(\partial S)/\min\{|S|, |S^c|\}$ (counting vertices, not volume). Show h and ϕ agree up to factors of the maximum and minimum degree, and exhibit a graph with a hub where h and ϕ are minimized by *different* sets. Explain, using the easy-direction proof, why ϕ – not h – is the one Cheeger's inequality is stated for.

Chapter 33

Spectral Clustering and Partitioning

The previous two chapters developed the algebra of the graph Laplacian: the matrix $\mathbf{L} = \mathbf{D} - \mathbf{A}$, its non-negative eigenvalues $0 = \lambda_1 \leq \lambda_2 \leq \cdots \leq \lambda_n$, and the quadratic form

$$x^\top \mathbf{L} x \;=\; \sum_{(i,j)\in E} w_{ij}(x_i - x_j)^2$$

which measures how steeply a scalar assignment x varies across connected edges. In this chapter, we put that algebra to work on one of the central challenges in network analysis: *graph partitioning*. Given a graph, how can we partition its vertices into clusters that are densely connected on the inside yet sparsely linked to the outside? Such clusters correspond to natural communities in a social network, coherent objects in computer vision, or functional modules in biological networks.

Finding an optimal balanced cut in a graph is NP-hard. Fortunately, relaxing the discrete labels to continuous coordinates opens up a clean geometric approach: if we drop the rigid binary requirement and allow vertices to sit at arbitrary real coordinates, the optimal placement is governed by the bottom eigenvectors of the Laplacian. By computing these eigenvectors in polynomial time, we can embed the graph into low-dimensional Euclidean space, where the natural clusters pull apart into easily separable geometric point clouds.

Intuition

Imagine pinning beads onto a taut wire, with elastic springs connecting any two beads that share an edge. Each spring exerts a restoring force pulling its endpoints together, with total potential energy $\sum_{(i,j)\in E}(x_i - x_j)^2$. If we simply release the beads, every bead rushes to a single point: an energy of zero that collapses all structure.

To extract meaningful geometry, we impose two physical constraints: the center of mass must remain fixed at zero ($\sum_i x_i = 0$), and the total spread must remain fixed ($\sum_i x_i^2 = n$). Under these rules, the beads cannot collapse. Instead, vertices that are tightly connected by many internal springs huddle close together, while the few sparse bridges between clusters are forced to stretch across the gap. The equilibrium positions that emerge are precisely the coordinates of the *Fiedler vector* v_2, the eigenvector of $\mathbf{L}$ corresponding to its second-smallest eigenvalue λ_2. Slicing the wire at a suitable threshold reveals the bottleneck cut.

We will develop this geometric viewpoint in three stages:

1. **Spectral Bisection**: Using the single Fiedler vector v_2 to project vertices onto a line, followed by a sweep threshold to find a clean balanced two-way cut.

2. **Normalized Cuts and Random Walks**: Normalizing by degree matrices so that cluster size reflects edge volume rather than bare vertex count, linking the cut directly to random-walk escape probabilities and Cheeger's inequality.

3. **k-Way Spectral Clustering**: Projecting vertices into $\mathbb{R}^k$ using the bottom k eigenvectors simultaneously, where communities become well-separated point clouds clustered by standard k-means.

Throughout, we trace every step by hand on a concrete running network so that the linear algebra and the geometry remain visible at every turn.

33.1 The cut we wish we could compute

Fix an undirected graph $G = (V, E)$ with nonnegative edge weights $w_{ij} \geq 0$. For disjoint vertex sets $A, B \subseteq V$ define the cut weight

$$\operatorname{cut}(A, B) \;=\; \sum_{i \in A,\, j \in B} w_{ij},$$

and for a single set S write $\operatorname{cut}(S) = \operatorname{cut}(S, \overline{S})$ where $\overline{S} = V \setminus S$. The *minimum cut* – smallest $\operatorname{cut}(S)$ over all nonempty proper S – is computable in polynomial time (Chapter 11), but it is useless for clustering: the cheapest cut usually peels off a single low-degree vertex. For clustering we need a *balanced* cut, one that leaves a substantial piece on *each* side.

Two classical balance-aware objectives. With $\operatorname{vol}(S) = \sum_{i \in S} d_i$ the volume (sum of weighted degrees) and $|S|$ the cardinality:

$$\operatorname{RatioCut}(A, B) \;=\; \operatorname{cut}(A, B)\left(\frac{1}{|A|} + \frac{1}{|B|}\right), \qquad \operatorname{NCut}(A, B) \;=\; \operatorname{cut}(A, B)\left(\frac{1}{\operatorname{vol}(A)} + \frac{1}{\operatorname{vol}(B)}\right).$$

Focus

Both objectives multiply the raw cut by a penalty that *blows up when one side is small*. Peeling off one vertex makes $|A| = 1$ (or $\operatorname{vol}(A) = d$ small), so the factor $1/|A|$ (or $1/\operatorname{vol}(A)$) is large and the objective is bad even though cut is tiny. The minimizer is forced to keep both sides fat. RatioCut balances by *count*; normalized cut balances by *volume*. They differ only in which notion of "size" they protect.

Figure 33.1 contrasts the two regimes on a seven-vertex graph: two dense triangles linked by two bridge edges, plus a degree-1 whisker attached to vertex 1. Global minimum cut severing the whisker achieves cut $= 1$ but isolates a single vertex (RatioCut $= 1 \cdot (\frac{1}{1} + \frac{1}{6}) \approx 1.17$), whereas cutting the two bridge edges pays cut $= 2$ to split the communities cleanly (RatioCut $= 2 \cdot (\frac{1}{4} + \frac{1}{3}) \approx 1.17$ and an even sharper win in NCut: $\frac{2}{7} + \frac{2}{8} = 0.54$ versus $\frac{1}{1} + \frac{1}{14} = 1.07$).

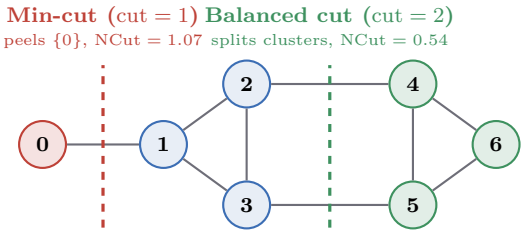


Figure 33.1: Why raw minimum cut fails for clustering: severing the single whisker edge $(0, 1)$ minimizes $\mathrm{cut}(A, B) = 1$ but produces a trivial 1-versus-6 split, whereas NCut prefers the balanced 4-versus-3 community split despite its higher edge cut.

Theorem 33.1 (Hardness)**.** *Minimizing* RatioCut *over all bipartitions of V is NP-hard, as is minimizing* NCut*.*

Because exact combinatorial optimization is NP-hard, we turn to continuous relaxation. Rather than forcing each coordinate to take one of two discrete values, we allow the coordinates to vary freely along the real line. The combinatorial objective then transforms into a continuous Rayleigh quotient involving the graph Laplacian $\mathbf{L}$. Solving this relaxation amounts to computing an eigenvector, which can be done in polynomial time. Once we have the continuous solution, we round the real-valued coordinates back to a discrete cut.

Our running graph

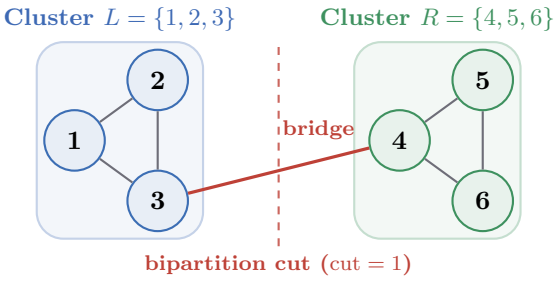


Figure 33.2: The running graph G_0: two dense triangles joined by a single sparse bridge $(3, 4)$. All edge weights are 1. The target cut is $L = \{1, 2, 3\} \mid R = \{4, 5, 6\}$ with cut weight 1, achieving optimal balance and conductance.

G_0 has $n = 6$ vertices and 7 edges. The degrees are $d_1 = d_2 = d_5 = d_6 = 2$ and $d_3 = d_4 = 3$ (the bridge endpoints carry one extra edge). The intended answer is obvious to the eye – the two triangles – and that is exactly why it is a good test: a correct algorithm must agree with the eye, and we can check every number by hand.

33.2 Bisection by the Fiedler vector

Take the RatioCut objective with the soft relaxation. Encode a bipartition (A, B) by the vector

$$f_i \;=\; \begin{cases} +\sqrt{|B|/|A|}, & i \in A, \\ -\sqrt{|A|/|B|}, & i \in B. \end{cases}$$

This unusual scaling is chosen so that two clean identities hold. A short computation (do it in the margin) gives

$$\sum_i f_i = |A|\sqrt{\tfrac{|B|}{|A|}} - |B|\sqrt{\tfrac{|A|}{|B|}} = 0, \qquad \|f\|^2 = |A| \cdot \tfrac{|B|}{|A|} + |B| \cdot \tfrac{|A|}{|B|} = n,$$

so $f \perp \mathbf{1}$ and $\|f\|^2 = n$. The point of the scaling is the next identity.

Lemma 33.1 (RatioCut as a quadratic form)**.** *With f defined above, $f^\top \mathbf{L} f = n \cdot \mathrm{RatioCut}(A, B)$.*

Proof. Using $x^\top \mathbf{L} x = \sum_{(i,j)\in E} w_{ij}(x_i - x_j)^2$, only edges crossing the cut contribute, and a crossing edge has $(f_i - f_j)^2 = \big(\sqrt{|B|/|A|} + \sqrt{|A|/|B|}\big)^2$. Expand:

$$\Big(\sqrt{\tfrac{|B|}{|A|}} + \sqrt{\tfrac{|A|}{|B|}}\Big)^2 = \tfrac{|B|}{|A|} + 2 + \tfrac{|A|}{|B|} = \tfrac{|A|+|B|}{|A|} + \tfrac{|A|+|B|}{|B|} = n\Big(\tfrac{1}{|A|} + \tfrac{1}{|B|}\Big).$$

Summing over the $\mathrm{cut}(A, B)$ crossing edges gives $f^\top \mathbf{L} f = \mathrm{cut}(A, B) \cdot n(\frac{1}{|A|} + \frac{1}{|B|}) = n \cdot \mathrm{RatioCut}(A, B)$. □

So minimizing RatioCut is *exactly*

$$\min_{A,B} \; f^\top \mathbf{L} f \quad \text{subject to} \quad f \text{ of the two-valued form above}, \quad f \perp \mathbf{1}, \quad \|f\|^2 = n.$$

The constraint forcing f to take only two discrete values is the sole source of combinatorial hardness. To overcome it, we relax the problem: we drop the two-valued requirement and allow $f \in \mathbb{R}^n$ to be an arbitrary real vector, retaining only the centering constraint $f \perp \mathbf{1}$ and the scale constraint $\|f\|^2 = n$:

$$\min_{f \in \mathbb{R}^n} \; f^\top \mathbf{L} f \quad \text{subject to} \quad f \perp \mathbf{1}, \; \|f\|^2 = n.$$

Why it works

The Rayleigh quotient and the Courant–Fischer theorem. For any symmetric matrix $\mathbf{M}$ with eigenvalues $\lambda_1 \le \lambda_2 \le \cdots \le \lambda_n$ and orthonormal eigenvectors $u_1, \ldots, u_n$, the *Rayleigh quotient*

$$R_{\mathbf{M}}(x) \;=\; \frac{x^\top \mathbf{M} x}{x^\top x}$$

measures the quadratic energy per unit norm. Expanding an arbitrary vector x in the eigenvector basis as $x = \sum_{i=1}^n c_i u_i$ gives $\|x\|^2 = \sum_i c_i^2$ and $x^\top \mathbf{M} x = \sum_i \lambda_i c_i^2$. The Rayleigh quotient is therefore an exact weighted average of the eigenvalues:

$$R_{\mathbf{M}}(x) \;=\; \frac{\sum_{i=1}^n \lambda_i c_i^2}{\sum_{i=1}^n c_i^2}.$$

Because the weights c_i^2 are non-negative, this weighted average can never be smaller than the minimum eigenvalue λ_1, nor larger than the maximum λ_n. The global minimum λ_1 is attained by choosing $x = u_1$.
What happens if we constrain x to be orthogonal to the first eigenvector u_1? That condition forces $c_1 = \langle x, u_1 \rangle = 0$. In the weighted average, the term $\lambda_1 c_1^2$ vanishes, leaving only eigenvalues $\lambda_2, \lambda_3, \ldots, \lambda_n$. The minimum of this restricted average is λ_2, achieved when $x = u_2$.
More generally, the *Courant–Fischer min-max theorem* states that for each index k,

$$\lambda_k \;=\; \min_{\dim S = k} \; \max_{0 \neq x \in S} R_{\mathbf{M}}(x) \;=\; \min_{x \perp u_1, \ldots, u_{k-1},\, x \neq 0} R_{\mathbf{M}}(x).$$

Applying this to the graph Laplacian $\mathbf{L}$:

- The smallest eigenvalue is always $\lambda_1 = 0$, with eigenvector $u_1 = \frac{1}{\sqrt{n}}\mathbf{1}$ (since $\mathbf{L}\mathbf{1} = 0$).
- The constraint $f \perp \mathbf{1}$ in our relaxed bisection is precisely $f \perp u_1$.

By Courant–Fischer, minimizing $f^\top \mathbf{L} f$ over all vectors orthogonal to $\mathbf{1}$ with $\|f\|^2 = n$ yields exactly $n\lambda_2$, and the optimal direction is the second eigenvector v_2. This eigenvector is the *Fiedler vector*, named after Miroslav Fiedler who in 1973 identified λ_2 as the *algebraic connectivity* of the graph.

We have proved:

Theorem 33.2 (Fiedler relaxation)**.** *The continuous relaxation of RatioCut bisection is solved by the Fiedler vector v_2, with optimal value $\lambda_2 = n^{-1} \min f^\top \mathbf{L} f$. Consequently* $\mathrm{RatioCut}^\star \geq \lambda_2$*:* the algebraic connectivity is a lower bound on the best ratio cut.

Now we must *round* the real vector v_2 back to a partition. The natural move: pick a threshold t and set $A = \{i : (v_2)_i > t\}$, B the rest. Two standard thresholds:

- **Sign threshold** ($t = 0$): split by the sign of $(v_2)_i$. Simple; works when the gap straddles zero.
- **Median / sweep threshold**: sort the entries of v_2, and among the $n-1$ "sweep cuts" (prefixes of the sorted order) pick the one with the smallest RatioCut. This is the *sweep*, and it is what good implementations use.

Algorithm 129: SpectralBisection(G) – Fiedler partition with sweep rounding

Input: Connected weighted graph $G = (V, E, w)$, $n = |V|$.
Output: A bipartition (A, B) of V.

Form the Laplacian $\mathbf{L} = \mathbf{D} - \mathbf{A}$.
Compute the eigenvector v_2 for the second-smallest eigenvalue λ_2.
Sort vertices so that $(v_2)_{\pi(1)} \leq (v_2)_{\pi(2)} \leq \cdots \leq (v_2)_{\pi(n)}$.
$best \leftarrow +\infty$; $A^\star \leftarrow \emptyset$.
for $k \leftarrow 1$ **to** $n - 1$ **do**
    $A \leftarrow \{\pi(1), \ldots, \pi(k)\}$, $B \leftarrow V \setminus A$.
    $q \leftarrow \mathrm{RatioCut}(A, B)$ `// or NCut for the normalized variant`
    if $q < best$ **then**
        $best \leftarrow q$; $A^\star \leftarrow A$.
return $(A^\star, V \setminus A^\star)$.

Common trap

The eigenvalue solver returns v_2 up to sign and scale. The sign is meaningless – $-v_2$ is just as valid an eigenvector – so *never* attach semantic meaning to "positive = cluster A." Always round through a sweep or a sign split and read off the partition, never the raw signs as labels across two runs. A second trap: if the graph is *disconnected*, $\lambda_2 = 0$ and v_2 is not unique; the relaxation silently returns a connected component, which may or may not be what you wanted. Always check connectivity first ($\lambda_2 > 0 \iff G$ connected, proved in Chapter 31).

Tracing G_0 by hand

The Laplacian of the running graph (order $1, 2, 3, 4, 5, 6$):

$$\mathbf{L} = \begin{pmatrix} 2 & -1 & -1 & 0 & 0 & 0 \\ -1 & 2 & -1 & 0 & 0 & 0 \\ -1 & -1 & 3 & -1 & 0 & 0 \\ 0 & 0 & -1 & 3 & -1 & -1 \\ 0 & 0 & 0 & -1 & 2 & -1 \\ 0 & 0 & 0 & -1 & -1 & 2 \end{pmatrix}.$$

The graph has a mirror symmetry swapping $L = \{1, 2, 3\}$ with $R = \{4, 5, 6\}$ (send $1 \leftrightarrow 6$, $2 \leftrightarrow 5$, $3 \leftrightarrow 4$). Eigenvectors are either symmetric or antisymmetric under this swap. The Fiedler vector is the antisymmetric one, constant on each triangle except at the bridge endpoints. Solving the 6×6 system (or just trusting the symmetry and a 2-parameter ansatz $f = (a, a, b, -b, -a, -a)$) yields, after normalizing,

$$v_2 \approx (0.46,\ 0.46,\ 0.26,\ -0.26,\ -0.46,\ -0.46)^\top, \qquad \lambda_2 \approx 0.44.$$

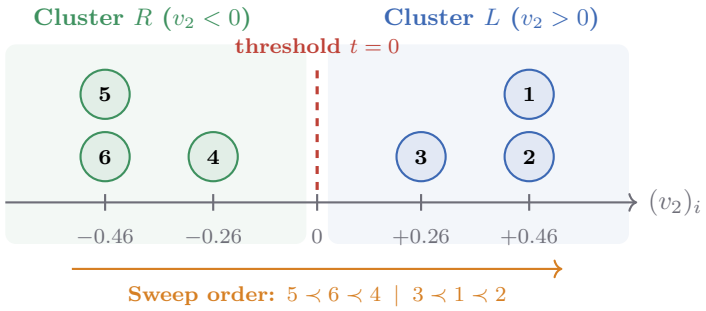


Figure 33.3: Vertices of G_0 embedded along the 1D line by their Fiedler coordinates $(v_2)_i$. The dense triangles collapse to tight groups on opposite sides of the origin, leaving a wide gap around 0. Thresholding at $t = 0$ cleanly isolates cluster $R = \{4, 5, 6\}$ from cluster $L = \{1, 2, 3\}$.

Run the sweep. Sorted order is $5, 6, 4, \mid 3, 1, 2$ (ties broken arbitrarily inside a triangle). The sweep cuts and their RatioCuts:

k	A (smallest k)	cut	$\frac{1}{\lvert A\rvert}+\frac{1}{\lvert B\rvert}$	RatioCut
1	$\{5\}$	2	$1+\frac{1}{5}$	2.40
2	$\{5,6\}$	2	$\frac{1}{2}+\frac{1}{4}$	1.50
3	$\{4,5,6\}$	**1**	$\frac{1}{3}+\frac{1}{3}$	**0.667**
4	$\{3,4,5,6\}$	2	$\frac{1}{4}+\frac{1}{2}$	1.50
5	$\{1,3,4,5,6\}$	2	$\frac{1}{5}+1$	2.40

The minimum is at $k=3$: $A=\{4,5,6\}$, RatioCut $=0.667$, exactly the two triangles. The lower bound from Theorem 33.2 was $\lambda_2\approx 0.44\le 0.667$ – the relaxation undershoots the true optimum, as it must, and the gap measures how much rounding cost we paid.

Trace it yourself

Redo this on the *path* P_4 (vertices $1-2-3-4$). Its Fiedler vector is $v_2\propto(1.31,\,0.54,\,-0.54,\,-1.31)$ and $\lambda_2=2-\sqrt{2}\approx 0.586$. The sign split gives $\{1,2\}\mid\{3,4\}$ – cut the middle edge, the only sensible bisection of a path. Confirm the sweep agrees, then try P_5 and watch the middle vertex land near zero, genuinely ambiguous.

33.3 Why the relaxation is good: gaps and Cheeger

A relaxation is only worth trusting if the rounded answer is provably close to optimal. For balanced cuts the guarantee is *Cheeger's inequality*, the central theorem of spectral partitioning. Define the *conductance* (a normalized, balance-aware cut quality) of a set S and of the graph:

$$\phi(S)\;=\;\frac{\operatorname{cut}(S)}{\min\{\operatorname{vol}(S),\,\operatorname{vol}(\overline{S})\}},\qquad \phi(G)\;=\;\min_{S\neq\emptyset,\,S\neq V}\phi(S).$$

Small conductance means a sparse, balanced cut – precisely a good cluster boundary. Let λ_2^{nrm} be the second-smallest eigenvalue of the *normalized* Laplacian $\mathbf{L}_{\mathrm{sym}}=\mathbf{D}^{-1/2}\mathbf{L}\,\mathbf{D}^{-1/2}$ (introduced next section).

Theorem 33.3 (Cheeger's inequality)**.** *For any connected graph,*

$$\frac{\lambda_2^{\mathrm{nrm}}}{2}\;\le\;\phi(G)\;\le\;\sqrt{2\,\lambda_2^{\mathrm{nrm}}}.$$

Why it works

The left inequality (the easy direction) says the eigenvalue lower-bounds the best conductance: no cut can be sparser than $\lambda_2^{\mathrm{nrm}}/2$, mirroring Theorem 33.2. The right inequality is the algorithmic payoff: it is *constructive*. Its proof takes the eigenvector v_2, runs exactly the *sweep* of Algorithm 129, and proves that some sweep cut achieves conductance $\le\sqrt{2\lambda_2^{\mathrm{nrm}}}$. So the simple "sort by eigenvector, try every prefix" rounding comes with a guarantee: it provably finds a cut within a quadratic factor of optimal.

Proof sketch of the hard direction. Let $g=\mathbf{D}^{1/2}v_2$, so the Rayleigh quotient $R(g)=\frac{g^{\top}\mathbf{L}g}{\sum_i d_i g_i^2}=\lambda_2^{\mathrm{nrm}}$. Order the vertices by g and consider the $n-1$ sweep sets $S_t=\{i: g_i>t\}$. The key combinatorial

lemma: if *every* sweep cut had conductance $> \phi$, then a probabilistic argument over a random threshold t (drawn with density proportional to $2|t|$) forces $R(g) > \phi^2/2$, contradicting $R(g) = \lambda_2^{\mathrm{nrm}}$ once we set $\phi = \sqrt{2\lambda_2^{\mathrm{nrm}}}$. Concretely, write the numerator and denominator of $R(g)$ as expectations over the random threshold; Cauchy–Schwarz relates the crossing-edge probability to the conductance of the realized sweep cut, and the bound on the minimum over sweep cuts pops out. The full proof appears in Chapter 32; the takeaway is that *the sweep over the eigenvector is the rounding the theorem analyzes.* □

Connection

Cheeger's inequality is the graph analogue of a Riemannian-geometry theorem (Cheeger 1970, Buser, Alon– Milman in the discrete setting): the smallest nonzero eigenvalue of the Laplace operator controls the narrowest "waist" of the space. On graphs it connects three subjects – **spectra** (companion *Linear Algebra* volume: eigenvalues of symmetric matrices), **cuts** (this volume: max-flow / min-cut, expansion), and **random walks** (companion *Probability* volume: λ_2^{nrm} is the spectral gap that governs mixing time, Chapters 35–36). A graph is an *expander* (Chapter 34) exactly when $\phi(G)$ is bounded below by a constant, equivalently $\lambda_2^{\mathrm{nrm}} = \Omega(1)$.

33.4 Normalized cuts and the random-walk view

RatioCut protects *cardinality* balance; normalized cut protects *volume* balance, which matches real data where dense clusters should be measured by total connection mass, not vertex count. The relaxation mirrors Section 33.2 but with $\mathbf{D}$-weighted inner products.

Encode a partition by $h_i = \sqrt{\mathrm{vol}(B)/\mathrm{vol}(A)}$ on A and $-\sqrt{\mathrm{vol}(A)/\mathrm{vol}(B)}$ on B. The same algebra as Lemma 33.1, now with the weighting $\mathbf{D}$, gives

$$h^\top \mathbf{L} h = \mathrm{vol}(V) \cdot \mathrm{NCut}(A,B), \qquad h^\top \mathbf{D} h = \mathrm{vol}(V), \qquad (\mathbf{D}h) \perp \mathbf{1}.$$

Substituting $h = \mathbf{D}^{-1/2} g$ turns the constrained problem into

$$\min_{g} \frac{g^\top \mathbf{D}^{-1/2} \mathbf{L}\, \mathbf{D}^{-1/2}\, g}{g^\top g} = \min_{g} \frac{g^\top \mathbf{L}_{\mathrm{sym}}\, g}{g^\top g} \quad \text{s.t. } g \perp \mathbf{D}^{1/2}\mathbf{1}.$$

The minimizing vector g is the second eigenvector of the *symmetric normalized Laplacian* $\mathbf{L}_{\mathrm{sym}} = \mathbf{D}^{-1/2}\mathbf{L}\,\mathbf{D}^{-1/2} = \mathbf{I} - \mathbf{D}^{-1/2}\mathbf{A}\mathbf{D}^{-1/2}$, and the cluster indicator is recovered as $h = \mathbf{D}^{-1/2} g$. There is a closely related *random-walk Laplacian* $\mathbf{L}_{\mathrm{rw}} = \mathbf{D}^{-1}\mathbf{L} = \mathbf{I} - \mathbf{D}^{-1}\mathbf{A}$, whose eigenvectors are exactly the h's and whose matrix $\mathbf{P} = \mathbf{D}^{-1}\mathbf{A}$ is the transition matrix of the natural random walk.

Intuition

Drop a random walker on the graph. It tends to get *trapped* inside a dense cluster: many edges keep it bouncing around inside, few edges let it escape. Normalized cut is, almost literally, the probability that a stationary walker crosses the boundary in one step. Minimizing NCut finds the partition the walker finds hardest to cross – the bottleneck. This is why $\mathbf{L}_{\mathrm{rw}}$, the walk's generator, is the right operator, and why its spectral gap is the walk's mixing rate.

Formal

The three Laplacians share eigenvalues in a tidy way. If $\mathbf{L}_{\text{sym}} g = \mu g$ then $\mathbf{L}_{\text{rw}}(\mathbf{D}^{-1/2} g) = \mu(\mathbf{D}^{-1/2} g)$ – same eigenvalue $\mu \in [0, 2]$, eigenvectors related by $\mathbf{D}^{-1/2}$. Use $\mathbf{L}_{\text{sym}}$ for computation (it is symmetric, so solvers are stable and the eigenvectors are orthonormal) and $\mathbf{L}_{\text{rw}}$ for interpretation (its eigenvectors are the cluster indicators h, not the rescaled g). The unnormalized $\mathbf{L}$ corresponds to RatioCut; the normalized pair corresponds to NCut. In practice the normalized version is preferred – it is more robust when degrees vary, a fact made precise by Cheeger's bound, which uses the *normalized* gap.

Tracing the normalized cut on G_0

For G_0 the degrees are $\mathbf{D} = \text{diag}(2, 2, 3, 3, 2, 2)$. Because the graph is nearly regular, the normalized Fiedler vector g is close to the unnormalized one but tilted: the degree-3 bridge vertices $3, 4$ get pulled toward zero a little harder (division by $\sqrt{3}$). Numerically the second eigenvalue of $\mathbf{L}_{\text{sym}}$ is $\mu_2 \approx 0.20$, and the recovered indicator $h = \mathbf{D}^{-1/2} g$ again changes sign exactly between $\{1, 2, 3\}$ and $\{4, 5, 6\}$. The NCut value of that split is

$$\text{NCut}(L, R) = \text{cut}(L, R)\left(\tfrac{1}{\text{vol}\, L} + \tfrac{1}{\text{vol}\, R}\right) = 1 \cdot \left(\tfrac{1}{7} + \tfrac{1}{7}\right) = \tfrac{2}{7} \approx 0.286,$$

since $\text{vol}\, L = \text{vol}\, R = 2 + 2 + 3 = 7$. Every other sweep cut scores worse (peeling a single degree-2 vertex costs $2 \cdot (\tfrac{1}{2} + \tfrac{1}{12}) \approx 1.17$), so the sweep again returns the two triangles. Cheeger's lower bound $\mu_2/2 \approx 0.10 \le \phi(G_0)$ is consistent with $\phi(G_0) = \text{cut}(L)/\,\text{vol}(L) = 1/7 \approx 0.143$.

33.5 k-way spectral clustering

For $k > 2$ clusters, recursively bisecting works but is myopic; the clean approach uses the *bottom k eigenvectors at once*. The idea generalizes the line embedding of Figure 33.3 from one dimension to k.

Intuition

One eigenvector lays the graph on a line. Two eigenvectors lay it on a plane; $k - 1$ of them lay it in $\mathbb{R}^{k-1}$. Each vertex i becomes a point $y_i \in \mathbb{R}^k$ whose coordinates are its entries in $v_1, \ldots, v_k$. Well-separated clusters in the graph become well-separated *point clouds* in this *spectral embedding*, and we finish with ordinary geometric clustering – k-means – in that space. The Laplacian has done the hard work of bending the graph into a shape where Euclidean distance means "hard to reach by walking on the graph."

Why the bottom k? Consider the ideal case of k *disconnected* clusters. Then $\mathbf{L}$ is block-diagonal, $\lambda_1 = \cdots = \lambda_k = 0$, and the eigenspace of 0 is spanned by the indicator vectors $\mathbf{1}_{C_1}, \ldots, \mathbf{1}_{C_k}$ of the components. Any orthonormal basis of that eigenspace gives an embedding in which *all vertices of one cluster map to a single point* of $\mathbb{R}^k$ – k distinct points total. k-means recovers the clusters exactly. When the clusters are merely well-separated rather than disconnected, the eigenvalues $\lambda_2, \ldots, \lambda_k$ lift slightly off 0 and the points spread into k tight blobs; matrix perturbation theory (the Davis–Kahan $\sin\Theta$ theorem in the companion *Linear Algebra* volume) bounds how much the embedding moves, hence how clean k-means stays.

Algorithm 130: NormalizedSpectralClustering(G, k) – Ng–Jordan–Weiss

Input: Weighted graph $G = (V, E, w)$, number of clusters k.
Output: A partition $V = C_1 \cup \cdots \cup C_k$.

1 $\mathbf{D} \leftarrow \operatorname{diag}(d_1, \ldots, d_n)$; $\mathbf{L}_{\text{sym}} \leftarrow \mathbf{I} - \mathbf{D}^{-1/2}\mathbf{A}\,\mathbf{D}^{-1/2}$.
2 Compute the k eigenvectors $u_1, \ldots, u_k$ of $\mathbf{L}_{\text{sym}}$ for its k *smallest* eigenvalues.
3 $U \leftarrow [\,u_1 \ \cdots \ u_k\,] \in \mathbb{R}^{n \times k}$ `// vertex i is row i of U`
4 **for** $i \leftarrow 1$ **to** n **do**
5 $\quad y_i \leftarrow U_{i,\cdot} \,/\, \|U_{i,\cdot}\|_2$ `// row-normalize onto the unit sphere`
6 $\{C_1, \ldots, C_k\} \leftarrow k$-Means$(\{y_1, \ldots, y_n\}, k)$.
7 **return** the partition assigning i to its k-means cluster.

Common trap

The row-normalization on line 5 (project each embedded point to the unit sphere) is the Ng–Jordan–Weiss trick and it matters: without it, high-degree and low-degree vertices land at different radii and k-means splits by radius instead of by cluster. A second subtlety: you need the k *smallest* eigenvalues, so use a solver tuned for the bottom of the spectrum (shift-and-invert Lanczos, or inverse iteration powered by the near-linear Laplacian solvers of Chapter 41), not a top-k method. Finally, k-means is non-convex; run it with several k-means++ seedings and keep the lowest-inertia result, or a bad initialization can spoil an otherwise good embedding.

A worked $k = 2$ embedding, and a $k = 3$ sketch

For $k = 2$ on G_0, $U = [\,u_1\ u_2\,]$ has first column $u_1 \propto \mathbf{D}^{1/2}\mathbf{1}$ (the constant direction, same for everyone up to degree) and second column u_2 = normalized Fiedler. After row-normalization every vertex sits on the unit circle; the three L-vertices cluster near one point and the three R-vertices near the antipodal point, because u_2 separates them while u_1 is shared. Two-means immediately recovers $\{1, 2, 3\}$ and $\{4, 5, 6\}$. This is the same partition the sweep found, now obtained geometrically; the same mechanism handles larger k.

To see $k = 3$, glue a *third* triangle $T_3 = \{7, 8, 9\}$ to G_0 with one bridge $(6, 7)$, giving a chain of three triangles. The bottom three eigenvalues are 0 and two small positives; the embedding $U \in \mathbb{R}^{9 \times 3}$ sends each triangle to a tight blob at a corner of a triangle in $\mathbb{R}^3$, and 3-means returns the three communities. The eigengap between λ_3 (small) and λ_4 (an order larger, governed by *within*-triangle structure) is the spectral signal that "$k = 3$ is the right number of clusters" – the *eigengap heuristic* for choosing k. Concretely the bottom of the spectrum is $\lambda_1 = 0,\ \lambda_2 \approx 0.21,\ \lambda_3 \approx 0.70,\ \lambda_4 = 3$, so the jump from λ_3 to λ_4 is the visible gap.

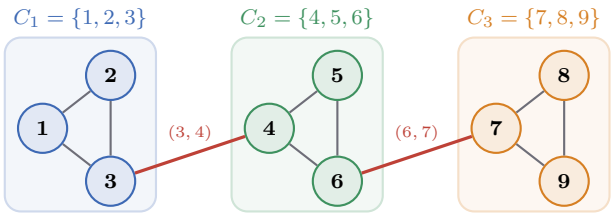


Figure 33.4: The three-triangle chain: three internally dense clusters $C_1 = \{1, 2, 3\}$, $C_2 = \{4, 5, 6\}$, and $C_3 = \{7, 8, 9\}$ linked only by sparse bottleneck bridge edges $(3, 4)$ and $(6, 7)$ (red).

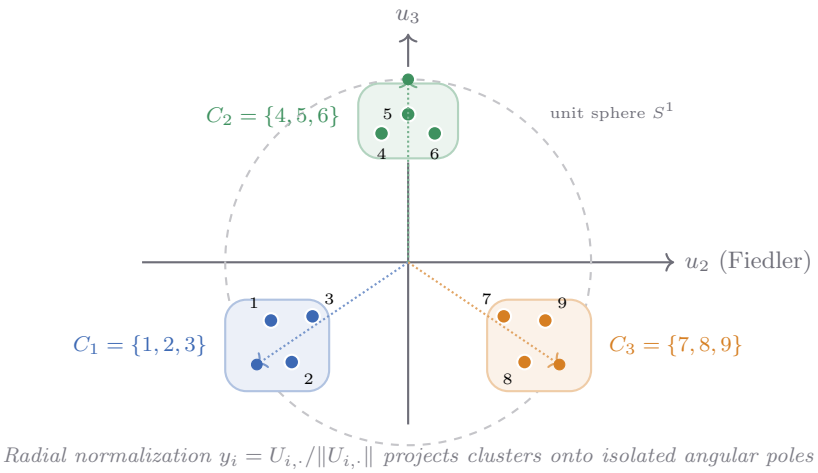


Figure 33.5: 2D spectral embedding of the three-triangle chain onto eigenvectors u_2 and u_3 of the normalized Laplacian. The nine vertices collapse into three tight, well-separated point clouds. Row-normalization projects each cluster onto the unit circle (dotted rays), where ordinary k-means cleanly isolates the three communities without ambiguity.

33.6 Complexity

The cost is dominated by the eigenvector computation. For a graph with m edges and target k clusters, an iterative bottom-eigenvector solver (Lanczos / LOBPCG) costs $O(m)$ per matrix–vector product and needs a number of iterations that grows with the conditioning, which depends on the spectral gap. Building the Laplacian is $O(m)$. The sweep in Algorithm 129 sorts ($O(n \log n)$) and then updates cut and vol incrementally as each vertex moves across, so all $n - 1$ sweep cuts cost $O(m)$ total, not $O(nm)$. The k-means stage on n points in $\mathbb{R}^k$ costs $O(nk \cdot I)$ per run for I iterations.

Cost summary

Spectral bisection (Alg. 129). Eigenpair (λ_2, v_2) by Lanczos: $\widetilde{O}(m/\sqrt{\lambda_2})$ to constant accuracy. Sweep rounding: $O(m + n \log n)$.

k-way (Alg. 130). Bottom k eigenvectors: $\widetilde{O}(mk/\sqrt{\text{gap}})$. Embedding and row-normalize: $O(nk)$. k-means: $O(nkI)$ per restart.

Exact eigendecomposition (dense, small graphs): $O(n^3)$ – what we used by hand on G_0.

With nearly-linear-time Laplacian solvers (Spielman–Teng and successors, Chapter 41), the eigenvector step can be made $\widetilde{O}(m)$, so the whole pipeline is near-linear in the graph size.

Application

Spectral clustering shows up wherever data forms a similarity graph. **Image segmentation**: pixels are vertices, edge weights are color/proximity similarities, NCut carves out objects (Shi–Malik). **Community detection** in social and biological networks. **Mesh and circuit partitioning** for parallel computation and VLSI layout, where a balanced cut minimizes inter-processor communication. **Manifold learning / dimensionality reduction**: Laplacian eigenmaps are literally the spectral embedding of Algorithm 130 used as features. **Document and gene clustering** via k-nearest-neighbor similarity graphs.

33.7 Building the graph: from data to similarity

Often the input is not a graph but n data points $x_1, \dots, x_n$. The clustering quality then hinges on how you build the similarity graph – a step at least as important as the spectral math.

- **ε-neighborhood graph**: connect i, j if $\|x_i - x_j\| \le \varepsilon$. Unweighted, symmetric; sensitive to ε and to varying density.
- **k-nearest-neighbor graph**: connect i to its k nearest points (then symmetrize by OR or AND). Adapts to local density; the usual default.
- **Fully connected Gaussian (RBF) graph**: $w_{ij} = \exp(-\|x_i - x_j\|^2 / 2\sigma^2)$. Smooth; the bandwidth σ sets the scale of "similar."

Common trap

The bandwidth σ and the neighbor count k are the real hyperparameters of spectral clustering, more consequential than anything in the eigensolver. Too large and everything connects to everything (λ_2 big, no clusters visible); too small and the graph fragments into singletons ($\lambda_2 \to 0$, spurious clusters). A robust default is to set σ to the median pairwise distance among k-nearest neighbors, or to use a self-tuning per-point bandwidth (Zelnik-Manor–Perona). When clusters fail to appear, suspect the graph construction before the eigenvectors.

33.8 Historical Context and Controversies

History

In the early 1970s, two communities working on very different problems arrived at the same mathematics. In Prague, Miroslav Fiedler was studying the physical mechanics of elastic trusses and electrical networks. He discovered that the second-smallest eigenvalue λ_2 of the graph Laplacian vanished if and only if the network fell into disconnected pieces, and its size measured structural cohesion. The signs of this eigenvector (the Fiedler vector) split the

vertices into two natural clusters. At about the same time, at IBM, Donath and Hoffman were trying to partition logic circuits across silicon chips to minimize inter-chip wire delays. Because balanced partitioning is NP-hard, they turned to the eigenvalues of the adjacency and Laplacian matrices for lower bounds.

Three decades later, in 2000, computer vision researchers encountered this same geometry when trying to segment digital photographs into distinct objects. Local edge-detection filters routinely failed because pixel noise broke contours apart. Jianbo Shi and Jitendra Malik realized an image is better viewed as a global similarity graph, where cut boundaries should minimize the normalized cut. Soon after, Andrew Ng, Michael Jordan, and Yair Weiss developed the modern normalized spectral clustering pipeline, showing that eigenvector embeddings reveal cluster structure that standard methods miss.

Early attempts ran into two natural pitfalls. The first was reaching for the polynomial-time min-cut algorithm from network flows, which in practice routinely isolates a single disconnected outlier or leaf vertex rather than splitting the graph into two meaningful halves. Penalizing cut sizes to avoid this imbalance makes the problem NP-hard. The second pitfall was running classic k-means directly on raw spatial coordinates, which assumes clusters are spherical and convex, failing completely on intertwined spirals or concentric rings. Spectral clustering bypasses both pitfalls: continuous eigenvector relaxation gives a smooth geometric embedding where simple thresholding recovers non-convex clusters, backed by Cheeger's discrete isoperimetric inequalities.

33.8.1 What We Can Now Do and Its Limitations

Focus

Capabilities Unlocked:

- **Non-Convex Manifold Clustering**: By mapping vertices into the eigenspace of $\mathbf{L}_{\text{sym}}$, complex non-linear structures (like concentric rings) are untangled into radially separated rays on the unit sphere, where simple k-means succeeds.
- **Global Relaxation**: Rather than getting trapped in local minima like greedy graph search, spectral methods solve for the global optimum of a continuous quadratic relaxation in polynomial time.

Honest Limitations and Downsides:

- **Computational Burden on Large Graphs**: Full exact eigendecomposition requires $O(n^3)$ operations. While sparse iterative solvers (Lanczos, LOBPCG) compute the bottom k eigenvectors in $\widetilde{O}(mk/\sqrt{\text{gap}})$, they stall when the eigengap is tiny.
- **The Cheeger Quadratic Loss**: When λ_2 is small, the Cheeger guarantee $\phi \leq \sqrt{2\lambda_2}$ can be loose. A graph with $\lambda_2 = 0.0001$ has conductance bounded only by $\sqrt{0.0002} \approx 0.014$, leaving room for an order-of-magnitude discrepancy between the continuous relaxation and the discrete cut.
- **Fragility to Degree Outliers**: In unnormalized clustering ($\mathbf{L}$), a single high-degree hub can dominate the quadratic form and pull the Fiedler vector toward itself, isolating the hub rather than uncovering balanced communities.

Connection

Throughout this chapter, the relaxations are *Rayleigh-quotient minimizations*, the guarantees rest on *Courant–Fischer* and *Davis–Kahan* from the companion *Linear Algebra* volume, and the operator is the *graph Laplacian* (Chapter 31) whose spectrum we analyzed in Chapter 32. Moving forward, **Chapter 34 (Expander Graphs)** studies the opposite extreme where λ_2 stays bounded away from zero so no sparse cut exists; **Chapter 35 (Random Walks and Electrical Networks)** interprets $\mathbf{L}_{\mathrm{rw}}$ as the walk generator and λ_2^{nrm} as the mixing rate; **Chapter 36 (PageRank and Markov Chains)** uses personalized random walks for localized clustering; and **Chapter 41 (Laplacian Solvers and Electrical Flows)** develops the Spielman–Teng solvers that compute Laplacian linear solves in nearly-linear $\widetilde{O}(m)$ time. In the companion *Probability* and *Machine Learning* volumes, these same embeddings reappear in k-means, Gaussian similarity kernels, and Laplacian eigenmaps.

33.9 Exercises

Exercise 33.1 (Verify the Fiedler trace)**.** Compute the full eigendecomposition of the Laplacian of G_0 (Figure 33.2) by hand or by computer. Confirm $\lambda_2 \approx 0.44$ and that the Fiedler vector is antisymmetric under the $L \leftrightarrow R$ mirror. Which eigenvalue and eigenvector correspond to the *symmetric* mode, and what cut, if any, does that mode suggest?

Exercise 33.2 (The barbell and the dumbbell)**.** Let G be two cliques K_t joined by a single edge (a barbell). Show $\lambda_2 = \Theta(1/t^2)$ and that the Fiedler vector cleanly separates the two cliques. Then replace the joining edge by a path of length ℓ (a dumbbell). What happens to λ_2 and to the Fiedler partition as ℓ grows? Where does the sweep cut the path?

Exercise 33.3 (Prove the easy direction of Cheeger)**.** Show $\phi(G) \geq \lambda_2^{\mathrm{nrm}}/2$ directly: plug the indicator-derived test vector h of the optimal conductance set into the Rayleigh quotient of $\mathbf{L}_{\mathrm{sym}}$ and bound it. Identify exactly where the factor of 2 is lost.

Exercise 33.4 (RatioCut vs. NCut diverge)**.** Construct a graph on which the optimal RatioCut bipartition and the optimal NCut bipartition are *different*. (Hint: make one side have many low-degree vertices and the other few high-degree vertices, so cardinality balance and volume balance disagree.) Compute both objectives for both partitions.

Exercise 33.5 (Sweep beats sign)**.** Exhibit a connected graph whose Fiedler vector has a sign split that is a *bad* cut, while some non-sign sweep cut is good. Conclude that the sign threshold of Algorithm 129 is not safe to use alone, justifying the full sweep.

Exercise 33.6 (Disconnected = degenerate)**.** Show that if G has exactly c connected components, then $\lambda_1 = \cdots = \lambda_c = 0$ and the 0-eigenspace is spanned by component indicators. Explain why k-way spectral clustering with $k = c$ then recovers the components *exactly*, for any choice of orthonormal basis in line 2 of Algorithm 130.

Exercise 33.7 (Implement and break)**.** Implement Algorithm 130. Run it on three Gaussian blobs (easy), then on two concentric circles (the classic case where k-means alone fails but spectral clustering with a k-NN graph succeeds). Now *break* it: shrink σ until the inner circle fragments. Report the λ spectrum at the breaking point and explain it via Section 33.7.

Exercise 33.8 (Eigengap heuristic)**.** For the three-triangle chain of Section 33.5, compute the bottom five eigenvalues and exhibit the gap between λ_3 and λ_4. Then add edges between the triangles until the gap closes; relate the closing gap to the merging of clusters and to Cheeger's inequality.

Exercise 33.9 (Recursive vs. simultaneous)**.** Compare recursive spectral bisection (bisect, then bisect each half) against the simultaneous k-eigenvector Algorithm 130 on a graph with three clusters of *very* different sizes. Find an instance where recursion makes the wrong first cut and never recovers, but the simultaneous method succeeds.

Exercise 33.10 (Normalized vs. unnormalized perturbation)**.** Take a graph with a few very high-degree "hub" vertices. Show empirically that unnormalized spectral clustering ($\mathbf{L}$) tends to isolate hubs while normalized clustering ($\mathbf{L}_{\text{sym}}$) does not. Explain using the role of $\mathbf{D}$ in the relaxation of Section 33.4.

Chapter 34

Expander Graphs

In Chapter 33, we searched for bottlenecks to partition graphs into clusters. An expander is the opposite extreme: a graph with no bottlenecks anywhere, where every cut is large and information spreads quickly.

A good road network has a peculiar property. No matter how you try to split the cities into two halves, you always have to cut a lot of roads. There is no narrow waist, no bottleneck you can sever with a few cuts to isolate a large region. Every set of cities, unless it is almost everybody or almost nobody, has a thick boundary leaking out to the rest. Networks like this are robust: knock out a handful of links and the graph stays connected; start a rumor anywhere and it floods the whole network in a logarithmic number of rounds; run a random walk and it forgets where it started almost immediately.

These are *expander graphs*, and they are among the most useful objects in theoretical computer science. They are sparse—only $O(n)$ edges—yet they behave, in every way that matters for routing, mixing, and information flow, like the complete graph on n vertices. This combination of dense-graph connectivity with a sparse graph's edge count is why they appear in so many places: in fault-tolerant networks, in derandomization, in error-correcting codes, in the analysis of Markov chains, and behind nearly optimal algorithms for cuts and flows.

This chapter develops expanders from three complementary angles that prove quantitatively equivalent. The *combinatorial* angle measures expansion by counting boundary edges. The *spectral* angle measures it by the gap between the first two Laplacian eigenvalues. The *probabilistic* angle shows that a random sparse regular graph is, with high probability, a strong expander. We tie the combinatorial and spectral views together with Cheeger's inequality and the Expander Mixing Lemma, sketch how to build expanders explicitly (the genuinely difficult step), and then turn to applications. Throughout we build on the Laplacian and spectral tools of Chapters 31–33.

34.1 Two ways to measure a bottleneck

Fix a graph $G = (V, E)$ with $n = |V|$ vertices. For most of this chapter we take G to be d-regular: every vertex has exactly d neighbors. Regularity is not strictly necessary, but it keeps the algebra clean, the eigenvalues comparable, and the constructions natural. The fundamental question is: *how hard is it to cut G into two substantial pieces?*

Intuition

Picture a cycle C_n versus the complete graph K_n. To split the cycle in half you snip exactly two edges; the cycle has a thin waist everywhere. To split K_n in half you must cut roughly $n^2/4$ edges—a quarter of all pairs. The cycle is the worst expander; the complete graph is the best. An expander is a sparse graph (like the cycle, $O(n)$ edges) that is hard to cut (like K_n). This chapter studies the tension between "few edges" and "hard to cut."

For a set $S \subseteq V$ write $\bar{S} = V \setminus S$. The *edge boundary* is

$$\partial S \;=\; \{(u,v) \in E : u \in S,\; v \in \bar{S}\},$$

the set of edges with exactly one endpoint in S. Cutting S away from the rest costs $|\partial S|$ edges.

Definition 34.1 (Edge expansion / Cheeger constant)**.** The *edge expansion* (or *conductance*, or *Cheeger constant*) of G is

$$h(G) \;=\; \min_{\substack{S \subseteq V \\ 0 < |S| \le n/2}} \frac{|\partial S|}{|S|}.$$

The restriction $|S| \le n/2$ matters: we normalize by the size of the *smaller* side. Without it, taking $S = \emptyset$ or $S = V$ would produce an uninformative ratio of 0. Reading the definition aloud: $h(G)$ is large exactly when *every* set of size at most $n/2$ has many edges escaping it. A large h means no bottleneck.

Definition 34.2 (Combinatorial expander)**.** A family $\{G_n\}$ of d-regular graphs (with $n \to \infty$, d fixed) is a *family of expanders* if there is a constant $\varepsilon > 0$ with $h(G_n) \ge \varepsilon$ for all n.

The word "family" is essential. A single finite graph always has *some* expansion; the content is that expansion stays bounded *away from zero* as the graph grows, while the degree d—and hence the edge count—stays constant. The surprise is that arbitrarily large graphs with only $dn/2$ edges can still guarantee that a fixed positive fraction of any set's volume leaks out.

Focus

Three normalizations float around the literature, and conflating them is the classic source of off-by-a-factor confusion.

- **Edge expansion** $h(G) = \min |\partial S|/|S|$ (boundary per vertex; what Definition 34.1 uses).
- **Conductance** $\phi(G) = \min |\partial S|/(d|S|)$ for $|S| \le n/2$ (boundary per unit *volume*; volume = sum of degrees). For d-regular graphs $\phi = h/d$.
- **Vertex expansion** counts boundary *vertices* rather than edges (below).

We use h for combinatorics and ϕ when we want a quantity in $[0,1]$ that matches random-walk mixing. Always check which one a source means.

There is a second, often stronger, notion that counts vertices instead of edges. Let $N(S)$ be the set of vertices outside S adjacent to something in S, the *vertex boundary*.

Definition 34.3 (Vertex expansion)**.** G is a (K, A) *vertex expander* if every set S with $|S| \le K$ satisfies $|N(S)| \ge A|S|$.

Vertex expansion with $A > 1$ says small sets have *more* outside neighbors than they have members—a strict "growth" condition. It is what bipartite expanders for error-correcting codes will need (Section 34.6.4). Edge and vertex expansion are related but not identical; a vertex of high degree can have a big edge boundary while touching few distinct vertices. For bounded degree d they sandwich each other within constant factors: $|\partial S|/d \le |N(S)| \le |\partial S|$, so $h/d \le$ (vertex expansion of small sets) $\le h$.

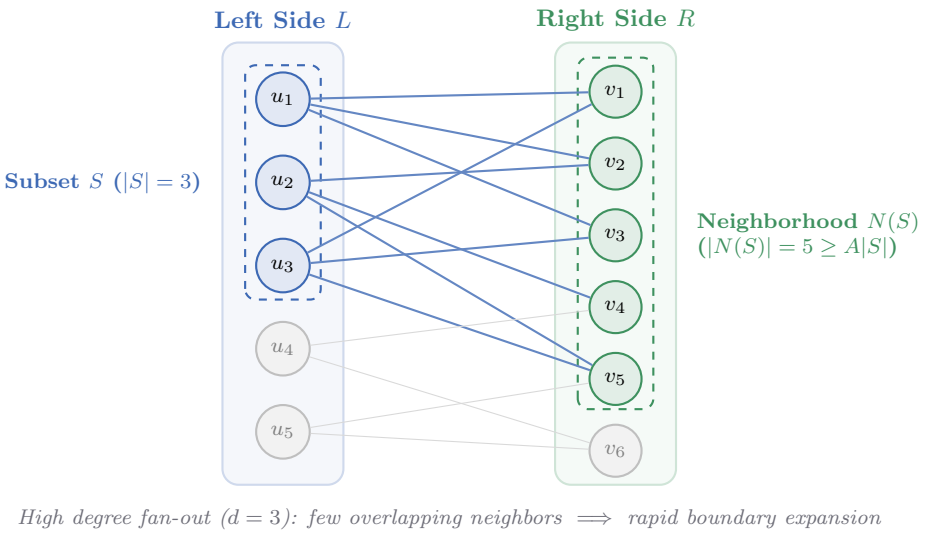


Figure 34.1: Bipartite expander neighbor fan-out. A subset S of 3 variable nodes on the left connects to $|N(S)| = 5$ distinct constraint nodes on the right via non-colliding edges. In a lossless expander, every small set S achieves expansion $|N(S)| \ge (1 - \varepsilon)d|S|$, guaranteeing that most neighbors touch S at a unique edge.

Trace it yourself

Take the 3-cube Q_3: vertices are bit strings $000, \dots, 111$, edges connect strings differing in one bit. It is 3-regular on 8 vertices. Pick $S = \{000, 001, 010, 011\}$ (all strings starting with 0). Its boundary edges are exactly the four "flip the first bit" edges 000–100, 001–101, 010–110, 011–111. So $|\partial S| = 4$, $|S| = 4$, ratio 1. Now try $S = \{000\}$: boundary $= 3$ (its three neighbors), ratio 3. Try $S = \{000, 111\}$ (an antipodal pair, not adjacent): each contributes 3 boundary edges, none shared, ratio $6/2 = 3$. The half-cube split is the cheapest, so $h(Q_3) = 1$. Verify by checking a couple more 4-sets that none beats ratio 1.

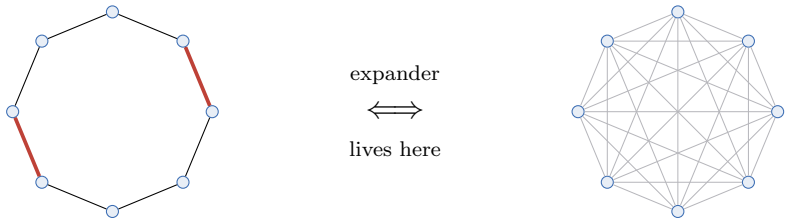


Figure 34.2: The two extremes. A cycle has a thin waist (small h); a complete graph has a thick waist (large h). An expander wants the cut-resistance of K_n at the edge-budget of the cycle.

34.2 The spectral viewpoint and Cheeger's inequality

Computing $h(G)$ exactly means minimizing over all 2^{n-1} subsets; the associated decision problem is NP-hard (it is essentially sparsest cut). The way around this, and the central idea of the chapter, is that a single eigenvalue of the Laplacian *estimates* h to within a quadratic factor, and that eigenvalue is computable in polynomial time.

Recall from Chapter 31 the Laplacian $\mathbf{L} = \mathbf{A}_D - \mathbf{A}$, where $\mathbf{A}_D$ is the diagonal degree matrix and $\mathbf{A}$ the adjacency matrix. For a d-regular graph $\mathbf{A}_D = dI$, so $\mathbf{L} = dI - \mathbf{A}$. The key identity is

$$x^\top \mathbf{L} x \;=\; \sum_{(u,v)\in E} \big(x_u - x_v\big)^2,$$

the sum of squared differences across edges—a measure of how non-smooth x is along the graph. $\mathbf{L}$ is symmetric positive semidefinite, so its eigenvalues are real and nonnegative:

$$0 = \lambda_1 \le \lambda_2 \le \cdots \le \lambda_n.$$

The all-ones vector $\mathbf{1}$ has $\mathbf{L}\mathbf{1} = 0$, giving $\lambda_1 = 0$. The multiplicity of 0 equals the number of connected components, so $\lambda_2 > 0$ iff G is connected. The quantity λ_2 is the *spectral gap* (for regular graphs; the algebraic connectivity or Fiedler value of Chapters 31 and 32). It is the single number that controls expansion.

Intuition

Why should an eigenvalue know about cuts? Suppose G had a sparse cut $(S, \bar{S})$. The indicator-like vector that is $+1/|S|$ on S and $-1/|\bar{S}|$ on $\bar{S}$, made orthogonal to $\mathbf{1}$, is nearly constant inside each side and only "pays" $x^\top \mathbf{L} x$ on the few boundary edges. So it is a cheap test vector, and by the variational (Courant–Fischer) characterization

$$\lambda_2 = \min_{x \perp \mathbf{1},\; x \neq 0} \frac{x^\top \mathbf{L} x}{x^\top x},$$

a cheap test vector forces λ_2 to be small. Sparse cut $\Rightarrow$ small λ_2. Cheeger's inequality supplies the converse, which is the harder direction: small λ_2 *forces* a sparse cut to exist, and you can find it by rounding the eigenvector.

Theorem 34.1 (Cheeger's inequality, discrete form)**.** *For any connected d-regular graph G,*

$$\frac{\lambda_2}{2} \;\le\; h(G) \;\le\; \sqrt{2\,d\,\lambda_2}.$$

Equivalently, in conductance $\phi = h/d$: $\lambda_2/(2d) \le \phi \le \sqrt{2\lambda_2/d}$.

The two sides have very different characters. The left inequality (easy) says expansion can be no larger than λ_2 permits. The right inequality (the Cheeger rounding) says that if λ_2 is bounded below by a constant, then h is too—which is exactly Definition 34.2. So:

A family of d-regular graphs is a family of expanders iff $\lambda_2 \ge \varepsilon'$ for a constant $\varepsilon' > 0$.

The spectral gap and combinatorial expansion are, up to the quadratic gap, the same phenomenon.

Why it works

Proof of the easy direction $\lambda_2 \le 2h$. Let S achieve $h = |\partial S|/|S|$ with $|S| \le n/2$. Build the test vector

$$x_v = \begin{cases} |\bar{S}| & v \in S, \\ -|S| & v \in \bar{S}. \end{cases}$$

Then $\sum_v x_v = |S||\bar{S}| - |\bar{S}||S| = 0$, so $x \perp \mathbf{1}$. An edge inside S or inside $\bar{S}$ contributes 0 to $x^\top \mathbf{L} x$; a boundary edge contributes $(|\bar{S}| + |S|)^2 = n^2$. Hence $x^\top \mathbf{L} x = |\partial S|\, n^2$. Meanwhile $x^\top x = |S|\,|\bar{S}|^2 + |\bar{S}|\,|S|^2 = |S||\bar{S}|\, n$. By Courant–Fischer,

$$\lambda_2 \le \frac{x^\top \mathbf{L} x}{x^\top x} = \frac{|\partial S|\, n^2}{|S||\bar{S}|\, n} = \frac{|\partial S|\, n}{|S||\bar{S}|}.$$

Since $|\bar{S}| \ge n/2$, we get $n/|\bar{S}| \le 2$, so $\lambda_2 \le 2|\partial S|/|S| = 2h$. □

The hard direction takes the second eigenvector x (the Fiedler vector), sorts vertices by their coordinate value, and considers only the $n-1$ *threshold cuts* $S_t = \{v : x_v > t\}$. A clever averaging argument—multiply and divide by $\sum |x_u^2 - x_v^2|$ and apply Cauchy–Schwarz across edges—shows at least one threshold cut has $|\partial S_t|/|S_t| \le \sqrt{2d\lambda_2}$. Algorithm 131 writes out the sweep procedure implicit in this proof, since it is what you actually run.

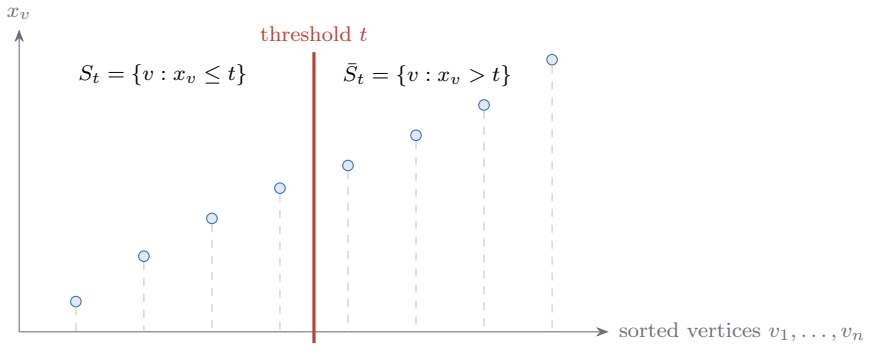


Figure 34.3: Cheeger rounding (Algorithm 131). Sort vertices by their Fiedler coordinate x_v; each of the $n-1$ gaps is a candidate threshold cut S_t. The sweep evaluates $|\partial S_t|/|S_t|$ at every gap and keeps the best; the analysis guarantees one of them is within the quadratic Cheeger bound $\sqrt{2d\lambda_2}$.

Algorithm 131: Spectral partitioning / Cheeger rounding

Input: connected d-regular graph $G = (V, E)$ on n vertices.
Output: a cut $S^\star$ with $|\partial S^\star|/|S^\star| \le \sqrt{2d\lambda_2}$.

```
1  Form L = dI − A;
2  Compute λ_2 and its eigenvector x (e.g. inverse power iteration deflating 1);
3  Sort vertices so that x_{v_1} ≤ x_{v_2} ≤ ··· ≤ x_{v_n};
4  best ← ∞;   S* ← ∅;
5  for t ← 1 to n − 1 do
6      S_t ← {v_1, ..., v_t};
7      compute |∂S_t| incrementally (add edges of v_t to outside, subtract those now inside);
8      r ← |∂S_t| / min(|S_t|, n − |S_t|);
9      if r < best then
10         best ← r;   S* ← S_t;
11 return S*;
```

Cost summary

The eigenvector dominates. With a sparse $\mathbf{L}$ (dn nonzeros) and a near-linear-time Laplacian solver, λ_2 and x come in $\tilde{O}(dn)$ time; classical inverse iteration is $O(\text{nnz} \cdot \text{iters})$. The sweep over threshold cuts (lines 5–11; see Figure 34.3) is $O(dn)$ total, since each vertex is inserted once and each of its d edges flips boundary status once. So the bottleneck is the eigenpair, and the guarantee on the returned cut is the quadratic Cheeger bound $\sqrt{2d\lambda_2}$.

Common trap

The Cheeger guarantee is *not* that you find the sparsest cut. You find a cut within a *quadratic* factor of optimal: if the true h is $\Theta(\lambda_2)$ you are fine, but when λ_2 is tiny the rounding can be off by a $\sqrt{\cdot}$ factor. The quadratic loss is real and, in the worst case, unavoidable for this rounding—it is the reason Chapter 38 reaches for SDP-based sparsest-cut approximations when tighter guarantees matter. Spectral partitioning is fast and usually works well in practice, but it is not an exact cut oracle.

Reading note

The discrete Cheeger inequality, transported from Riemannian manifolds by Dodziuk and by Alon and Milman (1984–1985), makes λ_2 a computable certificate of expansion. Alon's 1986 paper "Eigenvalues and expanders" made this spectral connection a foundation of expander analysis and Markov chain mixing.

34.2.1 The adjacency spectrum and the second eigenvalue

For d-regular graphs there is a parallel and equally common language using the *adjacency* eigenvalues. Since $\mathbf{L} = dI - \mathbf{A}$, if $\mathbf{A}$ has eigenvalues $d = \mu_1 \ge \mu_2 \ge \cdots \ge \mu_n \ge -d$ then $\mathbf{L}$ has eigenvalues $\lambda_i = d - \mu_i$. The top adjacency eigenvalue is always $\mu_1 = d$ (eigenvector $\mathbf{1}$), corresponding to $\lambda_1 = 0$. The spectral gap $\lambda_2 = d - \mu_2$ is large exactly when the second adjacency eigenvalue μ_2 is much smaller than d.

Define

$$\mu(G) \;=\; \max\bigl(|\mu_2|, |\mu_n|\bigr),$$

the largest *nontrivial* adjacency eigenvalue in absolute value. Pulling in μ_n as well controls the bipartite-like "near $-d$" eigenvalues that ruin mixing. A d-regular graph is called a *one-sided* (n, d, λ)-*expander* if $\mu_2 \le \lambda$, and *two-sided* if $\mu \le \lambda$ with $\lambda \ll d$. Smaller λ means a better expander. The next two sections show why the ratio λ/d is the parameter that matters.

Formal

How small can λ be? Not arbitrarily. The *Alon–Boppana bound* says any infinite family of d-regular graphs has $\mu_2 \ge 2\sqrt{d-1} - o(1)$. So $2\sqrt{d-1}$ is a wall: no family beats it asymptotically. Graphs that meet it, $\mu = \max(|\mu_2|, |\mu_n|) \le 2\sqrt{d-1}$, are called *Ramanujan graphs*—optimal expanders. They exist (Lubotzky–Phillips–Sarnak, 1988; Marcus–Spielman–Srivastava for bipartite, 2013), and they are as good as a sparse d-regular graph can ever be.

Figure 34.4 shows this geometry on the adjacency eigenvalue axis $[-d, d]$: the trivial eigenvalue sits at $\mu_1 = d$, while a two-sided expander traps all remaining eigenvalues $\mu_2, \dots, \mu_n$ inside $[-\mu, \mu]$ with a wide spectral gap $d - \mu$ on both ends.

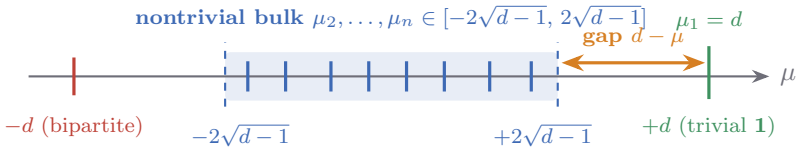


Figure 34.4: Adjacency spectrum of a d-regular Ramanujan expander. All nontrivial eigenvalues $\mu_2, \dots, \mu_n$ are confined within the Alon–Boppana interval $[-2\sqrt{d-1}, 2\sqrt{d-1}]$, keeping both the unmixing gap $d - \mu_2$ and the bipartite gap $d - |\mu_n|$ large.

34.3 The Expander Mixing Lemma

The central quantitative consequence of a spectral gap is that an expander graph *mimics a random graph*. In a truly random d-regular graph, the number of edges between two sets S, T should be about $\frac{d}{n}|S||T|$ (each of the $|S||T|$ ordered pairs is an edge with probability $\approx d/n$). The Expander Mixing Lemma says that in *any* graph with small λ, the edge counts are close to that ideal, with error controlled by λ.

Theorem 34.2 (Expander Mixing Lemma)**.** *Let G be d-regular with $\mu = \max(|\mu_2|, |\mu_n|)$. For all $S, T \subseteq V$,*

$$\Bigl|\, e(S,T) - \frac{d}{n}|S|\,|T| \,\Bigr| \;\le\; \mu\sqrt{|S|\,|T|},$$

where $e(S,T)$ counts edges with one endpoint in S and one in T (pairs in $S \cap T$ counted twice).

Why it works

Proof. Let $\mathbf{1}_S, \mathbf{1}_T$ be indicator vectors. Then $e(S,T) = \mathbf{1}_S^\top \mathbf{A}\, \mathbf{1}_T$ exactly, by definition of $\mathbf{A}$. Decompose each indicator into its component along $\mathbf{1}$ and the rest:

$$\mathbf{1}_S = \frac{|S|}{n}\mathbf{1} + s^\perp, \qquad \mathbf{1}_T = \frac{|T|}{n}\mathbf{1} + t^\perp,$$

with $s^\perp, t^\perp \perp \mathbf{1}$. Since $\mathbf{A}\mathbf{1} = d\mathbf{1}$,

$$\mathbf{1}_S^\top \mathbf{A}\, \mathbf{1}_T = \frac{|S||T|}{n^2}\, \mathbf{1}^\top \mathbf{A}\mathbf{1} + (s^\perp)^\top \mathbf{A}\, t^\perp = \frac{d}{n}|S||T| + (s^\perp)^\top \mathbf{A}\, t^\perp.$$

The cross terms vanish because $\mathbf{A}\mathbf{1} = d\mathbf{1} \perp s^\perp, t^\perp$. On the subspace orthogonal to $\mathbf{1}$, every eigenvalue of $\mathbf{A}$ has absolute value $\le \mu$, so $\|\mathbf{A}\, t^\perp\| \le \mu \|t^\perp\|$. By Cauchy–Schwarz,

$$|(s^\perp)^\top \mathbf{A}\, t^\perp| \le \mu\, \|s^\perp\| \|t^\perp\| \le \mu \sqrt{|S|}\sqrt{|T|},$$

using $\|s^\perp\|^2 = |S|(1 - |S|/n) \le |S|$ and likewise for T. Rearranging gives the claim. □

The proof is three lines once you see that $e(S,T)$ is literally a quadratic form in the adjacency matrix and that orthogonality to $\mathbf{1}$ kills the leading term. This is the cleanest illustration of the chapter's thesis: *combinatorial counting becomes linear algebra*.

Trace it yourself

Test the lemma on the cube Q_3 ($d = 3$, $n = 8$). Its adjacency eigenvalues are $3, 1, 1, 1, -1, -1, -1, -3$, so $\mu = \max(|1|, |-3|) = 3$. That equals d, so Q_3 is a *bad* expander (the -3 eigenvalue flags that it is bipartite). Indeed for S = even-weight strings, T = odd-weight strings ($|S| = |T| = 4$), every edge of Q_3 goes between them: $e(S,T) = 12$, while $\frac{d}{n}|S||T| = \frac{3}{8} \cdot 16 = 6$. The deviation is 6, against the stated bound $\mu\sqrt{|S||T|} = 3 \cdot 4 = 12$—so the lemma holds with a factor-of-two cushion. The slack is exactly the relaxation $\|s^\perp\|^2 \le |S|$ in the proof: here $\|s^\perp\|^2 = |S|(1 - |S|/n) = 4 \cdot \frac{1}{2} = 2$, and the tighter Cauchy–Schwarz form $\mu\|s^\perp\|\|t^\perp\| = 3 \cdot \sqrt{2} \cdot \sqrt{2} = 6$ *is* met with equality, driven by the $\mu_n = -3$ eigenvector (the bipartition sign vector). The lesson: bipartiteness ($\mu_n = -d$) obstructs rapid mixing, which is why the two-sided spectral bound depends on the absolute value $|\mu_n|$ alongside μ_2.

Corollary 34.1 (Spectral expansion controls edge expansion)**.** *For $|S| \le n/2$, setting $T = \bar{S}$ in Theorem 34.2 gives*

$$|\partial S| = e(S, \bar{S}) \ge \frac{d}{n}|S||\bar{S}| - \mu\sqrt{|S||\bar{S}|}.$$

When $\mu \ll d$ this already pins down balanced cuts: at $|S| = |\bar{S}| = n/2$ it reads $|\partial S| \ge \left(\frac{d}{2} - \mu\right)\frac{n}{2}$. For the uniform statement over all *$|S| \le n/2$ it is cleaner to route through the easy Cheeger direction: $d - \mu_2 = \lambda_2 \le 2h$, and since $\mu_2 \le \mu$,*

$$h(G) \;\ge\; \frac{\lambda_2}{2} \;=\; \frac{d - \mu_2}{2} \;\ge\; \frac{d - \mu}{2}.$$

A spectral gap $d - \mu$ bounded away from 0 forces constant edge expansion.

34.4 Random graphs are expanders

Existence is easy and instructive: almost every sparse regular graph is a near-optimal expander. The cleanest model is the *random d-regular graph* $G(n, d)$, drawn uniformly from all d-regular graphs on n labeled vertices (concretely, via the configuration / pairing model: give each vertex d half-edges and match the dn half-edges uniformly at random). We prove combinatorial expansion directly with a union bound; the spectral statement is deeper.

Theorem 34.3 (A random regular graph expands)**.** *Fix $d \geq 3$. There is a constant $\varepsilon > 0$ (depending only on d) such that a uniformly random d-regular graph on n vertices satisfies $h(G) \geq \varepsilon$ with probability $1 - o(1)$.*

Why it works

Proof sketch (first-moment / union bound). Call a set S with $|S| = k \leq n/2$ *bad* if $|\partial S| < \varepsilon k$, i.e. fewer than εk edges leave it. We bound the expected number of bad sets and show it is $o(1)$ when ε is small enough.
Fix k. There are $\binom{n}{k}$ candidate sets. For a fixed S of size k to be bad in the pairing model, most of its dk half-edges must match *within* S: at most εk may cross to $\bar{S}$. The probability that a given half-edge of S stays inside is roughly $dk/(dn) = k/n$, and the events are nearly independent, so the chance that all but εk of the dk half-edges stay inside is at most

$$\binom{dk}{\varepsilon k}\left(\frac{k}{n}\right)^{dk-\varepsilon k}.$$

Multiply by $\binom{n}{k} \leq (en/k)^k$ and sum over k. The dominant factor is $(en/k)^k \cdot (k/n)^{(d-\varepsilon)k} = \left[e\,(k/n)^{d-\varepsilon-1}\right]^k$. For $d \geq 3$ and ε small the exponent $d - \varepsilon - 1 \geq 1$, so $(k/n)^{d-\varepsilon-1} \leq k/n$, and the bracket is at most $e\,k/n < 1$ for $k \leq n/2$ once we absorb the binomial entropy. The geometric-type sum over k is $o(1)$. Hence with probability $1 - o(1)$ no bad set exists, i.e. $h \geq \varepsilon$. □

Intuition

Why does randomness help? A small set S has dk half-edges waiting to be paired. There are dn half-edges total but only dk inside S. With $k \leq n/2$, the overwhelming majority of partners live outside S, so a typical half-edge escapes. To trap a set (make it bad) you would need a wildly atypical matching that keeps almost all of S's half-edges internal—combinatorially rare. Expansion is the *generic* behavior; non-expanders (cycles, grids) are the special, structured exceptions.

The spectral counterpart is the celebrated result of Friedman (2008), settling Alon's conjecture: a random d-regular graph is *almost Ramanujan*.

Theorem 34.4 (Friedman, informal)**.** *For every $\varepsilon > 0$, a random d-regular graph satisfies $\mu(G) \leq 2\sqrt{d-1} + \varepsilon$ with probability $1 - o(1)$.*

So random graphs essentially meet the Alon–Boppana wall. This settles existence but does not help when you need a *specific*, checkable graph—e.g. wired into hardware or a code—and cannot afford to verify λ_2 on the fly or risk a bad draw. That is the motivation for explicit constructions.

	degree d	edges	$h(G)$ as $n \to \infty$	conductance $\phi = h/d$
cycle C_n	2	n	$\to 0$ ($\sim 4/n$)	$\to 0$ ($\sim 2/n$)
$\sqrt{n} \times \sqrt{n}$ grid	4	$\sim 2n$	$\to 0$ ($\sim 1/\sqrt{n}$)	$\to 0$
hypercube $Q_{\log n}$	$\log n$	$\frac{n \log n}{2}$	1 (half-cube cut)	$\Theta(1/\log n)$
random d-regular	d (fixed)	$dn/2$	$\Theta(1)$	$\Theta(1)$
complete K_n	$n-1$	$\binom{n}{2}$	$\Theta(n)$	$\to 1/2$

Table 34.1: Expansion across familiar graphs. Edge expansion h is not degree-normalized, so the degree-fair comparison is the conductance $\phi = h/d \in [0, 1]$. Only the random regular graph keeps ϕ bounded away from zero *at fixed degree and linear edge count*—the expander sweet spot. The hypercube has constant h but only because its degree $\log n$ grows, and its conductance still decays.

34.5 Explicit constructions, conceptually

A probabilistic existence proof gives an object you cannot point to. For many uses—hardware interconnects, deterministic algorithms, codes you ship—you need an *explicit* family: given n, write down the graph (or even just answer "who are vertex v's neighbors?") in $\text{poly}(\log n)$ time, and *prove* its expansion once and for all. There are three main approaches.

Algebraic constructions. The earliest explicit expanders come from group theory and number theory. A clean example is the *Margulis* construction: take vertices $\mathbb{Z}_m \times \mathbb{Z}_m$ and connect each (x, y) to a fixed handful of affine images such as $(x \pm y, y), (x, y \pm x), (x \pm y + 1, y), \ldots$. The eigenvalue bound is proved using harmonic analysis on the group. The strongest algebraic examples are the *Lubotzky–Phillips–Sarnak* (LPS) graphs: Cayley graphs of PGL_2 over finite fields whose Ramanujan bound $\mu \le 2\sqrt{d-1}$ follows from deep number theory (Deligne's proof of the Ramanujan–Petersson conjecture). These are optimal but the machinery is heavy.

The zig-zag product. Reingold, Vadhan, and Wigderson (2002) gave a purely combinatorial, self-contained route. The *zig-zag product* $G \circledz H$ combines a big graph G (good expansion, large degree) with a small graph H (fixed size, good expansion) to produce a graph on $|V(G)| \cdot |V(H)|$ vertices whose degree is that of H (small!) and whose spectral gap degrades only mildly.

Intuition

Replace every vertex of G by a copy ("cloud") of H, one cloud-vertex per edge of G. A step in the product is *zig–zag–zig*: take a small step inside your cloud (using H), then traverse the G-edge you landed on to jump to another cloud, then a small step inside the new cloud. The two little H-steps "randomize" which G-edge you cross, so even though each vertex now has only $\deg(H)^2$ neighbors, the long-range mixing of G is inherited. You trade G's big degree for H's small degree while keeping expansion. Iterating (square G to boost the gap before each zig-zag) builds an infinite family from one small gadget H.

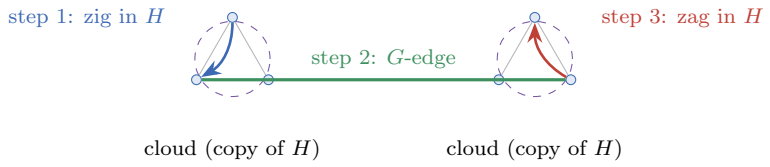


Figure 34.5: One step of the zig-zag product: a small H-step (zig), a G-edge crossing between clouds, then another H-step (zag). The two H-steps randomize the crossing edge, so a tiny degree suffices.

Lossless / bipartite expanders. For coding theory one wants *vertex* expansion close to the degree: every small left-set S has $|N(S)| \geq (1-\varepsilon)d|S|$ neighbors, nearly as many as the $d|S|$ edges leaving it. These "lossless" expanders (Capalbo–Reingold–Vadhan–Wigderson, 2002) are built from a randomness-conductor variant of zig-zag and are exactly what Section 34.6.4 needs. Spectral methods alone cannot reach A close to d (the Kahale barrier), so these need the combinatorial constructions.

Reading note

Explicit expanders evolved in stages: Margulis gave the first algebraic construction in 1973, Gabber and Galil made bounds explicit in 1981, and Lubotzky–Phillips–Sarnak reached spectral optimality in 1988. The zig-zag product (2002) introduced a purely combinatorial path, leading directly to Reingold's proof that $\mathsf{SL} = \mathsf{L}$ (undirected reachability in deterministic logspace). In 2013, Marcus, Spielman, and Srivastava proved the existence of bipartite Ramanujan graphs of every degree via interlacing polynomials.

34.6 Why expanders matter

We now turn to four applications, each using a different consequence of the spectral gap.

34.6.1 Fast mixing of random walks

Run a lazy random walk on a d-regular graph: from v, stay put with probability $1/2$, else step to a uniform neighbor. The transition matrix is $W = \frac{1}{2}I + \frac{1}{2}\frac{1}{d}\mathbf{A}$, with eigenvalues $w_i = \frac{1}{2} + \frac{\mu_i}{2d} \in [0,1]$. The stationary distribution is uniform, $\pi = \mathbf{1}/n$. How fast does the walk forget its start?

Theorem 34.5 (Mixing time of an expander)**.** *Let $w_\star = \max_{i \geq 2}|w_i|$ be the largest nontrivial walk eigenvalue in absolute value. Laziness keeps every $w_i = \frac{1}{2} + \frac{\mu_i}{2d} \in [0,1]$, so the maximum is attained at the most positive nontrivial adjacency eigenvalue μ_2:*

$$w_\star = \tfrac{1}{2} + \tfrac{\mu_2}{2d} \;\leq\; \tfrac{1}{2} + \tfrac{\mu}{2d} \;<\; 1.$$

For any start v and any t, the distribution p_t satisfies

$$\|p_t - \pi\|_1 \;\leq\; \sqrt{n} \cdot w_\star^t.$$

Hence after $t = O\big(\frac{1}{1-w_\star}\log n\big) = O\big(\frac{d}{d-\mu_2}\log n\big)$ steps the walk is within $1/n$ of uniform in total variation. (Replacing μ_2 by μ only loosens the bound, since $\mu_2 \leq \mu$.)

Why it works

Why. Write the start vector $e_v = \pi + \sum_{i \geq 2} c_i \psi_i$ in the eigenbasis $\{\psi_i\}$ of W. Applying W once scales the i-th component by w_i; the π-component (eigenvalue 1) is fixed. So

$$\|p_t - \pi\|_2^2 = \sum_{i \geq 2} c_i^2 w_i^{2t} \leq w_\star^{2t} \sum_{i \geq 2} c_i^2 \leq w_\star^{2t}.$$

Convert ℓ_2 to ℓ_1 with $\|\cdot\|_1 \leq \sqrt{n}\,\|\cdot\|_2$. The decay rate is governed entirely by the *spectral gap* $1 - w_\star = \frac{d - \mu_2}{2d}$: a constant gap (expander!) gives $O(\log n)$ mixing. □

Intuition

On a cycle the walk diffuses like Brownian motion, reaching the far side only after $\Theta(n^2)$ steps—the gap is $\Theta(1/n^2)$. On an expander every region leaks into every other immediately, so $\Theta(\log n)$ steps suffice. Fast mixing is the dynamical counterpart of expansion, and for sampling, $\log n$ versus n^2 steps is the difference between feasible and infeasible.

Application

Fast mixing is what makes *Markov chain Monte Carlo* work (Chapters 35 and 36). To sample from a complicated distribution—a random matching, a random coloring, a Gibbs state—you design a chain whose stationary distribution is the target and whose state graph is an expander; rapid mixing then means a short, provably good sample. The same gap bounds card-shuffling times and the convergence of PageRank.

34.6.2 Robust networks

Because every cut is large, expanders are hard to disconnect and hard to congest. Edge expansion $h \geq \varepsilon$ means you must delete at least ε edges *per vertex* to peel off any region: the edge-connectivity is high relative to the sparse degree, so random failures and even adversarial attacks leave a giant connected core. Expanders also make good *routing* networks: any permutation of n packets can be routed with low congestion and dilation $O(\log n)$, because the mixing property spreads load uniformly. This is why expander topologies appear in data-center and supercomputer interconnects and in peer-to-peer overlays, where you want maximal resilience and bandwidth from a bounded number of links per node.

34.6.3 Derandomization

Suppose a randomized algorithm decides correctly with probability $2/3$ using r random bits, and you want error 2^{-k}. Naively you repeat $O(k)$ times, spending $O(kr)$ fresh random bits. Random bits are an expensive resource in complexity theory; can we spend fewer?

Intuition

Build the *expander walk*. Make every r-bit string a vertex of a constant-degree expander on $N = 2^r$ vertices. Pick a start uniformly (r bits), then take a k-step random walk, each step

costing only $\log d = O(1)$ bits to choose a neighbor. Total randomness: $r + O(k)$ bits, not $O(kr)$. The walk visits k correlated-but-spread-out strings; use them as the k random seeds.

Theorem 34.6 (Ajtai–Komlós–Szemerédi / Gillman, expander Chernoff bound)**.** *Let $B \subseteq V$ be a "bad" set (seeds where the algorithm errs) with $|B|/n \le 1/3$, in a graph with $\mu/d = \beta$. For a random walk $v_1, \dots, v_k$,*

$$\Pr\Big[\#\{i : v_i \in B\} \ge \tfrac{1}{2}k\Big] \;\le\; e^{-\Omega((1-\beta)k)}.$$

So the fraction of walk steps landing in the bad set concentrates just as if the steps were independent, with a rate set by the spectral gap $1 - \beta$. Taking the majority vote of the k runs errs with probability $2^{-\Omega(k)}$—the same amplification as k independent trials, at a fraction of the randomness. This expander-walk sampler is a standard tool in derandomization and underlies the proof that error reduction costs essentially no extra random bits.

34.6.4 Error-correcting codes

Expanders give codes that encode, and especially *decode*, in linear time. Sipser and Spielman (1996) defined *expander codes* from a bipartite expander: n variable nodes on the left, m constraint nodes on the right, each constraint forcing the parity (or a small sub-code) of its neighborhood to be valid. A codeword is an assignment satisfying all constraints.

Why it works

Why expansion decodes. Use a lossless left expander: every small set S of corrupted variables has $|N(S)| \ge (1-\varepsilon)d|S|$ neighbors with $\varepsilon < 1/4$. Then most constraints adjacent to S touch S *only once*, so they are *unsatisfied* and visibly flag the error. The decoder is greedy: *find any variable that would fix more constraints than it breaks, and flip it.* Each flip strictly decreases the number of unsatisfied constraints, so the process terminates in linear time, and expansion guarantees that as long as the error set is small such a beneficial flip always exists until all errors are gone.

Theorem 34.7 (Sipser–Spielman, informal)**.** *A code from a $(\gamma n, (1-\varepsilon)d)$ lossless bipartite expander with $\varepsilon < 1/4$ has constant rate, corrects a constant fraction of adversarial errors, and admits a flip-decoder running in $O(n)$ time.*

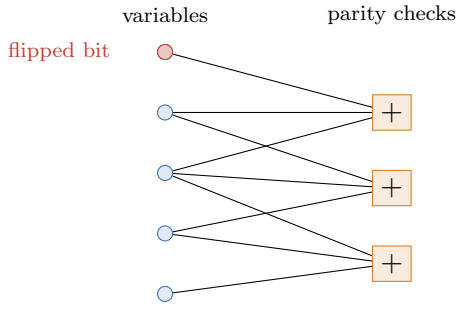


Figure 34.6: An expander (Tanner) code. Left vertices are bits, right vertices parity checks. A corrupted bit makes several checks fail; lossless expansion guarantees a greedy bit-flip always exists until decoding succeeds.

Connection

Each part of this chapter connects back to Chapter 31 and the companion *Linear Algebra* volume. Expansion = spectral gap = fast mixing = small effective resistances is one phenomenon viewed four ways. The Laplacian quadratic form (Chapter 31) gives Cheeger; the adjacency spectrum gives the Mixing Lemma; the walk matrix $W = \frac{1}{2}(I + \frac{1}{d}\mathbf{A})$ gives mixing time; and in the companion *Linear Algebra* volume the same gap appears as the condition number of the normalized Laplacian, which is precisely what makes expander *sparsifiers* and electrical-flow solvers fast. For expanders, the combinatorial, probabilistic, and spectral descriptions all express one property.

34.7 A fully worked example

Let us run the whole pipeline on one concrete graph: the *Petersen graph* P, which is 3-regular on 10 vertices and a standard small expander.

Intuition

P has an outer pentagon $0, 1, 2, 3, 4$, an inner pentagram $5, 6, 7, 8, 9$, and spokes i–$(i+5)$. It is vertex-transitive, has girth 5, and is a very good small expander. Its adjacency eigenvalues are 3 (once), 1 (five times), and -2 (four times). So $\mu = \max(|1|, |-2|) = 2$, the spectral gap is $d - \mu = 1$, and $\lambda_2 = d - \mu_2 = 3 - 1 = 2$.

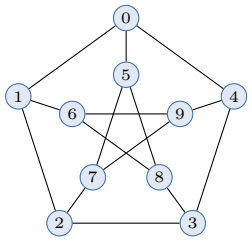


Figure 34.7: The Petersen graph: outer 5-cycle, inner pentagram, five spokes. A small $(10, 3, 2)$-expander, in fact Ramanujan since $\mu = 2 \le 2\sqrt{3-1} = 2\sqrt{2} \approx 2.83$.

Step 1: edge expansion by hand. By vertex-transitivity it suffices to test representative cuts. A single vertex has $|\partial S|/|S| = 3$. A spoke pair like $\{0, 5\}$: vertex 0 has neighbors $1, 4, 5$ and vertex 5 has neighbors $0, 7, 8$; the boundary is $\{1, 4, 7, 8\}$ via four edges (edge 0–5 is internal), ratio $4/2 = 2$. An outer edge $\{0, 1\}$: boundaries to $4, 5$ (from 0) and $2, 6$ (from 1), four edges, ratio 2. The sparsest balanced cuts of P achieve $h(P) = 1$: a 5/5 split can leave only 5 edges crossing, giving ratio $5/5 = 1$, and one checks the few balanced cuts that none drops below that.

Step 2: check Cheeger. Theorem 34.1 predicts

$$\frac{\lambda_2}{2} = \frac{2}{2} = 1 \;\le\; h(P) = 1 \;\le\; \sqrt{2d\lambda_2} = \sqrt{2 \cdot 3 \cdot 2} = \sqrt{12} \approx 3.46.$$

Both inequalities hold, and the *lower* bound is tight here: $h(P) = \lambda_2/2$ exactly. The upper Cheeger bound is loose by a factor ≈ 3.46, the usual quadratic slack.

Step 3: Corollary 34.1. The spectral cut bound gives $h \geq (d-\mu)/2 = (3-2)/2 = 1/2$, weaker than 1 but in the right ballpark. Here $\mu = 2$ is large relative to $d = 3$, so the raw mixing-lemma count is slack for small sets; the clean $h \geq (d-\mu)/2$ comes through the easy Cheeger step $d - \mu_2 = \lambda_2 \leq 2h$ (with $\mu_2 = 1 \leq \mu$). Using μ_2 instead recovers the tight value $h \geq (d-\mu_2)/2 = 1$.

Step 4: mixing. The walk eigenvalues are $w_i = \frac{1}{2} + \frac{\mu_i}{6}$, so $\mu_i = 1$ gives $\frac{2}{3}$ and $\mu_i = -2$ gives $\frac{1}{6}$. The largest nontrivial one is $w_\star = \frac{1}{2} + \frac{\mu_2}{2d} = \frac{1}{2} + \frac{1}{6} = \frac{2}{3}$ (the negative eigenvalue is tamed by laziness into the small $\frac{1}{6}$, not a near-1 value). The gap is $1 - w_\star = 1/3$, so from any start the walk is within $\Theta(1/n)$ of uniform after $O\big(\frac{d}{d-\mu_2} \log 10\big) = O(3 \log 10) \approx O(7)$ steps—a tiny graph mixes almost instantly, as expansion predicts.

Trace it yourself

Reproduce Step 4 numerically. Start $p_0 = e_0$ (all mass on vertex 0). One lazy step: p_1 has $1/2$ on vertex 0 and $1/6$ on each of $1, 4, 5$. Iterate $p_{t+1} = W p_t$ a dozen times (a 10×10 matrix multiply); watch $\|p_t - \frac{1}{10}\mathbf{1}\|_1$ fall by a factor near $2/3$ each step. Then deliberately break expansion: delete the five spokes, splitting P into two disjoint 5-cycles. Now $\mu = d$, the gap vanishes, and the walk never mixes across the (now nonexistent) cut. The experiment shows mixing failing as soon as the spectral gap closes.

Step 5: watch the error fall. By vertex-transitivity the only thing that distinguishes coordinates of p_t is the distance from the start, so we can collapse the 10-vector into three numbers: the mass on vertex 0 (distance 0), the mass on each of its three neighbors (distance 1), and the mass on each of the six remaining vertices (distance 2). The lazy step $p_{t+1} = W p_t$ becomes a 3×3 recurrence, and the total-variation distance to uniform $\frac{1}{2}\|p_t - \frac{1}{10}\mathbf{1}\|_1$ is now trivial to track by hand.

t	mass at dist. 0	each at dist. 1	each at dist. 2	$\frac{1}{2}\|p_t - \pi\|_1$
0	1.000	0.000	0.000	0.900
1	0.500	0.167	0.000	0.600
2	0.333	0.167	0.028	0.433
3	0.250	0.148	0.051	0.294
5	0.166	0.122	0.078	0.132
8	0.120	0.107	0.093	0.039
12	0.104	0.101	0.099	0.008

Table 34.2: Lazy walk on the Petersen graph from vertex 0, collapsed by distance. The total-variation distance to uniform contracts by roughly the predicted factor $w_\star = 2/3$ per step once the transient settles, dropping below $1/n = 0.1$ just after step 5 (it is 0.088 at step 6) and reaching 0.008 by step 12.

Read the last column down: after the first couple of steps each entry is about $2/3$ of the one above it, exactly the eigenvalue $w_\star$ predicted in Step 4. The distance crosses the $1/n$ threshold just after $t = 5$, comfortably inside the budget: the bound $\sqrt{n}\, w_\star^t \leq 1/n$ asks for only about 9 steps. Nothing in this table required diagonalizing a matrix; the spectral gap told us the contraction rate in advance, and the arithmetic merely confirmed it.

34.8 Historical Context and Controversies

History

The story of expander graphs began in telecommunications. In 1973, Mark Pinsker at the Institute for Information Transmission Problems in Moscow was studying telephone switching networks (routing systems that connect telephone callers without colliding). A telephone exchange needed to connect thousands of callers simultaneously without blocking, using as few internal switches and wires as possible. Pinsker realized that the ideal switching network was a sparse bipartite graph where every subset of inputs fanned out to a strictly larger set of outputs. Using the probabilistic method (proving an object exists by showing that a random choice succeeds with positive probability), Pinsker proved that a randomly chosen sparse graph is almost certainly an excellent expander.

However, Pinsker's proof was non-constructive: it showed that almost all sparse graphs are expanders, but gave no recipe to build even a single one deterministically. Wiring a physical exchange required an explicit blueprint. That same year, Gregory Margulis (1973) published the first explicit construction of constant-degree expanders. Because purely combinatorial techniques could not certify expansion, Margulis turned to representation theory, using Kazhdan's property (T) (a rigid representation-theoretic property of groups where the trivial representation is isolated) to prove a positive spectral gap. In 1988, Alexander Lubotzky, Ralph Phillips, and Peter Sarnak (LPS), and independently Margulis, constructed explicit *Ramanujan graphs* (optimal expanders matching the Alon–Boppana optimal spectral bound $2\sqrt{d-1}$) using Pierre Deligne's proof of the Ramanujan conjecture in automorphic forms.

For decades, the search for explicit expanders posed a classic paradox: a haystack made almost entirely of needles, yet hard to exhibit a single needle deterministically without heavy algebraic machinery. While dense graphs like K_n expand maximally, they overwhelm wiring and memory budgets with $O(n^2)$ edges; the engineering goal was to achieve constant degree $d = O(1)$ alongside linear boundary expansion. In 2002, Omer Reingold, Salil Vadhan, and Avi Wigderson developed the *zig-zag product* (an operation that replaces each vertex of a large graph with an entire small graph to amplify expansion without inflating degree), enabling the construction of constant-degree expanders via elementary graph-theoretic operations and leading directly to Reingold's 2005 deterministic logspace connectivity algorithm ($\mathbf{L} = \mathbf{SL}$). An open problem is to construct explicit non-bipartite Ramanujan graphs for every degree and size using purely combinatorial methods, bypassing heavy algebraic number theory.

34.8.1 What We Can Now Do and Its Limitations

Focus

Capabilities Unlocked:

- **Derandomization and Randomness Reduction**: Expanders allow randomized algorithms to be run with drastically fewer random bits. Rather than drawing k independent random seeds of length $\log n$ (costing $k \log n$ bits), a random walk on an expander generates k pseudorandom samples using only $\log n + O(k)$ total bits while achieving nearly identical error decay.

- **Linear-Time Error-Correcting Codes**: Sipser and Spielman (1996) used bipartite expanders to construct error-correcting codes that can be encoded and decoded in strictly linear time.
- **Fast Information Diffusion**: Gossip protocols and routing networks built on expanders guarantee that information reaches every node in $O(\log n)$ rounds with no single point of failure.

Honest Limitations and Downsides:

- **Rigid Algebraic Sizes**: Optimal Ramanujan constructions exist only for specific vertex counts and degrees related to primes (e.g., $d = p + 1$ for prime $p \equiv 1 \pmod 4$), making them cumbersome for arbitrary system sizes.
- **Absence of Efficient Routing**: An expander guarantees that short paths exist between every pair of vertices, but it does *not* provide a localized routing algorithm for packets to find those short paths without global topology tables.

Where We Go Next in the Book:

- In **Chapter 35 (Random Walks and Electrical Networks)**, we connect the expander spectral gap directly to electrical currents, effective resistances, and Ohm's law.
- In **Chapter 36 (PageRank and Markov Chains)**, we use the spectral gap of the Google matrix to prove exponential convergence of power iteration on web graphs.
- In **Chapter 41 (Laplacian Solvers and Electrical Flows)**, we use expander decompositions to build near-linear time Laplacian system solvers.

34.9 Exercises

Exercise 34.1 (Warm-up cuts)**.** For the 4-dimensional hypercube Q_4 (16 vertices, 4-regular), compute $|\partial S|/|S|$ for $S = \{\text{strings with first bit } 0\}$, for $S =$ a single vertex, and for $S =$ a Hamming ball of radius 1. Conjecture $h(Q_4)$ and argue the half-cube cut is optimal.

Exercise 34.2 (The easy direction, your turn)**.** Prove $\lambda_2 \le 2h(G)$ for non-regular graphs using the *normalized* Laplacian $\mathcal{L} = I - \mathbf{A}_D^{-1/2}\mathbf{A}\,\mathbf{A}_D^{-1/2}$ and conductance ϕ. Identify exactly where regularity was used in the proof of the easy direction of Theorem 34.1 and adapt it.

Exercise 34.3 (Mixing Lemma is tight)**.** Show the Petersen graph nearly saturates Theorem 34.2: find sets S, T where $|e(S,T) - \frac{d}{n}|S||T||$ is close to $\mu\sqrt{|S||T|}$. (Hint: use eigenvectors of $\mu_2 = 1$ or $\mu_n = -2$ to choose S, T.)

Exercise 34.4 (Bipartite poison)**.** Prove that a connected d-regular graph is bipartite iff $\mu_n = -d$. Conclude that bipartite graphs are never two-sided expanders, and explain why the lazy walk's $\frac{1}{2}I$ term is exactly the fix.

Exercise 34.5 (Implement Cheeger rounding)**.** Implement Algorithm 131. Test it on (a) a cycle C_{1000}, (b) a $\sqrt{n} \times \sqrt{n}$ grid, (c) a random 3-regular graph. Report the returned ratio versus $\lambda_2/2$ and $\sqrt{2d\lambda_2}$, and confirm the quadratic gap is visible on the cycle.

Exercise 34.6 (Alon–Boppana, baby case)**.** For $d = 3$, verify $2\sqrt{d-1} = 2\sqrt{2} \approx 2.83$ and check that the Petersen graph's $\mu = 2 < 2.83$ does not contradict the bound. Explain why a single small graph

can beat $2\sqrt{d-1}$ while no infinite family can. (Hint: Alon–Boppana is asymptotic; the $o(1)$ slack matters at small n.)

Exercise 34.7 (Expander Chernoff in code)**.** Empirically test Theorem 34.6. On a random 4-regular graph mark a random 1/3 of vertices "bad"; run 10^4 length-k walks for $k = 10, 20, 40$; plot the empirical tail of (fraction of steps in the bad set) against the independent-sampling Chernoff bound. Confirm the expander walk concentrates almost as tightly.

Exercise 34.8 (Break a code)**.** Build a tiny expander (Tanner) code from the bipartite graph of Figure 34.6 extended to 7 variables and 3 checks. Run the flip-decoder on a 1-bit error and on a 2-bit error. Construct an error pattern where greedy flipping gets stuck, and relate the failure to a set S that *fails* the lossless expansion condition $|N(S)| \geq (1-\varepsilon)d|S|$.

Exercise 34.9 (Zig-zag bookkeeping)**.** For the zig-zag product of G (D-regular on N vertices) with H (d-regular on D vertices), prove the product has ND vertices and is d^2-regular. Explain in one paragraph why squaring G before each zig-zag step is needed to keep the spectral gap from decaying to zero over the iteration.

Exercise 34.10 (Collapse the walk)**.** Justify the distance-collapse used in Table 34.2: because the Petersen graph is distance-transitive, the lazy walk from a fixed start keeps equal mass on all vertices at the same distance, so the 10-state chain reduces to a 3-state chain on distances $\{0, 1, 2\}$. Write down that 3×3 transition matrix explicitly, find its eigenvalues, and check that its nontrivial ones are $\{\frac{1}{2} + \frac{1}{6}, \frac{1}{2} - \frac{2}{6}\} = \{2/3, 1/6\}$, matching $w_i = \frac{1}{2} + \mu_i/(2d)$ for $\mu_i \in \{1, -2\}$. Why does the start-symmetry guarantee the collapse is exact rather than approximate?

Exercise 34.11 (Resistance and expansion)**.** Using the electrical-network view (Chapter 35), argue that in a d-regular expander the effective resistance between *any* two vertices is $\Theta(1/d)$ up to lower-order terms, in sharp contrast to the cycle where it grows linearly. Connect this to commute times via the identity $\text{commute}(u, v) = 2|E| \cdot R_{\text{eff}}(u, v)$.

Chapter 35

Random Walks and Electrical Networks

A token sits on a vertex. At each tick it picks a uniformly random edge out of where it stands and slides along it. That single rule defines a simple random walk on a graph, yet its behavior governs a wide range of structural quantities: how long until the token reaches a target vertex, how long until it visits every vertex, how much flow the graph can route between two endpoints, and where its bottlenecks lie. A one-line stochastic process connects all of these because it can be analyzed from three equivalent perspectives: as a finite Markov chain, as iterated multiplication by a normalized adjacency operator, and — the central theme of this chapter — as electric current flowing through a network of resistors.

The dictionary is exact: voltages are harmonic functions, and harmonic functions are hitting probabilities. Effective resistance between two vertices is, up to a factor of $2m$, the *commute time* of the walk. The energy a unit current dissipates is the reciprocal of conductance, and conductance lower-bounds how fast the walk mixes. Once you have the electrical picture, theorems that look like they need delicate probabilistic arguments fall out of Ohm's law and the fact that current minimizes energy. This chapter builds that dictionary from scratch, proves the central identities (commute time $= 2m\,R_{\text{eff}}$, Foster's theorem, the hitting-time formula via the Laplacian pseudoinverse), and traces every one of them by hand on a single small network.

Intuition

Replace each edge of the graph by a 1-ohm resistor and wire a 1-volt battery between two vertices s and t. Current flows through the network, splitting at each junction. Two facts make this physical picture useful for computation:

- The *voltage* at a vertex is the probability that a random walker started there reaches s before t (after rescaling). Voltages are *harmonic*: each interior vertex's voltage is the average of its neighbors', which is precisely the one-step equation of the walk.
- The *effective resistance* $R_{\text{eff}}(s,t)$ — the voltage you must apply to push one unit of current — equals $\frac{1}{2m}$ times the expected time for the walk to go from s to t *and back*.

So "how well-connected are s and t?" gets the exact same quantitative answer from physics (low effective resistance) and from probability (short commute time).

35.1 The simple random walk and its transition matrix

Fix a connected undirected graph $G = (V, E)$ with $n = |V|$ vertices and $m = |E|$ edges. For most of the chapter we take edges unweighted; the weighted case (conductances $w_{ij} > 0$) is a drop-in generalization that we flag as we go. Write $\deg(i)$ for the degree of vertex i, $\mathbf{A}$ for the adjacency matrix, $\mathbf{A}_D = \operatorname{diag}(\deg(1), \dots, \deg(n))$ for the degree matrix, and $\mathbf{L} = \mathbf{A}_D - \mathbf{A}$ for the graph Laplacian introduced in Chapter 31.

Definition 35.1 (Simple random walk)**.** The *simple random walk* on a graph G is the Markov chain $(X_0, X_1, X_2, \dots)$ on the vertex set V that, from vertex i, moves to a uniformly chosen neighbor:

$$\Pr[X_{t+1} = j \mid X_t = i] \;=\; \begin{cases} \dfrac{1}{\deg(i)} & \{i, j\} \in E, \\ 0 & \text{otherwise.} \end{cases}$$

The *transition matrix* is $\mathbf{A}_P \in \mathbb{R}^{n \times n}$ with $(\mathbf{A}_P)_{ij} = \Pr[X_{t+1} = j \mid X_t = i]$. Compactly, $\mathbf{A}_P = \mathbf{A}_D^{-1}\mathbf{A}$.

Each row of $\mathbf{A}_P$ sums to 1 (the walker must go somewhere): $\mathbf{A}_P$ is *row-stochastic*. If $p_t \in \mathbb{R}^n$ is the row vector of probabilities $\Pr[X_t = i]$, then one step is a vector–matrix product,

$$p_{t+1} \;=\; p_t\, \mathbf{A}_P, \qquad \text{so} \qquad p_t \;=\; p_0\, \mathbf{A}_P^{\,t}.$$

The whole evolution of the walk is described by the powers of one matrix. This is why so much about random walks reduces to the spectrum of $\mathbf{A}_P$, and why the Laplacian — so close to $\mathbf{A}_P$ — is never far away.

Example 35.1 (Our running network)**.** Throughout the chapter we trace a five-vertex graph: a triangle $1{-}2{-}3$ with a path $3{-}4{-}5$ hanging off it.

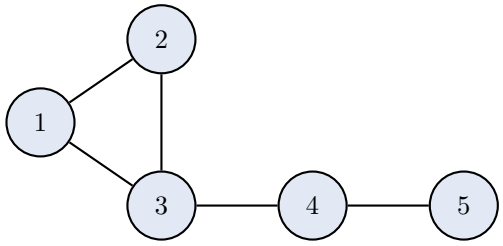


Figure 35.1: Running graph G: a triangle $\{1, 2, 3\}$ with a tail $3{-}4{-}5$. Degrees: $\deg(1) = \deg(2) = 2$, $\deg(3) = 3$, $\deg(4) = 2$, $\deg(5) = 1$. Edges $m = 5$.

Reading the edges $\{1,2\}, \{1,3\}, \{2,3\}, \{3,4\}, \{4,5\}$:

$$\mathbf{A} = \begin{pmatrix} 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 \end{pmatrix}, \quad \mathbf{A}_D = \operatorname{diag}(2, 2, 3, 2, 1), \quad \mathbf{A}_P = \begin{pmatrix} 0 & \frac{1}{2} & \frac{1}{2} & 0 & 0 \\ \frac{1}{2} & 0 & \frac{1}{2} & 0 & 0 \\ \frac{1}{3} & \frac{1}{3} & 0 & \frac{1}{3} & 0 \\ 0 & 0 & \frac{1}{2} & 0 & \frac{1}{2} \\ 0 & 0 & 0 & 1 & 0 \end{pmatrix}.$$

Vertex 5 is a dead end: whenever the walk reaches it, the next step is forced back to 4. Keep $\mathbf{A}_P$ in view; we will find its stationary distribution, then its hitting and commute times, all by hand.

35.1.1 The stationary distribution

Definition 35.2 (Stationary distribution)**.** A probability row vector π is *stationary* for the walk if $\pi \mathbf{A}_P = \pi$. It is a fixed point: start the walk distributed as π and it stays distributed as π forever.

For a simple random walk on an irregular graph, the stationary probability of a vertex is proportional to its degree rather than uniform. Hubs get visited more because more edges point at them.

Theorem 35.1 (Degree-proportional stationarity)**.** *On a connected graph the vector*

$$\pi_i \;=\; \frac{\deg(i)}{2m}$$

is stationary for the simple random walk, and it is the unique stationary distribution.

Proof. Compute the j-th entry of $\pi\mathbf{A}_P$:

$$(\pi\mathbf{A}_P)_j \;=\; \sum_i \pi_i (\mathbf{A}_P)_{ij} \;=\; \sum_{i:\{i,j\}\in E} \frac{\deg(i)}{2m}\cdot\frac{1}{\deg(i)} \;=\; \sum_{i:\{i,j\}\in E} \frac{1}{2m} \;=\; \frac{\deg(j)}{2m} \;=\; \pi_j.$$

The two $\deg(i)$ factors cancel, leaving one $\frac{1}{2m}$ per neighbor of j, of which there are $\deg(j)$. So $\pi\mathbf{A}_P = \pi$.

Intuition

Why is stationarity unique? A Markov chain is *irreducible* if every state can eventually reach every other state — which for an undirected graph simply means the graph is connected. By the Perron–Frobenius theorem, any irreducible stochastic matrix has a unique stationary probability distribution $\pi > 0$ with eigenvalue 1. The simple random walk also satisfies the *detailed balance* condition:

$$\pi_i(\mathbf{A}_P)_{ij} \;=\; \frac{\deg(i)}{2m}\cdot\frac{1}{\deg(i)} \;=\; \frac{1}{2m} \;=\; \frac{\deg(j)}{2m}\cdot\frac{1}{\deg(j)} \;=\; \pi_j(\mathbf{A}_P)_{ji}$$

for every edge $\{i,j\}\in E$. Detailed balance states that at equilibrium, the probability flux flowing from i to j exactly equals the reverse flux from j to i. This microscopic time-reversibility is the mathematical reason why reversible random walks correspond directly to electrical networks: detailed balance is the probabilistic counterpart of symmetric conductance ($c_{ij} = c_{ji}$) in Ohm's law.

□

Example 35.2 (Stationary distribution of G)**.** With $2m = 10$, the stationary distribution of our running graph is

$$\pi = \left(\tfrac{2}{10}, \tfrac{2}{10}, \tfrac{3}{10}, \tfrac{2}{10}, \tfrac{1}{10}\right) = (0.2,\, 0.2,\, 0.3,\, 0.2,\, 0.1).$$

The degree-3 hub vertex 3 is visited most; the leaf vertex 5 least. Sanity check one coordinate: $(\pi\mathbf{A}_P)_5 = \pi_4 \cdot (\mathbf{A}_P)_{45} = 0.2 \cdot \frac{1}{2} = 0.1 = \pi_5$. Good.

Common trap

Two failure modes break "$\pi_i = \deg(i)/2m$ and the walk converges to it." *Disconnected graph*: stationarity is no longer unique — each component carries its own π. *Bipartite graph*: the chain is periodic (period 2); π is still stationary, but p_t oscillates between the two sides and never settles. A single odd cycle, like the triangle in our running graph, kills bipartiteness and restores convergence. The *lazy* walk $\mathbf{A}'_P = \frac{1}{2}(\mathbf{A}_I + \mathbf{A}_P)$ — flip a coin, stay put on heads — is aperiodic on *any* graph and has the same π; it is the standard fix.

35.2 Hitting times and cover times

Definition 35.3 (Hitting and commute times)**.** For vertices a, b, let $T_b = \min\{t \geq 0 : X_t = b\}$ be the first time the walk reaches b. The *hitting time* is the expected first arrival starting from a,

$$H(a,b) \;=\; \mathbb{E}\big[T_b \mid X_0 = a\big].$$

The *commute time* is the round trip, $C(a,b) = H(a,b)+H(b,a)$. The *cover time* from a is $\mathbb{E}[\max_v T_v \mid X_0 = a]$, the expected time to visit *every* vertex; the cover time of G is the maximum over start vertices.

Hitting time is generally *not* symmetric: $H(a,b) \neq H(b,a)$. In our running graph, getting from the leaf 5 into the triangle is easy, but wandering out to the leaf 5 from the triangle is slow, because the walk keeps getting pulled back into the dense core. The commute time, being a round trip, is symmetric by construction — and that symmetry is the first hint of the resistance connection.

35.2.1 Hitting times by first-step analysis

The basic tool is *first-step analysis*: condition on the first move. From $a \neq b$ the walk takes one step (cost 1) to a uniform neighbor, then must still reach b:

$$H(a,b) \;=\; 1 + \frac{1}{\deg(a)} \sum_{c:\{a,c\}\in E} H(c,b), \qquad H(b,b) = 0. \tag{35.1}$$

Fixing the target b, this is a linear system in the $n-1$ unknowns $\{H(a,b) : a \neq b\}$. Writing h for the vector of hitting times to b and deleting the row/column for b, (35.1) reads $(\mathbf{A}_I - \mathbf{A}_P)\,h = \mathbf{1}$ on the non-b coordinates, a Laplacian-type system we solve below in closed form.

Example 35.3 (Hitting times on the tail of G)**.** Let us compute hitting times to vertex 5 by hand. By symmetry of the triangle, vertices 1 and 2 are interchangeable with respect to the target 5, so $H(1,5) = H(2,5)$. Write $h_a = H(a,5)$. The equations (35.1) are:

$$\begin{aligned} h_1 &= 1 + \tfrac{1}{2}h_2 + \tfrac{1}{2}h_3, \\ h_2 &= 1 + \tfrac{1}{2}h_1 + \tfrac{1}{2}h_3, \\ h_3 &= 1 + \tfrac{1}{3}h_1 + \tfrac{1}{3}h_2 + \tfrac{1}{3}h_4, \\ h_4 &= 1 + \tfrac{1}{2}h_3 + \tfrac{1}{2}\cdot 0. \end{aligned}$$

Using $h_1 = h_2$, the first equation gives $h_1 = 1 + \frac{1}{2}h_1 + \frac{1}{2}h_3$, so $h_1 = 2 + h_3$. Substituting into the third, $h_3 = 1 + \frac{2}{3}h_1 + \frac{1}{3}h_4 = 1 + \frac{2}{3}(2 + h_3) + \frac{1}{3}h_4$, i.e. $\frac{1}{3}h_3 = \frac{7}{3} + \frac{1}{3}h_4$, so $h_3 = 7 + h_4$. The fourth

gives $h_4 = 1 + \frac{1}{2}h_3$. Combine: $h_4 = 1 + \frac{1}{2}(7 + h_4) = \frac{9}{2} + \frac{1}{2}h_4$, hence $h_4 = 9$, then $h_3 = 16$, and $h_1 = h_2 = 18$.

So $H(1,5) = H(2,5) = 18$, $H(3,5) = 16$, $H(4,5) = 9$. The walk needs 18 steps on average to crawl out to the leaf from a far triangle vertex (Figure 35.2). We will recompute $H(4,5) = 9$ instantly from resistance in Example 35.5.

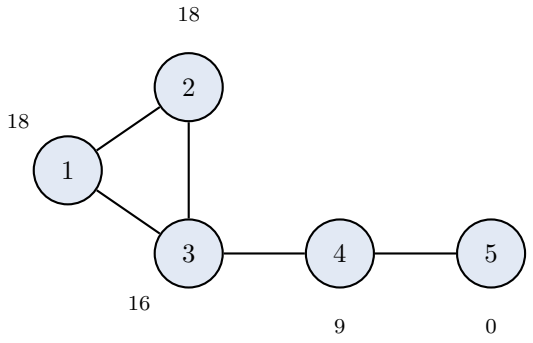


Figure 35.2: Hitting times $H(a,5)$ to the leaf, written beside each vertex. They grow monotonically with distance from 5 and are equal at the symmetric pair $\{1,2\}$. The target 5 has $H(5,5) = 0$.

Trace it yourself

Compute the hitting times *to* the hub vertex 3 on the running graph, $H(a,3)$ for $a \in \{1,2,4,5\}$, by first-step analysis. You should find $H(1,3) = H(2,3)$ small and $H(5,3)$ the largest. Then check that $H(3,5) + H(5,3)$ matches the commute time $C(3,5)$ we get from resistance in Section 35.4. Doing the two computations independently and watching them agree is the fastest way to believe the commute-resistance theorem.

35.2.2 Bounds on hitting and cover times

How large can these times be? For hitting time, a clean universal bound comes straight from commute time and resistance (Section 35.4); here we state the result.

Theorem 35.2 (Polynomial hitting time)**.** *On any connected graph with n vertices and m edges, $H(a,b) \le 2m \cdot (n-1) < n^3$ for all a, b, and there exist graphs (the "lollipop") where $H(a,b) = \Theta(n^3)$.*

Proof sketch. We show in Theorem 35.7 that $C(a,b) = 2m\, R_{\text{eff}}(a,b)$, and effective resistance is at most the graph distance $\le n-1$ (resistors in series). So $H(a,b) \le C(a,b) \le 2m(n-1) \le n(n-1)^2 < n^3$. The lollipop graph — a clique on $n/2$ vertices joined by a path of length $n/2$ — forces the walk to escape the sticky clique and then random-walk down the path, costing $\Theta(n^2)$ to leave times $\Theta(n)$ path length, i.e. $\Theta(n^3)$; this is the worst case (Figure 35.3). □

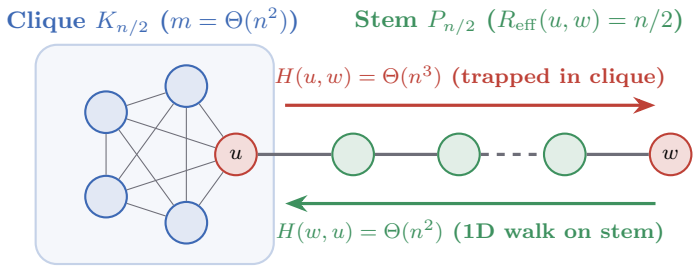


Figure 35.3: The lollipop graph achieves the worst-case $\Theta(n^3)$ hitting time and illustrates strong directional asymmetry: walking from the stem tip w into the portal u takes $\Theta(n^2)$ steps, whereas escaping the dense clique from u out to w takes $\Theta(n^3) = 2m\,R_{\text{eff}}(u,w)$ steps.

The cover time inherits a logarithmic overhead on top of the worst hitting time.

Theorem 35.3 (Matthews bound)**.** *Let $H_{\max} = \max_{a,b} H(a,b)$. The cover time satisfies*

$$\operatorname{cov}(G) \;\le\; H_{\max} \cdot \left(1 + \tfrac{1}{2} + \cdots + \tfrac{1}{n}\right) \;=\; H_{\max}\, O(\log n).$$

Idea. Order the vertices in a uniformly random order $v_1, \dots, v_n$ and let L_k be the time at which the k-th *distinct* "record" vertex (the last among $v_1, \dots, v_k$ to be hit) is covered. A coupon-collector argument bounds $\mathbb{E}[\text{cover time}]$ by $\sum_k \frac{1}{k} H_{\max}$, because the chance that v_k is the newest record is $1/k$. Summing the harmonic series gives the $O(\log n)$ factor. The matching lower bound (cover time $\ge H_{\min} \cdot H_n$ with $H_{\min}$ the min over a clever set) is the other half of Matthews' theorem. □

Cost summary

For an n-vertex, m-edge connected graph: hitting time $H(a,b) = O(mn) = O(n^3)$, tight for the lollipop. Cover time $\operatorname{cov}(G) = O(mn \log n)$; it is $\Theta(n \log n)$ for the complete graph and $\Theta(n^2)$ for the path/cycle. Computing one hitting time exactly costs one Laplacian solve, $O(n^3)$ by Gaussian elimination or $\tilde{O}(m)$ by the near-linear Laplacian solvers of Chapter 41.

35.3 The electrical-network analogy

We now change viewpoint and turn the graph into a resistor network: each edge $\{i,j\}$ becomes a resistor of resistance r_{ij}, equivalently a *conductance* $c_{ij} = 1/r_{ij}$. For the simple random walk all conductances are 1 (unit resistors); a weighted walk with edge weights w_{ij} corresponds to conductances $c_{ij} = w_{ij}$. The vertex conductance is $c_i = \sum_j c_{ij} = \deg(i)$ in the unit case.

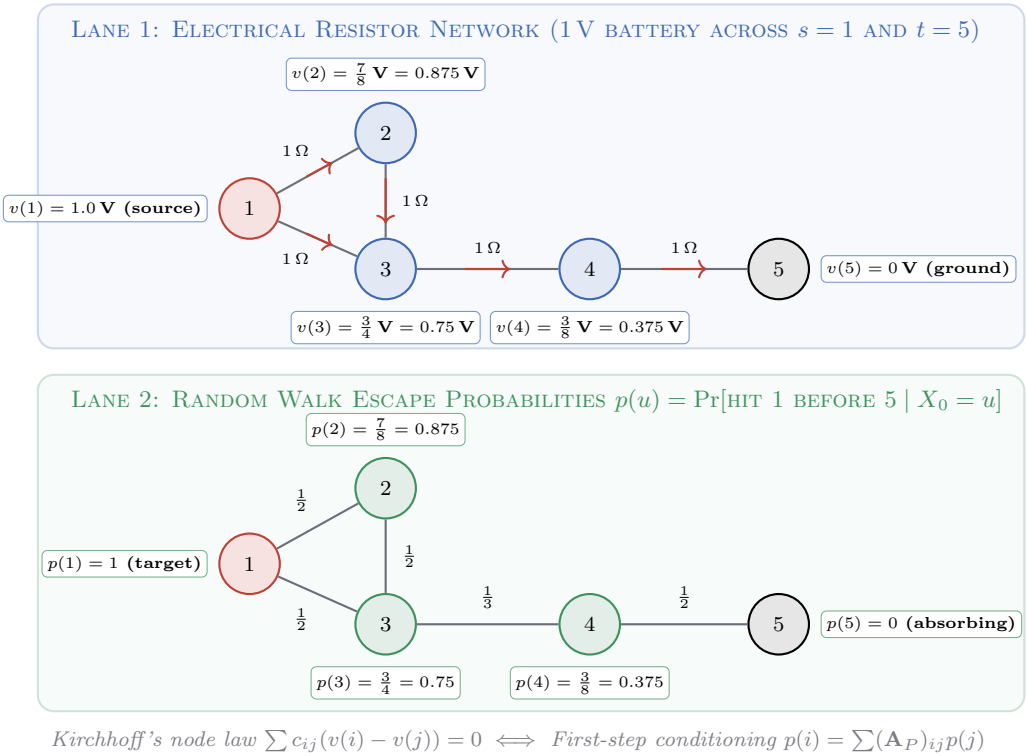


Figure 35.4: The electrical network analogy on our running graph G. Top: A 1 V battery is connected across $s = 1$ and $t = 5$ (grounded). The interior node voltages $v(u)$ settle to the unique harmonic values obeying Ohm's and Kirchhoff's laws. Bottom: The identical network viewed as a random walk. The probability that a walker started at u reaches target 1 before absorbing state 5 satisfies the exact same harmonic system, proving $p(u) \equiv v(u)$.

35.3.1 Voltages are harmonic functions

Attach a voltage $v(i)$ to each vertex. *Ohm's law* says the current along edge $\{i, j\}$ from i to j is $i_{ij} = c_{ij}(v(i) - v(j))$. *Kirchhoff's current law* says that at every vertex where no external current is injected, the net current is zero:

$$\sum_{j:\{i,j\}\in E} c_{ij}\big(v(i) - v(j)\big) = 0 \quad \Longleftrightarrow \quad v(i) = \frac{\sum_j c_{ij}\, v(j)}{\sum_j c_{ij}} = \sum_j (\mathbf{A}_P)_{ij}\, v(j).$$

The right-hand equation is the definition of a *harmonic* function: the value at i is the (conductance-weighted) average of its neighbors. And $\sum_j (\mathbf{A}_P)_{ij} v(j)$ is exactly one step of the random walk applied to v. So:

Why it works

A function is harmonic at vertex $i \iff$ Kirchhoff's law holds at $i \iff v(i) = (\mathbf{A}_P v)(i)$, i.e. v is unchanged by one step of the walk's averaging. The Laplacian packages all three: v is harmonic at i exactly when $(\mathbf{L}v)(i) = 0$, since $(\mathbf{L}v)(i) = \sum_j c_{ij}(v(i) - v(j))$ is the net current out of i. Harmonicity, zero net current, and invariance under one walk step are the same condition stated in three ways.

Now *inject* current: push a current of f in at vertex s and pull f out at t. Then $\mathbf{L}v = f(\mathbf{1}_s - \mathbf{1}_t)$, a Laplacian system whose right-hand side is the net injected current. Because $\mathbf{L}$ has the all-ones vector in its kernel, voltages are determined only up to an additive constant (ground one vertex), exactly as in physics.

Theorem 35.4 (Voltages solve the Dirichlet problem)**.** *Fix disjoint boundary sets with prescribed voltages (e.g. $v(s) = 1$, $v(t) = 0$). There is a unique function v that takes those boundary values and is harmonic at every interior vertex. Equivalently, the voltage $v(a)$ equals the probability that a random walk started at a reaches s before t.*

Proof. Existence/uniqueness. Harmonicity at interior vertices plus fixed boundary values is the linear system $(\mathbf{L}v)(a) = 0$ for interior a, with $v(s) = 1, v(t) = 0$. The interior block of $\mathbf{L}$ (rows/cols of interior vertices) is a symmetric *diagonally dominant* matrix that is strictly diagonally dominant in any row touching the boundary; on a connected graph it is invertible, so the system has a unique solution.

Probabilistic identity. Let $p(a) = \Pr[\text{reach } s \text{ before } t \mid X_0 = a]$. Clearly $p(s) = 1$, $p(t) = 0$. For interior a, first-step analysis gives $p(a) = \sum_j (\mathbf{A}_P)_{aj} p(j)$ — the walk steps to a random neighbor and continues — so p is harmonic with the same boundary values as v. By uniqueness, $p = v$. The voltage you read off a resistor network *is* a hitting probability. □

This is an instance of the maximum principle: a harmonic function attains its extremes on the boundary (no interior vertex can exceed all its neighbors, being their average), so the walk-as-voltage is squeezed between 0 and 1, as a probability must be.

35.3.2 Effective resistance and energy

Definition 35.4 (Effective resistance)**.** The *effective resistance* $R_{\text{eff}}(s,t)$ between s and t is the voltage drop $v(s) - v(t)$ needed to drive a *unit* current ($f = 1$) from s to t. Equivalently, $R_{\text{eff}}(s,t) = v(s) - v(t)$ where $\mathbf{L}v = \mathbf{1}_s - \mathbf{1}_t$.

Effective resistance replaces the whole network by a single equivalent resistor between s and t. Two familiar rules compute it for simple networks: resistors in *series* add ($R = r_1 + r_2$), resistors in *parallel* add reciprocally ($\frac{1}{R} = \frac{1}{r_1} + \frac{1}{r_2}$). More edges between two points can only lower resistance — which matches the intuition that more routes make s and t better connected.

There is a clean variational characterization. Recall the quadratic form $f^\top \mathbf{L} f = \sum_{\{i,j\}} c_{ij}(f_i - f_j)^2$ from Chapter 31; physically it is the power dissipated. Effective resistance is the minimum energy to route a unit flow:

Theorem 35.5 (Thomson's principle)**.** *Among all unit s–t flows (functions on oriented edges obeying conservation at interior vertices and net 1 out of s, into t), the electrical current is the unique minimizer of the dissipated energy $\mathcal{E}(i) = \sum_e r_e i_e^2$, and the minimum value equals $R_{\text{eff}}(s,t)$.*

Idea. Energy $\mathcal{E}$ is a strictly convex function on the affine space of unit flows, so it has a unique minimizer. Setting the gradient to zero subject to conservation produces exactly Kirchhoff's voltage law (the flow is a gradient of potentials), which characterizes the electrical current. Evaluating $\mathcal{E}$ at the current, $\sum_e r_e i_e^2 = \sum_e i_e \cdot (v\text{-drop}) = (\text{by conservation, telescoping}) \; 1 \cdot (v(s) - v(t)) = R_{\text{eff}}(s,t)$. □

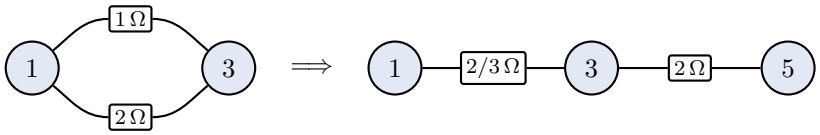
Thomson's principle has a dual, *Dirichlet's principle*: voltages minimize $f^\top \mathbf{L} f$ subject to the boundary conditions. The two are Lagrangian duals, and together they give the monotonicity law (Rayleigh): cutting an edge can only *raise* effective resistance; shorting two vertices can only lower it. We use this to bound resistances by hand.

Example 35.4 (Effective resistance in G)**.** Compute $R_{\text{eff}}(3,5)$ on the running graph. The path $3{-}4{-}5$ is two unit resistors in series, $r = 2$, and it is the *only* route from 3 to 5 (vertex 4 is a cut vertex), so $R_{\text{eff}}(3,5) = 2$.

Now $R_{\text{eff}}(1,2)$. There are two routes from 1 to 2: the direct edge ($r = 1$) and the detour $1{-}3{-}2$ ($r = 2$). The dangling tail $3{-}4{-}5$ carries no current (it is a dead end with no sink), so it does not affect the resistance. Two parallel resistors, 1 and 2:

$$\frac{1}{R_{\text{eff}}(1,2)} = \frac{1}{1} + \frac{1}{2} = \frac{3}{2} \quad \Longrightarrow \quad R_{\text{eff}}(1,2) = \frac{2}{3}.$$

Finally $R_{\text{eff}}(1,5)$: from 1 to 3 is the parallel combination of edge $1{-}3$ ($r = 1$) with the path $1{-}2{-}3$ ($r = 2$), giving $\frac{2}{3}$; then $3{-}4{-}5$ in series adds 2. So $R_{\text{eff}}(1,5) = \frac{2}{3} + 2 = \frac{8}{3}$, as Figure 35.5 shows step by step.



edge ∥ path 1 2 3 series: $2/3 + 2 = 8/3\,\Omega$

Figure 35.5: Reducing $R_{\text{eff}}(1,5)$. Left: between 1 and 3 the direct edge (r=1) is in parallel with the path $1{-}2{-}3$ (r=2), collapsing to $\frac{2}{3}$. Right: that equivalent resistor is in series with the tail $3{-}4{-}5$ (r=2), giving $R_{\text{eff}}(1,5) = \frac{8}{3}$.

35.3.3 Resistance via the Laplacian pseudoinverse

For a general network we need a formula, not a picture. Let $\mathbf{L}^+$ be the Moore–Penrose pseudoinverse of the Laplacian. Solving $\mathbf{L}v = \mathbf{1}_s - \mathbf{1}_t$ on the orthogonal complement of $\mathbf{1}$ gives $v = \mathbf{L}^+(\mathbf{1}_s - \mathbf{1}_t)$, and reading off the drop:

Theorem 35.6 (Resistance formula)**.** *With $\mathbf{1}_s$ the indicator of s,*

$$R_{\text{eff}}(s,t) \;=\; (\mathbf{1}_s - \mathbf{1}_t)^\top \mathbf{L}^+ (\mathbf{1}_s - \mathbf{1}_t) \;=\; \mathbf{L}^+_{ss} + \mathbf{L}^+_{tt} - 2\mathbf{L}^+_{st}.$$

Equivalently, $R_{\text{eff}}(s,t) = \sum_{k \geq 2} \dfrac{1}{\lambda_k} \big(u_k(s) - u_k(t)\big)^2$, where $0 = \lambda_1 < \lambda_2 \leq \cdots \leq \lambda_n$ are the Laplacian eigenvalues and u_k the orthonormal eigenvectors.

Proof. Write $\chi = \mathbf{1}_s - \mathbf{1}_t$, which is orthogonal to $\mathbf{1}$ (entries sum to 0), so $\mathbf{L}^+\mathbf{L}\chi = \chi$. Then $v = \mathbf{L}^+\chi$ solves $\mathbf{L}v = \chi$, and $R_{\text{eff}} = v(s) - v(t) = \chi^\top v = \chi^\top \mathbf{L}^+ \chi$. Expanding $\chi^\top \mathbf{L}^+ \chi = (\mathbf{1}_s - \mathbf{1}_t)^\top \mathbf{L}^+ (\mathbf{1}_s - \mathbf{1}_t)$ gives the entrywise form. The spectral form follows from $\mathbf{L}^+ = \sum_{k \geq 2} \lambda_k^{-1} u_k u_k^\top$ (the eigendecomposition of the Laplacian restricted to $\mathbf{1}^\perp$) and $\chi^\top u_k = u_k(s) - u_k(t)$. □

The spectral form connects resistance to spectral graph theory: resistance is a sum over the Laplacian spectrum, weighted by inverse eigenvalues, so small eigenvalues (poor connectivity, see Cheeger) inflate resistance, exactly as poor connectivity should.

Remark 35.1 (Resistance is a metric). R_{eff} is a genuine metric on V: symmetric, zero iff $s = t$, and it satisfies the triangle inequality (a one-line consequence of the energy formulation). The map $s \mapsto (\mathbf{L}^{+})^{1/2}\mathbf{1}_s$ embeds the vertices into Euclidean space so that squared distances are resistances — the *resistive embedding*, used in graph sparsification and machine learning.

35.4 Commute time equals $2m$ times resistance

We now prove the main theorem of the chapter. It ties the probabilistic quantity (commute time) to the electrical one (effective resistance) with a constant that depends only on the number of edges.

Theorem 35.7 (Chandra–Raghavan–Ruzzo–Smolensky–Tiwari)**.** *For every pair s, t in a connected graph,*

$$C(s,t) \;=\; H(s,t) + H(t,s) \;=\; 2m \cdot R_{\text{eff}}(s,t).$$

For a weighted graph with total conductance $2W = \sum_i c_i$, the constant $2m$ becomes $2W$.

Proof. The cleanest route superposes two electrical experiments. Fix a target t. Define a function $\phi_a = H(a,t)$, the hitting time to t from a. First-step analysis (35.1) says, for $a \neq t$,

$$\phi_a = 1 + \sum_j (\mathbf{A}_P)_{aj}\phi_j \iff \sum_j (\mathbf{A}_P)_{aj}(\phi_a - \phi_j) = 1 \iff \frac{1}{\deg(a)}\sum_{j\sim a}(\phi_a - \phi_j) = 1.$$

Multiply by $\deg(a)$: $\sum_{j\sim a}(\phi_a - \phi_j) = \deg(a)$, i.e. $(\mathbf{L}\phi)_a = \deg(a)$ for all $a \neq t$, while $\phi_t = 0$. So ϕ solves $\mathbf{L}\phi = \mathbf{d} - \alpha\,\mathbf{1}_t$ where $\mathbf{d}$ is the degree vector and α absorbs the deficit at t. Summing all entries of $\mathbf{L}\phi$ (which total 0, since every column of $\mathbf{L}$ sums to 0) forces $\alpha = \sum_a \deg(a) = 2m$. Hence

$$\mathbf{L}\,\phi \;=\; \mathbf{d} - 2m\,\mathbf{1}_t, \qquad \phi_a = H(a,t).$$

Read this electrically: $\phi = H(\cdot,t)$ is the voltage when we inject $\deg(a)$ units of current at every vertex a and extract all $2m$ units at t. By symmetry, $\psi_a = H(a,s)$ solves $\mathbf{L}\psi = \mathbf{d} - 2m\,\mathbf{1}_s$: inject $\deg(a)$ everywhere, extract $2m$ at s.

Subtract the two systems: $\mathbf{L}(\phi - \psi) = 2m(\mathbf{1}_s - \mathbf{1}_t)$. The injected-everywhere currents cancel, leaving a clean s–t current of magnitude $2m$. By Definition 35.4 (resistance scales the drop linearly with current), the voltage drop of $\phi - \psi$ between s and t is $2m\,R_{\text{eff}}(s,t)$. Evaluate that drop directly:

$$(\phi-\psi)_s - (\phi-\psi)_t = \big(\underbrace{H(s,t)}_{\phi_s} - \underbrace{H(s,s)}_{\psi_s=0}\big) - \big(\underbrace{H(t,t)}_{\phi_t=0} - \underbrace{H(t,s)}_{\psi_t}\big) = H(s,t) + H(t,s) = C(s,t).$$

Therefore $C(s,t) = 2m\,R_{\text{eff}}(s,t)$. □

Why this matters

Hitting time is an asymmetric, path-dependent expectation; nothing about its definition is symmetric in s and t. Yet its symmetrization, the commute time, equals $2m$ times a quantity (resistance) computed by Ohm's law with no probability in sight. The proof never simulates a walk: it solves two linear systems and subtracts. This works because the chain is reversible: a reversible walk *is* a resistor network, so the round-trip time is determined by an electrical quantity. Later hitting-time bounds in this book all rely on this identity.

Example 35.5 (Commute times in G, finished by hand)**.** We computed $R_{\text{eff}}(3,5) = 2$ (Example 35.4) and $2m = 10$, so

$$C(3,5) = H(3,5) + H(5,3) = 10 \cdot 2 = 20.$$

We already know $H(3,5) = 16$ from Example 35.3, so $H(5,3) = 20 - 16 = 4$. Getting from the leaf into the hub takes 4 steps on average; the reverse takes 16. The two directions differ by a factor of 4, yet the round trip is exactly $2m\, R_{\text{eff}}$.

Likewise $R_{\text{eff}}(4,5) = 1$ (single edge), so $C(4,5) = 10$, and since $H(5,4) = 1$ (forced step), $H(4,5) = 9$ — matching the brute-force value from Example 35.3 with no extra algebra. Finally $C(1,2) = 10 \cdot \frac{2}{3} = \frac{20}{3}$ and $C(1,5) = 10 \cdot \frac{8}{3} = \frac{80}{3}$.

35.4.1 Foster's theorem: a global resistance identity

The sum of the effective resistances over all edges depends only on the number of vertices, whatever the structure of the graph.

Theorem 35.8 (Foster)**.** *In any connected graph on n vertices,*

$$\sum_{\{i,j\} \in E} R_{\text{eff}}(i,j) \;=\; n - 1.$$

Proof. By the resistance formula, $R_{\text{eff}}(i,j) = (\mathbf{1}_i - \mathbf{1}_j)^\top \mathbf{L}^+ (\mathbf{1}_i - \mathbf{1}_j)$. For an edge, $\mathbf{1}_i - \mathbf{1}_j$ is the column of the signed incidence matrix $\mathbf{A}_B$ for that edge (up to sign), so $R_{\text{eff}}(i,j) = \big(\mathbf{A}_B^\top \mathbf{L}^+ \mathbf{A}_B\big)_{ee}$. Summing over edges is the trace:

$$\sum_e R_{\text{eff}}(e) = \operatorname{tr}\big(\mathbf{A}_B^\top \mathbf{L}^+ \mathbf{A}_B\big) = \operatorname{tr}\big(\mathbf{L}^+ \mathbf{A}_B \mathbf{A}_B^\top\big) = \operatorname{tr}(\mathbf{L}^+ \mathbf{L}) = \operatorname{rank}(\mathbf{L}) = n - 1,$$

using $\mathbf{L} = \mathbf{A}_B \mathbf{A}_B^\top$ (the incidence factorization) and $\operatorname{tr}(\mathbf{L}^+\mathbf{L}) = \operatorname{rank} \mathbf{L} = n - 1$ for a connected graph. □

Example 35.6 (Foster on G)**.** The five edges have resistances $R_{\text{eff}}(1,2) = R_{\text{eff}}(1,3) = R_{\text{eff}}(2,3) = \frac{2}{3}$ (each triangle edge sees the other two-edge path in parallel) and $R_{\text{eff}}(3,4) = R_{\text{eff}}(4,5) = 1$ (bridges). Sum:

$$3 \cdot \tfrac{2}{3} + 1 + 1 = 2 + 2 = 4 = n - 1. \quad \checkmark$$

A bridge always has $R_{\text{eff}} = 1$ (no parallel path), and indeed the two bridge edges contribute 2 while the triangle contributes 2, totaling exactly $n - 1 = 4$.

Connection

Foster's theorem underlies *spectral sparsification*. Spielman and Srivastava showed that sampling each edge with probability proportional to $w_e R_{\text{eff}}(e)$ — and Foster says these "effective weights" sum to $n - 1$, so $O(n \log n/\varepsilon^2)$ samples suffice — produces a sparse graph whose Laplacian approximates the original's quadratic form to within $1 \pm \varepsilon$. The same effective resistances drive the Laplacian-solver and electrical-flow algorithms of Chapter 41. Random walks, effective resistances, and spectral sparsifiers are closely linked tools. The companion *Probability* volume's chapter on Markov chains develops first-step analysis on general state spaces; here we specialize it to reversible walks on undirected graphs.

35.5 Computing the quantities: an algorithm

All the named quantities reduce to Laplacian solves. Below is the pipeline for hitting times, commute times, and resistances; it is the practical recipe behind the theory.

Algorithm 132: Random-walk quantities via Laplacian solves

Input: Connected graph $G = (V, E)$ with conductances c_{ij}; query vertices s, t.
Output: $R_{\text{eff}}(s,t)$, $C(s,t)$, and hitting times $H(\cdot, t)$.

Assemble $\mathbf{L} = \mathbf{A}_D - \mathbf{A}$ (weighted: $\mathbf{A}_D = \operatorname{diag}(c_i)$)
$2W \leftarrow \sum_i c_i$ `// = 2m in the unit-resistor case`
`// Effective resistance: one solve`
Solve $\mathbf{L}\, v = \mathbf{1}_s - \mathbf{1}_t$ for $v \perp \mathbf{1}$ (ground v at any vertex)
$R_{\text{eff}}(s,t) \leftarrow v(s) - v(t)$
$C(s,t) \leftarrow 2W \cdot R_{\text{eff}}(s,t)$
`// Hitting times to a fixed target` t`: one solve`
Solve $\mathbf{L}\,\phi = \mathbf{d} - 2W\,\mathbf{1}_t$ with $\phi_t = 0$ `// d = conductance/degree vector`
for *each* $a \in V$ **do**
  $H(a,t) \leftarrow \phi_a$
$H(s,t) \leftarrow \phi_s$; $\quad H(t,s) \leftarrow C(s,t) - H(s,t)$
return $R_{\text{eff}}(s,t)$, $C(s,t)$, ϕ

Why it works

Why does solving $\mathbf{L}\phi = \mathbf{d} - 2W\mathbf{1}_t$ give *all* hitting times to t at once? Because the derivation in Theorem 35.7 showed $H(\cdot, t)$ is precisely the voltage of the "inject-degree-everywhere, drain-at-t" experiment. One linear system yields all n hitting times. For *all-pairs* commute times you need the full pseudoinverse $\mathbf{L}^+$ (then $C(s,t) = 2m(\mathbf{L}^+_{ss} + \mathbf{L}^+_{tt} - 2\mathbf{L}^+_{st})$), a single $O(n^3)$ inversion or n solves.

Cost summary

Cost. Each Laplacian solve is $O(n^3)$ by dense Gaussian elimination, roughly $O(m^{1.5})$ by conjugate gradient with a good preconditioner, or $\tilde{O}(m) = O(m \log^c n \, \log(1/\varepsilon))$ by the Spielman–Teng / Koutis–Miller–Peng near-linear solvers. A single resistance/commute query: *two* solves, $\tilde{O}(m)$. All-pairs: one pseudoinverse, $O(n^3)$, or Johnson–Lindenstrauss plus $O(\log n/\varepsilon^2)$ solves to estimate *all* resistances to $1 \pm \varepsilon$ in $\tilde{O}(m/\varepsilon^2)$ total (Spielman–Srivastava).

35.6 Mixing time and the spectral gap

The last loose end is convergence: how fast does $p_t \to \pi$? For reversible chains this is governed by the spectrum of $\mathbf{A}_P$, which is similar to a symmetric *normalized* matrix and hence to the normalized Laplacian $\mathcal{L} = \mathbf{A}_I - \mathbf{A}_D^{-1/2}\mathbf{A}\mathbf{A}_D^{-1/2}$ of Chapter 32.

Theorem 35.9 (Spectral mixing bound)**.** *Let* $\mathbf{A}_P = \mathbf{A}_D^{-1}\mathbf{A}$ *have eigenvalues* $1 = \mu_1 > \mu_2 \geq \cdots \geq \mu_n \geq -1$*, and set* $\mu_\star = \max(\mu_2, |\mu_n|)$ *(the second-largest in absolute value). Then for the lazy walk*

and any start,

$$\|p_t - \pi\|_{\mathrm{TV}} \;\le\; \frac{1}{2}\sqrt{\frac{1-\pi_{\min}}{\pi_{\min}}}\,\mu_\star^{\,t}, \qquad t_{\mathrm{mix}}(\varepsilon) = O\Big(\frac{1}{1-\mu_\star}\log\frac{1}{\pi_{\min}\varepsilon}\Big).$$

The quantity $1-\mu_2$ *is the* spectral gap*; a larger gap means faster mixing.*

Idea. Conjugate $\mathbf{A}_P$ to the symmetric $\mathbf{A}_S = \mathbf{A}_D^{1/2}\mathbf{A}_P\mathbf{A}_D^{-1/2} = \mathbf{A}_D^{-1/2}\mathbf{A}\mathbf{A}_D^{-1/2} = \mathbf{A}_I - \mathcal{L}$, which shares $\mathbf{A}_P$'s eigenvalues and has an orthonormal eigenbasis. Expand the (rescaled) start vector in that basis; the $\mu_1 = 1$ component is the stationary part, and every other component decays by μ_k per step. The slowest survivor decays like $\mu_\star^t$. Converting the ℓ_2 bound to total variation contributes the $\sqrt{(1-\pi_{\min})/\pi_{\min}}$ factor. Laziness ($\mathbf{A}'_P = \frac{1}{2}(\mathbf{A}_I + \mathbf{A}_P)$) maps eigenvalues to $\frac{1}{2}(1+\mu) \in [0,1]$, killing the negative-eigenvalue (periodicity) problem so $\mu_\star = \mu_2$. □

By Cheeger's inequality (Chapters 32 and 34), the gap $1-\mu_2$ is sandwiched by the conductance Φ: $\frac{\Phi^2}{2} \le 1-\mu_2 \le 2\Phi$. So a graph mixes fast iff it has no sparse cut iff it is an expander. Expansion, fast mixing, and a large Laplacian gap describe one property, which is why random walks are central to sampling, undirected-connectivity in logspace, and PageRank (Chapter 36).

Trace it yourself

Estimate the mixing time of the running graph numerically. Build $\mathbf{A}_P$ from Example 35.1, lazify it, and iterate $p_{t+1} = p_t\mathbf{A}'_P$ from $p_0 = \mathbf{1}_5$ (start at the leaf). Watch p_t crawl toward $\pi = (.2, .2, .3, .2, .1)$. Plot $\|p_t - \pi\|_1$ on a log axis; the slope is $\log\mu_\star$. Then compute μ_2 of $\mathbf{A}_P$ directly (read it off the 5×5 matrix or get it numerically) and check the slope matches.

35.7 The full picture on one network

Viewing every quantity for the running graph side by side shows how both perspectives agree:

Quantity	By hand (probability)	By resistance	Value
π	first-step / fixed point	$\deg/2m$	$(.2, .2, .3, .2, .1)$
$H(3,5)$	Example 35.3	—	16
$H(5,3)$	$C - H(3,5)$	$2mR_{\mathrm{eff}} - 16$	4
$R_{\mathrm{eff}}(3,5)$	—	series 1+1	2
$C(3,5)$	$H(3,5)$+$H(5,3)$	$2m\,R_{\mathrm{eff}}$	20
$R_{\mathrm{eff}}(1,2)$	—	parallel $1 \parallel 2$	2/3
$\sum_e R_{\mathrm{eff}}(e)$	—	Foster	$4 = n-1$

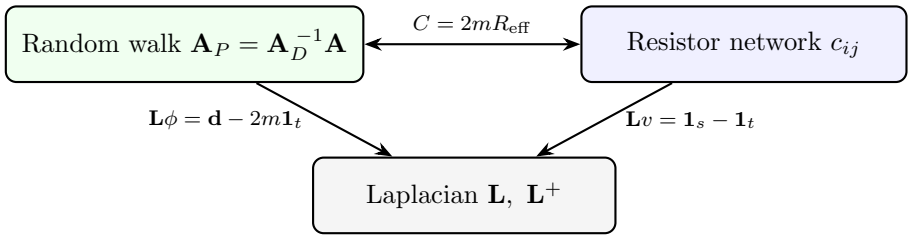


Figure 35.6: Three views of one object. Walk, circuit, and matrix are interconvertible; the commute–resistance identity is the top edge, and both sides reduce to Laplacian linear systems.

35.8 Historical Context and Controversies

History

The connection between random walks and electrical networks has a long history in applied mathematics. In 1847, Gustav Kirchhoff formulated his laws of electrical circuits alongside the matrix-tree theorem (which counts spanning trees via determinants of Laplacian submatrices), showing that the distribution of current through a network of wires can be expressed in terms of spanning trees. Thirty years later, Lord Rayleigh established his Monotonicity Principle in acoustics and electricity (pinching a wire or adding parallel pathways can never increase the overall effective resistance between two terminals).

The probabilistic half of the story developed in parallel. In 1921, George Pólya asked whether a wandering particle on an infinite grid $\mathbb{Z}^d$ would ever return to its starting point. He showed that random walks on a line or plane return with probability one, while on three-dimensional grids and higher, the walker can escape to infinity with non-zero probability. In 1944, Shizuo Kakutani connected Brownian motion to continuous Dirichlet problems (finding harmonic functions with fixed boundary conditions), and in 1984 Peter Doyle and J. Laurie Snell made the physical connection explicit: Pólya's recurrence theorem is exactly the statement that the infinite resistor grid has infinite effective resistance to infinity in dimensions $d \leq 2$, but finite resistance when $d \geq 3$.

Early algorithmic work ran into two traps. The first was trying to compute hitting times by summing geometric series over endless cycles, which quickly becomes unmanageable; solving the single linear system $\mathbf{L}v = b$ avoids the problem entirely. The second was assuming that hitting times behave like symmetric distances. On a lollipop graph, drifting down the stick into the clique takes quadratic time $O(n^2)$, but wandering out of the clique through the single bottleneck takes cubic time $\Theta(n^3)$. In 1989, Ashok Chandra, Prabhakar Raghavan, Walter Ruzzo, Roman Smolensky, and Prasoon Tiwari proved the commute-time identity $C(s,t) = 2m\, R_{\text{eff}}(s,t)$, demonstrating that summing the two directions restores the reciprocity of physical resistance and bounds the space needed to test connectivity in randomized logarithmic space. An active research problem is the localized estimation of hitting and commute times on massive directed networks where physical reciprocity breaks down.

35.8.1 What We Can Now Do and Its Limitations

Connection

Capabilities Unlocked:

- We can compute exact hitting times, escape probabilities, and commute times across arbitrary graphs in polynomial time by solving linear systems in the Laplacian matrix $\mathbf{L}$.
- Foster's theorem provides a global invariant: the sum of effective resistances across all edges in any connected graph is exactly $n-1$, establishing that edge resistances cannot all be simultaneously large.
- Effective resistance induces a metric space: $\sqrt{R_{\text{eff}}(u,v)}$ satisfies the triangle inequality and embeds vertices into Euclidean space where physical distance reflects multi-path connectivity.

Limitations:

- *Computational cost*: Calculating $R_{\text{eff}}(s,t)$ requires solving $\mathbf{L}v = \mathbf{1}_s - \mathbf{1}_t$. Exact matrix inversion via Gaussian elimination requires $O(n^3)$ operations, rendering exact resistance calculations infeasible on web-scale graphs with billions of vertices.
- *Reversibility failure on directed graphs*: The electrical network analogy requires detailed balance ($\pi_i P_{ij} = \pi_j P_{ji}$), which holds only for undirected (or reversible) Markov chains. On directed graphs, asymmetric transitions produce non-conservative flows with non-zero circulation. Standard resistor networks cannot model directed flows without active elements like diodes, breaking the voltage analogy.
- *Global sensitivity*: Effective resistance is a global property. Adding or deleting a single high-conductance edge far away can alter $R_{\text{eff}}(s,t)$ between vertices located nowhere near the change.

Where We Go Next: The electrical and reversible random-walk tools we built assume undirected edges: current flows both ways, and detailed balance holds. In **Chapter 36 (PageRank and Markov Chains)**, we step into directed, non-reversible networks. To rank web pages where hyperlinks are one-way arrows without physical reciprocity, we introduce Markov chains with random teleportation, the Perron–Frobenius theorem, and power iteration. (Later, in Chapter 41, we return to undirected resistor networks to solve Laplacian systems in nearly linear time.)

Exercises

Exercise 35.1 (Stationary by hand)**.** Take the path graph P_4 on vertices $1-2-3-4$. Write $\mathbf{A}_P$, find π from $\deg/2m$, and verify $\pi\mathbf{A}_P = \pi$ directly. Why does the (non-lazy) walk fail to converge from $X_0 = 1$? Identify the period.

Exercise 35.2 (Hitting times on a cycle)**.** For the n-cycle C_n, prove by symmetry and first-step analysis that $H(i,j) = d(n-d)$ where d is the graph distance from i to j. Conclude $C(i,j) = 2H$ when i,j are antipodal, and check it against $2m\,R_{\text{eff}}$ with $m = n$ and $R_{\text{eff}} = \frac{d(n-d)}{n}$ (two arcs in parallel).

Exercise 35.3 (The lollipop is worst)**.** The lollipop L_n is a clique $K_{n/2}$ with a path of length $n/2$ attached at one clique vertex u, ending at w. Show $H(w,u) = \Theta(n^2)$ (path) but $H(u,w) = \Theta(n^3)$ (escape the clique, then traverse). Conclude the $\Theta(n^3)$ worst-case hitting time and that hitting time can be highly asymmetric.

Exercise 35.4 (Resistance is a metric)**.** Using $R_{\text{eff}}(s,t) = \chi^\top \mathbf{L}^+ \chi$ with $\chi = \mathbf{1}_s - \mathbf{1}_t$, prove $\sqrt{R_{\text{eff}}}$ is a metric: show $R_{\text{eff}}(s,t) = \|(\mathbf{L}^+)^{1/2}\mathbf{1}_s - (\mathbf{L}^+)^{1/2}\mathbf{1}_t\|^2$ and apply the Euclidean triangle inequality. Where does connectivity get used?

Exercise 35.5 (Rayleigh monotonicity)**.** Prove that adding an edge to a graph cannot increase $R_{\text{eff}}(s,t)$ for any s,t, and that increasing any edge's resistance cannot decrease $R_{\text{eff}}(s,t)$. (Use Thomson's principle: a minimizer over a larger feasible set, or with smaller costs, can only do better.) Deduce that for a spanning subgraph $H \subseteq G$, $R^H_{\text{eff}}(s,t) \geq R^G_{\text{eff}}(s,t)$.

Exercise 35.6 (Foster, twice)**.** Verify Foster's theorem on (a) the complete graph K_4 and (b) the star $K_{1,3}$. For K_n, show every edge has $R_{\text{eff}} = 2/n$ and re-derive Foster from $\binom{n}{2} \cdot \frac{2}{n} = n - 1$.

Exercise 35.7 (Commute time from the pseudoinverse)**.** Compute $\mathbf{L}^+$ for the triangle K_3 explicitly (it is circulant). Read off $C(1,2) = 2m(\mathbf{L}^+_{11} + \mathbf{L}^+_{22} - 2\mathbf{L}^+_{12})$ and check against $2m\, R_{\text{eff}}(1,2)$ with $R_{\text{eff}} = 2/3$, $m = 3$. You should get $C = 4$.

Exercise 35.8 (Harmonic = hitting probability, coded)**.** Implement the Dirichlet solve: given a graph and two vertices s,t, set $v(s) = 1, v(t) = 0$ and solve $(\mathbf{L}v)(a) = 0$ at interior a. Then estimate the same hitting probabilities by Monte Carlo simulation of the walk. Plot solver vs. simulation; they should agree within sampling noise. (This is the cheapest empirical proof of Theorem 35.4.)

Exercise 35.9 (Spectral gap and a barbell)**.** The barbell graph is two cliques $K_{n/2}$ joined by a single edge. Argue informally that μ_2 is very close to 1 (a near-disconnected cut), so mixing is slow and the commute time across the bridge is $\Theta(n^3)$. Relate this to Cheeger: the bottleneck edge gives tiny conductance $\Phi = \Theta(1/n^2)$.

Exercise 35.10 (Weighted walks)**.** Generalize Theorem 35.7 to a weighted graph with conductances c_{ij}: define the walk with $\Pr[i \to j] = c_{ij}/c_i$, set $2W = \sum_i c_i$, and prove $C(s,t) = 2W\, R_{\text{eff}}(s,t)$ by repeating the two-experiment subtraction. Identify exactly where reversibility (detailed balance) is used.

Exercise 35.11 (Cover time of the path)**.** Show the cover time of the path P_n is $\Theta(n^2)$ (the walk must reach both ends; the far end is $\Theta(n^2)$ away). Then use the Matthews bound with $H_{\max} = \Theta(n^2)$ to get the loose upper bound $O(n^2 \log n)$, and explain why for the path the $\log n$ is *not* tight, whereas for the complete graph cover time $\Theta(n \log n)$ it is.

Chapter 36

PageRank and Markov Chains on Graphs

A web crawler in 1997 faced a hard problem: it could fetch billions of pages, but it had no idea which ones *mattered*. Keyword matching said the words "Stanford University" appeared on a hundred thousand pages – which one should a searcher see first? The answer Google adopted was simple to state. Treat the web as a directed graph, let a bored surfer click links at random forever, and ask: where does the surfer spend their time? Pages the surfer visits often are important. That long-run fraction of time is a *stationary distribution* of a Markov chain, and computing it on a graph with billions of nodes is what this chapter is about.

We will build the random-surfer model from scratch, show that PageRank is the stationary distribution of a carefully patched Markov chain, prove that power iteration converges and pin down its rate to the spectral gap, develop personalized PageRank, and study how well the algorithm holds up under adversarial link spam. Throughout, we use both the graph view and the linear-algebra view of the same object.

Connection

This chapter sits on top of three others. The Markov-chain machinery – stationary distributions, mixing, the Perron–Frobenius theorem – is developed in the companion *Probability* volume; we recall what we need. The matrix $\mathbf{M} = \mathbf{A}^\top \mathbf{D}^{-1}$ and its eigenstructure connect to the spectral chapters (Chapters 31–34) and to the random-walk/electrical-network material (Chapter 35). Personalized PageRank is, almost literally, a discounted random walk, so the walk-and-resistance lens carries over directly.

36.1 The web as a graph

Model the web as a directed graph $G = (V, E)$. Each page is a vertex; a hyperlink from page u to page v is a directed edge $u \to v$. Let $n = |V|$. For a vertex u let $\deg^+(u)$ be its *out-degree*, the number of links leaving u.

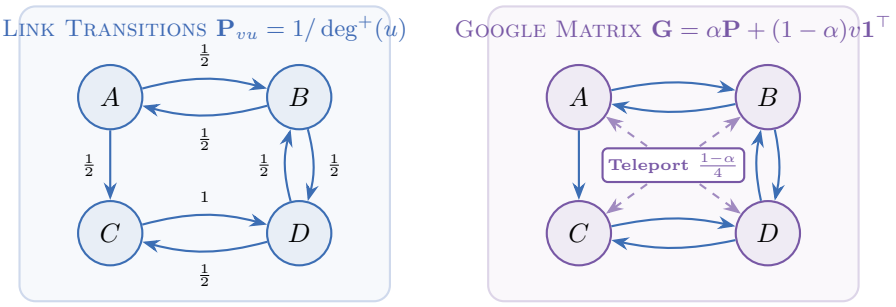


Figure 36.1: The four-page web and its PageRank Markov chain. Left: The raw link transition graph $\mathbf{P}$. Surfers follow links with uniform probabilities $1/\deg^+(u)$. Right: The regularized Google matrix $\mathbf{G}$. With probability α, the surfer follows a link; with probability $1-\alpha$, they teleport to a uniformly chosen page with probability $(1-\alpha)/n$. This guarantees that all transition probabilities are strictly positive, yielding an irreducible, aperiodic Markov chain with a unique stationary distribution.

Intuition

Ignore page content for now. A link $u \to v$ is a tiny vote: "u thinks v is worth visiting." But not all votes are equal. A link from a page that is itself widely linked-to should count for more than a link from an obscure page. This is recursive – importance is defined in terms of importance – and the random-surfer model makes the recursion well-posed and computable.

The recursive intuition has a long pedigree. Counting weighted citations to rank academic papers goes back to the 1970s (Pinski and Narin's *influence weights*), and eigenvector centrality predates the web by decades. What Page and Brin added was a model – the surfer – that makes the eigenvector both interpretable and numerically tame on a graph the size of the web.

36.2 The random-surfer model

Picture a surfer standing on page u. With each tick of the clock they pick one of the $\deg^+(u)$ outgoing links uniformly at random and follow it. This is a random walk on the directed graph. Where the surfer is after t steps is a random variable; the *distribution* of that random variable is a column vector $x_t \in \mathbb{R}^n$ with $x_t(v) = \Pr[\text{surfer at } v \text{ at time } t]$.

36.2.1 The transition matrix

Define the column-stochastic *transition matrix* $\mathbf{P} \in \mathbb{R}^{n\times n}$ by

$$\mathbf{P}_{vu} \;=\; \Pr[u \to v \text{ in one step}] \;=\; \begin{cases} \dfrac{1}{\deg^+(u)} & \text{if } (u,v) \in E, \\ 0 & \text{otherwise.} \end{cases}$$

Column u of $\mathbf{P}$ holds the surfer's next-step probabilities given they are at u; each nonzero column sums to 1. Equivalently, with $\mathbf{A}$ the directed adjacency matrix ($\mathbf{A}_{uv} = 1$ iff $u \to v$) and $\mathbf{D} = \operatorname{diag}(\deg^+(u))$,

$$\mathbf{P} \;=\; \mathbf{A}^\top \mathbf{D}^{-1}.$$

If we store distributions as *column* vectors x_t, one step advances them by $x_{t+1} = \mathbf{P}\,x_t$. (Some texts use row vectors and a row-stochastic $\mathbf{P}^\top$; we fix the column convention and stick to it.)

Example 36.1 (Transition matrix of Figure 36.1)**.** Order the vertices A, B, C, D. Out-edges: $A \to \{B, C\}$, $B \to \{A, D\}$, $C \to \{D\}$, $D \to \{B, C\}$. So

$$\mathbf{P} = \begin{pmatrix} 0 & \frac{1}{2} & 0 & 0 \\ \frac{1}{2} & 0 & 0 & \frac{1}{2} \\ \frac{1}{2} & 0 & 0 & \frac{1}{2} \\ 0 & \frac{1}{2} & 1 & 0 \end{pmatrix}, \qquad \begin{aligned} &\text{column } A: \text{ mass splits to } B, C \\ &\text{column } B: \text{ mass splits to } A, D \\ &\text{column } C: \text{ all mass to } D \\ &\text{column } D: \text{ mass splits to } B, C \end{aligned}$$

Each column sums to 1, so $\mathbf{P}$ is column-stochastic.

36.2.2 Two diseases: dangling nodes and traps

The raw walk has two pathologies that break the "long-run fraction of time" story.

Common trap

Dangling nodes (rank sinks). A page with no outgoing links ($\deg^+(u) = 0$) has an all-zero column in $\mathbf{P}$. The surfer who arrives there *vanishes* – probability leaks out of the system, and $\mathbf{P}$ is no longer stochastic. PDFs, image pages, and pages the crawler has not yet expanded are all dangling.

Traps (disconnected cliques). Even with every out-degree positive, a group of pages that link only among themselves swallows all probability that ever enters it. In Markov language the chain is *reducible* or *periodic*, and a unique stationary distribution need not exist.

Both diseases have the same cure: **teleportation**. Figure 36.2 contrasts the two structural traps on a raw web graph alongside the teleportation escape routes that restore ergodicity.

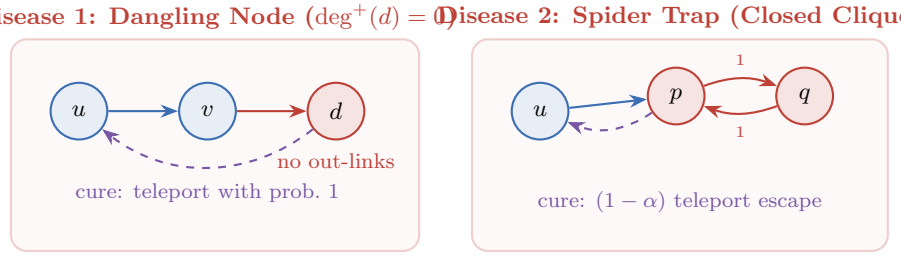


Figure 36.2: The two Markov-chain pathologies on raw web graphs. Left: A dangling node d has $\deg^+(d) = 0$, so arriving mass vanishes unless its zero column is patched to the teleport vector v. Right: A spider trap $\{p, q\}$ has no outgoing links to the rest of the web, absorbing all mass that enters until $(1 - \alpha)$ damping leaks surfers back out.

36.3 PageRank as a stationary distribution

Give the surfer a coin with bias α (the *damping factor*, classically $\alpha = 0.85$). At each step:

- with probability α, follow a uniformly random out-link (the old behavior);

- with probability $1-\alpha$, *teleport* to a page chosen from a distribution v (uniform, $v = \frac{1}{n}\mathbf{1}$, in the classic model).

A dangling node is handled by saying: if there are no out-links, teleport with probability 1. Algebraically, patch each all-zero column of $\mathbf{P}$ to the teleport vector v, giving a genuinely column-stochastic $\mathbf{P}'$. The *Google matrix* is then

$$\boxed{\mathbf{G} \;=\; \alpha\,\mathbf{P}' \;+\; (1-\alpha)\,v\,\mathbf{1}^{\top}.}$$

Here $v\,\mathbf{1}^{\top}$ is the rank-one matrix every column of which equals v: from anywhere, teleport lands you in v. The PageRank vector π is the stationary distribution of $\mathbf{G}$:

$$\boxed{\mathbf{G}\,\pi = \pi, \qquad \pi \geq 0, \qquad \mathbf{1}^{\top}\pi = 1.}$$

Intuition

Teleportation acts as a uniform damping mechanism on the link structure. A fraction $1-\alpha$ of the time the surfer ignores links entirely and jumps to a random page. This one change fixes both pathologies: it gives dangling nodes an exit, lets the walk leave traps that would otherwise hold probability inside closed cliques, and makes every entry of the Google matrix $\mathbf{G}$ strictly positive. By the Perron–Frobenius theorem, strict positivity guarantees a unique stationary distribution, and power iteration converges to it.

36.3.1 Why a unique stationary distribution exists

Theorem 36.1 (Existence and uniqueness of PageRank)**.** *For any $0 < \alpha < 1$ and any probability vector $v > 0$ (all entries positive), the Google matrix $\mathbf{G}$ has a unique stationary distribution π with $\pi > 0$, and it is the unique eigenvector of $\mathbf{G}$ with eigenvalue 1 up to scaling.*

Why it works

The proof applies Perron–Frobenius directly. Recall the statement: if $\mathbf{M}$ is a nonnegative *primitive* matrix (some power is strictly positive), then its spectral radius ρ is a simple eigenvalue, the only eigenvalue of modulus ρ, and its eigenvector can be chosen strictly positive. We check $\mathbf{G}$ qualifies.

Proof. Because $v > 0$ and $\alpha < 1$, every entry of $\mathbf{G} = \alpha\mathbf{P}' + (1-\alpha)v\mathbf{1}^{\top}$ satisfies $\mathbf{G}_{ij} \geq (1-\alpha)v_i > 0$. So $\mathbf{G}$ is strictly positive, hence irreducible and primitive (it is its own positive power). $\mathbf{G}$ is column-stochastic: $\mathbf{1}^{\top}\mathbf{G} = \alpha\,\mathbf{1}^{\top}\mathbf{P}' + (1-\alpha)(\mathbf{1}^{\top}v)\mathbf{1}^{\top} = \alpha\mathbf{1}^{\top} + (1-\alpha)\mathbf{1}^{\top} = \mathbf{1}^{\top}$, using $\mathbf{1}^{\top}\mathbf{P}' = \mathbf{1}^{\top}$ and $\mathbf{1}^{\top}v = 1$. A column-stochastic matrix has spectral radius exactly 1, with $\mathbf{1}$ a left eigenvector for eigenvalue 1. By Perron–Frobenius, 1 is a simple eigenvalue, it is the unique eigenvalue of modulus 1, and the corresponding right eigenvector π can be chosen strictly positive. Normalize $\mathbf{1}^{\top}\pi = 1$. Uniqueness of the normalized positive eigenvector follows from simplicity. □

Connection

This is the same Perron–Frobenius theorem that gives expander graphs (Chapter 34) their spectral gap and that governs the convergence of *any* aperiodic irreducible Markov chain in the companion *Probability* volume. The strict positivity of $\mathbf{G}$ is the discrete-graph analogue of an ergodic chain: every state reaches every other in one step. PageRank is, then, exactly the long-run occupancy of an ergodic chain, which makes the "fraction of time the surfer spends" story rigorous.

36.3.2 Handling dangling nodes without densifying

Patching every dangling column to v literally would destroy sparsity. The fix is to keep $\mathbf{P}$ sparse and carry the dangling correction as a scalar. Let $D \subseteq V$ be the dangling set and let $d \in \{0,1\}^n$ be its indicator. Then $\mathbf{P}' = \mathbf{P} + v\, d^{\top}$ (mass landing in D is redirected to v), and

$$\mathbf{G}\, x = \alpha \mathbf{P} x \;+\; \big(\alpha\, (d^{\top} x) + (1-\alpha)\big)\, v, \qquad \text{whenever } \mathbf{1}^{\top} x = 1.$$

The term $d^{\top} x = \sum_{u \in D} x_u$ is the total mass currently on dangling pages; it is a single dot product. We never form $\mathbf{G}$ or $\mathbf{P}'$ explicitly. This is the identity that makes PageRank scale.

36.4 Power iteration and convergence

How do we actually solve $\mathbf{G}\pi = \pi$ when n is in the billions? We do *not* invert anything. We iterate the surfer's step:

$$x^{(0)} = v, \qquad x^{(k+1)} = \mathbf{G}\, x^{(k)}.$$

This is power iteration. Because $\mathbf{G}$ has a unique dominant eigenvalue 1 with a spectral gap below it, $x^{(k)} \to \pi$.

Algorithm 133: PageRank by power iteration (sparse, dangling-aware)

Input: Out-adjacency lists of G; damping α; teleport vector v; tolerance ε
Output: PageRank vector π

$x \leftarrow v$;
repeat
  $x_{\text{old}} \leftarrow x$;
  $y \leftarrow \mathbf{0}$; `// accumulate` $\alpha \mathbf{P} x$
  foreach *vertex* u *with* $\deg^{+}(u) > 0$ **do**
    foreach $u \to w$ **do**
      $y_w \leftarrow y_w + \alpha\, x_u / \deg^{+}(u)$;
  $s \leftarrow \sum_{u \in D} x_u$; `// dangling mass`
  $\beta \leftarrow \alpha\, s + (1-\alpha)$; `// redistributed via teleport`
  $x \leftarrow y + \beta\, v$; `//` $\mathbf{1}^{\top} x = 1$ `automatically`
until $\|x - x_{\text{old}}\|_1 \le \varepsilon$;
return x;

Cost summary

Each iteration touches every edge once and every vertex a constant number of times: $\Theta(n+m)$ time, where $m = |E|$, and $\Theta(n)$ extra space for x, y. The number of iterations to reach ℓ_1-error ε is $O\big(\log(1/\varepsilon)/\log(1/\alpha)\big)$ (proved below), *independent of* n. For $\alpha = 0.85$, each iteration cuts the error by 0.85, so $\approx$ 50–100 iterations give roughly 4–7 correct digits on any graph (machine precision near 10^{-16} takes a couple hundred). Total: $O\big((n+m)\log(1/\varepsilon)/\log(1/\alpha)\big)$.

36.4.1 The convergence rate is the damping factor

The key fact is that the second-largest eigenvalue of $\mathbf{G}$ is bounded by α *regardless of the link graph.* The teleport term creates a spectral gap of at least $1-\alpha$.

Theorem 36.2 (Subdominant eigenvalue)**.** *Let $\lambda_1 = 1, \lambda_2, \dots, \lambda_n$ be the eigenvalues of the column-stochastic matrix $\mathbf{P}'$, ordered so $|\lambda_2| \geq \cdots \geq |\lambda_n|$. Then the eigenvalues of $\mathbf{G} = \alpha\mathbf{P}' + (1-\alpha)v\mathbf{1}^\top$ are 1 together with $\alpha\lambda_2, \alpha\lambda_3, \dots, \alpha\lambda_n$. In particular the subdominant eigenvalue of $\mathbf{G}$ has modulus at most α.*

Proof. Let $\mathbf{P}'$ have right eigenvector $u_1 = \pi'$ (its own stationary vector, eigenvalue 1, normalized $\mathbf{1}^\top u_1 = 1$). The left eigenvector of $\mathbf{P}'$ for eigenvalue 1 is $\mathbf{1}^\top$. For any other eigenvalue λ_i ($i \geq 2$) with right eigenvector u_i, left-eigenvector orthogonality gives $\mathbf{1}^\top u_i = 0$. Now compute $\mathbf{G}u_i = \alpha\mathbf{P}'u_i + (1-\alpha)v(\mathbf{1}^\top u_i) = \alpha\lambda_i u_i + 0 = \alpha\lambda_i u_i$. So each u_i ($i \geq 2$) is an eigenvector of $\mathbf{G}$ with eigenvalue $\alpha\lambda_i$. And $\mathbf{G}\pi = \pi$ supplies the eigenvalue-1 direction. Since $\mathbf{P}'$ stochastic forces $|\lambda_i| \leq 1$, we get $|\alpha\lambda_i| \leq \alpha$. □

Theorem 36.3 (Power-iteration convergence rate)**.** *With $x^{(k)} = \mathbf{G}^k v$ and π the PageRank vector,*

$$\|x^{(k)} - \pi\|_1 \;\leq\; 2\,\alpha^k.$$

Thus $O(\log(1/\varepsilon)/\log(1/\alpha))$ iterations suffice for ℓ_1-error ε. Figure 36.3 plots this decay for several damping factors.

Proof. Write $e^{(k)} = x^{(k)} - \pi$. Since $\mathbf{G}\pi = \pi$, $e^{(k+1)} = \mathbf{G}e^{(k)}$, so $e^{(k)} = \mathbf{G}^k e^{(0)}$. The error vector lives in the subspace $W = \{w : \mathbf{1}^\top w = 0\}$, because both $x^{(k)}$ and π are probability vectors and so $\mathbf{1}^\top e^{(k)} = 0$. On W, the action of $\mathbf{G}$ is governed by the eigenvalues $\alpha\lambda_2, \dots$ of Theorem 36.2, all of modulus $\leq \alpha$. The cleanest bound is the ℓ_1 contraction $\|\mathbf{G}w\|_1 \leq \alpha\|w\|_1$ for $w \in W$ (verified immediately below). Hence $\|e^{(k)}\|_1 \leq \alpha^k\|e^{(0)}\|_1 \leq 2\alpha^k$, since $\|x^{(0)} - \pi\|_1 \leq \|x^{(0)}\|_1 + \|\pi\|_1 = 2$. □

Why it works

The ℓ_1-contraction statement $\|\mathbf{G}w\|_1 \leq \alpha\|w\|_1$ for zero-sum w has a clean probabilistic reading. Couple two surfers with current distributions differing by w; with probability $1-\alpha$ both teleport to the *same* place v and their distributions agree there, killing that fraction of the disagreement; with probability α they take a link step, which (being stochastic) does not *increase* total-variation distance. So one step shrinks the disagreement by at least the teleport fraction $1-\alpha$ – equivalently, retains at most α of it. This coupling argument gives the spectral-gap bound without any diagonalization, and it works even when $\mathbf{G}$ is not diagonalizable.

Common trap

A bigger α (say 0.95) makes PageRank track link structure more closely but *slows convergence* (0.95^k decays lazily) and makes $\mathbf{G}$ more ill-conditioned and more spam-sensitive. A smaller α (0.7) converges fast and resists spam but washes out the link signal toward uniform. The classic 0.85 is an engineering compromise, not a theorem.

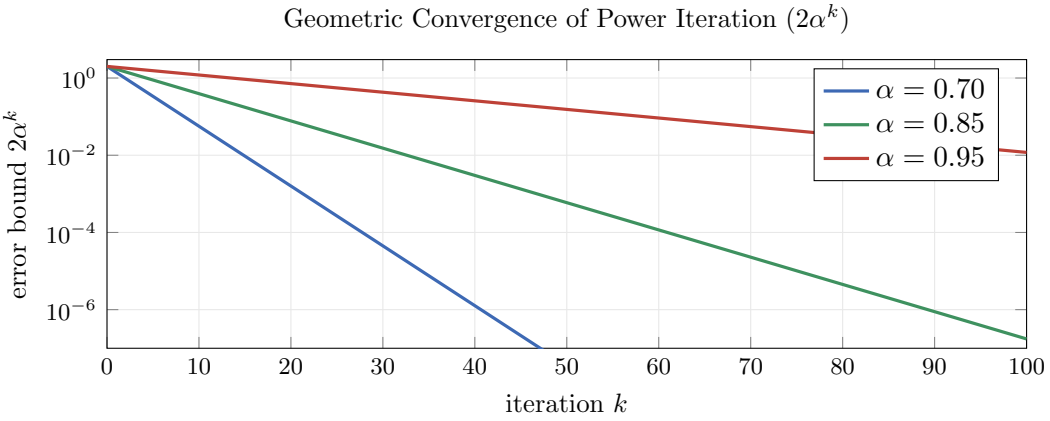


Figure 36.3: The guaranteed ℓ_1-error bound $\|x^{(k)} - \pi\|_1 \le 2\alpha^k$ of Theorem 36.3, on a log scale. The slope is $\log \alpha$ and nothing about the graph enters: the teleport-manufactured gap $1 - \alpha$ sets the decay. At $\alpha = 0.85$ roughly four dozen iterations buy three digits; $\alpha = 0.95$ needs about three times as many for the same accuracy.

36.4.2 The same thing as a linear solve

Power iteration is not the only method. Subtracting the teleport term turns the stationary equation into a sparse linear system. Starting from $\pi = \alpha \mathbf{P}'\pi + (1-\alpha)v$ and rearranging,

$$(\mathbf{I} - \alpha \mathbf{P}')\,\pi \;=\; (1-\alpha)\,v.$$

The coefficient matrix $\mathbf{I} - \alpha\mathbf{P}'$ is *strictly diagonally dominant by columns* (each off-diagonal column sum is $\alpha < 1$), hence invertible and well-conditioned away from $\alpha = 1$. So PageRank is the solution of an $n \times n$ sparse linear system, and any Krylov solver – GMRES, BiCGSTAB – or a stationary iteration like Gauss–Seidel applies. Power iteration is precisely the *Jacobi* iteration for this system, which is why its convergence factor is the spectral radius α of the iteration matrix $\alpha\mathbf{P}'$.

Why it works

Why bother, if power iteration already converges in $O(\log(1/\varepsilon)/\log(1/\alpha))$ steps? Two reasons. First, when α is pushed near 1 (sharper link sensitivity), the Jacobi factor α approaches 1 and power iteration crawls; a Krylov method's convergence depends on the full spectrum of $\mathbf{I}-\alpha\mathbf{P}'$, not merely its spectral radius, and can converge much faster. Second, the linear-system framing lets you reuse decades of preconditioning and parallel-solver engineering. The catch is memory: Krylov methods store several length-n vectors and lose the clean "one matrix–vector product per step" structure that makes power iteration straightforward to distribute across a cluster. At web scale that simplicity usually wins, which is why production PageRank stayed

close to power iteration even though the linear-algebra menu is larger.

Connection

The matrix $\mathbf{I} - \alpha\mathbf{P}'$ is the discrete resolvent of the walk, and on an undirected graph it is a reweighted, shifted Laplacian. The near-linear-time Laplacian solvers of Chapter 41 therefore apply directly to undirected PageRank – linking the Markov-chain view here to the spectral machinery of Chapters 31–35. The damping factor α sets an effective spectral gap of $1 - \alpha$, so slow power-iteration convergence as $\alpha \to 1$ and ill-conditioning of $\mathbf{I} - \alpha\mathbf{P}'$ reflect the exact same bottleneck.

36.5 Tracing PageRank by hand

Let us run Algorithm 133 on Figure 36.1, with $\alpha = 0.85$ and uniform teleport $v = \frac{1}{4}(1, 1, 1, 1)^\top$. There are no dangling nodes here, so $s = 0$ and $\beta = (1 - \alpha) = 0.15$ at every step; each step is $x \leftarrow 0.85\,\mathbf{P}x + 0.0375\,\mathbf{1}$ (since $\beta v = 0.15 \cdot \frac{1}{4}\mathbf{1} = 0.0375\,\mathbf{1}$).

Start at $x^{(0)} = (0.25,\ 0.25,\ 0.25,\ 0.25)^\top$. Recall from Example 36.1: A feeds B, C; B feeds A, D; C feeds D; D feeds B, C.

Step 1. Compute $\mathbf{P}x^{(0)}$ entrywise. Mass into each node:

$$(\mathbf{P}x)_A = \tfrac{1}{2}x_B = 0.125, \qquad (\mathbf{P}x)_B = \tfrac{1}{2}x_A + \tfrac{1}{2}x_D = 0.25,$$
$$(\mathbf{P}x)_C = \tfrac{1}{2}x_A + \tfrac{1}{2}x_D = 0.25, \qquad (\mathbf{P}x)_D = \tfrac{1}{2}x_B + x_C = 0.375.$$

Then $x^{(1)} = 0.85\,(0.125, 0.25, 0.25, 0.375)^\top + 0.0375\,\mathbf{1}$. Computing:

$$x^{(1)} = (0.14375,\ 0.25,\ 0.25,\ 0.35625)^\top.$$

(Check: the entries sum to 1.0.)

Step 2. Now feed $x^{(1)}$ through $\mathbf{P}$:

$$(\mathbf{P}x^{(1)})_A = \tfrac{1}{2}(0.25) = 0.125, \qquad (\mathbf{P}x^{(1)})_B = \tfrac{1}{2}(0.14375) + \tfrac{1}{2}(0.35625) = 0.25,$$
$$(\mathbf{P}x^{(1)})_C = 0.25, \qquad (\mathbf{P}x^{(1)})_D = \tfrac{1}{2}(0.25) + 0.25 = 0.375.$$

So $x^{(2)} = 0.85(0.125, 0.25, 0.25, 0.375)^\top + 0.0375\,\mathbf{1} = (0.14375, 0.25, 0.25, 0.35625)^\top$.

We hit a fixed point already: $x^{(2)} = x^{(1)}$. (This graph is symmetric enough that the $\mathbf{P}x$ profile stabilizes in one step; in general it would take a few dozen.) The PageRank vector is

$$\pi \approx (0.1438,\ 0.2500,\ 0.2500,\ 0.3563)^\top.$$

Trace it yourself

Read the answer. D ranks highest (0.356): it receives from both B and C, and C pours *all* its mass into D. B and C tie at 0.25. A is lowest (0.144): its only inbound link is from B, and that link is shared (B splits to A and D). Out-degree does not explain the ranking: A has the

same out-degree as B and D, but out-degree does not help a page; *in*-links from important pages do.
Now re-run it yourself with $\alpha = 0.5$. You should see the spread between π_D and π_A shrink: more teleportation pulls every score toward the uniform 0.25. Then add a dangling node E (a page A links to but that links nowhere) and watch $s = x_E$ become nonzero, feeding β.

Reading note

Sergey Brin and Larry Page described PageRank in 1998, punning on Larry Page's surname. The mathematical core is power iteration on a stochastic matrix (a technique Richard von Mises studied in the 1920s), applied to eigenvector centrality. Their main engineering contribution was the teleportation step, which ensures uniqueness while keeping the computation sparse on web graphs.

36.6 Personalized PageRank

The teleport vector v is a free parameter we set to uniform. Changing it shifts the ranking toward the pages v favors. *Personalized PageRank* (PPR) takes $v = e_s$, the indicator of a single source node s (or a small set): every teleport sends the surfer *home* to s. The stationary distribution then measures importance *as seen from* s – nodes a short, well-connected walk away from s score high.

$$\boxed{\text{ppr}_s \;=\; (1-\alpha)\,\big(\mathbf{I} - \alpha \mathbf{P}'\big)^{-1} e_s.}$$

Why it works

Where does the closed form come from? The stationary equation $x = \alpha \mathbf{P}' x + (1-\alpha) e_s$ rearranges to $(\mathbf{I} - \alpha \mathbf{P}')x = (1-\alpha)e_s$. Since $\alpha < 1$ and the column-stochastic $\mathbf{P}'$ has spectral radius 1, the matrix $\mathbf{I} - \alpha \mathbf{P}'$ is invertible with Neumann series $(\mathbf{I} - \alpha \mathbf{P}')^{-1} = \sum_{k \geq 0} \alpha^k \mathbf{P}'^k$. So

$$\text{ppr}_s = (1-\alpha) \sum_{k=0}^{\infty} \alpha^k \, \mathbf{P}'^k e_s.$$

Read the k-th term: $\mathbf{P}'^k e_s$ is the distribution after k link-steps from s, weighted by $(1-\alpha)\alpha^k$ – exactly the probability that a teleport-home surfer takes precisely k steps before the next teleport (a geometric$(1-\alpha)$ walk length). PPR is the *expected discounted visit count* of short random walks launched from s. This discounted-walk interpretation connects directly to Chapter 35.

Connection

That Neumann series is the discrete cousin of the resolvent $(\mathbf{I} - \alpha \mathbf{P})^{-1}$ and, on undirected graphs, of the Laplacian Green's function from Ch. 35. PPR vectors are the building block of local clustering: the Andersen–Chung–Lang algorithm sweeps a PPR vector to find a low-conductance cut *near* s without touching the whole graph – the local analogue of Cheeger's

inequality (Ch. 33). PPR is also how "related pages," friend recommendation in social graphs, and many graph-neural-network propagations (APPNP) are computed.

36.6.1 Computing PPR locally: push

For a single source we rarely want the whole vector; we want the few hundred high-PPR nodes near s. The Andersen–Chung–Lang *push* algorithm maintains an estimate p and a residual r (undelivered mass), and repeatedly pushes residual out of whichever node holds too much, touching only the explored frontier.

Algorithm 134: Approximate Personalized PageRank by push (Andersen–Chung–Lang)

Input: source s; damping α; threshold τ; out-adjacency of G
Output: estimate $p \approx \mathrm{ppr}_s$ with bounded residual

$p \leftarrow \mathbf{0}$; $\quad r \leftarrow e_s$;
while $\exists\, u$ *with* $r_u \geq \tau \deg^+(u)$ **do**
  $p_u \leftarrow p_u + (1-\alpha)\, r_u$; `// settle the keep-fraction at` u
  $m \leftarrow \alpha\, r_u$; $\quad r_u \leftarrow 0$;
  foreach $u \to w$ **do**
    $r_w \leftarrow r_w + m/\deg^+(u)$; `// spread the walk-fraction`
return p;

Why it works

Correctness invariant. Throughout, $\mathrm{ppr}_s = p + \mathrm{ppr}_r$, where $\mathrm{ppr}_r = \sum_u r_u \,\mathrm{ppr}_u$ is the PPR of the leftover residual (linearity of PPR in its teleport vector). Each push moves a node's residual into the part already accounted for by p plus residual sent downstream, preserving the invariant exactly. When all residuals are below threshold, p approximates ppr_s with ℓ_∞ error proportional to τ.
Complexity. Each push at u settles at least $(1-\alpha)\tau \deg^+(u)$ mass permanently into p, and total settled mass is ≤ 1. So the sum of $\deg^+(u)$ over all pushes is $O\big(\frac{1}{(1-\alpha)\tau}\big)$ – a bound *independent of* n. The work touches only nodes within reach of s, which is why push powers local clustering on graphs too large to load.

Application

Personalized PageRank is used in many large-scale recommendation and ranking systems: Twitter's "Who To Follow" ran PPR on the follow graph; Pinterest's Pixie serves related-pin recommendations from localized random-walk scores at interactive latency; and many e-commerce "customers also viewed" and graph-database proximity queries compute variants of PPR. The push algorithm's n-independent cost is exactly what makes these feasible on billion-edge graphs.

36.6.2 Tracing push by hand

Run Algorithm 134 on Figure 36.1 with source $s = A$, damping $\alpha = 0.85$, and a generous threshold $\tau = 0.05$. The push rule fires at u whenever $r_u \geq \tau \deg^+(u)$; settling deposits $(1-\alpha)r_u = 0.15\, r_u$ into p_u and spreads $\alpha r_u = 0.85\, r_u$ across u's out-edges. Recall the out-edges: $A \to \{B, C\}$, $B \to \{A, D\}$, $C \to \{D\}$, $D \to \{B, C\}$, each splitting mass into two halves except C, which sends everything to D.

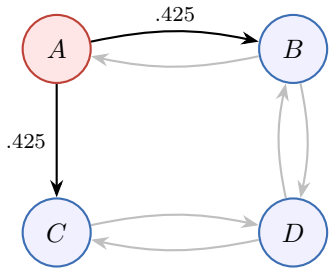


Figure 36.4: The first push, out of the source A. Its residual $r_A = 1$ settles 0.15 into p_A and sends $\alpha \cdot \frac{1}{2} = 0.425$ down each of A's two out-edges. Gray edges are not yet active; the frontier grows outward from s.

Initialize. $p = \mathbf{0}$, $r = e_A = (1, 0, 0, 0)$. Only A exceeds threshold ($r_A = 1 \geq 0.05 \cdot 2$).

Push A. Settle $p_A \mathrel{+}= 0.15 \cdot 1 = 0.15$; the walk-fraction is $m = 0.85$, split as 0.425 to each of B, C; zero out r_A. Now

$$p = (0.15,\ 0,\ 0,\ 0), \qquad r = (0,\ 0.425,\ 0.425,\ 0).$$

Push B ($r_B = 0.425 \geq 0.1$). Settle $p_B \mathrel{+}= 0.15 \cdot 0.425 = 0.06375$; spread $m = 0.85 \cdot 0.425 = 0.36125$ as 0.180625 to each of A, D:

$$p = (0.15,\ 0.06375,\ 0,\ 0), \quad r = (0.180625,\ 0,\ 0.425,\ 0.180625).$$

Push C ($r_C = 0.425 \geq 0.05$, since $\deg^+(C) = 1$). Settle $p_C \mathrel{+}= 0.06375$; all of $m = 0.36125$ goes to D (its only out-edge):

$$p = (0.15,\ 0.06375,\ 0.06375,\ 0), \quad r = (0.180625,\ 0,\ 0,\ 0.54125).$$

After three pushes p already orders the near neighborhood of A: the source keeps the most settled mass, B and C tie (each one hop from A), and D is filling up in the residual as the two-hop sink. Continuing to push D and the leftover residual at A drives every residual under $\tau \deg^+$, at which point p is the local PPR estimate. We never touched a node outside $\{A, B, C, D\}$ – on a billion-node web the same run would still settle only a handful of vertices.

Trace it yourself

Carry the trace to completion. Keep pushing the largest over-threshold residual until none remain, and record the final p; normalize it and compare against ppr_A obtained from the closed form $(1-\alpha)(\mathbf{I} - \alpha\mathbf{P}')^{-1}e_A$ (here $\mathbf{P}' = \mathbf{P}$ since the graph has no dangling node; use any linear-algebra library). Then lower τ to 0.005 and watch the number of pushes grow roughly like $1/\tau$, exactly as the complexity bound predicts. Finally, re-seed with $s = D$ and

confirm the estimate now concentrates on B, C – the nodes D feeds – showing how PPR is a *directional*, source-anchored centrality.

36.7 Spam and robustness

PageRank's openness – anyone can create pages and links – is also its attack surface. Because a page's score is fed by its in-links, an adversary who controls many pages can arrange links to inflate a target's rank. This is *link spam*.

36.7.1 The link farm

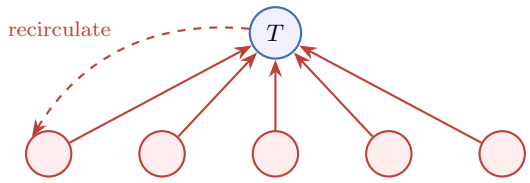


Figure 36.5: A link farm: many cheap pages funnel their teleport-seeded mass into target T. Each spam page collects $(1-\alpha)/n$ from teleport and forwards its share to T.

Intuition

Every page, however worthless, is *guaranteed* a baseline score of at least $(1-\alpha)/n$ from teleportation. Create f junk pages and point them all at T, and you have manufactured roughly $\alpha f \cdot (1-\alpha)/n$ of inflow for T out of thin air. Worse, if T links *back* into the farm, mass recirculates and amplifies (a geometric series in α). The defense the damping factor offers is the $(1-\alpha)$ leak: every cycle through the farm loses a $1-\alpha$ fraction to teleport, capping the amplification at $1/(1-\alpha)$.

Theorem 36.4 (Spam mass is bounded by the leak)**.** *Let $S \subseteq V$ be a set of pages an adversary fully controls (a link farm), and suppose all teleport is uniform. The total PageRank captured by S, namely $\pi(S) = \sum_{u \in S} \pi_u$, satisfies*

$$\pi(S) \;\le\; \frac{(1-\alpha)\,|S|/n \;+\; \alpha\,\mathrm{in}(S)}{1-\alpha},$$

where $\mathrm{in}(S) = \sum_{u \notin S} \pi_u \sum_{w \in S,\, u \to w} 1/\deg^+(u)$ is the legitimate inflow into S from outside. With no *legitimate inflow ($\mathrm{in}(S) = 0$), $\pi(S) \le \frac{|S|}{n}$: a farm of f pages cannot beat the score f* average *pages would get anyway.*

Proof sketch. Sum the stationary equation over $u \in S$. Mass entering S each step is: teleport $(1-\alpha)|S|/n$, plus α times link-mass from outside ($\alpha\,\mathrm{in}(S)$), plus α times link-mass that stays inside S ($\le \alpha\,\pi(S)$, since at most all of S's mass recirculates). At stationarity the mass on S equals the

mass entering it, so $\pi(S) \le (1-\alpha)|S|/n + \alpha\,\mathrm{in}(S) + \alpha\,\pi(S)$; solving for $\pi(S)$ gives the claim, and $\mathrm{in}(S) = 0$ yields $\pi(S) \le |S|/n$. □

Why it works

The theorem shows that teleportation by itself limits manipulation. Internal recirculation inside a farm is throttled by the factor $1/(1-\alpha)$; the only way to push $\pi(S)$ above the baseline $|S|/n$ is to acquire *legitimate* inflow $\mathrm{in}(S)$ – links from outside pages that themselves carry authentic PageRank. A spammer cannot easily automate getting links from high-reputation pages, so manipulating the ranking becomes expensive. A larger α raises the amplification cap $1/(1-\alpha)$, which is the second reason (beyond convergence speed) that production systems keep α moderate.

36.7.2 Defenses

- **TrustRank.** Seed teleport from a hand-vetted set of trustworthy pages instead of uniformly. Trust then propagates along links; spam farms, lacking inbound links from trusted seeds, never receive the seeded mass. This is personalized PageRank with a curated v.
- **Anti-TrustRank / BadRank.** Run the chain on the *reversed* graph from known-bad seeds to score how much a page links *toward* spam, then demote.
- **Spam-mass estimation.** Compare a page's PageRank under uniform teleport against its PageRank under a trusted teleport; a large gap flags artificially inflated pages (Gyongyi–Garcia-Molina).
- **Topical and supervised signals.** Modern ranking folds PageRank-like graph features into learned models with hundreds of other signals, so no single graph trick dominates – which itself dilutes the payoff of link spam.

Reading note

By the early 2000s, commercial search engines faced aggressive link-farm manipulation and "Google bombing" (coordinated anchor text designed to rank target pages for arbitrary queries). TrustRank (Gyöngyi, Garcia-Molina, and Pedersen, 2004) showed that teleporting toward vetted seed sites makes rankings more resistant to link-farm inflation.

36.8 The linear-algebra and Markov views, side by side

The table pairs each Markov-chain notion with its linear-algebra counterpart:

Markov-chain view	Linear-algebra view
Surfer's long-run occupancy	Dominant eigenvector of $\mathbf{G}$, eigenvalue 1
Ergodicity (every state reachable)	$\mathbf{G}$ strictly positive (Perron–Frobenius)
Mixing time of the chain	$O(1/\log(1/\alpha))$ from spectral gap $1-\alpha$
One simulated step of the surfer	One matrix–vector product $\mathbf{G}x$
Discounted walk from s	Neumann series $(1-\alpha)(\mathbf{I}-\alpha\mathbf{P}')^{-1}e_s$
Geometric$(1-\alpha)$ walk length	Eigenvalues scaled by α

Why this matters

Power iteration *is* simulating the chain in bulk: instead of running one surfer for a long time and counting visits (a Monte Carlo estimate of π), we propagate the entire probability cloud one step at a time and let the spectral gap collapse it onto π. The same object – the stationary distribution – is a fixed point to the algebraist, a long-run frequency to the probabilist, and a centrality score to the network scientist. PageRank has lasted partly because it makes sense in all three readings.

36.9 Historical Context and Controversies

History

The theoretical foundations of PageRank began in an early twentieth-century probability dispute. In 1906, Andrey Markov introduced sequences of dependent random variables to refute Pavel Nekrasov's theological claim that the law of large numbers applied only to completely independent events. To prove his point, Markov hand-tallied twenty thousand vowels and consonants across Alexander Pushkin's verse novel *Eugene Onegin*, showing that long-term stationary statistics govern systems with memory. Shortly after, Oskar Perron and Georg Frobenius established the spectral theory of non-negative matrices: any matrix with strictly positive real entries has a unique, simple maximum eigenvalue with a strictly positive eigenvector.

For nearly a century, Markov chains remained within physics and queueing theory. But in the late 1990s, the explosive growth of the World Wide Web created a serious problem for information retrieval. Early search engines ranked documents by text keyword frequency, which spammers quickly broke by stuffing popular words in invisible fonts on white backgrounds. Ranking by raw in-degree failed equally quickly, as bad actors could spin up thousands of dummy pages pointing to a scam site. At Cornell, Jon Kleinberg developed HITS (Hyperlink-Induced Topic Search) to separate hubs from authorities using alternating matrix iterates. At Stanford, Larry Page and Sergey Brin realized that links are endorsements, and modeled a web surfer wandering along links at random.

Raw web graphs immediately trip up a simple random walk with two structural flaws: dangling nodes (pages with no outgoing links) leak probability mass out of the walk, while spider traps (closed loops and dead-end cliques) swallow the walk permanently. Brin and Page's fix was to add a uniform restart chance: with probability $\alpha \approx 0.85$ the surfer follows a real hyperlink, and with probability $1-\alpha$ they teleport to an arbitrary page uniformly at random. This teleportation makes the entire transition matrix strictly positive. By the Perron–Frobenius

theorem, this ensures a unique, positive stationary distribution and creates a guaranteed spectral gap of $1-\alpha \approx 0.15$, allowing simple power iteration to converge to a robust global ranking in about fifty matrix multiplies regardless of web scale. What remains an open challenge is designing transparent ranking dynamics on adversarial, rapidly mutating graph streams without opening new manipulation vulnerabilities.

36.9.1 What We Can Now Do and Its Limitations

Connection

Capabilities Unlocked:

- We can compute a globally consistent, spam-resistant centrality score across billions of directed vertices in near-linear time $O(m \log(1/\varepsilon))$ using power iteration.
- The stationary distribution π is unique, strictly positive, and independent of initial conditions.
- Personalized PageRank and localized push algorithms allow computing targeted, local similarity scores centered at a query vertex without ever materializing or traversing the global graph.

Limitations:

- *Vulnerability to coordinated Sybil attacks*: While resistant to trivial spam, PageRank is vulnerable to coordinated link farms. A network of colluding pages with high internal reciprocal linking and a modest amount of incoming external authority can trap and amplify PageRank, requiring counter-measures like TrustRank or anti-collusion filtering.
- *Topic agnosticism and query blindness*: Global PageRank measures overall structural importance, not semantic relevance. Highly linked general hubs (like university or dictionary homepages) receive massive PageRank regardless of their relevance to a specialized technical query, requiring modern search systems to blend PageRank with semantic language models and click-through signals.
- *Continuous global recomputation*: As the web evolves continuously with millions of added and broken links every second, updating global PageRank requires massive distributed matrix-vector computation (pioneering distributed frameworks like MapReduce and Pregel).

Where We Go Next:

- In Chapter 37 (NP-Hard Graph Problems and Reductions), we turn to problems with no known polynomial-time algorithm. Spectral methods, random walks, and PageRank succeed because linear systems and eigenvector problems belong to the polynomial-time class **P**. But many important graph problems – finding Hamiltonian tours, minimum vertex covers, graph coloring, and maximum cuts – are **NP**-hard, so no polynomial-time algorithm for them is known.
- In Chapter 38 (Approximation Algorithms on Graphs), we learn how to design algorithms with provable performance guarantees for these **NP**-hard problems, using tools like semidefinite programming and randomized hyperplane rounding.

36.10 Exercises

Exercise 36.1 (Trace with teleportation changed)**.** Recompute the PageRank of Figure 36.1 with $\alpha = 0.5$, uniform teleport. Do two power-iteration steps by hand and report $x^{(2)}$. Confirm every score moved toward 0.25 relative to the $\alpha = 0.85$ answer, and explain why in one sentence.

Exercise 36.2 (Add a dangling node)**.** Insert a page E with an edge $A \to E$ and no out-edges of its own. Write the new sparse update rule including the dangling correction, then compute π to two iterations. What fraction of total mass sits on E at the fixed point, and where does E's mass go each step?

Exercise 36.3 (Eigenvalue check)**.** For the 4×4 matrix $\mathbf{P}$ of Example 36.1, verify numerically that the subdominant eigenvalue of $\mathbf{G} = 0.85\mathbf{P} + 0.15 \cdot \frac{1}{4}\mathbf{1}\mathbf{1}^\top$ has modulus ≤ 0.85, consistent with Theorem 36.2. (You may use any linear-algebra library.)

Exercise 36.4 (Prove the ℓ_1 contraction directly)**.** Show that for any zero-sum vector w ($\mathbf{1}^\top w = 0$), $\|\mathbf{G}w\|_1 \le \alpha\|w\|_1$, using the ℓ_1-contraction argument following Theorem 36.3 (split w into its positive and negative parts and track where each goes under one step of $\mathbf{G}$). Conclude Theorem 36.3 without diagonalizing.

Exercise 36.5 (Personalized PageRank as a walk)**.** For a source vertex s, prove that $\mathrm{ppr}_s(t) = (1-\alpha)\sum_{k\ge 0}\alpha^k \Pr[k\text{-step walk from } s \text{ ends at } t]$ equals the stationary distribution of the teleport-to-s chain. Then show $\mathrm{ppr}_s(t) = \mathrm{ppr}_t(s)$ does *not* generally hold on directed graphs, but does relate via a degree factor on undirected graphs.

Exercise 36.6 (Push correctness)**.** Prove the push invariant $\mathrm{ppr}_s = p + \sum_u r_u \,\mathrm{ppr}_u$ is preserved by one iteration of Algorithm 134. Then prove the $O\big(1/((1-\alpha)\tau)\big)$ bound on the total degree-weighted work by amortizing against settled mass.

Exercise 36.7 (Spam bound)**.** Complete the proof of Theorem 36.4 with full balance equations (do not skip the internal-recirculation term). Then construct a concrete farm of f pages with one legitimate inbound link of PageRank p_0 and compute the exact $\pi(S)$ as a function of α, f, n, p_0.

Exercise 36.8 (Break it, then fix it)**.** Design a small graph (at most 8 nodes) where a link farm of 3 nodes captures more than twice its fair share $3/n$ of PageRank by exploiting legitimate inflow plus internal recirculation. Then re-rank using a TrustRank teleport vector seeded on the two legitimate nodes and show the farm's captured mass collapses.

Exercise 36.9 (Monte Carlo PageRank)**.** Instead of power iteration, estimate π by simulating R independent surfers for T steps each on the Google matrix and tallying visit frequencies. Derive how large R must be for an additive ε estimate of a single π_u with confidence $1-\delta$ (Hoeffding). Compare the total work to power iteration and state when Monte Carlo wins (hint: when you only need a few coordinates).

Exercise 36.10 (Periodicity without teleport)**.** Build a directed graph whose raw walk matrix $\mathbf{P}$ is periodic (every node has period 2, e.g. a directed bipartite cycle) so raw power iteration oscillates and never converges. Show that any $\alpha < 1$ teleport restores convergence, and quantify how the oscillation amplitude decays as α^k.

Part VII

Hardness, Approximation, and the Modern Frontier

Chapter 37

NP-Hard Graph Problems and Reductions

Through the first thirty-six chapters, every algorithm ran in polynomial time: breadth-first search, Dijkstra, flows, matchings, and even the spectral sweeps and PageRank iterations of the preceding chapters operated within polynomial budgets. This chapter turns to the graph problems for which, despite decades of effort, no polynomial-time algorithm is known, and where complexity theory suggests none exists.

What is surprising is how *interconnected* these hard problems are. Hamiltonian cycle, traveling salesman, clique, independent set, vertex cover, graph coloring, max-cut, Steiner tree, longest path – these look like nine unrelated puzzles. Yet in a precise computational sense, they are polynomial-time translations of one another. A polynomial algorithm for any one of them would give polynomial algorithms for all of them, and for thousands of other problems besides. The equivalence comes from *reductions*: algorithms that translate one problem into another. Learning to build and read reductions is the most useful skill for telling, in practice, whether the problem on your desk is tractable or whether you should stop hunting for an exact polynomial algorithm and reach for approximation, parameterization, or heuristics.

We build the theory the way we built flows: pictures and intuition first, then the gadgets that make reductions work, then full proofs of the main hardness results, then the narrow boundary between easy and hard. The companion algorithms book introduced P, NP, and NP-completeness abstractly. Here we make it concrete and graphical.

37.1 What Hardness Means

We work with *decision problems*: questions with a yes/no answer. "Does G have a clique of size $\geq k$?" is a decision problem; "what is the largest clique?" is its optimization cousin. Decision is the right currency for complexity theory because yes/no answers compose cleanly under reduction, and because an optimization problem is rarely harder than polynomially many decision queries (binary search on k).

Definition 37.1 (The class NP)**.** A decision problem Π is in NP if there is a polynomial-time *verifier* V and a polynomial p such that for every yes-instance x there exists a *certificate* (witness) w with

$|w| \le p(|x|)$ and $V(x, w) = \mathsf{accept}$, and for every no-instance x no string w makes $V(x, w)$ accept.

Intuition

NP is the class of problems where a *yes* answer has a short, checkable proof. If I claim G has a Hamiltonian cycle, I hand you the cyclic order of the vertices; you verify in linear time that consecutive vertices are adjacent and that each vertex appears once. You do not have to find the cycle – only check the one I show you. Finding is (apparently) hard; checking is easy. The central question of P versus NP is whether "easy to check" always implies "easy to find."

$\mathsf{P} \subseteq \mathsf{NP}$: if you can solve in polynomial time you can verify by ignoring the certificate and re-solving. The million-dollar question is whether the inclusion is strict. Almost everyone believes $\mathsf{P} \neq \mathsf{NP}$, but it is unproven.

Definition 37.2 (Polynomial-time many-one reduction)**.** A problem A *reduces* to a problem B, written $A \le_p B$, if there is a polynomial-time computable function f mapping instances of A to instances of B such that

$$x \text{ is a yes-instance of } A \iff f(x) \text{ is a yes-instance of } B.$$

Read $A \le_p B$ as "A is no harder than B, up to polynomial overhead." If B has a polynomial algorithm, compose it with f to solve A in polynomial time. Contrapositive, the way we actually use it: if A is hard, then B is hard too. Reductions transport hardness *forward* along the arrow and tractability *backward*.

Common trap

The direction trips up everyone at least once. To prove your new problem B is hard, you reduce a *known*-hard problem A *to* B ($A \le_p B$): you show B is at least as hard as a problem already proved NP-hard. Reducing B to A proves nothing about B's hardness – it would only show B is easy if A is. Mnemonic: "reduce *from* the known hard problem *to* your target."

Definition 37.3 (NP-hard, NP-complete)**.** A problem B is NP-*hard* if $A \le_p B$ for every $A \in \mathsf{NP}$. It is NP-*complete* if additionally $B \in \mathsf{NP}$. The NP-complete problems are the hardest problems in NP: a polynomial algorithm for any one collapses $\mathsf{P} = \mathsf{NP}$.

Reading note

In 1971 Stephen Cook (and independently Leonid Levin) proved that Boolean satisfiability is NP-complete by simulating arbitrary Turing-machine verifiers with propositional formulas. Richard Karp (1972) then proved 21 foundational combinatorial problems are NP-complete via a web of polynomial-time reductions, establishing the standard reduction methodology.

Reduction is transitive: $A \le_p B$ and $B \le_p C$ give $A \le_p C$ (compose the two polynomial maps; a polynomial of a polynomial is a polynomial). So once *one* problem is known NP-hard, we never again climb all the way back to the definition. We reduce from a problem already in the club, so the club grows one reduction at a time. We take 3-SAT's NP-completeness as our anchor (it is Cook-Levin plus one standard step) and grow the graph wing of the club from there.

Focus

The plan of this chapter. Anchor at 3-SAT. Reduce 3-SAT → Independent Set → Vertex Cover ≡ Clique. Separately reduce 3-SAT → 3-Coloring. Reduce 3-SAT (via a Hamiltonicity gadget) to Hamiltonian Cycle → TSP, and to Longest Path. Touch Max-Cut and Steiner Tree. Then study the *boundary*: why 2-coloring, 2-SAT, shortest paths, and matching stay in P while their natural generalizations become NP-complete.

37.2 The Anchor: 3-SAT

A Boolean formula in *conjunctive normal form* (CNF) is an AND of clauses, each clause an OR of literals, each literal a variable x_i or its negation $\bar{x}_i$. 3-SAT asks: given a CNF formula with exactly three literals per clause, is there an assignment of true/false to the variables making every clause true?

Theorem 37.1 (Cook-Levin, 3-SAT form)**.** 3-SAT *is* NP*-complete.*

We do not reprove Cook-Levin from the Turing-machine tableau; the companion algorithms book does that. We take it as given and note only the easy half: 3-SAT$\in$ NP because a satisfying assignment is a certificate checkable in linear time. From here on, every hardness proof is a *gadget construction*: turn variables and clauses into pieces of a graph.

Intuition

A reduction from 3-SAT is a translation with two halves. *Variable gadgets* force a binary choice – true or false – one per variable. *Clause gadgets* check that at least one literal in each clause got satisfied, and wire into the variable gadgets so they can "see" the choices. The key is to make the gadgets local: each enforces its own little constraint, and the global structure of the target problem (a large independent set, a proper coloring, a Hamiltonian cycle) exists if and only if all the local constraints can be met simultaneously – which is exactly satisfiability.

37.3 Independent Set, Vertex Cover, and Clique

These three problems are essentially one problem. Fix a graph $G = (V, E)$.

- An *independent set* is a set $S \subseteq V$ with no edge inside it. Independent-Set: is there one of size $\geq k$?
- A *vertex cover* is a set $C \subseteq V$ touching every edge (each edge has at least one endpoint in C). Vertex-Cover: is there one of size $\leq k$?
- A *clique* is a set $K \subseteq V$ that is pairwise adjacent. Clique: is there one of size $\geq k$?

Lemma 37.1 (The complementarity triangle)**.** *For any graph G on n vertices and any $S \subseteq V$:*

1. *S is independent in G $\iff$ $V \setminus S$ is a vertex cover of G.*
2. *S is independent in G $\iff$ S is a clique in the complement $\bar{G}$.*

Hence G has an independent set of size $\geq k$ iff it has a vertex cover of size $\leq n-k$ iff $\bar{G}$ has a clique of size $\geq k$.

Proof. (1) S independent means no edge has both endpoints in S, i.e. every edge has an endpoint outside S, i.e. $V \setminus S$ covers every edge – and the chain of "i.e."s is reversible. (2) An edge inside S in G is a non-edge inside S in $\bar{G}$, so S edge-free in $G \iff S$ complete in $\bar{G}$. The size statements follow since $|V \setminus S| = n - |S|$, and complementing the graph is a polynomial-time, instance-preserving map. □

So these three are interreducible by trivial transformations: complement the graph, or complement the set. Figure 37.1 shows both transformations side by side on a four-vertex paw graph. Prove one hard and all three are hard. We prove Independent-Set hard by reducing from 3-SAT.

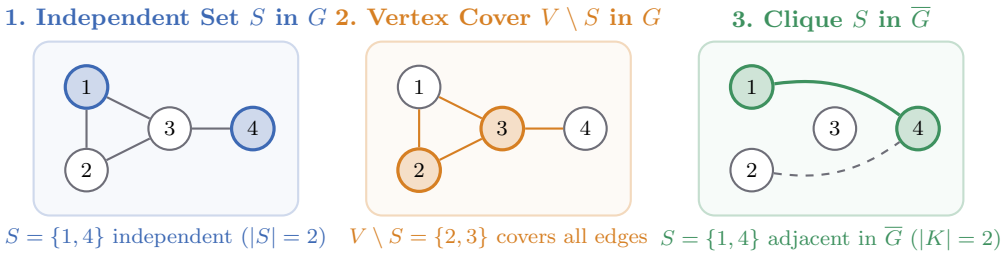


Figure 37.1: The complementarity triangle (Lemma 37.1) on a 4-vertex graph G. Taking the vertex complement turns the independent set $S = \{1, 4\}$ into the vertex cover $V \setminus S = \{2, 3\}$ of G; taking the edge complement turns the same set $S = \{1, 4\}$ into a clique in $\overline{G}$.

37.3.1 The gadget

Theorem 37.2. Independent-Set *is* NP*-complete.*

Proof. Membership: a set of k pairwise-nonadjacent vertices is a certificate checkable in $O(k^2)$ time. Hardness: reduce 3-SAT. Let φ have m clauses $C_1, \ldots, C_m$, each with three literals. Build a graph G_φ:

- **Clause triangles.** For each clause $C_j = (\ell_{j1} \vee \ell_{j2} \vee \ell_{j3})$ create three vertices, one per literal occurrence, and join them in a triangle. A triangle admits at most one vertex in any independent set – this forces us to pick exactly one "satisfying literal" per clause.
- **Conflict edges.** For every pair of literal-vertices that are *negations* of each other (x_i in one place, $\bar{x}_i$ in another), add an edge. This forbids choosing both x_i and $\bar{x}_i$ – it keeps our choices consistent with a single assignment.

Set $k = m$. We claim φ is satisfiable $\iff$ G_φ has an independent set of size m. □

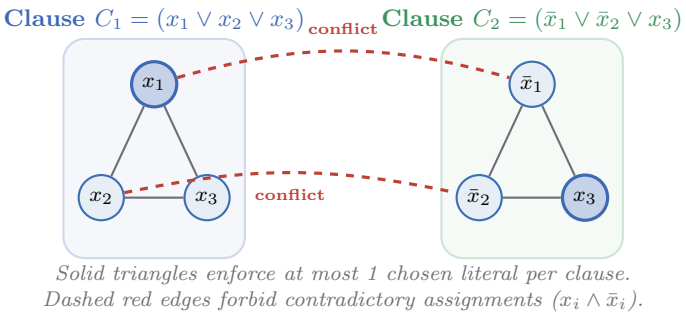


Figure 37.2: The 3-SAT to Independent Set reduction for $(x_1 \vee x_2 \vee x_3) \wedge (\bar{x}_1 \vee \bar{x}_2 \vee x_3)$. Each clause is a solid triangle requiring at least two vertices to be covered (hence at most one in the independent set). Dashed conflict edges connect complementary literals. An independent set of size $m = 2$ (shaded blue: x_1 and x_3) corresponds to a consistent satisfying assignment (setting $x_1 = \text{true}, x_3 = \text{true}$).

Why it works

Why the construction is correct. ($\Rightarrow$) Take a satisfying assignment. Each clause has at least one true literal; pick one such literal-vertex per triangle. That is m vertices, one per triangle, so no triangle edge is violated. Two chosen vertices can only conflict via a dashed edge, which connects x_i to $\bar{x}_i$ – but both being chosen would mean both literals are true under the same assignment, impossible. So the m vertices are independent.
($\Leftarrow$) Take an independent set S of size m. Each triangle holds at most one vertex of S, and there are m triangles, so S holds *exactly* one vertex per triangle. Define an assignment by setting each chosen literal true (if x_i's vertex is chosen set x_i true; if $\bar{x}_i$'s vertex is chosen set x_i false). No conflict edge is internal to S, so we never demand x_i both true and false – the assignment is well defined. Variables not pinned by any chosen literal get an arbitrary value. Every clause has its chosen literal true, so φ is satisfied.

The reduction builds $3m$ vertices and at most $3m + \binom{3m}{2}$ edges in $O(m^2)$ time – polynomial. By Lemma 37.1, Vertex-Cover and Clique are NP-complete too.

Trace it yourself

Trace the formula $\varphi = (x_1 \vee x_2 \vee x_3) \wedge (\bar{x}_1 \vee \bar{x}_2 \vee x_3)$ from Figure 37.2. We want an independent set of size 2. Try the assignment $x_3 = \text{true}$ (with x_1, x_2 free). Pick the x_3 vertex in C_1 and the x_3 vertex in C_2: no triangle edge between them, no conflict edge (both are the literal x_3, not negations), so $\{x_3^{C_1}, x_3^{C_2}\}$ is independent of size 2. The reduction confirms φ is satisfiable (indeed $x_3 =$true satisfies both clauses). Now try to force a conflict: pick x_1 in C_1 and $\bar{x}_1$ in C_2 – the dashed edge between them blocks it, exactly as a consistent assignment must.

Algorithm 135: REDUCE-3SAT-TO-INDEPENDENTSET(φ)

Input: CNF formula φ with clauses $C_1, \dots, C_m$, three literals each.
Output: Graph G and integer k with φ sat $\iff$ G has IS of size k.

```
V ← ∅;  E ← ∅
foreach clause C_j = (ℓ_j1 ∨ ℓ_j2 ∨ ℓ_j3) do
    create vertices v_j1, v_j2, v_j3 labeled by the literals
    add triangle edges {v_j1v_j2, v_j2v_j3, v_j3v_j1} to E
foreach pair u, v with label(u) = x_i, label(v) = x̄_i do
    add edge uv to E                                   // conflict edge
k ← m
return (G = (V, E), k)
```

Cost summary

The map runs in $O(m^2)$ time and produces $O(m)$ vertices, $O(m^2)$ edges. What it buys: INDEPENDENT-SET, VERTEX-COVER, and CLIQUE are NP-complete. No polynomial exact algorithm is known; the textbook brute force is $O(2^n \cdot \text{poly}(n))$, improvable to roughly $O(1.2^n)$ for INDEPENDENT-SET by clever branching.

37.4 Graph Coloring

A *proper c-coloring* assigns each vertex one of c colors so that adjacent vertices differ. c-COLORING: can G be properly colored with c colors? The chromatic number $\chi(G)$ is the least feasible c.

The jump from $c = 2$ to $c = 3$ is the cleanest easy/hard boundary in the subject, so it earns its own section (§37.9). Here we prove the hard side.

Theorem 37.3. 3-COLORING *is* NP*-complete.*

Construction. Membership: a coloring is a certificate, checked by scanning edges. Hardness: reduce 3-SAT. Colors are named T, F, B ("base"). The reduction uses three kinds of pieces.

Palette triangle. Three special vertices T, F, B joined in a triangle. In any 3-coloring they receive all three colors; rename so T gets T, F gets F, B gets B. They anchor the meaning of the colors.

Variable gadget. For each variable x_i, two vertices x_i and $\bar{x}_i$ joined to each other and both joined to B. The triangle $x_i, \bar{x}_i, B$ forces x_i and $\bar{x}_i$ to take the two colors $\{\mathsf{T}, \mathsf{F}\}$ in some order – exactly a Boolean choice, with x_i and $\bar{x}_i$ guaranteed opposite.

Clause gadget (OR gadget). For a clause $(a \vee b \vee c)$ attach a small widget that is 3-colorable *iff* at least one of a, b, c is colored T. The widget is the classic "OR tree": two binary OR-gadgets chained, the output forced adjacent to F and B. □

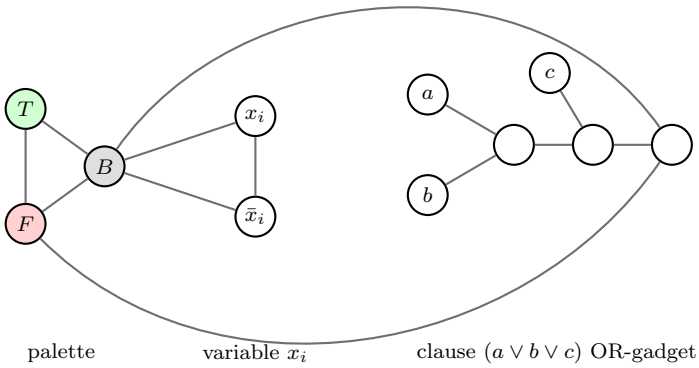


Figure 37.3: 3-coloring gadgets. The palette triangle fixes the three colors. Each variable's $x_i, \bar{x}_i$ are forced opposite by the triangle with B. The OR-gadget output can be colored T only when some input is T; tying the output to F and B forbids the all-F case.

Why it works

Why the OR-gadget works. The atomic OR-gadget is a small graph on inputs p, q and an output node r with the property: in any 3-coloring with palette fixed, r can be colored T iff p or q is T, and if both p, q are F then r is forced to F. Chaining two of them computes the OR of three literals. Finally we make the clause's output vertex adjacent to both F (color F) and B (color B), which forbids the output from being F or B – so it must be T, which is possible exactly when some literal of the clause is T, i.e. true. A formula is satisfiable iff every clause-output can be forced T iff the whole graph is 3-colorable.

The construction has $O(n + m)$ vertices and edges and is built in linear time. 3-Coloring is NP-complete, and for every fixed $c \geq 3$, c-Coloring is NP-complete (add a $(c-3)$-clique joined to the whole graph to lift 3 to c).

Application

Register allocation in compilers is graph coloring: vertices are program variables alive at the same time, edges mean "cannot share a register," colors are physical registers. Frequency assignment for radio towers, exam timetabling (colors are time slots, edges are conflicts), and Sudoku (extending a pre-colored 81-vertex conflict graph to a proper 9-coloring) are all direct instances of graph coloring. Because exact coloring is NP-hard, real compilers use the greedy + spilling heuristics of Chaitin-Briggs rather than optimal colorings.

37.5 Hamiltonian Cycle and the Traveling Salesman

A *Hamiltonian cycle* visits every vertex exactly once and returns to start. Ham-Cycle: does G have one? Compare it with the *Eulerian* cycle (visit every *edge* once), which is decidable in linear

time – a graph is Eulerian iff it is connected with all degrees even. Swapping "edge" for "vertex" turns a linear-time problem into an NP-complete one. The local-to-global structure that made Euler easy (degree parity is a purely local certificate) has no analogue for Hamiltonicity.

Reading note

The Hamiltonian cycle is named for William Rowan Hamilton's 1857 "Icosian Game" on a dodecahedron. Euler's 1736 bridge problem (visiting every edge) is solvable in linear time, but Hamilton's problem (visiting every vertex) is NP-complete.

Theorem 37.4. HAM-CYCLE *(directed and undirected) is* NP*-complete.*

Sketch with the key gadget. Membership: the cyclic vertex order is a linear-time-checkable certificate. Hardness: reduce 3-SAT (directed version; an undirected version follows by replacing each directed vertex with a 3-vertex path that enforces direction). The reduction uses a *choice-and-check* layout.

Variable chains. For each variable x_i build a horizontal chain of $2m$ vertices that the Hamiltonian cycle must traverse. Traversing it left-to-right encodes x_i = true; right-to-left encodes x_i = false. The chains are stacked and linked into one big loop, so the cycle threads each chain in one of two directions – a global binary choice per variable.

Clause vertices. For each clause C_j add a single vertex c_j. Wire c_j to a position on the chain of each variable appearing in C_j, oriented so that the cycle can detour into c_j and back *only* when the corresponding chain is traversed in the direction that satisfies the literal. Thus c_j can be visited iff at least one of its literals is true. □

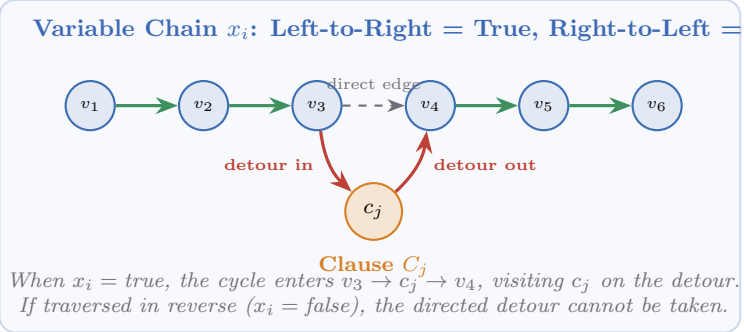


Figure 37.4: The Hamiltonian path/cycle gadget for 3-SAT. Each variable x_i is mapped to a horizontal directed chain. If x_i is set to true, the cycle moves left-to-right and can take the detour $v_3 \to c_j \to v_4$ to visit the clause vertex c_j. If the chain is traversed right-to-left (x_i = false), the directed arrows prevent taking this detour. Thus, c_j can be visited without breaking the cycle if and only if at least one of its literals evaluates to true.

Hamiltonicity leads to the best-known hard optimization problem.

Definition 37.4 (Traveling Salesman)**.** TSP: given n cities and pairwise distances $d(i, j)$ and a bound L, is there a tour visiting every city once with total length $\leq L$?

Theorem 37.5. TSP *is* NP*-complete, even in the* metric *case where distances obey the triangle inequality.*

Proof. Reduce Ham-Cycle. Given $G = (V, E)$, build a complete graph on V with

$$d(i,j) = \begin{cases} 1 & \{i,j\} \in E, \\ 2 & \{i,j\} \notin E, \end{cases}$$

and set the bound $L = n$. Distances are 1 or 2, so the triangle inequality holds automatically ($d(i,k) \le 2 \le d(i,j) + d(j,k)$), giving the metric case for free. A tour visits n edges. Its length is n iff every edge it uses has weight 1 iff every consecutive pair is a real edge of G iff the tour is a Hamiltonian cycle of G. So G is Hamiltonian $\iff$ the TSP instance has a tour of length $\le n$. The map is $O(n^2)$. □

Common trap

Ham-Cycle reduces to TSP the easy way (weights 1/2), which *also* settles metric TSP. But *general* TSP – arbitrary weights, no triangle inequality – is much worse to approximate: encode Hamiltonicity with weight 1 for edges and weight $n \cdot \rho$ for non-edges, and a tour shorter than $n\rho$ exists iff G is Hamiltonian. So a polynomial ρ-approximation to general TSP would decide Hamiltonicity – general TSP has *no* constant-factor approximation unless P = NP. Metric TSP, by contrast, has the Christofides 3/2-approximation. The triangle inequality is what separates them; do not conflate the two problems.

37.6 Longest Simple Path

Shortest path was solved in polynomial time in Chapters 7–9: BFS, Dijkstra, Bellman-Ford, Floyd-Warshall, all polynomial. Longest-Path – given G and k, is there a *simple* path with $\ge k$ edges? – is NP-complete.

Theorem 37.6. Longest-Path *is* NP*-complete, even on unweighted graphs.*

Proof. Membership: a path is a checkable certificate. Hardness: reduce Ham-Path (a Hamiltonian path, the open-ended cousin of Ham-Cycle, is also NP-complete by the same gadget without closing the loop). A graph on n vertices has a Hamiltonian path iff it has a simple path with $\ge n - 1$ edges. Set $k = n - 1$; the identity map is the reduction. □

Why it works

Why does "longest" break what "shortest" kept? The difference is the word *simple*. Shortest paths are automatically simple – a cycle only adds nonnegative length, so optimal solutions never repeat a vertex, and that lets dynamic programming over distances work: the prefix of a shortest path is a shortest path (optimal substructure). Longest *simple* paths have no such substructure: the longest path from s to v may not extend the longest path from s to u, because reusing vertices is forbidden and the optimal prefix may "waste" a vertex you needed later. The moment you drop simplicity to recover substructure, the longest *walk* blows up to $+\infty$ whenever a cycle is reachable (every traversal of it adds length), so the walk relaxation answers a different, useless question. Either way DP fails. Negative cycles break shortest paths for exactly the mirror-image reason; the two pathologies are dual.

Intuition

A simple way to see the boundary: longest path on a DAG *is* polynomial (topological-order DP), because a DAG has no cycles to repeat – simplicity is free. So Longest-Path is easy on DAGs and hard on general graphs. The hardness comes from the interaction between cycles and the simplicity constraint.

37.7 Max-Cut and Steiner Tree

Max-Cut. Partition V into $(S, \bar{S})$ to maximize the number of edges crossing. Decision: is there a cut of size $\geq k$? Its minimization counterpart, *Min-Cut* (Chapters 11–12), is polynomial via max-flow.

Theorem 37.7. Max-Cut *is* NP*-complete.*

Idea. Reduce from Not-All-Equal-3-SAT (NAE-3SAT), a known NP-complete variant of 3-SAT. In NAE-3SAT, a clause is satisfied if its three literals do not all take the same truth value – that is, they cannot be all-true or all-false. This requirement translates directly into a cut: think of the two sides of the cut $(S, \bar{S})$ as representing truth values true and false. A clause triangle with vertices for its three literals cuts at most 2 edges, and it cuts *exactly* 2 edges if and only if the three literals are not all placed on the same side (Figure 37.5). Thus, maximizing the cut size forces every clause triangle to contribute its maximum possible cut of 2, which happens precisely when a valid not-all-equal truth assignment exists. □

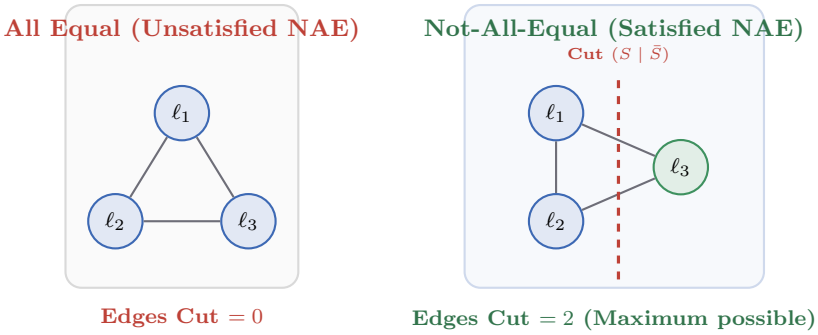


Figure 37.5: The clause triangle gadget in the reduction from NAE-3SAT to Max-Cut. Each clause is represented by a triangle of literal vertices. Left: If all three literals take the same truth value, they lie on the same side of the partition and 0 edges are cut. Right: If the clause is not-all-equal, the cut separates one literal from the other two, cutting exactly 2 edges. Maximizing the total cut forces every clause to achieve this maximum.

Connection

Max-Cut connects to two later chapters. Its *spectral* relaxation – maximize $\frac{1}{4}x^\top \mathbf{L}x$ over $x \in \{\pm 1\}^n$, where $\mathbf{L}$ is the graph Laplacian (Chapter 31) – connects cut size to the eigenvalues of $\mathbf{L}$ and to Cheeger-type bounds (Chapter 32). Relaxing $\{\pm 1\}^n$ to unit vectors gives the Goemans-Williamson semidefinite program and its famous 0.878-approximation (Chapter 38). So the same problem that is NP-hard here becomes a question about Laplacian quadratic

forms and rounding hyperplanes there.

Steiner Tree. Given a graph with edge weights and a subset $R \subseteq V$ of *terminals*, find a minimum-weight connected subgraph containing all of R. Decision: weight $\leq L$? When $R = V$ this is the (polynomial) minimum spanning tree; when $|R| = 2$ it is shortest path. The general case – choose which non-terminal "Steiner" vertices to include – is NP-complete, by reduction from Exact-Cover-by-3-Sets or from Vertex-Cover.

Intuition

Steiner tree shows that the hardness is not about "all" or "two" but about the combinatorial choice in between: which optional vertices to recruit. MST (recruit everyone) and shortest path (recruit no one) are the easy endpoints; the hardness comes from the exponentially many intermediate recruitment choices (Figure 37.6). There is a metric $\ln 4+\varepsilon \approx 1.39$-approximation (Byrka et al.) and a classic 2-approximation via the terminal metric's MST.

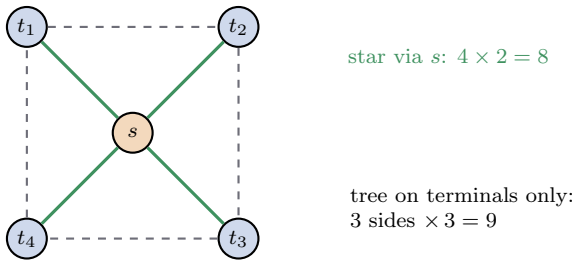


Figure 37.6: Why an optional vertex helps. Terminals $t_1, \ldots, t_4$ (blue) must be connected; s (orange) is a non-terminal that may be recruited. Solid green spokes cost 2 each, dashed perimeter edges cost 3 each. The Steiner star through s costs 8; the cheapest tree using only terminal edges needs three perimeter sides and costs 9. Choosing *whether* to include s is the combinatorial decision that makes the general problem hard.

37.8 A Map of the Reductions

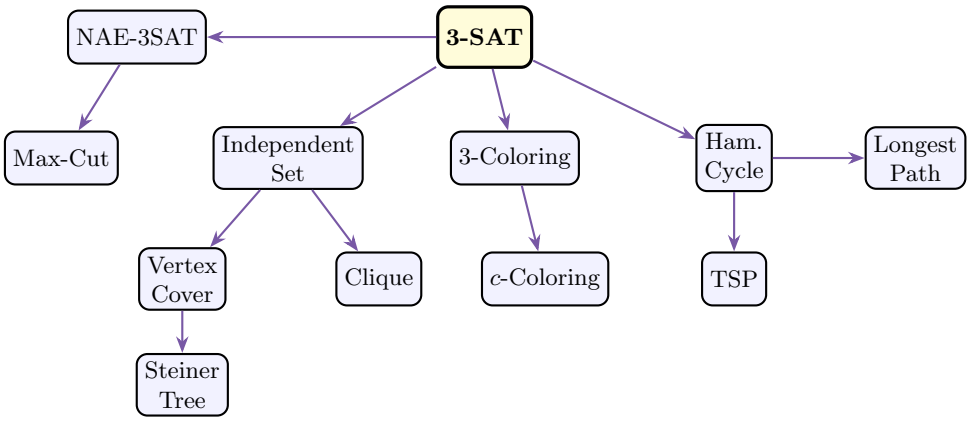


Figure 37.7: The graph wing of Karp's web. Every arrow is a polynomial reduction proved or sketched in this chapter; hardness flows along the arrows out of the 3-SAT anchor. A polynomial algorithm for any single box would imply $\mathsf{P} = \mathsf{NP}$.

37.9 The Boundary Between Easy and Hard

The most instructive part of this subject is *where* the line between easy and hard falls. Tiny changes in a problem statement – 2 versus 3, "edge" versus "vertex," shortest versus longest – flip tractability. We look at five boundary cases and the reason for each.

37.9.1 2-Coloring vs 3-Coloring

2-Coloring is polynomial: a graph is 2-colorable iff it is *bipartite* iff it has no odd cycle, all checkable by a single BFS/DFS that 2-colors greedily and reports failure on a conflicting edge (Figure 37.8).

Algorithm 136: Two-Color(G) – bipartiteness by BFS

Input: Undirected graph $G = (V, E)$.
Output: A proper 2-coloring, or not 2-colorable.

```
color[v] ← none for all v
foreach vertex s with color[s] = none do
    color[s] ← 0;   enqueue s
    while queue nonempty do
        u ← dequeue
        foreach neighbor w of u do
            if color[w] = none then
                color[w] ← 1 − color[u];   enqueue w
            else if color[w] = color[u] then
                return not 2-colorable
return color
```

Why it works

The algorithm is forced at every step: once s is color 0, all neighbors *must* be 1, theirs 0, and so on – the colors propagate with zero freedom. A conflict (an edge whose endpoints land on the same color) is reachable iff the BFS tree plus that edge forms an odd cycle, and odd cycles are precisely the obstruction to bipartiteness. With *three* colors the propagation is no longer forced: at a vertex with one colored neighbor you have two free choices, and those branching choices are exactly the combinatorial explosion that 3-SAT injects through the gadget of §37.4. Two colors leave no slack; three colors leave enough slack to encode arbitrary computation.

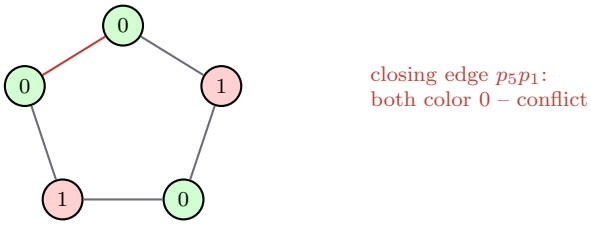


Figure 37.8: Two-Color on the 5-cycle. BFS from p_1 forces the colors $0, 1, 0, 1, 0$ around the path $p_1p_2p_3p_4p_5$; the closing edge p_5p_1 then joins two 0's, so Algorithm 136 reports not 2-colorable. The obstruction is exactly the odd cycle.

37.9.2 2-SAT vs 3-SAT

2-SAT (two literals per clause) is polynomial; 3-SAT is NP-complete. The difference is structural: a 2-clause $(a \vee b)$ is equivalent to two implications $\bar{a} \Rightarrow b$ and $\bar{b} \Rightarrow a$. Build the *implication graph* on the $2n$ literals; then φ is satisfiable iff no variable x_i has x_i and $\bar{x}_i$ in the same strongly connected component. That is a single SCC computation (Chapter 5), $O(n + m)$.

Intuition

A 2-clause forces a chain reaction: setting one literal cascades deterministically through implications, like the 2-coloring propagation. A 3-clause $(a \vee b \vee c)$ gives only the weak implication $(\bar{a} \wedge \bar{b}) \Rightarrow c$ – you must already know *two* values before it constrains the third. That "need two to pin one" is genuine disjunctive choice, and choice is what makes search exponential. The recurring pattern across all five boundaries: hardness appears exactly when local constraints stop forcing unique propagation and start offering free binary choices.

37.9.3 Shortest vs Longest Path

Covered in §37.6: shortest paths have optimal substructure (prefixes of optimal solutions are optimal) because nonnegative cycles never help; longest *simple* paths lose substructure to the simplicity constraint. On DAGs, where simplicity is automatic, longest path is polynomial again – the boundary is the presence of cycles.

37.9.4 Min-Cut vs Max-Cut

Min-Cut is polynomial (max-flow duality, Chapter 12; Karger's randomized global cut, Chapter 11). Max-Cut is NP-hard (§37.7). Minimizing a cut is a flow problem with LP duality and integral optima; maximizing the cut is equivalent to a ± 1 quadratic optimization with no such duality – the feasible region's geometry is convex for min and combinatorial for max.

37.9.5 Matching vs 3-Dimensional Matching

Maximum matching in graphs is polynomial – Edmonds' blossom algorithm (Chapter 17). 3-Dimensional-Matching (perfectly pair up triples from three disjoint sets) is NP-complete (Karp). Two-way matching has the augmenting-path / LP-integrality structure; the three-way version does not, and it is itself a primary source for reductions to Steiner tree, set cover, and scheduling.

Easy (in P)	**Hard (NP-complete)**	**What flips it**
2-Coloring (bipartite)	3-Coloring	forced vs free propagation
2-SAT	3-SAT	implication vs disjunction
Shortest path	Longest simple path	cycles + simplicity
Eulerian cycle	Hamiltonian cycle	edges vs vertices
Min-cut	Max-cut	flow duality vs ± 1 quadratic
Matching (2D)	3D matching	augmenting paths vs none
MST ($R = V$) / SP ($\lvert R\rvert = 2$)	Steiner tree	optional-vertex choice

Table 37.1: The easy/hard frontier. Each "hard" problem is one small modification away from a polynomial sibling; the right column names the structural feature that disappears.

37.10 What Hardness Means for Algorithm Design

A proof that your problem is NP-hard is useful guidance: it tells you to stop looking for an exact polynomial algorithm and pick one of the legitimate escape routes, each developed in its own later chapter.

- **Approximation** (Chapter 38). Settle for provably near-optimal. Vertex-Cover has a 2-approximation (take both endpoints of a maximal matching); metric TSP has Christofides' 3/2; Max-Cut has Goemans-Williamson's 0.878. Some problems resist: general TSP and Clique have no good approximation unless P = NP (the latter by the PCP theorem).
- **Parameterization** (Chapters 24–25). Confine the exponential blow-up to a parameter k that is small in practice. Vertex-Cover is fixed-parameter tractable in $O(1.27^k + kn)$ – a bounded-search-tree algorithm – so it is solvable exactly when the cover is small even if the graph is huge. Clique and Independent-Set, by contrast, are W[1]-hard: no such $f(k) \cdot \text{poly}(n)$ algorithm is expected.
- **Structural restriction.** Many hard problems collapse to polynomial on bounded-treewidth graphs (Chapter 24), planar graphs (Chapter 26; Baker's technique gives PTASs), or interval/chordal/perfect graphs (Chapter 30, where coloring and clique become polynomial). If your inputs have structure, exploit it.

- **Exact exponential algorithms.** When n is modest, an $O(2^n)$ Held-Karp DP solves TSP, and branch-and-bound with good pruning routinely handles thousands of cities. Hardness is asymptotic; small instances are fair game.
- **Heuristics and SAT/ILP solvers.** Encode back into SAT or integer programming and run an industrial solver. Modern CDCL SAT solvers and MIP engines solve very large structured instances despite worst-case hardness.

Why this matters

The main lesson of this chapter is how closely these problems are linked. Nine graph problems that arose in nine different decades and applications – routing, scheduling, register allocation, network design – turned out to be computationally equivalent, linked together by polynomial-time reductions. Proving one hard proves them all hard; finding one easy would make them all easy. That web is why "NP-complete" carries so much weight: it is a structural claim about thousands of problems at once. When you recognize your problem as a member, you inherit fifty years of accumulated knowledge about how to cope.

Connection

This chapter is the graph-flavored half of the NP-completeness chapter in the companion algorithms book. That book proves Cook-Levin from the Turing-machine tableau and develops SAT, subset-sum, and partition; here we took 3-SAT as a black box and grew the graph wing – Independent Set, Coloring, Hamiltonicity, TSP, Longest Path, Max-Cut, Steiner Tree – by gadget reductions. The escape routes (approximation, FPT, structural restriction) are the subject of Chapters 24, 25, 26, 30, and 38. Max-Cut in particular threads forward into the spectral and Laplacian chapters (31–32), where its relaxation is studied in terms of eigenvalues and electrical flows.

37.11 Historical Context and Controversies

History

The mathematical realization that certain computational problems might be intrinsically intractable began with Kurt Gödel. In a 1956 letter to John von Neumann, Gödel asked whether finding mathematical proofs of length n could be decided mechanically in $O(n)$ or $O(n^2)$ steps, noting that if so, the discovery of mathematical truth could be completely automated. Throughout the 1950s and 1960s, researchers in operations research repeatedly got stuck: while linear programming, spanning trees, and shortest paths yielded to polynomial algorithms, seemingly modest variations – such as requiring integer solutions, finding a traveling salesman tour, or seeking a maximum clique – stumped every algorithmic attempt.

In 1971, Stephen Cook formalized the class **NP** and proved that Boolean Satisfiability (SAT, deciding whether a propositional formula can be made true) is **NP**-complete, showing that every problem whose solutions can be verified in polynomial time reduces to SAT. Working independently in the Soviet Union, Leonid Levin published similar results in 1973 on universal search problems. The full impact of Cook's insight was demonstrated by Richard Karp in 1972, who proved that 21 central combinatorial problems – including Vertex Cover,

Clique, Hamiltonian Cycle, Coloring, and Max-Cut – are all **NP**-complete, showing that computational hardness is widespread in discrete mathematics rather than an oddity of logic.
Before Cook and Karp, researchers spent careers searching for polynomial-time algorithms for these problems, attributing failure to personal shortcoming rather than to a mathematical obstacle. A persistent trap is confusing the hardness of *search* with the ease of *verification*: checking a proposed certificate takes polynomial time, but finding it through exhaustive search requires exploring exponentially many configurations. Proving **NP**-completeness redirected theoretical energy toward approximation, fixed-parameter tractability, and heuristics. The main open question in theoretical computer science remains the **P** versus **NP** problem itself: proving mathematically that search is strictly harder than verification.

37.11.1 What We Can Now Do and Its Limitations

Connection

Capabilities Unlocked:

- We have a general method for proving that a graph problem is **NP**-hard. A polynomial reduction from a known **NP**-complete problem proves that finding an exact polynomial algorithm is as hard as solving every problem in **NP**.
- Proving **NP**-hardness tells algorithm designers to stop searching for exact polynomial algorithms and turn to practical alternatives: approximation algorithms, parameterized complexity (**FPT**), and structural exploitation (e.g., planarity, bounded treewidth).

Limitations:

- *Worst-case pessimism*: **NP**-completeness is strictly a worst-case classification. A problem can be **NP**-hard even though 99% of real-world instances encountered in practice can be solved in seconds by modern SAT solvers or integer programming packages.
- *Reduction discipline*: Beginners frequently execute reductions in reverse. Reducing a known polynomial problem (such as Max-Flow) to an unknown problem proves nothing about the unknown problem's hardness; hardness flows strictly *from* the known hard problem *to* the target problem.
- *Destruction of approximation ratios*: An exact polynomial reduction between two decision problems generally preserves only exact solvability, not approximate structure. For example, while Hamiltonian Cycle reduces to General TSP, Metric TSP admits a 3/2-approximation, whereas General TSP cannot be approximated within any constant factor unless $\mathbf{P} = \mathbf{NP}$.

Where We Go Next:

- In Chapter 38 (Approximation Algorithms on Graphs), we trade exactness for provable worst-case guarantees in polynomial time, developing LP rounding, greedy set cover, Christofides' 3/2-approximation for metric TSP, and Goemans–Williamson's 0.878-approximation for Max-Cut via semidefinite programming.
- In Chapter 39 (Dynamic Graph Algorithms), we study how to maintain connectivity, spanning forests, and shortest-path structures under online edge insertions and deletions

without recomputing from scratch.

37.12 Exercises

Exercise 37.1 (Direction discipline)**.** A classmate "proves" Clique is easy by reducing it to Max-Flow: "find a clique by computing a max-flow, which is polynomial." Explain in one paragraph exactly which arrow points the wrong way, and why a reduction *from* Clique *to* a polynomial problem proves nothing about Clique's difficulty.

Exercise 37.2 (Complementarity by hand)**.** Take the 5-cycle C_5. (a) Find its maximum independent set and its minimum vertex cover, and verify $|\text{IS}| + |\text{VC}| = 5$. (b) Draw $\overline{C_5}$ and find its maximum clique; confirm it equals the maximum independent set of C_5 via Lemma 37.1.

Exercise 37.3 (Run the IS gadget)**.** Apply Algorithm 135 to $\varphi = (x_1 \vee x_2 \vee \bar{x}_3) \wedge (\bar{x}_1 \vee x_2 \vee x_3) \wedge (x_1 \vee \bar{x}_2 \vee x_3)$. Draw the graph (three triangles plus conflict edges), find an independent set of size 3, and read off the satisfying assignment.

Exercise 37.4 (Why exactly one per triangle)**.** In the proof of Theorem 37.2, we asserted that an independent set of size m must take exactly one vertex from each clause triangle. Prove it carefully: show that "at most one per triangle" plus "m triangles" plus "$|S| = m$" forces "exactly one per triangle," and explain where the argument would break if some clause had only two literals.

Exercise 37.5 (Metric vs general TSP)**.** (a) Verify that the 1/2 weighting in Theorem 37.5 satisfies the triangle inequality. (b) Show that if general (non-metric) TSP had a polynomial ρ-approximation for any constant $\rho \geq 1$, then $\mathsf{P} = \mathsf{NP}$, using the heavy-non-edge reduction discussed after Theorem 37.5. (c) Why does the same argument fail to rule out the Christofides 3/2-approximation for metric TSP?

Exercise 37.6 (Longest path on a DAG)**.** Give an $O(V + E)$ algorithm for the longest path in a DAG. Then exhibit a small non-DAG (say, a graph on 5 vertices) where your DP returns a wrong answer, and pinpoint the step that assumed acyclicity.

Exercise 37.7 (2-SAT implication graph)**.** Solve $(x_1 \vee x_2) \wedge (\bar{x}_1 \vee x_3) \wedge (\bar{x}_2 \vee \bar{x}_3) \wedge (x_2 \vee x_3)$ by building the implication graph on the six literals, computing its strongly connected components, and checking that no x_i shares a component with $\bar{x}_i$. Read off a satisfying assignment from the condensation's reverse topological order.

Exercise 37.8 (Coloring lifts 3 to c)**.** Prove that c-Coloring is NP-complete for every fixed $c \geq 3$ by reducing 3-Coloring: add $c - 3$ new vertices forming a clique, each joined to every original vertex, and argue that the augmented graph is c-colorable iff the original is 3-colorable.

Exercise 37.9 (Break a gadget)**.** In the 3-coloring reduction (§37.4), suppose you *forgot* to make each clause-output vertex adjacent to the base color B (you only tied it to F). Construct a small unsatisfiable formula whose graph is nevertheless 3-colorable under the broken gadget, demonstrating that the reduction is no longer correct.

Exercise 37.10 (FPT is not free)**.** Vertex-Cover parameterized by solution size k has an $O(2^k(n+m))$ bounded-search-tree algorithm: pick any uncovered edge uv and branch on putting u or v in the cover. (a) Write this as pseudocode and prove correctness and the $O(2^k(n + m))$ bound. (b) Explain why the complementary trick ($\text{IS} = n - \text{VC}$) does *not* yield an $f(k) \cdot \text{poly}(n)$ algorithm for Independent-Set parameterized by the size of the independent set.

Exercise 37.11 (Max-Cut on a triangle and a square)**.** (a) Compute the max-cut of K_3 and of K_4 by hand. (b) For the 4-cycle C_4, exhibit a cut of all 4 edges and argue it is optimal. (c) Sketch why greedy local search (move any vertex that increases the cut) terminates and reaches a cut of size $\geq |E|/2$, a 2-approximation – and give a small graph where it gets stuck below optimal.

Exercise 37.12 (Reduction transitivity in practice)**.** Compose the chapter's reductions to give a *single direct* polynomial map from 3-SAT to TSP, and state the resulting bound L and number of cities in terms of the number of variables n and clauses m. Confirm the composed map is still polynomial.

Chapter 38

Approximation Algorithms on Graphs

The previous chapter left us in an uncomfortable place. Vertex cover, traveling salesman, max-cut, Steiner tree, set cover — each is NP-hard, so unless $\mathsf{P} = \mathsf{NP}$ there is no polynomial-time algorithm that always returns the optimum. But "no exact algorithm" is not the same as "no algorithm." For a great many of these problems we can compute, in polynomial time, a solution provably within a constant factor of optimal: a vertex cover at most twice the smallest one, a tour at most 3/2 the shortest one, a cut catching at least 87.8% of the optimal number of crossing edges. These are *approximation algorithms*, and the guarantee is a theorem rather than an empirical claim, proved against an optimum we cannot even compute.

This chapter develops four core paradigms: direct combinatorial constructions (vertex cover via maximal matching, Christofides for TSP, the double-tree heuristic for Steiner tree), the LP-relaxation-and-rounding method (vertex cover again, this time through duality), the greedy method with its $\ln n$ analysis (set cover), and semidefinite programming (Goemans–Williamson max-cut). We close with the most advanced topic in the chapter: sparsest cut, the $O(\log n)$ Leighton–Rao bound, and the $O(\sqrt{\log n})$ ARV result — which lead into the spectral and electrical-flow material in the chapters that follow.

38.1 What an approximation guarantee means

Definition 38.1 (Approximation ratio)**.** Let Π be an optimization problem and let $\mathrm{OPT}(I)$ denote the optimal objective value on instance I. A polynomial-time algorithm $\mathcal{A}$ is an *α-approximation* if for every instance I it returns a feasible solution of value $\mathcal{A}(I)$ with

$$\frac{\mathcal{A}(I)}{\mathrm{OPT}(I)} \le \alpha \quad \text{(minimization, } \alpha \ge 1\text{)}, \qquad \frac{\mathcal{A}(I)}{\mathrm{OPT}(I)} \ge \alpha \quad \text{(maximization, } \alpha \le 1\text{)}.$$

We call α the *approximation ratio*. A family of algorithms parameterized by $\varepsilon > 0$ giving ratio $1+\varepsilon$ (or $1-\varepsilon$) in time polynomial in the input for each fixed ε is a *polynomial-time approximation scheme* (PTAS); if the running time is also polynomial in $1/\varepsilon$ it is an *FPTAS*.

Intuition

The main difficulty is that we must bound our solution against an optimum *we cannot find*. The standard approach uses a third quantity — a *lower bound* $L \le \mathrm{OPT}$ (for minimization) that we *can* compute, and against which we sandwich our answer:

$$L \;\le\; \mathrm{OPT} \;\le\; \mathcal{A}(I) \;\le\; \alpha \cdot L.$$

If our output is at most αL and $\mathrm{OPT} \ge L$, then $\mathcal{A}(I) \le \alpha\,\mathrm{OPT}$ follows for free. Every algorithm below comes down to choosing L: a maximal matching, an LP value, a minimum spanning tree, an eigenvalue, an SDP. We never compute the optimum; the lower bound does all the work.

Reading note

The phrase "approximation algorithm" with a proven ratio dates to Graham's 1966 analysis of list scheduling, but graph approximation as a discipline took shape in the 1970s: Johnson's greedy set cover (1974), Christofides' TSP heuristic (1976), and Lovász's LP view of covering. The field's modern shape — LP rounding, primal-dual, SDP — was set by the 1990s, culminating in Goemans–Williamson (1995) and the hardness machinery (PCP theorem, 1992) that tells us when a ratio is the *best possible*.

38.2 Vertex cover: a 2-approximation three ways

Recall the problem. Given an undirected graph $G = (V, E)$, a *vertex cover* is a set $S \subseteq V$ touching every edge: for each $\{u, v\} \in E$, at least one of u, v lies in S. We want the smallest such S. Deciding whether a cover of size $\le k$ exists is NP-complete.

38.2.1 The maximal-matching algorithm

The simplest 2-approximation makes no direct reference to the optimum. Take *any* maximal matching — a set of edges, no two sharing an endpoint, that cannot be extended — and output both endpoints of every matched edge.

Algorithm 137: VertexCover-Matching($G = (V, E)$)

```
M ← ∅;   S ← ∅
foreach edge {u, v} ∈ E in arbitrary order do
    if u ∉ S and v ∉ S then
        M ← M ∪ {{u, v}}
        S ← S ∪ {u, v}
return S
```

Why it works

Feasibility. Suppose some edge $\{x, y\}$ were uncovered, i.e. $x, y \notin S$. Then when the loop reached $\{x, y\}$ neither endpoint was in S, so we would have added it to M and put x, y in S — contradiction. So S is a cover.

The lower bound. This is the key step. The matching M consists of vertex-disjoint edges. Any vertex cover must contain at least one endpoint from *each* matched edge (else that edge is uncovered), and since the edges are disjoint these endpoints are distinct. Hence

$$\mathrm{OPT} \;\geq\; |M|.$$

That is our computable lower bound $L = |M|$. Our output has $|S| = 2|M|$, so

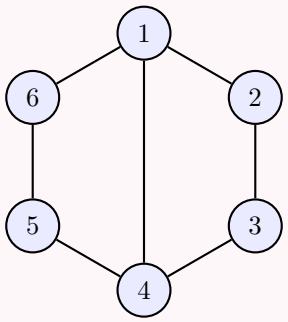
$$|S| = 2|M| \;\leq\; 2\,\mathrm{OPT}.$$

Theorem 38.1. *Algorithm 137 runs in $O(|V| + |E|)$ time and returns a vertex cover of size at most $2\,\mathrm{OPT}$.*

The running time is one pass over the edges with $O(1)$ membership tests using a boolean array.

Trace it yourself

Run it on the 6-cycle plus a chord. Label vertices 1–6 around a hexagon, edges $\{12, 23, 34, 45, 56, 61\}$, plus chord $\{14\}$.



Process edges in order $12, 23, 34, 45, 56, 61, 14$.

- $\{1, 2\}$: both free $\Rightarrow$ match it, $S = \{1, 2\}$.
- $\{2, 3\}$: $2 \in S$, skip.
- $\{3, 4\}$: both free $\Rightarrow$ match, $S = \{1, 2, 3, 4\}$.
- $\{4, 5\}$: $4 \in S$, skip. $\{5, 6\}$: both free $\Rightarrow$ match, $S = \{1, 2, 3, 4, 5, 6\}$.
- $\{6, 1\}$: covered. $\{1, 4\}$: covered.

Output $S = V$, size 6, with $|M| = 3$. The optimum here is $\{1, 3, 5\}$? Check: edge $\{2, 3\}$ has 3, $\{4, 5\}$ has 5, $\{6, 1\}$ has 1, $\{1, 2\}$ has 1, $\{3, 4\}$ has 3, $\{5, 6\}$ has 5, chord $\{1, 4\}$ has 1 — yes, size 3 is optimal. So we returned $6 = 2 \cdot 3$: the bound is tight on this instance. The order of edges matters for the constant in practice, never for the guarantee.

Common trap

A tempting "improvement": repeatedly pick the *highest-degree* vertex, add it to the cover, delete it, repeat. This greedy-by-degree heuristic feels smarter — and it is *not* a constant approximation. There are bipartite instances on which it returns a cover of size $\Theta(\log n)\cdot\mathrm{OPT}$. The matching algorithm wins because the matching supplies a lower bound to compare against; degree-greedy supplies none.

38.2.2 The LP relaxation and rounding

The second route makes the lower bound explicit as a linear program, and it generalizes to weighted vertex cover, where each vertex v has cost $w_v \geq 0$ and we minimize total cost. Write a 0/1 variable x_v for "v is in the cover."

$$\textbf{(IP)} \quad \min \sum_v w_v x_v \quad \text{s.t.} \quad x_u + x_v \geq 1 \;\; \forall \{u,v\} \in E, \quad x_v \in \{0,1\}.$$

Relax the integrality to $x_v \in [0,1]$ to get the LP. Its optimum $\mathrm{OPT}_{\mathrm{LP}}$ is a lower bound on the integral optimum, since every integral solution is a feasible fractional one: $\mathrm{OPT}_{\mathrm{LP}} \leq \mathrm{OPT}$. Solve the LP (polynomial time), then *round*: put v in the cover iff $x_v \geq \frac{1}{2}$.

Why it works

Feasibility. For any edge, $x_u + x_v \geq 1$ forces $\max(x_u, x_v) \geq \frac{1}{2}$, so at least one endpoint is rounded up. Every edge is covered.
Ratio. Rounding up at $\frac{1}{2}$ at most doubles each chosen coordinate: $x_v \geq \frac{1}{2} \Rightarrow 1 \leq 2x_v$. Therefore

$$\sum_{v \in S} w_v \;\leq\; \sum_{v \in S} w_v \cdot 2x_v \;\leq\; 2\sum_v w_v x_v \;=\; 2\,\mathrm{OPT}_{\mathrm{LP}} \;\leq\; 2\,\mathrm{OPT}. \qquad \blacksquare$$

This is the prototypical *relax-and-round* argument: solve a tractable continuous relaxation, then convert the fractional answer to an integral one while controlling the cost blow-up. The factor 2 here is exactly the "threshold" $1/\frac{1}{2}$.

Focus

Half-integrality (Nemhauser–Trotter). For unweighted vertex cover the LP has a notable half-integrality property: there is always an optimal solution with every $x_v \in \{0, \frac{1}{2}, 1\}$. The variables at 1 are in *every* optimal integral cover and the ones at 0 are in none; only the $\frac{1}{2}$ mass is genuinely undecided. This shrinks the problem to its "hard core" and is the basis of the Nemhauser–Trotter kernel in Chapter 25.

Connection

The LP dual of vertex cover is *fractional matching*: $\max \sum_e y_e$ subject to $\sum_{e \ni v} y_e \leq 1$. LP duality says $\mathrm{OPT}^{\mathrm{frac}}_{\mathrm{matching}} = \mathrm{OPT}^{\mathrm{frac}}_{\mathrm{VC}}$, and on bipartite graphs both relaxations are integral (Kőnig's theorem, Chapter 15), so vertex cover is *exactly* solvable there. The factor-2 gap arises only on non-bipartite graphs, where odd cycles cause the loss.

Reading note

Whether 2 is the best constant possible is one of the famous open questions. Under the Unique Games Conjecture, vertex cover is NP-hard to approximate within $2-\varepsilon$ for every $\varepsilon > 0$ (Khot–Regev, 2003). Without UGC, the best hardness is 1.36 (Dinur–Safra) improved toward $\sqrt{2}$. So the trivial matching algorithm may already be optimal — we just cannot prove it.

38.3 Set cover and the greedy $\ln n$ bound

Vertex cover is a special case of *set cover*, the canonical covering problem and the home of the greedy method. Given a universe U of n elements and a family $\mathcal{S} = \{S_1, \dots, S_m\}$ of subsets (with costs c_i, or unit cost in the simplest version) whose union is U, find a minimum-cost subfamily covering U. On a graph, "elements are edges, sets are the edges incident to each vertex" recovers vertex cover.

Intuition

Greedy is the obvious idea: keep buying the set that covers the most still-uncovered elements per dollar, until everything is covered. The analysis is the interesting part: greedy can be off by a factor of $\ln n$, and that factor is essentially unavoidable for any polynomial algorithm. The proof is a charging argument: spread each set's cost evenly over the new elements it covers, then sum the harmonic series.

Algorithm 138: GreedySetCover$(U, \mathcal{S}, c)$

```
C ← ∅;   R ← U                                   // R = uncovered elements
while R ≠ ∅ do
    pick S_i maximizing the coverage rate |S_i ∩ R| / c_i
    foreach e ∈ S_i ∩ R do
        price(e) ← c_i / |S_i ∩ R|
    C ← C ∪ {i};   R ← R \ S_i
return C
```

Theorem 38.2. GreedySetCover *returns a cover of cost at most* $H_n \cdot \mathrm{OPT}$, *where* $H_n = \sum_{k=1}^{n} \frac{1}{k} \le \ln n + 1$.

Why it works

The total cost of greedy equals $\sum_{e \in U} \mathrm{price}(e)$, because each set's cost is split exactly among the new elements it pays for. So it suffices to bound the prices.

Fix an optimal cover $\mathcal{O}$ of cost OPT. We claim $\sum_{e \in S} \mathrm{price}(e) \le H_{|S|} \cdot c(S)$ for every set $S \in \mathcal{O}$. Order the elements of S by the time greedy covered them: $e_1, e_2, \dots, e_k$ ($k = |S|$), most-recently covered last. Consider the moment just before e_j was covered. At that instant $e_j, \dots, e_k$ are all still uncovered, so S itself could cover at least $k - j + 1$ of R at rate $c(S)/(k-$

$j+1$). Greedy chose a set at least this cost-effective, so the price charged to e_j satisfies

$$\text{price}(e_j) \;\le\; \frac{c(S)}{k-j+1}.$$

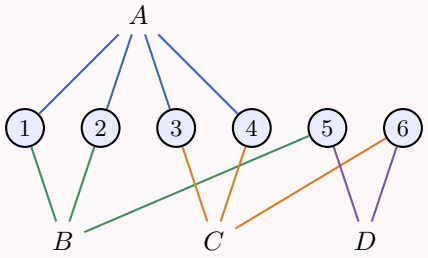
Summing over $j = 1, \dots, k$ gives $\sum_{e \in S} \text{price}(e) \le c(S)\, H_k$. Now sum over the optimal sets, and note each element is in at least one optimal set:

$$\text{greedy cost} = \sum_{e} \text{price}(e) \le \sum_{S \in \mathcal{O}} \sum_{e \in S} \text{price}(e) \le H_n \sum_{S \in \mathcal{O}} c(S) = H_n \cdot \text{OPT}. \qquad \blacksquare$$

Trace it yourself

Tiny unit-cost instance. Universe $U = \{1,2,3,4,5,6\}$, sets

$$A = \{1,2,3,4\}, \quad B = \{1,2,5\}, \quad C = \{3,4,6\}, \quad D = \{5,6\}.$$



Greedy first compares coverage: $|A| = 4$ wins. Pick A; remaining $R = \{5,6\}$. Now $|B \cap R| = 1$, $|C \cap R| = 1$, $|D \cap R| = 2$. Pick D; done. Greedy returns $\{A, D\}$, cost 2 — which is optimal here, so greedy was lucky. The classic bad instance stacks two big sets each covering "half plus one" against the cheap optimal pair, forcing the harmonic loss; build it in Exercise 38.3.

Cost summary

With a priority queue keyed by current coverage, greedy runs in $O(\sum_i |S_i| \log m)$. The $\ln n$ ratio is tight: Feige (1998) proved set cover is NP-hard to approximate within $(1 - o(1)) \ln n$, so greedy is essentially optimal among all polynomial algorithms. Few problems pin down their approximability so precisely.

38.4 Metric TSP, the double-tree bound, and Christofides

The traveling salesman problem: given a complete graph on V with edge weights w, find a minimum weight Hamiltonian cycle (a tour visiting every vertex once and returning). General TSP cannot be approximated within any constant unless P = NP (a tour-or-no-tour gadget makes even a ratio-2^{poly} approximation NP-hard). The fix is the *metric* assumption: weights satisfy the triangle inequality $w(u,w) \le w(u,v) + w(v,w)$. This is exactly the regime of real distances, and it is approximable.

38.4.1 The MST lower bound and double-tree

Intuition

Delete one edge from a tour and you have a Hamiltonian *path* — a spanning tree (a very thin one). So every tour contains a spanning tree, giving

$$\text{MST} \;\le\; \text{OPT}.$$

That is the lower bound. To turn a tree into a tour, walk around it (an Euler-tour traversal visiting each edge twice) and then "shortcut" past already-visited vertices. The walk costs $2\,\text{MST}$; shortcutting only helps because of the triangle inequality. Result: a 2-approximation.

Algorithm 139: DoubleTreeTSP(G, w) – metric

```
T ← minimum spanning tree of G
T' ← multigraph doubling every edge of T                    // all degrees even
W ← Eulerian circuit of T'                  // exists: connected, even degrees
walk W, output vertices in first-visit order, close the cycle
return the resulting Hamiltonian tour
```

Why it works

T' is connected and every vertex has even degree (each tree edge contributes 2), so an Eulerian circuit exists and has weight $2\,\text{MST}$. Reading off first visits and shortcutting replaces a detour $u \to \cdots \to v$ by the direct edge $u \to v$; the triangle inequality (applied repeatedly) guarantees the direct edge is no longer than the path it replaces. So the tour weighs at most $2\,\text{MST} \le 2\,\text{OPT}$.

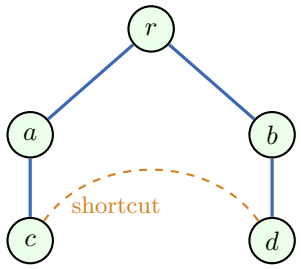


Figure 38.1: Double-tree on a small MST (solid). The Euler walk $r\,a\,c\,a\,r\,b\,d\,b\,r$ traverses each tree edge twice; reading off first visits r, a, c, b, d and closing the cycle replaces the back-tracking $c \to a \to r \to b \to d$ by the direct shortcut $c \to d$ (dashed), which the triangle inequality keeps no longer.

38.4.2 Christofides: fixing the parity instead of doubling

Doubling every edge is wasteful: we double the *whole* tree just to make degrees even, but only the odd-degree vertices were a problem. Christofides' idea is to repair parity with a minimum *matching* on exactly those vertices.

Focus

A graph has an Eulerian circuit iff it is connected and *every vertex has even degree*. In the MST T, let O be the set of odd-degree vertices. A classical fact (handshake lemma) says $|O|$ is even. Add a minimum-weight perfect matching M on O (using the metric distances): each vertex of O gains exactly one edge, flipping its parity to even, while even-degree vertices are untouched. Now $T \cup M$ is Eulerian.

Algorithm 140: CHRISTOFIDES(G, w) – metric

```
T ← minimum spanning tree
O ← vertices of odd degree in T                          // |O| even
M ← minimum-weight perfect matching on O (metric weights)
H ← T ∪ M                                   // multigraph; all degrees even, connected
W ← Eulerian circuit of H
shortcut W to a Hamiltonian tour and return it
```

Theorem 38.3. CHRISTOFIDES *is a* $\frac{3}{2}$*-approximation for metric TSP.*

Why it works

The Eulerian circuit weighs $w(T)+w(M)$, and shortcutting only decreases weight, so the tour costs at most $w(T) + w(M)$. We bound each piece against OPT.

Tree: $w(T) = \text{MST} \le \text{OPT}$, as before.

Matching: This is the harder part. Take an optimal tour and restrict attention to the vertices O, visited by the tour in some cyclic order $o_1, o_2, \ldots, o_{2t}, o_1$. Shortcutting the tour onto just these vertices gives a cycle $\mathcal{C}$ on O of weight at most OPT (triangle inequality). An even cycle splits into two perfect matchings — the odd-indexed edges and the even-indexed edges — whose weights sum to $w(\mathcal{C}) \le \text{OPT}$. The cheaper of the two weighs at most $\frac{1}{2}\text{OPT}$, and the minimum perfect matching M is no heavier:

$$w(M) \;\le\; \tfrac{1}{2}\,\text{OPT}.$$

Adding up: tour weight $\le w(T) + w(M) \le \text{OPT} + \frac{1}{2}\text{OPT} = \frac{3}{2}\text{OPT}$. ■

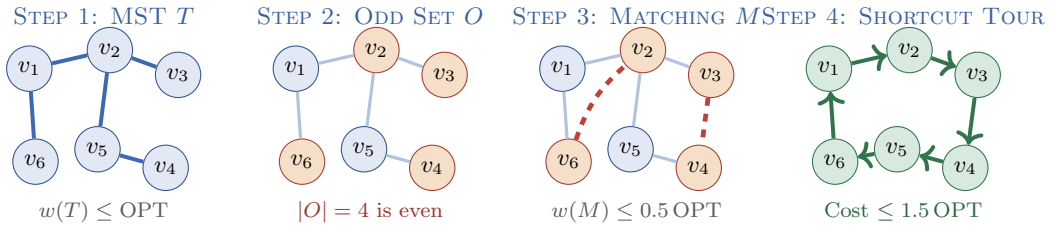
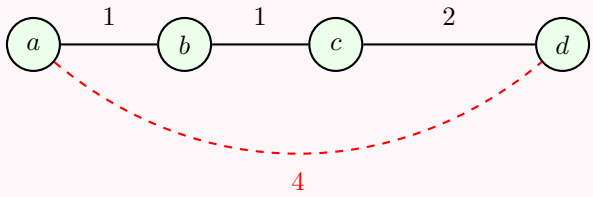


Figure 38.2: The four stages of Christofides' $\frac{3}{2}$-approximation algorithm for metric TSP. Step 1: compute minimum spanning tree T ($w(T) \le$ OPT). Step 2: identify odd-degree vertices $O = \{v_2, v_3, v_4, v_6\}$; the handshake lemma guarantees $|O|$ is even. Step 3: compute minimum-weight perfect matching M on O ($w(M) \le \frac{1}{2}\text{OPT}$), yielding an all-even Eulerian multigraph $T \cup M$. Step 4: trace an Eulerian walk and shortcut repeated vertices by the triangle inequality into a Hamiltonian cycle.

Trace it yourself

Four points on a line (a metric): $a = 0, b = 1, c = 2, d = 4$, distances = absolute differences. The MST is the path $a{-}b{-}c{-}d$, weight $1+1+2 = 4$. Odd-degree vertices: the two endpoints a and d (degree 1); b, c have degree 2.



The minimum matching on $O = \{a, d\}$ is just the single edge $\{a, d\}$ of weight 4 (red, dashed). Then $H = T \cup \{ad\}$ is a cycle $a{-}b{-}c{-}d{-}a$, already Hamiltonian, weight $4 + 4 = 8$. The optimal tour visits a, b, c, d in order and returns: $1 + 1 + 2 + 4 = 8$ too. So on this instance Christofides is exact. (Double-tree would have produced the Eulerian walk of weight $2 \cdot 4 = 8$ and shortcut to the same 8 — the gap between the methods shows only when the odd set is small relative to the tree.)

Reading note

Christofides published the 3/2 bound in 1976; it stood as the best known ratio for metric TSP for forty-four years (with 1.4 for graph metrics by Sebő and Vygen). In 2020, Karlin, Klein, and Oveis Gharan broke it by 10^{-36} using a randomized maximum-entropy distribution over spanning trees, showing that 3/2 is not the best achievable ratio.

Cost summary

MST is $O(n^2)$ on a dense metric (Prim with arrays). The matching dominates: a minimum-weight perfect matching on $|O| \le n$ vertices via blossom-based methods costs $O(n^3)$. So Christofides is $O(n^3)$. Double-tree is only $O(n^2 \log n)$ but gives the weaker factor 2.

38.5 Steiner tree: the metric closure and double-spanning-tree

Steiner tree generalizes MST. We are given a graph (or metric) and a subset $R \subseteq V$ of *terminals*; we must connect all terminals as cheaply as possible, but we are *allowed* to route through non-terminal *Steiner* vertices. Minimizing over which Steiner vertices to use is what makes it NP-hard. The natural 2-approximation reuses the doubling idea on the *metric closure*.

Intuition

Pretend the Steiner vertices do not exist: build the complete graph on the terminals R, where the weight of $\{s, t\}$ is the shortest-path distance $\text{dist}(s, t)$ in the original graph. Compute an MST of *that*. Expand each chosen "metric edge" back into its underlying shortest path. The result connects all terminals, possibly using Steiner vertices for free as waypoints, and the

doubling argument bounds it by twice the optimum.

Algorithm 141: SteinerMST(G, w, R)

1 compute the metric closure $\bar{G}$ on R: $\bar{w}(s,t) = \mathrm{dist}_G(s,t)$
2 $\bar{T} \leftarrow$ minimum spanning tree of $\bar{G}$
3 replace each edge $\{s,t\} \in \bar{T}$ by a shortest s–t path in G
4 remove cycles/redundant edges to get a tree; **return** it

Theorem 38.4. SteinerMST *returns a Steiner tree of weight at most* $2(1 - \frac{1}{|R|})\,\mathrm{OPT} < 2\,\mathrm{OPT}$.

Why it works

Let T^* be an optimal Steiner tree, weight OPT. Double its edges and take an Eulerian circuit W of weight $2\,\mathrm{OPT}$. Walk W and shortcut to the terminals in order of first visit: this yields a Hamiltonian cycle $\mathcal{H}$ *on the terminals only*, where consecutive terminals are joined by direct metric edges. Each such metric edge is at most the length of the path it shortcut (paths in W are at least metric distances), so $\bar{w}(\mathcal{H}) \le 2\,\mathrm{OPT}$. Deleting the heaviest edge of $\mathcal{H}$ leaves a spanning *path* of the terminals in $\bar{G}$, hence a spanning tree, of weight $\le 2\,\mathrm{OPT}$. The MST $\bar{T}$ is no heavier, so $\bar{w}(\bar{T}) \le 2\,\mathrm{OPT}$, and expanding edges to paths does not increase weight. The $(1 - 1/|R|)$ refinement comes from deleting the heaviest of $|R|$ cycle edges, which is at least the average. ■

Trace it yourself

A "plus sign": central Steiner vertex c joined to four terminals N, S, E, W each at distance 1; the terminals are pairwise at distance 2 through c (or directly, say, also 2). Metric closure on $\{N, S, E, W\}$ has all weights 2; its MST is any spanning tree, weight $3 \cdot 2 = 6$. Expanding, e.g., the path N–E–S–W routes through c and reuses the spoke to c. But the true optimum is the star: four spokes of weight 1, total 4. So this algorithm returns $6 \le 2 \cdot 4$, illustrating both the looseness and the role of the unused Steiner vertex. The famous worst case is the star, where ignoring Steiner points costs the full factor $2(1 - 1/|R|)$, as shown in Figure 38.3.

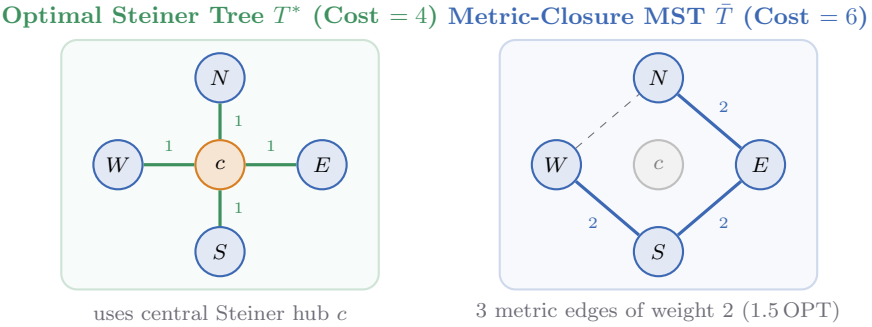


Figure 38.3: Why ignoring Steiner vertices costs up to $2(1-1/|R|)$. Left: Four terminals $\{N, S, E, W\}$ connected through a central Steiner vertex c at unit distance cost $\mathrm{OPT} = 4$. Right: In the metric closure $\bar{G}$ on the terminals alone, every pair has distance 2, so any spanning tree $\bar{T}$ uses $|R| - 1 = 3$ edges of weight 2 for total cost $6 = \frac{3}{2}\,\mathrm{OPT}$.

Connection

The integrality gap of this MST relaxation is exactly 2, but better relaxations exist. The bidirected-cut and hypergraph LPs, rounded by the iterative "loss-contracting" algorithm of Byrka–Grandoni–Rothvoß–Sanità (2010), achieve $\ln 4 + \varepsilon \approx 1.39$, the current best. The jump from combinatorial 2 to LP-based 1.39 mirrors the vertex-cover and TSP stories: a sharper lower bound buys a sharper ratio.

38.6 Max-cut: greedy, local search, and SDP rounding

Now a maximization problem. Given $G = (V, E)$ (unit weights for simplicity), partition V into (A, B) to maximize the number of *cut* edges — edges with one endpoint on each side. Max-cut is NP-hard, in sharp contrast to min-cut, which is polynomial.

38.6.1 Greedy and local search: factor one half

Intuition

A coin flip already does well. Put each vertex on side A or B independently with probability $\frac{1}{2}$. An edge is cut with probability $\frac{1}{2}$, so the expected cut size is $\frac{1}{2}|E|$. Since $\mathrm{OPT} \le |E|$, this is a $\frac{1}{2}$-approximation *in expectation*. Local search derandomizes it: while some vertex has more edges to its own side than across, flip it. Each flip strictly increases the cut, so it terminates, and at the end every vertex has at least half its edges cut — summing gives a cut of $\ge \frac{1}{2}|E|$.

Algorithm 142: LocalMaxCut(G)

```
(A, B) ← arbitrary partition
while some v has more neighbors on its own side than the other do
    move v to the other side
return (A, B)
```

Why it works

Let $d(v)$ be v's degree and $c(v)$ the number of its edges currently cut. At a local optimum $c(v) \ge d(v)/2$ for all v (else flipping v improves). Summing, $\sum_v c(v) \ge \frac{1}{2}\sum_v d(v)$. Each cut edge is counted twice on the left and each edge twice on the right, so $2\,|\mathrm{cut}| \ge |E|$, i.e. the cut has $\ge |E|/2 \ge \mathrm{OPT}/2$ edges. Each flip raises the integer cut value by at least 1, and the cut is bounded by $|E|$, so at most $|E|$ flips occur: $O(|V||E|)$ time. ■

38.6.2 Goemans–Williamson: the 0.878 SDP rounding

For decades nobody could beat the factor $\frac{1}{2}$. Michel Goemans and David Williamson did so by replacing discrete hypercube relaxations with a *semidefinite program* (SDP). The discrete problem assigns each vertex a scalar label $y_v \in \{-1, +1\}$; Goemans and Williamson lifted each vertex to a high-dimensional unit vector $x_v \in \mathbb{R}^n$ on the unit sphere.

Intuition

For a ± 1 labeling y_v, the indicator "$\{u,v\}$ is cut" equals $\frac{1-y_u y_v}{2}$. If we lift $y_v \in \{\pm 1\}$ to a unit *vector* $x_v \in \mathbb{R}^n$ and replace the scalar product $y_u y_v$ by the inner product $\langle x_u, x_v\rangle$, our goal becomes maximizing $\sum_{\{u,v\}\in E} \frac{1-\langle x_u, x_v\rangle}{2}$ over unit vectors.

What makes this geometric relaxation solvable in polynomial time? The variable is the Gram matrix $\mathbf{X} \in \mathbb{R}^{n\times n}$ of pairwise inner products, $X_{uv} = \langle x_u, x_v\rangle$. A symmetric matrix $\mathbf{X}$ is the Gram matrix of a set of vectors if and only if $\mathbf{X}$ is *positive semidefinite* (written $\mathbf{X} \succeq 0$). The condition $\mathbf{X} \succeq 0$ simply means that for every test vector $z \in \mathbb{R}^n$, the quadratic form satisfies $z^\top \mathbf{X} z \geq 0$, or equivalently, all eigenvalues of $\mathbf{X}$ are non-negative. The constraint that every vector is a unit vector is purely linear on the diagonal: $X_{vv} = \langle x_v, x_v\rangle = \|x_v\|^2 = 1$. The objective function $\sum_{\{u,v\}\in E} \frac{1-X_{uv}}{2}$ is also linear in the entries of $\mathbf{X}$. The set of positive semidefinite matrices with unit diagonal forms a convex spectrahedron, so the optimal Gram matrix $\mathbf{X}^*$ can be found to arbitrary precision ε in polynomial time using interior-point methods, and factorized into vectors x_v via Cholesky decomposition.

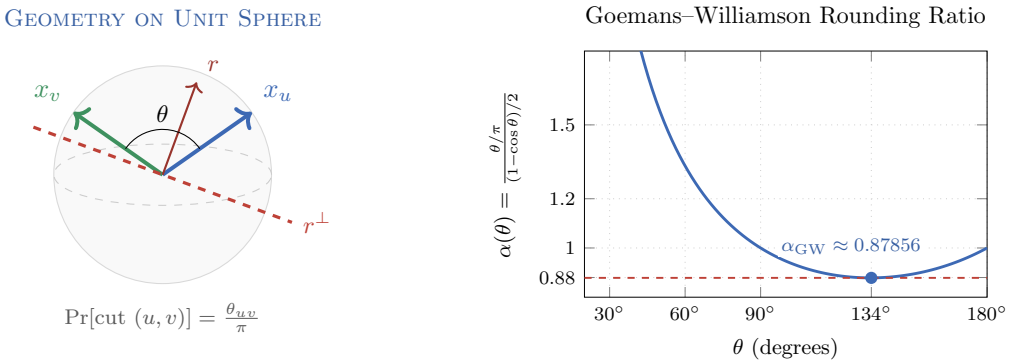


Figure 38.4: The Goemans–Williamson randomized rounding scheme for Max-Cut. Left: each vertex is mapped to a unit vector x_v in $\mathbb{R}^n$. A random hyperplane $r^\perp$ normal to Gaussian vector r cuts edge $\{u,v\}$ with probability θ_{uv}/π. Right: the ratio of cut probability to the SDP objective contribution $(1-\cos\theta)/2$ achieves its worst-case minimum of $\alpha_{\mathrm{GW}} \approx 0.87856$ at $\theta^* \approx 134°$, guaranteeing at least an 87.8% approximation.

Once the optimal unit vectors are computed, we round to discrete sets using a *random hyperplane*: pick a uniformly random direction r on the unit sphere (by sampling each coordinate from a standard Gaussian) and partition vertices by the sign of the inner product $\langle x_v, r\rangle$. A random hyperplane separates x_u and x_v exactly when it falls in the dihedral angle θ between them, which happens with probability θ/π.

Algorithm 143: GW-MaxCut(G)

1 solve the SDP: $\max \sum_{\{u,v\}\in E} \dfrac{1-\langle x_u, x_v\rangle}{2}$ over unit vectors $x_v \in \mathbb{R}^n$
2 draw a random Gaussian vector $r \in \mathbb{R}^n$
3 $A \leftarrow \{v : \langle x_v, r\rangle \geq 0\}, \quad B \leftarrow V \setminus A$
4 **return** (A, B)

Theorem 38.5 (Goemans–Williamson, 1995)**.** GW-MaxCut *returns a cut of expected value at*

least $\alpha_{\mathrm{GW}} \cdot \mathrm{OPT}$*, where*

$$\alpha_{\mathrm{GW}} = \min_{0 \le \theta \le \pi} \frac{\theta/\pi}{(1-\cos\theta)/2} \approx 0.87856.$$

Why it works

For an edge with vectors at angle $\theta_{uv} = \arccos\langle x_u, x_v\rangle$, the probability the random hyperplane cuts it is exactly θ_{uv}/π (the fraction of directions whose hyperplane falls between the two vectors). The edge's contribution to the SDP objective is $(1-\cos\theta_{uv})/2$. The pointwise ratio is bounded below by the constant $\alpha_{\mathrm{GW}} = \min_\theta \frac{\theta/\pi}{(1-\cos\theta)/2}$, so by linearity of expectation,

$$\mathbb{E}[\mathrm{cut}] = \sum_{\{u,v\}\in E} \frac{\theta_{uv}}{\pi} \ge \alpha_{\mathrm{GW}} \sum_{\{u,v\}\in E} \frac{1-\cos\theta_{uv}}{2} = \alpha_{\mathrm{GW}} \cdot \mathrm{SDP} \ge \alpha_{\mathrm{GW}} \cdot \mathrm{OPT}.$$

The minimizing angle is near $\theta^* \approx 2.331$ rad (134°), where the ratio bottoms out at ≈ 0.87856. ■

Common trap

The ratio is *not* achieved at the endpoints. At $\theta = \pi$ (antipodal vectors) it equals 1; as $\theta \to 0$ the ratio $\frac{2\theta}{\pi(1-\cos\theta)}$ actually *diverges* to $+\infty$, since the denominator $1-\cos\theta \sim \theta^2/2$ shrinks faster than the numerator. The worst case is the interior angle $\approx 134^\circ$. A common error is to "bound" the ratio by checking endpoints; you must minimize over the whole interval. Plot $f(\theta) = \frac{2\theta}{\pi(1-\cos\theta)}$ to see the single interior minimum.

Reading note

Before 1995 the best max-cut ratio was 1/2. Goemans and Williamson's semidefinite programming approach reached 0.87856. Under the Unique Games Conjecture, beating α_{GW} is NP-hard (Khot–Kindler–Mossel–O'Donnell, 2007), showing the relaxation constant matches the complexity barrier.

Cost summary

The SDP solves to additive error δ in time $\mathrm{poly}(n, \log\frac{1}{\delta})$ via interior-point or the matrix-multiplicative-weights method; the rounding is $O(n^2)$ for the dot products. Expected ratio 0.878; it can be made high-probability by repeating the hyperplane draw $O(\log n)$ times and keeping the best cut.

Connection

The SDP relaxation is an *eigenvalue*-flavored object: writing it in the Laplacian $\mathbf{L}$, the spectral relaxation $\frac{1}{4}\lambda_{\max}(\mathbf{L})\cdot$(a balance term) is a looser cousin, and the SDP is the tightest convex relaxation invariant under rotations. This is the same Laplacian whose *small* eigenvalues govern sparsest cut below and whose spectrum runs through Chapters 32 and 34. Max-cut wants the top of the spectrum; conductance wants the bottom.

38.7 Sparsest cut, conductance, and the Leighton–Rao and ARV bounds

We end with the problem in this chapter that connects approximation to spectra, metric embeddings, and electrical flows. The *sparsest cut* asks for the partition that is cheap in edges *and* balanced in vertices.

Definition 38.2 (Sparsest cut / conductance)**.** For $S \subseteq V$ with $0 < |S| \le |V|/2$, the (uniform, or edge-) *sparsity* is

$$\Phi(S) = \frac{|E(S, \bar{S})|}{|S|\,|\bar{S}|}, \qquad \text{and the related } \textit{conductance} \quad \phi(S) = \frac{|E(S, \bar{S})|}{\min(\mathrm{vol}(S), \mathrm{vol}(\bar{S}))},$$

where $\mathrm{vol}(S) = \sum_{v \in S} \deg(v)$. The sparsest cut is $\min_S \Phi(S)$; the *Cheeger constant* (edge expansion) is $\min_S \phi(S)$.

Intuition

Min-cut alone is fooled by a single pendant vertex: cutting it off costs one edge and is "sparse" but useless for clustering — it isolates almost nothing. Dividing by $|S||\bar{S}|$ (or by the volume) *rewards balance*, so the sparsest cut finds the genuine bottleneck of the graph: the dumbbell's thin neck, the two communities' weak interface. This is exactly the cut you want for divide-and-conquer, image segmentation, and clustering — and it is NP-hard to compute exactly.



38.7.1 The spectral bound (Cheeger) and where it fails

The bottom of the Laplacian spectrum gives a fast two-sided estimate. With λ_2 the second smallest eigenvalue of the normalized Laplacian, *Cheeger's inequality* reads

$$\frac{\lambda_2}{2} \;\le\; \phi_G \;\le\; \sqrt{2\lambda_2}.$$

Computing λ_2 and rounding its eigenvector (the "sweep cut") gives a cut within a *quadratic* factor of optimal — excellent when ϕ is constant, but as bad as $O(1/\phi)$ when the graph is a near-expander. The full proof appears in Chapters 32 and 34; here we only note its place as the cheapest sparsest-cut approximation.

38.7.2 Leighton–Rao: LP, multicommodity flow, and $O(\log n)$

Intuition

The LP relaxation of sparsest cut assigns a *distance* $d_{uv} \geq 0$ to each pair, forming a metric (d obeys the triangle inequality), minimizing $\sum_{\{u,v\}\in E} d_{uv}$ subject to total spread $\sum_{u,v} d_{uv} = 1$. By LP duality this equals the maximum *concurrent multicommodity flow* — route a unit of flow between every pair without exceeding edge capacities. The flow/cut gap is what we must round.

The rounding theorem (Linial–London–Rabinovich; Bourgain): *every n-point metric embeds into ℓ_1 with distortion $O(\log n)$.* An ℓ_1 metric is a nonnegative combination of cut metrics, and a cut metric is an actual cut. So we solve the LP, embed the optimal metric into ℓ_1 losing a factor $O(\log n)$, and read off a cut.

Theorem 38.6 (Leighton–Rao, 1988)**.** *There is a polynomial-time $O(\log n)$-approximation for sparsest cut, via the LP relaxation and Bourgain's ℓ_1 embedding. Equivalently, the max-concurrent-flow / min-sparsest-cut gap is $\Theta(\log n)$ for general demands.*

Why it works

Why ℓ_1 is the right target. A cut $(S, \bar{S})$ induces the *cut metric* $\delta_S(u, v) = \mathbf{1}[\,u, v \text{ on opposite sides}\,]$. Sparsity evaluated on a cut metric is literally $\Phi(S)$. The set of ℓ_1-embeddable metrics is exactly the cone spanned by cut metrics, so optimizing sparsity over ℓ_1 metrics *is* optimizing over (distributions of) cuts — no gap. The only loss is squeezing the LP's arbitrary metric into ℓ_1, and Bourgain's theorem caps that loss at $O(\log n)$. The matching lower bound (expanders give an $\Omega(\log n)$ flow/cut gap) shows $O(\log n)$ is best possible *for this LP.* ■

Focus

Bourgain's embedding, in brief. To embed a metric (V, d) into ℓ_1, pick $O(\log^2 n)$ random subsets A of V (at geometric scales of size) and map each vertex v to the vector of its distances $d(v, A) = \min_{a\in A} d(v, a)$, scaled. Distances contract by at most 1 and, in expectation across scales, expand by at least $\Omega(1/\log n)$ — giving distortion $O(\log n)$. The random subsets are doing exactly what random hyperplanes did for GW: turning a geometric object into combinatorial cuts.

38.7.3 ARV: the SDP and $O(\sqrt{\log n})$

Intuition

Leighton–Rao uses an LP (a metric). Arora–Rao–Vazirani (2004) tighten it to an *SDP*: embed vertices as vectors x_v with the extra constraint that squared distances $\|x_u - x_v\|^2$ themselves form a metric (the metric is "of negative type," i.e. ℓ_2^2). This rules out the degenerate configurations that cost Leighton–Rao a full $\log n$.

The main tool is a geometric *structure theorem*: in any such SDP solution, either there is an obvious sparse cut, or one can find two large, $\Omega(1/\sqrt{\log n})$-separated sets of vertices — Δ-separated "well-spread" sets — whose existence forces a good cut. Chaining these yields the bound.

Theorem 38.7 (Arora–Rao–Vazirani, 2004)**.** *The ℓ_2^2 (negative-type) SDP gives an approximation ratio of $O(\sqrt{\log n})$ for (uniform) sparsest cut, and the integrality gap of that SDP is $\Omega(\log \log n)$ but $o(\log n)$.*

We will not prove ARV — its structure theorem and the $\ell_2^2 \hookrightarrow \ell_1$ embedding with distortion $O(\sqrt{\log n})$ (Arora–Lee–Naor) would need a chapter of their own. The table below summarizes the ladder of relaxations and ratios.

Relaxation	Object	Sparsest-cut ratio
Spectral (Cheeger)	eigenvector of $\mathbf{L}$	quadratic ($\sqrt{\cdot}$ in ϕ)
Leighton–Rao (LP)	arbitrary metric $\to \ell_1$	$O(\log n)$
Arora–Rao–Vazirani (SDP)	ℓ_2^2 metric $\to \ell_1$	$O(\sqrt{\log n})$

Connection

This ladder recurs throughout the rest of the book. The spectral row connects directly to Chapters 31, 32, and 34, and the eigenvector sweep is the bisection algorithm of Chapter 33. The flow side of Leighton–Rao duality is multicommodity flow, while fast SDP and cut-matching solvers rely on *electrical flows* and near-linear-time Laplacian solvers — the Spielman–Teng machinery of Chapter 41. Sparsest cut is where combinatorial optimization, convex geometry, and spectral graph theory meet.

Reading note

Leighton and Rao's 1988 $O(\log n)$ result was the first to break the spectral barrier with a *provable* guarantee for general graphs and seeded the metric-embedding program in theoretical computer science. Arora, Rao, and Vazirani (2004) proved the $O(\sqrt{\log n})$ ratio using geometric measure concentration on the sphere.

38.8 A unifying picture

The algorithms in this chapter organize cleanly according to the lower bound on OPT each one uses.

Problem	Ratio	Lower bound used	Method
Vertex cover	2	maximal matching / LP	combinatorial / round
Set cover	$\ln n$	fractional / charging	greedy
Metric TSP	3/2	MST + matching	Christofides
Steiner tree	2 (1.39)	metric-closure MST	double tree / LP
Max-cut	0.878	SDP (Gram matrix)	hyperplane round
Sparsest cut	$O(\sqrt{\log n})$	LP / SDP metric	embed to ℓ_1

Why this matters

The table shows a progression. Combinatorial lower bounds (matchings, trees) give clean small constants but plateau. LP relaxations match or beat them and expose duality. SDP relaxations improve on problems where LPs have an inherent $\log n$ gap. Most of these ratios have a matching hardness result, often with the *same constant* (some conditional on the Unique Games Conjecture), which indicates that the relaxation *matches the limit of polynomial computation*. This correspondence between relaxation strength and approximability is the central lesson of the area.

38.9 Historical Context and Controversies

History

The formal study of approximation algorithms began in 1966, when Ronald Graham analyzed greedy list-scheduling heuristics for multiprocessor job assignment (distributing computing tasks across processors to minimize total finish time). Graham discovered that an arbitrary greedy order achieves a $(2 - 1/m)$-approximation of the optimal schedule, while uncovering curious scheduling anomalies where adding processors could paradoxically increase completion times. Following Cook and Karp's hardness results, David Johnson published his 1974 survey demonstrating that simple heuristics achieve a $(1 + \ln n)$ guarantee for Set Cover and a clean factor 2 for Vertex Cover. Two years later, in 1976, Nicos Christofides published his $\frac{3}{2}$-approximation for metric TSP, combining the minimum spanning tree with a minimum-weight perfect matching on odd-degree vertices.

For two decades, approximation relied almost entirely on combinatorial methods and linear programming relaxations. Then in 1995, Michel Goemans and David Williamson introduced their 0.878-approximation algorithm for Max-Cut using semidefinite programming (SDP, optimizing a linear function over the cone of positive semidefinite matrices) and randomized hyperplane rounding. By relaxing discrete signs to unit vectors on the sphere, Goemans and Williamson showed that high-dimensional geometry could handle problems where linear programming was limited by its integrality gap (the ratio between optimal integer and fractional solutions). The theoretical picture was deepened by the probabilistically checkable proofs (PCP) theorem in 1992 and Subhash Khot's 2002 Unique Games Conjecture (UGC), which implies that the Goemans–Williamson 0.878 ratio and the factor 2 for Vertex Cover are mathematically optimal if UGC holds.

Approximation theory repeatedly exposes plausible-looking heuristics that fail in the worst case. For example, greedily choosing the highest-degree vertex for Vertex Cover seems natural, yet bipartite counterexamples force it to suffer a $\Theta(\log n)$ approximation ratio, while picking both endpoints of an arbitrary maximal matching directly guarantees a factor of 2. Similarly, Christofides' $\frac{3}{2}$ barrier stood for forty-four years because the matching was chosen blindly after the tree was fixed; only in 2020 did Anna Karlin, Nathan Klein, and Shayan Oveis Gharan finally break it by sampling spanning trees from randomized maximum-entropy distributions. The central open question in graph approximation remains the Unique Games Conjecture; a proof would show that the factor 2 for Vertex Cover and the 0.878 ratio for Max-Cut are the best possible.

38.9.1 What We Can Now Do and Its Limitations

Connection

Capabilities Unlocked:

- We can approximate **NP**-hard optimization problems in polynomial time with proved worst-case bounds.
- Semidefinite programming and randomized vector rounding map discrete graph problems onto continuous spherical geometry, so that continuous optimization can be applied to combinatorial problems.
- LP duality provides certificate lower bounds (OPT $\geq$ LP), allowing algorithm designers to evaluate the proximity of a solution to optimality on individual instances.

Limitations:

- *Integrality gap bounds*: No rounding algorithm can beat the integrality gap of its underlying relaxation. If an LP relaxation has a gap of 2, no rounding technique based on that LP can guarantee a ratio better than 2.
- *Severe inapproximability barriers*: Not all **NP**-hard problems admit constant-factor approximations. General TSP cannot be approximated within any factor, and Maximum Clique cannot be approximated within $n^{1-\varepsilon}$ for any $\varepsilon > 0$ unless $\mathbf{P} = \mathbf{NP}$.
- *Computational overhead of SDPs*: While solvable in polynomial time, general interior-point methods for SDPs run in $\tilde{O}(n^{3.5})$ time, rendering exact SDP solvers impractical on graphs with millions of vertices without specialized first-order methods.

Where We Go Next:

- In Chapter 39 (Dynamic Graph Algorithms), edges are inserted and deleted online, making static recomputation too slow; we build Euler-tour trees and the Holm–de Lichtenberg–Thorup hierarchy to maintain connectivity in $O(\log^2 n)$ amortized time.
- In Chapter 40 (Streaming and Massive Graphs), the edge set is too large to store in RAM at all; we compute cuts, spanning forests, and approximate distances in $O(n \log^{O(1)} n)$ space using spanners and linear sketches.

38.10 Exercises

Exercise 38.1. Show that Algorithm 137 can return exactly 2 OPT by exhibiting an infinite family of graphs (hint: disjoint single edges, or a perfect matching) where it does. Then argue no analysis of this algorithm can beat the constant 2.

Exercise 38.2. Implement the LP-rounding weighted vertex cover and verify the 2-approximation on the weighted 5-cycle with vertex weights $1, 2, 1, 2, 1$. What does the LP optimum equal, and which vertices get fractional value $\frac{1}{2}$?

Exercise 38.3. Construct, for each k, a unit-cost set-cover instance on $n = 2^{k+1} - 2$ elements where greedy returns k sets but the optimum is 2. Conclude the analysis of Theorem 38.2 is tight up to a constant. (Hint: two "optimal" sets each of size $n/2$, plus a chain of greedily-tempting sets of sizes $n/2, n/4, n/8, \ldots$.)

Exercise 38.4. Build a metric instance where Christofides returns a tour of weight approaching $\frac{3}{2}\mathrm{OPT}$. (Hint: a long path with a carefully placed extra vertex so the odd set is {two endpoints} and the matching edge is expensive.) Then trace both Algorithm 139 and Algorithm 140 on it.

Exercise 38.5. Prove that for *general* (non-metric) TSP, the existence of a polynomial-time α-approximation for any constant α would imply $\mathsf{P} = \mathsf{NP}$. (Reduce from Hamiltonian cycle: weight 1 on graph edges, weight $\alpha n + 1$ on non-edges.)

Exercise 38.6. Show the metric-closure MST relaxation for Steiner tree has integrality gap exactly $2 - \frac{2}{|R|}$ on a star with one center and $|R|$ unit-distance terminals. Where does the $(1 - 1/|R|)$ in Theorem 38.4 come from in your example?

Exercise 38.7. Prove that LocalMaxCut (Algorithm 142) terminates in at most $|E|$ iterations and returns a cut of size $\geq |E|/2$. Give a graph where it gets stuck at a local optimum of value $\frac{1}{2}|E|$ while the global max-cut is $|E|$ (a bipartite example).

Exercise 38.8. Numerically minimize $f(\theta) = \dfrac{2\theta}{\pi(1 - \cos\theta)}$ over $(0, \pi]$ and confirm the minimizer $\theta^* \approx 2.331$ and value ≈ 0.87856. Why does the SDP objective being a *relaxation* let us replace SDP by OPT in the proof of Theorem 38.5?

Exercise 38.9. Verify that a cut metric δ_S embeds isometrically into ℓ_1 (one dimension suffices) and that sparsity evaluated on δ_S equals $\Phi(S)$ from Definition 38.2. Conclude that optimizing sparsity over the ℓ_1 cone of metrics has *no* integrality gap, so that all the Leighton–Rao loss is in the embedding step.

Exercise 38.10. On the n-vertex cycle C_n, compute the true sparsest cut, the Cheeger/spectral estimate (using $\lambda_2 = 2(1 - \cos\frac{2\pi}{n})$ for the unnormalized cycle Laplacian), and the Leighton–Rao LP value. Which approximation is sharper here, and why does the $O(\log n)$ bound look pessimistic on such a "nice" graph?

Exercise 38.11. Re-derive the vertex-cover 2-approximation as a *primal-dual* algorithm: grow dual edge variables y_e uniformly until some vertex's constraint $\sum_{e \ni v} y_e \leq w_v$ goes tight, add that vertex, repeat. Prove the cover cost is at most $2\sum_e y_e \leq 2\,\mathrm{OPT}$ and explain how this unifies the matching and LP views.

Chapter 39

Dynamic Graph Algorithms

In the preceding chapters, every algorithm operated on a fixed static graph: whether finding shortest paths, max-flow, or approximation guarantees, the graph held still while computation ran. But real graphs rarely hold still. A social network gains and loses friendships; a road map closes a bridge for repair; a circuit simulator toggles wires; a chess-like search adds and removes pieces on a board encoded as a graph. Recomputing from scratch after each change is the obvious thing to do, and for a single static query it is exactly right. When changes come in a long stream, recomputation is far too slow: an $O(m)$ rebuild after each of q updates costs $O(qm)$, and for $q \approx m$ that is quadratic.

Dynamic graph algorithms ask the sharper question. Given a graph that changes by small local edits – insert an edge, delete an edge, sometimes change a weight – maintain some property (connectivity, a spanning forest, the MST weight, the number of components, reachability, shortest-path distances) so that each **update** and each **query** is fast, ideally polylogarithmic. The goal is to store just enough auxiliary structure that a local change touches only a local, provably small part of it.

This chapter develops the core of the field. We fix vocabulary (incremental, decremental, fully dynamic; amortized versus worst-case), then build the two foundational data structures of the subject: Euler-tour trees for connectivity in a *forest*, and the Holm–de Lichtenberg–Thorup (HDLT) logarithmic-level structure that promotes forest connectivity to *fully dynamic* connectivity on general graphs in $O(\log^2 n)$ amortized time. We extend the same machinery to the dynamic minimum spanning tree. We trace a small graph through every structure by hand. Finally we turn to conditional lower bounds, organized around the Online Matrix–Vector (OMv) conjecture, that explain why so many dynamic problems are stuck at roughly $\sqrt{n}$ or polylog and not better.

39.1 A taxonomy of "dynamic"

Focus

Three update models, in increasing generality.

- **Incremental**: edges are only *inserted* (the graph grows).
- **Decremental**: edges are only *deleted* (the graph shrinks).
- **Fully dynamic**: both insertions and deletions, interleaved arbitrarily.

Two cost models, with different promises.

- **Amortized**: the bound holds *on average* over a sequence of operations; an individual operation may be slow as long as the running total is controlled.
- **Worst-case**: the bound holds for *every single* operation, no exceptions.

The split matters. Incremental connectivity is essentially free: a union-find (disjoint-set) forest answers "are u, v connected?" under edge insertions in $O(\alpha(n))$ amortized per operation, where α is the inverse Ackermann function (Chapter 6). Deletions are the hard direction, because union-find has no un-union; once two components merge, splitting them back apart on an edge deletion is exactly the operation the structure cannot do. Fully dynamic is hardest of all, and most of the work in this chapter goes into handling deletions.

Intuition

Why are deletions so much harder than insertions? An insertion can only *merge* information – two components become one, a smaller MST weight appears – and merging is monotone and easy to memoize. A deletion can *shatter* information: cutting one tree edge may or may not disconnect the graph, depending on whether some *other* edge, currently sitting idle, happens to reconnect the two halves. To answer that you must, in effect, search for a replacement edge, and finding that candidate edge without scanning the whole graph is the central difficulty in dynamic graph algorithms.

Reading note

Dynamic graph algorithms progressed from $O(\sqrt{m})$ via Frederickson's topology trees (1985) and $O(\sqrt{n})$ via sparsification (1992), to the polylogarithmic era launched by Henzinger and King (1995) and Holm, de Lichtenberg, and Thorup (1998); randomized worst-case polylog bounds arrived with Kapron, King, and Mountjoy (2013).

39.2 Warm-up: incremental connectivity and the limits of union-find

Start where it is easy. We maintain connected components under insertions only.

Algorithm 144: Incremental connectivity via union-find

```
Function Union(u, v):
    r_u ← Find(u);   r_v ← Find(v);
    if r_u = r_v then return;
    attach the smaller-rank root under the larger (union by rank);
Function Connected(u, v):
    return Find(u) = Find(v);
Function Find(x):
    if par[x] ≠ x then
        par[x] ← Find(par[x]) ;                          // path compression
    return par[x];
```

With union by rank and path compression, any sequence of m operations runs in $O(m\,\alpha(n))$ total, so $O(\alpha(n))$ amortized each. That is effectively constant, which settles the insertion-only case.

Common trap

The temptation is to "support deletion" by storing a `rank` and undoing the last union. That works only for a stack-like *rollback* pattern (union-find with rollback: no path compression, union by rank, undo in LIFO order, $O(\log n)$ per op). It does *not* give fully dynamic deletion of an *arbitrary* earlier edge, because the component that edge helped form may now be held together by many other edges, and removing the wrong internal pointer corrupts the forest. Rollback union-find is the right tool for *offline* dynamic connectivity (segment-tree-on-time, below), not for the online fully dynamic problem.

Connection

Rollback union-find plus a "segment tree over the timeline" solves *offline* fully dynamic connectivity: each edge is alive on a time interval $[t_{\text{in}}, t_{\text{out}})$; place it on the $O(\log q)$ canonical nodes of a segment tree over time, DFS the segment tree performing unions on entry and rollbacks on exit, and answer each query at its leaf. Total $O((m+q)\log q\,\log n)$. This is the dynamic-connectivity analogue of the offline trick in Chapter 21 (Euler tours) and is worth knowing because online methods are strictly harder. The rest of this chapter treats the online problem.

39.3 Euler-tour trees: connectivity in a forest

Before we can do general graphs, we need to do *forests* dynamically: support Link, Cut, and connectivity queries on a forest where the answer to a query never has to search for a replacement edge (there are no spare edges in a forest). Chapter 22's link-cut trees already do this, but for connectivity we prefer a structure that also makes *subtree* aggregates and "pick any edge incident to this tree" easy: the *Euler-tour tree* (ETT) of Henzinger and King, and Tarjan.

Intuition

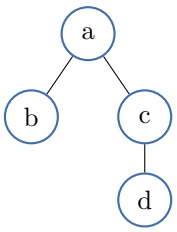
Recall the Euler tour of a tree (Chapter 21): walk the tree depth-first and write down each vertex every time you touch it. A tree with n vertices has $n-1$ edges; the closed Euler tour visits $2(n-1)$ edges and lists $2n-1$ vertex occurrences. The key move is to forget the tree shape and keep only this *sequence*. Then:

- "Are u and v in the same tree?" $\Leftrightarrow$ "do their occurrences lie in the same sequence?"
- LINK and CUT become *splits and concatenations* of sequences.

Store each sequence in a balanced BST keyed by position (a *balanced BST as a sequence*, i.e. an order-statistic / implicit-key treap or splay tree), and splits and concatenations are $O(\log n)$.

The Euler tour as a sequence

Represent the Euler tour as a list of *half-edges* and vertex visits. A clean encoding: for a tree, store the sequence of directed edges of the tour, plus one self-loop occurrence $\langle v, v\rangle$ per vertex so isolated vertices and endpoints have an anchor. Each undirected tree edge $\{u, v\}$ appears as two directed occurrences, $\langle u, v\rangle$ and $\langle v, u\rangle$. We keep, for each such occurrence, a pointer to its node in the balanced BST.



Euler tour (rooted at a)

a b a c d c a

as occurrences: $a_0\, b\, a_1\, c_0\, d\, c_1\, a_2$

edge $\{a, c\} \to$ occurrences
$\langle a, c\rangle$ (enter c), $\langle c, a\rangle$ (leave c)

Figure 39.1: A 4-node tree and one Euler tour. The tour string has $2n-1 = 7$ vertex occurrences. Connectivity of u, v is decided by whether their occurrences sit in the same balanced BST.

Link and Cut as sequence surgery

Why it works

Reroot. It is convenient to be able to make any vertex u the start of its tour. If the tour reads $S = X\,u\,Y$ where the marked u is some occurrence of u, then the rerooted tour is $u\,Y\,X\,u$. So rerooting at u is: split the sequence at u, swap the two halves, and reattach an occurrence of u at the end. One split, one concatenate.

Link(u, v) (join two different trees by edge $\{u, v\}$). Reroot u's tour at u and v's tour at v. Now u's tour starts with u and v's starts with v. The merged tour is

$$\underbrace{u \cdots}_{\text{tour}(u)}\ \langle u, v\rangle\ \underbrace{v \cdots}_{\text{tour}(v)}\ \langle v, u\rangle\ .$$

Concatenate tour(u), the new occurrence $\langle u, v\rangle$, tour(v), the new occurrence $\langle v, u\rangle$. A handful of splits/concatenations.

Cut$(\{u, v\})$**.** The edge contributes occurrences $\langle u, v\rangle$ and $\langle v, u\rangle$; between them sits exactly the Euler tour of v's subtree. Splitting the sequence just before $\langle u, v\rangle$ and just after $\langle v, u\rangle$ isolates that subtour as one piece; concatenating the outside two pieces removes the edge. Two splits, one concatenate.

Algorithm 145: Euler-tour tree operations (balanced-BST-as-sequence)

```
Function Reroot(u):
    let p be the BST node of any occurrence of u;
    (A, B) ← Split the sequence just before p ;          // A = X, B = u Y
    return Concat(B, A, new occ. u);
Function Link(u, v):
    T_u ← Reroot(u);   T_v ← Reroot(v);
    return Concat(T_u, ⟨u, v⟩, T_v, ⟨v, u⟩);
Function Cut(u, v):
    locate occurrences ⟨u, v⟩ and ⟨v, u⟩;
    split out the contiguous block from ⟨u, v⟩ to ⟨v, u⟩ inclusive;
    concatenate the two outside blocks; discard the two edge occurrences;
Function Connected(u, v):
    return TourRoot(u) = TourRoot(v) ;        // root of the BST containing an
     occurrence
```

Cost summary

Each ETT operation does $O(1)$ splits/concatenations on balanced sequences of length $O(n)$, hence $O(\log n)$ time (worst-case with a treap/red-black sequence; amortized with splay trees). Space is $O(n)$: there are $2n - 1$ occurrences per tree, summed to $O(n)$ over the forest. Connected is a "find the root of my BST" query, $O(\log n)$.

Trace it yourself

Trace LINK(b, d) on Figure 39.1 as if b and d were in *separate* trees (pretend the c–d side is its own tree containing d). Reroot tour$(b) = b$ at b: still b. Reroot the tour $c\,d\,c$ at d: it becomes $d\,c\,d$. Then the merged tour is $b\,\langle b, d\rangle\,d\,c\,d\,\langle d, b\rangle$. Write out the BST node sequence and confirm CONNECTED(b, c) now returns true while it was false before. Doing this once by hand shows why connectivity reduces to "same BST."

Why this matters

ETT also maintains *subtree* aggregates directly, because a subtree is a contiguous range of the tour (the block between $\langle u, v\rangle$ and $\langle v, u\rangle$). Augment the balanced BST with subtree sums and you get dynamic subtree-sum on a forest for free. What ETT does *not* give you, and link-cut trees do, are *root-to-node path* aggregates – a path is not a contiguous range of an Euler tour. Pick ETT for subtree/connectivity, LCT for path. This is a simple rule for choosing between the two dynamic-tree structures.

39.4 Fully dynamic connectivity: the HdLT logarithmic levels

We now want fully dynamic connectivity on a *general* graph: insert edge, delete edge, query "connected?" We maintain a spanning forest F of the current graph. An insertion is easy. The hard case is deleting a *tree* edge of F: we must decide whether some non-tree edge can replace it, and if so install that replacement, all without scanning everything.

Intuition

Keep a spanning forest F inside the graph G. Non-tree edges are "*spares*": they connect two vertices already connected through F.

- **Insert** $\{u, v\}$: if u, v already connected in F, store $\{u, v\}$ as a spare; else LINK it into F as a tree edge.
- **Query**: connectivity in F (an ETT query).
- **Delete a spare**: just remove it.
- **Delete a tree edge** $\{u, v\}$: CUT it, splitting F's tree into parts $T_u \ni u$ and $T_v \ni v$. Search for a spare edge with one endpoint in T_u and the other in T_v. If found, promote it to a tree edge (LINK); if none exists, the deletion genuinely disconnected the graph.

Everything hinges on finding that replacement spare cheaply. A naive search scans all spares incident to the smaller side, which can be $\Theta(m)$. The HdLT idea is a charging scheme that guarantees each spare is scanned a bounded number of times over its whole life.

Levels and the smaller-half rule

Focus

Assign every edge a **level** $\ell(e) \in \{0, 1, \ldots, L\}$ with $L = \lfloor \log_2 n \rfloor$. Maintain, for each level i, a spanning forest F_i of the subgraph G_i formed by edges of level $\geq i$. Two invariants:

(I1) $F_0 \supseteq F_1 \supseteq \cdots \supseteq F_L$; each F_i is a spanning forest of G_i, and $F := F_0$ is a spanning forest of all of G.

(I2) Every connected component of the subgraph induced by level-$\geq i$ edges has at most $\lfloor n/2^i \rfloor$ vertices.

A tree edge of F at level i is also a tree edge of $F_i, F_{i-1}, \ldots, F_0$. We keep an ETT for each level's forest F_i, augmented so each tour node knows whether its vertex has any incident *non-tree* edge at that level.

Invariant (I2) drives the analysis (Figure 39.2). Because a level-i component has $\leq n/2^i$ vertices and levels go up to $\log n$, an edge can be *promoted* (its level raised) at most $\log n$ times before its component would be too small to hold it. Each scan of an edge is paid for by its promotion. That is the amortized argument; the rest is bookkeeping.

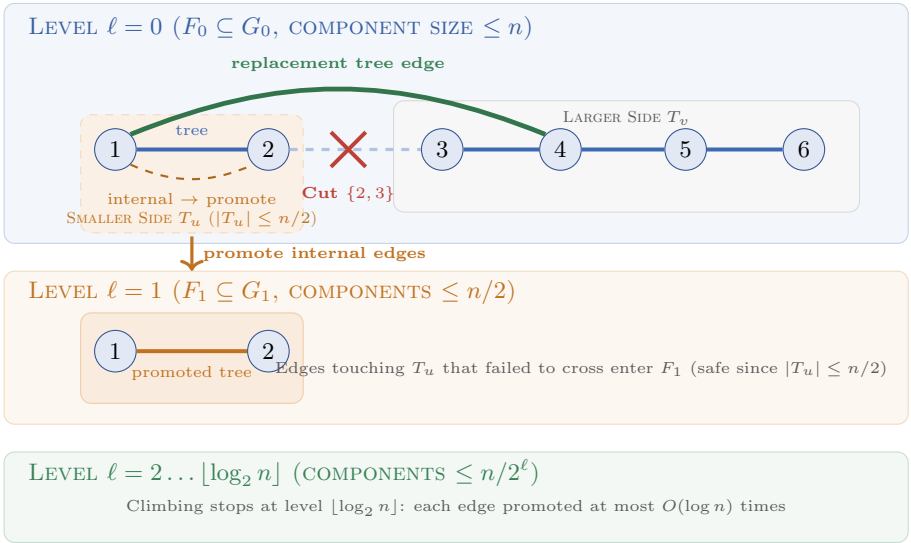


Figure 39.2: The Holm–de Lichtenberg–Thorup (HdLT) logarithmic level hierarchy. When tree edge $\{2, 3\}$ is cut at level 0, the smaller component T_u is inspected. Candidate edges with both endpoints in T_u are promoted to level 1 (invariant (I2) holds because $|T_u| \leq n/2$). A candidate edge $\{1, 4\}$ crossing between T_u and T_v is immediately selected as the replacement tree edge, reconnecting F_0. Because an edge climbs at most $\lfloor \log_2 n \rfloor$ levels, every edge scan is charged to a promotion, yielding $O(\log^2 n)$ amortized time.

Algorithm 146: HdLT fully dynamic connectivity

```
Function Connected(u, v):
    return u, v in same tree of F_0 (ETT query at level 0);
Function Insert(u, v):
    ℓ({u, v}) ← 0;
    if Connected(u, v) then store {u, v} as a non-tree edge at level 0;
    else add {u, v} to F_0 as a tree edge (ETT Link at level 0);
Function Delete(u, v):
    i ← ℓ({u, v});
    if {u, v} is a non-tree edge then
        remove it from the level-i adjacency;
    else
        remove {u, v} from F_i, F_{i−1}, ..., F_0 (ETT Cut at each level);
        Replace(u, v, i);
Function Replace(u, v, i):
    for j ← i downto 0 do
        let T_u, T_v be the two pieces of the level-j forest split by the cut;
        let T be the smaller of T_u, T_v (fewer vertices, via ETT subtree size);
        foreach level-j tree edge e inside T do
            raise ℓ(e) to j+1; add e to F_{j+1} ;          // promote tree edges
        foreach level-j non-tree edge e = {x, y} with x ∈ T do
            if y on the other side then
                add e to F_j, ..., F_0 as the replacement tree edge;
                return ;                                   // reconnected
            else
                raise ℓ(e) to j+1 ;          // both endpoints in T: promote
    // no replacement at any level: u, v are now genuinely disconnected
```

Why Replace is correct

Why it works

When we cut a level-i tree edge, we look for a replacement starting at the *highest* level i and descend. At level j we have split the level-j forest into T_u, T_v. We scan from the smaller side T. Two things can happen to each scanned edge:

- It is a non-tree edge crossing to the other side: it reconnects the two pieces. Use it, at level j, and stop. Correct because any spare crossing $T_u : T_v$ at this level restores connectivity, and a crossing edge of the *highest* possible level is safe to install without violating (I1)/(I2) below it.
- It is fully inside T (a tree edge of T, or a non-tree edge with both endpoints in T): we *promote* it to level $j{+}1$. This is allowed because T is the smaller half, so after the cut $|T| \le \lfloor \frac{1}{2}|T_u \cup T_v| \rfloor \le \lfloor \frac{1}{2}\lfloor n/2^j \rfloor \rfloor = \lfloor n/2^{j+1} \rfloor$, which is exactly the size budget (I2)

demands at level $j{+}1$ (the last step is the integer identity $\lfloor \lfloor x \rfloor /2 \rfloor = \lfloor x/2 \rfloor$). Promotion never breaks (I2).

If level j yields no crossing spare, every level-j spare touching T has been promoted, so none crosses; descend to $j{-}1$ and repeat on the coarser forest. If we exhaust level 0 without a crossing edge, then no edge of the graph crosses $T_u : T_v$ at all – the deletion truly disconnected the graph.

Formal

Theorem (HdLT). Fully dynamic connectivity is maintainable in $O(\log^2 n)$ amortized time per update and $O(\log n / \log\log n)$ time per query, using $O(m + n \log n)$ space.

Cost of a promotion. Each edge starts at level 0 and can be promoted at most $L = O(\log n)$ times, by (I2). Charge the cost of *scanning* an edge during REPLACE to a promotion: every edge we scan and keep is promoted one level (the only scanned edge we do *not* promote is the single crossing replacement, which we charge to the deletion that caused it). So the total number of edge-scan events over the whole run is $O(m \log n)$ scans, each costing an ETT update of $O(\log n)$ to move the edge between levels. That is $O(m \log^2 n)$ total promotion work, i.e. $O(\log^2 n)$ amortized per operation. Insertions and the L-fold CUT/LINK along the level forests add only $O(\log^2 n)$ each. The query bound uses a slightly cleverer ETT root-marking; the simple version gives $O(\log n)$.

Why it works

Finding a vertex of T that still has a level-j non-tree edge. We do not literally iterate all vertices of T to find incident spares. Augment each level-j ETT so every BST subtree records "does my range contain an occurrence whose vertex has a level-j non-tree edge?" Then "find any non-tree edge incident to T" is a single $O(\log n)$ descent into the BST of T. We pop edges from a per-vertex adjacency list one at a time, which is what lets us charge each examined edge to its promotion. This augmentation connects "ETT supports subtree aggregates" to "HdLT finds replacement edges fast."

Worked example

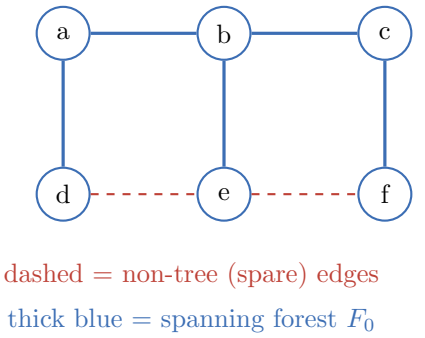


Figure 39.3: A graph on $\{a, \ldots, f\}$. Spanning forest F_0 is the five blue tree edges; $\{d, e\}$ and $\{e, f\}$ are spares (level 0).

Trace it yourself

Delete the tree edge $\{a, b\}$ in Figure 39.3. It is a level-0 tree edge, so CUT it; F_0 splits into $T_a = \{a, d\}$ and $T_b = \{b, c, e, f\}$. The smaller side is T_a (2 vertices). Scan level-0 edges touching T_a:

- tree edge $\{a, d\}$: inside T_a, promote it to level 1;
- spare $\{d, e\}$: endpoint $e \in T_b$ – *crossing*. Install it as the replacement tree edge.

F_0 becomes $\{a, d\}, \{d, e\}, \{b, e\}, \{b, c\}, \{c, f\}$, one connected tree again: the deletion did *not* disconnect the graph, and we found the replacement after touching just two edges from the smaller side. Now *also* delete $\{d, e\}$ and $\{e, f\}$ and then $\{a, b\}$ again, and watch a deletion finally report a real disconnection. Tracing this by hand is the fastest way to internalize the smaller-half charging.

Common trap

Three places people get HdLT wrong. (1) Scanning the *larger* half: the size-halving budget (I2) only holds for the smaller half; scan the wrong side and promotions can violate the level cap, destroying the amortized bound. (2) Forgetting to promote the level-j *tree* edges of T alongside the spares – if tree edges are left behind, F_{j+1} stops being a spanning forest of G_{j+1} and (I1) breaks. (3) Installing a replacement at the wrong level: the replacement edge must enter $F_j, \ldots, F_0$ (all levels at and below where it was found), or the lower forests lose connectivity.

Reading note

The smaller-half promotion charging mirrors union-by-size and small-to-large merging. HdLT combined this charging scheme with Euler-tour trees so that finding a replacement edge becomes a logarithmic-time BST descent, reducing the update bound from $O(\sqrt{n})$ to $O(\log^2 n)$.

39.5 Dynamic minimum spanning tree

Maintaining a minimum spanning forest under insertions and deletions is the same skeleton with one extra idea: replacement edges must be chosen by *weight*.

Intuition

Two classical facts (Chapter 6) drive everything.

- **Cycle rule.** Inserting edge e creates a cycle in the current MST T iff its endpoints are already connected. If so, e replaces the *maximum-weight* edge on the tree path between its endpoints, but only when e is lighter than that maximum; otherwise e becomes a spare.
- **Cut rule.** Deleting a tree edge e splits T into two sides; the correct replacement is the *minimum-weight* spare crossing the cut.

Figure 39.4 contrasts the two dual update primitives side by side: path-max replacement on an edge insertion versus lightest-crossing replacement on a tree-edge deletion.

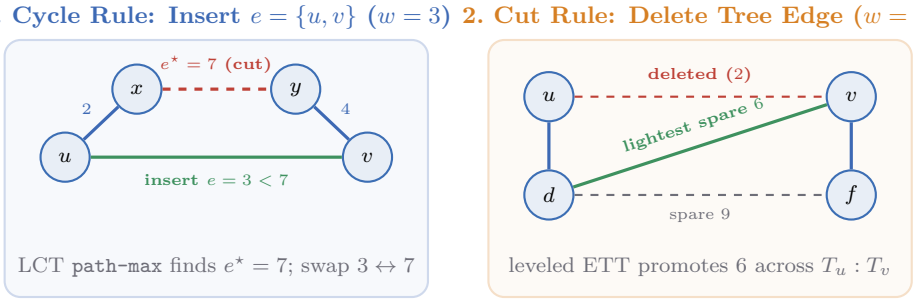


Figure 39.4: Dynamic MST update rules. Left (Cycle Rule): Inserting $e = \{u, v\}$ of weight 3 closes a cycle with the tree path u–x–y–v; a link-cut tree `path-max` query locates the heaviest path edge $e^\star = \{x, y\}$ (weight $7 > 3$) and replaces it with e. Right (Cut Rule): Deleting tree edge $\{u, v\}$ splits the tree into T_u and T_v; the leveled replacement search installs the lightest crossing spare (weight $6 < 9$).

Insertion: the cycle rule with path-max

Algorithm 147: Dynamic MST – edge insertion

```
Function InsertMST(u, v, w):
    if u, v in different trees of T then
        Link(u, v) with weight w ;                                  // forest grows
    else
        e* ← maximum-weight edge on the u–v tree path ;   // O(log n) via LCT path-max
        if w < weight(e*) then
            Cut(e*); demote e* to a spare; Link(u, v) with weight w;
        else store {u, v} as a spare of weight w;
```

The path-max query and the cut/link are exactly the link-cut tree primitives of Chapter 22: a link-cut tree stores the represented MST and answers "heaviest edge on the path u–v" in $O(\log n)$ amortized. So *incremental* MST is $O(\log n)$ per insertion. Deletion is the hard case.

Deletion: weighted replacement via leveled HdLT

Deleting a tree edge needs the *minimum-weight* crossing spare, which the plain HdLT replacement search does not order by weight. The fix (Holm–de Lichtenberg–Thorup, same paper) keeps the leveled-forest structure but, when searching for a replacement at a given level, takes the lightest crossing spare found. To make "lightest crossing" efficient, the levels are organized so that the search examines candidate spares in nondecreasing weight, and a careful argument shows the amortized cost remains $O(\log^2 n)$ per update for the minimum spanning *forest*.

Focus

Dynamic MSF result. The minimum spanning forest of a fully dynamic graph is maintainable in $O(\log^2 n)$ amortized time per insertion *and* per deletion (with the same leveled-ETT machinery, replacement edges chosen by weight). The MST *weight* is then available in $O(1)$ by keeping a running sum that is updated on every Link/Cut.

Why it works

Why weight-ordered replacement is correct. After cutting tree edge e, the two sides are T_u, T_v. By the cut rule, the unique correct replacement is the minimum-weight edge of G crossing $T_u : T_v$. Any *tree* edge of the old T other than e stays in the MST (removing e does not change which side those edges connect), so the only candidates are spares plus possibly nothing. Taking the lightest crossing spare therefore reproduces what Kruskal/Prim would pick on the reduced graph. The leveled structure only changes *how* we find that lightest crosser efficiently; correctness follows from the cut rule alone.

Trace it yourself

On Figure 39.3, attach weights: ab=1, bc=2, ad=3, be=4, cf=5 (tree), de=6, ef=7 (spares). Delete tree edge be (weight 4). Sides: the part containing e versus the rest. Crossing spares: de

(6) and ef (7). The cut rule installs the lighter, $de = 6$. New MSF weight $= 1+2+3+5+6 = 17$. Now instead delete ab (weight 1). Crossing spares across $\{a, d\} : \{b, c, e, f\}$: only $de = 6$. Replace; weight becomes $2+3+4+5+6 = 20$. Verify by recomputing the MST from scratch each time; the results should agree.

Connection

Dynamic MST is where Chapter 22 (link-cut trees, for the cheap incremental side and path-max) and this chapter's leveled ETT (for the hard decremental side) meet. The same path-max primitive powers Sleator–Tarjan dynamic-tree speedups of blocking-flow max-flow (Chapter 12–13). And the offline "segment tree over the timeline" from §39.2, combined with rollback union-find that tracks the maximum edge on each merge, gives a simple offline dynamic-MST that is often the practical choice when all updates are known in advance.

39.6 Sparsification: trading m for n

Many bounds above are stated in n, yet a graph can have $m = \Theta(n^2)$ edges. The reason updates do not degrade to m is *sparsification*, the Eppstein–Galil–Italiano–Nissenzweig technique, which applies to many problems and deserves a paragraph.

Intuition

For a property decided by a spanning forest (connectivity, MSF, bipartiteness, k-edge-connectivity for small k), the answer depends only on $O(n)$ *certificate* edges, not all m. Build a balanced binary tree over the edge set. Each leaf holds $O(n)$ edges; each internal node stores a *sparse certificate* of the union of its children's certificates – itself only $O(n)$ edges. An edge update touches one leaf and the $O(\log m)$ certificates on the path to the root, recomputing each from two $O(n)$-edge inputs. The dynamic structure then runs on a graph that always has $O(n)$ edges, so an $O(\sqrt{m})$ per-operation method becomes $O(\sqrt{n})$, and m disappears from polylog bounds entirely.

Cost summary

Sparsification multiplies the per-operation cost by $O(\log m)$ but replaces every m inside the base structure by n. It is the reason Frederickson's $O(\sqrt{m})$ topology trees became the $O(\sqrt{n})$ worst-case bound, and a clean way to keep m out of the polylog connectivity bounds even on dense graphs.

39.7 Worst-case versus amortized, and a glance at harder problems

HdLT is *amortized*: a single deletion can trigger many promotions and run in $\Theta(n)$ time, even though the average is $O(\log^2 n)$. For real-time systems (an individual operation must never stall) that is unacceptable, and worst-case bounds are genuinely harder to obtain.

Focus

Complexity bounds for fully dynamic connectivity.

- Amortized: $O(\log^2 n)$ (HdLT, deterministic), improved to $O(\log^2 n/\log\log n)$ (Wulff-Nilsen) and below with refinements.
- Worst-case: $O(\sqrt{n})$ deterministic (Frederickson + sparsification); $O(\text{poly}\log n)$ worst-case *randomized* via cut-set / XOR sketching (Kapron–King–Mountjoy 2013 and successors).
- Lower bound: $\Omega(\log n)$ per operation (Patrascu–Demaine), so no dynamic-connectivity structure can be truly constant-time.

Why this matters

The cut-set sketch, in one idea. Give each edge $\{u, v\}$ a random $O(\log n)$-bit id, and store at each vertex the XOR of ids of its incident edges. For a vertex set S, the XOR of the stored values over S cancels every edge with both endpoints inside S (each such id appears twice) and leaves the XOR of edges crossing the cut $(S, \bar{S})$. If exactly one edge crosses, that XOR *is* its id – you have recovered a crossing edge with no search at all. Layer $O(\log n)$ independent samplings to handle "more than one crosser," and you get a replacement edge in worst-case polylog time, with high probability. This idea underlies worst-case dynamic connectivity and is an example of linear sketching applied to graphs (Chapter 40).

39.8 Conditional lower bounds: the OMv wall

Why do so many dynamic problems plateau at $\sqrt{n}$ or polylog and not go faster despite decades of effort? The conjectured answer is that they cannot. The cleanest explanation is the *Online Matrix–Vector multiplication* (OMv) conjecture of Henzinger, Krinninger, Nanongkai and Saranurak (2015).

Focus

OMv problem. Fix an $n \times n$ Boolean matrix M (preprocessing allowed). Then receive vectors $v_1, \ldots, v_n \in \{0,1\}^n$ *online*: you must output Mv_t (Boolean matrix–vector product, $\vee/\wedge$) before seeing v_{t+1}.
OMv conjecture. No algorithm solves OMv in total time $O(n^{3-\varepsilon})$ for any constant $\varepsilon > 0$. (Even with arbitrary polynomial preprocessing of M.)

The naive algorithm computes each Mv_t in $O(n^2)$ for a total of $O(n^3)$; the conjecture says you cannot shave a polynomial factor off that when the queries are online. Why believe it? Because offline (all v_t known at once) it *is* beatable via fast matrix multiplication, yet decades of effort have not produced a truly subcubic *online* algorithm – the online constraint seems to forbid exactly the batching that fast matrix multiplication needs.

Why it works

From OMv to a dynamic lower bound: the reduction pattern (Figure 39.5). Encode the matrix M as a directed bipartite graph H on parts $A = \{a_1, \dots, a_n\}$, $B = \{b_1, \dots, b_n\}$ with a directed edge $b_j \to a_i$ iff $M_{ij} = 1$. To answer an online query vector v_t:

1. *Update phase*: for each j with $(v_t)_j = 1$, insert a directed edge from a fresh source s to b_j ("activate" column j). That is $O(n)$ edge insertions.
2. *Query phase*: for each i, a single dynamic query "is a_i reachable from s?" reveals whether $\bigvee_j (M_{ij} \wedge (v_t)_j)$, i.e. the i-th entry of Mv_t. That is $O(n)$ queries.
3. *Cleanup*: delete the $O(n)$ inserted s-edges to reset for v_{t+1}.

So one online matrix–vector product becomes $O(n)$ updates and $O(n)$ queries. If a dynamic reachability structure had $u(n)$ update and $q(n)$ query time, the whole OMv instance (with n query vectors) would run in $O(n \cdot n \cdot (u(n) + q(n))) = O(n^2(u + q))$. For this to beat $n^{3-\varepsilon}$ we would need $u(n) + q(n) = O(n^{1-\varepsilon})$. Hence: *under OMv, fully dynamic s–t reachability cannot have both update and query in $O(n^{1-\varepsilon})$.*

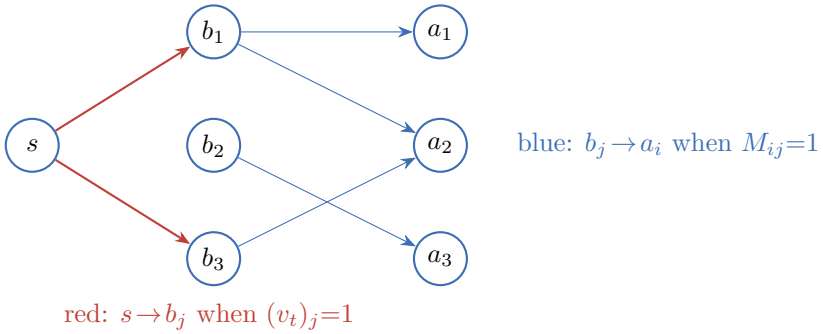


Figure 39.5: The OMv gadget for $v_t = (1, 0, 1)$. A directed query "is a_i reachable from s?" returns the i-th bit of Mv_t: here a_1 and a_2 are reachable (via b_1, and via b_1 or b_3), while a_3 is not, since its only in-edge is from the inactive b_2. So $Mv_t = (1, 1, 0)$.

Formal

Consequences (all conditional on OMv). There is no fully dynamic algorithm with $O(n^{1-\varepsilon})$ per operation (for constant $\varepsilon > 0$) for, among others:

- single-source / s–t **reachability** in directed graphs;
- **strong connectivity** (is the whole digraph one SCC?);
- maintaining whether the graph has a **perfect matching**;
- $O(\text{poly}\log)$-approximate shortest paths and many distance problems;
- subgraph-detection / triangle-type counting under updates.

The ε matters: these bounds rule out genuinely sublinear-per-operation algorithms, which is

exactly why $O(\sqrt{n})$ (for the undirected, easier problems that escape the reduction) or $O(n)$ (for the directed ones it traps) are the natural ceilings people hit.

Intuition

Undirected connectivity escapes OMv – the reduction needs the asymmetry of reachability, and undirected connectivity's spanning-forest structure is too well-behaved to encode matrix–vector products – which is precisely why we *can* do it in polylog. *Directed* reachability, exact shortest paths, and matching fall under the OMv reduction, which is why their dynamic versions are stuck near $O(n)$ per update. When a dynamic problem refuses to go below $\sqrt{n}$ or n after serious effort, suspect an OMv (or the related 3SUM / APSP) hardness reduction behind it.

Reading note

Conditional lower bounds based on the OMv conjecture unified dynamic hardness around 2015, alongside SETH, 3SUM, and APSP, proving that problems stuck at $\sqrt{n}$ or linear update times share common computational barriers under fine-grained complexity.

Connection

The OMv barrier is the dynamic-graph instance of *fine-grained complexity* (Chapter 37 covers NP-hardness; this is the polynomial-time analogue). It also explains the design of §39.6 and §39.4: we lean so hard on undirected, spanning-forest-based properties precisely because they are the ones that avoid the barrier. The streaming and sketching world of Chapter 40 attacks the same problems under a different resource (space), and the cut-set sketch of §39.7 belongs to both settings.

39.9 Putting it together

Cost summary

Problem	Model	Per-operation cost
Connectivity	incremental	$O(\alpha(n))$ amortized (union-find)
Connectivity (forest)	fully dynamic	$O(\log n)$ (Euler-tour tree)
Connectivity	fully dynamic	$O(\log^2 n)$ amortized (HdLT)
Connectivity	fully dynamic, w.c.	$O(\sqrt{n})$ det.; polylog rand. (sketch)
MST weight	incremental	$O(\log n)$ (link-cut, cycle rule)
Min spanning forest	fully dynamic	$O(\log^2 n)$ amortized (leveled ETT)
s–t reachability (dir.)	fully dynamic	$\Omega(n^{1-\varepsilon})$ under OMv

In summary: maintain a *spanning forest*, store everything else as spares, and handle deletions by a smaller-half charging argument that bounds how often any one edge is touched. Euler-tour trees

make "connectivity = same sequence" and "find an incident spare" both logarithmic; the leveled-forest wrapper turns the expensive replacement search into a bounded number of promotions. Where the spanning-forest idea does not apply – directed reachability, exact distances, matching – the OMv conjecture tells us, with high confidence, that no polylog method exists, and $\sqrt{n}$ or n is the best we can hope for.

39.10 Historical Context and Controversies

History

In the early decades of algorithm design, graph algorithms were strictly static. If a communication link failed or a new road opened, classic algorithms like Dijkstra's or Tarjan's DFS had to rerun from scratch, consuming linear time per update. The systematic pursuit of dynamic graph algorithms began in 1985, when Greg Frederickson introduced *topology trees* (hierarchical balanced tree clusterings), achieving $O(\sqrt{m})$ update times for minimum spanning trees. In 1992, David Eppstein, Zvi Galil, Giuseppe Italiano, and Amnon Nissenzweig developed sparsification trees, reducing the bound to $O(\sqrt{n})$ for general undirected graphs by recursively clustering vertices.

The breakthrough past the square-root barrier arrived in 1995, when Monika Henzinger and Valerie King introduced randomized Euler-tour trees (ETT, representing trees by their linear traversal sequences in balanced binary search trees) and edge sampling, achieving an amortized update time of $O(\log^3 n)$. In 1998, Jacob Holm, Kristian de Lichtenberg, and Mikkel Thorup (HdLT) designed the deterministic hierarchical level structure presented here, reaching $O(\log^2 n)$ amortized time per update. The limits became clearer in 2015 when Henzinger, Krinninger, Nanongkai, and Saranurak formulated the Online Matrix-Vector Multiplication (OMv) conjecture (hypothesizing that multiplying an online stream of vectors by a boolean matrix requires essentially cubic work), which gives a conditional, fine-grained explanation for thirty years of failed attempts to achieve polylogarithmic dynamic reachability on directed graphs.

Developing dynamic graph algorithms exposed subtle limitations in standard static data structures. Standard Disjoint-Set Union (DSU) handles insertions in near-constant time, but path compression irrevocably flattens tree structure; undoing operations via a rollback stack works only in offline LIFO order and fails when arbitrary edges are deleted online. In addition, when a tree edge is cut, searching blindly through non-tree edges to find a crossing spare can cost $\Omega(m)$ in dense graphs. The HdLT insight was charging replacement searches to edge promotions into the smaller half of the split, guaranteeing that an edge is promoted at most $\log n$ times before reaching its maximum level. An open problem is achieving deterministic worst-case polylogarithmic update times for general dynamic connectivity against an adaptive adversary.

39.10.1 What We Can Now Do and Its Limitations

Connection

Capabilities Unlocked:

- We can maintain undirected graph connectivity and minimum spanning forests dynamically under arbitrary edge insertions and deletions in $O(\log^2 n)$ amortized time.
- Euler-tour trees allow tree operations (Link, Cut, component membership, and subtree aggregates) in $O(\log n)$ worst-case time.
- The HdLT hierarchical edge-level framework prevents redundant edge examinations by guaranteeing that every unhelpful edge is permanently promoted to a higher level.

Limitations:

- *Amortized vs. worst-case latency*: The $O(\log^2 n)$ bound is strictly amortized. An adversarial deletion can trigger thousands of edge promotions across multiple levels, resulting in occasional $\Omega(n)$ worst-case latency spikes that are unacceptable in hard real-time systems.
- *The directed barrier*: Dynamic reachability and shortest paths on directed graphs cannot achieve polylogarithmic update times unless the OMv conjecture is false; they remain stuck at polynomial update times.
- *Memory footprint*: Maintaining $O(\log n)$ levels of Euler-tour trees and augmented balanced search trees incurs considerable pointer and memory overhead compared to static adjacency lists.

Where We Go Next:

- In Chapter 40 (Streaming and Massive Graphs), we consider graphs whose edge sets exceed main memory altogether. When edges arrive as a stream of insertions and deletions, we compress connectivity, cuts, and approximate distances into $O(n \operatorname{poly}\log n)$ space via spanners and linear L_0-sketches.
- In Chapter 41 (Laplacian Solvers and Electrical Flows), we explore how dynamic graph concepts interface with numerical optimization, where dynamic resistance updates accelerate electrical flow computations for maximum flow.

39.11 Exercises

Exercise 39.1 (Offline beats online)**.** Implement offline fully dynamic connectivity using a segment tree over the timeline plus rollback union-find (union by rank, no path compression, LIFO undo). Each edge is alive on an interval of operations; place it on the canonical segment nodes, DFS unioning on entry and rolling back on exit, and answer each query at its leaf. Prove the total time is $O((m+q)\log q \log n)$ and explain precisely why this technique fails when queries are *online*.

Exercise 39.2 (Euler-tour subtree sums)**.** Augment an Euler-tour tree so that each vertex carries a value and you support SubtreeSum(v) = sum of values in v's subtree, under Link, Cut, and point updates, all in $O(\log n)$. Argue why a subtree is a contiguous range of the tour, and explain why the same trick does *not* give you path sums (so you would need a link-cut tree for those).

Exercise 39.3 (The smaller-half budget)**.** In HdLT REPLACE, prove invariant (I2) is preserved: when you promote every level-j edge inside the smaller side T to level $j{+}1$, every level-$(j{+}1)$ component still has at most $\lfloor n/2^{j+1} \rfloor$ vertices. Then show that if you mistakenly scanned and promoted the *larger* half instead, (I2) can fail, and exhibit a sequence of operations whose amortized cost then degrades to $\Omega(\sqrt{n})$.

Exercise 39.4 (Counting components)**.** Extend HdLT so that NUMCOMPONENTS() returns the current number of connected components in $O(1)$. (Hint: a forest on n vertices with k trees has exactly $n-k$ tree edges; maintain the count of tree edges across all LINK/CUT.) State exactly which lines of Algorithm 146 must update the counter.

Exercise 39.5 (Incremental MST, the easy side)**.** Using a link-cut tree with path-max, implement INSERTMST (cycle rule) so that inserting an edge updates the MST in $O(\log n)$ amortized. Prove correctness from the cycle rule, and explain in one sentence why the *deletion* side cannot be handled by the link-cut tree alone (what does the cut rule need that a single tree cannot provide?).

Exercise 39.6 (Break the weight order)**.** Construct a small fully dynamic graph and a deletion where the HdLT replacement search, if it returns the *first* crossing spare it finds rather than the *minimum-weight* one, produces a spanning forest that is not minimum. Then fix the search to restore the cut rule and re-verify on your example.

Exercise 39.7 (The cut-set sketch)**.** Give each edge a uniformly random $2\lceil \log n \rceil$-bit id and store at each vertex the XOR of incident edge ids. Prove that for any vertex set S with exactly one edge crossing $(S, \bar{S})$, the XOR of stored values over S equals that edge's id. Then design an $O(\log n)$-layer scheme that, with high probability, recovers a crossing edge even when the number of crossers is unknown. Where does the failure probability come from?

Exercise 39.8 (OMv reduction, in full)**.** Write out the reduction of §39.8 for fully dynamic s–t reachability completely: specify the gadget graph, the update sequence per query vector v_t, how the n reachability queries reconstruct Mv_t, and the cleanup. Conclude that an $O(n^{1-\varepsilon})$-per-operation structure would refute OMv. Then identify exactly which step of your reduction breaks if the graph is *undirected* – and connect this to why undirected connectivity escapes the barrier.

Exercise 39.9 (Decremental only)**.** Suppose edges are only ever deleted (decremental connectivity), starting from a known graph. Show how to preprocess so that the *total* time over *all* deletions plus connectivity queries is $O(m \log^2 n)$, and argue why the decremental setting can sometimes beat the fully dynamic bound (hint: levels never need to go back down, so part of the bookkeeping disappears).

Exercise 39.10 (Worst-case via lazy rebuilding)**.** Sketch how to convert an amortized $O(\log^2 n)$ structure into one with $O(\sqrt{n})$ worst-case per operation by combining sparsification with periodic rebuilding of overloaded levels. Identify the trade-off: what do you give up in the amortized total to buy the worst-case guarantee?

Chapter 40

Streaming and Massive Graphs

In Chapter 39, we maintained connectivity and trees as edges were inserted and deleted, but we assumed the active graph could always be held in memory. That assumption breaks the moment the graph is a web crawl with 10^{11} edges, a social network arriving as a continuous stream of friend events, or a router log streaming past faster than any disk can absorb. In those regimes the graph never fully exists in RAM at once; we see edges one at a time in an order we cannot choose, and our working memory is only a small fraction of the input size. What graph properties can we still decide if we only have enough space to remember a few machine words per vertex?

This chapter develops the *semi-streaming* model ($O(n \operatorname{poly}\log n)$ space), showing how greedy forests, spanners, and linear sketches maintain structure, and where communication lower bounds prove that exact distances and triangle detection are impossible in small space.

40.1 The streaming model, precisely

Definition 40.1 (Graph stream)**.** Let $G = (V, E)$ have $n = |V|$ vertices, known in advance, labelled $1, \dots, n$. A *graph stream* is a sequence $\sigma = \langle a_1, a_2, \dots, a_m \rangle$ of *tokens* presented one at a time. In the *insertion-only* (or *cash-register*) model each token a_i is an edge $\{u, v\}$ to be added. In the *dynamic* (or *turnstile*) model each token is a pair $(\{u, v\}, \pm 1)$ that inserts or deletes an edge. An algorithm reads tokens left to right, keeps a memory state S of bounded size, and updates S per token. After the last token it answers a query about G.

Three knobs define the cost:

- **Space** s: bits of working memory, the scarce resource. Reading a token does not count against s; only what we choose to retain does.
- **Passes** p: how many times we are allowed to scan σ from the start. The default and most demanding setting is $p = 1$.
- **Per-token (update) time** and **post-processing time**: usually secondary, but we track them.

The input has n vertices and up to m edges, with m potentially as large as $\binom{n}{2} = \Theta(n^2)$. The aim is to use space $s \ll m$.

Definition 40.2 (Semi-streaming)**.** An algorithm is *semi-streaming* if it uses $O(n \operatorname{poly}\log n)$ bits of space and a small number of passes (ideally one).

Intuition

Why $n \operatorname{poly}\log n$ and not, say, $\sqrt{n}$ or $n^{1.5}$? Because $n \log n$ bits is exactly enough to store $O(n)$ machine words — one or a few per vertex. That is the natural "remember a little about everyone, forget the edges" frontier. Below it you cannot even hold a spanning forest; above it (n^{1+c}) you can hoard subquadratically many edges and the model stops being interesting. Semi-streaming is the regime where vertices fit but edges do not, and almost every positive result in this chapter is in it.

40.1.1 A running graph to keep in mind

Throughout, return to this little eight-vertex graph. Its edges will arrive in a scrambled order so we can watch an online algorithm cope.

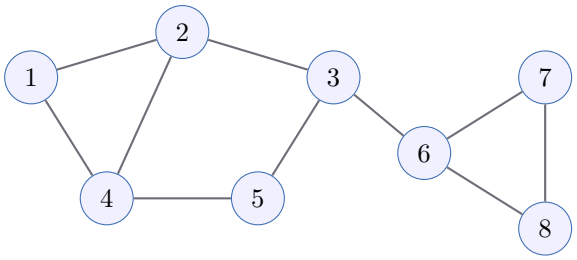


Figure 40.1: The running graph G: eight vertices, ten edges. Vertices $\{1,2,3,4,5\}$ form one cluster and $\{6,7,8\}$ a triangle, joined by the single bridge $\{3,6\}$. There is no edge 3–4, so the only triangle on the left is $\{1,2,4\}$; the only triangle on the right is $\{6,7,8\}$.

We will feed its edges in the adversarial order

$$\sigma = \langle\, (3,6), (1,2), (6,7), (2,4), (4,5), (1,4), (2,3), (3,5), (7,8), (6,8) \,\rangle.$$

40.2 Connectivity in one pass

The simplest semi-streaming result decides whether G is connected (and in fact maintains its connected components) in one pass and $O(n \log n)$ bits.

Intuition

A spanning forest already tells you the components, and a forest on n vertices has at most $n-1$ edges. So we never need to store more than $n-1$ edges. Maintain a forest incrementally: when an edge $\{u,v\}$ arrives, add it only if u and v are not already connected by what we have kept — otherwise it would close a cycle and is useless for connectivity. That is exactly Kruskal's forest-building, run online, with a union–find structure deciding "already connected?" in near-constant amortized time.

Algorithm 148: StreamingSpanningForest

Input: vertex set $V = \{1, \dots, n\}$; insertion stream of edges
Output: a spanning forest F; component labels

```
initialize union–find 𝒟 with each v in its own set
F ← ∅
foreach arriving edge {u, v} do
    if Find(u) ≠ Find(v) then
        F ← F ∪ {{u, v}}
        Union(u, v)
return F and the Find labels
```

Why it works

Correctness. Claim: at every moment F is a spanning forest of the graph seen so far, with the *same* connected components. Induction on tokens. Initially both are empty/trivial. When $\{u, v\}$ arrives: if u, v are already in the same tree of F, then by the induction hypothesis they were already connected in the seen graph, so adding the edge changes no component and skipping it changes no component — F stays a forest. If they are in different trees, the seen graph just merged two components, and adding $\{u, v\}$ to F merges exactly those two trees without creating a cycle (it joins two distinct trees). So F remains a forest realizing the same partition. After all tokens, G is connected iff F is a single tree iff $|F| = n - 1$.

Cost summary

F holds at most $n - 1$ edges $= O(n \log n)$ bits; union–find adds another $O(n \log n)$. Total space $O(n \log n)$ — semi-streaming with one pass. Per-token work is one Find/Union pair, $O(\alpha(n))$ amortized with path compression and union by rank. Post-processing is $O(n)$.

Trace it yourself

Run Algorithm 148 on σ from Figure 40.1. Write the forest edges kept and the running component count (start: 8 singletons).

token	action	kept?	#comp
$(3, 6)$	$3, 6$ separate	yes	7
$(1, 2)$	separate	yes	6
$(6, 7)$	separate	yes	5
$(2, 4)$	separate	yes	4
$(4, 5)$	separate	yes	3
$(1, 4)$	$1, 4$ already linked (1–2–4)	**no**	3
$(2, 3)$	joins $\{1, 2, 4, 5\}$ with $\{3, 6, 7\}$	yes	2
$(3, 5)$	$3, 5$ already linked	**no**	2
$(7, 8)$	joins 8 in	yes	1
$(6, 8)$	already linked	**no**	1

We kept exactly $7 = n - 1$ edges and ended at one component: G is connected. The three edges we dropped, $(1, 4), (3, 5), (6, 8)$, are precisely the cycle-closing ones — the "extra" density the

forest is allowed to forget.

40.2.1 Bipartiteness and k-edge-connectivity

The same skeleton answers more questions with a constant factor more space.

Bipartiteness. Keep the spanning forest *plus* a parity bit $\phi(v) \in \{0,1\}$ per vertex (the side of the 2-coloring relative to its tree root, maintained as union–find merges). When a non-tree edge $\{u,v\}$ arrives with $\textsc{Find}(u) = \textsc{Find}(v)$, check whether $\phi(u) = \phi(v)$; if so we have found an odd cycle and the graph is not bipartite. This is one pass, $O(n \log n)$ bits.

k-edge-connectivity. Maintain k edge-disjoint forests $F_1, \dots, F_k$ (Nagamochi–Ibaraki / Thurimella sparsification): insert each arriving edge into the first F_i where it does not close a cycle. The union $F_1 \cup \cdots \cup F_k$ is a *k-skeleton* preserving all cuts of size $\le k$, so it answers k-edge-connectivity exactly. Space $O(kn \log n)$: semi-streaming for $k = \operatorname{poly}\log n$.

Connection

The forest trick is Kruskal (Chapter 6) without sorting: the stream supplies an arbitrary order and we keep whatever extends the forest. The k-forest sparsifier is the combinatorial counterpart of the *spectral* sparsifier previewed in Chapter 35 and developed in Chapter 41: there we preserve every quadratic form $x^\top \mathbf{L} x$ to within $1 \pm \varepsilon$ using $O(n \log n/\varepsilon^2)$ *weighted* edges; here we preserve every cut of value $\le k$ exactly using kn *unweighted* edges. Both make the same point: a graph's connectivity structure is compressible to near-linear size.

40.3 Spanners: approximate distances in small space

A spanning forest forgets distances completely — it can stretch a length-1 edge into a length-$\Theta(n)$ detour. If we want approximate shortest paths in one pass, we need a *spanner*.

Definition 40.3 (Spanner)**.** A subgraph $H \subseteq G$ is a *t-spanner* (stretch t) if for all u, v,

$$\operatorname{dist}_G(u,v) \;\le\; \operatorname{dist}_H(u,v) \;\le\; t \cdot \operatorname{dist}_G(u,v).$$

Equivalently (it suffices to check edges): for every edge $\{u,v\} \in G$, $\operatorname{dist}_H(u,v) \le t$.

The "edges suffice" remark is worth a line: if every edge is t-stretched, then concatenating along any shortest path stretches the whole path by at most t. So a streaming spanner only has to guarantee, edge by edge, that it never drops an edge whose endpoints are currently far apart in H.

Intuition

Greedy spanner, online: when $\{u,v\}$ arrives, ask "are u and v already within distance t using the edges I have kept?" If yes, the new edge is redundant — drop it. If no, keep it. This guarantees stretch t by construction. The payoff is that the kept graph H then has *high girth*: every cycle in H has length $> t+1$, because the last edge of any short cycle would have been dropped. High girth forces sparsity.

Algorithm 149: StreamingGreedySpanner(t), $t = 2k-1$ odd

Input: stream of edges; stretch parameter $t = 2k-1$
Output: a t-spanner H

```
H ← ∅
foreach arriving edge {u, v} do
    compute d ← dist_H(u, v) by BFS in H (stop early at depth t)
    if d > t (includes d = ∞) then
        H ← H ∪ {{u, v}}
return H
```

Why it works

Stretch. For any kept edge, directly $\mathrm{dist}_H \le 1 \le t$. For any *dropped* edge $\{u,v\}$, we dropped it precisely because $\mathrm{dist}_H(u,v) \le t$ at that moment, and H only grows, so the bound persists. Hence every original edge is t-stretched, and by the edge-suffices remark H is a t-spanner.
Sparsity via girth. Suppose H contained a cycle C of length $\ell \le t+1 = 2k$. Consider the *last* edge $\{u,v\}$ of C to be added to H. At that instant the other $\ell - 1 \le t$ edges of C were already in H, forming a u–v path of length $\ell - 1 \le t$, so $\mathrm{dist}_H(u,v) \le t$ and we would have *dropped* $\{u,v\}$ — contradiction. Therefore H has girth $\ge t+2 = 2k+1$.

Lemma 40.1 (High-girth graphs are sparse, the Moore bound)**.** *A graph on n vertices with girth $\ge 2k+1$ has $O(n^{1+1/k})$ edges.*

Proof sketch. If the average degree is $d = 2|E|/n$, a standard cleaning gives a subgraph of minimum degree $\ge d/2$. Pick any vertex and grow a BFS tree to depth k; girth $\ge 2k+1$ means no two of these short paths collide before depth k, so the ball is a genuine tree and contains at least $1 + (d/2) + (d/2)^2 + \cdots$ down to depth k, i.e. $\gtrsim (d/2)^k$ vertices. This must be $\le n$, giving $d/2 \le n^{1/k}$, so $|E| = nd/2 \le n \cdot n^{1/k} = n^{1+1/k}$ up to the additive cleaning term. □

Cost summary

Algorithm 149 with $t = 2k-1$ stores an $O(n^{1+1/k})$-edge $(2k-1)$-spanner. Choosing $k = \log n$ gives stretch $O(\log n)$ and space $O(n^{1+1/\log n} \log n) = O(n \log n)$ bits — semi-streaming. The per-token cost is one truncated BFS, $O(|E(H)|)$ in the worst case; the additive-error and faster variants of Baswana–Sen reduce this to $O(1)$ expected per edge.

Trace it yourself

Build a 3-spanner ($t = 3$, $k = 2$) of the running graph in stream order σ. Rule: keep $\{u,v\}$ unless $\mathrm{dist}_H(u,v) \le 3$ already.

token	current $\text{dist}_H(u,v)$	keep?
$(3,6)$	∞	yes
$(1,2)$	∞	yes
$(6,7)$	∞	yes
$(2,4)$	∞	yes
$(4,5)$	∞	yes
$(1,4)$	$1\text{–}2\text{–}4 = 2 \le 3$	**no**
$(2,3)$	∞ ($\{1,2,4,5\}$ vs $\{3,6,7\}$)	yes
$(3,5)$	$3\text{–}2\text{–}4\text{–}5 = 3 \le 3$	**no**
$(7,8)$	∞	yes
$(6,8)$	$6\text{–}7\text{–}8 = 2 \le 3$	**no**

Kept: $\{(3,6),(1,2),(6,7),(2,4),(4,5),(2,3),(7,8)\}$ — seven edges, a tree here (the graph is small so the spanner coincides with a forest). On a denser instance the spanner would keep cycle edges that a forest discards, in exchange for the stretch guarantee. Sanity check on the dropped edge $(3,5)$: in H, $\text{dist}(3,5) = 3 = 3 \cdot \text{dist}_G(3,5)$, exactly the stretch ceiling. Figure 40.2 draws the kept and dropped edges.

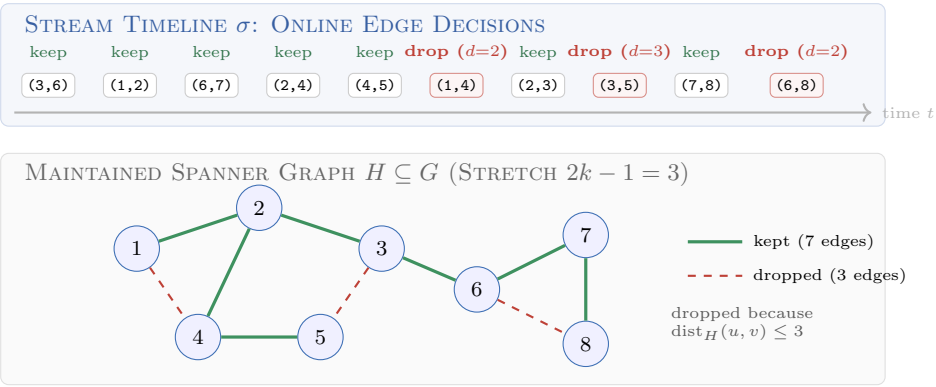


Figure 40.2: Semi-streaming greedy construction of a 3-spanner on the running graph under adversarial stream order σ. Top lane: sequential stream arrivals. Each token $\{u,v\}$ is inspected; if $\text{dist}_H(u,v) \le 3$ via existing edges, it is dropped as redundant, otherwise it is retained. Bottom lane: the resulting spanner H. Solid green edges form H (7 edges, preserving connectivity and bounding stretch to ≤ 3); dashed red edges $\{1,4\},\{3,5\},\{6,8\}$ were dropped and use no memory.

Common trap

"Isn't a spanning forest already a spanner?" Only with worst-case stretch $n-1$: deleting a single edge of an n-cycle leaves a spanning path whose endpoints jump from distance 1 to distance $n-1$. A spanning forest preserves *connectivity* in $n-1$ edges, whereas preserving *distances* within multiplicative stretch $2k-1$ requires keeping extra cycle edges and paying the $O(n^{1+1/k})$ space trade-off of Lemma 40.1.

40.4 Dynamic streams: when edges get deleted

Algorithms 148 and 149 fail under deletions. The spanning forest may keep edge $\{u, v\}$, and later a token $\big(\{u, v\}, -1\big)$ removes it — but the forest has no idea which *other* edge should take its place, because it deliberately threw those edges away. Greedy retention cannot be undone.

The fix is to store, instead of a chosen subset of edges, a small *linear* summary of the entire edge multiset. Linearity is what makes deletions easy: a deletion is just an insertion of -1, and the summary updates by subtraction.

Pause

Pause and think: Why does deleting an edge break a greedy forest so badly? In a static graph or an insertion stream, discarding cycle-closing edges is safe because the kept tree already connects their endpoints. But the moment a tree edge is deleted, the tree splits into two components. Any replacement edge that could reconnect them was discarded when it arrived earlier in the stream. The algorithm has no way to recover it.

Figure 40.3 traces this failure mode on the triangle $\{1, 2, 4\}$ from the running graph: once the cycle-closing edge $(1, 4)$ is discarded, a later deletion of the tree edge $(2, 4)$ leaves the forest disconnected even though $(1, 4)$ still bridges the cut in G.

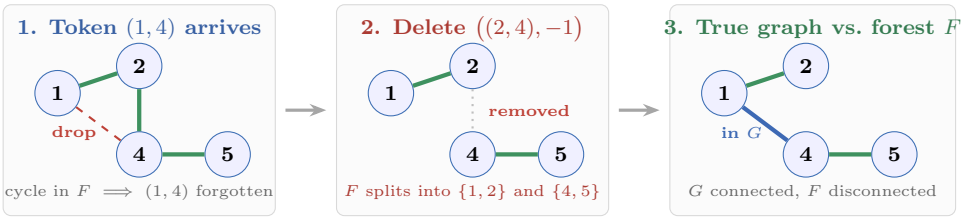


Figure 40.3: Why greedy spanning forests fail on turnstile streams. (1) When $(1, 4)$ arrives, 1 and 4 are already linked via 1–2–4, so $(1, 4)$ is discarded. (2) Later, token $\big((2, 4), -1\big)$ deletes the tree edge $(2, 4)$, splitting F. (3) Edge $(1, 4)$ is still present in G and reconnects the cut, but the algorithm discarded it in Step 1 and cannot recover it.

40.4.1 The vertex–edge incidence vector

Index the coordinate space by unordered pairs $\binom{V}{2}$, of dimension $\binom{n}{2}$. For a vertex v, define its *signed incidence vector* $x_v \in \mathbb{R}^{\binom{n}{2}}$: for each edge $\{u, v\}$ present in the graph, x_v has a $+1$ or -1 in coordinate $\{u, v\}$, with the sign chosen by a fixed global rule, say $+1$ if $v < u$ and -1 if $v > u$.

Lemma 40.2 (Boundary cancellation)**.** *For any vertex subset $S \subseteq V$, the support of $\sum_{v \in S} x_v$ is exactly the set of edges crossing the cut $(S, \bar{S})$.*

Proof. Coordinate $\{u, w\}$ receives a contribution from x_u and from x_w only. If both $u, w \in S$, the two contributions have opposite signs (one endpoint sees the edge as "smaller", the other as "larger") and cancel to 0. If both are outside S, neither contributes. If exactly one is in S, the coordinate survives with value ± 1. So the surviving coordinates are precisely the cut edges of S. □

Intuition

This uses the same signed vertex-edge incidence matrix B from Chapter 31 ($\mathbf{L} = B^\top B$), applied incrementally to an edge stream. Summing rows of the incidence matrix over S cancels internal edges and leaves the boundary ∂S. So if we could, for any S, recover *one* nonzero coordinate of $\sum_{v\in S} x_v$, we would learn one edge leaving S — enough to grow a spanning forest by repeatedly merging components and asking for an outgoing edge. The catch: $\sum_{v\in S} x_v$ lives in $\binom{n}{2}$ dimensions and we cannot store it. We store a tiny *sketch* of each x_v instead, and exploit that the sum of sketches is the sketch of the sum.

40.4.2 L_0 sampling

The primitive we need is a sampler that, from a high-dimensional vector accessed only through a small linear sketch, returns a uniformly random nonzero coordinate.

Definition 40.4 (L_0 sampler)**.** An L_0 sampler for vectors $x \in \mathbb{R}^N$ is a randomized linear map $\mathsf{Sk} : \mathbb{R}^N \to \mathbb{R}^s$ together with a recovery procedure Rec such that, from $\mathsf{Sk}(x)$ alone, with probability $\geq 1-\delta$, Rec outputs a coordinate i with $x_i \neq 0$, and conditioned on success i is (near-)uniform over $\mathrm{supp}(x)$. Such samplers exist with $s = O(\log^2 N \cdot \log(1/\delta))$.

The standard construction *subsamples by levels*. Hash each coordinate to a geometric level j with $\Pr[\text{level} \geq j] = 2^{-j}$. At level $j \approx \log\|x\|_0$ the survivors number $\Theta(1)$ in expectation; at that level keep a tiny linear "1-sparse recovery" gadget that can decode a single nonzero coordinate exactly. Run all $O(\log N)$ levels in parallel, read off the first level that is 1-sparse. Each gadget is linear, so the whole sampler is linear, so deletions are handled by subtraction. A 1-sparse recovery gadget stores three running linear combinations $\big(\sum_i x_i,\ \sum_i i\,x_i,\ \sum_i \mathrm{fp}(i)\,x_i\big)$; when the support is a single index $i^\star$ with value a, the first slot is a, the second is $a\,i^\star$, and dividing recovers $i^\star = (\text{slot2})/(\text{slot1})$, with the fingerprint slot detecting (whp) that the support really was size one.

Why it works

Why it is exactly linear, and why that matters. Every quantity stored — the level hashes are fixed in advance, and each gadget keeps only *sums* $\sum_i c_i x_i$ for fixed coefficients c_i. When token $(\{u,v\}, \pm 1)$ arrives we add ± 1 to two incidence vectors x_u, x_v, which adds $\pm c_{\{u,v\}}$ into the relevant sums. Insertion and deletion are the same operation with opposite sign. The sketch never "remembers which edges were kept"; it remembers numerical aggregates that automatically forget a deleted edge. This is why linear sketches work on turnstile streams while greedy forests do not.

40.4.3 Connectivity in dynamic streams (AGM sketching)

Now assemble the pieces. Maintain, for each vertex v, an independent collection of $O(\log n)$ L_0 sketches of its incidence vector x_v — one fresh sketch per round of a Borůvka-style component merge.

Algorithm 150: SketchConnectivity

Input: turnstile edge stream; $R = \lceil \log_2 n \rceil + 1$ rounds
Output: a spanning forest of the final graph

During the stream: for each v keep R independent L_0 sketches $\mathsf{Sk}_1(x_v), \dots, \mathsf{Sk}_R(x_v)$, updated linearly per token
After the stream (Borůvka):
$\mathcal{C} \leftarrow$ partition of V into singletons; $F \leftarrow \emptyset$
for $r = 1$ **to** R **do**
  foreach *component* $S \in \mathcal{C}$ **do**
    using *round-r* sketches, compute $\mathsf{Sk}_r\big(\sum_{v \in S} x_v\big) = \sum_{v \in S} \mathsf{Sk}_r(x_v)$
    run Rec to sample an edge leaving S (a nonzero coordinate)
  add every sampled inter-component edge to F and merge the components it joins, updating $\mathcal{C}$
return F

Why it works

Why $R = O(\log n)$ rounds suffice. Each Borůvka round, every current component samples (at least) one outgoing edge if one exists, so every non-maximal component merges with another. The number of components therefore at least halves each round; after $\lceil \log_2 n \rceil$ rounds it stops shrinking, and the partition equals the true connected components. Each round must use a *fresh independent* sketch: the components queried in round r depend on the random outcomes of rounds $1, \dots, r-1$, so reusing a sketch would correlate the query set with the sketch's randomness and void the L_0 guarantee. That is why we pay $O(\log n)$ sketches per vertex, not one.

Why the sum-of-sketches step is valid. By linearity $\sum_{v \in S} \mathsf{Sk}_r(x_v) = \mathsf{Sk}_r(\sum_{v \in S} x_v)$, and by Lemma 40.2 that argument vector is supported exactly on the cut ∂S. An L_0 sample from it is therefore a (uniform) edge leaving S, which is exactly the primitive Borůvka's algorithm needs.

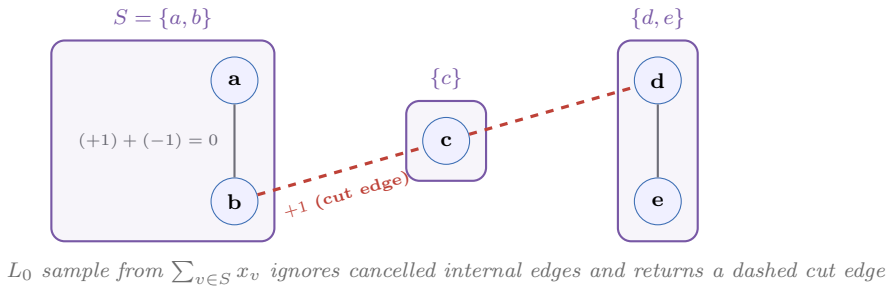


Figure 40.4: One Borůvka round under sketching. Summing the incidence sketches over a component S cancels its internal solid edges (Lemma 40.2) and leaves only the dashed boundary edges; the L_0 sampler returns one of them, merging S with a neighbor.

Cost summary

To make every one of the $O(n \log n)$ component-queries across all rounds succeed simultaneously, take the per-sketch failure to $\delta = 1/\operatorname{poly}(n)$, so each L_0 sketch is $O(\log^2 N \cdot \log(1/\delta)) = O(\log^3 n)$ bits (Definition 40.4 with $N = \binom{n}{2}$, $\log N = \Theta(\log n)$). Per vertex we keep $R = O(\log n)$ of them, giving $O(\log^4 n)$ bits per vertex and $O(n \log^4 n)$ bits total — semi-streaming, one pass, and it handles arbitrary insertions and deletions. Sharper L_0 samplers shave this to the $O(n \log^3 n)$ headline figure of the original AGM analysis. The classical insertion-only forest (Algorithm 148) is cheaper at $O(n \log n)$; sketching pays the extra log factors precisely to buy deletion-tolerance.

Connection

Lemma 40.2 applies the same boundary operator B (the signed incidence matrix) studied in Chapter 31: $\sum_{v \in S} x_v = B^\top \mathbf{1}_S$ is the indicator of the cut, and $\|B^\top \mathbf{1}_S\|_0 = |\partial S|$. Graph sketching runs Borůvka's MST/connectivity algorithm (Chapter 6) over a compressed coordinate space, with the cut/boundary algebra of Chapter 31 supplying the correctness. The same incidence-sketch idea extends (Ahn–Guha–McGregor) to spanners, sparsifiers, and even spectral sparsifiers in turnstile streams.

40.5 Approximate triangle counting

Let T be the number of triangles in G. Triangle counts measure clustering — the tendency of friends-of-friends to be friends — and feed network-science statistics like the global clustering coefficient. Computing T exactly in small space is impossible (Section 40.6); we estimate it.

40.5.1 A clean sampling estimator (one pass)

Intuition

A triangle is three edges $\{a,b\}, \{b,c\}, \{a,c\}$. Pick one edge at random as the "base", then a third vertex at random, and *watch* the rest of the stream to see whether the two edges that would complete the triangle on that vertex actually show up. If we structure the random choice so that each triangle has a known probability of being "set up" and then "confirmed", the indicator of confirmation is an unbiased $1/p$-scaled estimator of T.

A concrete one-pass estimator (Buriol et al.): sample a uniformly random position i in the stream and a uniformly random vertex $w \notin$ the edge $e_i = \{u, v\}$ at that position. Then scan the *rest* of the stream; let the indicator $X = 1$ if both edges $\{u, w\}$ and $\{v, w\}$ appear after position i. Each triangle $\{u, v, w\}$ is detected exactly when we sampled its earliest edge as e_i and picked the opposite vertex w, an event of probability $\frac{1}{m} \cdot \frac{1}{n-2}$. Hence

$$\mathbb{E}[X] \;=\; \frac{T}{m\,(n-2)}, \qquad \text{so} \qquad \widehat{T} \;=\; m\,(n-2)\,X$$

is unbiased: $\mathbb{E}[\widehat{T}] = T$. Averaging s independent copies and applying Chebyshev drives the variance down.

Pause

Pause and think: Why is exact triangle counting impossible in small space during a single pass? If we see edges $\{a, b\}$ and $\{b, c\}$ early on, we cannot predict whether $\{a, c\}$ will appear later. To detect every triangle exactly, an algorithm would have to remember almost every two-hop path in the graph, which requires storing nearly all the edges.

Algorithm 151: TRIANGLEESTIMATE (one independent copy)

Input: insertion stream $e_1, \dots, e_m$; n, m
Output: unbiased estimate $\widehat{T}$ of triangle count

pick uniform $i \in \{1, \dots, m\}$ via reservoir sampling
`// when token` $e_i = \{u, v\}$ `is the current reservoir choice:`
pick uniform $w \in V \setminus \{u, v\}$; set $a \leftarrow b \leftarrow$ false
foreach *token* e_j *with* $j > i$ **do**
    if $e_j = \{u, w\}$ **then** $a \leftarrow$ true
    if $e_j = \{v, w\}$ **then** $b \leftarrow$ true
$X \leftarrow [\, a \wedge b \,]$
return $\widehat{T} \leftarrow m\,(n-2)\,X$

Why it works

Unbiasedness. Fix a triangle $\tau = \{u, v, w\}$ and let e_i be its first edge in stream order, say $\{u, v\}$. The copy "catches" τ iff the reservoir landed on that very position *and* the random third vertex equals the remaining vertex w. These choices are uniform and independent, so $\Pr[\text{catch } \tau] = \frac{1}{m} \cdot \frac{1}{n-2}$, and given a catch both completing edges appear later (by definition of "first edge"), so $X = 1$. Catches of distinct triangles are disjoint events for a single copy, hence $\mathbb{E}[X] = \sum_{\tau} \frac{1}{m(n-2)} = \frac{T}{m(n-2)}$ and $\mathbb{E}[\widehat{T}] = T$.

Cost summary

One copy uses $O(1)$ words (reservoir index, the chosen edge, w, two bits). Reducing variance to get a $(1 \pm \varepsilon)$ estimate with confidence $1 - \delta$ requires $s = O\big(\frac{1}{\varepsilon^2} \cdot \frac{m\Delta}{T} \cdot \log \frac{1}{\delta}\big)$ parallel copies, where Δ bounds how many triangles share an edge — intuitively, the rarer the triangles, the more samples you must spend. With T large relative to m this is genuinely sublinear; with T tiny it degrades, which the lower bound below shows is unavoidable.

Trace it yourself

The running graph has exactly two triangles: $\{1, 2, 4\}$ and $\{6, 7, 8\}$, so $T = 2$, with $m = 10$, $n = 8$, giving $\widehat{T} = 10 \cdot 6 \cdot X = 60X$ per copy. The triangle $\{6, 7, 8\}$ has edges at positions $3\,(6, 7)$, $9\,(7, 8)$, $10\,(6, 8)$; its first edge is $(6, 7)$ at position 3. A copy catches it iff the reservoir picks $i = 3$ *and* $w = 8$: probability $\frac{1}{10} \cdot \frac{1}{6} = \frac{1}{60}$, contributing $60 \cdot \frac{1}{60} = 1$ to $\mathbb{E}[\widehat{T}]$. The triangle $\{1, 2, 4\}$ contributes another 1. Sum: $\mathbb{E}[\widehat{T}] = 2 = T$. A single copy almost always reports 0 and occasionally 60; only the *average* of many copies is informative, as is typical of high-variance estimators.

Figure 40.5 shows how one copy of Algorithm 151 catches the triangle $\{6, 7, 8\}$ on σ: once the

reservoir locks onto the earliest edge $e_3 = (6, 7)$ and selects $w = 8$, the remaining stream only needs to flip two boolean flags as $(7, 8)$ and $(6, 8)$ stream past.

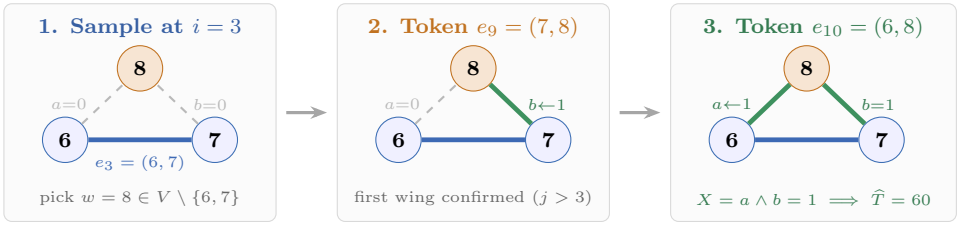


Figure 40.5: One-pass triangle estimation (Algorithm 151) catching $\{6, 7, 8\}$ in stream σ. (1) Reservoir sampling selects position $i = 3$ with edge $e_3 = (6, 7)$ and a uniform third vertex $w = 8$, initializing flags $a = b = 0$. (2) Later in the stream, $e_9 = (7, 8)$ sets $b \leftarrow 1$. (3) Finally $e_{10} = (6, 8)$ sets $a \leftarrow 1$, confirming the wedge ($X = 1$) and reporting $\widehat{T} = m(n - 2)X = 60$.

Application

Triangle and clustering-coefficient counts on streamed graphs drive spam and bot detection (fraud rings form unusually dense triangle clusters), recommendation ("triadic closure" predicts new links), and bioinformatics (motif counts in protein interaction networks). Real systems — think a social platform ingesting billions of edge events per day — cannot store the graph, so unbiased streaming estimators like Algorithm 151 (and its sketch-based turnstile cousins) are how these statistics get computed at all.

40.5.2 Frequency moments and the AMS sketch

Beyond reservoir sampling on edges, linear sketching gives a single framework for tracking frequencies, degrees, and higher-order subgraph counts in streams with both insertions and deletions. The basic construction is the Alon–Matias–Szegedy (AMS) sketch (1996).

Intuition

Suppose each edge e in the universe has an accumulated frequency f_e (in simple graphs $f_e \in \{0, 1\}$, while in dynamic multi-graphs $f_e \in \mathbb{Z}$). The second frequency moment $F_2 = \sum_e f_e^2$ is the squared ℓ_2 norm of the frequency vector.

To maintain F_2 in logarithmic space, choose a 4-wise independent hash function $h : E \to \{-1, +1\}$ drawn from a family with $\mathbb{E}[h(e)] = 0$ and $\mathbb{E}[h(e)^2] = 1$. Maintain a single scalar accumulator Z initialized to 0. Upon receiving an update (e, Δ) where $\Delta \in \{+1, -1\}$, update

$$Z \leftarrow Z + \Delta \cdot h(e).$$

At stream end, $Z = \sum_e f_e h(e)$. Its square $Y = Z^2$ is an unbiased estimator of F_2:

$$\mathbb{E}[Y] = \mathbb{E}\Big[\sum_e f_e^2 h(e)^2 + \sum_{e \neq e'} f_e f_{e'} h(e) h(e')\Big] = \sum_e f_e^2 \cdot 1 + 0 = F_2.$$

By 4-wise independence, pairwise cross-terms vanish and $\mathrm{Var}[Y] \leq 2F_2^2$. To drive the estimation error down to $\pm\varepsilon F_2$, we compute the average of $w = \lceil 2/\varepsilon^2 \rceil$ independent accumulators.

To drive the failure probability below δ, we take the median of $d = \lceil 36 \ln(2/\delta) \rceil$ such row averages. Because each cell is a linear combination of stream tokens, edge deletions are handled natively with no memory overhead.

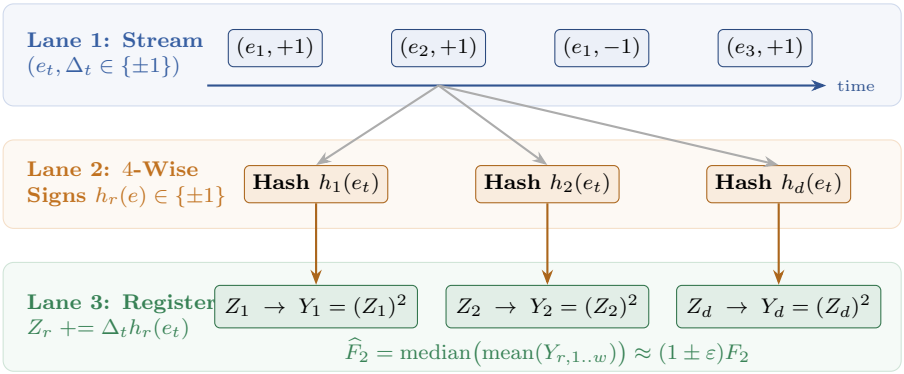


Figure 40.6: The Alon–Matias–Szegedy (AMS) sketch architecture for graph frequency moments. Lane 1: stream tokens arrive with edge identifier and sign ($\Delta_t = \pm 1$). Lane 2: d independent 4-wise hash functions assign uniform signs $h_r(e) \in \{-1, +1\}$ without storing lookup tables. Lane 3: scalar accumulators Z_r track the random projection. The squared values $Y_r = (Z_r)^2$ are averaged within rows of width $w = O(1/\varepsilon^2)$ to suppress variance, and the median across $d = O(\log(1/\delta))$ rows yields a $(1 \pm \varepsilon)$-approximation with probability at least $1 - \delta$.

40.6 What small space cannot do

The positive results above are nearly matched by impossibility theorems. The standard tool is *communication complexity*: split the stream between two players, Alice (the prefix) and Bob (the suffix); a streaming algorithm using s bits induces a protocol where Alice sends Bob her s-bit memory state, so any s-bit one-pass algorithm yields an s-bit one-way protocol. Lower-bounding the communication then lower-bounds s.

Theorem 40.1 (Connectivity needs $\Omega(n)$ space)**.** *Any (even randomized, constant-error) one-pass streaming algorithm that decides connectivity uses $\Omega(n)$ bits.*

Proof idea. Reduce from one-way Index. Alice holds a string $y \in \{0,1\}^N$ with $N = \Theta(n)$, Bob holds an index k, and Bob must output y_k; one-way communication forces $\Omega(N)$ bits. Encode y as the presence/absence of N candidate edges incident to a gadget so that the final graph is connected iff $y_k = 1$. Alice streams her edges and passes the memory state; Bob streams the query edges; the connectivity answer reveals y_k. Since the protocol transmits exactly the memory state, $s = \Omega(N) = \Omega(n)$. □

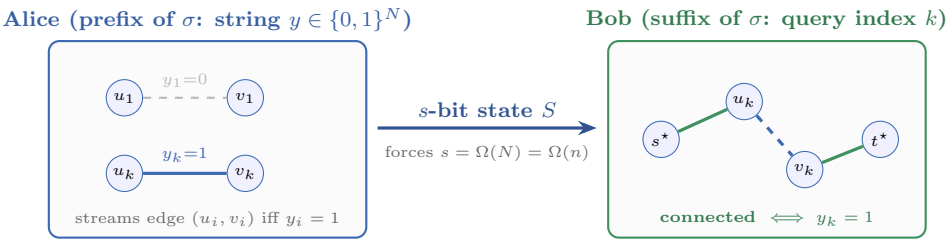


Figure 40.7: The communication reduction behind Theorem 40.1. Alice encodes her bitstring y as the prefix of the edge stream and forwards only the algorithm's s-bit memory state; Bob appends query edges for index k and reads y_k off the connectivity answer. A protocol this cheap forces $s = \Omega(N) = \Omega(n)$.

So semi-streaming connectivity ($O(n \log n)$) is optimal up to the logarithmic factor. Distances are much worse:

Theorem 40.2 (No exact distances, and stretch needs space)**.** *Any one-pass algorithm that, for a designated pair (s,t), distinguishes $\mathrm{dist}(s,t) \le d$ from $\mathrm{dist}(s,t) > d$ exactly requires $\Omega(n^2)$ bits in the worst case — essentially the whole graph. More generally, a one-pass $(2k-1)$-spanner-quality stretch bound forces $\Omega(n^{1+1/k})$ space, matching Algorithm 149 up to logs.*

Intuition

The spanner lower bound is the Moore bound (Lemma 40.1) read backwards. There exist graphs of girth $> 2k$ with $\Theta(n^{1+1/k})$ edges in which *every* edge is essential: delete any one and some pair's distance jumps. To certify stretch $< 2k+1$ you must remember $\Omega(n^{1+1/k})$ bits, because forgetting any edge of such a graph is detectable. Stretch and space trade off along exactly the same curve the greedy spanner achieves, so the greedy algorithm is optimal up to logarithmic factors.

Theorem 40.3 (Triangle detection needs $\Omega(n^2)$ space)**.** *Distinguishing $T = 0$ from $T \ge 1$ in one pass requires $\Omega(n^2)$ bits. Consequently any multiplicative $(1 \pm \varepsilon)$ approximation of T needs space growing as T shrinks; no algorithm in $o(n^2)$ space can certify triangle-freeness.*

Proof idea. Reduce from two-party Disjointness: Alice's set and Bob's set intersect iff a specific triangle exists in a bipartite-plus-query gadget. Disjointness needs $\Omega(N) = \Omega(n^2)$ communication even for randomized protocols, and again the protocol cost equals the memory state, so $s = \Omega(n^2)$. □

Common trap

A multiplicative approximation of T *cannot* tell $T = 0$ from $T = 1$ — $(1 \pm \varepsilon) \cdot 0 = 0$, yet you would need to detect the single triangle, which Theorem 40.3 forbids in small space. This is why Algorithm 151's sample complexity blows up as $T \to 0$: the problem itself is provably hard there. Always check which cases an approximation guarantee excludes.

40.6.1 Summary of the tradeoffs

The table summarizes the tradeoffs:

problem	model	space	passes	answer quality
connectivity	insertion	$O(n \log n)$	1	exact
connectivity	turnstile	$O(n \log^3 n)$	1	exact whp
bipartiteness	insertion	$O(n \log n)$	1	exact
$(2k-1)$-spanner	insertion	$O(n^{1+1/k} \log n)$	1	stretch $2k-1$
exact distance	either	$\Omega(n^2)$	1	exact (impossible small)
triangle count	insertion	$\tilde{O}(\frac{m\Delta}{\varepsilon^2 T})$	1	$(1\pm\varepsilon)$ whp
triangle $T>0$?	either	$\Omega(n^2)$	1	exact (impossible small)

Three resources trade off against each other; improving one costs another:

- **Space vs. accuracy.** Spanners trade more space ($n^{1+1/k}$) for less stretch ($2k-1$); triangle estimators trade more samples for smaller ε. These tradeoff curves are matched by lower bounds.
- **Space vs. passes.** Many problems collapse to $O(n)$ space if you allow $O(\log n)$ passes (e.g. exact shortest paths via repeated Bellman–Ford-style sweeps). One pass is the strict regime; each extra pass is a real-world cost (re-reading petabytes) but a real algorithmic lever.
- **Insertion vs. turnstile.** Deletions force linearity, which costs extra log factors and forces randomization (the sketches fail with small probability). Insertion-only lets you be greedy and deterministic. If your application never deletes, do not pay for sketching.

Why this matters

The results here share a pattern: *a graph's useful structure is near-linear-size and linearly compressible*, while its *exact* fine structure (precise distances, the presence of one specific triangle) needs quadratic space. Semi-streaming algorithms work on the compressible side of that divide, keeping one number per vertex, exploiting cut and boundary algebra, and accepting approximation exactly where the lower bounds say you must. The same compressibility underlies spectral sparsifiers and Laplacian solvers (Chapter 41), maintained in a single streaming pass.

40.7 Historical Context and Controversies

History

The mathematical foundation of streaming computation was established in 1996 by Noga Alon, Yossi Matias, and Mario Szegedy, who showed that statistical properties of massive numerical streams could be tracked using tiny randomized sketches — small linear projections that compress high-dimensional vectors into low-dimensional summaries. In the early 2000s, as web graphs and internet traffic grew rapidly, computer scientists realized that classical graph theory's core assumption — random access to the entire graph in RAM — had broken down. In 2005, Joan Feigenbaum, Sampath Kannan, Andrew McGregor, Siddharth Suri, and Jian Zhang introduced the *semi-streaming model*, proposing an $O(n \operatorname{poly}\log n)$ space budget: storing a few numbers per vertex is feasible on web graphs while storing every edge

is impossible.
The key result for dynamic streams came in 2012, when Kook Jin Ahn, Sudipto Guha, and Andrew McGregor (AGM) introduced *graph sketching*. AGM demonstrated that by representing a graph through its signed vertex-edge incidence matrix and computing linear L_0 sketches over its rows — sampling a random nonzero coordinate from an implicitly sketched vector — one could maintain spanning forests, connectivity, and cuts in a single pass under arbitrary edge insertions and deletions.
The transition to streaming exposed the limits of standard greedy approaches. In an insertion stream, keeping a spanning forest greedily by discarding cycle-forming edges is deterministic and uses only $O(n \log n)$ bits; but in a turnstile stream where edges can be deleted, greedy algorithms fail completely because discarded edges cannot be retrieved when a tree edge is removed. Overcoming this required linear sketches that store numerical aggregates that automatically undo themselves when an edge is deleted. Two-party communication complexity proved that computing exact shortest paths in a single pass requires $\Omega(n^2)$ bits, so exact distances need quadratic space and algorithm designers turned to metric spanners with stretch $2k-1$. What remains open includes closing the space gap for multi-pass streaming, optimizing per-token update time for dynamic sketches, and determining whether directed reachability in dynamic turnstile streams can ever beat trivial space bounds.

40.7.1 What We Can Now Do and Its Limitations

Connection

Capabilities Unlocked:

- We can process massive graphs with billions of edges in a single sequential pass using working memory proportional only to vertex count ($O(n \operatorname{polylog} n)$).
- Linear sketching (AGM) allows maintaining spanning forests and connectivity under arbitrary edge insertions and deletions (turnstile streams).
- Metric spanners provide provable $(2k-1)$ distance approximations with $O(n^{1+1/k})$ edges in an insertion stream.

Limitations:

- *Inability to solve fine-grained problems*: Exact distances, exact triangle counts, and exact subgraph detection provably require $\Omega(n^2)$ space in one pass; exact answers cannot be obtained in sublinear memory.
- *Randomization failure risk*: AGM sketches succeed with high probability (e.g., $1-1/n^c$). In rare adversarial configurations, hash collisions can cause false negative connectivity reports.
- *Per-token computational overhead*: While insertion-only streams process tokens in nanoseconds via union-find, turnstile linear sketching updates $O(\log^3 n)$ hash registers for every edge event, reducing throughput.

Where We Go Next:

- In Chapter 41 (Laplacian Solvers and Electrical Flows), we see how linear sketching and edge sampling ideas carry over to numerical graph algorithms. Spielman and Srivas-

tava use effective resistance sampling to construct $(1 \pm \varepsilon)$ *spectral sparsifiers* with only $O(n \log n/\varepsilon^2)$ edges, which leads to near-linear time Laplacian solvers.

- In Chapter 42 (Appendix: Math and Data Structures), we review the foundational data structures (DSU, segment trees) and linear algebra (Courant–Fischer min-max, matrix concentration) supporting these algorithms.

40.8 Exercises

Exercise 40.1 (Implement: components and a witness)**.** Implement Algorithm 148 with union by rank and path compression. Extend it so that, on a disconnected input, it outputs an actual vertex pair (u, v) in different components as a certificate of disconnection. Argue your extension stays within $O(n \log n)$ bits.

Exercise 40.2 (Prove: bipartiteness parity bits)**.** Prove that the parity-bit augmentation in Section 40.2 correctly reports non-bipartiteness exactly when an odd cycle exists. Show that maintaining $\phi(v)$ under union–find merges (flipping a whole subtree's parities when roots merge with conflicting orientation) is correct, and bound the total parity-update work.

Exercise 40.3 (Break: greedy forest under deletions)**.** Exhibit a turnstile stream on 4 vertices on which Algorithm 148, naively re-run with deletions removing edges from F, reports the wrong number of components. Explain precisely which step of the correctness proof in Section 40.2 fails, and why linearity in Section 40.4 repairs it.

Exercise 40.4 (Spanner stretch is tight)**.** For $t = 3$ ($k = 2$), construct a graph and a stream order on which Algorithm 149 produces an H where some original edge $\{u, v\}$ has $\mathrm{dist}_H(u, v) = 3$ exactly. Conclude the stretch bound $2k - 1$ is achieved, not merely upper-bounded.

Exercise 40.5 (Moore bound, the details)**.** Fill in the cleaning step of Lemma 40.1: prove that any graph with average degree d contains a subgraph with minimum degree $\geq d/2$ (repeatedly delete vertices of degree $< d/2$ and bound how many edges leave). Then complete the BFS-ball counting to get $|E| = O(n^{1+1/k})$.

Exercise 40.6 (1-sparse recovery)**.** A vector $x \in \mathbb{R}^N$ has at most one nonzero coordinate. You are told only $P = \sum_i x_i$ and $Q = \sum_i i\, x_i$. Show how to recover the index and value of the nonzero coordinate, and explain what goes wrong if the support has size two. Then describe how a random fingerprint slot detects (whp) a support of size ≥ 2 so the sampler can reject.

Exercise 40.7 (Why fresh sketches per round)**.** In Algorithm 150, suppose you reuse a single L_0 sketch across all R Borůvka rounds. Construct (or carefully argue) a scenario where reuse causes a correlated failure: the round-r query set $\{S \in \mathcal{C}\}$ depends on the sketch's own randomness from earlier rounds, breaking the uniformity guarantee. How many independent sketches are truly necessary, and why $\Theta(\log n)$?

Exercise 40.8 (Triangle estimator variance)**.** For Algorithm 151, compute $\mathrm{Var}[\widehat{T}]$ for a single copy in terms of m, n, T, and the maximum number Δ of triangles sharing an edge. Deduce the number of parallel copies needed for a $(1 \pm \varepsilon)$ estimate with probability $1 - \delta$, and verify it matches the $\widetilde{O}(m\Delta/(\varepsilon^2 T))$ figure quoted in Section 40.5.

Exercise 40.9 (Lower bound from INDEX)**.** Write out the reduction behind Theorem 40.1 in full: define the edge gadget encoding $y \in \{0,1\}^N$, specify Bob's query edges for index k, and prove the final graph is connected iff $y_k = 1$. State exactly where the one-way communication lower bound of $\Omega(N)$ enters.

Exercise 40.10 (Passes vs. space)**.** Show that with $O(\log n)$ passes and $O(n)$ space you can compute exact single-source BFS distances in an unweighted streamed graph (one BFS layer per pass). Contrast with the one-pass $\Omega(n^2)$ bound of Theorem 40.2 and explain, in your own words, what extra passes buy.

Exercise 40.11 (Extend: connectivity sketch to MST weight)**.** Sketch (informally) how to extend the AGM connectivity sketch of Algorithm 150 to estimate the weight of a minimum spanning tree of a turnstile-streamed weighted graph, using the "count components at each weight threshold" telescoping identity $w(\text{MST}) = \sum_i \big(\#\text{components using only edges of weight} \le i \big) - (\dots)$. Which pieces of this chapter does it reuse?

Chapter 41

Laplacian Solvers and Max-Flow via Electrical Flows

In Chapter 40, we saw that linear algebra and boundary operators compress massive data streams into tiny sketches. Earlier, in Part VI, we used the graph Laplacian $\mathbf{L}$ as an analytical tool to read off smoothness, connectivity, and random walk commute times. This chapter uses the Laplacian as an algorithmic tool. The single computational primitive

$$\text{given } b \text{ with } \mathbf{1}^\top b = 0, \quad \text{solve} \quad \mathbf{L}x = b$$

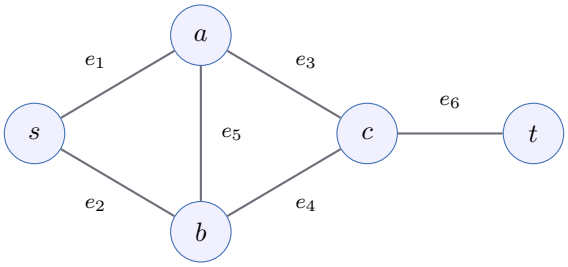
is, since the work of Spielman and Teng, solvable in time *nearly linear* in the number of edges — $O(m \log^c n \log(1/\varepsilon))$ for a fixed constant c, to relative accuracy ε. Once you can solve $\mathbf{L}x = b$ that fast, many graph problems reduce to it. Computing an electrical flow is one solve. Estimating effective resistances between many pairs is a few solves plus a random projection. In particular, *maximum flow* — the central combinatorial problem of Part III — can be approximated by solving a short sequence of electrical-flow problems, each a single Laplacian solve. This chapter develops that connection between electrical flows and discrete flows.

Intuition

Keep the spring-and-resistor picture from Chapters 31 and 35 in front of you. Pin a current of $+1$ into vertex s and pull -1 out of vertex t; the network settles into the unique current that obeys Ohm's and Kirchhoff's laws while minimizing dissipated energy. The vector of vertex potentials x that produces this current is exactly the solution of $\mathbf{L}x = b$ with $b = e_s - e_t$. So "solve a Laplacian" and "find the electrical flow" are the same computation seen from the two sides of Ohm's law: x is potentials, $\mathbf{L}x$ is the net current injected. Max-flow re-enters because an electrical flow already routes 1 unit from s to t; it just spreads it badly across edges, overloading some. If we make overloaded edges *stiffer* (raise resistance) and re-solve, current avoids them. Iterate, and the average of those electrical flows converges to a feasible near-maximum flow.

41.1 The running network

Fix one weighted graph for the whole chapter. Take five vertices $\{s, a, b, c, t\}$ and six edges, each with a *conductance* $w_e > 0$ (think: thickness of wire; resistance is $r_e = 1/w_e$). To keep arithmetic clean, start every conductance at 1.



Edges are numbered 1 through 6: $e_1 = \{s, a\}$, $e_2 = \{s, b\}$, $e_3 = \{a, c\}$, $e_4 = \{b, c\}$, $e_5 = \{a, b\}$, $e_6 = \{c, t\}$. Orient each edge from its lower-numbered endpoint to its higher one in the fixed order $s < a < b < c < t$; orientations are bookkeeping for signs, not directions of flow. With unit conductances the Laplacian $\mathbf{L} = \mathbf{D} - \mathbf{A}$ (degree matrix minus adjacency) is, ordering rows/columns s, a, b, c, t,

$$\mathbf{L} \;=\; \begin{pmatrix} 2 & -1 & -1 & 0 & 0 \\ -1 & 3 & -1 & -1 & 0 \\ -1 & -1 & 3 & -1 & 0 \\ 0 & -1 & -1 & 3 & -1 \\ 0 & 0 & 0 & -1 & 1 \end{pmatrix}.$$

Every row sums to zero, so $\mathbf{L}\mathbf{1} = 0$; the graph is connected, so the null space is exactly $\operatorname{span}\{\mathbf{1}\}$ and $\operatorname{rank}\mathbf{L} = 4$. We will inject $b = e_s - e_t = (1, 0, 0, 0, -1)^\top$ and solve $\mathbf{L}x = b$.

Focus

The incidence matrix makes the spring picture precise. Let $\mathbf{B} \in \mathbb{R}^{m \times n}$ have one row per oriented edge: row $e = (u \to v)$ is $e_u^\top - e_v^\top$. Let $\mathbf{W} = \operatorname{diag}(w_e)$. Then

$$\mathbf{L} \;=\; \mathbf{B}^\top \mathbf{W}\, \mathbf{B}, \qquad x^\top \mathbf{L} x \;=\; \sum_{e=\{u,v\}} w_e \,(x_u - x_v)^2.$$

This factorization is central to the chapter: it shows $\mathbf{L} \succeq 0$, exposes its null space ($\mathbf{B}x = 0 \iff$ x constant on each component), and — read right to left — says a Laplacian quadratic form *is* an electrical energy.

41.2 Electrical flows as energy minimization

Before solving anything, pin down what we are solving *for*. A flow is a vector $f \in \mathbb{R}^m$, one real number per oriented edge (negative means flow runs against the orientation). A flow *routes demand* b if it satisfies conservation $\mathbf{B}^\top f = b$: at every vertex, flow in minus flow out equals the injected demand. Among all flows routing b, the *electrical flow* is the one of least energy.

Definition 41.1 (Electrical flow)**.** The energy of a flow f is $\mathcal{E}(f) = \sum_e r_e f_e^2 = f^\top \mathbf{R} f$ where $\mathbf{R} = \mathbf{W}^{-1} = \mathrm{diag}(r_e)$. The *electrical flow* routing b is

$$f^\star \;=\; \arg\min_{f:\ \mathbf{B}^\top f = b} \sum_e r_e f_e^2.$$

This is a convex quadratic with linear constraints; it has a unique minimizer. The Lagrangian optimality conditions are Ohm's and Kirchhoff's laws.

Theorem 41.1 (Ohm + Kirchhoff = a Laplacian solve)**.** *Let x solve $\mathbf{L}x = b$ (any solution; all differ by a multiple of $\mathbf{1}$). Then*

$$f^\star \;=\; \mathbf{W}\mathbf{B}\,x, \qquad \textit{i.e.} \qquad f_e^\star \;=\; w_e\,(x_u - x_v) \quad \textit{for } e = (u \to v),$$

is the electrical flow routing b, and it is unique. Conversely the minimizing flow determines the potentials x up to an additive constant.

Why it works

The proof is one line of Lagrange multipliers, and the multipliers are the potentials. Minimize $f^\top \mathbf{R} f$ subject to $\mathbf{B}^\top f = b$. The Lagrangian is $f^\top \mathbf{R} f - 2x^\top(\mathbf{B}^\top f - b)$ with multiplier vector $x \in \mathbb{R}^n$. Stationarity in f: $2\mathbf{R}f - 2\mathbf{B}x = 0$, so $f = \mathbf{R}^{-1}\mathbf{B}x = \mathbf{W}\mathbf{B}x$. That is *Ohm's law*: flow on an edge equals conductance times potential drop. Feasibility $\mathbf{B}^\top f = b$ becomes $\mathbf{B}^\top \mathbf{W}\mathbf{B}\,x = \mathbf{L}x = b$, which is *Kirchhoff's current law*: the net flow out of each vertex equals its injection. The two physical laws are precisely the two optimality conditions, and together they are the linear system $\mathbf{L}x = b$.

Proof. The feasible set $\{f : \mathbf{B}^\top f = b\}$ is a nonempty affine subspace (nonempty because $b \perp \mathbf{1}$ lies in the range of $\mathbf{B}^\top$), and $f \mapsto f^\top \mathbf{R} f$ is strictly convex on it after translating to the linear subspace $\ker \mathbf{B}^\top$, since $\mathbf{R} \succ 0$. Hence a unique minimizer exists and is characterized by first-order stationarity of the Lagrangian. As computed above, stationarity gives $f = \mathbf{W}\mathbf{B}x$ and feasibility gives $\mathbf{L}x = b$. Since $\mathbf{L}x = b$ has a solution (as $b \perp \ker \mathbf{L} = \mathrm{span}\,\mathbf{1}$) unique up to $+\tau\mathbf{1}$, and $\mathbf{B}\mathbf{1} = 0$, the flow $f = \mathbf{W}\mathbf{B}x$ is independent of the choice of solution. Uniqueness of $f^\star$ follows from strict convexity. □

Trace it yourself

Solve $\mathbf{L}x = e_s - e_t$ on the running network by hand. Fix the gauge $x_t = 0$ and drop the last row/column (grounding t), leaving the 4×4 system in (x_s, x_a, x_b, x_c):

$$\begin{pmatrix} 2 & -1 & -1 & 0 \\ -1 & 3 & -1 & -1 \\ -1 & -1 & 3 & -1 \\ 0 & -1 & -1 & 3 \end{pmatrix} \begin{pmatrix} x_s \\ x_a \\ x_b \\ x_c \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \\ 0 \\ 0 \end{pmatrix}.$$

By the $a \leftrightarrow b$ symmetry of the network, $x_a = x_b$. Set $x_a = x_b = p$. Row 2: $-x_s + 3p - p - x_c = 0 \Rightarrow x_s + x_c = 2p$. Row 4: $-2p + 3x_c = 0 \Rightarrow x_c = \frac{2}{3}p$. Then $x_s = 2p - x_c = \frac{4}{3}p$. Row 1: $2x_s - 2p = 1$, i.e. $\frac{8}{3}p - 2p = \frac{2}{3}p = 1$, so $p = \frac{3}{2}$, giving $x_a = x_b = \frac{3}{2}$, $x_s = 2$, $x_c = 1$, $x_t = 0$.

Now read off currents by Ohm's law $f_e = w_e\,(x_u - x_v) = x_u - x_v$:

$$f_{sa} = 2 - \tfrac{3}{2} = \tfrac{1}{2}, \quad f_{sb} = \tfrac{1}{2}, \quad f_{ab} = 0, \quad f_{ac} = \tfrac{3}{2} - 1 = \tfrac{1}{2}, \quad f_{bc} = \tfrac{1}{2}, \quad f_{ct} = 1 - 0 = 1.$$

Check Kirchhoff at s: out $= \frac{1}{2} + \frac{1}{2} = 1 = b_s$. At c: in $\frac{1}{2} + \frac{1}{2} = 1$, out 1. The symmetric edge $\{a, b\}$ carries *nothing* — there is no potential difference to push it. Energy $\mathcal{E} = \sum r_e f_e^2 = 4 \cdot (\frac{1}{2})^2 + 0 + 1^2 = 2$. And indeed $\mathcal{E} = b^\top x = x_s - x_t = 2$. That identity — energy = effective resistance times current^2 — is the next definition.

Figure 41.1 plots these solved vertex potentials x_v and the resulting Ohm's-law currents along a horizontal voltage axis from $x_s = 2$ down to $x_t = 0$.

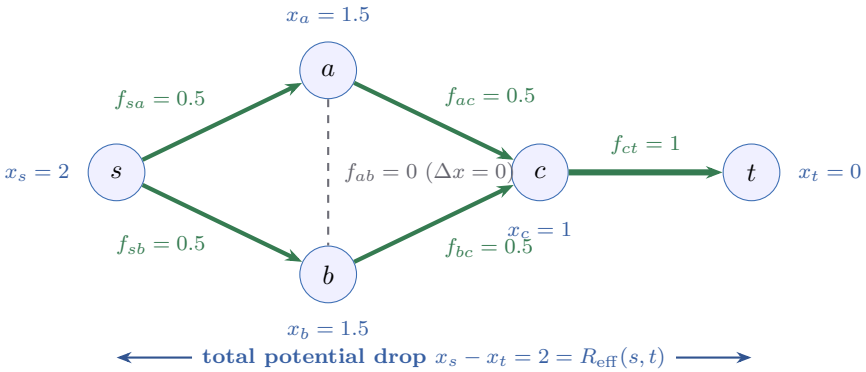


Figure 41.1: Solved vertex potentials x and electrical currents $f_e = w_e(x_u - x_v)$ for unit demand $b = e_s - e_t$ on the running network. Symmetry pins $x_a = x_b = 1.5$, so edge $\{a, b\}$ sees zero potential drop and carries zero current. The total voltage drop $x_s - x_t = 2$ equals both the dissipated energy $\mathcal{E}(f^\star)$ and the effective resistance $R_{\text{eff}}(s, t)$.

41.2.1 Effective resistance

Definition 41.2 (Effective resistance)**.** For $s \neq t$, the *effective resistance* is $R_{\text{eff}}(s,t) = (e_s - e_t)^\top \mathbf{L}^+ (e_s - e_t)$, where $\mathbf{L}^+$ is the Moore–Penrose pseudoinverse. It equals the energy of the unit electrical flow from s to t, and equals the potential drop $x_s - x_t$ when one unit is injected.

In the worked calculation above, $R_{\text{eff}}(s,t) = x_s - x_t = 2$. Effective resistance is a metric on vertices (the triangle inequality is a short consequence of energy minimization), it governs hitting and commute times of the random walk (proved in Chapter 35), and — the fact we need for sparsification — the effective resistance of an edge times its weight, $w_e R_{\text{eff}}(u,v)$, is the *leverage score* of that edge: its sampling importance.

Pause

Pause and think: Why does the effective resistance between two endpoints u and v never exceed the resistance r_e of a direct edge between them? Because adding parallel pathways through the rest of the graph provides alternative routes for current, which can only decrease the total energy of the electrical flow.

Connection

The same quantity appears in three areas. *Linear algebra:* $R_{\text{eff}}(s,t)$ is a quadratic form in $\mathbf{L}^{+}$, hence a Schur-complement / Gram quantity. *Random walks:* commute time between s and t equals $2m\,R_{\text{eff}}(s,t)$ in the unweighted case, so resistance *is* round-trip walk time up to scale. *Spanning trees:* by the matrix-tree theorem, $w_e R_{\text{eff}}(e)$ is exactly the probability that edge e appears in a uniformly random (weight-proportional) spanning tree. The same number governs solver preconditioning, walk mixing, and tree sampling — which is why one fast solver helps with all three.

41.3 Solving $\mathbf{L}x = b$ in nearly linear time

Gaussian elimination costs $O(n^3)$ and destroys sparsity (fill-in). For graphs we want $O(m \operatorname{poly}\log n)$. Two facts make it possible: $\mathbf{L}$ is symmetric positive semidefinite, so an *iterative* method converges at a rate set by the condition number; and we can build a cheap *preconditioner* whose condition number relative to $\mathbf{L}$ is small. The preconditioner is built from a spanning tree, repaired by sparsification. We assemble the pieces.

41.3.1 Preconditioned conjugate gradient

Restrict to the subspace $\mathbf{1}^{\perp}$ where $\mathbf{L}$ is positive definite; write its condition number as $\kappa(\mathbf{L}) = \lambda_{\max}/\lambda_{\min}$ over the nonzero eigenvalues. Plain conjugate gradient solves $\mathbf{L}x = b$ to relative error ε in $O(\sqrt{\kappa(\mathbf{L})}\,\log(1/\varepsilon))$ matrix–vector products, each costing $O(m)$. For graphs κ can be as large as $\operatorname{poly}(n)$ (a path graph has $\kappa = \Theta(n^2)$), so this alone is not near-linear.

Preconditioning replaces $\mathbf{L}x = b$ by the equivalent system $\mathbf{P}^{-1}\mathbf{L}x = \mathbf{P}^{-1}b$ for an SPD matrix $\mathbf{P}$ that (i) is cheap to apply $\mathbf{P}^{-1}$ to a vector and (ii) *approximates* $\mathbf{L}$ spectrally. Spectral approximation is the right notion of "close."

Definition 41.3 (Spectral approximation)**.** Matrices $\mathbf{P}, \mathbf{L} \succeq 0$ satisfy $\mathbf{P} \approx_{\kappa} \mathbf{L}$ when

$$\tfrac{1}{\kappa}\, x^{\top}\mathbf{P}\,x \;\le\; x^{\top}\mathbf{L}\,x \;\le\; \kappa\, x^{\top}\mathbf{P}\,x \qquad \text{for all } x.$$

If $\mathbf{P} \approx_{\kappa} \mathbf{L}$ then the preconditioned operator $\mathbf{P}^{-1}\mathbf{L}$ has all eigenvalues in $[1/\kappa, \kappa]$, so preconditioned CG (PCG) converges in $O(\sqrt{\kappa}\,\log(1/\varepsilon))$ iterations, each costing one multiply by $\mathbf{L}$ ($O(m)$) plus one *solve* against $\mathbf{P}$. The algorithmic task is to find a preconditioner $\mathbf{P}$ with small condition number κ against which linear systems $\mathbf{P}z = r$ can be solved rapidly.

Algorithm 152: PRECONDITIONEDSOLVE($\mathbf{L}, b, \mathbf{P}, \varepsilon$)

Input: Laplacian $\mathbf{L}$, demand $b \perp \mathbf{1}$, SPD preconditioner $\mathbf{P}$ with $\mathbf{P} \approx_\kappa \mathbf{L}$, accuracy ε.
Output: x with $\|x - \mathbf{L}^+ b\|_{\mathbf{L}} \le \varepsilon \|\mathbf{L}^+ b\|_{\mathbf{L}}$.

$x \leftarrow 0$; $r \leftarrow b$; $z \leftarrow \textsc{Solve}_{\mathbf{P}}(r)$; $p \leftarrow z$
for $k = 0, 1, 2, \ldots$ *until* $\|r\| \le \varepsilon \|b\|$ **do**
  $q \leftarrow \mathbf{L}p$
  $\alpha \leftarrow (r^\top z)/(p^\top q)$
  $x \leftarrow x + \alpha p$; $r_{\text{new}} \leftarrow r - \alpha q$
  $z_{\text{new}} \leftarrow \textsc{Solve}_{\mathbf{P}}(r_{\text{new}})$
  $\beta \leftarrow (r_{\text{new}}^\top z_{\text{new}})/(r^\top z)$
  $p \leftarrow z_{\text{new}} + \beta p$; $r \leftarrow r_{\text{new}}$; $z \leftarrow z_{\text{new}}$
return x

41.3.2 Low-stretch spanning trees: the first preconditioner

The simplest preconditioner is a spanning tree T of G, with its tree Laplacian $\mathbf{L}_T$. Solving $\mathbf{L}_T z = r$ is trivial: a tree Laplacian (grounded at a root) is, after a permutation, triangular — one upward pass accumulates currents into the root, one downward pass sets potentials, total $O(n)$. So $\textsc{Solve}_{\mathbf{L}_T}$ is linear. The question is how well $\mathbf{L}_T$ approximates $\mathbf{L}$.

Since $T \subseteq G$, dropping edges only lowers the quadratic form: $x^\top \mathbf{L}_T x \le x^\top \mathbf{L} x$, giving the easy side $\mathbf{L}_T \preceq \mathbf{L}$. The hard side asks how much energy the missing edges can have that the tree must absorb. The right accounting is *stretch*.

Definition 41.4 (Stretch)**.** For a non-tree edge $e = \{u, v\}$ with resistance r_e, let $P_T(u, v)$ be the unique tree path. Its stretch is $\mathrm{st}_T(e) = \big(\sum_{f \in P_T(u,v)} r_f\big)/r_e$, the tree-path resistance over the edge's own resistance. The total stretch is $\mathrm{st}_T(G) = \sum_{e \in E} \mathrm{st}_T(e)$ (tree edges contribute 1 each).

Lemma 41.1 (Tree preconditioning bound)**.** $\mathbf{L}_T \preceq \mathbf{L} \preceq \big(1 + \mathrm{st}_T(G)\big)\,\mathbf{L}_T$, *so* $\mathbf{L}_T \approx_\kappa \mathbf{L}$ *with* $\kappa \le 1 + \mathrm{st}_T(G)$.

Why it works

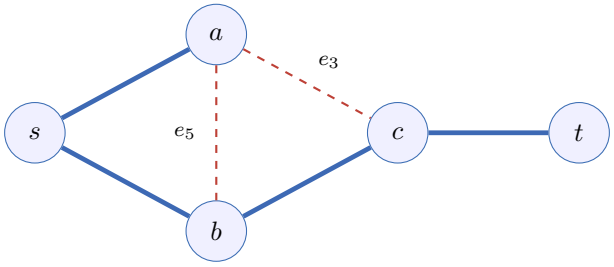
Why stretch is the multiplier. Route each non-tree edge $e = \{u, v\}$, carrying a unit of "demand" $e_u - e_v$, along its tree path. This embeds G into T as a flow. The congestion this puts on a tree edge, summed and weighted by resistances, is captured by stretch: a non-tree edge of small resistance whose tree detour is long forces a lot of current through the tree, so the tree must be "soft" enough to carry it, which is precisely a large ratio in the spectral inequality. Summing the contribution of all non-tree edges, the worst blow-up of $x^\top \mathbf{L} x$ over $x^\top \mathbf{L}_T x$ is bounded by $1 + \mathrm{st}_T(G)$. The bound is loose (it is a sum, not a max), but it is exactly what makes the next idea — sampling — pay off.

Trace it yourself

Compute stretch on the running network. Take the spanning tree $T = \{e_1, e_2, e_4, e_6\} = \{\{s, a\}, \{s, b\}, \{b, c\}, \{c, t\}\}$ (four edges, five vertices — a tree). All resistances are 1. The two non-tree edges are $e_3 = \{a, c\}$ and $e_5 = \{a, b\}$.

- $e_5 = \{a, b\}$: tree path $a \to s \to b$ uses $\{s, a\}, \{s, b\}$, resistance 2, so $\mathrm{st}_T(e_5) = 2/1 = 2$.
- $e_3 = \{a, c\}$: tree path $a \to s \to b \to c$ uses $\{s, a\}, \{s, b\}, \{b, c\}$, resistance 3, so $\mathrm{st}_T(e_3) = 3$.

The four tree edges contribute 1 each, so $\mathrm{st}_T(G) = 4 + 2 + 3 = 9$ and Lemma 41.1 gives $\kappa \le 1+9 = 10$. Now try the better-balanced tree $T' = \{e_1, e_3, e_4, e_6\} = \{\{s, a\}, \{a, c\}, \{b, c\}, \{c, t\}\}$: non-tree edges are $e_2 = \{s, b\}$ (path $s \to a \to c \to b$, resistance 3, stretch 3) and $e_5 = \{a, b\}$ (path $a \to c \to b$, resistance 2, stretch 2), so $\mathrm{st}_{T'}(G) = 4 + 3 + 2 = 9$ again — this tiny graph is too symmetric to separate the trees, but on a path or grid the choice of T swings stretch by orders of magnitude, which is why Theorem 41.2 matters.



The spanning tree $T = \{e_1, e_2, e_4, e_6\}$ (solid blue) of the running network; the two non-tree edges $e_3 = \{a, c\}$ and $e_5 = \{a, b\}$ (dashed red) are the ones that get stretched along their tree paths. Edge e_3 detours $a \to s \to b \to c$ (resistance 3) and e_5 detours $a \to s \to b$ (resistance 2), for total stretch 9.

A breadth-first or shortest-path tree can have stretch $\Theta(n^2)$. The key fact is that *low-stretch* trees exist and are constructible.

Theorem 41.2 (Low-stretch spanning trees)**.** *Every weighted graph has a spanning tree T with $\mathrm{st}_T(G) = O(m \log n \log\log n)$, constructible in $O(m \log n \log\log n)$ time. (Alon–Karp–Peleg–West first achieved $m^{O(\sqrt{\log\log n})}$; Abraham–Bartal–Neiman and Abraham–Neiman drove it to the near-linear bound.)*

These trees come from recursively partitioning the graph into low-diameter clusters (a "star decomposition") and recursing, so that on average no edge is stretched by more than a poly log factor. With such a tree, $\kappa \le 1 + O(m \log n \log\log n)$ and PCG needs $O(\sqrt{m}\ \mathrm{poly}\log)$ iterations — better than nothing, still not near-linear because of the $\sqrt{m}$. The fix is to add a few well-chosen non-tree edges back, sampled by stretch, and to do so *recursively*.

41.3.3 From a tree to an ultrasparsifier

The plan that closes the gap (Spielman–Teng, streamlined by Koutis–Miller–Peng) is: take the low-stretch tree T, then *sample* non-tree edges with probability proportional to their stretch, scaling kept edges up to stay unbiased. Adding t sampled edges produces an *ultrasparsifier* H: a graph with only $n - 1 + t$ edges that nonetheless satisfies $\mathbf{L}_H \approx_\kappa \mathbf{L}$ with a condition number you control by choosing t.

Theorem 41.3 (Ultrasparsifier)**.** *For any $\kappa > 1$ there is a graph H with $n - 1 + O\big(\frac{\mathrm{st}_T(G)}{\kappa} \log n\big)$ edges and $\mathbf{L}_H \approx_\kappa \mathbf{L}$, computable in near-linear time.*

Why this matters: H is a tree plus a near-tree, so its own Laplacian system can be solved by *eliminating* all degree-1 and degree-2 vertices (a partial Cholesky that does not fill in), shrinking H to a graph on $O\big(\frac{\mathrm{st}_T}{\kappa}\log n\big)$ vertices, and recursing the entire solver on that smaller graph. Each recursion level multiplies the vertex count by a factor < 1, so the recursion has $O(\log n)$ depth and the total work telescopes to near-linear.

Algorithm 153: LAPLACIANSOLVE$(\mathbf{L}, b, \varepsilon)$ — recursive preconditioned solver

Input: Laplacian $\mathbf{L}$ of $G = (V, E)$, $b \perp \mathbf{1}$, accuracy ε.
Output: $x \approx \mathbf{L}^{+} b$.

if $|V| \le c_0$ **then**
  return direct dense solve of $\mathbf{L}x = b$
$T \leftarrow$ LOWSTRETCHTREE(G) `// Theorem 41.2`
$H \leftarrow$ ULTRASPARSIFY(G, T, κ) `// tree + stretch-sampled edges, Thm. 41.3`
$G' \leftarrow$ GREEDYELIMINATE(H) `// remove deg-1/deg-2 vertices, no fill-in`
build $\mathbf{P} = \mathbf{L}_H$; let SOLVE$_{\mathbf{P}}(r)$ recurse via LAPLACIANSOLVE$(\mathbf{L}_{G'}, \cdot)$ on the reduced system
return PRECONDITIONEDSOLVE$(\mathbf{L}, b, \mathbf{P}, \varepsilon)$ `// Algorithm 152`

Figure 41.2 traces one level of this recursive reduction: adding t stretch-sampled off-tree edges to T creates the ultrasparsifier H, and greedy Gaussian elimination of all degree-1 leaves and degree-2 series paths collapses H onto a much smaller core G' without introducing fill-in.

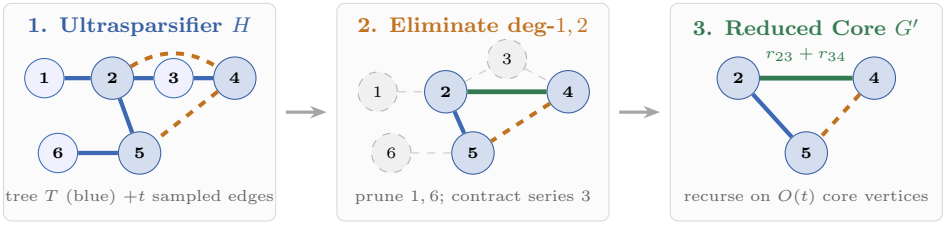


Figure 41.2: One level of the recursive preconditioned solver (Algorithm 153). (1) The ultrasparsifier H augments a low-stretch tree T with t stretch-sampled off-tree edges. (2) Degree-1 leaves $(1, 6)$ are eliminated directly and degree-2 paths (2–3–4) collapse into single series edges with zero fill-in. (3) The solver recurses on the compact core G'.

Cost summary

With κ chosen at each level so that PCG needs $O(\sqrt{\kappa})$ inner iterations and the reduced graph shrinks by a constant factor, the recursion solves to accuracy ε in

$$O\big(m \log^{c} n \log(1/\varepsilon)\big)$$

time for a small constant c. The original Spielman–Teng bound had a large c; Koutis–Miller–Peng brought it to $c \approx 1$–2 and made the constants practical; later work (Cohen–Kyng–Miller–Pachocki–Peng–Rao–Xu and others) sharpened it further and gave the simpler "sparsified Cholesky" route. For this book treat the solver as a black box returning $\tilde{x}$ with $\|\tilde{x} - \mathbf{L}^{+} b\|_{\mathbf{L}} \le \varepsilon \|\mathbf{L}^{+} b\|_{\mathbf{L}}$ in time $O(m \,\mathrm{poly}\log n \log(1/\varepsilon))$.

Common trap

The accuracy is *relative in the* $\mathbf{L}$*-norm*, not entrywise, and the answer is defined only modulo $\mathbf{1}$. Two traps follow. First, always project b onto $\mathbf{1}^\perp$ before solving; a stray component along $\mathbf{1}$ has no solution and CG will not converge. Second, when you chain solves (as max-flow does), ε-errors can compound: a downstream routine that divides by a small potential difference will amplify the residual. The cure throughout is to demand ε *polynomially small* ($\varepsilon = 1/\operatorname{poly}(n)$) — the $\log(1/\varepsilon)$ cost makes this almost free — and to re-project onto the feasible flow space after each solve.

41.4 Spectral sparsification

Ultrasparsifiers keep a tree intact. A *spectral sparsifier* is the unconstrained version: replace G (with m edges) by a reweighted subgraph $\widetilde{G}$ with only $O(n \log n/\varepsilon^2)$ — or even $O(n/\varepsilon^2)$ — edges such that $\mathbf{L}_{\widetilde{G}} \approx_{1+\varepsilon} \mathbf{L}$. Spectral sparsifiers are the reason "Laplacian solver" and "cut/flow algorithm" became near-linear at all: sparsify once, then run anything on $O(n)$ edges.

Theorem 41.4 (Spielman–Srivastava: sample by effective resistance)**.** *Sample $q = O(n \log n/\varepsilon^2)$ edges i.i.d., each edge e chosen with probability $p_e \propto w_e R_{\text{eff}}(e)$ (its leverage score), and give a sampled edge weight $w_e/(q\,p_e)$. With high probability the resulting $\widetilde{G}$ satisfies $\mathbf{L}_{\widetilde{G}} \approx_{1+\varepsilon} \mathbf{L}$.*

Why it works

Write $\mathbf{L} = \mathbf{B}^\top \mathbf{W} \mathbf{B} = \sum_e w_e\, b_e b_e^\top$ as a sum of rank-one terms, $b_e = \mathbf{B}^\top e_e$. Conjugate by $\mathbf{L}^{+/2}$: the matrix $\Pi = \mathbf{L}^{+/2}\mathbf{L}\mathbf{L}^{+/2}$ is a projection (it equals $\sum_e v_e v_e^\top$ with $v_e = \sqrt{w_e}\,\mathbf{L}^{+/2} b_e$), and the squared norm evaluates directly to $\|v_e\|^2 = w_e\, b_e^\top \mathbf{L}^+ b_e = w_e R_{\text{eff}}(e)$ — precisely the leverage score of edge e. So sparsifying $\mathbf{L}$ is exactly the matrix problem "approximate a sum of rank-one projections $\sum v_e v_e^\top$ by sampling few terms," and the textbook answer is to sample each term proportional to $\|v_e\|^2$. A matrix Chernoff bound then gives concentration with $O(n \log n/\varepsilon^2)$ samples. The leverage scores $w_e R_{\text{eff}}(e)$ sum to exactly $n-1$ (the trace of a rank-$(n-1)$ projection), which is why a near-linear number of samples suffices.

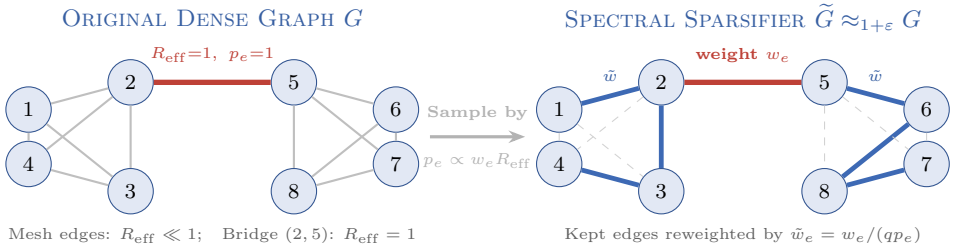


Figure 41.3: Spielman–Srivastava spectral sparsification by effective resistance sampling. Left: original graph G with dense clusters connected by a critical bridge edge. The bridge edge has effective resistance $R_{\text{eff}}(e) = 1$ (a cut edge), so its leverage score is 1 and it is retained with probability $p_e = 1$. Redundant mesh edges have small effective resistances $R_{\text{eff}}(e) \ll 1$ and are sampled lightly. Right: the resulting sparsifier $\widetilde{G}$ contains only $O(n \log n/\varepsilon^2)$ edges, each retained edge reweighted by $\tilde{w}_e = w_e/(qp_e)$ to guarantee unbiased quadratic forms: $(1-\varepsilon)x^\top \mathbf{L}_G x \le x^\top \mathbf{L}_{\widetilde{G}} x \le (1+\varepsilon)x^\top \mathbf{L}_G x$ for all x.

Computing all m effective resistances exactly would need m solves — too slow. The fix is a Johnson–Lindenstrauss projection: $R_{\text{eff}}(e) = \|\mathbf{W}^{1/2}\mathbf{B}\,\mathbf{L}^{+}b_e\|^2$ is the squared length of a vector; project the $m \times n$ matrix $\mathbf{W}^{1/2}\mathbf{B}\,\mathbf{L}^{+}$ down to $O(\log n/\varepsilon^2)$ random rows, preserving all pairwise lengths. Each projected row is computed with one Laplacian solve. So $O(\log n)$ solves estimate *all* leverage scores, and sparsification is itself near-linear — closing the loop, since the solver used inside needs only a crude preconditioner to bootstrap.

Connection

Solvers and sparsifiers improve each other: a crude tree preconditioner gives a slow solver, which is fast enough to sample effective resistances, which yields a spectral sparsifier, which yields a faster solver. Batson, Spielman, and Srivastava (2009) proved that optimal $O(n/\varepsilon^2)$-edge sparsifiers exist via a deterministic barrier method, while Spielman and Srivastava's effective-resistance sampling (2008) made sparsification practical. Koutis, Miller, and Peng later simplified the solver construction, while Cohen et al. extended it to generalized regression. This line of work shows how continuous spectral methods can answer discrete questions about graphs.

41.5 Maximum flow via a sequence of electrical flows

We now approximate maximum s–t flow in an *undirected, capacitated* graph using only electrical-flow computations — each a single Laplacian solve. Capacities $u_e > 0$ are given; we want the largest F such that F units can be routed from s to t with $|f_e| \le u_e$ on every edge. We describe the Christiano–Kelner–Madry–Spielman–Teng (CKMST, 2011) algorithm, which computes a $(1-\varepsilon)$-approximate max-flow in $O(m^{4/3}\,\mathrm{poly}(\varepsilon^{-1}, \log n))$ time — at the time, the first sub-$m^{1.5}$ undirected max-flow, and the template for the later almost-linear-time results.

41.5.1 Why electrical flow is the right relaxation

A feasible flow must obey $|f_e| \le u_e$ — an ℓ_∞ constraint, the natural home of combinatorial flow. Electrical flow instead minimizes $\sum r_e f_e^2$ — an ℓ_2 objective, the natural home of linear algebra. The strategy is the classic one for turning ℓ_∞ feasibility into a sequence of ℓ_2 solves: *multiplicative weights*. Run an electrical flow; wherever it exceeds capacity, raise that edge's resistance so the next electrical flow shies away; repeat; average the flows. The average is nearly feasible and nearly maximum.

Intuition

Picture the running network with s and t. The electrical flow of Section 41.2 already sent 1 unit from s to t, splitting it $\frac{1}{2}/\frac{1}{2}$ across the two paths $s \to a \to c \to t$ and $s \to b \to c \to t$. Suppose every edge had capacity 1 except $\{c, t\}$, which carries the full unit and so is at capacity. If we wanted to push 1.2 units, the bottleneck $\{c, t\}$ would overflow. Multiplicative weights says: make $\{c, t\}$ more resistive, but there is only one way to t, so the scheme cannot route around it — it instead discovers that $\{c, t\}$ is a true min-cut edge. Electrical flow, by spreading current to minimize energy, automatically finds *where* the bottlenecks are: congested edges are exactly the high-energy ones, and reweighting turns that signal into an algorithm.

Pause

Pause and think: Why can't a single electrical flow solve max-flow directly? An electrical flow minimizes total ℓ_2 energy $\sum r_e f_e^2$, treating an edge that carries twice its capacity as paying four times the energy rather than violating a hard cutoff. It lacks a hard ceiling to enforce $|f_e| \le u_e$. That is why we must iterate, penalizing overloaded edges until the flow respects the capacity constraints.

41.5.2 The multiplicative-weights outer loop

Fix a target flow value F (binary-search it, or scale so the optimum is near 1). Maintain a weight $w_e \ge 1$ per edge; the resistance fed to the electrical solver is $r_e = w_e/u_e^2$, so that an edge carrying its full capacity u_e contributes a fixed unit of "congestion energy" $r_e u_e^2 = w_e$. Each round, compute the electrical flow routing F units $s \to t$ under current resistances, measure each edge's *congestion* $\rho_e = |f_e|/u_e$, and bump weights multiplicatively by congestion. Average the rounds.

Algorithm 154: ElectricalMaxFlow$(G, u, s, t, F, \varepsilon)$

Input: Undirected graph G, capacities u, source/sink s, t, target value F, accuracy ε.
Output: A flow of value F with congestion $\le 1 + O(\varepsilon)$, or a certificate that F exceeds the max-flow.

$w_e \leftarrow 1$ for all e; $\bar{f} \leftarrow 0$; $N \leftarrow \Theta(\varepsilon^{-2}\sqrt{m}\log m)$ `// rounds;` $\Theta(\mu\varepsilon^{-2}\log m)$ `with width` $\mu \le \sqrt{m}$
for $i = 1$ **to** N **do**
  $r_e \leftarrow w_e/u_e^2$ for all e; $W \leftarrow \sum_e w_e$
  $x \leftarrow$ LaplacianSolve($\mathbf{L}_r$, $F(e_s - e_t)$, poly small ε) `// one solve,` $\mathbf{L}_r = \mathbf{B}^\top \mathbf{R}^{-1}\mathbf{B}$
  $f_e^{(i)} \leftarrow (x_u - x_v)/r_e$ `// electrical flow routing` F
  if $\mathcal{E}(f^{(i)}) = \sum_e r_e (f_e^{(i)})^2 > (1+\varepsilon)\, W$ **then**
    return "F is infeasible" `// energy too high ⇒ no flow of value` F `fits`
  $\rho_e \leftarrow |f_e^{(i)}|/u_e$ for all e
  $w_e \leftarrow w_e\left(1 + \frac{\varepsilon}{\mu}\rho_e\right)$ for all e `//` $\mu = \max_e \rho_e$ `controls the step`
  $\bar{f} \leftarrow \bar{f} + \frac{1}{N} f^{(i)}$
return $\bar{f}$ `// average flow: value` F`, congestion` $\le 1 + O(\varepsilon)$

Why it works

Two invariants drive the analysis. *(1) Electrical energy lower-bounds congestion.* The electrical (minimum-energy) flow $f^{(i)}$ has energy $\mathcal{E}(f^{(i)}) \le \mathcal{E}(g)$ for every flow g routing F units. If a feasible flow of value F exists with all $|g_e| \le u_e$, then $\mathcal{E}(g) = \sum_e (w_e/u_e^2) g_e^2 \le \sum_e w_e = W$. So whenever the electrical energy exceeds $(1+\varepsilon)W$, *no* capacity-respecting flow of value F exists — that is the infeasibility certificate. *(2) Weights track congestion.* The potential $W = \sum_e w_e$ grows by a factor $(1 + \frac{\varepsilon}{\mu}\rho_e)$ per edge per round, so $\log W$ accumulates total congestion. The standard MWU regret bound forces the time-average congestion on every edge toward the best achievable, i.e. toward 1. Averaging the per-round electrical flows yields $\bar{f}$ whose congestion is $1 + O(\varepsilon)$, and rescaling by $1/(1 + O(\varepsilon))$ gives a feasible flow of value $(1 - O(\varepsilon))F$.

41.5.3 The width problem and the $m^{1/3}$ fix

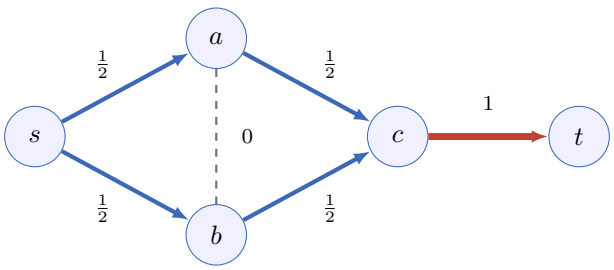
The catch is *width* $\mu = \max_e \rho_e$: with the width-scaled step size ε/μ used in Algorithm 154, MWU needs $\Theta(\mu\,\varepsilon^{-2}\log m)$ rounds, and a single electrical flow can overload one edge by a factor as large as $\sqrt{m}$ (all the current funnels through a low-resistance bottleneck). That would force $\Theta(\sqrt{m}\,\varepsilon^{-2}\log m)$ rounds — too many. CKMST's key idea controls width by *removing* the worst offenders: any edge whose electrical congestion exceeds a threshold $\rho^\star$ has, by an energy argument, large effective resistance, so it must lie near a sparse cut. Delete such edges (only $O(m/\rho^{\star 2})$ of them across the run, charged to energy) and re-solve. Setting $\rho^\star = m^{1/3}$ balances "few rounds" against "few deletions," giving $O(m^{1/3}\ \mathrm{poly}(\varepsilon^{-1}, \log m))$ rounds and total time $O(m^{4/3}\ \mathrm{poly}(\varepsilon^{-1}, \log m))$.

Theorem 41.5 (CKMST)**.** *Algorithm 154, augmented with the width-reduction step (delete edges of congestion $> m^{1/3}$ and re-solve), computes a $(1-\varepsilon)$-approximate maximum s–t flow in an undirected capacitated graph in time $O(m^{4/3}\ \mathrm{poly}(\varepsilon^{-1}, \log m))$, using $O(m^{1/3}\ \mathrm{poly}(\varepsilon^{-1}, \log m))$ Laplacian solves.*

Proof sketch. Correctness of the inner loop is the MWU regret bound together with invariant (1) above: with the width-scaled step ε/μ, after $N = \Theta(\mu\,\varepsilon^{-2}\log m)$ rounds the averaged flow $\bar f$ has congestion $\le 1 + O(\varepsilon)$, or some round certified infeasibility. For width, observe that in any single electrical flow the energy is $\mathcal{E} = \sum_e r_e f_e^2 \le (1+\varepsilon)W$, while an edge of congestion ρ_e contributes $r_e f_e^2 = w_e \rho_e^2 \ge \rho_e^2$. Hence the number of edges with $\rho_e > \rho^\star$ in a given round is at most $(1+\varepsilon)W/\rho^{\star 2}$, and a charging argument over all rounds bounds the *total* deletions by $O(m/\rho^{\star 2} \cdot \mathrm{poly\,log})$. Each deleted edge can be shown to lie across an s–t cut of value $\le (1+\varepsilon)F$, so deleting it loses at most a $(1+\varepsilon)$ factor in feasibility, absorbed into ε. With $\rho^\star = m^{1/3}$: rounds $= O(m^{1/3}\varepsilon^{-2}\,\mathrm{poly\,log})$, each a solve at near-linear cost, total $O(m^{4/3}\ \mathrm{poly}(\varepsilon^{-1}, \log m))$ after the balanced charging. The careful bookkeeping is in the CKMST paper; the takeaway is that the only oracle used is a Laplacian solve. □

Trace it yourself

Run two rounds on the running network with all capacities $u_e = 1$, target $F = 1$, step ε moderate. Round 1: weights $w_e = 1$, resistances $r_e = 1$, so this is the electrical flow we already computed — congestions $\rho_{sa} = \rho_{sb} = \rho_{ac} = \rho_{bc} = \frac{1}{2}$, $\rho_{ab} = 0$, $\rho_{ct} = 1$. Energy $\mathcal{E} = 2$; with $W = 6$ the test $\mathcal{E} \le (1+\varepsilon)W$ passes, so $F = 1$ is feasible. The bottleneck $\{c, t\}$ is at congestion 1, every other edge at $\le \frac{1}{2}$, so the weight bump raises w_{ct} the most. Round 2: now $r_{ct} > 1$, so the next electrical flow finds $\{c, t\}$ stiffer — but $\{c, t\}$ is the *only* edge into t, so its current cannot drop below 1; what changes is upstream, where current keeps splitting evenly because a, b stay symmetric. The averaged flow $\bar f$ converges to: $\frac{1}{2}$ on each of $\{s, a\}, \{s, b\}, \{a, c\}, \{b, c\}$, 0 on $\{a, b\}$, 1 on $\{c, t\}$ — a feasible unit flow, which is in fact the max-flow here (min-cut $= \{c, t\}$ alone, value 1). Note that the bottleneck was already visible as the unique congestion-1 edge in round 1.



The electrical flow routing one unit from s to t: arrow thickness tracks current. The symmetric edge $\{a, b\}$ (dashed) carries nothing, and the lone edge $\{c, t\}$ (red) carries the full unit at congestion 1 — the bottleneck whose weight the multiplicative-weights loop raises most.

Connection

The CKMST result showed that maximum flow can be solved as a sequence of regularized electrical flows. Following this result, Sherman and Kelner–Lee–Orecchia–Sidford achieved $O(m \text{ poly}(\varepsilon^{-1}, \log n))$ for undirected max-flow, Madry pushed exact unit-capacity max-flow to $O(m^{10/7} \text{ poly} \log n)$ via electrical flows inside interior-point methods, and Lee and Sidford deepened the interior-point connection. This line of work led to Chen, Kyng, Liu, Peng, Probst Gutenberg, and Sachdeva's 2022 almost-linear time ($m^{1+o(1)}$) algorithm for general min-cost flow.

41.6 Applications of near-linear Laplacian solvers

A single near-linear $\mathbf{L}x = b$ solver serves as a general-purpose algorithmic primitive across graph theory. Each reduction follows the same pattern: express a graph problem via ℓ_2 energy minimization in the Laplacian, then extract the answer from one or more linear solves.

- **Electrical flow / effective resistance.** One solve (Theorem 41.1); the basic primitive.
- **Spectral sparsification.** $O(\log n)$ solves via JL leverage-score estimation (Section 41.4); then the remaining work runs on *only* $O(n)$ edges.
- **Max-flow / min-cut (undirected, approximate).** $O(m^{1/3} \text{ poly log})$ solves (Theorem 41.5); generalizes to min-cost flow.
- **Random spanning tree sampling.** Reductions to effective-resistance estimates (via Kirchhoff's matrix-tree identity in Section 41.2.1).
- **Graph semi-supervised learning / heat diffusion.** Solving $(\mathbf{L} + \tau I)x = b$ is label propagation; a single regularized solve.
- **Spectral clustering bootstrap.** Approximate Fiedler vectors via inverse power iteration, each step a solve.
- **Interior-point methods for flows and matchings.** Each Newton step is a re-weighted Laplacian solve; this is how almost-linear-time flow is achieved.

Why this matters

Classical graph algorithms are *combinatorial*: they make discrete local moves (augment a path, relax an edge, contract a blossom) and argue by potential functions over those moves. The Laplacian paradigm is *continuous*: it relaxes the discrete problem to a convex program over $\mathbb{R}^n$, solves the program with linear algebra, and rounds back. The relaxation that works is almost always the *electrical* one — ℓ_2 energy — because ℓ_2 minimization is a linear solve, and the linear solve is the thing we made fast. Max-flow is the main example because it is so combinatorial: the fact that even *it* can be solved with a sequence of electrical flows is strong evidence that "solve $\mathbf{L}x = b$" belongs beside "sort an array" as a basic subroutine.

Application

These ideas are used in practice. Resistance-based sampling underlies practical graph-sparsification libraries used to speed up GNN training and large-scale spectral clustering. Combinatorial-multigrid and lean algebraic-multigrid solvers — direct descendants of the tree-preconditioner idea — solve Laplacian and SDD systems in production for image segmentation, mesh processing, and circuit simulation (the original electrical-network application). And the interior-point-over-Laplacian-solves template is now the standard route to state-of-the-art running times for min-cost flow, bipartite matching, and optimal transport.

41.7 Historical Context and Controversies

History

For over fifty years, maximum flow was a central problem of combinatorial graph algorithms. Beginning with Ford and Fulkerson (1956), Edmonds and Karp (1972), Dinitz (1970), and Goldberg and Tarjan (1988, push-relabel), researchers approached maximum flow through augmenting paths, capacity scaling, and blocking flows, but all combinatorial methods remained bottlenecked near $O(m\sqrt{n})$ or $O(m^{3/2})$ time. In 2004, Daniel Spielman and Shang-Hua Teng introduced a nearly-linear time algorithm for solving symmetric diagonally dominant (SDD) linear systems $\mathbf{L}x = b$ in $\widetilde{O}(m \log(1/\varepsilon))$ time, bypassing classical $O(n^3)$ Gaussian elimination by recursively sparsifying graphs and constructing low-stretch spanning tree preconditioners. In 2008–2011, Daniel Spielman and Nikhil Srivastava introduced spectral sparsification via effective resistance sampling, showing that randomly sampling $O(n \log n/\varepsilon^2)$ edges proportional to their leverage scores $w_e R_{\text{eff}}(e)$ — the importance of an edge in preserving electrical energy — preserves all cuts and quadratic forms to within $1\pm\varepsilon$. Building on these foundations in 2011, Christiano, Kelner, Madry, Spielman, and Teng (CKMST) beat the long-standing combinatorial running times for approximate undirected max-flow by coupling electrical flow computations with multiplicative weights updates, computing $(1-\varepsilon)$-approximate maximum flow in $\widetilde{O}(m^{4/3}\varepsilon^{-3})$ time. In 2022, Chen, Kyng, Liu, Peng, Probst Gutenberg, and Sachdeva achieved almost-linear time $(m^{1+o(1)})$ for general min-cost flow by combining continuous interior-point methods with dynamic Laplacian solvers.

This combination of continuous and discrete methods was at first met with skepticism. In the 1990s, many researchers regarded maximum flow as an inherently discrete problem; the idea that continuous ℓ_2 electrical currents (which minimize quadratic energy $\sum r_e f_e^2$ and ignore edge capacity limits) could solve discrete ℓ_∞ capacity-constrained max-flow seemed implausible. The main technical difficulty was width: electrical current concentrates on low-resistance shortcuts and puts heavy congestion on bottleneck edges. CKMST resolved this through an energy-congestion invariant: heavily congested edges dissipate a lot of energy, allowing the algorithm to remove or heavily penalize them and converge in only $\widetilde{O}(m^{1/3})$ rounds. What remains open is developing practical implementations that eliminate large polylogarithmic overhead on real hardware, and obtaining near-linear algorithms for general directed maximum flow.

41.7.1 What We Can Now Do and Its Limitations

Connection

Capabilities Unlocked:

- We can solve linear systems in graph Laplacians $\mathbf{L}x = b$ in nearly-linear time $\tilde{O}(m \log(1/\varepsilon))$, which makes electrical potentials and effective resistances cheap to compute.
- Every dense graph can be compressed into a $(1 \pm \varepsilon)$ spectral sparsifier with only $O(n \log n/\varepsilon^2)$ edges in near-linear time.
- Maximum flow and minimum cut can be approximated to within $(1 \pm \varepsilon)$ in near-linear time via iterated electrical flows, bypassing classical combinatorial barriers.

Limitations:

- *Approximation requirement*: Continuous Laplacian solvers produce $(1 \pm \varepsilon)$ approximate flows and potentials; recovering exact integer flows requires additional rounding or interior-point iterations.
- *Asymptotic constants and implementation complexity*: While asymptotically near-linear, full theoretical Laplacian solvers feature large poly-logarithmic factors and intricate recursive tree decompositions. Practical implementations often rely on algebraic multigrid (AMG) or lean combinatorial heuristics.
- *Symmetric Diagonally Dominant (SDD) restriction*: The solver applies strictly to symmetric diagonally dominant matrices. General asymmetric linear systems or directed flows cannot be directly solved without specialized symmetrizations.

Where We Go Next:

- In Chapter 42 (Appendix: Math and Data Structures), we collect the mathematical tools used throughout this book: the Courant–Fischer min-max theorem, Rayleigh quotients, matrix concentration inequalities, and the core data structures (Euler-tour trees, segment trees, and disjoint-set union) needed to implement these algorithms.

Exercises

Exercise 41.1 (Hand-solve a second demand)**.** On the running network of Section 41.1 with unit conductances, solve $\mathbf{L}x = e_a - e_b$ (inject at a, extract at b). Use symmetry to guess x_s, x_c, x_t first, then verify. Compute the electrical flow and $R_{\text{eff}}(a, b)$. Explain why $R_{\text{eff}}(a, b) < 1$ even though edge $\{a, b\}$ has resistance 1.

Exercise 41.2 (Energy equals resistance)**.** Prove that for the unit flow from s to t, the minimum energy equals $R_{\text{eff}}(s, t) = (e_s - e_t)^\top \mathbf{L}^+(e_s - e_t)$, and that this equals the potential drop $x_s - x_t$. (Hint: plug $f^\star = \mathbf{W}\mathbf{B}x$ into $\mathcal{E}$ and use $\mathbf{L}x = e_s - e_t$.)

Exercise 41.3 (Stretch of a cycle)**.** For the unweighted cycle C_n, let T be C_n minus one edge e. Compute $\text{st}_T(G)$ exactly, and the resulting condition-number bound from Lemma 41.1. Compare to the true $\kappa(\mathbf{L}_{C_n}) = \Theta(n^2)$ and comment on how loose the stretch bound is for this single graph.

Exercise 41.4 (Leverage scores sum to $n-1$)**.** Prove $\sum_e w_e R_{\text{eff}}(e) = n - 1$ for any connected graph

on n vertices. (Hint: $w_e R_{\text{eff}}(e) = \operatorname{tr}(\mathbf{L}^+ w_e b_e b_e^\top)$; sum over e and recognize the trace of a projection.) Explain why this makes $O(n \log n)$ samples enough in Theorem 41.4.

Exercise 41.5 (PCG iteration count)**.** Suppose $\mathbf{P} \approx_\kappa \mathbf{L}$. Show that the eigenvalues of $\mathbf{P}^{-1}\mathbf{L}$ (restricted to $\mathbf{1}^\perp$) lie in $[1/\kappa, \kappa]$, and conclude that PCG (Algorithm 152) reaches relative $\mathbf{L}$-norm error ε in $O(\sqrt{\kappa}\log(1/\varepsilon))$ iterations. Where does the $\sqrt{\kappa}$, rather than κ, come from?

Exercise 41.6 (Width is the bottleneck)**.** Construct an undirected unit-capacity graph and an s, t pair where a single unit electrical flow puts congestion $\Theta(\sqrt{m})$ on one edge. (Hint: one short low-resistance edge in parallel with many long paths.) Explain why this forces fixed-step MWU (unit step, round count $\Theta(\mu^2\varepsilon^{-2}\log m)$) to use $\Theta(m)$ rounds, and how the CKMST edge-deletion step rescues the count.

Exercise 41.7 (Infeasibility certificate)**.** In Algorithm 154, prove that if some round has electrical energy $\mathcal{E}(f^{(i)}) > (1+\varepsilon)W$ then no capacity-respecting flow of value F exists. (Hint: any feasible g has $\mathcal{E}(g) \le W$ and the electrical flow minimizes energy.) What min-cut lower bound does this give?

Exercise 41.8 (Implement the solver loop)**.** Implement Algorithm 152 with $\mathbf{P} = \mathbf{L}_T$ for a spanning tree T, where $\textsc{Solve}_{\mathbf{L}_T}$ is the two-pass tree solve. Test on a random grid graph and plot iteration count versus grid size for (a) a BFS tree and (b) a low-stretch tree from a star-decomposition heuristic. Confirm the $\sqrt{\text{st}_T}$ scaling.

Exercise 41.9 (Effective resistance is a metric)**.** Prove that R_{eff} satisfies the triangle inequality: $R_{\text{eff}}(s,t) \le R_{\text{eff}}(s,u) + R_{\text{eff}}(u,t)$. (Hint: a unit flow $s \to t$ can be built by composing unit flows $s \to u$ and $u \to t$; energy is convex, and Thomson's principle says the electrical flow has least energy among all routings.)

Exercise 41.10 (From electrical flow to a cut)**.** Given the electrical potentials x from a unit s–t solve, describe a sweep-cut rounding: sort vertices by potential and threshold. Show on the running network that the best threshold cut recovers $\{c, t\}$ as the min-cut. State (without full proof) why a single electrical flow already gives a constant-factor cut approximation, and why iterating is needed for $(1+\varepsilon)$.

Exercise 41.11 (Regularized solve as diffusion)**.** Show that $(\mathbf{L} + \tau I)x = b$ has a unique solution for any $\tau > 0$ and any b (no $\mathbf{1}^\perp$ restriction). Interpret x as one step of heat diffusion / a personalized-PageRank-like vector, and argue that the same near-linear solver applies because $\mathbf{L} + \tau I$ is SDD. Relate $\tau \to 0$ to the pure Laplacian solve.

Part VIII

Background

Chapter 42

Appendix: Math and Data-Structure Background

The chapters in this book rely on a small set of recurring mathematical tools: a way to talk about how fast an algorithm runs, a handful of core data structures that show up over and over, and just enough linear algebra and probability to follow the spectral and randomized material. None of it is graph theory proper, which is why it lives here at the back rather than scattered through the main text. Every tool here is used directly in the chapters ahead. Union-find powers Kruskal's minimum-spanning-tree algorithm and almost every offline connectivity computation. Heaps drive Dijkstra and Prim. Segment trees and Fenwick trees power the range queries that heavy-light decomposition and Euler-tour techniques reduce to. Eigenvalues of the Laplacian *are* the content of Chapters 31–35, culminating in Chapter 41's electrical flows. And the probability section supplies what you need to follow Karger's contraction algorithm, streaming sketches, and Chapters 35–36 (random walks and PageRank).

The treatment here is self-contained but brisk. Where a topic deserves a whole book of its own – the linear algebra especially – we build the intuition and prove the facts the graph algorithms actually use, then point to the companion *Linear Algebra* volume for the full development. Read this appendix once front to back to calibrate notation, or dip in when a main chapter sends you here.

42.1 Asymptotics: Big-O, Big-Theta, Big-Omega

We never care about the exact running time of an algorithm down to the machine cycle. We care how the cost grows as the input grows, and we want a notation that throws away the clutter – constant factors, lower-order terms, the speed of the particular computer – and keeps the growth rate. That is what asymptotic notation does.

Intuition

Think of two algorithms as two functions $f(n)$ and $g(n)$ giving the number of basic operations on inputs of size n. Big-O is a claim about the *ceiling*: $f = O(g)$ says "f grows no faster than g, ignoring constants." Big-Omega is the *floor*: $f = \Omega(g)$ says "f grows at least as fast as g." Big-Theta is both at once: $f = \Theta(g)$ pins the growth rate exactly. When we say merge sort

is $\Theta(n \log n)$ we mean its cost is sandwiched between two constant multiples of $n \log n$ for all large n.

Definition 42.1 (Asymptotic classes)**.** Let $f, g : \mathbb{N} \to \mathbb{R}_{\geq 0}$. Then

$$f = O(g) \iff \exists\, c > 0,\ n_0 :\ \forall n \geq n_0,\ f(n) \leq c\, g(n),$$

$$f = \Omega(g) \iff \exists\, c > 0,\ n_0 :\ \forall n \geq n_0,\ f(n) \geq c\, g(n),$$

$$f = \Theta(g) \iff f = O(g) \text{ and } f = \Omega(g).$$

We write $f = o(g)$ when $f(n)/g(n) \to 0$, a *strictly* smaller growth rate.

The "=" in $f = O(g)$ is a historical abuse; $O(g)$ is really a *set* of functions and "$f = O(g)$" means $f \in O(g)$. You can chain it ($n^2 + 3n = n^2 + O(n)$) but never read it as an equation you can flip.

Example 42.1 (Discarding the small stuff)**.** Let $f(n) = 3n^2 + 50n + 1000$. Then $f = \Theta(n^2)$. Upper bound: for $n \geq 50$ we have $50n \leq n^2$ and $1000 \leq n^2$, so $f(n) \leq 3n^2 + n^2 + n^2 = 5n^2$, giving $O(n^2)$ with $c = 5$, $n_0 = 50$. Lower bound: $f(n) \geq 3n^2$ always, giving $\Omega(n^2)$ with $c = 3$. The linear and constant terms vanish into the constant; only the leading n^2 survives.

A few facts we use constantly, all immediate from the definition:

- **Transitivity.** $f = O(g)$ and $g = O(h)$ imply $f = O(h)$.
- **Sum rule.** $O(f) + O(g) = O(\max(f, g))$. Two phases of an algorithm cost the larger of the two.
- **Product rule.** If $f_1 = O(g_1)$ and $f_2 = O(g_2)$ then $f_1 f_2 = O(g_1 g_2)$. A loop of $O(g_1)$ iterations each costing $O(g_2)$ runs in $O(g_1 g_2)$.
- **Logs.** $\log_a n = \Theta(\log_b n)$ for any constant bases, so we drop the base inside O. But the base *matters* in an exponent: 2^n and 3^n are not asymptotically equal.

Common trap

For a loop running i from 1 to n with variable inner cost, multiplying the iteration count by the maximum inner cost can severely overcount. When the inner work is $O(i)$, the exact sum is $\sum_{i=1}^{n} O(i) = O(\sum i) = O(n^2/2) = O(n^2)$, which happens to match $O(n \cdot n)$ here, whereas inner work that drops geometrically (such as $O(n/2^i)$ across tree levels) sums to $O(n)$ rather than $O(n \log n)$. Always sum the actual per-iteration cost instead of multiplying the worst-case iteration by the loop count.

Connection

The notation O dates to Bachmann (1894) and Edmund Landau in analytic number theory. Donald Knuth imported it into algorithm analysis in the 1960s and, in a 1976 note, defined the matching Ω and Θ so that algorithm designers could state lower bounds and tight classifications cleanly rather than abusing O to mean "order exactly." The convention throughout this text follows Knuth: O for upper bounds, Ω for lower bounds, and Θ when the growth rate is pinned tightly from above and below.

42.1.1 A hierarchy to memorize

For large n, these grow in strictly increasing order, each o of the next:

$$1 \prec \log\log n \prec \log n \prec n^{\varepsilon} \prec n \prec n\log n \prec n^2 \prec n^3 \prec 2^n \prec n! \prec n^n,$$

where $0 < \varepsilon < 1$. Polynomials beat any polylog; exponentials beat any polynomial. The boundary between $\text{poly}(n)$ (tractable) and $2^{\Omega(n)}$ (intractable) is the line the complexity chapters draw.

42.2 Amortized Analysis

Some data structures are cheap on average but occasionally expensive. A dynamic array doubles when full: most insertions are $O(1)$, but the doubling insertion copies everything. Worst-case-per-operation says $O(n)$, which is true but misleading – a *sequence* of n insertions costs only $O(n)$ total, so $O(1)$ *amortized*. Amortized analysis is the rigorous way to charge for a sequence rather than for the worst single step.

Intuition

Imagine a prepaid coffee card. Each cheap operation overpays a little, banking credit. When an expensive operation arrives, it spends the banked credit instead of charging you a spike. If you can guarantee the bank never goes negative, then the total you actually paid (cheap overpayments) upper-bounds the total cost, and dividing by the number of operations gives a smooth per-operation price even though individual operations were lumpy.

There are three standard methods; they give the same bounds but feel different.

Aggregate method. Bound the total cost $T(n)$ of any sequence of n operations, then declare the amortized cost $T(n)/n$. For the doubling array, the copies happen at sizes $1, 2, 4, \ldots, 2^{\lfloor \log n \rfloor}$, summing to $\leq 2n$ copies total, so n insertions cost $O(n)$ and the amortized cost is $O(1)$.

Accounting method. Assign each operation an *amortized charge* $\widehat{c}_i$, possibly different from its real cost c_i. The charges must dominate every prefix: $\sum_{i=1}^{k} \widehat{c}_i \geq \sum_{i=1}^{k} c_i$ for all k. Then $\sum \widehat{c}_i$ bounds the true total. For the doubling array, charge each insertion three credits: one for its own write, and two banked to pay for copying itself and one earlier element when the next doubling hits. The bank is always nonnegative, so $\widehat{c} = O(1)$.

Potential method. The slickest. Define a *potential* function Φ mapping each data-structure state to a nonnegative real, with $\Phi(D_0) = 0$. The amortized cost of the i-th operation is

$$\widehat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1}).$$

Summing telescopes:

$$\sum_{i=1}^{n} \widehat{c}_i = \sum_{i=1}^{n} c_i + \Phi(D_n) - \Phi(D_0) = \sum_{i=1}^{n} c_i + \Phi(D_n) \geq \sum_{i=1}^{n} c_i,$$

using $\Phi(D_n) \geq 0 = \Phi(D_0)$. So the amortized total bounds the real total. The hard part is choosing Φ so that it absorbs the spikes.

Pause

Pause and think: Why does charging 3 credits per insertion in a dynamic array guarantee that the bank balance never goes negative? After the array doubles from size k to $2k$, every new insertion contributes 2 banked credits. By the time k new items arrive and the array fills, exactly $2k$ credits have been saved – precisely enough to pay for copying all $2k$ items into the next doubled array.

Figure 42.1 visualizes this credit cycle across a doubling sequence: while the true insertion cost c_i spikes whenever the array resizes, the banked potential $\Phi(D_i) = 2n - \text{cap}$ climbs steadily by +2 on cheap insertions and drops to 0 to pay for each copy spike, leaving a flat amortized charge $\widehat{c}_i = 3$.

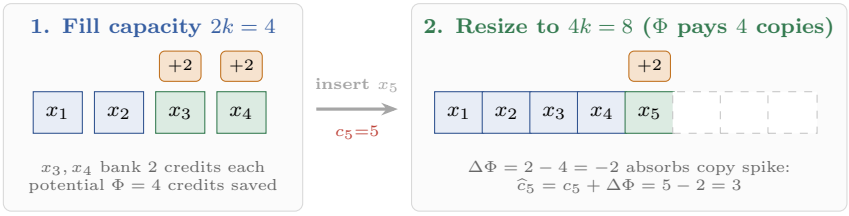


Figure 42.1: Amortized accounting and potential method on a doubling array ($\widehat{c}_i = 3$ credits per insertion). Left: after resizing from 2 to 4, inserting x_3 and x_4 uses 1 credit each for the write and banks 2 credits each ($\Phi = 4$). Right: when x_5 triggers a resize to capacity 8 (true cost $c_5 =$ 4 copies + 1 write = 5), the 4 banked credits pay for copying $x_1 \dots x_4$, while x_5's own 3 credits pay 1 for writing x_5 and bank 2 fresh credits ($\Phi = 2$).

Example 42.2 (Binary counter)**.** Increment a k-bit binary counter n times from zero. A single increment can flip many bits (e.g. $0111 \to 1000$ flips four). Worst case per increment is $O(k)$. Let Φ be the number of 1 bits. An increment that flips t trailing ones to zero and sets one zero to one has real cost $c_i = t + 1$ and changes the bit count by $-t + 1$, so

$$\widehat{c}_i = (t + 1) + (-t + 1) = 2 = O(1).$$

The potential exactly cancels the cascade. Total cost of n increments is $O(n)$, not $O(nk)$.

Why it works

Why is the potential method trustworthy? Because the telescoping sum is an identity, not an estimate. The only inequality used is $\Phi(D_n) \geq \Phi(D_0)$, which holds whenever Φ is nonnegative and starts at zero. So *any* valid potential yields a correct amortized bound; a cleverer Φ just yields a tighter one. This is exactly the mechanism behind the $O(\log n)$ amortized bounds for splay trees and the near-constant bounds for union-find below.

42.3 Union-Find (Disjoint-Set Union)

Maintain a partition of $\{1, \dots, n\}$ into disjoint sets under two operations: FIND(x) returns a canonical representative of x's set, and UNION(x, y) merges the two sets. Equality of representatives answers "are x and y in the same set?" This is precisely what Kruskal's algorithm needs to test whether an

edge would close a cycle, and what offline dynamic-connectivity and many percolation computations are built on.

Intuition

Represent each set as a rooted tree whose root is the representative. FIND walks up to the root. UNION hangs one root under the other. Two tricks keep the trees flat. *Union by rank* always hangs the shorter tree under the taller, so heights grow slowly. *Path compression* re-points every node visited during a FIND straight at the root, so future finds are instant. Together they make the forest almost completely flat.

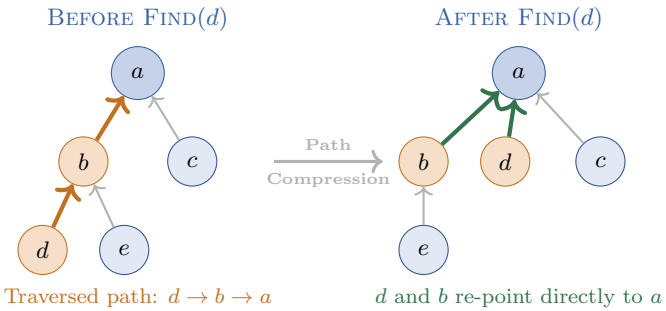


Figure 42.2: Disjoint-Set Union (DSU) path compression during FIND(d). Left: walking up from leaf d to root representative a visits nodes d and b. Right: as the recursion unwinds, all traversed nodes (d and b) re-point their parent pointers directly to root a, flattening the tree for all subsequent operations. The untouched branch e remains attached to b.

Algorithm 155: Union-Find with union by rank and path compression

```
function MakeSet(x):
    parent[x] ← x;   rank[x] ← 0
function Find(x):
    if parent[x] ≠ x then
        parent[x] ← Find(parent[x])                    // path compression
    return parent[x]
function Union(x, y):
    rx ← Find(x);   ry ← Find(y)
    if rx = ry then return
    if rank[rx] < rank[ry] then
        swap rx ↔ ry                                   // attach smaller under larger
    parent[ry] ← rx
    if rank[rx] = rank[ry] then rank[rx] ← rank[rx] + 1
```

Trace it yourself

Trace it. Start with singletons $1 \dots 6$, all rank 0. Process the edge set $\{(1,2),(3,4),(2,4),(5,6),(2,6)\}$ as Kruskal would.

- UNION$(1,2)$: ranks tie, 2 hangs under 1, $rank[1] = 1$. Sets: $\{1,2\}$.

- UNION$(3,4)$: 4 under 3, *rank*$[3] = 1$. Sets: $\{1,2\},\{3,4\}$.
- UNION$(2,4)$: FIND$(2) = 1$, FIND$(4) = 3$, ranks tie (1 each), 3 hangs under 1, *rank*$[1] = 2$. Set: $\{1,2,3,4\}$.
- UNION$(5,6)$: 6 under 5, *rank*$[5] = 1$. Sets: $\{1,2,3,4\},\{5,6\}$.
- UNION$(2,6)$: FIND$(2) = 1$ (rank 2), FIND$(6) = 5$ (rank 1), so 5 hangs under 1. Now FIND(6) next time compresses 6 straight to 1. One set: $\{1,2,3,4,5,6\}$.

Notice how rank tracks an *upper bound* on height, and how a later FIND flattens the chain $6 \to 5 \to 1$ into $6 \to 1$.

42.3.1 Why it is fast

Theorem 42.1 (Tarjan, 1975)**.** *Starting from n singletons, any sequence of $m \geq n$* FIND *and* UNION *operations with union by rank and path compression runs in $O(m\,\alpha(n))$ time, where α is the inverse Ackermann function.*

The function $\alpha(n)$ grows so slowly it is at most 4 for every n that could be written down in this universe; for practical purposes union-find is constant-time per operation. A full proof is intricate; we prove the clean intermediate bounds and state the sharp one.

Pause

Pause and think: If path compression flattens find paths so aggressively, why isn't the amortized cost per operation strictly $O(1)$? Because unions can link two deep trees without flattening them; an adversary can interleave unions and finds so that compressing one path leaves many long paths untouched in other branches, preventing the overall cost from collapsing to a constant.

Lemma 42.1 (Rank bounds)**.** *Under union by rank (even without path compression), a root of rank r has at least 2^r nodes in its tree, and the height of any tree is $O(\log n)$. Ranks of nodes along any root path strictly increase.*

Proof. By induction on the number of unions. A rank-0 root has its $1 \geq 2^0$ node. A root reaches rank r only by uniting two rank-$(r-1)$ roots, each with $\geq 2^{r-1}$ nodes by induction, so the merged tree has $\geq 2^r$ nodes. Hence $2^r \leq n$, i.e. $r \leq \log_2 n$. Path compression only moves nodes closer to the root and never raises a rank, so it preserves the node-count-per-rank bound. Ranks strictly increase from child to parent because a node is attached only under a strictly-or-equally-ranked root, and the equal case bumps the parent's rank. □

So *without* path compression each operation is already $O(\log n)$. Path compression pushes this down to $O(\log^* n)$ (iterated logarithm) by a simpler argument, and to the $\alpha(n)$ of Theorem 42.1 by Tarjan's full potential argument. The companion fact to carry away:

Cost summary

Union-find. Space $O(n)$. With union by rank *and* path compression, m operations in $O(m\,\alpha(n))$ – effectively $O(1)$ each. With only one of the two tricks, $O(\log n)$ per operation. Naive (neither), $O(n)$ worst case.

Application

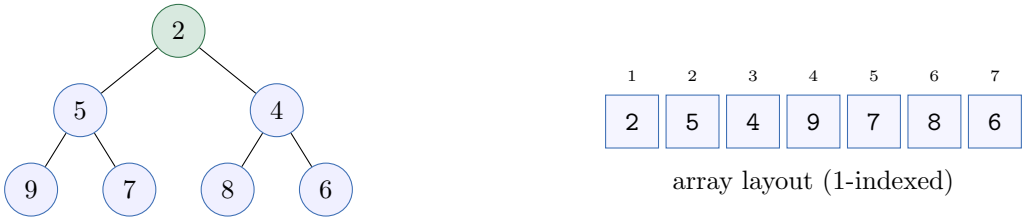
Kruskal's MST sorts edges then does m union-finds: the sort dominates at $O(m \log m)$, the connectivity bookkeeping is essentially free. Union-find also drives offline LCA (Tarjan's algorithm), incremental connected components, cycle detection in undirected graphs, and the merge step in Borůvka's algorithm. Whenever a problem is "maintain a partition under merges," reach for it.

42.4 Heaps and Priority Queues

A priority queue stores keyed elements and supports Insert, ExtractMin (remove and return the smallest key), and often DecreaseKey. Dijkstra and Prim *are* priority queues with a relaxation loop wrapped around them, so the heap's cost profile directly sets the shortest-path and MST bounds.

Intuition

A *binary heap* is a complete binary tree stored in an array, with the heap order: every node's key is $\leq$ its children's. The minimum sits at the root, index 1. Children of i live at $2i$ and $2i+1$; the parent of i is $\lfloor i/2 \rfloor$. To insert, drop the new key at the end and *sift up* swapping with the parent while it is smaller. To extract the min, move the last element to the root and *sift down* swapping with the smaller child while it violates order. Both touch one root-to-leaf path: $O(\log n)$.



Algorithm 156: Binary-heap sift operations (min-heap, array A, size s)

```
function SiftUp(i):
    while i > 1 and A[⌊i/2⌋] > A[i] do
        swap A[i] ↔ A[⌊i/2⌋];   i ← ⌊i/2⌋
function SiftDown(i):
    while 2i ≤ s do
        j ← 2i;   if j + 1 ≤ s and A[j + 1] < A[j] then j ← j + 1
        if A[i] ≤ A[j] then break
        swap A[i] ↔ A[j];   i ← j
function Insert(x):
    s ← s + 1; A[s] ← x; SiftUp(s)
function ExtractMin():
    m ← A[1]; A[1] ← A[s]; s ← s − 1; SiftDown(1); return m
```

Why it works

Correctness rests on a single invariant: *after every operation,* $A[\lfloor i/2 \rfloor] \le A[i]$ *for all* $i \ge 2$. SiftUp restores it by repairing the one edge that a fresh leaf could violate, walking the violation upward; SiftDown repairs the one edge a displaced root could violate, walking it downward. Because the tree is complete, its height is $\lfloor \log_2 n \rfloor$, so each sift does $O(\log n)$ comparisons. The root being the global minimum follows by transitivity along any root-to-node path.

Trace it yourself

Build a heap by inserting $5, 3, 8, 1, 4$ into an empty min-heap.

- Insert 5: $[5]$.
- Insert 3: $[5, 3] \to$ sift up $\to [3, 5]$.
- Insert 8: $[3, 5, 8]$, no swap.
- Insert 1: $[3, 5, 8, 1] \to$ sift up: $1 < 5$ swap $\to [3, 1, 8, 5]$; $1 < 3$ swap $\to [1, 3, 8, 5]$.
- Insert 4: $[1, 3, 8, 5, 4] \to$ sift up: parent of index 5 is index 2 $(= 3)$, $4 > 3$, stop.

Now ExtractMin: returns 1, moves 4 to root $\to [4, 3, 8, 5]$, sift down: smaller child is 3, swap $\to [3, 4, 8, 5]$; children of index 2 are 5 (index 4) only, $4 < 5$ stop. Result $[3, 4, 8, 5]$, min is 3. The trace matches the invariant at every step.

Heapify in linear time. Building a heap from an unsorted array of n keys looks like n inserts at $O(n \log n)$, but bottom-up SiftDown from index $\lfloor n/2 \rfloor$ down to 1 is only $O(n)$. The reason: a node at height h costs $O(h)$, and there are $\le n/2^{h+1}$ nodes at height h, so the total is

$$\sum_{h \ge 0} \frac{n}{2^{h+1}} O(h) = O\Big(n \sum_{h \ge 0} \frac{h}{2^h}\Big) = O(2n) = O(n),$$

since $\sum_{h \ge 0} h/2^h = 2$. Most nodes are leaves, where sifting down does nothing.

Cost summary

Heaps. Binary heap: INSERT, EXTRACTMIN, DECREASEKEY all $O(\log n)$; build $O(n)$; space $O(n)$. Binomial and pairing heaps trade constants. *Fibonacci heaps* achieve $O(1)$ amortized INSERT and DECREASEKEY with $O(\log n)$ amortized EXTRACTMIN, which is what gives Dijkstra its $O(m + n \log n)$ bound. In practice a binary or d-ary heap usually wins on constants.

Connection

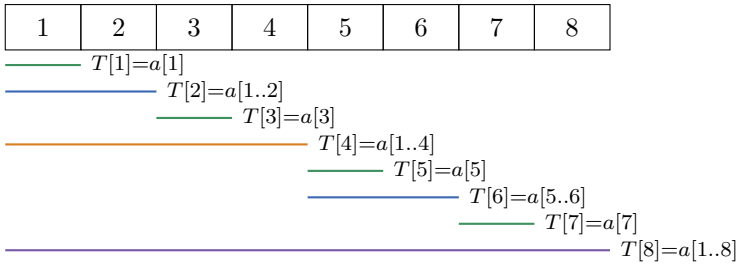
Williams introduced the binary heap in 1964 as the core structure of heapsort; Floyd gave the linear-time bottom-up build the same year. Fibonacci heaps (Fredman and Tarjan, 1984) were designed precisely to speed up Dijkstra's algorithm and minimum spanning trees, showing how amortized potential functions absorb cascading cuts.

42.5 Fenwick Trees (Binary Indexed Trees)

Maintain an array $a[1 \dots n]$ under two operations: *point update* $a[i] \mathrel{+}= v$ and *prefix sum* $\sum_{j \le i} a[j]$. A plain array does updates in $O(1)$ but prefix sums in $O(n)$; a prefix-sum array flips that. The Fenwick tree does both in $O(\log n)$ with a single array of size n and a bit trick: isolating the lowest set bit.

Intuition

Let $\text{lowbit}(i) = i \,\&\, (-i)$ be the lowest set bit of i (in two's complement, $-i$ flips and adds one, so $i \,\&\, (-i)$ isolates the rightmost 1). Store at index i the partial sum of the $\text{lowbit}(i)$ entries ending at i, namely $a[i - \text{lowbit}(i) + 1 \dots i]$. Then a prefix sum up to i is assembled by repeatedly subtracting lowbit: each step jumps over a block of size a power of two, and the blocks tile $[1, i]$ exactly. The number of jumps is the number of set bits in i, at most $\log_2 n$.



Algorithm 157: Fenwick tree (array $T[1 \dots n]$ initialized to 0)

```
function Update(i, v):
    while i ≤ n do
        T[i] ← T[i] + v;   i ← i + lowbit(i)        // climb to covering blocks
function PrefixSum(i):
    s ← 0
    while i > 0 do
        s ← s + T[i];   i ← i − lowbit(i)           // descend over tiling blocks
    return s
```

A range sum $a[l \dots r]$ is `PrefixSum(`r`) − PrefixSum(`l`-1)`. By swapping the roles – range update and point query – via a difference array, or running two Fenwick trees, you get range-update range-sum, still $O(\log n)$.

Why it works

Why do the descending jumps tile $[1, i]$ exactly? Index i covers $(i - \text{lowbit}(i), i]$. Set $i' = i - \text{lowbit}(i)$; this clears the lowest set bit, so i' covers a block ending exactly where the previous block began, with no gap and no overlap. Iterating clears set bits one at a time until i reaches 0, so the visited blocks partition $[1, i]$. Symmetrically, $i + \text{lowbit}(i)$ in UPDATE walks to every block whose range contains i, which are exactly the cells that must change. Both loops do one iteration per set bit encountered, bounded by $\lfloor \log_2 n \rfloor + 1$.

Trace it yourself

Let $n = 8$, all zero. `Update(3, 5)`: visits $3\,(\text{lowbit}\,1) \to 4\,(\text{lowbit}\,4) \to 8\,(\text{lowbit}\,8) \to 16 > 8$ stop. So $T[3], T[4], T[8]$ += 5. Now `PrefixSum(6)`: visits $6\,(+T[6]{=}0) \to 4\,(+T[4]{=}5) \to 0$ stop, returns 5. Correct: only $a[3] = 5$ lies in $[1, 6]$. `PrefixSum(2)` visits $2 \to 0$, returns 0. Range sum $a[3..6]$ = `PrefixSum(`6`) − PrefixSum(`2`)` $= 5 - 0 = 5$. The bit jumps did three array touches instead of scanning six cells.

42.6 Segment Trees

The Fenwick tree is a specialist: fast, tiny, but tied to invertible aggregates like sums. The *segment tree* is the generalist. It answers "aggregate over $a[l \dots r]$" for *any* associative operation – min, max, gcd, sum, matrix product – and supports point and (with lazy propagation) range updates, all in $O(\log n)$.

Intuition

Build a balanced binary tree over the index range $[1, n]$. The root owns $[1, n]$; each internal node owning $[l, r]$ splits at $m = \lfloor (l + r)/2 \rfloor$ into children $[l, m]$ and $[m + 1, r]$; leaves own single indices. Each node caches the aggregate of its range. A query $[ql, qr]$ recurses, returning a node's cached value whole when its range lies inside $[ql, qr]$, and recursing only when the query straddles the split. At most $O(\log n)$ nodes are ever fully inside while their parent straddles,

so the query visits $O(\log n)$ nodes.

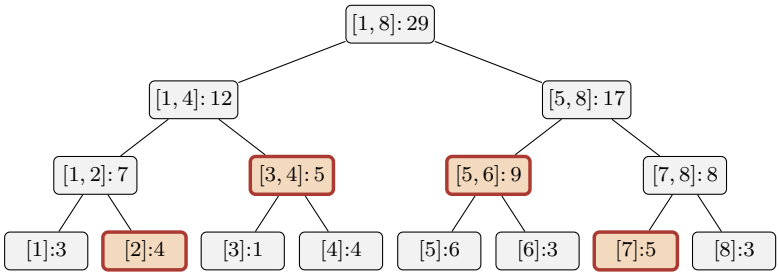


Figure 42.3: Segment tree range query decomposition for query range $[2, 7]$ on array $a = [3, 4, 1, 4, 6, 3, 5, 3]$. The query recursively traverses the tree and collects the maximal disjoint canonical subtrees completely contained in $[2, 7]$ (highlighted in orange with red borders): nodes [2]=4, [3, 4]=5, [5, 6]=9, and [7]=5. Summing these 4 cached values immediately yields $\sum a[2 \dots 7] = 4 + 5 + 9 + 5 = 23$ in $O(\log n)$ time without touching all 6 individual leaf elements.

The leaves hold $a = [3, 4, 1, 4, 6, 3, 5, 3]$; each internal node holds the sum of its subtree. A query for $\sum a[2 \dots 7]$ stitches together [2]=4, [3, 4]=5, [5, 6]=9, [7]=5, totaling 23 – four cached nodes, no full scan.

Algorithm 158: Segment tree: build and range query (sum; node p owns $[l, r]$)

```
1  function Build(p, l, r):
2      if l = r then t[p] ← a[l]; return
3      m ← ⌊(l + r)/2⌋
4      Build(2p, l, m);   Build(2p+1, m+1, r)
5      t[p] ← t[2p] + t[2p+1]
6  function Query(p, l, r, ql, qr):
7      if qr < l or r < ql then return 0                    // disjoint:  identity
8      if ql ≤ l and r ≤ qr then return t[p]                // fully inside
9      m ← ⌊(l + r)/2⌋
10     return Query(2p, l, m, ql, qr) + Query(2p+1, m+1, r, ql, qr)
11 function Update(p, l, r, i, v):
12     if l = r then t[p] ← v; return
13     m ← ⌊(l + r)/2⌋
14     if i ≤ m then Update(2p, l, m, i, v)
15     else Update(2p+1, m+1, r, i, v)
16     t[p] ← t[2p] + t[2p+1]
```

Theorem 42.2 (Query cost)**.** *A range query visits $O(\log n)$ nodes.*

Proof. At each depth of the tree, consider the nodes the recursion enters. A node is entered only if its range overlaps $[ql, qr]$ partially (a fully-inside or fully-outside node returns without recursing). Claim: at most two nodes per depth are entered with a *partial* overlap – one containing the left endpoint ql, one containing qr. Any node strictly between them is fully inside and stops; any node outside is disjoint and stops. The two boundary nodes each spawn at most two children at the next

depth, but again only the boundary-touching ones recurse. So the number of recursing nodes per level is $O(1)$, and the tree has depth $O(\log n)$, giving $O(\log n)$ visited nodes total. □

Common trap

The classic bug is an off-by-one in the disjoint test. Writing $qr < l$ *or* $r < ql$ is correct; writing $qr \le l$ accidentally drops the boundary index when $ql = qr = l$. Equally common: forgetting that the identity element depends on the operation – 0 for sum, $+\infty$ for min, $-\infty$ for max, the empty product for products. Return the wrong identity on the disjoint branch and a single query silently corrupts the whole answer.

42.6.1 Lazy propagation

To support *range* updates (add v to all of $a[l \dots r]$) without touching every leaf, store a pending *lazy* value at each node: "this whole subtree still owes this update." Apply it to the node's cached aggregate immediately, but push it down to children only when the recursion actually needs to descend past this node. A node fully inside the update range absorbs the change in $O(1)$ and parks the lazy tag; the $O(\log n)$ boundary argument of Theorem 42.2 still bounds the work.

Why it works

The lazy tag is correct because the cached aggregate is kept consistent with all applied-but-not-pushed updates: a range-add of v over a node owning k indices raises its sum by $k \cdot v$ instantly, and the children's caches stay valid relative to *their* own tags. Pushing down before recursing guarantees a child is never read while an ancestor's tag is still owed to it. This is the same "pay lazily, settle on demand" idea as the amortized accounting method, applied to subtrees instead of to a sequence of operations.

Cost summary

Range structures. Fenwick: $O(\log n)$ per op, $O(n)$ space, invertible aggregates only, smallest constants. Segment tree: $O(\log n)$ per op, $O(n)$ space (array of size $\le 4n$), any associative aggregate, range updates via lazy propagation. Sparse table (static, idempotent ops like min): $O(n \log n)$ build, $O(1)$ query, no updates.

Connection

The tree chapters rely on these structures throughout. Heavy-light decomposition (Chapter 19) and the Euler-tour technique (Chapter 21) reduce path and subtree queries to *range* queries on an array, then hand them to a segment tree. The LCA-as-RMQ reduction (Chapter 23) feeds a sparse table or segment tree. Link-cut trees (Chapter 22) generalize the segment-tree idea to a forest that changes shape. If a tree algorithm in this book runs in $O(n \log n)$, a range structure is almost certainly why.

42.7 Linear Algebra for the Spectral Chapters

The Laplacian, spectral-clustering, expander, and electrical-flow chapters all speak the language of symmetric matrices and their eigenvalues. We collect the facts they use. This is a summary, not a course; the companion *Linear Algebra* volume develops everything below from scratch, and we cite it freely.

42.7.1 Symmetric matrices and the spectral theorem

A real matrix $\mathbf{M} \in \mathbb{R}^{n \times n}$ is *symmetric* if $\mathbf{M}^\top = \mathbf{M}$. Adjacency matrices of undirected graphs and graph Laplacians are symmetric, which is exactly why graphs have real spectra. A nonzero vector x is an *eigenvector* with *eigenvalue* λ if $\mathbf{M}x = \lambda x$: the matrix merely stretches x by λ without rotating it.

Theorem 42.3 (Spectral theorem, real symmetric case)**.** *Every symmetric $\mathbf{M} \in \mathbb{R}^{n \times n}$ has n real eigenvalues $\lambda_1 \le \lambda_2 \le \cdots \le \lambda_n$ (with multiplicity) and an associated* orthonormal *basis of eigenvectors $u_1, \ldots, u_n$. Equivalently*

$$\mathbf{M} = \sum_{i=1}^{n} \lambda_i \, u_i u_i^\top = \mathbf{U} \Lambda \mathbf{U}^\top,$$

where $\mathbf{U} = [u_1 \ \cdots \ u_n]$ is orthogonal ($\mathbf{U}^\top \mathbf{U} = \mathbf{I}$) and $\Lambda = \operatorname{diag}(\lambda_1, \ldots, \lambda_n)$.

We do not prove it here – see the companion *Linear Algebra* volume – but the consequences run through every spectral chapter. The eigenvectors give an orthonormal coordinate system in which $\mathbf{M}$ acts diagonally; problems that look coupled in the standard basis decouple into n independent one-dimensional problems, one per eigenvalue.

Intuition

Picture $\mathbf{M}$ as a deformation of space. The spectral theorem says a symmetric deformation has n perpendicular "principal axes" (the u_i) along which it is pure stretch by λ_i, no shear. The Laplacian's smallest eigenvalue is always 0 with eigenvector the all-ones vector, because $\mathbf{L}\mathbf{1} = 0$; the *number* of zero eigenvalues counts connected components, and the second-smallest λ_2 – the Fiedler value – measures how hard the graph is to cut. That single number is central to Chapters 33 and 34 (spectral clustering and expanders).

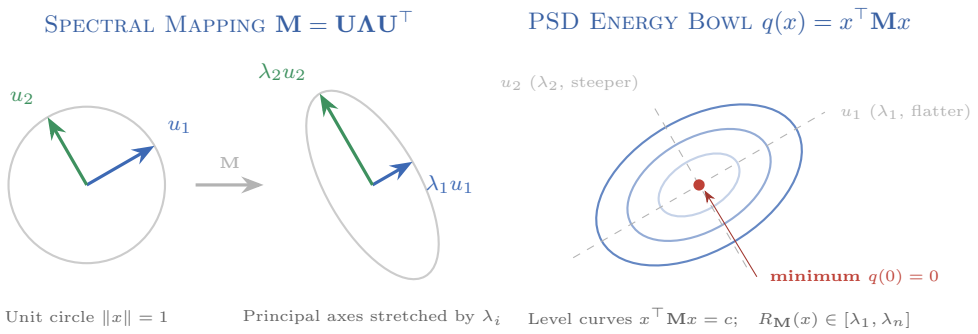


Figure 42.4: Geometry of symmetric and positive semidefinite matrices. Left: the Spectral Theorem states that a symmetric matrix $\mathbf{M}$ acts by pure stretch along mutually orthogonal eigenvectors u_1, u_2 without rotation or shearing, mapping the unit circle to an aligned ellipse. Right: when $\mathbf{M} \succeq 0$, the quadratic energy surface $q(x) = x^\top \mathbf{M} x \geq 0$ forms a convex parabolic bowl whose concentric level sets are ellipsoids, with curvature λ_i along direction u_i. The Rayleigh quotient $R_\mathbf{M}(x) = \frac{x^\top \mathbf{M} x}{\|x\|^2}$ smoothly interpolates between minimum curvature λ_1 and maximum curvature λ_n.

A symmetric $\mathbf{M}$ maps the unit circle to an ellipse whose axes are the orthonormal eigenvectors u_i, each scaled by its eigenvalue λ_i. No rotation of the axes occurs; that is what "symmetric means real spectrum, orthogonal eigenvectors" looks like geometrically.

42.7.2 Positive semidefiniteness and the Rayleigh quotient

Definition 42.2 (PSD and PD)**.** A symmetric $\mathbf{M}$ is *positive semidefinite* (PSD), written $\mathbf{M} \succeq 0$, if $x^\top \mathbf{M} x \geq 0$ for all $x \in \mathbb{R}^n$; it is *positive definite* (PD, $\mathbf{M} \succ 0$) if the inequality is strict for all $x \neq 0$.

Proposition 42.1 (Equivalent characterizations of PSD)**.** *For symmetric $\mathbf{M}$ the following are equivalent: (i) $\mathbf{M} \succeq 0$; (ii) every eigenvalue $\lambda_i \geq 0$; (iii) $\mathbf{M} = \mathbf{B}^\top \mathbf{B}$ for some matrix $\mathbf{B}$.*

Sketch. (i)⇒(ii): take $x = u_i$, then $u_i^\top \mathbf{M} u_i = \lambda_i \geq 0$. (ii)⇒(iii): with $\lambda_i \geq 0$ set $\mathbf{B} = \Lambda^{1/2}\mathbf{U}^\top$, so $\mathbf{B}^\top \mathbf{B} = \mathbf{U}\Lambda\mathbf{U}^\top = \mathbf{M}$. (iii)⇒(i): $x^\top \mathbf{B}^\top \mathbf{B} x = \|\mathbf{B}x\|^2 \geq 0$. □

The graph Laplacian is the headline example: $\mathbf{L} = \sum_{(u,v)\in E} (e_u - e_v)(e_u - e_v)^\top$, which is manifestly of the form $\mathbf{B}^\top \mathbf{B}$ with $\mathbf{B}$ the signed incidence matrix, hence PSD. Concretely, $x^\top \mathbf{L} x = \sum_{(u,v)\in E} (x_u - x_v)^2 \geq 0$, the "Dirichlet energy" minimized in Chapters 31, 33, 35, and 41.

Definition 42.3 (Rayleigh quotient)**.** For symmetric $\mathbf{M}$ and $x \neq 0$, the *Rayleigh quotient* is

$$R_\mathbf{M}(x) = \frac{x^\top \mathbf{M} x}{x^\top x}.$$

Theorem 42.4 (Variational characterization of eigenvalues)**.** *With $\lambda_1 \leq \cdots \leq \lambda_n$,*

$$\lambda_1 = \min_{x \neq 0} R_\mathbf{M}(x), \qquad \lambda_n = \max_{x \neq 0} R_\mathbf{M}(x),$$

and more generally (Courant–Fischer)

$$\lambda_k = \min_{\dim S = k} \max_{0 \neq x \in S} R_\mathbf{M}(x).$$

Proof. Expand $x = \sum_i c_i u_i$ in the orthonormal eigenbasis. Then $x^\top x = \sum_i c_i^2$ and $x^\top \mathbf{M} x = \sum_i \lambda_i c_i^2$, so

$$R_{\mathbf{M}}(x) = \frac{\sum_i \lambda_i c_i^2}{\sum_i c_i^2}$$

is a weighted average of the eigenvalues with nonnegative weights c_i^2. A weighted average lies between the smallest and largest values, attaining λ_1 at $x = u_1$ and λ_n at $x = u_n$. The Courant–Fischer extension follows by the same averaging restricted to k-dimensional subspaces. □

Why it works

Much of spectral graph theory starts from this theorem. It turns "find the second eigenvalue λ_2 of the Laplacian" into "minimize $\sum_{(u,v)\in E}(x_u - x_v)^2$ over unit vectors x orthogonal to $\mathbf{1}$." That optimization *is* the relaxed balanced-cut problem: assign each vertex a real number so that adjacent vertices land close, subject to a spread constraint. Cheeger's inequality then sandwiches the true cut quality between $\lambda_2/2$ and $\sqrt{2\lambda_2}$, which is how the Fiedler eigenvector becomes a graph partitioner. Every appearance of "the energy of a vector" or "the Rayleigh quotient" in the later chapters traces back to Theorem 42.4.

Example 42.3 (Trace and determinant from the spectrum)**.** Two identities used repeatedly: $\operatorname{tr}(\mathbf{M}) = \sum_i \lambda_i$ and $\det(\mathbf{M}) = \prod_i \lambda_i$. The first counts, e.g., closed walks: $\operatorname{tr}(\mathbf{A}^k)$ equals both the number of closed walks of length k and $\sum_i \lambda_i^k$. The second underlies the matrix-tree theorem: the number of spanning trees equals $\frac{1}{n}\prod_{i\geq 2} \lambda_i$ over the nonzero Laplacian eigenvalues.

Connection

Pointers into the main text. Chapter 31 builds $\mathbf{L}$ and proves the component count equals $\dim\ker \mathbf{L}$. Chapter 32 develops the full spectrum and the matrix-tree theorem. Chapter 33 runs k-means on the bottom Fiedler vectors. Chapter 34 ties λ_2 to expansion. Chapter 35 reads off effective resistance from the pseudoinverse $\mathbf{L}^+$, and Chapter 41's Laplacian solvers approximate $\mathbf{L}^{-1}b$ in near-linear time. For derivations of the spectral theorem, PSD factorizations, pseudoinverses, and Courant–Fischer, see the companion *Linear Algebra* volume, in the chapters on symmetric operators and quadratic forms.

42.8 Probability for the Randomized Chapters

Karger's contraction algorithm, randomized min-cut, random walks, PageRank, and the streaming sketches all rely on a small amount of discrete probability, collected here.

42.8.1 Expectation, linearity, and variance

A discrete random variable X takes values x with probabilities $\mathbb{P}[X = x]$. Its *expectation* is $\mathbb{E}[X] = \sum_x x\, \mathbb{P}[X = x]$. The single most-used fact is *linearity of expectation*: for *any* random variables (independent or not) and constants,

$$\mathbb{E}\left[\sum_i a_i X_i\right] = \sum_i a_i\, \mathbb{E}[X_i].$$

No independence needed – this is what makes it so useful. Counting arguments that look hopeless ("expected number of edges crossing a random cut") collapse to summing one indicator at a time.

Example 42.4 (Indicator trick)**.** To count the expected number of events that occur, write the count as a sum of indicator variables $X_i = \mathbf{1}[\text{event } i]$, note $\mathbb{E}[X_i] = \mathbb{P}[\text{event } i]$, and sum. For a uniformly random cut of a graph with m edges, each edge is cut with probability $1/2$, so the expected number of cut edges is $m/2$ – the one-line proof that a max cut of size $\geq m/2$ always exists.

The *variance* $\mathrm{Var}[X] = \mathbb{E}[(X - \mathbb{E}[X])^2] = \mathbb{E}[X^2] - \mathbb{E}[X]^2$ measures spread, and for *independent* X, Y, $\mathrm{Var}[X + Y] = \mathrm{Var}[X] + \mathrm{Var}[Y]$.

42.8.2 Concentration: from average to almost-sure

Expectation tells you the center; concentration inequalities tell you a random variable rarely strays far. Three, in increasing strength and assumption:

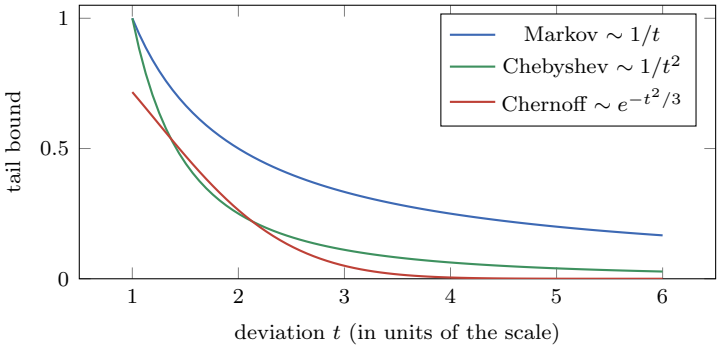
Theorem 42.5 (Markov, Chebyshev, Chernoff)**.** *Let $X \geq 0$.* (Markov) $\mathbb{P}[X \geq t] \leq \mathbb{E}[X]/t$. *If X has finite variance,* (Chebyshev) $\mathbb{P}[\,|X - \mathbb{E}[X]| \geq t\,] \leq \mathrm{Var}[X]/t^2$. *If $X = \sum_{i=1}^{n} X_i$ is a sum of independent $\{0,1\}$ variables with $\mu = \mathbb{E}[X]$, then* (Chernoff) *for $0 < \delta \leq 1$,*

$$\mathbb{P}[\,|X - \mu| \geq \delta\mu\,] \leq 2\exp\Big(-\frac{\delta^2\mu}{3}\Big).$$

Markov, and the bounds from it. Since $X \geq 0$, $\mathbb{E}[X] = \sum_x x\,\mathbb{P}[X = x] \geq \sum_{x \geq t} x\,\mathbb{P}[X = x] \geq t\,\mathbb{P}[X \geq t]$, giving Markov. Chebyshev is Markov applied to $(X - \mathbb{E}[X])^2$ with threshold t^2. Chernoff applies Markov to the moment generating function e^{sX} and optimizes over s; the exponential decay is what makes "repeat and take the majority" so effective. □

Intuition

The qualitative ladder: Markov needs only nonnegativity and decays like $1/t$ – weak but universal. Chebyshev needs variance and decays like $1/t^2$ – the basis of the median trick. Chernoff needs independence and decays *exponentially* – this is why a randomized algorithm with success probability $2/3$ can be boosted to $1 - 2^{-k}$ by running it $O(k)$ times and voting. The error shrinks geometrically in the number of repetitions.



The three bounds in increasing strength: Markov decays polynomially and slowly, Chebyshev faster, and Chernoff drops off exponentially. The price for the stronger bound is a stronger hypothesis – nonnegativity, then finite variance, then independence.

Application

Karger's algorithm contracts a random edge repeatedly; a fixed min cut survives all contractions with probability $\geq 2/n^2$ (a linearity-and-conditioning computation). Running it $O(n^2 \log n)$ times makes the failure probability $(1 - 2/n^2)^{\Theta(n^2 \log n)} \leq 1/n$ by the inequality $1-x \leq e^{-x}$ – pure Chernoff-style boosting. Chapters 35 and 36 (random walks and PageRank) use the same machinery to bound mixing times: a walk's distribution converges to stationary at a rate governed by the spectral gap $1 - \lambda_2$ of the transition matrix, tying the probability section straight back to the linear algebra one.

Common trap

Independence is not optional in Chernoff. Linearity of expectation survives arbitrary dependence, but variance addition and the exponential tail bound do *not*: highly correlated indicators can have the right mean and still pile all their mass at the extremes. When the randomized chapters invoke Chernoff, check that the summands really are independent (or use a martingale/Azuma variant when they are not). Mixing up "$\mathbb{E}$ behaves nicely" with "the sum concentrates" is the most common error in randomized analysis.

42.9 Historical Context, Controversies, and the Unified Curriculum

History

Each of the mathematical and algorithmic tools in this appendix grew out of a concrete problem in data organization, continuous physics, or statistical inference. Asymptotic notation began with Paul Bachmann (1894) and Edmund Landau in number theory, before Donald Knuth imported O, Ω, and Θ in the 1970s to give algorithm analysis a standard classification system. The analysis of Disjoint-Set Union (DSU) produced one of the most famous surprises in theoretical computer science. Introduced by Bernard Galler and Michael Fischer in 1964, the runtime of union-find with path compression remained open for a decade until Robert Tarjan published his 1975 analysis. Tarjan proved that m operations on n elements require $\Theta(m\,\alpha(m, n))$ time, where α is the functional inverse of Ackermann's hyper-exponential growth function – showing that the cost per operation is provably non-constant yet grows so slowly that $\alpha(n) \leq 4$ for all n within physical universe bounds. Priority queues saw a parallel evolution: J. W. J. Williams introduced the binary heap in 1964, Robert Floyd devised the linear-time bottom-up build that same year, and Michael Fredman and Robert Tarjan (1984) created Fibonacci heaps to accelerate Dijkstra's and Prim's algorithms via amortized potential functions.

Range query data structures emerged from geometric computing and data compression. In 1977, Jon Bentley introduced segment trees to solve Klee's measure problem in computational geometry, pioneering the canonical decomposition of arbitrary intervals into logarith-

mic dyadic nodes. In 1994, Peter Fenwick introduced the Binary Indexed Tree while developing arithmetic data compression algorithms, discovering that low-bit manipulation in two's-complement arithmetic creates a tree structure with zero pointer overhead. In parallel, the spectral foundations trace to Ernst Fischer (1905) and Richard Courant (1920), who formulated the Courant–Fischer min-max theorem to characterize eigenvalues as solutions to variational Rayleigh quotient optimizations over subspaces.

Several of these results corrected widespread misconceptions. Before Tarjan, algorithm designers were divided between believing path compression achieved $O(1)$ amortized time or suspecting an unyielding $\Omega(\log \log n)$ lower bound; both were wrong, and the true bound turned out to lie outside the elementary functions. When random matrices emerged in spectral graph theory, researchers routinely made the error of applying scalar Chernoff bounds element-by-element to matrices and taking a union bound over n^2 entries, which gives loose bounds, until Joel Tropp (2012) developed Matrix Chernoff bounds via non-commutative operator inequalities. What remains open includes developing truly cache-oblivious range query structures that match pointer-free theoretical bounds without empirical constant overhead, and simplifying non-commutative matrix concentration inequalities for graph applications.

42.9.1 Summary of the Book and Companion Volumes

Connection

What This Book Has Built: Across forty-two chapters, this book has covered graph algorithms from the basics up:

1. *Discrete Foundations*: From elementary traversals (BFS, DFS) and shortest paths (Dijkstra, Bellman–Ford) to trees, topological sorting, and disjoint sets.
2. *Combinatorial Optimization*: Maximum flows, minimum cuts, bipartite matching, link-cut trees, and heavy-light decomposition.
3. *NP-Hardness and Approximation*: Karp's web of reductions, Christofides' metric TSP, and Goemans–Williamson semidefinite hyperplane rounding.
4. *Dynamic and Streaming Algorithms*: Real-time forest maintenance via HdLT level hierarchies and sublinear-space processing via AGM linear sketches.
5. *Continuous and Spectral Methods*: The Laplacian matrix, Cheeger's isoperimetric inequality, spectral clustering, random walk electrical reciprocity, and the Spielman–Teng nearly-linear time Laplacian solver.

Where These Tools Go Next in the 8-Book Curriculum:

- In **Book 3 (Linear Algebra From the Ground Up)**, the eigenvalues, singular values, and Rayleigh quotients introduced here are developed from geometric first principles as linear transformations.
- In **Book 4 (Probability and Statistics From the Ground Up)**, the Markov chains, stationary distributions, and Chernoff concentration inequalities used in PageRank and random walks are explored in depth.
- In **Book 5 (Optimization From the Ground Up)**, the semidefinite programs, multiplicative weights updates, and interior-point methods that broke the max-flow barrier

become our central study.

42.10 Exercises

Exercise 42.1 (Asymptotic sorting)**.** Order these by growth rate, grouping any that are Θ of each other: $n \log n$, $2^{\log_2 n}$, $n^{1/\log_2 n}$, $\log(n!)$, $n^{1.001}$, $(\log n)^{\log n}$, $2^{\sqrt{\log n}}$. (Hint: $\log(n!) = \Theta(n \log n)$ by Stirling; $n^{1/\log_2 n} = 2$.)

Exercise 42.2 (Potential method)**.** A stack supports PUSH, POP, and MULTIPOP(k) which pops $\min(k, \text{size})$ elements. Using the potential $\Phi =$ stack size, prove every operation has $O(1)$ amortized cost, so any sequence of n operations on an initially empty stack runs in $O(n)$.

Exercise 42.3 (Union-find lower bound by example)**.** Show that *without* union by rank, path compression alone can produce a single FIND costing $\Omega(n)$: give an adversarial sequence of unions building a path of length n, then exhibit the expensive find. Then show union by rank alone (no compression) keeps every find $O(\log n)$, citing Lemma 42.1.

Exercise 42.4 (Heapify is tight)**.** Prove the bottom-up build is $\Theta(n)$, not merely $O(n)$: exhibit an input on which $\Omega(n)$ swaps actually occur. Contrast with the top-down "insert n times" build and give an input forcing it to do $\Theta(n \log n)$ work.

Exercise 42.5 (Fenwick from scratch)**.** Implement a Fenwick tree supporting range-add and range-sum using two Fenwick trees over a difference array. Prove the formula $\sum_{j=1}^{i} a[j] = (i+1) \cdot B_1.\text{QUERY}(i) - B_2.\text{QUERY}(i)$ for the auxiliary arrays B_1, B_2 you maintain, and verify it on $n = 4$ with one range update by hand.

Exercise 42.6 (Segment tree, non-sum aggregate)**.** Adapt the segment tree of Section 42.6 to answer range-*minimum* queries with point updates. State the identity element, rewrite the combine step, and trace a query for $\min a[2 \ldots 7]$ on the array $[3, 4, 1, 4, 6, 3, 5, 3]$, listing the cached nodes stitched together.

Exercise 42.7 (Break the lazy tag)**.** Here is a buggy lazy segment tree for range-add/range-sum: it pushes the lazy tag to children *after* recursing instead of before. Construct a sequence of two overlapping range-adds and one query that returns the wrong sum, and explain which invariant the misordering violates.

Exercise 42.8 (Rayleigh quotient by hand)**.** Let $\mathbf{M} = \left(\begin{smallmatrix} 2 & 1 \\ 1 & 2 \end{smallmatrix}\right)$. Compute its eigenvalues and orthonormal eigenvectors. Verify Theorem 42.4 by evaluating $R_{\mathbf{M}}(x)$ at $x = (1, 0)$, $x = (1, 1)$, $x = (1, -1)$ and checking each lies in $[\lambda_1, \lambda_2]$, with equality exactly at the eigenvectors.

Exercise 42.9 (Laplacian is PSD)**.** For the path graph on three vertices $1-2-3$, write $\mathbf{L}$ explicitly, exhibit it as $\mathbf{B}^\top \mathbf{B}$ for the signed incidence matrix $\mathbf{B}$, and confirm $x^\top \mathbf{L} x = (x_1 - x_2)^2 + (x_2 - x_3)^2 \geq 0$. Find $\ker \mathbf{L}$ and relate its dimension to the number of connected components.

Exercise 42.10 (Indicator trick)**.** A uniformly random permutation of $\{1, \ldots, n\}$ has a *fixed point* at i if it maps $i \mapsto i$. Using indicator variables and linearity, show the expected number of fixed points is exactly 1, independent of n. Then use Chebyshev to bound the probability of seeing $\geq t$ fixed points.

Exercise 42.11 (Boosting by repetition)**.** A Monte Carlo algorithm answers correctly with probability $2/3$ on every input. You run it $2k+1$ independent times and take the majority vote. Using the Chernoff bound of Theorem 42.5, show the majority is wrong with probability at most $e^{-\Omega(k)}$, and deduce how many repetitions guarantee error below $1/n$.

Exercise 42.12 (Karger survival, warm-up)**.** In a graph with min cut of size c, every vertex has degree $\geq c$, so $m \geq nc/2$. Show that a uniformly random edge avoids a fixed min cut with probability $\geq 1 - \frac{2}{n}$. Chain these across $n-2$ contractions to derive the $\geq 2/(n(n-1))$ survival probability quoted in Section 42.8.